Shiply

Accounting System

Developer Guide: architecture, calculations, postings, screens and user interface

Shiply Now pilot platform

Version of 12 November 2026

Confidential. Prepared for an engineering partner.

Contents

1 Introduction 6
- 1.1 Purpose of this document 6
- 1.2 What Shiply Now is 6
- 1.3 Roles 6
- 1.4 The apps 7
- 1.5 How to read this document 8
- 1.6 Conventions 9

2 Architecture 11
- 2.1 Monorepo layout 11
- 2.2 How a request flows 12
- 2.3 Background jobs 14
- 2.4 Deployment 15
- 2.5 Testing 15
- 2.6 Accounting code map 16

3 The ledger core 28
- 3.1 Tables 28
- 3.2 post(): the only way to write 30
- 3.3 Worked example: a delivered order 34
- 3.4 Database guards 34
- 3.5 Balances 36
- 3.6 Holding money: lockedBalance and mustHold 36
- 3.7 Cairo days and months 37
- 3.8 Closing the books 38
- 3.9 Closing a year 38

4 Chart of accounts 39
- 4.1 Categories 39
- 4.2 Groups 39
- 4.3 Accounts by code 40
- 4.4 System accounts and party wallets 44
- 4.5 Which accounts a journal entry may touch 46
- 4.6 Opening balances 47

5 Controls and access 48
- 5.1 Access profiles 48
- 5.2 The second person rule 51
- 5.3 Books lock and reopening 55
- 5.4 Audit log and History 55
- 5.5 Idempotency 56
- 5.6 Error answers 57

6 Merchant fees: contracts, prices and the postings of each event 63
- 6.1 Conventions used by every fee 63
- 6.2 Merchant contracts v2: versions and the version in force 64
- 6.3 Standard prices from the settings 65
- 6.4 Resolving a price field by field 66
- 6.5 Volume tiers 67
- 6.6 Fee kinds 67

6.7 The delivery posting 68
6.8 Postal fee 70
6.9 The failed delivery rule 71
6.10 Receiving fee 71
6.11 Storage, charged daily with catch up 72
6.12 Return fee 73
6.13 Monthly minimum fee 73
6.14 VAT on top 74
6.15 Damage compensation and rider charge 74
6.16 Credit limit 75
6.17 The contract calculator 75
6.18 Where the code lives 75

7 Cash on delivery, riders, store boxes and rider pay 76
7.1 Accounts used 76
7.2 COD collection 77
7.3 Rider cash limit at handover to the rider 77
7.4 Cash age and the overdue alert 77
7.5 Cash handover to the store box 78
7.6 Bank deposit from a store box 79
7.7 Petty cash from a store box 79
7.8 Basata deposits 80
7.9 Rider pay per delivery 82
7.10 The withdrawal hold 82
7.11 The wallet figures 82
7.12 Withdrawals 83
7.13 What riders owe (1350) and penalties 84
7.14 Rider supply offices 84
7.15 Where the code lives 86

8 Expenses, suppliers, bills and the assets we pay for 87
8.1 Rules shared by every movement of money 87
8.2 Expenses paid at once 88
8.3 Suppliers 88
8.4 Supplier bills 89
8.5 Paying a bill and withholding tax 90
8.6 Bills in another currency 91
8.7 Cheques 91
8.8 Purchase orders 93
8.9 Packaging stock at average cost 95
8.10 Recurring expenses 95
8.11 Prepayments 96
8.12 Equipment, depreciation and disposal 96
8.13 Accruals 97
8.14 Bank accounts in another currency 98
8.15 Loans 99
8.16 Cost tags 100
8.17 Expense claims 100
8.18 Attachments chosen inside the forms 101
8.19 Where the code lives 102

9 Merchant money: wallets, payouts, receivables, invoices and profit 103
9.1 The merchant wallet and its sign 103

9.2 Merchant statements 104
9.3 Payouts 104
9.4 Receipts and adjustments 108
9.5 Who owes us, by age (first in, first out) 109
9.6 Payment reminders 109
9.7 Tax invoices, credit notes and debit notes 110
9.8 ETA electronic invoices 111
9.9 Security deposits 112
9.10 Merchant profit report 112
9.11 Where the code lives 113

10 Payroll and staff costs 114
10.1 Who is on the payroll 114
10.2 Salary packages 114
10.3 The rules, all from settings 114
10.4 The payslip calculation 115
10.5 The salary run and its posting 116
10.6 Advances and bonuses 117
10.7 Sales commission in payroll 118
10.8 Unpaid leave and violation deductions 118
10.9 Outsourced staff 118
10.10 Payslips 118
10.11 A full worked payslip 119
10.12 Where the code lives 120

11 Taxes, reports, reconciliation and closing 121
11.1 Periods and Cairo time 121
11.2 VAT report (per month of a year) 121
11.3 Monthly VAT return 121
11.4 Withholding tax, form 41 122
11.5 Salary tax and social insurance lists 122
11.6 Company income tax 123
11.7 Paying taxes 123
11.8 Financial statements 123
11.9 Drill down and comparisons 124
11.10 Profit per store and the fully loaded cost per order 125
11.11 Cash flow (direct method) 125
11.12 Four week cash forecast 125
11.13 Forecast: 12 month scenarios 126
11.14 Coming up and due 128
11.15 Budgets 129
11.16 Bank reconciliation 130
11.17 Month end checklist and closing the books 131
11.18 Closing a year 132
11.19 Exports 132
11.20 Where the code lives 134

12 UI development 135
12.1 App router structure 135
12.2 Fetching data 136
12.3 Translations, Arabic and right to left 137
12.4 Theme and brand tokens 137
12.5 Layout: apps, top bar and menus 140

12.6 ListView 142
12.7 DocumentFrame and DocumentPage 147
12.8 Forms: FormFooter and unsaved changes 152
12.9 DateInput, Amount and money parsing 153
12.10 StatCard, WalletCard and charts 154
12.11 DrillReportPage 157
12.12 InfoTip and the Accounting search 159
12.13 Printing and PDF generation 160
12.14 Push notifications 162

13 Screens: Dashboard and Money in 164
13.1 Entering Accounting: the app home 164
13.2 The top bar of Accounting 166
13.3 Dashboard 172
13.4 Approvals 175
13.5 Coming up and due 189
13.6 Merchant wallets 192
13.7 Pay merchants (the guided run) 196
13.8 Payouts list 204
13.9 A payout page: statement, proof, checks and mark paid 208
13.10 Tax invoices 225
13.11 Who owes us, by age (Receivables aging) 232
13.12 Cheques register 235
13.13 Emails to merchants 249
13.14 The merchant's side: Wallet with SOA and POT 250
13.15 Status summary 254

14 Screens: Money out 256
14.1 Record an expense 258
14.2 Suppliers 268
14.3 Supplier bills 274
14.4 Purchase orders 287
14.5 Money movements 302
14.6 Petty cash 313
14.7 Rider withdrawals 322
14.8 Expense claims 331
14.9 Riders (rider wallets) 340
14.10 Rider offices 342
14.11 Salaries and payslips 345
14.12 Recurring expenses 348
14.13 Prepaid and equipment (assets) 349

15 Screens: Cash and bank 354
15.1 Cash position 354
15.2 Store cash (handover differences and bank deposits) 357
15.3 Bank matching 362
15.4 Basata deposits 377
15.5 A Basata deposit 381
15.6 Record Basata transfer 385
15.7 Frozen riders 388
15.8 Missing receipts 391
15.9 Loans 393
15.10 A loan page 400

15.11 Rider app: My wallet 405
15.12 Rider app: Deposit at Basata 414
15.13 Store app: Cash from riders 422
15.14 Store app: Petty cash 427
15.15 Statuses on these screens at a glance 432

16 Screens: Reporting and Planning 434
16.1 All reports 434
16.2 Profit and loss, Balance sheet and Trial balance (the drill down report page) 435
16.3 Cash flow and the four week forecast 450
16.4 Profit per store 452
16.5 Merchant profit and the what if 453
16.6 Costs and income by tag 456
16.7 Payables aging 457
16.8 Compare periods (financial statements) 460
16.9 The journal list 461
16.10 Searching and filtering the journal 469
16.11 The journal entry page 476
16.12 Journal entry status 487
16.13 Forecast 493
16.14 Budgets 506

17 Screens: Month end, Setup and common parts 513
17.1 Close the month (guided run) 513
17.2 Month end checklist 518
17.3 Close the books 520
17.4 Year end and export 523
17.5 Taxes 525
17.6 Chart of accounts 531
17.7 Company tax details 536
17.8 Merchant contracts and deposits 538
17.9 Cost tags 540
17.10 Payout checks 541
17.11 Opening balances 543
17.12 Accounting settings 544
17.13 Common part: the list view 549
17.14 Common part: the document page frame 554
17.15 Common part: the attachment field and viewer 561
17.16 Common part: the form footer 563
17.17 Common part: the settings menu 564
17.18 Statuses and their colors 565
17.19 Access: the same screen for view and for use 567
17.20 Phone layout 570
17.21 Arabic, right to left 573
17.22 Dark theme 576

1 Introduction

1.1 Purpose of this document

This document explains how the accounting system of Shiply Now is built and how its user interface works. It is written for an outside developer who is building a similar system and wants to understand the design decisions, the data model, the rules the code enforces and the screens that sit on top of them. It is not a user manual: it describes how the parts fit together and why, with short excerpts of the real code.

Every statement in this document was checked against the source code of the monorepo. Where the code and an older design note differ, the code wins. Where a feature was still being finished when this document was written, the text says so in a marked note instead of describing it.

1.2 What Shiply Now is

Shiply Now is a 60 minute delivery service for merchants who sell on Shopify, run from small dark stores in Greater Cairo. A dark store is a small warehouse that is not open to the public. A merchant sends stock to a dark store before any customer orders. When a customer orders, the store only picks, packs and hands the parcel to a rider, so the delivery can reach the customer within the promised time.

The system is a standalone pilot. It covers the whole chain:

- dark stores (and hubs, which are sorting facilities) with delivery zones built from areas of Greater Cairo and opening hours;
- the merchant catalog, matched to Shopify variants by SKU;
- stock per store and bin, changed only through an append only log of stock movements;
- a Shopify app that registers one fulfillment location per merchant shop and dark store, receives fulfillment requests and reports stock and tracking;
- the order queue of each store with service level timers, and the staff screens that receive, pick, pack and hand over parcels;
- the rider screens that deliver, collect cash on delivery and take proof photos;
- an admin desk for operations, inventory, merchants, customers, human resources and sales;
- an accounting system: a double entry ledger in piastres, cash control, merchant payouts, supplier bills, taxes, payroll and reports.

Cash on delivery is common in Egypt, so a large part of the accounting is about cash that riders collect for merchants, how it reaches a store box or a bank, and how and when the merchant is paid back what is theirs after our fees.

1.3 Roles

Every user account has one role. The role decides which area of the web app a person lands in. Team roles then receive a finer access profile per app (see the chapter on controls and access).

Role	Who	Main area
`admin`	Team members who run the business (owner, managers, office staff). Their rights come from an access profile.	`/home` app tiles, then `/admin/...` and `/finance/...`
`finance`	Accounting staff. Also a team role with an access profile; by default they see only the Accounting app.	`/finance`
`store_staff`	People who work in a dark store or hub: receiving, picking, packing, cash handover, petty cash.	`/store/<store id>`
`rider`	Delivery riders. They see their deliveries, their cash, their wallet of earnings and their cash deposits.	`/rider`
`merchant`	Users of a merchant company (owner or member with rights). They see orders, products, shipments, wallet, invoices and contract of their own merchant only.	`/merchant`

The three team roles (`admin` , `finance` , `store_staff`) are listed in `TEAM_ROLES` in `packages/shared/src/access.ts` . Their real rights come from a per app access level (none, view, use, manage), so two admins can have very different rights. Riders and merchants keep their own fixed areas.

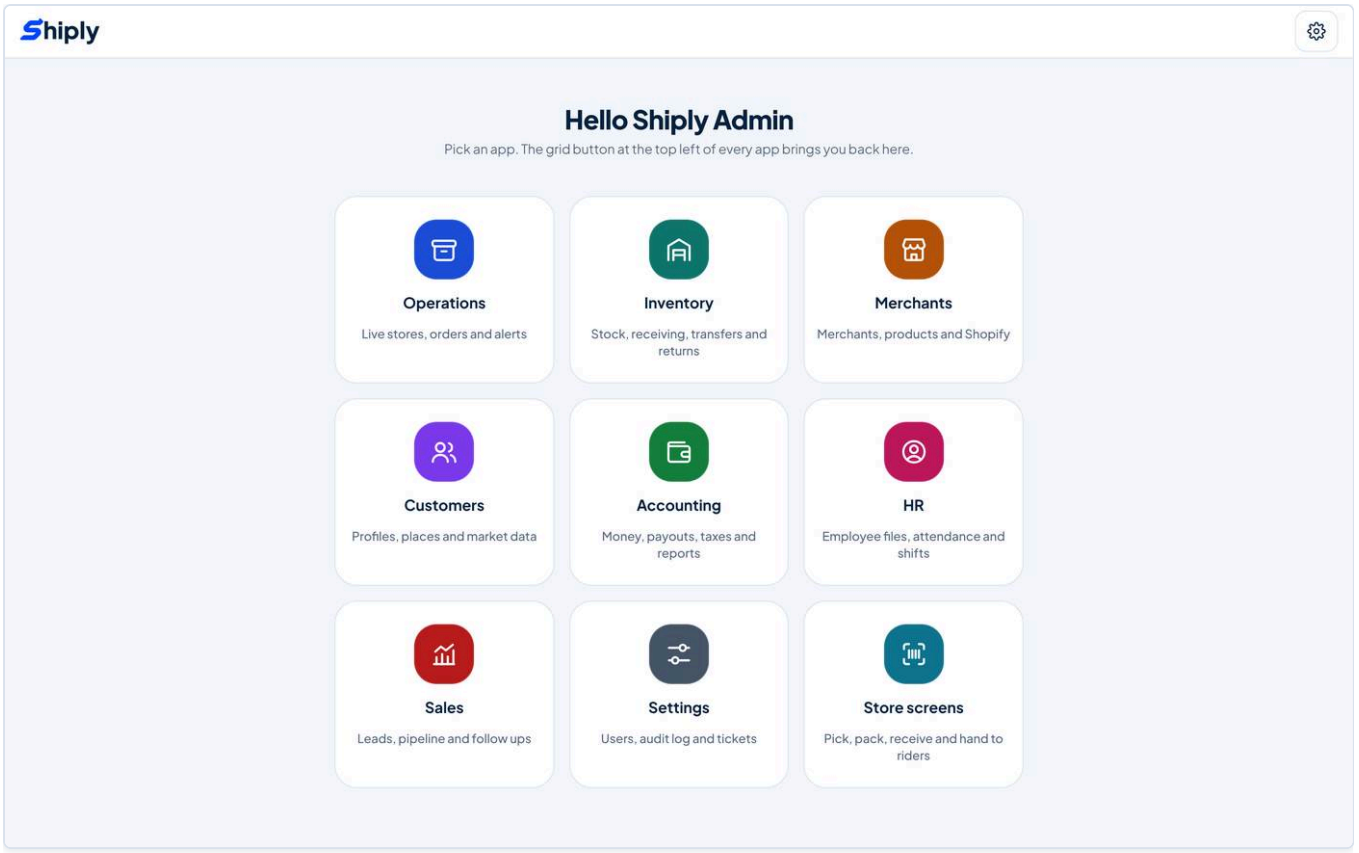
1.4 The apps

The desk for team members is organised like Odoo: a home screen of app tiles at `/home` , and inside each app a top bar with that app's menus. The apps are defined once in `apps/web/src/lib/apps.ts` :

App	Access app	What it holds
Operations	`operations`	Overview dashboard, orders, alerts, stores, zones, addresses.
Inventory	`inventory`	Stock levels, shipments received, transfers, returns to merchants, damage, counts, packaging.
Merchants	`merchants`	Merchants and their contracts, products, Shopify stores.
Customers	`customers`	Customer profiles (needs the customer data switch), places, market data.
Accounting	`accounting`	Everything under `/finance` : dashboard, money in, money out, cash and bank, reports, planning, month end, setup.
HR	`hr`	Employees, time and attendance, leave, discipline, inbox, organisation.
Sales	`sales`	Pipeline, leads, companies, proposals, targets, commission.
Settings	`settings`	Users and access, audit log, system settings.

A person who may use the store screens also gets a Store screens tile. Each app keeps its own Configuration page for its settings at `/admin/settings/<app>` , visible only with settings manage.

Merchants, riders and store staff do not use the app tiles. They have their own mobile first areas: `/merchant` (with a sidebar), `/rider` and `/store` . Every team member and rider also has My HR at `/my-file` (leave, payslips, expense claims). There is a public company website at `/` and a public tracking page for each order at `/t/<token>` .



The home screen at `/home` for a person with every app open: the grid of app tiles (Operations, Inventory, Merchants, Customers, Accounting, HR, Sales, Settings and Store screens), each tile in its own color with its icon.

1.5 How to read this document

The document moves from the foundation upward:

1. The architecture chapter shows the monorepo layout, how one request travels from the browser to the database and back, where background jobs run, how the system is deployed and tested, and a full map of every accounting file.
2. The ledger core chapter explains the double entry ledger: its tables, the one function that writes to it, the database triggers that guard it, how balances are read, and how the books are closed.
3. The chart of accounts chapter lists every account group and code and explains which accounts are opened by the system, which by finance, and which ones a manual journal entry may touch.
4. The controls and access chapter covers access profiles, the second person rule, the books lock, the audit log, idempotency and the error answers.
5. The UI development chapter explains how the web app is built: routing, data fetching, translation, the brand tokens, the app frame and every reusable building block (list views, document pages, forms, reports, charts, printing, push).

Each chapter can be read on its own, but the ledger core chapter is the base for everything about money. Code excerpts are short and real; file paths are relative to the repository root. Placeholders marked as screenshots show where a picture of the running interface belongs.

1.6 Conventions

Money in piastres

> **RULE**
> Money is stored as integers in piastres (1 Egyptian pound, EGP, is 100 piastres). Never as a floating point number. Values are converted to EGP only for display.

Every money column in the database ends in `_piastres` (for example `amount_piastres` on `ledger_lines`) and is an integer. The ledger refuses any line whose amount is not a safe integer. Percentages are stored as basis points, where 10,000 basis points are 100%: a fee of 2.5% is stored as `250` and VAT of 14% as `1400`. A share of an amount is computed in whole piastres, rounded half up, by one helper:

```
/** A share in basis points of an amount, rounded half up, in whole piastres. */
export function share(amount: number, basisPoints: number): number {
  return Math.floor((amount * basisPoints + 5_000) / 10_000);
}
```

Amounts typed by a person are converted from text to piastres without going through a float (`parseEgp` in `apps/web/src/lib/money.ts`, described in the UI chapter). Amounts are shown with two decimals and Latin digits, kept left to right inside Arabic text.

Display in EGP

The screens show EGP with two decimals, for example `1,249.50`. A negative amount shows in the danger color. Exports to CSV and Excel write money as EGP numbers with two decimals so that a spreadsheet can add them up.

Cairo time

The business runs on Cairo time, and Egypt has summer time, so the offset from UTC is not fixed (two hours in winter, three in summer). The rules are:

- Instants are stored in UTC (`occurred_at`, `created_at`).
- A business day is a Cairo calendar day written as `YYYY-MM-DD`. The helpers `cairoDay(instant)` and `cairoDayStart(day)` in `packages/shared/src/hours.ts` convert between the two and honour summer time. A fixed `+02:00` is never used.
- In SQL, a Cairo day or month is computed by converting from UTC first: `(occurred_at AT TIME ZONE 'UTC') AT TIME ZONE 'Africa/Cairo'`.
- Admin and accounting date filters take Cairo days. A money event dated on a past day is booked at noon Cairo time of that day.
- Date boxes in the interface are typed and shown day first (`dd/mm/yyyy`), as in Egypt, and sent to the API as `YYYY-MM-DD`.

VAT

VAT is added on top of our fees. Its rate is one global setting, `fee.vat_bp`, in basis points. The setting's code default is 0; the owner set it to 1400 (14%) on 11 November 2026. The same rate applies to rider office bills when the office has a tax number. VAT is never stored per merchant. When a fee is charged, the merchant's balance

goes down by the fee plus its VAT, the fee is booked as income and the VAT is booked as owed to the tax authority (account 2301). Fees and VAT are computed separately, each rounded to whole piastres.

Document numbers

Accounting documents carry gapless serial numbers with a prefix, padded to six digits: `JV-000001` for a journal entry, `PAY-` for payouts, `CH-` for cash handovers, `BILL-`, `INV-`, `CN-`, `DN-`, `PO-`, `CHQ-`, `PC-`, `RW-`, `BS-`, `EC-`, `BD-` and others. Numbers are taken inside the transaction that creates the document, under a database lock, so there are no gaps and no duplicates.

Language and wording

The interface exists in English (left to right) and Arabic (right to left). Every string goes through translation files; none is written into a component. The code and its comments use plain words. This document follows the same habit, and uses the words of the code (for example "wallet" for an account that belongs to one merchant, rider, supplier or team member).

The latest accounting features

The last group of features added to Accounting is described in full where each belongs:

- Merchant payouts carry a statement of account (`SOA-`) frozen when they are prepared and a proof of transfer (`POT-` , the bank slip with its amount and reference) uploaded before they are paid; Accounting ticks payout checks in between, one of them by a second person who will not pay. See Merchant money (chapter 13) and Controls and access.
- Suppliers have their own list and document page, and purchase orders can buy assets: each unit received becomes a fixed asset depreciated from the month it arrived (chapter 12).
- Receipts and documents are chosen inside every money form and linked to the record in the same transaction; every journal entry has its own page with a printable voucher (chapter 12).
- The postal fee: the postal authority's share of our fees per shipment, our cost, paid monthly from Taxes (chapter 10).
- Budgets are planned per quarter, a 12 month forecast works with named scenarios, and Coming up and due lists every known payment with phone reminders (chapter 15).

2 Architecture

Shiply Now is one TypeScript monorepo managed with pnpm workspaces. Strict mode is on everywhere. One database (PostgreSQL 16 with PostGIS) holds every record, and one Docker image runs both servers. This chapter describes the layout, the path a request takes, background jobs, deployment and tests, then maps every file that belongs to the accounting system.

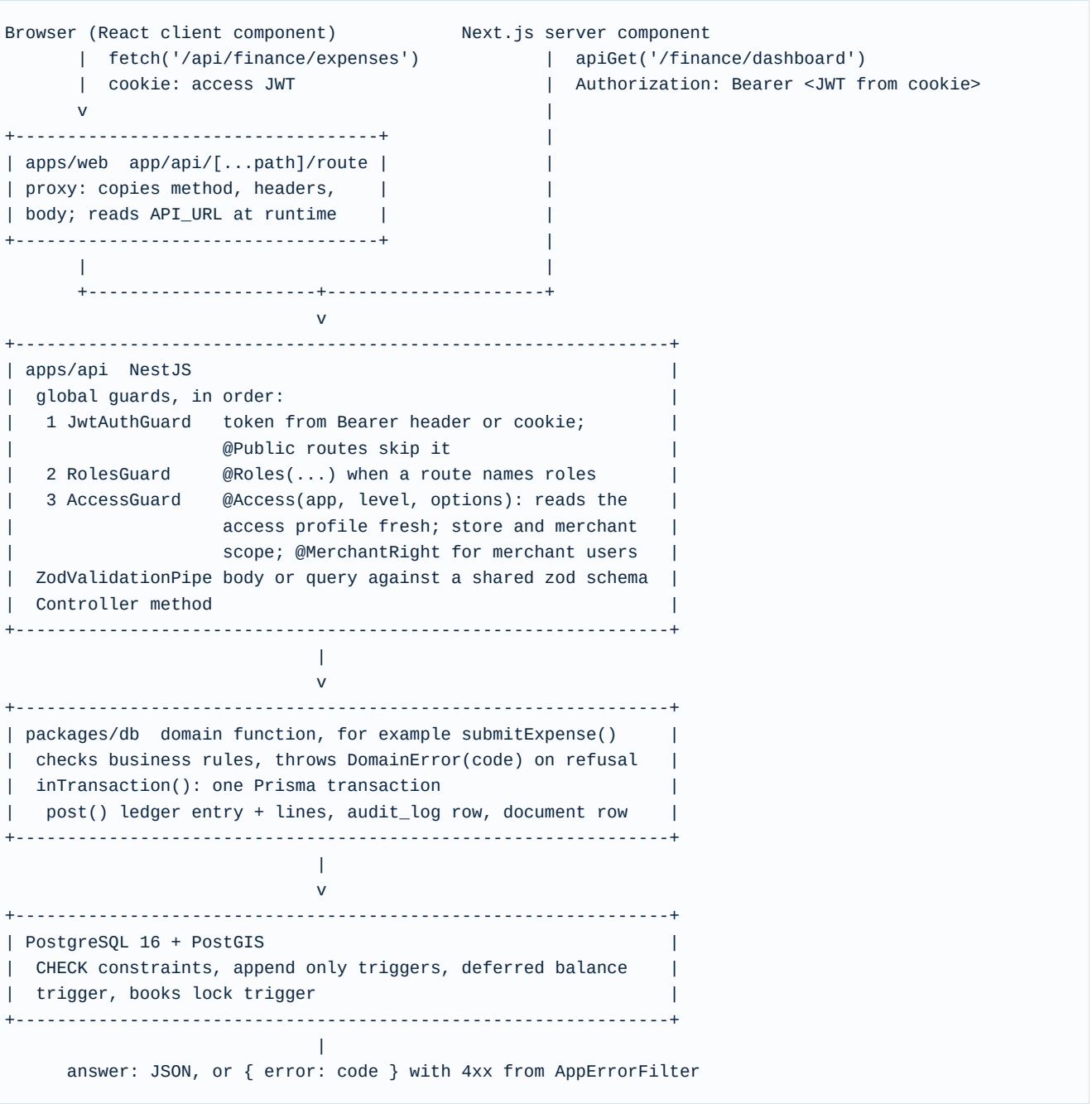
2.1 Monorepo layout

Folder	Package	What it is
`apps/api`	`@shiplynow/api`	The REST API, built with NestJS 11. Controllers check access and validate input, then call domain functions from `packages/db`. OpenAPI documentation is generated at `/docs`. Background jobs run inside this process.
`apps/web`	`@shiplynow/web`	The Next.js app (app router). It holds every screen: the public site, `/home` and the admin apps, `/finance`, `/store`, `/rider`, `/merchant`, `/my-file` and the public tracking page. It never talks to the database; it calls the API.
`apps/shopify`		The Shopify app configuration and the theme app extension (the storefront zone widget). The server side of the Shopify integration lives in `apps/api/src/shopify`.
`packages/db`	`@shiplynow/db`	The Prisma schema and migrations, and every domain function that reads or writes the database: stock, orders, the ledger and the rest of accounting, HR, sales, push. Each change runs in a transaction.
`packages/shared`	`@shiplynow/shared`	Pure code used by both the API and the web app: zod schemas of request bodies, constants, setting definitions with their defaults, access rules, Cairo time helpers, list view state, pure calculations (tax bands, contract prices, aging buckets, bank match ranking, export files).
`deploy`		Start scripts of the production container (`start.mjs`, `prepare.mjs`, `migrations-current.mjs`).
`docs`		Design notes (architecture, access, HR, sales, zones, deployment, backups, load test, standard operating procedures).
`scripts`		Tools: the Shopify tunnel, the merchant app generator, the load test, the electronic invoice signer for the finance PC, and the SOP builder.

The dependency direction is strict: `shared` depends on nothing of ours; `db` depends on `shared`; `api` depends on both; `web` depends only on `shared` (its data comes over HTTP). Because business rules live in `packages/db`, the seed, the demo loader, the API and the background jobs all use exactly the same implementation of each rule.

2.2 How a request flows

The browser only ever talks to the web app's origin. The web app forwards every `/api/...` call to the API, so the authentication cookies stay first party. Server components of the web app call the API directly with the user's token.



The /api proxy

The file `apps/web/src/app/api/[...path]/route.ts` is a route handler that exports the same function for every HTTP method. It rebuilds the target address from `API_URL` (read at runtime, so one build works on any host), removes hop by hop headers, forwards the body as bytes, and copies the answer back keeping each `Set-Cookie` header separate. A network failure answers `502 { error: 'api_unreachable' }`. A Next.js rewrite is not used because it would freeze the API address at build time.

Middleware

`apps/web/src/middleware.ts` runs on the Node.js runtime (so the JWT secret is read at runtime and never bundled) for `/admin`, `/store`, `/rider`, `/merchant`, `/finance` and `/home`. It reads the access token's claims and calls `decideAccess` (`apps/web/src/lib/access.ts`), which answers allow, send to login, refresh the session first, or redirect to the person's own area. On allow it adds the path in a request header so the app frame can show not found for a page the person may not open.

Authentication

Sign in sets an httpOnly access JWT cookie (HS256) and an opaque refresh token stored hashed and rotated on every use. `JwtAuthGuard` (`apps/api/src/auth/jwt-auth.guard.ts`) accepts the token from an `Authorization: Bearer` header (used by server components) or from the cookie (used through the proxy), verifies it and puts `{ sub, email, role }` on the request. A missing or bad token answers `401 { error: 'unauthenticated' }`.

Guards and decorators

The three guards are registered globally in `apps/api/src/auth/auth.module.ts` with `APP_GUARD`, so no route can forget them. The decorators in `apps/api/src/auth/decorators.ts` are:

- `@Public()`: skips authentication. Used for routes checked another way (Shopify signatures, Slack signatures, the public tracking page, the website lead form).
- `@Roles(...roles)`: the route is for these roles only.
- `@Access(app, level?, options?)`: the route is for team members whose access profile reaches `level` in `app`. Without a level, `GET` and `HEAD` need view and other methods need use. Options add `customerData`, `salaries`, a store or merchant route parameter to check, and further levels in other apps (`also`). A method decorator overrides the class decorator.
- `@MerchantRight(right)`: a merchant member needs that right (owners have all); `'owner'` limits the route to owners.
- `@CurrentUser()` and `@CurrentAccess()`: parameter decorators that give the handler the claims and the access profile the guard loaded.

Every accounting controller carries `@Access('accounting')` on the class, and stronger levels on single methods:

```
@Access('accounting')
@Controller('finance')
export class ControlsController {
  @Post('requests/:id/decide')
  @Access('accounting', 'manage')
  async decide(@Param('id') id: string, @Body(new ZodValidationPipe(decideRequestSchema)) body,
@CurrentUser() user) {
    ...
  }
}
```

A test (`apps/api/test/route-access.test.ts`) fails when any route has none of `@Public`, `@Access` or `@Roles`, unless it is on an explicit list of routes open to any signed in user.

Domain functions and transactions

A controller does little more than call one domain function with the caller's user id. Domain functions take a `Db`, which is either the Prisma client or a transaction client, and run their work through `inTransaction`:

```
/** Either the full client or the client handed to a $transaction callback. */
export type Db = PrismaClient | Prisma.TransactionClient;

/** Runs fn in a transaction; joins the caller's transaction when db already is one. */
export function inTransaction<T>(db: Db, fn: (tx: Prisma.TransactionClient) => Promise<T>) {
  if ('$transaction' in db) return db.$transaction(fn);
  return fn(db);
}
```

This lets a delivery, its ledger postings, its order events and its audit rows share one transaction: `completeDelivery` in `orders.ts` calls `postDeliveryCompleted(tx, ...)`, which calls `post(tx, ...)` several times, all inside the same transaction. Either everything is written or nothing is.

Refusals and errors

A business rule refusal throws `DomainError(code, status = 409, details?)` from `packages/db/src/domain-error.ts`. The global filter `AppErrorFilter` (`apps/api/src/common/domain-error.filter.ts`) turns it into `{ error: code, ...details }` with that status. Validation failures answer `400 { error: 'validation_failed', issues }`. HTTP errors keep their usual answer. Anything unexpected answers 500 and raises one `server_error` admin alert per source and message through `ErrorReporter`, storing only the method and path, never bodies or query strings.

2.3 Background jobs

There is no queue service and no Redis. `JobsService` (`apps/api/src/jobs/jobs.service.ts`) starts two timers when the API boots: one every 30 seconds for the business jobs and one every 10 seconds for web push. Each job:

- opens a transaction and takes a Postgres transaction level advisory lock with `pg_try_advisory_xact_lock(<fixed id>)`; if another API instance holds it, this run does nothing;
- is idempotent: every ledger posting it makes has an idempotency key (for example `storage:<day>:<store>:<merchant>`), so running twice changes nothing;
- remembers in memory the last Cairo day or slot it finished, to skip needless work, but never relies on that memory for correctness;
- reports a thrown error through `ErrorReporter` instead of crashing.

```
const result = await this.prisma.$transaction(async (tx) => {
  const [{ locked }] = await tx.$queryRaw`SELECT pg_try_advisory_xact_lock(${STORAGE_LOCK}) AS locked`;
  if (!locked) return null;
  return chargeStorageDays(tx, now);
}, { timeout: 60_000 });
```

The accounting jobs are: the daily storage charge (with catch up of missed days), the monthly finance run (recurring expenses prepared as requests, prepayment releases and depreciation, accrual reversals), contract monthly minimum fees and contract payout schedules, the rider cash overdue check every ten minutes, payment reminders to merchants in debit, and the Basata cash deposit deadline. The electronic invoice sync

(`EtaSync` in `apps/api/src/finance/eta.ts`) has its own one minute timer and sends nothing without its credentials. Merchant emails are sent from an outbox by the same job loop. Jobs can be switched off with `JOBS_ENABLED` ; they are off by default in tests.

2.4 Deployment

One Docker image runs everything. `deploy/start.mjs` :

1. checks that the database address and the JWT secret are set, and optionally swaps the database name (`DATABASE_NAME`);
2. runs `deploy/migrations-current.mjs` ; when every migration folder is already applied it skips `prisma migrate deploy` (a sleeping free instance wakes faster), otherwise it applies the migrations;
3. runs `deploy/prepare.mjs` in a separate process: the one time fresh start when asked, the seed (which only adds missing rows and never overwrites admin edits), and the demo month when `DEMO_LOAD` is true;
4. starts the API on `127.0.0.1:API_PORT` (inside the container only) and the web app on `$PORT` (the only public door). If either stops, the container stops so the host restarts it.

One `.env` file at the repository root serves every app in development; `.env.example` lists every variable. Secrets are never committed. Backups are the database provider's restore window plus a nightly encrypted dump kept as a CI artifact (see `docs/BACKUP.md`).

2.5 Testing

Package	Runner	What runs
`packages/shared`	Vitest	Pure unit tests (tax bands, aging buckets, list state, report periods, contract pricing, bank statements).
`packages/db`	Vitest against a real PostgreSQL with PostGIS	Domain tests per area (`finance.test.ts` , `finance-controls.test.ts` , `books.test.ts` , `database-rules.test.ts` and many more). They use `TEST_DATABASE_URL` , never the normal database address, because every file wipes the tables. A global setup applies the migrations first. Files run one after another.
`apps/api`	Vitest with SWC and Supertest	The whole Nest app is built with `createTestApp` (`apps/api/test/helpers.ts`), which can replace each external boundary with a fake: `Clock` , `Geocoder` , `ShopifyAdmin` , `EtaClient` , `Slack` , `PushSender` , `Mailer` , `Overpass` . Requests go through the real guards and filters.
`apps/web`	Vitest	Pure helpers (list view, money parsing, report view, close run), the app menu rules, the translations (same keys in both languages, no dots in keys) and the WCAG contrast test of the brand tokens.

Every step of work ends with lint, typecheck and tests (`pnpm lint` , `pnpm typecheck` , `pnpm test`). The project rules also name Playwright for key web flows, but no Playwright suite is present in the repository at the time of writing.

2.6 Accounting code map

The tables below list every file that belongs to the accounting system, with what it holds and its main exported functions.

packages/db/src/finance

File	What it holds	Main exports
`ledger.ts`	The ledger core: account references, system accounts with their codes, default accounts, account code sequences, gapless numbers, the only posting function, balances.	`post`, `assertPeriodOpen`, `balanceOf`, `lockedBalance`, `balancesByType`, `share`, `nextAccountCode`, `nextNumber`, `accountKey`, `ensureDefaultAccounts`, `DEFAULT_ACCOUNTS`, `CATEGORY`, `assertActiveTag`
`books.ts`	The company books on top of the ledger: chart of accounts, opening and closing accounts, money movements between banks and boxes, expenses, owner capital and drawings, other income, free journal entries, account statement, trial balance, profit and loss, balance sheet.	`chartOfAccounts`, `createAccount`, `updateAccount`, `openAccount`, `mustHold`, `transferMoney`, `recordExpense`, `recordCapital`, `recordOtherIncome`, `checkJournalEntry`, `recordJournalEntry`, `accountStatement`, `trialBalance`, `profitAndLoss`, `balanceSheet`, `moneyAccounts`, `isWallet`, `eventTime`, `natural`, `ownerNames`
`postings.ts`	Postings made by operations events in the event's own transaction: delivery completed (COD to merchant, every fee kind, rider pay, office commission), failed delivery fee, receiving fee, return fee, daily storage.	`postDeliveryCompleted`, `postDeliveryFailed`, `postReceivingFee`, `postReturnFee`, `accrueStorage`, `lastClosedDay`
`operations.ts`	Rider cash and handovers to stores, bank deposits from store boxes, what is owed to merchants, payouts (create, mark paid, cancel), merchant receipts and adjustments.	`riderCash`, `cashToHandIn`, `recordCashHandover`, `reviewCashHandover`, `recordBankDeposit`, `merchantOwed`, `payoutDueOn`, `createPayout`, `assertPayoutOwed`, `markPayoutPaid`, `cancelPayout`, `recordMerchantReceipt`, `recordMerchantAdjustment`, `bankAccount`, `riderBonusOwed`

File	What it holds	Main exports
`controls.ts`	Controls on the books: books lock, the second person rule (requests), payout approval, receipt photos, opening balances, and the submit wrappers that post at once or queue a request.	`booksLock`, `closeBooks`, `approvalThreshold`, `queueRequest`, `submitMovement`, `submitExpense`, `submitDrawings`, `submitCapitalIn`, `submitOtherIncome`, `submitMerchantReceipt`, `submitMerchantAdjustment`, `submitJournalEntry`, `submitAdvance`, `submitPayroll`, `submitBillPayment`, `submitTaxPayment`, `submitPrepayment`, `submitAssetPurchase`, `submitPackagingPurchase`, `submitStaffBonus`, `submitCreditNote`, `submitAssetDisposal`, `listFinanceRequests`, `decideFinanceRequest`, `cancelFinanceRequest`, `payoutNeedsApproval`, `approvePayout`, `attachReceipt`, `receiptsFor`, `missingReceipts`, `recordOpeningBalances`, `RECEIPT_KINDS`
`reports.ts`	Wallet lists, merchant statements, ledger entry list, cash handover list, payout list.	`financeWallets`, `merchantStatement`, `listLedgerEntries`, `listCashHandovers`, `listPayouts`
`payout-run.ts`	The guided Pay merchants run; nothing is stored for the run itself.	`prepareDuePayouts`, `payoutRun`, `markPayoutsPaid`, `PAY_RUN_STEPS`
`payables.ts`	Suppliers, bills (also in another currency), bill payments with withholding tax, bill ageing, withholding report, taxes owed and tax payments.	`createSupplier`, `updateSupplier`, `listSuppliers`, `recordBill`, `cancelBill`, `payBill`, `withholdingFor`, `listBills`, `billAgeing`, `whtReport`, `taxesOwed`, `payTax`, `rateMicro`, `centsToPiastres`, `TAXES`
`assets.ts`	Prepayments and equipment, monthly release and straight line depreciation.	`recordPrepayment`, `recordAssetPurchase`, `runMonthlyCharges`, `chargeDepreciation`, `listPrepayments`, `listFixedAssets`, `monthlyPart`
`tools.ts`	Merchant bank details, the bulk payout bank file, equipment sold or scrapped, accrual reversal.	`updateMerchantBank`, `payoutBankFile`, `disposeAsset`, `reverseAccruals`
`recurring.ts`	Monthly recurring expenses, prepared as requests on their day.	`createRecurring`, `setRecurringActive`, `listRecurring`, `runRecurring`
`bank.ts`	Bank statement import with fingerprints, automatic and manual matching, bank charges, setting lines aside, reconciliation view.	`importStatement`, `autoMatch`, `matchBankLine`, `matchBankLines`, `reopenBankLine`, `ignoreBankLine`, `bookBankCharge`, `listStatements`, `reconciliation`, `bankMatching`
`period.ts`	Period reports and the year end: VAT report, budget against actual, cash flow and four week forecast, year close, journal lines export, receipt photos for the audit pack.	`vatReport`, `setBudget`, `budgetVsActual`, `cashFlow`, `cashForecast`, `yearCloseStatus`, `closeYear`, `journalLines`, `receiptPhotos`, `VAT_PURCHASE_KINDS`

File	What it holds	Main exports
`budgets.ts`	Budgets planned per quarter (equal split of a quarter into its months, months changed by hand), the year against actual per month, quarter and year, copying a year with a percent.	`setQuarterBudget`, `budgetYear`, `copyBudgetYear`
`drill.ts`	Drill down reports (profit and loss, balance sheet, trial balance with comparison) and the ledger lines list behind any report line.	`drillReport`, `ledgerLinesList`, `LEDGER_LINES_DEF`, `ledgerLinesGaps`
`profit.ts`	Profit by store, merchant or rider from the line tags; cost per delivered order.	`profitBy`, `costPerOrder`
`merchant-profit.ts`	What each merchant earns us in a Cairo month against what serving them costs.	`merchantProfitability`, `merchantProfitDetail`
`cost-tags.ts`	User defined cost tags and the report by tag.	`listCostTags`, `createCostTag`, `updateCostTag`, `costTagReport`
`fx.ts`	Bank accounts in USD or EUR: rates, both balances, book value transfers, month end revaluation.	`fxRateMicro`, `impliedRateMicro`, `bothBalances`, `bookValueOut`, `foreignBalances`, `revalueForeignBank`, `foreignBanks`
`loans.ts`	Bank loans: one 2700 account each, instalment schedule, payments in order, interest.	`recordLoan`, `submitLoan`, `payLoanInstalment`, `submitLoanPayment`, `listLoans`, `loanDetail`, `loanInstalmentsDue`
`cheques.ts`	The cheques register: book, deposit, clear, bounce, cancel.	`registerCheque`, `bookCheque`, `depositCheque`, `clearCheque`, `bounceCheque`, `cancelCheque`, `dropChequeRequest`, `listCheques`, `chequesDue`, `getCheque`
`contracts.ts`	Merchant contracts as versions with every price, tiers, terms; prices in force on a day; minimum fees; credit limit; security deposits.	`contractPricesAt`, `versionInForce`, `pricesForDay`, `standardPrices`, `productPricesFor`, `payoutMinimumFor`, `deliveryAttemptsFor`, `createContractDraft`, `updateContractDraft`, `activateContractVersion`, `applyContractSchedules`, `chargeMinimumFees`, `assertWithinCreditLimit`, `recordDeposit`, `submitDeposit`, `listContracts`
`invoices.ts`	Monthly tax invoices per merchant from booked fees, credit notes, company tax profile, the electronic invoice (ETA) flow.	`prepareInvoices`, `monthlyFees`, `issueCreditNote`, `creditNoteVat`, `listInvoices`, `getInvoice`, `companyProfile`, `updateCompanyProfile`, `updateMerchantTax`, `etaReadiness`, `documentsToSign`, `saveSignatures`, `recordEtaResult`, `resendInvoice`, `nextDocumentNumber`, `FEE_KINDS`
`debit-notes.ts`	Debit notes linked to an invoice.	`debitNoteAmounts`, `issueDebitNote`, `submitDebitNote`

File	What it holds	Main exports
`returns.ts`	Tax return worksheets: VAT return, withholding list (form 41), payroll tax and insurance lists, company income tax estimate and booking, comparative statements.	`vatReturn`, `withholdingPayments`, `payrollForms`, `corporateTaxEstimate`, `bookCorporateTax`, `comparativeStatements`
`payroll.ts`	Salary run from HR packages with Egyptian deductions, staff advances, staff bonuses, payslips.	`payrollRules`, `listStaffPay`, `giveAdvance`, `previewPayroll`, `runPayroll`, `payslips`, `payStaffBonus`, `myPayslips`, `myPayslip`
`expense-claims.ts`	Employee expense claims: manager step, accounting pays.	`submitExpenseClaim`, `managerDecideClaim`, `cancelExpenseClaim`, `rejectExpenseClaim`, `payExpenseClaim`, `claimCategories`, `setClaimableCategory`, `myExpenseClaims`, `expenseClaimDetail`, `claimsWaitingForManager`
`rider-wallet.ts`	Rider pay wallet: hold days, what a rider owes, withdrawals (request, approve, reject, pay), shortage and penalty charges.	`riderWallet`, `riderWalletTotals`, `riderOwed`, `requestWithdrawal`, `cancelWithdrawal`, `approveWithdrawal`, `rejectWithdrawal`, `payWithdrawal`, `listRiderWithdrawals`, `riderWalletSummaries`, `chargeShortageToRider`, `chargeRiderPenalty`, `chargeRiderPenalties`
`rider-offices.ts`	Rider supply offices: commission at each delivery, month end office bills, daily office account.	`postRiderOfficeCommission`, `setRiderOffice`, `prepareOfficeBills`, `listRiderOffices`, `riderOfficeDetail`, `riderOfficeAccount`, `riderOfficeOf`
`cash-control.ts`	Rider cash limit and age, cash position, petty cash at stores, the month end checklist.	`checkRiderCashLimit`, `checkRiderCashOverdue`, `ridersHoldingCash`, `storeRiderCash`, `cashPosition`, `submitPettyCash`, `postApprovedPettyCash`, `reversePettyCash`, `listPettyCash`, `pettyCategories`, `setPettyCategory`, `monthEndChecklist`, `markMonthEndItem`, `unmarkMonthEndItem`
`basata.ts`	Rider cash deposits through Basata branches, the daily deadline, frozen riders, Basata transfers to the bank.	`submitBasataDeposit`, `approveBasataDeposit`, `rejectBasataDeposit`, `runDepositDeadline`, `assertNotFrozen`, `unfreezeRider`, `listFrozenRiders`, `recordBasataSettlement`, `listBasataSettlements`, `basataPosition`
`storage.ts`	Units per store, merchant and Cairo day; daily storage charge with catch up.	`unitsAtDayEnds`, `unitsForDays`, `chargeStorageDays`, `unitDays`
`aging.ts`	Receivables and payables by age as of a Cairo day.	`receivablesAging`, `payablesAging`
`receivables.ts`	Merchants whose wallet turned debit and the payment request notification.	`merchantsOwingUs`, `requestMerchantPayment`

File	What it holds	Main exports
`reminders.ts`	Daily payment reminders to merchants in debit.	`reminderRules`, `runReceivableReminders`, `sendReminderNow`
`lists.ts`	Server side of the Accounting list views: one definition and loader per list.	`FINANCE_LIST_DEFS`, `queryFinanceList`, `financeListGaps`
`documents.ts`	Document page descriptors for every accounting document.	`accountingDocument`
`dashboard.ts`	The Accounting dashboard: today strip and journal cards with 30 day sparklines.	`financeDashboard`, `SPARK_DAYS`
`search.ts`	The one search of the Accounting app.	`financeSearch`
`bulk.ts`	Bulk approve of requests and payouts, each record on its own.	`bulkDecideRequests`, `bulkApprovePayouts`
`payments-due.ts`	Coming up and due: every known payment (recurring expenses, bills, loan instalments, salaries, taxes, postal fees, merchant payouts) with its due Cairo day, grouped overdue, 7 days, 30 days, later. Read only.	`paymentsDue`, `dueGroupOf`
`forecast.ts`	The 12 month forecast scenarios: opening balances, the suggestion from the last complete months, drivers and overrides, approval, comparison, use as budget. Never posts.	`forecastOpening`, `forecastSuggestion`, `listForecasts`, `forecastDetail`, `createForecast`, `saveForecastDrivers`, `saveForecastOverrides`, `renameForecast`, `setForecastStatus`, `deleteForecast`, `compareForecasts`, `forecastBudgetPreview`, `forecastToBudget`
`payout-proof.ts`	Payout statement of account (frozen at prepare), proof of transfer, payout checks, the ready to pay check, the statement CSV.	`buildPayoutStatement`, `freezePayoutStatement`, `getPayoutStatement`, `addPayoutProof`, `getPayoutProofFile`, `ensurePayoutChecks`, `listPayoutChecks`, `createPayoutCheck`, `updatePayoutCheck`, `markPayoutCheck`, `assertReadyToPay`, `payoutControls`, `payoutStatementCsv`
`attachments.ts`	Receipts and documents chosen inside finance forms: upload unlinked, link in the record's transaction, files of entries, payouts and requests, hourly purge of unused uploads.	`saveFinanceFile`, `discardFinanceFile`, `linkAttachments`, `withAttachments`, `claimRecordPhoto`, `movementTarget`, `attachmentsFor`, `photoAttachment`, `purgeUnlinkedUploads`

File	What it holds	Main exports
`index.ts`	Exports again the public functions of the folder for `@shiplynow/db`.	

packages/db/src, outside the finance folder

File	Accounting role
`client.ts`	`Db` type and `inTransaction`.
`domain-error.ts`	`DomainError`, the business refusal with a code and status.
`audit.ts`	`recordAudit`: one `audit_log` row in the caller's transaction; the audit list used by History.
`settings.ts`	`getSetting`: a setting's value (store override, then global, then the code default).
`orders.ts`	Calls `postDeliveryCompleted` and `postDeliveryFailed` inside the delivery transaction.
`inbound.ts`	Calls `postReceivingFee` when a shipment is closed.
`returns.ts`	Calls `postReturnFee` when a return is handed to the merchant.
`damage.ts`	Posts `damage_compensation` and `damage_rider_charge` when admin settles a damage report.
`packaging.ts`	Packaging stock bought, used (`packaging_used` at pack time) and adjusted.
`purchasing.ts`	Purchase orders with packaging, cost and asset lines: receipts (`po_receipt`, one fixed asset per asset unit) and the bill from an order (`supplier_bill`).
`records.ts`	Chatter back end: record references and scope, notes, scheduled activities.
`list-query.ts`	`queryList`, `listStateOf`, `listGaps`, `ListScope`: the list helper every list endpoint uses.
`push.ts`	Web push dispatch, including finance approvals, rider wallet notices and the `payment_due` reminders of Coming up and due.
`email.ts`, `email-templates.ts`	Merchant email outbox (statements, payout paid with the SOA PDF and the POT file attached, invoices, payment reminders).
`crm-sales.ts`	Sales commission, paid through the salary run or as a staff bonus.
`hr-pay.ts`, `hr-exit.ts`, `hr-violations.ts`	Salary packages read by payroll, the exit settlement, final violation deductions.
`fresh-start.ts`, `demo/`, `seed/`	The one time wipe before real use, the demo month loader, and the seed (default accounts through `ensureDefaultAccounts`).

packages/shared/src

File	What it holds
`accounts.ts`	Account categories and groups (`ACCOUNT_GROUPS` , `accountGroup` , `debitNormal`), document number formats (`entryNumber` , `payoutNumber` , `handoverNumber` , `billNumber`), zod schemas of accounts, transfers, expenses, capital, other income, journal entries, suppliers, bills, payments, taxes, recurring, prepayments, assets, budgets, company profile, merchant tax and bank details, credit notes, asset disposal.
`finance.ts`	Payout schedules, schemas of handovers, deposits, payouts, merchant receipts and adjustments, books lock, request decisions, receipts, opening balances, advances, bonuses, payroll; `LEDGER_KINDS` .
`settings.ts`	`SETTING_DEFINITIONS` : every business value with its zod schema, default, description and whether a store may override it (fees, VAT, approval limit, cash limits, payroll rates).
`hours.ts`	Cairo time: `cairoDay` , `cairoDayStart` , `cairoAt` , `addDays` , `isCalendarDay` .
`access.ts`	Access apps and levels, job role templates, `canDo` , `canUseStore` , `canUseMerchant` .
`record-apps.ts`	`RECORD_APPS` : which app's people may read a record's history, notes and activities.
`list-view.ts`	List view state in the address, `runList` , `ListDef` , favourites and bulk schemas.
`finance-lists.ts`	`FINANCE_LISTS` : each Accounting list's filters, groupings, sorts, sums and default filters.
`app-lists.ts`	`APP_LISTS` : the same for lists of the other apps.
`documents.ts`	`ACCOUNTING_DOCS` , `DocDescriptor` , `DOC_PREFIXES` , `parseDocNumber` , `DOC_LISTS` .
`report-period.ts`	Report periods, presets and comparison ranges; drill report types; ledger lines query.
`bank-match.ts`	`rankSuggestions` , `sumMatches` : bank line match suggestions.
`statement.ts`	Reading bank statement dates and amounts in many shapes; import schemas.
`export-file.ts`	`exportCsv` and `exportXlsx` (a stored zip written by hand, no library).
`aging.ts`	Aging bands and buckets, oldest first matching of open debits, reminder rules, debit note schema.
`budgets.ts`	Quarters, `splitQuarter` , `isEvenSplit` , `scaleBudget` , `variancePercentBp` , quarter and copy schemas.
`cash-control.ts`	Cash age buckets, next payout day, petty cash schemas, month end items.
`rider-wallet.ts`	Withdrawal methods and statuses, hold rules, `availableFrom` , `walletTotals` , `withdrawalDeductions` .
`loans.ts`	Loan schedule (declining or flat), Egyptian banks list, cost tag schemas, currencies.
`basata.ts`	Basata deposit and settlement numbers, schemas, deadline helpers.
`expense-claims.ts`	Expense claim numbers, statuses and schemas; payslip line type.
`payroll.ts`	Egyptian salary deductions from settings: `yearlyIncomeTax` , `monthlyPayslip` .

File	What it holds
`purchasing.ts`	Email kinds, packaging schemas, purchase order numbers, statuses and schemas.
`contracts.ts`	Cheques register types and schemas, contract prices and tiers, failed delivery rule, contract calculator and what if.
`eta.ts`	The ETA electronic invoice document and its canonical text for signing.
`forecast.ts`	The forecast model: drivers, `forecastCompute` (profit and loss, balance sheet, cash flow per month in whole piastres), periods, budget rows, `mulDiv`, `FC-` numbers and schemas.
`payments-due.ts`	Payment due kinds and groups, `paymentsDueQuerySchema`.
`payout-proof.ts`	`soaNumber`, `proofNumber`, proof file types and size, `SECOND_PERSON_CHECK`, check schemas, `PayoutStatementData`, `STATEMENT_KIND_ORDER`, `payoutMatches`.
`attachments.ts`	`attachmentIdsField` (at most 10 ids, used by every finance form schema).

apps/api/src/finance and related controllers

File	Routes and role
`finance.module.ts`	Registers every finance controller and the ETA sync.
`finance.controller.ts`	`/finance`: overview, cash handovers and their review, bank deposits, payouts (list, create, prepare due, mark paid, cancel), merchant receipts and adjustments, payout schedule, merchant statements and CSV, owed amount, ledger entries and CSV.
`books.controller.ts`	Chart of accounts, money accounts, open and update accounts, account statement and CSV, transfers, expenses, capital, other income, journal entries, profit and loss, balance sheet, trial balance.
`controls.controller.ts`	Controls summary, books lock (manage), requests list, decide (manage), cancel, payout approve (manage), attachments, missing receipts, opening balances (accounting and settings manage).
`views.controller.ts`	Dashboard, payments due, journal voucher PDF, list views `GET /finance/lists/:id`, document pages `GET /finance/documents/:type/:id`, search, bulk decide requests and bulk approve payouts (manage).
`runs.controller.ts`	Drill reports and their PDF, ledger lines, bank matching and match many, the Pay merchants run and its mark paid.
`payables.controller.ts`	Suppliers, bills, bill payments, ageing, taxes and withholding, tax payments, recurring expenses, prepayments, assets, receivables and payment requests.
`period.controller.ts`	Bank statements (read file, import, list), reconciliation and line actions, VAT report, budgets, cash flow and forecast, year close (manage), journal CSV, audit pack zip, tax return worksheets, comparative statements.
`invoices.controller.ts`	Invoices, prepare a month, credit notes, resend, ETA sync and readiness, company profile, signer key (owners), merchant tax details, documents to sign and signatures; `EtaSignerController` at `/eta-signer` for the signing program.

File	Routes and role
`cheques.controller.ts`	Cheques register and actions, contracts overview, security deposits, merchant profit.
`cash-control.controller.ts`	Cash position and CSV, petty cash list, categories, reversal, month end checklist and marks.
`profit.controller.ts`	Profit per tag, cost per order, stores list, staff pay, commissions, bonuses, payslips, advances, payroll preview and run (with the salaries switch).
`loans-tags-fx.controller.ts`	Loans and instalment payments, cost tags and their report, foreign banks and revaluation.
`rider-withdrawals.controller.ts`	Rider withdrawals and wallets; approve, reject, pay.
`rider-offices.controller.ts`	Rider offices, their daily account and CSV, commission rate, prepare office bills.
`expense-claims.controller.ts`	Claimable categories, claim detail, pay, reject, bulk approve.
`basata.controller.ts`	Basata deposits review, settlements, frozen riders and unfreeze.
`aging.controller.ts`	Receivables and payables aging with PDF, reminder now, debit notes.
`purchasing.controller.ts`	Purchase orders: decide, ordered, receive, bill, close.
`tools.controller.ts`	Payout bank file and CSV, merchant bank details, asset disposal.
`budgets.controller.ts`	`/finance/budgets` : the year against actual (view), set a quarter and copy a year (use).
`forecast.controller.ts`	`/finance/forecasts` : list, suggestion, compare, detail and budget preview (view); create, rename, delete, drivers, overrides (use); approve, reopen, use as budget (manage).
`payout-proof.controller.ts`	Payout controls, statement JSON, PDF and CSV, proof upload and file, ticking checks (use); payout checks list (view), create and edit (manage).
`payout-statement-pdf.ts`	The statement of account PDF in English or Arabic, statement CSV and proof file answers.
`files.controller.ts`	`POST /finance/files` and `DELETE /finance/files/:id` : uploads chosen in forms (accounting use or store use).
`voucher-pdf.ts`	`journalVoucherPdf` : the A4 journal voucher in English or Arabic, served by `GET /finance/journal/:id/voucher.pdf` in `views.controller.ts` .
`report-pdf.ts` , `aging-pdf.ts`	PDF of drill reports and aging reports in English or Arabic.
`spreadsheet.ts`	`readSpreadsheet` : reads an uploaded .xlsx or CSV bank statement without a spreadsheet library.
`finance-csv.ts`	CSV answer helper with a Cairo time stamp in the file name.
`eta.ts`	`HttpEtaClient` (the boundary to the tax authority) and `EtaSync` (submits signed documents and polls results every minute).
`../admin/records.controller.ts`	Chatter: notes and activities of any record, my activities, list favourites and column preferences.

File	Routes and role
`../admin/history.controller.ts`	`GET /history/:entityType/:entityId` from `audit_log`.
`../admin/lists.controller.ts`	`GET /lists/:app/:id` for the other apps' list views.
`../merchant/merchant-finance.controller.ts`	The merchant's wallet: payout schedule, statement and CSV, each payout's SOA (JSON, PDF, CSV) and POT once paid, invoices, bank details.
`../rider/rider.controller.ts`	The rider's wallet, Basata deposits and withdrawals.
`../store/store-cash.controller.ts`, `../store/store-petty.controller.ts`	Cash handovers counted at the store, cash withdrawals paid by the store manager, petty cash.
`../hr/my-pay.controller.ts`	My payslips and my expense claims; managers approve their team's claims.
`../common/pdf.ts`	`PdfWriter`: pdfkit pages with right to left word layout for Arabic.
`../common/bulk.ts`	`eachRecord`: runs a single record action for each id of a bulk action.

apps/web: pages, components and helpers

Pages under `apps/web/src/app/finance` are thin server components: they fetch from the API with `apiGet` and render a component. The table lists the folders.

Page folder	Main component
`layout.tsx`	`AppShell` with `AccountingSearch` and menu badges (approvals waiting, missing receipts).
`page.tsx` (`/finance`)	Accounting dashboard: today strip, journal cards with `Sparkline`.
`accounts`, `accounts/[id]`	`AccountsManager`; account statement with History.
`approvals`, `approvals/[id]`	`FinanceListPage` (list `approvals`); `DocumentPage` with `Approvals` actions.
`payouts`, `payouts/[id]`, `payouts/run`	List with `BankFilePanel` and `PayoutsManager`; document page with `PayoutControls` (statement, proof, checks, mark paid); `PayMerchantsRun` with a proof per row (`ProofUpload`).
`bills`, `bills/[id]`, `suppliers`, `suppliers/[id]`	Lists and document pages with `Payables` forms.
`journal`, `journal/[id]`	Journal list view; entry document page with the printable voucher and the PDF buttons.
`cheques`, `cheques/[id]`	List and register form; cheque document page.
`rider-withdrawals`, `rider-withdrawals/[id]`, `riders`	Withdrawal list and page; `RiderWallets`.
`petty-cash`, `petty-cash/[id]`	List and `PettyCashDesk`.

Page folder	Main component
`invoices`, `invoices/[id]`, `company`	Invoice list, `InvoiceView` with debit notes and print, company tax details.
`purchase-orders`, `purchase-orders/[id]`	`PurchaseOrders`.
`expense-claims`, `expense-claims/[id]`	List with `ClaimCategories`; `ClaimActions`.
`basata`, `basata/[id]`, `basata/transfer`, `frozen-riders`	Deposit list (`AppListPage`), `BasataActions`, `BasataSettlementForm`, `FrozenRiders`.
`cash`, `cash-position`, `bank`, `receipts`, `movements`, `expenses/new`	`CashDesk`, cash position with `StatCard`, `BankMatching`, `MissingReceipts`, `MoneyMovements`, `QuickExpense`.
`merchants`, `merchants/[id]`, `receivables`, `payables-aging`, `contracts`, `merchant-profit`	`MerchantWallets`, `StatementView`, `Aging`, `Contracts`, `MerchantProfit`.
`profit-and-loss`, `balance-sheet`, `trial`, `reports`, `reports/lines`, `reports/tags`, `money-flow`, `profit`, `statements`	`DrillReportPage`, report index with `InfoTip`, ledger lines `ListView`, cost tag report, cash flow, profit per store, comparative statements.
`month-end`, `month-end/run`, `close`, `year-end`, `taxes`, `vat`, `tax-returns`	`MonthEndChecklistView`, `CloseMonthRun`, `CloseBooks`, `YearEnd`, taxes with tabs (the last two redirect to tabs).
`payroll`, `payroll/payslips`, `rider-offices`, `loans`, `cost-tags`, `recurring`, `assets`, `opening`, `emails`	`Payroll`, payslips print, `RiderOffices`, `LoansFx`, `Payables` forms, `OpeningBalances`, `EmailOutbox`.
`budgets`, `forecast`, `forecast/[id]`, `due`	`BudgetPlan` (quarters opening into months, copy a year); `Forecast` (scenario list, editor with drivers, overrides, statements and use as budget); `PaymentsDue` (the dashboard card uses `PaymentsDueGroups` too).
`payout-checks`	`PayoutChecksEditor`.
`/merchant/wallet`, `/merchant/wallet/payouts/[id]`	The merchant's payouts with SOA and POT side by side and the Matches mark.

Components in `apps/web/src/components/finance`: `AccountsManager`, `Aging`, `Amount`, `Approvals`, `BankDetailsForm`, `BankFilePanel`, `BankMatching`, `BankSelect`, `AttachmentField` (files chosen inside a form, `useAttachments`), `BasataActions`, `BasataSettlementForm`, `BudgetPlan`, `CashDesk`, `Cheques`, `ClaimActions`, `ClaimCategories`, `CloseBooks`, `CloseMonthRun`, `Contracts` (deposit form), `CorporateTaxForm`, `CostTagSelect`, `DebitNoteForm`, `EmailOutbox`, `Forecast`, `FrozenRiders`, `InvoiceView`, `Invoices`, `LoansFx`, `MerchantProfit`, `MerchantWallets`, `MissingReceipts`, `MoneyMovements`, `MonthEndChecklist`, `OpeningBalances`, `PayMerchantsRun`, `Payables`, `PaymentsDue`, `PayoutChecksEditor`, `PayoutControls`, `PayoutsManager`, `ProofUpload`, `Payroll`, `PeriodReports`, `PettyCashDesk`, `PrintButton`, `PurchaseOrders`, `QuickExpense`, `ReceiptField`, `ReportTable`, `RiderOffices`, `RiderWallets`, `RiderWithdrawals`, `StatementView`, `useFinanceAction` (sends one action, shows the translated result, refreshes the page), and `reports/` (`DrillReportPage`, `DrillReportView`, `LinesFilters`, `ReportFilterBar`).

Components in `apps/web/src/components/odoo` are the generic building blocks described in the UI chapter: `ListView`, `FinanceListPage`, `AppListPage`, `DocumentFrame`, `DocumentPage`, `Chatter`, `RecordNav`, `DocEntries`, `Attachments` (`AttachmentTile`, `AttachmentsSmart`: one file opens directly, several open a

viewer; `AddAttachments` adds files to a saved record), `InfoTip`, `AccountingSearch`, and the configuration files `finance-lists.ts`, `app-lists.ts` and `list-formatters.ts`.

Helpers in `apps/web/src/lib` used by accounting: `money.ts` (`parseEgp`), `format.ts` (`formatEgp`, Cairo date formats), `day-first.ts`, `list-view.ts`, `workflows.ts`, `report-view.ts`, `close-run.ts`, `pay-run.ts`, `quick-expense.ts`, `foreign-money.ts`, `rider-wallet.ts`, `my-pay.ts`, `legacy-list.ts`, `csv.ts`, and the answer types `finance-types.ts`, `aging-types.ts`, `cash-control-types.ts`, `my-pay-types.ts`, `basata.ts`, `purchasing-types.ts`.

3 The ledger core

All money in Shiply Now lives in one double entry ledger. Every money event, from a rider collecting cash at a door to the year end close, is one ledger entry made of two or more lines whose amounts add up to zero. Balances are never stored; they are always summed from the lines. This chapter describes the tables, the one function that writes to them, the database rules that guard them, how balances are read and checked, and how periods are closed.

> **RULE**
> Money is a double entry ledger in piastres. Every entry's lines add up to zero (a deferred database trigger refuses anything else) and ledger rows are append only; corrections are new entries. Balances are always summed from lines, never stored. Debit is positive.

3.1 Tables

Three tables hold the ledger. They are defined in `packages/db/prisma/schema.prisma` (models `LedgerAccount`, `LedgerEntry`, `LedgerLine`) and created by the migration `20261027090000_finance`, with columns added by later migrations (profit dimensions, cost tags, foreign currency).

```
ledger_accounts 1 ──────────────< ledger_lines >──────────── 1 ledger_entries
  id, key, code, type,              id, entry_id, account_id,       id, number (JV),
  category, group_code,             amount_piastres (+ debit,       kind, memo,
  names, bank fields,               − credit), occurred_at,         occurred_at,
  currency, is_active,              dark_store_id, merchant_id,     order_id,
  owner columns                     rider_id, user_id,              idempotency_key,
  (merchant_id, rider_id,           cost_tag_id,                    data (json),
   dark_store_id, user_id,          foreign_cents, fx_rate_micro    created_by_id
   supplier_id)
```

ledger_accounts

One row per account: a company account (a bank, a cash box, an expense category) or a wallet that belongs to one party (a merchant, a rider, a store's box, a supplier, a team member).

Column	Meaning
`key` (unique)	A stable key the code uses to find the account: `merchant:<id>`, `rider_cash:<id>`, `store_cash:<id>`, `revenue`, `vat_payable`, `equity:capital`, or `acct:<code>` for accounts finance opens by hand.
`code` (unique)	Chart of accounts number: `1101` for a company account, `2100-0001` for a party wallet. Given from the group's sequence.
`type`	Enum `ledger_account_type`: `merchant`, `rider_cash`, `rider_bonus`, `rider_dues`, `store_cash`, `bank`, `cash_box`, `revenue`, `other_income`, `vat_payable`, `rider_bonus_expense`, `cash_difference`, `damage_expense`, `expense`, `other_asset`, `other_liability`, `equity`, `staff_advance`, `supplier`, `merchant_deposit`.

Column	Meaning
`category`	Enum `account_category` : `asset` , `liability` , `equity` , `revenue` , `expense` . Fixed by the type through the `CATEGORY` map in `ledger.ts` . It is the first digit of the code (1 to 5).
`group_code`	The group the code was taken from (1100 banks, 5400 operating expenses...).
`name_en` , `name_ar`	Names in both languages. Party and store accounts have none; they show their owner's name.
`bank_name` , `account_number` , `iban` , `note`	Bank details (required for group 1100).
`currency`	`EGP` by default; a bank account may be held in `USD` or `EUR` . Ledger amounts stay in EGP.
`petty` , `claimable`	An expense category store staff may pick for petty cash, or employees may claim back.
`is_active`	Closed accounts are kept; only an empty account can be closed.
`merchant_id` , `rider_id` , `dark_store_id` , `user_id` , `supplier_id`	The owner of a wallet. An account with any of them set is a wallet (`isWallet`).

ledger_entries

One row per money event.

Column	Meaning
`number` (unique)	Gapless serial number shown as `JV-000001` .
`kind`	What happened: `cod_collected` , `order_fee` , `cash_handover` , `merchant_payout` , `expense` , `journal` , `year_close` and about sixty more (see below).
`memo`	A short text (an order number, a reference, the reason of a journal entry).
`occurred_at`	When the event happened (UTC). A backdated event is booked at noon Cairo time of its day.
`order_id`	The order the entry belongs to, when there is one.
`idempotency_key` (unique)	Same key means the same event: a second post returns the first entry.
`data`	JSON details of the event (fee basis, tier, handover counts, the day of a storage charge, `reverseOn` of an accrual).
`created_by_id`	The user who caused it, or null for jobs.

ledger_lines

One side of an entry. Debit is positive, credit is negative, in piastres. A line of zero is refused by a CHECK constraint (`ledger_lines_amount_not_zero`).

Column	Meaning
`entry_id`, `account_id`	The entry and the account.
`amount_piastres`	Signed integer: positive is a debit, negative a credit.
`occurred_at`	Copied from the entry, so balances at a moment need no join.
`dark_store_id`, `merchant_id`, `rider_id`	Tags: where and for whom the line counts. They drive profit per store, merchant and rider, cost per order and every report filter by store or merchant.
`user_id`	The team member a salary or advance line is for.
`cost_tag_id`	The user's own analytic tag (a campaign, a project), at most one per line.
`foreign_cents`, `fx_rate_micro`	For a line on a bank account in another currency: the amount in that currency's cents (signed like the line) and the rate used, EGP for one unit times 1,000,000.

Indexes exist on `(account_id, occurred_at)` and on each tag with `occurred_at`, because nearly every report sums lines of an account, or of a tag, over a period.

3.2 post(): the only way to write

Every domain function that moves money calls `post` in `packages/db/src/finance/ledger.ts`. Nothing else inserts into the ledger tables. Its input:

```
export interface PostInput {
  kind: string;
  idempotencyKey: string;
  occurredAt?: Date;
  memo?: string;
  orderId?: string | null;
  data?: Prisma.InputJsonValue;
  createdById?: string | null;
  /** Where the whole entry counts; worked out from the order and wallets when left out. */
  darkStoreId?: string | null;
  merchantId?: string | null;
  riderId?: string | null;
  /** Debit positive, credit negative. Lines of zero are dropped. A line's own tags win. */
  lines: { account: AccountRef; amount: number; tags?: LineTags; foreign?: ForeignAmount }[];
}
```

An account is named by an `AccountRef`: by kind for wallets and system accounts (`{ type: 'merchant', merchantId }`, `{ type: 'vat_payable' }`) or by id for any account (`{ id }`). Accounts named by kind are created the first time they are used (see the chart of accounts chapter).

Step by step

1. Drop zero lines and check integers. Lines of zero are removed. Any amount that is not a safe integer is refused with `amount_not_integer`. If no line is left, `post` returns null and writes nothing. This is why a fee of 0 under a contract books nothing.
2. Check the balance in code. The sum of the lines must be zero and there must be at least two lines; otherwise `post` throws a plain `Error` (a programming mistake, not a business refusal), which answers 500.
3. Open a transaction (or join the caller's) with `inTransaction`.

4. Idempotency. If an entry with the same `idempotency_key` exists, return it and write nothing.
5. Books lock. `assertPeriodOpen(tx, occurredAt)` refuses a moment before `books_lock.locked_before` with `books_closed`.
6. Line extras. `checkLineExtras`: a line on a bank account in another currency must carry its foreign amount (`foreign_currency_account` otherwise), an EGP account never carries one, foreign cents must be integers, and every cost tag named must exist.
7. Serial number. `nextNumber(tx, 'ledger_entries')` takes a transaction level advisory lock on `hashtext('number:ledger_entries')` and returns `MAX(number) + 1`. The lock is held until the transaction ends, so two postings cannot take the same number and no number is skipped.
8. Insert the entry.
9. Resolve accounts. Each `AccountRef` becomes an account id, creating the account when a wallet or system account is used for the first time.
10. Work out the tags with `entryTags` (below).
11. Insert the lines, each with its tags (a line's own tags win over the entry's), cost tag and foreign amount.
12. Commit. At commit the deferred trigger checks again that the entry balances.

```
export async function post(db: Db, input: PostInput) {
  const lines = input.lines.filter((l) => l.amount !== 0);
  for (const l of lines)
    if (!Number.isSafeInteger(l.amount)) throw new DomainError('amount_not_integer', 400);
  if (lines.length === 0) return null;
  const total = lines.reduce((s, l) => s + l.amount, 0);
  if (total !== 0 || lines.length < 2) throw new Error(`unbalanced ledger entry ${input.kind}`);

  return inTransaction(db, async (tx) => {
    const seen = await tx.ledgerEntry.findUnique({ where: { idempotencyKey: input.idempotencyKey } });
    if (seen) return seen;
    const occurredAt = input.occurredAt ?? new Date();
    await assertPeriodOpen(tx, occurredAt);
    await checkLineExtras(tx, lines);
    const entry = await tx.ledgerEntry.create({
      data: { number: await nextNumber(tx, 'ledger_entries'), kind: input.kind, ... },
    });
    ...
  });
}
```

> Because `nextNumber` serialises every posting on one advisory lock, two postings never run their number step at the same time. For a pilot volume this is the price of gapless numbers. A system with much higher volume could number entries at the end of the day instead.

Tag inference order

Every line carries a store, merchant and rider tag when one can be found, so reports can split income and costs without extra bookkeeping. `entryTags` fills each tag in this order, stopping at the first value found:

1. what the caller passed on the entry (`darkStoreId`, `merchantId`, `riderId`);
2. the order of the entry (`orders.dark_store_id` and `orders.merchant_id`);
3. the owners of the wallets on the entry's lines (a merchant wallet gives the merchant, a rider wallet the rider, a store box the store);
4. for the store only: the rider's own store.

A tag given on a single line (`lines[i].tags`) wins over the entry's tag for that line. The `user_id` and `cost_tag_id` of a line come only from the line's own tags. Lines with no store are company wide.

Entry kinds

The `kind` is free text so new events need no migration, but every value comes from code. The kinds posted today, grouped by source:

Source	Kinds
Deliveries (`postings.ts`)	`cod_collected` , `shipping_fee` , `product_handling` , `order_fee` , `cod_fee` , `packaging_charge` , `postal_fee` , `rider_bonus` , `failed_fee` , `receiving_fee` , `storage_fee` , `return_fee`
Contracts	`minimum_fee` , `deposit_received` , `deposit_refunded`
Cash and payouts (`operations.ts`)	`cash_handover` , `cash_difference` , `bank_deposit` , `merchant_payout` , `merchant_receipt` , `adjustment`
Riders	`rider_withdrawal` , `rider_shortage_charge` , `rider_penalty` , `rider_office_commission` , `rider_office_shortage` , `basata_deposit` , `basata_deadline_charge` , `basata_settlement`
Damage and packaging	`damage_compensation` , `damage_rider_charge` , `packaging_purchase` , `packaging_used` , `packaging_adjustment`
Books (`books.ts`)	`money_transfer` , `expense` , `capital_in` , `capital_out` , `other_income` , `journal` , `accrual` , `accrual_reversal` , `opening_balance`
Payables and taxes	`supplier_bill` , `supplier_bill_cancel` , `bill_payment` , `tax_payment` , `po_receipt` , `corporate_tax`
Assets and loans	`prepayment` , `prepayment_release` , `asset_purchase` , `depreciation` , `asset_disposal` , `loan_received` , `loan_payment` , `fx_revaluation`
Cheques	`cheque_received` , `cheque_cleared` , `cheque_bounced` , `cheque_cancelled`
Invoices	`credit_note` , `debit_note`
People	`payroll` , `staff_advance` , `staff_bonus` , `expense_claim` , `petty_cash` , `petty_cash_reversal`
Year end	`year_close`

Idempotency keys

Each posting's key names the event it belongs to, so running the same code twice (a retry, a job that overlaps, a double tap) posts once. Keys built from records:

Event	Key
Cash collected at delivery	`delivery:<delivery id>:cod`
Each fee of a delivered order	`order:<order id>:order_fee` , `:cod_fee` , `:shipping_fee` , `:product_handling` , `:packaging_charge`

Event	Key
Rider pay	`delivery:<delivery id>:bonus`
Failed attempt fee	`delivery:<delivery id>:failed_fee`
Receiving fee	`manifest:<manifest id>:receiving_fee`
Storage of a day	`storage:<day>:<store id>:<merchant id>`
Cash handover	`handover:<handover id>`
Payout paid	`payout:<payout id>`
Salary run	`payroll:<YYYY-MM>`
Recurring expense	`recurring:<id>:<month>`
Year close	`year_close:<year>`
Opening balances	`opening_balance` (one per company)

Postings started by a person (an expense, a transfer, a journal entry) use the client generated idempotency key of the request, with a prefix: `expense:<key>`, `transfer:<key>`, `journal:<key>`. A form in the web app creates its key with `newActionKey()` (`apps/web/src/lib/ids.ts`, a random UUID with a fallback for plain http) and keeps it until the action succeeds, so a double click or a retry after a lost answer posts once.

3.3 Worked example: a delivered order

WORKED EXAMPLE

A merchant's contract charges an order fee of 15.00 EGP fixed plus 2% of the products value, no COD fee, no shipping fee. VAT is 14% (`fee.vat_bp` = 1400). Rider pay is 20.00 EGP per delivered order. A rider delivers an order whose products are worth 500.00 EGP and collects 550.00 EGP in cash. `postDeliveryCompleted` runs inside the delivery transaction and posts:

Entry kind	Account	Debit	Credit
`cod_collected`	1300 rider cash (rider's wallet)	550.00	
`cod_collected`	2100 merchant (merchant's wallet)		550.00
`order_fee`	2100 merchant	28.50	
`order_fee`	4101 delivery and service fees		25.00
`order_fee`	2301 VAT payable		3.50
`postal_fee`	5103 Postal fees	2.50	
`postal_fee`	2307 Postal fees owed		2.50
`rider_bonus`	5101 rider pay per delivered order	20.00	
`rider_bonus`	2200 rider bonus (rider's wallet)		20.00

The order fee is `1500 + share(50000, 200) = 1500 + 1000 = 2500` piastres and its VAT is `share(2500, 1400) = 350` piastres. The postal fee, our cost and never the merchant's, is `share(2500, fee.postal_bp = 1000) = 250` piastres (chapter 10). The COD fee, shipping fee, per product prices and packaging charge are 0 here, so their posts write nothing. Every line is tagged with the order's store and merchant; the lines of the two entries that touch a rider wallet (`cod_collected` and `rider_bonus`) are also tagged with the rider, because the tags are worked out per entry. Afterwards the merchant's wallet holds a credit of 521.50 EGP: the company owes the merchant the cash collected minus the fee and its VAT.

The fee lines come from one helper, so every fee kind charges VAT the same way:

```
async function feeLines(tx: Tx, merchantId: string, fee: number) {
  const vat = share(fee, await getSetting(tx, 'fee.vat_bp'));
  const merchant: AccountRef = { type: 'merchant', merchantId };
  return [
    { account: merchant, amount: fee + vat },
    { account: { type: 'revenue' } as AccountRef, amount: -fee },
    { account: { type: 'vat_payable' } as AccountRef, amount: -vat },
  ];
}
```

3.4 Database guards

The code checks every rule first so it can answer with a clear code, but the database enforces the hard rules too, whatever the caller.

Append only

```
-- Ledger rows are history: never changed or removed (corrections are new entries).
CREATE TRIGGER "ledger_entries_append_only" BEFORE UPDATE OR DELETE ON "ledger_entries"
  FOR EACH ROW EXECUTE FUNCTION "reject_append_only_change"();
CREATE TRIGGER "ledger_lines_append_only" BEFORE UPDATE OR DELETE ON "ledger_lines"
  FOR EACH ROW EXECUTE FUNCTION "reject_append_only_change"();
```

`reject_append_only_change()` (from the first migration) raises an error with `restrict_violation`. The same function guards `stock_movements`, `order_events`, `audit_log`, ticket messages, order notes, record notes and packaging movements. A mistake in the books is corrected by a new entry that turns the old one round (a reversal) or by an adjustment with a written reason.

Every entry balances

```
CREATE FUNCTION "ledger_entry_must_balance"() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE total BIGINT; lines INT;
BEGIN
  SELECT COALESCE(SUM("amount_piastres"), 0), COUNT(*) INTO total, lines
    FROM "ledger_lines" WHERE "entry_id" = NEW."entry_id";
  IF total <> 0 OR lines < 2 THEN
    RAISE EXCEPTION 'ledger entry % does not balance (total %, % lines)', NEW."entry_id", total, lines;
  END IF;
  RETURN NULL;
END $$;

CREATE CONSTRAINT TRIGGER "ledger_lines_balance" AFTER INSERT ON "ledger_lines"
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION "ledger_entry_must_balance"();
```

The trigger is deferred to commit time, so `post` can insert the lines one by one; at commit each entry touched must add up to zero with at least two lines, or the whole transaction fails.

Closed periods

```
CREATE FUNCTION "ledger_line_period_open"() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE lock_at TIMESTAMP(3);
BEGIN
  SELECT "locked_before" INTO lock_at FROM "books_lock" WHERE "id" = 'books';
  IF lock_at IS NOT NULL AND NEW."occurred_at" < lock_at THEN
    RAISE EXCEPTION 'books_closed: % is inside a closed period', NEW."occurred_at";
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER "ledger_lines_period_open" BEFORE INSERT ON "ledger_lines"
  FOR EACH ROW EXECUTE FUNCTION "ledger_line_period_open"();
```

Other money tables carry CHECK constraints too (for example a cash handover's difference must equal expected minus counted, and a payout's amount must be positive). Tax invoices and frozen contract versions are protected by their own triggers.

3.5 Balances

No balance column exists anywhere. A balance is the sum of an account's lines, optionally before a moment:

```
/** Debit minus credit of one account, optionally up to (not including) a moment. */
export async function balanceOf(db: Db, ref: AccountRef, before?: Date): Promise<number> {
  const result = await db.ledgerLine.aggregate({
    where: {
      account: 'id' in ref ? { id: ref.id } : { key: accountKey(ref) },
      occurredAt: before ? { lt: before } : undefined,
    },
    _sum: { amountPiastres: true },
  });
  return result._sum.amountPiastres ?? 0;
}
```

`balancesByType` sums many accounts in one grouped query. Reports such as the trial balance sum debits and credits per account over a period with one SQL query (`sumsByAccount` in `books.ts`), optionally filtered by the store or merchant tag and optionally leaving out the `year_close` entry.

Sign convention

Raw balances are debit minus credit. Assets and expenses normally have a debit (positive) balance; liabilities, equity and income normally have a credit (negative) balance. Screens and reports show every balance as an accountant reads it, positive on its normal side:

```
/** Debit balances are normal for assets and expenses; the rest are shown with the sign flipped. */
export function debitNormal(category: AccountCategoryName) {
  return category === 'asset' || category === 'expense';
}

/** The balance as an accountant reads it: assets and expenses debit, the rest credit. */
export const natural = (category, raw: number) => (debitNormal(category) ? raw : 0 - raw);
```

So what we owe a merchant (a credit on their 2100 wallet) is shown as a positive amount owed: `merchantOwed` returns the balance with its sign flipped. When a merchant's wallet turns to a debit balance (fees and charges passed the cash collected), the merchant owes us; the receivables and aging screens list those merchants.

3.6 Holding money: lockedBalance and mustHold

> **RULE**
> Money cannot leave a bank or cash box that does not hold it: neither at the movement's own moment (a past day for a backdated movement) nor at any later moment up to now.

Two payments from the same box at the same time must not both spend the same money. Every check that a bank or box holds an amount takes a row lock on the account first, inside the transaction that posts the payment:

```
export async function lockedBalance(tx: Tx, ref: AccountRef) {
  const found = 'id' in ref ? { id: ref.id }
    : await tx.ledgerAccount.findUnique({ where: { key: accountKey(ref) }, select: { id: true } });
  if (!found) return { accountId: null, balance: 0 };
  await tx.$queryRaw`SELECT id FROM ledger_accounts WHERE id = ${found.id} FOR UPDATE`;
  return { accountId: found.id, balance: await balanceOf(tx, { id: found.id }) };
}
```

`openAccount` in `books.ts` takes the same `FOR UPDATE` lock and checks that the account belongs to the expected groups and is open. `mustHold` then checks the amount against the running balance, which matters for backdated movements: paying 1,000 EGP dated last Tuesday must not make last Wednesday's balance negative, even if the box holds enough today.

```
export async function mustHold(tx: Tx, account: Pick<LedgerAccount, 'id'>, amount: number, at = new
Date()) {
  await assertPeriodOpen(tx, at);
  const rows = await tx.$queryRaw`
    WITH moments AS (
      SELECT occurred_at, SUM(amount_piastres)::bigint AS amount FROM ledger_lines
      WHERE account_id = ${account.id} GROUP BY occurred_at
    ), running AS (
      SELECT occurred_at, (SUM(amount) OVER (ORDER BY occurred_at))::bigint AS balance FROM moments
    )
    SELECT
      (SELECT balance FROM running WHERE occurred_at <= ${at} ORDER BY occurred_at DESC LIMIT 1) AS
at_balance,
      (SELECT MIN(balance) FROM running WHERE occurred_at > ${at}) AS later_min`;
  const atBalance = Number(rows[0]?.at_balance ?? 0);
  const laterMin = rows[0]?.later_min == null ? atBalance : Number(rows[0].later_min);
  if (amount > Math.min(atBalance, laterMin)) throw new DomainError('not_enough_money');
}
```

Lines at the same moment count together. The check refuses with `not_enough_money`. A day inside closed books answers `books_closed` first, whatever the balance.

3.7 Cairo days and months

A movement entered for a past day is booked at noon Cairo time of that day; one entered for today is booked now. A future day is refused:

```
/** When a money event happened: a past Cairo day at noon, or now for today and no day. */
export function eventTime(day: string | undefined, now: Date = new Date()) {
  if (!day || day === cairoDay(now)) return now;
  if (!isCalendarDay(day)) throw new DomainError('invalid_date', 400);
  if (day > cairoDay(now)) throw new DomainError('future_date', 400);
  return new Date(cairoDayStart(day).getTime() + 12 * 3600_000);
}
```

A period of Cairo days `[from, to]` is turned into instants as `cairoDayStart(from)` (included) to `cairoDayStart(addDays(to, 1))` (excluded). In SQL, grouping by Cairo day or month always converts from UTC first, because `occurred_at` is stored without a time zone:

```
SELECT to_char((l.occurred_at AT TIME ZONE 'UTC') AT TIME ZONE 'Africa/Cairo', 'YYYY-MM') AS month, ...
```

Writing only `AT TIME ZONE 'Africa/Cairo'` on a value without a time zone would read it as Cairo time and shift it the wrong way; the double conversion was made a rule after a review found this mistake.

3.8 Closing the books

The table `books_lock` has a single row (`id = 'books'`, enforced by a CHECK) holding `closed_through` (a Cairo day) and `locked_before` (the instant the next Cairo day starts). Nothing may be posted before `locked_before`: `assertPeriodOpen` answers `books_closed` and the database trigger refuses it as well.

`closeBooks` in `controls.ts`:

- accepts only a day already over in Cairo (`day_not_over` otherwise);
- takes the advisory lock `books_lock` so two people cannot move it at once;
- moving the date forward closes more days; moving it back reopens days and needs an admin (in the API: a person with settings manage) and a reason of at least five characters (`reopen_admin_only`, `reason_required`);
- writes an audit row `finance.books_close` or `finance.books_reopen` with the old and new day.

Jobs respect the lock: a storage day, a depreciation month or a recurring month that falls inside closed books is skipped or charged on the first open day, as each job's rules say.

3.9 Closing a year

`closeYear` in `period.ts` moves the year's profit into retained earnings:

1. Only a year already over can be closed (`year_not_over`), each year once (`year_already_closed`, the entry key is `year_close:<year>`) and in order (`earlier_year_open` when income or expense accounts still carry balances from before 1 January of that year).
2. Under the `books_lock` advisory lock, it sums every income and expense account up to the end of the year.
3. The closing entry is dated one millisecond before the new year (in Cairo time). If the books are locked past that moment, the lock is moved just before it for this one entry, in the same transaction.
4. It posts one `year_close` entry: each income and expense balance turned round to zero, and the total against 3401 retained earnings.
5. It sets the books lock through 31 December (or keeps a later lock), and writes the audit row `finance.year_close`.

Profit and loss reports leave the `year_close` entry out, so a closed year still shows its income and costs. The balance sheet shows the profit of the current year (not yet closed) as its own line next to equity, so assets always equal liabilities plus equity plus earnings.

4 Chart of accounts

Every ledger account has a code taken from its group. The first digit of a code is the account's category; the first four digits are its group. Groups and their rules live in `ACCOUNT_GROUPS` (`packages/shared/src/accounts.ts`); the accounts the system itself uses, with their preferred codes and names, live in `SYSTEM_ACCOUNTS` and `DEFAULT_ACCOUNTS` (`packages/db/src/finance/ledger.ts`). The chart was introduced by the migration `20261108090100_chart_of_accounts`, which also numbered every account that existed before.

4.1 Categories

First digit	Category	Normal balance	Shown as
1	`asset`: what the company owns or is owed (banks, cash, rider cash, advances, equipment)	Debit	As is
2	`liability`: what the company owes (merchants, riders, taxes, suppliers, loans)	Credit	Sign flipped
3	`equity`: owner capital, drawings, opening balances, retained earnings	Credit	Sign flipped
4	`revenue`: fee income and other income	Credit	Sign flipped
5	`expense`: running costs, rider pay, damage, depreciation, taxes on profit	Debit	As is

The category of an account is fixed by its type (`CATEGORY` in `ledger.ts`). Accumulated depreciation (1701) is an asset account that normally carries a credit balance, so it shows as a negative amount among the assets and reduces the value of equipment.

4.2 Groups

Each group says whether each party has its own wallet in it (party), whether finance can open accounts in it by hand (and of which type), and whether it holds money that can be moved (banks and cash boxes).

Group	Category	Name	Party wallets	Opened by finance as	Money
1100	asset	Banks	no	`bank`	yes
1150	asset	Cheques under collection	no		no
1200	asset	Cash boxes (company boxes and store boxes)	no	`cash_box`	yes
1300	asset	Cash with riders	yes		no
1350	asset	Rider dues (what a rider owes us)	yes		no
1400	asset	Other assets	no	`other_asset`	no

Group	Category	Name	Party wallets	Opened by finance as	Money
1500	asset	Staff advances	yes		no
1600	asset	Fixed assets (equipment and vehicles)	no		no
1700	asset	Accumulated depreciation	no		no
2100	liability	Merchants	yes		no
2200	liability	Rider bonuses (rider pay earned, not yet withdrawn)	yes		no
2300	liability	Taxes	no		no
2400	liability	Other liabilities	no	`other_liability`	no
2450	liability	Cheques payable	no		no
2500	liability	Suppliers	yes		no
2600	liability	Merchant security deposits	yes		no
2700	liability	Loans (one account per loan)	no		no
3100	equity	Owner capital	no	`equity`	no
3200	equity	Owner drawings	no	`equity`	no
3300	equity	Opening balances	no		no
3400	equity	Retained earnings	no		no
4100	revenue	Fees	no		no
4200	revenue	Other income	no	`other_income`	no
5100	expense	Rider costs	no		no
5200	expense	Damage	no		no
5300	expense	Cash differences	no		no
5400	expense	Operating expenses	no	`expense`	no
5500	expense	Bank charges	no	`expense`	no
5600	expense	Depreciation	no		no
5700	expense	Staff costs	no		no
5800	expense	Company income tax	no		no
5900	expense	Other costs	no		no

4.3 Accounts by code

The table lists every account the system knows by key, with the code it takes when first used and what posts to it. English names are the ones stored in `name_en`; every account also has an Arabic name.

Code	Key	Name	Purpose
1101	`bank`	The company's main bank account (named CIB in the code)	The default bank for deposits, payouts, receipts and bonus payouts when no other bank is named. Further banks get 1102, 1103 and so on.
1151	`cheques_under_collection`	Cheques under collection	Cheques received and booked but not yet paid by the bank.
12xx	`store_cash:<store>`	The store's cash box	Cash counted at handover; deposits to the bank and petty cash leave from it. Opened automatically, next free code in 1200.
12xx	`basata_transit`	Money at Basata, on its way to the bank	Approved rider deposits at Basata branches until Basata's daily transfer reaches the bank. Next free code in 1200.
`1300-nnnn`	`rider_cash:<rider>`	Rider's cash	Cash on delivery the rider collected and still carries.
`1350-nnnn`	`rider_dues:<rider>`	Rider's dues	What a rider owes the company (shortages kept on them, damage charges, penalties), taken from their next withdrawal.
1401	`prepaid`	Prepaid expenses	Money paid once for several months (a year's rent), released monthly.
1402	`packaging_stock`	Packaging stock	Bags, boxes, labels and tape bought, at average cost, until used at packing.
`1500-nnnn`	`staff_advance:<user>`	Staff advance of a person	Money advanced to a team member, recovered from salary or withdrawal.
1601	`fixed_assets`	Equipment and vehicles	Equipment bought, at cost: paid directly, or received on a purchase order asset line (one fixed asset per unit).
1701	`accumulated_depreciation`	Accumulated depreciation	Straight line depreciation charged so far (credit balance).
`2100-nnnn`	`merchant:<merchant>`	Merchant's wallet	What we owe the merchant: cash collected for them minus our fees and VAT, minus payouts. A debit balance means the merchant owes us.
`2200-nnnn`	`rider_bonus:<rider>`	Rider's earnings	Rider pay per delivered order, until withdrawn.

Code	Key	Name	Purpose
2301	`vat_payable`	VAT payable	VAT on our fees (output) less VAT claimed back on purchases (input).
2302	`wht_payable`	Withholding tax payable	Withholding tax kept back from supplier payments.
2303	`payroll_tax_payable`	Salary income tax payable	Tax deducted from salaries.
2304	`social_insurance_payable`	Social insurance payable	Employee and company shares of social insurance.
2305	`martyrs_fund_payable`	Martyrs fund payable	The martyrs fund deduction.
2306	`corporate_tax_payable`	Company income tax payable	The year's company income tax agreed with the accountant.
2307	`postal_fee_payable`	Postal fees owed	The postal authority's share of our fees per shipment (`fee.postal_bp`), owed until paid once a month from Taxes (tax kind `postal`). Type `other_liability` .
2401	`goods_received`	Goods received, not billed yet	Purchase order receipts until the supplier's bill arrives.
2402	`rider_office_accrued`	Rider office commission accrued, not billed yet	Used by the first version of rider office commission; new commission goes straight to the office's supplier wallet.
2451	`cheques_payable`	Cheques payable	Cheques we wrote that the bank has not paid yet.
`2500-nnnn`	`supplier:<supplier>`	Supplier's wallet	What we owe the supplier for bills not yet paid (rider offices are suppliers too).
`2600-nnnn`	`merchant_deposit:<merchant>`	Merchant's security deposit	A deposit held under the merchant's contract, paid back when it ends.
2701, 2702...	one per loan	The loan	Principal still owed on each bank loan, opened with the loan.
3101	`equity:capital`	Owner capital	Money the owner puts in.
3201	`equity:drawings`	Owner drawings	Money the owner takes out.
3301	`equity:opening`	Opening balances	The other side of the one time opening balances entry.
3401	`retained_earnings`	Retained earnings	Profit of closed years.
4101	`revenue`	Delivery and service fees	Every fee charged to merchants (one income account; the entry kind tells the fee kind).

Code	Key	Name	Purpose
4201	`other_income:general`	Other income	Income that is not a fee. Finance can open more 4200 accounts.
5101	`rider_bonus_expense`	Rider pay per delivered order	Rider pay, and the credit side of rider penalties.
5102	`rider_office_commission`	Rider office commission	The office's commission on rider pay, at each delivery.
5103	`postal_fee_expense`	Postal fees	The postal fee booked at each delivered order and each charged failed attempt: our cost, never charged to the merchant, without VAT. Type `expense`.
5201	`damage_expense`	Damage paid by us	Merchant compensation for units damaged in our care.
5301	`cash_difference`	Cash shortages and surpluses	Handover differences written off.
5401 to 5409	`expense:rent`, `expense:salaries`, `expense:fuel`, `expense:packaging`, `expense:marketing`, `expense:utilities`, `expense:maintenance`, `expense:government`, `expense:other`	Rent; Salaries and wages; Fuel and transport; Packaging and supplies; Marketing; Utilities and internet; Maintenance; Government fees; Other expenses	Running costs. Salaries post to 5402; packaging used at packing posts to 5404. Finance can open more 5400 categories.
5501	`expense:bank_charges`	Bank charges	Bank fees, also booked from bank statement lines.
55xx	`basata_fees`	Basata fees	What Basata keeps from each daily transfer. Next free code in 5500.
5601	`depreciation_expense`	Depreciation	Monthly depreciation of equipment.
5701	`social_insurance_expense`	Social insurance, company share	The company's share of social insurance.
5702	`staff_bonus_expense`	Staff bonuses	Bonuses paid directly (outsourced people).
5801	`income_tax_expense`	Company income tax	The year's company income tax.
5901	`fx_difference`	Exchange rate differences	Differences on foreign currency bills, transfers and revaluation.
5902	`asset_disposal`	Loss or gain on equipment sold	The rest when equipment is sold or scrapped.
5904	`loan_interest_expense`	Loan interest	Interest part of loan instalments.

The codes in this table are the preferred codes. When a preferred code is already taken by another account, the account takes the next free code of its group instead (both in `accountId` when a system account is first used, and in `ensureDefaultAccounts` for the default accounts).

> A few names stored in an older migration differ from the current code (for example 5101 was first named "Rider delivery bonuses" and is now "Rider pay per delivered order" in `SYSTEM_ACCOUNTS`). Accounts keep the name they were created with unless finance renames them; the code only gives a name when it creates an account.

4.4 System accounts and party wallets

There are three kinds of accounts:

- System accounts exist once and are found by key (`revenue`, `vat_payable`, `bank`, `retained_earnings`...). The code posts to them by kind: `{ account: { type: 'vat_payable' } }`. They can be renamed but never closed: their keys are in `LOCKED_KEYS` in `books.ts`.
- Party wallets belong to one merchant, rider, supplier, team member or store. They carry the owner in `merchant_id`, `rider_id`, `supplier_id`, `user_id` or `dark_store_id`, have no stored name (they show their owner's name) and can be neither renamed nor closed. `isWallet(account)` is true for them.
- Company accounts opened by finance: banks, cash boxes, other assets and liabilities, equity accounts, income and expense categories. Their key is `acct:<code>`.

A wallet or system account is created the first time a posting names it. The function `accountId` in `ledger.ts` looks the key up; when it is missing it chooses the code (the preferred one, or the next free one), and creates the row with `upsert` so two transactions creating the same wallet at once do not collide:

```
const WALLET_GROUPS = {
  store_cash: { group: 1200, party: false },
  rider_cash: { group: 1300, party: true },
  merchant: { group: 2100, party: true },
  rider_bonus: { group: 2200, party: true },
  rider_dues: { group: 1350, party: true },
  staff_advance: { group: 1500, party: true },
  supplier: { group: 2500, party: true },
  merchant_deposit: { group: 2600, party: true },
};
```

How codes are numbered

`nextAccountCode(tx, group, party)` takes the advisory lock `ledger_account_code` for the transaction, then:

- for a party group, finds the highest four digit suffix used in that group and adds one: `2100-0001`, `2100-0002`... It refuses `group_full` after `9999`;
- for a company group, finds the highest plain numeric code in the group and adds one: `1101`, `1102`... It refuses `group_full` after the group's 99th code.

```
export async function nextAccountCode(tx: Tx, group: number, party: boolean) {
  await tx.$executeRaw`SELECT pg_advisory_xact_lock(hashtext('ledger_account_code'))`;
  if (party) {
    const prefix = `${group}-`;
    const rows = await tx.$queryRaw`
      SELECT MAX(CAST(substr(code, 6) AS INTEGER)) AS max FROM ledger_accounts
      WHERE group_code = ${group} AND code LIKE ${prefix + '%'}`;
    const next = (rows[0]?.max ?? 0) + 1;
    if (next > 9999) throw new DomainError('group_full');
    return `${prefix}${String(next).padStart(4, '0')}`;
  }
  const rows = await tx.$queryRaw`
    SELECT MAX(CAST(code AS INTEGER)) AS max FROM ledger_accounts
    WHERE group_code = ${group} AND code ~ '^[0-9]+$'`;
  const next = Math.max(rows[0]?.max ?? group, group) + 1;
  if (next > group + 99) throw new DomainError('group_full');
  return String(next);
}
```

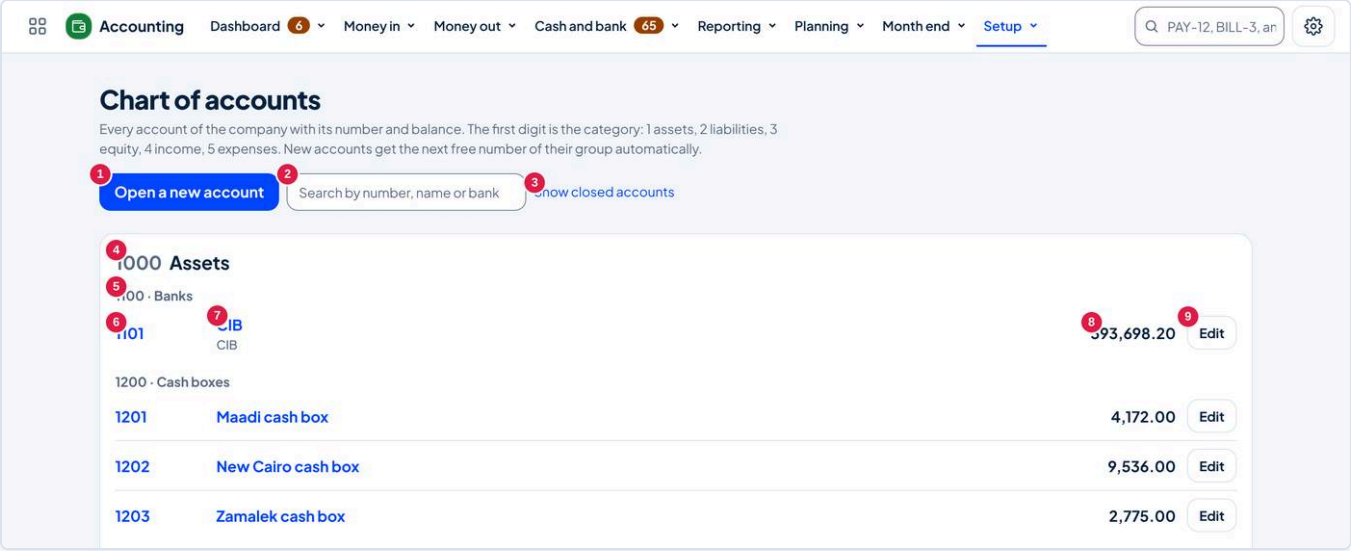
Store cash boxes are company accounts (not party wallets) of group 1200, so they take plain codes (1201 onward when they were first numbered, the next free code later). Loans call `nextAccountCode(tx, 2700, false)` when a loan is recorded.

Opening, renaming and closing accounts

`createAccount` in `books.ts` (route `POST /finance/accounts`, accounting use) opens an account in a group whose `manualType` is set: 1100, 1200, 1400, 2400, 3100, 3200, 4200, 5400 or 5500. Any other group answers `group_not_open`. The rules:

- a bank (1100) needs a bank name and an account number (`bank_details_required`), and the same bank and number cannot be entered twice (`bank_account_exists`);
- only a bank may be held in USD or EUR (`currency_banks_only`);
- names are unique inside the group in either language (`account_name_taken`);
- the code comes from `nextAccountCode`; the key is `acct:<code>`;
- an audit row `finance.account_create` is written.

`updateAccount` (`PATCH /finance/accounts/:id`) renames an account and edits bank details. Wallets cannot be renamed or closed (`account_locked`). A system account cannot be closed (`account_locked`). Any other account can be closed only when its balance is zero (`account_has_balance`). A closed account cannot receive new lines (`account_inactive`), but its history stays.



The chart of accounts at `/finance/accounts` : accounts listed in code order grouped by category, each with its code, name and balance, and the form to open a new account with the group chosen.

4.5 Which accounts a journal entry may touch

A free journal entry is the accountant's tool for accruals, reclassifications and compound payments. Because a journal entry could otherwise create or hide money, `checkJournalEntry` in `books.ts` limits the accounts it may use. It runs when the entry is entered and again when it is posted (also after a second person approves it).

Account	Allowed?	Why
System and company accounts (income, expenses, taxes, other assets and liabilities, equity, loans)	Yes	This is what journal entries are for.
Banks (1100) and company cash boxes (1200), including store cash boxes	Yes, with conditions	Allowed since the owner asked for compound entries such as rent and electricity paid together from the bank. Money out of a bank or box is checked with `mustHold` at the entry's moment; money in waits for a second person while `finance.approve_money_in` is 1; money out waits from the approval limit (see `submitJournalEntry`).
Banks and boxes in an accrual (an entry reversed by itself next month)	No: `accrual_money_not_allowed`	Real money never comes back by itself.
Wallets of merchants, riders, suppliers, team members and merchant deposits (2100, 2200, 1300, 1350, 1500, 2500, 2600)	No: `wallet_not_allowed`	Each party keeps its own screens, so its statement always tells the whole story (merchant adjustments, rider charges, bills, advances).

Account	Allowed?	Why
Rider cash (1300)	No	Rider cash accounts are wallets, so they are refused as wallets; the code also refuses group 1300 with `money_account_not_allowed` as a second guard. Rider cash moves only through handovers and deposits.
Closed accounts	No: `account_inactive`	

```
for (const l of lines) {
  const account = await db.ledgerAccount.findUnique({ where: { id: l.accountId } });
  if (!account) throw new DomainError('account_not_found', 404);
  if (isWallet(account) && account.darkStoreId == null) throw new DomainError('wallet_not_allowed');
  if (account.groupCode === 1300) throw new DomainError('money_account_not_allowed');
  if (!account.isActive) throw new DomainError('account_inactive');
  await assertActiveTag(db, l.costTagId);
  ...
}
if (input.reverseNextMonth && money.size) throw new DomainError('accrual_money_not_allowed');
```

Other checks of a journal entry: a written reason of at least five characters (`reason_required`), between two and twenty lines each with either a debit or a credit (`invalid_line`), debits equal to credits (`not_balanced`), a day that is not in the future (`future_date`), and active cost tags. Its idempotency key is `journal:<client key>` ; its kind is `journal` , or `accrual` with `data.reverseOn` set to the first day of the next month, when the daily job posts the reversal (`accrual_reversal:<id>`).

4.6 Opening balances

The balances the books start from are posted once by an owner (accounting manage and settings manage) with `recordOpeningBalances` . Amounts are typed as an accountant reads them (positive on the account's normal side) and turned into debits and credits with `debitNormal` ; the difference goes to 3301 opening balances. The entry is dated at the very start of the first Cairo day and keyed `opening_balance` , so a second attempt answers `opening_already_posted` ; corrections are journal entries.

5 Controls and access

The ledger guarantees that the books always balance. The controls in this chapter guarantee that the right people move money, that large or sensitive movements are seen by a second person, that closed periods stay closed, and that every change can be traced. They are implemented in `packages/shared/src/access.ts`, `apps/api/src/auth`, `packages/db/src/finance/controls.ts` and `packages/db/src/audit.ts`.

5.1 Access profiles

Team members (roles `admin`, `finance`, `store_staff`) each have one access profile, stored in `user_access` (one row per user) and `user_stores` (the stores of a person limited to some stores). A profile holds:

- a level per app: `store`, `operations`, `inventory`, `merchants`, `customers`, `accounting`, `hr`, `sales`, `settings`. Levels are `none` < `view` < `use` < `manage`. The store screens have no view level and settings has only none and manage;
- `all_stores`: every store, or only the listed ones;
- switches: `customer_data` (named customer profiles), `salaries` (salaries, payslips, payroll), `own_merchants_only` (account managers see only their merchants), `own_deals_only` (sales reps see only their deals);
- `job_role`: the template the profile was last filled from, or `custom`.

Job roles are templates in `JOB_ROLE_TEMPLATES`. The ones that matter for accounting:

Job role	Accounting	Other apps	Salaries
`owner`	manage	manage everywhere, settings manage	yes
`finance_manager`	manage	operations, inventory, merchants view	yes
`finance_officer`	use	operations, inventory, merchants view	no
`accountant`	view	operations, inventory, merchants view	no
`store_manager`	none	store manage (pays approved cash withdrawals from its store box and may let a rider carry cash above the limit, with a reason), operations and inventory use	no

What the accounting levels mean in practice:

Level	May do
view	Open every Accounting page and report, export lists and reports. Every `GET` route under `/finance`.
use	Prepare and record: expenses, transfers, journal entries, payouts (prepare, mark paid), bills and payments, cheques, rider withdrawals (approve, pay), expense claims (pay), petty cash, invoices, month end ticks by hand. Every `POST`, `PUT`, `PATCH`, `DELETE` without a stronger level.
manage	Decide what a second person decides: approve or reject requests, approve payouts (also in bulk), close the books, close a year, reverse a petty cash expense, open cost tags, revalue foreign banks, set a rider office's commission, book the company income tax, edit the company tax profile.

Level	May do
manage plus settings manage	Owner only actions: post the opening balances, create the electronic invoice signer key, reopen closed days.

The routes added with the payout controls, finance files and planning follow the same levels:

Routes	Level
`GET /finance/payouts/:id/controls`, `.../statement`, `.../statement.pdf`, `.../statement.csv`, `.../proof`	accounting view, and the payout's merchant inside the person's merchants
`POST /finance/payouts/:id/proof`, `POST /finance/payouts/:id/checks/:checkId`	accounting use (the second person check is also refused to the payer when paying)
`GET /finance/payout-checks`	accounting view
`POST /finance/payout-checks`, `PATCH /finance/payout-checks/:id`	accounting manage
`POST /finance/files`, `DELETE /finance/files/:id`	accounting use or store use (store staff attach petty cash receipts); a person may delete only their own upload that no form saved
`GET /finance/journal/:id/voucher.pdf`, `GET /finance/payments-due`, `GET /finance/lists/suppliers`, `GET /finance/documents/supplier/:id`	accounting view (payments due: a person limited to some stores sees their stores' items and company wide items, never merchant payouts)
`POST /finance/suppliers`, `PATCH /finance/suppliers/:id`	accounting use
`GET /finance/forecasts`, `.../suggestion`, `.../compare`, `.../:id`, `.../:id/budget-preview`	accounting view
`POST /finance/forecasts`, `PATCH` and `DELETE /finance/forecasts/:id`, `PUT .../:id/drivers`, `PUT .../:id/overrides`	accounting use
`POST /finance/forecasts/:id/approve`, `.../reopen`, `.../to-budget`	accounting manage
`GET /finance/budgets/year`	accounting view; scope `all` or `company` needs all stores, a store id must be one of the person's stores
`PUT /finance/budgets/quarter`, `POST /finance/budgets/copy`	accounting use, with the same store rule
`GET /merchant/finance/payouts/:id/statement`, `.../statement.pdf`, `.../statement.csv`, `.../proof`	merchant users with the `money` right, own payouts only; the proof once the payout is paid

Routes that read or change salaries (payroll preview and run, staff pay, payslips, advances, bonuses, payroll tax lists) also need the `salaries` switch, declared as `@Access('accounting', 'use', { salaries: true })`.

The old `users.finance_level` column (view, prepare, approve) still exists in the database but no longer decides anything; the accounting level replaced it.

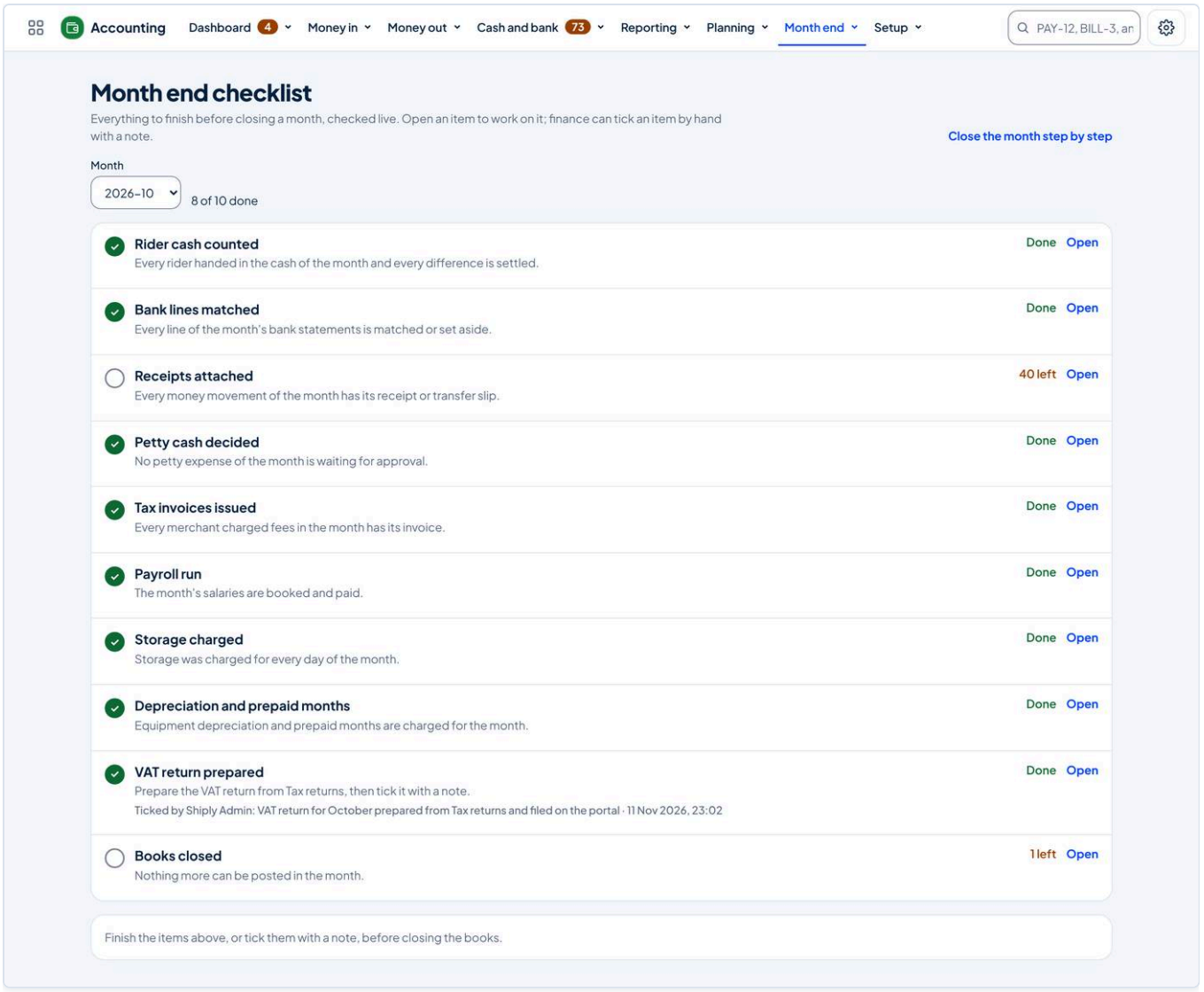
How a route is protected

The `AccessGuard` (`apps/api/src/auth/access.guard.ts`) runs on every request after the JWT guard. For a route with an `@Access` rule:

1. A rider or merchant passes only where the route's `@Roles` names them; otherwise `403 { error: 'forbidden' }`.
2. For a team member it loads the access profile fresh from the database, once per request (`loadAccess`), so a change made by an admin applies at once without signing in again.
3. The needed level is the rule's level, or view for `GET` and `HEAD` and use for the rest. The person must reach it in one of the rule's apps, and every level listed in `also`; otherwise `403 { error: 'no_access' }`.
4. `customerData` and `salaries` in the rule need the switches.
5. Any store id the request names (`:storeId`, the rule's `storeParam`, `?darkStoreId`) must be one of the person's stores (`store_not_allowed`), and any merchant id must be one of theirs when they are limited (`merchant_not_allowed`).
6. For merchant users, `@MerchantRight` checks the member's rights.

Lists and reports go further than the guard: a person limited to some stores sees only rows of their stores. List loaders receive a `ListScope` (`{ userId, storeIds, merchantIds }`, null meaning all) built by `scopeOf(user, access)`; single records outside the scope answer `store_not_allowed`. The cash position and the dashboard show such a person only their boxes, riders and handovers.

The web app applies the same rules to decide what to show: each page in `apps/web/src/lib/apps.ts` names the level it needs, menus hide pages a person may not open, and opening one directly shows not found. The API remains the authority.



An Accounting screen seen by a person with the view level: buttons that change data are hidden. Each person's levels, stores and switches are set in the access editor on the users page (`/admin/users` , Settings app).

5.2 The second person rule

> **RULE**
>
> Money said to come in, and changes to what we owe a merchant, wait for a second person while `finance.approve_money_in` is 1. Money going out waits for a second person from `finance.approval_threshold_piastres` . Nobody approves their own request or payout.

The reason is simple: if one person could record money coming in on paper and then pay it out as a payout or an expense, money could be created and taken. So every claimed money in is checked by someone else, and every large payment too.

Setting	Default	Meaning
`finance.approve_money_in`	1	Owner capital, other income, merchant payments to us, merchant balance changes, cheques received, deposits received, loans received, asset disposals and debit notes wait for a second person (1) or post at once (0).

Setting	Default	Meaning
`finance.approval_threshold_piastres`	1,000,000 (10,000 EGP)	Payouts, expenses, drawings and other money going out from this amount need a second person; 0 turns it off.

Post now or wait

Every money screen calls a `submit...` function instead of the posting function directly. `submit` in `controls.ts` decides:

```
const reviewed =
  payload.kind === 'credit_note' ||
  (alwaysReviewed(payload)
    ? (await getSetting(db, 'finance.approve_money_in')) === 1
    : await needsApproval(db, amount));
if (!reviewed) {
  const entry = await execute(db, payload, actorUserId);
  if (receiptPhotoId) await attachReceipt(db, { entryId: entry.id, photoId: receiptPhotoId, actorUserId
});
  return { status: 'posted' as const, entry };
}
return { status: 'pending' as const, request: await queueRequest(db, payload, actorUserId,
receiptPhotoId) };
```

The answer tells the screen what happened: `{ status: 'posted', entry }` or `{ status: 'pending', request }`. The web helper `useFinanceAction` shows "sent for approval" for a pending answer.

```
money in (capital, other income, merchant receipt, adjustment, cheque in,
          deposit in, loan in, disposal, debit note)
  └─ finance.approve_money_in = 1 ──▶ request: pending
  └─ finance.approve_money_in = 0 ──▶ posted at once

credit note ──────────────────────────▶ request: pending (always)

money out (expense, drawings, bill payment, tax payment, prepayment, asset,
           packaging, advance, bonus, payroll, petty cash, cheque out,
           deposit out, loan payment)
  └─ amount < limit (or limit 0) ──▶ posted at once
  └─ amount ≥ limit ──────────────▶ request: pending

request: pending ──approve (another person, accounting manage)──▶ approved ──▶ posted
                 ──reject (another person, with a note)─────────▶ rejected
                 ──cancel (the person who asked)────────────────▶ cancelled
```

Journal entries follow the same rule through `submitJournalEntry`: an entry that touches no bank or cash box posts at once; one that brings money into a bank or box waits while `finance.approve_money_in` is 1; one that takes money out waits from the limit. All checks of the entry run before it is queued, so a bad entry is refused when it is entered, not when the approver opens it.

finance_requests

A waiting movement is a row of `finance_requests`: `kind`, `amount_piastres`, `payload` (the movement exactly as it was entered, as JSON), `status` (`pending`, `approved`, `rejected`, `cancelled`), the requester, the decider,

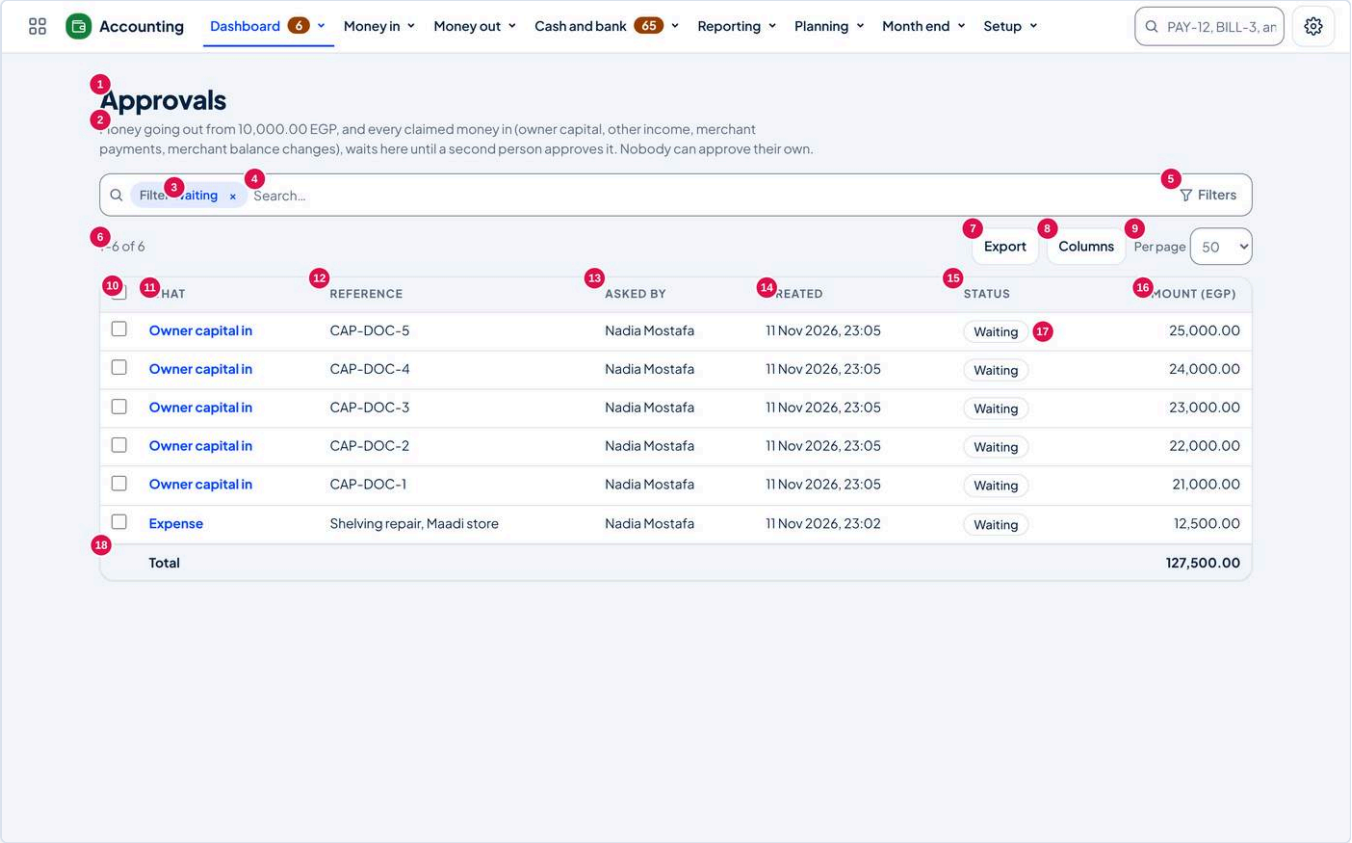
the time, the decision note and, once approved, the ledger entry it posted. Receipt photos can be attached to a request and follow it.

Request kinds (`RequestPayload` in `controls.ts`):

Direction	Kinds
Money in or merchant balance (always reviewed while approve_money_in is 1)	`capital_in` , `other_income` , `merchant_receipt` , `merchant_adjustment` , `cheque_in` , `deposit_in` , `loan_in` , `asset_disposal` , `debit_note` , and `deposit_out` when the deposit goes back into the merchant's balance
Always reviewed	`credit_note`
Money out (reviewed from the limit)	`expense` , `drawings` , `journal` (money part), `staff_advance` , `staff_bonus` , `payroll` , `bill_payment` , `tax_payment` , `prepayment` , `asset_purchase` , `packaging_purchase` , `petty_cash` , `cheque_out` , `deposit_out` , `loan_payment`

Rules of the lifecycle:

- One request per key. `queueRequest` takes an advisory lock on the movement's idempotency key and returns an existing request with the same key while it is pending or approved. A rejected or cancelled request never blocks asking again, so a corrected month's salaries can be sent after a rejection.
- Checked when entered. The day is checked with `eventTime` before the request is stored, so a future date fails at once.
- Never one's own. `decideFinanceRequest` refuses when the decider is the requester: `403 same_person` . Rejecting needs a note (`note_required`).
- Claimed once. The decision first runs an `updateMany ... where status = 'pending'` ; if it changed no row, someone else decided first and the answer is `request_not_pending` . Two approvers clicking at once cannot both post.
- Posted as entered, by the requester. On approval the stored payload is executed in the same transaction, with the requester as the author of the posting. Every check of the movement runs again then (a bank may no longer hold the money).
- Cancel by the requester only (`not_your_request`), while pending. A cancelled or rejected cheque request also cancels its cheque.
- Audited: `finance.request_create` , `finance.request_approve` , `finance.request_reject` , `finance.request_cancel` .



The approvals list at `/finance/approvals` : waiting requests and payouts with kind, amount, requester and day, a few rows ticked and the bulk bar showing Approve and Reject.

Payouts

Payouts to merchants follow the same idea without a request row. A payout is prepared as a draft (one open draft per merchant). From the approval limit, `approvePayout` must be called by another person than the one who prepared it (`same_person`), and `markPayoutPaid` refuses an unapproved large payout with `approval_needed` . Marking paid also checks that the merchant is still owed at least the payout (`payout_more_than_owed`) and that the bank holds the money (`not_enough_money`). Bulk approval approves only payouts at or above the limit (`approval_not_needed` for the others) and never one's own.

Marking a payout paid by transfer has its own second person control on top of the approval. When the payout is prepared its statement of account (`SOA-000001`) is frozen; before it is marked paid, the bank slip is uploaded as its proof of transfer (`POT-000001`) with the amount and reference typed from it, and every active payout check is ticked. `assertReadyToPay()` in `packages/db/src/finance/payout-proof.ts` , called by `markPayoutPaid` , refuses `pot_required` (no proof), `pot_amount_mismatch` (the proof's amount is not the payout's), `soa_mismatch` (no statement, or its net is not the payout's amount), `checks_incomplete` (an active check not ticked) and `403 same_person` when the person marking it paid is the one who ticked the built in check `second_person_viewed` ("Second person viewed the statement and the proof of transfer"). That check cannot be switched off and can be ticked only once the proof and the statement exist. Replacing the proof of a draft clears every tick; once paid, the proof and the ticks are fixed by a database trigger. A payout paid by cheque uses the cheque register as its proof instead. The full flow is in Merchant money, payouts (chapter 13).

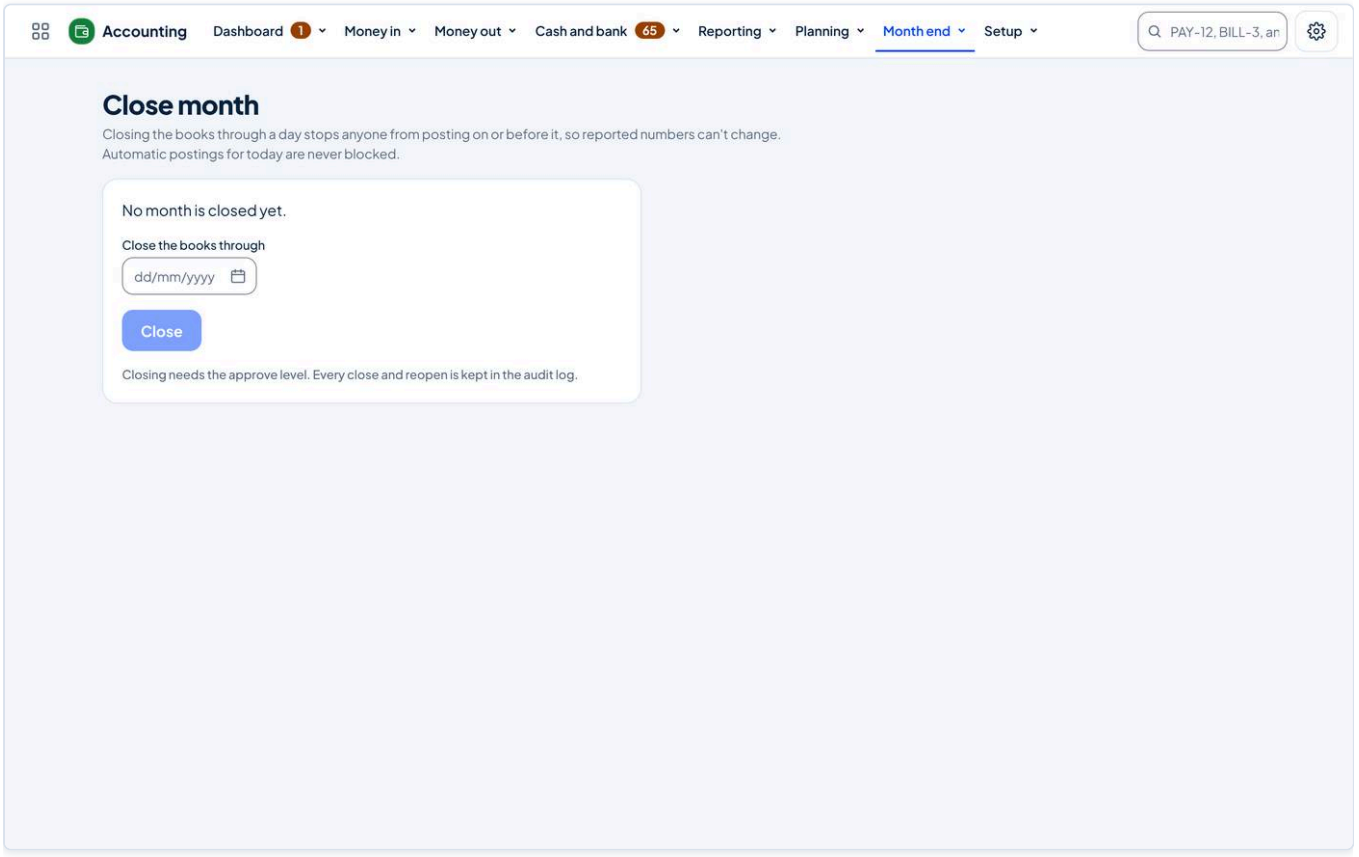
Other two person checks

Several flows have their own form of the rule, each refusing self decisions:

- Rider withdrawals: a rider never approves their own (`own_withdrawal`); from the approval limit the payer must not be the approver (`same_person`).
- Expense claims: nobody decides their own; from the limit the payer is not the approving manager.
- Basata deposits: finance never decides a deposit of their own (`own_deposit`).
- Purchase orders: approved at once below the limit, otherwise by another person with accounting manage.
- Petty cash: above `finance.petty_cash_limit_piastres` (500 EGP) or when the box would keep less than `finance.petty_cash_float_piastres` (1,000 EGP), the expense waits as a `petty_cash` request.

5.3 Books lock and reopening

The books are closed through a Cairo day (the ledger core chapter explains the table and the trigger). In the API, `POST /finance/books-lock` needs accounting manage. Moving the date back reopens days: the controller passes the role `admin` only when the person also has settings manage, and `closeBooks` then requires a reason of at least five characters. Both directions are audited (`finance.books_close` , `finance.books_reopen`) with the old and the new day. The month end checklist and the Close the month run end with this same lock; closing a year sets it through 31 December.



The Close the books page at `/finance/close` : the day the books are closed through, who closed them and when, the day picker, and the reason box that appears when an earlier day is chosen (reopening).

5.4 Audit log and History

> **RULE**
> Every action is audited: who, what, when, before and after.

`audit_log` rows hold `actor_user_id`, `action`, `entity_type`, `entity_id`, `before` and `after` (JSON) and `created_at`. The table is append only (trigger `audit_log_append_only`). Rows are written with `recordAudit(tx, ...)` in the same transaction as the change, so a change without its audit row cannot exist.

```
export function recordAudit(tx: Prisma.TransactionClient, entry: AuditEntry) {
  return tx.auditLog.create({
    data: {
      actorUserId: entry.actorUserId,
      action: entry.action,
      entityType: entry.entityType,
      entityId: entry.entityId,
      before: entry.before ?? undefined,
      after: entry.after ?? undefined,
    },
  });
}
```

Accounting actions are named `finance.<what>`: for example `finance.expense`, `finance.journal`, `finance.money_transfer`, `finance.payout_create`, `finance.payout_approve`, `finance.payout_paid`, `finance.payout_cancel`, `finance.cash_handover`, `finance.cash_handover_review`, `finance.bill_create`, `finance.bill_payment`, `finance.cheque_book`, `finance.credit_note`, `finance.debit_note`, `finance.invoice_issue`, `finance.receipt_attach`, `finance.account_create`, `finance.account_update`, `finance.opening_balances`, `finance.year_close`, `finance.month_end_mark`, `finance.payroll`. Contract changes use `merchant.contract_*`, chatter uses `record.note` and `record.activity`.

The History panel of a document page reads `GET /history/:entityType/:entityId` (`apps/api/src/admin/history.controller.ts`), newest first, up to 100 rows. Who may read it is decided by `RECORD_APPS` (`packages/shared/src/record-apps.ts`): money records (`payout`, `ledger_entry`, `bill`, `finance_request`, `cheque`, `tax_invoice`, `rider_withdrawal`, `petty_cash`, `expense_claim`, `basata_deposit`, `books` and others) need accounting view; merchants and riders can be read by their own app or accounting. Record types not listed are for settings manage. A known record must also be inside the person's stores and merchants. The full audit log is browsable in the Settings app at `/admin/audit`.

5.5 Idempotency

> **RULE**
> Every staff action sends a client generated idempotency key, so double taps and retries apply once. Background jobs are idempotent and safe to run twice.

Idempotency is enforced at three levels:

1. The ledger: `ledger_entries.idempotency_key` is unique and `post` returns the existing entry for a known key (see the ledger core chapter).
2. Documents: tables created by a person's action (cash handovers, rider withdrawals, Basata deposits, petty cash and others) store the request's idempotency key with a unique index, and the domain function returns the existing row for a repeat. Some movements answer a repeat with the first entry and write one audit row only (deposits, merchant receipts, adjustments).
3. Requests: `queueRequest` keeps one pending or approved request per key, under an advisory lock.

Jobs use keys built from the record and the period (a storage day, a month, a deadline day), so a job that runs on two instances at once, or again after a restart, changes nothing.

5.6 Error answers

A refusal answers JSON `{ error: code }` with a 4xx status (409 by default, 400 for bad input, 403 for access and same person, 404 for missing records). Some refusals carry plain values beside the code (for example `payout_more_than_owed` carries `amountPiastres` and `owedPiastres`). Input that fails its zod schema answers `400 { error: 'validation_failed', issues }`. The web app shows the English or Arabic message for the code from `finance.merchants.errors` in the messages files, or a generic message for an unknown code. The table lists every accounting error code with its English message as shown to the user.

Code	English message
`nothing_to_pay`	Nothing is owed to this merchant up to that day.
`payout_already_open`	This merchant already has a payout waiting.
`reason_required`	Write a reason of at least 5 characters.
`invalid_amount`	Enter an amount.
`more_than_cash_in_store`	That is more than the cash in this store.
`more_than_owed`	That is more than the rider has earned.
`handover_not_pending`	Someone already reviewed this.
`payout_not_draft`	This payout was already handled.
`validation_failed`	Check the fields.
`generic`	Something went wrong. Try again.
`account_name_taken`	An account with that name already exists in this group.
`bank_account_exists`	That bank account is already in the chart.
`bank_details_required`	Enter the bank name and account number.
`group_not_open`	Accounts in this group are opened by the system.
`group_full`	This group has no free numbers left.
`account_locked`	The system uses this account, so it cannot be closed or renamed here.
`account_has_balance`	Move its balance out first; only an empty account can be closed.
`account_inactive`	That account is closed.
`account_not_found`	Choose a bank or cash box.
`bank_account_not_found`	Choose a bank account.
`expense_account_not_found`	Choose an expense category.
`income_account_not_found`	Choose an income account.
`equity_account_not_found`	Choose an equity account.

Code	English message
`not_enough_money`	There is not that much money in the account it comes from.
`same_account`	From and to must be different accounts.
`future_date`	The date cannot be in the future.
`invalid_vat`	VAT must be less than the amount.
`not_balanced`	Debits must equal credits.
`invalid_line`	Each line needs an account and either a debit or a credit.
`wallet_not_allowed`	Merchant and rider wallets are changed from their own pages.
`money_account_not_allowed`	Riders' cash can't be changed by a journal entry. Use the cash handover screens.
`accrual_money_not_allowed`	An accrual can't include a bank or cash box, because it is reversed by itself next month.
`books_closed`	This date is in a closed month. Choose a later date, or ask an admin to reopen the month.
`approval_needed`	This payout needs a second person's approval first.
`same_person`	You prepared this, so another person must approve it.
`already_approved`	Already approved.
`finance_level_too_low`	Your finance access doesn't allow this. Ask an admin.
`request_not_pending`	Someone already handled this.
`not_your_request`	Only the person who asked can withdraw it.
`note_required`	Write why you reject it.
`reopen_admin_only`	Only an admin can reopen a closed month.
`day_not_over`	You can close only days that are over.
`opening_already_posted`	Opening balances were already posted. Correct them with a journal entry.
`account_twice`	An account appears twice.
`nothing_to_post`	Enter at least one amount.
`store_not_found`	Store not found.
`salary_not_set`	Set this person's salary first.
`more_than_advance`	That is more than the person owes as an advance.
`net_below_zero`	Deductions are larger than the salary.
`payroll_already_run`	This month's salaries were already paid.
`future_month`	A month that has not started yet cannot be paid.
`invalid_month`	Choose a valid month.
`person_twice`	A person appears twice.

Code	English message
`user_not_found`	Person not found.
`supplier_exists`	A supplier with this name already exists.
`supplier_not_found`	Supplier not found or closed.
`bill_exists`	This supplier invoice number is already entered.
`bill_not_found`	Bill not found.
`bill_not_open`	This bill is already paid or cancelled.
`expense_account_required`	Choose the expense category (or set one on the supplier).
`due_before_bill`	The due date cannot be before the bill date.
`invalid_wht`	Withholding tax must be between 0% and 100%.
`invalid_salvage`	What it is worth at the end must be less than its price.
`invalid_months`	Number of months is not valid.
`nothing_owed`	This merchant does not owe us anything.
`end_before_start`	The last month cannot be before the first.
`invalid_day_of_month`	Day of month must be 1 to 28.
`recurring_not_found`	Recurring expense not found.
`merchant_not_found`	Merchant not found.
`invalid_quarter`	Quarter not valid.
`invalid_date`	Date not valid.
`bank_line_not_found`	Bank line not found.
`bank_line_not_open`	This bank line is already matched or set aside.
`ledger_line_not_found`	Entry line not found.
`ledger_line_matched`	That entry already matches another bank line.
`amounts_differ`	The amounts are different.
`not_money_out`	Only money out can be bank charges.
`nothing_to_import`	No lines to import.
`year_not_over`	This year is not over yet.
`year_already_closed`	This year is already closed.
`earlier_year_open`	Close the earlier year first.
`invalid_year`	Year not valid.
`range_too_long`	Choose at most about a year.
`invalid_range`	The end is before the start.

Code	English message
`outsourced_not_on_payroll`	This person is paid by their outsourcing company, not by our payroll.
`employee_bonus_in_payroll`	Employees get bonuses in the salary run, where they are taxed.
`staffing_company_required`	Choose the outsourcing company.
`not_their_company`	This person is hired through another company.
`no_person_on_bill`	Advances are taken back only on a bill for one person.
`month_not_over`	That month is not over yet.
`invoice_not_found`	Invoice not found.
`more_than_invoiced`	More than is left on this invoice.
`invoice_not_rejected`	Only a rejected or invalid invoice can be sent again.
`invalid_tax_number`	A tax registration number has 9 digits.
`invalid_activity_code`	An activity code has 4 digits.
`invalid_national_id`	A national ID has 14 digits.
`invalid_iban`	An Egyptian IBAN is EG followed by 27 digits.
`invalid_rate`	Write the rate as a number, for example 48.50.
`invalid_currency`	Currency not valid.
`asset_not_found`	Equipment not found.
`asset_already_disposed`	Already sold or scrapped.
`disposal_before_charged_month`	Depreciation is already booked for that month or later: choose a later day.
`not_enough_cash_in_box`	The store box does not hold that much cash.
`petty_cash_reversed`	This petty expense was already reversed.
`petty_cash_not_posted`	Only a posted petty expense can be reversed.
`invalid_item`	This item cannot be ticked by hand.
`mark_not_found`	This item was not ticked.
`cheque_exists`	This cheque (bank and number) is already in the register.
`cheque_not_due`	This cheque is post dated: it can be cleared on its due day.
`cheque_not_open`	This cheque cannot do that in its current state.
`cheque_not_cancellable`	Only a cheque not yet deposited or cleared can be cancelled.
`cheque_amount_mismatch`	The bill changed since this cheque was written. Cancel it and write a new one.
`cheque_egp_only`	Cheques pay bills in EGP only.
`cheque_not_found`	Cheque not found.
`bank_name_required`	Enter the bank the cheque is drawn on.

Code	English message
`bank_account_required`	Choose our bank account the cheque is drawn on.
`party_name_required`	Enter who wrote the cheque.
`purpose_direction`	This purpose does not fit the direction.
`due_before_issue`	The due date cannot be before the issue date.
`more_than_deposit`	That is more than the deposit we hold.
`merchant_over_credit_limit`	The merchant is over the credit limit.
`tier_twice`	Each band needs its own starting order.
`tier_empty`	Give each band a fixed fee, a percentage or both.
`payout_not_found`	Payout not found.
`commission_not_payable`	This commission is not approved, already paid, or for another amount.
`below_payout_minimum`	What we owe is below the payout minimum of the merchant's contract; it waits until it reaches it.
`payout_more_than_owed`	The merchant is owed less than this payout now (fees, adjustments or credit notes came after it was prepared). Cancel it and prepare it again.
`more_than_available`	That is more than you can withdraw now.
`withdrawal_not_approved`	Approve the request before paying it.
`withdrawal_already_paid`	This withdrawal is already paid.
`withdrawal_not_requested`	This request can no longer be changed.
`withdrawal_not_open`	This request is already closed.
`own_withdrawal`	You cannot decide your own withdrawal.
`withdrawal_not_found`	Withdrawal not found.
`rider_office_not_found`	This supplier is not a rider office.
`later_month_prepared`	A later month is already prepared for this office. Months are prepared in order.
`office_has_riders`	Riders are still linked to this office. Unlink them on their HR file first.
`supplier_has_staff`	People on a monthly salary are hired through this supplier. A rider office takes only riders paid per delivery.
`invalid_commission`	The commission must be between 0 and 100%.
`more_than_they_owe`	That is more than the supplier owes us.
`bill_from_rider_office`	A rider office's commission bill cannot be cancelled.
`foreign_currency_account`	This bank account is in another currency: only transfers with the amount in that currency, bills in the same currency and loans paid in EGP elsewhere can use it.
`foreign_amount_required`	Type the amount in the other currency too.
`currency_mismatch`	The two accounts (or the bill and the account) are in different currencies.

Code	English message
`foreign_payment_unsupported`	A bill with withholding tax or an advance taken back is paid from an EGP account.
`currency_banks_only`	Only bank accounts can be held in another currency.
`not_foreign_account`	This account is in EGP: there is nothing to revalue.
`already_revalued`	This account was already revalued for that month.
`cost_tag_inactive`	This cost tag is switched off.
`cost_tag_not_found`	This cost tag no longer exists.
`cost_tag_exists`	A cost tag with this name already exists.
`loan_repaid`	This loan is fully repaid.
`instalment_paid`	This instalment is already paid.
`pay_in_order`	Pay the instalments in their order.
`first_payment_too_early`	The first instalment must come after the day the money came in.
`lender_required`	Choose the bank that lent the money.
`loan_not_found`	This loan was not found.
`invalid_fee_kind`	Choose a fee for every line.
`invalid_lines`	Add between 1 and 20 lines.
`invalid_quantity`	Enter a quantity of at least 1.
`merchant_not_allowed`	This merchant is not one of yours.
`store_not_allowed`	This store is not one of yours.
`budget_year_empty`	The year to copy from has no budget here.
`budget_year_not_empty`	That year already has a budget here.
`same_year`	Choose two different years.
`pot_required`	Upload the proof of transfer first.
`pot_amount_mismatch`	The amount on the slip is not the payout amount.
`soa_mismatch`	The statement net is not the payout amount.
`checks_incomplete`	Tick every payout check first.

Some screens keep their own short messages for codes in their context: the bank matching screen (`finance.bankMatch.errors`), the Pay merchants run (`finance.payRun.errors` , including `approval_not_needed`), the Record an expense form (`finance.quickExpense.errors` , checked in the browser before sending), rider pay (`riderPay.errors` , including `below_minimum` , `withdrawal_already_open` , `invalid_instapay`), Basata (`basata.errors` , including `own_deposit` , `deposit_not_pending` , `more_than_cash`), the store cash screens (`storeCash.errors` , `cashControl.pettyStore.errors`), the chatter (`documents.chatter.errors`) and bulk results in list views (`listView.errors`).

6 Merchant fees: contracts, prices and the postings of each event

This chapter explains how the system decides what a merchant pays for every delivered order, failed attempt, received unit, stored unit, returned unit and month, and exactly which ledger entries each event writes. Every amount is an integer number of piastres (100 piastres = 1 EGP). Every percentage is an integer number of basis points (bp: 10,000 bp = 100%). No floating point number ever reaches money.

6.1 Conventions used by every fee

The rounding helper

All percentage shares of money in the fee code use one helper, `share()` in `packages/db/src/finance/ledger.ts` (identical to `bpShare()` in `packages/shared/src/contracts.ts`):

```
share(amount, bp) = floor((amount * bp + 5000) / 10000)
```

This is "round half up" to the whole piastre for non negative amounts. Example: 12.5 piastres becomes 13, 12.49 becomes 12.

One entry per fee kind, VAT on each entry

Each fee is its own ledger entry with its own entry kind and its own idempotency key. The VAT of a fee is computed on that fee alone: `vat = share(fee, fee.vat_bp)`. The entry has three lines, built by `feeLines()` in `packages/db/src/finance/postings.ts`:

Account	Debit	Credit	Tags
2100 merchant wallet (`2100-nnnn`)	fee + vat		store, merchant, order (when the event has an order)
4101 Delivery and service fees		fee	same
2301 VAT payable		vat	same

Because VAT is rounded per entry, the VAT of an order can differ by a piastre from 14% of the sum of its fees. A developer rebuilding the system must keep the per entry rounding, because the monthly tax invoice adds up the VAT that was actually booked (chapter 13).

Rules every posting obeys

- Lines of 0 are dropped by `post()`. When every line is 0 nothing is written and `post()` returns null. So a fee of 0 (for example a shipping fee on a contract that has none) leaves no entry at all.
- The lines of an entry must add up to 0, otherwise `post()` throws; a deferred database trigger refuses an unbalanced entry as well.
- The same idempotency key posts once: a repeat returns the first entry. All keys in this chapter are unique per event, so a retried transaction or a job that runs twice changes nothing.

- Sign convention in the ledger: debit is positive, credit is negative. A merchant wallet is a liability; a credit balance means we owe the merchant, a debit balance means the merchant owes us.
- Tags: when the caller gives no store or merchant tag, `post()` takes them from the entry's order, then from the wallets on its lines, then from the rider's store (`entryTags()` in `ledger.ts`).

6.2 Merchant contracts v2: versions and the version in force

Each merchant has at most one contract (table `merchant_contracts` , numbered `CTR-000001` onward without gaps). A contract is made of versions (table `merchant_contract_versions` , numbered 1, 2, 3 per contract). A version holds the whole deal:

Group	Fields	Meaning of null
Nine prices	`shippingFeePiastres` , `orderFixedPiastres` , `orderPercentBp` , `codPercentBp` , `failedDeliveryPiastres` , `receivingPerUnitPiastres` , `storagePerUnitMonthPiastres` , `returnPerUnitPiastres` , `packagingChargePiastres`	null means the standard price (the global setting); 0 is an explicit price of zero
Failed share	`failedDeliveryPercentBp`	null: see the failed delivery rule (10.8)
Per product prices	rows in `merchant_contract_version_products` (version, product, price per unit), at most 5,000	a product without a row is charged nothing for this fee
Volume tiers	rows in `merchant_contract_version_tiers` , at most 20, each with `fromOrder` and nine nullable prices	null keeps the version's price
Payment terms	payout schedule, minimum payout, monthly minimum fee, security deposit, credit limit	null: the merchant's own schedule or the global setting
Promises	promise minutes (informative), delivery attempts included, stores and areas (informative)	null: the store setting `delivery.max_attempts`
Dates	`startDate` , `endDate` , `noticeDays` , `effectiveFrom`	no end date: open ended

Statuses and activation

```
draft ──activate(effectiveFrom)──▶ active ──a later version activated──▶ superseded
  │                                  (shown as "ended" once endDate has passed)
  └── deleted (only a draft)
```

`activateContractVersion()` in `packages/db/src/finance/contracts.ts` refuses: a version that is not a draft; an effective day before today (Cairo); an effective day before the effective day of the last activated version (`effective_before_last_version`); an effective day inside closed books (`books_closed`); an effective day after the version's end date. The previous active version becomes superseded. Activated versions and their product prices are frozen by a database trigger. The action is audited as `merchant.contract_activate` with the full terms before and after. When the version takes effect today, its agreed payout schedule is written on the merchant at once; a later effective day leaves it to the daily job `applyContractSchedules()` , once per version.

The version in force

> **RULE**
> Every charge reads the version in force on the Cairo calendar day of its event. `versionInForce(db, merchantId, day)` returns the activated version (status active or superseded) with the latest `effectiveFrom` not after that day; when two share the day, the higher version number wins. No version means standard prices for everything.

The event day for each fee:

Fee	Event moment
Shipping, per product, order, COD, packaging charge	the delivery moment (rider marks delivered)
Failed delivery	the failed attempt
Receiving	staff close the shipment (inbound manifest)
Storage	the last millisecond of the stored Cairo day
Return	staff hand the return parcel to the collector
Monthly minimum	each day of the month separately (10.12)

The version's dates (`startDate` to `endDate`) are different from its effective day. Tiers and the monthly minimum only count on days the dates cover (`covers()` in `contracts.ts`); prices of the version still apply outside them as long as it is the version in force.

6.3 Standard prices from the settings

The standard price of every fee is a global setting (scope `global` only; a merchant scope is refused with `scope_not_allowed`). Definitions and defaults live in `packages/shared/src/settings.ts` ; `standardPrices()` in `contracts.ts` reads them.

Setting key	Unit	Default in code	Used for
`fee.shipping_fee_piastres`	piastres per delivered order	0	shipping fee (often all inclusive)
`fee.order_fixed_piastres`	piastres per delivered order	0	order fee, fixed part
`fee.order_percent_bp`	bp of products value	0	order fee, percent part
`fee.cod_percent_bp`	bp of cash collected	0	COD fee
`fee.failed_delivery_piastres`	piastres per failed attempt	0	fixed failed delivery fee

Setting key	Unit	Default in code	Used for
`fee.failed_delivery_percent_bp`	bp of shipment charges	10000 (100%)	failed share for contracts with a shipping fee
`fee.receiving_per_unit_piastres`	piastres per unit received	0	receiving fee
`fee.storage_per_unit_month_piastres`	piastres per unit per month	0	storage fee (charged daily)
`fee.return_per_unit_piastres`	piastres per unit returned	0	return fee
`fee.packaging_charge_piastres`	piastres per delivered order	0	packaging charge (0: packaging is our own cost)
`fee.postal_bp`	bp	1000 (10%)	The postal authority's share of our fees before VAT per shipment; our cost, never charged to the merchant
`fee.vat_bp`	bp	0 in code; set to 1400 (14%) by the owner on 11 November 2026	VAT added on top of every fee
`finance.payout_minimum_piastres`	piastres	0	minimum payout when the contract has none (chapter 13)

The mapping between each price, its setting, its column on a tier and its entry kind is the constant `CONTRACT_PRICES` in `packages/shared/src/contracts.ts`.

6.4 Resolving a price field by field

`resolvePrices(version, tier, standard)` resolves each of the nine prices independently:

```
price[k] = tier[k]  ?? version[k]  ?? standard[k]
           (band)     (contract)     (global setting)
```

The function used by the postings is `contractPricesAt(db, { merchantId, at, orderId? })`, which calls `pricesForDay()`. It returns the resolved prices, the failed delivery rule (10.8), the id of the version (for per product prices), the contract label, the band used (`tierFrom`) and the order's place in the month (`nth`).

The all inclusive preset

Most merchants pay one shipping fee that covers everything. The preset `allInclusivePrices()` (constant `ALL_INCLUSIVE_ZERO_KEYS`) sets order fixed, order percent, COD, receiving, storage, return and packaging charge to an explicit 0 (null would mean the standard price), clears the fixed failed fee (null) and sets `failedDeliveryPercentBp` to 10000 (failed attempts cost 100% of the shipment charges).

6.5 Volume tiers

A tier says: "from the nth delivered order of the Cairo month, these prices replace the version's". `tierFor(tiers, nth)` picks the tier with the highest `fromOrder` not above `nth`; before the first band there is none.

> **RULE**
> The order that crosses into a band and every later order of the month pay the band's prices. Earlier orders are never charged again and never rebated. The count restarts each Cairo month.

How nth is counted

- For a delivered order: the merchant row is locked (`SELECT ... FOR UPDATE`) so two deliveries at the same moment count one after the other, then `nth = (delivered orders of the merchant with deliveredAt from the start of the Cairo month up to now, this order excluded) + 1`.
- For any other charge (failed delivery, receiving, storage, return): `nth` is the place the next delivered order would take at that moment, the same count without locking.
- `nth` is only computed when the version in force has tiers and its dates cover the day; otherwise no tier applies.
- Per product prices never take tiers.

> **WORKED EXAMPLE**
> Version: order fixed 15 EGP (1,500 piastres), tier from order 501 with fixed fee 10 EGP (1,000). In November the merchant's 500th delivered order pays 1,500 piastres; the 501st and every later November order pay 1,000; on 1 December the count restarts and the first order pays 1,500 again. Orders 1 to 500 are not recharged.

6.6 Fee kinds

Entry kind	Formula (before VAT)	Charged when	Idempotency key
`shipping_fee`	price `shippingFeePiastres`	delivered	`order:<orderId>:shipping_fee`
`product_handling`	sum over delivered (picked) lines of `pickedQty * productPrice`; products without a price add 0	delivered	`order:<orderId>:product_handling`
`order_fee`	`orderFixed + share(products, orderPercentBp)`, products = sum of `pickedQty * unitPricePiastres`	delivered	`order:<orderId>:order_fee`
`cod_fee`	`share(cashCollected, codPercentBp)`	delivered	`order:<orderId>:cod_fee`
`packaging_charge`	price `packagingChargePiastres`	delivered	`order:<orderId>:packaging_charge`

Entry kind	Formula (before VAT)	Charged when	Idempotency key
`failed_fee`	see 10.8	each failed attempt	`delivery:<deliveryId>:failed_fee` (entries made before this key existed keep `order:<orderId>:failed_fee`)
`receiving_fee`	`sum(receivedQty) * receivingPerUnit`	shipment closed	`manifest:<manifestId>:receiving_fee`
`storage_fee`	see 10.10	each Cairo day, per store and merchant	`storage:<day>:<storeId>:<merchantId>`
`return_fee`	`sum(pickedQty) * returnPerUnit`	return handed over	`return:<returnId>:return_fee`
`minimum_fee`	top up to the monthly minimum (10.12)	first run after the month ends	`minimum_fee:<merchantId>:<YYYY-MM>`

Units cancelled before delivery (`pickedQty` 0) are never charged by the products based fees. The customer's delivery fee (what the shopper paid for 60 minute delivery) is not ours: it is part of the cash collected and goes to the merchant wallet.

6.7 The delivery posting

When the rider marks an order delivered (`packages/db/src/orders.ts`), the same database transaction updates the delivery (status, `codCollectedPiastres`), moves the order to delivered, raises a `cod_mismatch` alert when the collected cash differs from the expected COD, and calls `postDeliveryCompleted()` in `postings.ts`. That function writes these entries in this order:

1. `cod_collected`: the cash now belongs to the merchant.
2. `shipping_fee`, `product_handling`, `order_fee`, `cod_fee`, `packaging_charge` (each skipped when 0).
3. `postal_fee`: our cost, the postal authority's share of those five fees (see Postal fee below).
4. `rider_bonus`: the rider's pay for the delivery.
5. `rider_office_commission`, only when the rider is linked to an active rider office (chapter 11).

Account	Debit	Credit	Tags
1300 rider cash (`1300-nnnn`)	cash collected		store, merchant (from the order), rider
2100 merchant wallet		cash collected	same

Idempotency key: `delivery:<deliveryId>:cod`; entry kind: `cod_collected`. A prepaid order has cash 0, so no COD entry and a COD fee of 0.

Account	Debit	Credit	Tags
5101 Rider pay per delivered order	pay		store, merchant, rider

Account	Debit	Credit	Tags
2200 rider wallet (`2200-nnnn`)		pay	same

Idempotency key: `delivery:<deliveryId>:bonus` ; entry kind: `rider_bonus` . Pay = setting `rider.bonus_per_delivery_piastres` (default 0, store override allowed) of the order's store.

WORKED EXAMPLE

Contract in force: order fee 15 EGP fixed plus 2% of products, COD fee 1%, shipping fee 0, packaging charge 0, no per product prices, VAT 14%. Order: 2 units at 250 EGP, products value 50,000 piastres. The customer pays 550 EGP cash (products plus a 50 EGP delivery fee set by the merchant): cash collected 55,000 piastres. Rider pay 25 EGP (2,500). The rider belongs to an office with a 10% commission.

- Order fee = 1,500 + share(50,000, 200) = 1,500 + 1,000 = 2,500; VAT = share(2,500, 1400) = 350.
- COD fee = share(55,000, 100) = 550; VAT = share(550, 1400) = 77.
- Postal fee = share(2,500 + 550, 1000) = 305.
- Office commission = share(2,500, 1000) = 250.

Account	Debit	Credit	Tags
1300 rider cash	550.00		order, store, merchant, rider
2100 merchant wallet		550.00	same
2100 merchant wallet	28.50		order fee entry
4101 fee income		25.00	
2301 VAT payable		3.50	
2100 merchant wallet	6.27		COD fee entry
4101 fee income		5.50	
2301 VAT payable		0.77	
5103 Postal fees	3.05		postal fee entry
2307 Postal fees owed		3.05	
5101 rider pay	25.00		rider bonus entry
2200 rider wallet		25.00	
5102 rider office commission	2.50		office commission entry
2500 office supplier wallet		2.50	

Six separate entries (amounts in EGP). Keys: `delivery:D:cod` , `order:O:order_fee` , `order:O:cod_fee` , `delivery:D:postal_fee` , `delivery:D:bonus` , `delivery:D:office_commission` . The merchant's balance moves by 55,000 – 2,850 – 627 = 51,523 piastres (515.23 EGP) in their favour. The shipping fee, per product and packaging entries are not written because they are 0.

6.8 Postal fee

> **RULE**
> The postal authority takes `fee.postal_bp` (default 1000 = 10%) of our fees before VAT on every shipment. It is our cost: it is never charged to the merchant and carries no VAT.

`postPostalFee()` in `packages/db/src/finance/postings.ts` runs in the same transaction as the event, right after the fees of that shipment:

```
delivered order:      base = shippingFee + productHandling + orderFee + codFee + packagingCharge
                      (the five fees just charged on the order, each before VAT)
each charged failed attempt:  base = failedFee (before VAT)
storage, receiving and return fees: no postal fee (they are not shipments)
postal = share(base, fee.postal_bp)          (the same rounding helper as every fee, 10.1)
```

Account	Debit	Credit	Tags
5103 Postal fees	postal		order, store, merchant (taken from the order by `post()`)
2307 Postal fees owed		postal	same

Idempotency keys: `delivery:<deliveryId>:postal_fee` for a delivered order and `delivery:<deliveryId>:failed_postal_fee` for a failed attempt; entry kind `postal_fee`; memo the order number; data holds the delivery, the base and the rate used. A base or rate of 0 writes nothing (lines of 0 are dropped), and a repeat changes nothing. The setting is global only (no store override), read at the moment of the event.

> **WORKED EXAMPLE**
> A delivered order whose fees before VAT add up to 100.00 EGP (10,000 piastres), VAT 14%. The merchant's wallet goes down by 114.00 EGP exactly as without the postal fee (fees 100.00 to 4101, VAT 14.00 to 2301). Separately the postal entry books share(10,000, 1000) = 1,000 piastres: Dr 5103 Postal fees 10.00, Cr 2307 Postal fees owed 10.00. Our income stays 100.00; the postal authority is owed 10.00. In the example of 10.7 (order fee 2,500 and COD fee 550) the postal fee is share(3,050, 1000) = 305 piastres.

Paying: the postal fees owed are paid once a month from Taxes as tax kind `postal` (`payTax()` in `packages/db/src/finance/payables.ts`, `TAXES.postal = postal_fee_payable`, through `submitTaxPayment()`): never more than owed (`more_than_owed`) nor more than the bank or box holds (`not_enough_money`), the second person rule from the approval limit, and files chosen in the form. The entry is kind `tax_payment` (Dr 2307, Cr bank or box), memo "Postal fees" and the reference, key `tax_payment:<clientKey>`. `taxesOwed()` shows the amount owed as `postalPiastres`.

Where else it shows: Coming up and due lists it as kind `postal` (what was owed at the start of the month due at its end, the rest at the end of next month; chapter 15); the four week cash forecast counts it with the taxes in week 0; merchant profit shows it as a cost line of each merchant (lines on 5103 tagged with the merchant, chapter 13); the forecast suggests `postal_per_order` from the books and budgets the line to 5103 (chapter 15); the journal list puts `postal_fee` in the deliveries family. It is never part of the VAT report or VAT return, never on an invoice and never on the merchant's statement.

6.9 The failed delivery rule

A failed attempt is charged on every attempt (one entry per delivery row), by `postDeliveryFailed()` in `postings.ts`, inside the same transaction as the failure. The rule is `failedDeliveryRule(version, tier, resolvedPrices, standardPercentBp)` in `packages/shared/src/contracts.ts`, applied in this order:

1. A fixed failed fee set on the band or on the version (0 included) is used as it is.
2. Else the version's `failedDeliveryPercentBp`, as a share of the shipment charges.
3. Else, when the resolved shipping fee is above 0, `fee.failed_delivery_percent_bp` (default 10000) of the shipment charges.
4. Else the resolved fixed failed fee (the standard `fee.failed_delivery_piastres`); contracts without a shipping fee behave as before shipping fees existed.

The shipment charges are computed by `shipmentCharges()`:

```
shipment = shippingFee
         + productHandling(picked units at their product prices)
         + orderFixed
         + share(productsValue, orderPercentBp)
failedFee = (mode fixed)   ? fixedPiastres
          : (mode percent) ? share(shipment, bp)
```

The COD fee and the packaging charge are never part of the shipment charges (no cash was collected; packaging may be reused). Prices come from `contractPricesAt()` without an order id, so the band is the one the next delivered order of the month would take.

Account	Debit	Credit	Tags
2100 merchant wallet	fee + VAT		order, store, merchant
4101 fee income		fee	
2301 VAT payable		VAT	

Idempotency key: `delivery:<deliveryId>:failed_fee`; entry kind: `failed_fee`; data holds the reason, the mode and, for percent mode, `percentBp` and `shipmentPiastres`. A failed attempt earns the rider nothing. Each charged attempt also books the postal fee on its failed fee (key `delivery:<deliveryId>:failed_postal_fee`, see Postal fee).

> **WORKED EXAMPLE**
> All inclusive contract: shipping fee 60 EGP (6,000 piastres), everything else 0, failed share 100%. Shipment charges = 6,000 + 0 + 0 + share(50,000, 0) = 6,000. Failed fee = share(6,000, 10000) = 6,000; VAT = share(6,000, 1400) = 840. The merchant is charged 68.40 EGP for the attempt. With a share of 5,000 bp the fee would be 3,000 and VAT 420. With a fixed failed fee of 20 EGP set on the version (and no share), the fee is 2,000 whatever the shipment.

6.10 Receiving fee

`postReceivingFee()` runs when staff close an inbound manifest (`packages/db/src/inbound.ts`), in the closing transaction. Units are all `receivedQty` of the manifest lines, including extra units and products not listed. Price

is `receivingPerUnitPiastres` of the version in force at the closing moment, at the band of the next delivered order.

```
fee = sum(line.receivedQty) * receivingPerUnitPiastres
vat = share(fee, fee.vat_bp)
```

Posting: the standard fee lines (merchant wallet debit, fee income and VAT credit), store tag = the manifest's store. Idempotency key `manifest:<manifestId>:receiving_fee`; kind `receiving_fee`.

WORKED EXAMPLE

120 units received at 2 EGP: fee 24,000 piastres, VAT 3,360; merchant debited 27,360 (273.60 EGP).

6.11 Storage, charged daily with catch up

Formula

`accrueStorage(db, day, units)` in `postings.ts` charges one Cairo day for every store and merchant pair. Units are every unit of the merchant in every bin of the store at the end of the day, sellable or not (receiving, damaged and held bins included).

```
daysInMonth = number of days of the calendar month of `day`
fee = floor((units * storagePerUnitMonthPiastres + floor(daysInMonth / 2)) / daysInMonth)
vat = share(fee, fee.vat_bp)
skip the pair when fee <= 0
```

This is the monthly price divided by the days of the month, rounded half up per store, merchant and day. The prices are those of the version in force on that day at the band the month's deliveries reached (next delivered order). The entry is dated the last millisecond of the day (`cairoDayStart(day + 1) - 1 ms`), so statements show it on that day.

Account	Debit	Credit	Tags
2100 merchant wallet	fee + VAT		store, merchant
4101 fee income		fee	store, merchant
2301 VAT payable		VAT	store, merchant

Idempotency key `storage:<YYYY-MM-DD>:<storeId>:<merchantId>`; kind `storage_fee`; data holds day, store, units and monthly price. A day inside closed books is skipped silently (`postUnlessClosed()` swallows `books_closed`), never retried.

WORKED EXAMPLE

300 units at 1.50 EGP per unit per month (150 piastres). In November (30 days): floor((45,000 + 15) / 30) = 1,500 piastres per day. In December (31 days): floor((45,000 + 15) / 31) = 1,452 piastres, VAT share(1,452, 1400) = 203.

Units of a past day and the catch up

The free hosting sleeps, so the daily job may run late. `chargeStorageDays()` in `packages/db/src/finance/storage.ts`:

1. finds the last day already done (the newest `storage_days` row, or before that table existed, the newest day in a `storage_fee` entry);
2. charges every day after it up to the last finished Cairo day, at most `storage.catch_up_days` days (default 7, global; older missed days stay uncharged);
3. for each day, reads the units of that day from the snapshot table `stock_unit_days`, or works them out once from the stock and writes the snapshot (first writer wins);
4. charges the day and writes a `storage_days` row (also for a day skipped by closed books).

Units of a past day are exact because stock only changes through append only stock movements: `unitsAtDayEnds()` takes the units held now per store and merchant and undoes every movement (into or out of a bin) made after the end of that day. A negative result counts as 0. The snapshot is written for every merchant, also those paying no storage, because the merchant profit report shares rent by these unit days (chapter 13).

The API job `runStorageCharge()` (`apps/api/src/jobs/jobs.service.ts`) runs it under a Postgres advisory lock (`pg_try_advisory_xact_lock`), at most once per finished day per process.

6.12 Return fee

`postReturnFee()` runs when staff hand a return parcel to the merchant's collector (`packages/db/src/returns.ts`). Units are the sum of `pickedQty` of the return lines.

```
fee = sum(line.pickedQty) * returnPerUnitPiastres
```

Standard fee lines; store tag = the return's store. Idempotency key `return:<returnId>:return_fee`; kind `return_fee`.

6.13 Monthly minimum fee

A version may set `minimumMonthlyFeePiastres`. After a month ends, `chargeMinimumFees()` in `contracts.ts` (job `runContractMinimums()`, once per Cairo day, under an advisory lock) tops the month up.

```
for each day d of the month:
    v = versionInForce(merchant, d)
    if v has a minimum and v's dates cover d: sum += v.minimum; days += 1
minimum = days > 0 ? round(sum / daysInMonth) : 0        (Math.round)
fees    = fee income (4101, before VAT) tagged with the merchant in the month,
          minimum_fee entries excluded (credit notes and adjustments reduce it)
topUp   = minimum − fees;   charge only when topUp > 0
vat     = share(topUp, fee.vat_bp)
```

A minimum is therefore shared out by days: a contract that starts on the 16th of a 30 day month with a 3,000 EGP minimum gives a minimum of round(15 × 300,000 / 30) = 150,000 piastres. Before charging on the first

day after the month, the month's last storage day is charged first (`accrueStorage(db, last)`), so it counts in the fees. The entry is dated the last millisecond of the month; a month inside closed books is skipped.

Account	Debit	Credit	Tags
2100 merchant wallet	topUp + VAT		merchant
4101 fee income		topUp	merchant
2301 VAT payable		VAT	merchant

Idempotency key `minimum_fee:<merchantId>:<YYYY-MM>` ; kind `minimum_fee` .

> **WORKED EXAMPLE**
> Minimum 5,000 EGP for all of November; fees booked 3,200 EGP. Top up 180,000 piastres, VAT share(180,000, 1400) = 25,200. The merchant is debited 2,052 EGP on 30 November 23:59:59.999 Cairo time.

6.14 VAT on top

VAT is never part of a contract. It is the single global setting `fee.vat_bp` read at the moment each fee is booked, added on top of every fee kind (including the minimum top up). Contract screens show it but cannot change it. The VAT credited to 2301 on fees is the output VAT of the VAT report and return (chapter 15).

6.15 Damage compensation and rider charge

Units go to the damaged bin only through a damage report with a cause (`arrived_damaged` , `store_handling` , `delivery` , `customer` , `expired`). Admin settles it with `settleDamage()` in `packages/db/src/damage.ts` , confirming or changing the cause (a change needs a note). The money moves in the same transaction; settling twice is refused.

```
companyPays  = cause in ('store_handling', 'delivery')        (COMPANY_CAUSES)
compensation = companyPays ? quantity * product.pricePiastres : 0
riderCharge  = chargeRider ? compensation : 0
               (chargeRider allowed only for cause 'delivery' with a rider on the report)
```

No VAT: compensation is not a fee.

Account	Debit	Credit	Tags
5201 Damage paid by us	compensation		store, merchant
2100 merchant wallet		compensation	store, merchant

Idempotency key `damage:<reportId>:compensation` ; kind `damage_compensation` .

Account	Debit	Credit	Tags
1350 rider dues (`1350-nnnn`)	riderCharge		store, rider
5201 Damage paid by us		riderCharge	store, rider

Idempotency key `damage:<reportId>:rider_charge`; kind `damage_rider_charge`. The rider owes it and it is taken from their next withdrawal (chapter 11).

> **WORKED EXAMPLE**
> 3 units of a product priced 200 EGP, cause delivery, rider charged: compensation 60,000 piastres credited to the merchant; 60,000 moved onto the rider's dues; the net cost to the company is 0.

6.16 Credit limit

`assertWithinCreditLimit()` refuses a new shipment while the merchant owes us more than the version's `creditLimitPiastres` (error `merchant_over_credit_limit`), and raises one `merchant_over_credit_limit` alert per merchant and Cairo day (key `credit_limit:<merchantId>:<day>`). No limit set means no check.

6.17 The contract calculator

The contract editor, the sales proposals and the merchant profit what if use pure functions in `packages/shared/src/contracts.ts` that mirror the postings:

- `orderCharge(terms, standard, vatBp, input)`: one shipment at its band, every fee and the VAT computed per fee (as booked), plus what the same shipment costs if the attempt fails.
- `monthEstimate(...)`: orders spread evenly over the month (bands walked from where each starts), storage day by day at the band reached so far (`soFar = floor(orders * d / days)`), the minimum topping the fees up, VAT on the month total.
- `whatIf(base, params)`: recomputes shipping, order and COD fees for a booked month, keeps other fees, applies a minimum.

6.18 Where the code lives

Concern	File and function
Fee postings	`packages/db/src/finance/postings.ts`: `feeLines`, `postDeliveryCompleted`, `postDeliveryFailed`, `postPostalFee` (not exported), `postReceivingFee`, `postReturnFee`, `accrueStorage`
Contract resolution	`packages/db/src/finance/contracts.ts`: `standardPrices`, `versionInForce`, `contractPricesAt`, `pricesForDay`, `productPricesFor`, `activateContractVersion`, `minimumForMonth`, `chargeMinimumFees`, `assertWithinCreditLimit`
Pure price rules	`packages/shared/src/contracts.ts`: `CONTRACT_PRICES`, `tierFor`, `resolvePrices`, `failedDeliveryRule`, `shipmentCharges`, `productHandlingCharge`, `bpShare`
Storage days	`packages/db/src/finance/storage.ts`: `chargeStorageDays`, `unitsAtDayEnds`, `unitsForDays`
Rider office commission	`packages/db/src/finance/rider-offices.ts`: `postRiderOfficeCommission`
Damage	`packages/db/src/damage.ts`: `settleDamage`

7 Cash on delivery, riders, store boxes and rider pay

This chapter follows the physical cash from the customer's hand to the bank, and the rider's earnings from the delivery to the withdrawal. It also covers what riders owe, Basata deposits, petty cash paid from store boxes and the rider supply offices. Amounts are integer piastres; `share(a, bp) = floor((a * bp + 5000) / 10000)` as in chapter 10.

```
customer pays COD
       |  cod_collected (at delivery)
       ▼
1300 rider cash ──cash_handover──▶ 1200 store box ──bank_deposit──▶ 1100 bank
       |                                  |
       |                                  └─petty_cash──▶ 54xx/55xx expense
       |
       ├──basata_deposit (approved)──▶ 1200 Money at Basata ──basata_settlement──▶ 1100 bank
       |                                                          └─▶ 55xx Basata fees
       ├──difference written off──▶ 5301 cash shortages and surpluses
       ├──shortage kept on rider──▶ 1350 rider dues
       └──deadline charge──▶ 1350 rider dues, or 2500 the rider office's wallet

rider earns per delivery: 5101 rider pay ──▶ 2200 rider wallet ──rider_withdrawal──▶ bank or box
                                                  (dues 1350 and advance 1500 netted)
```

7.1 Accounts used

Code	Account	Owner	Normal balance
`1300-nnnn`	Rider cash: COD collected and not handed in	one per rider	debit (asset)
`1350-nnnn`	Rider dues: what the rider owes the company	one per rider	debit (asset)
`12nn`	Store cash box	one per dark store (next free code of group 1200)	debit
`12nn`	Money at Basata, on its way to the bank (system key `basata_transit`)	company	debit
`2200-nnnn`	Rider wallet: pay earned and not withdrawn	one per rider	credit (liability)
`1500-nnnn`	Staff advance	one per team member	debit
`2500-nnnn`	Supplier wallet, also used for a rider office	one per supplier	credit
5101	Rider pay per delivered order	company	expense

Code	Account	Owner	Normal balance
5102	Rider office commission	company	expense
5301	Cash shortages and surpluses	company	expense
`55nn`	Basata fees (next free code of group 5500)	company	expense

7.2 COD collection

At delivery the rider enters the cash collected. The delivery transaction posts `cod_collected` (Dr 1300 rider cash, Cr 2100 merchant wallet, key `delivery:<deliveryId>:cod`, see chapter 10). When the collected amount differs from the expected COD a `cod_mismatch` alert is created; the posting always uses what was collected.

7.3 Rider cash limit at handover to the rider

Before a cash order is handed to a rider, `checkRiderCashLimit()` in `packages/db/src/finance/cash-control.ts` locks the rider row and checks:

```
cash     = max(0, balance(1300 rider cash) − pending Basata deposits)
carrying = sum(codExpectedPiastres) of the rider's deliveries in status assigned or picked_up
total    = cash + carrying + this order's COD
refuse with rider_cash_limit when total > finance.rider_cash_limit_piastres
```

Setting `finance.rider_cash_limit_piastres`: default 500,000 (5,000 EGP), store override, 0 turns the check off. Prepaid orders are not checked. A store manager (store manage) may hand it over anyway with a reason of at least 5 characters; the override is audited as `order.rider_cash_override` with the amounts. A frozen rider (11.8) gets no orders at all.

7.4 Cash age and the overdue alert

Cash is handed in oldest first, so the cash a rider still holds is made of the newest collections. `heldPieces(balance, collections)` in `packages/shared/src/cash-control.ts` walks the rider's positive 1300 lines newest first and takes from each until the balance is covered; any balance with no collection behind it counts as collected at time 0 (very old).

```
pieces = []
left = balance
for c in collections (newest first):
    take = min(left, c.amount); pieces.push(take at c.at); left −= take
if left > 0: pieces.push(left at epoch)
```

Buckets for Cash position: fresh (younger than `finance.rider_cash_warn_hours`, default 4), aging (up to the maximum), overdue (older than `finance.rider_cash_max_hours`, default 12, store override). The job `checkRiderCashOverdue()` runs every ten minutes and raises one `rider_cash_overdue` alert per rider and Cairo day (key `rider_cash_overdue:<riderId>:<day>`) when any piece is older than the maximum. Pending Basata deposits count as handed in for this check (they cover the oldest cash).

7.5 Cash handover to the store box

What the rider is expected to hand in

`cashToHandIn()` in `packages/db/src/finance/operations.ts`:

```
expected = max(0, balance(1300 rider cash)
                  − sum(differencePiastres of this rider's handovers still pending)
                  − sum(amountPiastres of this rider's Basata deposits still pending))
```

A pending difference stays in rider cash until finance decides, so it must not be expected again.

Recording the handover

Staff count the cash and enter the total. `recordCashHandover()` refuses when `cash.store_handover_enabled` is 0 for the store (`store_handover_closed`) or the rider belongs to another store, locks the rider row, computes `difference = expected − counted` and creates a handover row `CH-000001` onward (gapless) with status `balanced` when the difference is 0, otherwise `pending`. The counted cash moves at once:

Account	Debit	Credit	Tags
12nn store cash box	counted		store (from the box), rider
1300 rider cash		counted	

Idempotency key `handover:<handoverId>`; kind `cash_handover`. The staff action also carries its own client idempotency key, stored on the handover row.

Finance decides a difference

`reviewCashHandover()` takes a pending handover, a decision and a note:

- `write_off`: the difference becomes a company loss (shortage) or gain (surplus).
- `charge_rider`: a shortage moves to the rider's dues and is taken from their next withdrawal; a surplus is only marked (it stays in rider cash, no entry).

Account	Debit	Credit	Tags
5301 cash shortages and surpluses	difference (shortage)	|difference| (surplus)	rider, store
1300 rider cash	|difference| (surplus)	difference (shortage)	

Write off: key `handover:<id>:write_off`, kind `cash_difference`.

Account	Debit	Credit	Tags
1350 rider dues	shortage		rider, store
1300 rider cash		shortage	

Charge rider: key `handover:<id>:charge_rider`, kind `rider_shortage_charge` (`chargeShortageToRider()` in `rider-wallet.ts`).

WORKED EXAMPLE

The rider holds 1,200 EGP (120,000 piastres), no pending items. Staff count 1,150 EGP. Handover entry: Dr store box 115,000, Cr rider cash 115,000; difference 5,000 pending (rider cash still shows 5,000). Finance charges the rider: Dr 1350 5,000, Cr 1300 5,000. Rider cash is now 0 and the rider owes 50 EGP, netted from the next withdrawal.

7.6 Bank deposit from a store box

`recordBankDeposit()`: finance takes cash from a store box to a bank (default CIB 1101). The box row is locked and the amount may not exceed what it holds (`more_than_cash_in_store`).

Account	Debit	Credit	Tags
1101 bank (or the chosen 11nn)	amount		store (from the box)
12nn store cash box		amount	

Idempotency key `deposit:<clientKey>`; kind `bank_deposit`. A repeat answers the first entry.

7.7 Petty cash from a store box

Store staff (store use) pay a small expense from their store box with `submitPettyCash()` (`cash-control.ts`): an expense category finance marked as petty (an active account of groups 5400 or 5500 with `petty` on), a note of at least 3 characters, a required receipt photo (photo kind `receipt`) and an optional cost tag. Number `PC-000001` onward.

```
box        = locked balance of the store box
refuse not_enough_cash_in_box when amount > box
overLimit  = limit > 0 and amount > finance.petty_cash_limit_piastres   (default 50,000, store
override)
belowFloat = box − amount < finance.petty_cash_float_piastres           (default 100,000, store
override)
if overLimit or belowFloat: wait as a petty_cash finance request (second person)
else: post at once
```

Account	Debit	Credit	Tags
chosen expense category	amount		store, cost tag
12nn store cash box		amount	store

Idempotency key `petty:<pettyId>`; kind `petty_cash`. When approved later, the box is checked again. Reversal by accounting manage with a reason of at least 5 characters: the same lines turned round, key `petty_reversal:<pettyId>`, kind `petty_cash_reversal`.

WORKED EXAMPLE

Limit 500 EGP, float 1,000 EGP, the box holds 1,300 EGP. A 250 EGP expense leaves 1,050: posted at once. A 400 EGP expense would leave 900, below the float: it waits for finance.

7.8 Basata deposits

Riders may deposit cash at a Basata branch instead of the store (`packages/db/src/finance/basata.ts`). Both channels can be closed per store: `cash.basata_enabled` and `cash.store_handover_enabled` (default 1, store override).

The rider sends a deposit

`submitBasataDeposit()` : amount, Basata reference (spaces removed, upper case, used once across all deposits), deposit time (not in the future, one minute of slack), the rider's own photo of the receipt (kind `basata_receipt`). Number `BD-000001` onward. The most the rider may deposit:

```
max = cashToHandIn(rider) + deadline deductions not given back yet
```

Each deposit records the Cairo day whose deadline it counts for (`deadlineDayOf()`): the first deadline at or after the moment it was sent. No money moves until Accounting decides.

Accounting approves

`approveBasataDeposit()` (accounting use, never one's own). With `held` = rider cash less pending handover differences:

```
heldPart = min(amount, held)
rest     = amount − heldPart        (a late deposit of cash a deadline already took)
for each open deadline deduction, oldest first, while rest > 0:
    give = min(rest, deduction.amount − deduction.givenBack)
    if the deduction was charged to an office: credit the office wallet give
    else: credit 1350 dues up to what the rider still owes; the remainder credits 2200 wallet
```

Account	Debit	Credit	Tags
12nn Money at Basata	amount		rider, store
1300 rider cash		heldPart	rider, store
2500 office wallet, 1350 rider dues or 2200 rider wallet		given back parts	rider, store

Idempotency key `basata_deposit:<depositId>` ; kind `basata_deposit` . The receipt photo is attached to the entry. Rejection needs a reason of at least 5 characters and moves no money, except that when the deposit's deadline has already passed (and the deadline rule is on) the cash is taken from the rider at once, as at the deadline (key `basata_rejected:<depositId>`).

The daily deadline

Every rider must have handed in or deposited the day's cash by `cash.deposit_deadline_hour` (default 23, Cairo, global) while `cash.deposit_deadline_enabled` is 1. The job `runDepositDeadline()` works from the books as of the deadline instant, so a run after the server slept sees what was true then:

```
heldAtDeadline = max(0, balance(1300) before the deadline
                      − differences of handovers made before it and still pending then
                      − deposits sent before it and not decided before it)
charged = max(0, min(heldAtDeadline, cashToHandIn(rider) now))
```

Account	Debit	Credit	Tags
2500 rider office wallet (rider linked to an office) or 1350 rider dues (no office)	charged		rider, store
1300 rider cash		charged	rider, store

Idempotency key `basata_deadline:<riderId>:<YYYY-MM-DD>`; kind `basata_deadline_charge`. A moment inside closed books is booked at the first open moment. The deduction is kept in `rider_cash_deductions` (amount and what later deposits gave back). The rider is frozen (reason `deposit_deadline` or `deposit_rejected`) and one `rider_frozen` alert is raised. The first run ever only marks its starting day and takes nothing. Each Cairo day is processed once (`cash_deadline_runs`); while the rule is off the days are marked skipped.

A frozen rider gets no orders at handover and cannot withdraw (`rider_frozen`), but can sign in and deposit. Only Accounting (accounting use) unfreezes, with a reason of at least 5 characters.

WORKED EXAMPLE

At 23:00 a rider still holds 800 EGP and is linked to an office: Dr office wallet 80,000, Cr rider cash 80,000; the office will have it deducted from its next payment. Without an office: Dr 1350 80,000. If the rider deposits the 800 EGP the next morning and Accounting approves it, heldPart is 0, rest is 80,000, and the whole amount credits back the office wallet (or the dues).

Basata's daily transfer

`recordBasataSettlement()`: finance enters the bank, the day the money arrived, the amount and the approved deposits it covers. Number `BT-000001` onward.

```
total = sum(amount of the chosen approved deposits)
fee   = total − amountReceived          (refused when negative: more_than_deposits)
refuse not_enough_money when total > balance(Money at Basata)
```

Account	Debit	Credit	Tags
11nn bank	amount received		
55nn Basata fees	fee		
12nn Money at Basata		total	

Idempotency key `basata_settlement:<clientKey>`; kind `basata_settlement`. A past day is booked at its noon (Cairo). No second person (money between our own accounts) but audited; the bank line is then matched in bank matching (chapter 15).

WORKED EXAMPLE

Three approved deposits total 6,000 EGP; the bank receives 5,985 EGP. Dr bank 598,500, Dr Basata fees 1,500, Cr Money at Basata 600,000.

7.9 Rider pay per delivery

Riders earn only `rider.bonus_per_delivery_piastres` (default 0, store override, shown as "Rider pay per delivered order") per delivered order. It is posted at delivery (Dr 5101, Cr 2200 rider wallet, key `delivery:<deliveryId>:bonus`, kind `rider_bonus`; chapter 10). Failed attempts earn nothing. There is no salary, tax or insurance for riders paid per delivery; they are never on the salary run (`employees.pay_basis = per_delivery`).

7.10 The withdrawal hold

Each earning is on hold for `rider.withdraw_hold_working_days` (default 5, global) working days after its Cairo delivery day and is available from the start of the next day. `availableFrom(day, rules)` in `packages/shared/src/rider-wallet.ts`:

```
d = deliveryDay; left = holdDays
while left > 0:
    d = d + 1 day
    if weekday(d) in weekend or d is an official holiday: continue
    left -= 1
availableOn = d + 1 day          (with holdDays 0: the day after the delivery)
```

Weekend days come from `rider.weekend_days` (default 5): each decimal digit is a JavaScript weekday of the Cairo date, 0 Sunday to 6 Saturday. 5 is Friday, 56 is Friday and Saturday; Sunday is written last (50) because a leading 0 would be lost in the integer. Official holidays are HR's holiday rows. Weekdays are taken from the calendar day itself (`new Date(day + 'T00:00:00Z').getUTCDay()`).

WORKED EXAMPLE

Delivery on Wednesday 11 November 2026, hold 5, weekend Friday. Working days counted: Thu 12, (Fri 13 skipped), Sat 14, Sun 15, Mon 16, Tue 17. Available from Wednesday 18 November. If Sunday 15 November were an official holiday, it would be available from Thursday 19 November.

7.11 The wallet figures

Nothing is stored: `riderWalletTotals()` (`packages/db/src/finance/rider-wallet.ts`) reads the rider's 2200 lines (credits are earnings, debits are amounts paid out including deductions and old `rider_bonus_payout` entries), the dues (1350) and advance (1500) balances, and open withdrawals, then `walletTotals()` computes:

```
earnedAll = sum of all earnings
matured   = sum of earnings whose availableOn <= today
onHold    = earnedAll − matured
paid      = sum of debits on the 2200 wallet
owed      = max(0, dues) + max(0, advance)
pending   = sum(amount) of open withdrawals (requested or approved)
available = max(0, matured − paid − owed − pending)
balance   = earnedAll − paid
```

What the rider owes is netted before they ask, so the amount they ask for is what they receive.

7.12 Withdrawals

Request

`requestWithdrawal()` (the rider, from the rider app; idempotency key; number `RW-000001` onward, gapless). It locks the rider and refuses: a disabled account; a frozen rider; more than `rider.max_open_withdrawals` open (default 1); more than available; less than `rider.min_withdrawal_piastres` (default 10,000) unless it is everything available. Methods: cash at the store, InstaPay (name@instapay or a mobile), mobile wallet (Vodafone Cash, Etisalat Cash, Orange Cash, WE Pay; an Egyptian mobile), bank (IBAN `EG` plus 27 digits). The owed amounts are set aside on the request, dues first, then the advance, never counting twice what other open requests already set aside:

```
setAside = sum(deduction) of other open requests
dues     = max(0, owedDues − setAside)
advance  = max(0, owedAdvance − max(0, setAside − owedDues))
```

Approve, reject, cancel

Accounting (accounting use, never the rider) approves; rejection needs a reason (requested or approved); the rider may cancel before approval. Rejection and cancel free the amount at once.

Pay

`payWithdrawal()` refuses the rider's own request, an unapproved one, and, from `finance.approval_threshold_piastres` (default 1,000,000; 0 turns it off), a payer who is also the approver (`same_person`). The source is the chosen open bank or box, else the store box for cash at the store, else CIB 1101; a store manager pays cash only from their store's box. The source row is locked and must hold the amount. Deductions are capped by what is still owed now; the wallet must still cover everything leaving it:

```
dues     = min(storedDues, owedDues now)
advance  = min(storedAdvance, owedAdvance now)
free     = matured − paid − (amount + deductions of the other open requests)
refuse more_than_available when amount + dues + advance > free
```

Account	Debit	Credit	Tags
2200 rider wallet	amount + dues + advance		rider, store
1350 rider dues		dues	rider, store
1500 staff advance (the rider's user)		advance	rider, store, user
bank or cash box		amount	rider, store

Idempotency key `rider_withdrawal:<withdrawalId>` ; kind `rider_withdrawal` .

WORKED EXAMPLE

Matured 1,000 EGP, already paid 300, dues 50 (a cash shortage), advance 100, no open request. Available = 1,000 − 300 − 150 = 550 EGP. The rider asks for 550 EGP by InstaPay. Paid from CIB: Dr 2200 70,000; Cr 1350 5,000; Cr 1500 10,000; Cr 1101 55,000. After it the dues and advance are 0 and the wallet's available is 0.

7.13 What riders owe (1350) and penalties

The 1350 rider dues wallet receives: shortages finance keeps on the rider (11.5), damage charged to the rider (chapter 10), deadline charges of riders without an office (11.8), and final violation deductions of riders paid per delivery. A staff advance given to a rider stays in 1500 and is netted the same way.

Penalty: `chargeRiderPenalty()` (job `chargeRiderPenalties()`, safe to run twice). For a final violation with a deduction:

```
amount = penalty == deduction_amount ? violation.amountPiastres
       : floor((round(days * 100) * rider.penalty_day_piastres + 50) / 100)
         (days come in quarters; worked in hundredths so no float reaches money)
```

`rider.penalty_day_piastres`: default 20,000, store override.

Account	Debit	Credit	Tags
1350 rider dues	amount		rider, store
5101 rider pay		amount	rider, store

Idempotency key `violation:<violationId>:rider_penalty`; kind `rider_penalty`. The violation is marked as taken (also when the amount is 0) so it never reaches the salary run.

> **WORKED EXAMPLE**
> Deduction of 1.25 days at 200 EGP per day: floor((125 × 20,000 + 50) / 100) = 25,000 piastres.

7.14 Rider supply offices

An office is a supplier with `is_rider_office` and its own `commission_bp` (accounting manage, audited). A rider paid per delivery links to one office through HR (employment outsourced, staffing supplier = the office). We pay the rider as usual; the office earns a commission on that pay. Code: `packages/db/src/finance/rider-offices.ts`.

Commission at each delivery

```
commission = share(riderPay, office.commissionBp)     (the rate at that moment)
```

Account	Debit	Credit	Tags
5102 rider office commission	commission		order, store, merchant, rider
2500 office supplier wallet		commission	same

Idempotency key `delivery:<deliveryId>:office_commission`; kind `rider_office_commission`; data holds supplier, rider, pay and rate. Deliveries booked before 11 November 2026 credited 2402 "commission accrued, not billed yet" instead of the wallet; the month end bill clears what is left there.

Month end: shortages moved to the office, then the bill

"Prepare office bills" (accounting use) runs `prepareOfficeBills()` for a month that is over, in month order (`later_month_prepared` otherwise). For each office, under an advisory lock, dated the month's last second (or now when the books are closed by then):

1. For each linked rider, what they still owed at the end of the month beyond everything their wallet held then moves onto the office:

   ```
   duesAtEnd   = balance(1350) before the month end
   holdsAtEnd  = 0 − balance(2200) before the month end
   uncovered   = duesAtEnd − max(0, holdsAtEnd)
   moved       = min(uncovered, dues now)     (only when > 0)
   ```

   Posting Dr office 2500 wallet, Cr 1350 rider dues; key `office_shortage:<riderId>:<YYYY-MM>`; kind `rider_office_shortage`.

2. One bill per office and month (`BILL-` numbering), from the month's commission:

   ```
   net      = sum of 5102 lines of the office's commission entries in the month
   accrued  = part of it still sitting in 2402 (older deliveries)
   vat      = office has a tax number ? share(net, fee.vat_bp) : 0
   total    = net + vat
   onWallet = net − accrued
   ```

Account	Debit	Credit	Tags
2402 commission accrued		accrued	
2301 VAT payable (input VAT)		vat	
2500 office supplier wallet			accrued + vat

Idempotency key `bill:office_bill:<supplierId>:<YYYY-MM>`; kind `supplier_bill`; data stores `onWalletPiastres` so the VAT return still counts the whole purchase (chapter 15). No bill when nothing was earned; a bill is never negative, cannot be cancelled, and is due the day it is prepared. Shortages larger than the commission leave the office wallet in debit, carried to the next month.

Paying the office with an offset

`payBill()` (chapter 12) accepts `offsetPiastres`: what the supplier owes us beyond its open bills (wallet balance plus open bills' remaining amounts, under a lock per supplier). The bill is settled in full, withholding tax stays computed on the whole amount settled, and less money leaves the bank.

```
cash = settled − withholding − advanceRecovery − offset   (refused when negative)
```

WORKED EXAMPLE

November: the office's riders made deliveries paying 125,000 EGP; commission 10%: 12,500 EGP (1,250,000 piastres) credited to the office wallet delivery by delivery. The office has a tax number: VAT 175,000. One rider owed 300 EGP at month end while his wallet held 100 EGP: 200 EGP (20,000) moves onto the office.

Account	Debit	Credit	Tags
2500 office wallet	200.00		shortage entry
1350 rider dues		200.00	rider
2301 VAT payable	1,750.00		office bill entry
2500 office wallet		1,750.00	

Bill total 14,250 EGP. The office has 1% withholding: withholding = round(1,250,000 × 100 × 1,425,000 / (10,000 × 1,425,000)) = 12,500. Paid with an offset of 20,000: cash = 1,425,000 − 12,500 − 20,000 = 1,392,500. Payment entry: Dr office wallet 1,405,000 (settled minus offset); Cr bank 1,392,500; Cr 2302 withholding 12,500. The office wallet ends at 0.

7.15 Where the code lives

Concern	File and function
Handover, deposit to bank	`packages/db/src/finance/operations.ts` : `cashToHandIn` , `recordCashHandover` , `reviewCashHandover` , `recordBankDeposit`
Cash limit, age, petty cash	`packages/db/src/finance/cash-control.ts` : `checkRiderCashLimit` , `riderCashPieces` , `checkRiderCashOverdue` , `submitPettyCash` , `postApprovedPettyCash` , `reversePettyCash` ; `packages/shared/src/cash-control.ts` : `heldPieces` , `cashAgeBucket`
Basata	`packages/db/src/finance/basata.ts` : `submitBasataDeposit` , `approveBasataDeposit` , `rejectBasataDeposit` , `runDepositDeadline` , `riderCashAtDeadline` , `recordBasataSettlement` ; `packages/shared/src/basata.ts` : `depositDeadline` , `deadlineDayOf`
Rider wallet	`packages/db/src/finance/rider-wallet.ts` : `riderWalletTotals` , `requestWithdrawal` , `payWithdrawal` , `chargeShortageToRider` , `chargeRiderPenalty` ; `packages/shared/src/rider-wallet.ts` : `availableFrom` , `weekendDays` , `walletTotals` , `withdrawalDeductions`
Rider offices	`packages/db/src/finance/rider-offices.ts` : `riderOfficeOf` , `postRiderOfficeCommission` , `prepareOfficeBills`

8 Expenses, suppliers, bills and the assets we pay for

This chapter covers money going out to suppliers and for the company's own costs: expenses paid at once, supplier bills with withholding tax, bills in another currency, cheques, purchase orders, packaging stock, recurring expenses, prepayments, equipment and depreciation, accruals, loans, cost tags, bank accounts in another currency and expense claims. Amounts are integer piastres.

8.1 Rules shared by every movement of money

Money accounts

Money leaves or enters only through banks (group 1100, CIB is 1101) and cash boxes (group 1200, store boxes included). Before money leaves, the account row is locked (`SELECT ... FOR UPDATE`) and its balance read, so two payments at the same moment are checked one after the other. Most movements use `mustHold(tx, account, amount, at)` in `packages/db/src/finance/books.ts` :

```
running balance of the account at each distinct moment (lines at one moment count together)
atBalance = running balance at the movement's own moment (latest moment <= at)
laterMin  = lowest running balance at any moment after it (atBalance when none)
refuse not_enough_money when amount > min(atBalance, laterMin)
```

A backdated payment therefore never turns an earlier or a later balance negative. A moment inside closed books answers `books_closed` first.

Dates

`eventTime(day)` : no day or today means now; a past Cairo day is booked at its noon (Cairo time); a future day is refused (`future_date`).

The second person rule

`submit()` in `packages/db/src/finance/controls.ts` either posts at once or stores a `finance_requests` row that another person decides (`decideFinanceRequest()` ; nobody decides their own request: `same_person` ; a rejection needs a note).

Request kinds	When it waits
`capital_in` , `other_income` , `merchant_receipt` , `merchant_adjustment` , `cheque_in` , `deposit_in` , `loan_in` , `debit_note` , `asset_disposal` , and `deposit_out` through the merchant's wallet	always, while `finance.approve_money_in` is 1 (default 1)
`credit_note`	always, whatever the setting

Request kinds	When it waits
everything else (expenses, drawings, bill payments, tax payments, prepayments, asset purchases, packaging purchases, advances, bonuses, payroll, loan payments, issued cheques, journal entries moving money out)	when the amount is at least `finance.approval_threshold_piastres` (default 1,000,000 = 10,000 EGP; 0 turns it off)

The request keeps the client's idempotency key; one request per key exists while it is pending or approved (a rejected one can be sent again). When approved, the same posting function runs with its own ledger key, so it still posts once.

8.2 Expenses paid at once

`recordExpense()` in `books.ts`, through `submitExpense()`. The amount entered is what was paid, VAT included; the VAT is typed from the receipt (it is never computed) and must be less than the amount. The category is an open account of groups 5400 or 5500. Optional: store (none is company wide), cost tag, receipt photo.

Account	Debit	Credit	Tags
expense category (54nn or 55nn)	amount – vat		store, cost tag
2301 VAT payable (input VAT claimed back)	vat		store
bank or cash box		amount	store

Idempotency key `expense:<clientKey>`; kind `expense`.

> **WORKED EXAMPLE**
>
> Internet bill paid from CIB, 1,140 EGP with 140 EGP VAT on the tax invoice: Dr 5406 Utilities and internet 100,000; Dr 2301 14,000; Cr 1101 114,000.

8.3 Suppliers

A supplier has a name, a tax id, a default expense account and a withholding tax rate `whtBp` (basis points of the amount before VAT, 0 to 10,000). Its wallet is `2500-nnnn` (credit: we owe it). Rider offices are suppliers with `is_rider_office` (chapter 11).

Suppliers have their own pages, apart from the bills. A supplier's fields (`supplierSchema` in `packages/shared/src/accounts.ts`): name, tax id (at most 30 characters), phone, email (checked when given), withholding rate `whtBp` (default 0) and an optional default expense account proposed on new bills; `isActive` can be switched off on an update. `createSupplier()` and `updateSupplier()` (`POST /finance/suppliers`, `PATCH /finance/suppliers/:id`, accounting use) are audited (`finance.supplier_create` on creation); a name already used by another supplier, in any letter case, answers `supplier_exists`, and a default expense account must be an open account of groups 5400 or 5500.

The list is the Accounting list view `suppliers` (`loadSuppliers` in `packages/db/src/finance/lists.ts`, `GET /finance/lists/suppliers`). One row per supplier:

```
kind     = rider_office      when is_rider_office
         = staffing_company  when people not yet left are employed through it
         = supplier          otherwise
balance  = 0 − sum of the lines of its wallet accounts     (what we owe them; below 0 they owe us)
openBills, people (staff not left), whtPercent = whtBp / 100, accountCode = its 2500 wallet code
```

Part	Values
Search	name, tax id, phone, email, wallet code
Filters (groups joined by "and", values in a group by "or")	status: `active` (the default), `inactive`; kind: `rider_office`, `staffing_company`; tax: `has_tax_number`, `no_tax_number`; balance: `owes_us` (balance < 0), `we_owe_them` (balance > 0)
Group by	kind, status
Sort	name (default, A to Z), balance, open bills
Totals	balance

The supplier document (`/finance/suppliers/<id>` , `supplierDoc` in `packages/db/src/finance/documents.ts`) shows the fields (name, tax id, phone, email, withholding %, default expense account, wallet account, commission % for a rider office, people employed through it, open bills, what we owe, created on), edited on the page itself, and smart buttons: bills (the bills list searched by the supplier's name), payments (the `bill_payment` entries of its bills, opened in the journal), purchase orders, the balance (opening the ledger lines of its wallet), cheques when it has any, and the rider office page for a rider office. The document carries the chatter like every Accounting document. The bills page lists only bills; its New bill form links to New supplier. The screens are shown in the chapter Screens: Money out, section "Suppliers".

8.4 Supplier bills

`recordBill()` in `packages/db/src/finance/payables.ts` . A bill books its cost and VAT when it arrives, and the supplier is owed until paid. Number `BILL-000001` onward (gapless). Net and VAT are entered (VAT never computed); the bill date gives the entry's time through `eventTime()` ; the due date may not be before it. A bill may name a team member (an outsourced person's month, only from their own outsourcing company: `not_their_company`); the cost line then carries the person, their store and rider tags. Supplier, reference and person together are unique (`bill_exists`).

Account	Debit	Credit	Tags
expense category	net		store, person, rider, cost tag
2301 VAT payable (input VAT)	vat		store
2500 supplier wallet		net + vat	store

Idempotency key `bill:<clientKey>` ; kind `supplier_bill` ; the bill row stores net, VAT, total, `whtBp` (copied from the supplier at that moment) and the entry.

Cancelling a bill

`cancelBill()` : only an open bill with nothing paid, with a reason of at least 5 characters. Not for bills from a purchase order or a rider office. A new entry reverses it (rows are append only):

Account	Debit	Credit	Tags
2500 supplier wallet	total		store
expense category		net	
2301 VAT payable		vat	

Idempotency key `bill_cancel:<billId>` ; kind `supplier_bill_cancel` .

8.5 Paying a bill and withholding tax

`payBill()` , through `submitBillPayment()` (second person from the limit). A bill can be paid in several payments. The amount entered is how much of the bill the payment settles, withholding tax included.

Withholding exact to the piastre

Withholding is a share of the amount before VAT. It is computed on the running total paid, so the payments together always withhold exactly `round(net × whtBp / 10,000)` once the bill is fully paid:

```
withheldAt(paid) = round(net * whtBp * paid / (10000 * total))      (Math.round)
wht(this payment) = withheldAt(paidBefore + settled) − withheldAt(paidBefore)
cash = paid − wht − advanceRecovery − offset          (refused when negative)
```

Other options: `advanceRecoveryPiastres` for a bill naming a person (an advance we gave them, taken back from what we pay their company; never more than their 1500 balance), `offsetPiastres` (what the supplier owes us beyond its open bills, chapter 11), `viaChequesPayable` (paid by a cheque we wrote: credit 2451 instead of a bank). The bank or box must hold the cash.

Account	Debit	Credit	Tags
2500 supplier wallet	settled – offset		store
5901 exchange differences	fxDiff (when positive)	–fxDiff (when negative)	
bank, cash box or 2451 cheques payable		cash	
2302 Withholding tax payable		wht	
1500 staff advance of the person		advanceRecovery	user

Idempotency key `bill_payment:<clientKey>` ; kind `bill_payment` ; data stores `gross` (settled), `wht` , `recovery` , `offset` , `fxDiff` and `net = round(bill.net × settled / bill.total)` , the amount before VAT used by form 41. The bill becomes `paid` when `paidPiastres` reaches the total.

WORKED EXAMPLE

Bill: net 10,000 EGP, VAT 1,400 EGP, total 1,140,000 piastres, withholding 3% (300 bp). Full withholding = 30,000.

- Payment 1 settles 3,333.33 EGP (333,333): withheldAt(333,333) = round(1,000,000 × 300 × 333,333 / 11,400,000,000) = round(8,771.92) = 8,772. Cash 324,561.
- Payment 2 settles the remaining 806,667: withheldAt(1,140,000) = 30,000, so wht = 30,000 − 8,772 = 21,228. Cash 785,439.

Withheld in total 30,000 exactly. Each payment: Dr supplier (settled), Cr bank (cash), Cr 2302 (wht).

8.6 Bills in another currency

A bill may be in another currency (`bills.currency` , three upper case letters). The rate is typed as text and stored exactly as `fx_rate_micro` = EGP per unit × 1,000,000 (`rateMicro()` : up to 6 decimals, no float). Foreign amounts are cents.

```
centsToPiastres(cents, micro) = floor((cents * micro + 500000) / 1000000)     (half up)
bill.net   = centsToPiastres(foreignNetCents, billMicro)
bill.vat   = centsToPiastres(foreignVatCents, billMicro)
foreignTotal = foreignNetCents + foreignVatCents
```

A payment is entered in the bill's cents at today's rate:

```
settled = (cents == foreignTotal − foreignPaid) ? total − paidPiastres          (last payment settles
exactly)
                                                : centsToPiastres(cents, billMicro)
paid    = centsToPiastres(cents, todayMicro)
          (or, paid from a bank account held in that currency, the cents' book value, 12.14)
fxDiff  = paid − settled      → 5901 exchange differences (loss positive, gain negative)
```

WORKED EXAMPLE

A USD 1,000.00 bill (100,000 cents, no VAT) at 48.75: net = floor((100,000 × 48,750,000 + 500,000) / 1,000,000) = 4,875,000 piastres. Paid in full when the rate is 49.10 from an EGP bank: paid = 4,910,000; settled = 4,875,000; fxDiff = 35,000. Entry: Dr supplier 4,875,000; Dr 5901 35,000; Cr bank 4,910,000.

8.7 Cheques

The register is `packages/db/src/finance/cheques.ts` ; numbers `CHQ-000001` onward (gapless). Purposes: received (merchant balance, security deposit, supplier refund, other income) and issued (a bill, a payout, a deposit paid back).

```
received: awaiting_approval —book→ received —deposit→ deposited —clear→ cleared
                                        └──────────────────bounce / cancel——→ bounced / cancelled
issued:   awaiting_approval —book→ issued —clear→ cleared
                                        └──bounce / cancel—→ bounced / cancelled
```

Booking

Registering a received cheque submits a `cheque_in` request (always reviewed while `finance.approve_money_in` is 1); an issued cheque submits `cheque_out` (from the approval limit); a payout cheque books at once (the payout's own approval applies). Until booked the cheque is `awaiting_approval`; a rejected request cancels it.

Booking	Debit	Credit	Key and kind
Received	1151 Cheques under collection	party: merchant wallet, merchant deposit 2600, supplier wallet, or the income account	`cheque:<id>:book`, `cheque_received`
Issued for a bill	`payBill()` with `viaChequesPayable`: Dr supplier, Cr 2451 Cheques payable (cash part), Cr 2302 (withholding)		`bill_payment:cheque:<id>`, `bill_payment`
Issued for a payout	2100 merchant wallet	2451 Cheques payable	`payout:<payoutId>`, `merchant_payout`
Issued, deposit paid back	2600 merchant deposit	2451 Cheques payable	`cheque:<id>:book`, `deposit_refunded`

For a bill the cheque's amount is the cash part; `settledFor()` finds the amount of the bill it settles: it starts from `round(cash × total / (total − net × whtBp / 10,000))` and walks by single piastres until `settled − wht(settled) = cash` (refused with `cheque_amount_mismatch` if no amount fits, for example after a partial payment meanwhile).

Clearing

A received cheque cannot clear before its due day (`cheque_not_due`); an issued one needs the bank to hold the amount.

Direction	Debit	Credit
Received	bank	1151 Cheques under collection
Issued	2451 Cheques payable	bank

Idempotency key `cheque:<id>:clear`; kind `cheque_cleared`. Depositing a received cheque at the bank moves no money (status only).

Bounce or cancel

A bounce is possible while a received cheque is received or deposited, or an issued one is issued (reason of at least 3 characters). A cancel (note of at least 3 characters) is possible while the cheque is awaiting approval (its request is dropped, nothing was posted) or booked but not yet deposited or cleared. Both post the booking entry's lines turned round, with their tags (key `cheque:<id>:reverse`, kind `cheque_bounced` or `cheque_cancelled`). For a bill, its `paidPiastres` goes down by the amount settled and it is open again; a payout paid by the cheque is cancelled. A bounce raises a `cheque_bounced` alert. Withholding lists leave out payments whose cheque bounced or was cancelled (chapter 15).

8.8 Purchase orders

`packages/db/src/purchasing.ts`. Numbers `PO-000001` onward. Flow: asked (accounting use), approved at once below `finance.approval_threshold_piastres`, otherwise by another person with accounting manage (`same_person`); ordered; received in parts; billed; closed. A line is packaging (a packaging item), expense (an expense category) or asset (a life in months).

Receiving

Each delivery is recorded once per key (`purchase_order_receipts`). Packaging lines enter packaging stock at the ordered price; the liability waits in 2401 until the bill arrives.

Account	Debit	Credit	Tags
1402 Packaging stock	sum(quantity × unit price) of packaging lines		store
1601 Equipment and vehicles	sum(quantity × unit price) of asset lines		store
2401 Goods received, not billed yet		both	store

Idempotency key `po_receipt:<clientKey>`; kind `po_receipt`. Expense lines book nothing on receipt.

Billing

`billPurchaseOrder()` bills everything received and not yet billed. The tolerance check:

```
expected = sum((receivedQty − billedQty) * unitPrice) of the open lines
refuse bill_above_order when net > expected + share(expected, finance.po_bill_tolerance_bp)
        (default 200 bp = 2%)
```

Account	Debit	Credit	Tags
2401 Goods received (packaging and asset lines)	ordered value		store
expense categories of expense lines	ordered value		store
difference account: the first expense line's account, else 5404 Packaging when the order has packaging, else 5409 Other expenses	net – expected (may be negative)		store
2301 VAT payable	vat		
2500 supplier wallet		net + vat	

Idempotency key `bill:po:<clientKey>`; kind `supplier_bill`. Paid like any bill; it cannot be cancelled by reversal (`bill_from_purchase_order`). Closing: cancelled when nothing came, refused while receipts are unbilled.

Asset lines

A purchase order can buy anything the company needs: each line is exactly one of packaging (a packaging item), a cost (an expense account of groups 5400 or 5500) or an asset (equipment such as a laptop or a scale). The kind is stored on the line (`purchase_order_lines.kind`) and worked out by `purchaseLineKind()` in `packages/db/src/purchasing.ts` from what the line names: a packaging item, an expense account or an asset life. A line naming none or more than one is refused (`packaging_or_expense`).

- An asset line carries `assetLifeMonths` (1 to 600, else `invalid_months`) and may name its own store, `assetDarkStoreId` (only on asset lines; else the order's store; an unknown store answers `store_not_found`).
- Its unit price must be above 0 (`invalid_amount`): a free asset has nothing to depreciate.

Receiving an asset line, in the same `po_receipt` entry as the packaging lines of that delivery:

Account	Debit	Credit	Tags
1601 Equipment and vehicles	sum(quantity received × unit price) of asset lines		the order's store
2401 Goods received, not billed yet		the same	the order's store

Idempotency key `po_receipt:<clientKey>`; kind `po_receipt` (one entry per delivery with the packaging part, as above). Then, in the same transaction, one fixed asset is opened per unit received, so each can later be sold or scrapped on its own:

```
for each asset line received, u = 1 .. quantity received:
    fixed_assets row:
        name          = description, or "description (u/quantity)" when several units
        cost          = unit price
        salvage       = 0
        lifeMonths    = the line's assetLifeMonths
        startMonth    = the Cairo month of the receipt entry
        darkStoreId   = the line's assetDarkStoreId, else the order's store
        entryId       = the po_receipt entry ; purchaseOrderLineId = the line
    audited finance.asset_purchase
```

Depreciation then runs exactly as for equipment bought directly (12.12): `runMonthlyCharges()` charges each month once it is over, starting with the month the asset arrived, Dr 5601 Cr 1701, key `depr:<assetId>:<YYYY-MM>`. The receipt's `purchase_order_receipts.valuePiastres` holds packaging and asset value together.

Billing an order with asset lines: packaging and asset lines were booked against 2401 when received, so the bill debits 2401 for them at the ordered price; cost lines go to their own expense accounts. The difference between the bill's net and the ordered value of what it covers never changes the assets (they keep their ordered cost); it goes to the difference account:

```
differenceAccount = the account of the order's first cost (expense) line
                  else 5404 Packaging and supplies, when the order has a packaging line
                  else 5409 Other expenses (opened from the default accounts when missing)
```

WORKED EXAMPLE

An order for 3 laptops at 25,000 EGP each, life 36 months, for store ZAM, and nothing else. Received in full on 5 November: Dr 1601 7,500,000; Cr 2401 7,500,000 piastres; three fixed assets "Laptop (1/3)", "(2/3)", "(3/3)", each cost 2,500,000, start month November, store ZAM. The supplier's bill is 75,600 EGP net plus 14% VAT 10,584 EGP; tolerance 2% allows up to 76,500 EGP. Bill entry: Dr 2401 7,500,000; Dr 5409 Other expenses 60,000 (the 600 EGP difference, since the order has no cost or packaging line); Dr 2301 1,058,400; Cr 2500 supplier wallet 8,618,400. Each laptop is depreciated from November: monthlyPart(2,500,000, 36, i) = 69,444 piastres a month, and 69,460 in the 36th month.

The purchase order document lists, under each asset line, the fixed assets it opened. The screens are shown in the chapter Screens: Money out, section "Purchase orders".

8.9 Packaging stock at average cost

Packaging (bags, boxes, labels, tape) is our own stock, never `stock_movements` (`packages/db/src/packaging.ts`): `packaging_levels` keeps units and value per store and item; `packaging_movements` is append only (receive, consume, adjust, count). Each movement is written first, then the level changes with a guarded update that never lets units or value go below zero.

```
valueOut(level, qty) = qty >= level.onHand ? level.value                (the last unit takes what is
left)
                                          : round(level.value * qty / level.onHand)
average cost shown = round(value / onHand)
on receipt: item.unitCostPiastres = round(value / qty)    (used for found units in adjustments)
```

Bought and paid at once

`recordPackagingPurchase()` through the second person rule (from the limit): Dr 1402 Packaging stock, Cr bank or box; key `packaging_purchase:<clientKey>`; kind `packaging_purchase`; units received at the amount paid.

Used when an order is packed

`consumePackagingForOrder()` uses the most specific recipe (merchant at the store, merchant, store, everyone) plus the box staff chose. Packing is never blocked: missing units raise one `packaging_short` alert; what is available is used.

Account	Debit	Credit	Tags
5404 Packaging and supplies	sum of valueOut		order, store, merchant
1402 Packaging stock		same	same

Idempotency key `packaging:<orderId>`; kind `packaging_used`. The order tag makes the cost appear in merchant profit and cost per order. `stock.packaging_low_units` (default 50, store override, 0 off) raises one `packaging_low` alert per store and item. Adjustments post kind `packaging_adjustment`: lost units leave at average cost, found units come in at the item's unit cost.

> **WORKED EXAMPLE**
>
> A store holds 200 bags valued 3,000 EGP (300,000). An order uses 3 bags: valueOut = round(300,000 × 3 / 200) = 4,500. Dr 5404 4,500; Cr 1402 4,500. The level becomes 197 bags, 295,500.

8.10 Recurring expenses

`packages/db/src/finance/recurring.ts`. A recurring expense (rent, internet, software) is entered once: category, money account, amount, optional VAT, store, memo, reference, day of month 1 to 28, start month and optional end month. Once its day of the month has come (the daily job checks `today >= dayOfMonth`, so a late run still catches it), the job `runRecurring()` prepares that month's expense as a request, always queued for a second person (never posted by itself) with idempotency key `recurring:<id>:<YYYY-MM>` and the day set to that month's day. On approval it posts as an ordinary expense (key `expense:recurring:<id>:<YYYY-MM>`, kind `expense`).

8.11 Prepayments

`recordPrepayment()` in `packages/db/src/finance/assets.ts` : months paid ahead (1 to 120), first month charged, expense category, optional store. Through the second person rule from the limit.

Account	Debit	Credit	Tags
1401 Prepaid expenses	amount		store
bank or cash box		amount	

Key `prepayment:<clientKey>` ; kind `prepayment` . Each month that is over, `runMonthlyCharges()` moves one part to the expense:

```
monthlyPart(amount, months, i) = i == months − 1 ? amount − floor(amount / months) * (months − 1)
                                                 : floor(amount / months)
```

Posting Dr expense category, Cr 1401; key `prepaid:<prepaymentId>:<YYYY-MM>` ; kind `prepayment_release` ; dated the month's last day at 23:00 Cairo (`monthEnd()`), or the first open moment when that month is closed.

> **WORKED EXAMPLE**
> Rent of 120,000 EGP paid for 7 months: floor(12,000,000 / 7) = 1,714,285 for months 1 to 6 and 12,000,000 − 6 × 1,714,285 = 1,714,290 for month 7.

8.12 Equipment, depreciation and disposal

`recordAssetPurchase()` : cost, salvage (0 to less than cost), life 1 to 600 months, start month, store. Dr 1601 Equipment and vehicles, Cr bank or box; key `asset:<clientKey>` ; kind `asset_purchase` ; through the second person rule from the limit.

Straight line depreciation

```
base   = cost − salvage
charge month i (0 based) = monthlyPart(base, lifeMonths, i)
```

Posting Dr 5601 Depreciation, Cr 1701 Accumulated depreciation (an asset account with a credit balance, shown as a negative asset); key `depr:<assetId>:<YYYY-MM>` ; kind `depreciation` ; dated as prepayments. Only months that are over are charged; a disposed asset is never charged for its disposal month or later.

Sold or scrapped

`disposeAsset()` in `packages/db/src/finance/tools.ts` , through the second person rule (always reviewed while money in needs approval). It refuses a disposal dated in or before a month already depreciated (`disposal_before_charged_month`), first charges the months before the disposal month that were not charged, then:

```
depreciated = sum of the asset's depreciation entries
loss        = cost − depreciated − salePrice      (negative is a gain)
```

Account	Debit	Credit	Tags
1701 Accumulated depreciation	depreciated		store
bank or cash box	sale price		
5902 Loss or gain on equipment sold	loss	(gain)	
1601 Equipment and vehicles		cost	

Key `asset_disposal:<clientKey>` ; kind `asset_disposal` .

WORKED EXAMPLE

A van bought for 36,000 EGP over 36 months, no salvage: 100,000 piastres a month. Sold after 12 charged months for 20,000 EGP: depreciated 1,200,000; loss = 3,600,000 – 1,200,000 – 2,000,000 = 400,000. Dr 1701 1,200,000; Dr bank 2,000,000; Dr 5902 400,000; Cr 1601 3,600,000.

8.13 Accruals

A journal entry may be an accrual (kind `accrual` , data `reverseOn` = the first day of the next month). The daily job `reverseAccruals()` posts the same lines with the signs turned and the same tags on that day at 00:00 Cairo (or the first open moment when that day is in closed books); key `accrual_reversal:<entryId>` ; kind `accrual_reversal` . An accrual may not touch a bank or cash box (`accrual_money_not_allowed`).

Entering a journal entry

Journal entries by hand are entered on the Journal entry tab of Money movements and sent to `POST /finance/journal-entries` (`journalEntrySchema`): 2 to 20 lines, each with an account, a debit or a credit in piastres and an optional cost tag; a reason of 5 to 300 characters (the memo); the day; `reverseNextMonth` for an accrual; the client's idempotency key; and the files chosen in the form (`attachmentIds`). `checkJournalEntry()` in `books.ts` checks the whole entry before it is posted or queued:

Code	When
`reason_required`	The reason is shorter than 5 characters.
`invalid_line`	A line has both a debit and a credit, or neither, or fewer than 2 lines remain.
`not_balanced`	Debits and credits differ.
`account_not_found` , `account_inactive`	An unknown or closed account.
`wallet_not_allowed`	A merchant, rider, supplier or staff advance wallet (each has its own screens).
`money_account_not_allowed`	A rider's cash (group 1300).
`accrual_money_not_allowed`	An accrual that touches a bank or cash box.

`submitJournalEntry()` then posts at once (kind `journal` , or `accrual`), or queues a `journal` request when money comes into a bank or box (while `finance.approve_money_in` is 1) or leaves one from the approval limit; money never leaves a bank or box that does not hold it (chapter 5).

The journal entry page

Every entry opens at `/finance/journal/<id>`, whatever its kind, and every `JV-` number shown in Accounting links there. The page is a document page fed by `GET /finance/documents/ledger_entry/:id`: the kind in words, the Cairo day, the memo and reference, the source document the entry was posted for (bill, payout, invoice, cheque, petty cash, withdrawal, claim, handover, request) and its order; who prepared it (the requester when it went through a second person, else the poster) and who approved it and when; the amount (the sum of the debits); and the lines with the account (a wallet shows its owner: merchant, rider with badge, store, supplier or person), store, merchant, rider, debit and credit, with totals. Smart buttons open the lines, the files (one file opens directly, several open a viewer), the source document, the approval request and the order. The chatter holds notes, reminders, the files and the audit history.

The entry can be printed as an A4 journal voucher (the browser's print hides the page and shows the voucher) or downloaded as a PDF in English or Arabic made on the API: `GET /finance/journal/:id/voucher.pdf?lang=en|ar&kind=` (`apps/api/src/finance/voucher-pdf.ts`, `journalVoucherPdf()`; accounting view; `kind` is the entry type in words as the page shows it, printed as text only, at most 80 characters). The voucher shows the company name and tax registration number, "Journal voucher" with the number, date, type, description and what it was posted for, the lines with store, merchant or rider, debit, credit and totals ("Amounts in EGP"), and three signature boxes: Prepared by, Checked by, Approved by. The screens are shown in the chapter Screens: Reporting and Planning, section "The journal entry page", and the form in Screens: Money out, section "Money movements".

8.14 Bank accounts in another currency

`packages/db/src/finance/fx.ts`. The books stay in EGP; each line on a USD or EUR bank account carries its EGP amount plus `foreign_cents` (signed like the line) and the rate used (`fx_rate_micro`). `post()` refuses a line on such an account without its foreign amount (`foreign_currency_account`), so payouts, withdrawals and petty cash can never use one. Each account has two balances: EGP (the books) and cents (the bank).

```
bookValueOut(account, cents) = cents == heldCents ? egpBalance
                             : floor((egpBalance * cents + floor(heldCents / 2)) / heldCents)
impliedRateMicro(piastres, cents) = floor((|piastres| * 1000000 + |cents| / 2) / |cents|)
```

Movement	Lines
Buy currency	Dr foreign bank (EGP paid, +cents), Cr EGP bank
Sell currency	Dr EGP bank (EGP received), Dr or Cr 5901 (book value − received), Cr foreign bank (book value, −cents)
Foreign to foreign (same currency)	Dr receiving bank (book value, +cents), Cr sending bank (book value, −cents)
Pay a bill in that currency	Dr supplier (bill rate), Dr or Cr 5901, Cr foreign bank (book value, −cents); no withholding or advance recovery allowed
Month end revaluation	Dr or Cr foreign bank (0 cents), Cr or Dr 5901

Transfers use key `transfer:<clientKey>`, kind `money_transfer`. Revaluation, once per account and month that is over, at the month's last millisecond:

```
target = floor((foreignCentsAtMonthEnd * rateMicro + 500000) / 1000000)
diff   = target − egpBalanceAtMonthEnd
```

Key `fx_reval:<accountId>:<YYYY-MM>`; kind `fx_revaluation`.

WORKED EXAMPLE

A USD account holds 50,000.00 USD (5,000,000 cents) booked at 2,430,000 EGP (243,000,000 piastres). At month end the rate is 48.90: target = floor((5,000,000 × 48,900,000 + 500,000) / 1,000,000) = 244,500,000; diff 1,500,000. Dr USD bank 1,500,000 (0 cents); Cr 5901 1,500,000 (a gain of 15,000 EGP shown as a credit on the expense account).

8.15 Loans

`packages/db/src/finance/loans.ts` and `packages/shared/src/loans.ts`. A loan is borrowed from a bank (new loans name their bank; older supplier and owner lenders stay readable). On approval it gets `LN-000001` and its own liability account (group 2700: 2701, 2702...). The money comes into a bank (group 1100 only). Money in waits for a second person (`loan_in`).

Account	Debit	Credit	Tags
bank		principal	
27nn the loan			principal

Key `loan:<clientKey>`; kind `loan_received`.

The schedule (whole piastres, worked out once)

Monthly rate r = bp / 120,000. `mulDivRound(a, b, c) = (2ab + c) / (2c)` in BigInt, which is round half up of a × b / c.

```
declining (equal instalment, annuity):
    A = (120000 + bp)^n ;  B = 120000^n
    instalment = mulDivRound(P * bp, A, 120000 * (A − B))     (bp 0: mulDivRound(P, 1, n))
    each month: interest = mulDivRound(remaining, bp, 120000)
                principal = instalment − interest (last month, or when larger than remaining:
remaining)
flat:
    totalInterest = mulDivRound(P, bp * n, 120000)
    interestEach  = floor(totalInterest / n); principalEach = floor(P / n)
    last month takes what is left of both
due days: the same day each month; a missing day becomes the month's last day
```

WORKED EXAMPLE

100,000 EGP (10,000,000 piastres) at 24% a year (2,400 bp) over 12 months, first payment 31 January 2026.

Method	Instalment 1	Interest 1	Principal 1	Instalment 12
Declining	945,596	200,000	745,596	945,597 (interest 18,541, principal 927,056)
Flat	1,033,333	200,000	833,333	1,033,337 (principal 833,337)

The second declining instalment is due 28 February 2026 (no 31st).

Paying an instalment

`payLoanInstalment()` : instalments are paid in order (`pay_in_order`), the money account must hold the instalment (`mustHold`), second person from the limit. Interest is booked when paid.

Account	Debit	Credit	Tags
27nn the loan		principal part	
5904 Loan interest		interest part	
bank or cash box			instalment

Key `loan_payment:<loanId>:<seq>` ; kind `loan_payment` .

8.16 Cost tags

Besides the store, merchant, rider and person tags every line carries, accounting manage may open cost tags (a campaign, a project). Expenses, journal entry lines, supplier bills and petty cash may put one tag on their cost line (`ledger_lines.cost_tag_id` ; a new choice must be active; a tag switched off keeps counting on its old lines). The report "Costs and income by tag" and the ledger lines filter read it (`packages/db/src/finance/cost-tags.ts`).

8.17 Expense claims

`packages/db/src/finance/expense-claims.ts` . An employee claims a work cost paid from their own pocket: a category finance marked claimable, store or company wide, the day paid (not in the future, not older than `claims.max_age_days` , default 60), a required receipt photo and an optional VAT from a proper tax invoice (less than the amount). Number `EC-000001` . The manager in force approves (none: straight to accounting); accounting use pays from a bank or box, or either rejects with a reason; the person may cancel before any approval; nobody decides their own claim; from the approval limit the payer is not the approving manager (`same_person`). The money account must hold it (`mustHold`).

Account	Debit	Credit	Tags
expense category	amount – vat		store, user
2301 VAT payable (input VAT)	vat		store, user

Account	Debit	Credit	Tags
bank or cash box		amount	store, user

Key `expense_claim:<claimId>`; kind `expense_claim`. The person is always paid the full amount; the VAT only splits the cost. The receipt photo stays with the entry.

8.18 Attachments chosen inside the forms

Every finance form that makes a record of money chooses its receipts and documents inside the form, before saving (`packages/db/src/finance/attachments.ts`, `apps/api/src/finance/files.controller.ts`, the web field `AttachmentField`). The flow:

1. Each file is uploaded on its own as soon as it is chosen: `POST /finance/files?name=`, the body is the file (accounting use or store use). `saveFinanceFile()` accepts `image/jpeg`, `image/png` and `image/webp` up to 1,500,000 bytes (the browser shrinks phone photos first) and `application/pdf` up to 5,242,880 bytes. It refuses `file_type_not_allowed`, `file_empty`, `file_too_large`, and `file_not_matching_type` when the first bytes are not those of the claimed type. The file is stored as a `photos` row of kind `receipt`, with its cleaned file name (no path, no control characters, at most 120 characters), marked `unlinked`, owned by the uploader.
2. The form keeps the returned ids and sends them with its body as `attachmentIds` (`attachmentIdsField`: at most 10). A file removed from the form before saving is deleted at once: `DELETE /finance/files/:id`, only one's own unlinked upload (`attachment_not_found` otherwise). The save waits while uploads are still running.
3. The controller runs the save through `withAttachments(db, { ids, actorUserId }, work, target)`: the record and the links are made in one transaction. `target` names where the files go: the posted entry, or the finance request when the movement waits for a second person (`movementTarget()`), or the payout for a payout marked paid. `linkAttachments()` claims the uploads with one guarded update (unlinked, own, kind receipt), so two saves naming the same file cannot both take it, writes one `finance_receipts` row per file (with `entryId`, `payoutId` or `requestId`) and audits `finance.receipt_attach`.

Code	When
`too_many_attachments` (400)	More than 10 distinct ids.
`attachment_not_found` (404)	An id is unknown or is not a finance file.
`attachment_not_yours` (403)	The upload was made by someone else.
`attachment_already_linked`	The upload is already linked to another record.
`attachment_target_missing` (400)	The save made nothing the files can be linked to.

Any refusal undoes the record as well: a movement is never saved with half its files. A save without files runs as before.

Forms that take `attachmentIds`: money transfers, expenses, owner capital and drawings, other income, journal entries, supplier bills, bill payments, tax payments, prepayments, equipment purchases, bank deposits from store boxes, payouts marked paid, merchant receipts, cheques, security deposits, Basata transfers, loans and loan instalments, and petty cash (where the store's receipt photo may itself be an upload: `claimRecordPhoto()`). A request carries its files to the entry it posts when approved: `attachmentsFor()` finds

the files of an entry directly, through the request it was posted from, and through the payout it pays, each file once, oldest first. Files can still be added later from the document page (the earlier "add receipt later" route).

Uploads that no form saved are deleted by an hourly API job (`runUploadPurge` calling `purgeUnlinkedUploads()`): unlinked files older than 24 hours (`UNLINKED_UPLOAD_HOURS`) that nothing points to (no finance receipt, petty cash, cheque, expense claim, Basata deposit, order or delivery). It is safe to run twice. The field and the viewer are shown in the chapter Screens: Month end, Setup and common parts, section "Common part: the attachment field and viewer".

8.19 Where the code lives

Concern	File and function
Money checks, expenses, transfers	`packages/db/src/finance/books.ts`: `eventTime`, `mustHold`, `recordExpense`, `transferMoney`
Second person rule	`packages/db/src/finance/controls.ts`: `submit`, `queueRequest`, `decideFinanceRequest`, `approvalThreshold`
Suppliers, bills, payments, taxes	`packages/db/src/finance/payables.ts`: `recordBill`, `cancelBill`, `withholdingFor`, `payBill`, `payTax`, `rateMicro`, `centsToPiastres`
Cheques	`packages/db/src/finance/cheques.ts`: `registerCheque`, `bookCheque`, `settledFor`, `clearCheque`, `bounceCheque`, `cancelCheque`
Purchase orders, packaging	`packages/db/src/purchasing.ts`, `packages/db/src/packaging.ts`
Prepayments, assets	`packages/db/src/finance/assets.ts`: `monthlyPart`, `runMonthlyCharges`, `chargeDepreciation`; `tools.ts`: `disposeAsset`, `reverseAccruals`
Foreign currency	`packages/db/src/finance/fx.ts`: `bookValueOut`, `revalueForeignBank`
Loans	`packages/shared/src/loans.ts`: `loanSchedule`; `packages/db/src/finance/loans.ts`: `recordLoan`, `payLoanInstalment`

9 Merchant money: wallets, payouts, receivables, invoices and profit

This chapter covers what happens to a merchant's balance after the fees of chapter 10 are booked: how the balance is read, how payouts are prepared and paid, how a merchant who owes us is followed up, how the monthly tax invoice and its credit and debit notes are made, how documents reach the Egyptian Tax Authority (ETA), how security deposits are held, and how the merchant profit report shares costs.

9.1 The merchant wallet and its sign

Each merchant has one wallet account (type `merchant`, code `2100-nnnn`, category liability). In the ledger debit is positive, so:

```
raw balance  = sum(amount_piastres) of the wallet's lines
owed to the merchant = 0 − raw balance          (merchantOwed() in operations.ts)
owed > 0 : we owe the merchant (credit balance)
owed < 0 : the merchant owes us (debit balance, a receivable)
```

What moves the wallet:

Event	Effect on what we owe	Entry kind
COD collected	up by the cash	`cod_collected`
Every fee plus its VAT, minimum top up	down	fee kinds of chapter 10
Damage compensation	up	`damage_compensation`
Payout paid	down	`merchant_payout`
Payment received from the merchant	up	`merchant_receipt`
Adjustment (credit or charge)	up or down	`adjustment`
Credit note	up (fees and VAT given back)	`credit_note`
Debit note	down	`debit_note`
Security deposit taken from or paid back into the wallet	down or up	`deposit_received`, `deposit_refunded`
Cheque received against the balance	up	`cheque_received`

The customer's delivery fee is part of the cash collected, so it stays with the merchant. Balances are never stored: every screen sums the lines.

9.2 Merchant statements

`merchantStatement(db, merchantId, from, to)` in `packages/db/src/finance/reports.ts` reads the wallet from the merchant's side:

```
openingOwed = 0 − sum(wallet lines before from)
for each line in [from, to), ordered by time then id:
    amount (merchant's side) = 0 − line.amount
    running = running − line.amount
closingOwed = running
byKind[kind] = sum of amounts per entry kind
```

Each line keeps the entry kind, memo and order number. The merchant reads it at `/merchant/wallet`. When a payout is marked paid, the statement of the payout period and its CSV are queued as an email (`queuePayoutEmails()`, sent by the email outbox job through the Mailer boundary).

9.3 Payouts

Schedules

`merchants.payout_schedule`: `daily`, `twice_weekly` (Sunday and Wednesday) or `weekly` (Sunday). The merchant chooses it (audited); finance can change it; an activated contract version may set it once when it takes effect. `payoutDueOn(schedule, weekday)` in `operations.ts` (weekday 0 is Sunday, of the Cairo date); `nextPayoutDay()` in `packages/shared/src/cash-control.ts` gives the next due day.

Preparing a draft

`createPayout(db, { merchantId, cutoffAt })` locks the merchant row and:

```
refuse payout_already_open when the merchant has a draft (one open draft per merchant)
owedAtCutoff = merchantOwed(merchant, before cutoffAt)
amount       = min(owedAtCutoff, merchantOwed(merchant) now)      (fees after the cutoff may have
lowered it)
refuse nothing_to_pay when amount <= 0
minimum      = version in force on the cutoff day: minimumPayoutPiastres,
               else finance.payout_minimum_piastres (default 0)
refuse below_payout_minimum when amount < minimum
```

The cutoff is the end of a Cairo day (`cairoDayStart(day + 1)`). The draft gets `PAY-000001` onward (gapless). No money moves for a draft.

Approval

From `finance.approval_threshold_piastres` (default 1,000,000; 0 off), a second person approves (`approvePayout()` in `controls.ts`): never the one who prepared it (`same_person`), only once.

Statement of account (SOA)

Every payout carries a statement of account frozen when it is prepared. `createPayout()` calls `freezePayoutStatement()` (`packages/db/src/finance/payout-proof.ts`) in the same transaction as the draft, so a draft never exists without its statement. The statement is built by `buildPayoutStatement()` from

`merchantStatement()` (13.2), the statement the merchant reads in the wallet, so its numbers are the wallet's numbers:

```
from     = cutoffAt of the merchant's last paid payout whose cutoff is not after this one,
           else the merchant's creation time (never after this cutoff)
to       = this payout's cutoffAt
s        = merchantStatement(merchant, from, to)
for each wallet line of s:
    vat  = the vat_payable lines of the same entry, shown once (on the entry's first line)
orders   = lines that name an order, except merchant_payout and merchant_receipt:
           cod = sum of cod_collected ; fees[kind] = sum of every other kind ; vat ; net
other    = every remaining line (storage, receiving, receipts, adjustments, notes, payouts)
opening  = s.openingOwed
closing  = s.closingOwed                    (what was owed at the cutoff)
net      = payout.amountPiastres            (what is transferred)
heldBack = closing − net                    (charges booked after the cutoff that lowered the amount)
refuse soa_mismatch when net differs from the payout's amount
```

The whole statement (`PayoutStatementData` , version 1, in `packages/shared/src/payout-proof.ts`) is stored as JSON in `payout_statements.data` , with the period, opening, closing, held back and net also in their own columns. Numbers `SOA-000001` onward are gapless, taken under an advisory lock inside the transaction. A database trigger refuses every `UPDATE` or `DELETE` of a statement (`payout_statement_frozen`) and a `CHECK` keeps its net above 0. The freeze is audited as `finance.payout_statement` . A draft prepared before statements existed gets its statement when its proof is uploaded; a payout that has one keeps it.

The statement is served three ways, all from the frozen JSON:

- JSON: `GET /finance/payouts/:id/statement` .
- PDF in English or Arabic (`GET /finance/payouts/:id/statement.pdf?lang=` , `apps/api/src/finance/payout-statement-pdf.ts` , pdfkit through `PdfWriter`): the summary by kind with the VAT inside the fees, every order of the period, the other movements and, once there is a proof, the proof of transfer with a line saying whether the three amounts match.
- CSV (`statement.csv`): a heading row (statement, payout, merchant), then one row per movement with the Cairo date and time, type, order, reference, amount, VAT and running balance, framed by the opening and closing balances, then the amount held back after the cutoff and the net to transfer. House CSV rules apply (byte order mark, formula prefixes neutralised, money as EGP text from integers).

Proof of transfer (POT)

`addPayoutProof()` stores the bank slip of the transfer. The upload is `POST /finance/payouts/:id/proof` : the body is the file itself, the query carries its `name` , the `amountPiastres` and the bank `reference` typed from the slip (`payoutProofQuerySchema`). It refuses:

Code	When
`file_type_not_allowed`	The type is not PDF, JPEG or PNG, or the first bytes of the file do not match the type it claims.
`file_too_large`	The file is empty or above 5 MB (`PAYOUT_PROOF_MAX_BYTES`).
`pot_required`	The reference is shorter than 2 characters.
`payout_not_draft`	The payout is paid or cancelled.

Code	When
`pot_amount_mismatch` (409)	The amount typed from the slip is not the payout's amount (both amounts are returned).

The system compares the typed amount with the payout; it never reads the image. That is why the second person check below asks a person to open the slip. A payout has at most one proof (`payout_proofs` , numbers `POT-000001` onward, gapless). While the payout is a draft, uploading again replaces the file, amount and reference, keeps the POT number, and deletes every check already ticked on the payout, because those ticks confirmed another file. The upload is audited as `finance.payout_proof` , a replacement as `finance.payout_proof_replace` with the old and new file, amount and reference. Once the payout is paid a trigger (`payout_paid_frozen`) refuses any change or deletion of its proof and of its check marks.

Payout checks

`payout_checks` lists what Accounting confirms before a payout is paid; `payout_check_marks` records each tick (payout, check, person, moment, optional note of at most 300 characters).

- The system check `second_person_viewed` ("Second person viewed the statement and the proof of transfer") is inserted by the migration and repaired by `ensurePayoutChecks()` if the table was emptied. It cannot be switched off (`check_is_system`). It can be ticked only once the payout has its proof (`pot_required`) and its statement (`soa_mismatch`), since it confirms both were seen.
- Accounting manage adds checks (`createPayoutCheck()` : a code of lower case letters, digits and underscores starting with a letter, an English and an Arabic label, an order; a code used twice answers `check_code_taken`) and changes labels, order or the active switch (`updatePayoutCheck()`). Both are audited (`finance.payout_check_create` , `finance.payout_check_update`).
- Only active checks count. A new active check is needed at once on every payout still a draft.
- `markPayoutCheck()` ticks one active check on a draft (accounting use; `check_not_found` for an unknown or inactive check), audited as `finance.payout_check` . Ticking the same check again returns the first tick and changes nothing.

Ready to pay: the flow and the refusals

```
prepare (createPayout) ──▶ draft PAY-nnnnnn with its SOA frozen
   │
   ├── amount ≥ finance.approval_threshold_piastres ──▶ approvePayout() by another person
   ├── upload the POT (amount = payout amount, bank reference)
   ├── tick every active check; second_person_viewed by someone who will not pay
   ▼
markPayoutPaid() ──▶ assertReadyToPay() ──▶ posting (below) ──▶ payout emails with SOA and POT
```

Approval and the proof are independent: the proof may be uploaded before or after the approval, but marking paid needs both. `assertReadyToPay(tx, payout, payerId)` runs inside `markPayoutPaid()` after the approval check and before the owed and bank checks, and refuses in this order:

Code	Status	When
`pot_required`	409	No proof of transfer.
`pot_amount_mismatch`	409	The proof's amount is not the payout's amount.
`soa_mismatch`	409	No frozen statement, or its net is not the payout's amount.

Code	Status	When
`checks_incomplete`	409	An active check is not ticked; the answer lists the missing codes, comma separated.
`same_person`	403	The person marking it paid is the one who ticked `second_person_viewed`.

It returns the proof's reference, which becomes the payout's reference when the person marking it paid types none. The posting itself did not change. `payoutControls()` (`GET /finance/payouts/:id/controls`) gives the payout page everything at once: the statement summary, the proof, each check with who ticked it and when, `matches` (statement net, proof amount and payout amount are the same number of piastres: `payoutMatches()`) and `blockers`, the refusal codes the person looking would get if they marked it paid now.

A payout paid by an issued cheque does not go through `markPayoutPaid()`: the cheque register is its proof, and no proof upload or checks are asked.

What the merchant sees

Merchant users with the money right read their own payouts' documents under `/merchant/finance/payouts/:id/`: `statement`, `statement.pdf` (Arabic unless `lang=en`), `statement.csv` and `proof`; another merchant's payout answers 404, and the proof only once the payout is paid (`proof_not_found` before). `/merchant/wallet` lists each payout with its SOA and POT side by side and a Matches mark. When a payout is marked paid, `queuePayoutEmails()` attaches to the payout paid email the SOA as a PDF in the merchant's email language and the POT file, both made or read only when the email is sent (references `payout_soa_pdf:<payoutId>:<en|ar>` and `payout_proof:<payoutId>`).

The screens are shown in the chapter Screens: Dashboard and Money in (sections "A payout page: statement, proof, checks and mark paid" and "The merchant's side: Wallet with SOA and POT"); the checks are edited on the Payout checks screen of Screens: Month end, Setup and common parts.

Marking a payout paid

`markPayoutPaid()`: the draft must be approved when it needed approval (`approval_needed`); it must pass `assertReadyToPay()` (proof, statement and checks, above); it is never more than what is owed now (`assertPayoutOwed()`, error `payout_more_than_owed`); the bank (default CIB 1101) is locked and must hold the amount (`not_enough_money`); a past Cairo day is booked at its noon; the books lock applies.

Account	Debit	Credit	Tags
2100 merchant wallet	amount		merchant
11nn bank		amount	merchant

Idempotency key `payout:<payoutId>`; kind `merchant_payout`; memo = bank reference (the one typed when marking paid, else the proof's). Files chosen in the mark paid form are linked to the payout in the same transaction (see Attachments in chapter 12). A payout can also be paid by a cheque (chapter 12: Dr wallet, Cr 2451, same key). A draft may be cancelled with a note.

The Pay merchants run

`prepareDuePayouts()` in `packages/db/src/finance/payout-run.ts` prepares a draft for every active merchant whose schedule is due on today's Cairo weekday, of what is owed up to the end of a chosen day (default yesterday); merchants with nothing owed, below their minimum or with an open draft are listed as skipped with the reason. Nothing is stored for the run itself; its state is read from the payouts. The run's "mark

paid" step (`markPayoutsPaid()`) calls `markPayoutPaid()` once per payout in its own transaction, with one bank and one day, so one refusal never stops the others. Each payout needs its own proof of transfer and its checks (a row without a proof answers `pot_required`); its reference is the one typed on its proof unless one reference is given for all. A payout already paid (with the given reference, when one is given) counts as done, so pressing twice changes nothing. The run shows for each draft its SOA number, its proof, how many active checks are ticked and whether the person looking ticked the second person check (they may not pay it then).

The bulk transfer file

`payoutBankFile()` in `packages/db/src/finance/tools.ts` lists draft payouts that are ready (approved when they needed approval, and with bank details: an IBAN, or a bank name and account number). The CSV (`GET /finance/payouts/bank-file.csv` , `apps/api/src/finance/tools.controller.ts`) has one row per payout:

Column	Value
Beneficiary name	the merchant's beneficiary name, else legal name, else name
Bank	bank name
Account number	bank account number
IBAN	`EG` plus 27 digits
Amount	EGP with two decimals, integers only (`egp()`)
Currency	`EGP`
Reference	the payout number `PAY-nnnnnn`
Merchant	merchant name

The columns are the usual bulk transfer columns, to be matched to CIB's own template. CSV files follow the house rules: `UTF-8` byte order mark, and a cell starting with `=` , `+` , `-` or `@` is prefixed with an apostrophe unless it is a plain number.

9.4 Receipts and adjustments

A merchant who owes us (fees above their COD) pays by bank transfer: `recordMerchantReceipt()` , always reviewed by a second person while `finance.approve_money_in` is 1.

Account	Debit	Credit	Tags
1100 bank	amount		merchant
2100 merchant wallet		amount	merchant

Key `receipt:<clientKey>` ; kind `merchant_receipt` .

An adjustment (`recordMerchantAdjustment()`) needs a reason of at least 5 characters and is always reviewed while money in needs approval. A positive amount credits the merchant against fee income; a negative one charges them.

Account	Debit	Credit	Tags
4101 fee income	amount (credit to the merchant)	|amount| (charge)	merchant

Account	Debit	Credit	Tags
2100 merchant wallet	|amount| (charge)	amount (credit)	merchant

Key `adjustment:<clientKey>`; kind `adjustment`. No VAT: a correction with VAT is made with a credit or debit note (13.7).

9.5 Who owes us, by age (first in, first out)

`receivablesAging()` in `packages/db/src/finance/aging.ts` lists, as of the end of a Cairo day, merchants whose wallet is in debit and received cheques still under collection. A wallet has no due dates, so each debit counts from the day it was booked, and every credit pays the oldest debit still open (`openDebits()` in `packages/shared/src/aging.ts`):

```
open = []; credit = 0
for each wallet line in time order (debit positive):
    if amount > 0: used = min(credit, amount); credit −= used
                   if amount > used: open.push(amount − used on its day)
    if amount < 0: pay = −amount
                   while pay > 0 and open not empty: take from open[0] (oldest)
                   credit += pay left over
age of an open debit = Cairo days from its day to the as of day
band: age <= 0 current (not yet due), 1 to 30, 31 to 60, 61 to 90, over 90
```

The open debits always add up to the debit balance. The debit period starts at the first line after the balance was last zero or in credit. Cheques under collection are aged by their due day. Rider cash, rider dues and staff advances are not listed: they are taken back from handovers, withdrawals or salaries. Exports: CSV, Excel and PDF. `/finance/payables-aging` ages open supplier bills by due day the same way.

WORKED EXAMPLE

As of 30 November. Wallet lines: 1 Oct storage and fees +3,000 EGP; 5 Oct COD −1,000; 20 Oct fees +2,500; 25 Nov fees +1,200. FIFO: the 1,000 credit pays part of the 1 October debit, leaving 2,000 (1 Oct, 60 days: band 31 to 60), 2,500 (20 Oct, 41 days: 31 to 60), 1,200 (25 Nov, 5 days: 1 to 30). Balance 5,700 EGP owed to us.

9.6 Payment reminders

`packages/db/src/finance/reminders.ts`. A reminder is a notification in the merchant panel, a `payment_reminder` email to the billing emails (unless opted out) and a web push. The daily job runs once `receivables.reminder_hour` (default 10, Cairo) has passed, while `receivables.reminders_enabled` is 1:

```
reminderDue(sinceDay, today, sent, lastDay):
    sent >= receivables.reminder_max (default 4)        → no
    sent == 0 → days(sinceDay, today) >= receivables.reminder_first_days (default 3)
    else      → days(lastDay, today)  >= max(receivables.reminder_every_days (default 7), 1)
```

Counted per debit period (`merchant_payment_reminders.period_start`); a wallet back to zero or credit ends the period. Each automatic reminder's key holds the merchant, the period and its number, so a second run

sends nothing twice. "Send reminder now" is sent even past the maximum, counts toward it and restarts the wait.

9.7 Tax invoices, credit notes and debit notes

`packages/db/src/finance/invoices.ts` and `debit-notes.ts`. The fees were booked when they happened; an invoice is only the document for them. Documents live in `tax_invoices` with types `invoice` (`INV-000001`), `credit_note` (`CN-000001`) and `debit_note` (`DN-000001`), each numbered without gaps under an advisory lock. Amounts and lines are fixed by a database trigger once issued.

Monthly invoice

`prepareInvoices(db, { month })` runs only for a month that is over, once per merchant and month. `monthlyFees()` reads the ledger:

```
entries of the fee kinds (shipping_fee, product_handling, order_fee, cod_fee, failed_fee,
  receiving_fee, storage_fee, return_fee, adjustment, minimum_fee, packaging_charge)
  that have a 4101 line tagged with the merchant inside the Cairo month
per kind: quantity = number of entries
          net = −sum(4101 lines) ; vat = −sum(2301 lines)
kinds with net <= 0 are left out
invoice net = sum(net); vat = sum(vat); total = net + vat
```

The invoice VAT is the VAT actually booked entry by entry, not 14% of the total. An invoice notice email is queued with a link to `/merchant/invoices`.

Credit note

`issueCreditNote()` gives back part of an invoice's fees, always through a second person. The amount (fees before VAT) may not exceed what is left uncredited. VAT comes from the running total, like withholding:

```
creditNoteVat(invoice, before, amount):
    n = before.net + amount
    total = n >= invoice.net ? invoice.vat : round(n * invoice.vat / invoice.net)
    return max(total − before.vat, 0)
```

Account	Debit	Credit	Tags
4101 fee income	amount		merchant
2301 VAT payable	vat		merchant
2100 merchant wallet		amount + vat	merchant

Key `credit_note:<clientKey>`; kind `credit_note`; one document line of kind `adjustment` with the reason.

> **WORKED EXAMPLE**
>
> Invoice net 12,345.67 EGP (1,234,567), VAT 1,728.39 (172,839). Credit note 1: 100,001 → VAT round(100,001 × 172,839 / 1,234,567) = 14,000. Credit note 2 for the rest, 1,134,566 → total reaches the invoice net, VAT = 172,839 − 14,000 = 158,839. Together exactly the invoice VAT.

Debit note

`issueDebitNote()` charges a merchant more after an invoice (a missed fee, a correction). It always names an invoice of the merchant; one line per fee kind (the invoice kinds plus an adjustment) with its amount before VAT; VAT per line at `fee.vat_bp` read when the note is asked for (frozen on the request so the approver sees the exact amounts):

```
line.vat = share(line.net, vatBp); net = sum; vat = sum; total = net + vat
```

Account	Debit	Credit	Tags
2100 merchant wallet	total		merchant, store (when given)
4101 fee income		net	
2301 VAT payable		vat	

Key `debit_note:<clientKey>`; kind `debit_note`. It waits for a second person like every merchant balance change. Its kind is not a monthly fee kind, so the next invoice never counts it again; it is output VAT in the VAT report and return.

9.8 ETA electronic invoices

```
tax_invoices row ──buildDocument()──▶ ETA document v1.0 (JSON) ──stored──▶ read back
       ──etaCanonical()──▶ canonical text ──/eta-signer──▶ signing program on the finance PC
       (USB token, CAdES BES) ──signature──▶ saveSignatures() ──EtaSync every minute──▶ ETA
       ──▶ submitted (uuid) ──▶ valid | invalid | rejected ──fix──▶ resendInvoice() ──▶ not_sent
```

- `etaDocument()` (`packages/shared/src/eta.ts`) builds the document: type I (invoice), C (credit note) or D (debit note); issuer from `company_profile` (type B, branch id); receiver type B when the merchant has a tax registration number (then a full address is required, else `merchant_address_missing`), else P; one line per document line with item type EGS, the item code per fee kind from `company_profile`, unit EA, tax T1 subtype V009 at the rate the fee was actually charged (`round(vat × 10,000 / net) / 100` percent); amounts are EGP numbers with two decimals (`piastres / 100`); `dateTimeIssued` in UTC without fractions. A note references the original invoice's uuid and waits until the invoice was sent (`original_not_sent`).
- `etaCanonical()`: every property name upper case and quoted, followed by its value; an array gives its name once and again before each element; plain values quoted as written; the `signatures` property left out. Because the database may reorder JSON keys and ETA rebuilds the canonical text from what it receives, the document is first stored, read back, and the canonical text is made from the stored copy (`documentsToSign()`). The text is fixed the first time it is handed out.
- The signing program (`scripts/eta-signer`, PowerShell) asks `/eta-signer` (public, checked with the signer key from `company_profile`, shown once) for texts and returns signatures.
- `EtaSync` (`apps/api/src/finance/eta.ts`) every minute submits signed documents and asks the status of submitted ones through the `EtaClient` boundary (environment names `ETA_ENV`, `ETA_CLIENT_ID`, `ETA_CLIENT_SECRET`; nothing is sent without them).
- A rejected or invalid document is corrected (tax details, an item code) and sent again with `resendInvoice()`: its text, signature and document are cleared and it waits for a new signature.

9.9 Security deposits

A contract may ask for a security deposit. It is held in the merchant's deposit wallet (type `merchant_deposit`, code `2600-nnnn`, a liability). `recordDeposit()` in `contracts.ts` through `submitDeposit()`:

Direction and channel	Lines	Checks and approval
In, by bank	Dr bank, Cr 2600	always reviewed (money in)
In, from the wallet	Dr 2100 wallet, Cr 2600	never more than we owe the merchant (`more_than_owed`); always reviewed
Out, by bank	Dr 2600, Cr bank	never more than held (`more_than_deposit`); the bank must hold it; from the approval limit
Out, into the wallet	Dr 2600, Cr 2100 wallet	never more than held; always reviewed

Key `deposit:<clientKey>`; kinds `deposit_received` and `deposit_refunded`. Deposits can also come or go by cheque (chapter 12).

9.10 Merchant profit report

`merchantProfitability(db, { month })` in `packages/db/src/finance/merchant-profit.ts`, per merchant and Cairo month (the `year_close` entry is always left out):

Part	How it is computed
Fees by kind	–sum of revenue lines tagged with the merchant, per entry kind (before VAT)
Rider pay	expense lines of type `rider_bonus_expense` tagged with the merchant
Rider office commission	lines on 5102 tagged with the merchant
Postal fees	lines on 5103 tagged with the merchant (the postal authority's share of the fees on the merchant's shipments, chapter 10)
Damage	lines of type `damage_expense` tagged with the merchant
Packaging	lines on 5404 tagged with the merchant
Other direct	any other expense line tagged with the merchant
Rider salaries share	each store's expense lines tagged with a rider user and no merchant, shared by the merchant's part of the store's delivered orders
Store costs share	each store's other untagged expense lines (rent apart, no rider user), shared the same way
Storage space share	each store's rent (5401) shared by unit days actually stored (`stock_unit_days`, chapter 10); when a store has rent but no day of the month is over yet, the units held now

```
shareOf(pool, part, whole) = whole > 0 ? round(pool * part / whole) : 0
margin          = fees − costs
marginPerOrder  = round(margin / deliveredOrders)
marginBp        = round(margin * 10000 / fees)
```

The detail view adds a what if (`whatIf()` in `packages/shared/src/contracts.ts`): shipping, order and COD fees recomputed on the month's delivered orders, products value and cash with other values; other fees kept; a minimum tops up; costs do not change.

WORKED EXAMPLE

A store's rent in November is 30,000 EGP and its merchants stored 90,000 unit days, of which merchant A 27,000: A's storage space share = round(3,000,000 × 27,000 / 90,000) = 900,000 (9,000 EGP), whether or not A pays storage fees.

9.11 Where the code lives

Concern	File and function
Wallet, payouts, receipts, adjustments	`packages/db/src/finance/operations.ts` : `merchantOwed` , `payoutDueOn` , `createPayout` , `assertPayoutOwed` , `markPayoutPaid` , `cancelPayout` , `recordMerchantReceipt` , `recordMerchantAdjustment`
Statement of account, proof of transfer, payout checks	`packages/db/src/finance/payout-proof.ts` : `buildPayoutStatement` , `freezePayoutStatement` , `getPayoutStatement` , `addPayoutProof` , `getPayoutProofFile` , `listPayoutChecks` , `createPayoutCheck` , `updatePayoutCheck` , `markPayoutCheck` , `assertReadyToPay` , `payoutControls` , `payoutStatementCsv` ; `packages/shared/src/payout-proof.ts` : `soaNumber` , `proofNumber` , `payoutMatches` , `STATEMENT_KIND_ORDER` ; `apps/api/src/finance/payout-proof.controller.ts` , `payout-statement-pdf.ts`
Payout run and bank file	`packages/db/src/finance/payout-run.ts` ; `packages/db/src/finance/tools.ts` : `payoutBankFile` , `updateMerchantBank`
Statements	`packages/db/src/finance/reports.ts` : `merchantStatement` , `financeWallets`
Aging and reminders	`packages/shared/src/aging.ts` : `openDebits` , `agingBand` , `reminderDue` ; `packages/db/src/finance/aging.ts` , `reminders.ts`
Invoices and notes	`packages/db/src/finance/invoices.ts` : `monthlyFees` , `prepareInvoices` , `creditNoteVat` , `issueCreditNote` , `buildDocument` , `documentsToSign` ; `debit-notes.ts` : `debitNoteAmounts` , `issueDebitNote`
ETA	`packages/shared/src/eta.ts` : `etaDocument` , `etaCanonical` ; `apps/api/src/finance/eta.ts` : `EtaSync` , `EtaClient`
Deposits	`packages/db/src/finance/contracts.ts` : `recordDeposit` , `submitDeposit` , `depositHeld`
Merchant profit	`packages/db/src/finance/merchant-profit.ts` : `merchantProfitability` , `merchantProfitDetail`

10 Payroll and staff costs

This chapter explains how a month's salaries are worked out and booked: salary packages kept by HR, Egyptian social insurance, the martyrs fund and salary income tax computed from settings, advances, bonuses, sales commission, unpaid leave and violation deductions, outsourced staff billed by their company, and payslips. All arithmetic is in integer piastres. The deduction helper is the same half up share used everywhere:

```
share(amount, bp) = floor((amount * bp + 5000) / 10000)
```

10.1 Who is on the payroll

Person	How they are paid	On the salary run
Employee paid a monthly salary (`employees.employment = employee`, `pay_basis = monthly_salary`)	the monthly salary run	yes
Rider paid per delivery (`pay_basis = per_delivery`; only riders may be)	rider wallet, per delivered order (chapter 11); no tax, no insurance	no (`per_delivery_not_on_payroll`)
Outsourced person (`employment = outsourced` with a staffing supplier)	their outsourcing company, by a supplier bill tagged with them; we may give them advances and bonuses directly	no (`outsourced_not_on_payroll`)

`listStaffPay()` in `packages/db/src/finance/payroll.ts` lists active admin, finance, store staff and rider accounts with no employee file or a monthly salary file, with the package in force today and the advance they owe.

10.2 Salary packages

HR keeps salary packages (`salary_packages` with `salary_allowances`): from a day, a basic salary, an insured wage, and allowances (kind, label, amount). History is kept. Accounting never edits them.

```
packageInForce(employee, day) = the package with the latest effectiveFrom <= day,
                                else (before the first package) the first package
packageGross(package)         = basic + sum(allowances)
```

The run reads the package in force on the month's last day (`packages/db/src/hr-pay.ts`).

10.3 The rules, all from settings

Every rate, band and limit is a global setting the accountant edits (`packages/shared/src/settings.ts`); `payrollRules()` reads them. The defaults follow the law as known in 2026 and must be confirmed.

Setting	Default	Meaning
`payroll.si_employee_bp`	1100 (11%)	employee share of the insured wage
`payroll.si_employer_bp`	1875 (18.75%)	company share of the insured wage
`payroll.si_min_wage_piastres`	270,000 (2,700 EGP)	lowest monthly insured wage
`payroll.si_max_wage_piastres`	1,670,000 (16,700 EGP)	highest monthly insured wage
`payroll.martyrs_fund_bp`	5 (0.05%)	martyrs fund on gross
`payroll.personal_exemption_piastres`	2,000,000 (20,000 EGP a year)	yearly personal exemption
`hr.payroll_days_per_month`	30	a day of pay = basic / this

Yearly salary tax bands

Band	Yearly taxable income up to (setting `payroll.tax_band_n_to_piastres`)	Rate (setting `payroll.tax_band_n_bp`)
1	40,000 EGP (4,000,000)	0%
2	55,000 EGP (5,500,000)	10% (1000)
3	70,000 EGP (7,000,000)	15% (1500)
4	200,000 EGP (20,000,000)	20% (2000)
5	400,000 EGP (40,000,000)	22.5% (2250)
6	1,200,000 EGP (120,000,000)	25% (2500)
above		27.5% (`payroll.tax_top_bp` 2750)

High earners lose the low bands

Setting	Default	Above it
`payroll.tax_skip_1_from_piastres`	600,000 EGP (60,000,000)	band 1 taxed at band 2's rate
`payroll.tax_skip_2_from_piastres`	700,000 EGP (70,000,000)	bands 1 to 2 at band 3's rate
`payroll.tax_skip_3_from_piastres`	800,000 EGP (80,000,000)	bands 1 to 3 at band 4's rate
`payroll.tax_skip_4_from_piastres`	900,000 EGP (90,000,000)	bands 1 to 4 at band 5's rate
`payroll.tax_skip_5_from_piastres`	1,200,000 EGP (120,000,000)	bands 1 to 5 at band 6's rate

A threshold of 0 turns that step off. The highest threshold the income passes decides how many low bands are skipped.

10.4 The payslip calculation

Pure functions in `packages/shared/src/payroll.ts` ; the run is `previewPayroll()` then `runPayroll()` in `packages/db/src/finance/payroll.ts` .

```
base       = line.grossPiastres given by finance, else packageGross(package in force on the month's
last day)
bonus      = taxed bonus entered for the month (overtime, target bonus)
commission = sum of approved, unpaid sales commissions of months <= this month
deduction  = penalties or unpaid days entered by finance
hrItems    = unpaid leave days and final violation deductions HR hands over (14.8)
gross      = base + bonus + commission − deduction − hrItems      (refused when negative)

insured    = min(max(package.insuredWage ?? gross, siMinWage), siMaxWage)
siEmployee = gross > 0 ? share(insured, siEmployeeBp) : 0
siEmployer = gross > 0 ? share(insured, siEmployerBp) : 0
martyrs    = share(gross, martyrsBp)

yearlyTaxable = (gross − siEmployee) * 12 − personalExemption
incomeTax     = floor((yearlyIncomeTax(yearlyTaxable) + 6) / 12)     (the yearly tax / 12, half up)
afterTax      = gross − siEmployee − martyrs − incomeTax
net           = afterTax − advanceRecovery                            (refused when negative)
```

Yearly income tax

```
yearlyIncomeTax(taxable):
    if taxable <= 0: return 0
    rates = band rates
    skipped = 0
    for k in 0..4: if skipFrom[k] > 0 and taxable > skipFrom[k]: skipped = k + 1
    for i in 0..skipped−1: rates[i] = rates[skipped] (or the top rate)
    tax = 0; lower = 0
    for each band i: if taxable <= lower: stop
                     tax += share(min(taxable, band.to) − lower, rates[i]); lower = band.to
    if taxable > lower: tax += share(taxable − lower, topBp)
    return tax
```

Each band's tax is rounded half up to the piastre on its own.

WORKED EXAMPLE

High earner check: yearly taxable 650,000 EGP (65,000,000). It passes the first threshold only, so band 1 is taxed at 10%:

- band 1: 4,000,000 × 10% = 400,000
- band 2: 1,500,000 × 10% = 150,000
- band 3: 1,500,000 × 15% = 225,000
- band 4: 13,000,000 × 20% = 2,600,000
- band 5: 20,000,000 × 22.5% = 4,500,000
- band 6: 25,000,000 × 25% = 6,250,000

Yearly tax 14,125,000 piastres (141,250 EGP).

10.5 The salary run and its posting

`submitPayroll()` checks everything with `previewPayroll()` (month not in the future: `future_month`; each person once; salary set: `salary_not_set`; recovery not above the advance owed: `more_than_advance`), then posts at once below `finance.approval_threshold_piastres` or waits for a second person (the amount

compared is the total net). The run happens once per month: `payroll_already_run`. The money account (bank or box) is locked and must hold the total net.

The entry is dated the month's last day at 23:00 Cairo when the month is over, else now (the current month may be paid while it runs). One entry for everyone:

Account	Debit	Credit	Tags
5402 Salaries and wages (one line per person)	gross		user, store, rider
5701 Social insurance, company share (per person)	siEmployer		user, store, rider
1500 staff advance (per person)		advanceRecovery	user, store
2303 Salary income tax payable		sum(incomeTax)	
2304 Social insurance payable		sum(siEmployee + siEmployer)	
2305 Martyrs fund payable		sum(martyrs)	
bank or cash box		sum(net)	

Idempotency key `payroll:<YYYY-MM>`; kind `payroll`. The entry's data keeps every person's figures (base, basic, allowances, bonus, commission, deductions, HR items, gross, insured wage, both insurance shares, martyrs, tax, recovery, net): payslips and the tax and insurance lists are read from it. In the same transaction the HR items are marked as taken (`markHrPayrollItems()`, each once) and the commissions are marked paid. The taxes owed are paid later from Taxes (chapter 15).

10.6 Advances and bonuses

Staff advances

`giveAdvance()`, through the second person rule from the limit. The person's 1500 wallet (`1500-nnnn`, an asset) holds what they owe.

Account	Debit	Credit	Tags
1500 staff advance	amount		user, store, rider
bank or cash box		amount	

Key `advance:<clientKey>`; kind `staff_advance`. It is taken back from a salary run (employees), from the payment of an outsourced person's bill (`advanceRecoveryPiastres`, chapter 12) or from a rider withdrawal (chapter 11).

Bonuses

An employee's bonus goes through the run (taxed, part of gross). An outsourced person's bonus is paid directly with `payStaffBonus()` (refused for employees: `employee_bonus_in_payroll`), through the second person rule from the limit:

Account	Debit	Credit	Tags
5702 Staff bonuses	amount		user, store, rider
bank or cash box		amount	

Key `staff_bonus:<clientKey>`; kind `staff_bonus`. It may pay an approved sales commission of the same amount (`commission_not_payable` otherwise).

10.7 Sales commission in payroll

Sales commission is computed by the Sales app (`packages/db/src/crm-sales.ts`, `packages/shared/src/crm-sales.ts`). Once a Cairo month is over, for each sales rep (key `commission:<rep>:<month>`):

```
counted fees = fees before VAT (revenue lines tagged with the merchant) of the rep's qualified won
               merchants inside their commission months
hit        = a fees target above 0 exists and counted fees >= target
commission = hit ? sum over deals of commissionOn(dealFees, deal.commission_bp) : 0
commissionOn(fees, bp) = share(max(0, fees), bp)
```

A hit is a draft approved (or adjusted with a reason) by another sales manager. Approved commissions of months up to the run's month (`payableCommissions()`) are added to the employee's gross in the salary run, taxed; outsourced reps are paid by a direct staff bonus.

10.8 Unpaid leave and violation deductions

`hrPayrollItems()` in `packages/db/src/hr-inbox.ts` hands the run, for an employee, every unpaid leave day and every final violation deduction up to the month's last day that no run has taken:

```
dayDeduction(basic, days, perMonth) = round(basic * days / perMonth)    (perMonth =
hr.payroll_days_per_month)
unpaid leave day item  = dayDeduction(basic, day fraction, perMonth)
violation item         = penalty deduction_amount ? violation.amountPiastres
                                                  : dayDeduction(basic, violation.days, perMonth)
```

They reduce the gross before tax. Outsourced people's deductions are only listed for their company. Riders paid per delivery are charged to their wallet instead (chapter 11).

10.9 Outsourced staff

An outsourced person's month is a supplier bill from their own outsourcing company (chapter 12) naming the person: the cost line carries the user, store and rider tags, so the person's cost appears in profit per store and merchant profit. Paying it can take back their advance (Cr 1500). The bill's expense account is the one chosen (for example 5402 Salaries and wages).

10.10 Payslips

Employees on the salary run see only their own payslips in My HR (`myPayslips()`, `myPayslip()`), read from the posted `payroll:<month>` entry; printable in English or Arabic; a `payslip_ready` push tells them. Riders paid per delivery see their wallet instead.

10.11 A full worked payslip

WORKED EXAMPLE

Month: October 2026. Employee with a package: basic 12,000 EGP, transport allowance 2,600 EGP, insured wage 10,000 EGP. Finance adds a 800 EGP target bonus. HR hands over 1 unpaid leave day. The employee owes a 500 EGP advance, recovered in full.

Step	Formula	Piastres	EGP
Base	1,200,000 + 260,000	1,460,000	14,600.00
Bonus		80,000	800.00
Unpaid day	round(1,200,000 × 1 / 30)	40,000	400.00
Gross	1,460,000 + 80,000 − 40,000	1,500,000	15,000.00
Insured wage	min(max(1,000,000, 270,000), 1,670,000)	1,000,000	10,000.00
Employee insurance	share(1,000,000, 1100)	110,000	1,100.00
Company insurance	share(1,000,000, 1875)	187,500	1,875.00
Martyrs fund	share(1,500,000, 5) = floor(750.5)	750	7.50
Yearly taxable	(1,500,000 − 110,000) × 12 − 2,000,000	14,680,000	146,800.00
Band 1	4,000,000 at 0%	0	
Band 2	1,500,000 at 10%	150,000	
Band 3	1,500,000 at 15%	225,000	
Band 4	7,680,000 at 20%	1,536,000	
Yearly tax		1,911,000	19,110.00
Monthly income tax	floor((1,911,000 + 6) / 12)	159,250	1,592.50
After tax	1,500,000 − 110,000 − 750 − 159,250	1,230,000	12,300.00
Advance recovered		50,000	500.00
Net paid		1,180,000	11,800.00

Posting (this person alone, from CIB), dated 31 October 2026 23:00 Cairo, key `payroll:2026-10`, kind `payroll`:

Account	Debit	Credit	Tags
5402 Salaries and wages	15,000.00		user, store
5701 Social insurance, company share	1,875.00		user, store
1500 staff advance		500.00	user, store
2303 Salary income tax payable		1,592.50	
2304 Social insurance payable		2,975.00	
2305 Martyrs fund payable		7.50	
1101 CIB		11,800.00	

Debits 16,875.00 = credits 16,875.00. The cost to the company is gross plus the company insurance: 16,875.00 EGP.

For comparison, a gross of 60,000 EGP with no insured wage on the package: the insured wage is clamped to 16,700 EGP, employee insurance 1,837.00, company insurance 3,131.25, martyrs 30.00, income tax 12,353.25, after tax 45,779.75 EGP.

10.12 Where the code lives

Concern	File and function
Deduction rules	`packages/shared/src/payroll.ts` : `yearlyIncomeTax` , `monthlyPayslip`
Run, advances, bonuses, payslips	`packages/db/src/finance/payroll.ts` : `payrollRules` , `listStaffPay` , `giveAdvance` , `previewPayroll` , `runPayroll` , `payStaffBonus` , `payslips` , `myPayslips`
Second person	`packages/db/src/finance/controls.ts` : `submitPayroll` , `submitAdvance` , `submitStaffBonus`
Packages	`packages/db/src/hr-pay.ts` : `packageInForce` , `packageGross`
HR deductions	`packages/db/src/hr-inbox.ts` : `hrPayrollItems` , `markHrPayrollItems` ; `packages/shared/src/hr-leave.ts` : `dayDeduction`
Commission	`packages/db/src/crm-sales.ts` : `payableCommissions` , `markCommissionsPaid` ; `packages/shared/src/crm-sales.ts` : `commissionOn` , `commissionMonthIndex`

11 Taxes, reports, reconciliation and closing

This chapter explains how the tax worksheets are built from the ledger (VAT, withholding, salary tax and insurance, company income tax), how tax is paid, how the financial statements and drill down reports compute and compare, how profit per store and the fully loaded cost per order share costs, how cash flow and the four week forecast work, how bank lines are matched, what the month end checklist checks, how a year is closed, and what the exports contain. Every report reads the ledger lines; nothing is stored as a balance.

11.1 Periods and Cairo time

Reports take Cairo calendar days. A day `D` covers `[cairoDayStart(D), cairoDayStart(D + 1))`; `cairoDayStart()` (`packages/shared`) applies Egypt's summer time, never a fixed offset. In SQL a moment's Cairo day or month is always taken as `(occurred_at AT TIME ZONE 'UTC') AT TIME ZONE 'Africa/Cairo'`. Drill down reports refuse ranges longer than 400 days (`range_too_long`).

11.2 VAT report (per month of a year)

`vatReport(db, year)` in `packages/db/src/finance/period.ts` reads every line on 2301 VAT payable in the year, by Cairo month. Columns follow the entry's kind, not only the line's sign:

```
purchase kinds = supplier_bill, supplier_bill_cancel, expense, expense_claim
output = −sum(lines) where kind = credit_note
                       or (kind not tax_payment, not a purchase kind, and line < 0)
input  =  sum(lines) where kind is a purchase kind
                       or (kind not tax_payment, not credit_note, not a purchase kind, and line > 0)
paid   =  sum(lines) where kind = tax_payment
due    = output − input
```

So a credit note lowers output, a cancelled bill lowers input, a debit note raises output, a rider office bill (kind `supplier_bill`) is input, and other entries (journal entries) go by sign.

11.3 Monthly VAT return

`vatReturn(db, month)` in `packages/db/src/finance/returns.ts` builds the return worksheet from the entries of the month that have a 2301 line (year close and tax payments excluded), grouped by kind:

```
for each kind: revenue = −sum(4101 lines); vat = −sum(2301 lines)
               expense = sum(lines on expense accounts, 2401 goods received and 2402 commission
accrued)
credit_note        : creditNotes += −revenue ; creditNoteVat += −vat
purchase kinds     : purchases += expense + sum(data.onWalletPiastres) ; inputVat += −vat
revenue ≠ 0 or vat > 0 (fees, minimum, debit notes...):
                     sales += revenue ; outputVat += vat
                     (debit notes also shown apart as "of which")
otherwise          : purchases += expense ; inputVat += −vat
carriedIn = max(0, balance of 2301 before the month)        (a debit balance is credit carried in)
due       = outputVat − creditNoteVat − inputVat − carriedIn
duePiastres = max(due, 0) ; carriedOut = due < 0 ? −due : 0
paidSince = VAT tax payments booked after the month
```

A rider office's commission since 11 November 2026 is booked on the office wallet at each delivery, so its month end bill entry carries only the VAT; `onWalletPiastres` in the bill entry's data adds the commission back to purchases (counted once, since the entry has one 2301 line).

WORKED EXAMPLE

November: fees 400,000 EGP with VAT 56,000; one credit note of 1,000 EGP with VAT 140; bills and expenses with VAT claimed 9,000; a debit balance of 2,000 carried in from October. Due = 56,000 – 140 – 9,000 – 2,000 = 44,860 EGP; nothing carried out.

The month end checklist has a hand ticked item for the VAT return (15.15).

11.4 Withholding tax, form 41

`withholdingPayments(db, { year, quarter })` lists every bill payment of the quarter that kept withholding tax, one row per payment, leaving out payments made by a cheque that later bounced or was cancelled (`reversedPaymentIds()`):

```
row: day, voucher JV number, supplier, tax id,
     net (amount before VAT of the payment = data.net = round(bill.net * settled / bill.total)),
     rate = net ? round(wht * 10000 / net) : supplier.whtBp,
     withheld = data.wht
```

The CSV (`form41-<year>-Q<n>.csv`) has the columns: Tax registration number, Name, Date, Amount before VAT, Rate % (two decimals), Withheld, Voucher. `whtReport()` in `payables.ts` gives the same quarter per supplier, flagging suppliers with withholding and no tax id.

11.5 Salary tax and social insurance lists

`payrollForms(db, month)` reads the month's `payroll:<month>` entry (chapter 14) and joins each person's national id and social insurance number from their employee file. Totals: gross, income tax, insured wages, both insurance shares, martyrs fund; `missingIds` counts people without an id or number. Two CSVs (salaries switch needed): the salary tax list and the social insurance list (Insurance number, National ID, Name, Insured wage, Employee share, Company share, Total).

11.6 Company income tax

```
corporateTaxEstimate(year):
    pl              = profit and loss of the year (year close entry left out)
    booked          = the 5801 Company income tax line of that year
    profitBeforeTax = pl.profit + booked
    estimate        = profitBeforeTax > 0 ? round(profitBeforeTax * finance.corporate_tax_bp / 10000) :
0
                        (setting default 2250 = 22.5%)
```

The accountant settles the amount (the taxable profit may differ from accounting profit). `bookCorporateTax()` books it before the year is closed, only for a year that is over, and only the difference with what is booked:

Account	Debit	Credit	Tags
5801 Company income tax	difference		
2306 Company income tax payable		difference	

Idempotency key `corporate_tax:<year>:<n>` where n is the number of earlier corporate tax entries of the year plus one; kind `corporate_tax`; dated two milliseconds before the year ends. A lower amount posts a negative difference (the lines turn round).

WORKED EXAMPLE

Profit before tax 2,400,000 EGP: estimate round(240,000,000 × 2,250 / 10,000) = 54,000,000 piastres (540,000 EGP). The accountant settles on 520,000 EGP: Dr 5801 52,000,000, Cr 2306 52,000,000. Changed later to 530,000 EGP: a second entry Dr 5801 1,000,000, Cr 2306 1,000,000.

11.7 Paying taxes

`payTax()` in `payables.ts`, through the second person rule from the limit. The tax is one of: VAT (2301), withholding (2302), salary income tax (2303), social insurance (2304, both shares), martyrs fund (2305), company income tax (2306). Never more than owed now (`more_than_owed`, under an advisory lock per tax); the money account must hold it.

Account	Debit	Credit	Tags
the tax payable account	amount		
bank or cash box		amount	

Key `tax_payment:<clientKey>`; kind `tax_payment`; data `{ tax, reference }`. `taxesOwed()` shows what is owed per tax (0 – balance).

11.8 Financial statements

All three are in `packages/db/src/finance/books.ts`; they sum lines per account (`sumsByAccount()`) with optional store and merchant filters. A balance is shown "as an accountant reads it": `natural(category, debit - credit)` keeps the sign for assets and expenses and flips it for liabilities, equity and income.

Report	Lines read	Result
Trial balance	all lines of the period	per account: debits, credits, balance; total debits always equal total credits (unfiltered)
Profit and loss	income and expense accounts in the period, the `year_close` entry left out	income, expenses, profit = income – expenses
Balance sheet	all lines before the end of the chosen day	assets, liabilities, equity, plus earnings = income – expenses not yet closed into 3401; wallets shown as one line per group (store boxes kept apart)

Accumulated depreciation (1701) is an asset account with a credit balance, so it appears as a negative asset. With a store or merchant filter only the lines tagged with them count, so a filtered report may not balance (the screen says so).

11.9 Drill down and comparisons

`drillReport()` in `packages/db/src/finance/drill.ts` runs the same three reports once for the period and once for the comparison, every line keyed by its account; a click opens the ledger lines behind the line (`GET /finance/ledger-lines`), whose total is the report line exactly. For the balance sheet the earnings drill starts after the last year close.

Comparison periods

`comparisonRange(range, mode)` in `packages/shared/src/report-period.ts` :

- `last_year` : the same days a year earlier (a day the month lacks becomes its last day).
- `previous` , in this order:
  1. a year to date (from 1 January to a day in a later quarter of the same year): the same days a year earlier;
  2. a quarter to date (from a quarter's first day to a later month of that quarter): the quarter before, the same number of days into it, never past that quarter's end (a quarter's last day stays a quarter's last day);
  3. any other range starting on the 1st: moved back by the number of months it covers (a month's last day stays a month's last day);
  4. otherwise: moved back by its own length in days.

WORKED EXAMPLE

Range	Previous
1 to 11 November 2026 (month to date)	1 to 11 October 2026
1 October to 11 November 2026 (quarter to date)	1 July to 11 August 2026
1 October to 31 December 2026 (whole quarter)	1 July to 30 September 2026
1 January to 11 November 2026 (year to date)	1 January to 11 November 2025
5 to 11 November 2026 (7 days)	29 October to 4 November 2026

Change and percent: `change = current − previous` ; `percent = previous == 0 ? none : round(change / |previous| × 1000) / 10` (one decimal). Presets: this month, last month, this quarter, this year, last year, custom. Exports: PDF (English or Arabic, made on the API), CSV and Excel.

11.10 Profit per store and the fully loaded cost per order

`profitBy(db, { from, to, by })` in `packages/db/src/finance/profit.ts` groups income (−sum of revenue category lines) and costs (sum of expense category lines) by the line's store, merchant or rider tag, `year_close` left out; lines without the tag form one company wide row.

`costPerOrder()`:

```
orders(s)       = delivered orders of store s in the period (deliveredAt in range)
allOrders       = sum over stores
companyCosts    = costs of the untagged (company wide) row
shared(s)       = allOrders > 0 ? round(companyCosts * orders(s) / allOrders) : 0
incomePerOrder  = round(income(s) / orders(s))
directPerOrder  = round(directCosts(s) / orders(s))
fullCostPerOrder= round((directCosts(s) + shared(s)) / orders(s))
profitPerOrder  = round((income(s) − directCosts(s) − shared(s)) / orders(s))
(null when the store delivered nothing)
```

WORKED EXAMPLE

Store A delivered 3,000 orders, store B 1,000. Company wide costs 200,000 EGP. A's share = round(20,000,000 × 3,000 / 4,000) = 15,000,000 (150,000 EGP). A's direct costs 240,000 EGP: direct cost per order 80 EGP, fully loaded (240,000 + 150,000) / 3,000 = 130 EGP.

11.11 Cash flow (direct method)

`cashFlow(db, { from, to, darkStoreId? })` in `period.ts`. Money is every line on groups 1100 (banks), 1200 (cash boxes, Money at Basata included) and 1300 (rider cash).

```
opening = sum of money lines before from ; closing = sum before to
per entry: net = sum of its money lines in the period
per kind : in  = sum of net over entries with net > 0
           out = −sum of net over entries with net < 0
section  : asset_purchase, asset_disposal → investing
           loan_received, loan_payment, capital_in, capital_out → financing
           every other kind → operating
```

Because each entry is netted first, a move between our own money accounts (a bank deposit from a box, a handover from rider cash to a box, a transfer between banks) nets to 0 and does not appear. Money borrowed and repaid is financing: a whole loan instalment, interest included, counts as financing (IAS 7 allows it); buying and selling equipment is investing; owner capital in and drawings out are financing; everything else is operating (`SECTION` in `period.ts`). With a store, only money lines tagged with that store count.

11.12 Four week cash forecast

`cashForecast(db, now)` in `period.ts`:

```
planned kinds = expense, bill_payment, payroll, tax_payment, staff_advance, prepayment,
                asset_purchase, capital_in, capital_out, other_income
trading       = sum(in − out) of moneyByKind over the last 56 days, planned kinds left out
weeklyTrading = round(trading / 8)
money now     = sum of money lines (1100, 1200, 1300) up to now
for week w = 0..3 (7 days from today):
    bills     = remaining of open bills due in the week (week 0 also takes overdue ones)
    recurring = amount of each active recurring expense whose day of month falls in the week
                (within its start and end months)
    salaries  = (number of month ends in the week) × gross of every package in force today
                for active employees on our payroll
    taxes     = week 0 only: VAT, withholding and postal fees owed now (never below 0)
    closing   = opening + weeklyTrading − (bills + recurring + salaries + taxes)
```

In week 0 the taxes also include the postal fees owed (2307): the query sums `vat_payable`, `wht_payable` and `postal_fee_payable`.

11.13 Forecast: 12 month scenarios

Accounting, Planning, Forecast holds named scenarios of the next 12 Cairo months, apart from the four week cash forecast above. Nothing in it posts to the ledger. Code: `packages/shared/src/forecast.ts` (the model, pure, whole piastres only, so the browser recalculates live while editing and the API gives the same numbers), `packages/db/src/finance/forecast.ts` (suggestion, storage, budget), `apps/api/src/finance/forecast.controller.ts` (`/finance/forecasts`).

Scenarios

A scenario (`forecasts`, numbers `FC-000001` onward, gapless) has a name (2 to 80 characters), a start month (the current Cairo month when it was made), the months its suggestion read, the opening balances at the start of the first month, its drivers (`forecast_drivers`: month, key, value) and its line overrides (`forecast_overrides`: month, line, amount, note, who and when). Status `draft` or `approved`. `createForecast()` starts either from the system's suggestion or as a copy of another scenario (`copyFromId`: drivers, overrides, start month and opening copied). Drafts are edited (`saveForecastDrivers()`, `saveForecastOverrides()`, one transaction each, audited `finance.forecast_edit`), renamed or deleted; an approved scenario refuses edits (`forecast_approved`) until accounting manage opens it again (`setForecastStatus()`, audited `finance.forecast_approve` and `finance.forecast_reopen`). A month outside the scenario's 12 months answers `month_outside_forecast`, an unknown key `invalid_driver` or `invalid_line`. Up to 6 scenarios can be compared side by side (`compareForecasts()`: each month's orders, revenue, costs, profit and cash at the end).

The suggestion

`forecastSuggestion()` reads the last 3 complete Cairo months of the books (fewer when the books are younger, none before the first ledger line or delivered order) and writes drivers for the first month:

```
growth_bp          = average month on month change of delivered orders (months whose previous month had
orders)
orders             = last month's delivered orders × (10000 + growth_bp) / 10000
orders_per_new_merchant = last month's orders / merchants with orders last month
cod_share_bp       = cash orders × 10000 / orders           cod_per_order = cash collected / cash
orders
fee:<kind>         = fee income of that entry kind / orders  (per delivered order, before VAT)
rider_pay_per_order, office_commission_per_order, postal_per_order, packaging_per_order
                   = 5101, 5102, 5103, 5404 over the months / orders
salaries           = gross of the salary packages in force
rent, fixed:<account> = monthly average of every other income and cost account
                     (recurring expenses instead where set up; company income tax left out)
loan_principal, loan_interest, depreciation = from the loan schedules and the asset register
vat_bp             = fee.vat_bp            vat_input = average VAT claimed back per month
merchant_hold_days = days of last month's net cash the merchants' balance holds (at most the month's
days)
```

Every division is `mulDiv(a, b, c)`: a × b ÷ c rounded half away from zero, in whole numbers.

Drivers and the model

Level drivers (`growth_bp`, `orders_per_new_merchant`, `cod_share_bp`, `cod_per_order`, the four per order costs, `salaries`, `rent`, `vat_bp`, `vat_input`, `merchant_hold_days`, `rider_cash_end`, every `fee:<kind>` and `fixed:<account>`) keep their last set value in later months. Flow drivers (`orders`, `new_merchants`, `capital_in`, `drawings`, `loans_in`, `loan_principal`, `loan_interest`, `capex`, `depreciation`) count only in the month they are set. `forecastCompute()`, month by month:

```
base        = the month's own orders value when set, else (after month 1) base × (10000 + growth_bp) /
10000
newMerchants += new_merchants
orders      = max(0, base + newMerchants × orders_per_new_merchant)
codOrders   = orders × cod_share_bp / 10000 ;  codCollected = codOrders × cod_per_order
fee:<kind>  = orders × driver   ; rider_pay, office_commission, postal_fee, packaging = orders × per
order driver
salaries, rent, fixed:<id>, depreciation, loan_interest = the driver
each line   = its override for the month when one is set, else the computed amount
profit      = revenue lines − cost lines ; feeIncome = sum of fee lines
VAT         vatOutput = feeIncome × vat_bp / 10000 ; last month's VAT owed (if positive) is paid this
month;
            vatEnd = vatOwed − paid + vatOutput − vat_input   (a credit is carried, never paid back)
merchants   net = codCollected − feeIncome − vatOutput ; merchantsEnd = net × merchant_hold_days / days
in month
            payouts = merchants opening + net − merchantsEnd
rider cash  = rider_cash_end, else the opening rider cash
operating   = profit + depreciation + Δmerchants + ΔVAT − Δrider cash
investing   = −capex
financing   = loans_in − loan_principal + capital_in − drawings
cash        += operating + investing + financing
balance sheet: cash, rider cash, equipment (+capex − depreciation), other assets (as opened) =
               merchants, VAT, loans, other liabilities (as opened) + capital (+capital_in − drawings)
+ retained (+profit)
```

Warnings name the months where cash goes below 0 (`cash_below_zero`), loans below 0 (`loans_below_zero`) or merchants owe us (`merchants_owe_us`). Periods are each month, the scenario's quarters (Q1 its first three months) and the year: flows add up, balances are those of the period's last month.

Overrides

Any profit and loss line of any month (a cost line, `fee:<kind>` or `fixed:<account>`, the account an income or expense account) can be set by hand with a note; the line is then marked edited and no longer computed. Clearing the override (`null`) computes it again. Each change is audited with the line, month, before, after and note.

Use as budget

`forecastBudgetPreview()` (accounting view) lists, per month and ledger account, what the scenario would write into the company wide budgets next to the budget there now: fee lines go to 4101, rider pay 5101, office commission 5102, postal fee 5103, packaging 5404, salaries 5402, rent 5401, depreciation 5601, loan interest 5904, `fixed:<id>` to its account. `forecastToBudget()` (accounting manage, only an approved scenario, else `forecast_not_approved`) writes them in one transaction through `setBudget()`, with `mode` `replace` (existing rows are overwritten) or `skip` (existing rows kept). A negative amount is never a budget and is skipped; system accounts not opened yet are opened. The result counts written and skipped rows; audited `finance.forecast_to_budget`. Rows written this way count as months set by hand (below). The screens are shown in the chapter Screens: Reporting and Planning, section "Forecast".

11.14 Coming up and due

`paymentsDue(db, now, { stores, days, kinds })` in `packages/db/src/finance/payments-due.ts` lists every payment the company already knows it will make, with its due Cairo day, the amount and the screen that pays it. It is read only. Served by `GET /finance/payments-due?days=&kinds=` (accounting view; days 0 to 90, default 30; kinds comma separated): the dashboard card asks 30 days, the page `/finance/due` 90 days with a filter by kind.

Kind	Due day and amount
`recurring`	Each active recurring expense on its day of each month in the window (between its start and end months), its amount. A month whose request was approved or rejected is done; one still pending is shown as waiting for approval and links to it.
`bill`	Open supplier bills by due date, what is still unpaid (total – paid).
`loan`	Unpaid loan instalments on their due day, the instalment.
`salaries`	The salary run on each month's last day until `payroll:<month>` is posted; an estimate, the gross of the packages in force that day. Last month is included when it was never run, once a salary run exists at all.
`tax`	VAT, withholding, salary tax, social insurance and martyrs fund, each: what was owed at the start of this month less tax payments since is due at the end of this month (never more than owed now), the rest at the end of next month. There is no legal calendar in the code, so this "end of next month" rule is used for every tax.
`postal`	The postal fees owed (2307), by the same rule as the taxes.
`payout`	A merchant's prepared draft payout is due from the day it was prepared; otherwise a merchant we owe money is due on their next payout day (`nextPayoutDay(schedule, today)`) with what is owed now.

```
group(dueDay) = overdue  when dueDay < today
              = week     when dueDay ≤ today + 7
              = month    when dueDay ≤ today + 30
              = later    otherwise (only when the window is longer than 30 days)
order: due day, then the larger amount first; each group has its count and total
```

Items with a store (bills and recurring expenses of a store) reach only people of that store; company wide items everyone with Accounting view; merchant payouts only people with all stores.

Push reminders (type `payment_due`, texts `payment_due_soon` and `payment_due_today`): the push job reads the list at most once every 10 minutes (`PAYMENT_DUE_SCAN_MS`, claimed across instances) and tells people with Accounting use of items due exactly `finance.due_reminder_days` (default 3) days ahead and of items due today; overdue items are never pushed again. Each item and send day is one key, `payment_due:<kind>:<id>:<day>`, so a second run sends nothing; the merchant payouts of one due day go out as one notification. Store items go to that store's people, payouts to people with all stores. The screens are shown in the chapter Screens: Dashboard and Money in, section "Coming up and due".

11.15 Budgets

Budgets are per income or expense account, month and store (or company wide); `budgetVsActual()` in `period.ts` shows the variance for a month (with a store: only that store's budgets and lines; without: the whole company).

Planning by quarter

Budgets are planned per quarter first (Q1 January to March, Q2 April to June, Q3 July to September, Q4 October to December), then by month; rows are still stored per month, so budget against actual per month keeps working (`packages/shared/src/budgets.ts`, `packages/db/src/finance/budgets.ts`, `apps/api/src/finance/budgets.controller.ts`).

```
splitQuarter(total) : each = floor(total / 3) ; months = [each, each, total − 2 × each]
                      (whole piastres; the last month takes the remainder)
setQuarterBudget(year, quarter, account, store or company wide, amount)
    → the three months get splitQuarter(amount), manual = false
setBudget(month, ...)  (the existing month route, PUT /finance/budgets)
    → that month only, manual = true
quarter amount = always the sum of its three months
```

WORKED EXAMPLE

Q1 rent of a store planned at 100,000.00 EGP (10,000,000 piastres): January 3,333,333, February 3,333,333, March 3,333,334. Changing March by hand to 4,000,000 marks it manual; the quarter now reads 10,666,666 and is shown as edited.

The account must be an income or expense account (`account_not_found`), the year 2020 to 2100 (`invalid_year`), the quarter 1 to 4 (`invalid_quarter`), the amount a whole number of piastres not below 0. Setting a quarter is audited as `finance.budget_quarter_set` with the months before and after; a month as `finance.budget_set` with its manual flag.

`budgetYear(year, scope, groupCode?)` (`GET /finance/budgets/year?year=&scope=&group=`) gives every income and expense account (active, or holding a budget) with budget, actual, variance and percent per month, per quarter and for the year. Scope `all` is every store and the company wide part together, `company` only company wide budgets and lines with no store, or a store id. Actual leaves the `year_close` entry out. The variance is positive when better than plan: actual – budget for income, budget – actual for costs; the percent is `variancePercentBp(variance, budget)` (none without a budget). A quarter is flagged `edited` when one of its months was changed by hand and `uneven` when its months are not the equal split.

`copyBudgetYear(fromYear, toYear, scope, percentBp, replace)` (`POST /finance/budgets/copy`) copies a year into another, optionally raised or lowered by a percent in basis points (500 = 5%; at least –10,000):

```
scaleBudget(amount, bp) = round half up(amount × (10000 + bp) / 10000), never below 0   (BigInt, no
floats)
per account, store and quarter of the source year:
    no month changed by hand → splitQuarter(scaleBudget(quarter total, bp)), manual = false
    otherwise                → each month scaleBudget(month, bp), keeping its manual flag
```

It refuses `same_year`, `budget_year_empty` when the source year has nothing in the scope, and `budget_year_not_empty` when the target year already has rows in the scope unless `replace` is set, which deletes them first. Audited `finance.budget_copy`. Reading needs accounting view, setting and copying accounting use; people limited to some stores must name one of their stores. The screens are shown in the chapter Screens: Reporting and Planning, section "Budgets".

11.16 Bank reconciliation

Importing statements

Finance uploads the bank's statement as .xlsx or CSV (read on the server without a spreadsheet library; old .xls is refused with a "save as" hint), picks the columns (guessed from English and Arabic headers; debit and credit columns or one signed amount). Dates and amounts are parsed by `parseStatementDate()` and `parseStatementAmount()` (`packages/shared/src/statement.ts`: day first dates, month names, Excel day numbers; brackets, a minus or DR make an amount negative; no floats). Each upload is a statement `BS-000001` onward. Each line gets a fingerprint:

```
base = day | amount | balance | description | reference
n    = how many times the same base appeared earlier in this file (1, 2...)
fingerprint = sha256(base | n)
```

Lines whose fingerprint already exists for the account are skipped, so overlapping uploads add nothing, while two equal transfers on one day in one file are both kept. A future day or an amount of 0 is refused.

Automatic matching

`autoMatch()` in `packages/db/src/finance/bank.ts` runs after an import and when the screens open. For each unmatched bank line, oldest first, it takes the ledger lines of the same account with exactly the same amount within 3 days that prove no bank line yet; when exactly one is the closest it is matched; two equally close ones are left for a person. Matching never records money.

Suggestions for a person

`rankSuggestions(bankLine, openLines)` in `packages/shared/src/bank-match.ts`:

Rank	Reason	Rule	One click match
0	`same_day`	same amount, same Cairo day	yes
1	`within_3_days`	same amount, at most 3 days apart	yes
2	`sum`	two or three open lines (same direction, smaller, within 7 days; at most 40 candidates, at most 6 sums) adding up exactly	yes
3	`same_amount`	same amount, further than 3 days	yes
4	`near_amount`	same sign, within max(100 piastres, round(5% of the bank amount)), within 7 days	no
5	`partial`	same direction, smaller, within 7 days, not part of a sum and not near	no

Sorted by rank, then by distance from the bank amount, then by day gap, then by ids; at most 12 shown. Lines picked by hand match when they add up exactly (`sumMatches()`; the extra lines are kept in `bank_line_matches`; a trigger lets a ledger line prove only one bank line; reopening removes them). Money out with no entry can be booked as bank charges in one tap (`bookBankCharge()`: an expense on 5501 Bank charges through the second person rule, key `expense:bankcharge:<bankLineId>`, then matched); a line that needs no entry is set aside with a reason of at least 5 characters. Money in is never created from a statement line.

Reconciliation view

`reconciliation(db, { accountId, day })`: book balance at the end of the day, the last bank balance printed on a statement line up to that day, the difference, bank lines not in the books, and book lines not on any statement since the first day a statement covers.

11.17 Month end checklist and closing the books

`monthEndChecklist(db, month)` in `packages/db/src/finance/cash-control.ts` checks a month live:

Item	Counted as left	Waits for the month's end
`cash_handovers`	pending handover differences created before the month's end, plus riders still holding cash collected before it	no
`bank_lines`	unmatched statement lines dated in the month	no
`receipts`	entries of receipt kinds in the month with no receipt photo (on the entry, its request or its payout)	no
`petty_cash`	petty expenses created before the month's end still waiting for approval	no
`invoices`	merchants with fee income in the month and no invoice for it	yes
`payroll`	1 when no `payroll:<month>` entry and some salary package exists	yes

Item	Counted as left	Waits for the month's end
`storage`	days of the month (up to yesterday, from the first storage charge ever) with no storage entry	no
`depreciation`	prepayment and asset months due for the month whose key (`prepaid:` or `depr:`) has no entry	yes
`vat_return`	always ticked by hand	yes
`books_closed`	1 until the books are closed through the month's last day	yes

Any item except `books_closed` can be ticked by hand with a note of at least 3 characters (`month_end_marks` , audited). The month is ready to close when it is over and every other item is done. Closing (`closeBooks()` in `controls.ts` , accounting manage) sets `books_lock.closedThrough` to a day already over and `lockedBefore` to the start of the next day; `post()` and a database trigger then refuse lines before it (`books_closed`). Moving the date back reopens days: admin only, with a reason of at least 5 characters, audited as `finance.books_reopen` .

11.18 Closing a year

`closeYear(db, { year })` in `period.ts` (approve level), once per year, years in order (refused while an earlier year still has income or expense balances: `earlier_year_open`), only a year that is over:

```
balances = per income and expense account, sum of lines before the year's end (non zero only)
total    = sum(balances)          (= −profit, since income is a credit)
lines    = each account: −balance ; 3401 Retained earnings: +total
```

Account	Debit	Credit	Tags
each income account		its credit balance	
each expense account		its debit balance	
3401 Retained earnings	loss	profit	

Key `year_close:<year>` ; kind `year_close` ; dated the year's last millisecond (Cairo). A lock past that moment is lifted for this entry and put back in the same transaction; the books are then closed through 31 December (or later if they already were). Profit and loss, profit per store and merchant profit leave the `year_close` entry out, so the closed year still shows its result. Book the company income tax (15.6) before closing.

11.19 Exports

Journal CSV

`journalLines(db, { from, to })` returns every ledger line between two Cairo days, one row per line, ordered by entry number then amount (debits first). The CSV (`GET /finance/export/journal.csv` , `apps/api/src/finance/period.controller.ts`) is importable by Odoo, Zoho Books, QuickBooks and Daftra:

Column	Value
Date	Cairo day of the line
Journal number	`JV-nnnnnn`
Kind	entry kind
Description	entry memo
Account code	chart code
Account name	English name, else the owner's name (merchant, store, user, supplier)
Account name (Arabic)	Arabic name, else as above
Debit	EGP with two decimals when the line is a debit, else empty
Credit	EGP with two decimals when the line is a credit, else empty
Store	store code of the line's tag
Merchant	merchant name of the line's tag

Audit pack

`GET /finance/export/audit-pack.zip` (a stored zip, no library) holds, in a folder `audit-<from>-<to>`:

- `journal.csv`: the journal above for the period;
- `trial-balance.csv`: Account code, Account name, Debit, Credit for the period;
- `chart-of-accounts.csv`: Code, Name, Name (Arabic), Category, Group, Active, Balance now (inactive accounts included);
- `vat-<year>.csv`: Month, VAT on our fees, VAT claimed back, Due, Paid for the year of the period's last day;
- `receipts/JV-nnnnnn-k.jpg` (or png, webp): every receipt photo attached to an entry, its request or its payout in the period, named by voucher.

All CSVs follow the house rules: `UTF-8` byte order mark, formula prefixes neutralised, money as EGP text from integers.

The journal list and the entry page

The journal is browsed at `/finance/journal`, the Accounting list view `journal` (`GET /finance/lists/journal`, rows inside the person's stores). Search covers numbers, memos and references. Filters: dates (today and the other date presets; the default is the last 7 days), kind families (`deliveries`: the fee, COD, rider pay and postal fee kinds; `cash`; `payouts`: merchant payouts, rider bonus payouts and rider withdrawals; `bills`; `manual`: journal entries and adjustments), entries with an order, and entries without a receipt. Group by kind, day, month or who made it; sort by number, day or amount; the amount column is totalled. Every row opens `/finance/journal/<id>`, the entry page with its lines, source document, files and the printable voucher (chapter 12, "The journal entry page"). The screens are shown in the chapter Screens: Reporting and Planning, sections "The journal list", "Searching and filtering the journal" and "The journal entry page".

11.20 Where the code lives

Concern	File and function
Budgets by quarter	`packages/shared/src/budgets.ts` : `splitQuarter` , `quarterMonths` , `scaleBudget` , `variancePercentBp` ; `packages/db/src/finance/budgets.ts` : `setQuarterBudget` , `budgetYear` , `copyBudgetYear`
Forecast	`packages/shared/src/forecast.ts` : `forecastCompute` , `forecastPeriods` , `forecastBudgetRows` , `mulDiv` ; `packages/db/src/finance/forecast.ts` : `forecastSuggestion` , `createForecast` , `saveForecastDrivers` , `saveForecastOverrides` , `setForecastStatus` , `compareForecasts` , `forecastBudgetPreview` , `forecastToBudget`
Coming up and due	`packages/db/src/finance/payments-due.ts` : `paymentsDue` , `dueGroupOf` ; `packages/shared/src/payments-due.ts` ; push source `paymentDueEvents` in `packages/db/src/push.ts`
VAT report, cash flow, forecast, year close, journal export	`packages/db/src/finance/period.ts` : `vatReport` , `cashFlow` , `cashForecast` , `closeYear` , `journalLines` , `receiptPhotos` , `budgetVsActual`
Tax returns	`packages/db/src/finance/returns.ts` : `vatReturn` , `withholdingPayments` , `payrollForms` , `corporateTaxEstimate` , `bookCorporateTax` , `comparativeStatements`
Tax payments, withholding report	`packages/db/src/finance/payables.ts` : `payTax` , `taxesOwed` , `whtReport`
Statements	`packages/db/src/finance/books.ts` : `trialBalance` , `profitAndLoss` , `balanceSheet` , `natural`
Drill down and periods	`packages/db/src/finance/drill.ts` ; `packages/shared/src/report-period.ts` : `comparisonRange` , `presetRange` , `changeOf`
Profit	`packages/db/src/finance/profit.ts` : `profitBy` , `costPerOrder`
Bank	`packages/db/src/finance/bank.ts` : `importStatement` , `autoMatch` , `bookBankCharge` , `reconciliation` ; `packages/shared/src/bank-match.ts` : `rankSuggestions` , `sumMatches`
Month end and books lock	`packages/db/src/finance/cash-control.ts` : `monthEndChecklist` , `markMonthEndItem` ; `controls.ts` : `closeBooks`
Export endpoints	`apps/api/src/finance/period.controller.ts`

12 UI development

The web app (`apps/web`) is a Next.js app router project in TypeScript with Tailwind CSS. It holds no business rule of its own: every number and every decision comes from the API, and the screens only collect input, show answers and keep state in the address bar. This chapter explains how pages are built, how text and direction are handled, the brand tokens, the app frame, and every reusable building block, with the props a developer needs.

12.1 App router structure

Pages live in `apps/web/src/app` . Each area is a folder with a `layout.tsx` that draws the frame and pages under it:

Folder	Area	Frame
`app/page.tsx` , `how-it-works` , `solutions` , `contact` , `team`	Public company site	`SiteShell` (its own scoped styles)
`home`	App tiles for team members	None (a grid of tiles)
`admin`	Operations, Inventory, Merchants, Customers, HR, Sales, Settings	`AppShell`
`finance`	Accounting	`AppShell` with the Accounting search and badges
`store`	Dark store staff (mobile first)	`store/[storeId]/layout.tsx` with `BottomTabs`
`rider`	Riders (mobile first)	Each page draws `AppHeader`
`merchant`	Merchant users	`DashboardShell` (Navy sidebar)
`my-file`	My HR for every team member and rider	
`t/[token]` , `p/[token]` , `setup/[token]`	Public tracking page, public proposal, one time password setup	
`api/[...path]`	The proxy to the API	Route handler

Accounting pages follow one pattern: a server component page reads its `searchParams` , fetches what it needs with `apiGet` , and renders building blocks. Most pages set `export const dynamic = 'force-dynamic'` because their data is always live.

```
// apps/web/src/app/finance/layout.tsx
export const dynamic = 'force-dynamic';

/** The Accounting app. */
export default async function FinanceLayout({ children }: { children: ReactNode }) {
  const controls = await apiGet<FinanceControls>('/finance/controls');
  return (
    <AppShell
      search={<AccountingSearch />}
      badges={{
        '/finance/approvals': controls.waiting + controls.payoutsToApprove,
        '/finance/receipts': controls.missingReceipts,
      }}
    >
      {children}
    </AppShell>
  );
}
```

12.2 Fetching data

Server components

`apps/web/src/lib/api-server.ts` (marked `server-only`) reads the access token from the cookie and calls the API directly at `API_URL` with an `Authorization: Bearer` header and `cache: 'no-store'`:

- `apiGet<T>(path)`: a 401 redirects to `/login`; a 404 or 403 shows the not found page (a record of another store, an old link); other errors throw.
- `apiTry<T>(path)`: for a secondary part of a page; answers null instead of failing, so the rest of the page still shows.
- `getMyAccess()`: `GET /me/access`, cached once per request with React `cache`, so every layout and page asking gets one call. `getMyProfile()` returns the team member's access profile; `getMerchantRights()` and `requireMerchantRight()` serve merchant pages.

Client components

Client components (files starting with `'use client'`) call the API through the same origin proxy, so the httpOnly cookie is sent: `apiSend(method, path, body)` and `apiGetClient(path)` in `apps/web/src/lib/api-client.ts`. Both return `{ ok: true, data }` or `{ ok: false, error, details? }` and never throw; a network failure answers `error: 'network'`. After a successful change the component calls `router.refresh()` so the server components fetch fresh data. The accounting helper `useFinanceAction()` wraps this: it sends one action, shows the translated result or error message, says "sent for approval" when the answer is `{ status: 'pending' }`, and refreshes.

The /api proxy and the middleware

The browser never calls the API host. `app/api/[...path]/route.ts` forwards every method to `API_URL` read at runtime, so one build runs on any host (described in the architecture chapter). `middleware.ts` runs on the Node.js runtime for the signed in areas, decides with `decideAccess` whether to allow, send to login, refresh the session or redirect to the person's own area, and passes the path in a request header so `AppShell` can show not found for a page the person may not open.

12.3 Translations, Arabic and right to left

> **RULE**
> The UI supports Arabic (right to left) and English (left to right). Strings go through translation files, never written into components.

Translations use `next-intl`. The locale comes from the cookie `shiply_locale` (`en` or `ar`, default `en`), not from the address, so areas keep plain paths like `/finance`. `apps/web/src/i18n/request.ts` loads `messages/<locale>.json` for each request. The theme is the cookie `shiply_theme` (`system`, `light`, `dark`). The language and theme menu sets the cookie with `document.cookie` and calls `router.refresh()`.

Messages are one JSON file per language with top level namespaces per area: `common`, `admin`, `store`, `merchant`, `rider`, `finance`, `workflow`, `listView`, `lists`, `documents`, `infoTips`, `apps`, `hr`, `sales`, `cashControl`, `contracts`, `riderPay`, `basata` and others. Server components use `getTranslations('finance.journal')`; client components use `useTranslations('listView')`. Values take ICU placeholders: `t('label', { term })`. `t.has(key)` is used to fall back when a code may have no message (for example unknown error codes fall back to `generic`).

> **RULE**
> Translation keys never contain dots, because `next-intl` treats dots as nesting. Setting names such as `sla.pick_minutes` become keys like `sla__pick_minutes`.

A test (`apps/web/src/lib/messages.test.ts`) fails when Arabic and English do not have exactly the same keys, or when any key contains a dot.

Direction is set once on `<html>`: the root layout writes `lang` and `dir` (`rtl` for Arabic) from the locale. Components then use logical Tailwind classes that follow the direction (`ms-`, `me-`, `ps-`, `start-`, `end-`, `border-s`) and the `rtl:` variant where a picture must turn (a chevron uses `rtl:rotate-180`). Numbers, phones, amounts, document numbers and order numbers stay left to right inside Arabic text with the class `ltr-iso`:

```
/* Numbers, phones and order IDs stay left to right inside Arabic text. */
.ltr-iso {
  direction: ltr;
  unicode-bidi: isolate;
}
```

Amounts and dates always use Latin digits (the date formatters use `ar-EG-u-nu-latn`). Arabic text never takes letter spacing, because it breaks the joining of letters. Charts keep time running left to right in Arabic too. The brand's S arrow mark is never mirrored.

12.4 Theme and brand tokens

Colors, type, space, radius and shadow are CSS custom properties in `apps/web/src/styles/shiply-tokens.css`, named by role, not by color. A Tailwind preset (`apps/web/src/styles/shiply.tailwind.js`) maps Tailwind names to these variables, so dark mode needs no extra classes in page code:

```
colors: {
  shiply: { cream: '#FAF1E8', navy: '#082341', blue: '#0046FC', lime: '#CFF905' },
  bg: 'var(--shiply-bg)',
  surface: 'var(--shiply-surface)',
  text: 'var(--shiply-text)',
  'text-muted': 'var(--shiply-text-muted)',
  primary: 'var(--shiply-primary)',
  'on-primary': 'var(--shiply-on-primary)',
  // Lime: one highlight per screen at most, always with on-accent (Navy) text.
  accent: 'var(--shiply-accent)',
  'on-accent': 'var(--shiply-on-accent)',
  ...
}
```

Palette

Brand color	Hex	Use
Cream	`#FAF1E8`	Brand grounds, text on Navy cards
Navy	`#082341`	Text, the dark theme ground, the merchant sidebar, wallet cards
Shiply Blue	`#0046FC`	Actions, links, charts
Lime	`#CFF905`	Accent: at most one highlight per screen, never text on white or Cream, text on Lime is always Navy

Role	Light	Dark
`bg` (page)	`#F3F6FA`	`#082341`
`surface` (cards)	`#FFFFFF`	`#0D2B4D`
`surface-raised`	`#F3F6FA`	`#133559`
`line`	`#DFE5EE`	`#24476F`
`line-strong` (form fields, 3:1)	`#7F8798`	`#6B85AB`
`text`	`#082341`	`#FAF1E8`
`text-muted`	`#5B6B82`	`#BFCBE0`
`primary`	`#0046FC`	`#3366FF`
`link`	`#0046FC`	`#9DB5FF`
`accent` / `on-accent`	`#CFF905` / `#082341`	same
`danger`	`#B42318`	`#FF8A7A`
`status-new`	`#4E5A72`	`#B3BFD0`
`status-in-progress`	`#6A3FD6`	`#B9A2FF`
`status-heading-to-customer`	`#A8188E`	`#F28BDE`
`status-delivered`	`#0F6A35`	`#5FD38D`

Role	Light	Dark
`status-needs-action`	`#9A4100`	`#FFB35C`
`status-returning`	`#B42318`	`#FF8A7A`
`hero` / `hero-ink` / `hero-accent` (wallet cards)	`#082341` / `#FAF1E8` / `#CFF905`	`#0046FC` / `#FFFFFF` / `#CFF905`

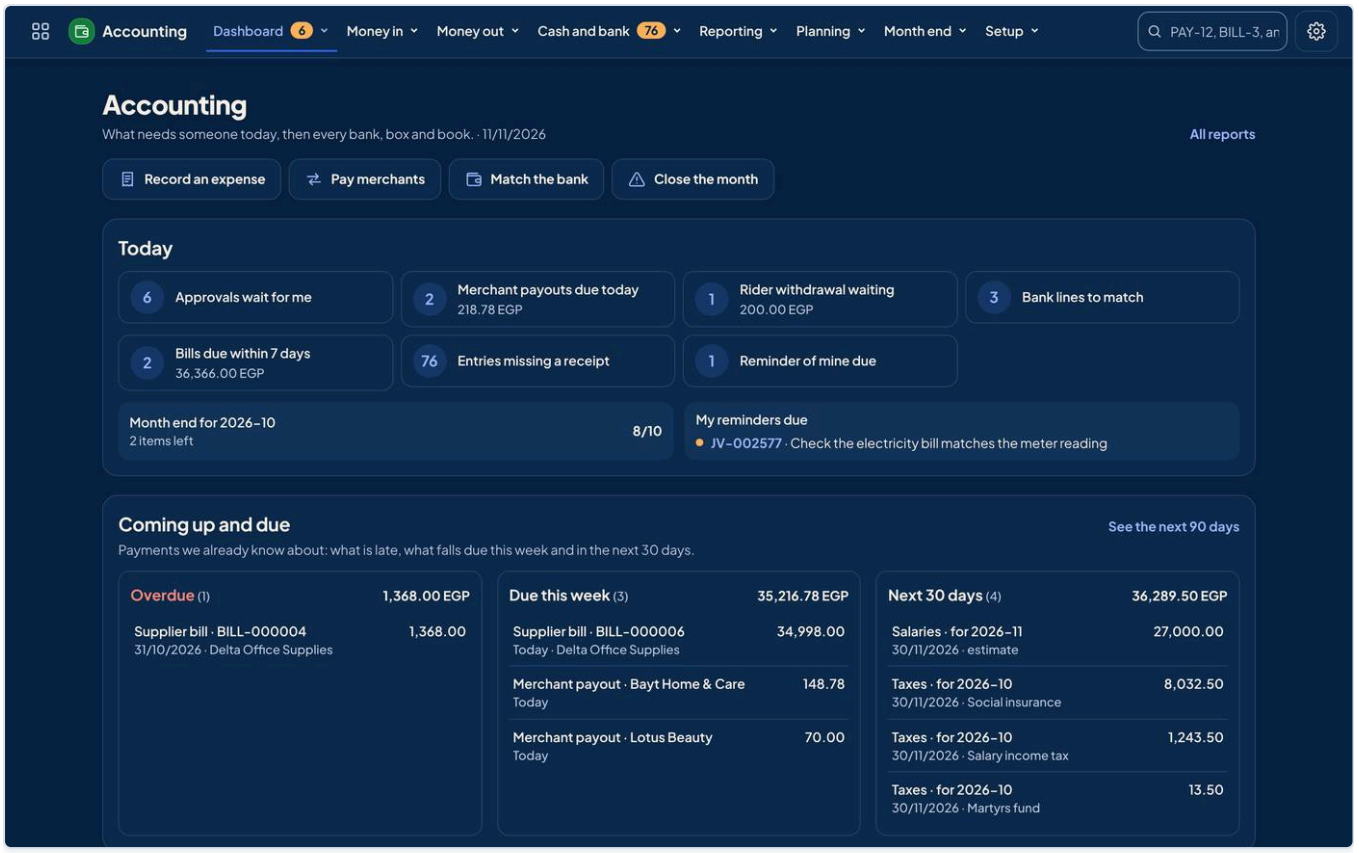
The dark theme applies with `[data-theme="dark"]` on `<html>` (an explicit choice) or with the operating system setting when the person chose system (`@media (prefers-color-scheme: dark)` on `:root:not([data-theme='light'])`). The two dark blocks are identical, and `.shiply-on-navy` applies the same dark roles inside a light page (the merchant sidebar). Buttons are component classes in `globals.css` : `btn-primary` (Blue), `btn-secondary` (Navy outline), `btn-accent` (Lime with Navy text). Status pills use the status color as text on a 14% tint of itself.

Fonts and type

Fonts are self hosted woff2 files: Plus Jakarta Sans 400 to 800 for Latin and IBM Plex Sans Arabic 400 to 700 for Arabic pages (its 700 also serves 800). Type sizes are tokens and Tailwind sizes: `display` 56px, `h1` 40px, `h2` 28px, `h3` 20px, `body-lg` 18px, `body` 16px, `body-sm` 14px, `caption` 13px, `label` 12px with letter spacing, and `number-lg` 28px for big numbers. Tables of numbers use the class `tabular` (tabular figures).

Contrast test

`apps/web/src/styles/tokens.test.ts` parses the token file and checks every text and interface pair the components use against WCAG AA in both themes: 4.5:1 for text on every ground (text, muted text, link, danger and each status color on `bg` , `surface` and `surface-raised`), 3:1 for form borders, focus rings and buttons against the page, plus `on-primary` on `primary` , `on-accent` on `accent` , `on-status` on every status color and the hero pairs. It also checks that the system dark block, the explicit dark block and `.shiply-on-navy` are identical, and that each app tile's icon color (white or Navy, chosen by `tileInk`) is readable on the tile.



The Accounting dashboard at `/finance` shown twice side by side: in the light theme (grey page, white cards, Navy text) and in the dark theme (Navy grounds, Cream text), with the same cards and sparklines.

12.5 Layout: apps, top bar and menus

Team members land on `/home`, a screen of app tiles. The tiles come from `appsFor(profile)`: an app shows when the person has at least view on its access app (Settings needs manage) and at least one page they may open. A person with store use also gets the Store screens tile.

Inside an app, `AppShell` (a server component) draws the `AppBar` and the page. It reads the profile, the theme and the path header; when the path is a page on a menu the person may not open, it answers not found. `AppBar` holds the grid button back to `/home`, the app's name, its menus and the account menu. A menu with one page is a plain link; a menu with several opens a list. Counts (`badges`) show next to a page and its menu. On phones (below the `lg` breakpoint) the menus fold into one Menu button that lists them all.

Every app, menu and page is defined once in `apps/web/src/lib/apps.ts`. Pages keep their addresses; the file only says which app and menu each belongs to and what access it needs:

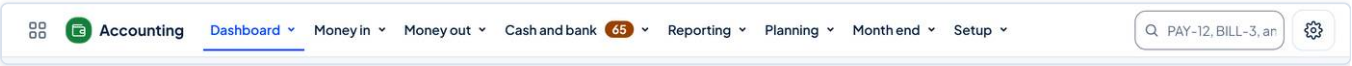
```
export interface PageNeed {
  app?: AccessApp;        // the app whose level is needed (default: the page's app)
  level?: AccessLevel;    // default view
  customerData?: boolean;
  salaries?: boolean;
  owner?: boolean;        // accounting manage and settings manage
}
export interface AppPage { href: string; label: string; exact?: boolean; need?: PageNeed }

// part of the Accounting app
{
  label: 'moneyOut',
  pages: [
    { href: '/finance/expenses/new', label: 'recordExpense', need: { level: 'use' } },
    { href: '/finance/bills', label: 'bills' },
    { href: '/finance/payroll', label: 'payroll', exact: true, need: { salaries: true } },
    ...
  ],
}
```

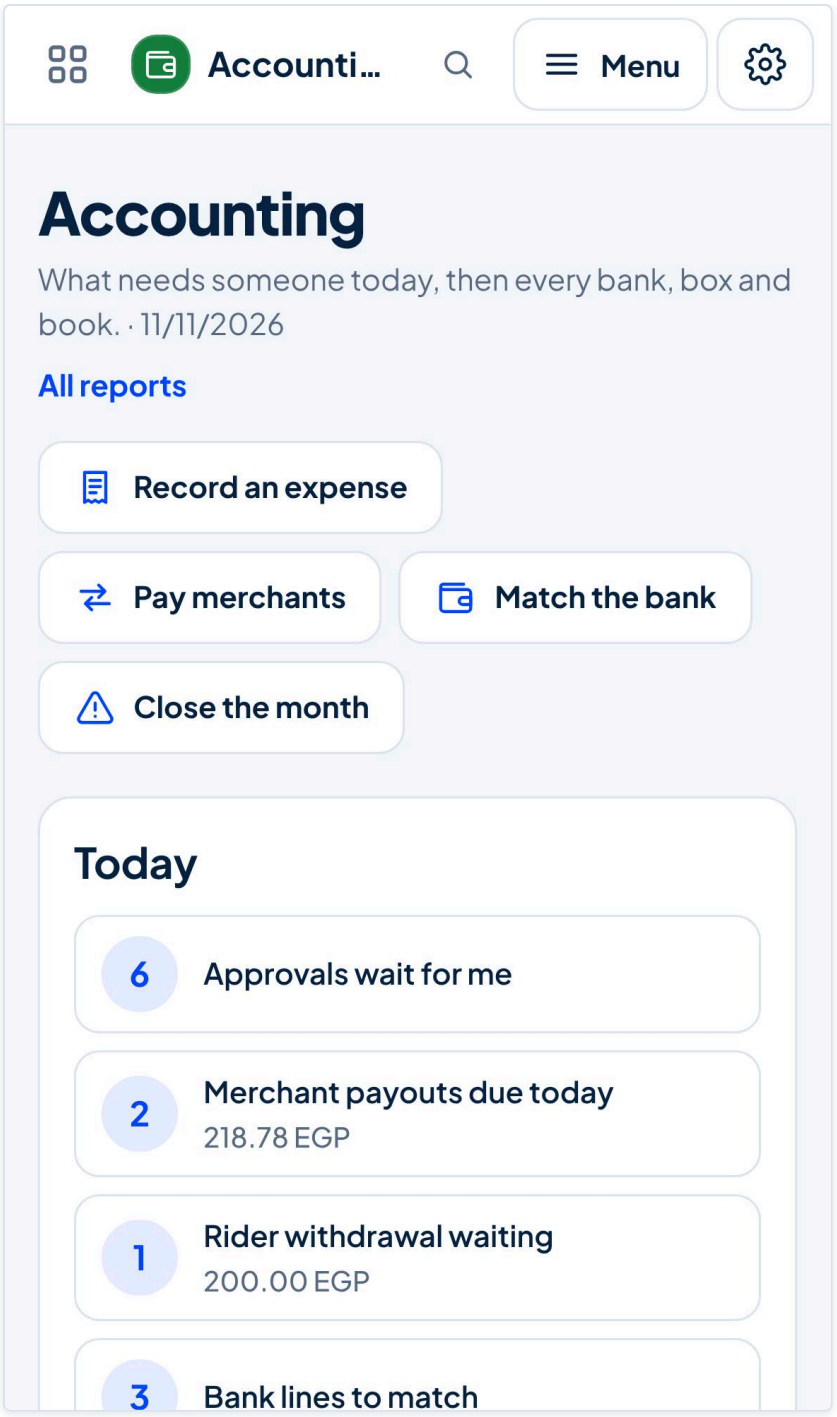
The Accounting menus are organised by job: Dashboard, Money in, Money out, Cash and bank, Reports, Planning, Month end and Setup. `locate(pathname)` finds the page with the longest matching address, so a document such as `/finance/bills/<id>` lights the Bills menu. `canOpenPage` applies the page's need with `canDo` from `packages/shared`, the same function the API uses. A test (`apps/web/src/lib/apps.test.ts`) checks which apps and pages each job role sees.

The account menu (`AccountMenu`) is one Settings button on every signed in screen. It opens Help (the standard operating procedure of the person's role), Notifications, Tickets (team members and riders only), language, theme and Sign out, so these never take a row of a phone screen. It shows the person's photo from My HR when there is one.

Grid cells never grow wider than their column (`.grid > * { min-width: 0 }` in `globals.css`), so a wide table scrolls inside its own box and pages never slide sideways on a phone.



The Accounting top bar on a desktop at `/finance/bills`: the grid button, the Accounting name, the menus (Dashboard, Money in, Money out with its list open, Cash and bank, Reports, Planning, Month end, Setup), the search box and the account menu button; the Approvals count badge visible.



The Accounting dashboard on a phone: the top bar keeps the grid button and the app name, and the menus fold into one Menu button.

12.6 ListView

`ListView` (`apps/web/src/components/odoo/ListView.tsx` , a client component) is the Odoo style list: a search bar with facet chips, a Filters, Group by and Favourites panel, a column chooser, row selection with a bulk action bar, CSV and Excel exports, totals of money and count columns, paging, and cards on phones.

```
<ListView config={config} data={data} toolbar={<NewBillButton />} />
```

Prop	Type	Meaning
`config`	`ListViewConfig`	The list's columns, filters, value filters, groupings, labels, row link, bulk actions, default filters and export name.
`data`	`ListResult`	The rows of the current page (or the groups), totals and counts, fetched by the server page for the current address.
`toolbar`	React node	The page's own buttons next to the search (New bill, an export of lines).

The configuration type (`apps/web/src/lib/list-view.ts`):

```
export interface ListViewConfig {
  listId: string;                // favourites and chosen columns are kept per list id
  endpoint: string;              // GET, the list state as query
  columns: ListColumn[];         // key, label, kind, optional, sort, primary, format
  filters: { key: string; group: string; label: string }[];
  values?: ListValueFilter[];    // a store, a merchant, a first or last day
  groupBys: { key: string; label: string }[];
  valueLabels: Record<string, Record<string, string>>;   // statuses and kinds in words
  rowHref?: string;              // each row opens this page; {id} or any {field} is filled in
  bulkActions?: BulkAction[];
  defaultFilters: string[];
  exportName: string;
  navIdKey?: string; navTrail?: { label: string; href: string }[]; navLabel?: string;
}
```

Column kinds are `text` , `ltr` , `money` , `number` , `date` , `datetime` , `status` and `bool` . A column marked `optional` is hidden until chosen; `primary` is the card title on phones; `format` names a formatter that builds a sentence from the row (an alert as one sentence).

State in the address

The list's whole state lives in the address, so Back works and a view can be shared as a link. The parameters, read and written by `parseListState` and `serializeListState` in `packages/shared/src/list-view.ts` :

Parameter	Meaning	Example
`q`	Typed search, matched across the row's search fields with Arabic letter forms and case folded	`q=nile`
`f`	Predefined filters, comma separated. Always written, even empty, so removing the last filter does not bring the defaults back	`f=open,overdue`
`g`	Group by, one or two levels	`g=supplier,month`
`sort` , `desc`	Sort key and direction	`sort=due_day&desc=1`
`page` , `size`	Page (from 1) and page size (25, 50, 100 or 200; 50 by default)	`page=2&size=100`
`v`	Value filters as `key:value` pairs: a store, a merchant, a first or last Cairo day	`v=store:abc,from:2026-11-01`

Filters in the same group are joined by "or"; filter groups and value filters are joined by "and", as in Odoo. Unknown keys are dropped when the state is read, so an old shared link still opens. Any change of what is

shown goes back to page 1. Other parameters of the address (a tab) are kept.

Search, filters, grouping and favourites

The search bar shows one chip per active facet: the typed search, one chip per filter group (its filters joined by "or"), the value filters and the grouping. Backspace in an empty search box removes the last chip. The Filters panel lists the predefined filters by group and the value filters: pick lists apply at once, days apply with an Apply days button. The Group by panel toggles up to two levels; a grouped list first shows the groups with counts and subtotals, and opening a group fetches its rows with `open=<keys joined by a newline>`.

Favourites save the current view under a name, per person and per list (`user_list_favorites`, routes `GET/POST/DELETE /me/list-favorites`, audited `list.favorite_save`). One favourite can be the default: it opens when the address names no list state. The column chooser saves the visible columns per person (`PUT /me/list-prefs/:list`) and caches them in `localStorage` (`shiply-cols:<listId>`), wrapped in try and catch because storage may be off.

Selection and bulk actions

Ticking rows shows a sticky bulk bar with the list's actions. A bulk action is declarative:

```
export interface BulkAction {
  key: string;
  label: string;
  endpoint: string;               // POSTed with { ids, note?, ...body }
  body?: Record<string, unknown>;
  note?: 'required' | 'optional'; // a reason asked before it runs
  tone?: 'primary' | 'danger';
  choice?: { field: string; label: string; options: { value: string; label: string }[] };
}
```

The endpoint answers `{ results: [{ id, ok, error }] }`. On the API, `eachRecord` (`apps/api/src/common/bulk.ts`) runs the single record action for each id in its own transaction, so each record still goes through every rule (never one's own, the approval limit, scope) and one refusal never stops the others. The bar then shows how many succeeded and why the others were refused, in the reader's language (`listView.errors`).

Exports, totals and paging

Export CSV and Export Excel fetch every row that passes the current state (`all=1`, up to `LIST_MAX_ROWS`, 5,000), keep only the ticked rows when some are ticked, and build the file in the browser with `exportCsv` or `exportXlsx` from `packages/shared/src/export-file.ts`:

- CSV starts with a `UTF-8` byte order mark so Excel shows Arabic, uses Windows line ends, and prefixes cells starting with `=`, `+`, `-` or `@` with a quote (unless they are plain numbers) so a spreadsheet never runs them as formulas;
- Excel is a minimal .xlsx written by hand: one sheet, inline strings and numbers, stored in a zip with its own CRC32, no library; an Arabic export sets the sheet right to left;
- money columns are written as EGP numbers with two decimals, dates as `YYYY-MM-DD`, statuses in the reader's words.

Totals of the summed columns cover every row that passed the filters, not only the page. Paging shows the counts and the page size choice. On phones (below `md`) rows become cards with the primary column as title.

Server side: runList, queryList and FINANCE_LISTS

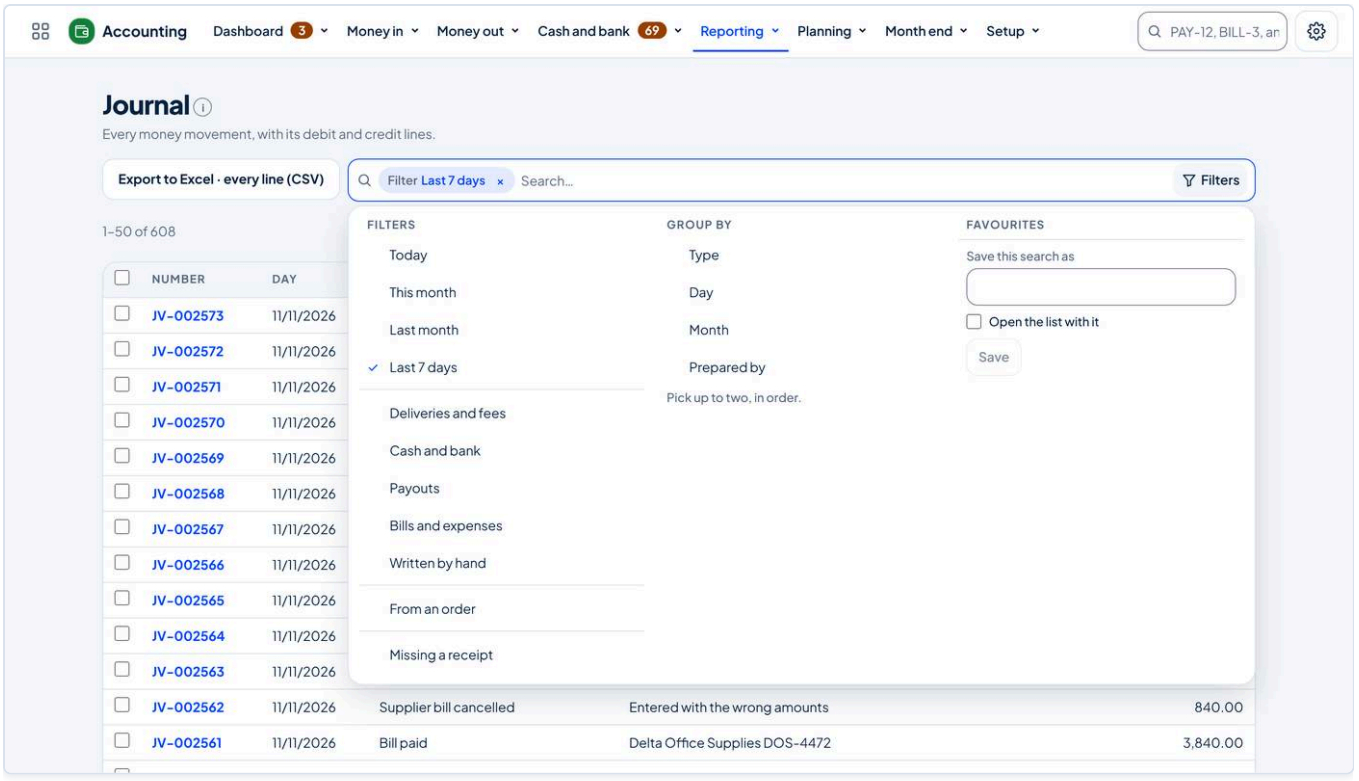
The filter, grouping and sort keys of each Accounting list are declared once in `FINANCE_LISTS` (`packages/shared/src/finance-lists.ts`); the other apps' lists in `APP_LISTS` :

```
bills: {
  filters: { open: 'status', paid: 'status', cancelled: 'status',
             overdue: 'due', due_7_days: 'due', this_month: 'date', last_month: 'date', last_7_days:
'date' },
  groupBys: ['supplier', 'status', 'store', 'account', 'month'],
  sorts: ['number', 'supplier', 'bill_day', 'due_day', 'total', 'left'],
  sums: ['total', 'vat', 'paid', 'left'],
  defaultFilters: ['open'],
},
```

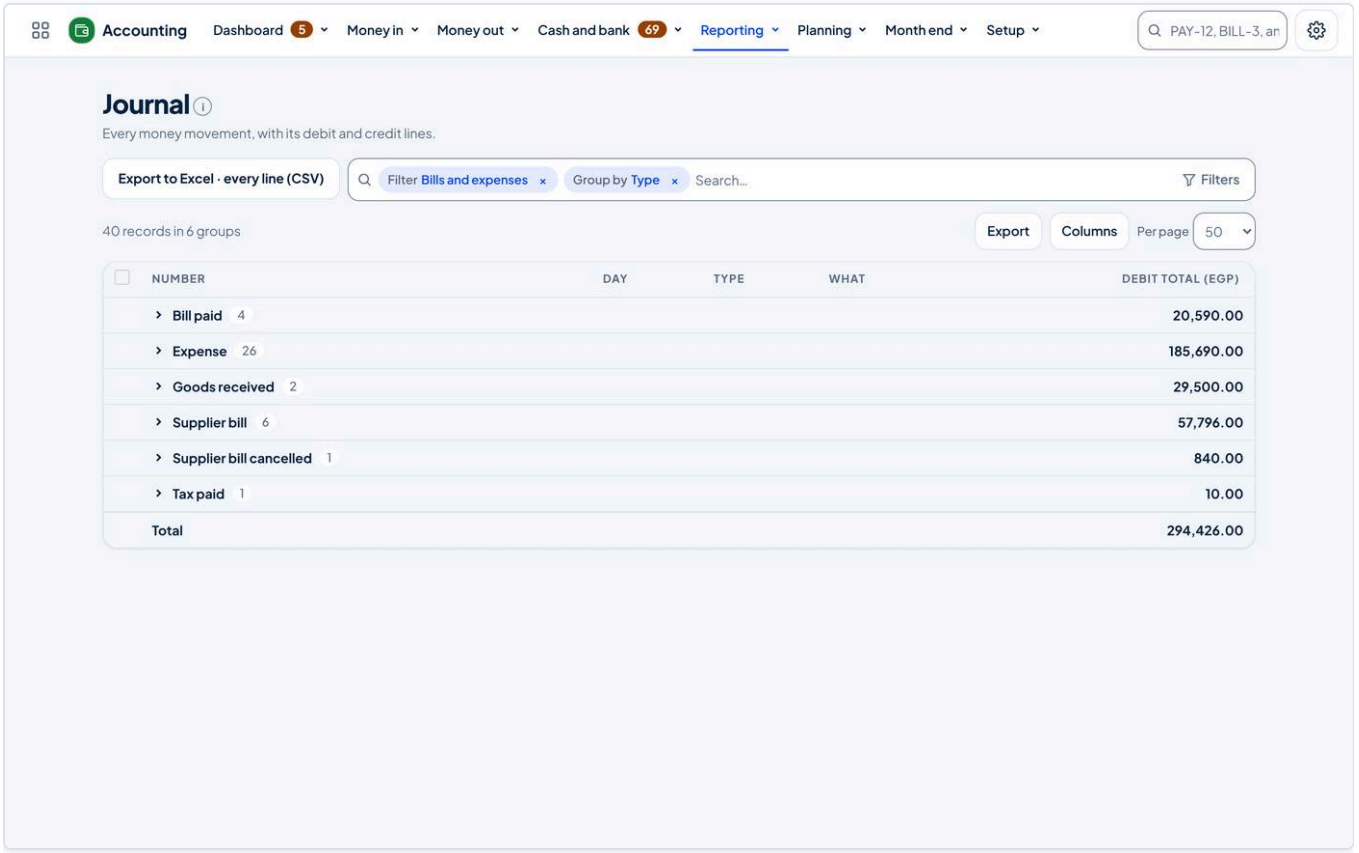
The web builds its menus from these keys (`components/odoo/finance-lists.ts` adds columns and labels). The API implements them in `FINANCE_LIST_DEFS` (`packages/db/src/finance/lists.ts`): for each list a `ListDef` (filter tests, group functions, value filters, search fields, sort functions, sums) and a loader. `queryList` (`packages/db/src/list-query.ts`) reads the state from the query, loads the newest rows inside the person's scope (narrowed in the database by a chosen store, merchant or days before the 5,000 row cap), and `runList` applies search, filters, value filters, grouping with counts and subtotals, totals, sorting and paging. `financeListGaps()` and `appListGaps()` prove in tests that both sides know exactly the same keys. The endpoint is `GET /finance/lists/:id` (accounting view) or `GET /lists/:app/:id` .

A page uses the list through `FinanceListPage` , which fetches the configuration and the data in parallel and renders the title, an intro sentence, the page's own panels and the list:

```
export default async function FinanceJournalPage({ searchParams }) {
  const [params, t] = await Promise.all([searchParams, getTranslations('finance.journal')]);
  return (
    <FinanceListPage id="journal" searchParams={params}
      title={<>{t('title')}<InfoTip term="journal" /></>} intro={t('intro')} />
  );
}
```

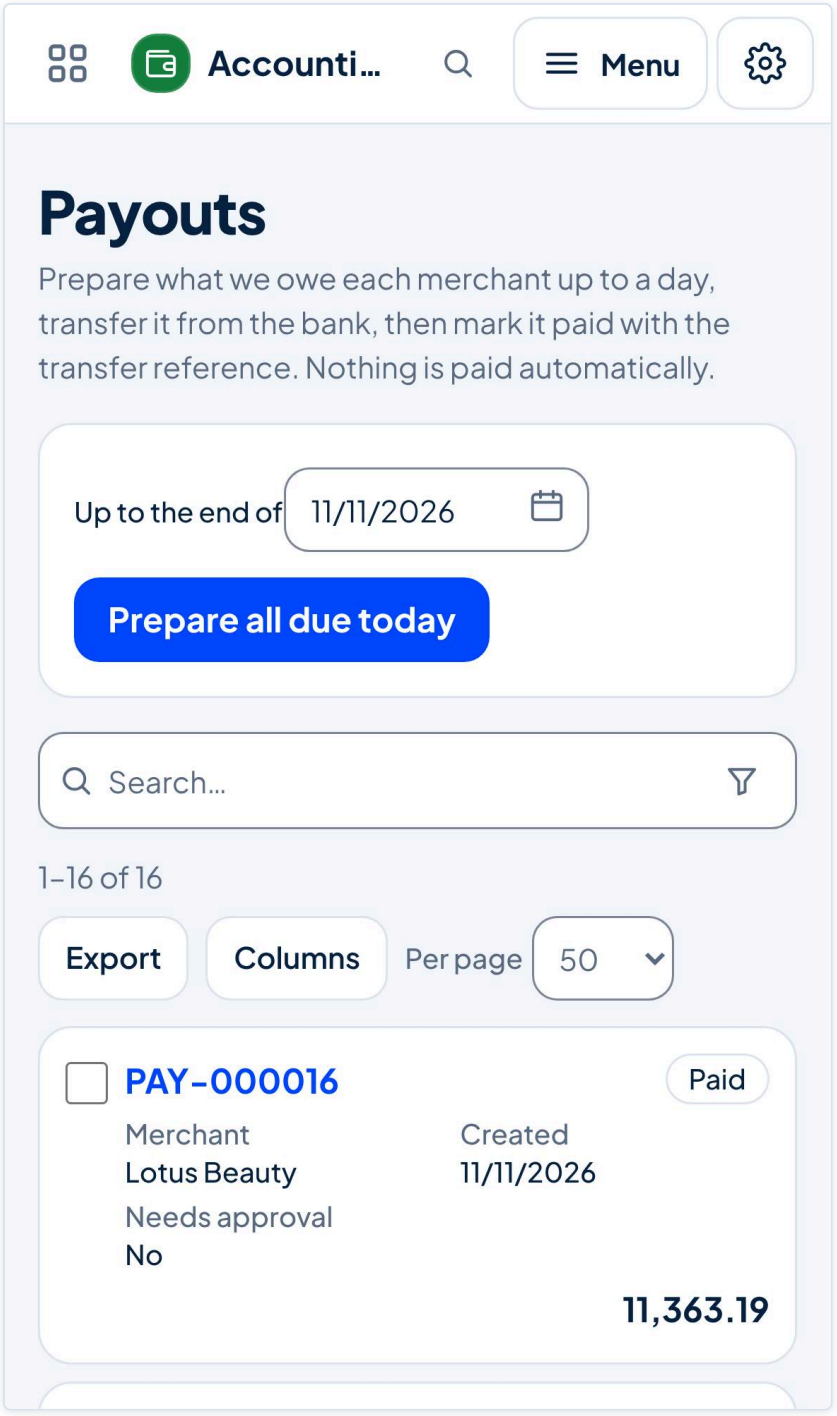


The bills list at `/finance/bills` with the Filters panel open: filters grouped (status, due, date), the store and merchant pick lists, facet chips in the search bar, and the totals row under the money columns.



The journal at `/finance/journal` grouped by kind and then by day: group rows with counts and subtotals, one group opened showing its entries.

The approvals list at `/finance/approvals` with three rows ticked: the sticky bulk bar with Approve and Reject, and after running, the result line saying how many were approved and why one was refused.



The payouts list at `/finance/payouts` on a phone width: rows shown as cards with the payout number as title, status and amount.

12.7 DocumentFrame and DocumentPage

Every record with its own page (a bill, a payout, a journal entry, an order, a merchant, an employee) is shown in one frame, `DocumentFrame` (`components/odoo/DocumentFrame.tsx`, a server component). From top to bottom:

1. Breadcrumbs: the app's home, then the list the person came from, with its filters kept, then the record number.
2. Previous and next (`RecordNav`): when a row is opened from a list view, `ListView` stores the ids of the rows the person saw, the list address and the row link template in `sessionStorage` under `shiply-doc-nav` . The document page reads it and shows "3 / 25" with previous and next links, like Odoo.
3. Header and status bar: the number, the title and the `StatusBar` of the record's workflow.
4. Actions: the document's action buttons (approve, mark paid, cancel), components of the page.
5. Smart buttons: counts of related records linking to them (a payout's merchant, entries, receipts, emails; a bill's supplier, payments, entries, attachments). Three to a row on phones.
6. Content and, beside it from the `lg` breakpoint (or `xl` for wide pages), the chatter; below the content on narrower screens.

Prop	Meaning
`root` , `list`	Breadcrumb links: the app's home and the document's list.
`id` , `number` , `numberLtr` , `title`	The record and how it is named.
`status` , `statusLabel` , `flow`	The status and the workflow name; without a flow the status shows as a pill.
`actions` , `smart` , `attachments`	Action buttons, smart buttons (`DocSmartButton[]`), photos with what each shows.
`entityType` , `canWrite`	The audit entity type the chatter and history hang off, and whether the person may write notes.
`notes`	Notes from another thread (an order shows its team notes instead of record notes).
`asideFrom` , `header` , `historyPath`	Where the chatter moves beside the content, a custom header, another history endpoint.

For accounting documents, `DocumentPage` (`components/odoo/DocumentPage.tsx`) builds the whole page from a `DocDescriptor` that the API returns at `GET /finance/documents/:type/:id` (built by `accountingDocument` in `packages/db/src/finance/documents.ts`): fields with a kind (`money` , `date` , `datetime` , `status` , `ltr` , `percent` , `multiline` , `text`), optional lines with columns, smart buttons, attachments, the actions the reader may take now and the values those actions need. The page passes its own action components in `actions` and any custom content (an invoice as printed, a cheque register) as children.

```
<DocumentPage doc={doc} actions={<PayBill bill={bill} accounts={accounts} />} />
```

Status bar and workflows

`apps/web/src/lib/workflows.ts` holds the path and side states of every kind of document:

```
payout: { path: ['draft', 'paid'], side: ['cancelled'] },
bill: { path: ['open', 'paid'], side: ['cancelled'] },
rider_withdrawal: { path: ['requested', 'approved', 'paid'], side: ['rejected', 'cancelled'] },
finance_request: { path: ['pending', 'approved'], side: ['rejected', 'cancelled'] },
petty_cash: { path: ['pending', 'posted'], side: ['rejected', 'cancelled', 'reversed'] },
cheque_received: { path: ['awaiting_approval', 'received', 'deposited', 'cleared'], side: ['bounced',
'cancelled'] },
purchase_order: { path: ['requested', 'approved', 'ordered', 'received', 'billed', 'closed'], side:
['rejected', 'cancelled'] },
```

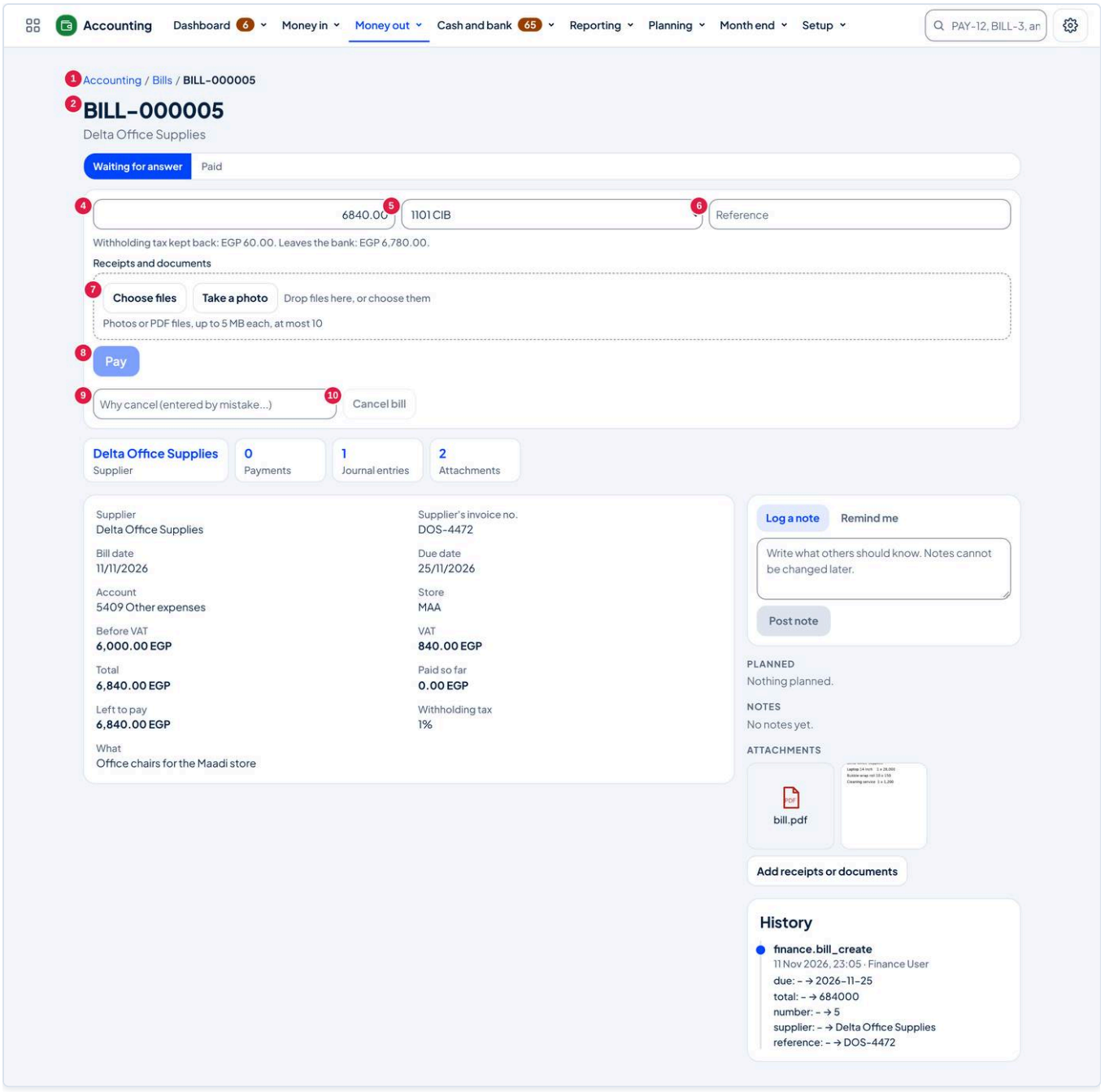
`StatusBar` draws the path with done steps ticked, the current step lit in the primary color and later steps muted. A side state replaces the end of the path in red. On phones only the step before, the current step and the next one show.

Chatter

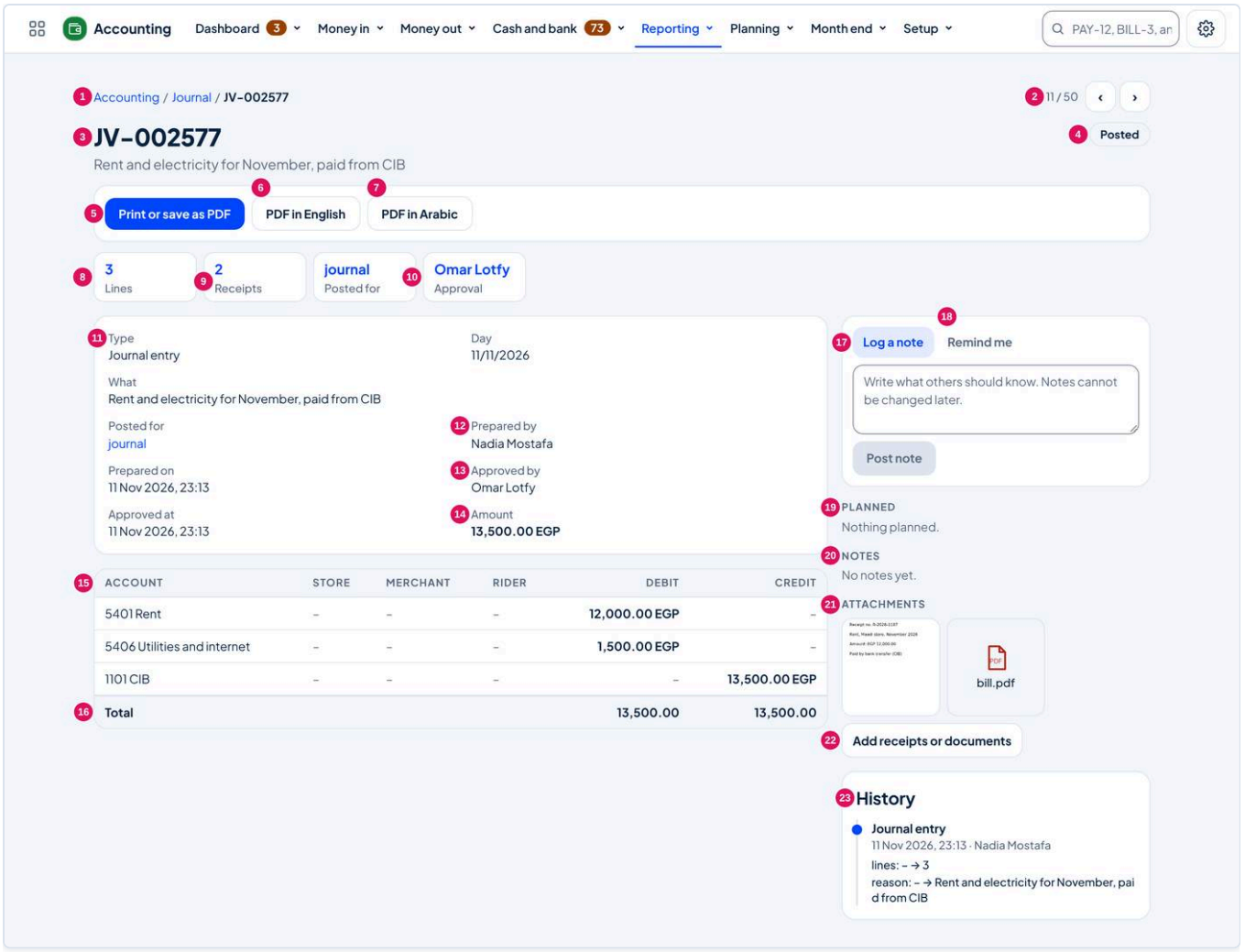
The chatter sits beside the content. It has four parts:

- Notes: append only (trigger on `record_notes`), audited `record.note`. Routes `GET/POST /records/:entityType/:entityId/notes`.
- Activities ("Remind me"): a Cairo day, a note and a person who may open the record (`GET /records/:entityType/:entityId/people` lists them). Done by that person or whoever planned it (`POST /records/activities/:id/done`). A web push `activity_due` is sent once on its day, and the person's activities are listed on the Accounting dashboard (`GET /me/activities`).
- Attachments: receipt photos and other photos of the record, each labelled with what it shows.
- History: the audit rows of the record (`RecordHistory`, from `GET /history/:entityType/:entityId`).

Reading needs view on the record's app (`RECORD_APPS`) and the record inside the person's stores and merchants; writing needs use. Customer records also need the customer data switch.



A bill document page at `/finance/bills/<id>`: breadcrumbs Accounting / Bills / BILL number, the "3 / 25" previous and next control, the status bar (Open, Paid), the Pay and Cancel actions, smart buttons (supplier, payments, entries, attachments), the fields, and the chatter on the right with a note, a reminder and the history.



A journal entry at `/finance/journal/<id>` : the JV number, kind, day and memo fields, the lines table with account, debit and credit, smart buttons to the source document and the order, and the Print button.

The journal entry page and its voucher

`apps/web/src/app/finance/journal/[id]/page.tsx` renders every ledger entry with `DocumentPage` (fed by `GET /finance/documents/ledger_entry/:id`) and adds three actions: `PrintButton` and two links to `/api/finance/journal/<id>/voucher.pdf?lang=en|ar&kind=` , the kind label translated in each language by the page. The printed voucher is a server component (`Voucher`) rendered in the same page with `hidden print:block` , while the screen part is `print:hidden` : company name and tax number, "Journal voucher" and the number, date (day first), type, description, posted for, the lines (account in the page's language, store, merchant or rider, debit, credit), totals, "Amounts in EGP", and the Prepared by, Checked by and Approved by boxes. The API's PDF (`voucher-pdf.ts`) prints the same with `PdfWriter` . Every list row opens its record from anywhere on the row, and every `JV-` number shown in Accounting links to `/finance/journal/<id>` .

Attachments inside every money form

`AttachmentField` (`components/finance/AttachmentField.tsx`) is the receipts and documents field of every finance creation form: several files, drag and drop, Choose files and a Take a photo button (`capture="environment"`). Photos (JPEG, PNG, WebP) are shrunk in the browser first (`shrinkPhoto`), PDFs are sent as they are up to 5 MB, at most 10 files. Each file is uploaded at once to `POST /api/finance/files?name=` and shown as a tile (busy, done or error); removing a tile deletes the upload (`DELETE /finance/files/:id`). The hook `useAttachments()` gives the form `ids` (sent as `attachmentIds`), `busy` (the save waits while uploads run)

and `reset()` after a save. On document pages `components/odoo/Attachments.tsx` shows the files: `AttachmentTile` (a thumbnail for a photo, a PDF tile with its name, opening in a new tab), `AttachmentsSmart` (the receipts smart button: no file jumps to the attachments section, one file opens it directly, several open the viewer "All attachments") and `AddAttachments` (adds files to a saved record). The server side is described in chapter 12, "Attachments chosen inside the forms"; the screens in Screens: Month end, Setup and common parts, section "Common part: the attachment field and viewer".

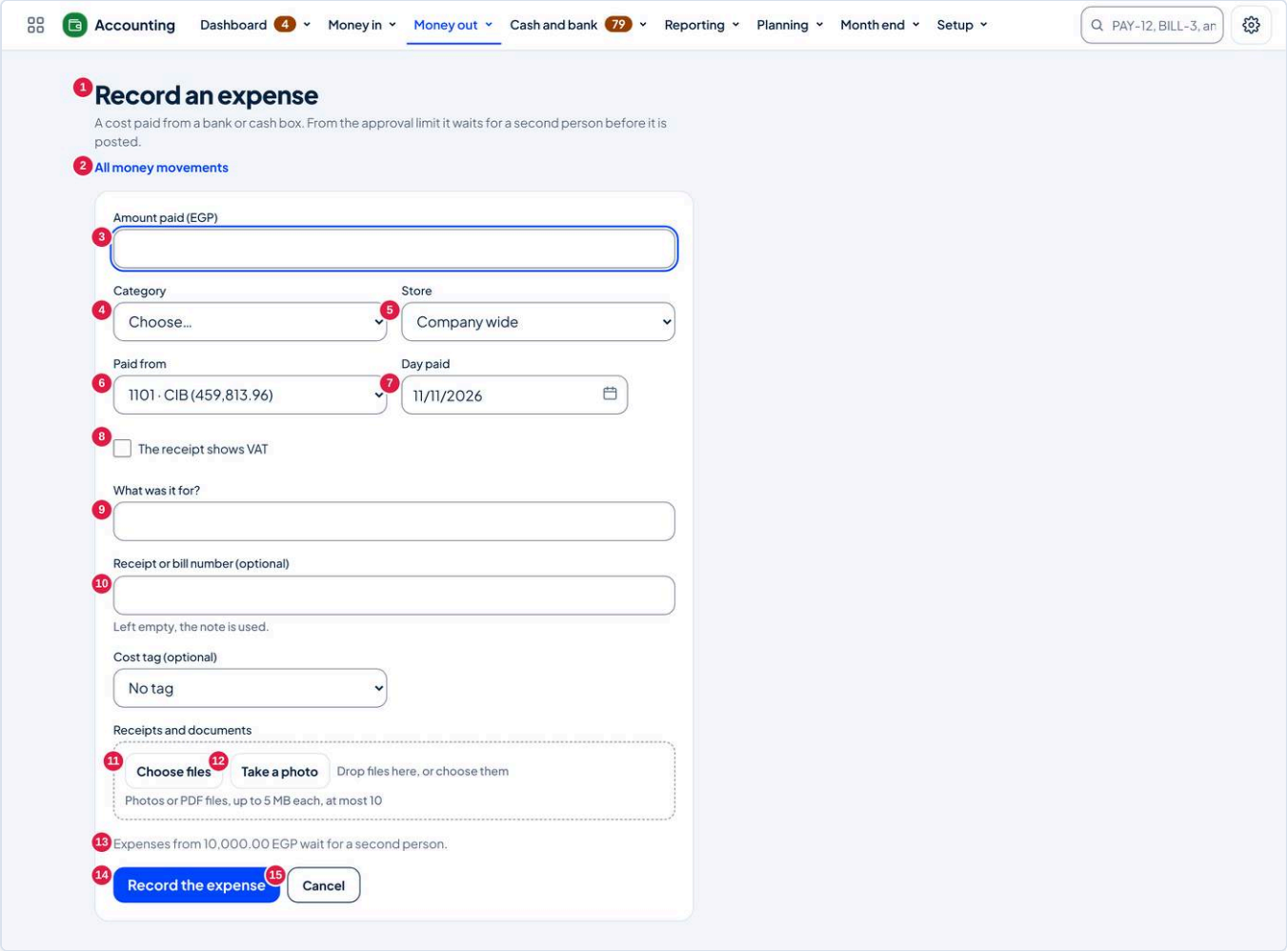
12.8 Forms: FormFooter and unsaved changes

Creation and edit forms end with `FormFooter` (`components/workflow/FormFooter.tsx`): Save, Cancel and an "Unsaved changes" marker.

Prop	Meaning
`dirty`	The form has changes; shows the marker and turns on the leave guard.
`busy`	Saving; disables both buttons and turns the guard off.
`saveLabel` , `disabled`	Another label for Save; Save disabled.
`cancelHref` , `onCancel`	Where Cancel goes, or what it runs; without both it goes back one page.
`onSave`	Without it, Save submits the surrounding form.

Cancel asks before throwing changes away. The hook `useUnsavedChanges(dirty)` asks before leaving: the browser's own question on closing or reloading the tab (`beforeunload`), and a confirm on any link click inside the app (a click listener in the capture phase that cancels the navigation when the person says no).

Money forms create one idempotency key per action with `newActionKey()` (`apps/web/src/lib/ids.ts`), send it with the body, and make a new key only when the person starts another action, so a double tap posts once. Forms that move money take a receipt photo through `ReceiptField` , which uploads the photo at once (`POST /photos?kind=receipt`) and sends its id with the form.



The Record an expense form at `/finance/expenses/new` with values typed: amount, category, store, paid from (with its balance shown), day, VAT, note, receipt photo, the line saying whether it will post now or wait for approval, and the footer with Save, Cancel and "Unsaved changes".

12.9 DateInput, Amount and money parsing

DateInput

`DateInput` (`components/DateInput.tsx`) is a text box that always reads day first (`dd/mm/yyyy`), as in Egypt, whatever language the browser runs in (the browser's own date box follows the device and may show month first). The form still receives `YYYY-MM-DD` under `name` , and a calendar button opens the device's own picker. Props: `name` , `value` or `defaultValue` , `onChange` (receives `{ target: { value } }` like an input), `max` , `className` . An unreadable date marks the box invalid. The conversions are `isoToDayFirst` and `dayFirstToIso` in `lib/day-first.ts` , which refuse days that do not exist.

Amount and formatting

```
/** An EGP amount, kept left to right in Arabic, red when negative. */
export function Amount({ piastres, strong = false }: { piastres: number; strong?: boolean }) {
  const value = piastres === 0 ? 0 : piastres;   // -0 shows as 0.00, never "-0.00"
  return (
    <span className={`ltr-iso ${strong ? 'font-bold' : ''} ${value < 0 ? 'text-danger' : ''}`}>
      {formatEgp(value)}
    </span>
  );
}
```

`formatEgp` (`lib/format.ts`) shows piastres as EGP with two decimals and Latin digits. Other formatters show Cairo times and dates (`formatCairoTime` , `formatCairoDateTime` , `formatCutoffDay`).

Parsing typed amounts without floats

```
/** EGP typed by a person ("1,249.50", "-20") to integer piastres without going through a float. */
export function parseEgp(text: string, { allowNegative = false } = {}): number | null {
  const clean = text.trim().replace(/,/g, '');
  const match = /^(-)?(\d+)(?:\.(\d{1,2}))?$/.exec(clean);
  if (!match || (match[1] && !allowNegative)) return null;
  const value = Number(match[2]) * 100 + Number((match[3] ?? '').padEnd(2, '0'));
  return match[1] ? 0 - value : value;
}
```

The integer part and the decimals are read separately, so "0.29" is 29 piastres exactly, where a float would give 28.999. Contract price imports on the API use the same idea (`egpTextToPiastres` in `packages/shared/src/contracts.ts`).

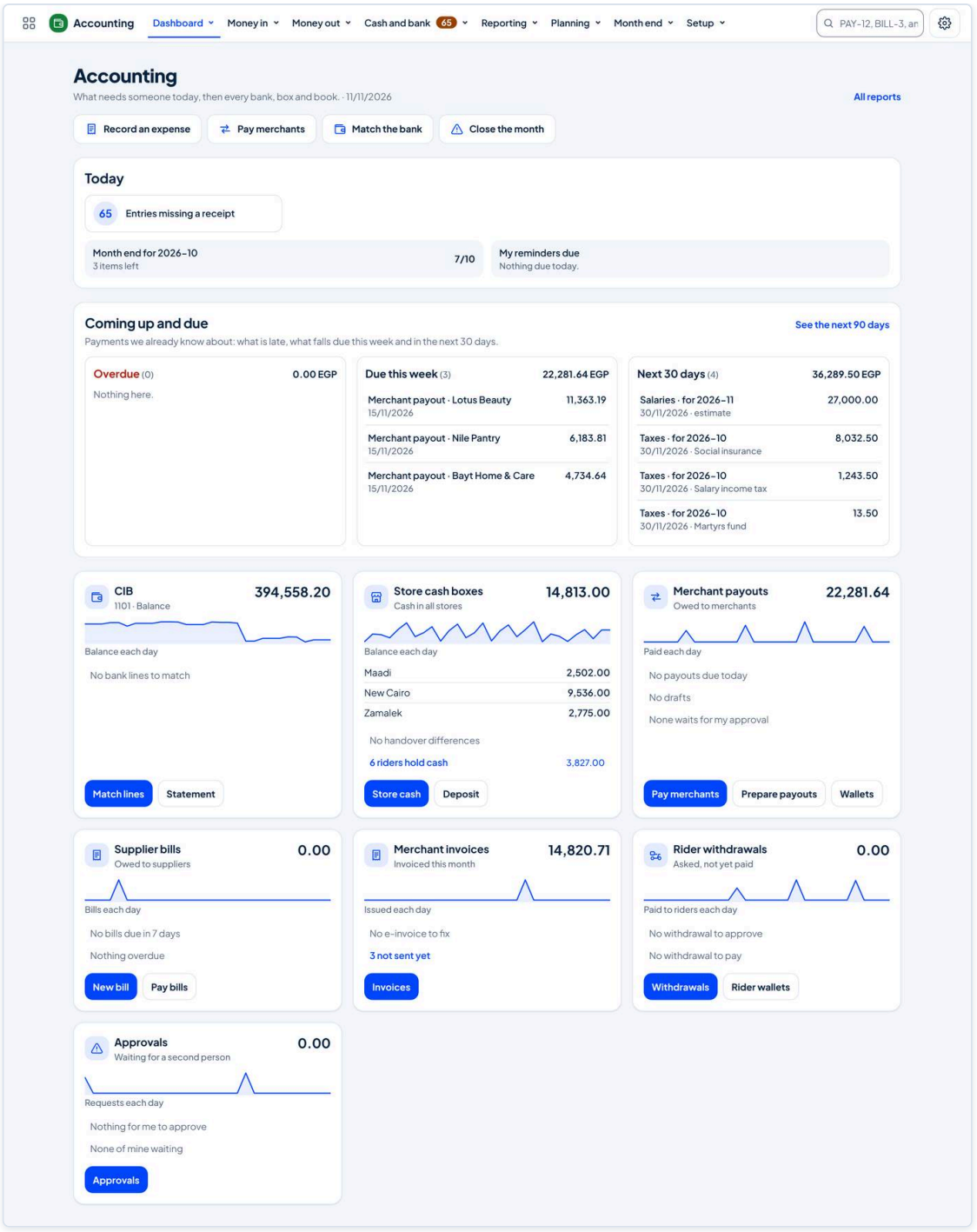
12.10 StatCard, WalletCard and charts

`StatCard` (`components/StatCard.tsx`) is the dashboard number card after the brand guide's product screen: a soft rounded card, the icon on top in the status color (or a short color mark without an icon), the large tabular number, then the label. Props: `label` , `value` (already formatted), `hint` , `tone` (`primary` , `new` , `in-progress` , `heading-to-customer` , `delivered` , `needs-action` , `returning`), `icon` , and `alert` (the number takes the needs action color).

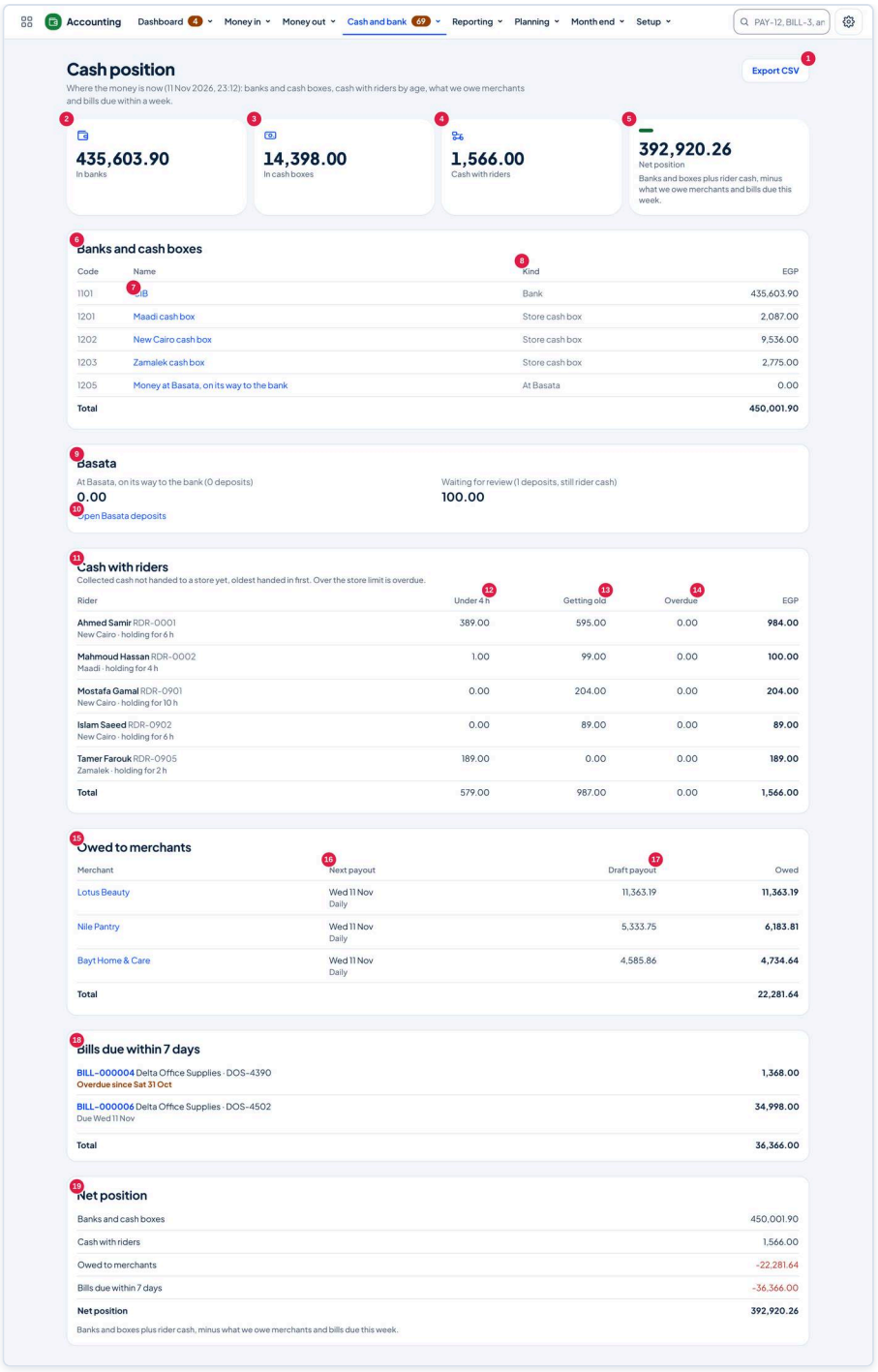
`WalletCard` (`components/WalletCard.tsx`) is the money balance card after the guide's cash on delivery balance card: a Navy ground (Shiply Blue in the dark theme), a small spaced label, the amount large in Cream, the note in Lime and the S arrow mark faint in a corner. Props: `label` , `value` , `hint` , `icon` , `large` . It is used for wallets and balances only.

Charts are drawn in HTML and SVG without a chart library, one series in the primary color:

- `Sparkline` (`components/charts/Sparkline.tsx`): a small SVG line of the last days for dashboard cards. Props: `values` , `label` (the summary screen readers get), `width` , `height` . Time runs left to right in Arabic too.
- `ColumnChart` : one series of columns over time, in HTML so text stays crisp in both languages; each column shows its exact value on hover or focus, and a hidden table gives screen readers the same numbers.
- `TargetBars` : progress against targets (sales).



The Accounting dashboard at `/finance`: the Today strip (approvals waiting for me, payouts due today, rider withdrawals, handovers with a difference, bank lines to match, bills due in 7 days, entries missing a receipt), the quick actions, and the journal cards (bank, cash boxes, store boxes, payouts, bills, invoices, withdrawals, approvals) each with its balance and 30 day sparkline.



The cash position at `/finance/cash-position` showing several StatCards with icons and tones, next to a merchant wallet balance shown as a WalletCard on `/merchant/wallet`.

The dashboard (`apps/web/src/app/finance/page.tsx`) also shows the Coming up and due card: it asks `GET /finance/payments-due?days=30` and renders `PaymentsDueGroups` (`components/finance/PaymentsDue.tsx`): Overdue, Due this week and Next 30 days, each with its count and total, every item with its due day, label, counterparty, amount and a link to the screen that pays it, and a link "See the next 90 days". The page `/finance/due` asks 90 days (adding the Later group), offers one chip per kind (`?kind=`, sent as `kinds`) and a folded "How is this counted?" text. The rules are in chapter 15, "Coming up and due"; the screen in Screens: Dashboard and Money in, section "Coming up and due".

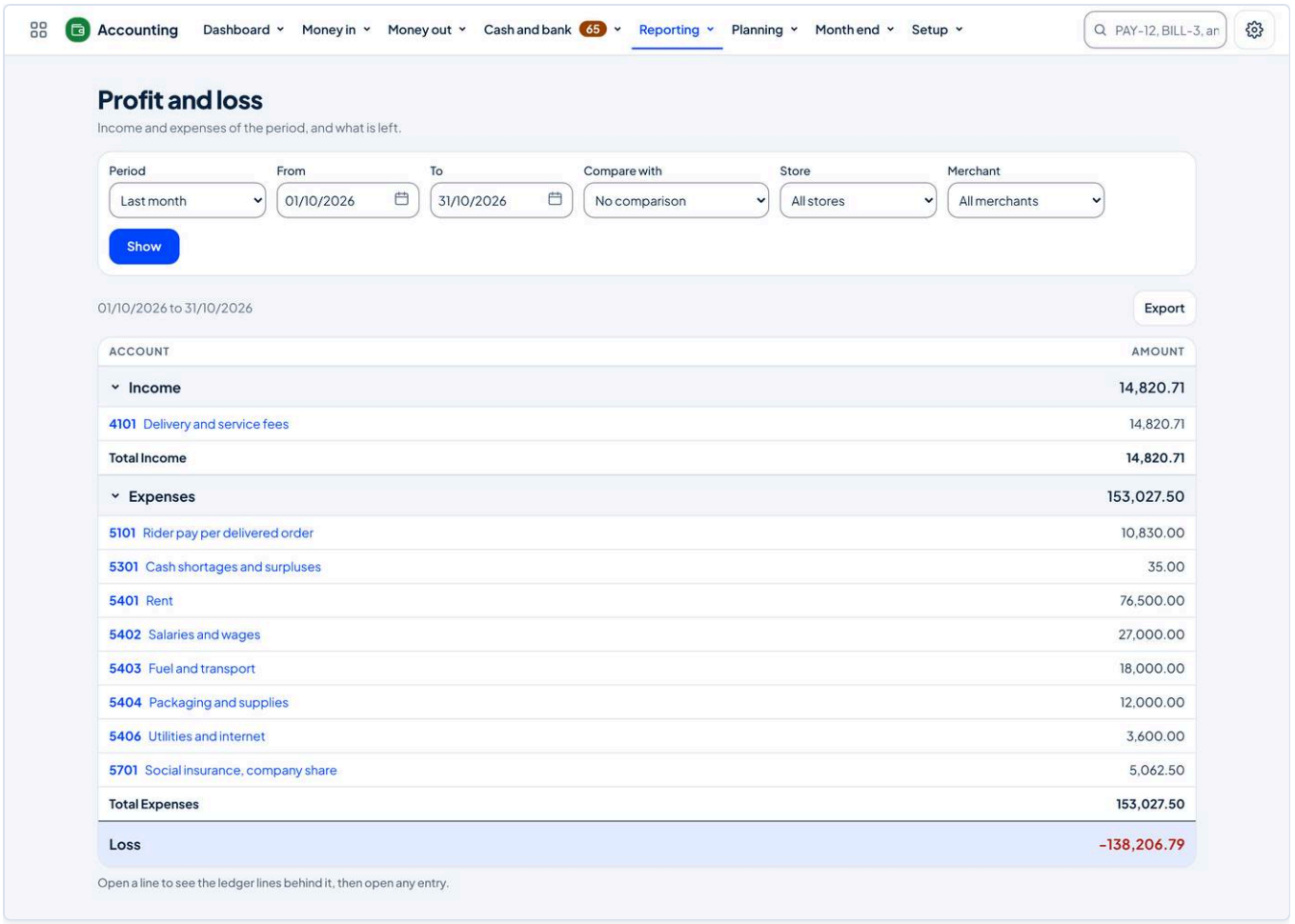
12.11 DrillReportPage

Profit and loss, the balance sheet and the trial balance are one component, `DrillReportPage` (`components/finance/reports/DrillReportPage.tsx`, a server component):

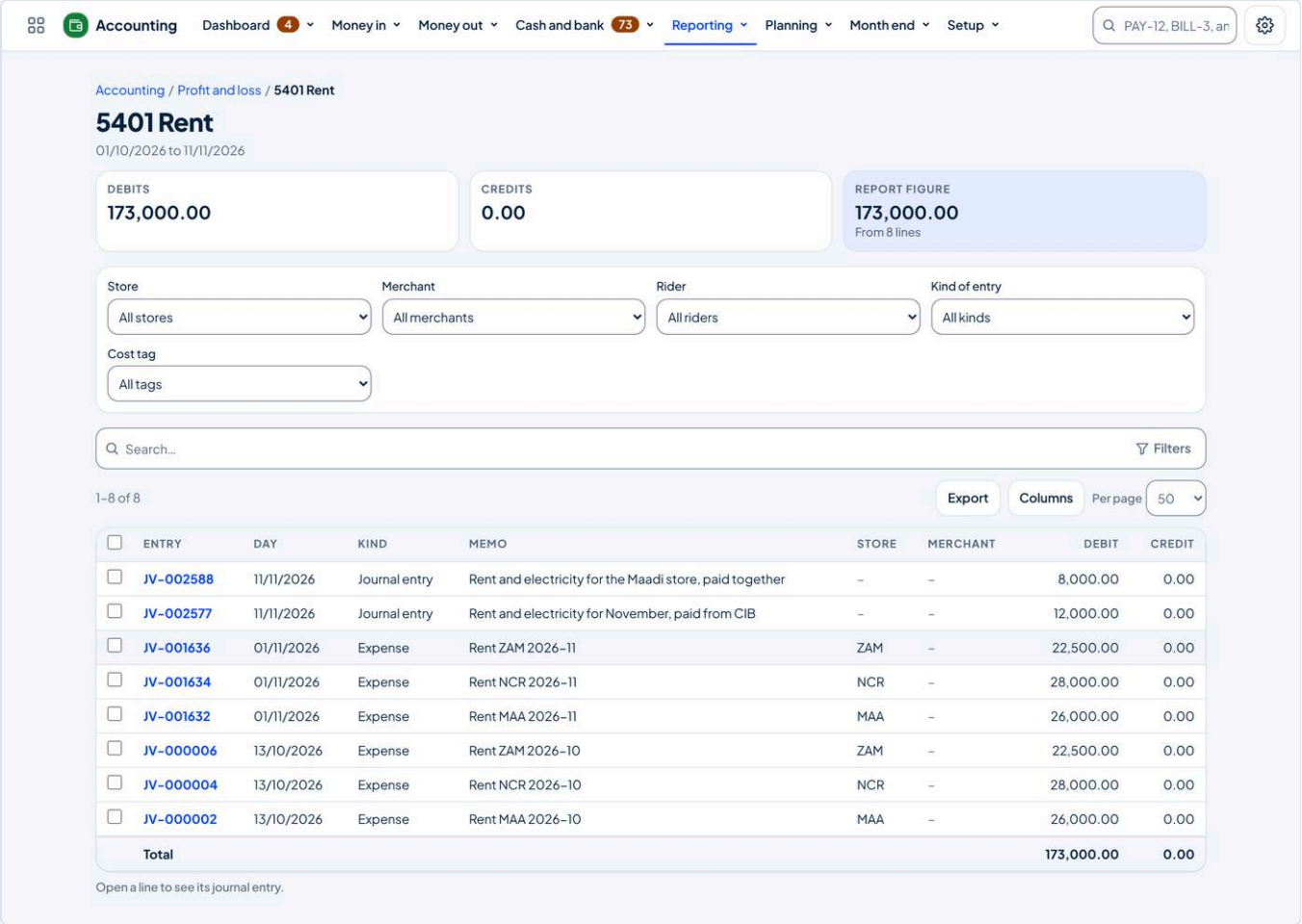
```
<DrillReportPage report="profit_and_loss" searchParams={params}
  title={t('title')} intro={t('intro')} term="balanceSheet" before={(data) => ...} />
```

Prop	Meaning
`report`	`profit_and_loss`, `balance_sheet` or `trial_balance`.
`searchParams`	The address, holding `from`, `to`, `compare` (`none`, `previous`, `last_year`), `store` and `merchant`.
`title`, `intro`, `term`	Heading, a plain sentence, and an `InfoTip` term.
`before`	A status line above the report (the balance sheet says whether it balances).

It reads the state with `readReportState` (`lib/report-view.ts`), fetches `GET /finance/reports/:report` (built by `drillReport` in `packages/db/src/finance/drill.ts`) and the store and merchant choices, and renders `ReportFilterBar` (period presets this month, last month, this quarter, this year, last year or chosen Cairo days, the comparison, store and merchant) and `DrillReportView`. Sections fold; each line shows the comparison amount, the change and the percent, and links to `/finance/reports/lines`, a list view of the ledger lines behind it with the same filters (`GET /finance/ledger-lines`), whose rows open the journal entry. Breadcrumbs lead back to the report with its filters. A report filtered by store or merchant counts only lines tagged with them, so it may not balance, and says so. Exports are CSV, Excel and a PDF in English or Arabic made on the API.



Profit and loss at `/finance/profit-and-loss?compare=previous`: the filter bar with this month and the comparison chosen, income and expense sections with each account, the compared amount, the change and the percent, and the export buttons.

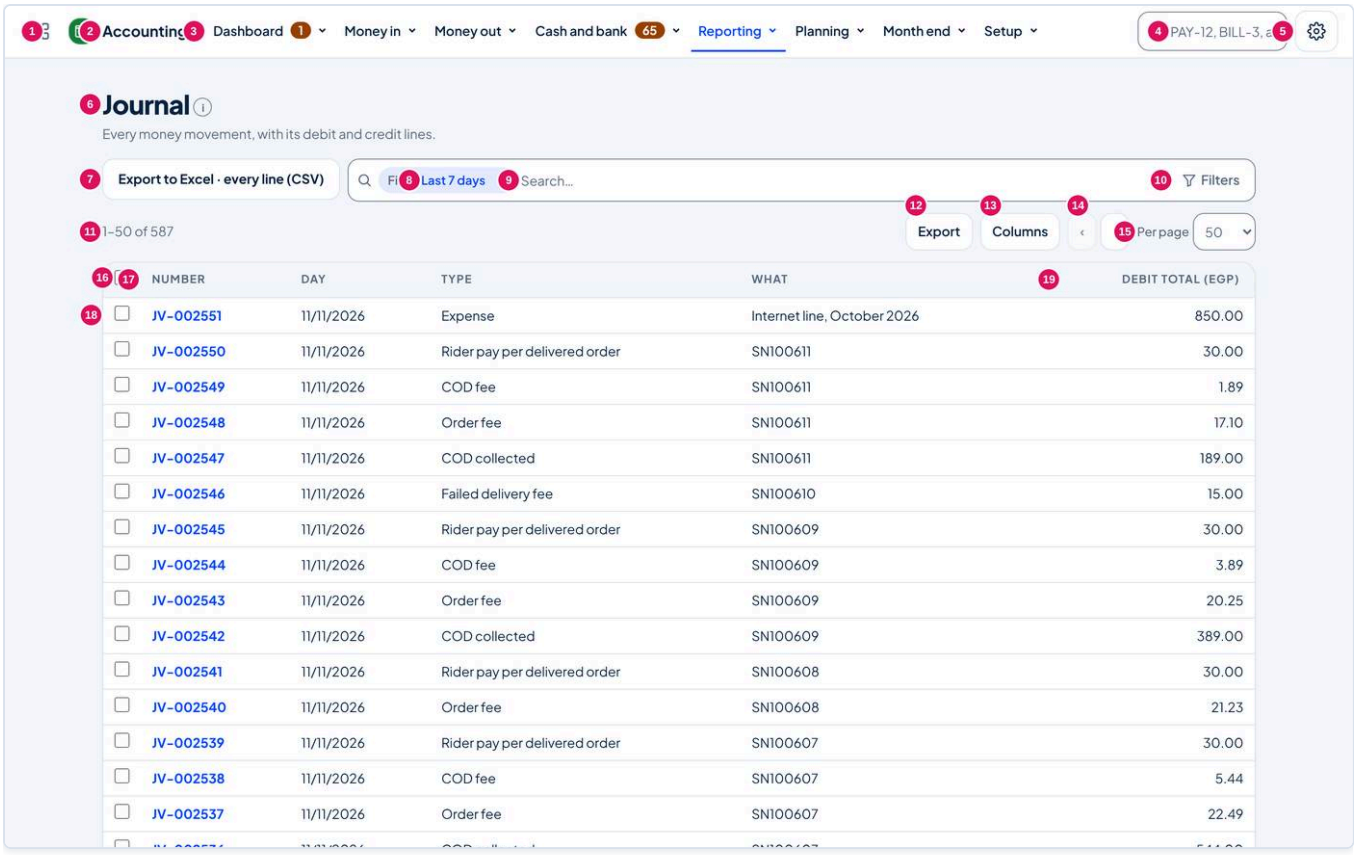


The ledger lines behind one account at `/finance/reports/lines`, opened from a profit and loss line: breadcrumbs back to the report, the lines list with entry number, day, store, merchant and amount, and the total.

12.12 InfoTip and the Accounting search

`InfoTip` explains an accountant's word in plain language: a small "i" next to a title that opens a tooltip on hover, focus or tap. Its prop `term` is one of `journal`, `trialBalance`, `receivables`, `accrual`, `withholding`, `balanceSheet`, `cashFlow`; the words live under `infoTips` in the messages.

`AccountingSearch` is the one search box in the Accounting top bar (`GET /finance/search`). It understands document numbers typed loosely: `parseDocNumber` reads `PAY-000123`, `pay-123`, `pay 123` or `PAY123` as payout 123, for every prefix (`JV`, `PAY`, `CH`, `BILL`, `INV`, `CN`, `DN`, `PO`, `CHQ`, `PC`, `RW`, `BS`, `EC`, `BD`). It also finds order numbers and merchant and supplier names, inside the person's scope.



The journal list: the title carries an InfoTip that explains the accountant word, and the search box with its chips sits under it.

12.13 Printing and PDF generation

Two ways produce paper:

- Browser print for invoices, payslips, statements and comparative statements: `PrintButton` calls `window.print()`, the person saves as PDF from the print window. Menus, buttons and the chatter carry `print:hidden`, the main area drops its width limit, and `@media print` in `globals.css` prints black on white.
- Server PDF with pdfkit for documents that must look the same everywhere: the drill reports (`GET /finance/reports/:report/export.pdf?lang=en|ar`, `report-pdf.ts`), the aging reports (`aging-pdf.ts`) and the merchant contract.

pdfkit shapes Arabic letters through fontkit, but it lays a line out as one left to right run. `PdfWriter` in `apps/api/src/common/pdf.ts` therefore lays words out itself: each word is measured and drawn on its own, in visual order. Words with Arabic letters read right to left inside their run, numbers and Latin words left to right, and an Arabic page places the runs from the right. Brackets are mirrored inside Arabic runs, and lines wrap by words. The fonts are the brand fonts (`apps/api/assets/fonts`).

```
/** A line's words in the order they are drawn from left to right. */
export function visualWords(text: string, rtl: boolean): { word: string; arabic: boolean }[] {
  const words = text.split(/\s+/).filter(Boolean);
  const runs: { arabic: boolean; words: string[] }[] = [];
  for (const word of words) {
    const arabic = LETTER_OR_DIGIT.test(word) ? ARABIC.test(word) : (runs[runs.length - 1]?.arabic ??
rtl);
    const last = runs[runs.length - 1];
    if (last && last.arabic === arabic) last.words.push(word);
    else runs.push({ arabic, words: [word] });
  }
  const ordered = rtl ? [...runs].reverse() : runs;
  return ordered.flatMap((r) => (r.arabic ? [...r.words].reverse() : r.words).map((word) => ({ word,
arabic: r.arabic })));
}
```

الأرباح والخسائر

من 2026-10-01 إلى 2026-11-11

مقارنة بالفترة من 2026-07-01 إلى 2026-08-11

الإيرادات

الحساب	المبلغ	المقارنة	التغيير	%
4101 إيرادات رسوم التوصيل والخدمات	21,739.56	0.00	21,739.56	–
الإجمالي الإيرادات	**21,739.56**	**0.00**	**21,739.56**	**–**

المصروفات

الحساب	المبلغ	المقارنة	التغيير	%
5101 أجر المناديب عن كل طلب مُسلَّم	17,040.00	0.00	17,040.00	–
5201 تعويضات التلفيات	75.00	0.00	75.00	–
5301 عجز وزيادة النقدية	45.00	0.00	45.00	–
5401 إيجار	153,000.00	0.00	153,000.00	–
5402 مرتبات وأجور	27,000.00	0.00	27,000.00	–
5403 وقود ومواصلات	24,000.00	0.00	24,000.00	–
5404 تغليف ومستلزمات	12,000.00	0.00	12,000.00	–
5406 مرافق وإنترنت	7,200.00	0.00	7,200.00	–
5701 حصة الشركة في التأمينات	5,062.50	0.00	5,062.50	–
الإجمالي المصروفات	**245,422.50**	**0.00**	**245,422.50**	**–**
الربح (الخسارة)	**-223,682.94**	**0.00**	**-223,682.94**	**–**

شيبلي ناو · الأرباح والخسائر · صفحة 1 من 1

A drill down report (profit and loss) downloaded as a PDF in Arabic: pages laid out right to left by the API.

12.14 Push notifications

The app is an installable web app with web push for every role. The parts:

- Browser: `PushRegistrar` (in the root layout, production builds with push keys only) registers `/sw.js` and, for a signed in person who already allowed notifications, sends the device's subscription and language to `POST /me/push/subscriptions` again when it changed. The `/notifications` page (`PushSettings`) asks for permission and lets the person switch each type on or off. On an iPhone it needs Add to Home Screen.
- Server: devices are stored in `push_subscriptions` (deleted when the push service answers 404 or 410), choices in `notification_preferences` (on by default). `dispatchPush` (`packages/db/src/push.ts`) runs every 10 seconds from the API jobs, reads the last 15 minutes of events, and notifies only people who may

open the linked page, in the device's language. Each event key is claimed once in `push_sent`, so overlaps send nothing twice. Sending sits behind the `PushSender` boundary (`HttpPush` with the VAPID keys; `FakePush` in tests); without keys nothing is sent.

Types that concern accounting: `finance_approval` (a request or withdrawal waiting), `cash_difference`, `rider_cash_overdue`, `rider_wallet` (approved, paid or rejected withdrawal), `merchant_payout`, `activity_due` (a chatter reminder), `payslip_ready`, `expense_claim`, `basata_deposit`, `rider_frozen`. The texts in both languages live in `packages/shared/src/push.ts`, and `pushTypesFor` decides which types a person can receive from what they may open.

The type `payment_due` (for people with Accounting use) tells of a known payment `finance.due_reminder_days` (default 3) days before its due day (text `payment_due_soon`: "Due on {day}: {kind}") and on the day itself (`payment_due_today`), linking to the item's screen, or to `/finance/due` when several merchant payouts of one day go out as one notification. The source is `paymentDueEvents` in `packages/db/src/push.ts`, keyed `payment_due:<kind>:<id>:<day>` (chapter 15).

13 Screens: Dashboard and Money in

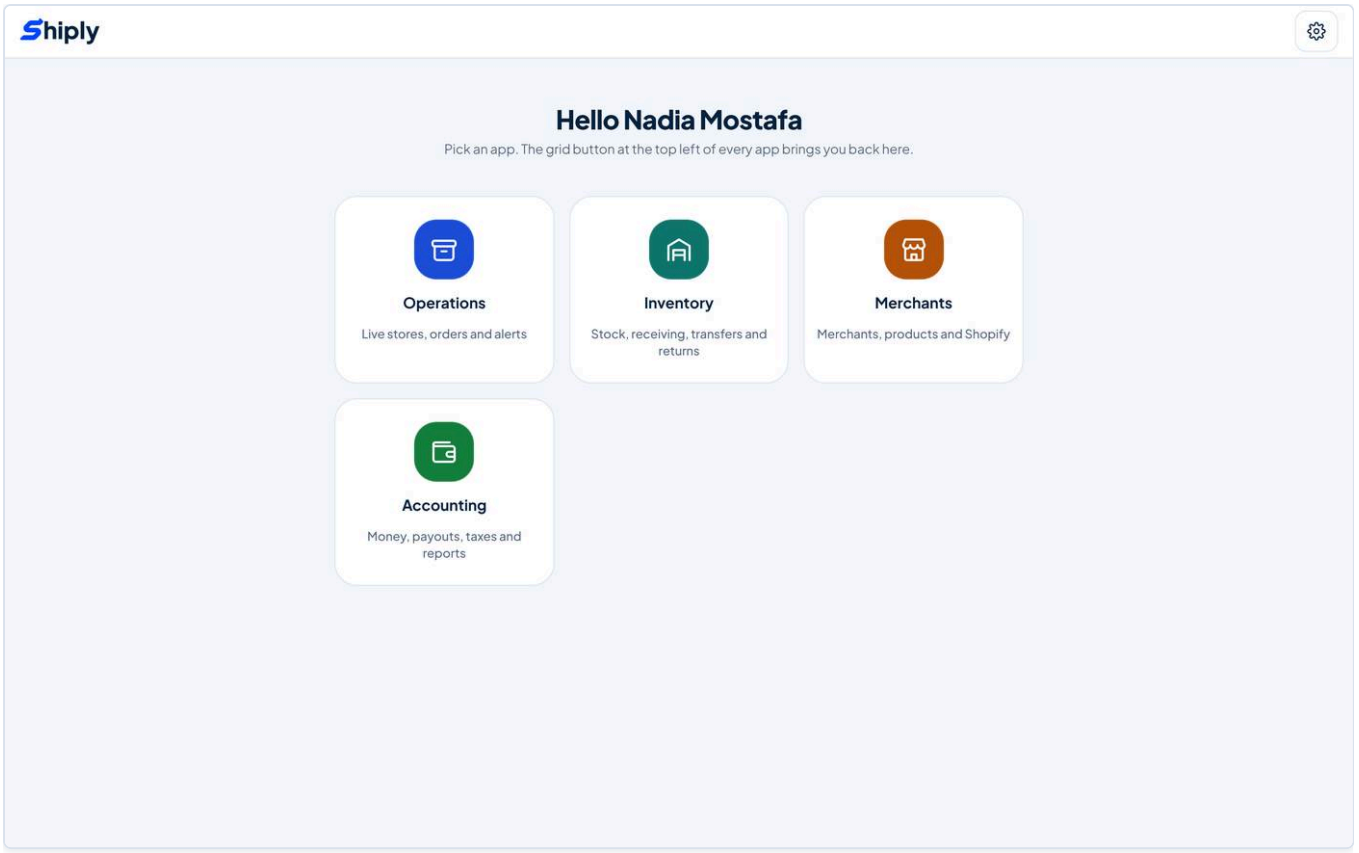
This chapter walks through the Accounting app exactly as a finance user sees it, from the app tile to the last screen of the Money in menu, and then shows the merchant's side of the same payout. Every screenshot was taken on the local demo copy on 11/11/2026 with two finance users: the preparer (accounting use) and a second finance user (accounting manage) who approves and confirms. Every action shown was really performed, so the results (entries, numbers, statuses) are the system's own answers.

> Access words used below. View: may open the screen and read. Use: may prepare, upload, mark paid, register and cancel. Manage: may also approve what waits for a second person (payouts above the limit, Approvals requests, bulk approve). Every route of the Accounting API is guarded by `@Access('accounting')`; reading needs view, writing needs use, and the routes listed as manage carry `@Access('accounting', 'manage')` (`requests/:id/decide`, `requests/bulk-decide`, `payouts/:id/approve`, `payouts/bulk-approve`, payout checks setup). On top of the level, the second person rule is checked by the server: nobody approves what they asked for, and the person who confirmed the statement and slip of a payout cannot be the one who marks it paid.

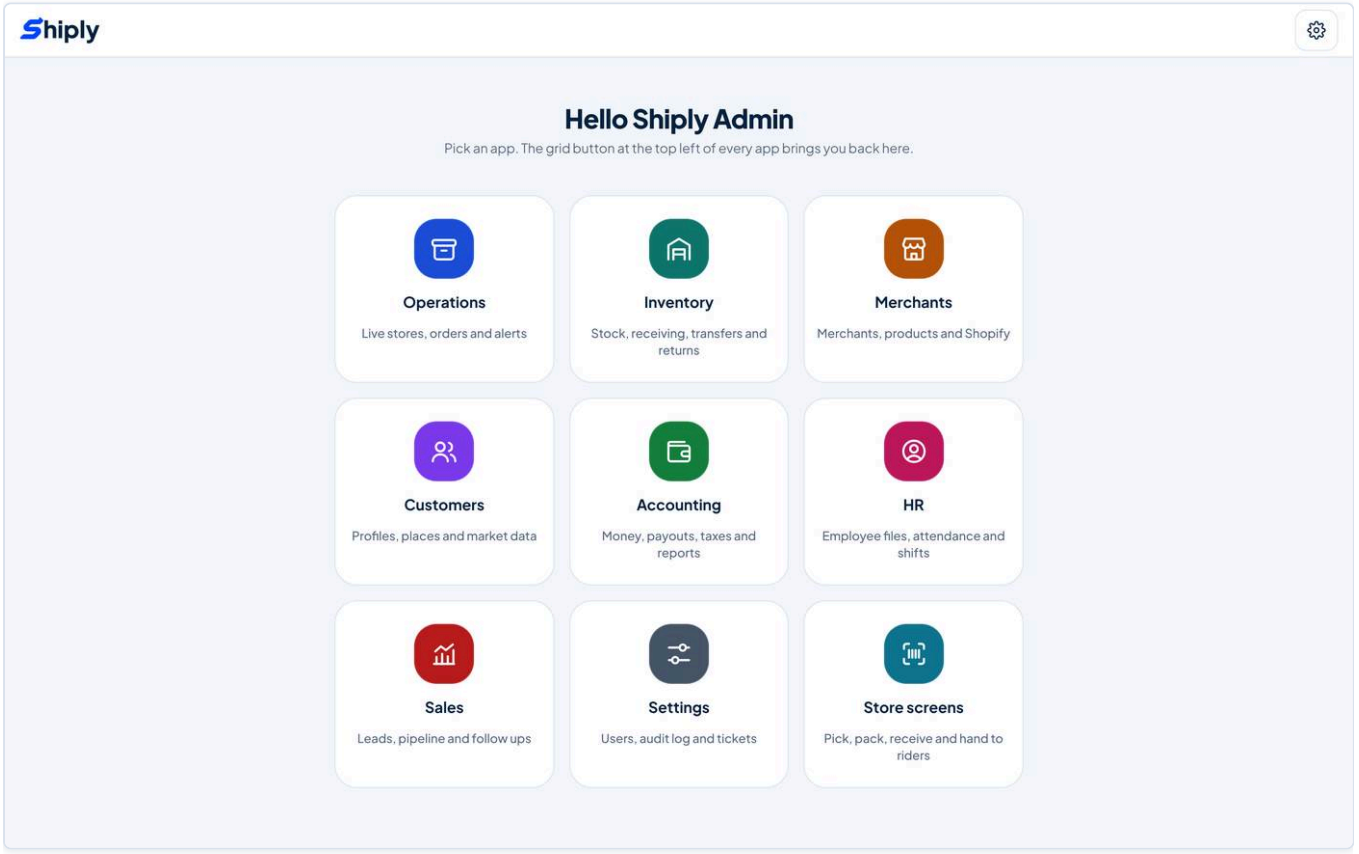
> VAT shows 0.00 on every fee, invoice and note in these screenshots because the demo database holds `fee.vat_bp` = 0. With the owner's 14% the same screens show the VAT column filled.

13.1 Entering Accounting: the app home

Address: `/home`. Who: every team member with an account; a finance user lands here after sign in. Each tile is shown only when the person's access profile allows that app (the pages and their needed level are defined once in `apps/web/src/lib/apps.ts`). The demo finance user has view on Operations, Inventory and Merchants and manage on Accounting, so four tiles show; the admin sees every app.



The app home of the finance user: a greeting with the person's name and one tile per app the access profile allows. Accounting is the green wallet tile. Address `/home`.

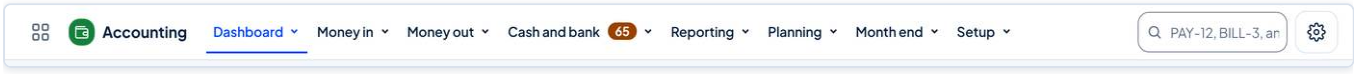


The same home for the admin: every app tile is shown. Address `/home`.

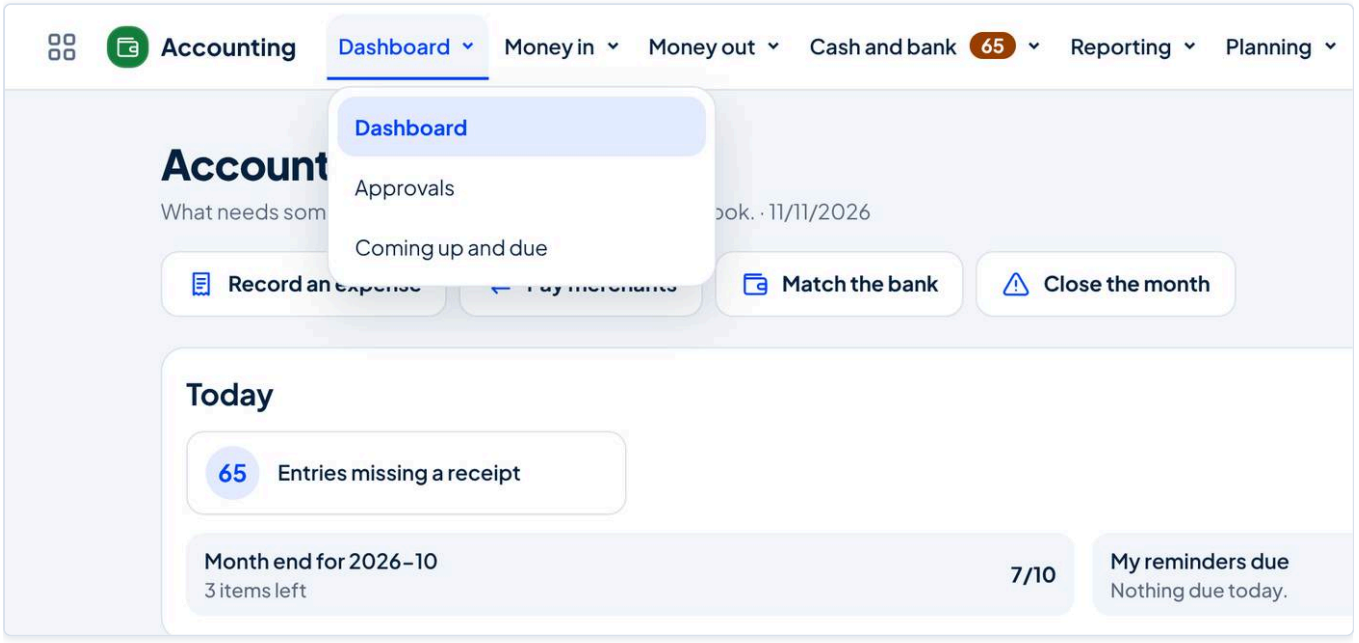
Control	What it does	Rule / limits	Result
Greeting	Shows "Hello" and the person's name	Name from the user account	None
App tile (icon, name, one line description)	Opens the app's first page	Shown only if the profile has at least view on that app	Accounting opens `/finance`, the dashboard
Settings button (bottom of the tile page and every app top bar)	Opens the account menu	Same for every signed in person	See "The top bar" below

13.2 The top bar of Accounting

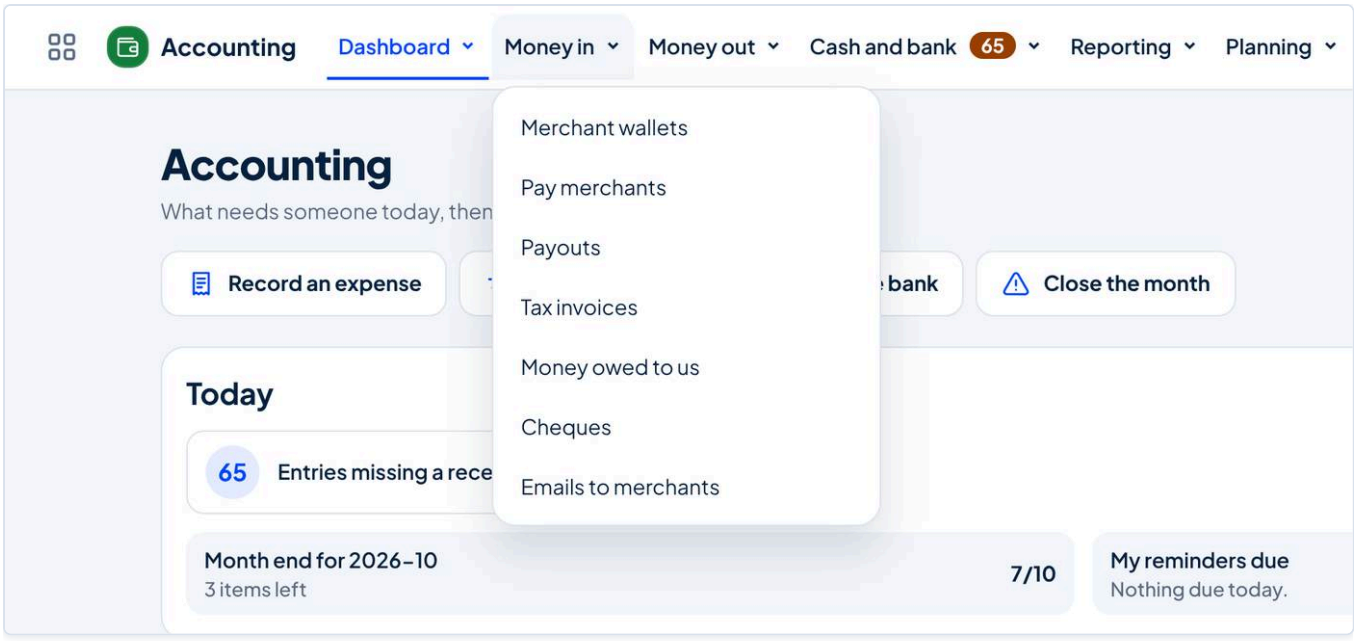
Address: on every page under `/finance`. Who: anyone with accounting view. The bar is `AppShell` with the Accounting search and badges.



The Accounting top bar: grid button, app name, the eight menus with their badges, the search box and the Settings button.



The Dashboard menu open: Dashboard, Approvals and Coming up and due. The badge on the menu counts what waits in it for this person.



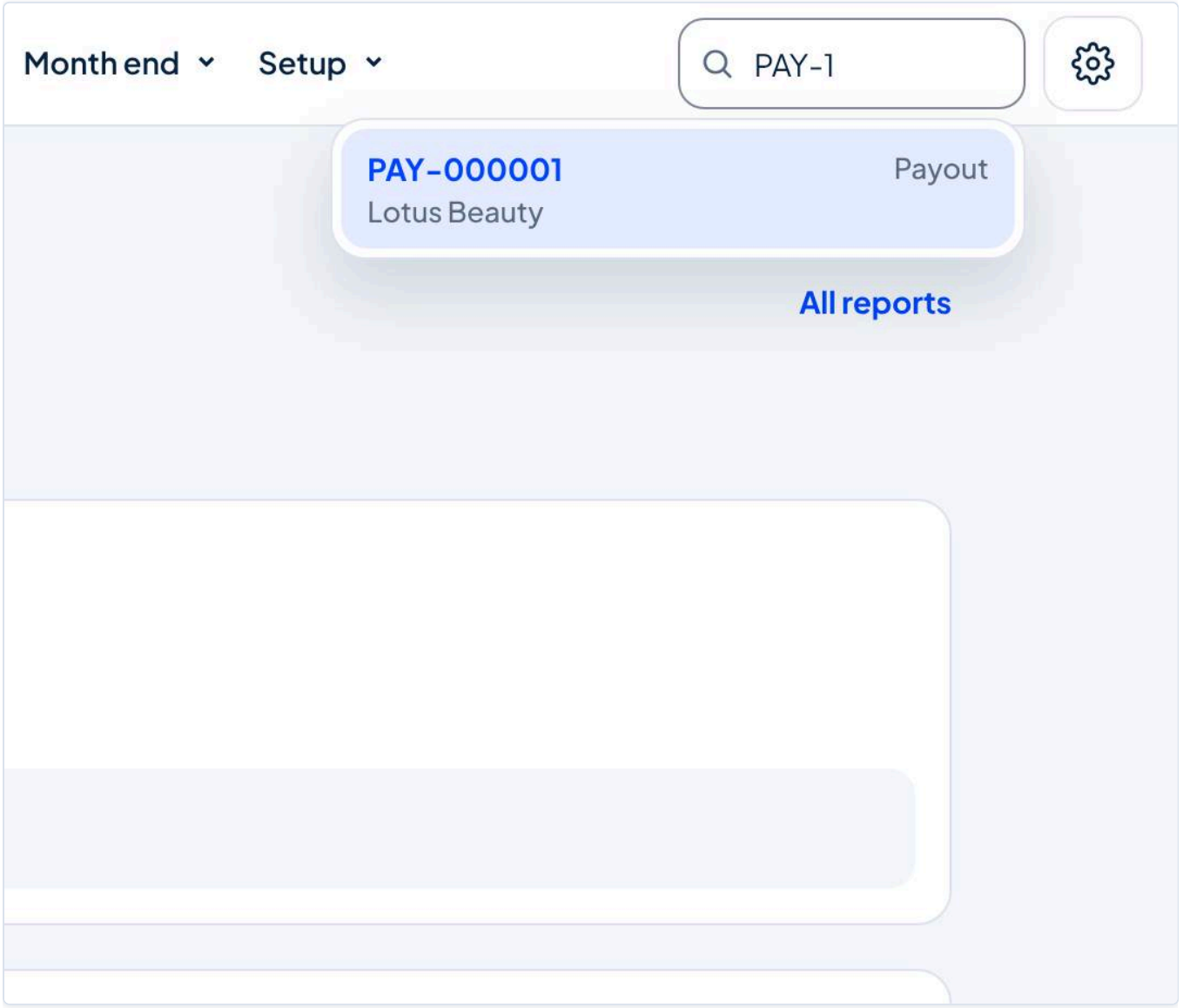
The Money in menu open: Merchant wallets, Pay merchants, Payouts, Tax invoices, Money owed to us, Cheques, Emails to merchants.

Control	What it does	Rule / limits	Result
Grid button (four squares, label "All apps")	Goes back to the app tiles	Always shown	Opens `/home`
Green wallet icon and "Accounting"	Opens the dashboard	Always shown	Opens `/finance`
Dashboard menu	Dashboard, Approvals, Coming up and due	Badge (brown pill, white number) counts approvals waiting for me	The underline in Shiply Blue marks the menu of the current page
Money in menu	The seven pages of this chapter	Each item is shown only if the profile allows it	Opens the page
Money out, Cash and bank, Reporting, Planning, Month end, Setup	Menus of the other chapters	Cash and bank shows a badge with the entries missing a receipt	Opens a list of pages
Search box (placeholder `PAY-12, BILL-3, an order or a merchant`)	Finds a document, order or merchant	Starts after two characters, waits 200 ms after typing, asks `GET /finance/search`; arrows move, Enter opens the first hit, Escape closes; on phones it opens from a magnifier button	A list of hits with the number in blue and the kind on the right; "Nothing found." when empty
Settings button (gear)	Opens the account menu	Same menu on every signed in screen	Help, Notifications, Tickets, My HR, language (English or Arabic), theme (System, Light, Dark), Sign out

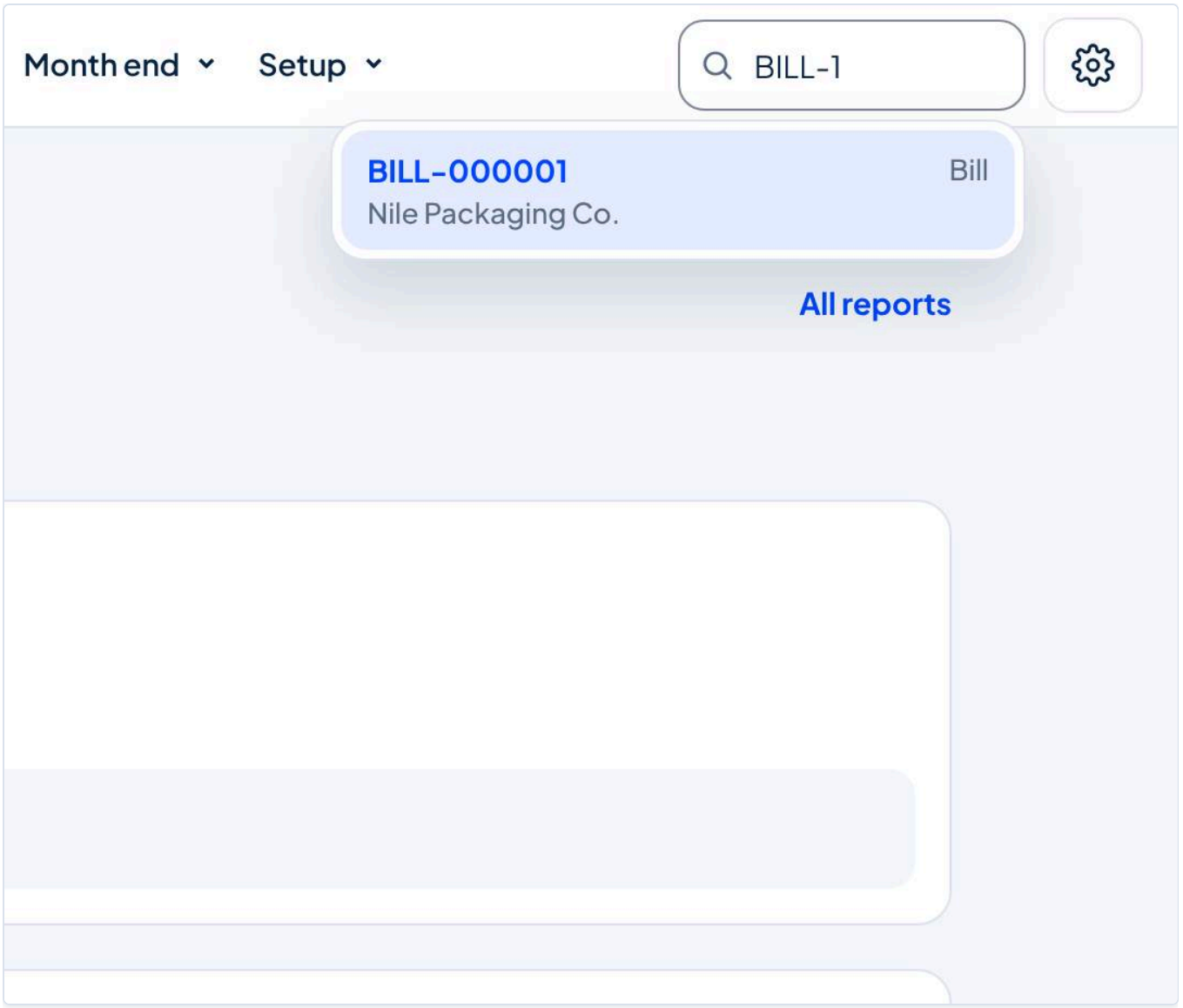
Search examples

Numbers are read in loose forms by `parseDocNumber`: `PAY-1`, `pay 1` and `PAY-000001` all find the same payout. The search covers JV, PAY, CH, BILL, INV, CN, DN, PO, CHQ, PC, RW, BS, EC, BD numbers, order numbers and

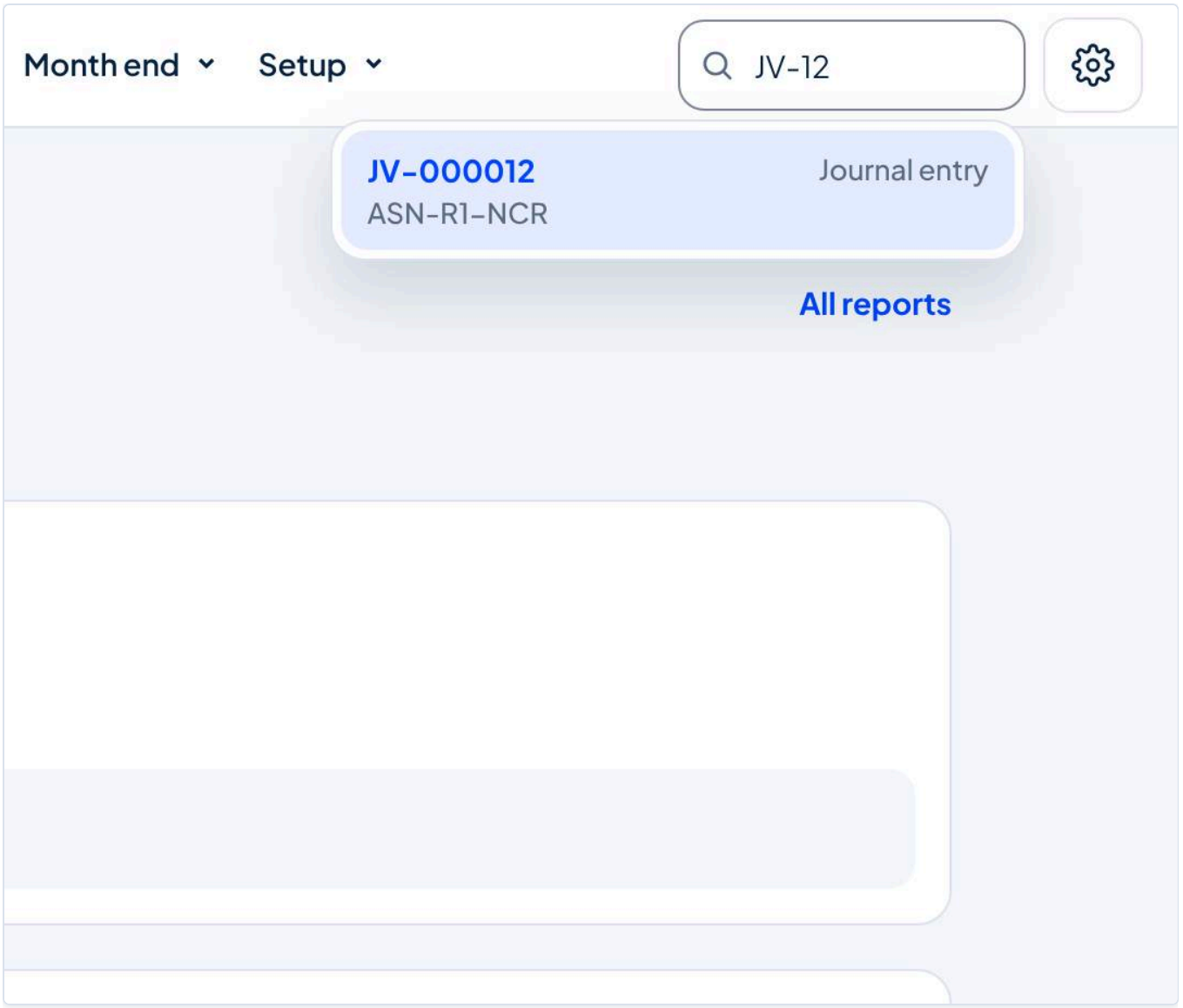
merchant and supplier names, always inside the person's stores and merchants.



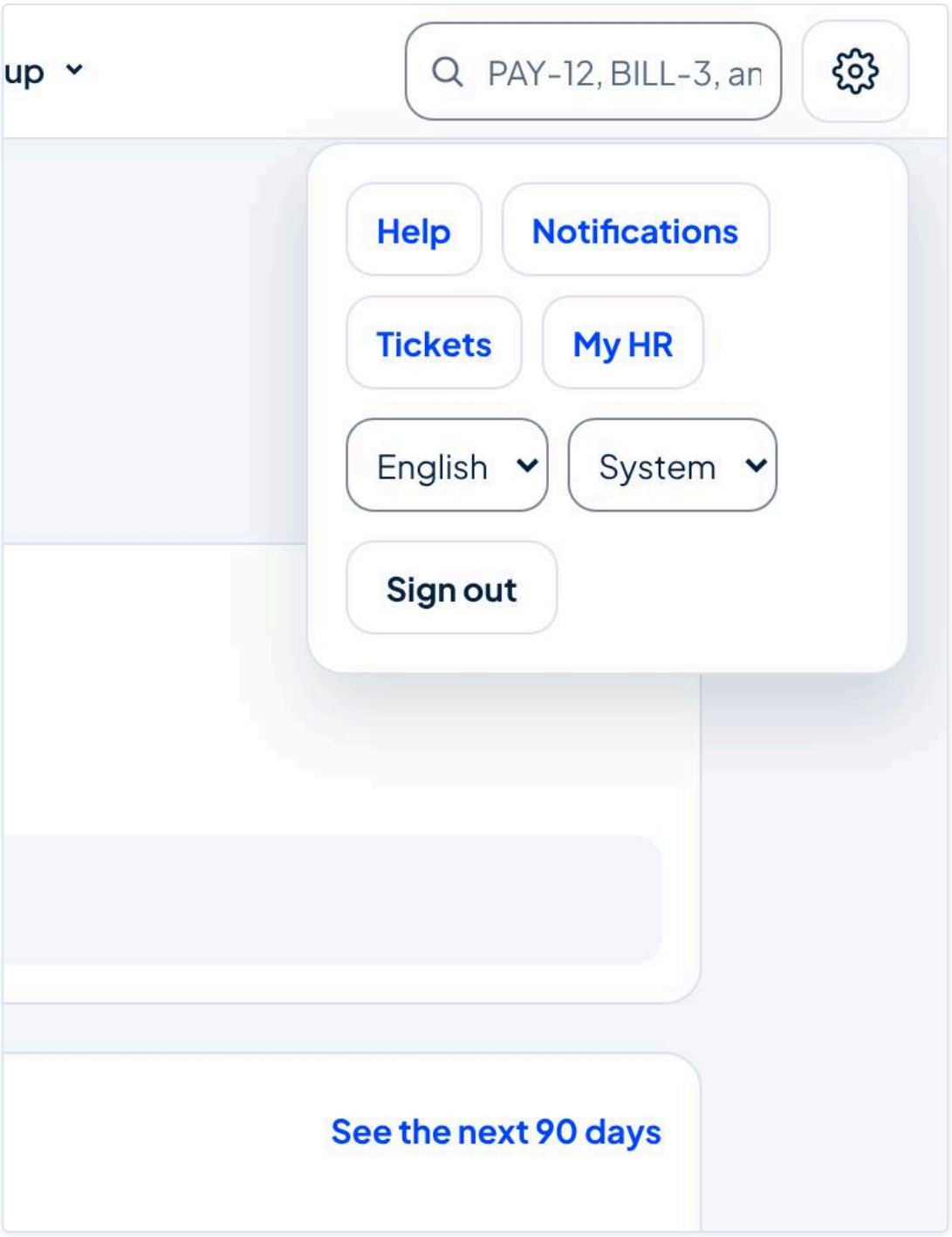
Typing `PAY-1` finds `PAY-000001`, kind Payout, with the merchant below.



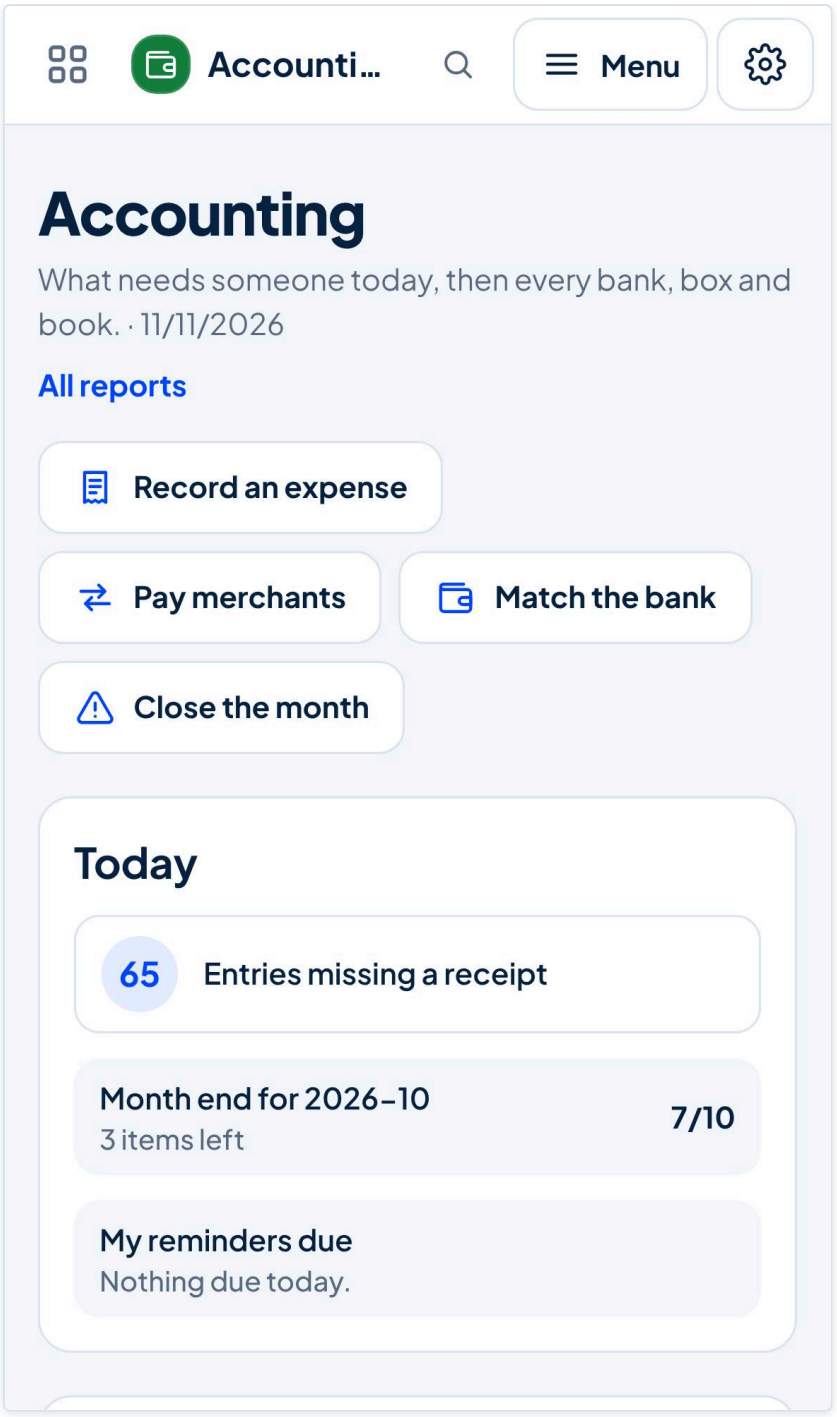
Typing `BILL-1` finds `BILL-000001`, kind Bill, with the supplier below.



Typing `JV-12` finds journal entry `JV-000012` with its memo below.



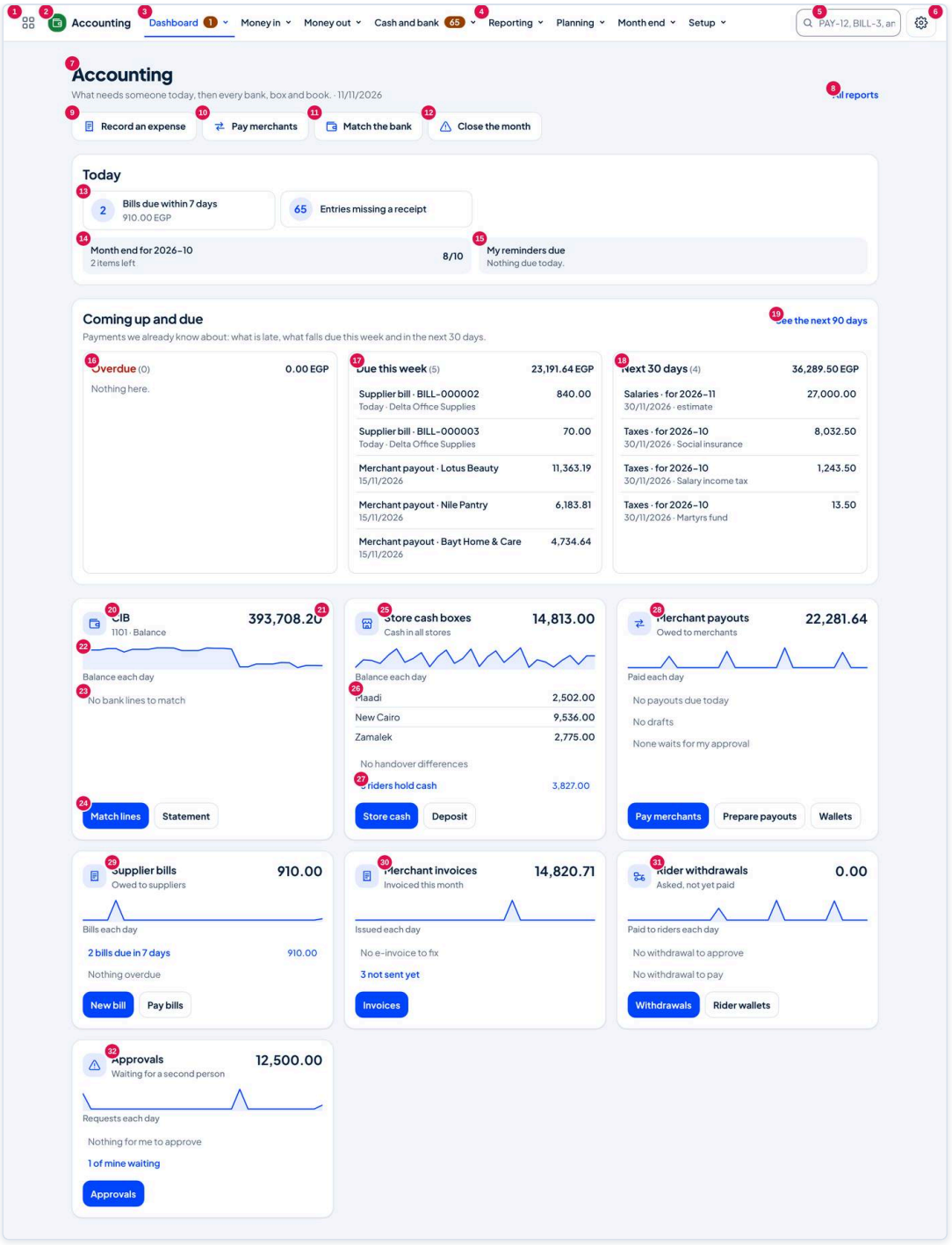
The Settings menu opened from the gear: Help, Notifications, Tickets, My HR, language, theme and Sign out.



On a phone the menus fold into one Menu button and the search becomes a magnifier button; the quick actions wrap. Address `/finance`.

13.3 Dashboard

Address: `/finance`, menu Dashboard > Dashboard. Who: accounting view. The quick action Record an expense is shown only with accounting use; card actions that change the books are hidden without use. People limited to some stores see only their boxes, riders and handovers. Data comes from `GET /finance/dashboard` and `GET /finance/payments-due?days=30`.



The Accounting dashboard with numbered parts (legend below). Address `/finance`.

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Grid button	Four grey squares	Back to `/home`
2	App name	Green wallet tile and "Accounting" in Navy	Reloads the dashboard
3	Dashboard menu with badge	Blue text and blue underline (current menu); brown pill with the count of approvals waiting for me	Opens the menu
4	Cash and bank badge	Brown pill: entries missing a receipt	Opens the menu
5	Search box	Grey outline, magnifier, placeholder	See the top bar

#	Part	What it shows (colors, icons, wording)	What clicking it does
6	Settings	Gear in a rounded square	Account menu
7	Title and intro	"Accounting", "What needs someone today, then every bank, box and book." and today's Cairo date (dd/mm/yyyy)	None
8	All reports	Blue link at the right	Opens `/finance/reports`
9	Record an expense	White button, blue receipt icon; only with accounting use	Opens `/finance/expenses/new`
10	Pay merchants	White button, blue arrows icon	Opens the guided run `/finance/payouts/run`
11	Match the bank	White button, wallet icon	Opens `/finance/bank`
12	Close the month	White button, warning triangle	Opens `/finance/month-end/run?month=` of last month
13	Today strip items	One tile per waiting item with a count in a blue circle and, where it applies, an EGP amount below; only items with a count above 0 are shown (approvals for me, payouts due today, rider withdrawals, handovers with a difference, bank lines to match, bills due within 7 days, entries missing a receipt, my reminders). When nothing waits: "Nothing waits for anyone. Well done."	Opens the list filtered to that item (for example bills with `?f=due_7_days`)
14	Month end tile	Grey tile "Month end for `2026-10`" with "n items left", "Everything is ready: close the books." or "The books are closed for this month.", and done/total on the right	Opens the Close the month run on that month
15	My reminders due	Grey tile; each reminder has a dot (red when overdue, amber when due today), the record label in blue and the note; "Nothing due today." when empty	Opens the record
16	Overdue	Red title with the count, total in EGP; rows of payments past their day, or "Nothing here."	A row opens the screen that pays it
17	Due this week	Navy title, count and total; at most five rows (kind · label, day or "Today" · detail, amount); "n more" link when longer	Merchant payout rows open Pay merchants; bills open the bill
18	Next 30 days	Same layout for the rest of the 30 days; salaries say "estimate"	Salaries open `/finance/payroll`, taxes `/finance/taxes`
19	See the next 90 days	Blue link	Opens `/finance/due`
20	Bank card (one per bank and company cash box)	Icon in a blue square, name, code and "Balance"	None
21	Card balance	Large number in EGP; red when below zero	None
22	Sparkline	Shiply Blue line with a light fill of the last 30 days and its caption ("Balance each day", "Paid each day"...)	None

#	Part	What it shows (colors, icons, wording)	What clicking it does
23	Waiting lines	Blue and bold when the count is above 0, grey when 0 (for example "No bank lines to match")	Opens the filtered list
24	Card actions	First action blue filled, the others white outlined	Bank: Match lines, Statement; Store boxes: Store cash, Deposit; Payouts: Pay merchants, Prepare payouts, Wallets
25	Store cash boxes card	Total of all store boxes, sparkline	None
26	Store parts	Up to five stores with their box balance	None
27	Riders holding cash	"6 riders hold cash" with the amount, blue when above 0	Opens the cash position
28	Merchant payouts card	Owed to merchants; waits: payouts due today, drafts to pay, waiting for my approval	Pay merchants, Prepare payouts, Wallets
29	Supplier bills card	Owed to suppliers; bills due in 7 days, overdue	New bill, Pay bills
30	Merchant invoices card	Invoiced this month; e invoices to fix, not sent yet	Invoices
31	Rider withdrawals card	Asked, not yet paid; to approve, to pay	Withdrawals, Rider wallets
32	Approvals card	Amount waiting for a second person; "Nothing for me to approve", "1 of mine waiting"	Approvals

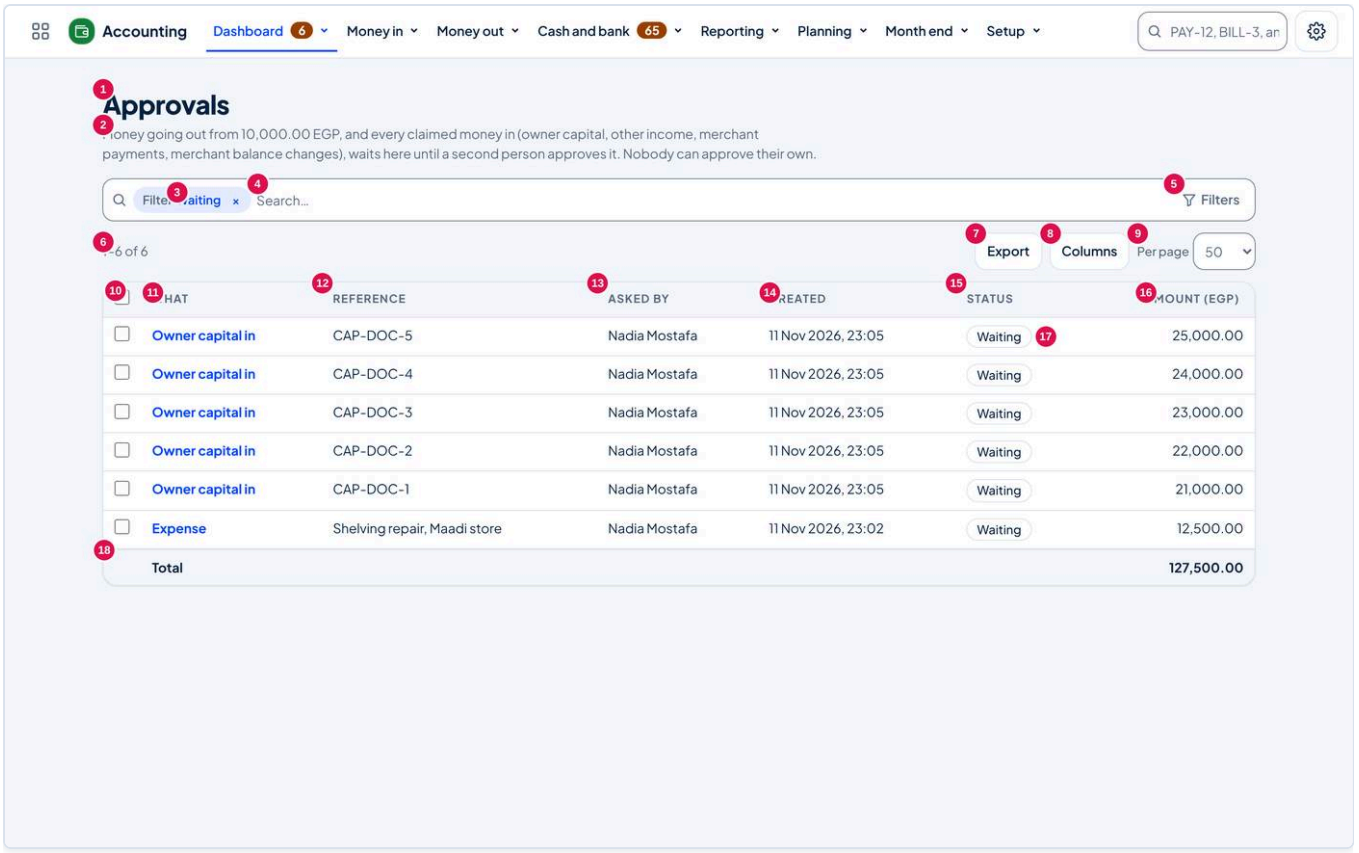
RULE

The dashboard posts nothing. Every number is read from the ledger and the documents at request time; balances are always summed from lines, never stored.

13.4 Approvals

Address: `/finance/approvals`, menu Dashboard > Approvals. Who: accounting view reads the list; accounting use may withdraw their own request; accounting manage approves and rejects (one at a time or in bulk). The list is a list view fed by `GET /finance/lists/approvals`.

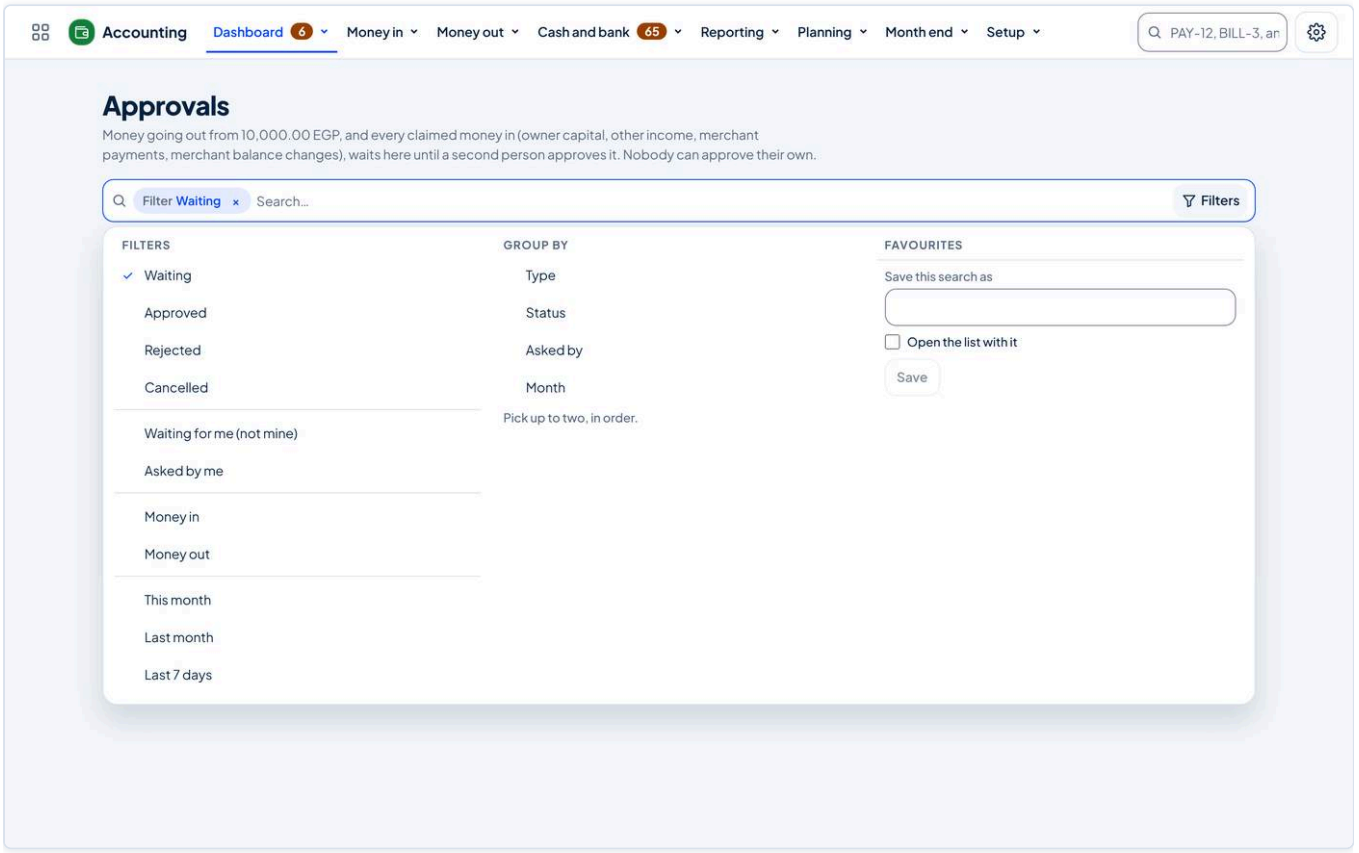
What lands here: money going out from `finance.approval_threshold_piastres` (10,000.00 EGP in the demo) and every claimed money in (owner capital, other income, merchant payments, merchant balance changes, cheques received, credit and debit notes). The intro sentence prints the live limit. When payouts wait for a manager, a yellow outlined banner "n payouts wait for a second person: open them" links to `/finance/payouts?f=for_me_to_approve`.



Approvals opened by the second finance user: the default filter Waiting is applied. Address `/finance/approvals`.

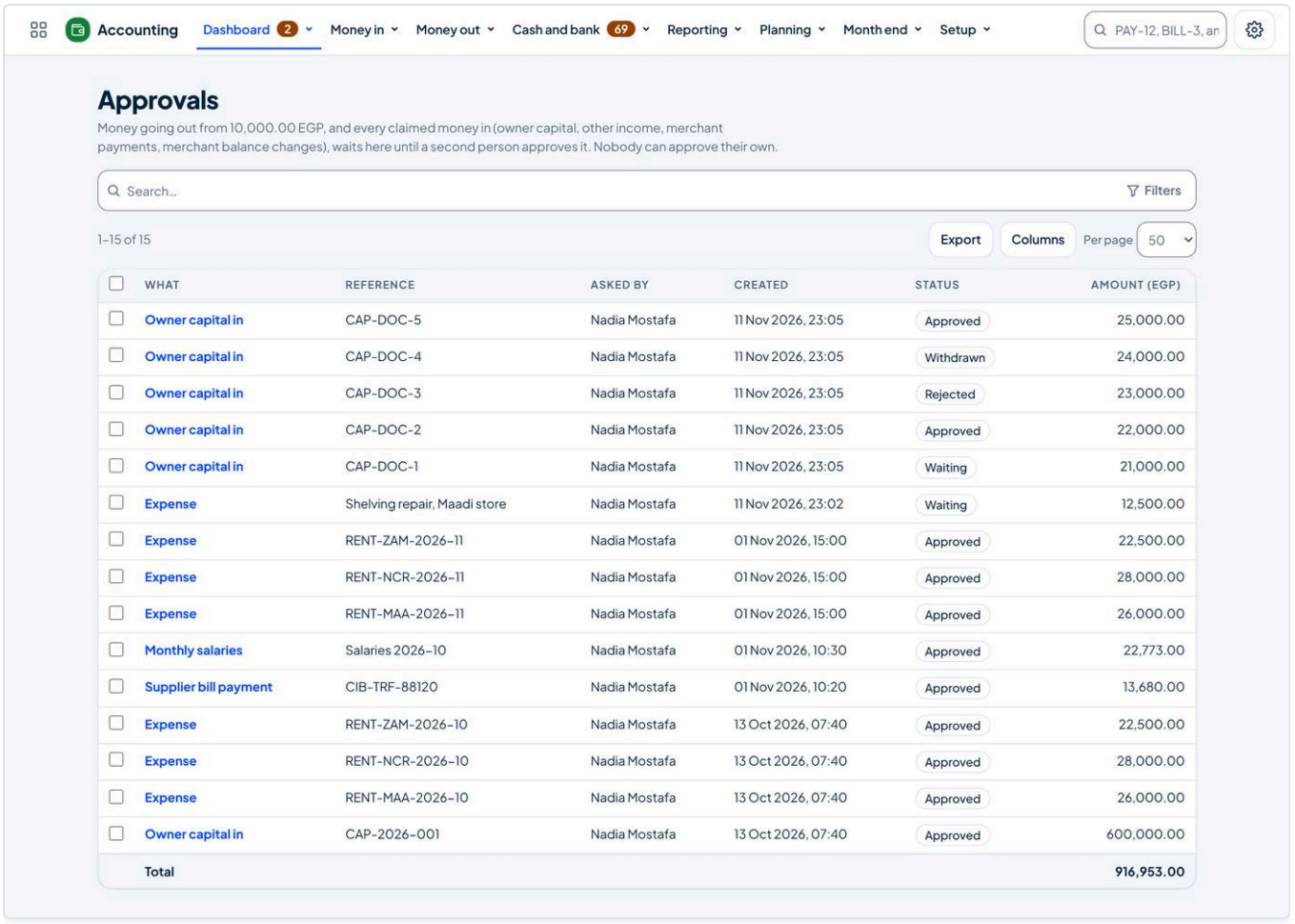
#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Title	"Approvals"	None
2	Intro	The rule with the current limit	None
3	Facet chip	Light blue chip "Filter Waiting" with a blue ×	× removes the filter; Backspace in the search removes the last chip
4	Search	"Search…": searches kind, reference and person	Adds a Search chip
5	Filters	Funnel icon	Opens Filters, Group by and Favourites
6	Range	"1–6 of 6"	None
7	Export	White button	CSV or Excel workbook; only ticked rows when some are ticked
8	Columns	White button	Shows the optional columns Decided by, Entry, Receipts
9	Per page	25, 50, 100, 200	Changes the page size
10	Select all	Checkbox in the header	Ticks every row of the page and shows the bulk bar
11	What	The request kind in blue (Owner capital in, Expense, Merchant balance change, Cheque received, Credit note to a merchant…); sortable	Opens the request page
12	Reference	The reference typed by the asker	None

#	Part	What it shows (colors, icons, wording)	What clicking it does
13	Asked by	The person who asked	None
14	Created	Day and time in Cairo; sortable	Sorts
15	Status	Status pill	None
16	Amount (EGP)	Right aligned; sortable	Sorts
17	Status pill	Waiting, Approved, Rejected or Withdrawn: dark text in a thin grey outlined pill (the four states look the same in the list; only the word differs)	None
18	Total row	Sum of Amount over every passing row, not only the page	None



The Filters panel: status (Waiting, Approved, Rejected, Withdrawn), who (Waiting for me (not mine), Asked by me), direction (Money in, Money out), dates (This month, Last month, Last 7 days); Group by Type, Status, Asked by, Month; Favourites to save the search.

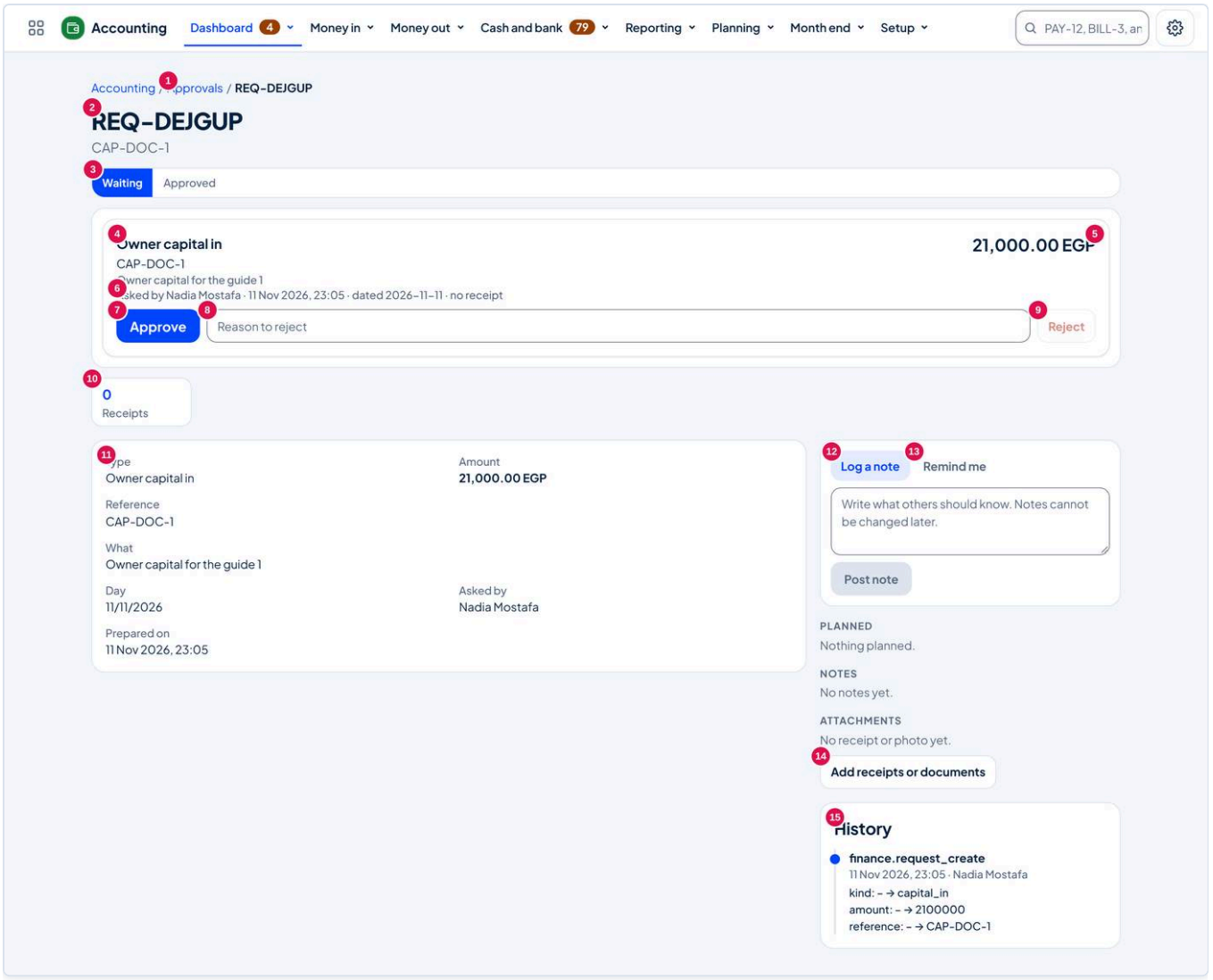
Filter or grouping	Rule
Waiting, Approved, Rejected, Withdrawn	One group: ticked ones are joined with "or"
Waiting for me (not mine)	Pending requests asked by someone else
Asked by me	Requests I asked for
Money in, Money out	By the direction of the request kind
This month, Last month, Last 7 days	Cairo days of creation
Group by	Up to two levels, in order, with counts and subtotals
Favourites	"Save this search as", "Open the list with it" (default), Save; kept per person



With the Waiting chip removed every state shows: Approved, Withdrawn, Rejected and Waiting pills side by side.

An approval request page

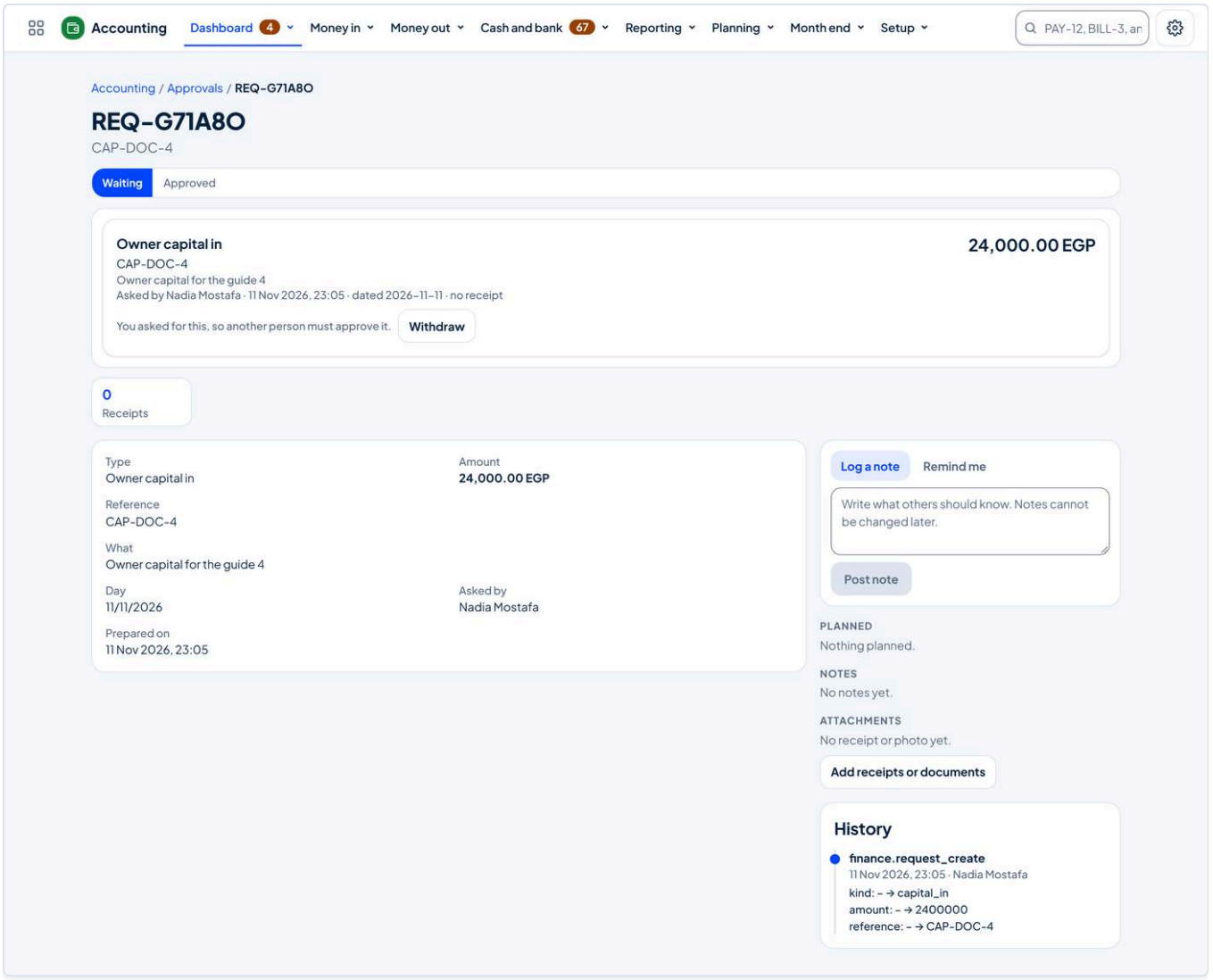
Address: `/finance/approvals/<id>`. The page is a `DocumentPage` fed by `GET /finance/documents/finance_request/<id>`; the action card appears only while the request is pending and the reader has accounting use.



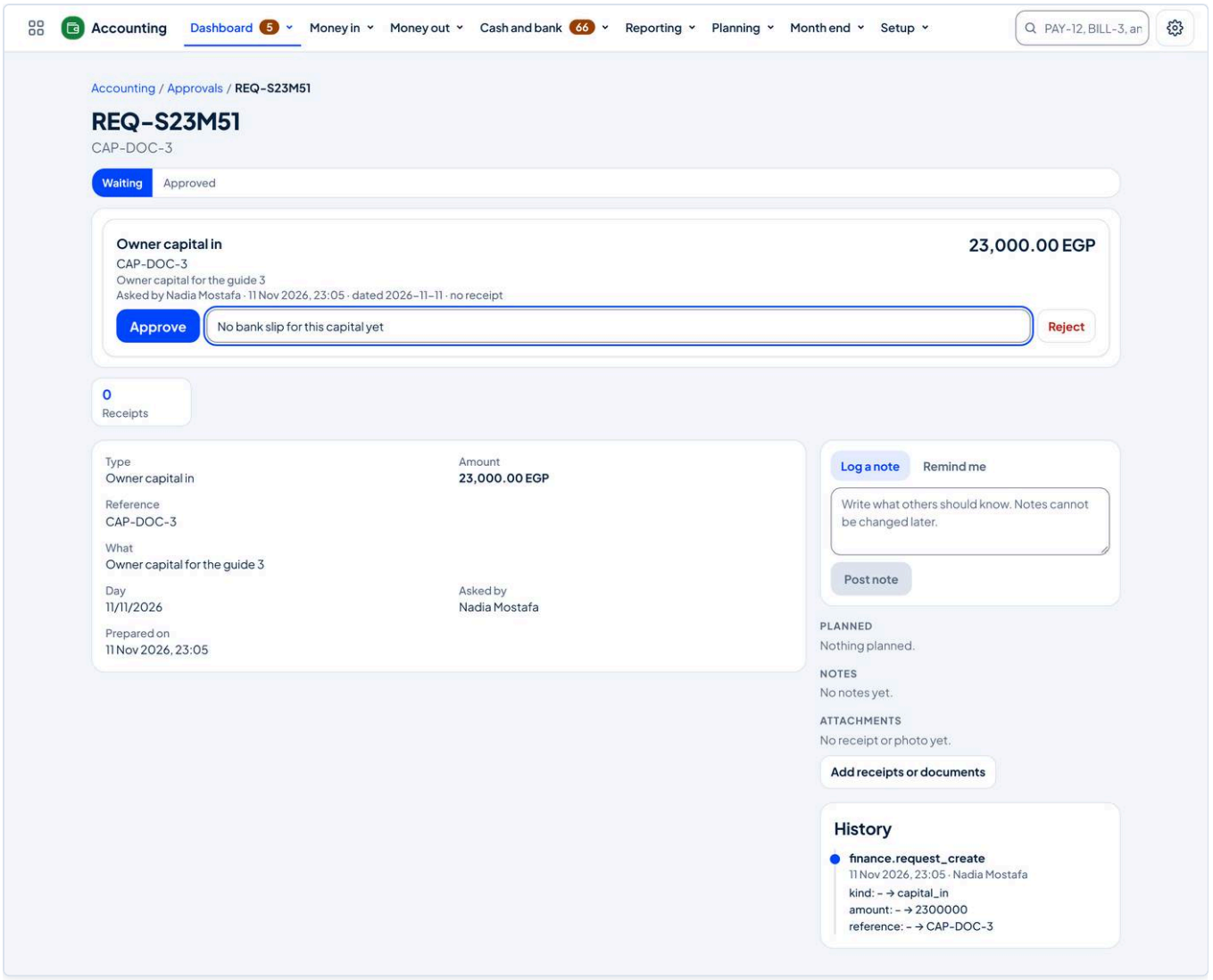
A pending request as the second finance user sees it, with numbered parts. Address `/finance/approvals/<id>`.

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Breadcrumbs	Accounting / Approvals / `REQ-xxxxxx` (the last six characters of the id)	Back to the list with its filters
2	Title	The request label and its reference below	None
3	Status bar	Waiting then Approved; the current step is a Shiply Blue filled segment with white text; a passed step has a tick; Rejected and Withdrawn appear as a red filled segment at the end	None
4	Kind	Bold kind ("Owner capital in"), reference, memo	None
5	Amount	Large "21,000.00 EGP"	None
6	Asked line	"Asked by name · time · dated day · no receipt / n receipts"	None
7	Approve	Blue filled button; shown only to someone other than the asker	Posts `POST /finance/requests/<id>/decide {approve: true}`: the entry is posted and the request becomes Approved

#	Part	What it shows (colors, icons, wording)	What clicking it does
8	Reason to reject	Text field	Needed for Reject
9	Reject	White button with red text; disabled until the reason has 3 characters	Rejects with the reason; nothing is posted
10	Smart button Receipts	Blue count and label	Opens the attachments
11	Fields	Type, Amount, Reference, What, Day, Asked by, Prepared on, and once decided Decided by, Decided on, Decision note, Entry	Entry opens the journal entry
12	Log a note	Chatter tab; "Notes cannot be changed later"	Posts an append only note
13	Remind me	Chatter tab	Plans a reminder on a Cairo day for a person who may open the record
14	Add receipts or documents	White button under Attachments	Adds files to the request
15	History	Blue dots, newest first, each audit row with who, when and before → after	None

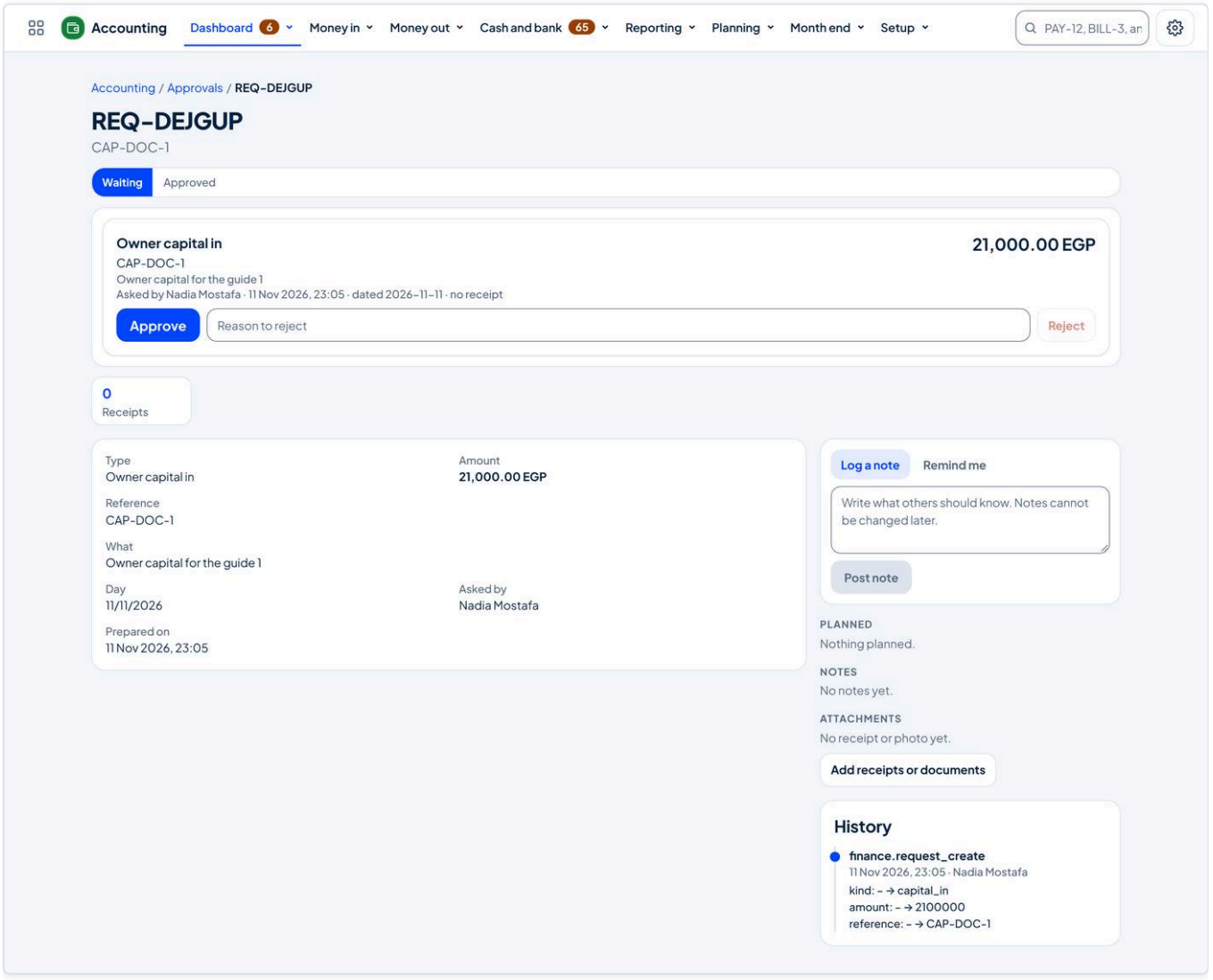


The asker opening their own request: "You asked for this, so another person must approve it." and a Withdraw button instead of Approve and Reject.

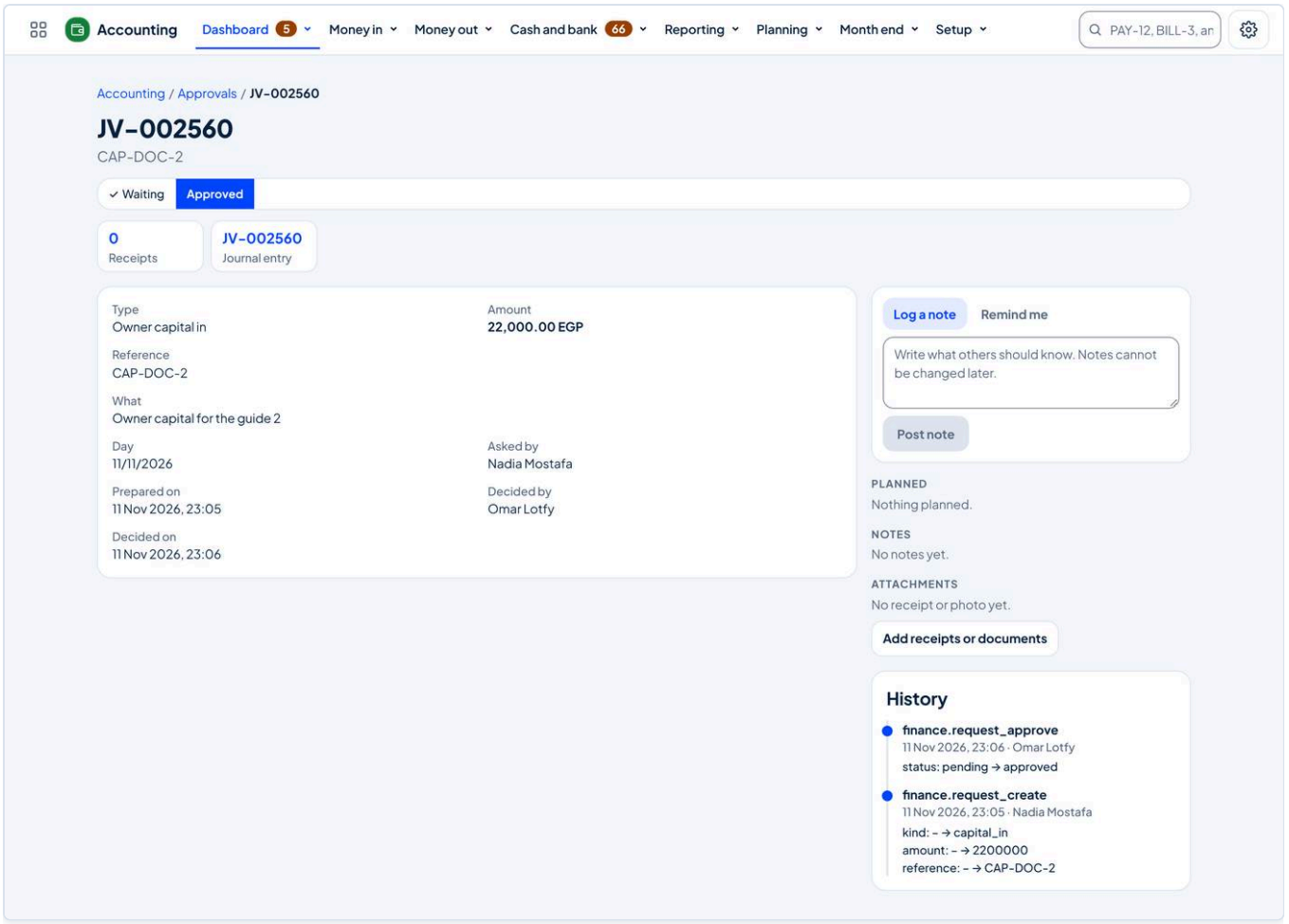


The second person typed a reason; Reject is now enabled (red text).

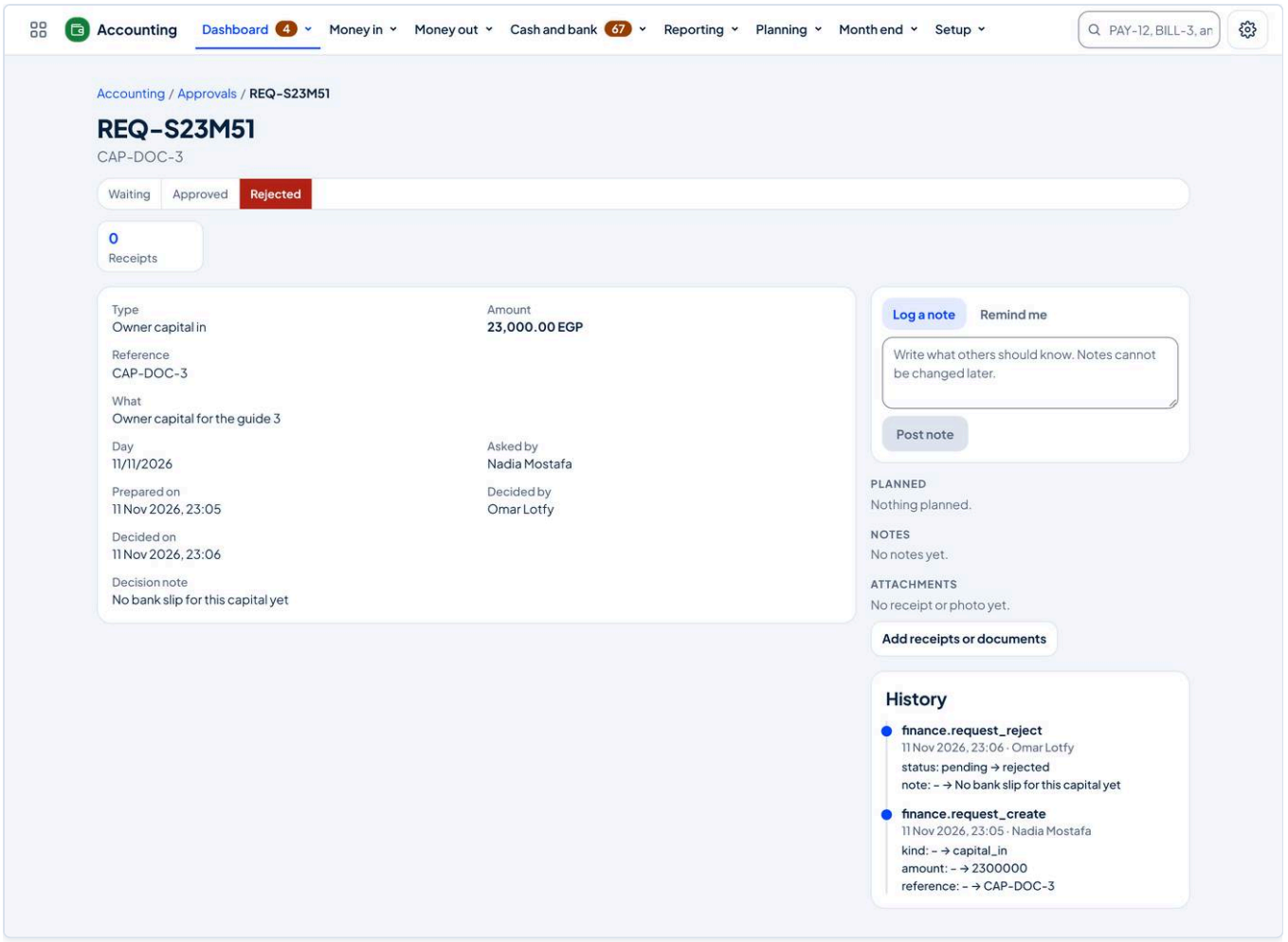
The four states of a request



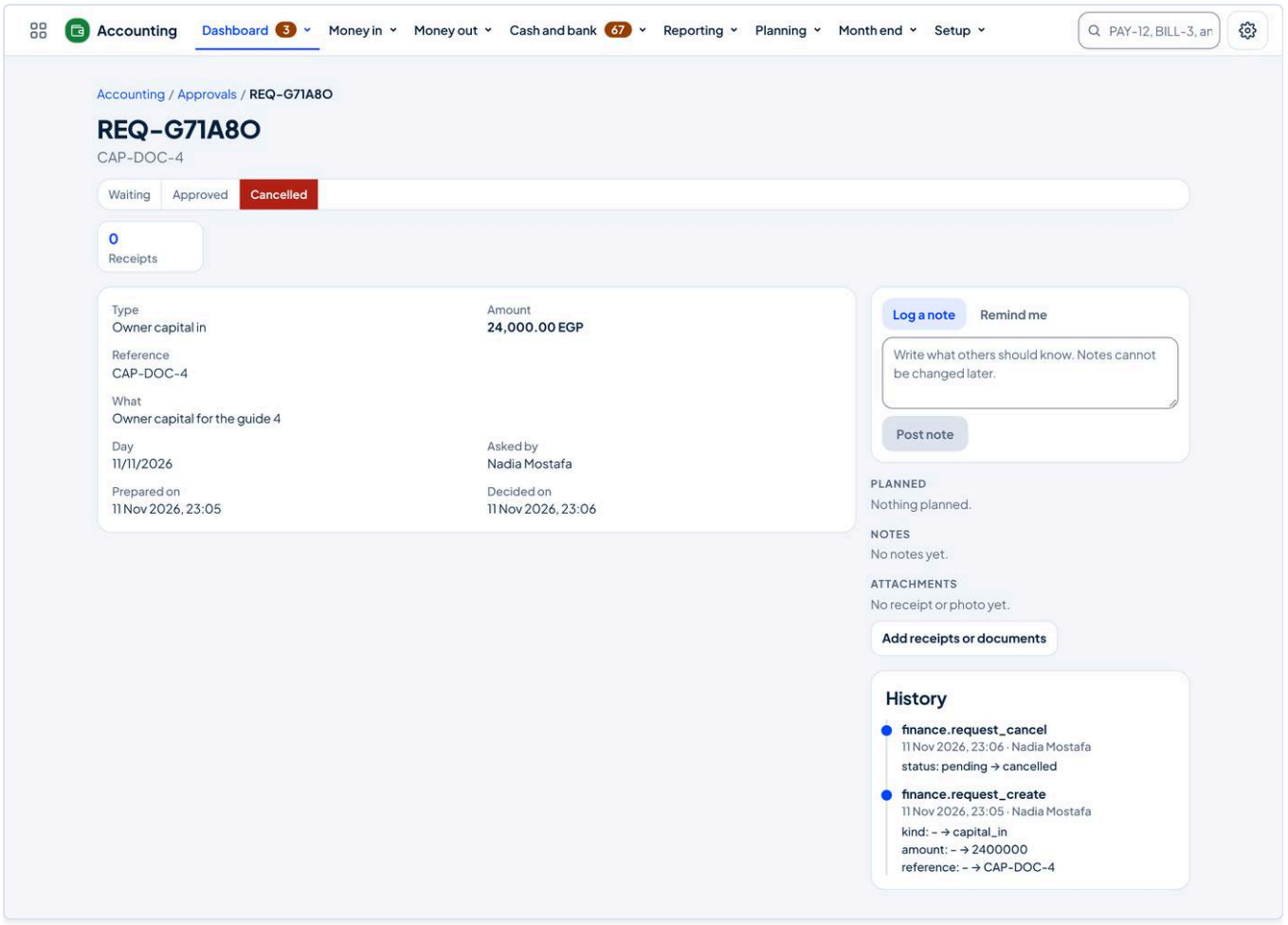
Waiting: blue Waiting segment, the action card with Approve, the reason field and Reject.



Approved by the second user: the status bar shows Waiting ticked and Approved filled blue; the fields show Decided by, Decided on and the entry it posted.



Rejected: a red Rejected segment at the end of the bar; Decision note shows the reason; nothing was posted.



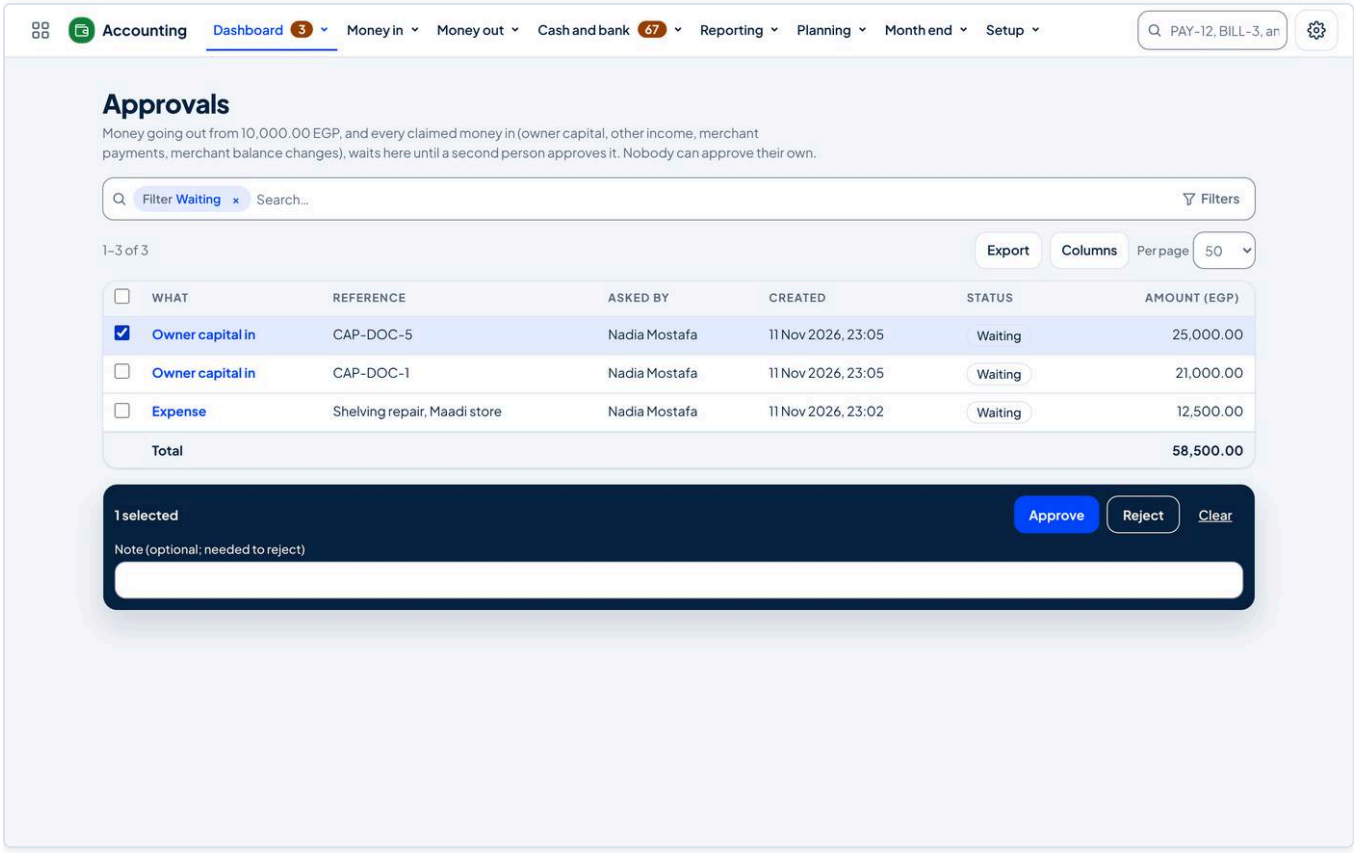
Withdrawn by the asker: a red Withdrawn segment; the request can no longer be decided.

Control	What it does	Rule / limits	Result
Approve	`POST /finance/requests/<id>/decide`	Accounting manage; never one's own (`same_person`); the request must still be pending (`request_not_pending` : "Someone already handled this."); money checks run again at approval (a bank must still hold the money)	Entry posted (JV number), status Approved, audit `finance.request_approve`
Reject	Same route with `approve: false`	Reason of at least 3 characters (`note_required` : "Write why you reject it.")	Status Rejected, nothing posted; a cheque waiting on it is cancelled
Withdraw	`POST /finance/requests/<id>/cancel`	Only the asker (`not_your_request` : "Only the person who asked can withdraw it.")	Status Withdrawn

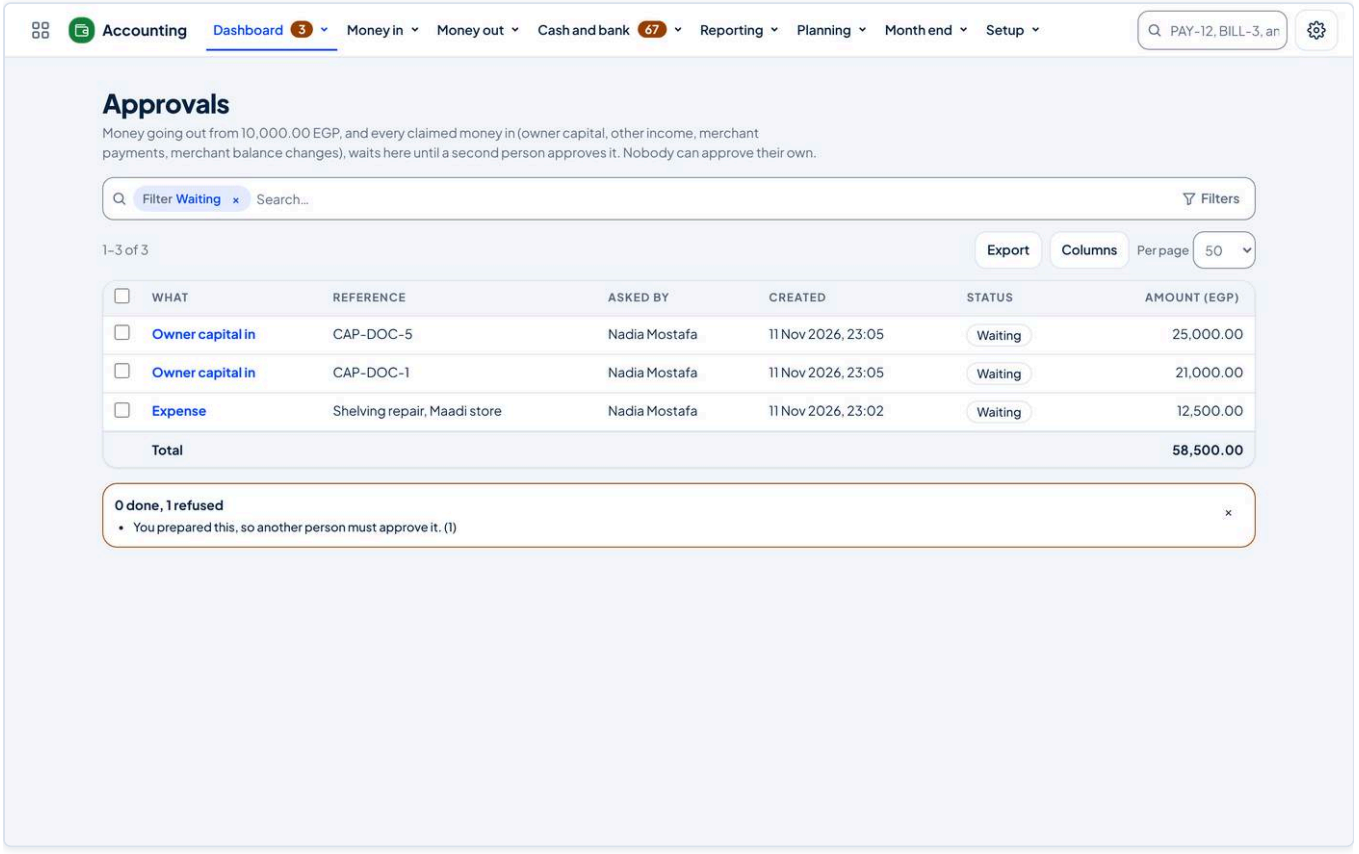
Bulk approve and reject

Ticking rows shows the sticky bulk bar in Navy at the bottom: "n selected", Approve (blue), Reject (outlined), Clear, and a note field ("Note (optional; needed to reject)"). The bar is shown only with accounting manage.

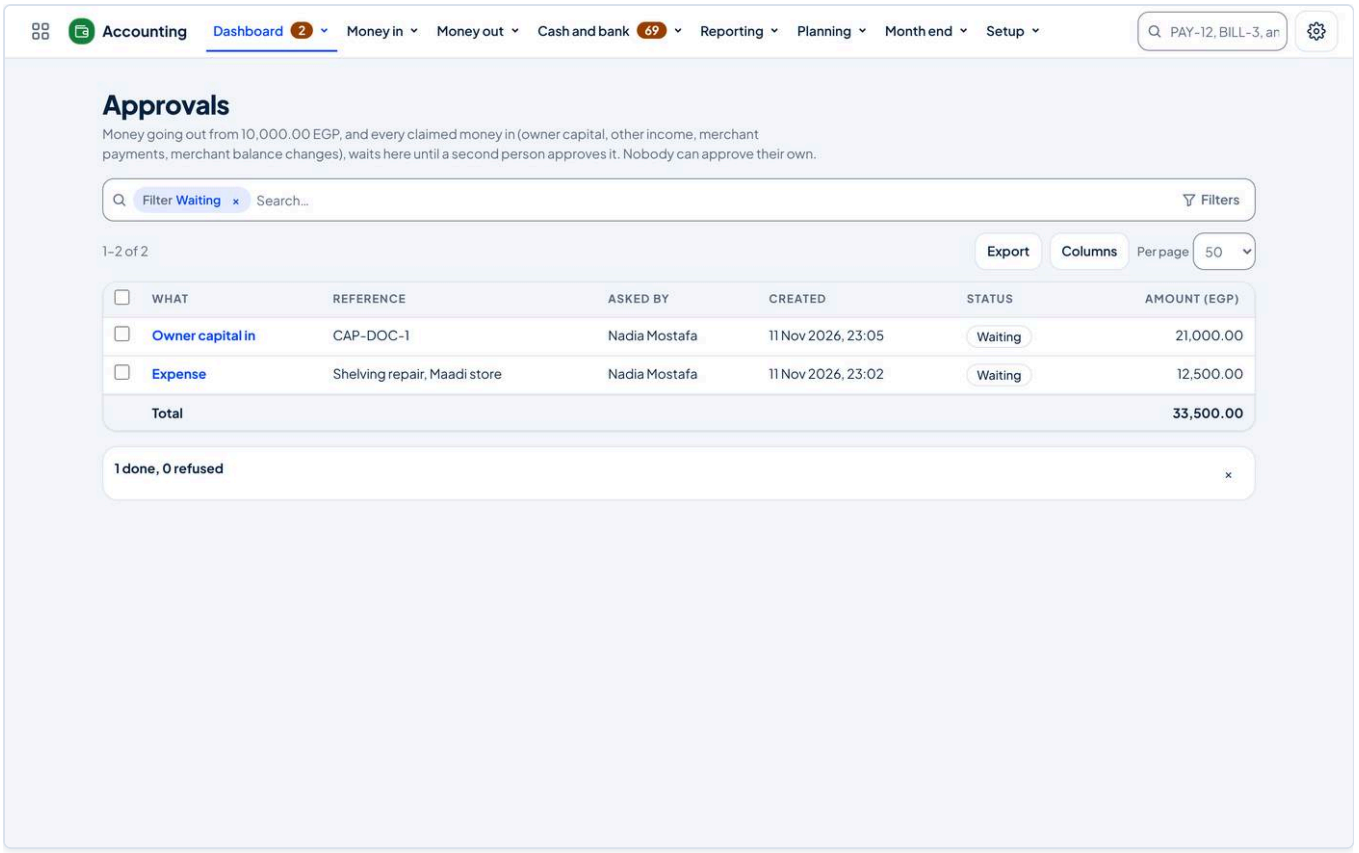
Each request is still checked on its own by `POST /finance/requests/bulk-decide`; the answer lists every row as done or refused.



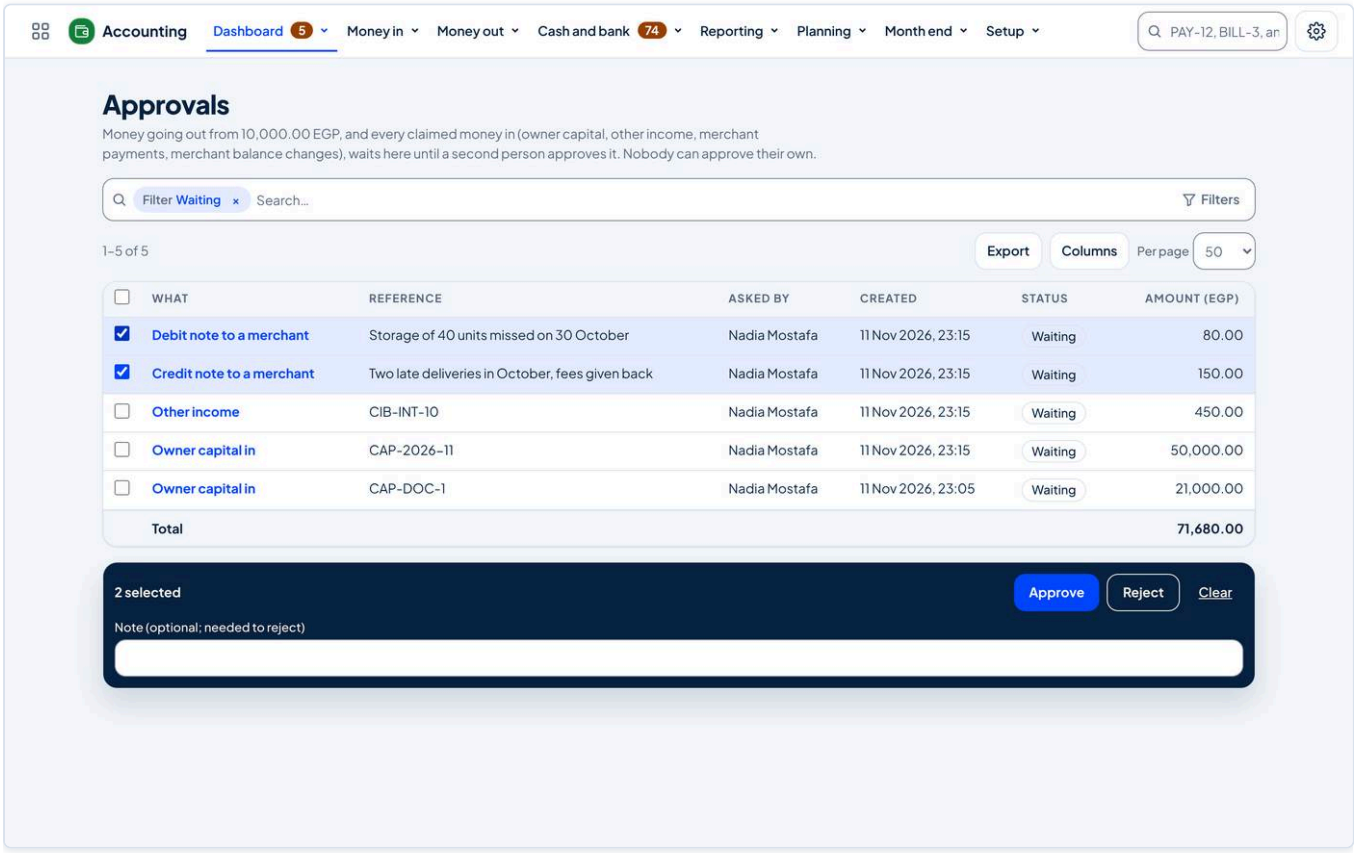
The preparer ticked one of her own requests: the Navy bulk bar shows "1 selected", Approve, Reject and Clear.



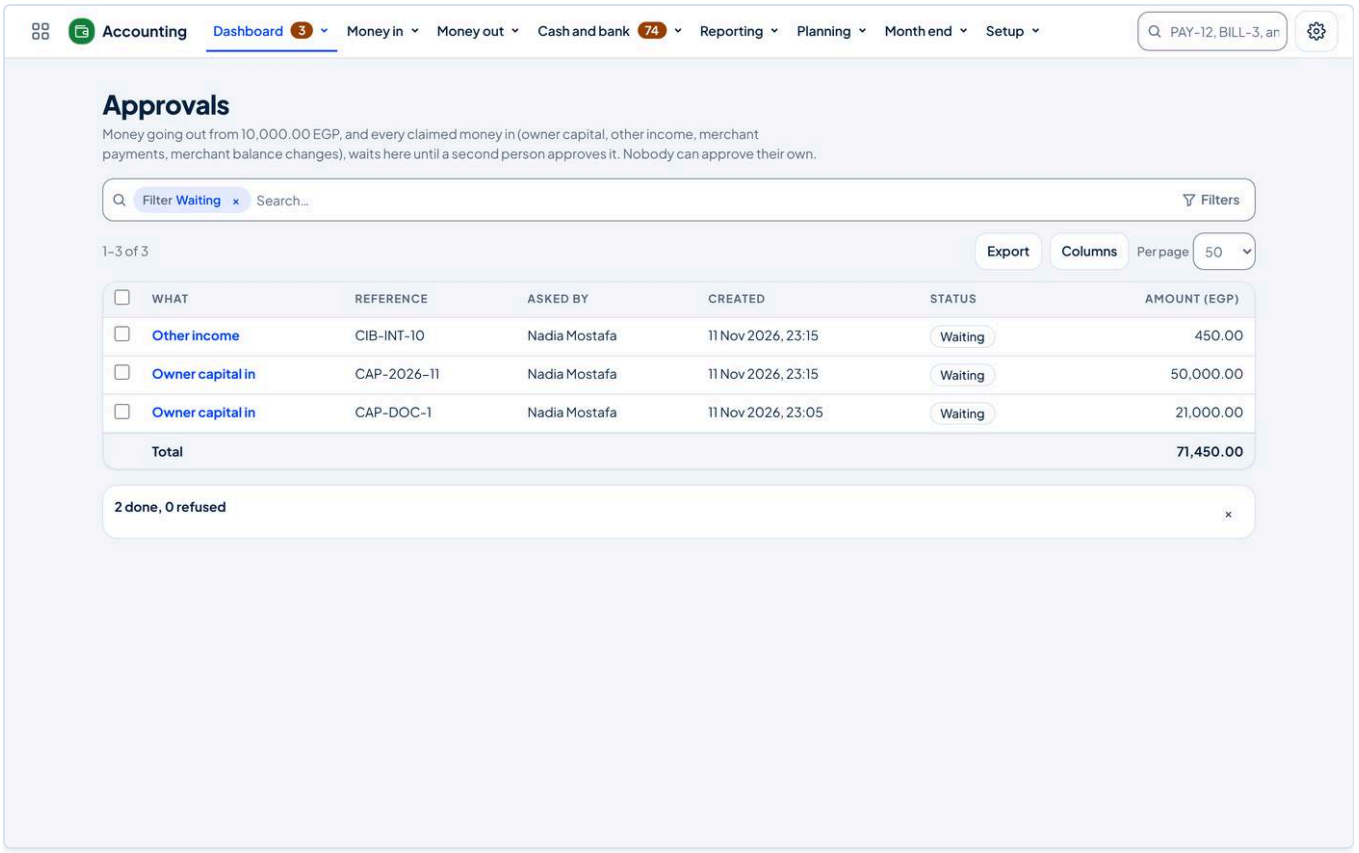
Bulk approve of one's own request: "0 done, 1 refused" with the reason "You prepared this, so another person must approve it. (1)" in an amber outlined box.



The second finance user approved the same request in bulk with the note "Bank slip seen": "1 done, 0 refused" and the row left the Waiting list.



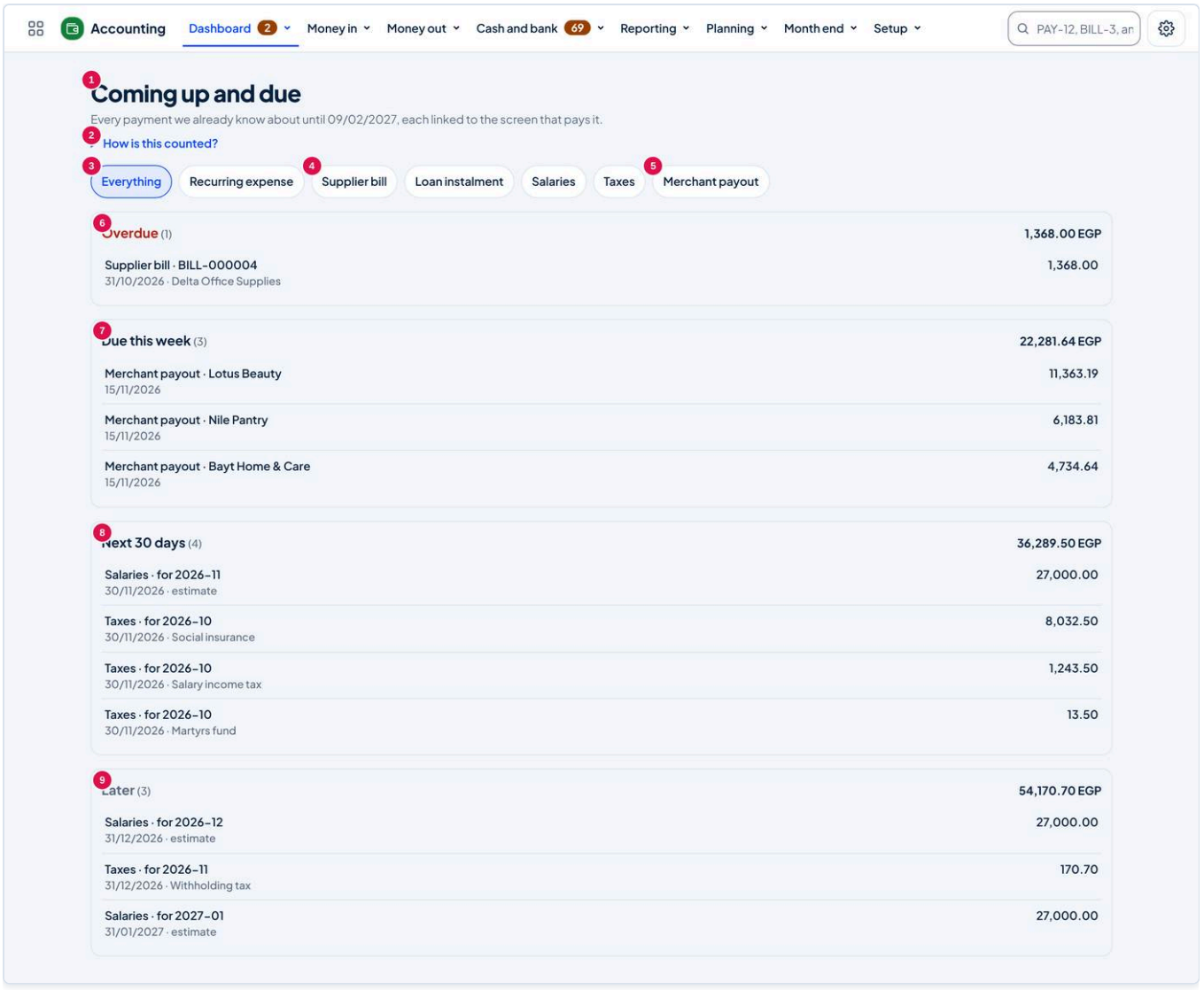
The second user ticks a credit note and a debit note request (see Invoices) before approving both at once.



Both notes approved in one tap: each was checked and posted on its own.

13.5 Coming up and due

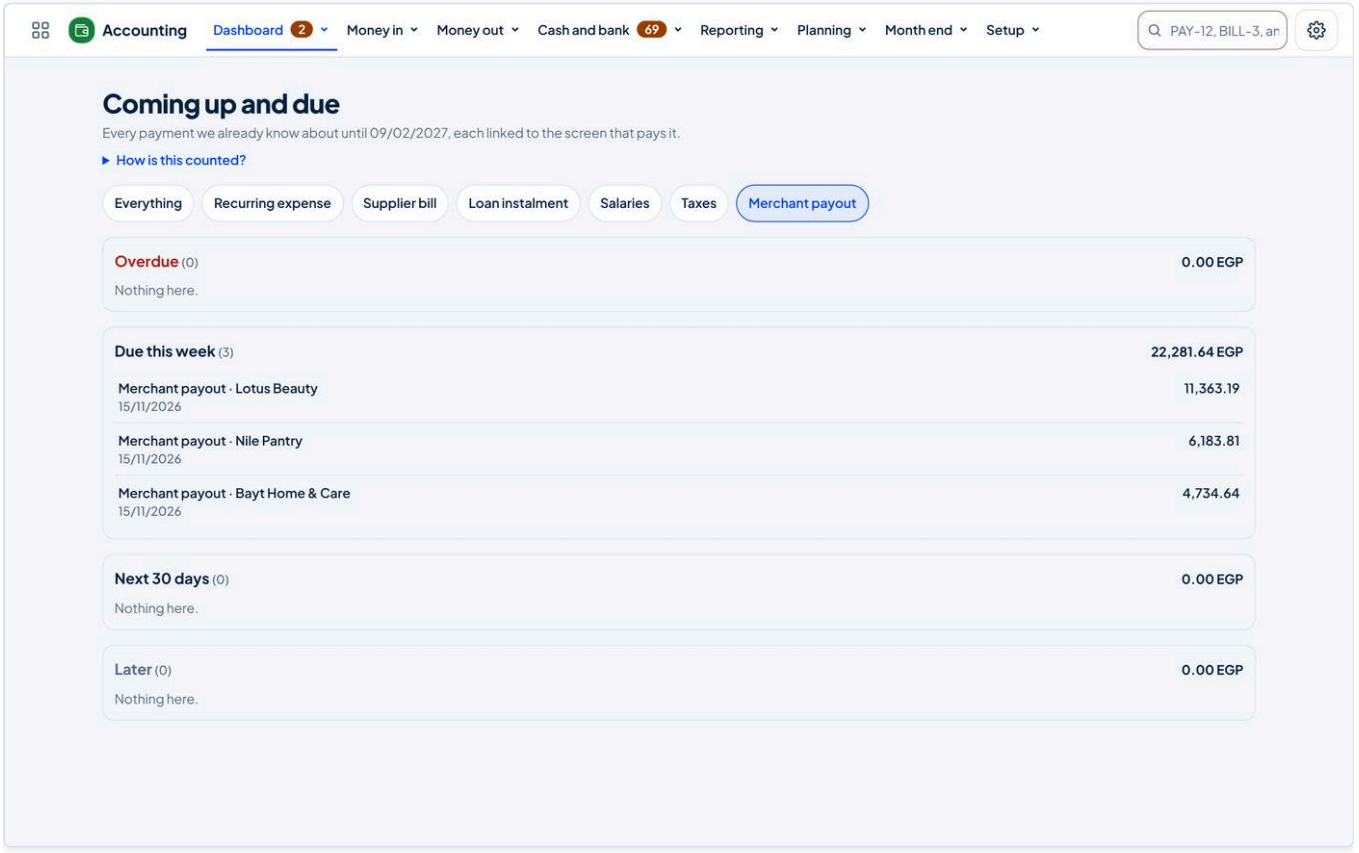
Address: `/finance/due`, menu Dashboard > Coming up and due. Who: accounting view. Data: `GET /finance/payments-due?days=90`, with `kinds=` when a kind is chosen. Nothing can be paid from this page; each row links to the screen that pays it.



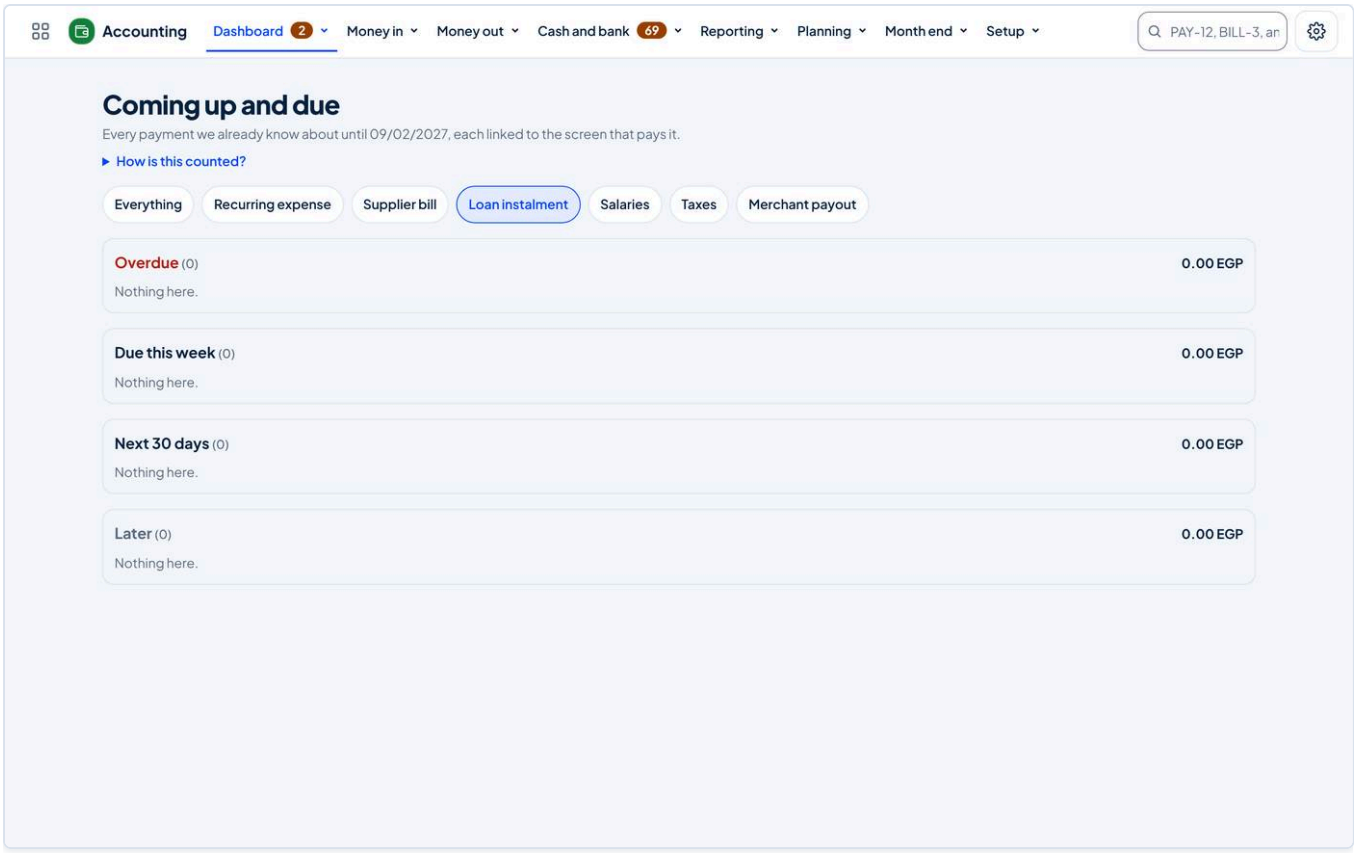
Every known payment of the next 90 days, grouped Overdue, Due this week, Next 30 days and Later. Address `/finance/due`.

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Title and intro	"Every payment we already know about until dd/mm/yyyy, each linked to the screen that pays it."	None
2	How is this counted?	Blue fold link	Opens the method text (below)
3	Everything chip	Selected chip: blue outline, light blue fill, blue text	Removes the kind filter
4	Kind chips	Recurring expense, Supplier bill, Loan instalment, Salaries, Taxes, Merchant payout; white when not chosen	Opens `/finance/due?kind=<kind>`
5	Merchant payout chip	Example of a kind chip	Shows only payouts
6	Overdue	Red title, count and total	Rows open the payer screen
7	Due this week	Navy title, count and total	Same
8	Next 30 days	Navy title	Same
9	Later	Grey title, up to day 90	Same

The method under "How is this counted?": recurring expenses until approved, open bills by due day, unpaid loan instalments, salaries estimated from the packages in force, taxes owed at the end of this or next month, merchant payouts on each merchant's next payout day.



Filtered to Merchant payout: only the payout rows remain; the chip is selected. Address `/finance/due?kind=payout`.

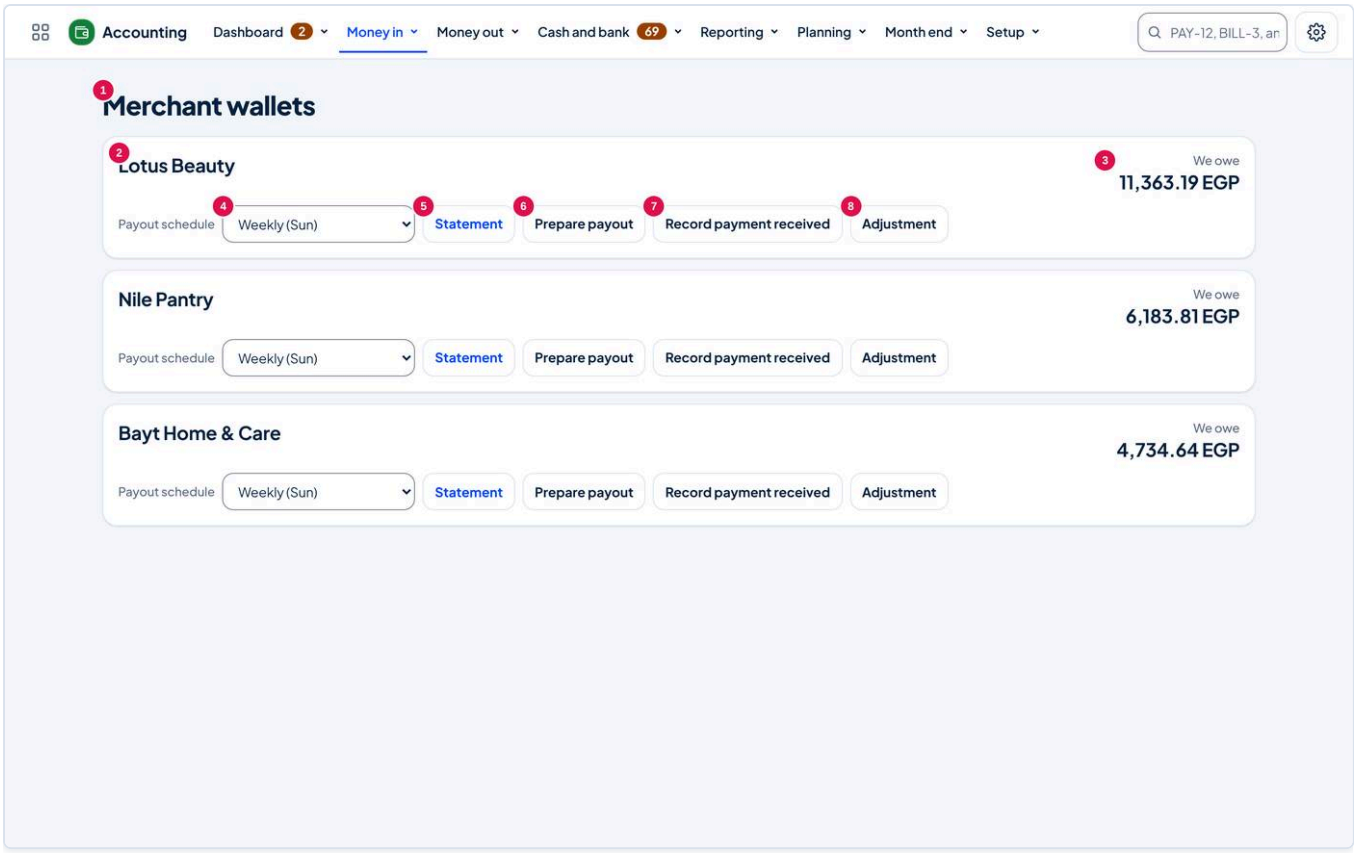


Empty state: filtered to Loan instalment with no loans, every group reads "Nothing here." with 0.00 EGP.

> **RULE**
> People limited to some stores see their stores' items and company wide ones, but not merchant payouts.

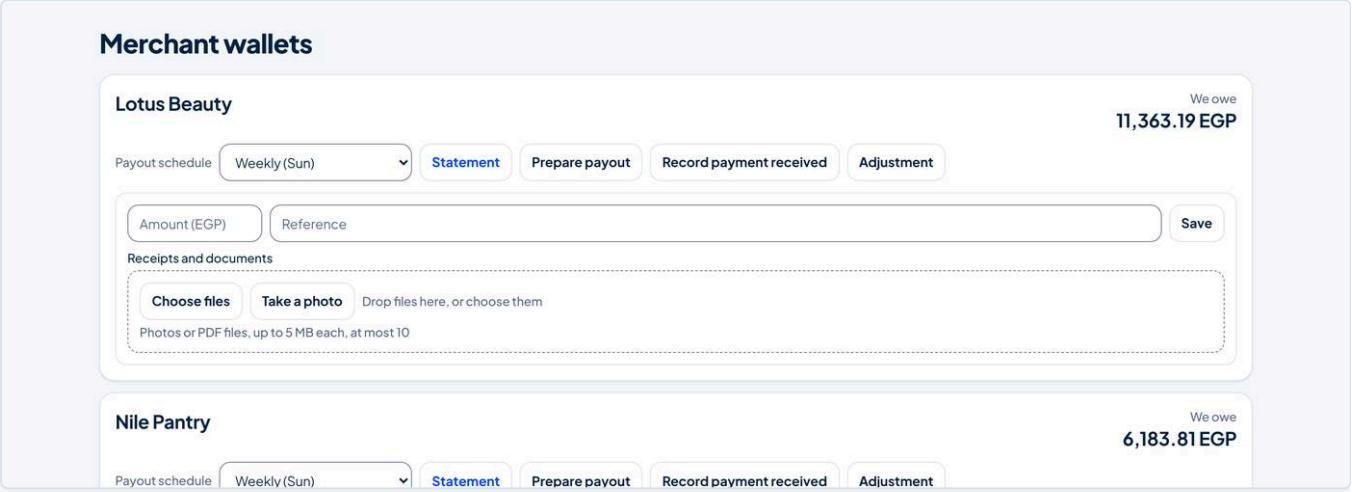
13.6 Merchant wallets

Address: `/finance/merchants`, menu Money in > Merchant wallets. Who: accounting view reads; the schedule, Prepare payout, Record payment received and Adjustment need accounting use. Data: `GET /finance/overview`. For the money rules see the chapter "Merchant money: wallets, payouts, receivables, invoices and profit".

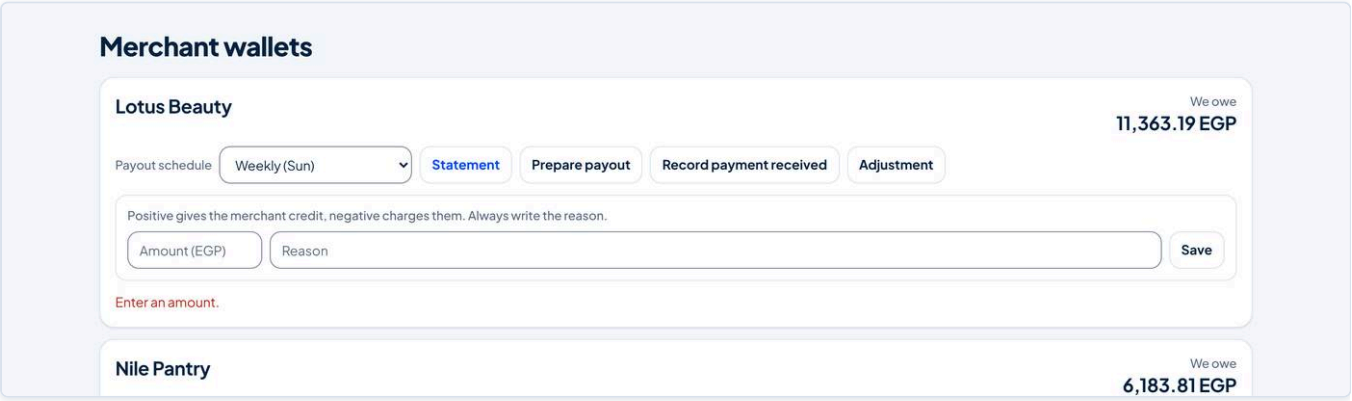


One card per merchant with what we owe them and the actions. Address `/finance/merchants`.

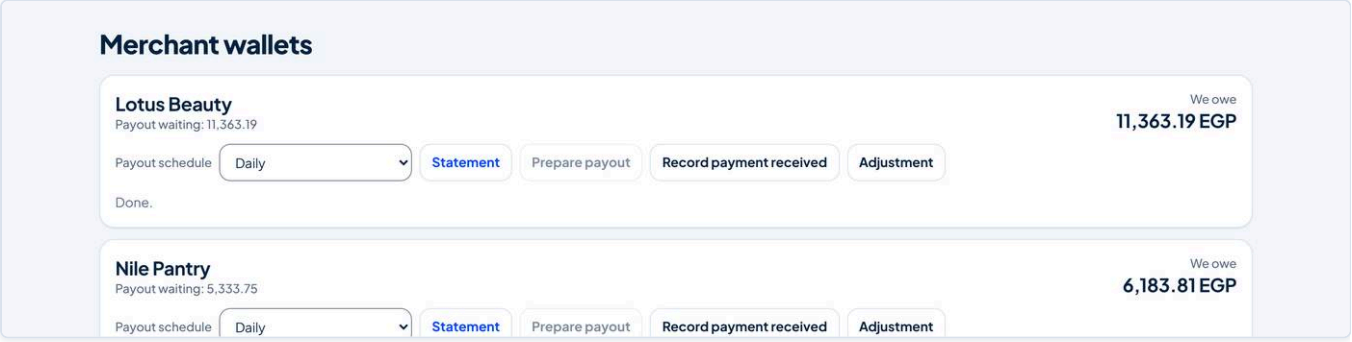
#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Title	"Merchant wallets"	None
2	Merchant name	Navy heading; under it "Payout waiting: amount" while a draft payout exists	None
3	Balance	"We owe" (credit) or "Owes us" (debit) above the amount in EGP, always shown positive	None
4	Payout schedule	Daily, Twice a week (Sun, Wed), Weekly (Sun)	Saves at once with `PATCH /finance/merchants/<id>/payout-schedule` (audited); a message "Done." appears
5	Statement	White button with blue text	Opens the wallet statement `/finance/merchants/<id>`
6	Prepare payout	White button; greyed when nothing is owed or a draft already exists	Prepares one payout up to the end of today (`POST /finance/payouts`); the SOA is frozen in the same transaction
7	Record payment received	White button	Opens the receipt panel
8	Adjustment	White button	Opens the adjustment panel



Record payment received: Amount (EGP), Reference, Save and the Receipts and documents field (Choose files, Take a photo, drop zone; photos or PDF up to 5 MB each, at most 10). A bank picker appears only when more than one bank is open.



Adjustment with the hint "Positive gives the merchant credit, negative charges them. Always write the reason." Saving with no amount shows the red message "Enter an amount."



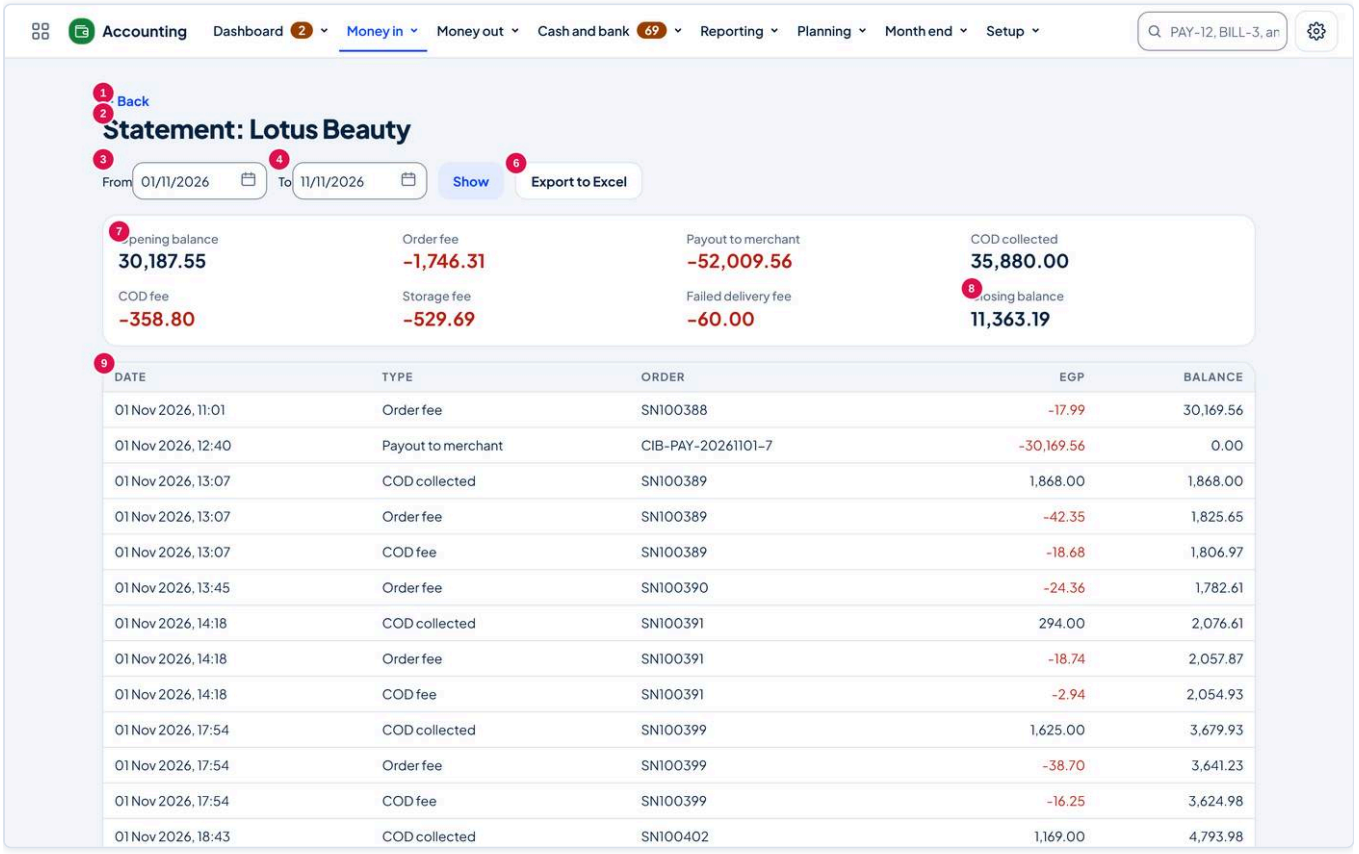
After Prepare payout: "Done.", the line "Payout waiting: 11,363.19" under the name, and Prepare payout greyed.

Control	What it does	Rule / limits	Result
Prepare payout	`POST /finance/payouts {merchantId, cutoffDay: today}`	`nothing_to_pay` : "Nothing is owed to this merchant up to that day."; `payout_already_open` : "This merchant already has a payout waiting."; below the contract minimum it is skipped	Draft payout `PAY-` with its frozen `SOA-` ; needs approval when at or above the limit

Control	What it does	Rule / limits	Result
Record payment received: Save	`POST /finance/receipts` with an idempotency key and the files	Always waits for a second person while `finance.approve_money_in` is 1	"Saved and sent for approval..." and a row in Approvals (Merchant paid us)
Adjustment: Save	`POST /finance/adjustments`	Reason of at least 5 characters (`reason_required`); waits for a second person	A Merchant balance change request in Approvals; once approved, entry kind `adjustment`

A merchant wallet statement

Address: `/finance/merchants/<id>?from=&to=` (default the last 30 Cairo days). Data: `GET /finance/statements/<id>` , the same `merchantStatement` the merchant reads.



The wallet statement of Lotus Beauty from 01/11/2026 to 11/11/2026, numbered parts in the legend. Address `/finance/merchants/<id>` .

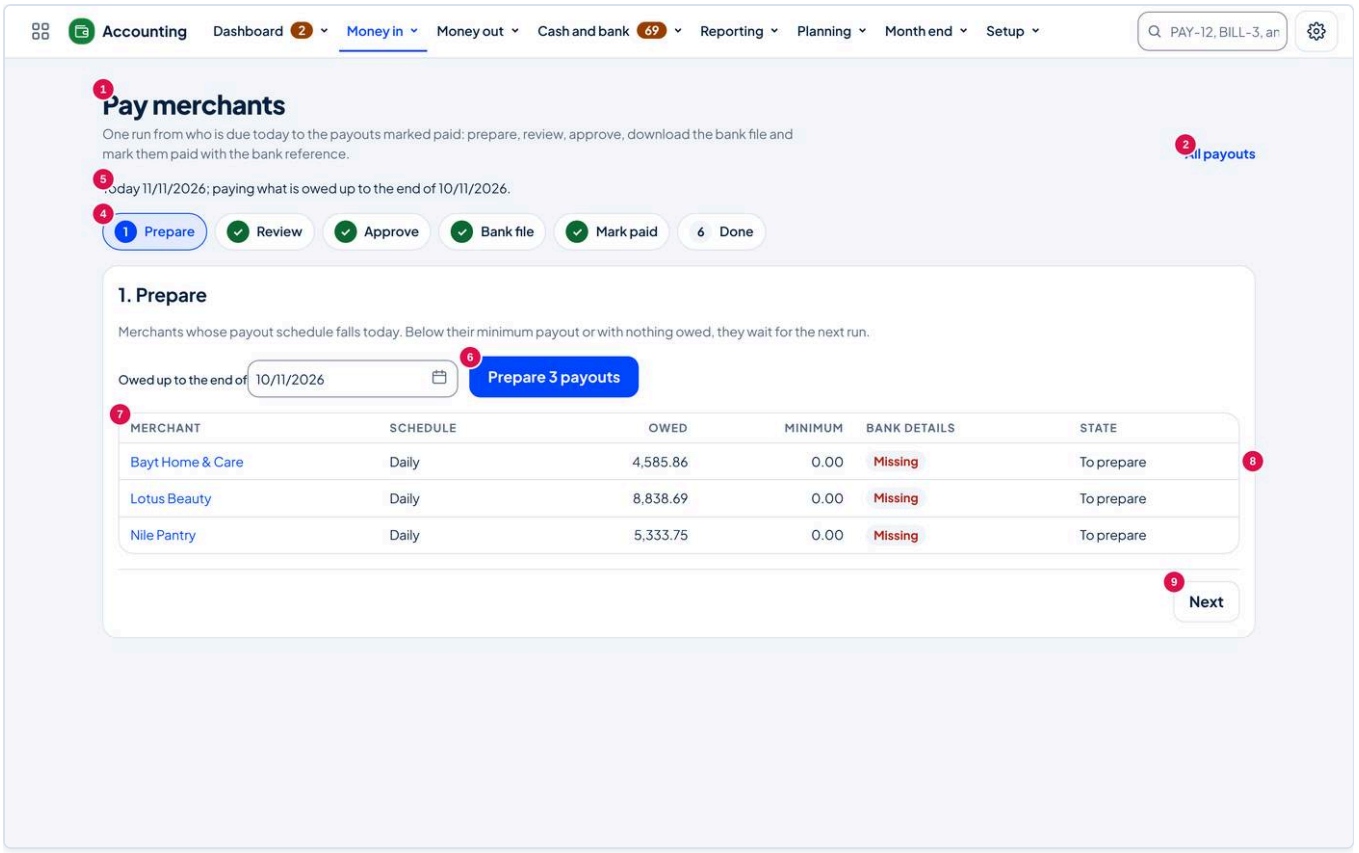
#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Back	Blue link with an arrow	Back to Merchant wallets
2	Title	"Statement: merchant"	None
3	From	Date box typed and shown dd/mm/yyyy with a calendar icon	Sets the first day
4	To	Same	Sets the last day

#	Part	What it shows (colors, icons, wording)	What clicking it does
(next to 4)	Show	Light blue button	Reloads with the days in the address
6	Export to Excel	White button	Downloads `/api/finance/statements/<id>/export.csv` (UTF 8 with byte order mark)
7	Opening balance	What we owed at the start; then one figure per entry kind (Order fee, Payout to merchant, COD collected, COD fee, Storage fee, Failed delivery fee...) with charges in red and minus	None
8	Closing balance	What we owe at the end	None
9	Lines	Date, Type, Order or reference, EGP (red when it lowers what we owe), Balance after the line	None

13.7 Pay merchants (the guided run)

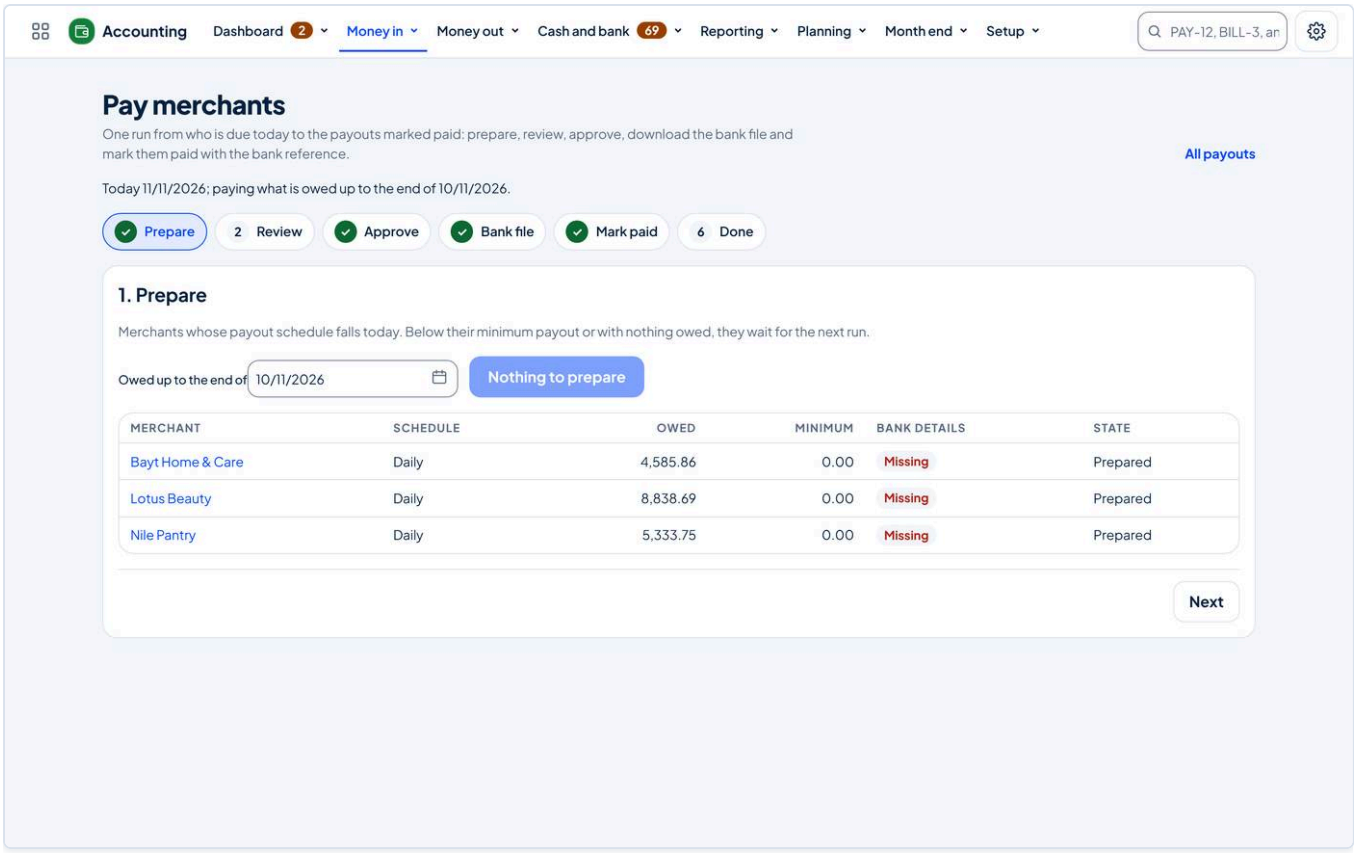
Address: `/finance/payouts/run?step=&cutoff=`, menu Money in > Pay merchants. Who: accounting view reads every step; Prepare, Upload proof and Mark paid need accounting use; Approve needs accounting manage. Data: `GET /finance/payouts/run` (nothing is stored for the run; where it stands is read from the payouts). The page opens on the step the payouts say is next; any step can be opened from the step chips. The money rules are in "Merchant money, payouts".

To show a full run the three demo merchants were set to the Daily schedule (on 11/11/2026, a Wednesday, the weekly Sunday schedule makes nobody due) and two merchants got bank details.

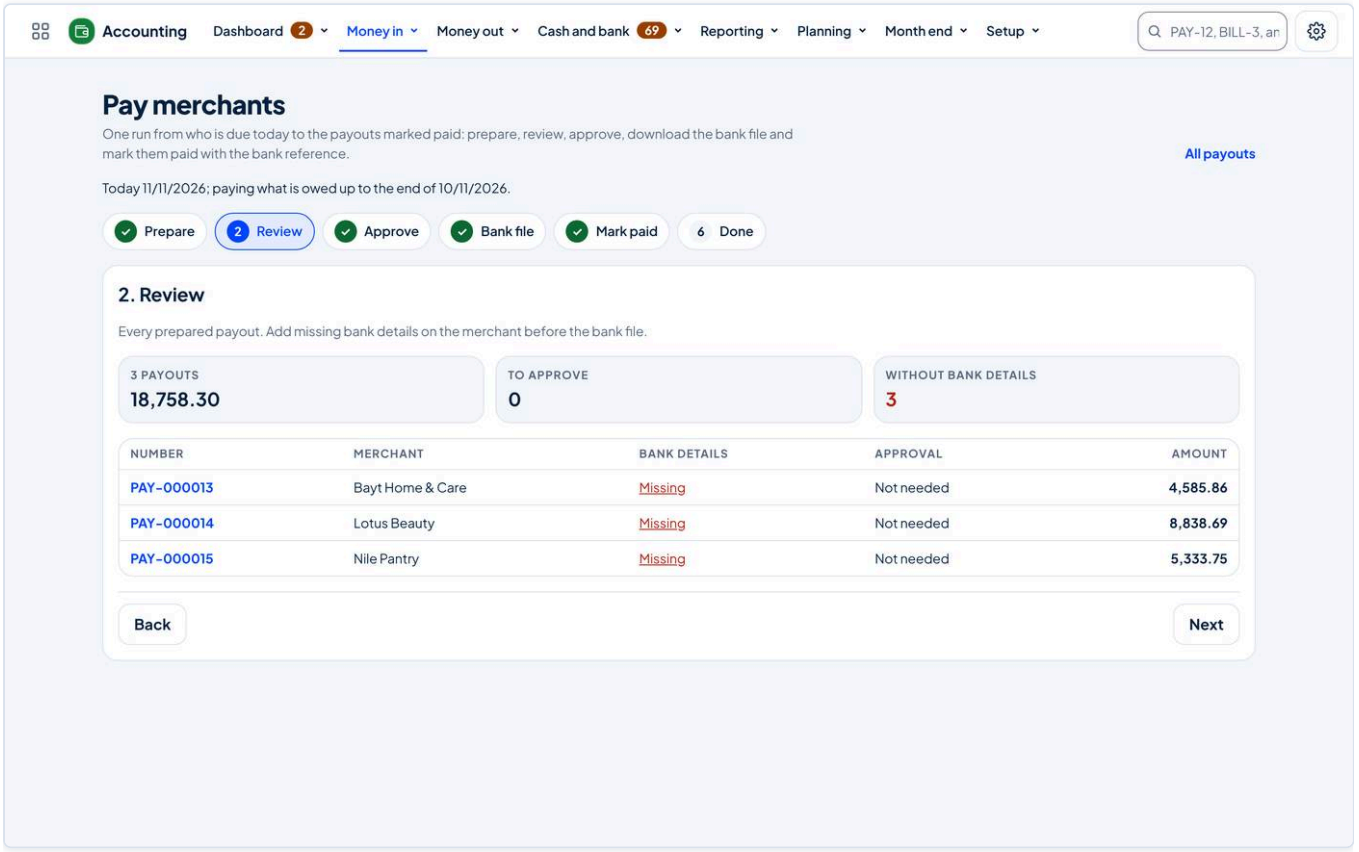


Step 1 Prepare with numbered parts: who is due today, what is owed up to the cutoff day and whether bank details are ready. Address `/finance/payouts/run?step=prepare`.

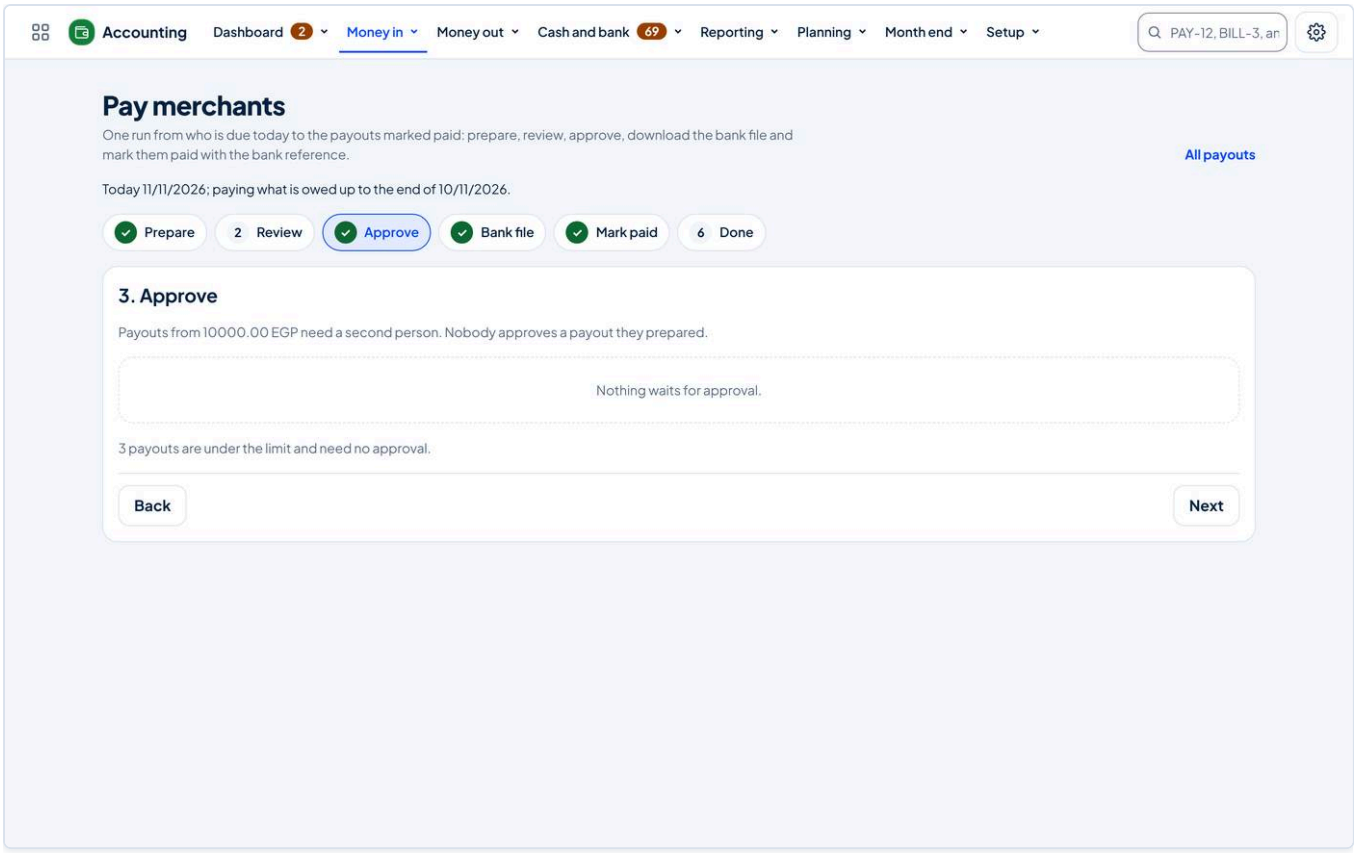
#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Title and intro	"Pay merchants"	None
2	All payouts	Blue link	Opens `/finance/payouts`
5	Today line	"Today 11/11/2026; paying what is owed up to the end of 10/11/2026."	None
4	Step chips	Prepare, Review, Approve, Bank file, Mark paid, Done. The open step has a blue outline and a blue number; a step with nothing waiting shows a green tick; a step with work shows its grey number	Opens that step (`?step=`)
(in the card)	Owed up to the end of	Date box, default yesterday, never after today	Changes the cutoff (`?cutoff=`)
6	Prepare n payouts	Blue button; "Nothing to prepare" when none is ready	Prepares a payout for every row in state To prepare
7	Table	Merchant (blue link to the wallet), Schedule, Owed, Minimum, Bank details (green "Ready" or red "Missing" pill), State	Merchant opens its statement
8	State	To prepare, Prepared, Paid today, Below the minimum, Nothing owed	None
9	Next	White button (Back on later steps)	Next step



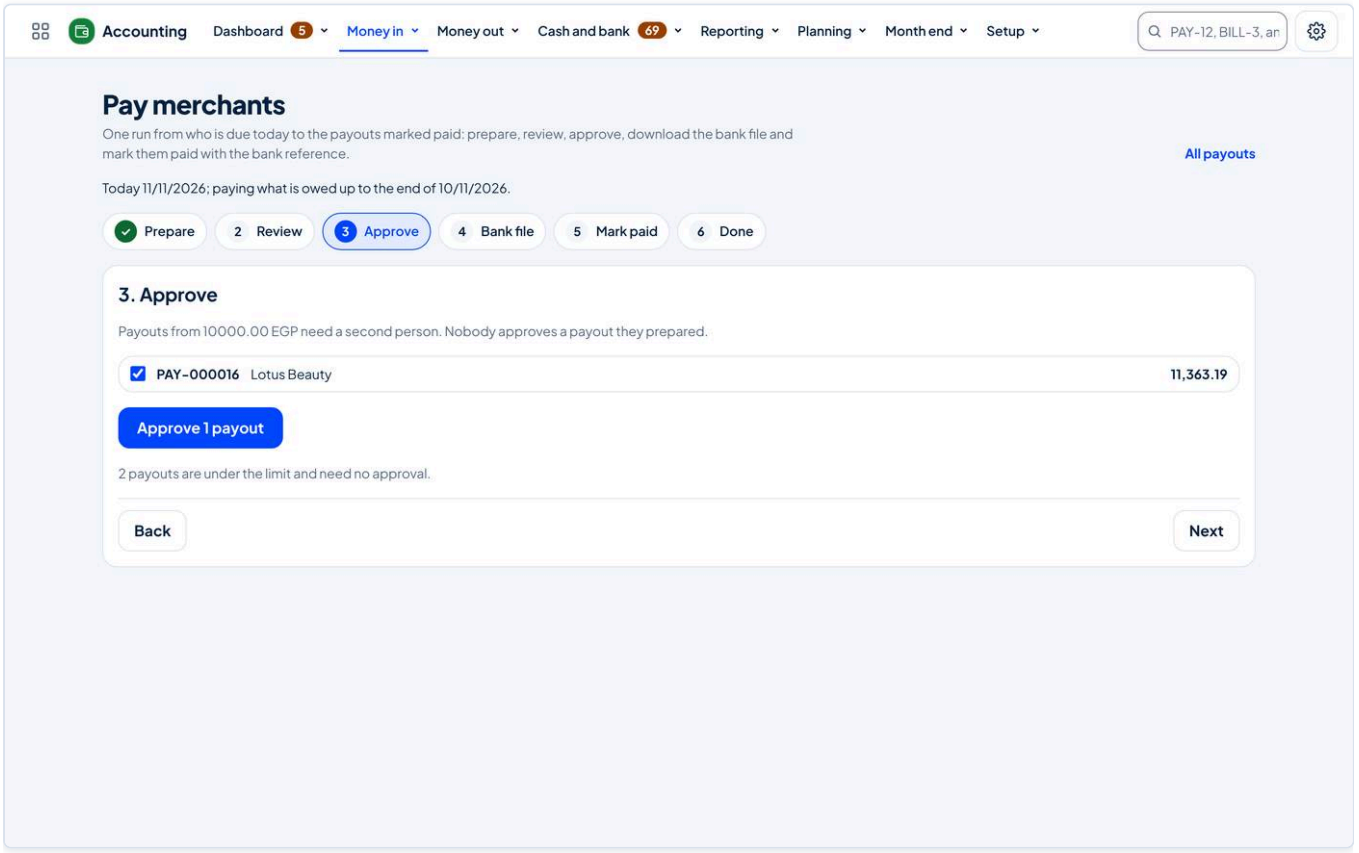
After Prepare 3 payouts: the message "3 prepared, 0 skipped." and the rows now read Prepared.



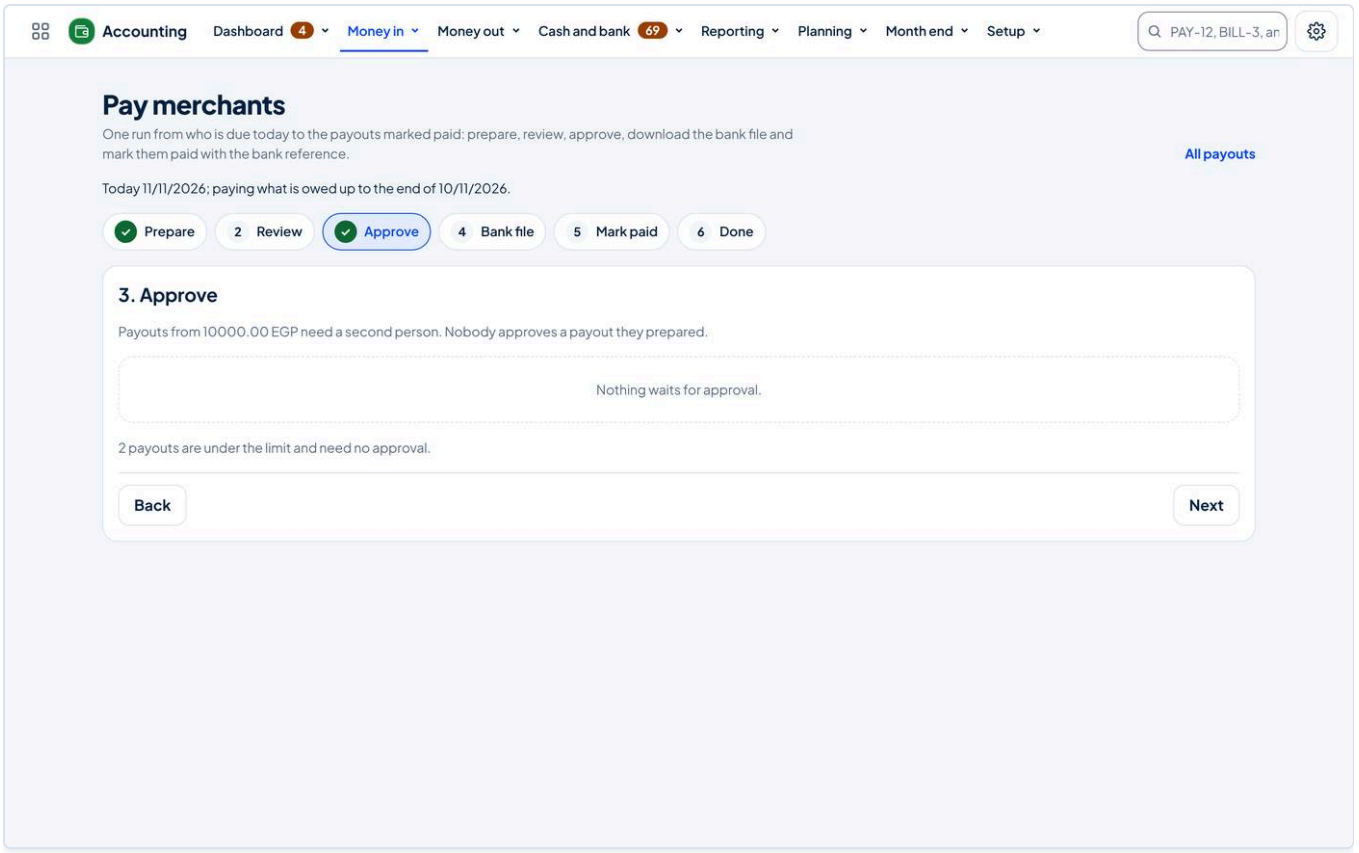
Step 2 Review: three stat boxes (n payouts with the total, To approve, Without bank details in red), then each payout with Bank details (red underlined "Missing" links to the merchant) and Approval (Not needed, Waits for approval, Approved by name).



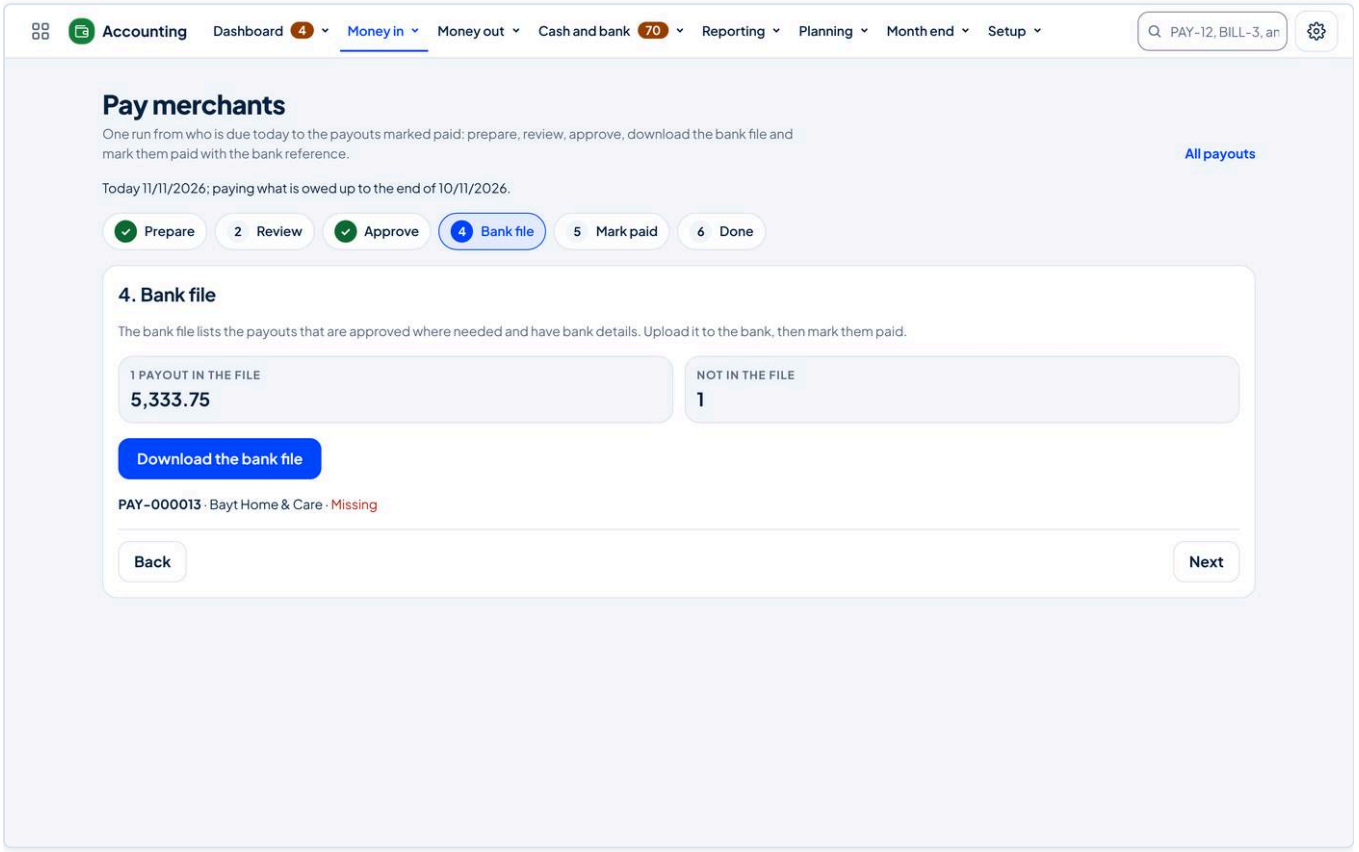
Step 3 Approve when nothing waits: the dashed box "Nothing waits for approval." and "3 payouts are under the limit and need no approval."



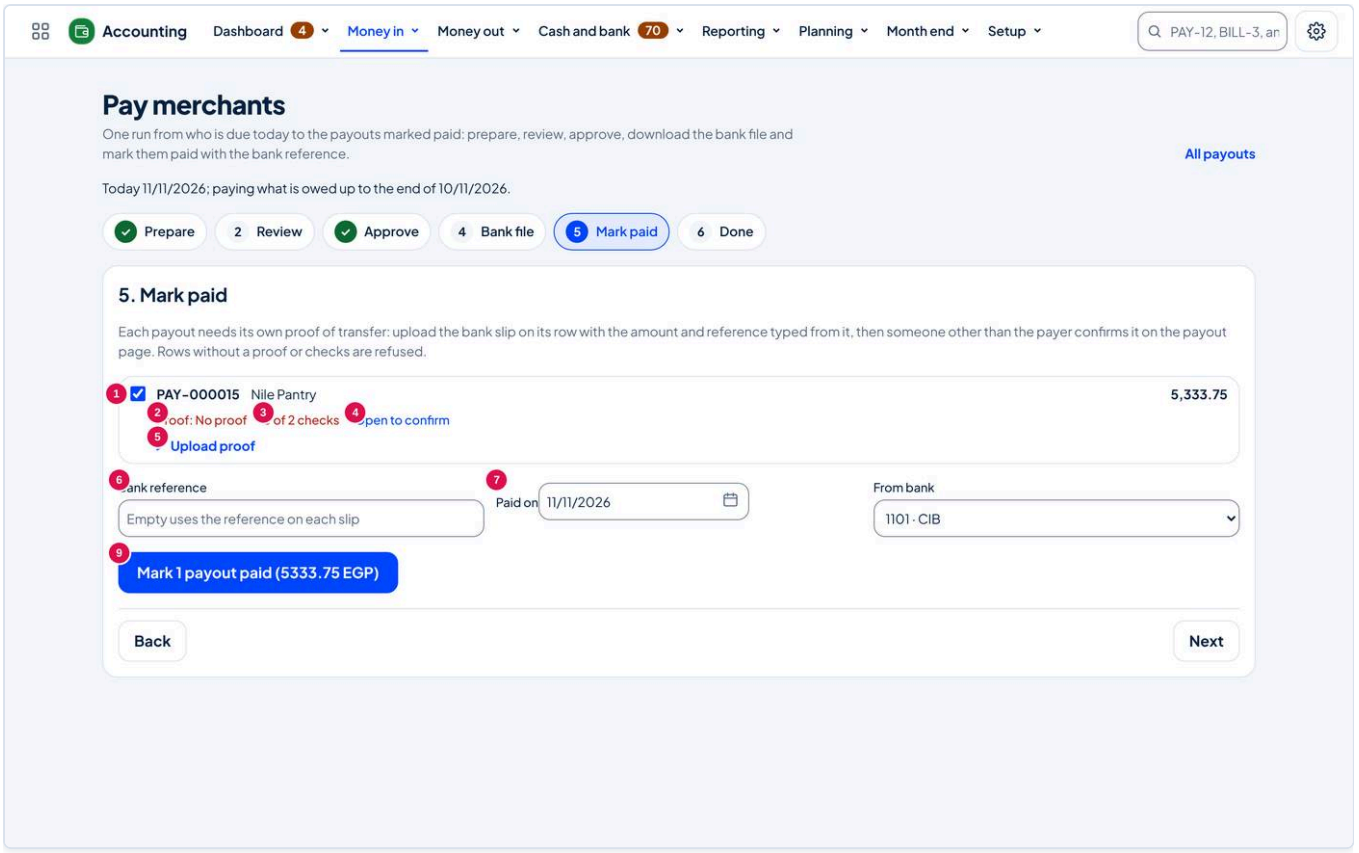
Step 3 Approve as the second finance user: one payout of 11,363.19 EGP is above the 10,000 EGP limit; its checkbox is ticked and the button reads "Approve 1 payout". A payout one prepared shows "You prepared it, so someone else approves it." with no checkbox.



After approving: "1 approved, 0 refused."

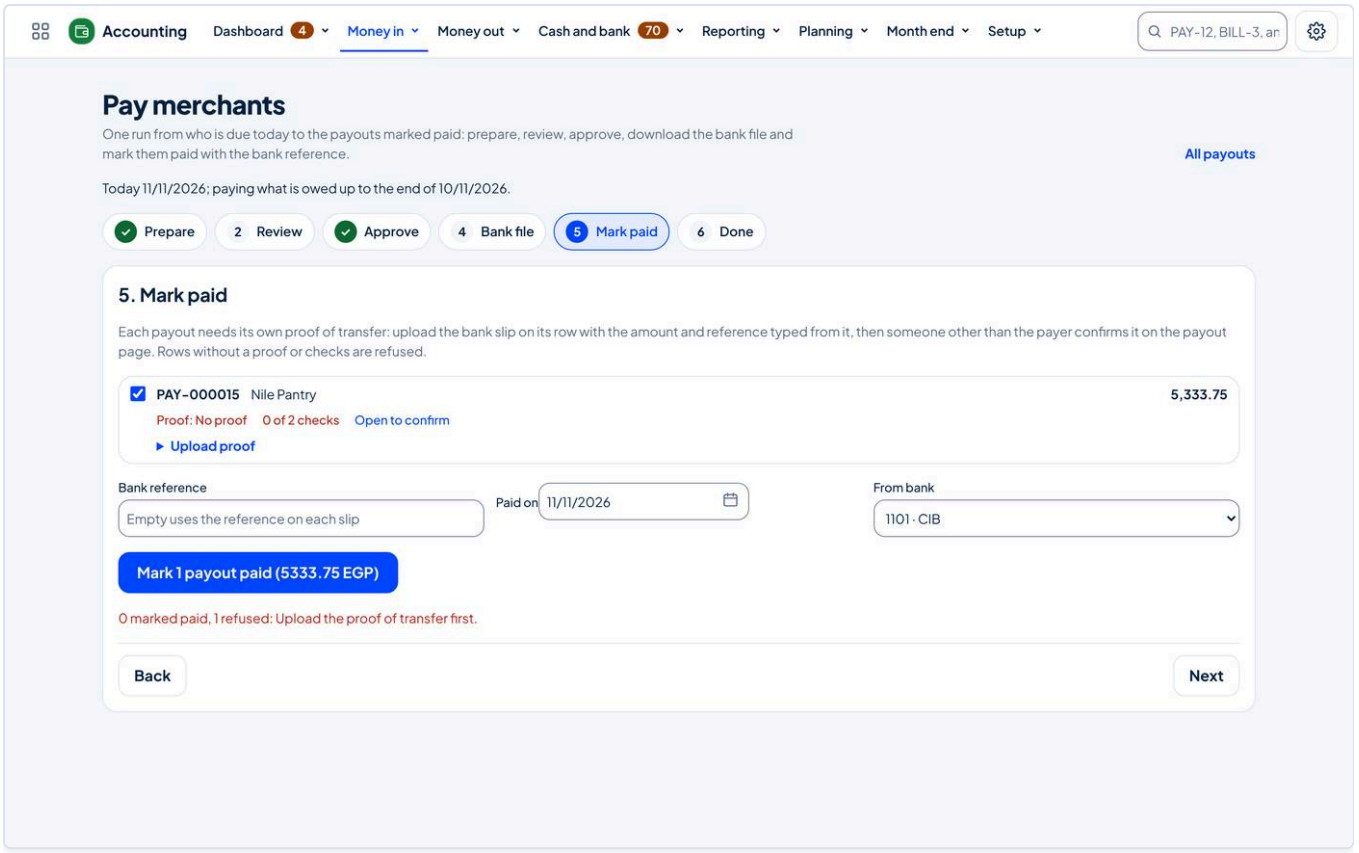


Step 4 Bank file: "1 payout in the file" with its total, "Not in the file 1", Download the bank file (blue) and below each payout left out with the reason (red "Missing" for bank details, or Waits for approval).

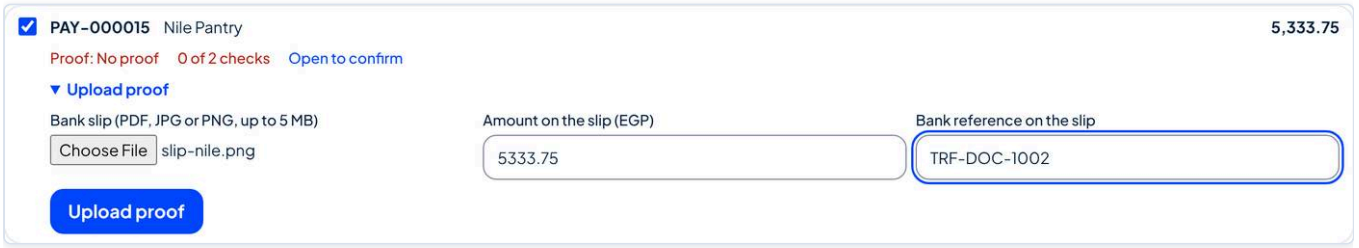


Step 5 Mark paid with numbered parts (legend below).

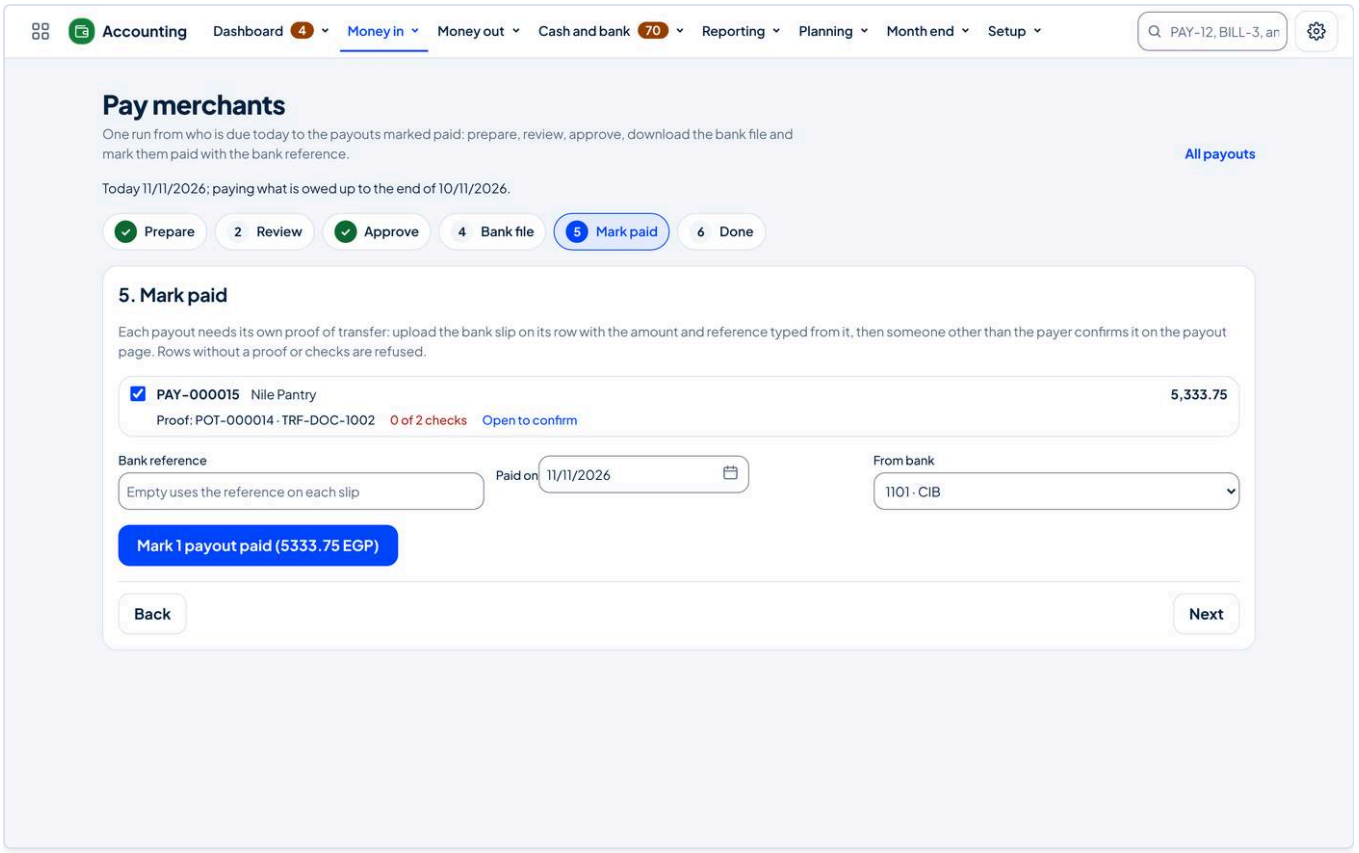
#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Row checkbox	Ticked by default for every payout ready to pay (approved where needed, bank details present)	Includes or leaves out the row
2	Proof	"Proof: `POT-000014 · TRF-DOC-1002` ", or red "Proof: No proof"	None
3	Checks	"n of m checks", red until all are ticked	None
4	Open to confirm	Blue link	Opens the payout page at its Statement, proof and checks section
5	Upload proof	Blue fold, shown while the row has no proof	Opens the compact proof form (file, amount prefilled, reference)
6	Bank reference	"Empty uses the reference on each slip"	One reference for all ticked rows
7	Paid on	Date box, default today, never in the future	A past day is booked at its noon; closed books refuse it
8	From bank	Every open bank account ("1101 · CIB")	The bank the payouts leave from
9	Mark n payouts paid (amount EGP)	Blue button	`POST /finance/payouts/run/mark-paid` : `markPayoutPaid` once per payout in its own transaction



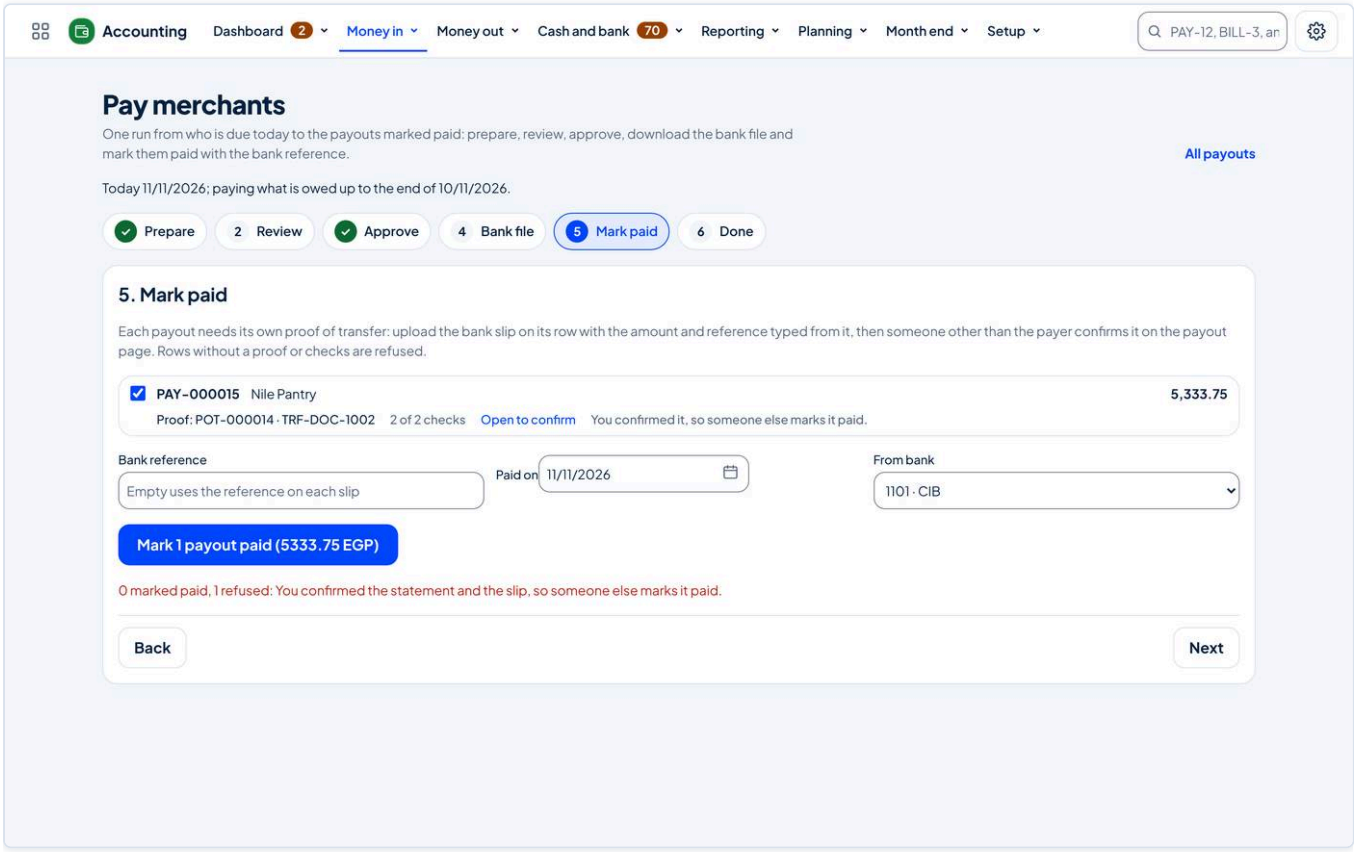
Refusal, no proof: the row shows red "Proof: No proof" and "0 of 2 checks"; Mark 1 payout paid answers in red "0 marked paid, 1 refused: Upload the proof of transfer first."



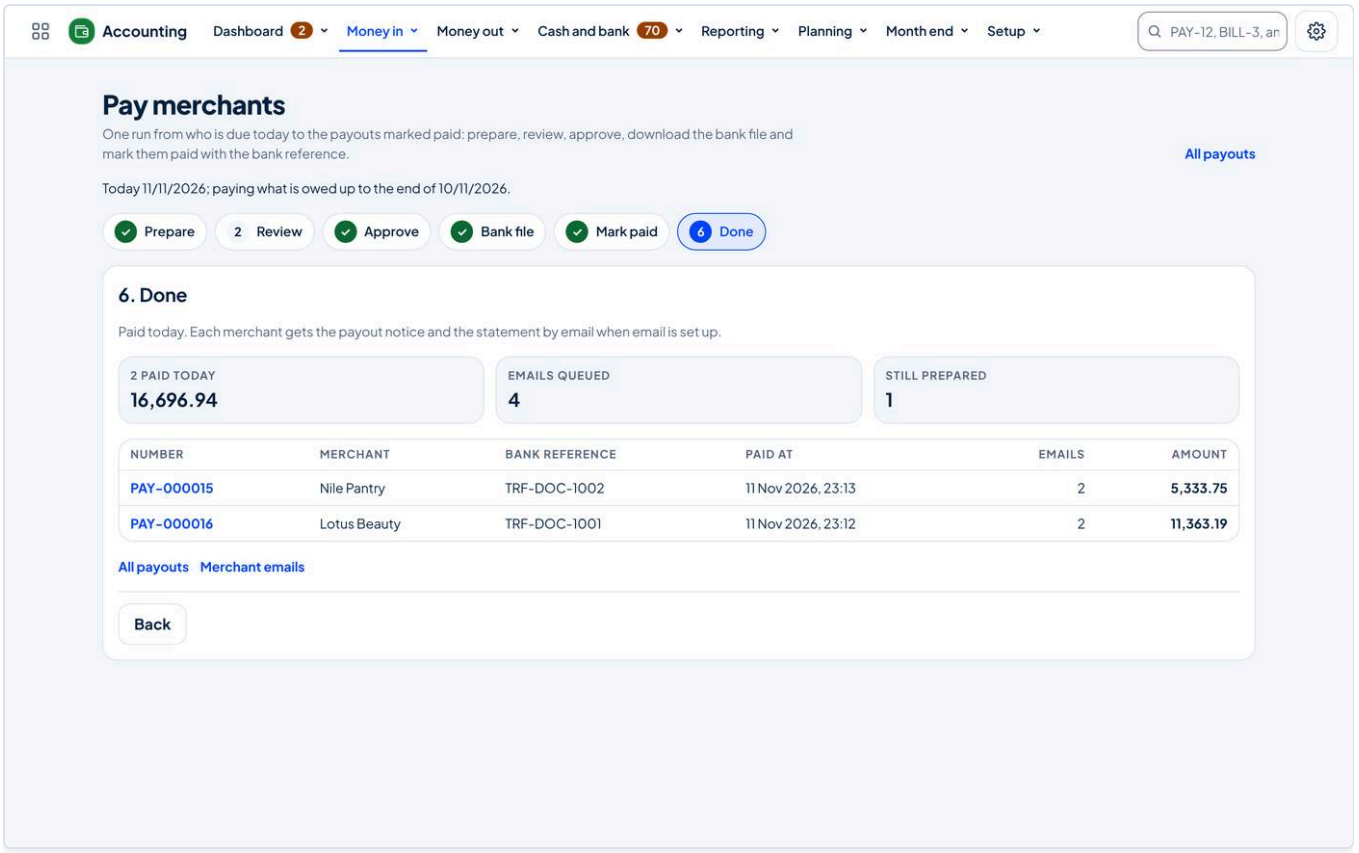
Upload proof on the row: a PNG bank slip chosen, the amount prefilled with the payout amount and the reference typed from the slip.



After the upload the row shows the POT number and reference; the checks are still to be ticked on the payout page.



Refusal, same person: the preparer confirmed the statement and slip herself ("You confirmed it, so someone else marks it paid." on the row) and then pressed Mark paid: "0 marked paid, 1 refused: You confirmed the statement and the slip, so someone else marks it paid."

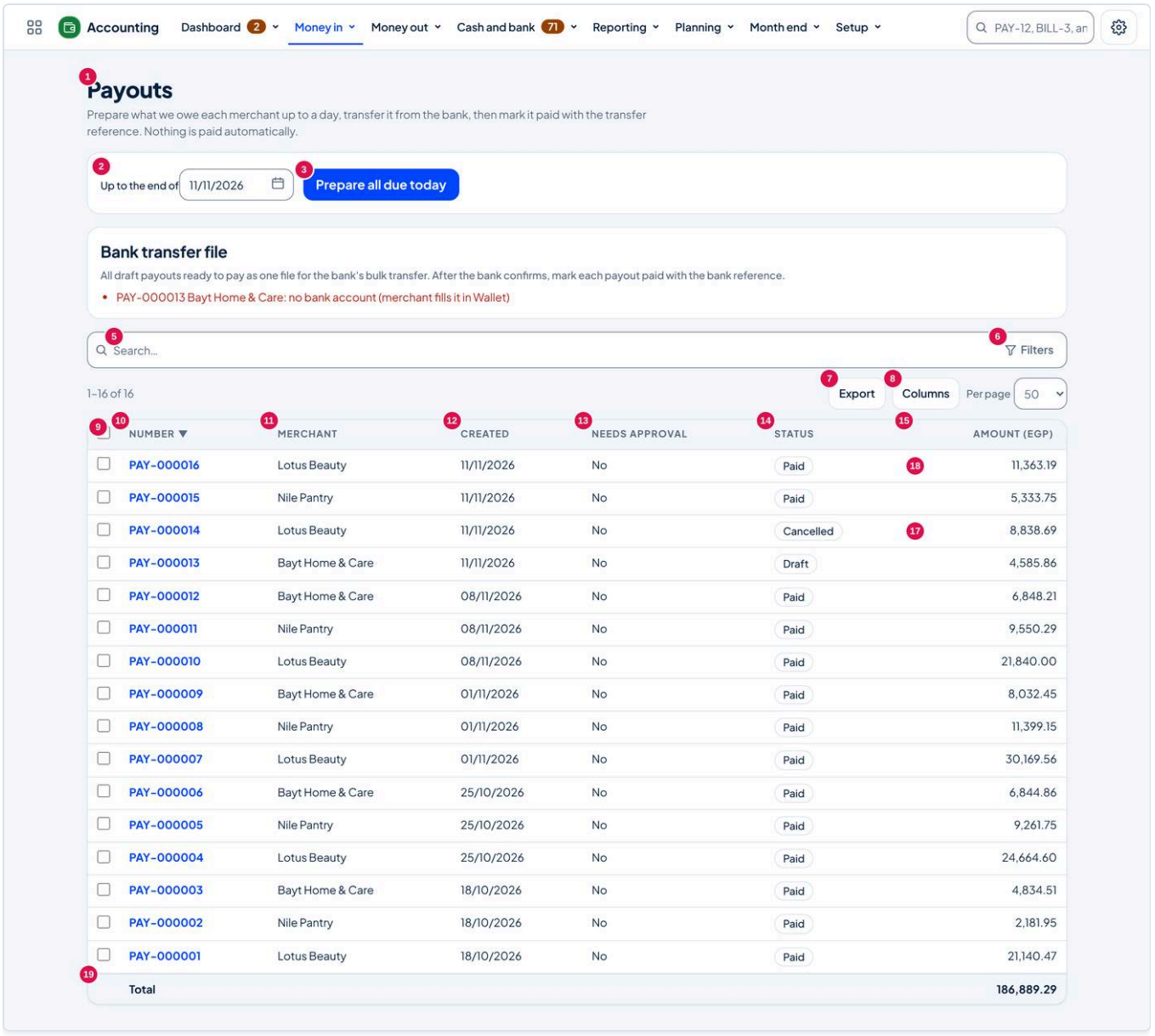


Step 6 Done after the second finance user marked it paid: n paid today with the total, Emails queued, Still prepared, and each paid payout with its bank reference, time and the number of emails queued; links All payouts and Merchant emails.

> **RULE**
>
> Marking paid by transfer needs, beyond the approval and the bank balance: a proof of transfer (`pot_required`), the proof amount equal to the payout (`pot_amount_mismatch`), the frozen statement net equal to the payout (`soa_mismatch`), every active check ticked (`checks_incomplete`) and the second person check ticked by someone other than the payer (`same_person`). These are checked by `assertReadyToPay` in `packages/db/src/finance/payout-proof.ts` .

13.8 Payouts list

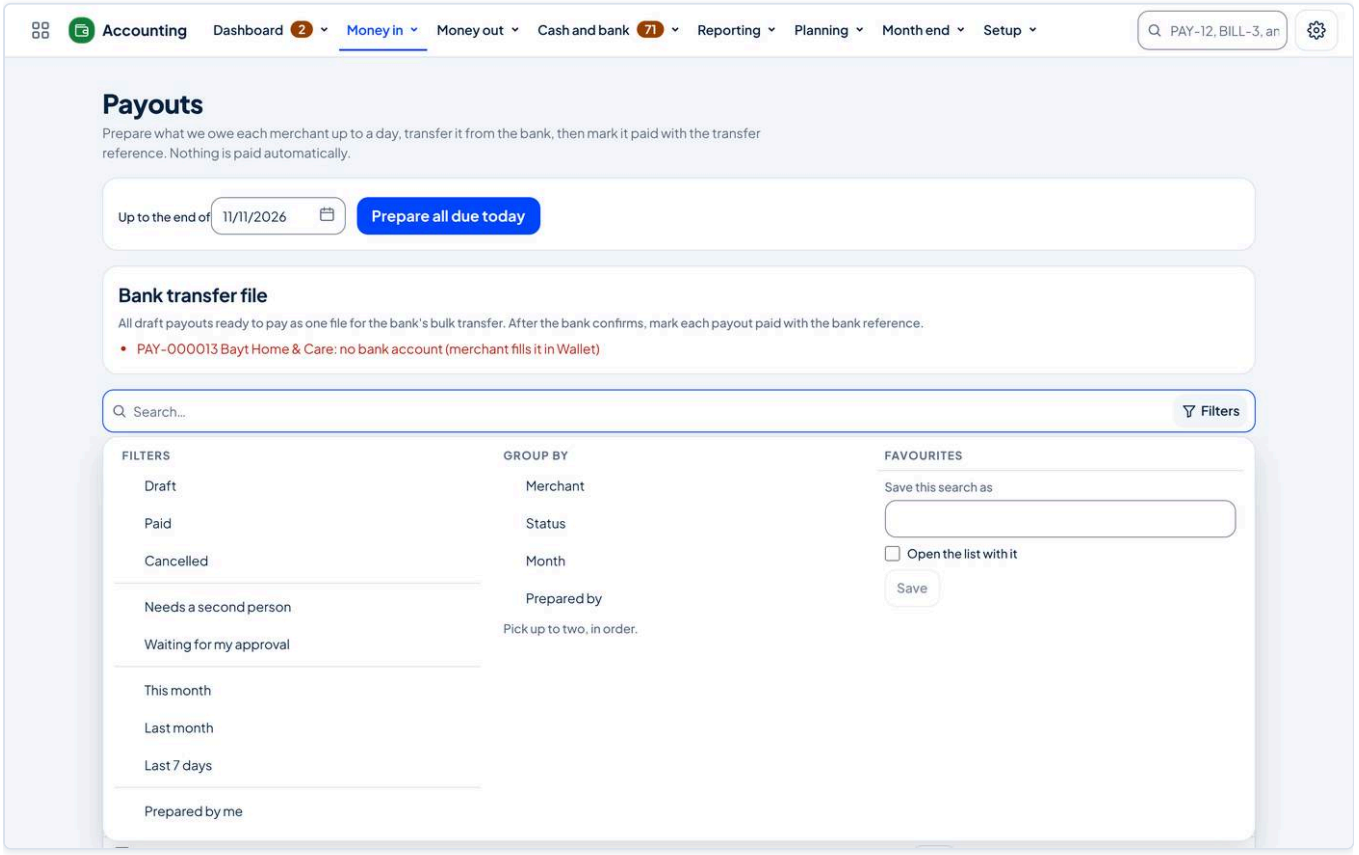
Address: `/finance/payouts` , menu Money in > Payouts. Who: accounting view reads; the prepare form needs accounting use; bulk Approve payouts needs accounting manage. List data: `GET /finance/lists/payouts` .



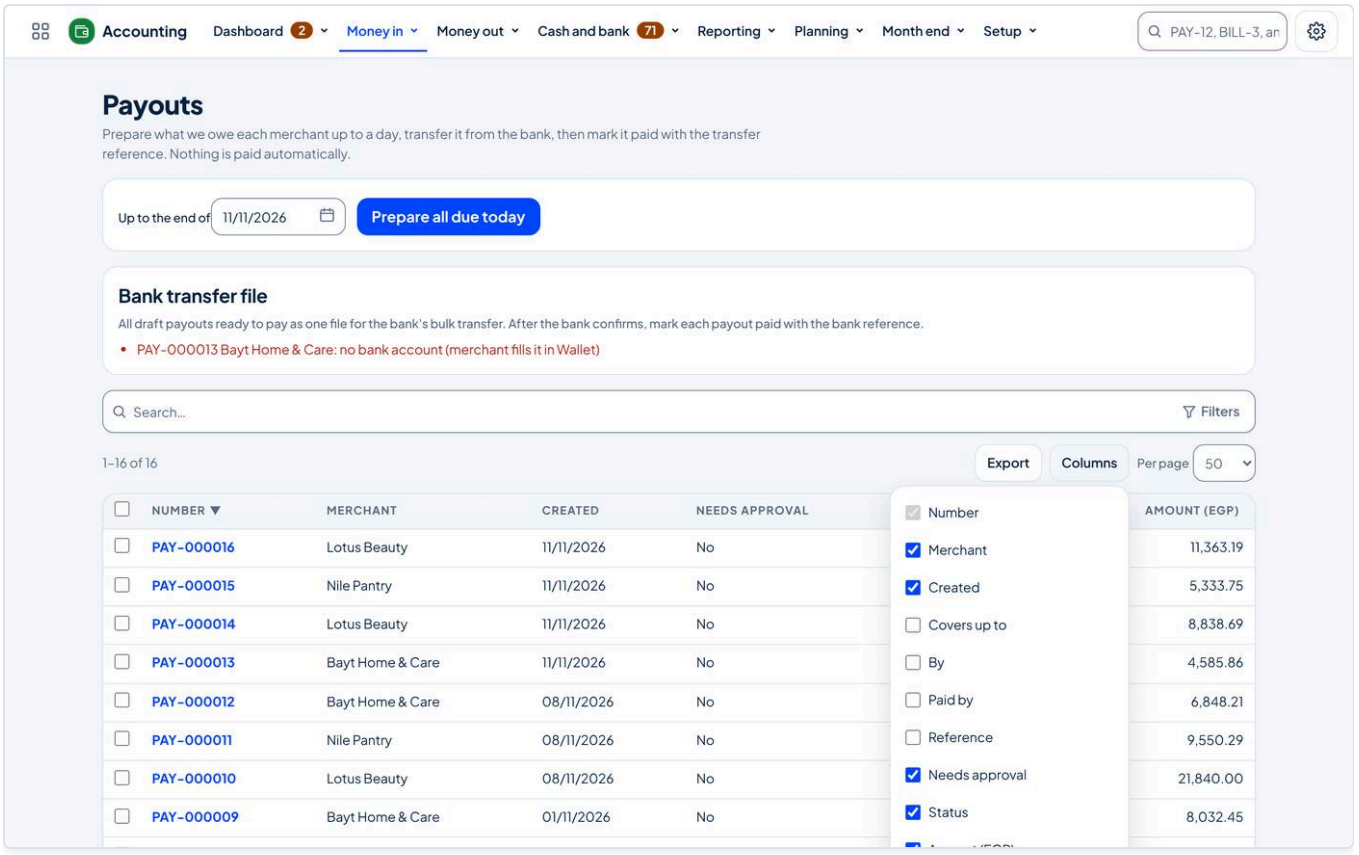
Payouts with every state (Draft, Cancelled, Paid), sorted by number, newest first. Address `/finance/payouts?sort=number&desc=1`.

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Title and intro	"Nothing is paid automatically."	None
2	Up to the end of	Date box, default today, never after today	Sets the cutoff
3	Prepare all due today	Blue button	`POST /finance/payouts/due` : one payout per merchant whose schedule falls today; "n payouts prepared." or "No merchant is due today with money owed."
(card)	Bank transfer file	Explains the bulk file; lists in red each draft left out (`PAY-000013` Bayt Home & Care: no bank account (merchant fills it in Wallet)); a download link when drafts are ready	Downloads the CIB bulk CSV
5	Search	Number, merchant, reference	Adds a Search chip

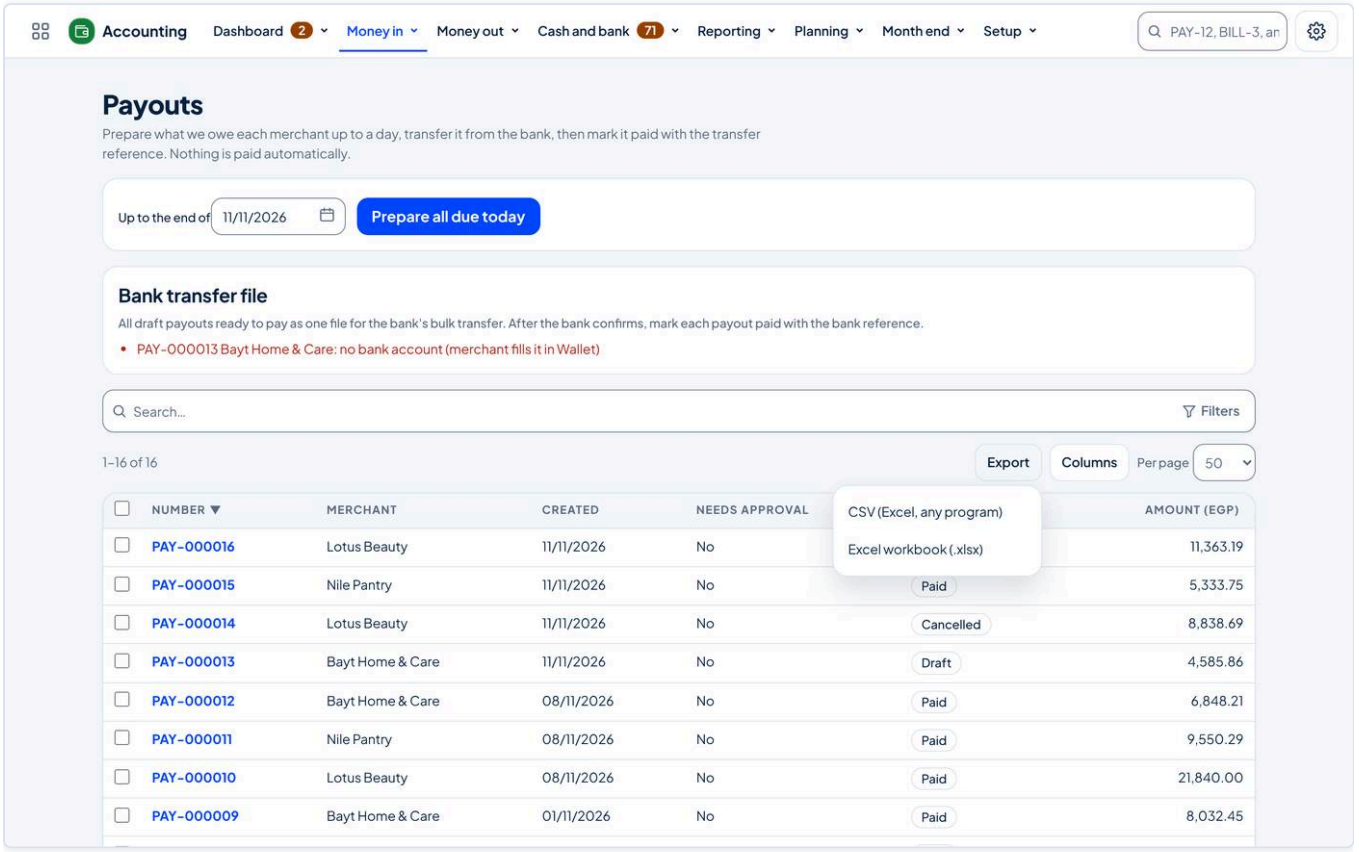
#	Part	What it shows (colors, icons, wording)	What clicking it does
6	Filters	Funnel	Opens the panel
7	Export	White button	CSV or Excel
8	Columns	White button	Optional Covers up to, By, Paid by, Reference
9	Select all	Checkbox	Shows the bulk bar (Approve payouts) for managers
10	Number	`PAY-` in blue; the arrow shows the sort	Opens the payout page
11	Merchant	Name	Sorts
12	Created	dd/mm/yyyy	Sorts
13	Needs approval	Yes while a draft still waits for a second person; No otherwise (also after it was approved)	None
14	Status	Pill	None
17	Cancelled pill	Grey outlined pill "Cancelled"	None
18	Paid pill	Grey outlined pill "Paid"; a draft shows "Draft"	None
15	Amount (EGP)	Right aligned, sortable	Sorts
19	Total	Sum over every passing row	None



Filters: Draft, Paid, Cancelled; Needs a second person, Waiting for my approval; This month, Last month, Last 7 days; Prepared by me. Group by Merchant, Status, Month, Prepared by. Favourites.



The Columns chooser: optional columns can be shown or hidden; the choice is kept per person.



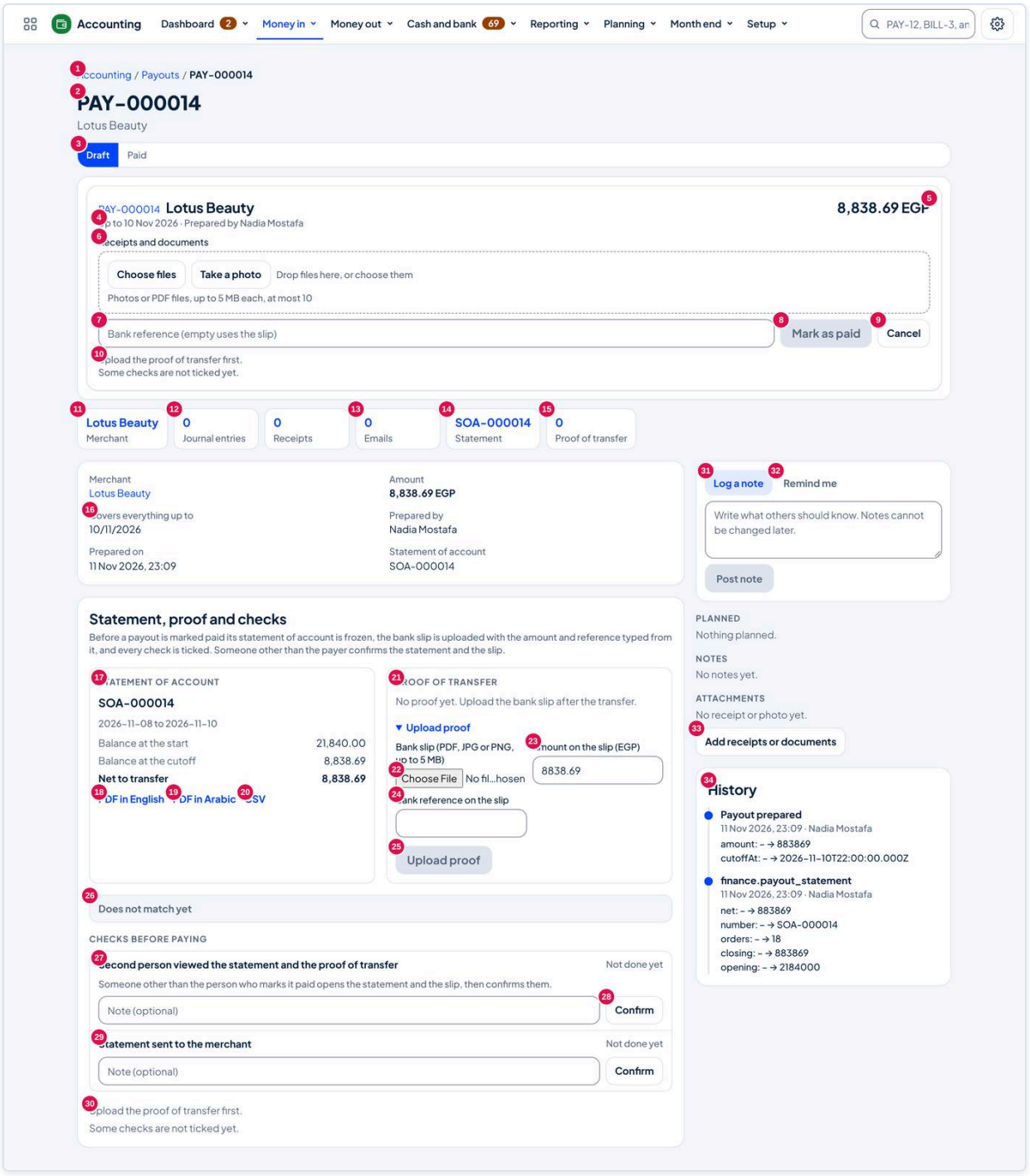
The Export menu: CSV (Excel, any program) and Excel workbook (.xlsx); "Only the n selected" when rows are ticked.

RULE

Bulk Approve payouts (`POST /finance/payouts/bulk-approve`) approves only payouts at or above the limit (`approval_not_needed` : "Below the approval limit: no approval needed") and never one's own.

13.9 A payout page: statement, proof, checks and mark paid

Address: `/finance/payouts/<id>` . Who: accounting view reads; uploading, ticking a check, Mark as paid and Cancel need accounting use; Approve needs accounting manage. Data: `GET /finance/documents/payout/<id>` and `GET /finance/payouts/<id>/controls` . The flow, as the page's own intro says: the statement of account is frozen when the payout is prepared, the bank slip is uploaded with the amount and reference typed from it, every check is ticked, and someone other than the payer confirms the statement and the slip.



A draft payout with numbered parts, before any proof. Address `/finance/payouts/<id>`.

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Breadcrumbs	Accounting / Payouts / `PAY-000014`	Back to the list with its filters
2	Title	Payout number and merchant	None
3	Status bar	Draft (blue filled) then Paid; Cancelled appears as a red segment at the end	None
4	Cutoff line	"Up to 10 Nov 2026 · Prepared by name"	None
5	Amount	Large EGP amount	None

#	Part	What it shows (colors, icons, wording)	What clicking it does
(above 6)	Approval banner	Amber outlined box "This payout is above the approval limit: another person must approve it before it is paid." with a blue Approve button; after approval "Approved by name"	Approve: `POST /finance/payouts/<id>/approve`
6	Receipts and documents	Optional extra files (Choose files, Take a photo, drop zone)	Attached to the payout entry when paid
7	Bank reference	"Bank reference (empty uses the slip)"	Overrides the slip's reference
8	Mark as paid	Blue button, greyed while anything blocks (approval, proof, checks, same person)	`POST /finance/payouts/<id>/paid`
9	Cancel	White button	Opens "Why cancel?" and a red Cancel
10	Blockers	Grey lines: "Upload the proof of transfer first.", "Some checks are not ticked yet."	None
11	Smart button Merchant	Merchant name in blue	Opens the merchant page
12	Journal entries	Count of entries (1 once paid)	Opens them
13	Emails	Emails queued for this payout	Opens Emails to merchants
14	Statement	The `SOA-` number	Opens the statement section
15	Proof of transfer	0 or the `POT-` number	Opens the proof section
16	Fields	Merchant, Amount, Covers everything up to, Prepared by, Prepared on, Statement of account; later Approved by, Paid by, Paid on, Reference, Proof of transfer, Reference on the slip, Note (cancel reason)	Merchant opens its page
17	Statement of account box	`SOA-` number, period (`yyyy-mm-dd` to `yyyy-mm-dd`), Balance at the start, Balance at the cutoff, "Charges after the cutoff, held back" when not zero, Net to transfer in bold	None
18	PDF in English	Blue link	Downloads `statement.pdf?lang=en`
19	PDF in Arabic	Blue link	Downloads `statement.pdf?lang=ar`
20	CSV	Blue link	Downloads `statement.csv`
21	Proof of transfer box	"No proof yet. Upload the bank slip after the transfer."; once uploaded the `POT-` number, amount, reference, "Uploaded by name, time" and View slip; for a cheque payout "Paid by cheque: the cheque register is its proof."	None

#	Part	What it shows (colors, icons, wording)	What clicking it does
22	Bank slip	File input "Bank slip (PDF, JPG or PNG, up to 5 MB)"	Chooses the file
23	Amount on the slip (EGP)	Prefilled with the payout amount; the person checks it against the slip	None
24	Bank reference on the slip	Text, at least 2 characters	None
25	Upload proof	Blue button, greyed until a file is chosen and the reference has 2 characters; "Replace proof" once a proof exists	`POST /finance/payouts/<id>/proof` with the file as the body
26	Match line	Grey box "Does not match yet"; green outlined box "✓ Matches: statement net, slip amount and payout amount are equal"	None
27	Second person check	"Second person viewed the statement and the proof of transfer" with the hint below; "Not done yet" or "name, time" with a tick	None
28	Confirm	White button with an optional note field	`POST /finance/payouts/<id>/checks/<checkId>`
29	Other checks	Checks finance added in Setup > Payout checks (here "Statement sent to the merchant")	Confirm ticks it
30	Ready line	The same blockers, or "Ready to be marked paid."	None
31	Log a note	Chatter	Append only note
32	Remind me	Chatter	Reminder on a day
33	Add receipts or documents	Attachments	Adds files
34	History	Audit rows: Payout prepared, statement frozen, proof, approve, checks, Payout paid	None

The frozen Statement of Account

The SOA is made by `freezePayoutStatement` in the same transaction as the payout (numbers `SOA-000001` onward, gapless under an advisory lock) from the merchant's own statement: from the previous paid payout's cutoff to this cutoff, per order and by kind with the VAT inside each fee. It never changes afterwards; the PDF says so on its first lines.

Statement of account

SOA-000016 · Payout PAY-000016
Merchant: Lotus Beauty
From 2026-11-08 to the end of 2026-11-11 (Cairo days)
Frozen on 2026-11-11 when the payout was prepared; it never changes.

Summary

Balance at the start	**21,840.00 EGP**
Cash collected for you	**12,303.00 EGP**
Order fees	**-631.54 EGP**
Cash on delivery fees	**-123.03 EGP**
Failed delivery fees	**-30.00 EGP**
Storage fees	**-155.24 EGP**
Payouts	**-21,840.00 EGP**
VAT included in the fees above	**0.00 EGP**
Balance at the cutoff	**11,363.19 EGP**
Net to transfer	**11,363.19 EGP**

Orders

Order	Cash collected	Fees	VAT	Net
SN100539	0.00	-23.63	0.00	-23.63
SN100541	0.00	-42.36	0.00	-42.36
SN100542	320.00	-22.33	0.00	297.67
SN100540	694.00	-31.68	0.00	662.32
SN100543	694.00	-46.68	0.00	647.32
SN100557	220.00	-19.83	0.00	200.17
SN100560	519.00	-27.30	0.00	491.70
SN100561	469.00	-26.05	0.00	442.95
SN100568	994.00	-39.18	0.00	954.82
SN100570	1,392.00	-49.13	0.00	1,342.87
SN100578	0.00	-30.08	0.00	-30.08
SN100579	694.00	-31.68	0.00	662.32
SN100584	444.00	-25.43	0.00	418.57
SN100586	0.00	-29.61	0.00	-29.61
SN100589	1,443.00	-50.40	0.00	1,392.60
SN100588	0.00	-35.22	0.00	-35.22
SN100593	993.00	-39.15	0.00	953.85
SN100594	720.00	-32.33	0.00	687.67
SN100599	0.00	-17.78	0.00	-17.78

Shiply Now · SOA-000016 · page 1 of 2

The SOA PDF in English, page 1: number, payout, merchant, period in Cairo days, "Frozen on ... when the payout was prepared; it never changes.", the Summary (Balance at the start, Cash collected for you, each fee kind, Payouts, VAT included in the fees, Balance at the cutoff, Net to transfer) and the Orders table (Cash collected, Fees, VAT, Net). Made by `payout-statement-pdf.ts`.

كشف حساب

SOA-000016 · الدفعة PAY-000016

التاجر: Lotus Beauty

من 2026-11-08 حتى نهاية 2026-11-11 (بتوقيت القاهرة)

ثُبّت في 2026-11-11 عند تجهيز الدفعة ولا يتغير بعد ذلك.

الملخص

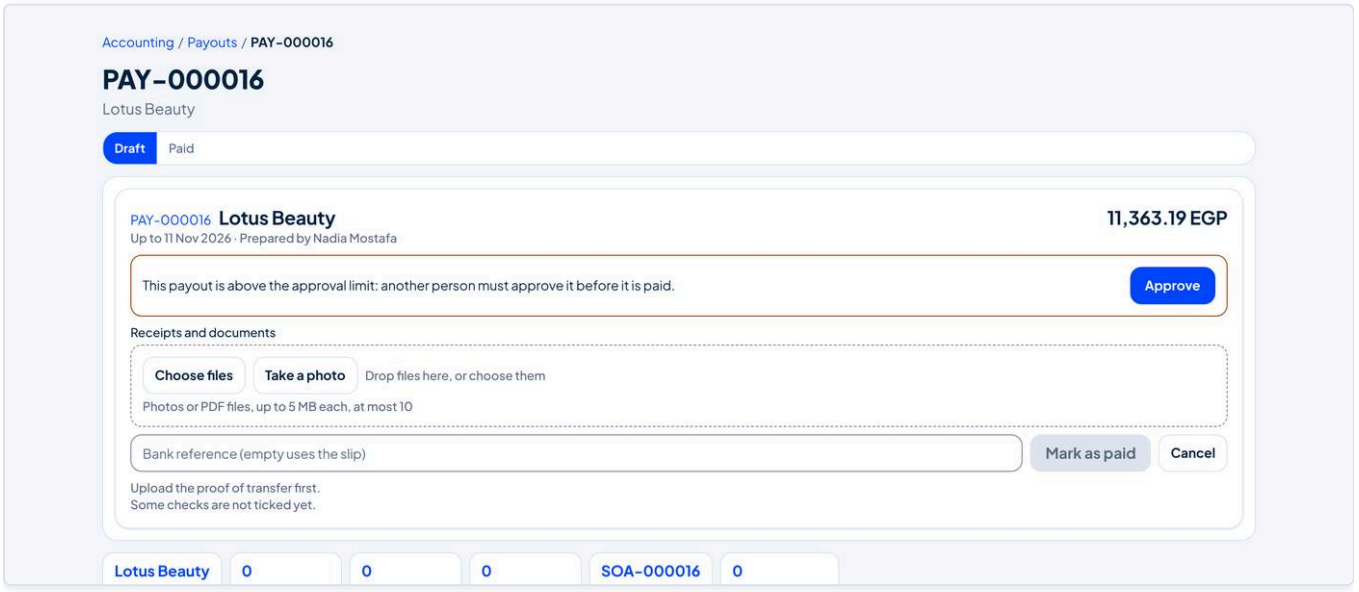
الرصيد في البداية	**21,840.00 ج.م**
نقدية محصلة لك	**12,303.00 ج.م**
رسوم الطلبات	**-631.54 ج.م**
رسوم الدفع عند الاستلام	**-123.03 ج.م**
رسوم التوصيل الفاشل	**-30.00 ج.م**
رسوم التخزين	**-155.24 ج.م**
الدفعات	**-21,840.00 ج.م**
ضريبة القيمة المضافة ضمن الرسوم أعلاه	**0.00 ج.م**
الرصيد عند تاريخ القطع	**11,363.19 ج.م**
الصافي المحوّل	**11,363.19 ج.م**

الطلبات

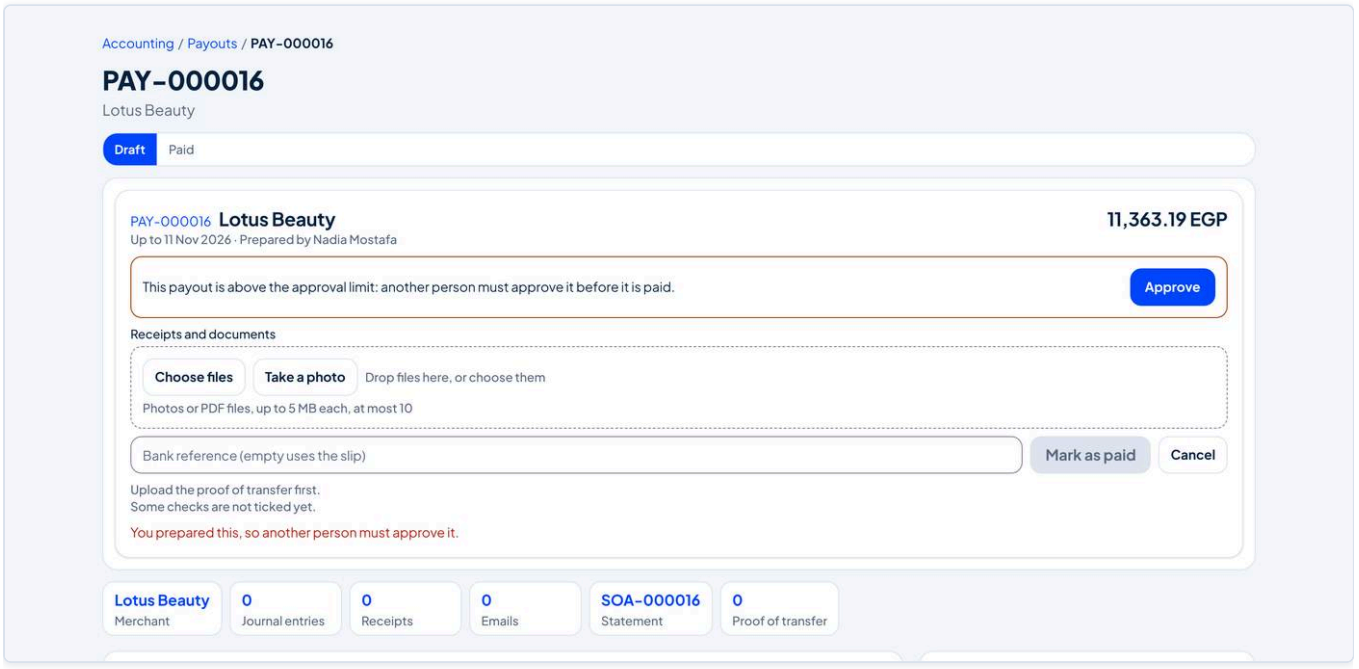
الطلب	النقدية المحصلة	الرسوم	الضريبة	الصافي
SN100539	0.00	-23.63	0.00	-23.63
SN100541	0.00	-42.36	0.00	-42.36
SN100542	320.00	-22.33	0.00	297.67
SN100540	694.00	-31.68	0.00	662.32
SN100543	694.00	-46.68	0.00	647.32
SN100557	220.00	-19.83	0.00	200.17
SN100560	519.00	-27.30	0.00	491.70
SN100561	469.00	-26.05	0.00	442.95
SN100568	994.00	-39.18	0.00	954.82
SN100570	1,392.00	-49.13	0.00	1,342.87
SN100578	0.00	-30.08	0.00	-30.08
SN100579	694.00	-31.68	0.00	662.32
SN100584	444.00	-25.43	0.00	418.57
SN100586	0.00	-29.61	0.00	-29.61
SN100589	1,443.00	-50.40	0.00	1,392.60

The same SOA in Arabic, laid out right to left with IBM Plex Sans Arabic.

Approving a payout above the limit

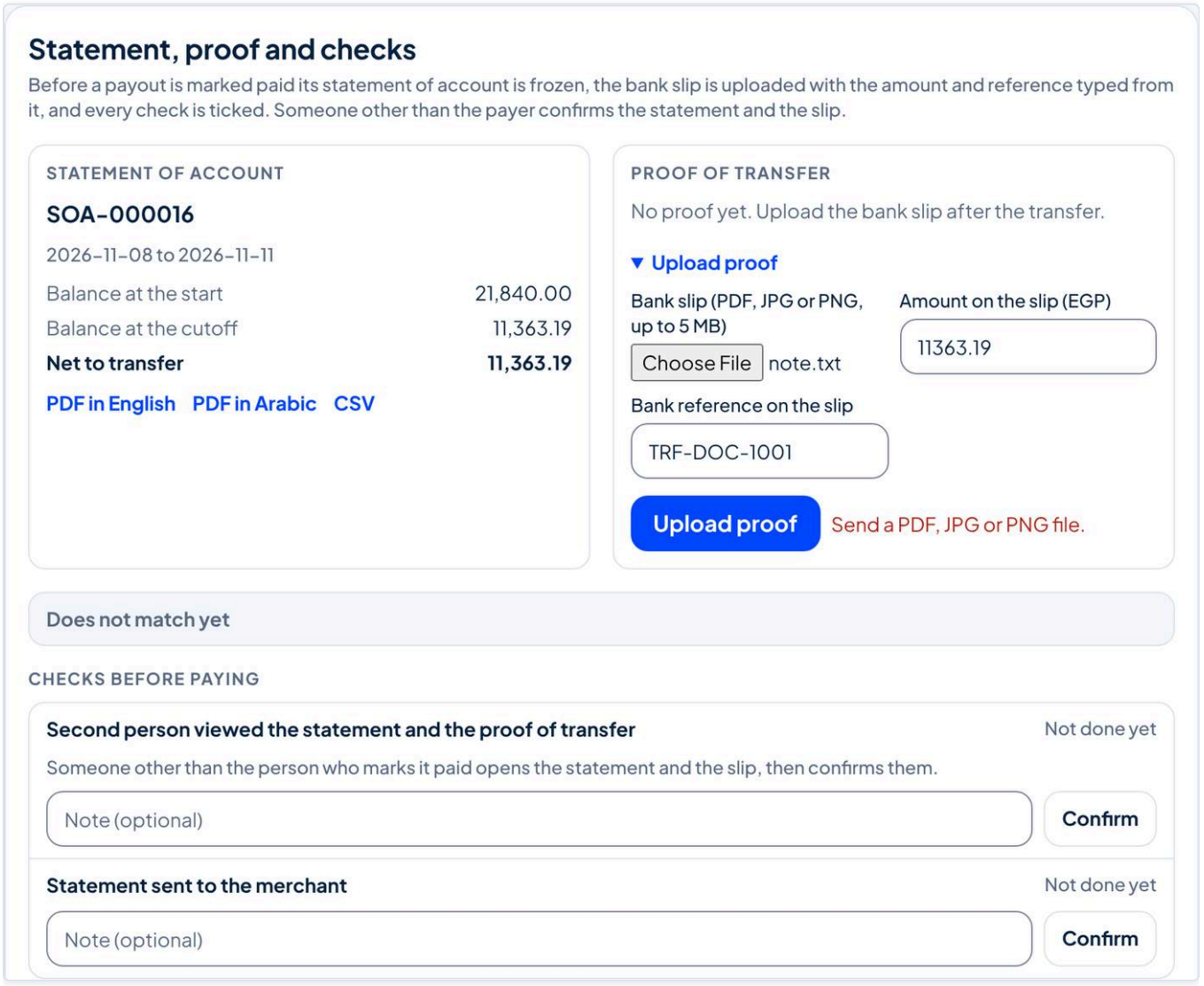


A payout of 11,363.19 EGP prepared from the wallet: the amber approval banner with Approve; Mark as paid is greyed.

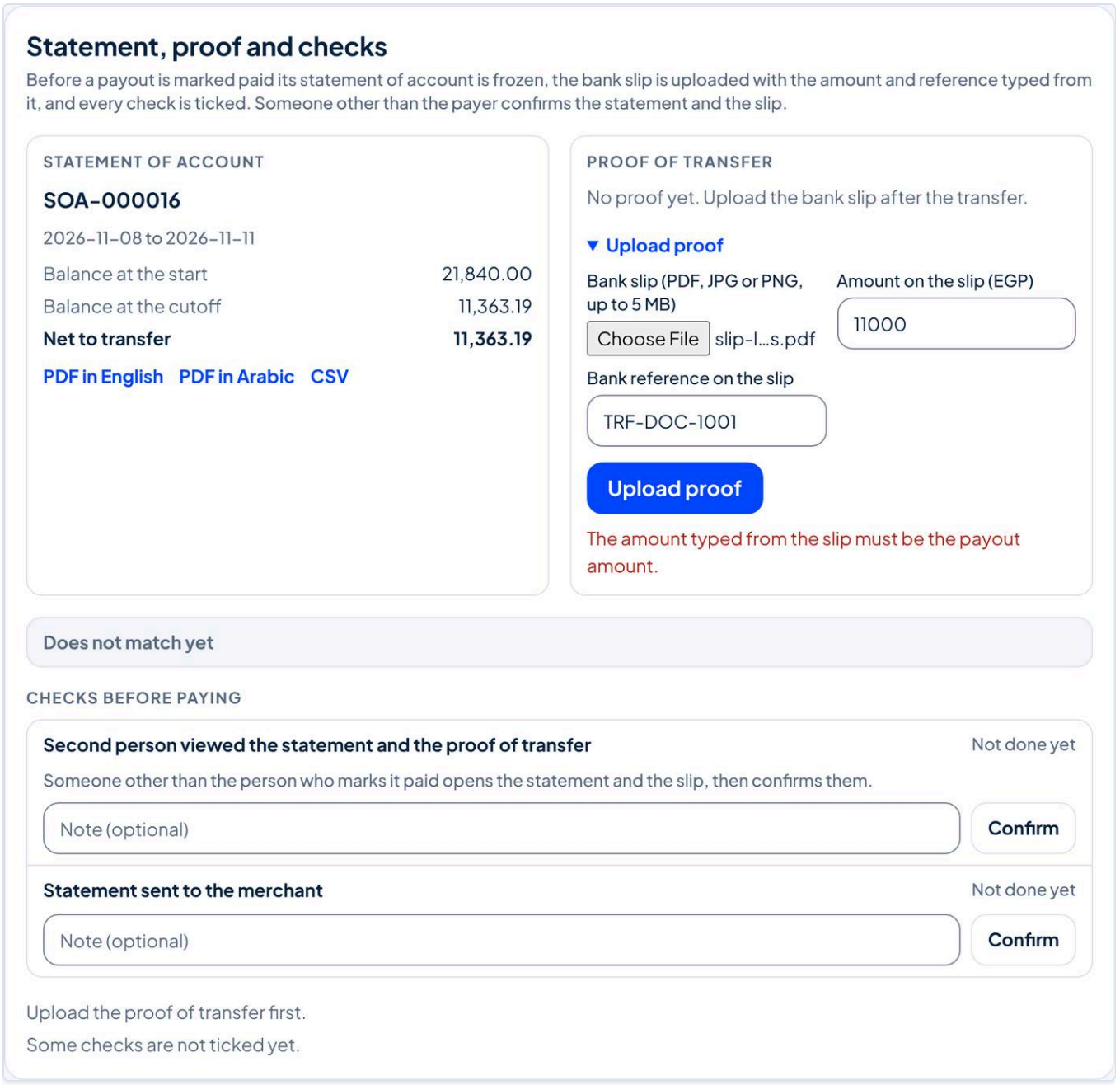


The preparer pressing Approve on her own payout: the red message "You prepared this, so another person must approve it." The second finance user approves it from the run (step 3) or from this page.

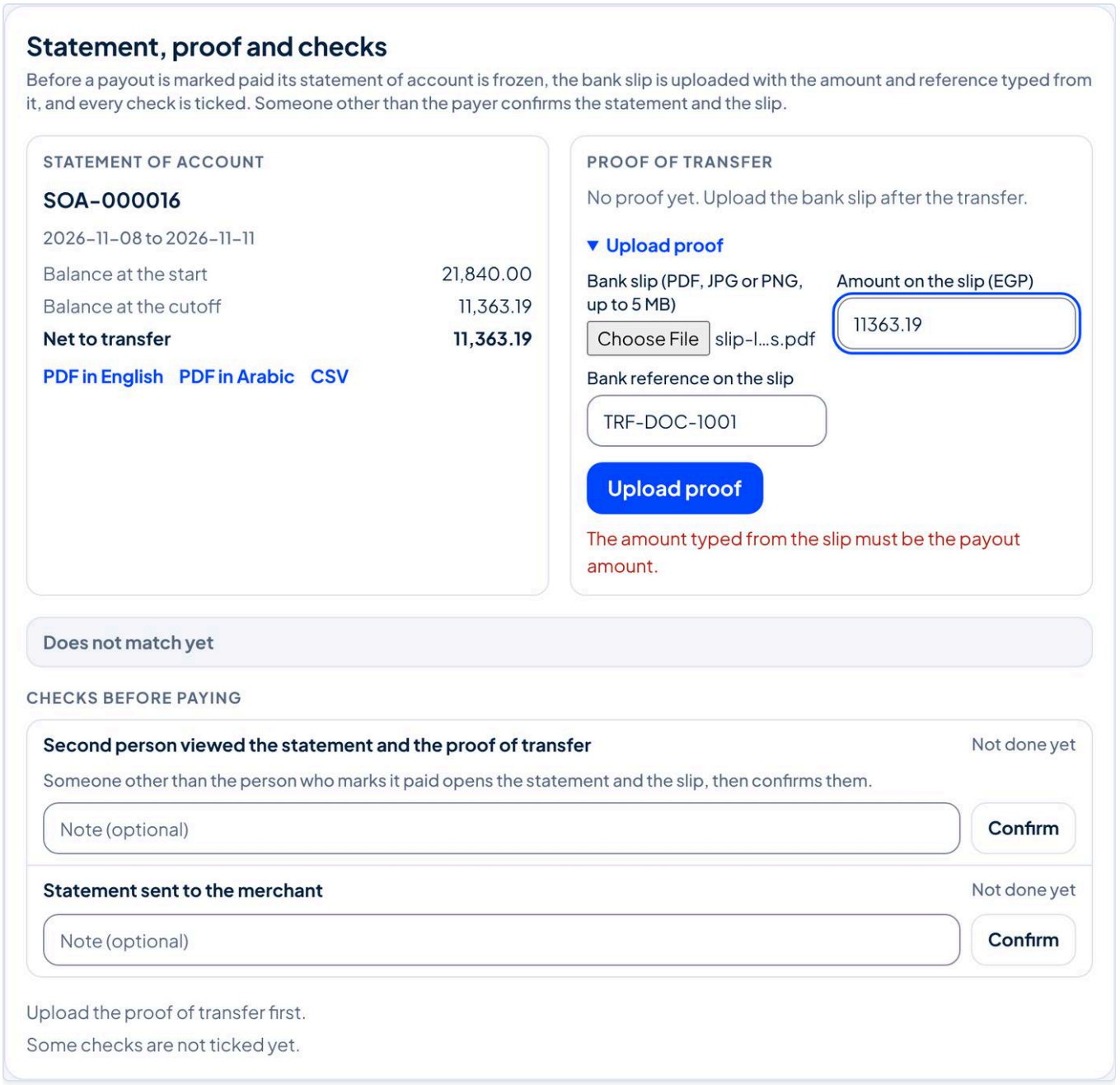
Uploading the Proof of Transfer and its refusals



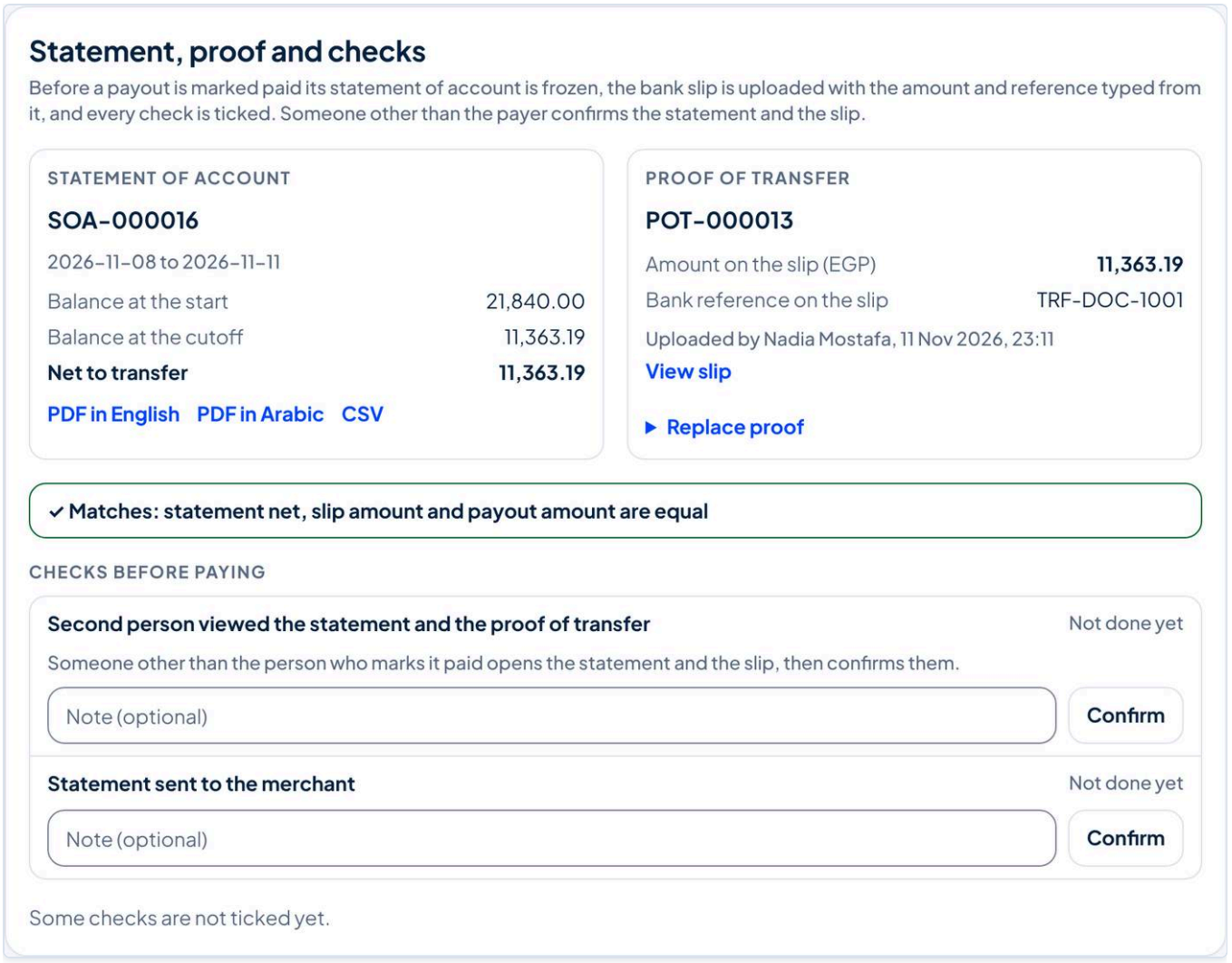
Refusal, wrong file type: a .txt file is chosen; the red message "Send a PDF, JPG or PNG file." (checked in the browser, and again on the server by the file's first bytes).



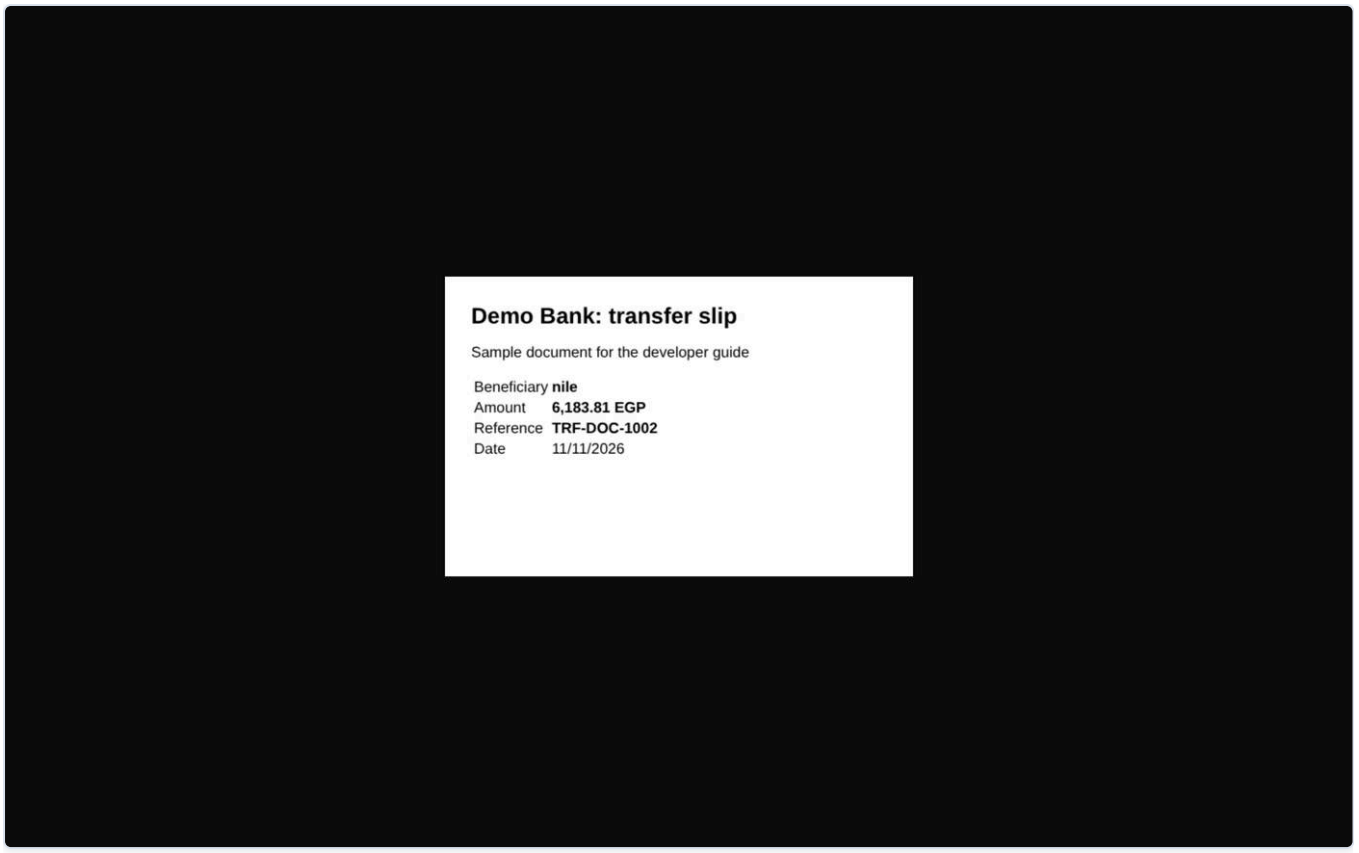
Refusal, amount mismatch: the slip amount typed as 11000 while the payout is 11,363.19; the server answers `pot_amount_mismatch`, shown as "The amount typed from the slip must be the payout amount." Nothing is saved.



The correct entry: the PDF slip chosen, amount 11363.19, reference `TRF-DOC-1001`.



Proof saved as `POT-000013`: amount, reference, "Uploaded by name, time", View slip and the fold Replace proof. The match line turned green: "✓ Matches: statement net, slip amount and payout amount are equal". Still "Some checks are not ticked yet."



View slip opens the stored file in a new tab (`GET /finance/payouts/<id>/proof`); here a PNG slip.

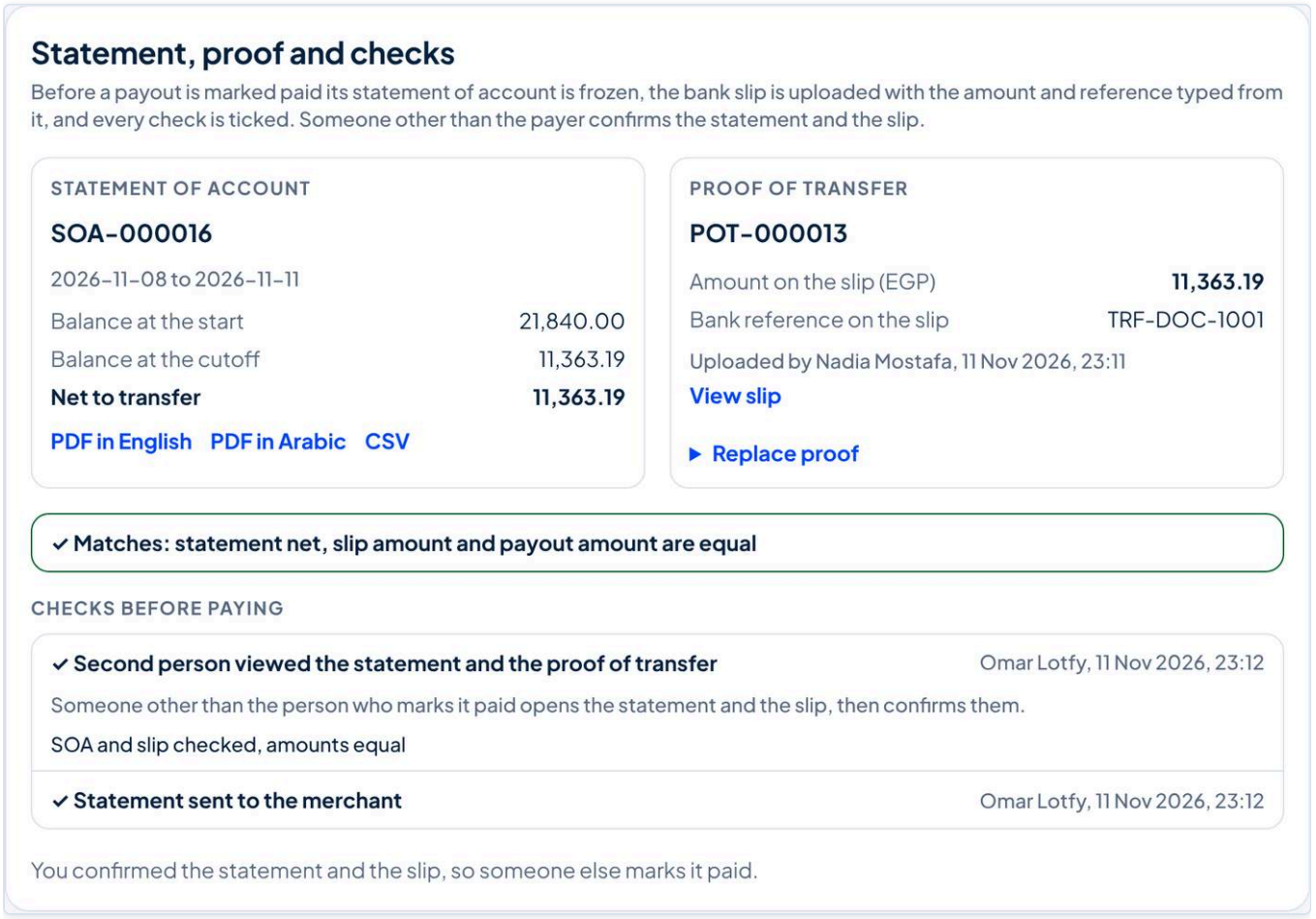
The system compares the amount typed from the slip with the payout; it does not read the image. In the demo the Nile Pantry slip picture shows another amount than the one typed: the second person check exists exactly to catch that, by a person opening the slip.

Refusal	Code	Message on screen
File not PDF, JPG or PNG	`file_type_not_allowed`	Send a PDF, JPG or PNG file.
File over 5 MB or empty	`file_too_large`	The file is larger than 5 MB.
Amount typed is not a number	`invalid_amount` (browser)	Type the amount in EGP, for example 1250.50.
Amount typed differs from the payout	`pot_amount_mismatch`	The amount typed from the slip must be the payout amount.
Reference under 2 characters	`pot_required`	Upload the proof of transfer first.
Payout no longer a draft	`payout_not_draft`	This payout is no longer waiting.

RULE

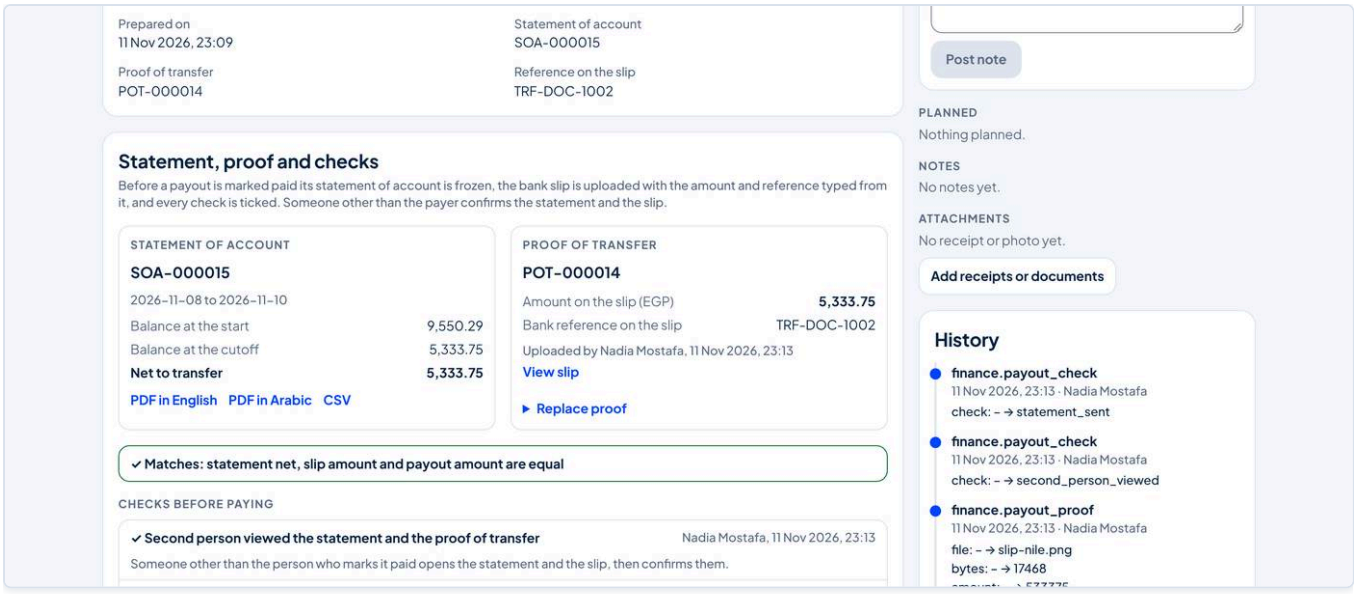
While the payout is a draft a wrong proof can be replaced: its POT number is kept, the file, amount and reference change, and every check already ticked is cleared ("Replacing the proof clears the ticks given on the old one."). Audited as `finance.payout_proof_replace` .

The checks, ticked by the second user



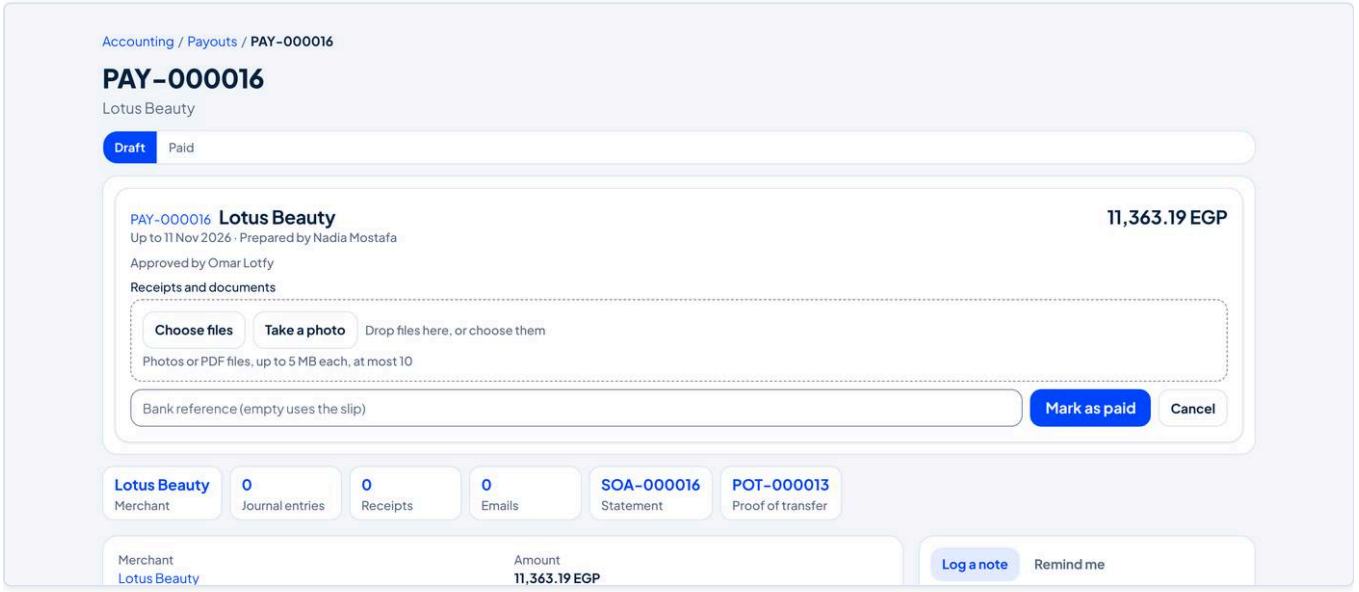
The second finance user opened the statement and the slip, then confirmed both checks with a note. Each check shows a tick, the name and time. The bottom line tells this user "You confirmed the statement and the slip, so someone else marks it paid."

Control	What it does	Rule / limits	Result
Confirm (second person check)	Ticks the check with the optional note	Accounting use; only on a draft; needs the proof and the statement first (`pot_required`); ticking twice changes nothing; cannot be switched off in setup (`check_is_system`)	Audit `finance.payout_check` ; this person can no longer mark the payout paid
Confirm (other checks)	Ticks a check added in setup	Any accounting user; a new check applies to every payout still a draft	Audit row

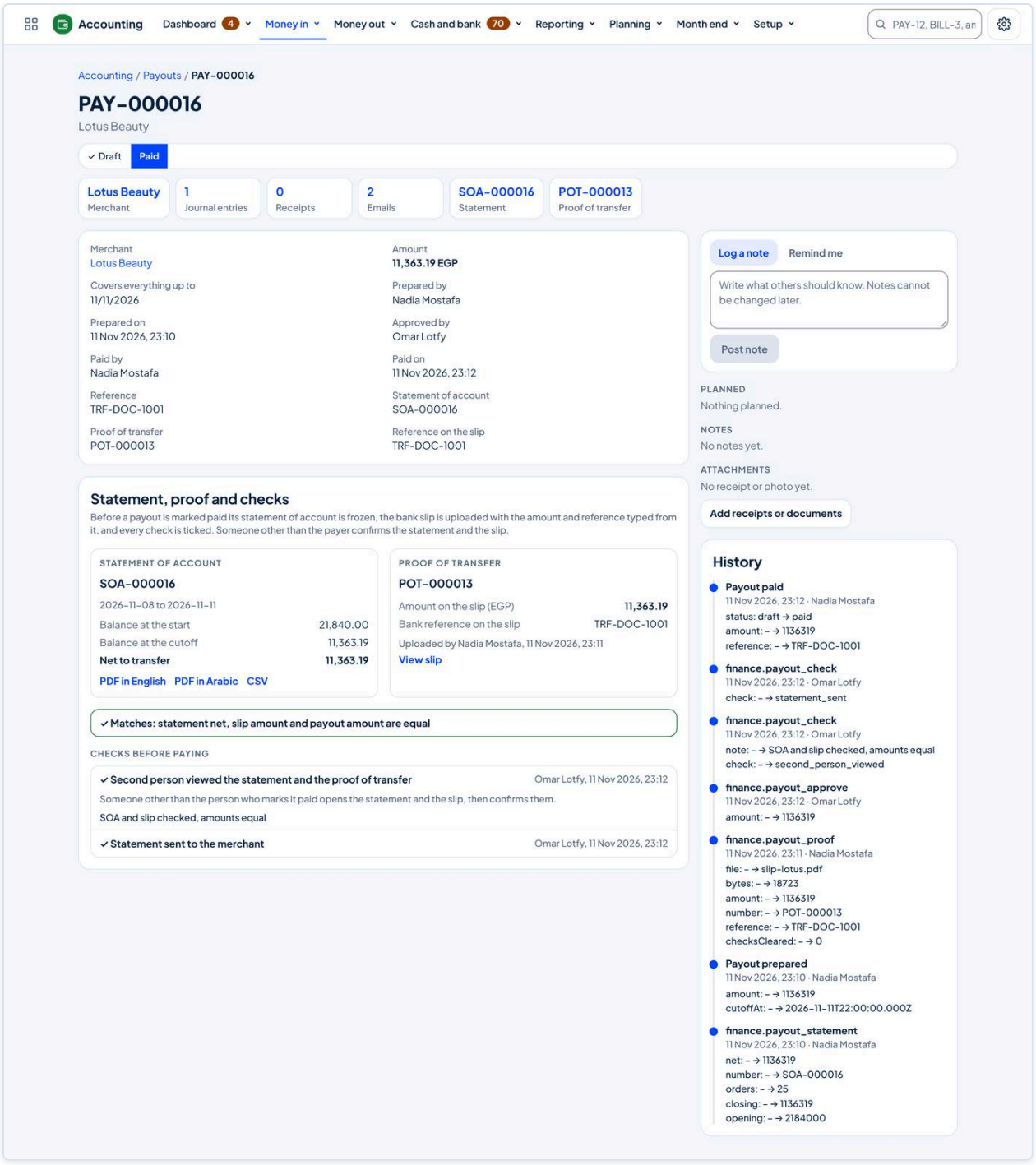


The same person refusal on the payout page: the preparer confirmed the checks herself on Nile Pantry's payout; Mark as paid stays greyed with "You confirmed the statement and the slip, so someone else marks it paid."

Mark paid and the paid state



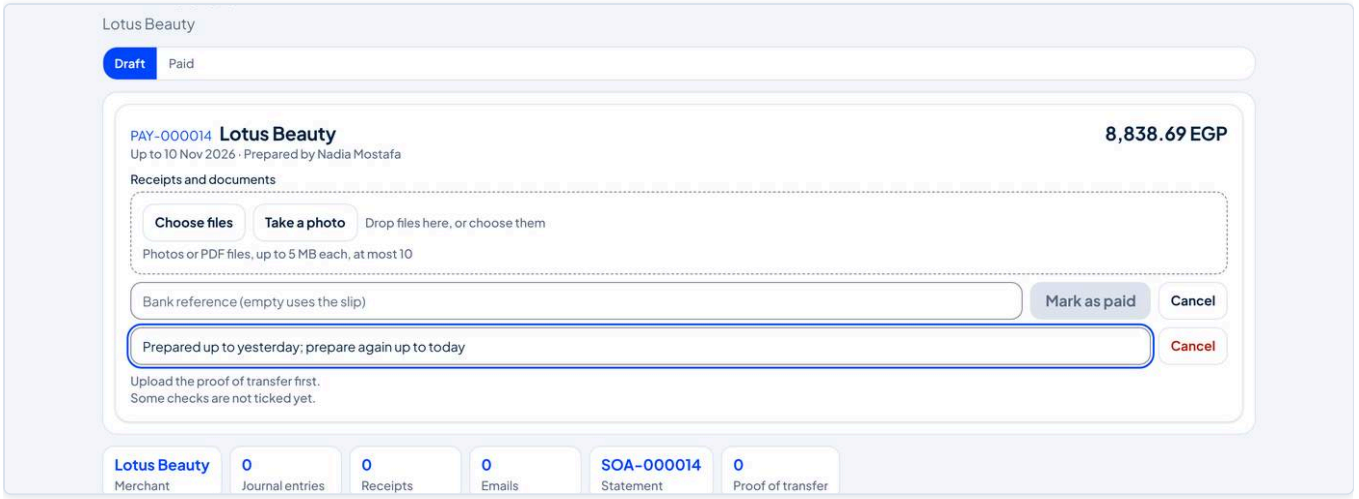
Ready: approved, proof matches, both checks ticked by the second user; for the preparer Mark as paid is now blue.



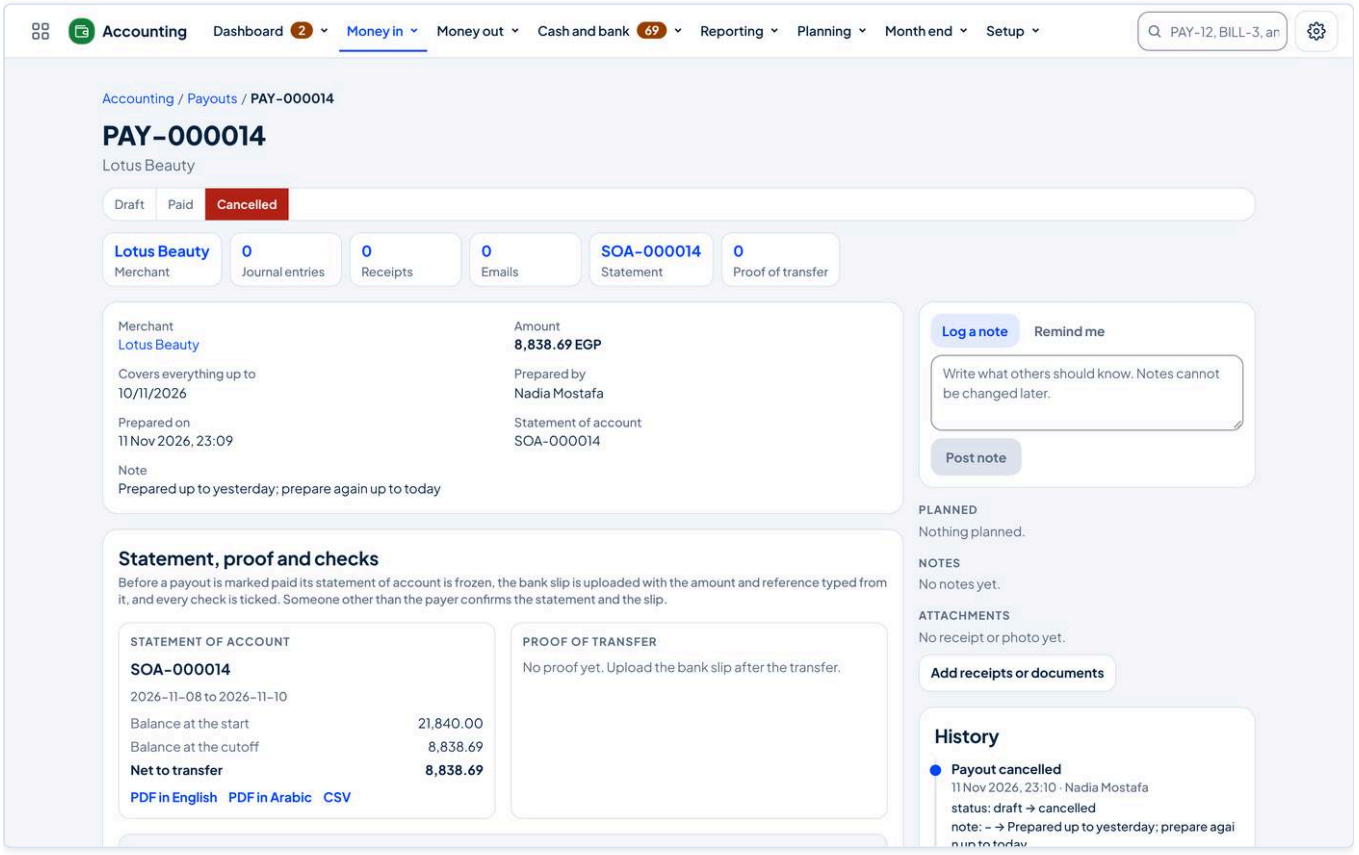
Paid: the status bar shows Draft ticked and Paid filled blue; smart buttons show 1 journal entry, 2 emails (payout notice with SOA and POT attached, statement with its CSV), the SOA and POT numbers; fields show Approved by, Paid by, Paid on, Reference (taken from the slip), Proof of transfer and Reference on the slip. The action card is gone.

Control	What it does	Rule / limits	Result
Mark as paid	`POST /finance/payouts/<id>/paid` with the reference (or the slip's), bank and files	Approval where needed (`approval_needed`), proof, amounts, checks, not the second person (`same_person`), the merchant still owed at least the amount (`payout_more_than_owed`), the bank holds it (`not_enough_money`), books open (`books_closed`)	Entry `merchant_payout` (Dr merchant wallet, Cr bank), status Paid, emails queued; see "Merchant money, payouts"

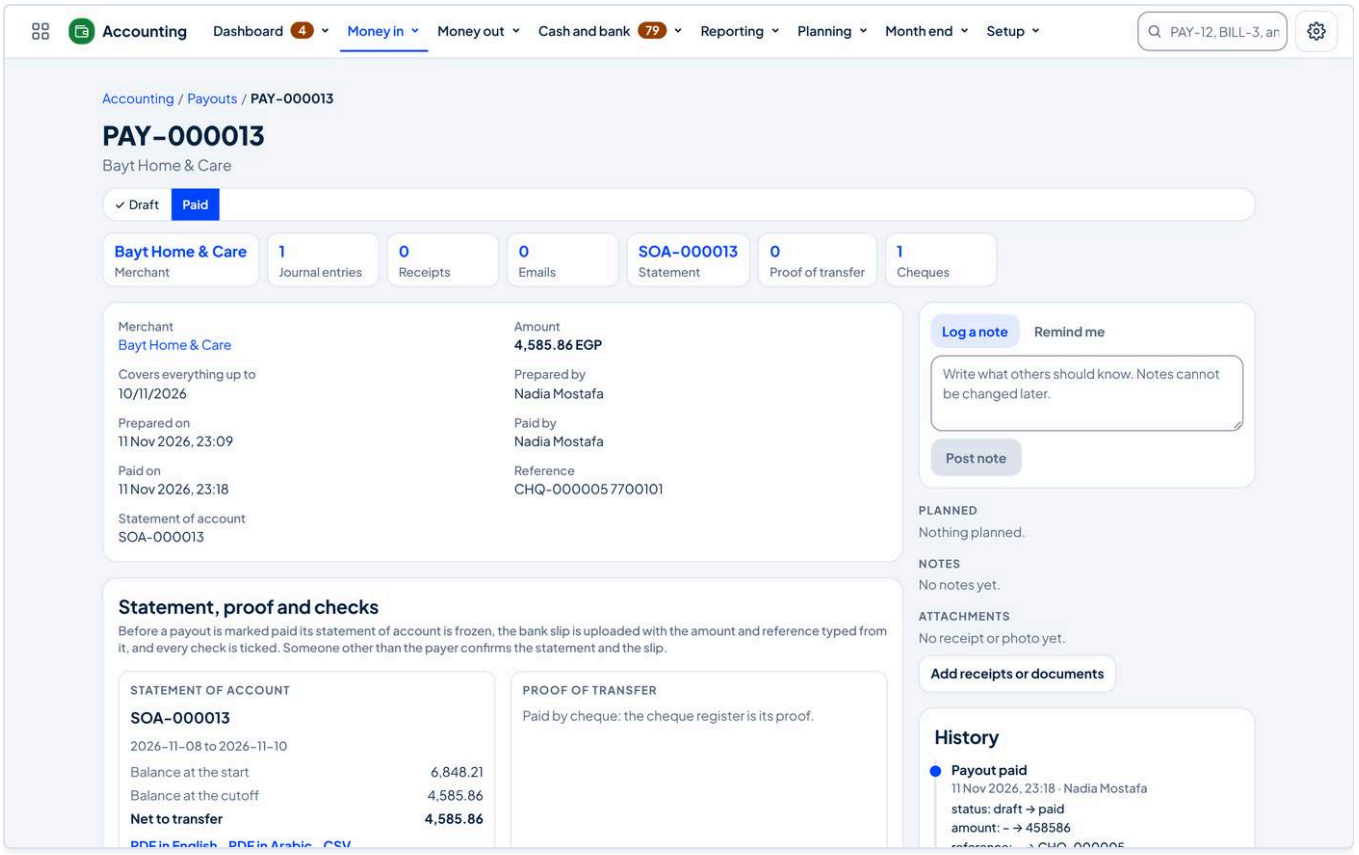
Control	What it does	Rule / limits	Result
Cancel	`POST /finance/payouts/<id>/cancel {note}`	Only a draft; a reason of at least 3 characters	Status Cancelled, nothing posted; the merchant can be prepared again



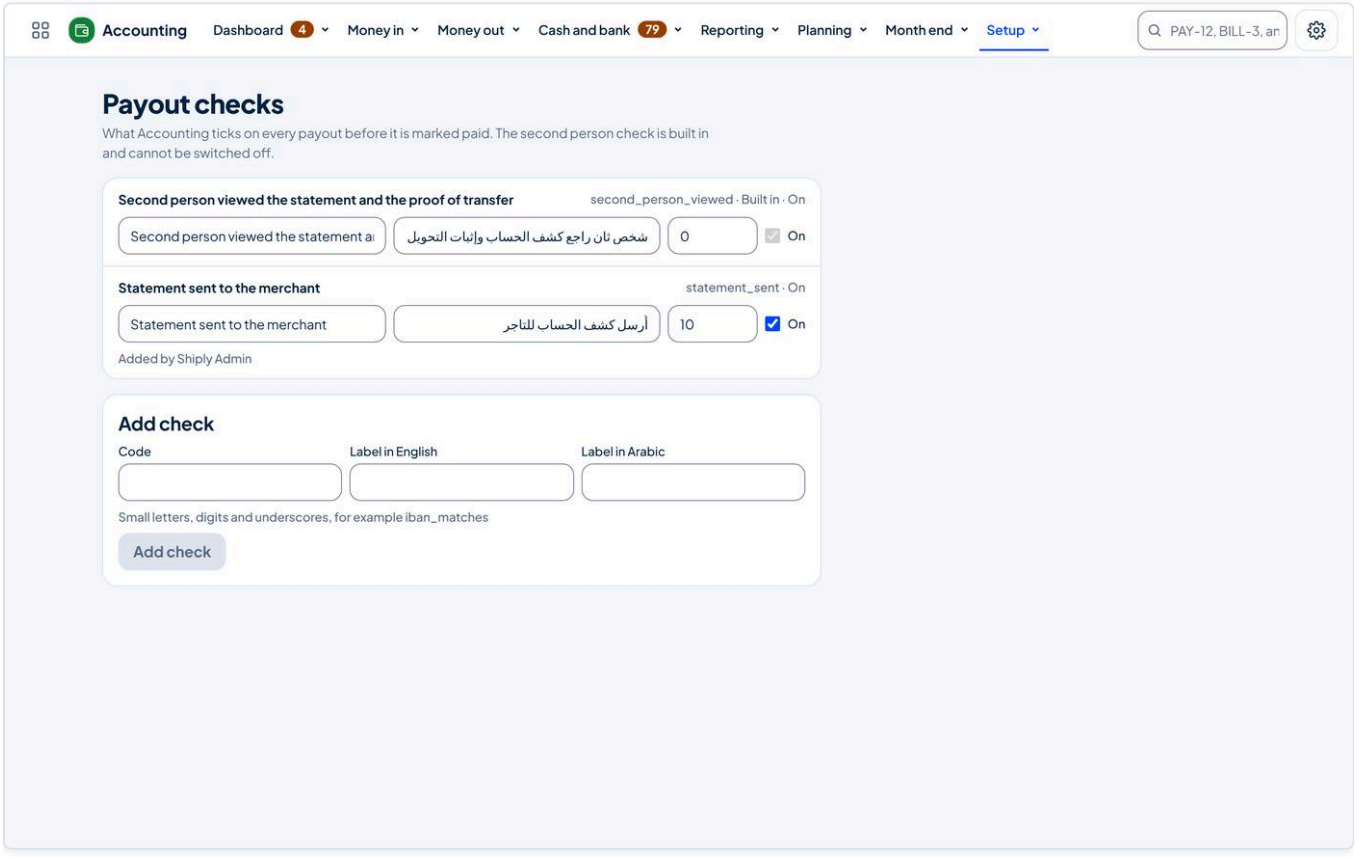
Cancel opens "Why cancel?" and a red Cancel button, enabled once the reason has 3 characters.



Cancelled: a red Cancelled segment at the end of the status bar, the reason under Note, the SOA still kept; no proof, no entry.



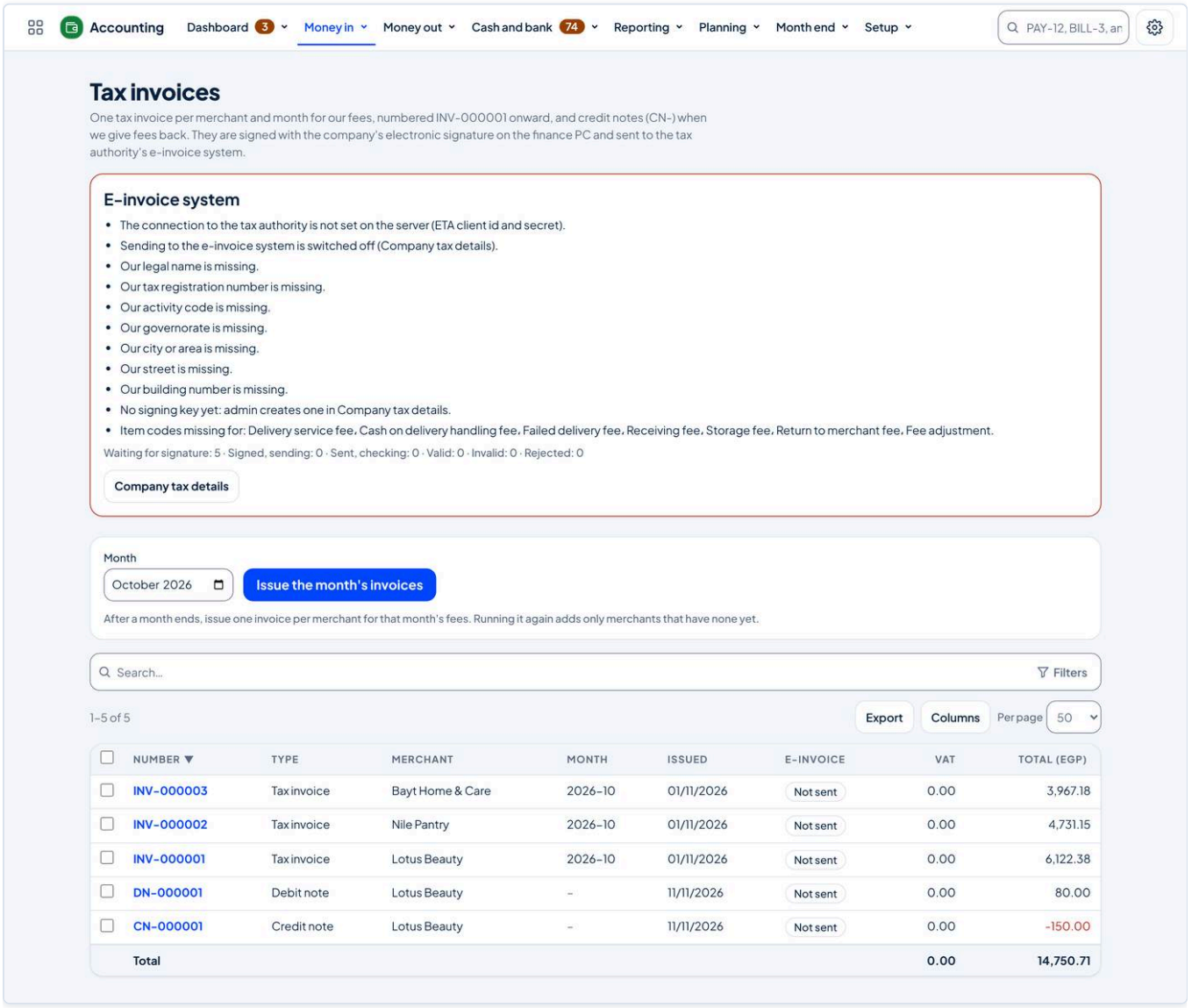
A payout paid by an issued cheque: Draft ticked and Paid, smart button Cheques 1, Reference `CHQ-000005 7700101` and the proof box "Paid by cheque: the cheque register is its proof."



Setup > Payout checks (`/finance/payout-checks` , accounting manage): the built in second person check (cannot be switched off) and the checks finance adds with code, English and Arabic labels, order and On.

13.10 Tax invoices

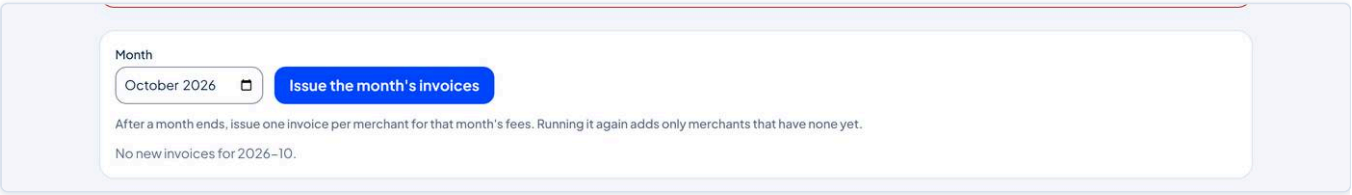
Address: `/finance/invoices`, menu Money in > Tax invoices. Who: accounting view reads; Issue the month's invoices, credit and debit notes need accounting use; the notes then wait for a second person with accounting manage. List data: `GET /finance/lists/invoices`; the readiness box: `GET /finance/eta/readiness`. The rules are in "Merchant money, invoices" and "Merchant money, ETA".



Tax invoices with the readiness box, the month form and the list holding invoices, a credit note and a debit note. Address `/finance/invoices`.

Control	What it does	Rule / limits	Result
`E-invoice` system box	Red outlined box listing what stops sending: the connection (ETA client id and secret), the switch, each missing company field, the signing key, missing item codes; then the count per state	Green "Everything is set..." when nothing is missing	Company tax details button opens `/finance/company`; "Send and check now" appears when sending is on
Month	Month picker	Only a month that is over	None

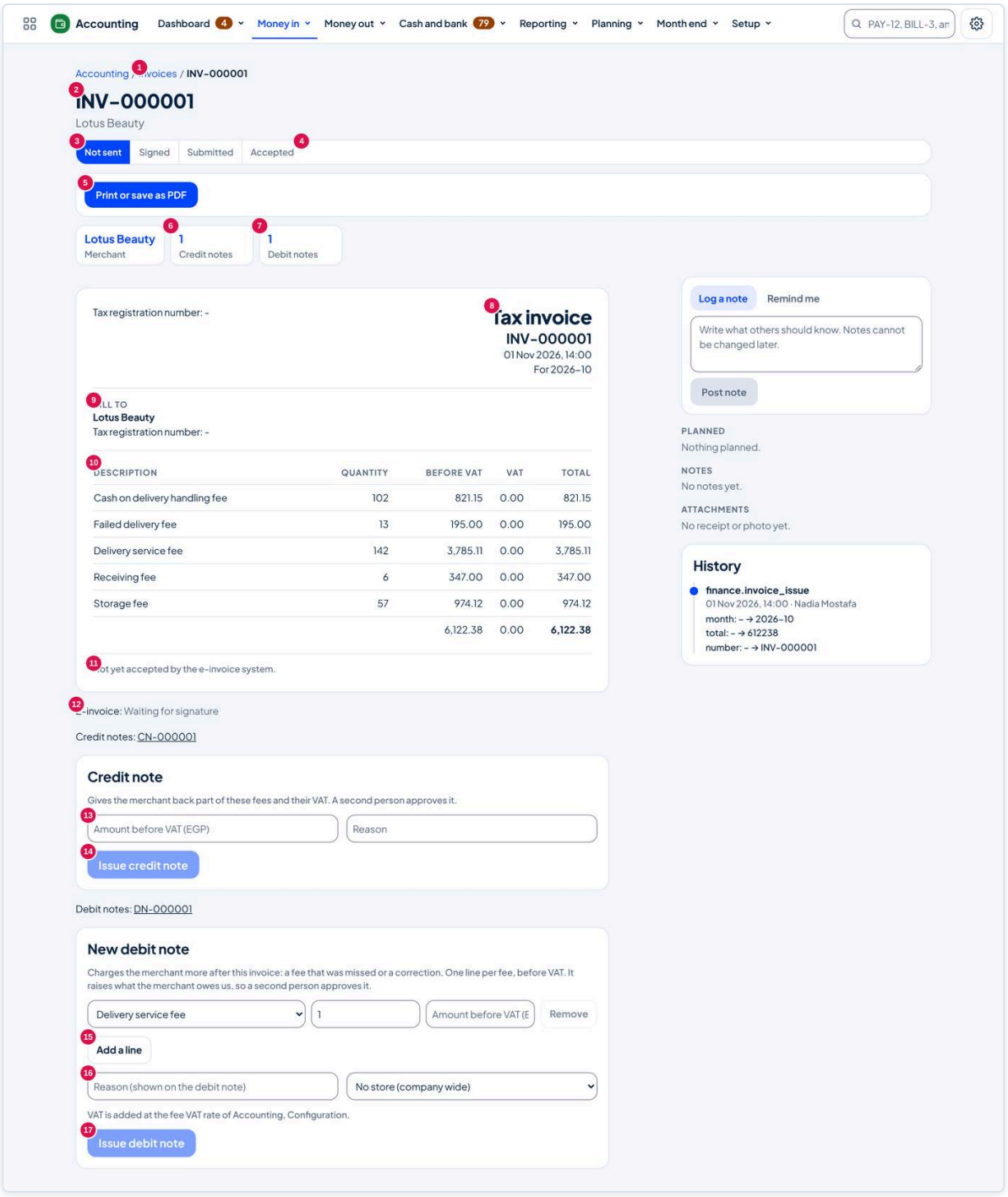
Control	What it does	Rule / limits	Result
Issue the month's invoices	`POST /finance/invoices/prepare`	One invoice per merchant and month; running again adds only merchants without one	"n invoices issued for month" or "No new invoices for month"
Number	`INV-` , `CN-` , `DN-` in blue	Gapless	Opens the document
Type	Tax invoice, Credit note, Debit note	Filter by type	None
Merchant, Month, Issued	Text, `yyyy-mm` , dd/mm/yyyy	Sortable merchant, issued	None
`E-invoice` state pill	Not sent, Signed, Submitted, Valid, Invalid, Rejected	Filters Not sent to the tax authority, Sent, waiting, Accepted, Needs a fix	None
Before VAT (optional), VAT, Total (EGP)	Money columns with totals	Sums over every passing row	None



Running Issue the month's invoices again for October: no new invoice, because every merchant already has one.

An invoice page

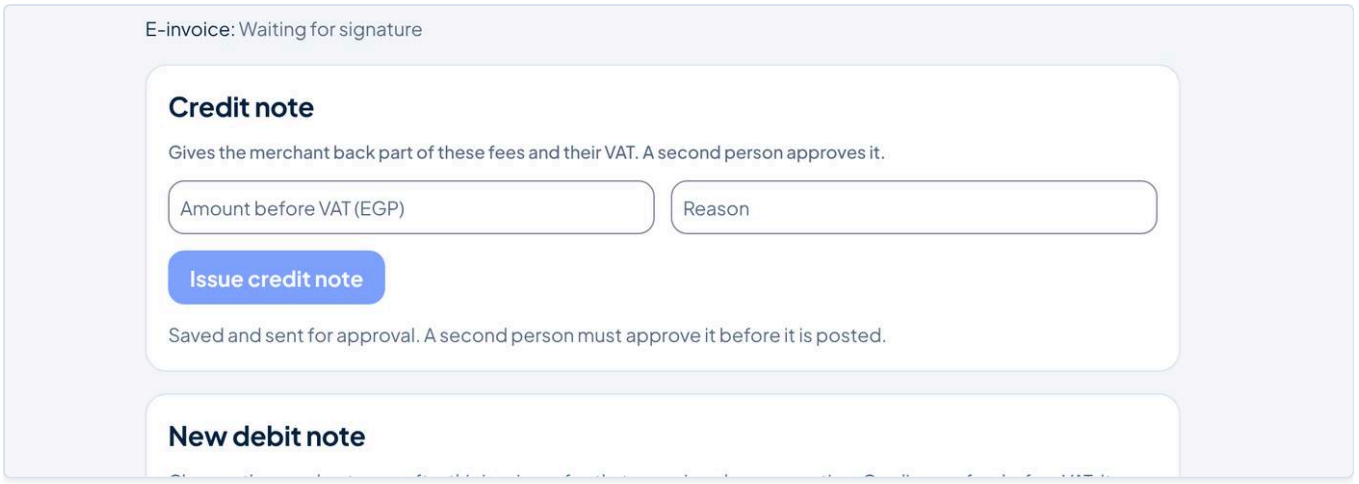
Address: `/finance/invoices/<id>` .



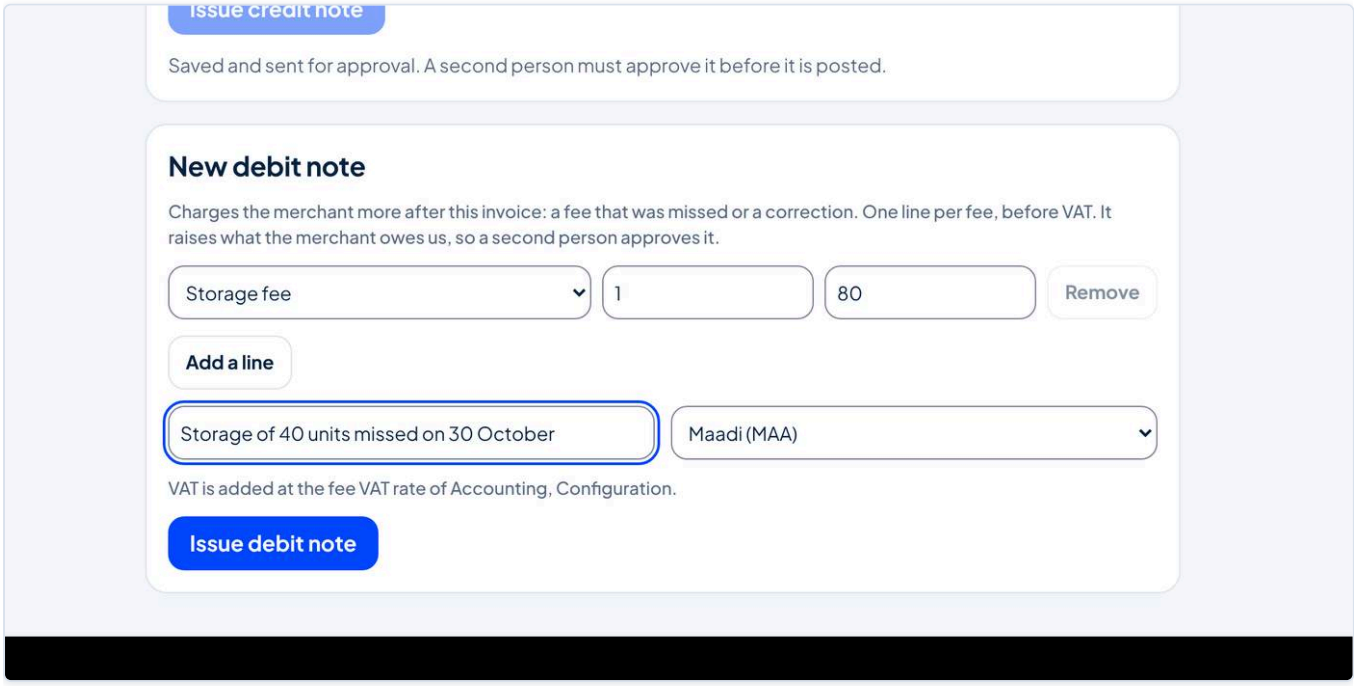
Invoice `INV-000001` with numbered parts, after a credit note and a debit note were issued against it.

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Breadcrumbs	Accounting / Invoices / number	Back to the list
2	Title	Number and merchant	None
3	Status bar, current	Not sent in blue; the steps are Not sent, Signed, Submitted, Accepted	None
4	Last step	Accepted (the tax authority answered valid); Invalid and Rejected appear as red segments	None

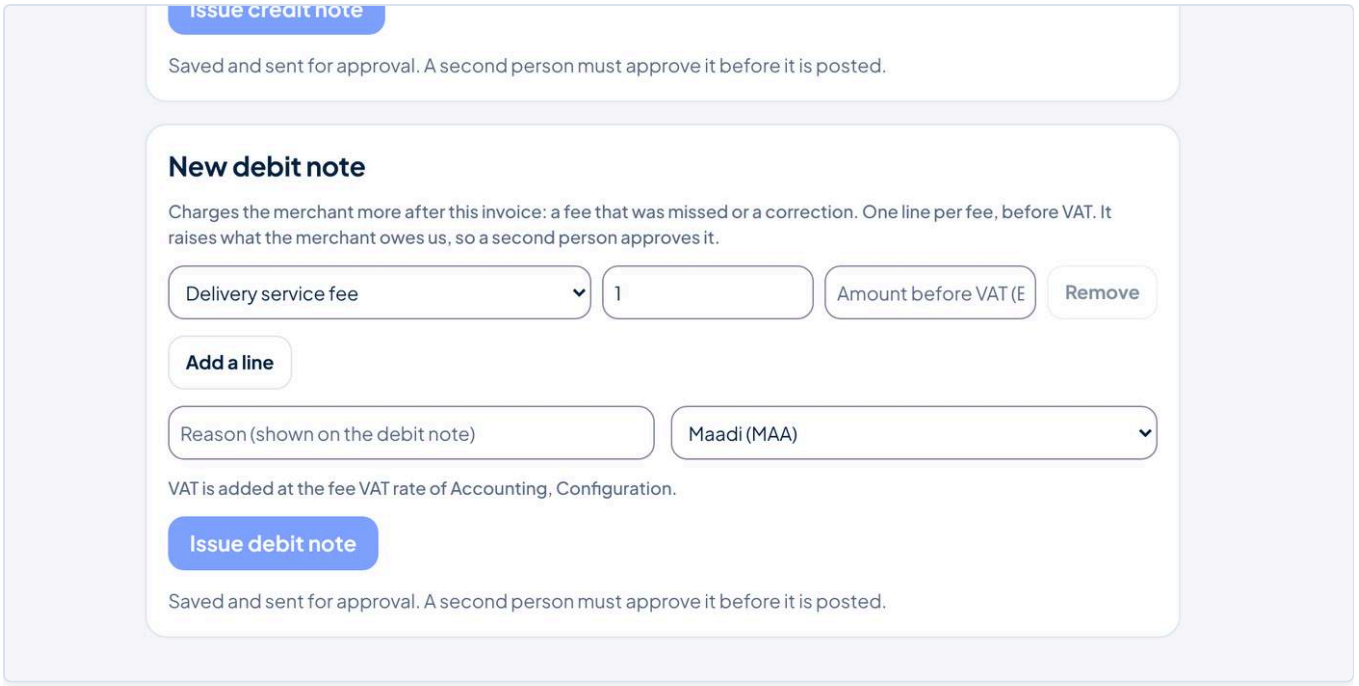
#	Part	What it shows (colors, icons, wording)	What clicking it does
5	Print or save as PDF	Blue button	Opens the browser print; printing hides the menus
6	Credit notes	Count	Lists them
7	Debit notes	Count	Lists them
8	Document head	"Tax invoice", number, issue time, "For `yyyy-mm` "; our tax registration number at the left	None
9	Bill to	Merchant and its tax number (a dash without one: invoiced as a person)	None
10	Lines	One line per fee kind: Description, Quantity, Before VAT, VAT, Total, then the totals in bold	None
11	Acceptance line	"Not yet accepted by the e invoice system." or the ETA document id	None
12	`E-invoice:` state	Waiting for signature, Signed, sending, Sent, checking, Valid, Invalid, Rejected; for Invalid or Rejected a hint and Send again	Send again: `POST /finance/invoices/<id>/resend`
(above 13)	Credit notes: and Debit notes: links	Underlined numbers	Open the note
13	Credit note amount and reason	Amount before VAT (EGP), Reason	None
14	Issue credit note	Light blue until both fields are filled	`POST /finance/invoices/<id>/credit-notes` ; always waits for a second person
15	Debit note lines	Fee kind, Quantity, Amount before VAT, Remove; Add a line	Builds the lines
16	Reason and store	Reason (shown on the debit note); store or "No store (company wide)"	None
17	Issue debit note	Light blue until filled	`POST /finance/invoices/<id>/debit-notes` ; waits for a second person



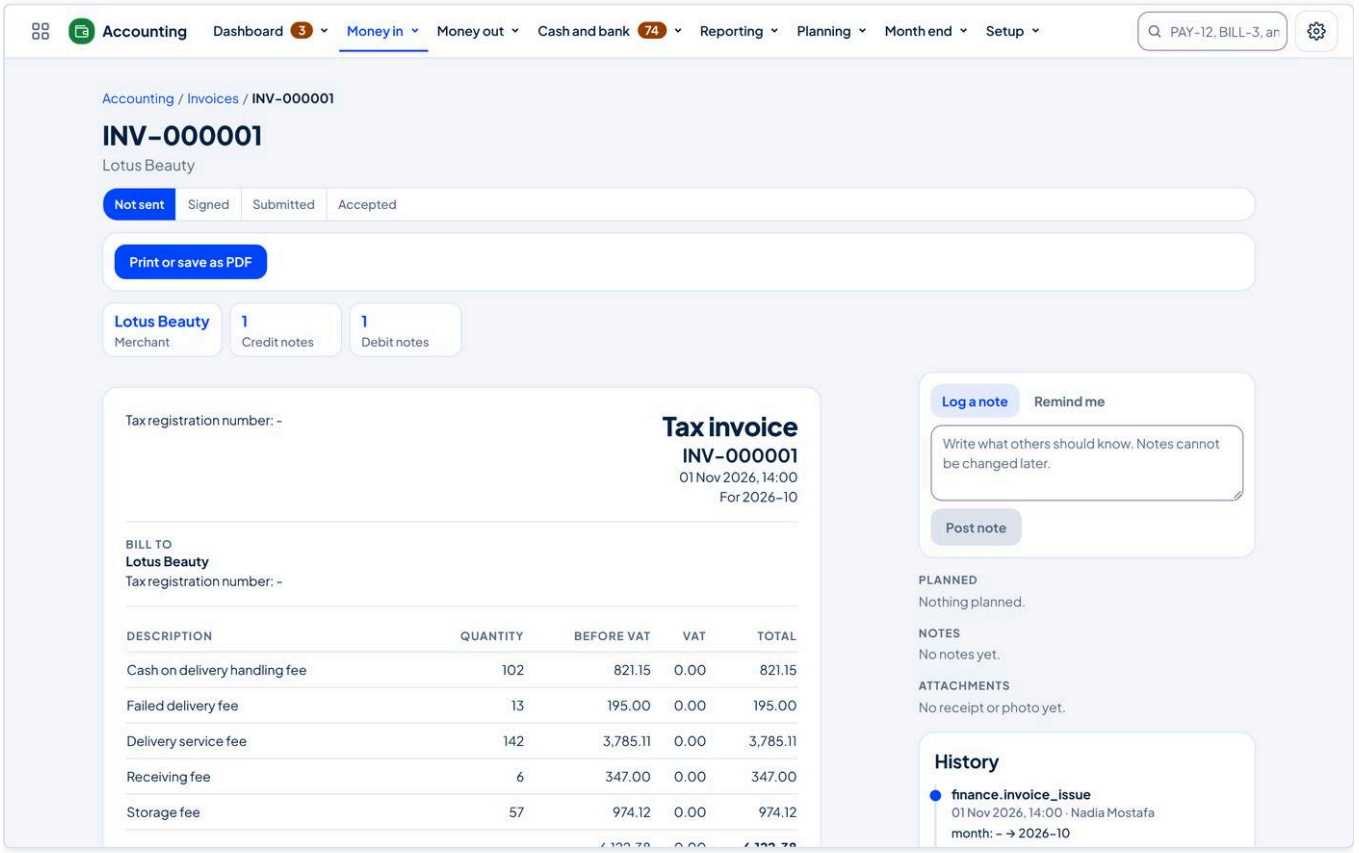
Credit note sent: "Saved and sent for approval. A second person must approve it before it is posted."



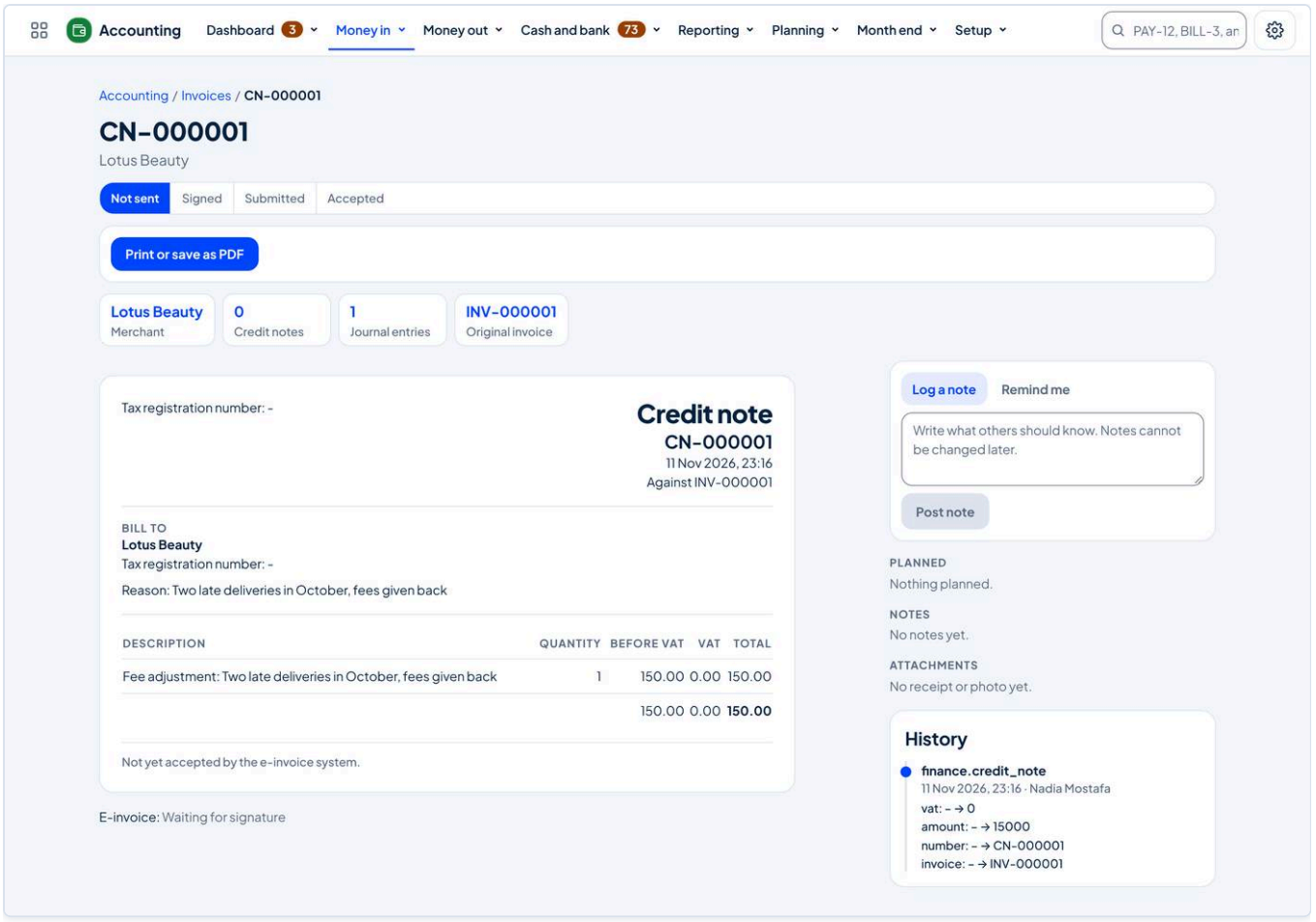
The debit note form filled: one Storage fee line of 80 EGP, the reason and the Maadi store.



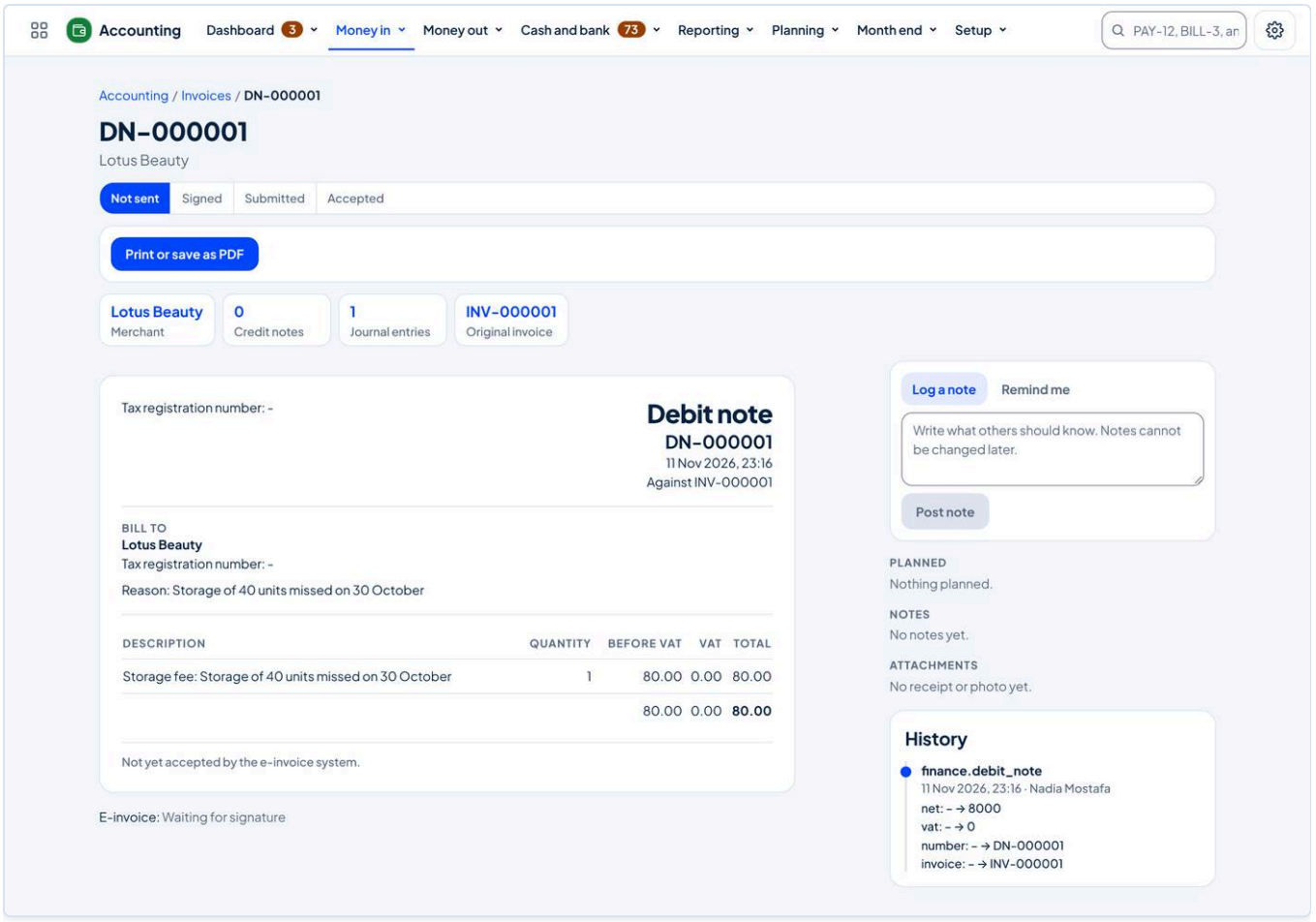
Debit note sent for approval; the form resets.



After the second user approved both notes: the smart buttons count 1 credit note and 1 debit note.



Credit note `CN-000001`: "Credit note", "Against `INV-000001`", the reason, one line giving back 150.00 before VAT with proportional VAT, smart buttons to the journal entry and the original invoice.

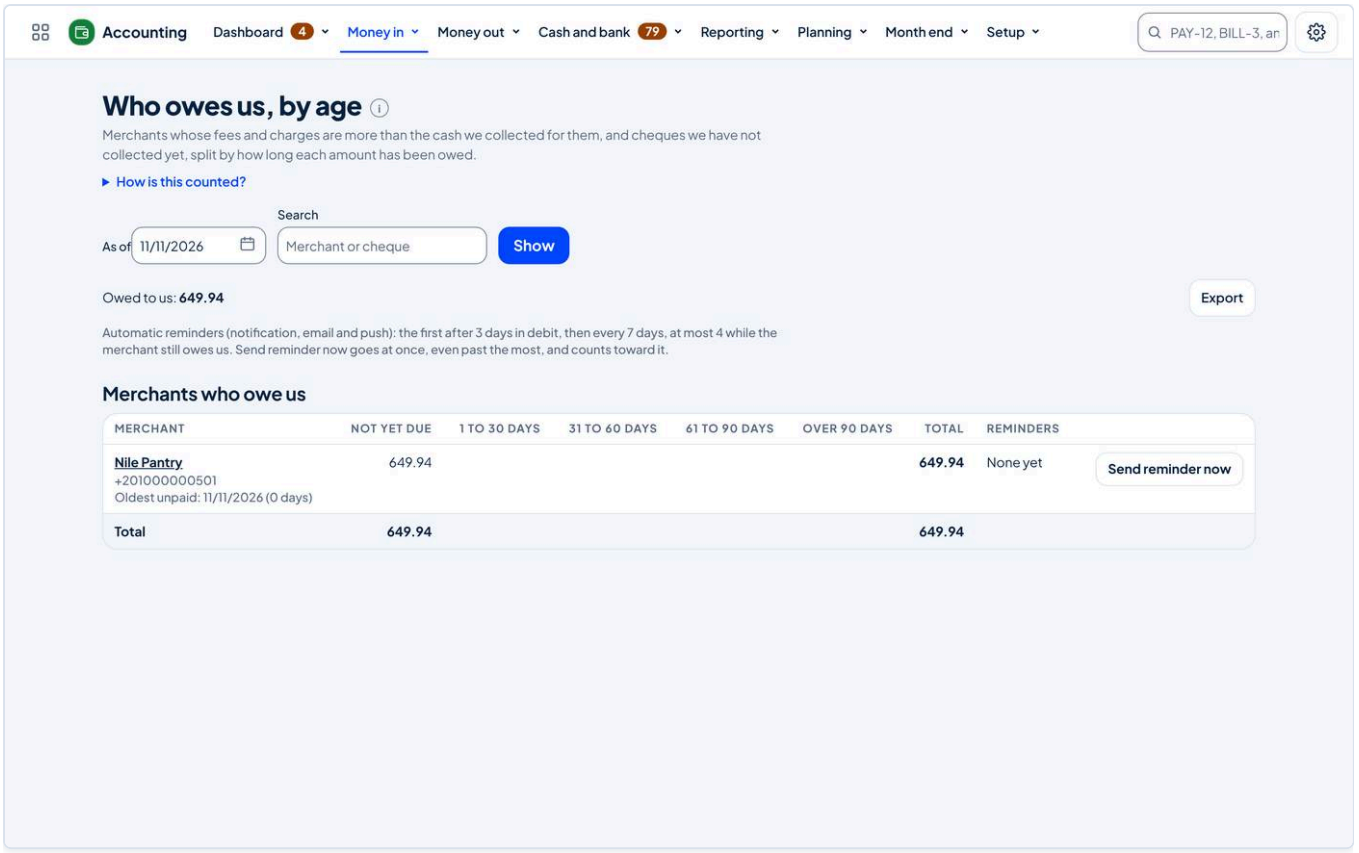


Debit note `DN-000001` : "Debit note", "Against `INV-000001` ", "Storage fee: Storage of 40 units missed on 30 October", 80.00 before VAT; status bar Not sent; ETA document type D is sent only after the invoice itself.

Refusal	Rule
Credit note above what is left	A credit note gives back VAT from the running total, never more than the invoice's VAT and fees
Same person approving	Credit and debit notes always wait for someone other than the asker
Fixed amounts	Once issued, invoice and note amounts and lines are fixed by a database trigger

13.11 Who owes us, by age (Receivables aging)

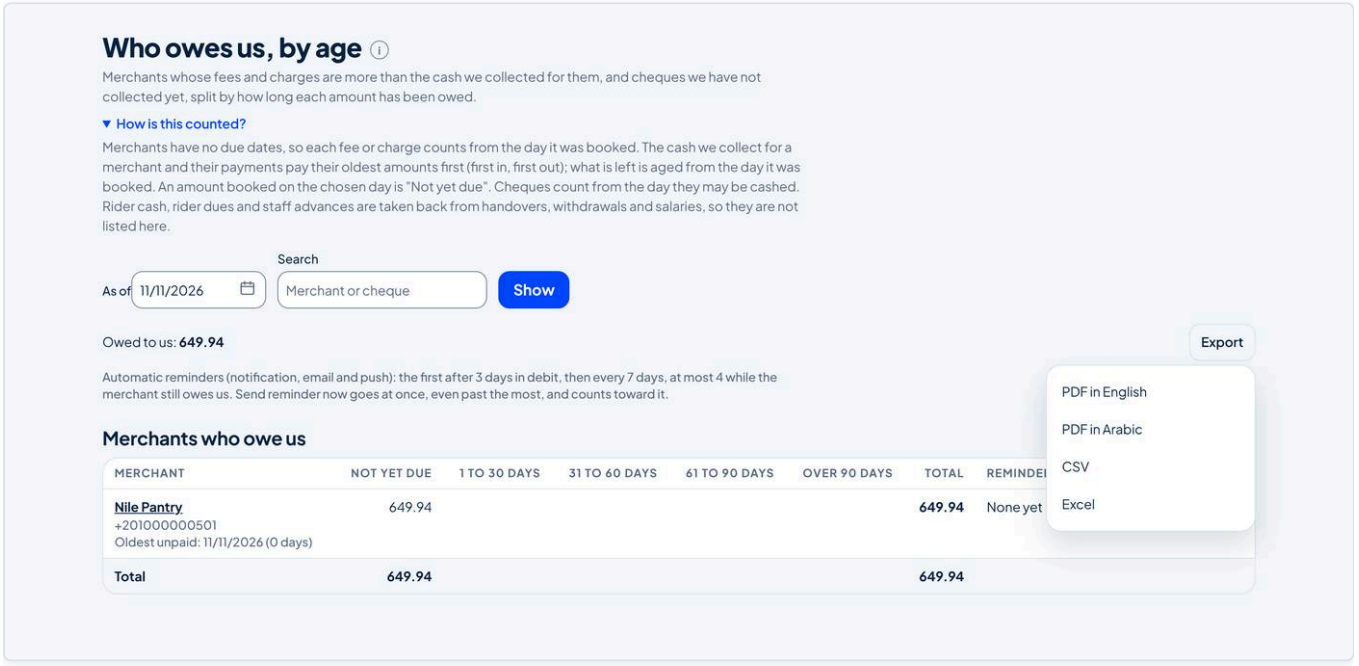
Address: `/finance/receivables?asOf=&q=` , menu Money in > Money owed to us. Who: accounting view reads; Send reminder now needs accounting use. To show a row, a merchant charge of 2,500 EGP (an Adjustment) was asked and approved for Nile Pantry.



One merchant owes 649.94 EGP, not yet due. Address `/finance/receivables`.

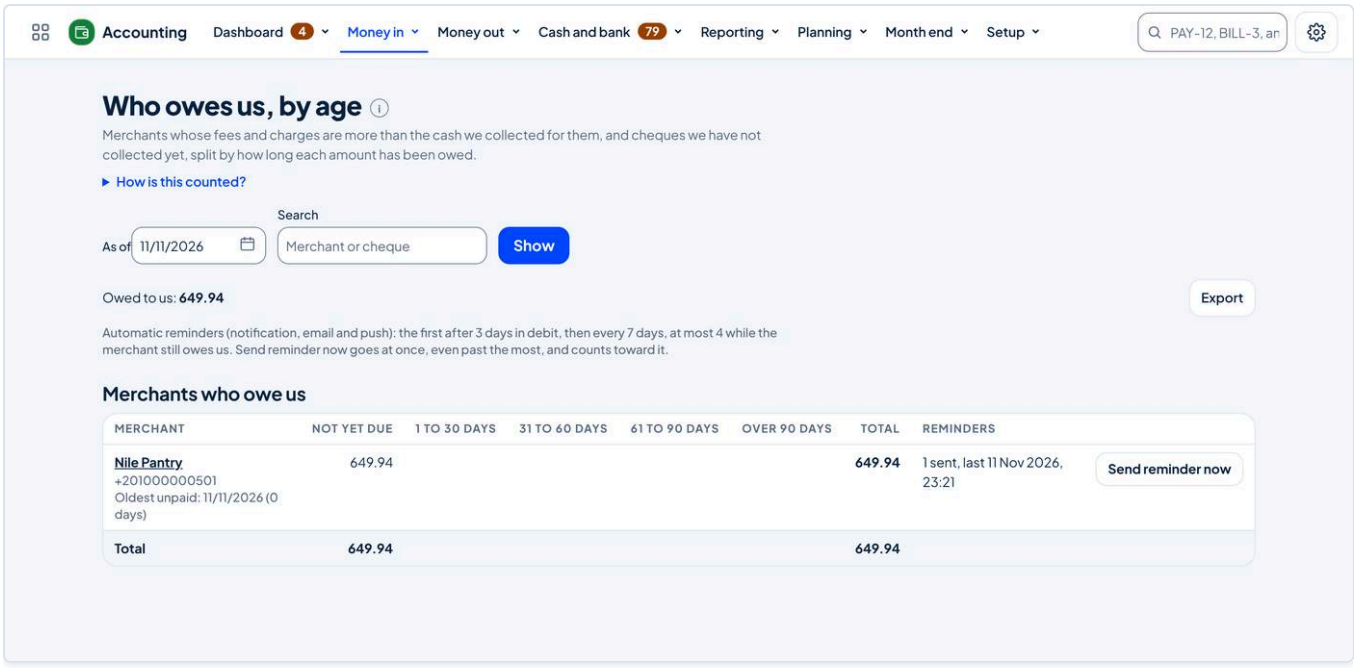
Control	What it does	Rule / limits	Result
Title with the i tip	Explains "receivables"	InfoTip	A small explanation
How is this counted?	Fold with the method	Each charge counts from the day it was booked; cash collected and payments pay the oldest first (first in, first out); cheques count from the day they may be cashed	Text
As of	Date box	Any past day	Ages as of that day
Search	"Merchant or cheque"	Text	Narrows the rows
Show	Blue button	Puts both in the address	Reloads
Owed to us:	Grand total	Merchants plus cheques under collection	None
Export	Menu: PDF in English, PDF in Arabic, CSV, Excel	Same rows	Downloads `who-owes-us`
Reminder rule line	"Automatic reminders ... the first after 3 days in debit, then every 7 days, at most 4"	From `receivables.*` settings; "switched off" text when disabled	None
Merchants who owe us table	Merchant (link, phone, "Oldest unpaid: day (n days)"), Not yet due, 1 to 30 days, 31 to 60, 61 to 90, Over 90, Total, Reminders	Buckets by age	The merchant link opens the wallet statement

Control	What it does	Rule / limits	Result
Send reminder now	Sends notification, email and push at once	Counts toward the most; goes even past it	"Reminder n sent." and the Reminders cell "1 sent, last time"
Cheques waiting to be collected	Second table when cheques are under collection	By due day	Opens the cheque



The method folded open and the Export menu (PDF in English, PDF in Arabic, CSV, Excel).

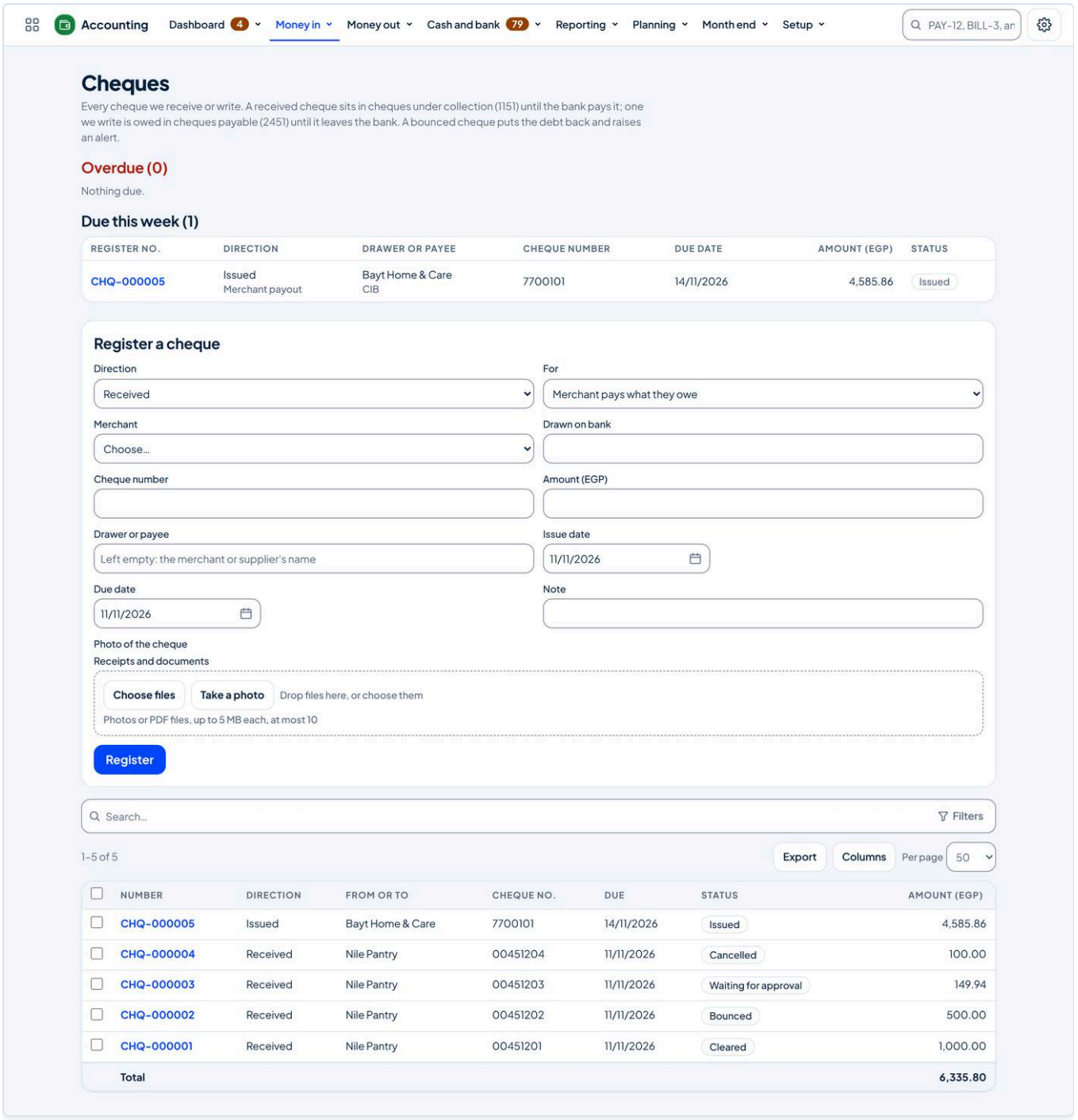
After Send reminder now: "Reminder 1 sent." under the button and "1 sent, last 11 Nov 2026, 23:21" in Reminders.



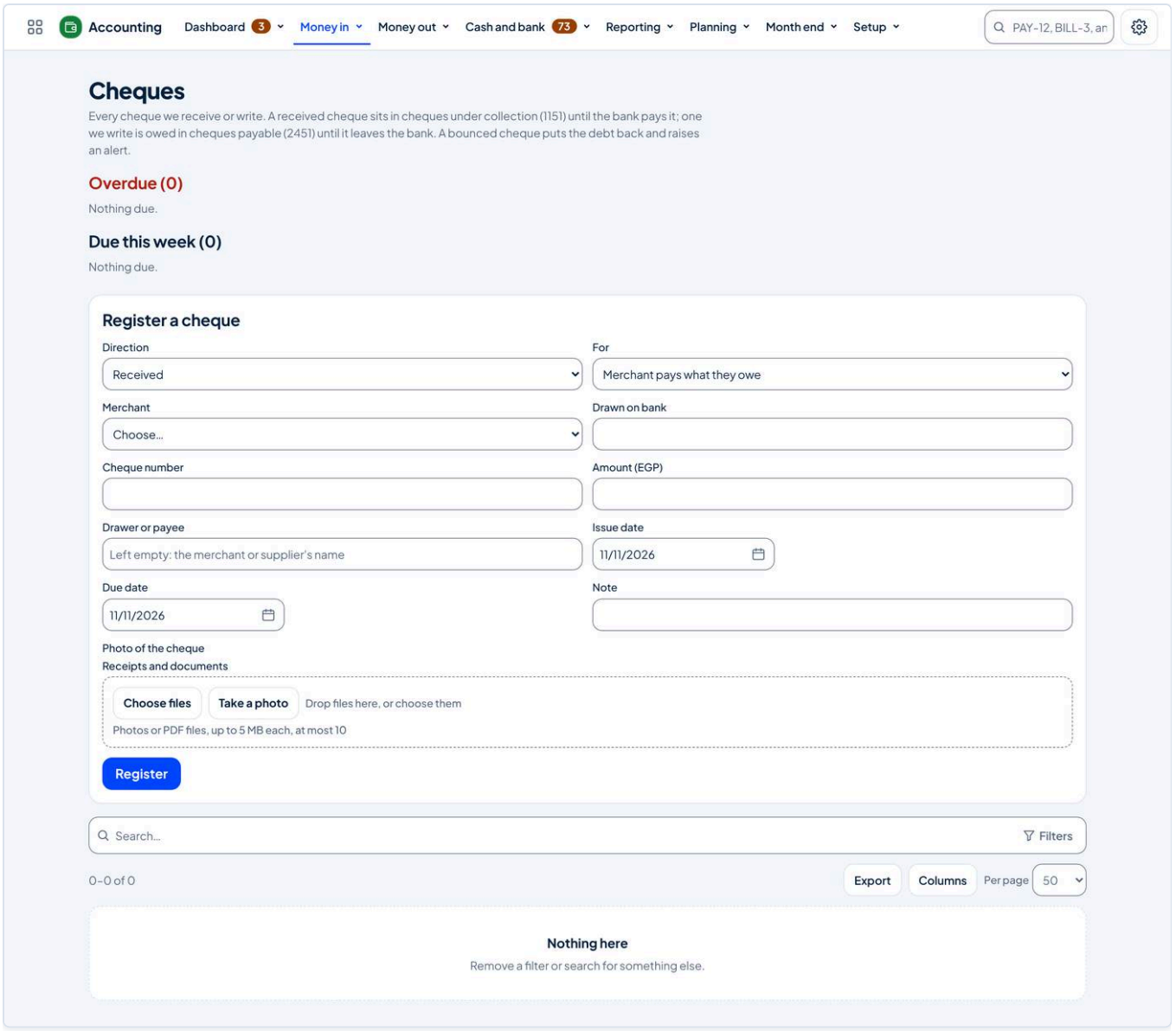
Empty state as of an earlier day: "No merchant owes us anything on this day."

13.12 Cheques register

Address: `/finance/cheques`, menu Money in > Cheques. Who: accounting view reads; Register, Deposit at bank, Bank paid it, Bounced and Cancel cheque need accounting use; a received cheque then waits for a second person with accounting manage. The rules are in "Cash and riders" and "Merchant money".



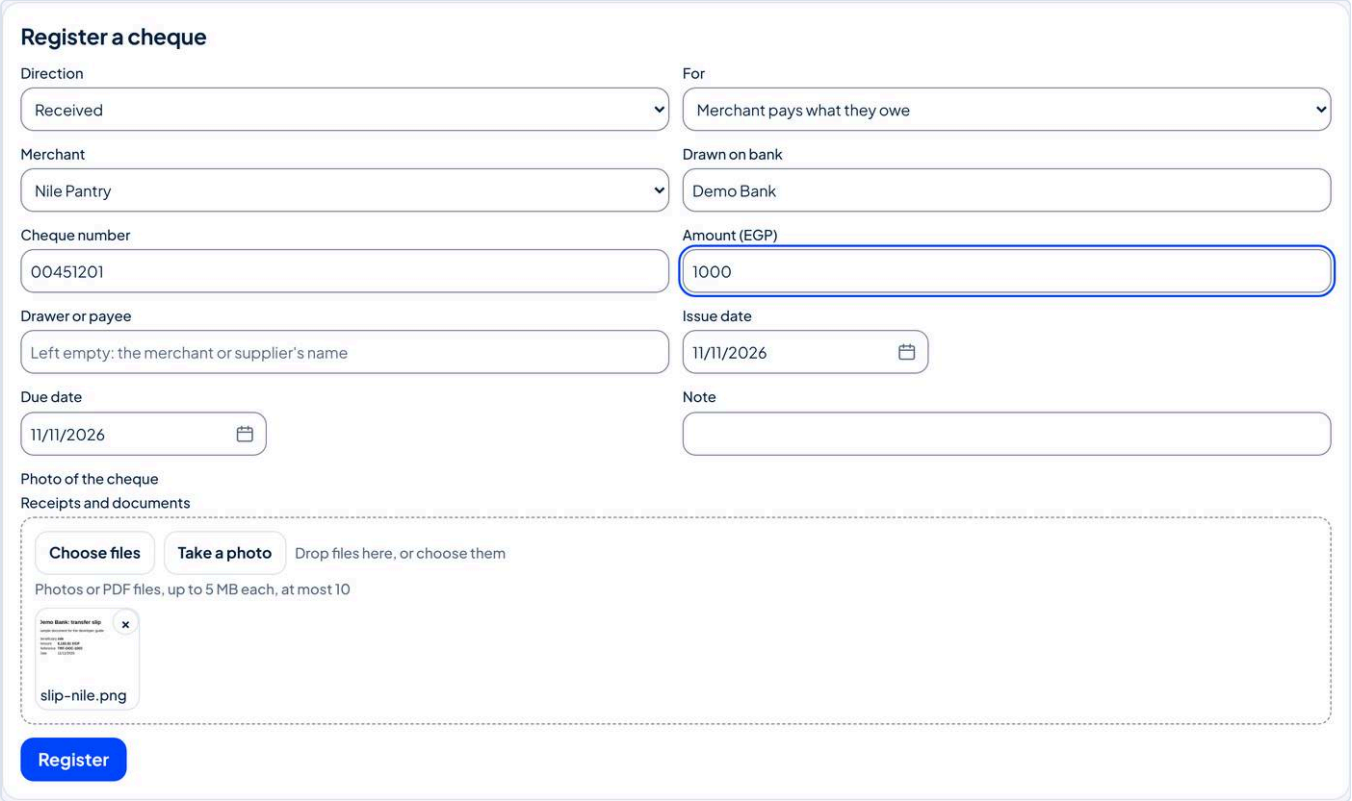
The register with every state: the due lists (Overdue in red, Due this week), the Register a cheque form, and the list with Issued, Cancelled, Waiting for approval, Bounced and Cleared. Address `/finance/cheques`.



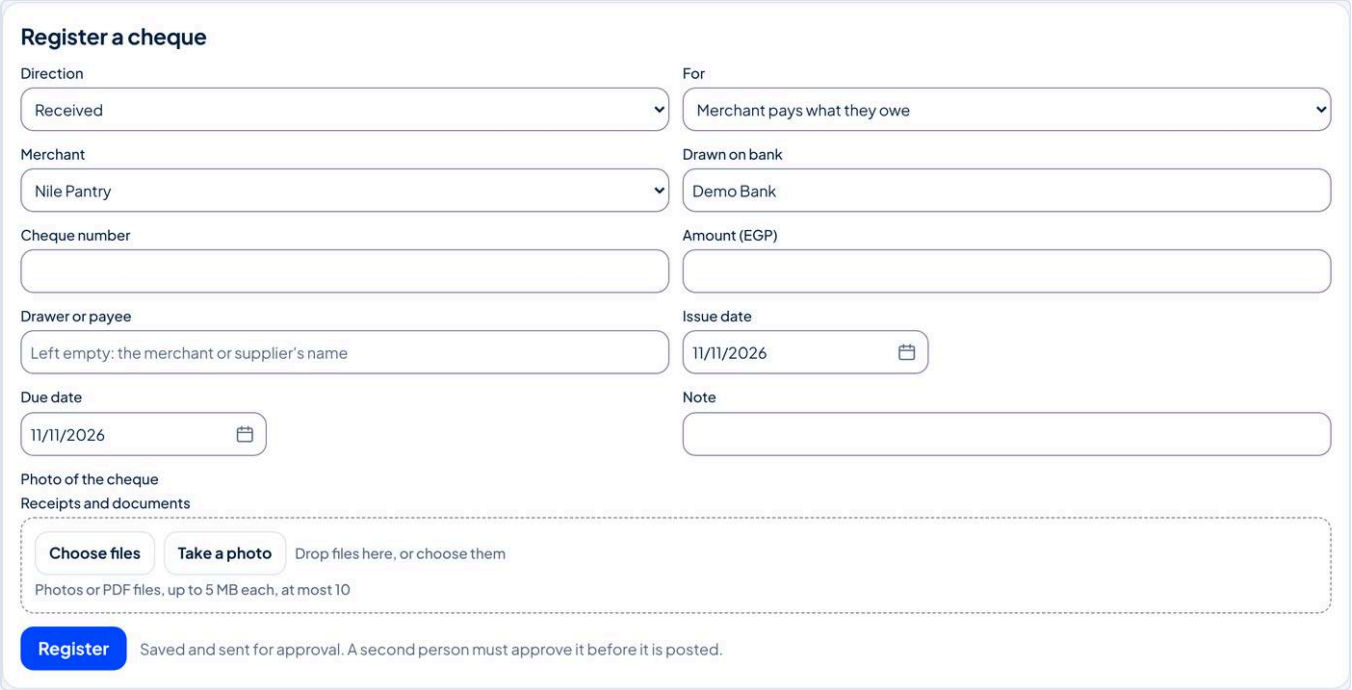
Before any cheque: "Nothing due." under both due lists and an empty list.

Control	What it does	Rule / limits	Result
Overdue (n)	Red title; open cheques past their due day	Received not cleared, issued not cleared	Row opens the cheque
Due this week (n)	Register no., Direction (with the purpose below), Drawer or payee (bank below), Cheque number, Due date, Amount, Status pill	Next 7 days	Same
Direction	Received or Issued	Changes the For list	None
For	Received: Merchant pays what they owe, Merchant security deposit, Supplier pays us back, Other income. Issued: Supplier bill, Merchant payout, Security deposit paid back	Shows the matching picker (Merchant, Supplier, Income account, Bill, Draft payout)	None
Drawn on bank	Received only	2 to 80 characters	None
Cheque number	The printed number	Required	None

Control	What it does	Rule / limits	Result
Amount (EGP)	For a bill: "Settles of the bill (EGP, tax kept back included)"; a payout cheque takes the payout amount	Above 0	None
Drawer or payee	"Left empty: the merchant or supplier's name"	Optional	None
Issue date, Due date	Date boxes, default today	Due not before issue (`due_before_issue`)	None
Our bank account	Issued only	An open bank	None
Note, Photo of the cheque (Receipts and documents)	Optional	Photos or PDF up to 5 MB, at most 10	Attached to the booking entry or its request
Register	`POST /finance/cheques`	Received: always waits for a second person; issued: from the approval limit (a payout cheque follows the payout's own approval)	"Saved and sent for approval..." or "Cheque registered and booked."
List	Number, Direction, From or to, Cheque no., Due, Status, Amount; optional Bank	Filters Received, Issued, Awaiting approval, Open, Cleared, Bounced or cancelled, Overdue, Due in 7 days; group by Direction, Status, From or to, Bank, Month	Opens the cheque



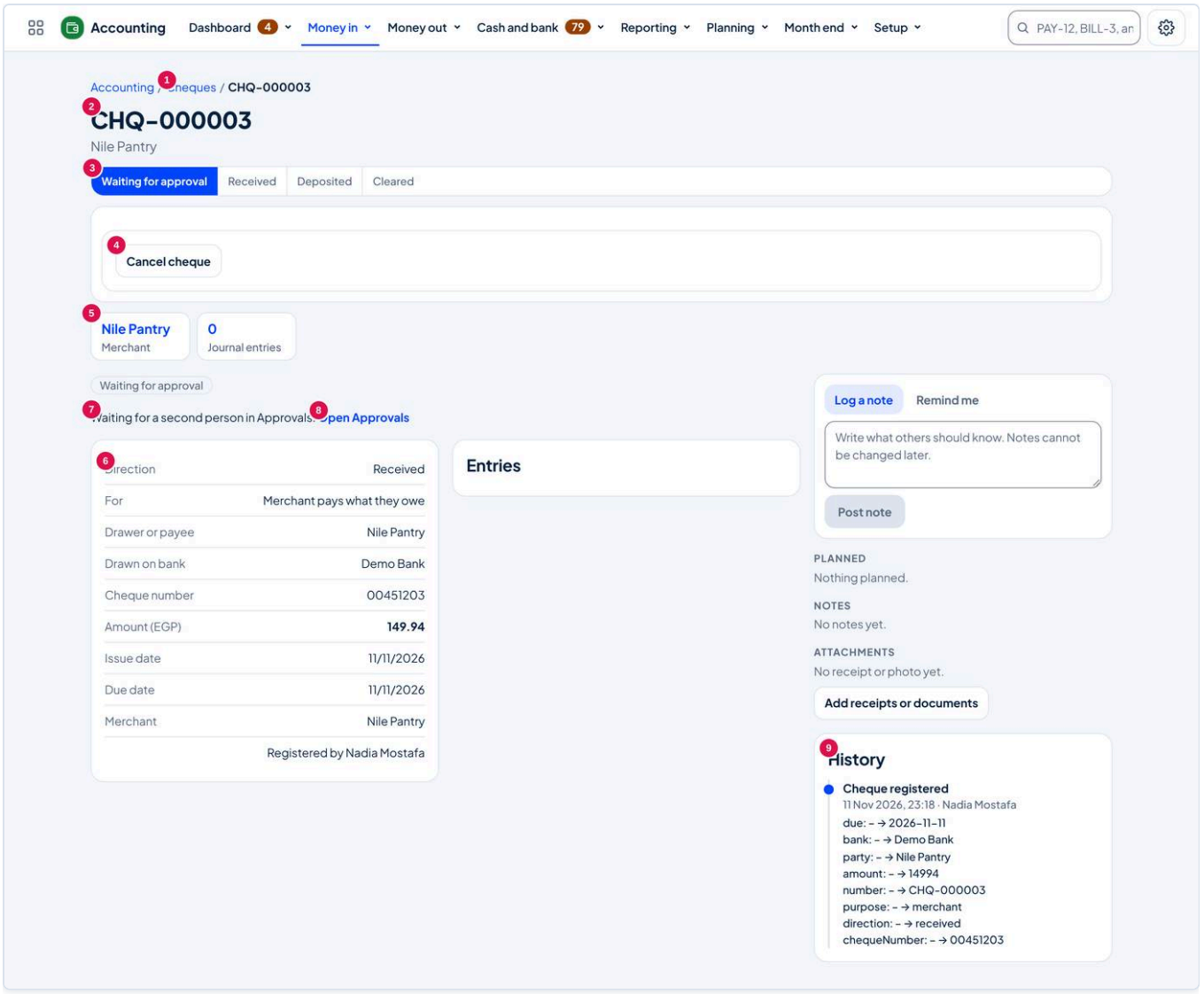
Register a cheque filled: Received, Merchant pays what they owe, Nile Pantry, Demo Bank, number and amount.



After Register: "Saved and sent for approval. A second person must approve it before it is posted."

A cheque page and its states

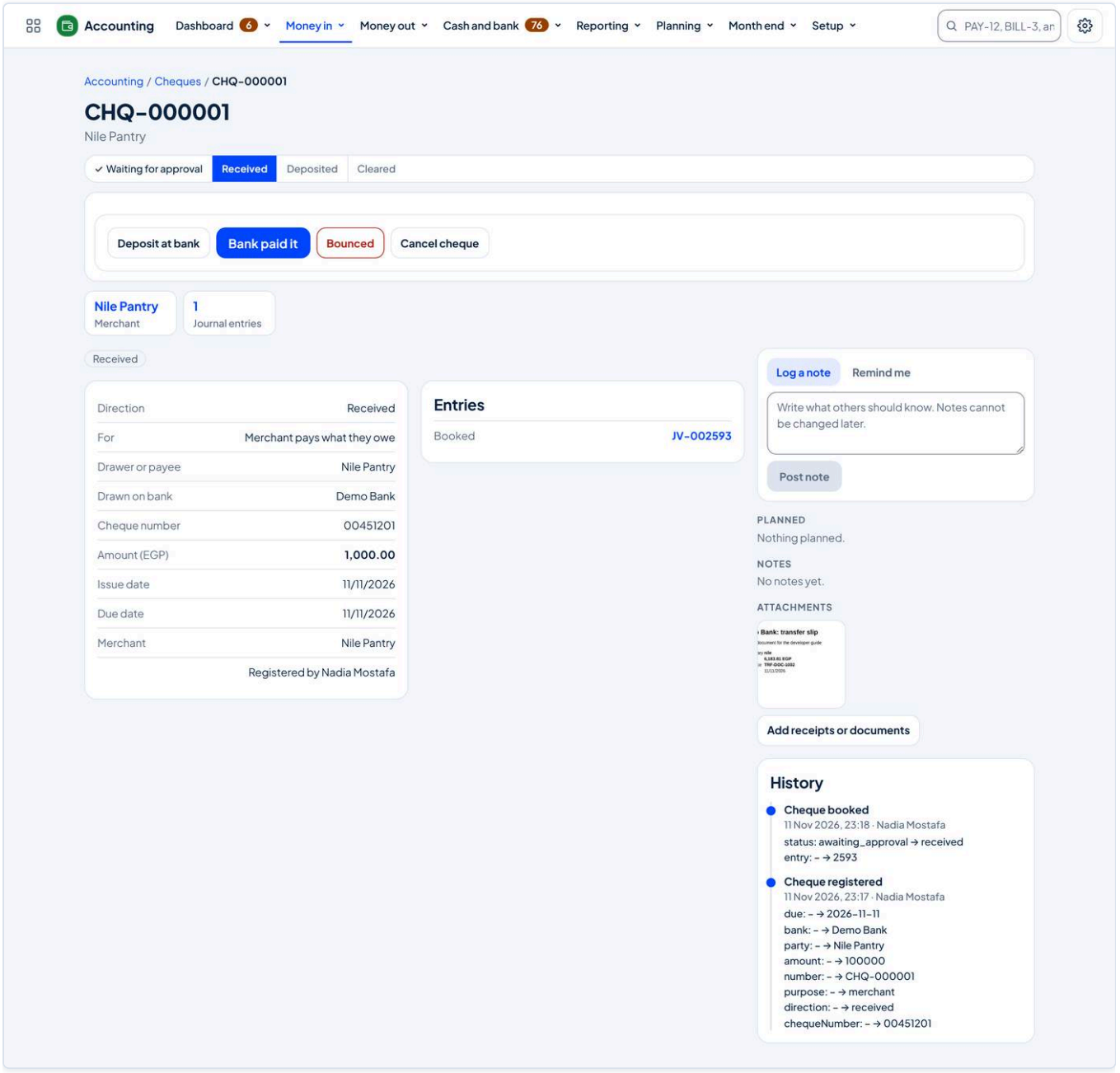
Address: `/finance/cheques/<id>`. The status bar of a received cheque is Waiting for approval, Received, Deposited, Cleared; an issued cheque is Issued, Cleared; Bounced and Cancelled appear as red segments. The action card shows only the actions the state allows.



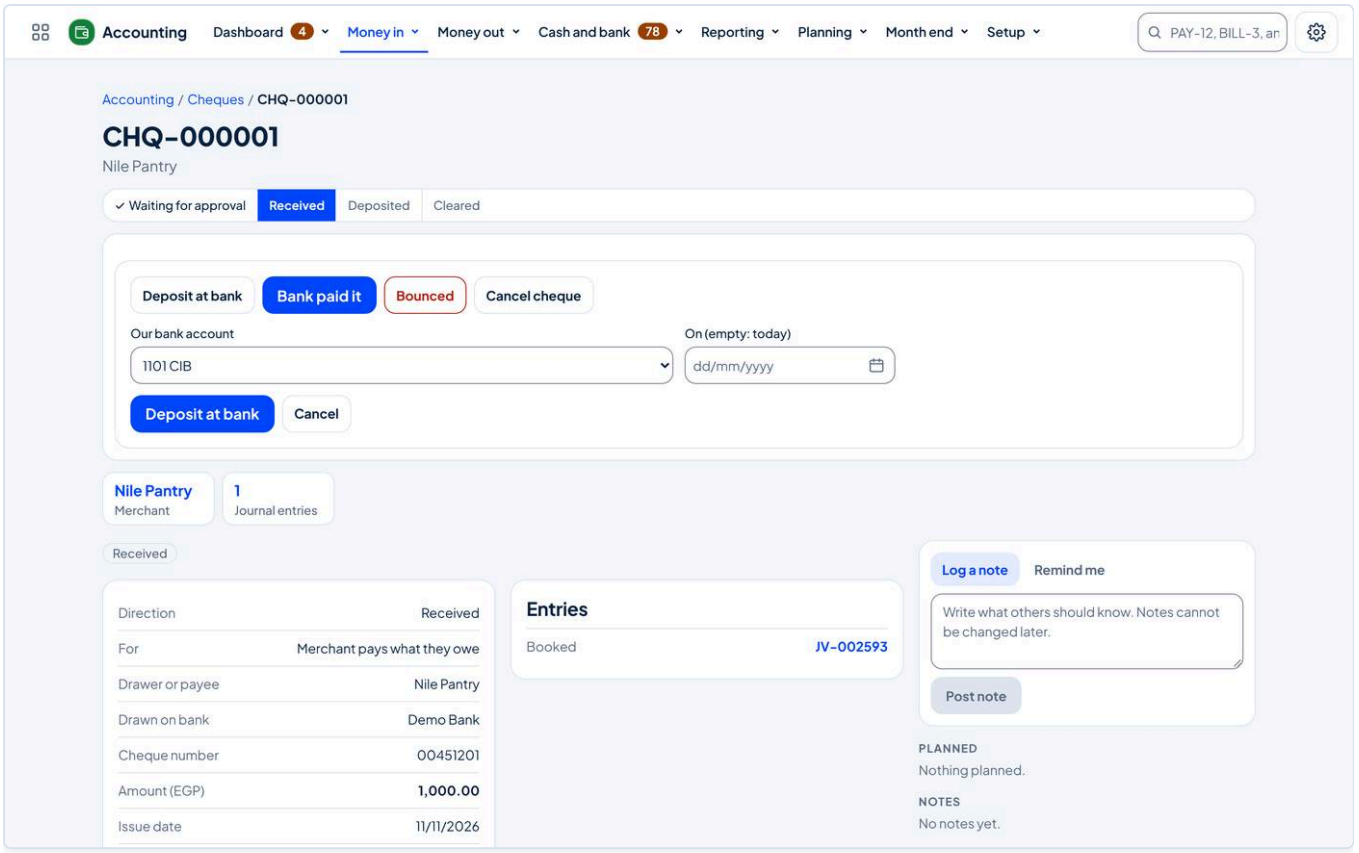
Waiting for approval, with numbered parts (legend below).

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Breadcrumbs	Accounting / Cheques / `CHQ-000003`	Back to the register
2	Title	Register number and party	None
3	Status bar	Current step blue filled; passed steps ticked; Bounced or Cancelled red	None
4	Actions	Deposit at bank (white, received only), Bank paid it (blue), Bounced (red), Cancel cheque (white), each opening a small form with Our bank account, On (empty: today), and the reason where needed	See the table below
5	Smart buttons	Merchant or supplier, Journal entries count	Open them
6	Fields	Direction, For, Drawer or payee, Drawn on bank, Cheque number, Amount, Issue date, Due date, Merchant, Our bank account, Status (with the bounce reason or cancel note), Registered by	None
7	Waiting line	"Waiting for a second person in Approvals." with a grey pill	None
8	Open Approvals	Blue link	Opens `/finance/approvals`

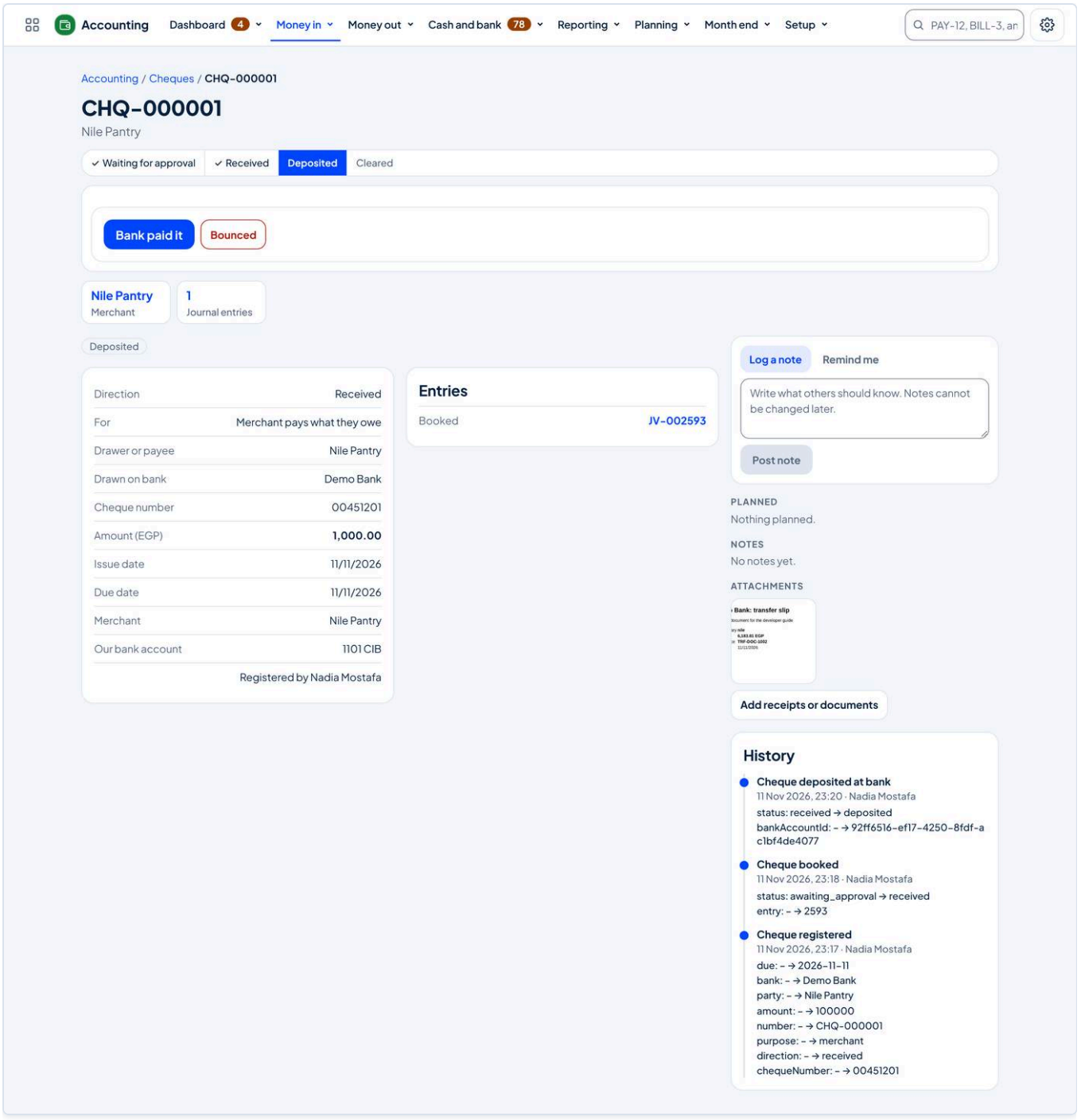
#	Part	What it shows (colors, icons, wording)	What clicking it does
9	History	Cheque registered, booked, deposited, cleared, bounced, cancelled	None



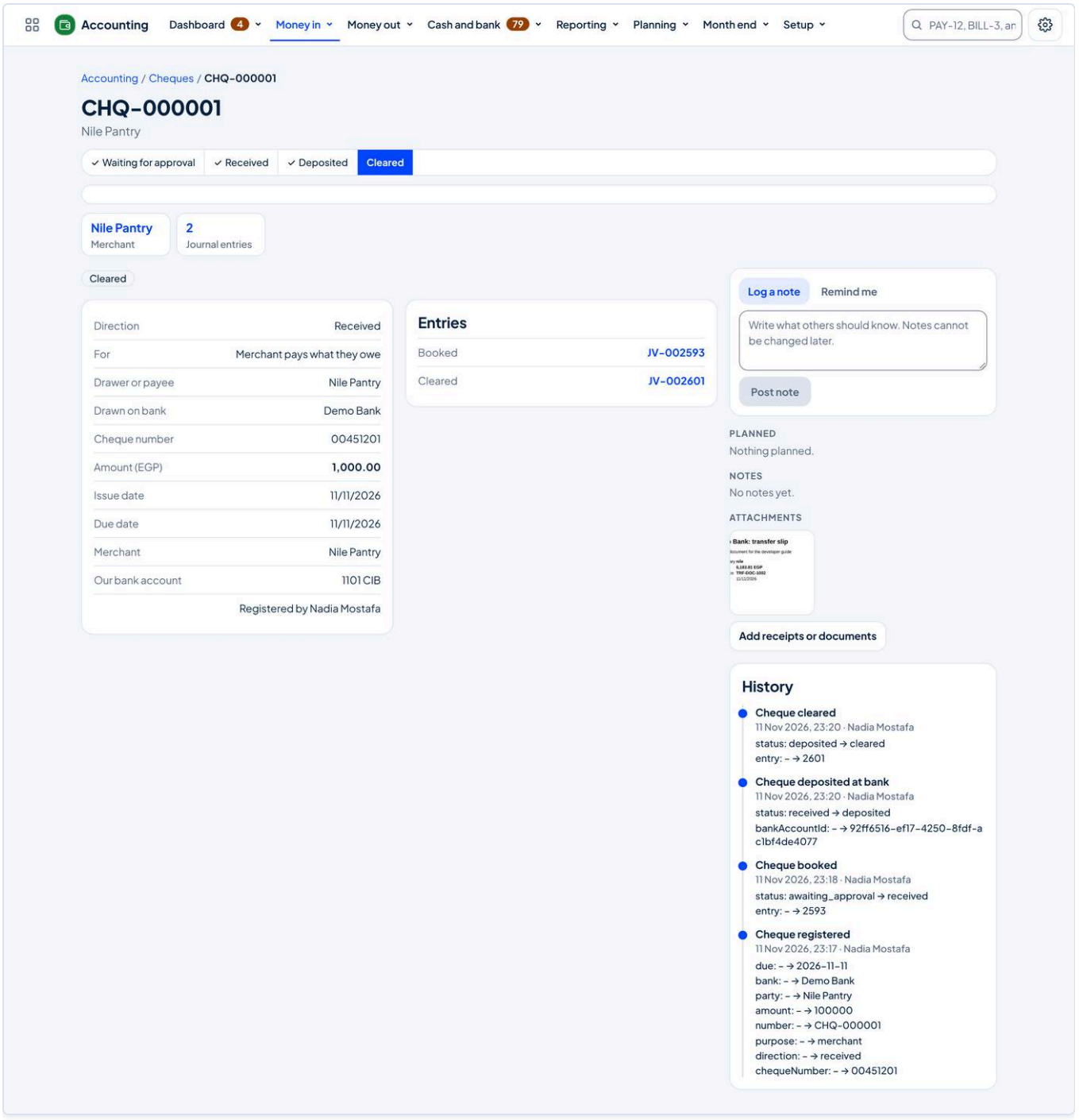
Received (approved by the second user): entry Booked `JV-002593` (Dr 1151 cheques under collection, Cr the merchant wallet); actions Deposit at bank, Bank paid it, Bounced, Cancel cheque.



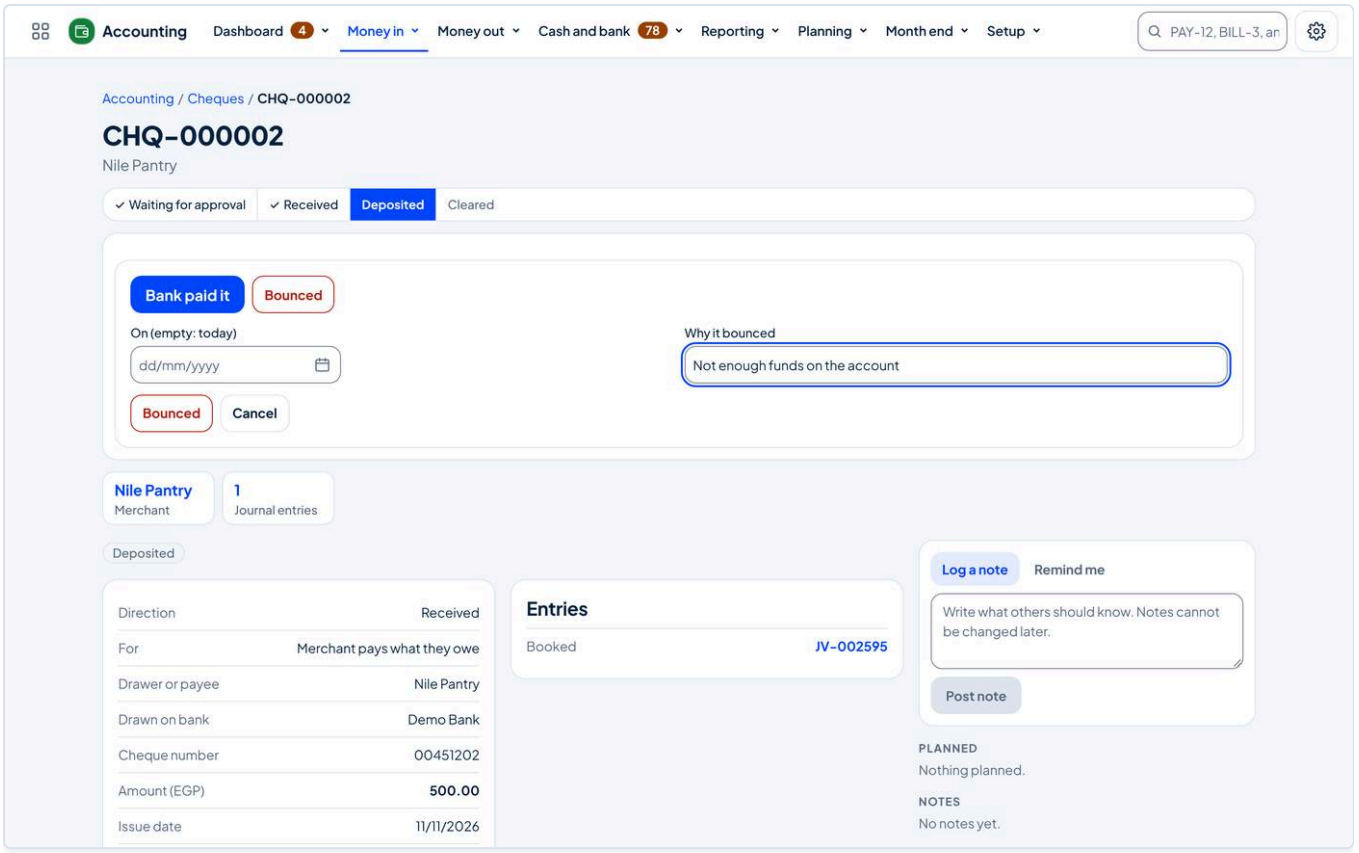
Deposit at bank opens Our bank account and On (empty: today), with Deposit at bank and Cancel.



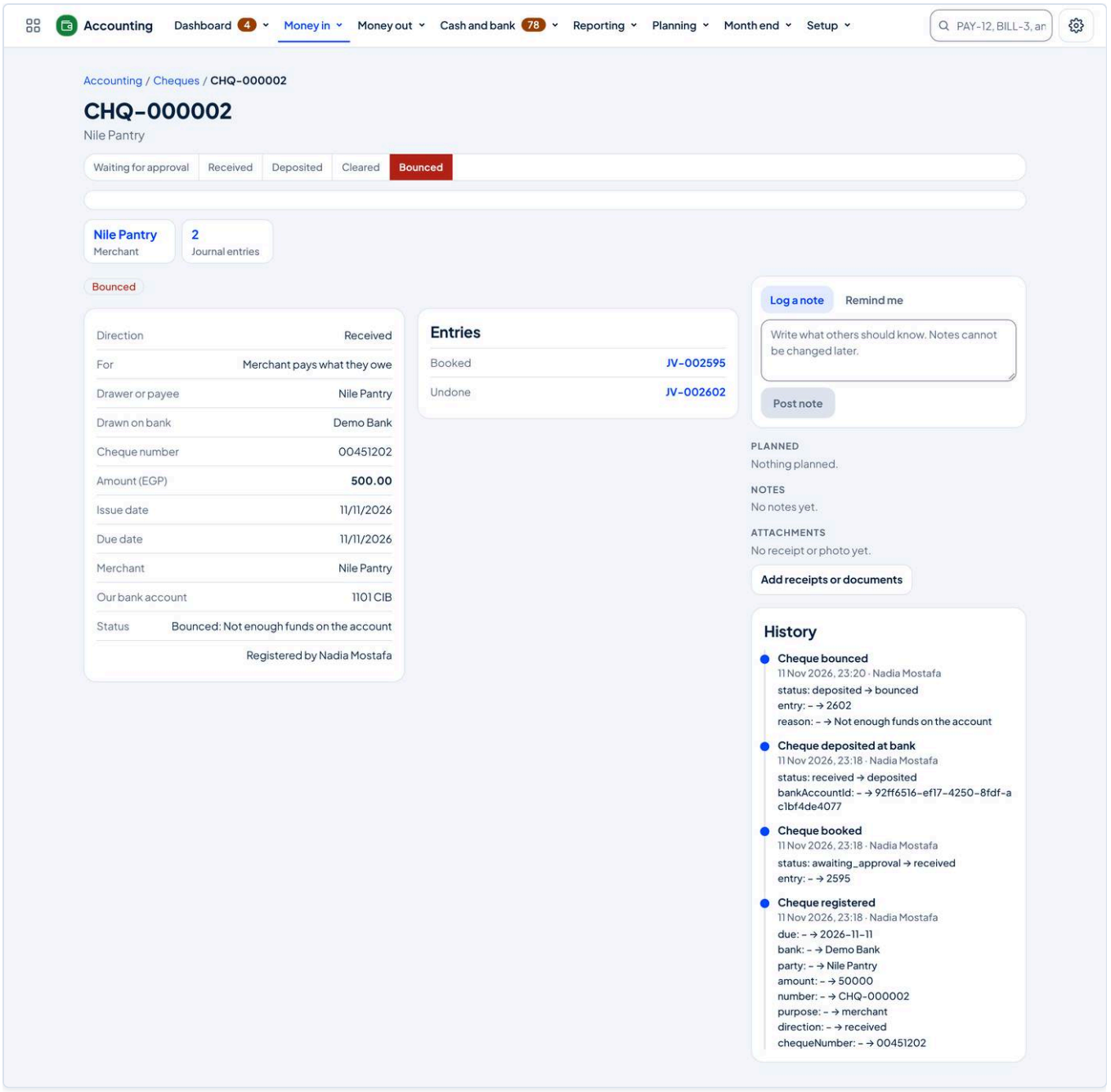
Deposited: Received ticked, Deposited filled; Cancel cheque is no longer offered.



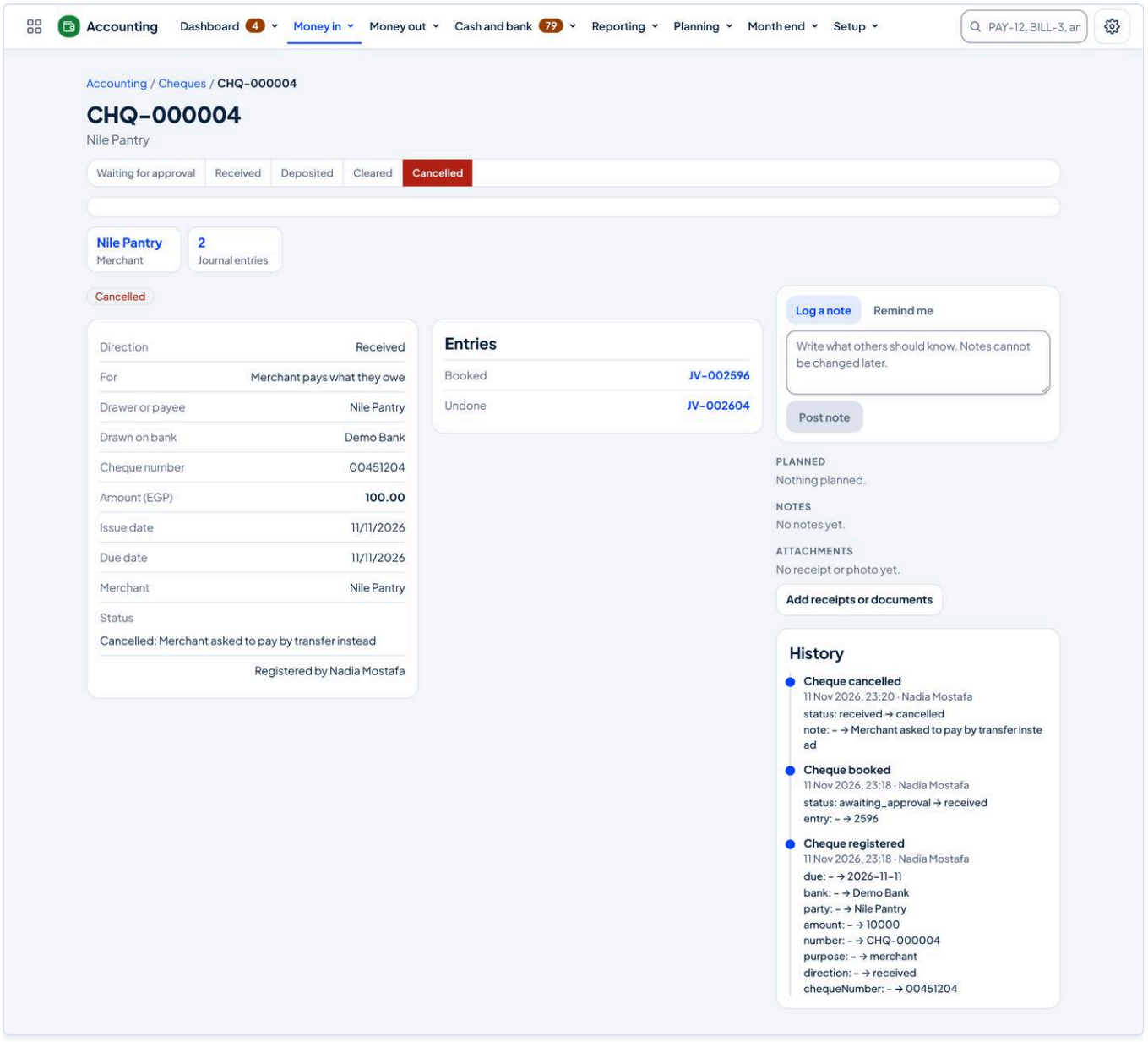
Cleared: every step ticked and Cleared filled; Entries Booked and Cleared (Dr bank, Cr 1151); no actions left.



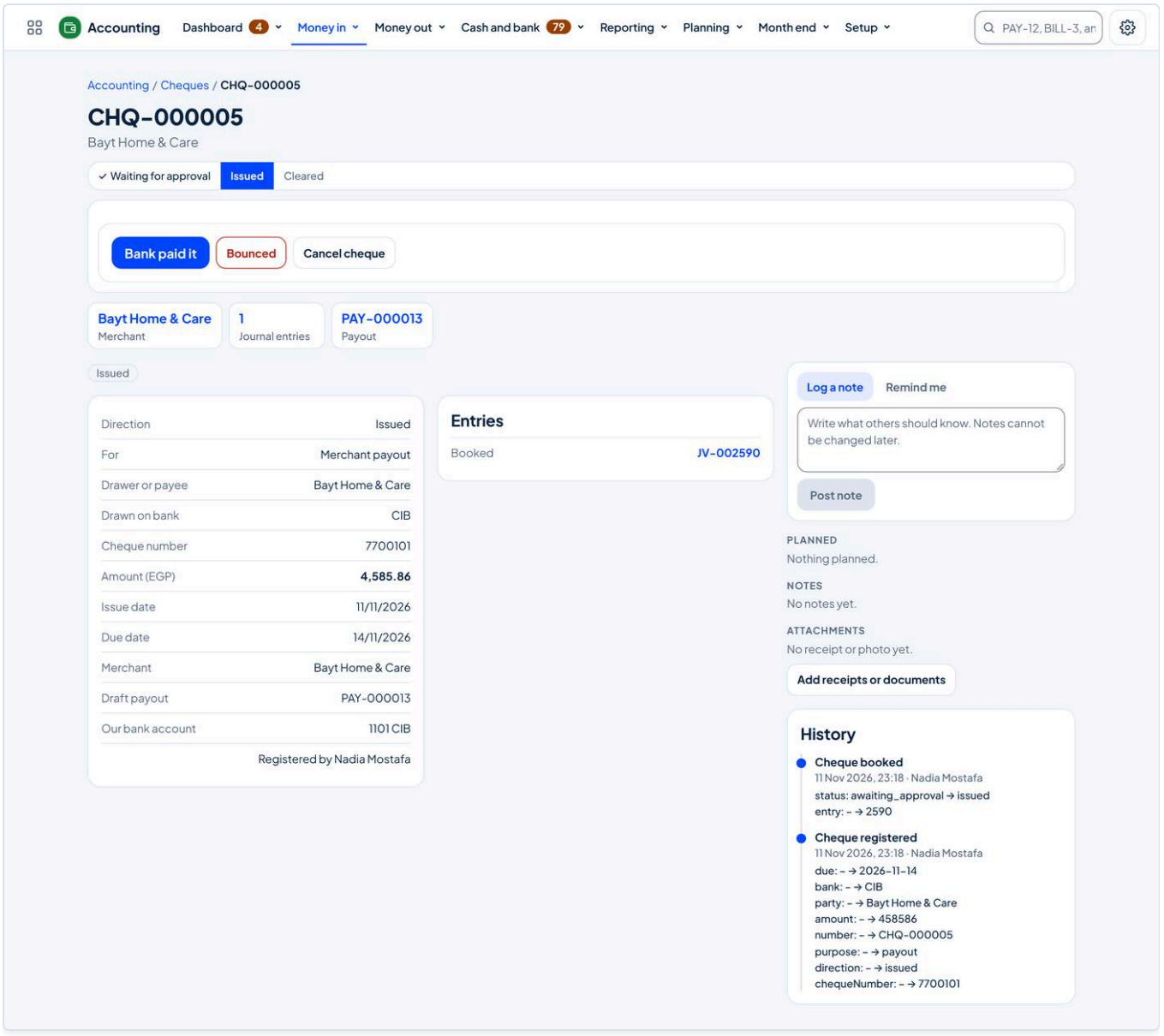
Bounced opens "Why it bounced" and On; the red Bounced button is enabled after 3 characters.



Bounced: a red Bounced segment, a red Bounced pill, Status "Bounced: Not enough funds on the account", entries Booked and Undone (the booking turned round); a `cheque_bounced` alert is raised and the debt is back on the merchant.



Cancelled before deposit: a red Cancelled segment, "Cancelled: Merchant asked to pay by transfer instead", entries Booked and Undone.

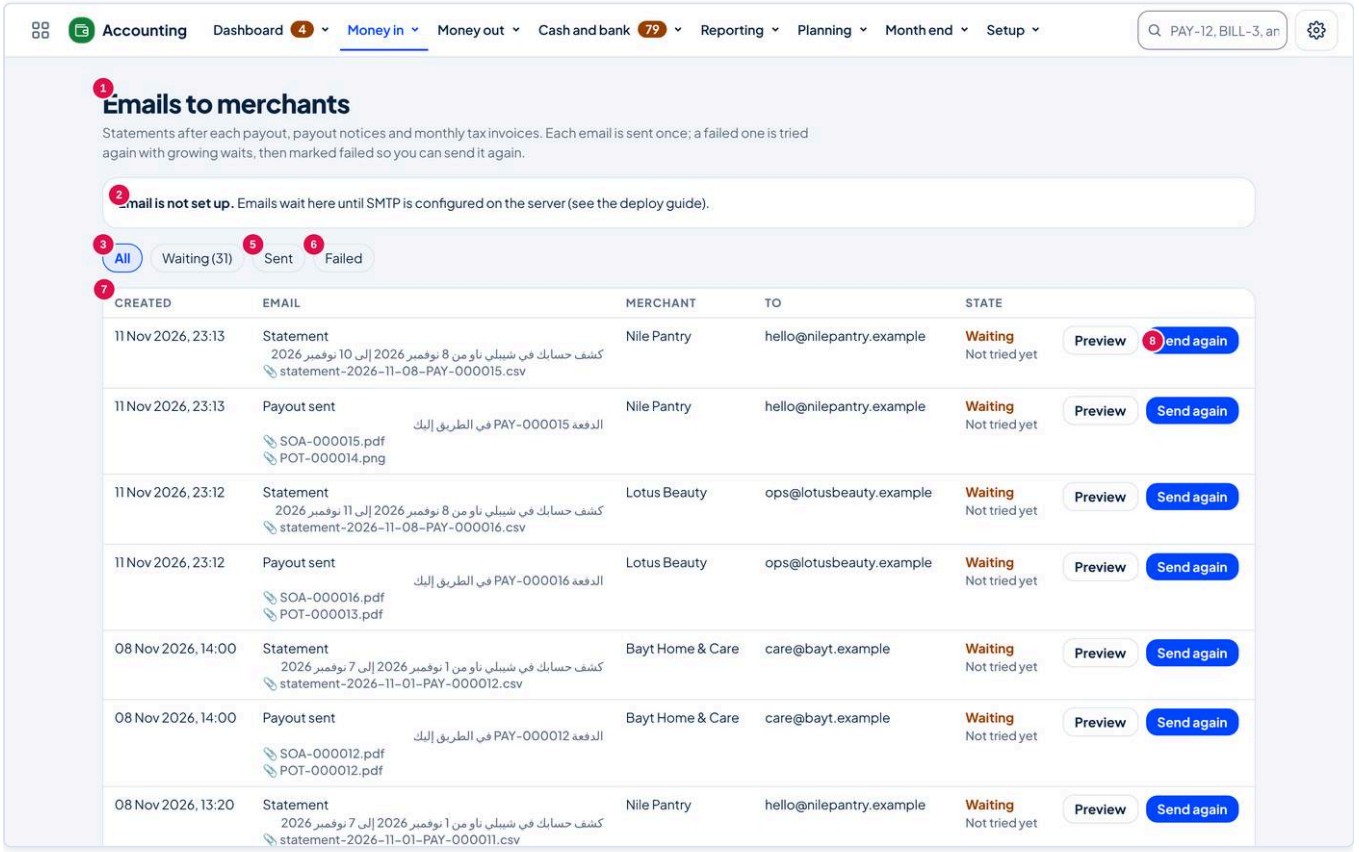


Issued (for a merchant payout): status bar Issued then Cleared; booked against 2451 cheques payable; the payout became Paid at once (see the payout page above).

Action	Route	Rule / limits	Result
Deposit at bank	`POST /finance/cheques/<id>/deposit`	Received cheques in state Received	Deposited, no money moves yet
Bank paid it	`.../clear`	Received: never before its due day, into the bank; issued: out of the bank, which must hold it	Cleared with its entry
Bounced	`.../bounce {reason}`	Open cheques; reason of 3 characters	Booking reversed, bill reopened or payout cancelled, alert
Cancel cheque	`.../cancel {note}`	Waiting for approval, Received or Issued only (before deposit or clear)	Booking reversed where booked

13.13 Emails to merchants

Address: `/finance/emails?status=`, menu Money in > Emails to merchants. Who: accounting view reads; Send again needs accounting use. Emails are written to `email_outbox` in the same transaction as their event and sent by the API job through the Mailer; without SMTP nothing is sent.



Emails to merchants with numbered parts: the payout of 11/11 queued a Statement (with its CSV) and a Payout sent notice (with the SOA PDF and the POT file). Address `/finance/emails`.

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Title and intro	"Each email is sent once; a failed one is tried again with growing waits, then marked failed"	None
2	Not set up box	"Email is not set up. Emails wait here until SMTP is configured on the server"	None
3	All	Selected chip in blue	Shows every email
(next)	Waiting (n)	Chip with the count	`?status=pending`
5	Sent	Chip	`?status=sent`
6	Failed	Chip	`?status=failed`
7	Table	Created, Email (kind: Statement, Payout sent, Tax invoice, Payment reminder; the subject in the merchant's chosen language; each attachment with a clip icon), Merchant, To, State (Waiting in amber, Sent, Failed in red) with "Not tried yet" or "n tries"	None

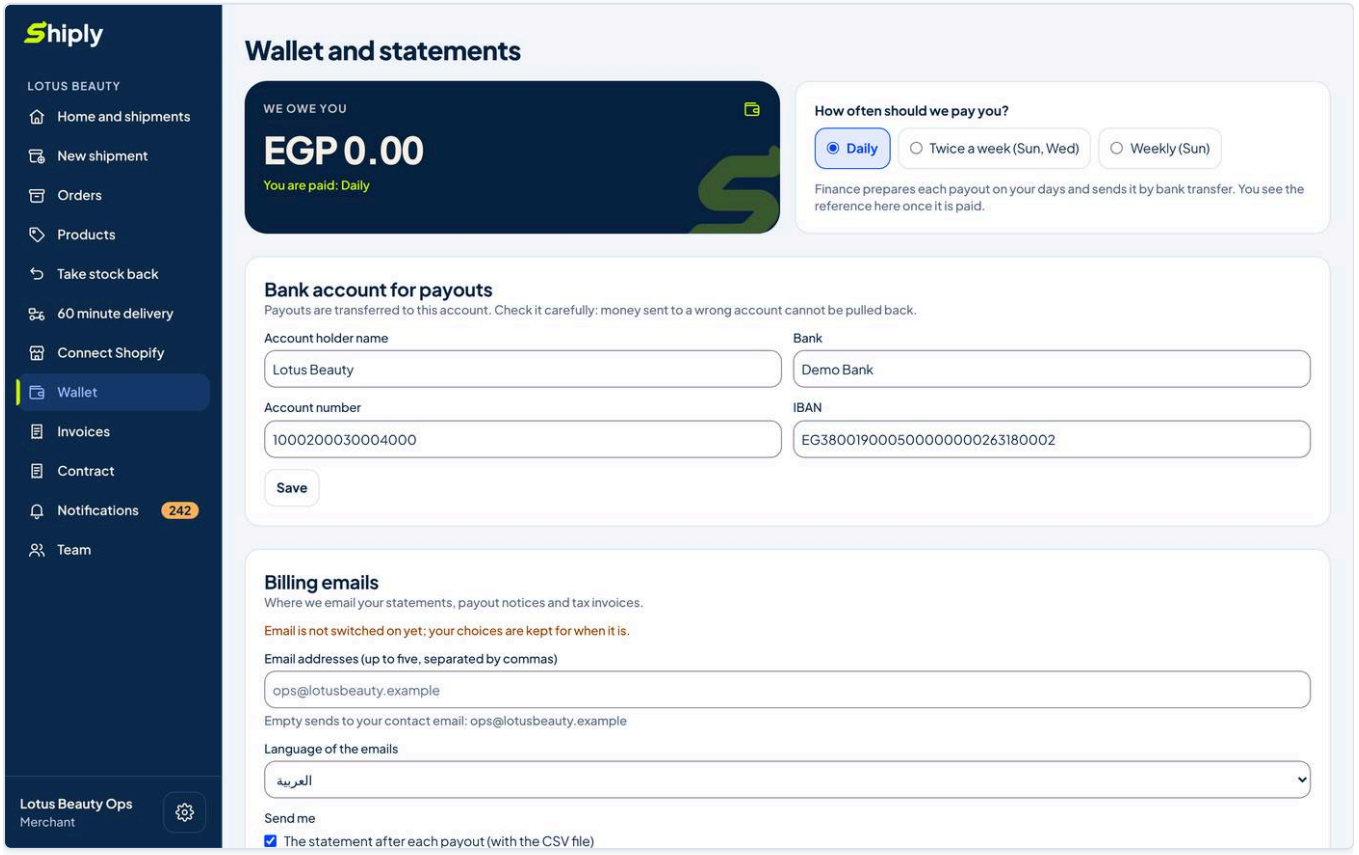
#	Part	What it shows (colors, icons, wording)	What clicking it does
8	Preview and Send again	White Preview button; blue Send again	Preview shows the email below the table with Close; Send again queues it again ("It will be sent again in a moment.")

Preview of a queued email shown under the list, with Close.

13.14 The merchant's side: Wallet with SOA and POT

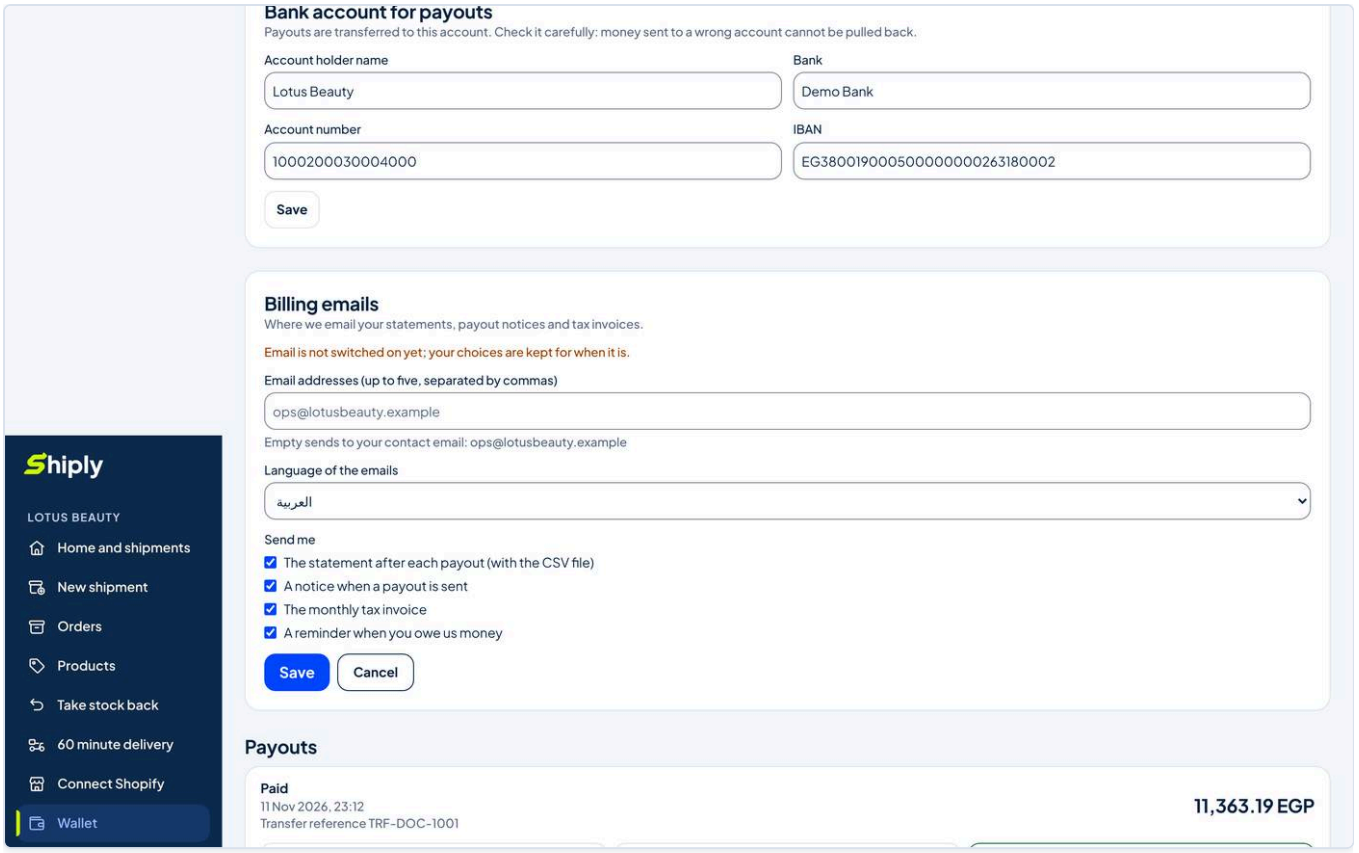
Address: `/merchant/wallet`, merchant sidebar > Wallet. Who: a merchant user with the money right (owners always). Data: `/merchant/finance/*`, limited to the merchant's own payouts; a proof file is served to the

merchant only once the payout is paid.



The top of the merchant wallet: the Navy balance card "We owe you EGP 0.00" with "You are paid: Daily" in Lime, the schedule choice, the bank account for payouts and the billing emails. Address `/merchant/wallet`.

Control	What it does	Rule / limits	Result
Balance card	"We owe you" or "You owe us" with the amount	Summed from the wallet lines	None
How often should we pay you?	Daily, Twice a week (Sun, Wed), Weekly (Sun)	`PATCH /merchant/finance/payout-schedule`, audited	"Saved. Your next payout follows this schedule."
Bank account for payouts	Account holder name, Bank, Account number, IBAN, Save	IBAN is EG and 27 digits; finance sees the same values	Saved; payouts without it stay out of the bank file
Billing emails	Addresses (up to five), language, which emails to send	Empty sends to the contact email	Saved

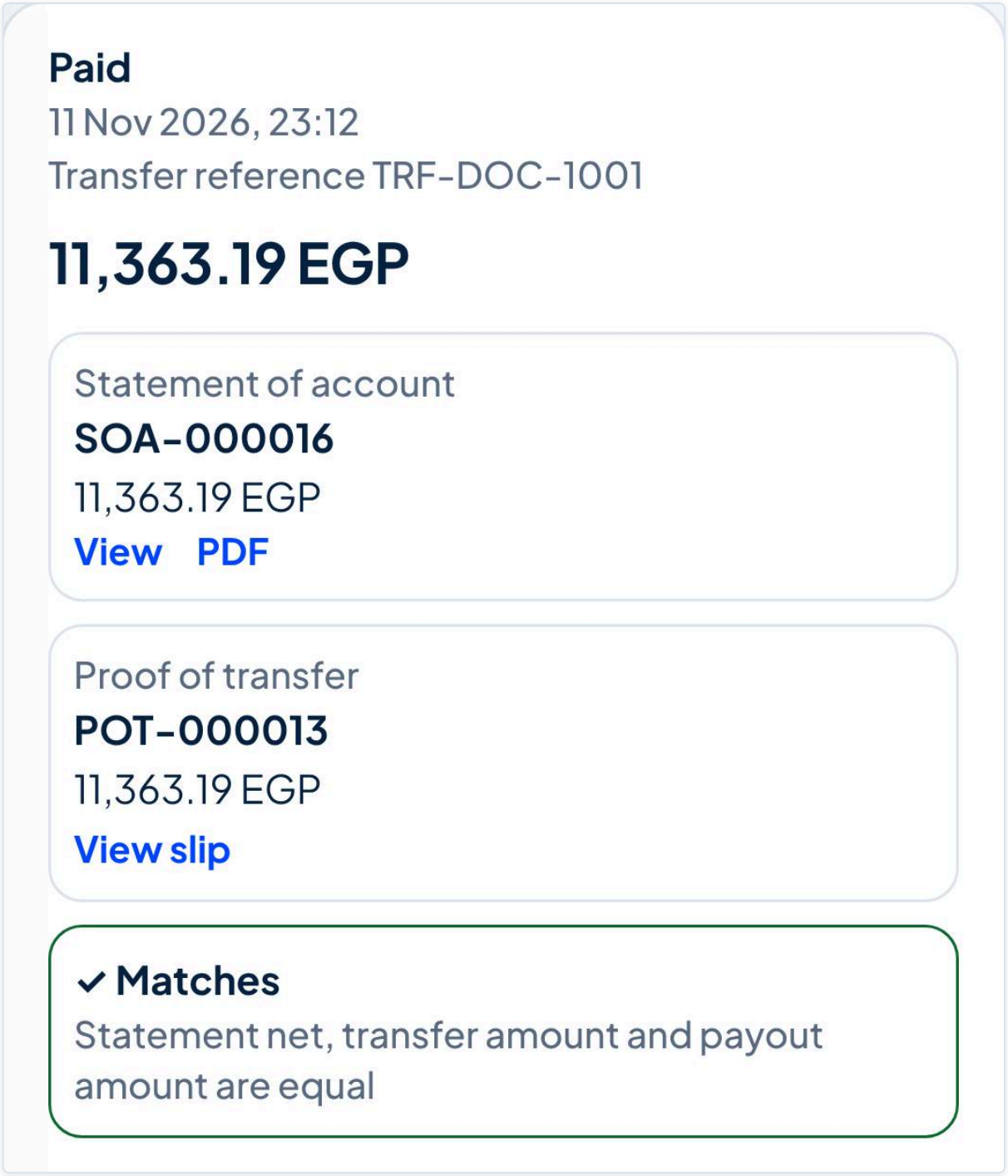


The Payouts section: every payout with its statement and proof side by side.

One paid payout as the merchant sees it, with numbered parts (legend below).

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	State	"Paid" in bold, or "Being prepared" for a draft	None
2	Paid time and reference	"Transfer reference `TRF-DOC-1001` "	None
3	Amount	Large EGP amount	None
4	Statement of account box	`SOA-` number and its net	None
5	View	Blue link	Opens `/merchant/wallet/payouts/<id>`
6	PDF	Blue link	Downloads the SOA PDF in the page language
7	Proof of transfer box	`POT-` number and its amount; "Shown once paid" while a draft; "Paid by cheque" for a cheque payout	None
8	View slip	Blue link	Opens the bank slip

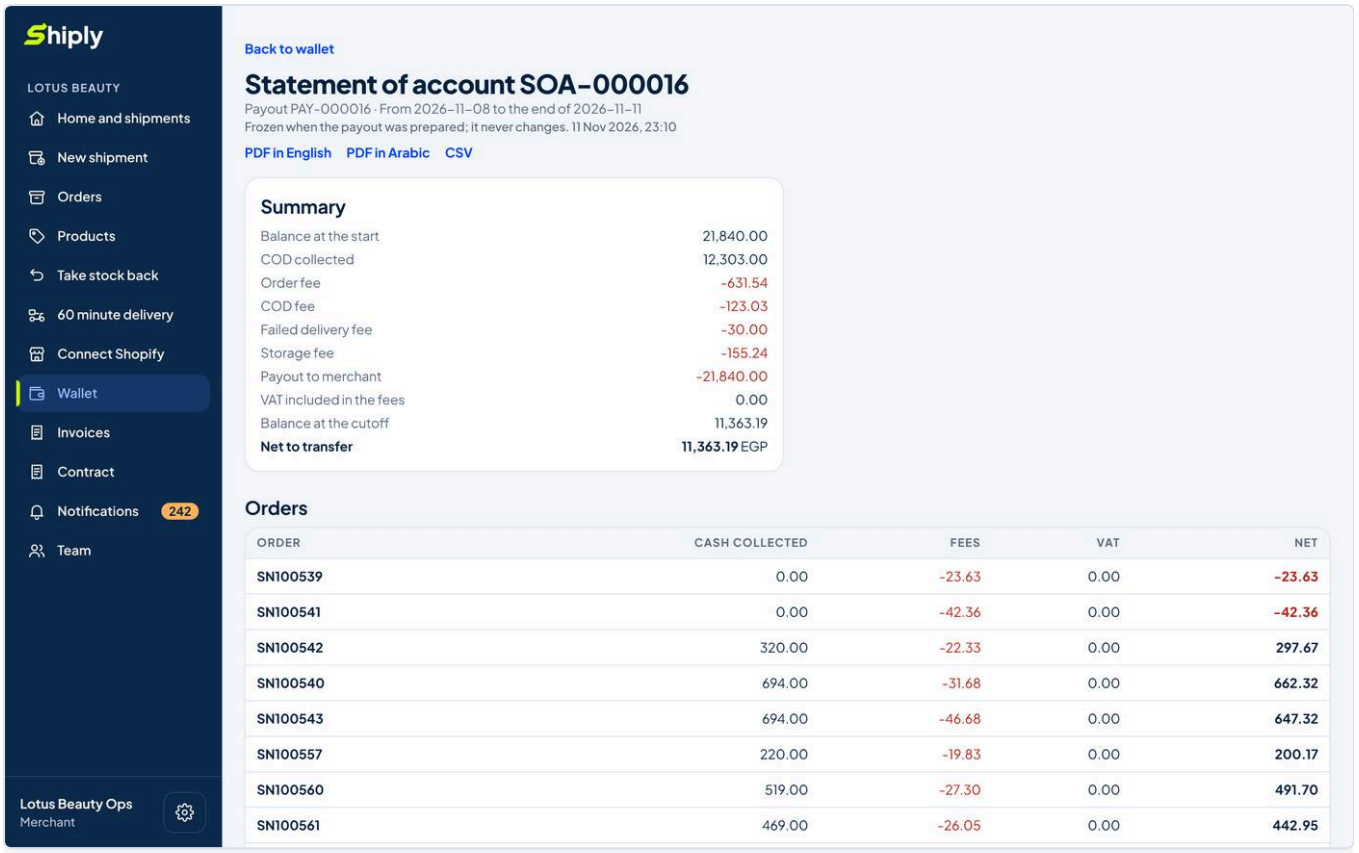
#	Part	What it shows (colors, icons, wording)	What clicking it does
9	Matches mark	Green outlined box "✓ Matches" and "Statement net, transfer amount and payout amount are equal"; otherwise "Check with us" (paid without a match) or "Shown once paid"	None



The same payout card on a phone: statement, proof and the Matches mark stack in one column.

The merchant's SOA page

Address: `/merchant/wallet/payouts/<id>`. It reads the frozen statement (`getPayoutStatement` with the merchant's id; another merchant's payout answers 404).



Statement of account `SOA-000016` as the merchant sees it: Back to wallet, payout and period, "Frozen when the payout was prepared; it never changes." with the time, PDF in English, PDF in Arabic, CSV, the Summary (charges in red) and the Orders table (Cash collected, Fees, VAT, Net in bold, red when below zero); Other movements below.

Control	What it does	Rule / limits	Result
Back to wallet	Returns to `/merchant/wallet`	None	None
PDF in English, PDF in Arabic, CSV	Download the frozen statement	Same file finance sends by email	File
Summary	Balance at the start, each kind, VAT included in the fees, Balance at the cutoff, Charges after the cutoff held back (when any), Net to transfer	Frozen numbers	None
Orders	Order, Cash collected, Fees, VAT, Net	One row per order of the period	None
Other movements	Day, Type, Reference, amount (payouts, receipts, adjustments)	"Nothing in this period." when empty	None

13.15 Status summary

Record	States and how they look
Approval request	Waiting (blue segment), Approved (blue segment, Waiting ticked), Rejected and Withdrawn (red segment); in the list every state is a grey outlined pill with its word
Payout	Draft (blue segment; list pill "Draft"), Paid (blue segment, Draft ticked), Cancelled (red segment); the merchant sees "Being prepared" or "Paid"

Record	States and how they look
Tax invoice, credit note, debit note	Not sent (Waiting for signature), Signed (Signed, sending), Submitted (Sent, checking), Accepted (Valid); Invalid and Rejected in red with Send again. Only Not sent could be shown: the demo has no ETA connection
Cheque received	Waiting for approval, Received, Deposited, Cleared; Bounced and Cancelled in red
Cheque issued	Waiting for approval (above the limit), Issued, Cleared; Bounced and Cancelled in red
Email	Waiting (amber), Sent, Failed (red)

14 Screens: Money out

This chapter walks through every screen of the Accounting app's **Money out** menu exactly as a user sees it: what is on the page, what each control does, the rule behind it, who may use it and what the result looks like. The menu holds, in this order: Record an expense, Suppliers, Bills (page title Supplier bills), Purchase orders, Expenses and transfers (page title Money movements), Petty cash, Rider withdrawals, Expense claims, Riders (page title Rider wallets), Rider offices, Payroll (page title Salaries) and Payslips, both only with the salaries switch, Recurring expenses and Equipment (page title Prepaid and equipment). Every page keeps its own address, so a link can be shared.

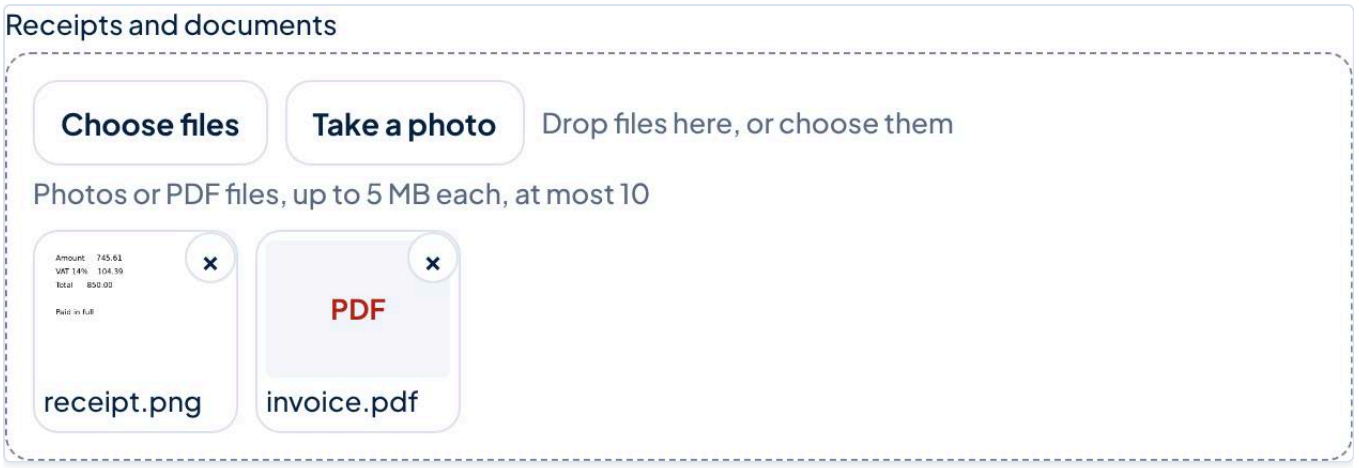
The screenshots come from the demo database. The demo team members' names are replaced by role labels (Finance User, Finance Approver, Store Staff A, Rider B and so on) and phone numbers are hidden. Red numbered circles on a screenshot point to the parts described in the legend table under it; they are not part of the app.

Rules shared by every Money out screen

- **Access.** Every screen needs the Accounting app at `view` level. Saving anything needs `use`; a few decisions need `manage` (named per control below). People limited to some stores see only those stores' rows, boxes and riders. The API checks every call again (AccessGuard), so a hidden button is never the only protection: a refused call answers `403 no_access` or `store_not_allowed`.
- **The approval limit.** Money going out (expenses, bill payments, drawings, advances, prepayments, equipment, journal entries taking money out of a bank or box) is posted at once below `finance.approval_threshold_piastres` (default 10,000.00 EGP; 0 turns the rule off). From the limit it waits in *Dashboard > Approvals* for a second person. Money coming in (owner capital, other income, journal entries putting money into a bank or box) always waits while `finance.approve_money_in` is 1. Selling or scrapping equipment always waits. Nobody approves their own request (`same_person`). See the chapter Controls and access for the request kinds.
- **Money must exist.** A bank or cash box never pays more than it holds at the moment of the payment or at any later moment (`not_enough_money`), and nothing is dated in closed books (`books_closed`) or in the future (`future_date`). See the chapter Expenses, suppliers, bills and the assets we pay for, rules shared by every movement of money.
- **Idempotency.** Every save sends a fresh key made when the form opens, so a double tap or a retry posts once.
- **Amounts.** Typed in EGP with up to two decimals (`1250.50`), sent to the API in piastres, never as floats.

The attachment field (Receipts and documents)

Most Money out forms carry the same field to attach receipts, invoices and slips *before* saving (component `AttachmentField`). It is described once here and named "attachment field" in the tables below.



The attachment field with a PNG receipt and a PDF invoice chosen, before the form is saved (/finance/expenses/new).

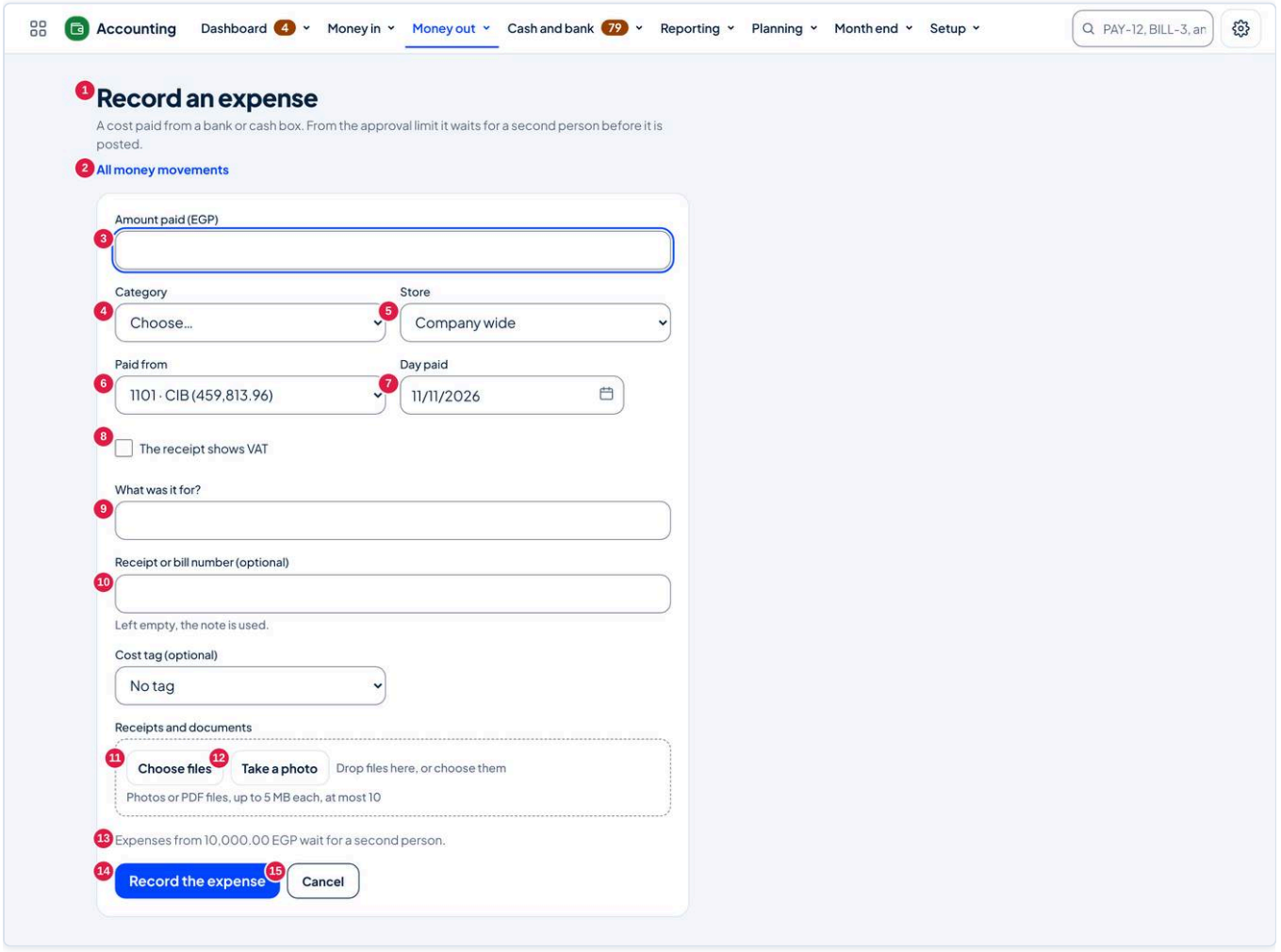
Part	What it shows	What it does and its limits
Label "Receipts and documents"	Plain caption above a dashed box	The whole box is also a drop zone: files dragged onto it are added.
Choose files	White button with a grey border	Opens the file picker (several files at once). Accepts JPEG, PNG and WebP photos and PDF files only; anything else shows "*name* is not a photo or a PDF file."
Take a photo	White button	Opens the phone camera (on a computer, the file picker).
Hint	"Photos or PDF files, up to 5 MB each, at most 10"	A PDF above 5 MB shows "*name* is larger than 5 MB."; an 11th file shows "At most 10 files." Photos are shrunk in the browser like every receipt photo.
File tiles	A photo shows its picture; a PDF shows a red "PDF" label; the file name under it; a round × button in the corner	Each file is uploaded at once (`POST /finance/files`) and waits unlinked. While a file uploads the tile says "Uploading…" and the form cannot be saved ("Wait until the files finish uploading."). A failed upload says "Upload failed. Try again." The × removes the file and deletes the upload.
On save	The tiles disappear when the form is reset	The form sends the ids as `attachmentIds` ; the save links them to the record (journal entry, bill, payment, request) in the same transaction. They then appear under Attachments in the record's chatter and in its Receipts smart button.

The list view

Suppliers, Supplier bills, Petty cash, Rider withdrawals and Expense claims are list views (component `ListView` , see the chapter UI development). Their shared controls are: the **search bar** with facet chips (a light blue rounded chip such as "Filter Open ×"; the × removes it; Backspace in an empty search box removes the last chip), **Filters** (opens a panel of three columns: Filters, Group by, Favourites), **Export** (CSV or Excel of the rows passing the filters, or only the ticked rows), **Columns** (shows or hides optional columns, remembered per person), **Per page** (20, 50, 100), the count "1–5 of 5", a tick box per row and one in the header (select all), sortable column headers (click once ascending, again descending), a **Total** row summing money columns over every passing row, and cards instead of a table on phones. The list state lives in the address (`f` filters, `g` grouping, `q` search, `sort` , `page`), so the view can be bookmarked. Filters inside one group are joined by "or", groups by "and". Favourites saves the current search under a name ("Open the list with it" makes it the default).

14.1 Record an expense

Address: `/finance/expenses/new` . **Menu:** Accounting > Money out > Record an expense (shown only to people with Accounting `use`). Also opened from the dashboard quick action and from a cash box card. **Who:** Accounting `use` to save; the page itself needs `use` . The form is the short version of the Expense tab of Money movements and calls the same `POST /finance/expenses` .



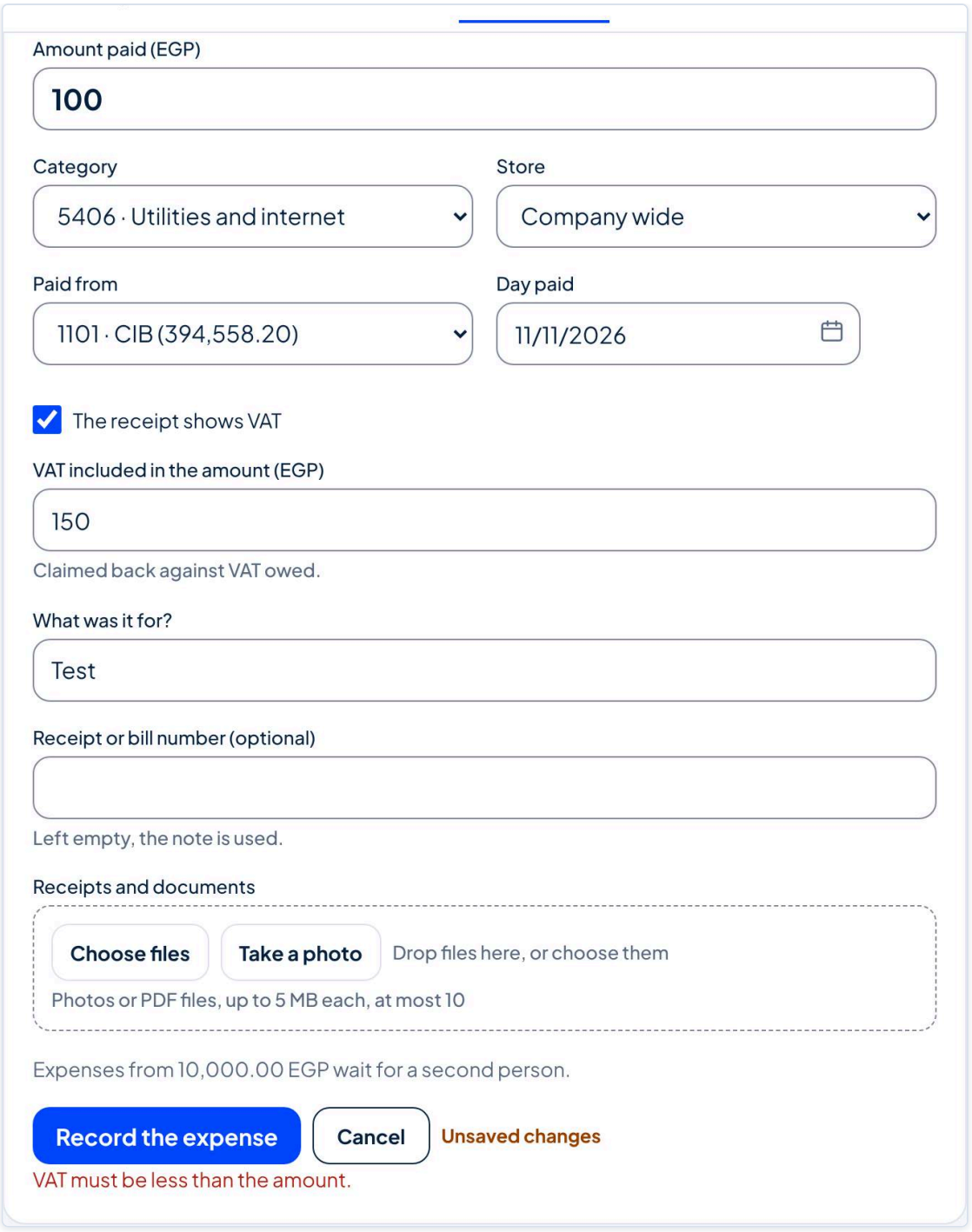
Record an expense, empty, with numbered parts (/finance/expenses/new).

#	Part	What it shows (colors, wording)	What clicking it does
1	Title and intro	"Record an expense", grey text: a cost paid from a bank or cash box; from the approval limit it waits for a second person	Nothing.
2	All money movements	Blue link	Opens Money movements on its Expense tab (the full form with cost tag, store and VAT).
3	Amount paid (EGP)	Large input, focused when the page opens (blue ring)	The total paid, VAT included.
4	Category	Select "Choose...", then every open account of groups 5400 (operating expenses) and 5500 (other expenses), as "5406 · Utilities and internet"	The expense account debited.

#	Part	What it shows (colors, wording)	What clicking it does
5	Store	Select, "Company wide" first, then dark stores and hubs "Maadi (MAA)"	Tags every line of the entry with the store, for profit per store. Company wide leaves it empty.
6	Paid from	Select of open banks and cash boxes with their balance "1101 · CIB (394,558.20)"; CIB is chosen by default	The money account credited. When the amount is above the balance a red line appears: "That is more than this account holds."
7	Day paid	Date box shown day first (dd/mm/yyyy), today by default, a calendar button	A past day is booked at that day's noon, Cairo time. A future day is refused.
8	The receipt shows VAT	Tick box, off	When ticked, a field "VAT included in the amount (EGP)" appears with the hint "Claimed back against VAT owed."
9	What was it for?	Required text, 3 to 300 characters	Becomes the entry's memo.
10	Receipt or bill number (optional)	Text, up to 100 characters; hint "Left empty, the note is used."	Becomes the entry's reference.
11, 12	Choose files, Take a photo	The attachment field	See The attachment field above.
13	Limit line	Grey: "Expenses from 10,000.00 EGP wait for a second person." When the amount reaches the limit it turns orange and bold: "From 10,000.00 EGP this waits in Approvals for a second person."	Nothing; it tells in advance whether the save posts or waits. Hidden when the limit is 0.
14	Record the expense	Blue button; reads "Send for approval" when the amount reaches the limit	Checks the form, then sends it. An "Unsaved changes" marker in orange appears beside the buttons once anything is typed.
15	Cancel	White outlined button	Asks before throwing away typed values, then empties the form and makes a new idempotency key.

Checks before sending and error messages

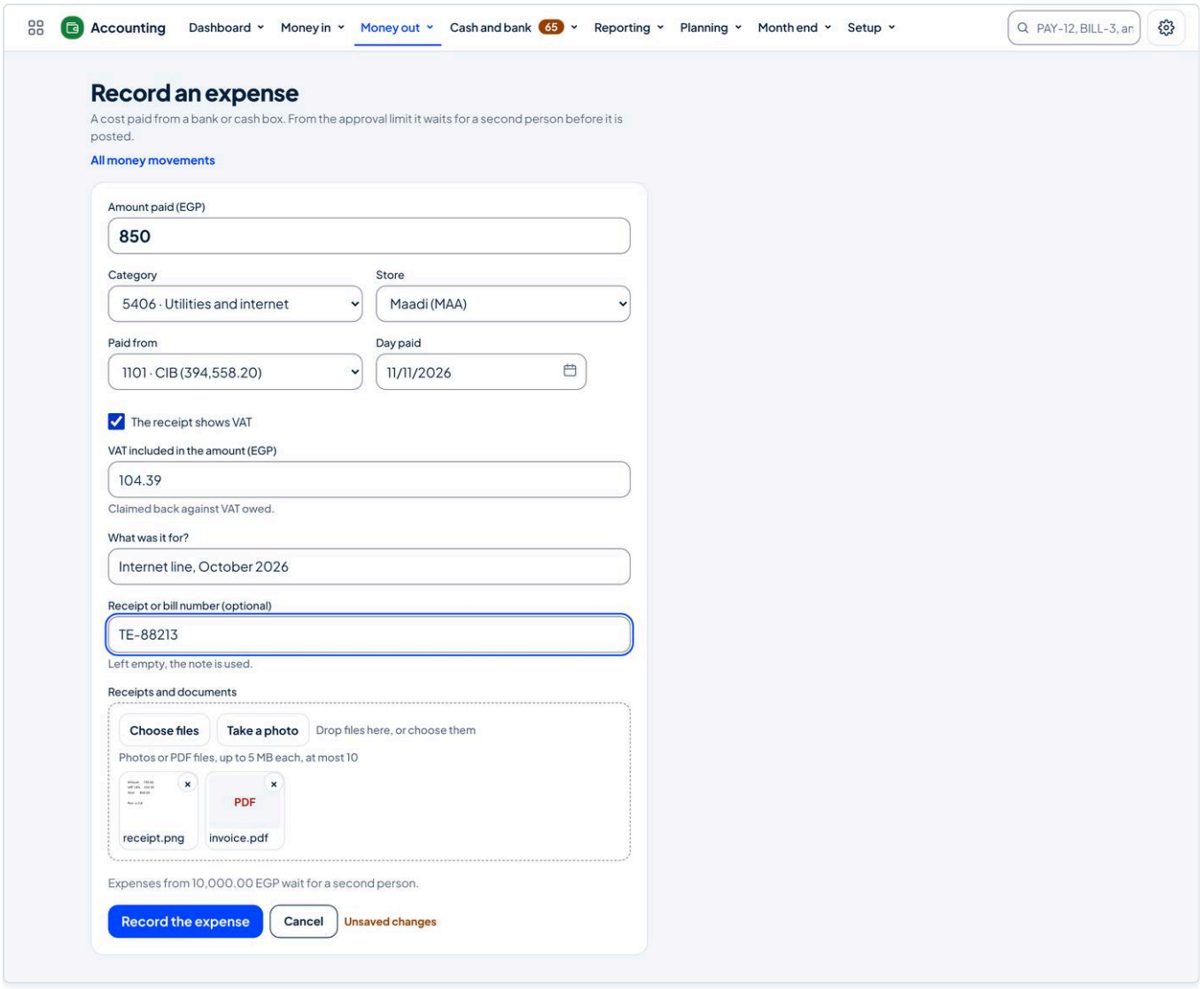
Situation	Message (red, under the buttons)
Amount empty or zero	Enter the amount paid.
VAT not an amount	Enter the VAT as an amount in EGP.
VAT equal to or above the amount	VAT must be less than the amount.
No category / no money account	Choose a category. / Choose where it was paid from.
Day not valid / in the future	Choose a valid day. / The day cannot be in the future.
Note shorter than 3 characters	Write what it was for (at least 3 characters).
API refusals	`not_enough_money`, `books_closed` and the other finance errors, in plain words.



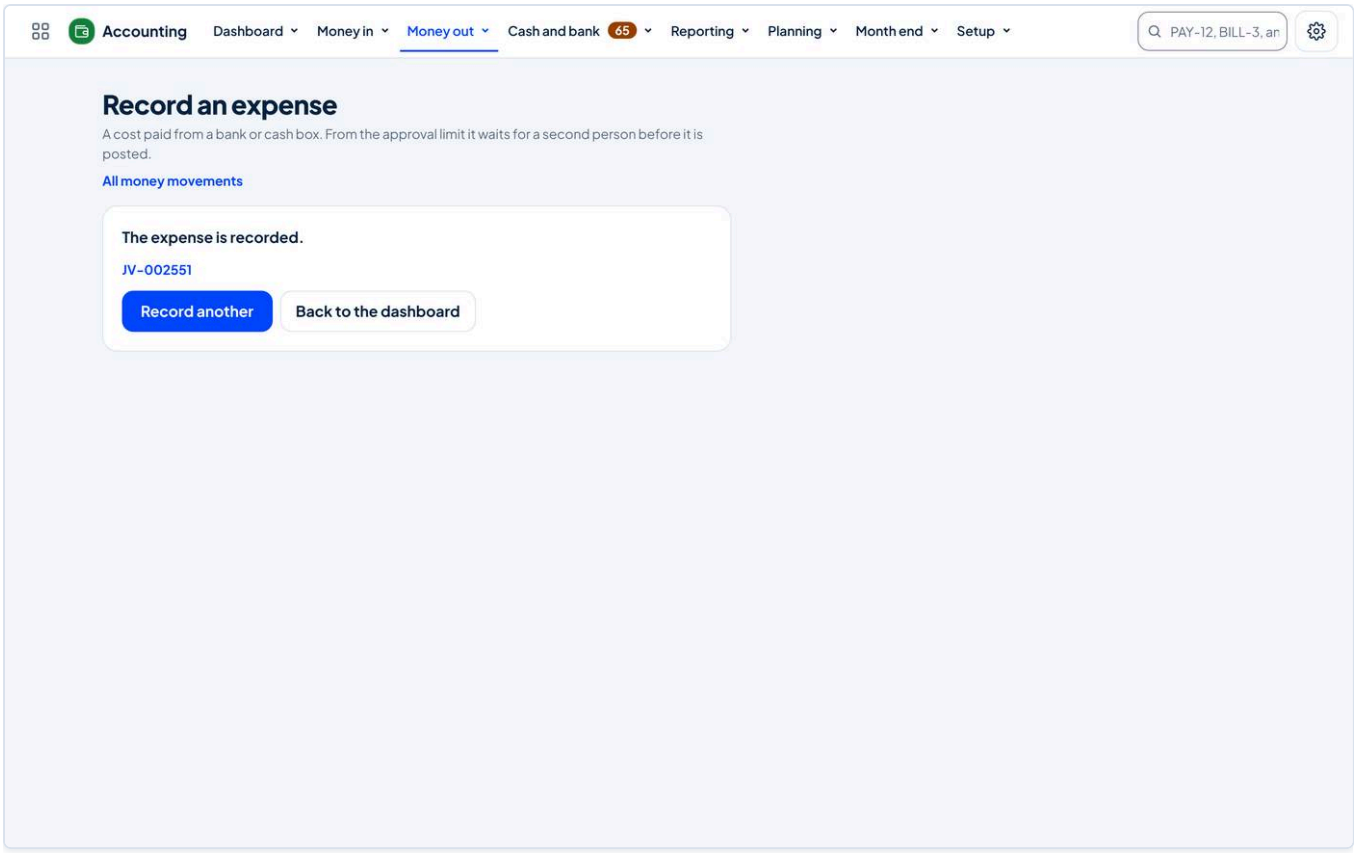
An error before sending: VAT of 150 on an amount of 100 gives "VAT must be less than the amount." The orange "Unsaved changes" marker shows the form was edited.

Posted at once

An internet bill of 850.00 EGP with 104.39 EGP VAT, tagged to Maadi, paid from CIB, with a PNG receipt and a PDF invoice, is below the limit and is posted at once. The form is replaced by a confirmation card: "The expense is recorded.", the voucher number as a blue link, and two buttons, *Record another* (empties the form with a new key) and *Back to the dashboard*.

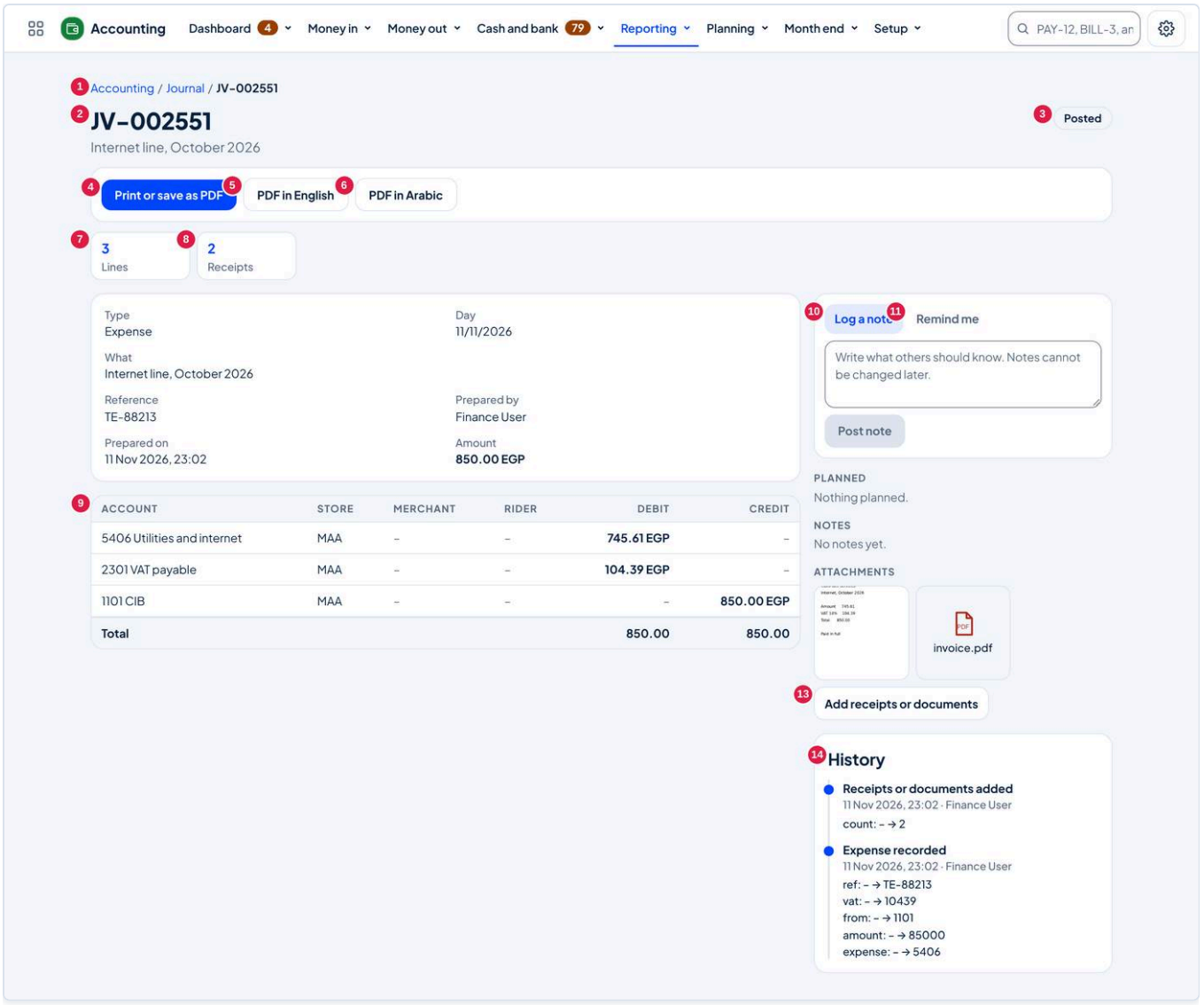


The form filled in, VAT ticked and two files attached, before saving.



The confirmation after posting, with the voucher JV-002551 as a link.

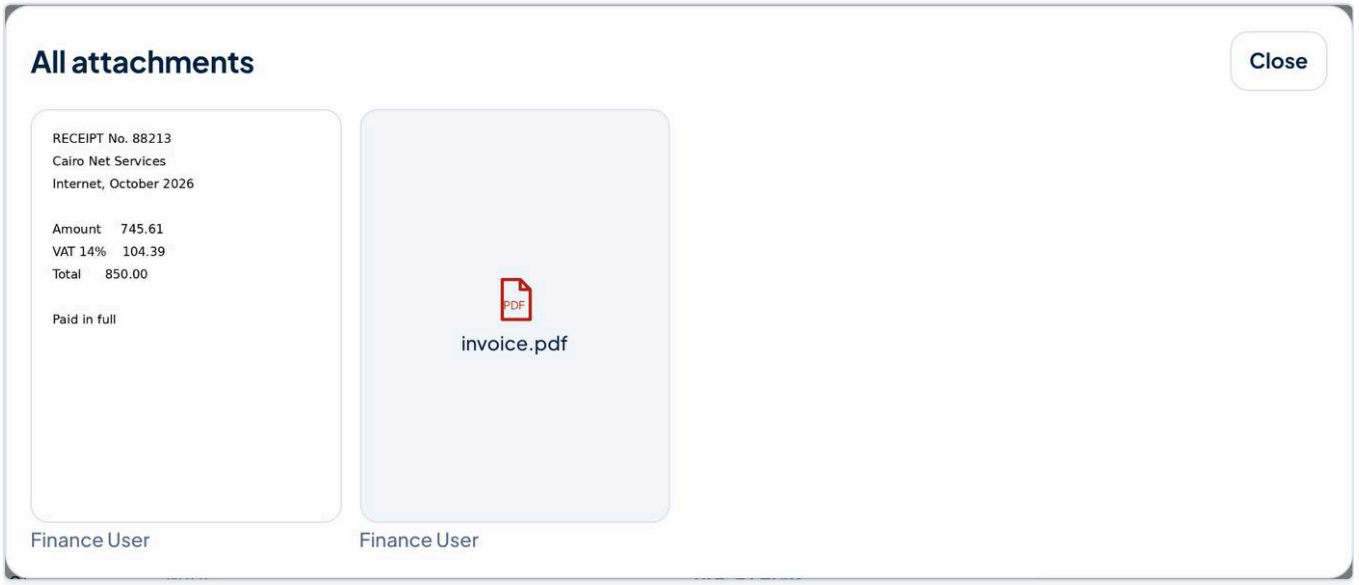
The entry (`expense` kind) is: Dr 5406 Utilities and internet 745.61, Dr 2301 VAT payable 104.39 (input VAT claimed back), Cr 1101 CIB 850.00, every line tagged MAA. See the chapter Expenses, suppliers, bills and the assets we pay for, expenses paid at once.



The resulting journal entry page (/finance/journal/<id>) with its two files under Attachments.

#	Part	What it shows	What clicking it does
1	Breadcrumbs	Accounting / Journal / JV-002551, blue links	Back to the journal list with the filters it had.
2	Voucher number and memo	"JV-002551", grey memo under it	Nothing.
3	Status pill	"Posted", grey outlined pill on the right	Nothing.
4	Print or save as PDF	Blue button	Prints the voucher from the browser (the menu is hidden in print).
5, 6	PDF in English, PDF in Arabic	White buttons	Download the voucher as a PDF made on the API.
7	Lines smart button	"3 Lines"	Scrolls to the lines table.
8	Receipts smart button	"2 Receipts"	With one file it opens the file; with several it opens the attachment viewer (below); with none it jumps to the attachments section.
9	Lines table	Account, Store, Merchant, Rider, Debit, Credit; amounts in bold with EGP; a Total row	Nothing; the lines never change after posting.

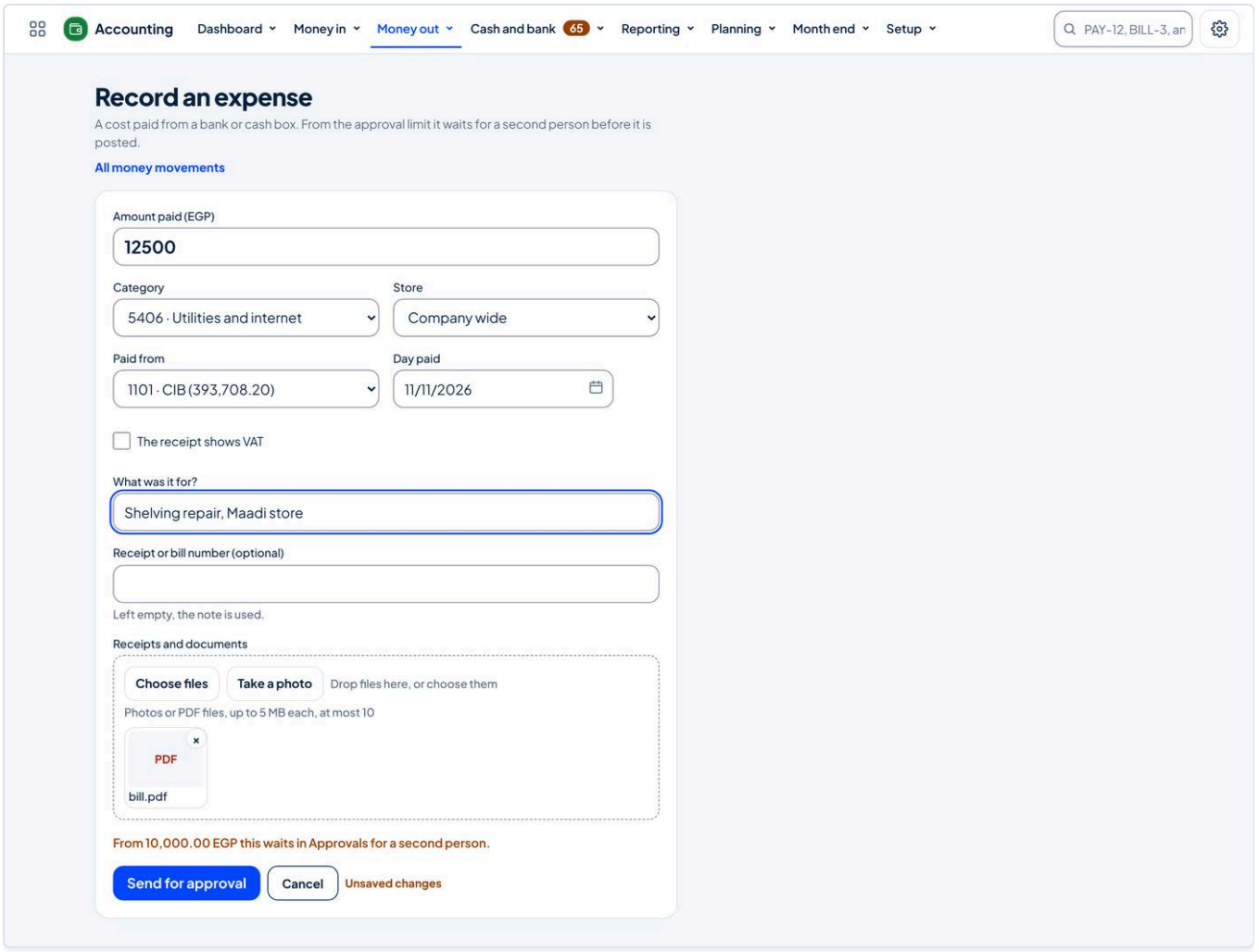
#	Part	What it shows	What clicking it does
10, 11	Log a note, Remind me	Chatter tabs; a text box "Notes cannot be changed later" and a grey Post note button	Adds an append only note, or plans a reminder on a Cairo day for a person who may open the entry.
12	Attachments	Thumbnails: the PNG as a picture, the PDF as a red document icon with its name	Opens the file in a new tab.
13	Add receipts or documents	White button	Attaches more files after posting.
14	History	Blue dots, newest first: "Receipts or documents added" (count 2) and "Expense recorded" with the values (amounts in piastres)	Nothing; read from `audit_log`.



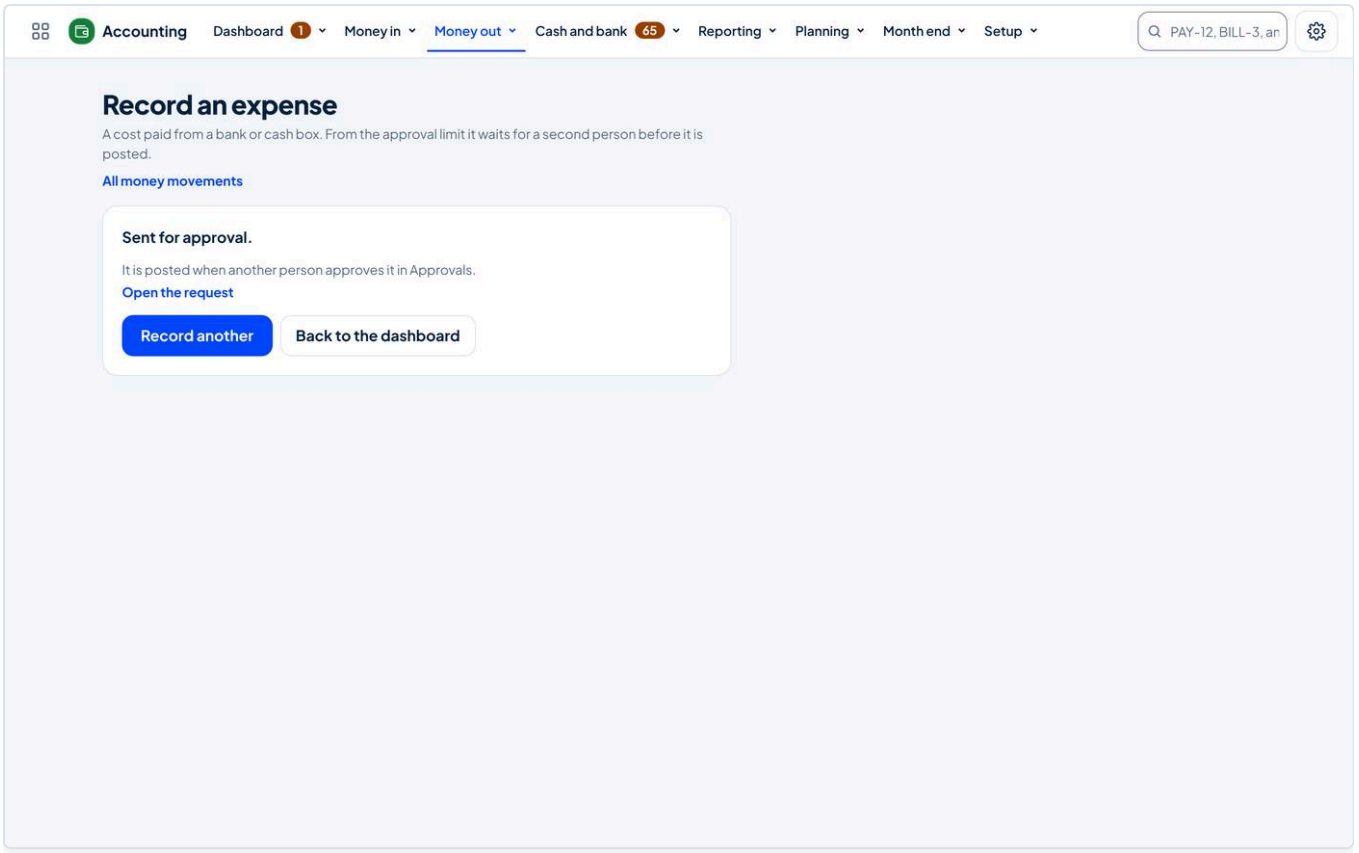
The attachment viewer opened from the "2 Receipts" smart button: one tile per file with its name, and a Close button. Clicking outside the dialog closes it.

Waiting in Approvals above the limit

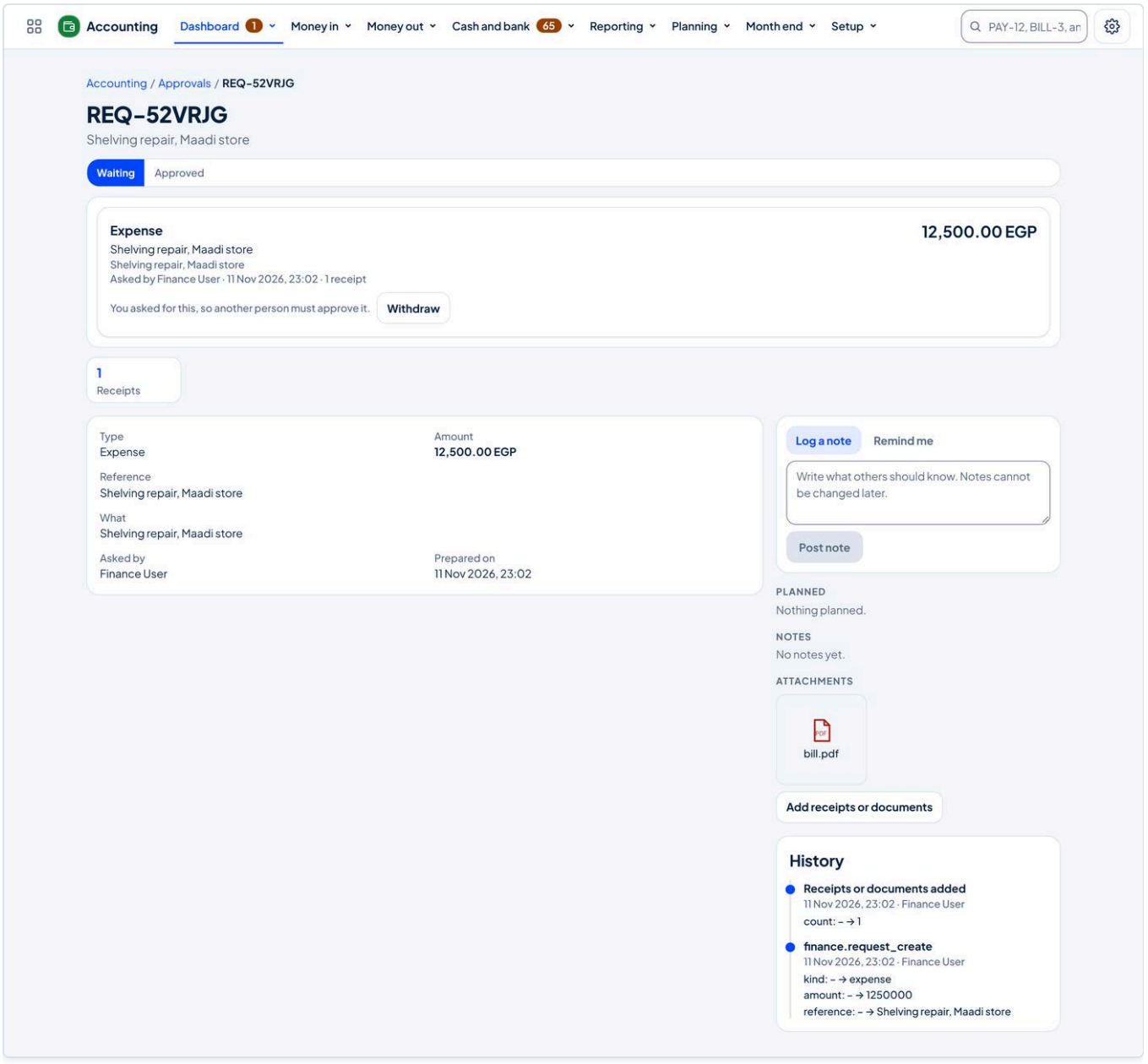
A shelving repair of 12,500.00 EGP reaches the limit: the limit line turns orange and the button reads *Send for approval*. After sending, the card reads "Sent for approval. It is posted when another person approves it in Approvals." with *Open the request*.



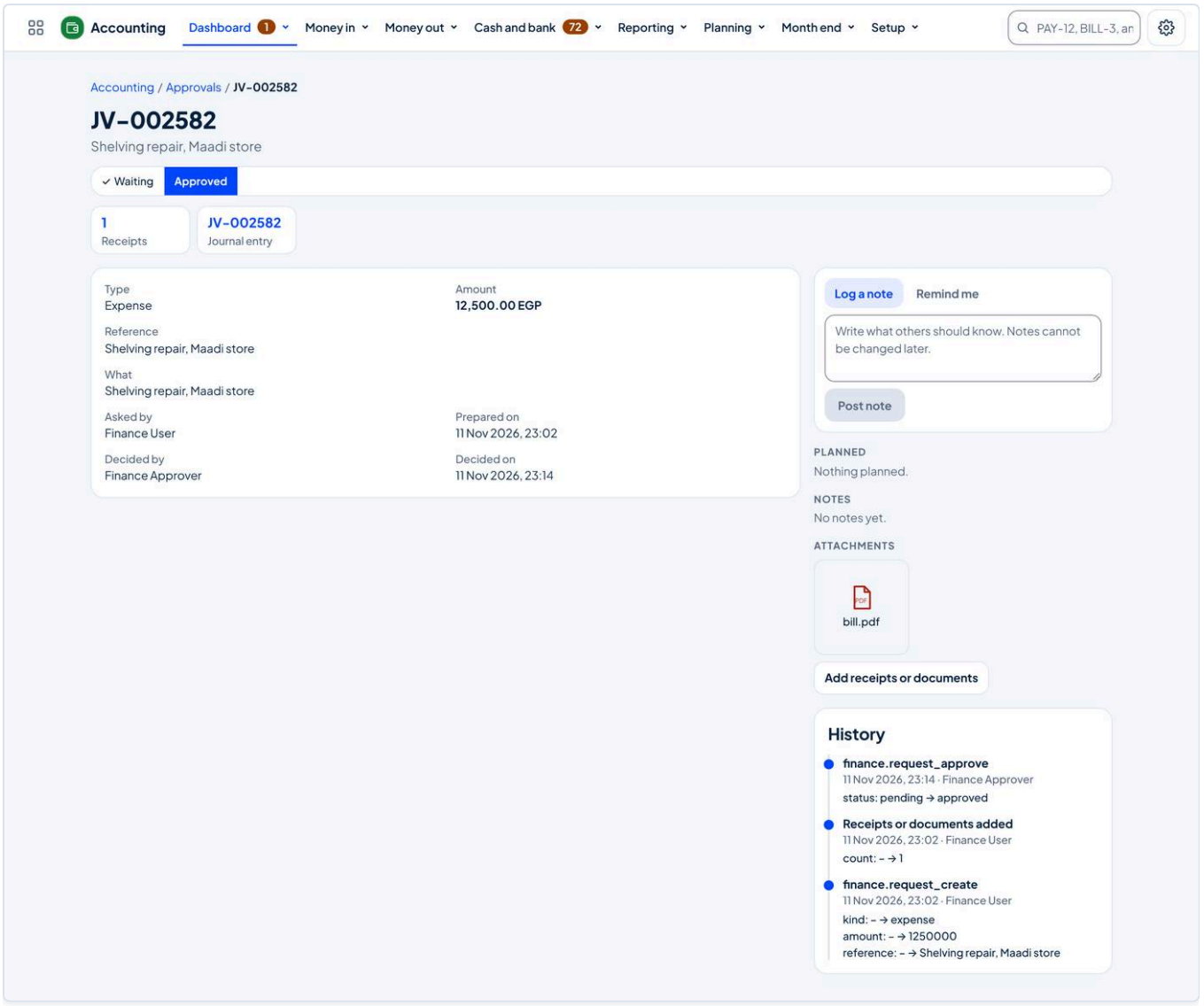
An amount above the limit: orange warning and the "Send for approval" button, with a PDF attached.



The confirmation after sending for approval.



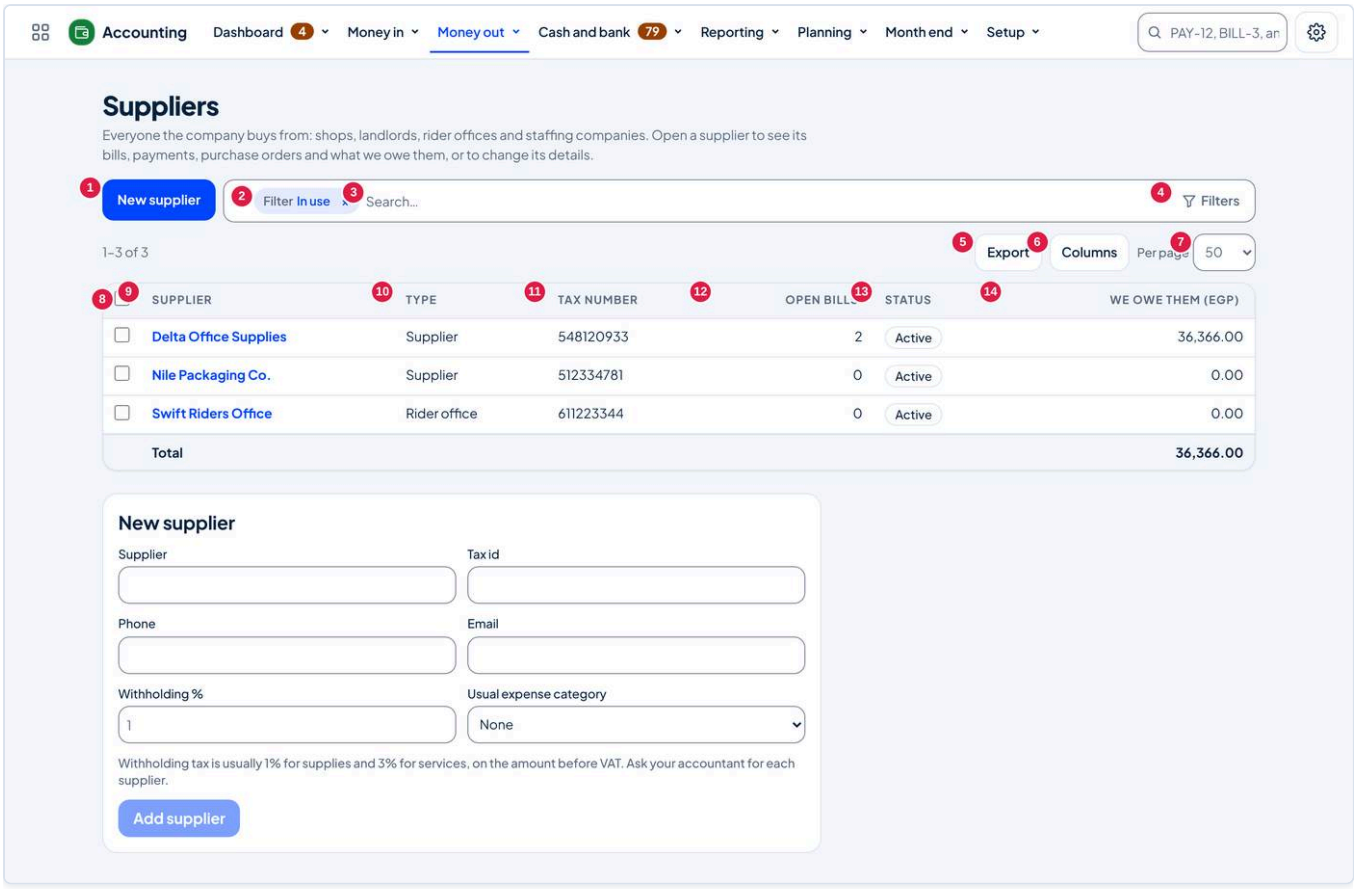
The request REQ-52VRJG as the asker sees it: status bar Waiting, "You asked for this, so another person must approve it." and a Withdraw button; the PDF is already attached.



After the second person approved: the status bar reads Approved and a smart button opens the journal entry it posted.

14.2 Suppliers

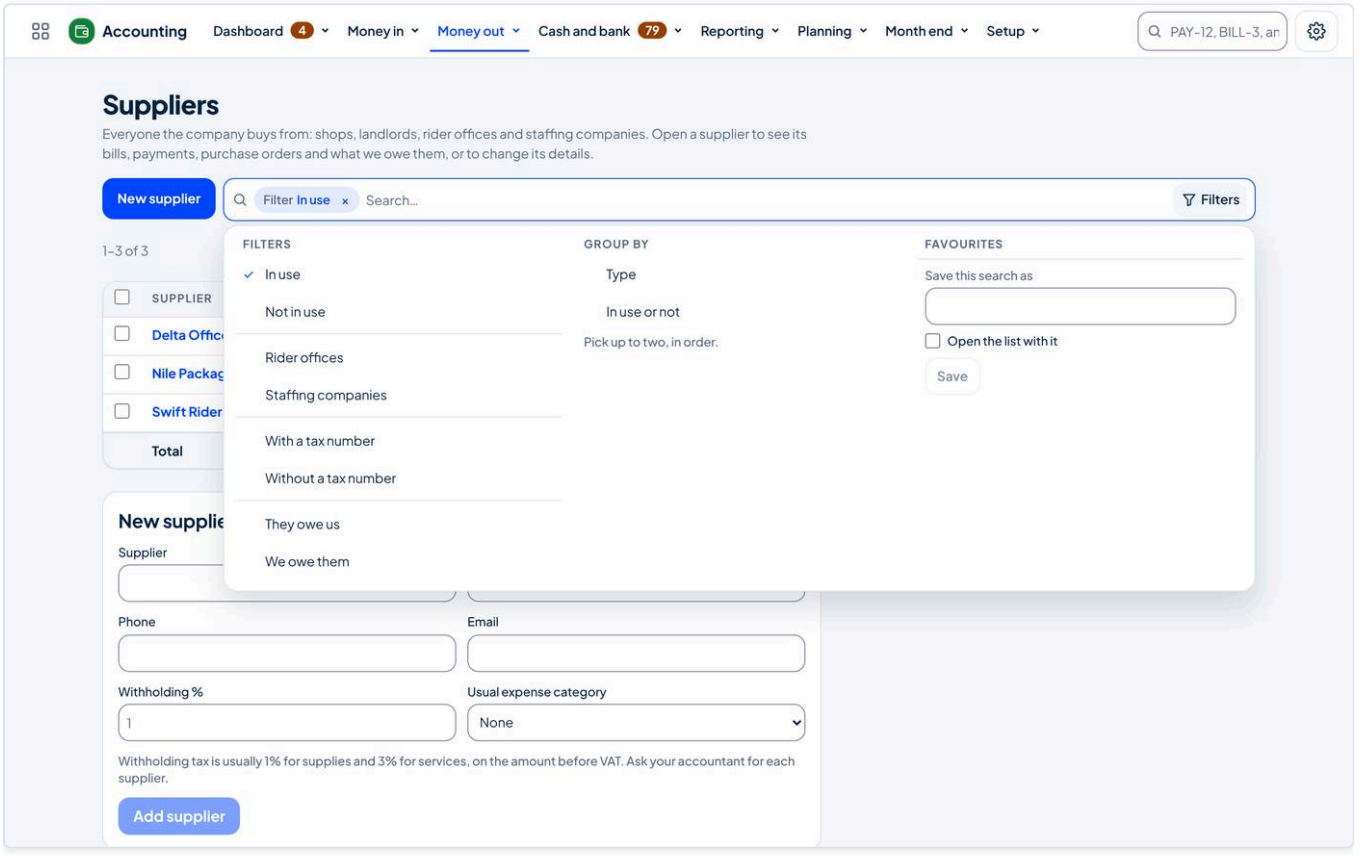
Address: `/finance/suppliers` ; one supplier at `/finance/suppliers/<id>` . **Menu:** Accounting > Money out > Suppliers. **Who:** Accounting `view` to read; `use` to add or edit (the New supplier button and form are hidden without it). Rider offices and staffing companies are suppliers too.



The suppliers list, opening on the default filter "In use" (/finance/suppliers).

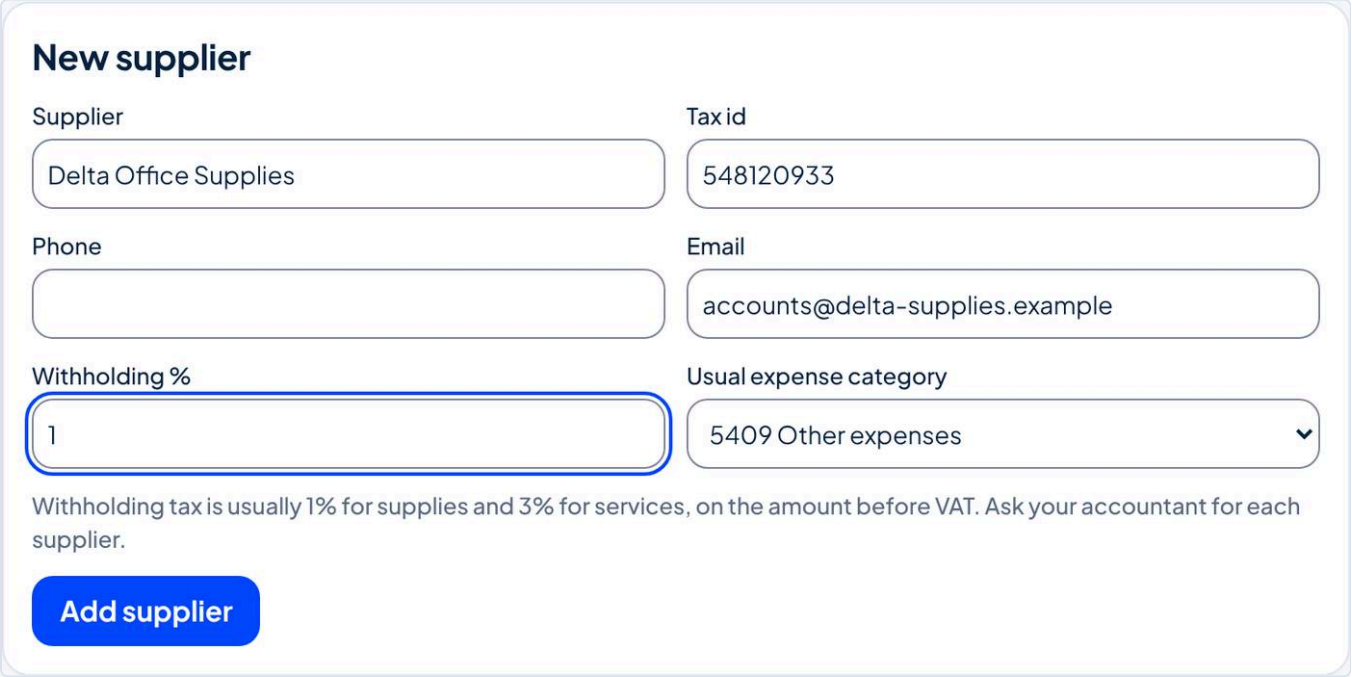
#	Control	What it does	Rule / limits	Result
1	New supplier	Blue button; jumps to the New supplier form under the list	Accounting `use`	The form scrolls into view.
2	Chip "Filter In use"	The default filter	Removing it shows suppliers not in use too	The list reloads.
3	Search	Searches name, tax number, phone and email	Loose match, case ignored	A "Search" chip.
4	Filters	Opens the panel	Filters: In use, Not in use; Rider offices, Staffing companies; With a tax number, Without a tax number; They owe us, We owe them. Group by: Type, In use or not	See the panel below.
5, 6, 7	Export, Columns, Per page	As in every list view	Optional columns: Phone, Email, Withholding %, Account, People (riders or staff linked)	File `suppliers.csv` or `.xlsx`.
8	Select all	Ticks every row on the page	No bulk action on suppliers; ticking limits the export	Only ticked rows are exported.
9	Supplier	Name, blue link; sortable		Opens the supplier page.
10	Type	Supplier, Rider office or Staffing company		
11	Tax number	As entered		

#	Control	What it does	Rule / limits	Result
12	Open bills	Count of open bills; sortable		
13	Status	Grey pill "Active" or "Inactive"		
14	We owe them (EGP)	Balance of the supplier's 2500 wallet, credit shown positive; sortable; summed in Total	Below zero the supplier owes us (a rider office's shortages)	



The Filters panel of Suppliers: Filters (ticked ones carry a check mark), Group by ("Pick up to two, in order") and Favourites with "Save this search as", "Open the list with it" and Save.

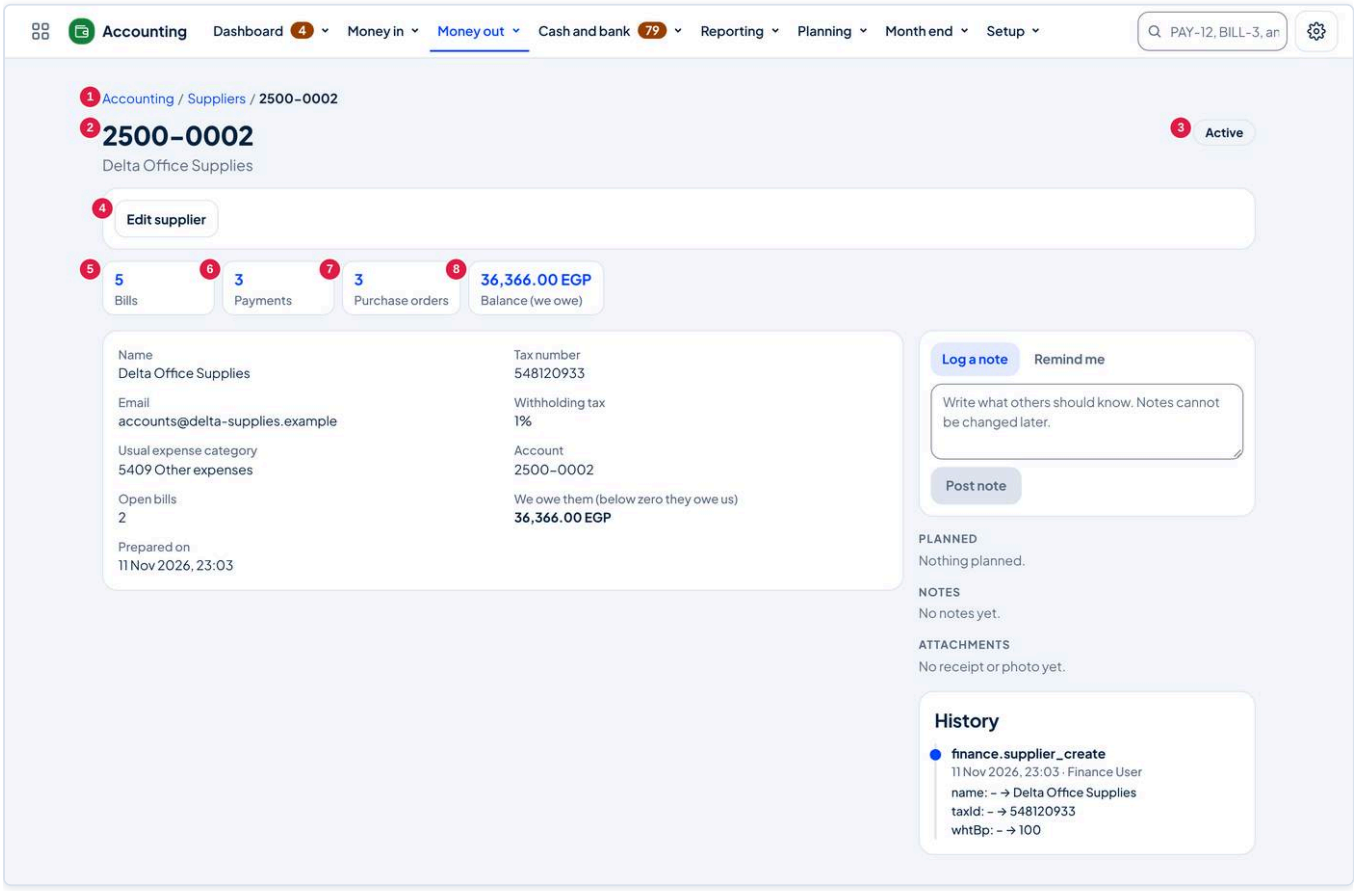
New supplier



The New supplier form filled in.

Control	What it does	Rule / limits	Result
Supplier	The name	Required, 2 to 80 characters	
Tax id	The supplier's tax registration number	Optional, up to 30 characters. A rider office with a tax number gets VAT on its monthly bill	
Phone, Email	Contact details	Optional; an email must look like one (`invalid_email`)	
Withholding %	Withholding tax kept back from each payment, on the amount before VAT	0 to 100, stored as basis points (1% = 100); default 1. Hint: "usually 1% for supplies and 3% for services"	Used by every bill payment.
Usual expense category	Default category proposed on a new bill	Open 5400 or 5500 accounts; None allowed	
Add supplier	Saves (`POST /finance/suppliers`)	Disabled (light blue) until a name is typed; Accounting `use` ; audited `finance.supplier_create`	Opens the new supplier's page. Its wallet account (`2500-000n`) is opened when it is first needed.

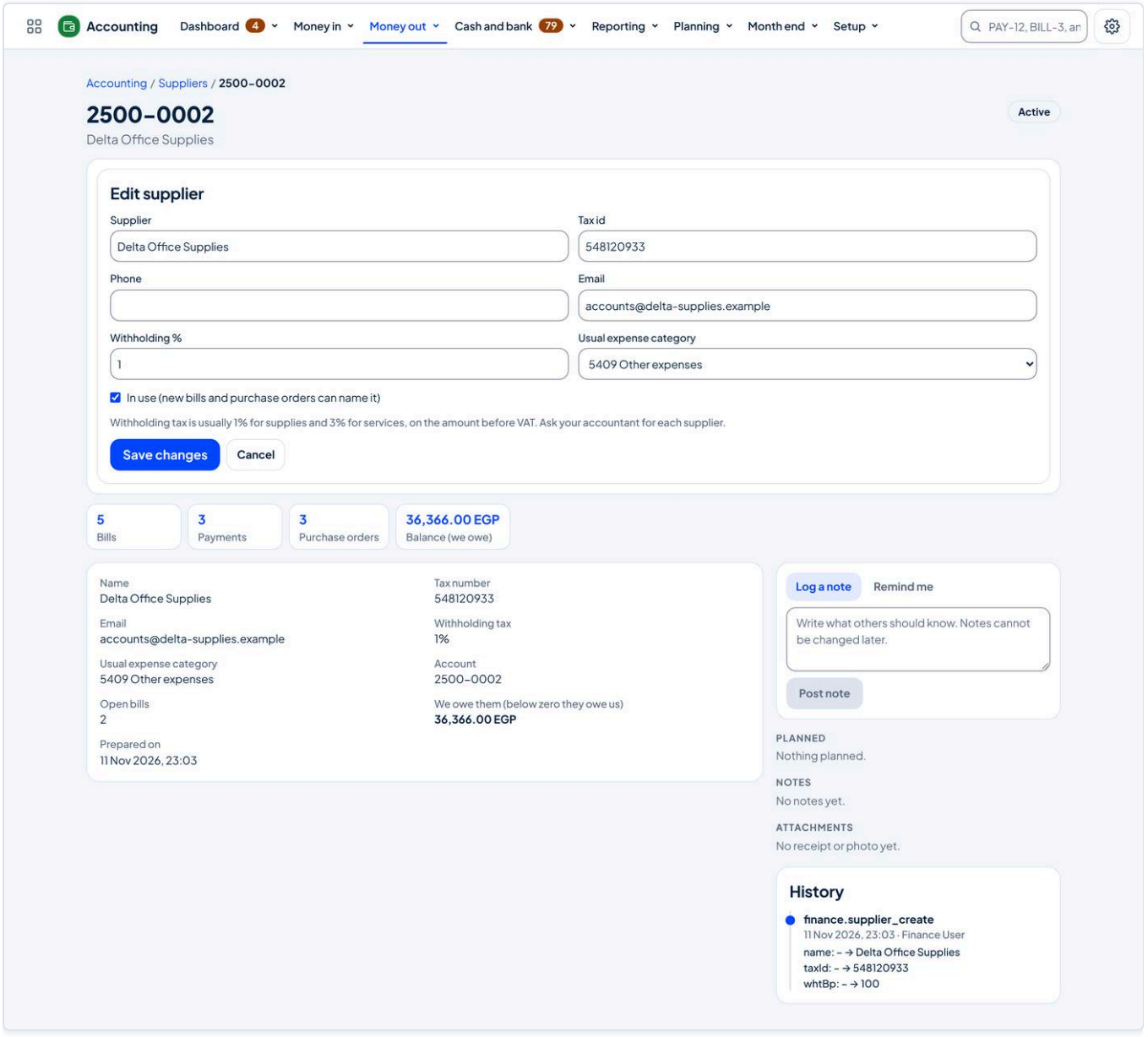
The supplier page



A supplier page with two open bills (/finance/suppliers/<id>).

#	Part	What it shows	What clicking it does
1	Breadcrumbs	Accounting / Suppliers / title	Back to the list with its filters.
2	Title	The supplier's wallet code (2500-0002) once it has one, else its name; the name in grey under it	Nothing.
3	Status pill	"Active" or "Inactive"	Nothing.
4	Edit supplier	White button in the top bar (Accounting `use`)	Opens the edit form in place (below).
5	Bills smart button	Count of all its bills, cancelled included	Opens Supplier bills searched on the supplier's name, every status (`/finance/bills?q=<name>&f=`).
6	Payments smart button	Count of bill payments	Opens the journal searched on the supplier's name.
7	Purchase orders smart button	Count	Opens Purchase orders for this supplier ("Orders of *name*." with "Show every supplier").
8	Balance (we owe)	Blue amount	Opens the ledger lines report of the supplier's wallet account (`/finance/reports/lines?account=<id>`).

#	Part	What it shows	What clicking it does
	Details	Name, tax number, email, withholding tax, usual category, account, open bills, "We owe them (below zero they owe us)", prepared on	Nothing.
	Chatter	Log a note, Remind me, Attachments, History	As on every document page.

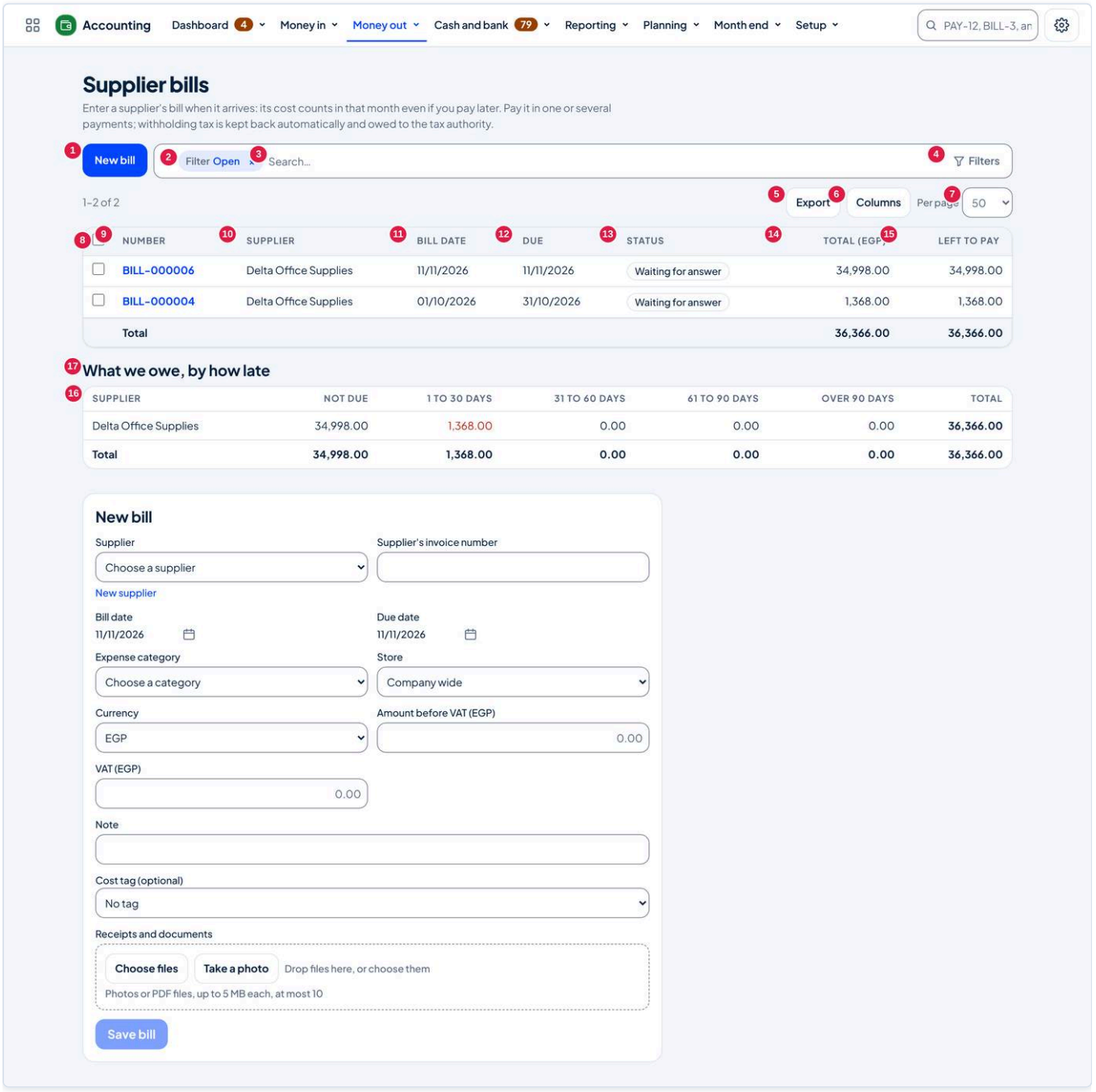


Edit supplier opened: the same fields as the new form plus "In use (new bills and purchase orders can name it)", Save changes and Cancel.

Untick *In use* to retire a supplier: it disappears from the New bill and New purchase order choices; its bills and history stay. Saving (`PATCH /finance/suppliers/<id>`) shows "Saved." and is audited.

14.3 Supplier bills

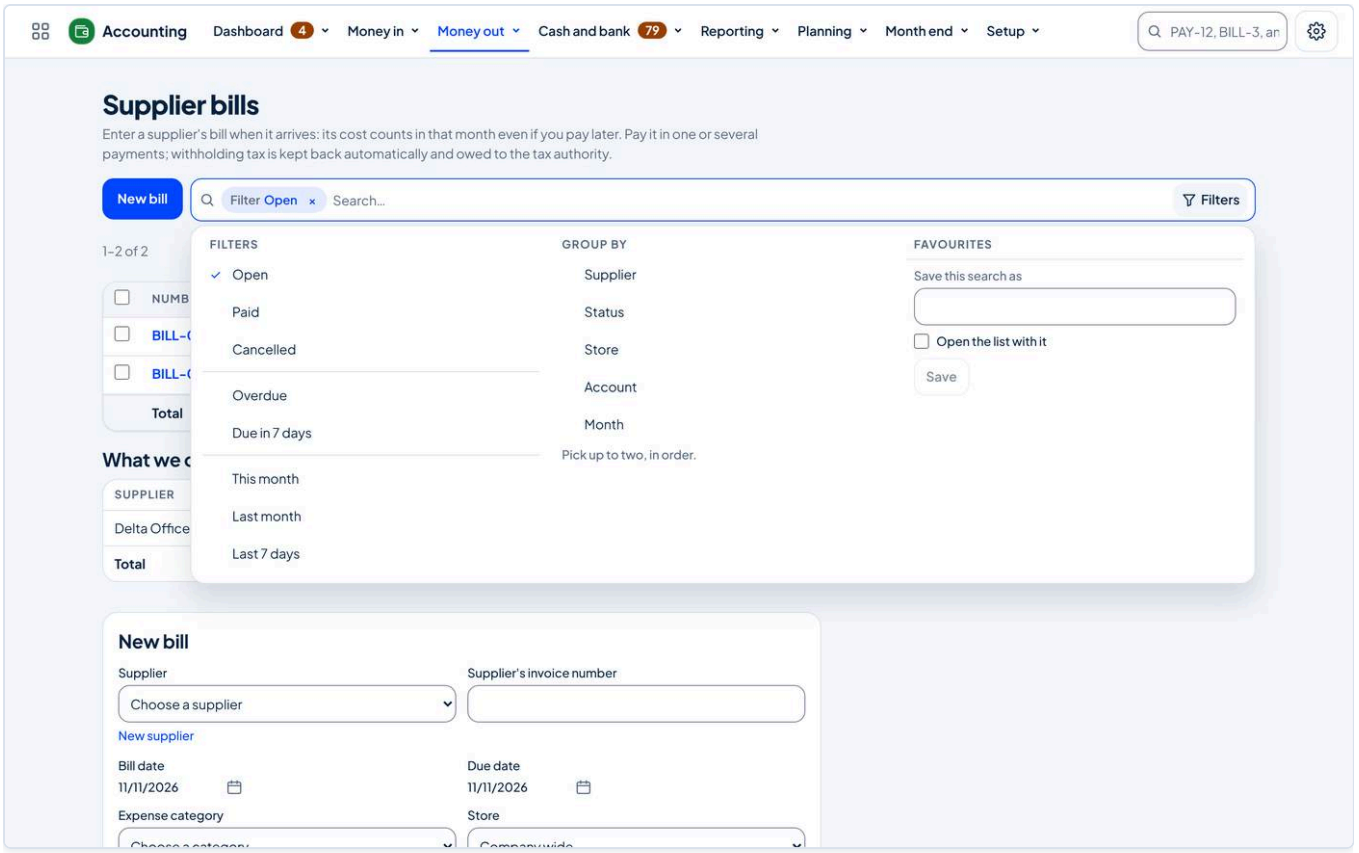
Address: `/finance/bills` ; one bill at `/finance/bills/<id>` . **Menu:** Accounting > Money out > Bills. **Who:** Accounting `view` to read; `use` to enter, pay or cancel a bill. A bill books its cost when it arrives (Dr expense and VAT, Cr the supplier's 2500 wallet) and is paid later in one or more payments that keep back withholding tax. See the chapter Expenses, suppliers, bills and the assets we pay for, supplier bills and bill payments.



Supplier bills opening on the default filter "Open", with the ageing table under the list (/finance/bills).

#	Control	What it does	Rule / limits	Result
1	New bill	Blue button; jumps to the New bill form	Accounting `use` to save	

#	Control	What it does	Rule / limits	Result
2	Chip "Filter Open"	Default filter	Remove it to see paid and cancelled bills	
3	Search	Bill number, supplier, supplier's invoice number, note		
4	Filters	Filters: Open, Paid, Cancelled; Overdue, Due in 7 days; This month, Last month, Last 7 days. Group by: Supplier, Status, Store, Account, Month	OR inside a group, AND across	Grouped rows show counts and subtotals.
5, 6, 7	Export, Columns, Per page	Optional columns: Reference, Store, Account, VAT, Paid		`bills.csv` / `.xlsx`
8	Select all	Ticks rows	No bulk action; limits the export	
9	Number	`BILL-000001` onward, gapless; blue link; sortable		Opens the bill page.
10	Supplier	Sortable		
11, 12	Bill date, Due	Day first; sortable		
13	Status	Grey outlined pill. An open bill shows the state label "Waiting for answer", then Paid, Cancelled		
14, 15	Total (EGP), Left to pay	Bill total with VAT, and what is still unpaid; both summed in Total; sortable	Withholding kept back counts as paid	
16, 17	What we owe, by how late	Ageing per supplier: Not due, 1 to 30 days, 31 to 60, 61 to 90, Over 90, Total. Late amounts are red. With nothing owed: "We owe no supplier anything."	By due day, Cairo time	The same figures as Who we owe, by age.

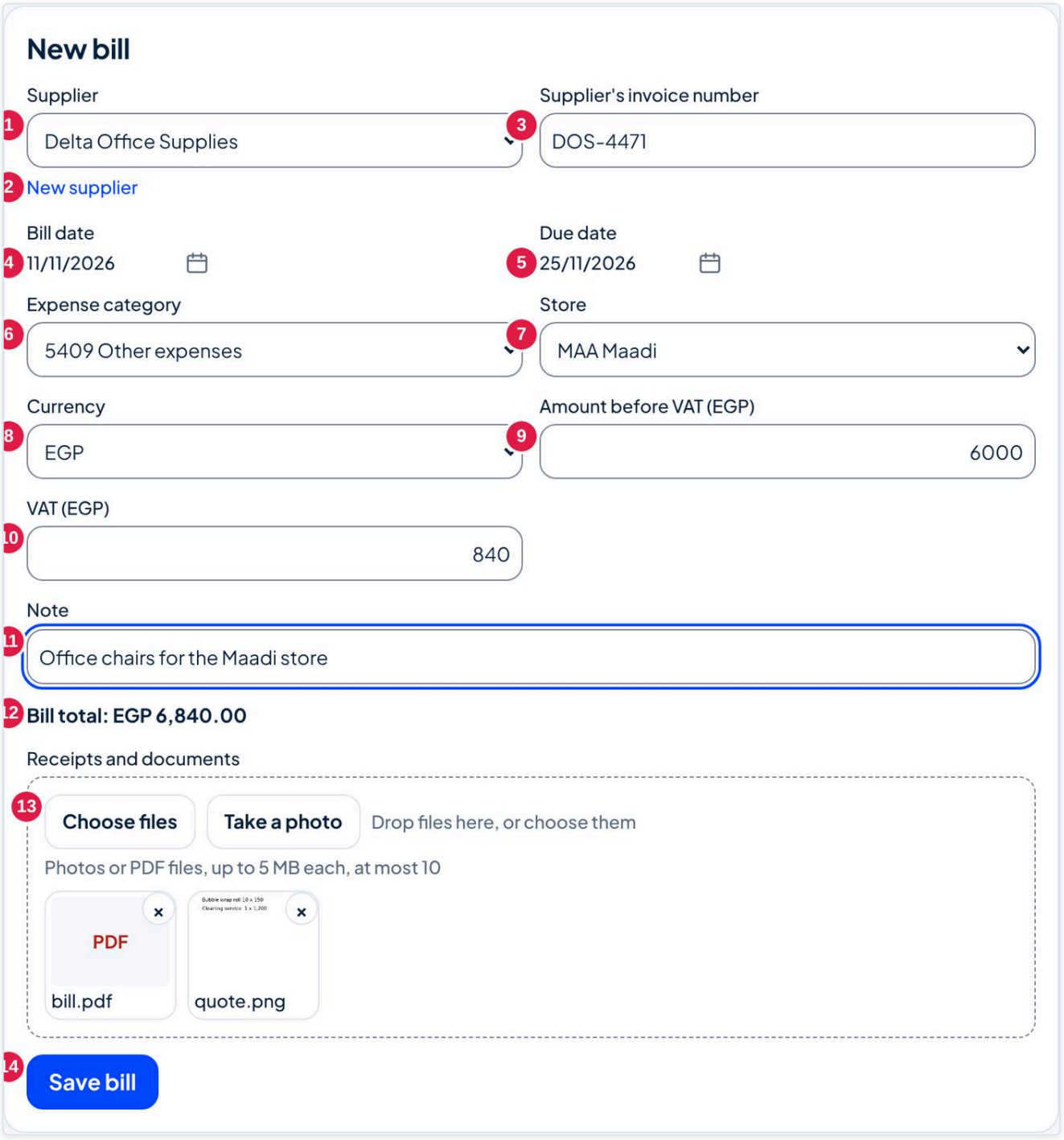


The Filters panel of Supplier bills.

NUMBER	SUPPLIER	BILL DATE	DUE	STATUS	TOTAL (EGP)	LEFT TO PAY
BILL-000006	Delta Office Supplies	11/11/2026	11/11/2026	Waiting for answer	34,998.00	34,998.00
BILL-000005	Delta Office Supplies	11/11/2026	25/11/2026	Paid	6,840.00	0.00
BILL-000004	Delta Office Supplies	01/10/2026	31/10/2026	Waiting for answer	1,368.00	1,368.00
BILL-000003	Delta Office Supplies	11/11/2026	11/11/2026	Paid	70.00	0.00
BILL-000002	Delta Office Supplies	11/11/2026	11/11/2026	Cancelled	840.00	0.00
BILL-000001	Nile Packaging Co.	17/10/2026	16/11/2026	Paid	13,680.00	0.00
Total					**57,796.00**	**36,366.00**

Every status at once (filter chip removed): two open bills, one paid, one paid in two parts and one cancelled.

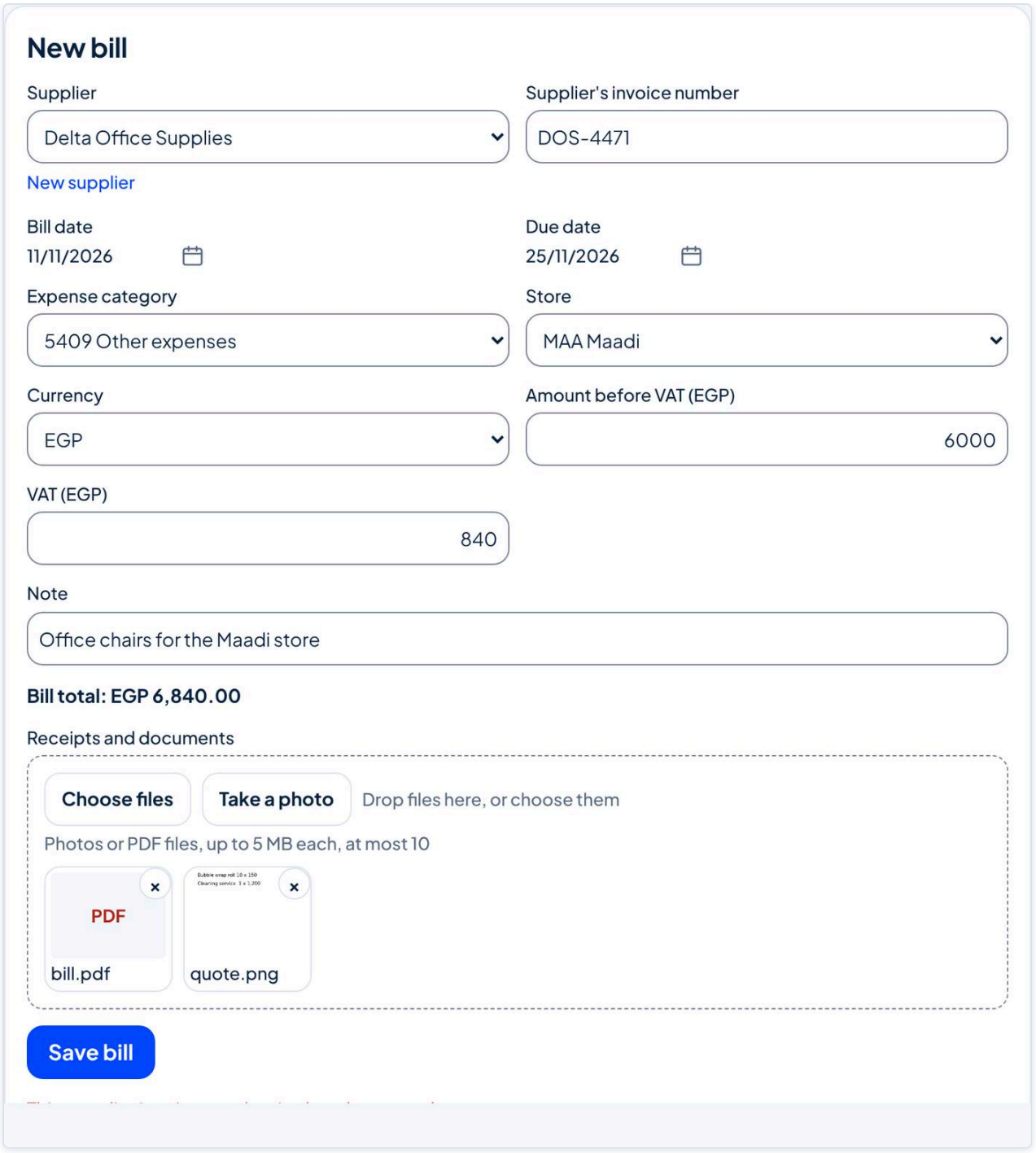
New bill



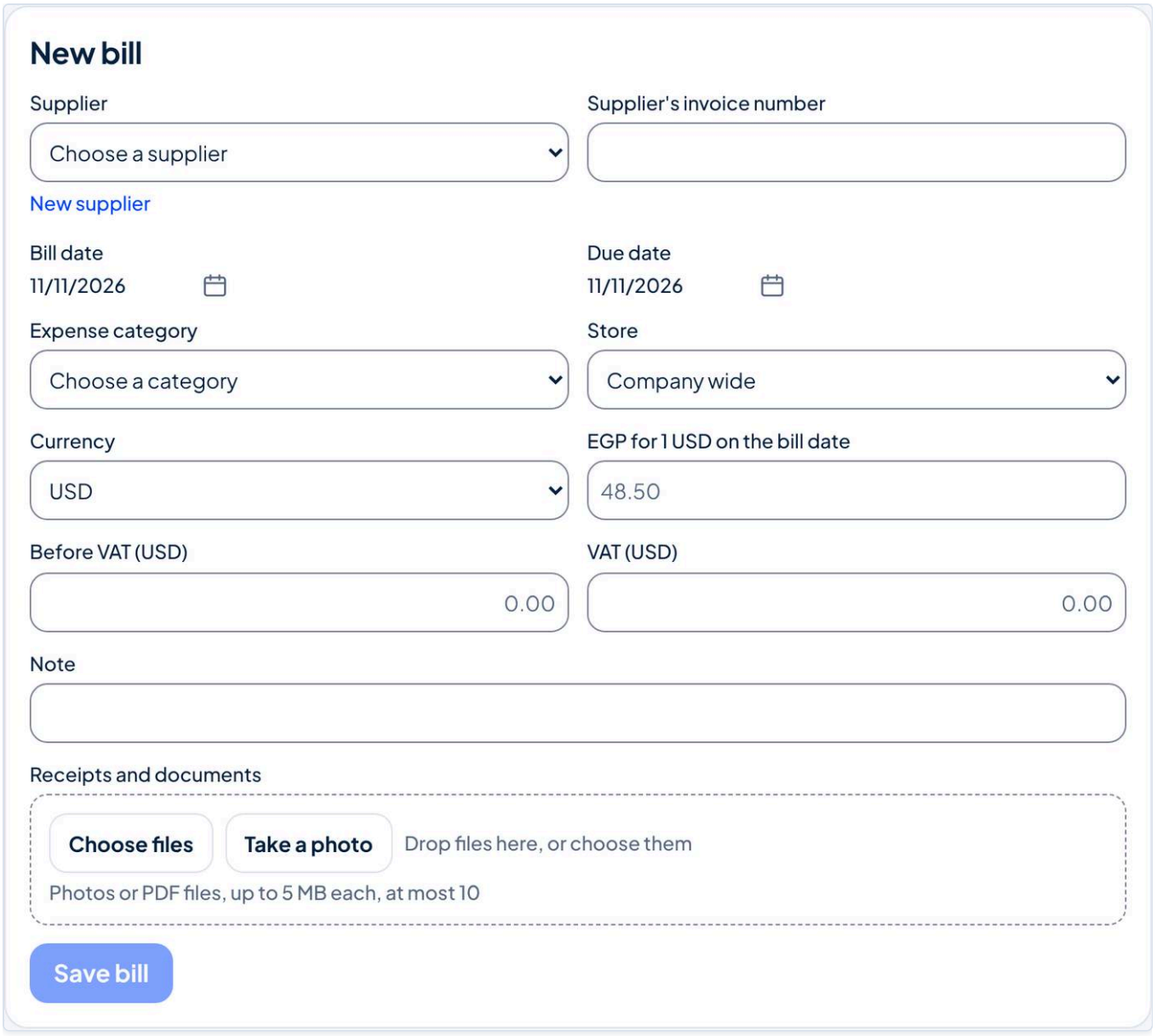
The New bill form filled in, with a PDF invoice and a PNG quote attached.

#	Control	What it does	Rule / limits	Result
1	Supplier	"Choose a supplier", then suppliers in use	Required	Its usual category is proposed.
2	New supplier	Blue link		Opens Suppliers at its New supplier form.
3	Supplier's invoice number	The supplier's own number	Required, 2 to 100 characters; once per supplier: a second bill with the same number answers "This supplier invoice number is already entered."	

#	Control	What it does	Rule / limits	Result
4, 5	Bill date, Due date	Day first date boxes, today by default	The bill date may not be in the future or in closed books; the due date may not be before the bill date (`due_before_bill`)	The cost counts in the bill date's month; ageing uses the due date.
6	Expense category	Open 5400 and 5500 accounts	Required	The account debited.
7	Store	"Company wide" or a store		Tags the cost line.
8	Currency	EGP, USD, EUR, GBP, SAR, AED	Another currency adds "EGP for 1 *USD* on the bill date" and the amounts are typed in that currency	Booked in EGP at the bill's rate; see bills in another currency.
9	Amount before VAT (EGP)	Net amount, right aligned	Above 0	
10	VAT (EGP)	VAT on the invoice	0 or more	Booked to 2301 as input VAT.
11	Note	Free text up to 300 characters		
	Cost tag (optional)	Shown when active cost tags exist		Tags the cost line.
12	Bill total	"Bill total: EGP 6,840.00", bold, updated as you type		
13	Attachment field	The invoice and any documents	See above	Linked to the bill.
14	Save bill	Saves (`POST /finance/bills`)	Disabled until the required fields are filled	"Saved as BILL-000005." under the button; the form empties.

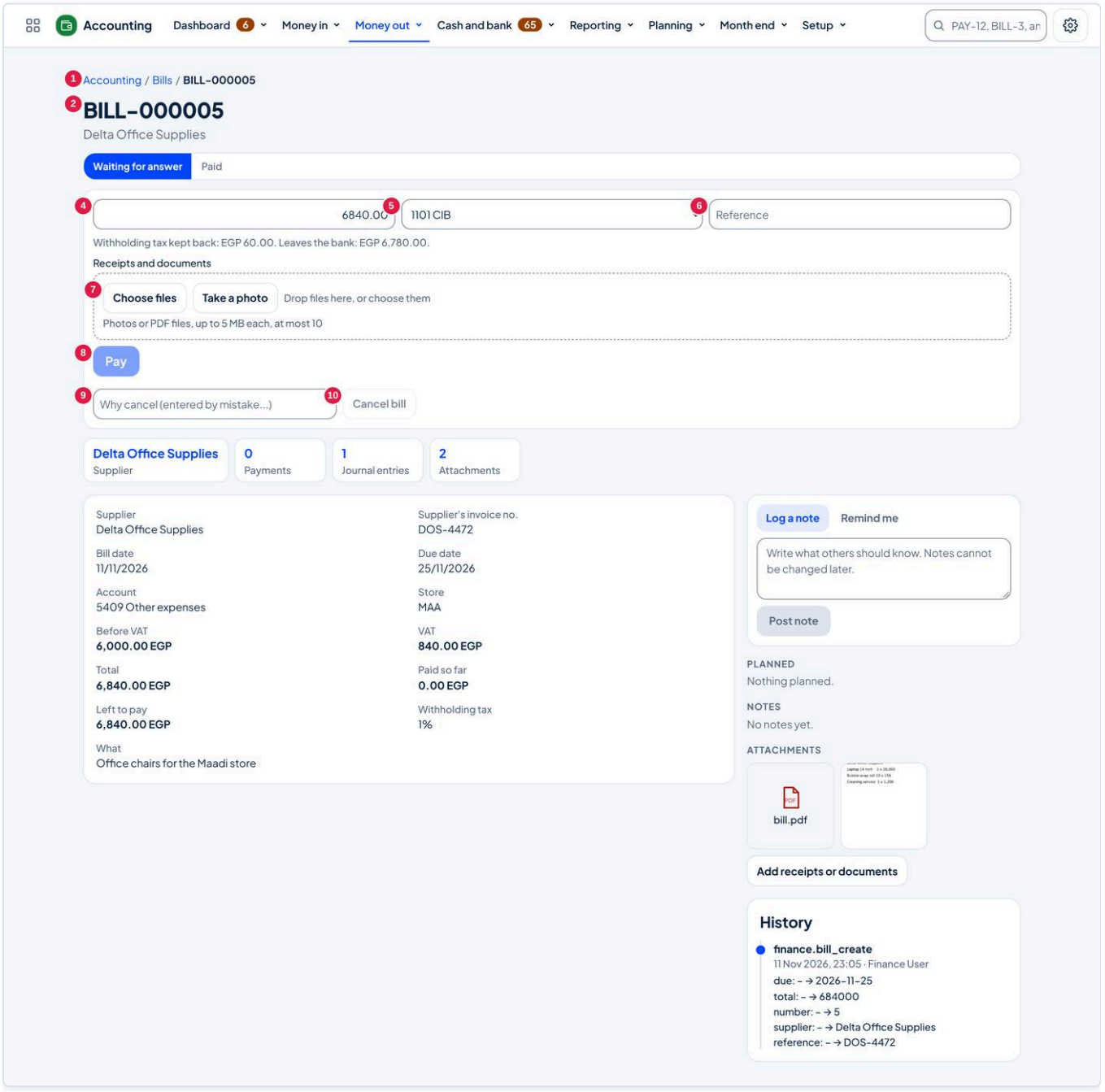


The error shown when the supplier's invoice number was already entered for that supplier.



Currency set to USD: the rate on the bill date is asked and the amounts are typed in dollars.

The bill page



An open bill of 6,840.00 EGP before any payment (/finance/bills/<id>).

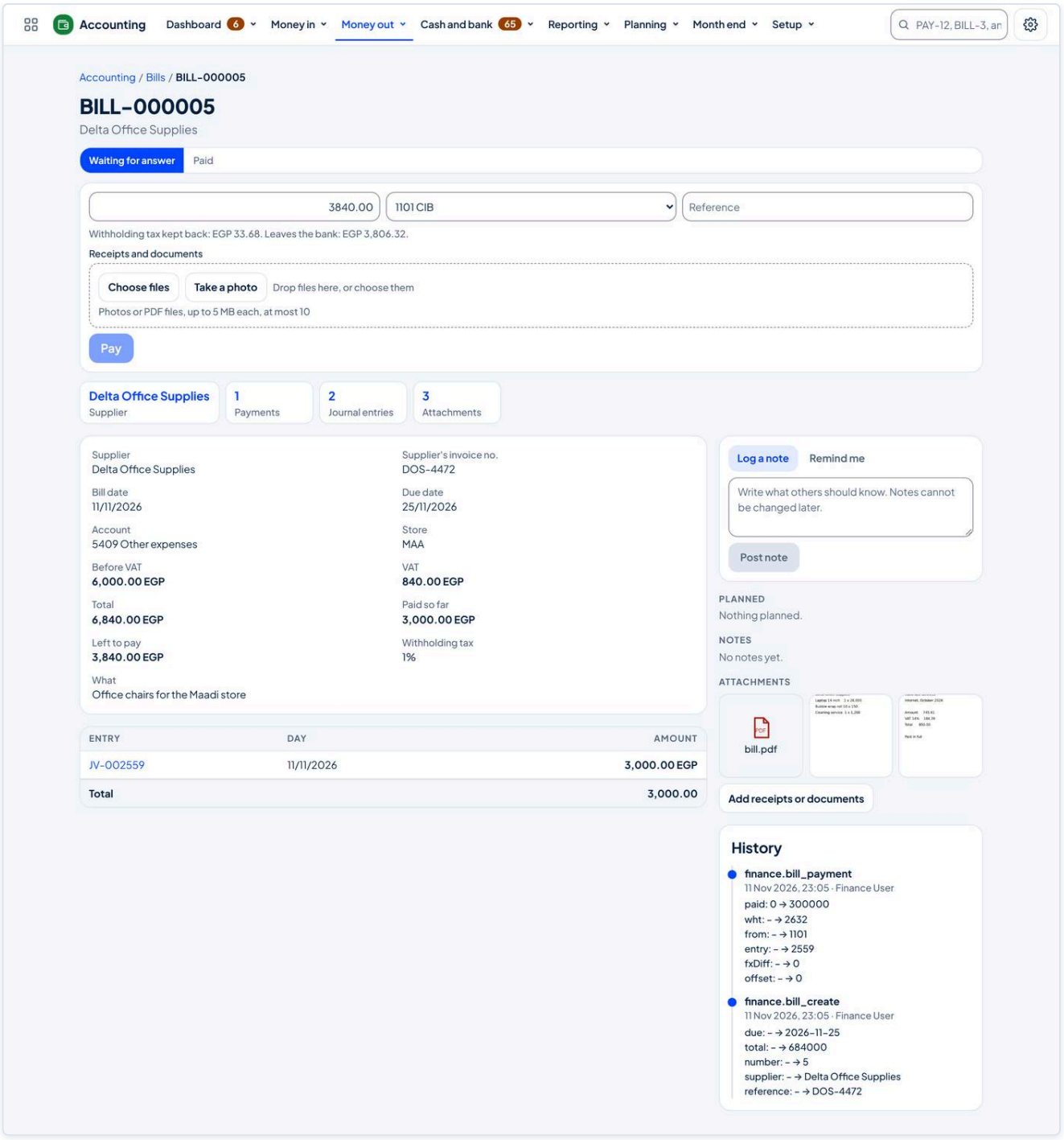
#	Part	What it shows	What clicking it does
1	Breadcrumbs	Accounting / Bills / BILL-000005	Back to the list with its filters (previous and next records of that list are kept).
2	Number and supplier	Bill number, supplier name in grey	
	Status bar	Open step in blue (reads "Waiting for answer"), then Paid; Cancelled appears in red only when cancelled	
4	Amount	Prefilled with what is left to pay	Change it to pay in parts.

#	Part	What it shows	What clicking it does
5	Paid from	Open banks and cash boxes, CIB first	
6	Reference	Bank reference, placeholder only	Required, at least 2 characters; the Pay button stays light blue until then.
	Withholding line	Grey: "Withholding tax kept back: EGP 60.00. Leaves the bank: EGP 6,780.00."	Shown when the supplier has a withholding rate; recalculated as you type.
7	Attachment field	The transfer slip	Linked to the payment entry.
8	Pay	Blue button	Posts the payment (`POST /finance/bills/<id>/payments`), or sends it to Approvals from the approval limit.
9	Why cancel	Placeholder "Why cancel (entered by mistake...)"	At least 5 characters.
10	Cancel bill	White button with grey text	Only for a bill with nothing paid and not a rider office bill; posts a reversing entry.
	Smart buttons	Supplier; Payments (count); Journal entries (count); Attachments (count)	Supplier opens the bills searched by the supplier; Payments opens the journal searched by the supplier's invoice number; Journal entries opens the journal searched by the bill number.
	Details	Supplier, supplier's invoice no., bill date, due date, account, store, before VAT, VAT, total, paid so far, left to pay, withholding tax %, what	
	Payments table	Entry (link), day, amount, Total; shown once paid	Opens the payment's journal entry.

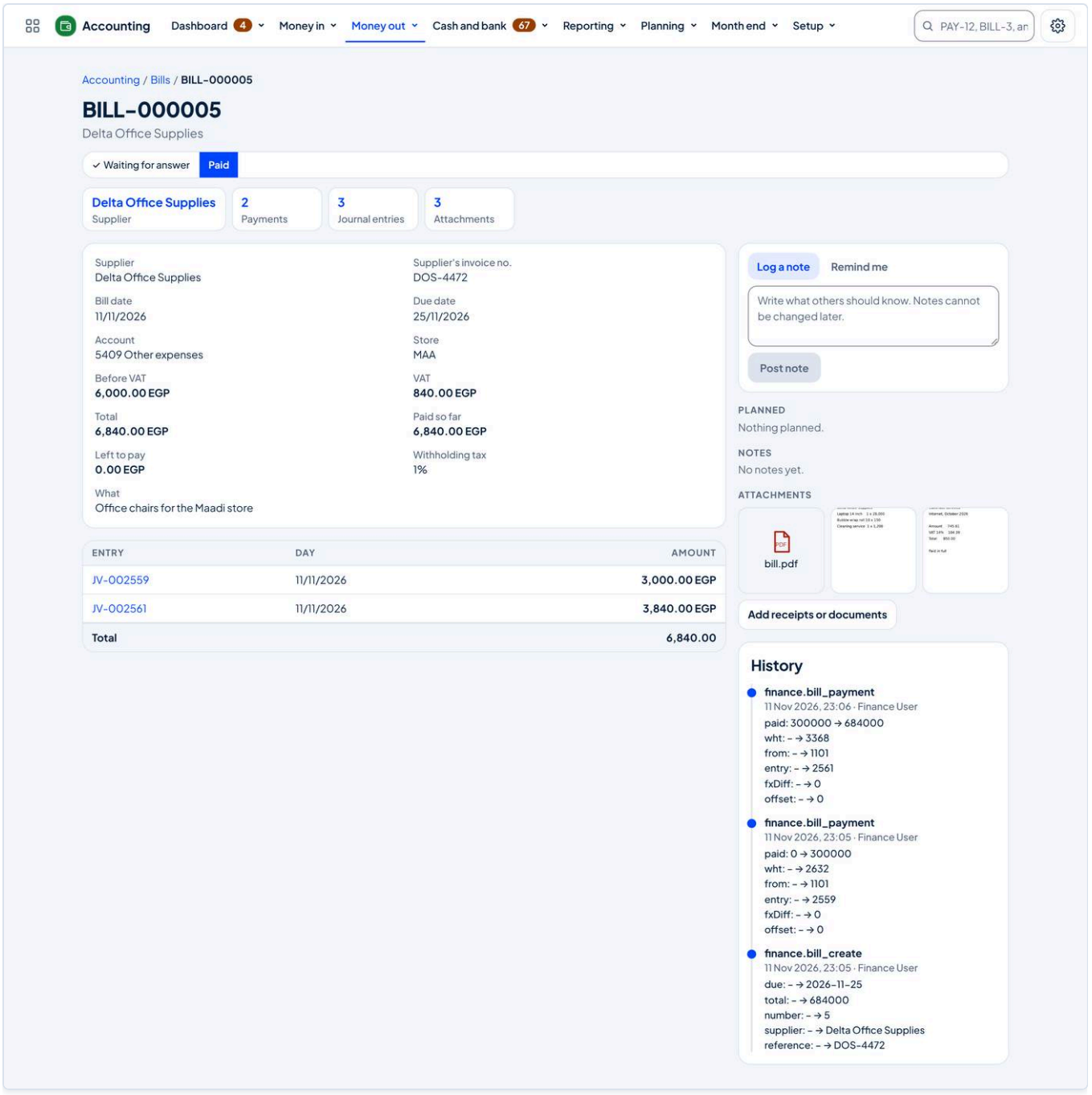
Paying with withholding tax

The supplier keeps 1% withholding on the amount before VAT. Each payment keeps back its share: `wht = net × rate × paid ÷ total`, rounded to the piastre, and the last payment makes the total exact. Paying 3,000.00 of 6,840.00 keeps back 26.32 (6,000 × 1% × 3,000 ÷ 6,840) and 2,973.68 leaves CIB; paying the remaining 3,840.00 keeps back 33.68 and 3,806.32 leaves CIB; 60.00 in all, exactly 1% of 6,000.00. Each payment entry is Dr supplier wallet, Cr 2302 withholding tax payable, Cr the bank. See the chapter Expenses, suppliers, bills and the assets we pay for, bill payments, and Taxes for the quarterly form 41.

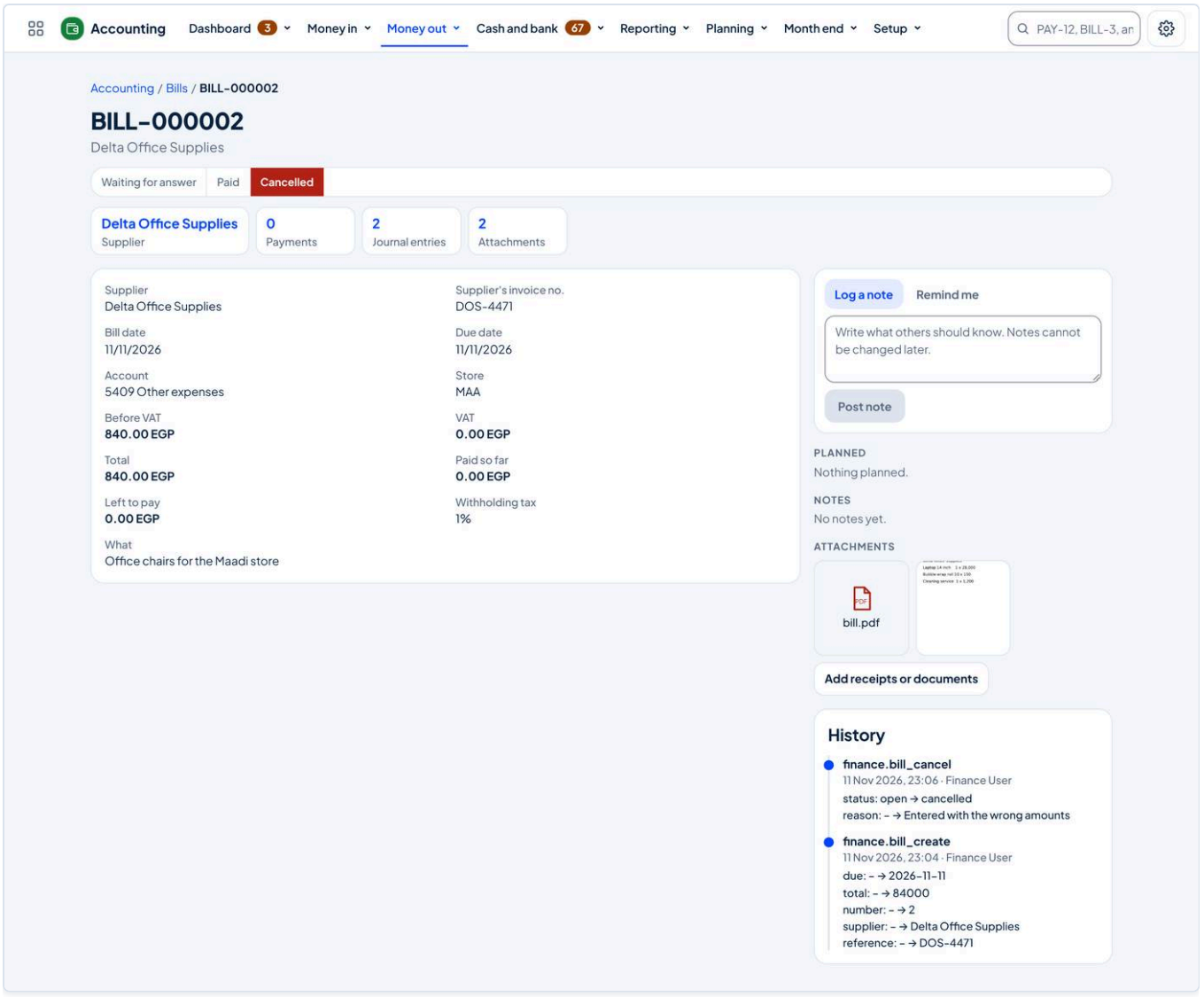
The payment form for the rest of the bill: 3,840.00, CIB, a reference, and the withholding line "EGP 33.68. Leaves the bank: EGP 3,806.32."



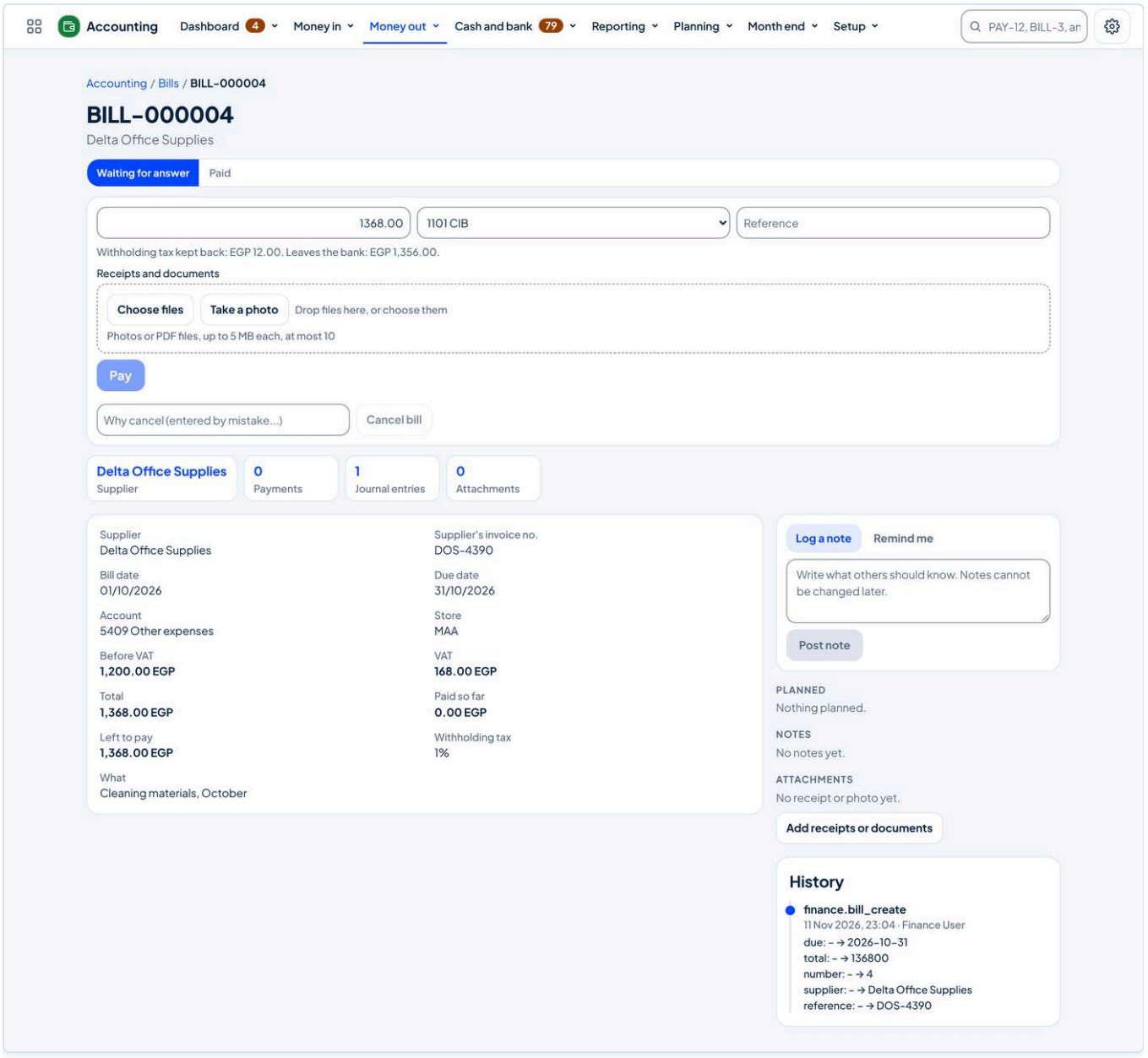
After the first payment: paid so far 3,000.00, left to pay 3,840.00, one payment entry, the slip under Attachments. The bill stays open.



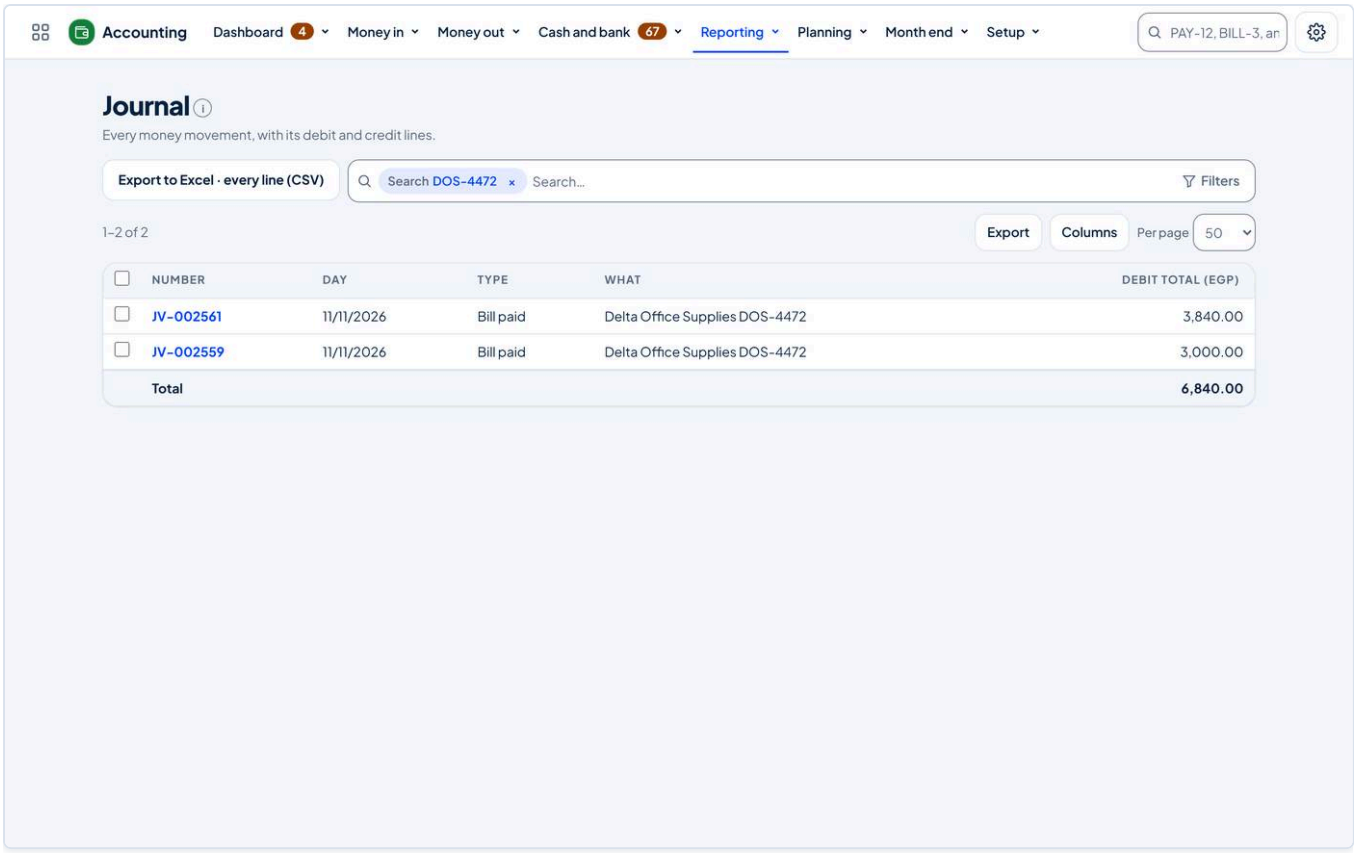
Fully paid: the status bar moves to Paid (blue), the pay and cancel forms disappear, two payment entries add up to 6,840.00.



A cancelled bill: Cancelled in red at the end of the status bar, left to pay 0.00, and the reason in the history (finance.bill_cancel).



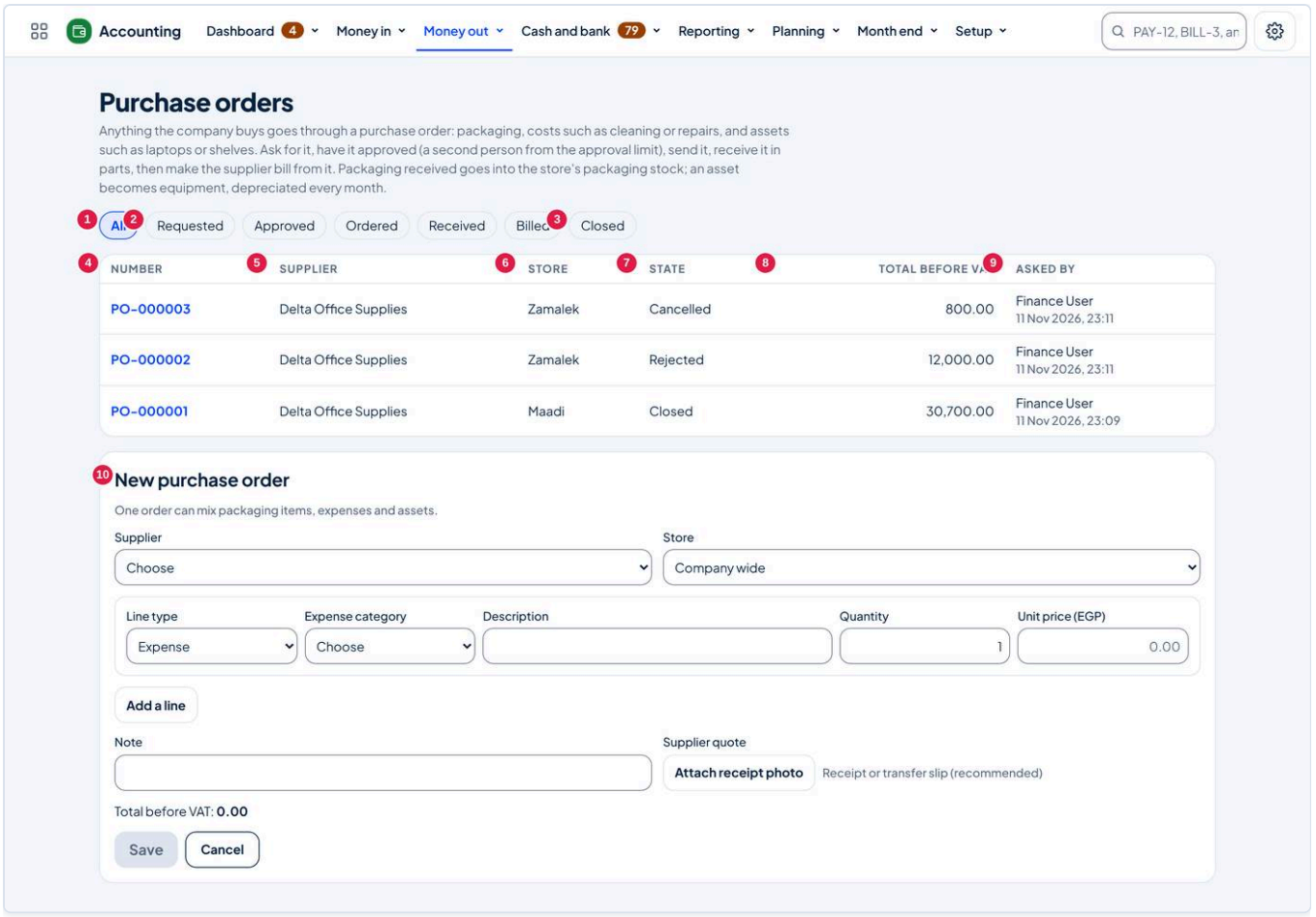
An overdue open bill (due 31/10/2026): it still shows the pay form; the ageing table lists it in red under 1 to 30 days.



The Payments smart button opens the journal searched on the supplier's invoice number, listing the two payment entries.

14.4 Purchase orders

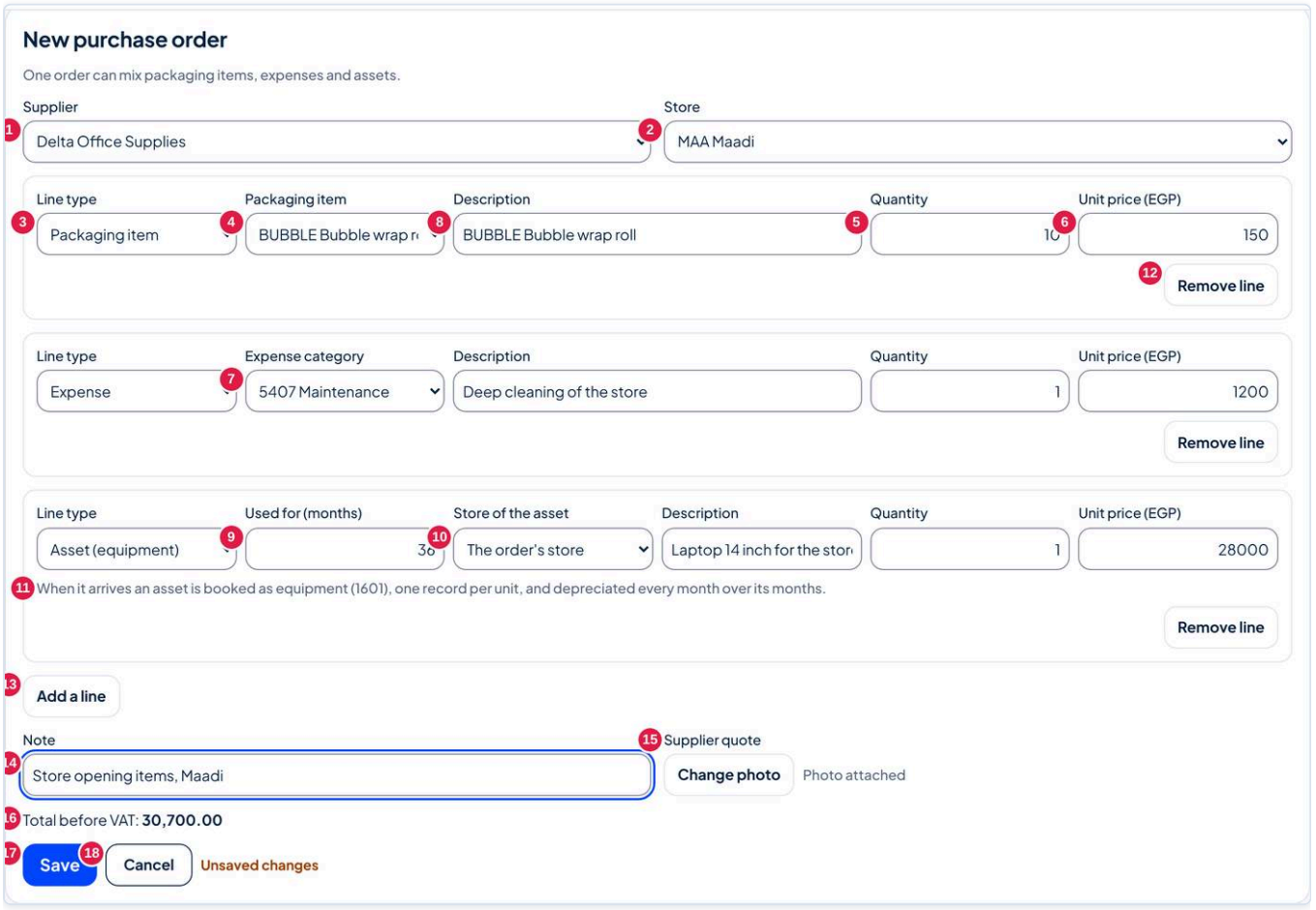
Address: `/finance/purchase-orders` (filter `?status=`, supplier `?supplierId=`); one order at `/finance/purchase-orders/<id>`. **Menu:** Accounting > Money out > Purchase orders. **Who:** Accounting `use` asks for, sends, receives (Inventory `use` may receive too), bills and closes an order; approving or rejecting needs Accounting `manage` and another person than the asker. An order below the approval limit is approved at once ("Below the approval limit"). See the chapter Expenses, suppliers, bills and the assets we pay for, purchase orders.



Purchase orders with one order closed, one rejected and one cancelled (/finance/purchase-orders).

#	Control	What it does	Rule / limits	Result
1 to 3	Show: All, Requested, Approved, Ordered, Received, Billed, Closed	Rounded filter buttons; the chosen one is blue	Each button shows one state; rejected and cancelled orders appear under All only	Reloads with `?status=`.
4	Number	`PO-000001` onward, gapless, blue link		Opens the order.
5, 6	Supplier, Store	Store or "Company wide"		
7	State	Plain text: Requested, Approved, Ordered (with a grey share such as "60% received" while partly received), Received, Billed, Closed, Rejected, Cancelled		
8	Total before VAT	Sum of the lines		
9	Asked by	Person and time		
10	New purchase order	The form below the list	Accounting `use`	See below.

New purchase order: a packaging line, an expense line and an asset



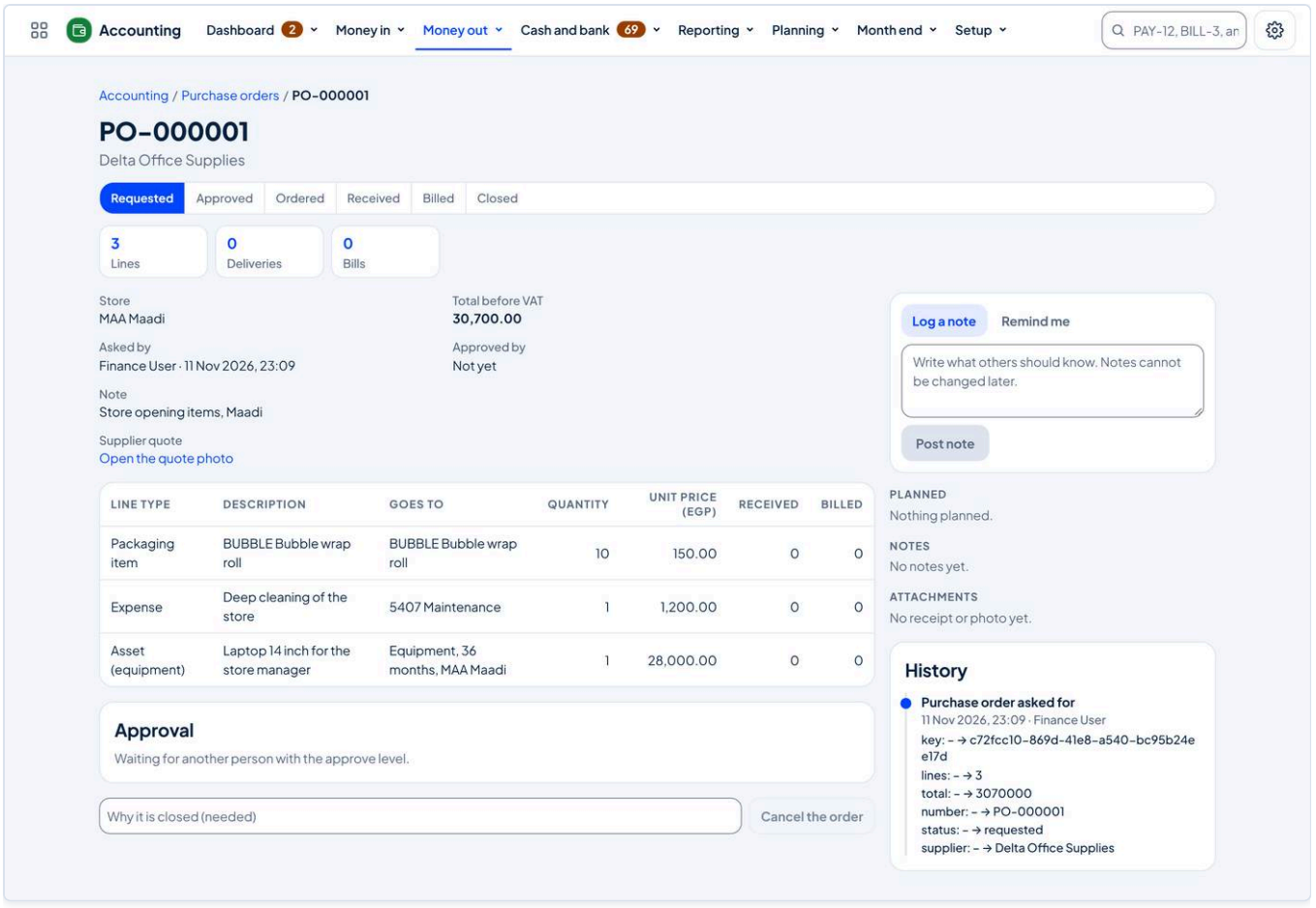
A new order mixing a packaging item (bubble wrap), an expense (deep cleaning) and an asset (a laptop, 36 months), with the supplier's quote photo.

#	Control	What it does	Rule / limits	Result
1	Supplier	Active suppliers	Required (`supplier_not_found` "Choose an active supplier.")	
2	Store	"Company wide" or a dark store	Packaging needs a store ("Packaging goes into a store: choose the store.")	Packaging received goes into that store's packaging stock.
3	Line type	Packaging item, Expense, Asset (equipment)	One type per line	Changes the fields of the line.
4	Packaging item	Active packaging items "BUBBLE Bubble wrap roll"	Items are created in Inventory > Packaging; with none the list is empty	Fills the description and the expected unit cost.
5, 6	Quantity, Unit price (EGP)	Whole quantity, price per unit	An asset needs a price above zero	
7	Expense category	On an expense line: open 5400 and 5500 accounts		Booked when the bill is made.
8	Description	Free text		
9	Used for (months)	On an asset line: its life	1 to 600 ("Enter the months the asset is used, 1 to 600.")	Depreciated straight line over these months.

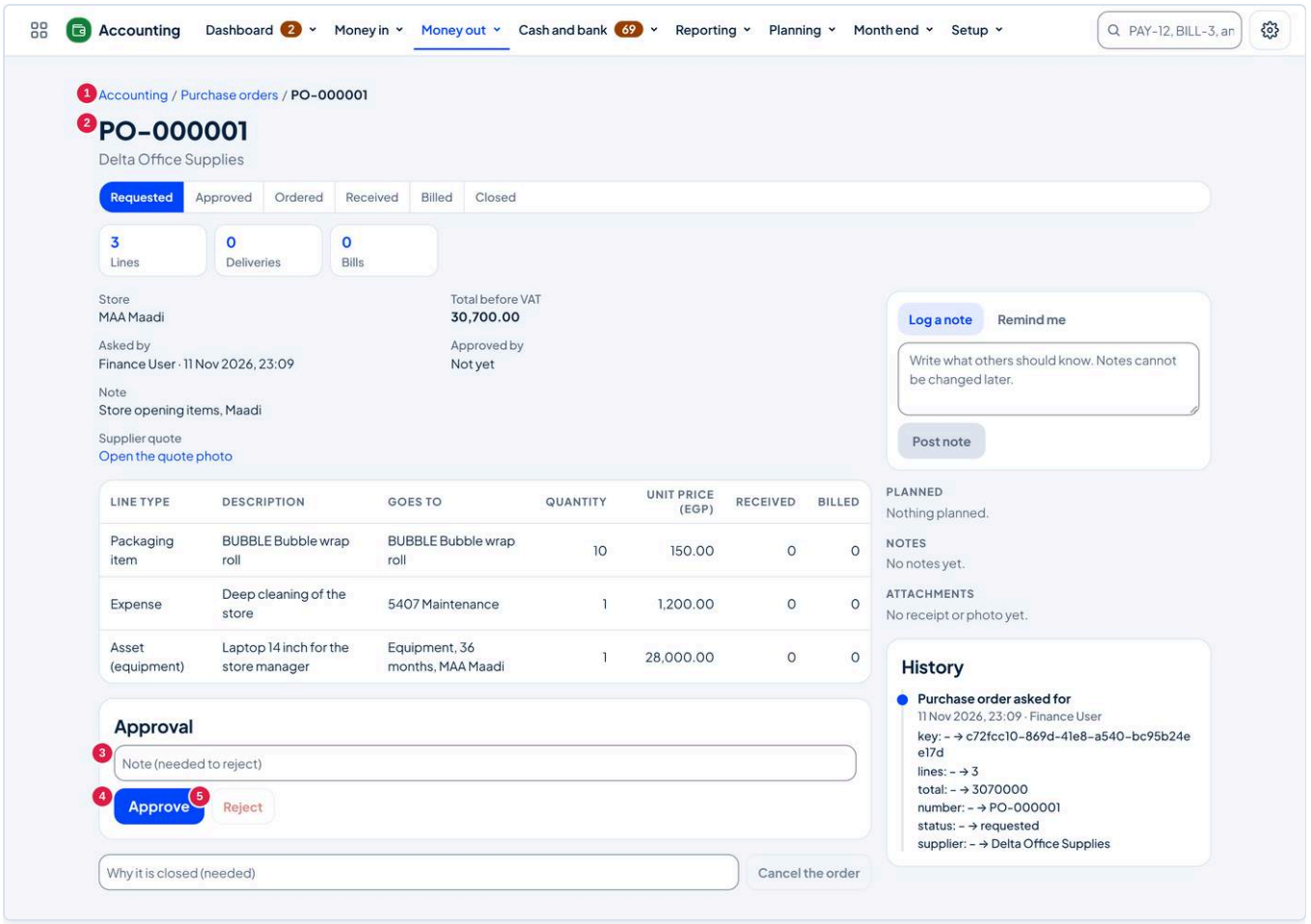
#	Control	What it does	Rule / limits	Result
10	Store of the asset	"The order's store" or another store		Tags the asset and its depreciation.
11	Asset hint	"When it arrives an asset is booked as equipment (1601), one record per unit, and depreciated every month over its months."		
12	Remove line	White button on each line	At least one line stays	
13	Add a line	White button		A new expense line.
14	Note	Free text		
15	Supplier quote	"Attach receipt photo" (one photo; "Change photo" and "Photo attached" once chosen)	A photo only; this field is not the multi file attachment field	Shown as "Open the quote photo" on the order.
16	Total before VAT	Live total (30,700.00)		
17, 18	Save, Cancel	FormFooter with "Unsaved changes"	Save is grey until the order is valid	Opens the new order.

The purchase order page and its states

The status bar runs Requested, Approved, Ordered, Received, Billed, Closed; done steps carry a check mark, the current one is blue, and Rejected or Cancelled appear in red. Smart buttons: Lines (count), Deliveries (receipts), Bills. The details show the store, total before VAT, "Asked by" with the time, "Approved by" (the person, "Below the approval limit" or "Not yet"), the note and "Open the quote photo". The lines table shows Line type, Description, Goes to (the packaging item, the expense account, or "Equipment, 36 months, MAA Maadi" with "1 asset in the register"), Quantity, Unit price, Received and Billed.

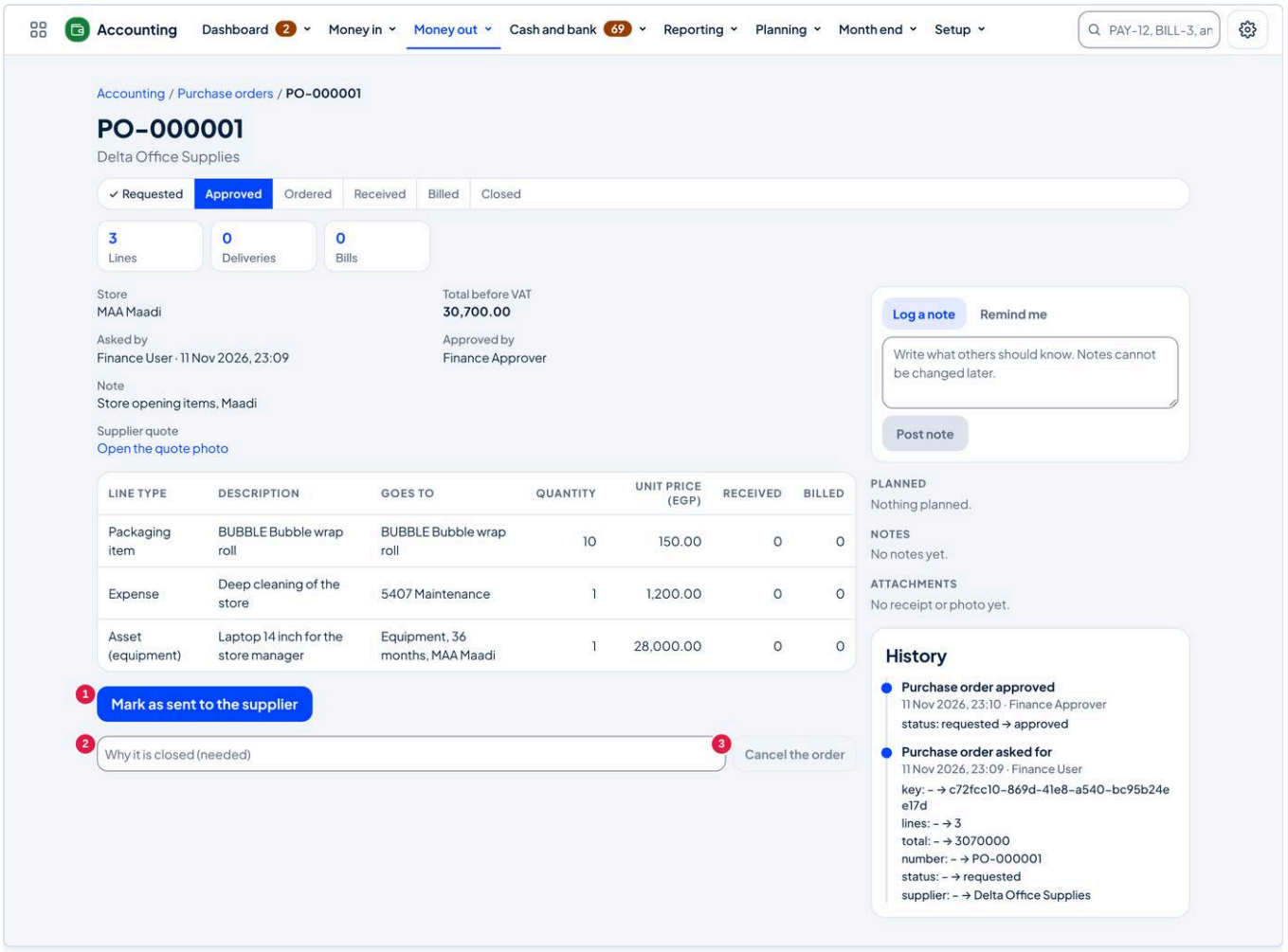


Requested, as the asker sees it: "Waiting for another person with the approve level." and the close field.

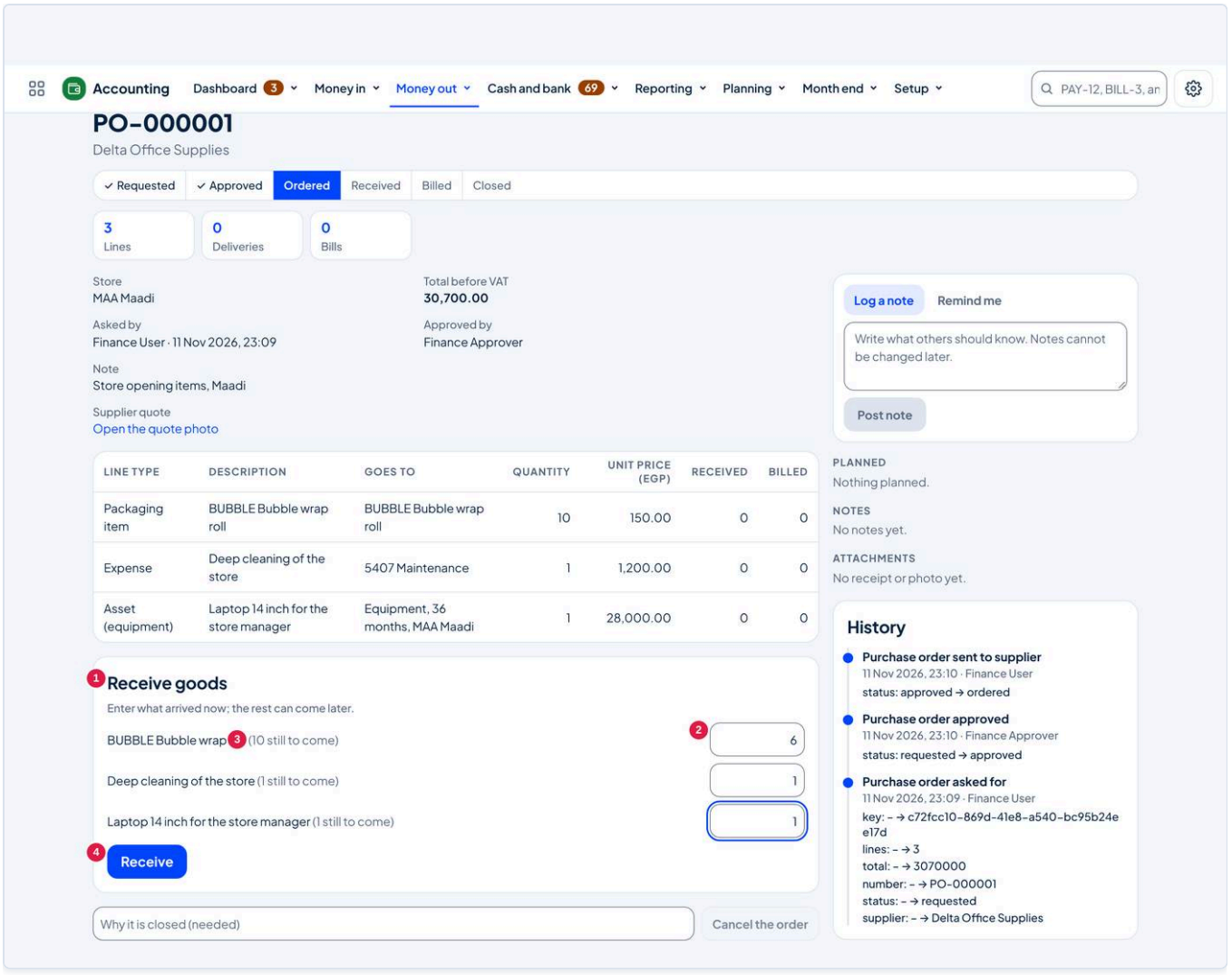


Requested, as a second person with Accounting manage sees it.

#	Part	What it shows	What clicking it does
1	Breadcrumbs	Accounting / Purchase orders / PO-000001	Back to the list.
2	Number	PO number, supplier in grey	
3	Note (needed to reject)	Text box in the Approval card	Required only to reject.
4	Approve	Blue button	"Approved." The order moves to Approved. The asker gets "Someone else must approve what you asked for." if they try.
5	Reject	White button with red text, disabled until a note is typed	"Rejected." The order ends in red Rejected.

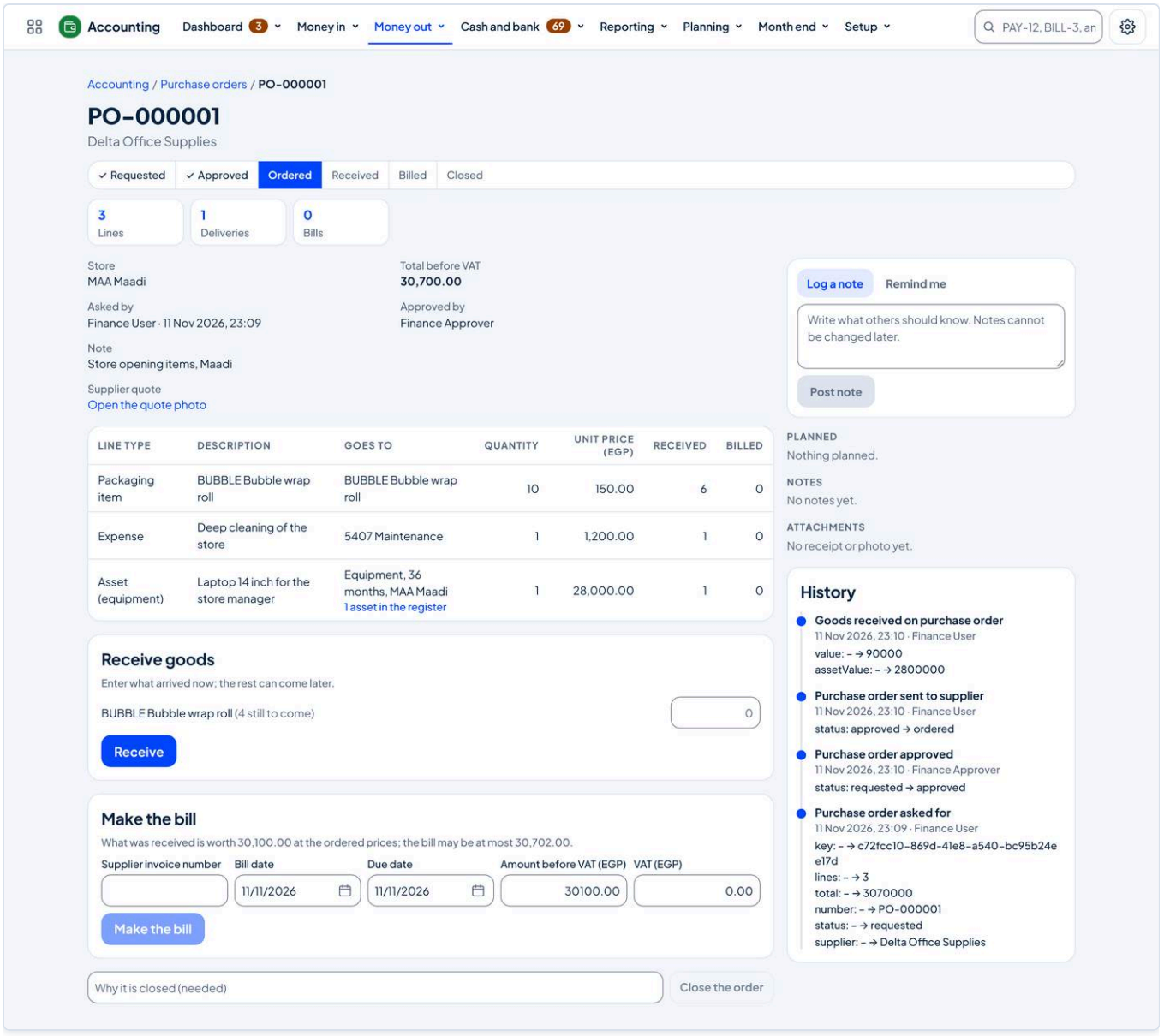


Approved: "Mark as sent to the supplier" (Accounting use) and the close field with "Cancel the order".

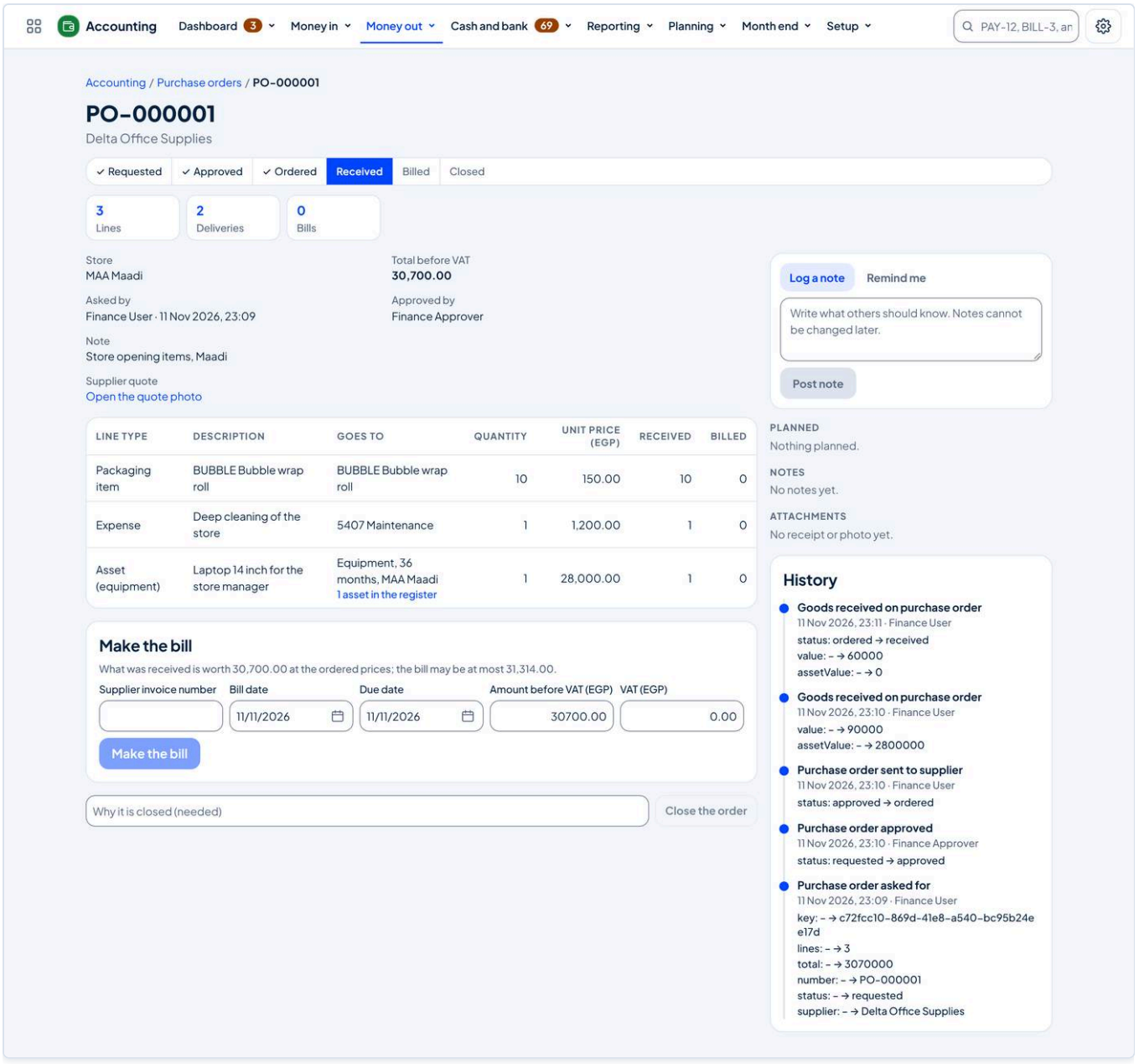


Ordered: the Receive goods card, one quantity box per line with "(10 still to come)".

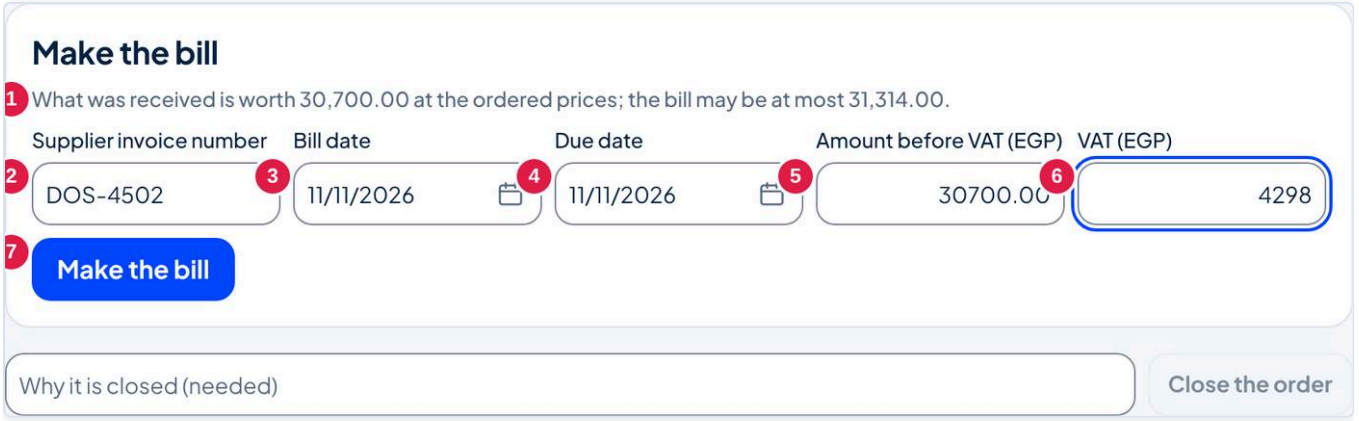
#	Part	What it shows	What clicking it does
1	Receive goods	Heading and "Enter what arrived now; the rest can come later."	
2	Quantity box	One per line still to come	Whole numbers; more than ordered answers "More than was ordered."
3	Still to come	Grey "(10 still to come)"	
4	Receive	Blue button	"Received." Packaging goes into the store's packaging stock (Dr 1402 Packaging stock, Cr 2401 Goods received not billed, at the ordered price); each asset unit becomes an equipment record (Dr 1601, Cr 2401). Each delivery is booked once per key.



Partly received: 6 of 10 rolls, the cleaning and the laptop received; the laptop line links "1 asset in the register"; the Make the bill card appears.

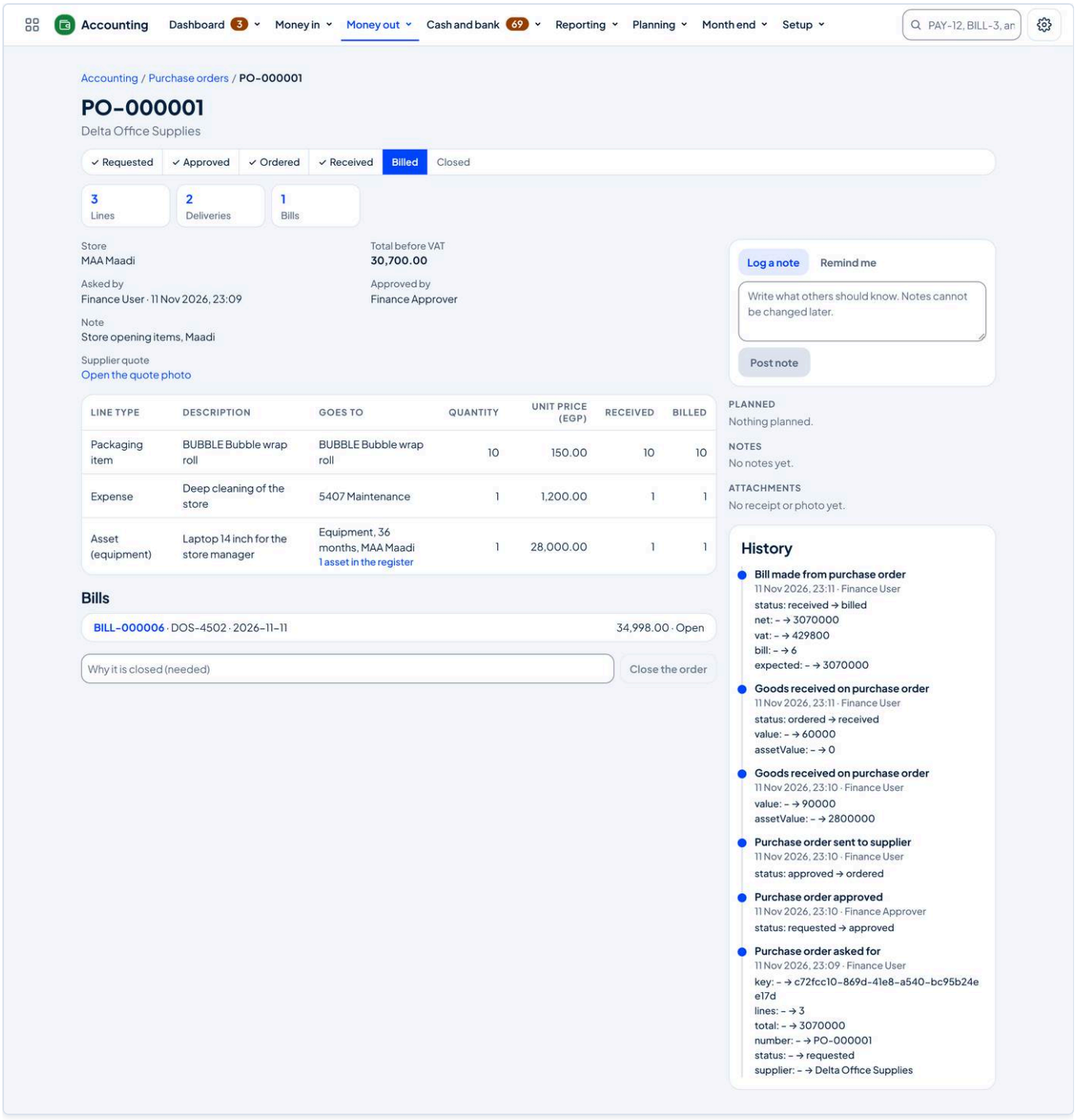


Received: every line arrived; only Make the bill and Close remain.

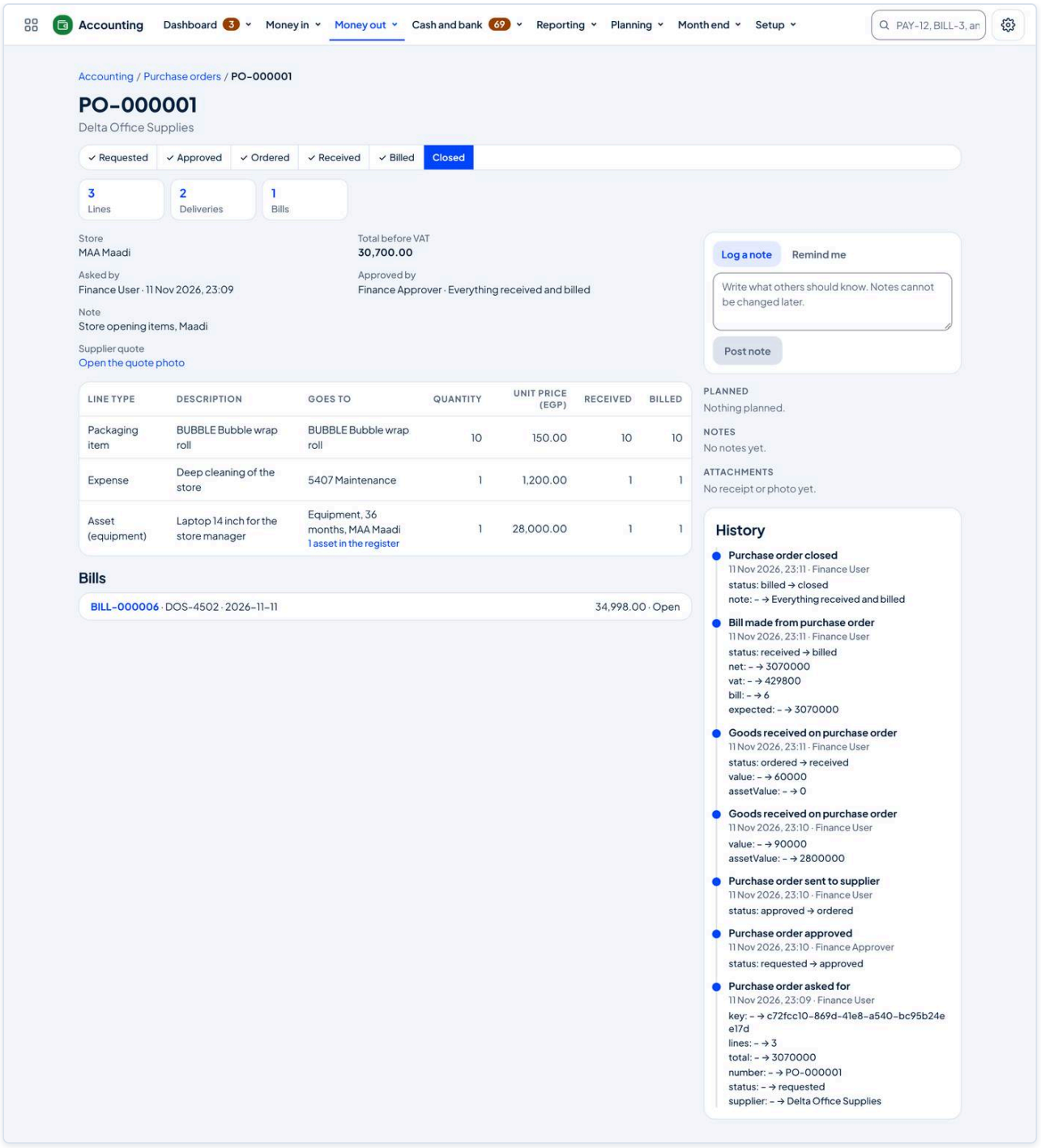


Make the bill from the order.

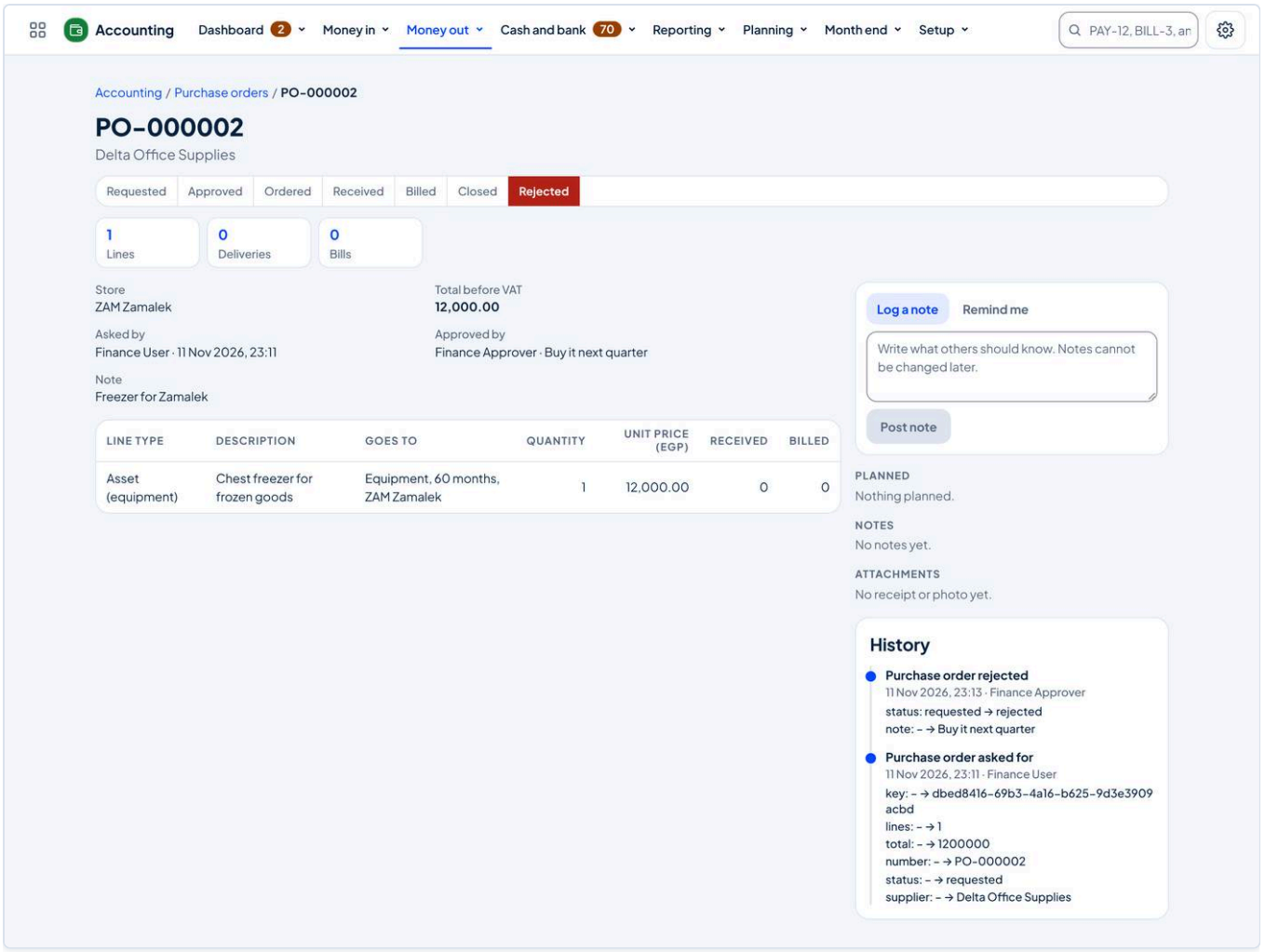
#	Control	What it does	Rule / limits	Result
1	Hint	"What was received is worth 30,700.00 at the ordered prices; the bill may be at most 31,314.00."	At most `finance.po_bill_tolerance_bp` above (default 2%)	Above it: "The bill is above the ordered prices by more than allowed."
2	Supplier invoice number		Once per supplier ("This supplier invoice number is already recorded.")	
3, 4	Bill date, Due date	Day first		
5	Amount before VAT (EGP)	Prefilled with the received value		The difference to the ordered value goes to the order's expense.
6	VAT (EGP)			
7	Make the bill	Blue button	Accounting use	"Bill recorded. Pay it from Bills." A normal bill (BILL-000006, 34,998.00 with VAT 4,298.00) that clears 2401, books the cleaning to 5407 and VAT to 2301, against the supplier's wallet. It is paid from Supplier bills and cannot be cancelled by reversal.



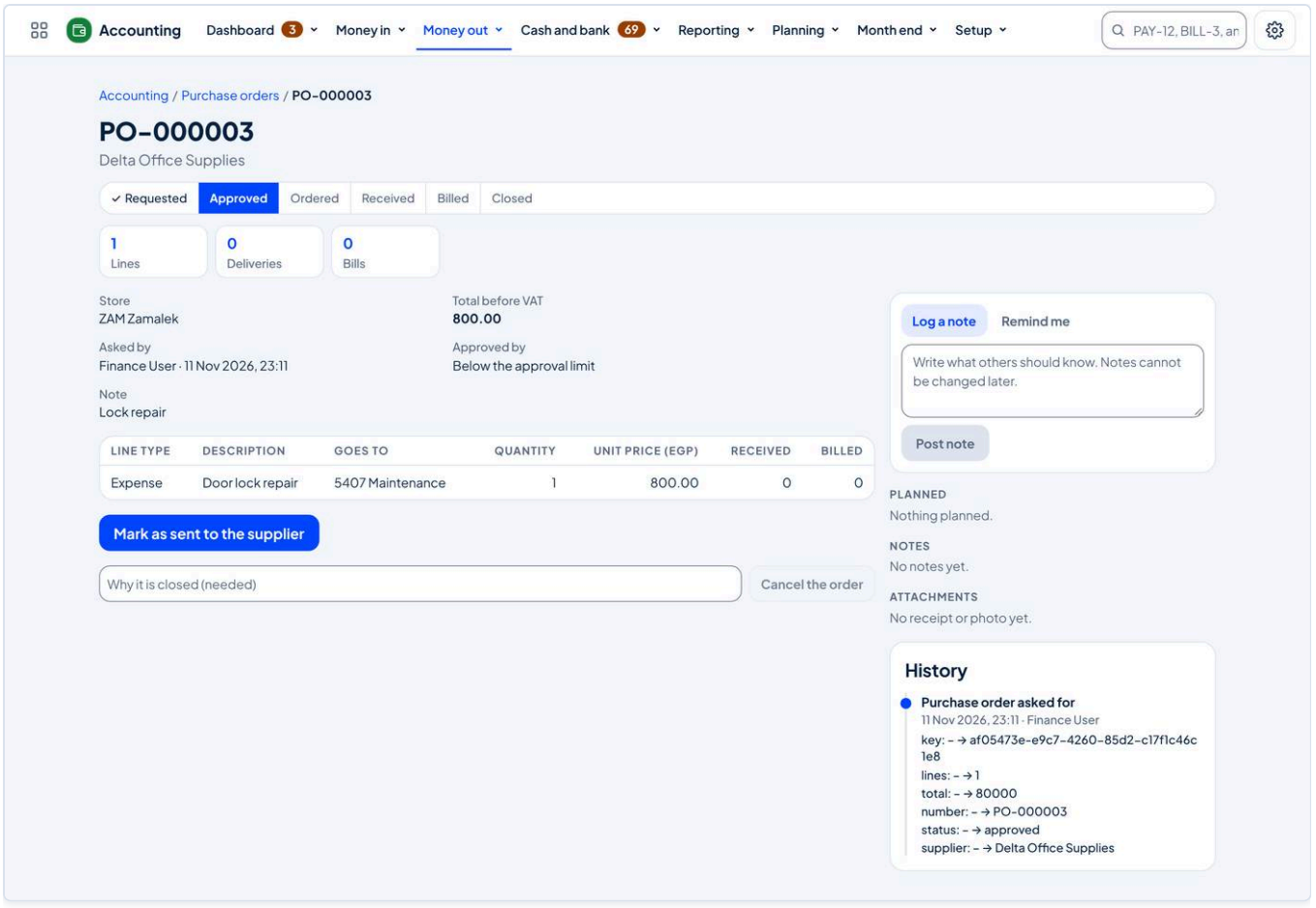
Billed: the Bills table lists BILL-000006 with its amount and "Open"; the close field reads "Close the order".



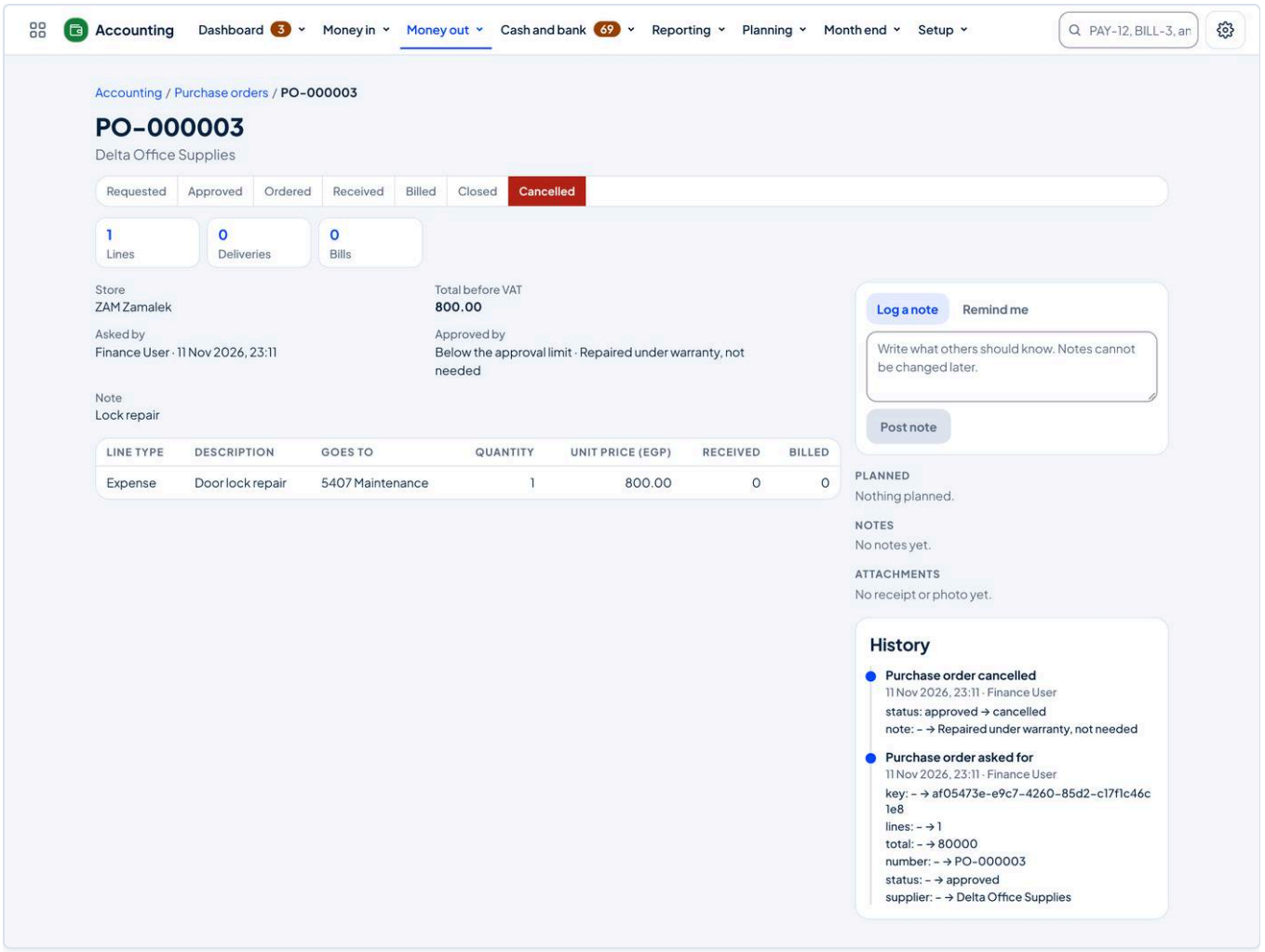
Closed with a note; the close note is shown with the approval details.



Rejected by the second person, red Rejected at the end of the status bar; the line "Approved by" shows the person who decided and the note.



An 800.00 EGP order approved at once: "Approved by: Below the approval limit".

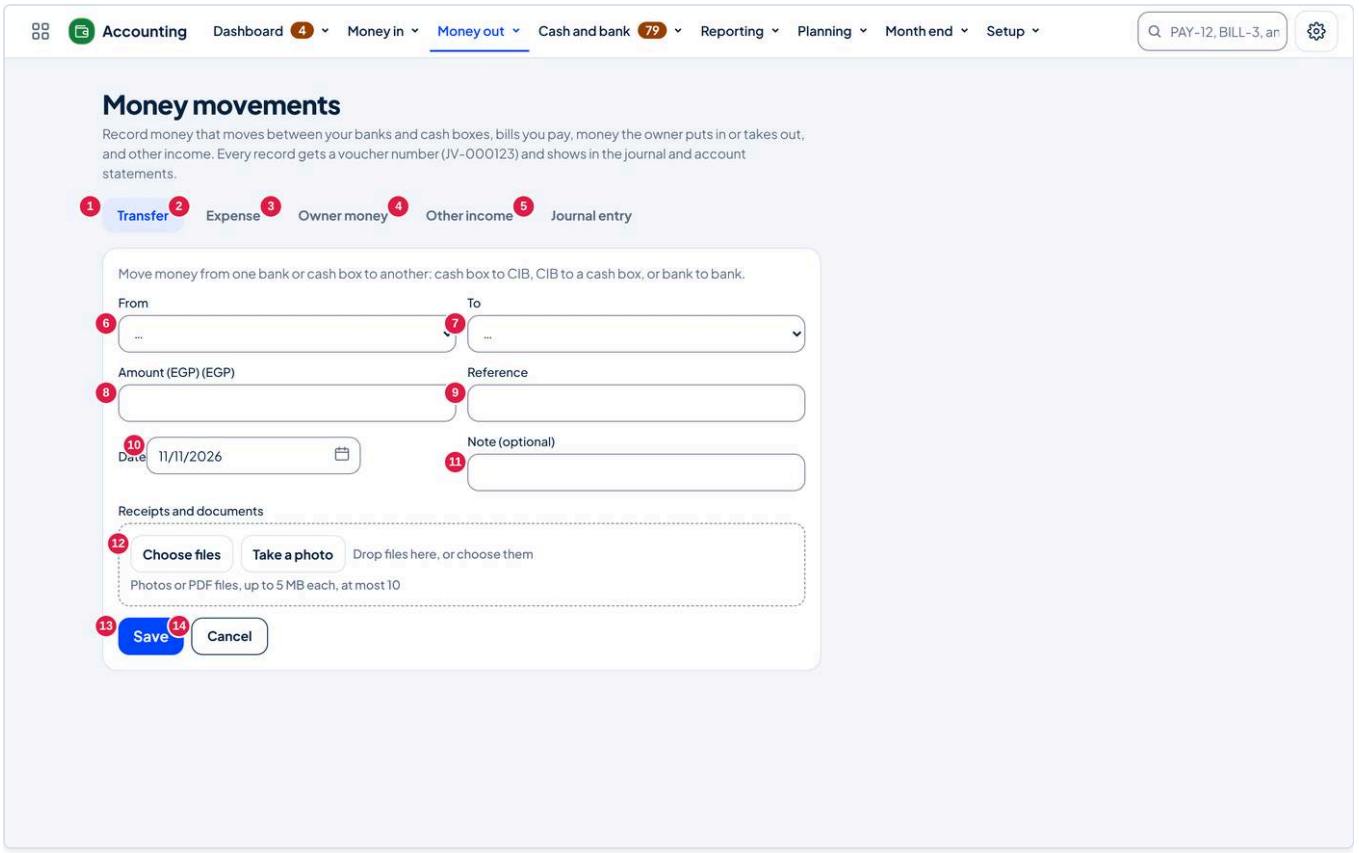


Cancelled: closed before anything arrived; the button read "Cancel the order" and ends in red Cancelled.

Close rules: "Why it is closed (needed)" must be filled; closing is refused while received goods are not billed ("Bill what was received before closing."). With nothing received the button reads *Cancel the order* and the order ends Cancelled; otherwise *Close the order* and it ends Closed.

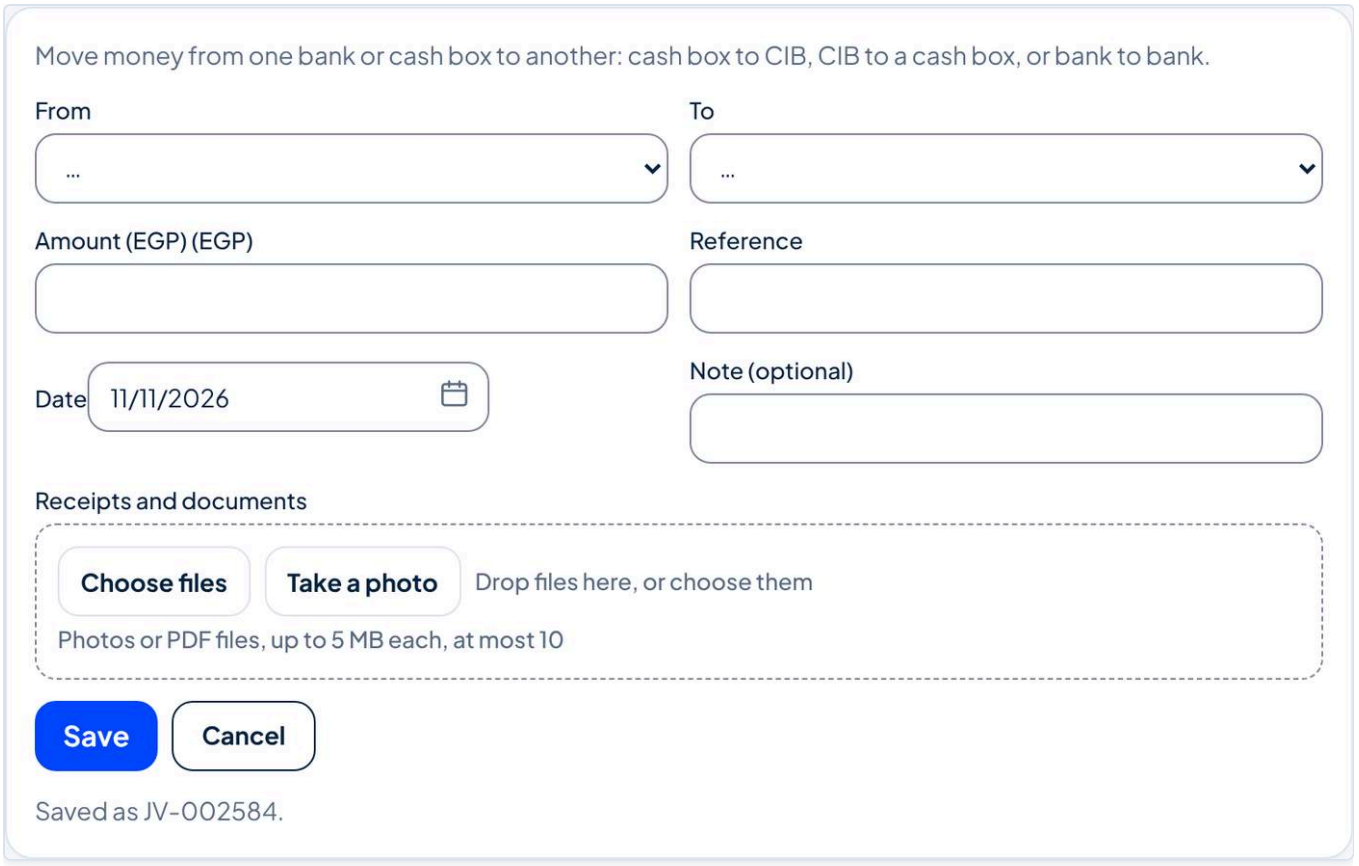
14.5 Money movements

Address: `/finance/movements?tab=transfer|expense|capital|income|journal`. **Menu:** Accounting > Money out > Expenses and transfers. **Who:** Accounting `use` to save. Five tabs, each a form that posts a voucher (JV number) or sends a request to Approvals. Every tab ends with the attachment field and Save / Cancel (FormFooter). After a save the message under the buttons reads "Saved as JV-002584." or "Saved and sent for approval. A second person must approve it before it is posted.", and the form empties.



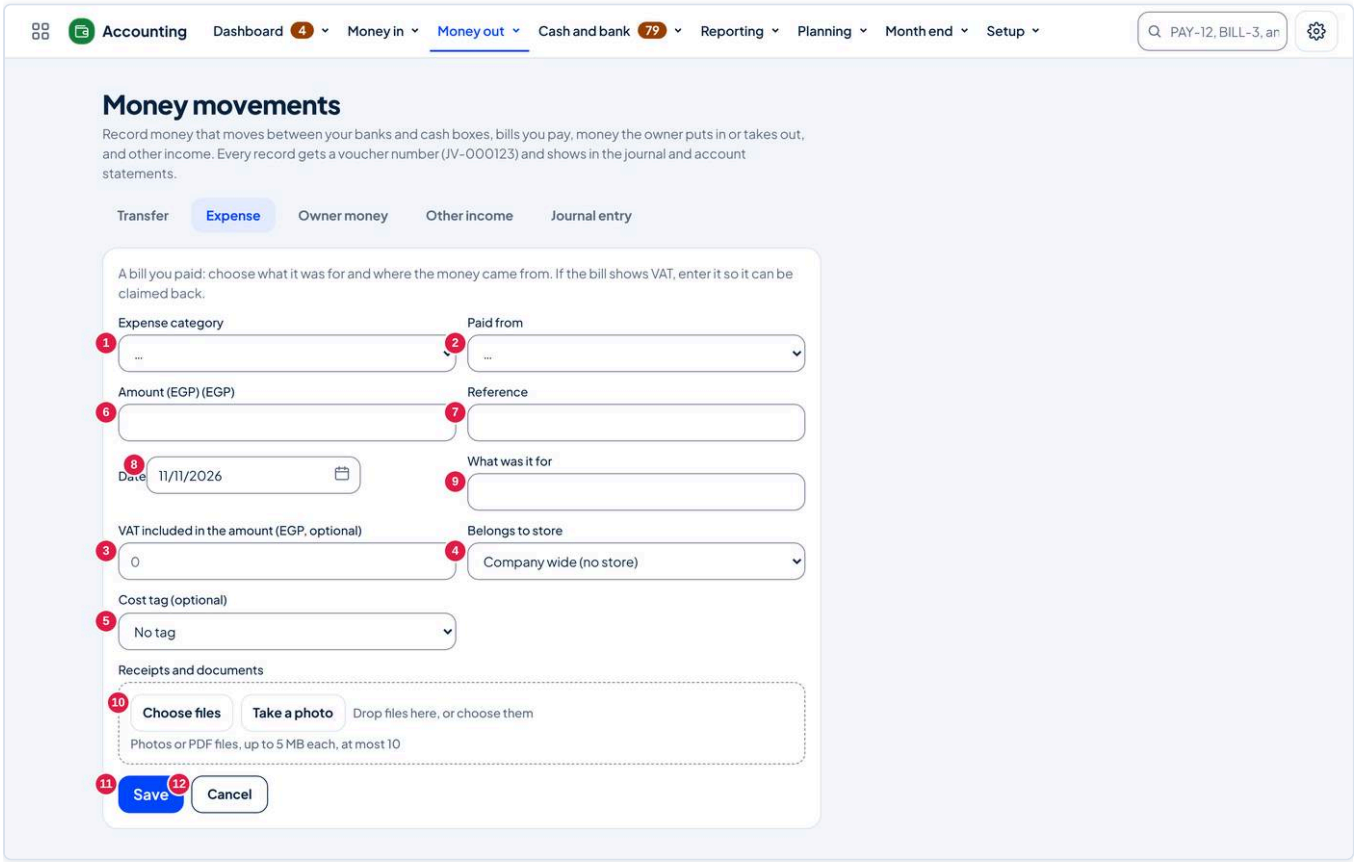
The Transfer tab with the five tabs (/finance/movements?tab=transfer).

#	Control	What it does	Rule / limits	Result
1 to 5	Tabs Transfer, Expense, Owner money, Other income, Journal entry	The chosen tab has a light blue ground and blue text; each tab has a grey help line on top of its form		Changes `?tab=`.
6	From	Banks and cash boxes with balances	Required; must hold the amount	Credited.
7	To	Banks and cash boxes	Required; not the same account	Debited.
8	Amount (EGP)	The amount	"Enter an amount in EGP, for example 1250.50."	
9	Reference	Required, 2 characters or more		
10	Date	Day first, today by default	Never future, never in closed books	
11	Note (optional)	Memo		
12	Attachment field			
13, 14	Save, Cancel		From the approval limit the transfer waits; a foreign currency account adds "Foreign amount"	Transfer of 2,000.00 from CIB to the Maadi box: "Saved as JV-002584." (Dr 1201, Cr 1101).



The transfer saved: the form empties and shows the voucher number.

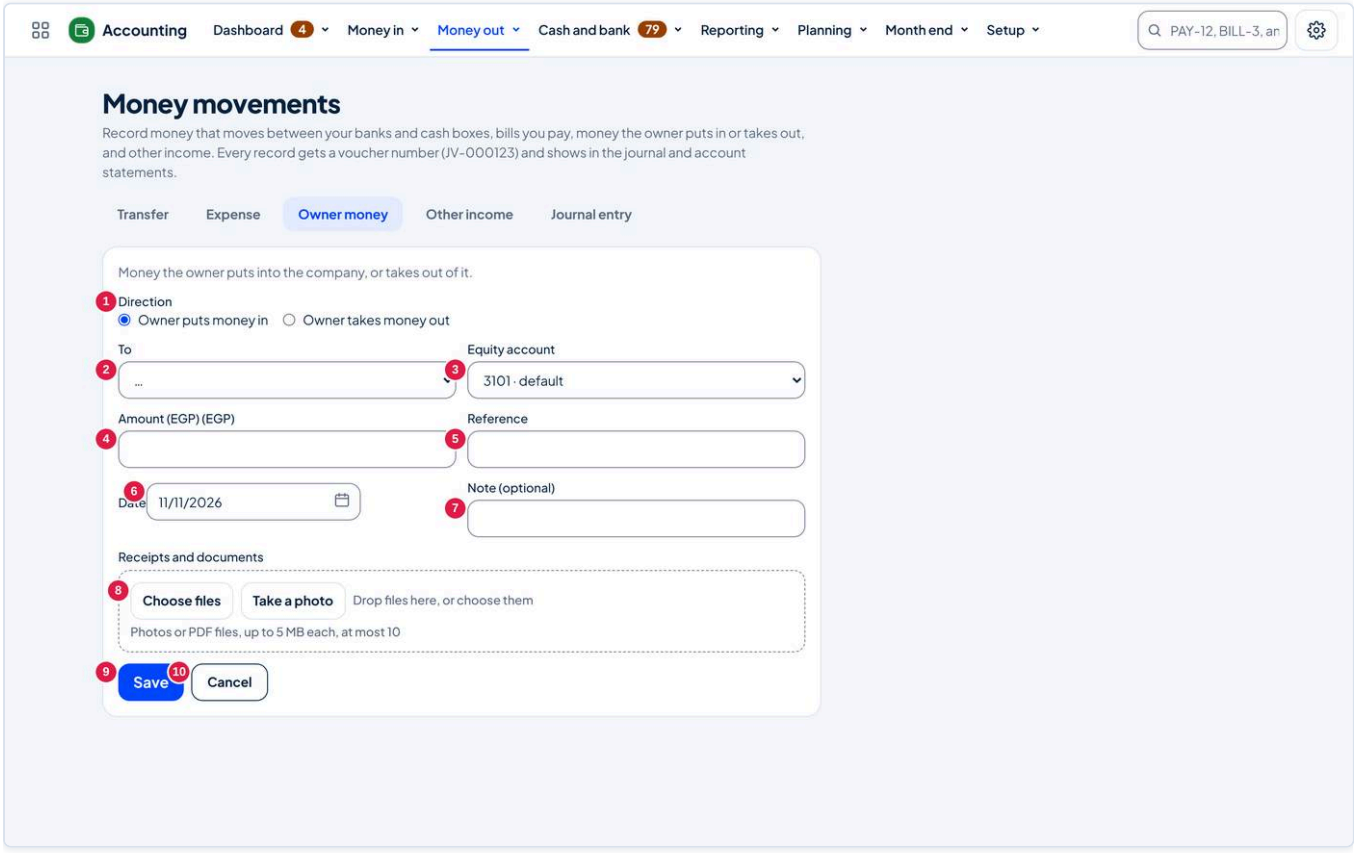
Expense tab



The Expense tab: the full expense form.

#	Control	What it does	Rule / limits
1	Expense category	Open 5400 and 5500 accounts	Required
2	Paid from	Banks and boxes with balance	Must hold the amount
3	VAT included in the amount (EGP, optional)	Input VAT to claim back	Below the amount
4	Belongs to store	"Company wide (no store)" or a store	
5	Cost tag (optional)	"No tag" or an active tag	
6 to 9	Amount, Reference, Date, What was it for	Here the note is required ("What was it for", 3 characters or more)	
10 to 12	Attachment field, Save, Cancel	Same posting as Record an expense	Posted, or waiting from the limit

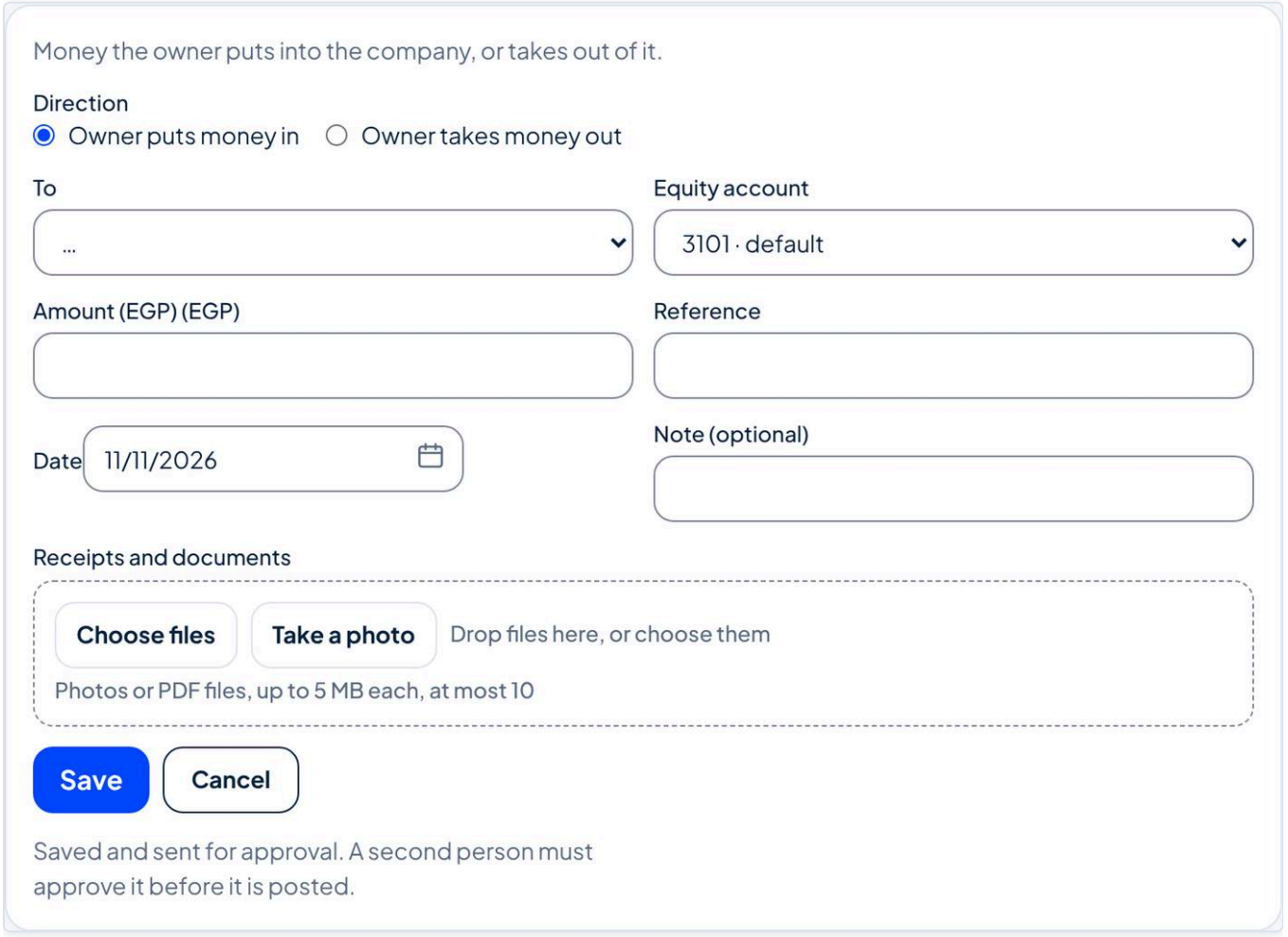
Owner money tab (capital and drawings)



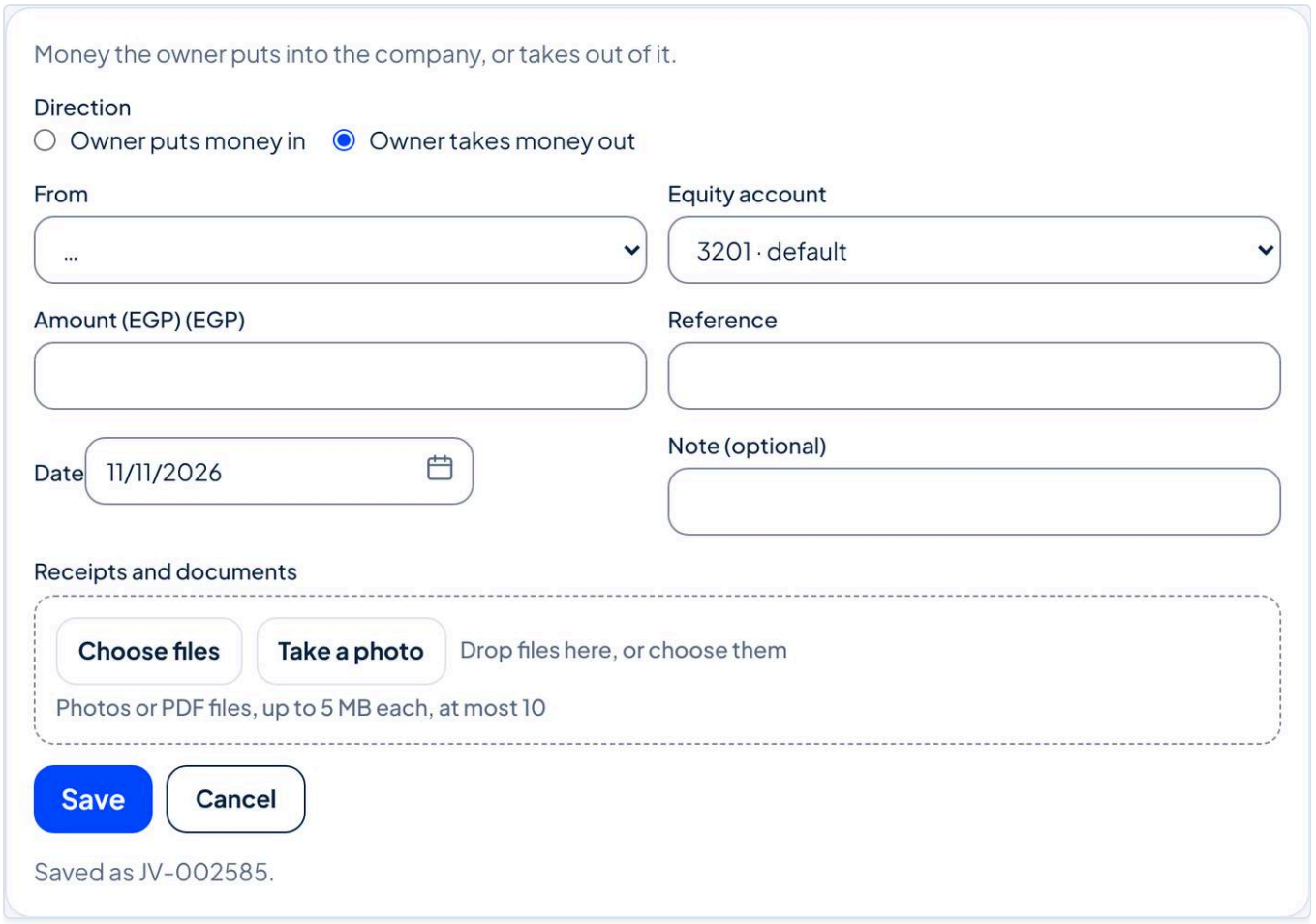
Owner money with "Owner puts money in" chosen.

#	Control	What it does	Rule / limits	Result
1	Direction	Radio: Owner puts money in / Owner takes money out		Switches the money label between To and From and the default equity account.
2	To / From	The bank or box	Money out must be held	

#	Control	What it does	Rule / limits	Result
3	Equity account	"3101 · default" for money in (capital), "3201 · default" for money out (drawings), or another 3100 or 3200 account		
4 to 10	Amount, Reference, Date, Note, attachment field, Save, Cancel		Money in always waits for a second person; drawings wait from the limit	50,000.00 in: "Saved and sent for approval…". 1,500.00 out: "Saved as JV-002585." (Dr 3201, Cr 1101).

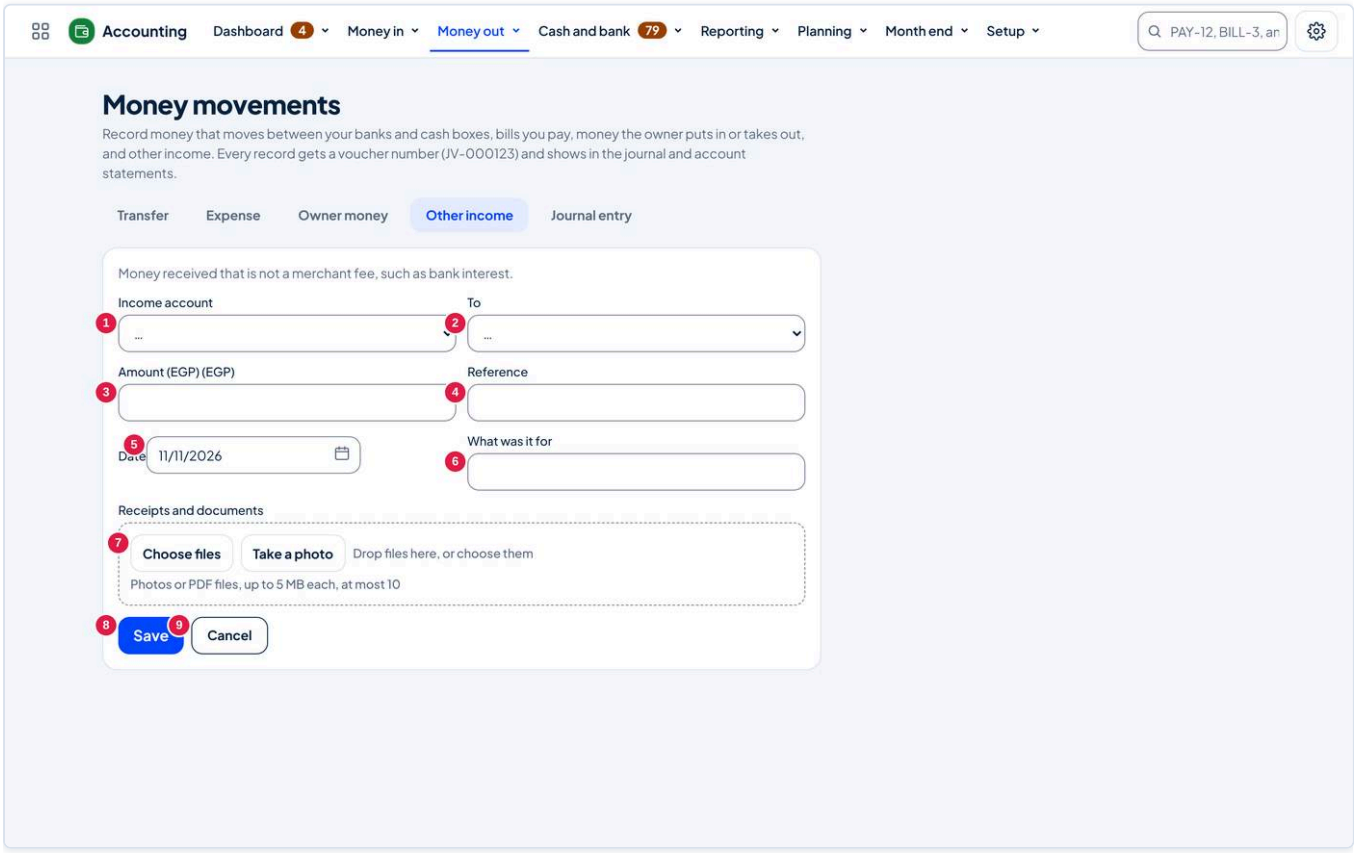


Owner capital in with a PDF attached, sent for approval.



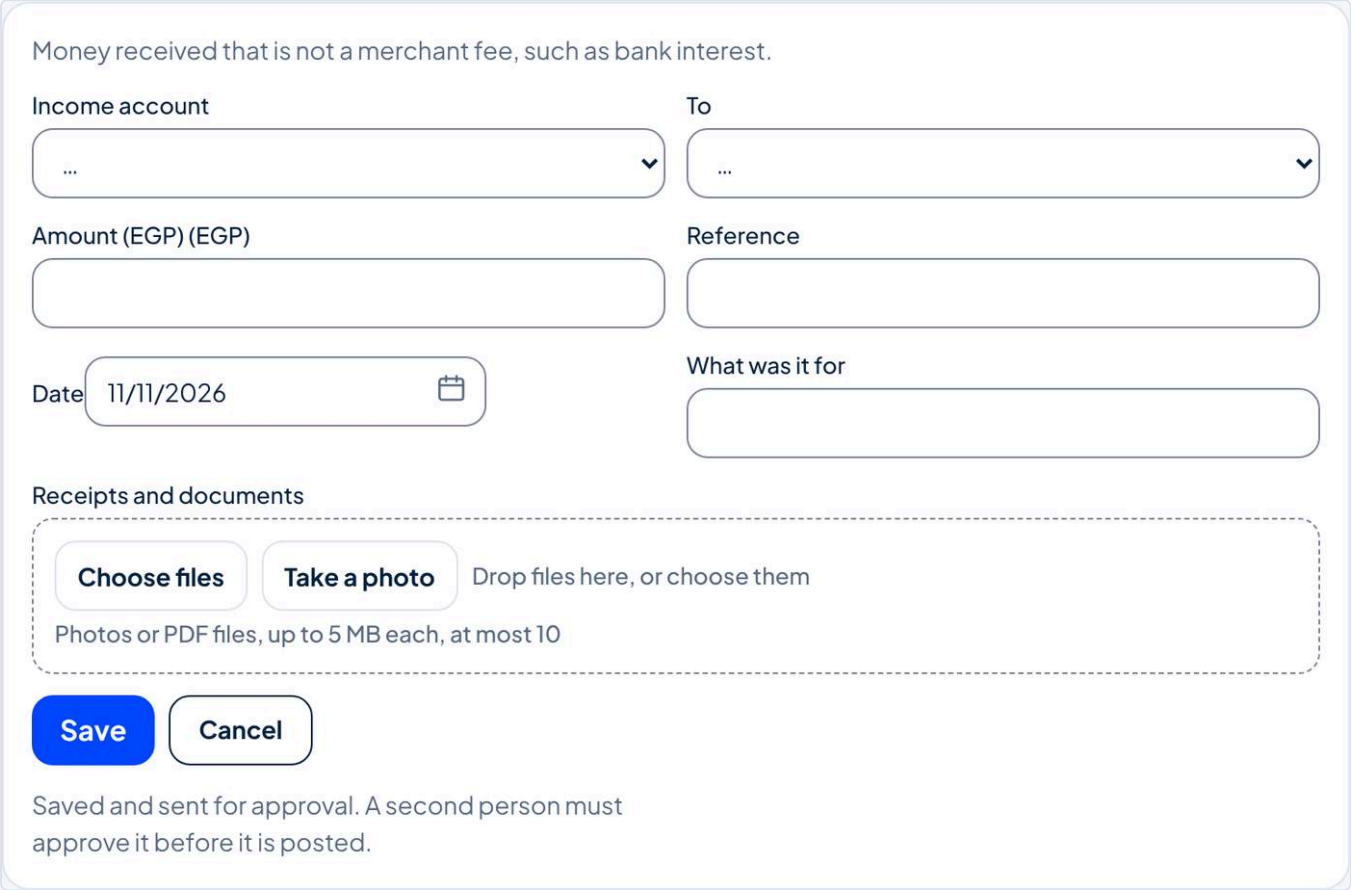
Drawings ("Owner takes money out", 3201 default) posted at once below the limit.

Other income tab



Other income: income account (4200 group), the bank or box it came into, amount, reference, date, "What was it for" (required), attachments.

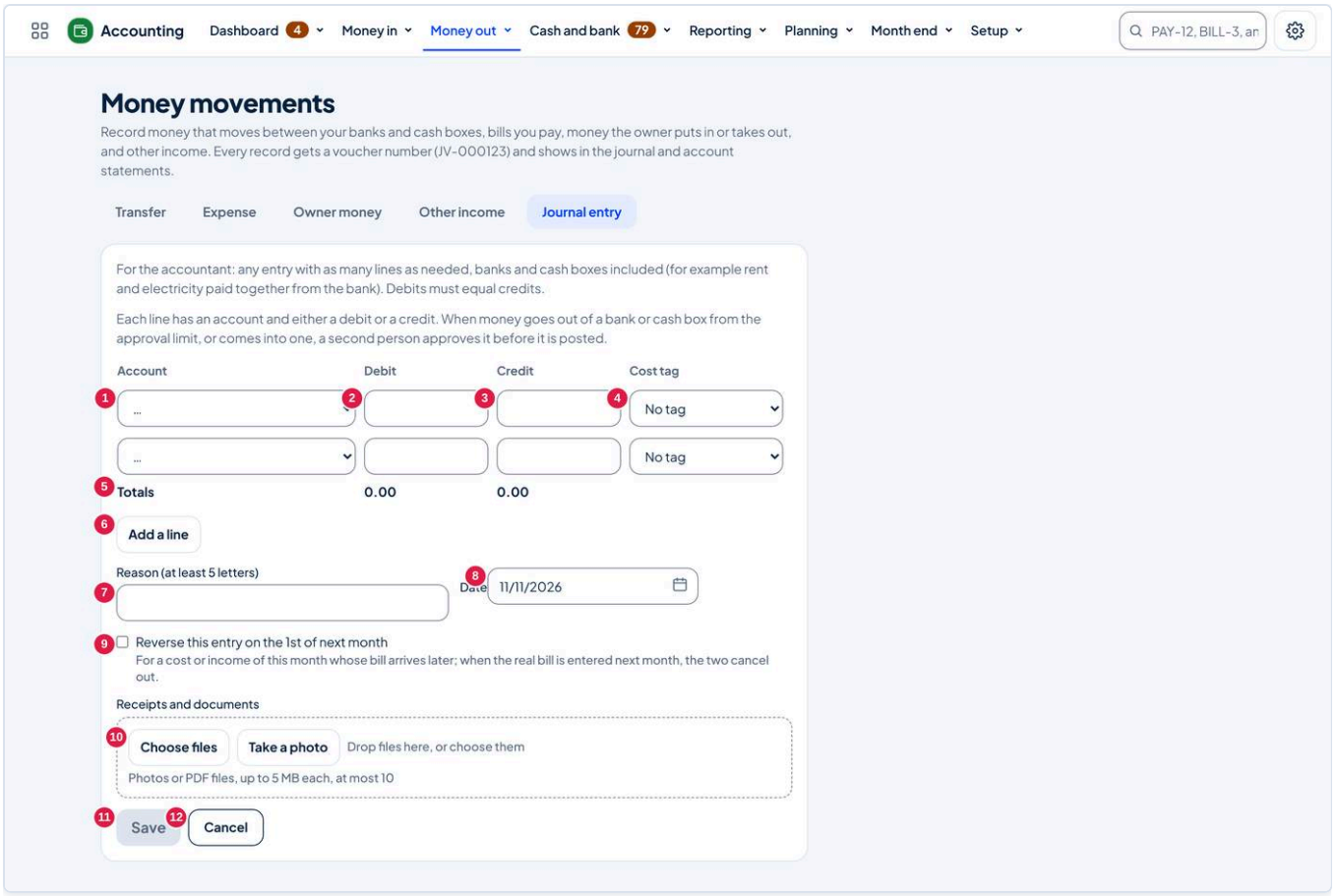
Money in always waits for a second person while `finance.approve_money_in` is 1. Bank interest of 450.00 EGP was sent for approval.



Other income sent for approval.

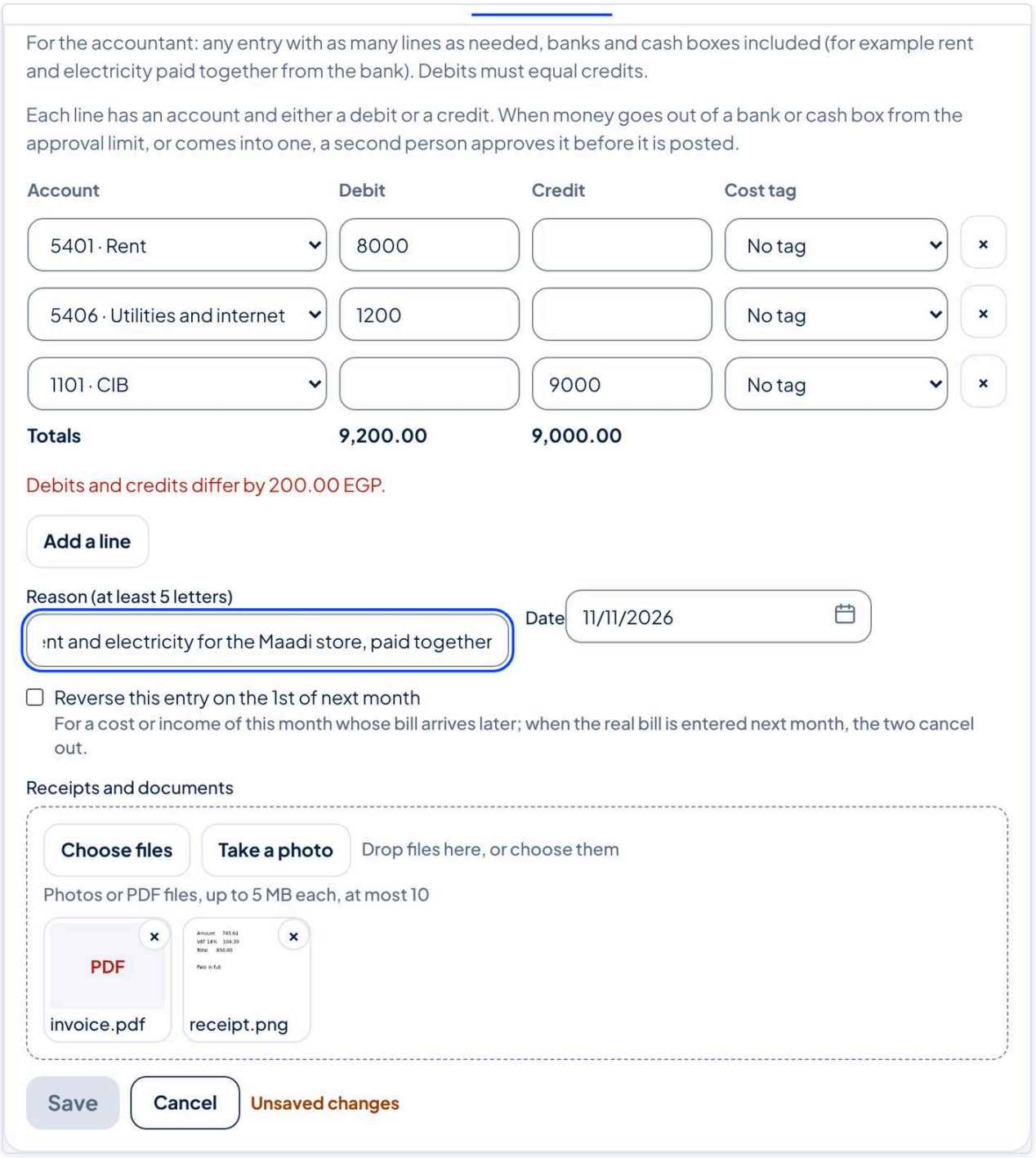
Journal entry tab

For the accountant: any balanced entry with as many lines as needed, banks and cash boxes included (for example rent and electricity paid together from the bank). It is checked in full before saving or queuing (`checkJournalEntry`): debits equal credits, at least two lines, each line either a debit or a credit, no merchant or rider wallet, no staff advance wallet, no rider cash (`money_account_not_allowed`), money out never above what a bank or box holds, an accrual may not touch a bank or box (`accrual_money_not_allowed`). Money coming into a bank or box waits for a second person; money going out waits from the approval limit; otherwise it posts at once.

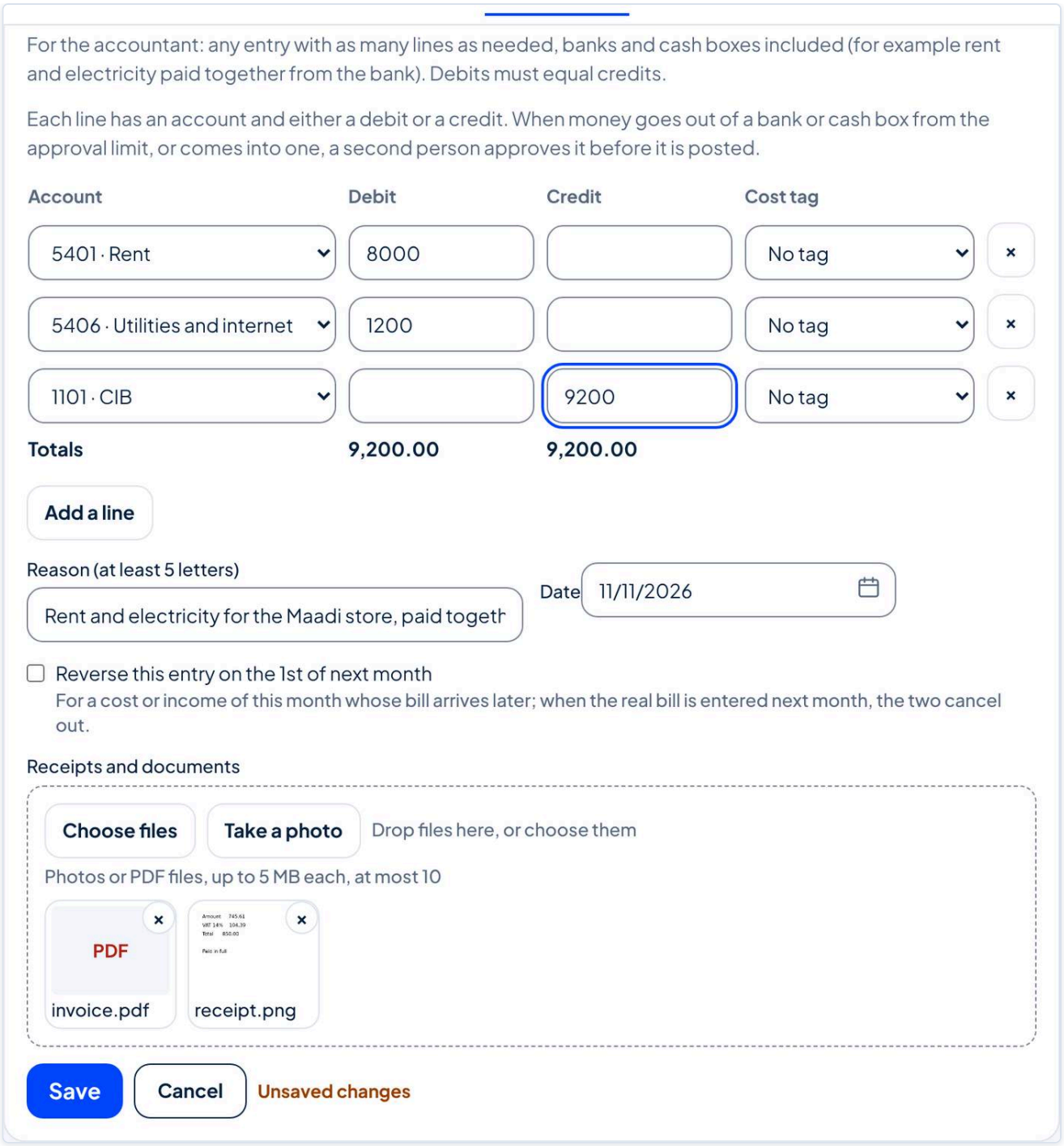


The Journal entry tab, empty, with two lines.

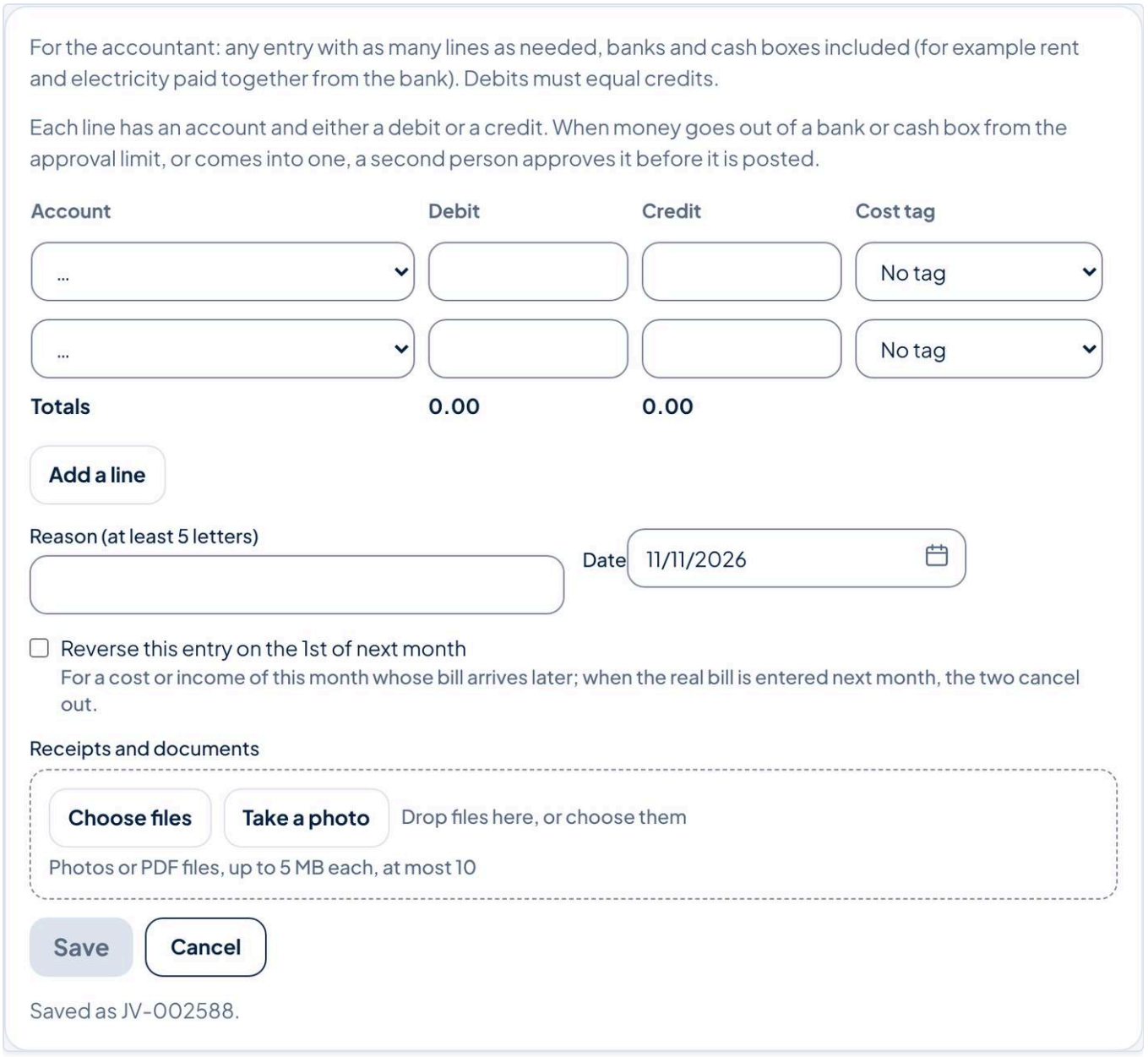
#	Control	What it does	Rule / limits
1	Account	Every account allowed in a journal entry, "5401 · Rent"	Wallets and rider cash are left out
2, 3	Debit, Credit	Typing in one empties the other	Amounts in EGP
4	Cost tag	"No tag" or an active tag, per line	Column shown when cost tags exist
	× Remove line	Shown on every line once there are three or more	
5	Totals	Debit and credit totals; when they differ a red line "Debits and credits differ by 200.00 EGP." and Save stays grey	
6	Add a line	Adds an empty line	
7	Reason (at least 5 letters)	The entry's memo	Required
8	Date	Day first	Not future, not in closed books
9	Reverse this entry on the 1st of next month	Tick box with the hint "For a cost or income of this month whose bill arrives later…"	An accrual; the daily job reverses it on the first day of next month
10	Attachment field		
11, 12	Save, Cancel		



Three lines (rent 8,000.00 and utilities 1,200.00 debited, CIB 9,000.00 credited): the red difference line and a grey Save.



Balanced at 9,200.00 with a PDF and a PNG attached; Save turns blue.

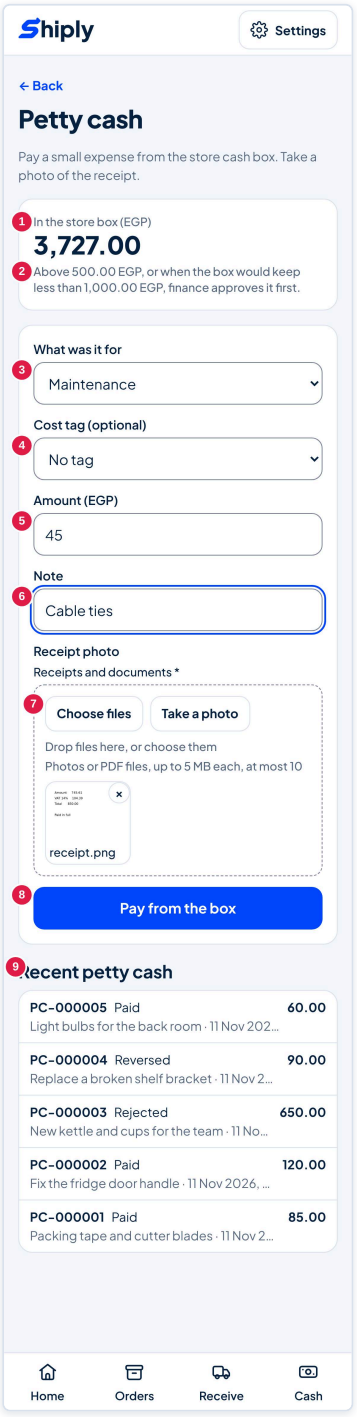


Saved as JV-002588: money out of 9,200.00 is below the limit, so it posted at once.

14.6 Petty cash

Address: `/finance/petty-cash` ; one record at `/finance/petty-cash/<id>` ; the store side at `/store/<storeId>/petty-cash` . **Menu:** Accounting > Money out > Petty cash; staff open it from the store home. **Who:** store staff with Store `use` pay from their store box; Accounting `view` reads the list; Accounting `use` opens or closes petty cash categories; reversing needs Accounting `manage` . A payment above `finance.petty_cash_limit_piastres` (500.00 EGP), or one that would leave the box below `finance.petty_cash_float_piastres` (1,000.00 EGP), waits in Approvals as a petty_cash request. See the chapter Cash and riders, petty cash.

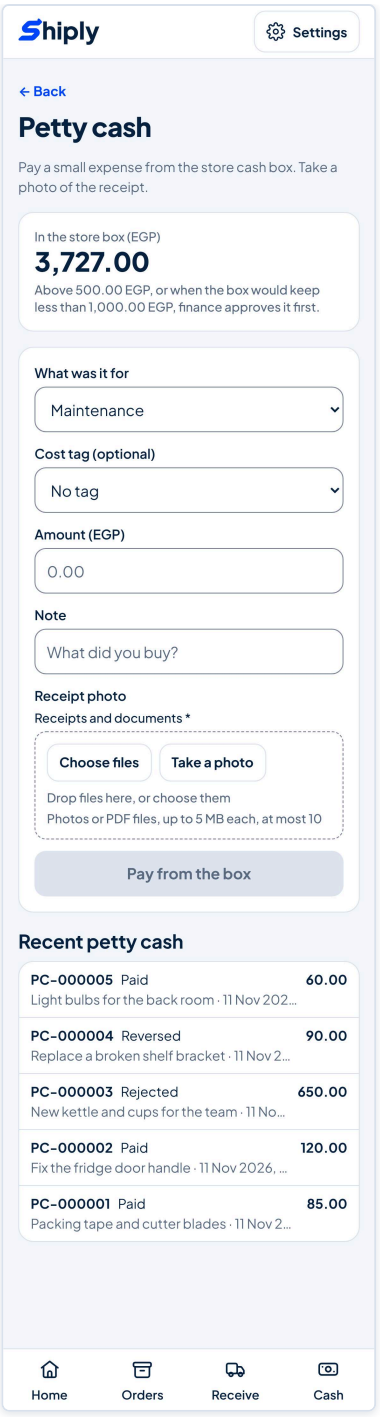
The store side (phone)



Petty cash on a store staff phone, filled in with a receipt photo (/store/<storeId>/petty-cash).

#	Part	What it shows	What it does
1	In the store box (EGP)	White card with the box balance in large bold figures	Read from the store's 1200 cash box.
2	Limit hint	"Above 500.00 EGP, or when the box would keep less than 1,000.00 EGP, finance approves it first."	
3	What was it for	Only the categories finance opened to stores (by name, without code)	With none: "Finance has not opened any petty cash category yet."

#	Part	What it shows	What it does
4	Cost tag (optional)	"No tag" or an active tag	
5	Amount (EGP)		"Enter the amount paid."; above the box: "The box does not hold that much cash."
6	Note	Placeholder "What did you buy?"	Required ("Write what it was for.")
7	Receipt photo	The attachment field, required (marked with *)	"Pay from the box" stays grey until a receipt is added.
8	Pay from the box	Blue button	"Saved as PC-000002. Paid from the box." or "Saved as PC-000003. Waiting for finance to approve it."
9	Recent petty cash	Number, status in green (Paid) or grey (Waiting for approval, Rejected, Reversed), note, person, time, amount	

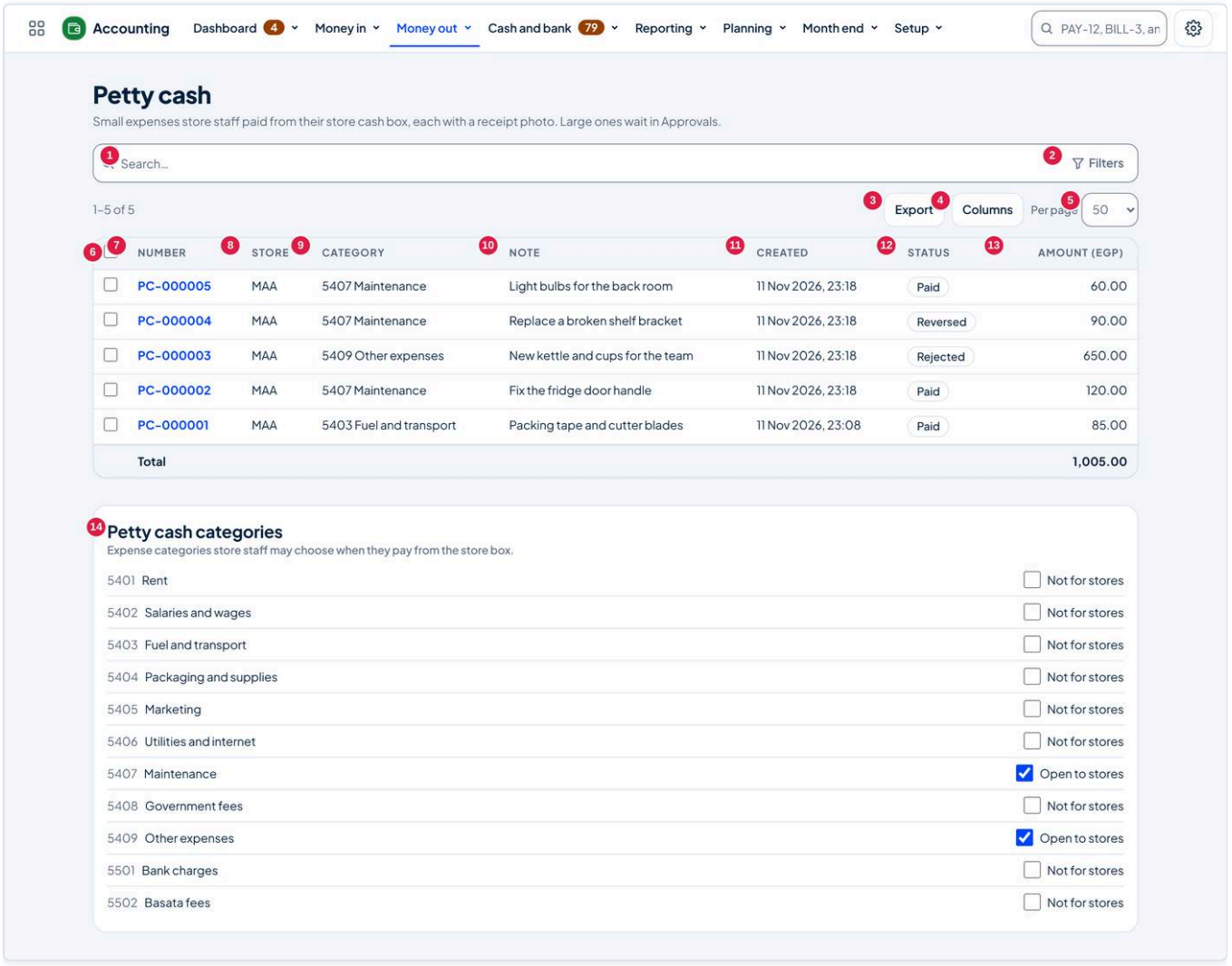


The recent list after several payments: paid, waiting for approval, rejected and reversed.

A posted payment is Dr the category, Cr the store's box, tagged with the store and the cost tag. The store's *Cash* screen (`/store/<storeId>/cash`) is where staff count riders' cash handovers and a store manager pays approved cash withdrawals; it is covered in the cash chapter.

The store's Cash from riders screen on a phone, reached from the Cash tab (/store/<storeId>/cash).

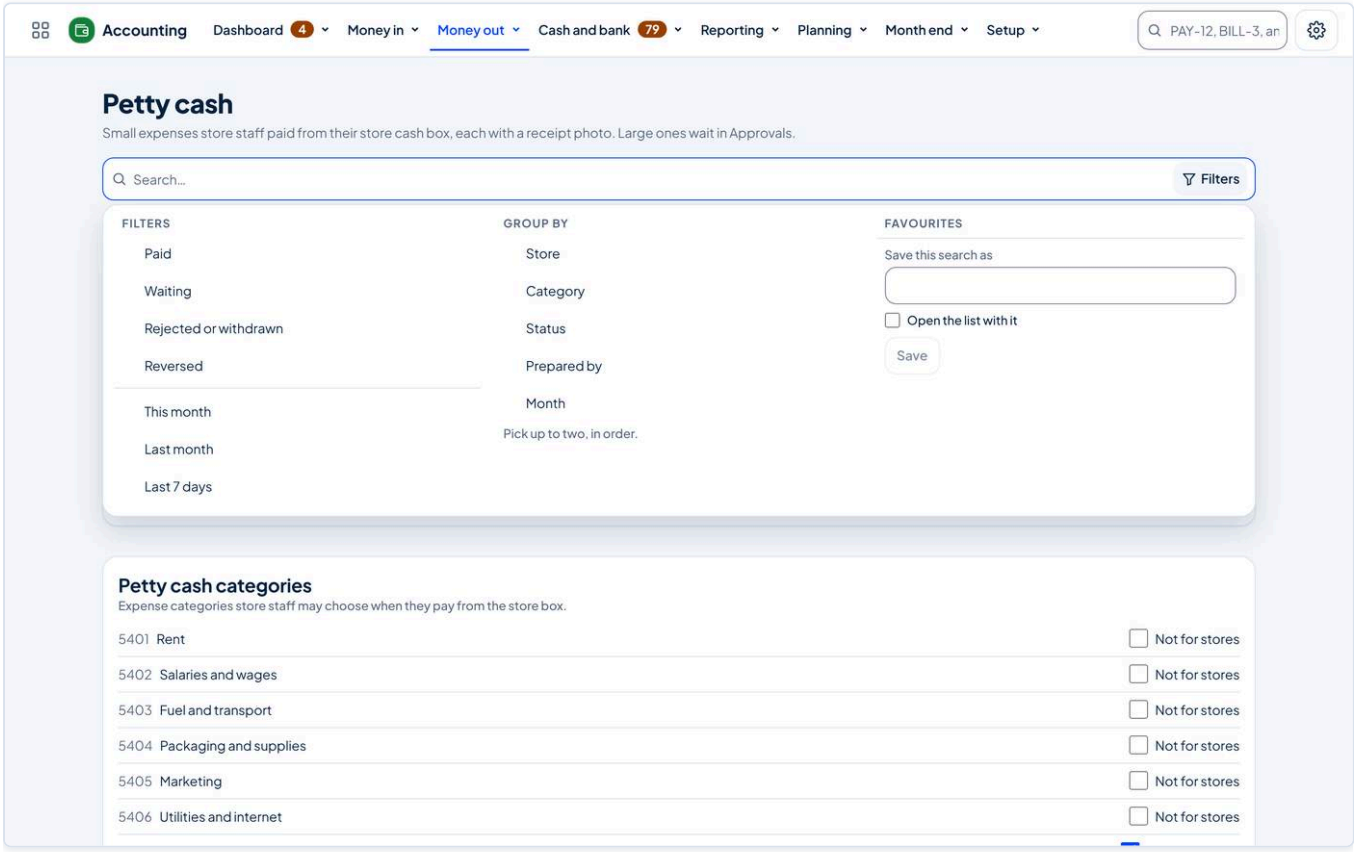
The finance list



Petty cash in Accounting, no default filter, and the petty cash categories under it (/finance/petty-cash).

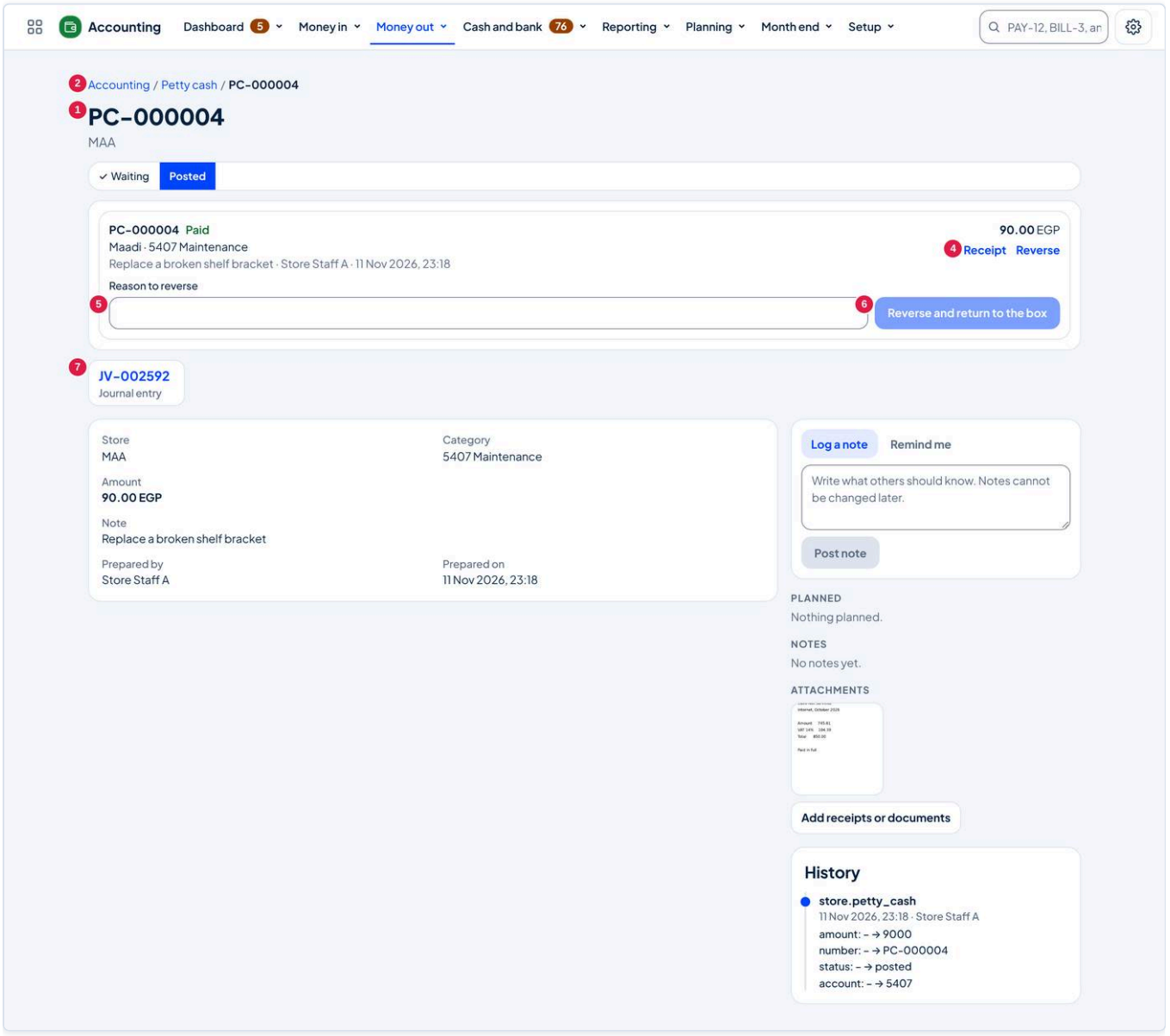
#	Control	What it does	Rule / limits
1	Search	Number, note, category, store	
2	Filters	Paid, Waiting for approval, Rejected or withdrawn, Reversed; This month, Last month, Last 7 days. Group by Store, Category, Status, Prepared by, Month	
3 to 5	Export, Columns, Per page	Optional column Prepared by	
6	Select all	No bulk action	
7	Number	`PC-000001` onward, gapless; opens the record	
8 to 11	Store, Category, Note, Created	Store code, account code and name, note, Cairo time	

#	Control	What it does	Rule / limits
12	Status	Grey outlined pill: Paid, Waiting for approval, Rejected, Withdrawn, Reversed	
13	Amount (EGP)	Summed in Total	
14	Petty cash categories	Every 5400 and 5500 account with a tick box "Open to stores" or "Not for stores"	Accounting `use`; each change is saved at once (`POST /finance/petty-cash/categories/<id>`) and audited



The Filters panel of Petty cash.

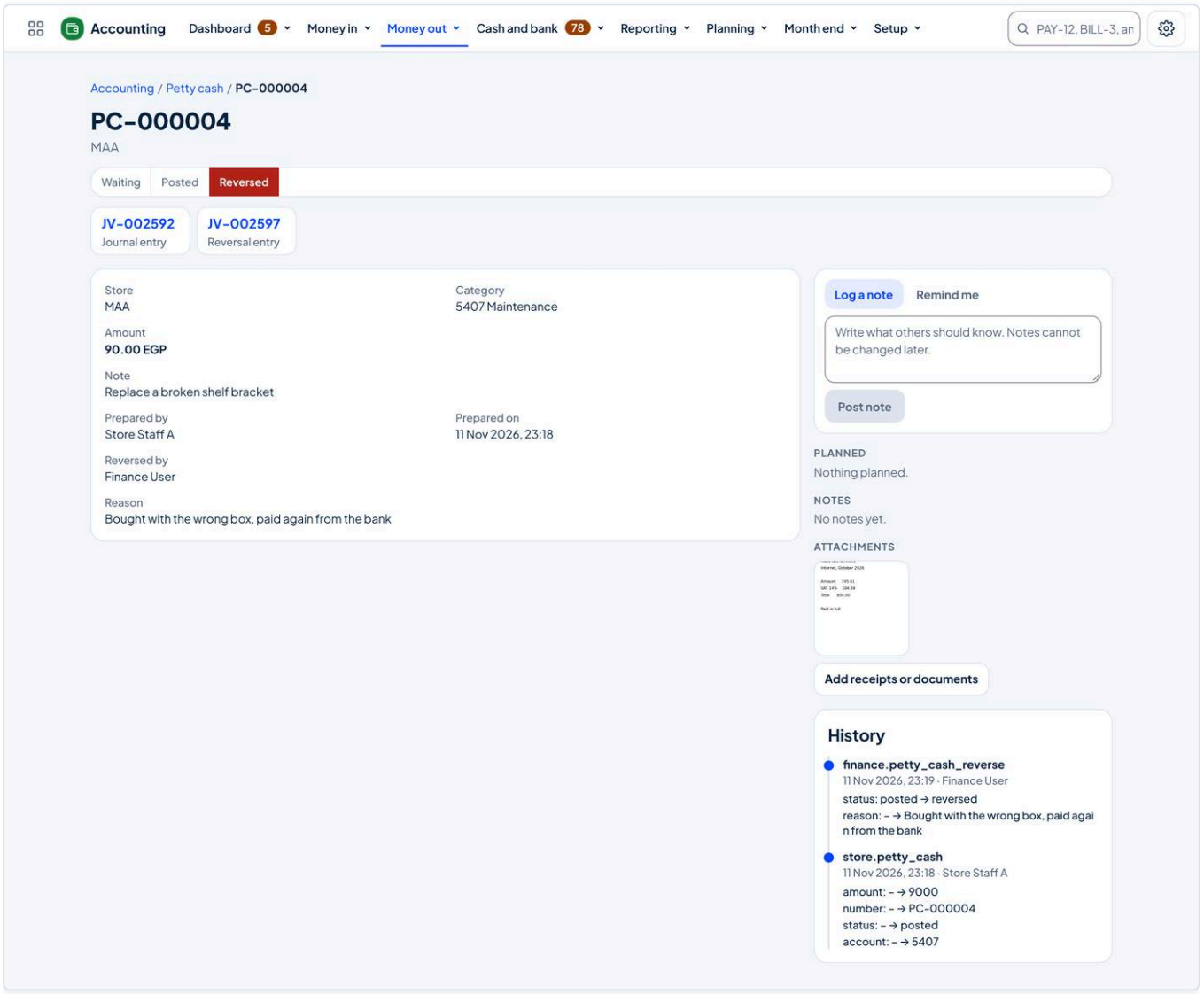
The petty cash page and its states



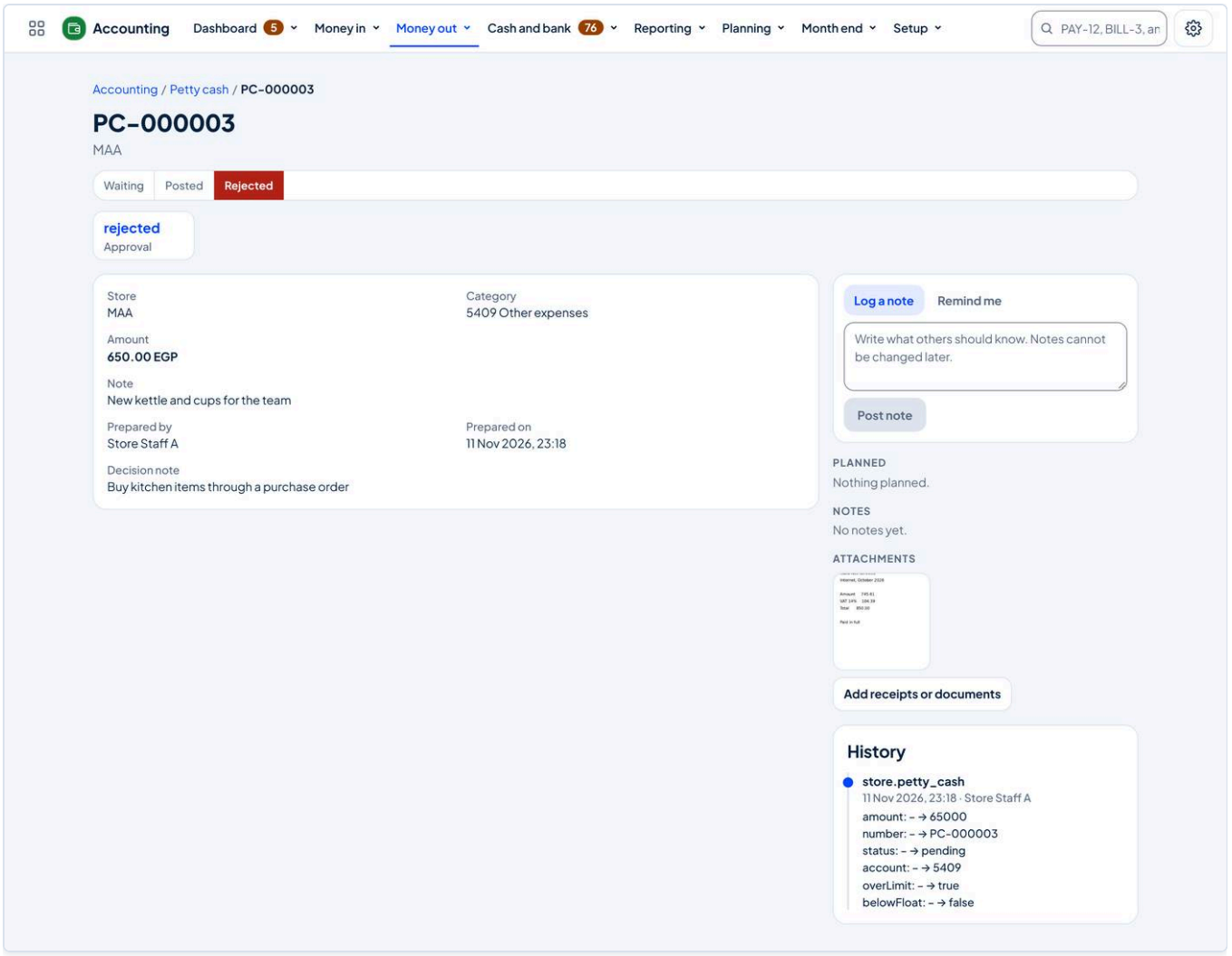
A paid record with the reverse form opened (/finance/petty-cash/<id>).

#	Part	What it shows	What clicking it does
1	Title	PC number, store code in grey	
2	Breadcrumbs	Accounting / Petty cash / PC-000004	Back to the list.
	Status bar	Waiting, Posted; Rejected or Reversed in red	
	Summary card	Number, status word in green "Paid", store, category, note, person, time, amount in bold	
4	Receipt, Reverse	Blue links on the right	Receipt opens the photo; Reverse (Accounting `manage`) opens the reason box. A waiting record shows "Approvals" instead.
5	Reason to reverse	Text box	Required.

#	Part	What it shows	What clicking it does
6	Reverse and return to the box	Blue button, light until a reason is typed	Posts the reversal (the lines turned round) and the record becomes Reversed.
7	Journal entry smart button	The JV number	Opens the entry.



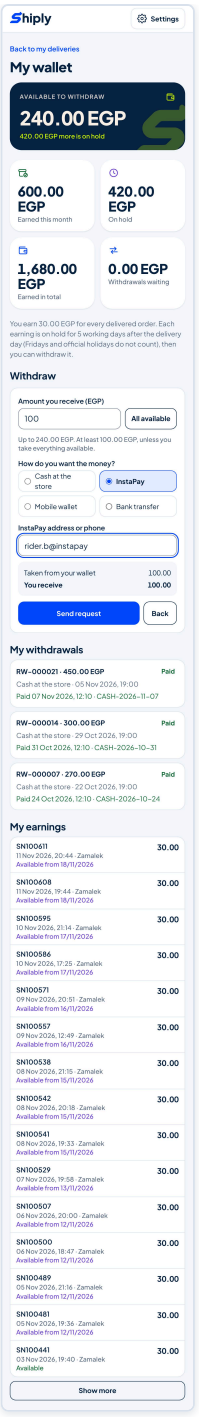
Reversed: red Reversed in the status bar, two smart buttons (Journal entry and Reversal entry), "Reversed by" and the reason.



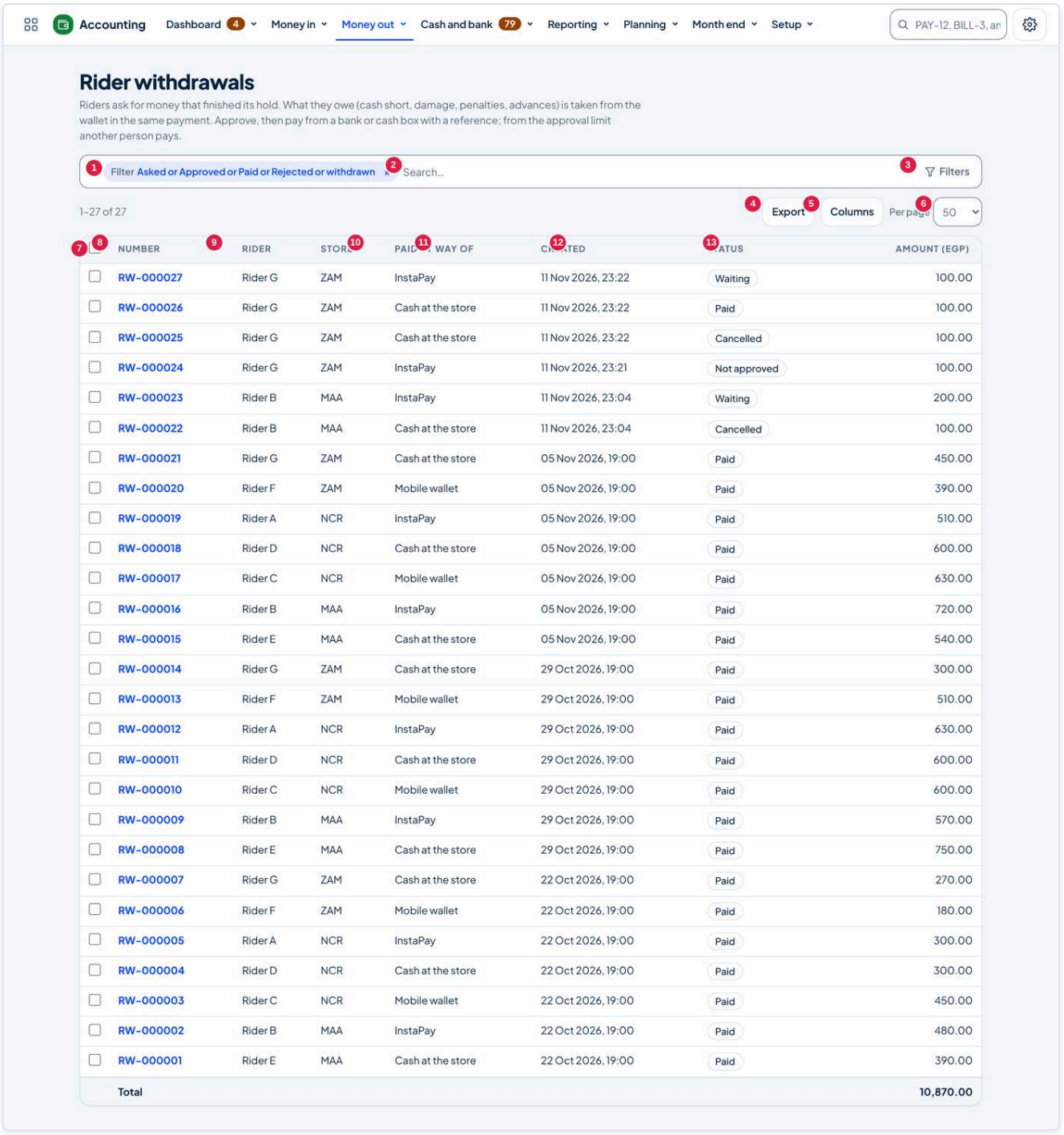
Rejected in Approvals: red Rejected, the decision note, and no journal entry. The smart button shows the request status word ("rejected"; a waiting record shows "pending").

14.7 Rider withdrawals

Address: `/finance/rider-withdrawals` ; one request at `/finance/rider-withdrawals/<id>` ; the rider side at `/rider/wallet` . **Menu:** Accounting > Money out > Rider withdrawals. **Who:** Accounting `use` approves, rejects and pays; never the rider. From the approval limit the payer must not be the approver (`same_person`). A store manager may pay an approved cash request at the store from the store box. See the chapter Cash and riders, rider pay and withdrawals.



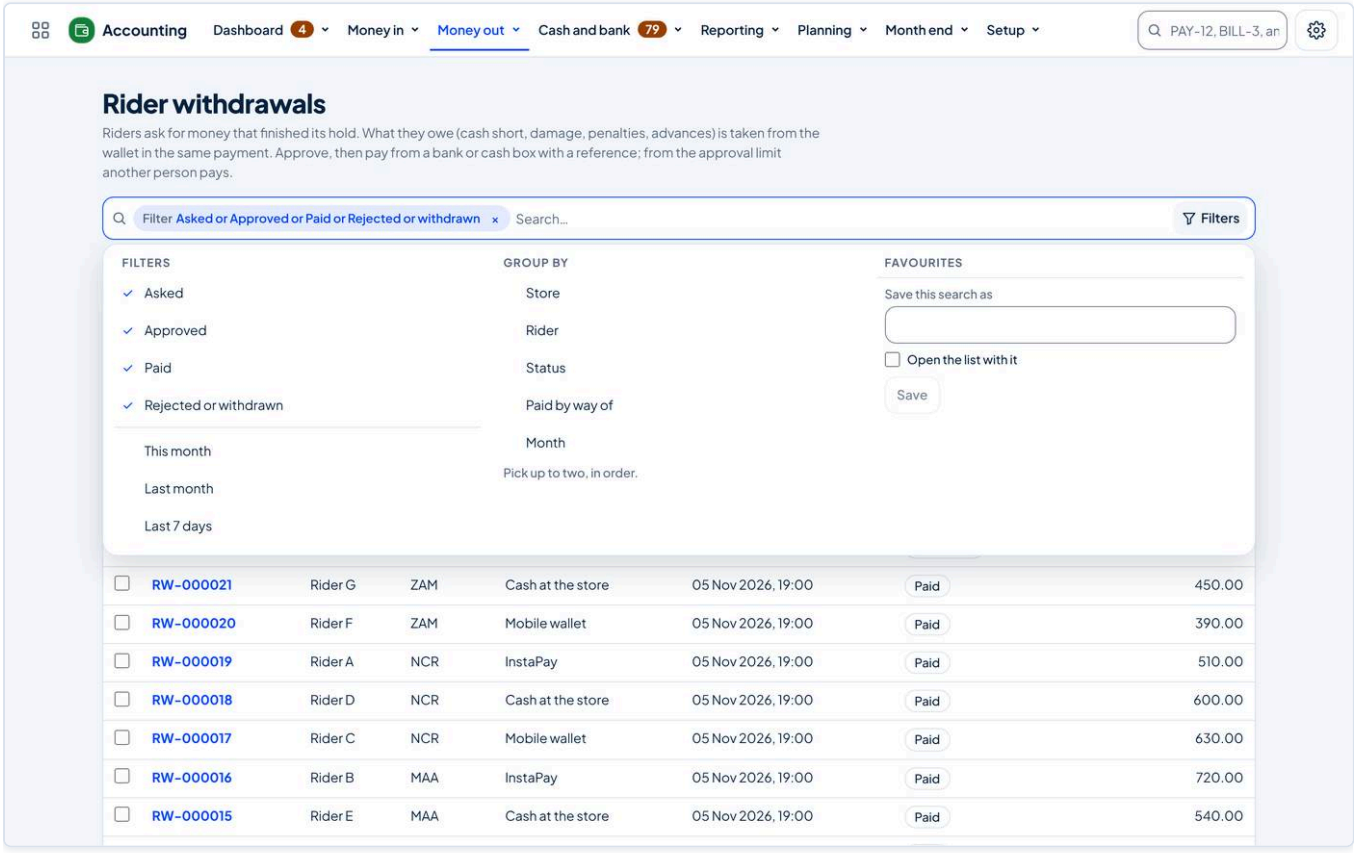
The rider asks on the phone: amount (with "All available"), the way to be paid (Cash at the store, InstaPay, Mobile wallet, Bank transfer), the InstaPay address, and "Taken from your wallet / You receive" before Send request.



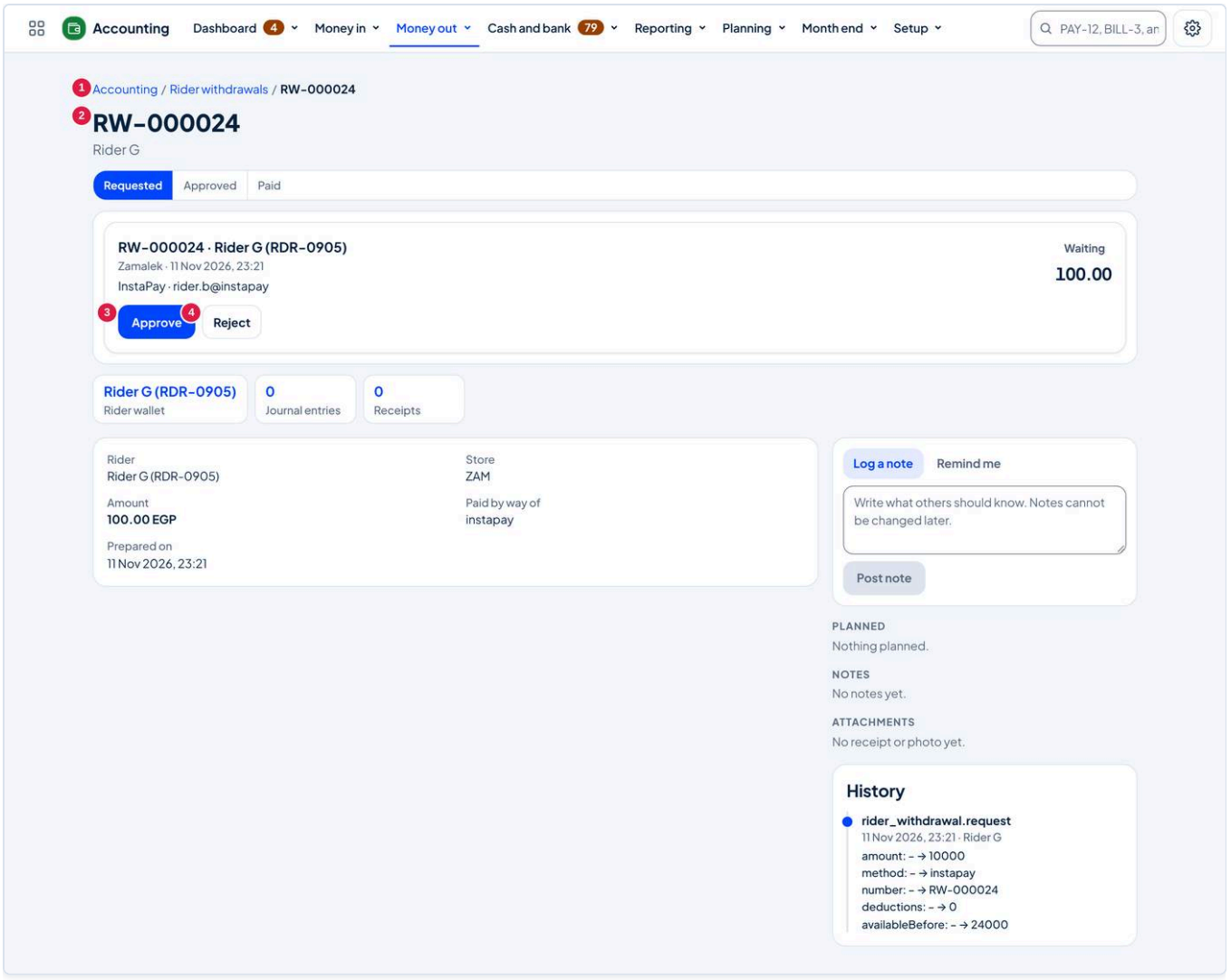
Rider withdrawals with every status shown (/finance/rider-withdrawals; the default filter is Asked or Approved).

#	Control	What it does	Rule / limits
1	Chip	Default "Asked or Approved"; here every status	
2	Search	Number, rider, badge	
3	Filters	Asked, Approved, Paid, Rejected or cancelled; dates. Group by Store, Rider, Status, Way of payment, Month	
4 to 6	Export, Columns, Per page	Optional columns Badge and Deducted (what the rider owed, taken first)	
7	Select all	No bulk action	

#	Control	What it does	Rule / limits
8 to 12	Number (`RW-000001` , opens the request), Rider, Store, Paid by way of, Created	Sortable number, rider, created and amount	
13	Status	Pill: Waiting, Approved, Paid, Not approved, Cancelled	
	Amount (EGP)	What the rider receives; summed	

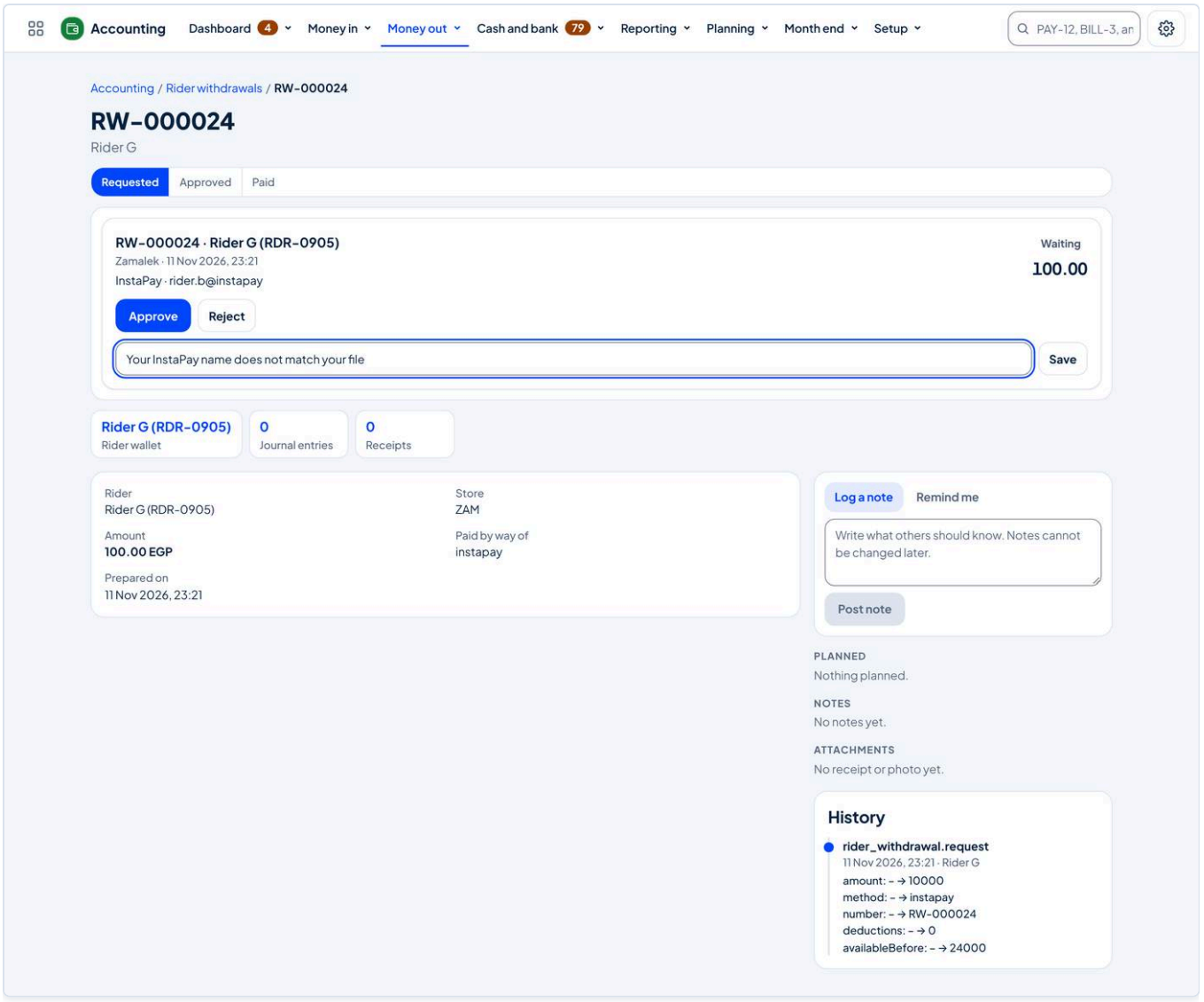


The Filters panel of Rider withdrawals.

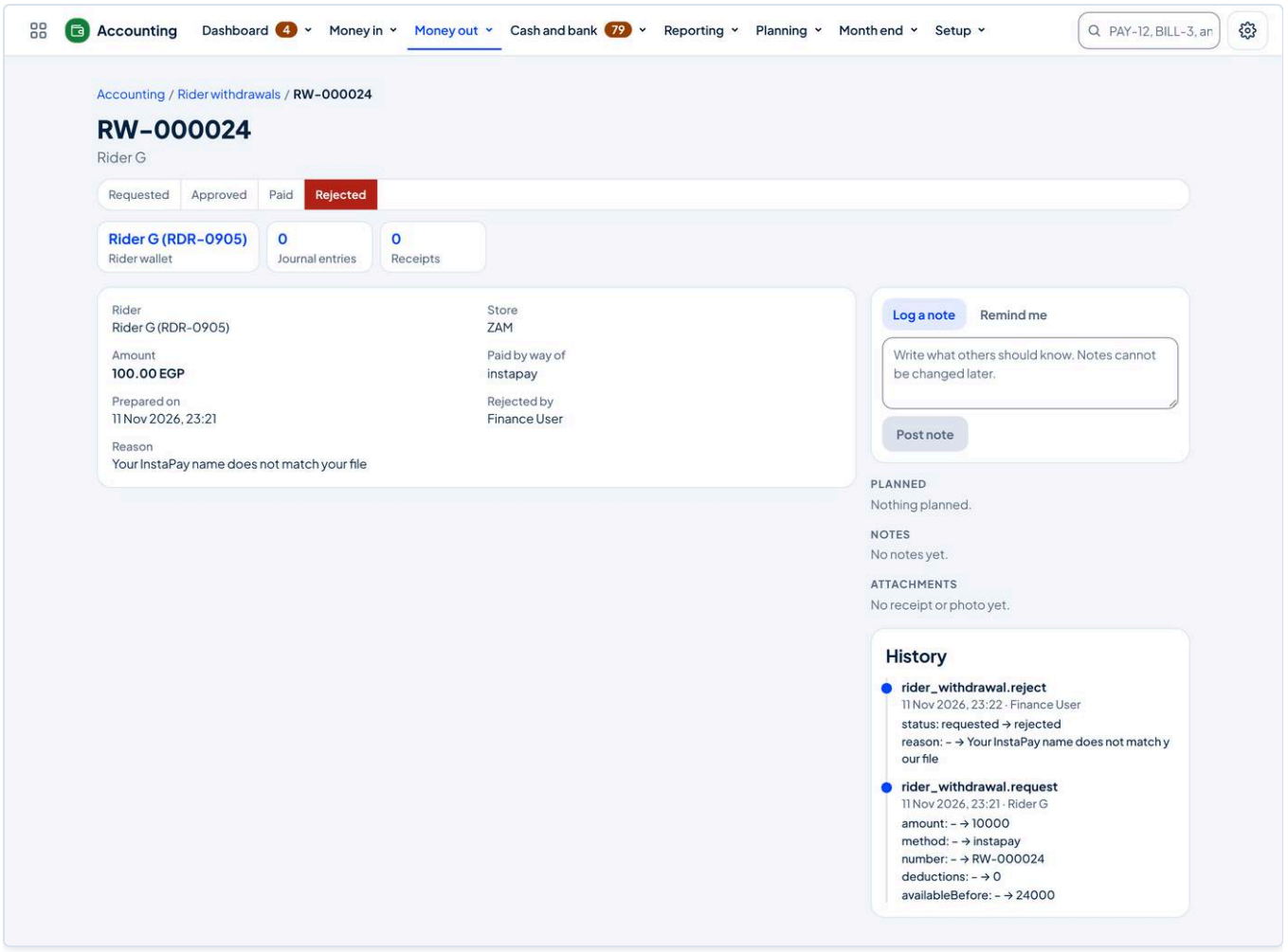


A request waiting (status bar Requested).

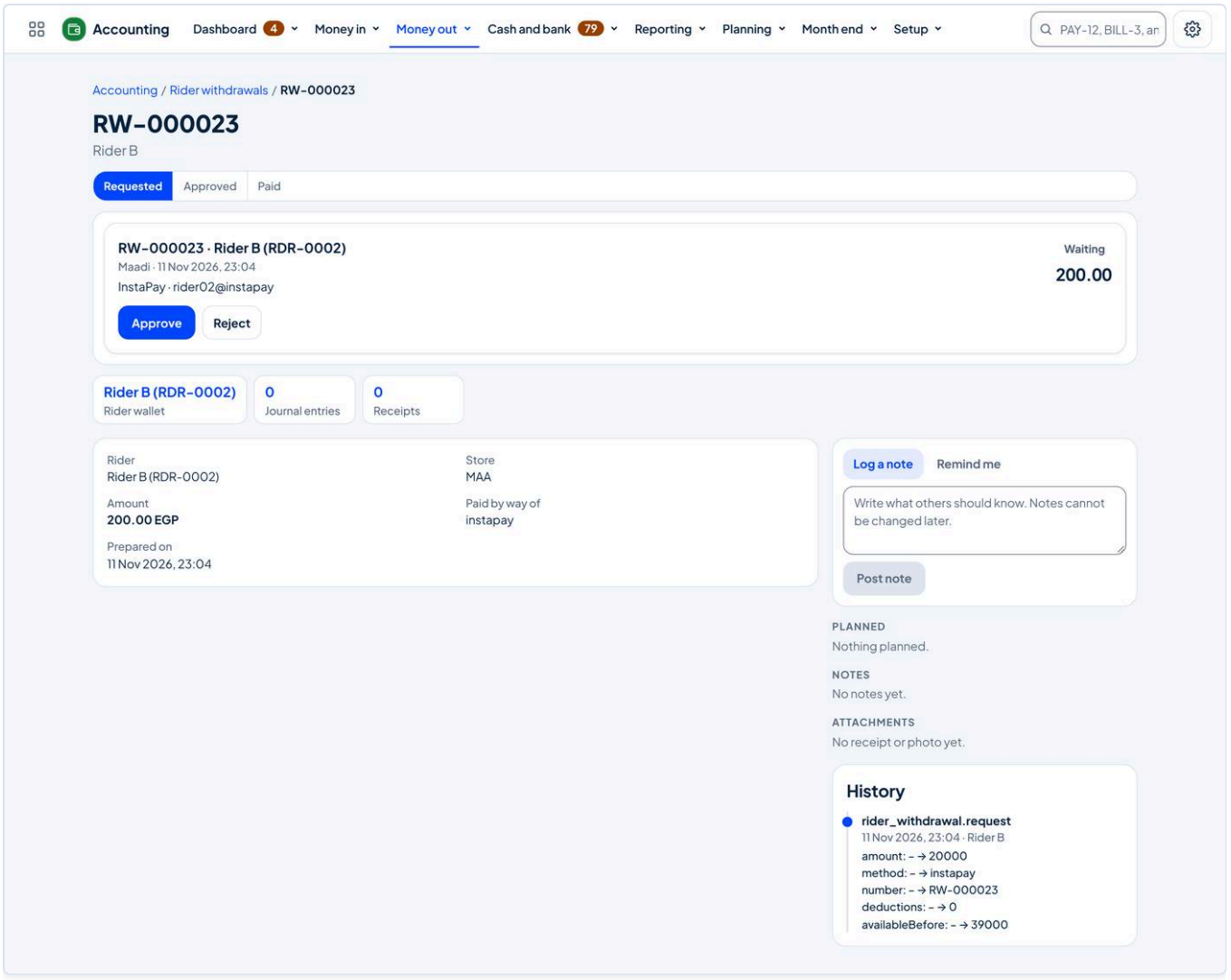
#	Part	What it shows	What clicking it does
1, 2	Breadcrumbs, number	RW number, rider label in grey	
	Card	RW number, rider and badge, store, time, way of payment and address; status word and amount on the right; deductions line when the rider owed something	
3	Approve	Blue button	"Approved." The request moves to Approved; the amount stays set aside.
4	Reject	White button	Opens "Why (the rider sees this)" and Save; at least 3 characters. The amount is freed at once and the rider sees "Not approved: *reason*".
	Smart buttons	Rider wallet (badge), Journal entries, Receipts	Rider wallet opens the rider's wallet account lines.



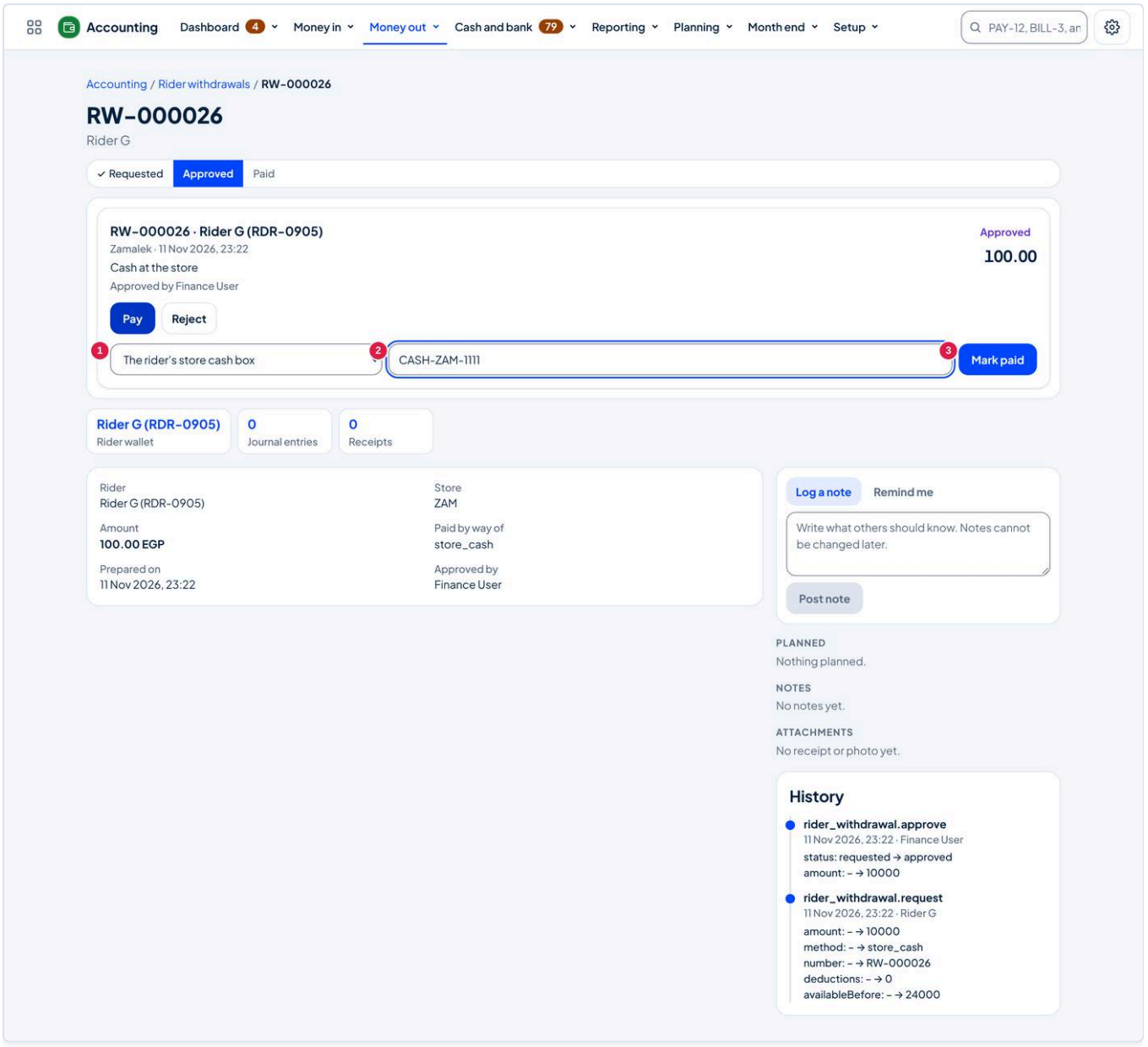
Reject opened with the reason the rider will see.



Rejected: "Not approved" in red at the end of the status bar.

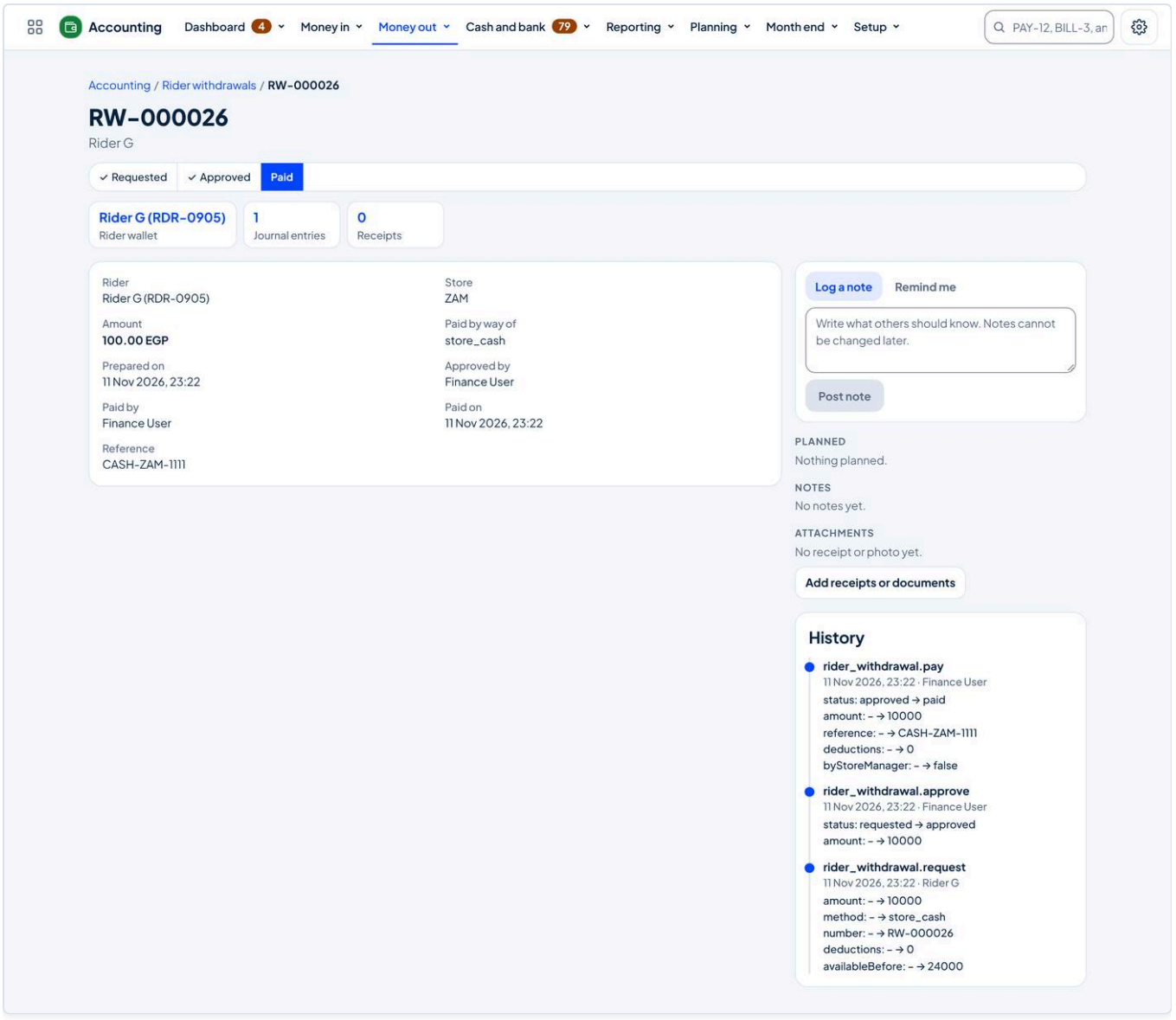


Cancelled by the rider before approval.



Approved, with Pay opened.

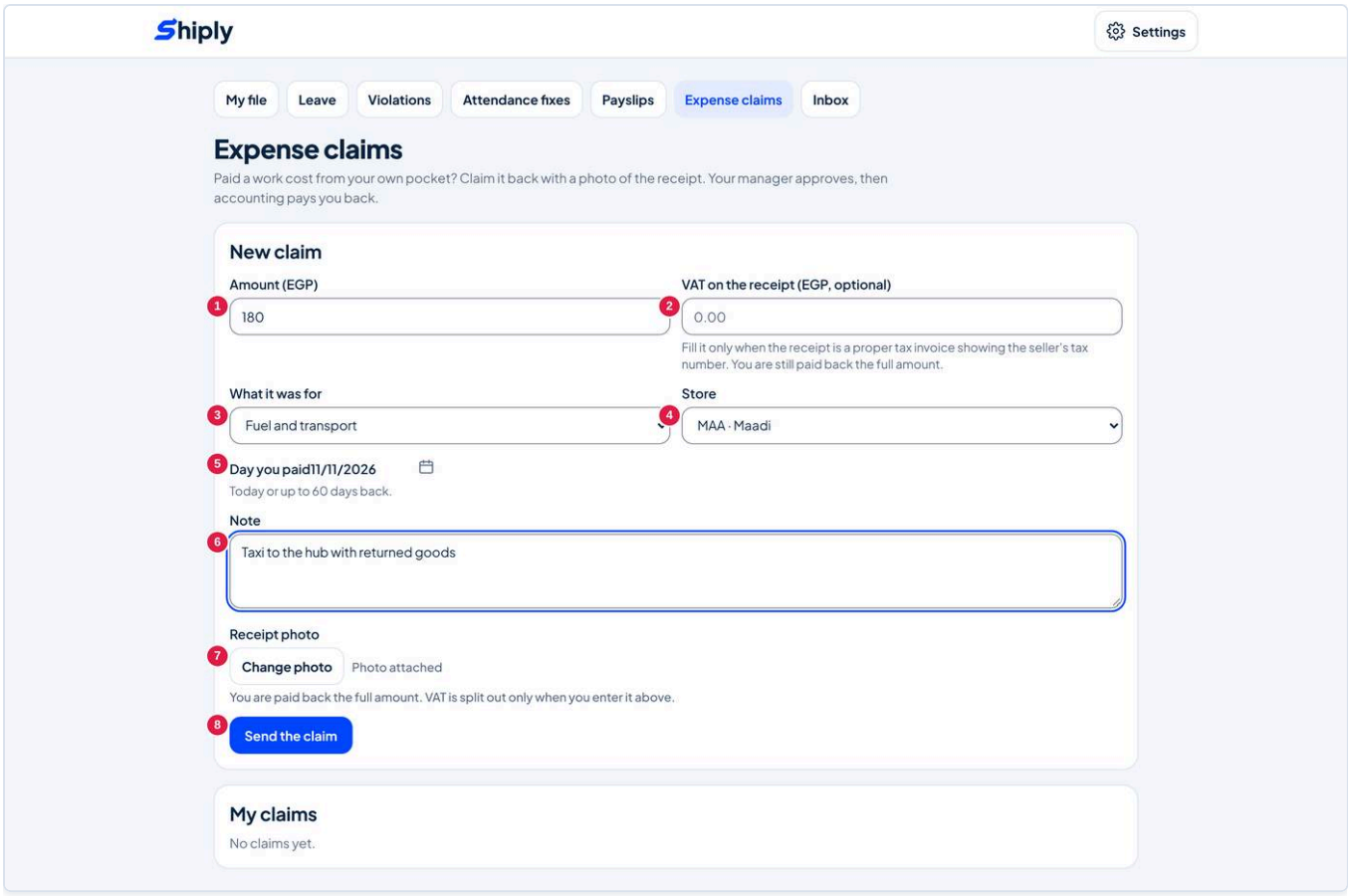
#	Control	What it does	Rule / limits
1	Pay from	Default "The rider's store cash box" for cash at the store, "The main bank (CIB)" for the others; any open bank or box may be chosen	Must hold the amount
2	Reference	Bank or cash reference	At least 2 characters
3	Mark paid	Posts Dr 2200 rider wallet (amount plus deductions), Cr 1350 dues, Cr 1500 advance, Cr the bank or box	"Paid." From the approval limit the approver cannot pay ("You approved this, so another person must pay it.")



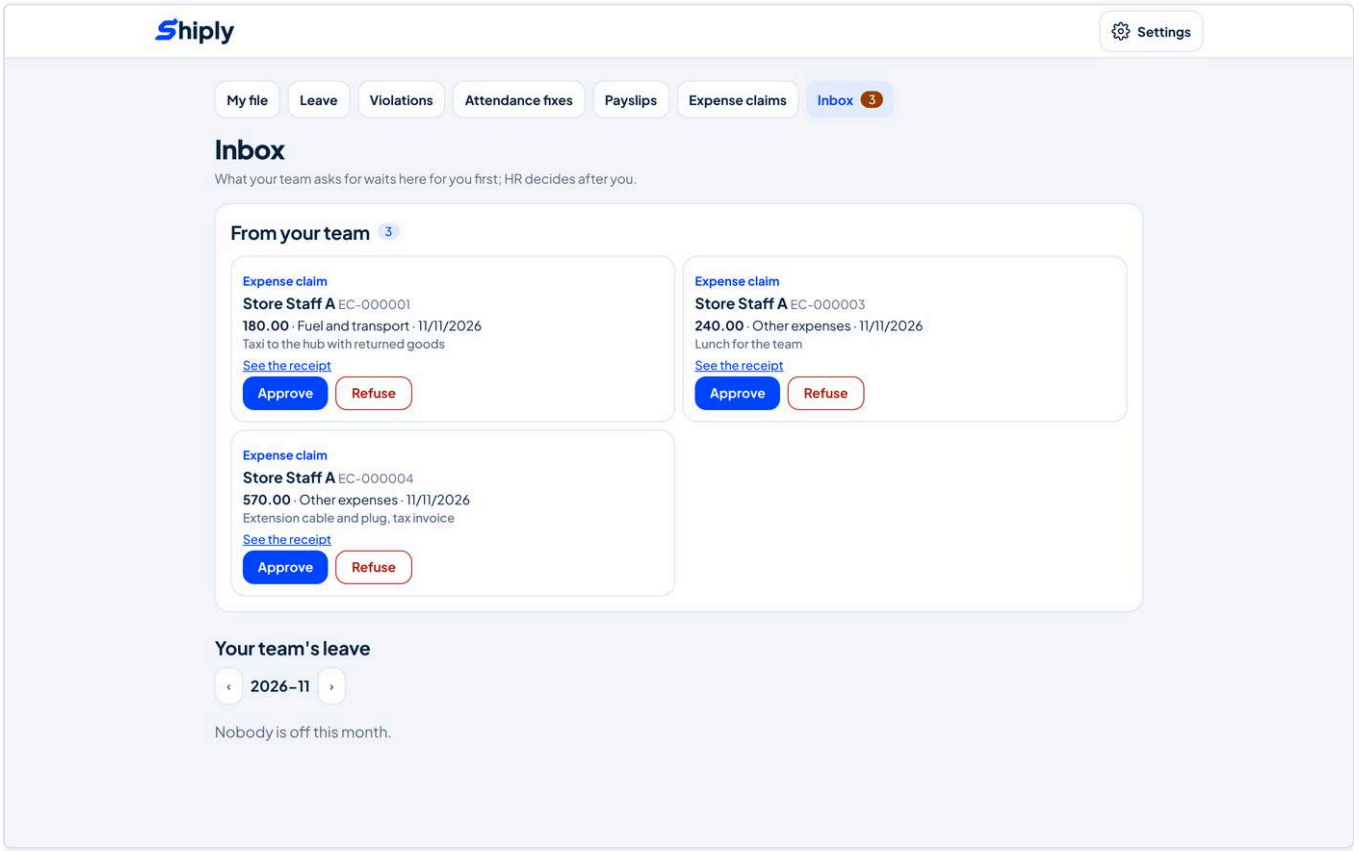
Paid: Paid in blue, one journal entry, "Paid by", "Paid on" and the reference. "Paid by way of" shows the method code (store_cash).

14.8 Expense claims

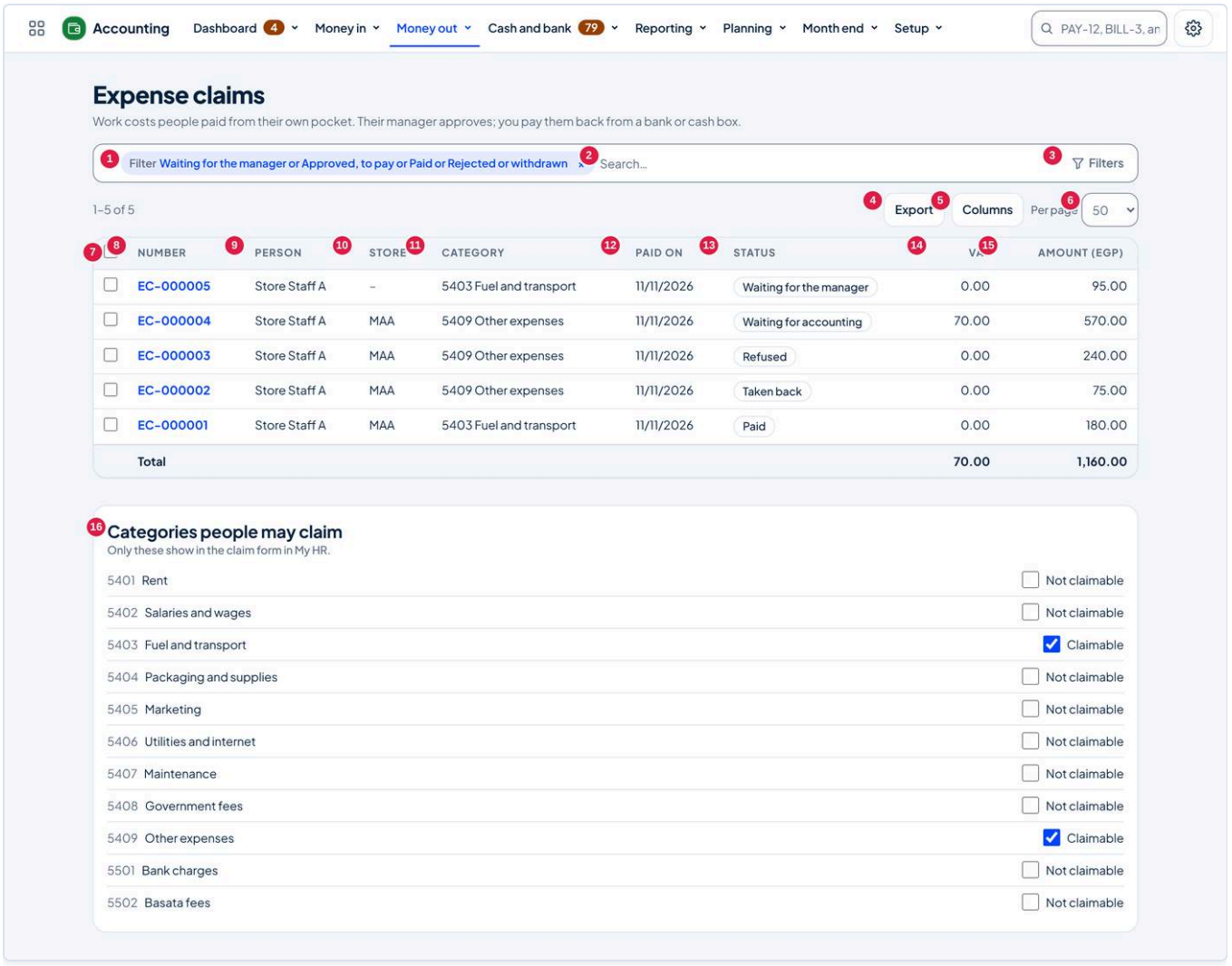
Address: `/finance/expense-claims`; one claim at `/finance/expense-claims/<id>`; the employee side at `/my-file/claims`; the manager decides in `/my-file/inbox`. **Menu:** Accounting > Money out > Expense claims. **Who:** any employee claims; the manager in force approves first (none: straight to accounting); Accounting `use` pays or refuses; choosing which categories may be claimed needs Accounting `manage`. Nobody decides their own claim; from the approval limit the payer is not the approving manager. See the chapter Expenses, suppliers, bills and the assets we pay for, expense claims.



The employee's claim form in My HR: amount, VAT on the receipt (optional), what it was for, store, day paid (today or up to 60 days back), note and receipt photo.



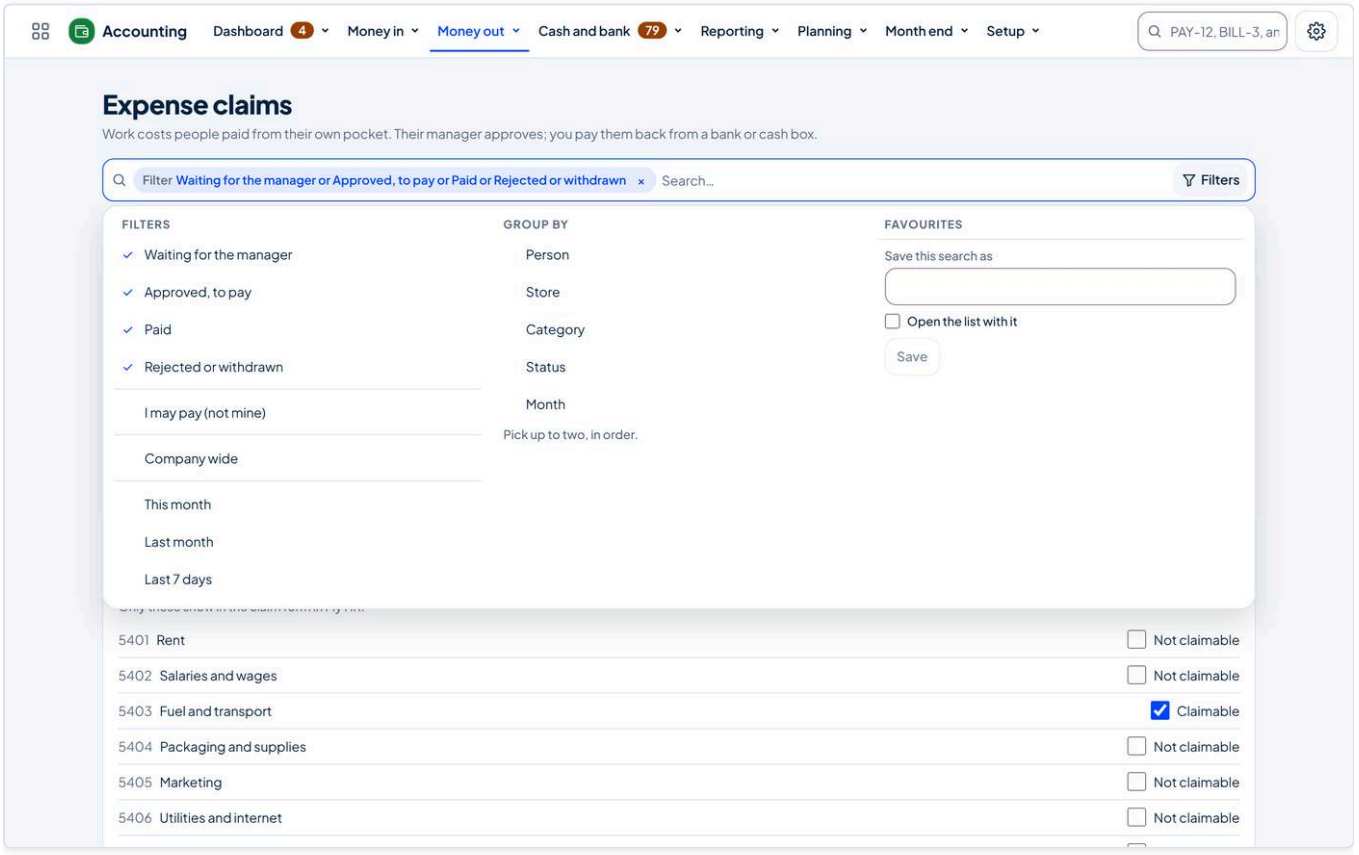
The manager's Inbox: one card per claim with "See the receipt", Approve and Refuse.



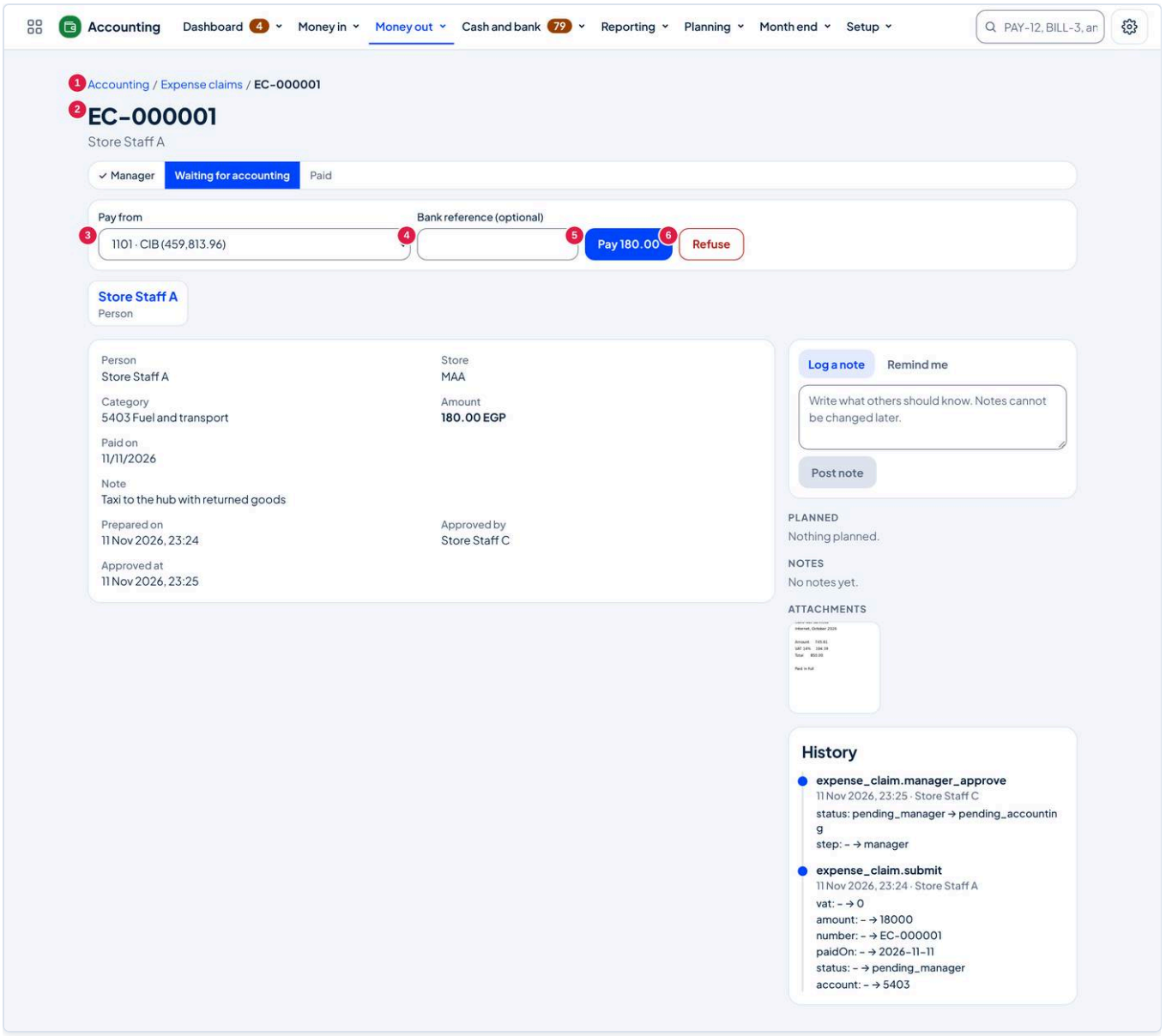
Expense claims with every status (/finance/expense-claims; the default filter is "Approved, to pay").

#	Control	What it does	Rule / limits
1	Chip	Default "Approved, to pay"	
2, 3	Search, Filters	Filters: Waiting for the manager, Approved to pay, Paid, Rejected or withdrawn; "I may pay (not mine)"; Company wide; dates. Group by Person, Store, Category, Status, Month	
4 to 6	Export, Columns, Per page	Optional columns Note, Created, Approved by, Paid by	
7	Select all	For a manager, the bulk action "Approve claims" approves the manager step of people they manage	Never one's own
8 to 12	Number (`EC-000001`), Person, Store, Category, Paid on		
13	Status	Pill: Waiting for the manager, Waiting for accounting, Paid, Refused, Taken back	
14, 15	VAT, Amount (EGP)	Both summed	

#	Control	What it does	Rule / limits
16	Categories people may claim	Tick box per 5400 and 5500 account: Claimable or Not claimable	Accounting `manage` ; only claimable ones show in the claim form



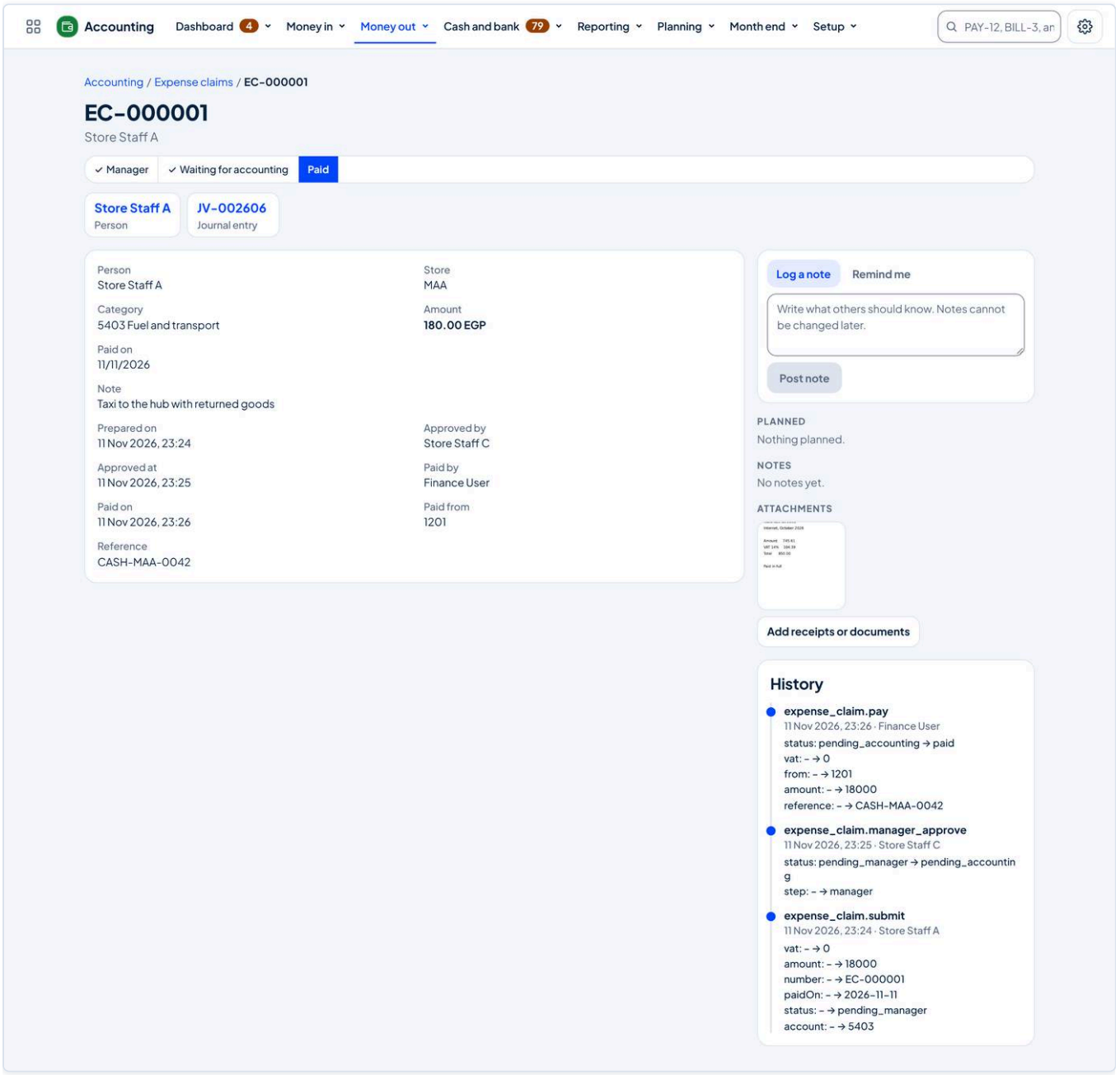
The Filters panel of Expense claims.



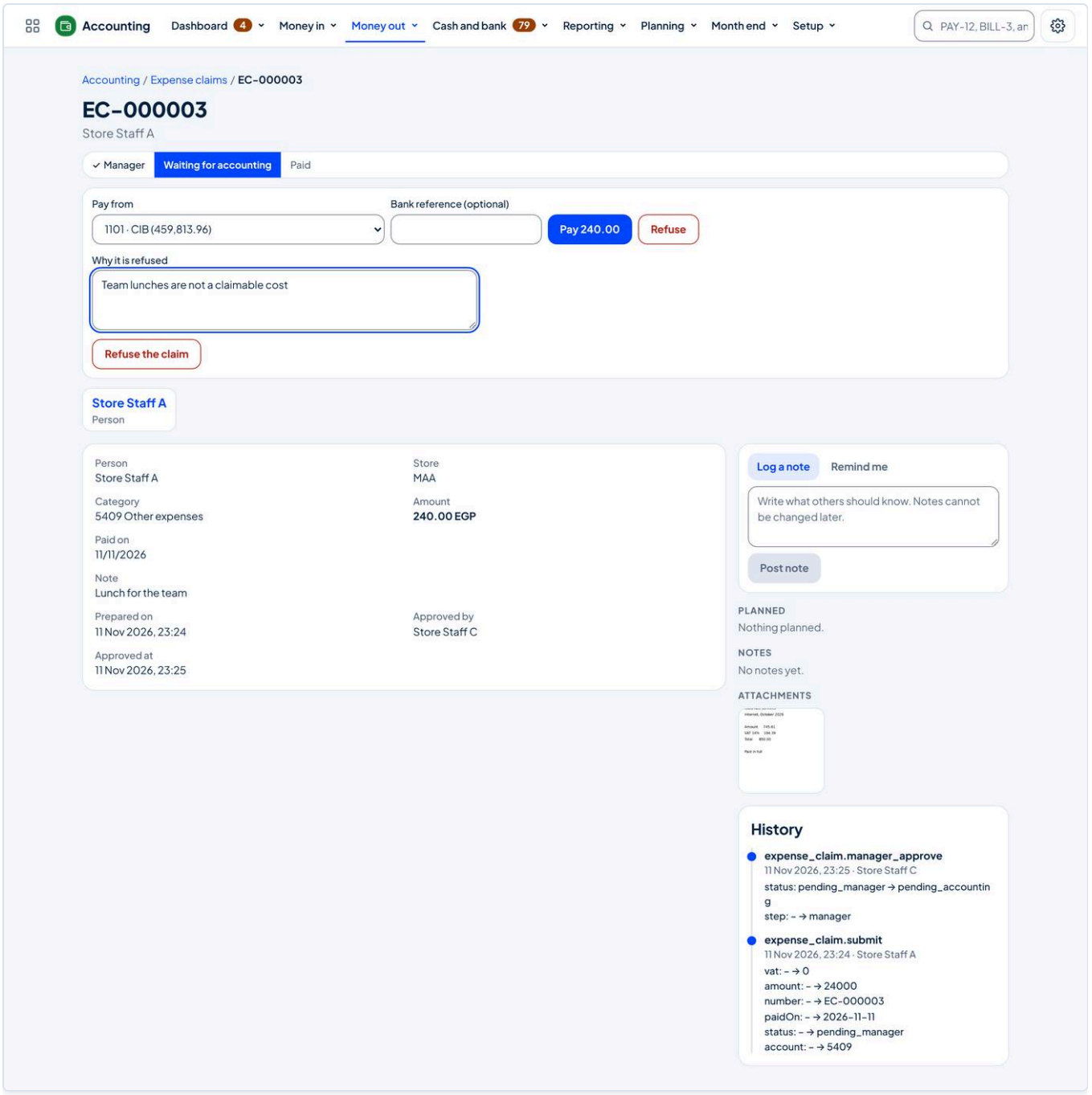
A claim approved by the manager and waiting for accounting.

#	Part	What it shows	What clicking it does
1, 2	Breadcrumbs, number	EC number and person	
	Status bar	Manager, Waiting for accounting, Paid; Rejected or Cancelled in red	
3	Pay from	Open banks and boxes with balances	Must hold the amount.
4	Bank reference (optional)		
5	Pay 180.00	Blue button with the amount	Posts Dr the category (and Dr 2301 for VAT on a tax invoice), Cr the bank or box, store and person tags. The person is paid the full amount.
6	Refuse	White button with red border	Opens "Why it is refused" and "Refuse the claim" (at least 3 characters).

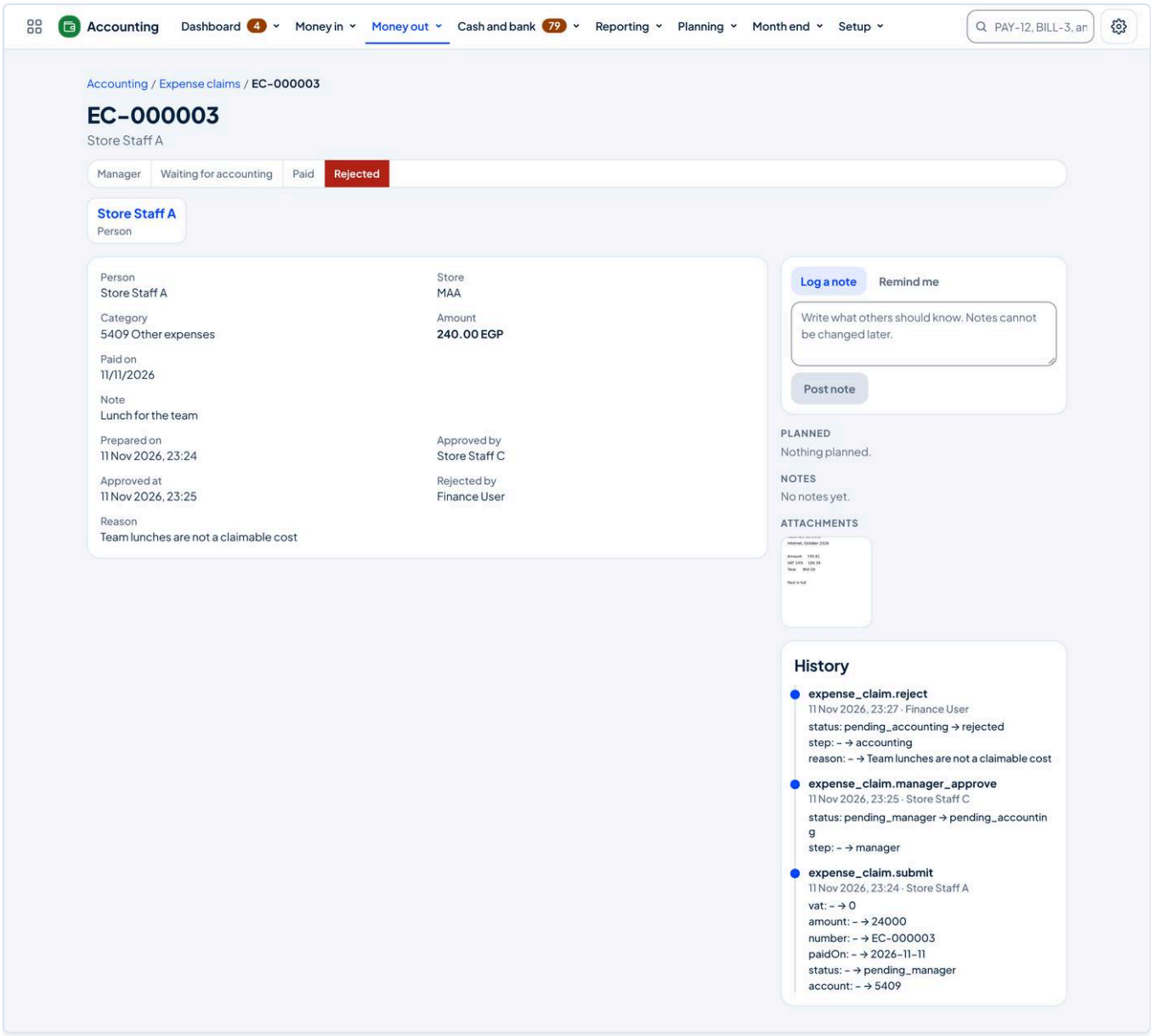
#	Part	What it shows	What clicking it does
	Smart buttons	Person (opens the list searched on them), Journal entry once paid	



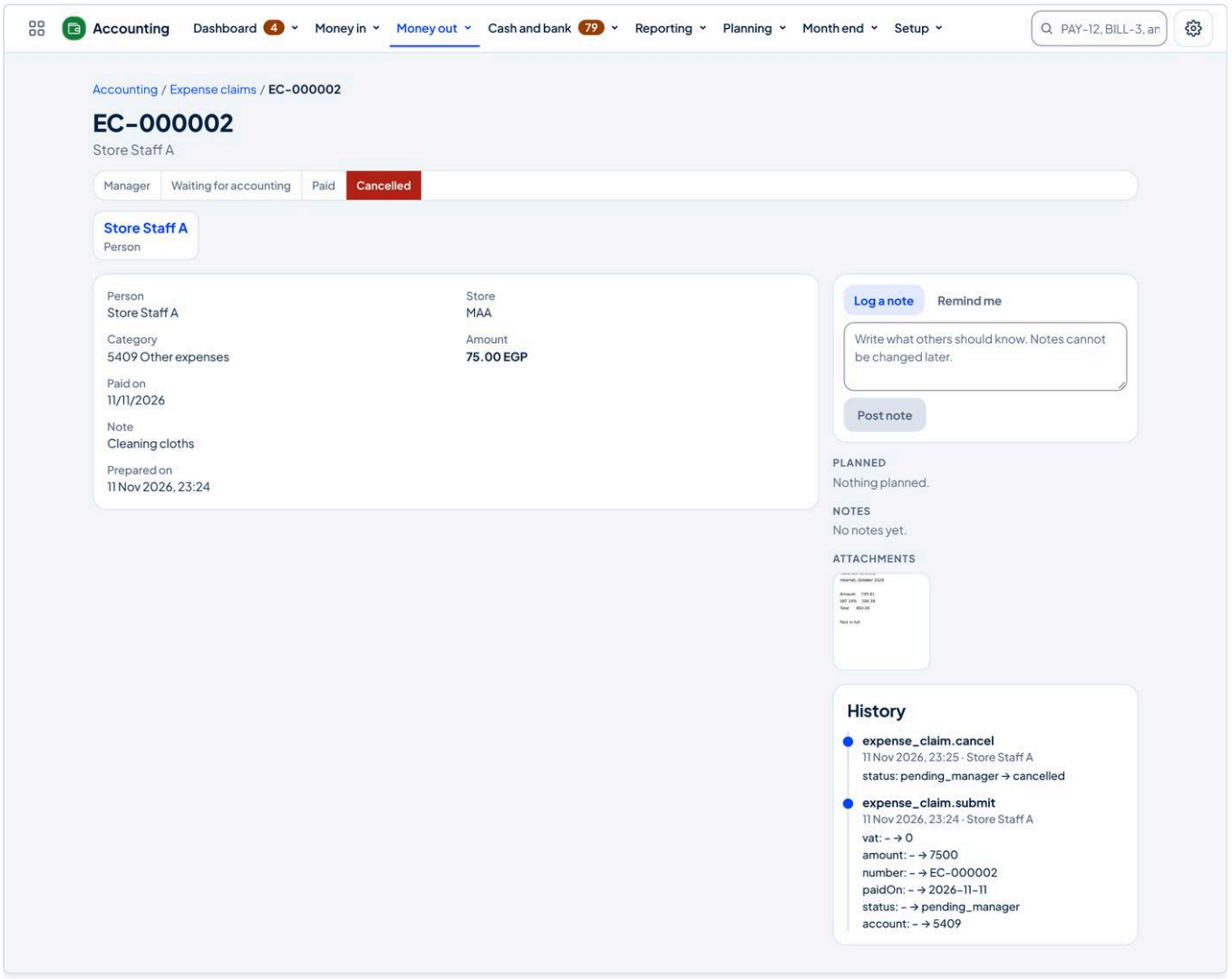
Paid from the Maadi box: Paid in blue, the journal entry, "Paid from 1201" and the reference.



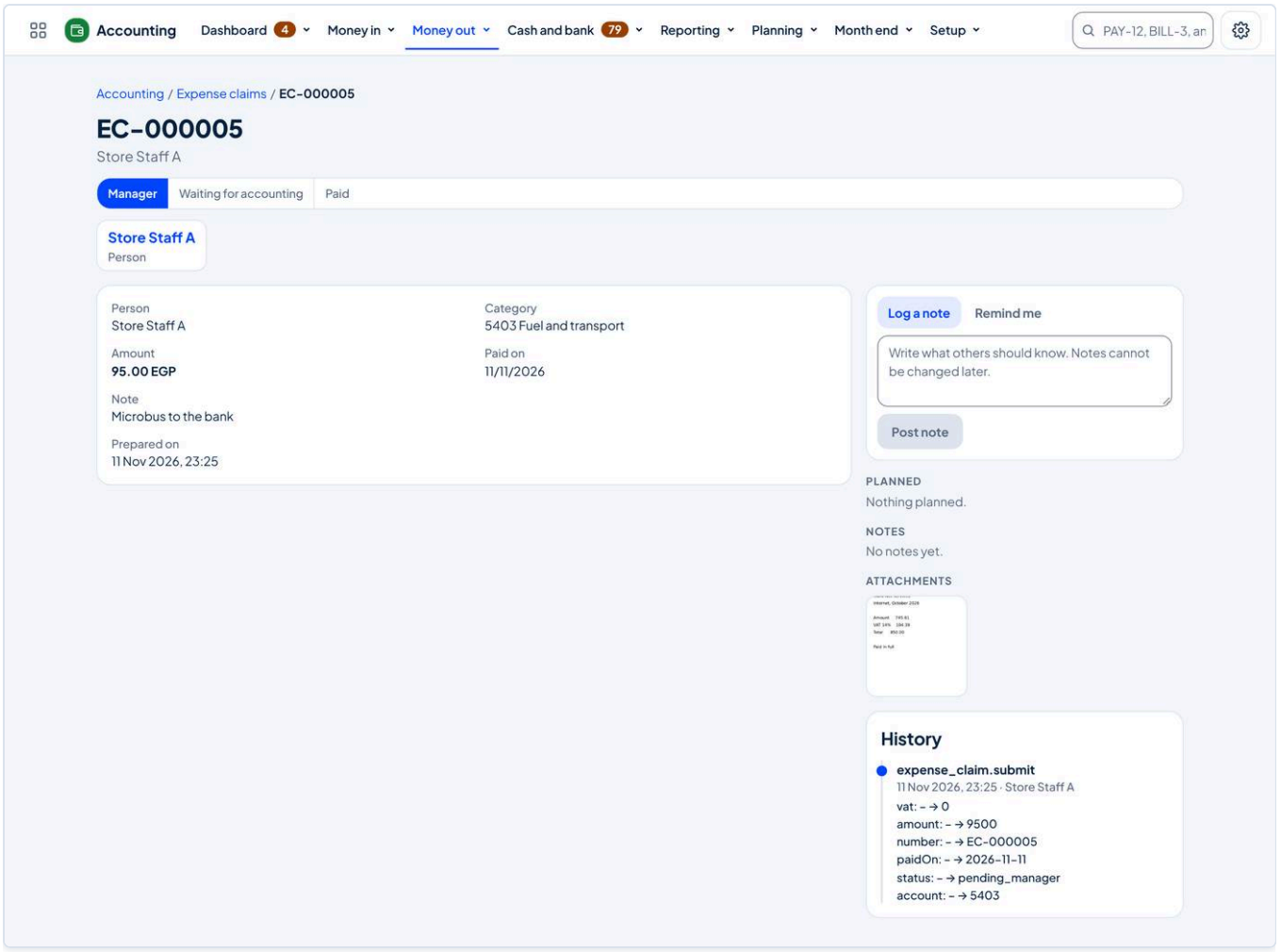
Refuse opened with its reason.



Refused: Rejected in red, "Rejected by" and the reason.



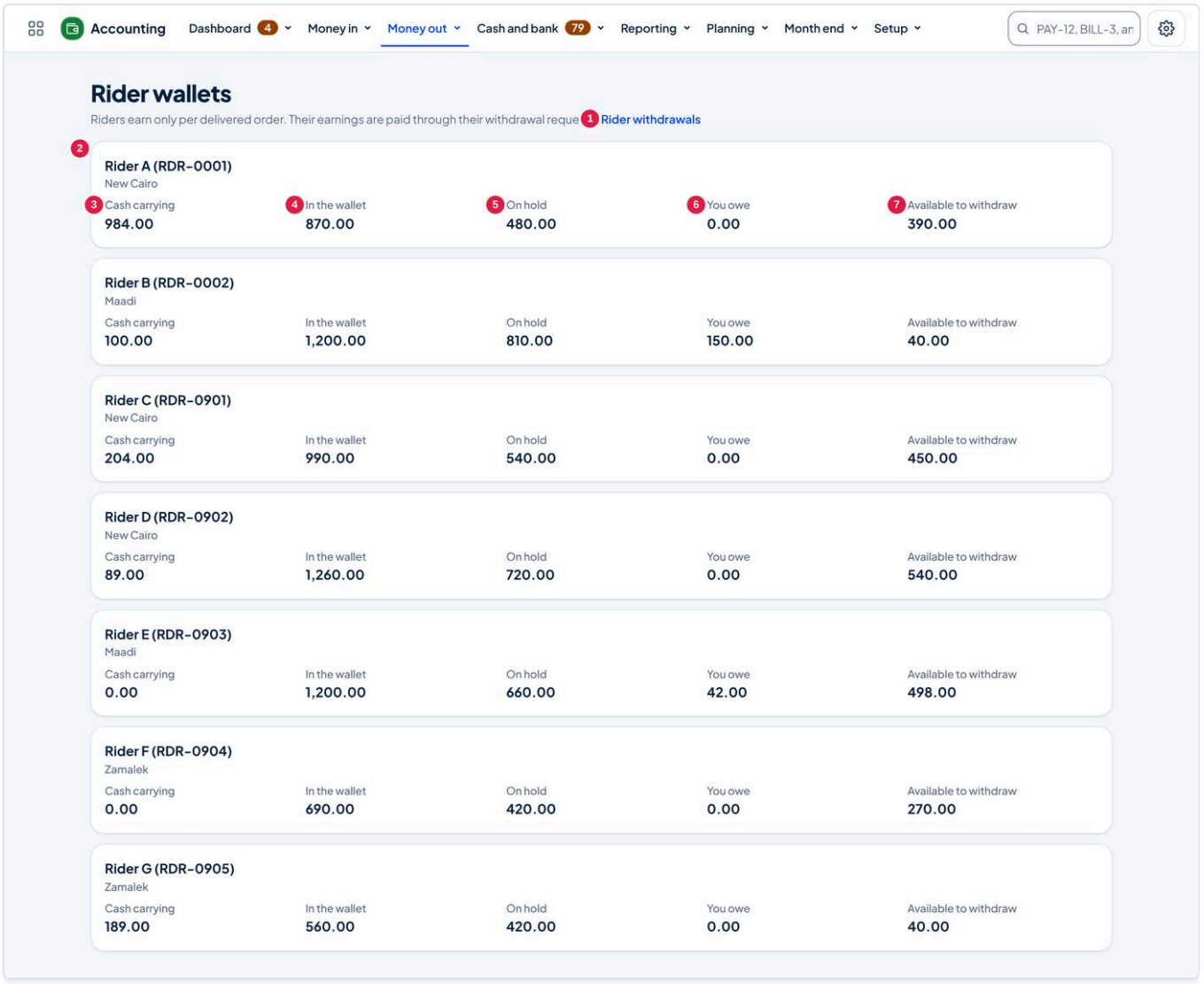
Taken back by the employee before any decision: Cancelled in red.



Still with the manager: no buttons for accounting yet.

14.9 Riders (rider wallets)

Address: `/finance/riders`. **Menu:** Accounting > Money out > Riders (page title Rider wallets). **Who:** Accounting `view`; read only. One card per rider in the person's stores.

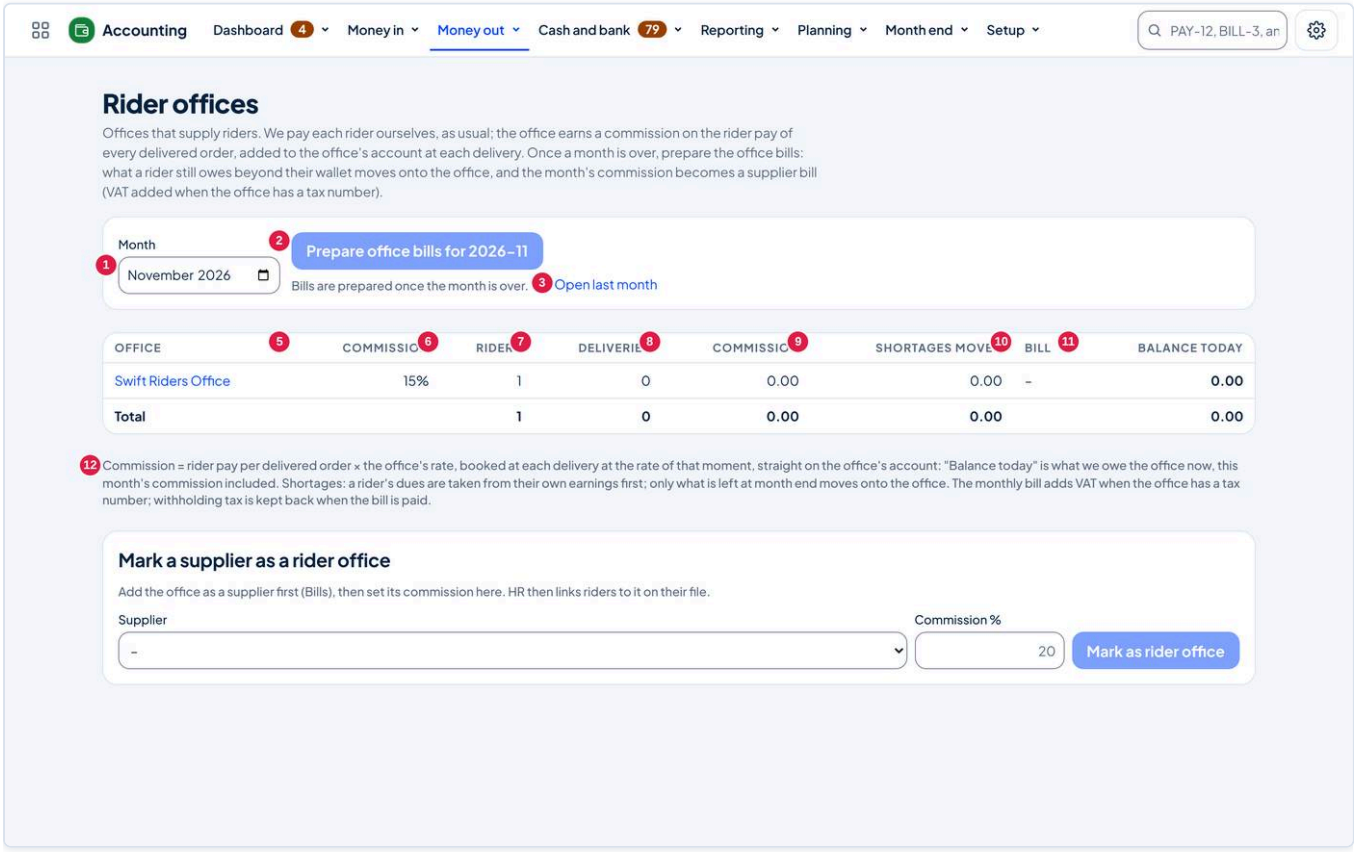


Rider wallets (/finance/riders).

#	Part	What it shows	Rule
1	Rider withdrawals	Blue link in the intro	Opens Rider withdrawals.
2	Card title	Rider, badge, store	
3	Cash carrying	Cash on delivery in the rider's 1300 account	Handed in at the store or deposited at Basata.
4	In the wallet	Balance of the 2200 rider pay wallet	Rider pay per delivered order, less what was paid out.
5	On hold	Earnings still in their hold	`rider.withdraw_hold_working_days` working days after the delivery day.
6	You owe	Dues (1350) plus any staff advance	Taken first from the next withdrawal.
7	Available to withdraw	Matured earnings less paid out, owed and open withdrawals	Always summed from the ledger, never stored.

14.10 Rider offices

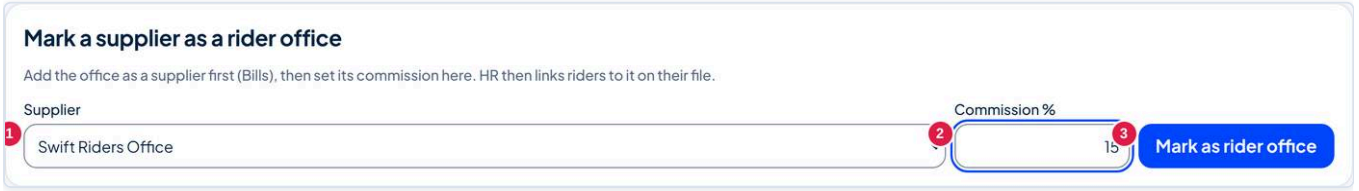
Address: `/finance/rider-offices` (`?month=YYYY-MM`); one office at `/finance/rider-offices/<id>` . **Menu:** Accounting > Money out > Rider offices. **Who:** Accounting `view` reads; `use` prepares the month end bills; `manage` marks a supplier as an office and changes its rate. HR links riders to an office on their file. See the chapter Cash and riders, rider supply offices.



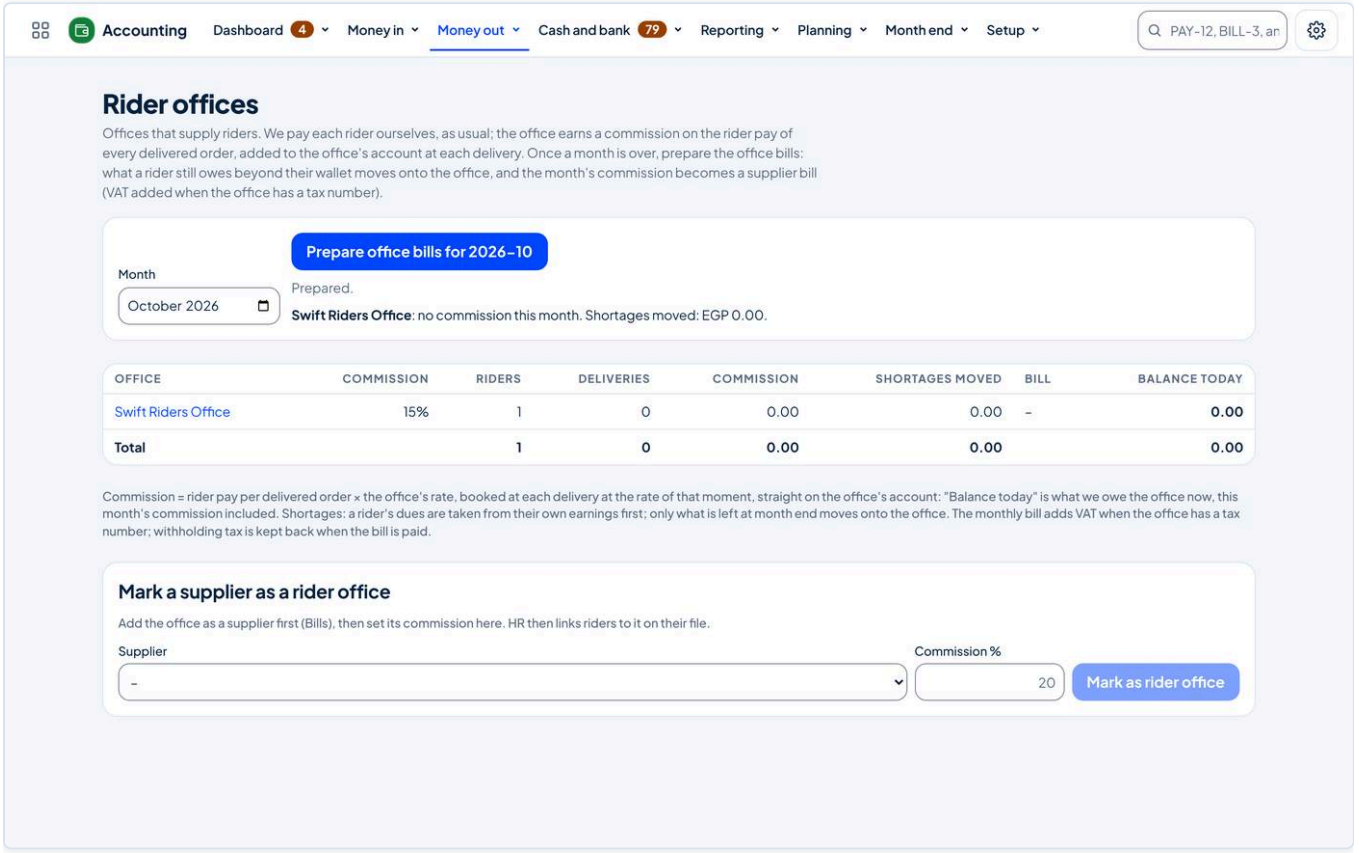
Rider offices with one office and one linked rider (/finance/rider-offices).

#	Control	What it does	Rule / limits	Result
1	Month	Month picker		The table shows that month.
2	Prepare office bills for *month*	Blue button, light while the month is not over ("Bills are prepared once the month is over.")	Accounting `use` ; months in order	For each office: moves what linked riders still owe beyond their wallet onto the office, then makes one bill from the month's commission (VAT only with a tax number). The result lists each office, e.g. "no commission this month. Shortages moved: EGP 0.00."
3	Open last month	Blue link		Sets `?month=` to last month.
	Office	Name (link); "No tax number: no VAT" in grey when it has none		Opens the office page.

#	Control	What it does	Rule / limits	Result
5 to 11	Commission, Riders, Deliveries, Commission, Shortages moved, Bill, Balance today	Rate, linked riders, the month's deliveries and commission, shortages moved, the bill number or a dash, what we owe now; Total row		
12	How it works	Grey paragraph on commission and shortages		



Mark a supplier as a rider office: Supplier, Commission % (default 20) and "Mark as rider office" (Accounting manage).



After "Prepare office bills for 2026-10": "Prepared." and one line per office.

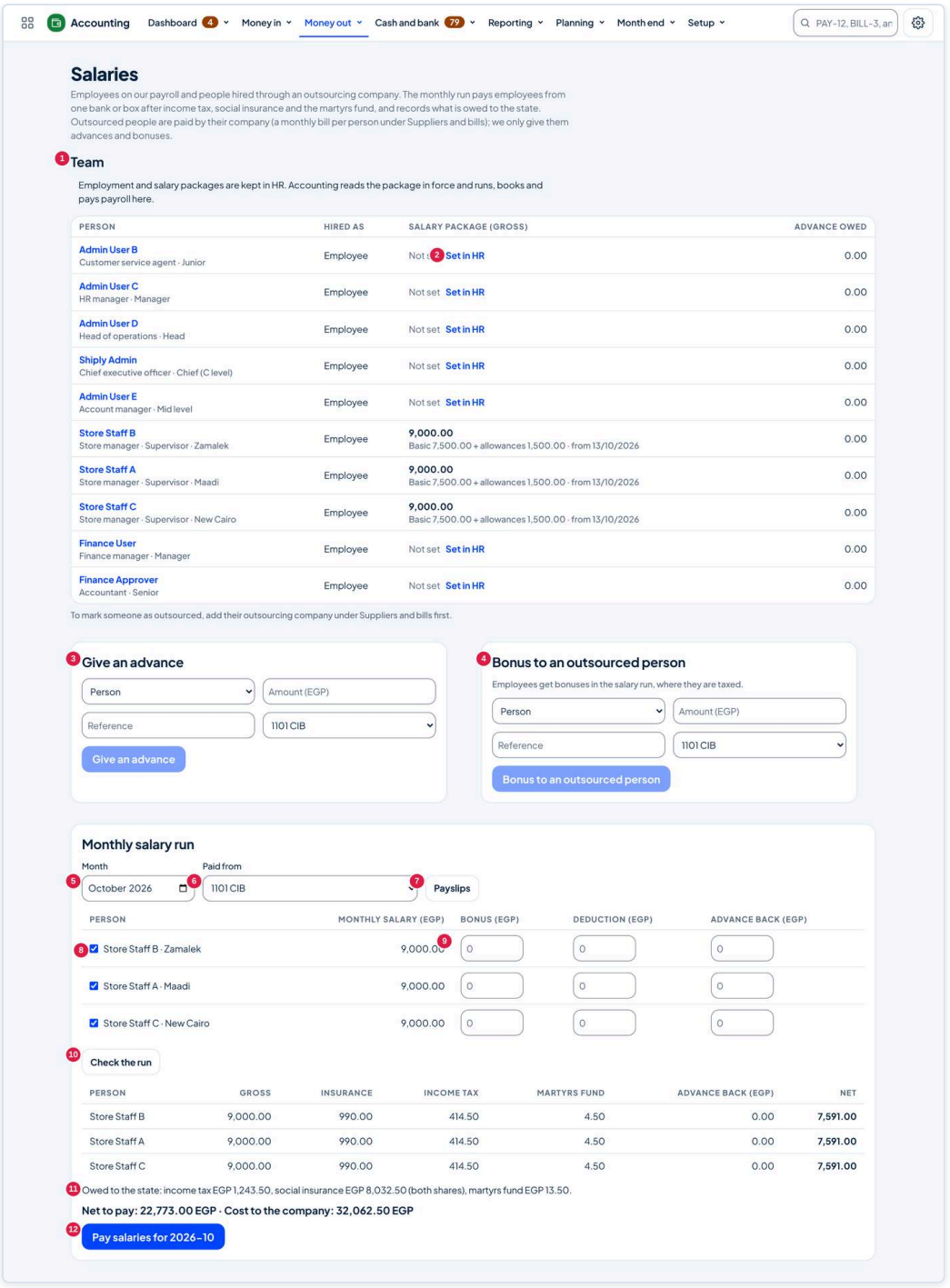
An office page with its daily account (/finance/rider-offices/<id>). The demo office was linked today, so no delivery has earned commission yet.

#	Part	What it shows	What clicking it does
1	Back to rider offices	Blue link	Back to the list.
2	Title	Office name; "Tax number 611223344 · Withholding 3%"	
3 to 6	Commission, Balance today, Commission not billed yet (in the balance), Open bills	White stat cards	

#	Part	What it shows	What clicking it does
7, 8	Commission %, Save rate	Rate box and button (Accounting `manage`); hint "A new rate counts for deliveries from now on; earlier deliveries keep the rate they were booked at."	Audited.
9	Daily account	"What we owe the office, day by day": balance before the month, commission, deliveries, balance today; a table Day, Deliveries, Commission, Shortages moved, Bills (VAT), Payments, We owe; "Nothing moved on the account this month." when empty	
10	Month	Month picker of the daily account	Reloads with `?month=`.
11	Download CSV	White button	The daily account as CSV.
12	Show days without movements	Tick box	Adds empty days.
13	Riders	Rider (link), store, deliveries and commission this month, rider wallet, rider owes	The rider opens their file.
14	Month by month	Last 12 months: deliveries, rider pay, commission, shortages moved, bill ("Month running", "Not prepared" or the bill)	
15	Prepare this bill	The oldest month not prepared, then the button	Prepares that month for this office.
16	Bills	Bill, month, net, VAT, bill total, left to pay, status; "No bills yet."	The bill opens its page, where it is paid with the office's shortages set off.

14.11 Salaries and payslips

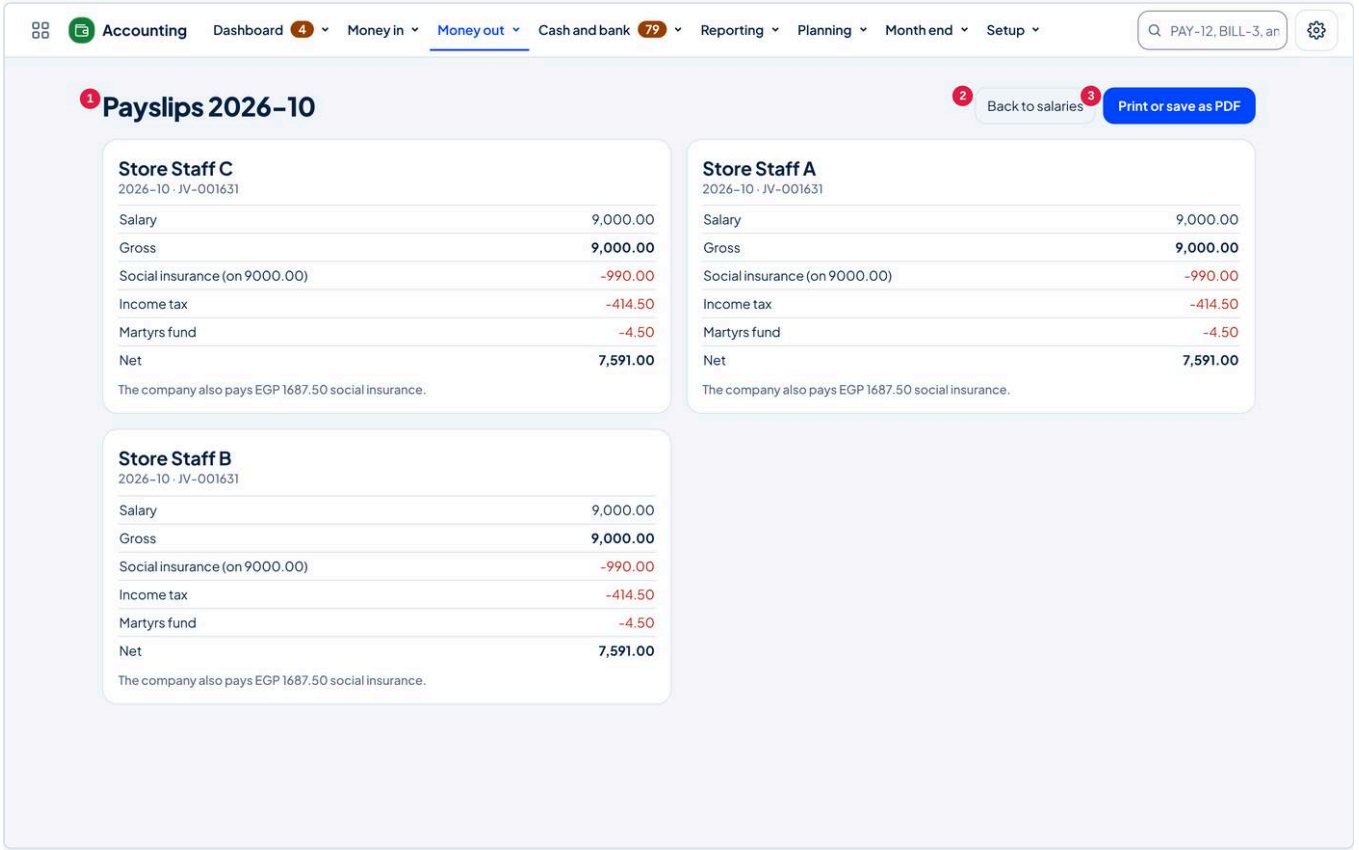
Address: `/finance/payroll` and `/finance/payroll/payslips?month=YYYY-MM`. **Menu:** Accounting > Money out > Payroll and Payslips, shown only with the salaries switch. **Who:** Accounting `view` with salaries to read; `use` with salaries to give advances, bonuses and run payroll. Packages are kept in HR. See the chapter Payroll and staff costs.



Salaries with the October run checked (/finance/payroll).

#	Part	What it shows	What it does
1	Team	Person (link to the HR file), Hired as (Employee or "Outsourced: *company*"), Salary package (gross with "Basic … + allowances … from *day*"), Advance owed	Per delivery riders are not listed.
2	Set in HR	"Not set" and a blue link	Opens the person's HR file, Pay tab.
3	Give an advance	Person, Amount, Reference, money account, button	Dr 1500 staff advance wallet, Cr the bank or box; from the limit it waits.

#	Part	What it shows	What it does
4	Bonus to an outsourced person	Same fields	Dr 5702 staff bonus; employees get bonuses in the run instead, where they are taxed.
5	Month	Month picker of the run	No month after the current one (`future_month`).
6	Paid from	Bank or box	
7	Payslips	White button	Opens the payslips of the month.
8	Include tick box	One per employee	Leaves a person out of this run.
9	Bonus, Deduction, Advance back	Amounts per person	
10	Check the run	White button	Preview: gross, insurance, income tax, martyrs fund, advance back, net per person (with commission and HR deductions noted).
11	Owed to the state, Net to pay, Cost to the company	Totals of the preview	
12	Pay salaries for *month*	Blue button	Books the run once per month (key `payroll:YYYY-MM`); waits for a second person from the limit.

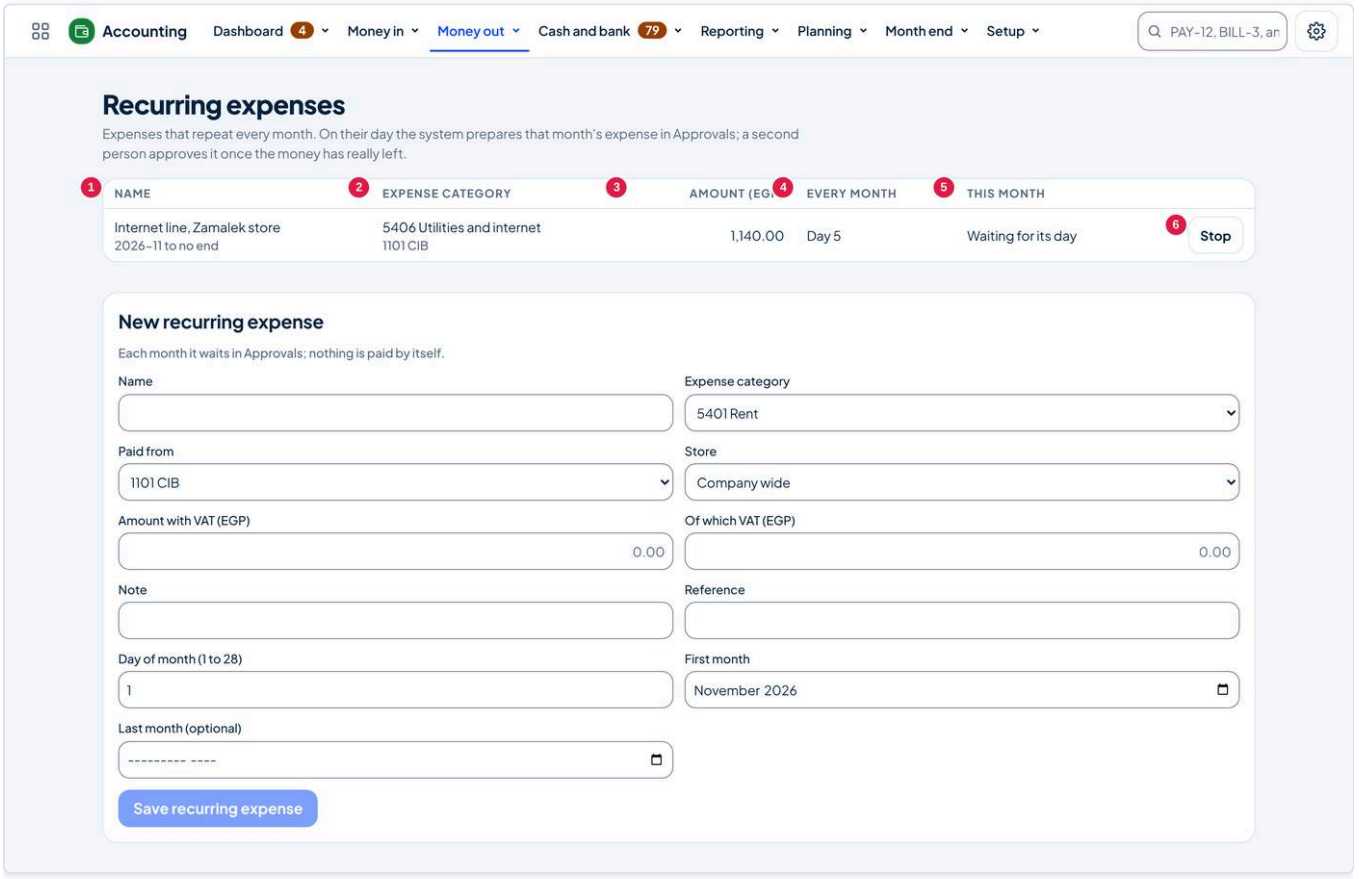


Payslips of a month: one card per employee with salary, gross, social insurance, income tax, martyrs fund (negatives in red) and net, the voucher number, and Print or save as PDF.

#	Part	What it does
1	Title "Payslips 2026-10"	The month from `?month=`
2	Back to salaries	Returns to Salaries.
3	Print or save as PDF	Prints every card; the menu is hidden in print.

14.12 Recurring expenses

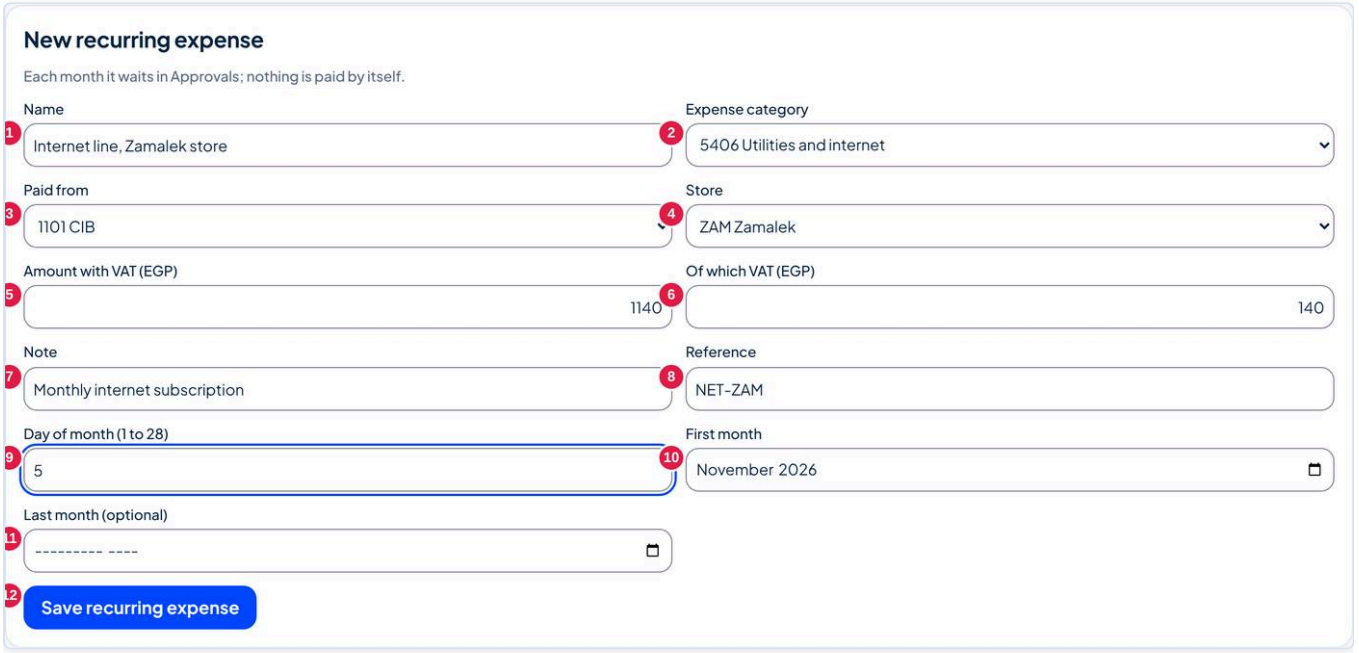
Address: `/finance/recurring` . **Menu:** Accounting > Money out > Recurring expenses. **Who:** Accounting `use` to add, stop or start again. On its day each month the job prepares that month's expense as a request in Approvals; nothing is paid by itself.



Recurring expenses with one item (/finance/recurring).

#	Part	What it shows	What it does
1	Name	Name and "2026-11 to no end"	
2	Expense category	Account and the money account under it	
3	Amount	With VAT	
4	Every month	"Day 5"	
5	This month	"Prepared, in Approvals", "Waiting for its day" or "Stopped"	

#	Part	What it shows	What it does
6	Stop / Start again	White button	Switches the item off or on (audited).

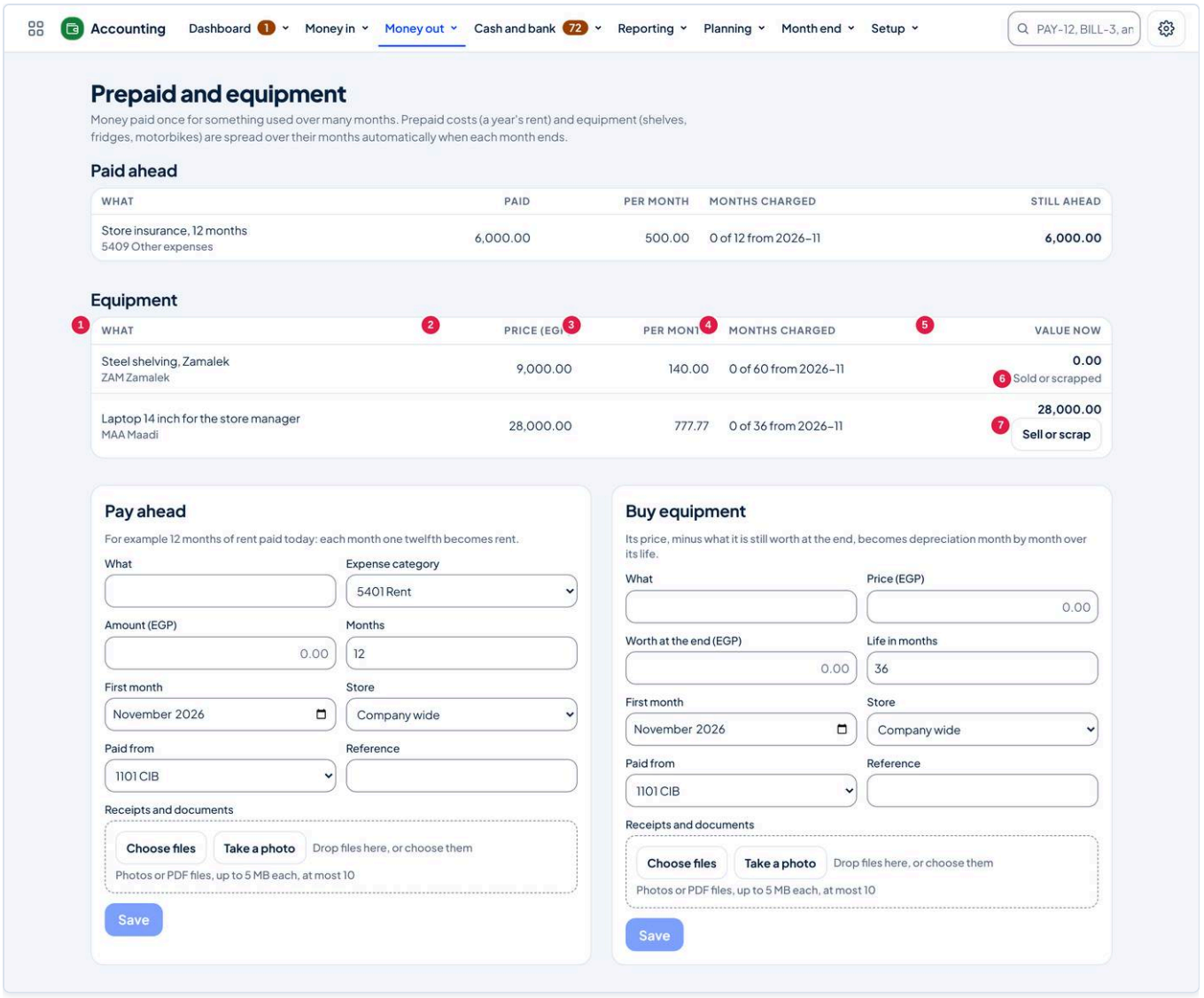


New recurring expense filled in.

#	Control	Rule / limits
1	Name	2 to 80 characters
2	Expense category	5400 or 5500 account
3	Paid from	Bank or box
4	Store	Company wide or a store
5, 6	Amount with VAT, Of which VAT	VAT below the amount
7, 8	Note, Reference	Note 3 to 300 characters; reference required
9	Day of month (1 to 28)	1 to 28, so every month has it
10, 11	First month, Last month (optional)	The last month not before the first (`end_before_start`)
12	Save recurring expense	Accounting `use` ; the new item appears in the list with "Waiting for its day"

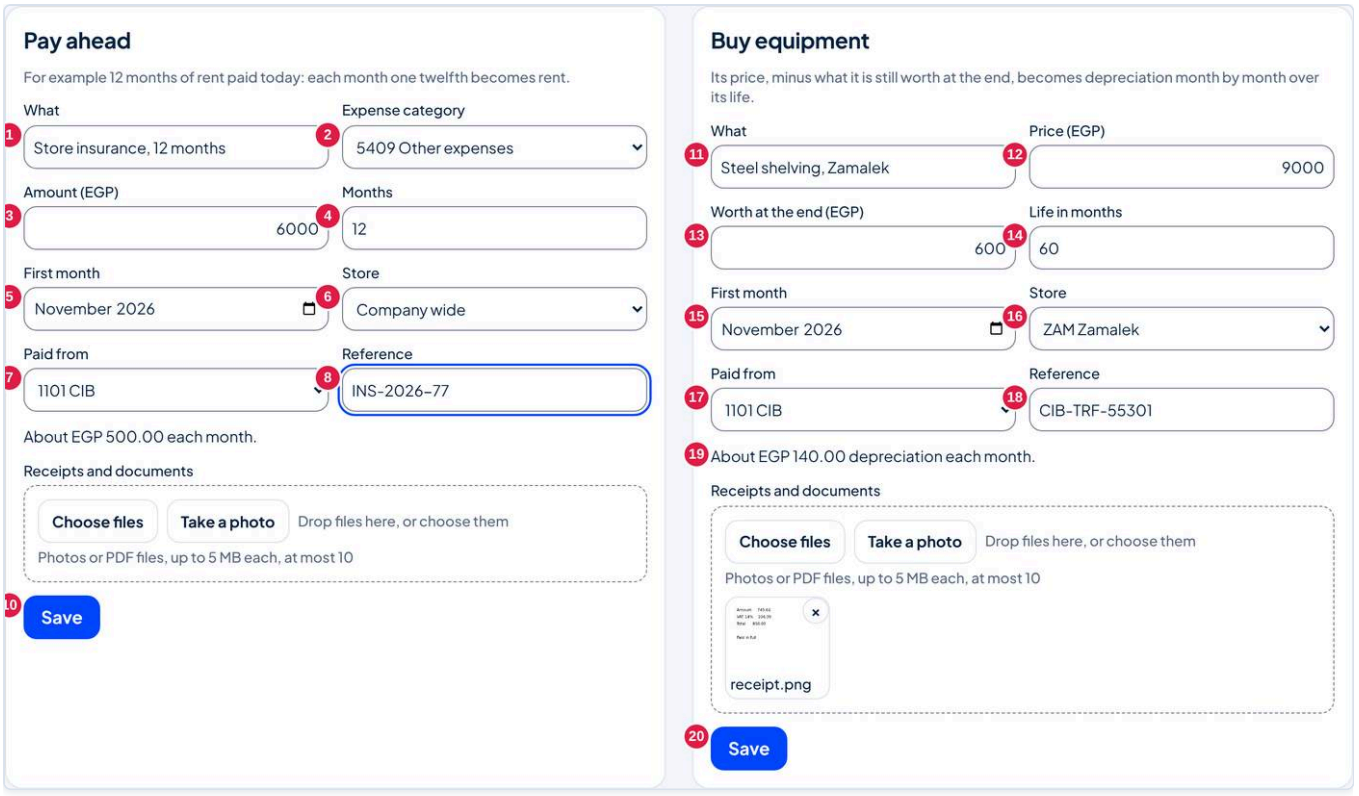
14.13 Prepaid and equipment (assets)

Address: `/finance/assets` . **Menu:** Accounting > Money out > Equipment. **Who:** Accounting `view` reads; `use` pays ahead, buys equipment and sends a sale or scrapping. There is no separate page per asset: each asset is one row of the Equipment register, and its entries are reached through the journal (reference) or the ledger lines of 1601. Equipment received on a purchase order appears here automatically. See the chapter Expenses, suppliers, bills and the assets we pay for, prepayments, equipment and depreciation.



The register after a purchase order laptop, a bought shelving unit (then sold) and an insurance paid ahead (/finance/assets).

#	Part	What it shows	What clicking it does
	Paid ahead	What, paid, per month, months charged ("0 of 12 from 2026–11"), still ahead; "Nothing paid ahead." when empty	
1	What	Equipment name and store	
2	Price (EGP)	Cost	
3	Per month	Depreciation each month: (price – worth at the end) ÷ months, in whole piastres (28,000 ÷ 36 = 777.77; the last month takes the rest)	
4	Months charged	"0 of 36 from 2026–11"	Charged once each month is over (key `depr:<id>:<month>`).
5	Value now	Cost less depreciation charged, bold	
6	Sold or scrapped	Grey text, value 0.00	The asset left the books.
7	Sell or scrap	White button under the value	Opens the disposal form.



Pay ahead and Buy equipment, filled in.

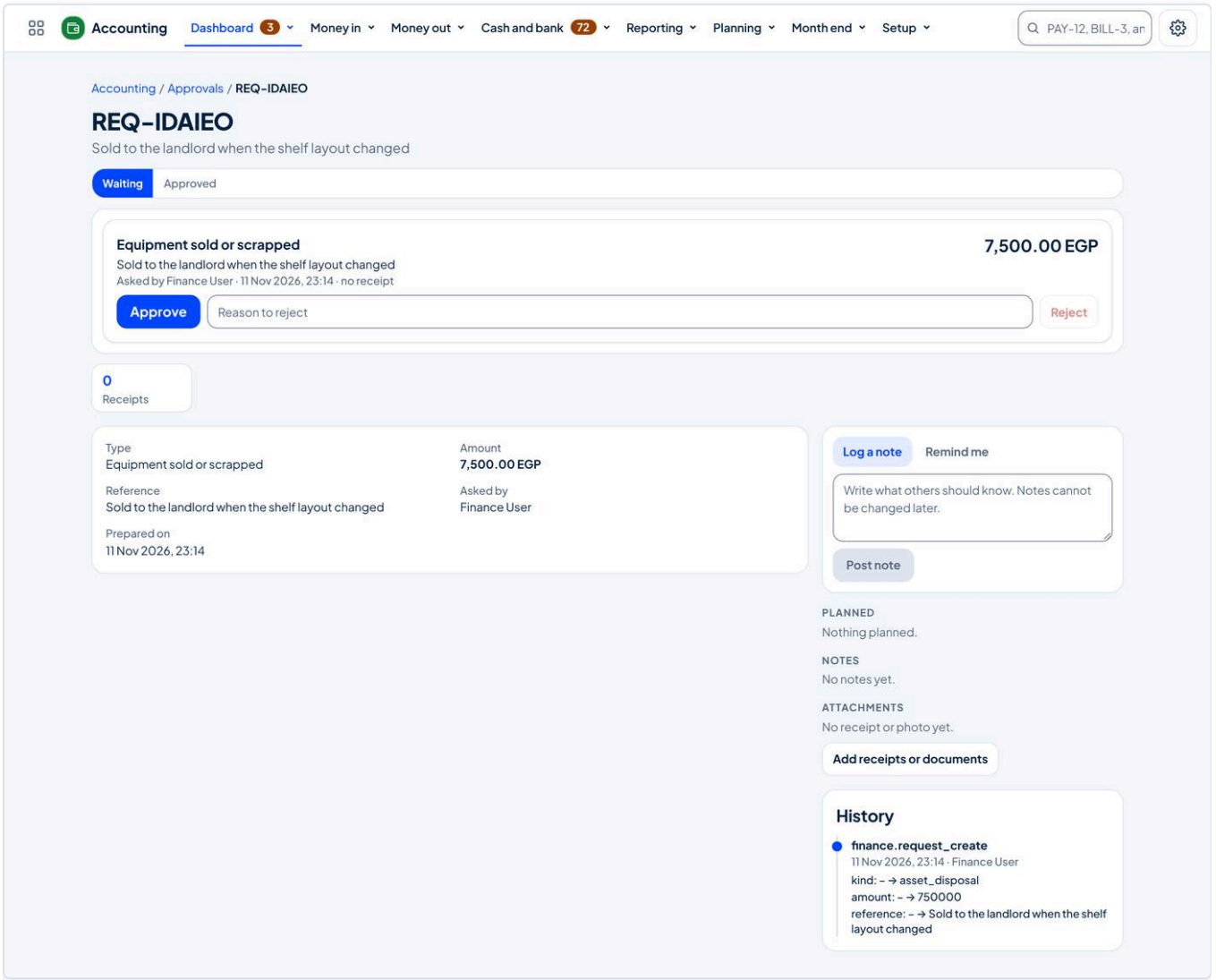
#	Control	Rule / limits	Result
1 to 8	Pay ahead: What, Expense category, Amount, Months, First month, Store, Paid from, Reference	Months 1 to 120; the bank or box must hold it; from the limit it waits	Dr 1401 Prepayments, Cr the bank; then one share a month to the category.
9	"About EGP 500.00 each month."	Live hint	
10	Save	With the attachment field above it	
11 to 18	Buy equipment: What, Price, Worth at the end, Life in months, First month, Store, Paid from, Reference	Worth at the end below the price (`invalid_salvage`); life 1 to 600 months	Dr 1601 Equipment, Cr the bank.
19	"About EGP 140.00 depreciation each month."	(9,000 – 600) ÷ 60	
20	Save	With the attachment field	

Selling or scrapping an asset

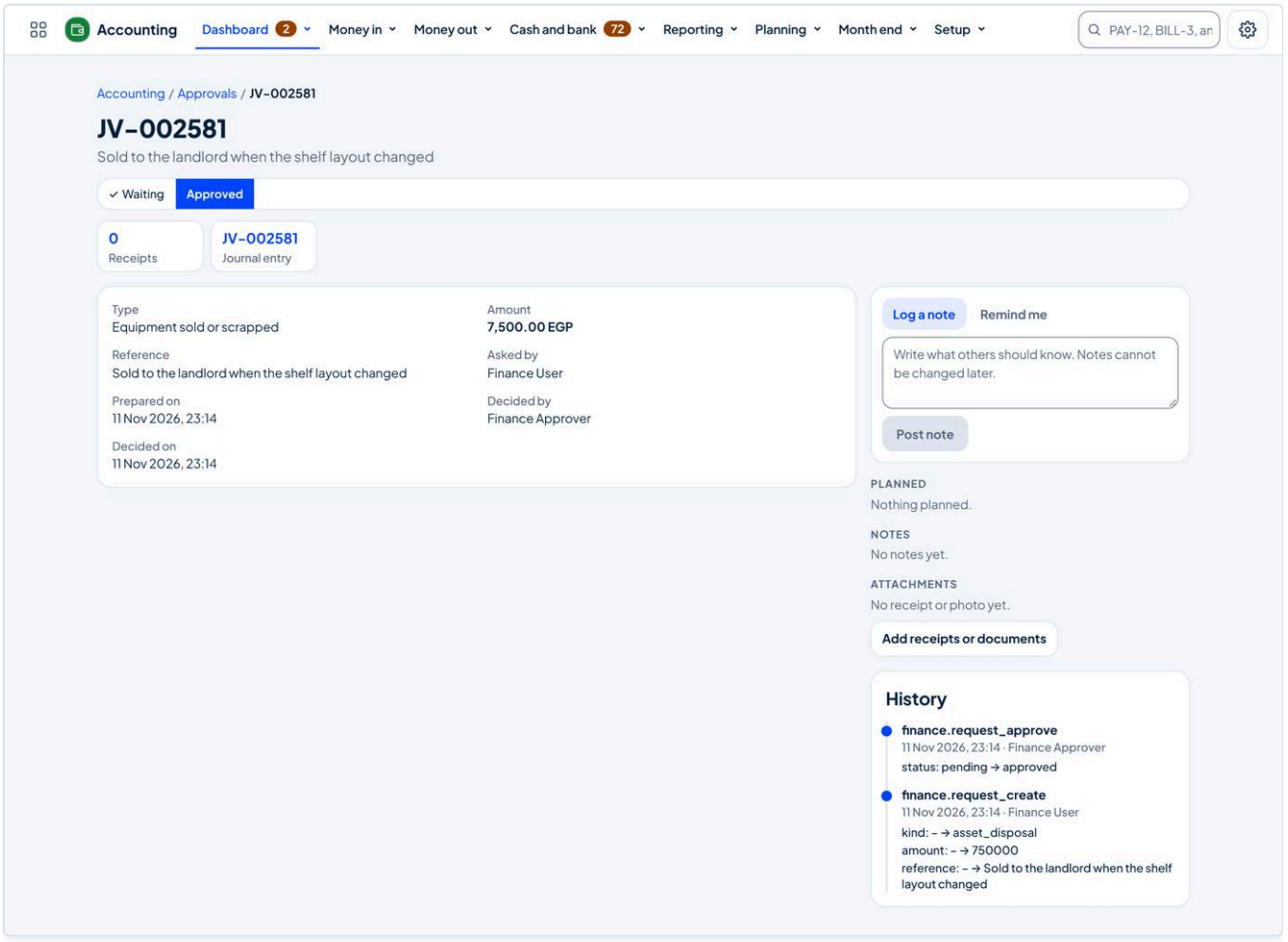
Sell or scrap opened: Sold for (EGP, 0 if scrapped), the account the money comes into (shown only when sold for more than 0), the reason, the hint and "Take off the books".

After sending: "Saved and sent for approval. A second person must approve it before it is posted."

A disposal always waits for a second person (`asset_disposal`). On approval the cost and accumulated depreciation leave the books, the sale money comes into the bank, and the rest goes to 5902 (here a loss of 1,500.00: value 9,000.00, sold for 7,500.00). Months before the disposal month are charged first; a disposal in or before a month already charged is refused (`disposal_before_charged_month`).



The disposal request as the second person sees it: Approve, "Reason to reject" and Reject.



Approved: the request shows its journal entry; the register then shows "Sold or scrapped".

15 Screens: Cash and bank

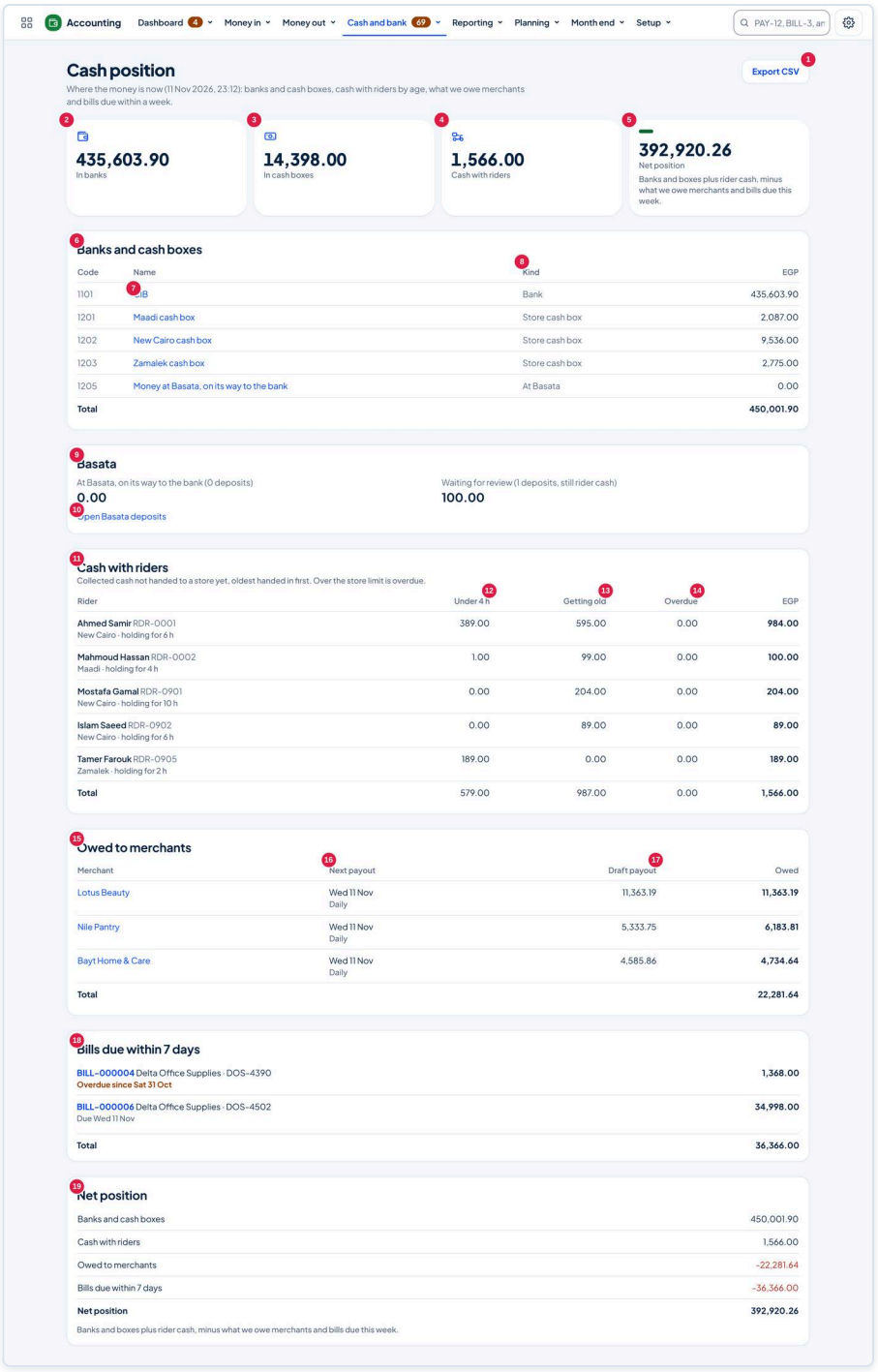
This chapter walks through every screen of the **Cash and bank** menu of the Accounting app, then the two rider screens and the two store screens that feed it. Every screenshot was taken on the local demo copy (Cairo time 11 Nov 2026, about 23:00) after performing the actions shown, so each figure also shows the real result of the action. The money rules behind these screens (accounts, postings, ages, deadlines) are explained in chapter 11, *Cash on delivery, riders, store boxes and rider pay*, and in chapter 12, *Payables and expenses* (sections Loans and Bank accounts in another currency). The red numbered circles on the screenshots were added for this guide; each figure with circles is followed by a legend table.

> Access in one line: every page of this menu opens with Accounting `view` (GET routes). Every button that changes something needs Accounting `use` (POST routes default to use in `AccessGuard`), except where a row says otherwise. People limited to some stores see only the riders, boxes, handovers and Basata deposits of their stores; banks, merchants, bills and loans stay company wide.

The menu path for all Accounting screens below is **Accounting app tile, top bar menu Cash and bank**. The orange counter beside *Cash and bank* in the top bar adds up the items waiting on the pages of that menu (open bank lines, Basata deposits waiting, money movements without a receipt and so on), so it changes while you work.

15.1 Cash position

Address `/finance/cash-position`. Menu: Cash and bank, Cash position. Opens with Accounting view; there is no button that changes data on this page. It answers one question: where is the money right now. Data comes from `GET /finance/cash-position` (function `cashPosition` in `packages/db/src/finance/cash-control.ts`), the CSV from `GET /finance/cash-position/export.csv`.



Cash position at /finance/cash-position with every part numbered. One deposit (100.00 EGP) is waiting for review in the Basata block; merchants have draft payouts; two supplier bills are due.

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Export CSV	White button with blue text, top right.	Downloads the CSV described below (one row per bank, box, rider, merchant and bill, then the net).
2	In banks	Stat card, blue wallet icon, the sum of all open bank accounts (group 1100) in EGP.	Nothing.
3	In cash boxes	Stat card, blue cash icon, the sum of company cash boxes and store boxes (group 1200, Money at Basata included).	Nothing.

#	Part	What it shows (colors, icons, wording)	What clicking it does
4	Cash with riders	Stat card, blue scooter icon, all rider cash (1300 wallets). When any rider has overdue cash the card turns to the needs action color (orange) with a dot and the hint "N EGP overdue".	Nothing.
5	Net position	Stat card with a short green bar on top when the net is zero or more, orange when negative. Hint text explains the formula.	Nothing.
6	Banks and cash boxes	Table Code, Name, Kind, EGP with a bold Total. A bank in USD or EUR shows its foreign balance in small grey text under the EGP amount.	Section only.
7	Account name	Blue link.	Opens the account page `/finance/accounts/<id>` with its lines.
8	Kind	Grey text: Bank, Cash box, Store cash box or At Basata.	Nothing.
9	Basata block	Shown only while something is at Basata or waiting. Left: "At Basata, on its way to the bank (N deposits)" with the amount approved but not yet transferred; right: "Waiting for review (N deposits, still rider cash)".	Nothing.
10	Open Basata deposits	Blue link.	Opens `/finance/basata`.
11	Cash with riders	One row per rider holding cash: bold name, grey badge, store and "holding for N h" (hours since the oldest collection still held), then the cash split by age and the total in bold. Empty state: "No rider is holding cash."	Section only.
12	Under 4 h	Cash younger than `finance.rider_cash_warn_hours` (4 in the demo; the header shows the setting).	Nothing.
13	Getting old	Cash between the warn hours and `finance.rider_cash_max_hours` (12).	Nothing.
14	Overdue	Cash older than the max hours of the rider's store; a value above zero is bold orange.	Nothing.
15	Owed to merchants	Merchants we owe, with a grey line "Merchants owe us N EGP (not counted as cash)" when some wallets are in debit. Empty state: "We owe no merchant anything."	Merchant names are blue links to `/finance/merchants/<id>`.
16	Next payout	The next day of the merchant's payout schedule (short weekday and date) and the schedule below in grey: Daily, Sunday and Wednesday, Sunday.	Nothing.
17	Draft payout	Amount of an open draft payout, or a dash.	Nothing.
18	Bills due within 7 days	Each open supplier bill due in 7 days or already overdue: blue BILL number, supplier and reference, then "Due Wed 11 Nov" in grey or "Overdue since Sat 31 Oct" in bold orange, amount left on the right. Empty state: "No bill is due this week."	The BILL number opens the bills list `/finance/bills`.

#	Part	What it shows (colors, icons, wording)	What clicking it does
19	Net position table	Banks and cash boxes, plus Cash with riders, minus Owed to merchants (red, negative), minus Bills due (red), equals Net position in bold.	Nothing.

Controls on this screen

Control	What it does	Rule / limits	Result
Export CSV	Downloads `cash-position.csv`	Same scope as the page (store limited people get only their boxes and riders). UTF 8 with byte order mark, formula safe cells.	Columns `section, code, name, date, amount_egp, detail, note`. Sections: `bank`, `store_box`, `cash_box`, `basata`, `rider_cash` (date = held since, detail "fresh / aging / overdue"), `merchant_owed` (negative, date = next payout day, detail = schedule), `bill_due` (negative, note "overdue" when late) and one `net_position` row.
Store scope banner	Shown only to people limited to some stores	Text: "You see only the cash of your stores. Banks, merchants, bills and the net position are company wide."	Rider and box rows are filtered by store.

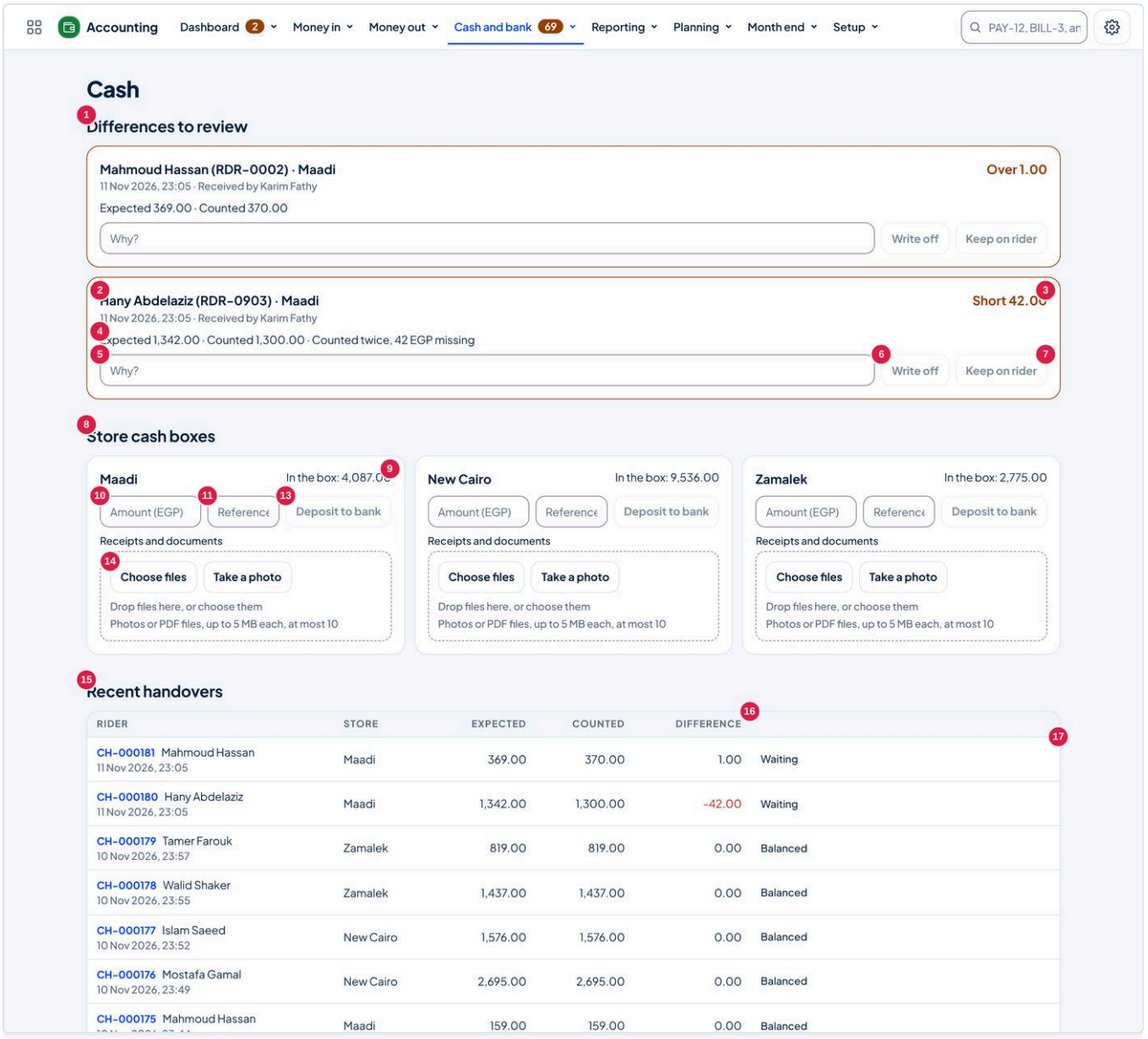
RULE

The intro line prints the moment the figures were read ("Where the money is now (11 Nov 2026, 23:12)"). Rider cash is aged oldest first: cash handed in or deposited pays off the oldest collections, so the age comes from the newest collections still covering the balance (chapter 11, Cash age and the overdue alert).

15.2 Store cash (handover differences and bank deposits)

Address `/finance/cash`, page title **Cash**. Menu: Cash and bank, Store cash. Opens with Accounting view; the decision buttons and Deposit to bank need Accounting use. Data: `GET /finance/handovers?status=pending`, `GET /finance/handovers` (the 50 newest are shown) and `GET /finance/overview` for the store boxes. Component `CashDesk` (`apps/web/src/components/finance/CashDesk.tsx`).

The two differences on the figure were created on the store side (see Store app: Cash from riders): Hany counted 1,300.00 against 1,342.00 expected, and Mahmoud handed in 370.00 against 369.00.



Store cash at /finance/cash with two handover differences waiting (a shortage and a surplus), the three store boxes and the recent handovers with their statuses.

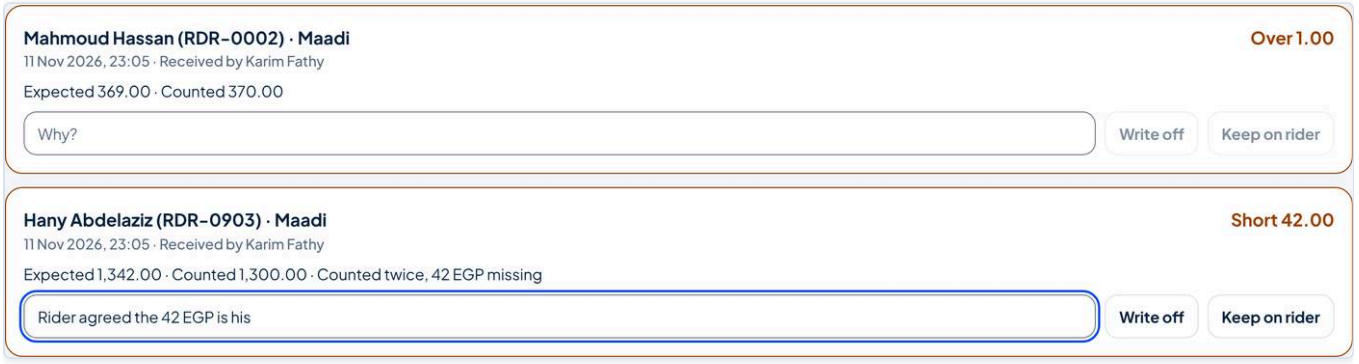
#	Part	What it shows	What clicking it does
1	Differences to review	One card per handover whose counted cash differed from what was expected (status pending). Each card has an orange border. Empty state: "No differences waiting."	Section only.
2	Rider line	Rider name, badge in brackets, store; under it the time and "Received by" the staff member who counted.	Nothing.
3	Difference	Bold orange "Short 42.00" (less cash than expected) or "Over 1.00" (more).	Nothing.
4	Expected and counted	"Expected 1,342.00 · Counted 1,300.00" and the staff note if one was written.	Nothing.
5	Why?	Text box for the decision note.	Typing at least 3 characters enables the two buttons.

#	Part	What it shows	What clicking it does
6	Write off	Grey outline button, disabled until the note has 3 characters.	Posts the difference to 5301 cash shortages and surpluses (a shortage is a loss, a surplus income) and closes the card.
7	Keep on rider	Same style.	A shortage moves from rider cash to the rider's 1350 dues wallet (taken from his next withdrawal); a surplus stays in rider cash. Closes the card.
8	Store cash boxes	One card per dark store.	Section only.
9	In the box	The store box balance.	Nothing.
10	Amount (EGP)	Amount to take to the bank. The deposit fields appear only when the box holds cash.	Typing.
11	Reference	Bank slip reference, at least 2 characters.	Typing.
12	Bank (not visible here)	A bank list appears beside the reference only when more than one bank account exists; empty means the main bank CIB 1101.	Chooses the bank.
13	Deposit to bank	Outline button, disabled until amount and reference are valid and files finished uploading.	Posts the bank deposit (Dr bank, Cr store box) with the files attached; message "Deposit recorded."
14	Receipts and documents	The shared attachment field: Choose files, Take a photo, drop zone, hint "Photos or PDF files, up to 5 MB each, at most 10".	Uploads at once; thumbnails appear with a cross to remove.
15	Recent handovers	Table of the 50 newest handovers: blue CH number with rider name and time, store, Expected, Counted, Difference, status.	Nothing (the CH number is styled as a link but is plain text).
16	Difference	Counted minus expected: 1.00 for a surplus, red −42.00 for a shortage, 0.00 when balanced.	Nothing.
17	Status	Column without a header. Plain text, no pill: Balanced, Waiting, Written off, On rider; the decision note in grey under a decided status.	Nothing.

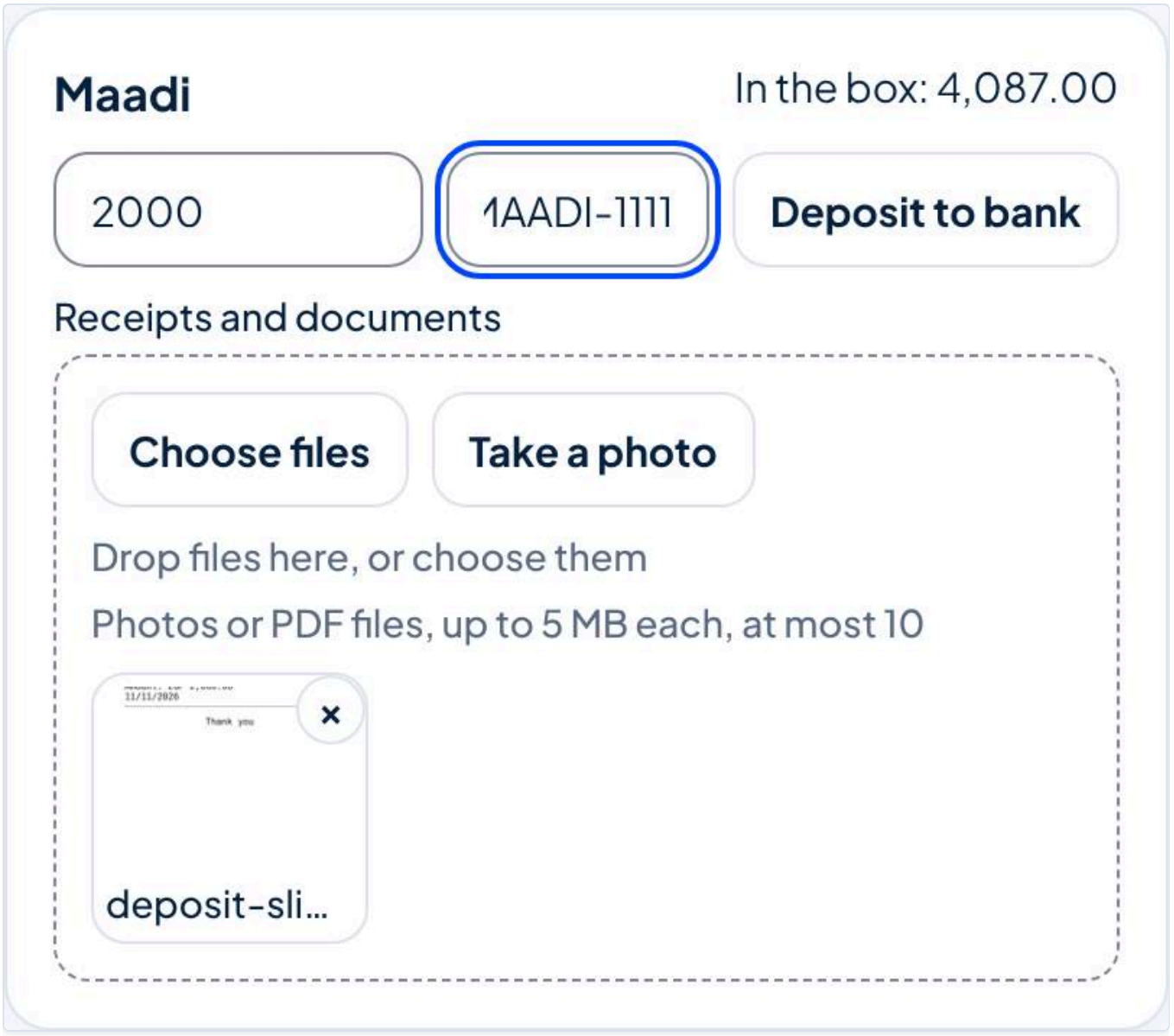
Controls on this screen

Control	What it does	Rule / limits	Result
Why? + Write off	`POST /finance/handovers/:id/review` with `decision: write_off`	Note 3 to 300 characters; only a pending handover (`handover_not_pending` otherwise); Accounting use.	Entry kind `cash_difference` , key `handover:<id>:write_off` ; status Written off; audit `finance.cash_handover_review` .
Why? + Keep on rider	Same route with `decision: charge_rider`	Same rules.	Entry kind `rider_shortage_charge` (Dr 1350 rider dues, Cr 1300 rider cash) for a shortage; status On rider; the rider sees "You owe" in his wallet.

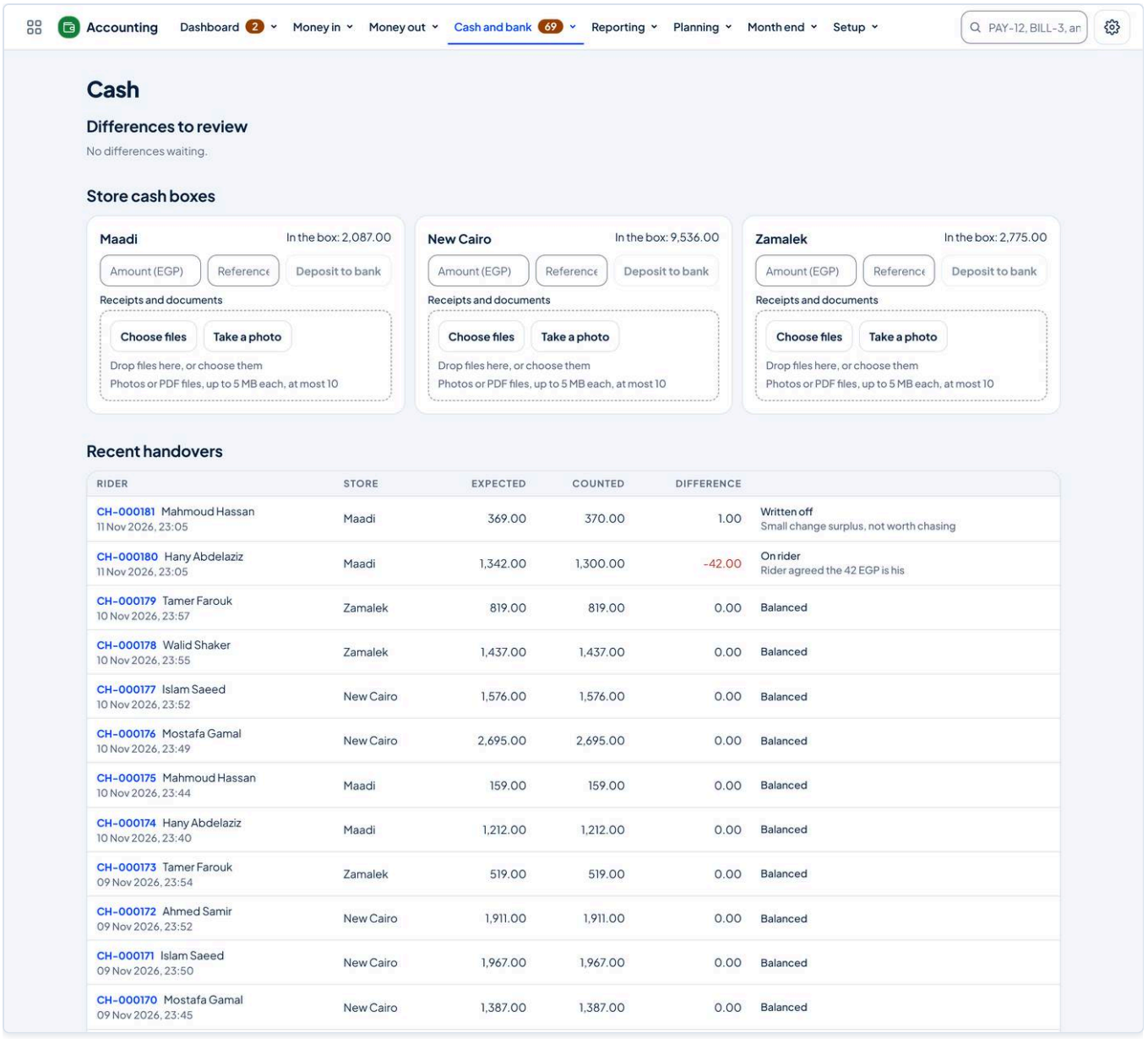
Control	What it does	Rule / limits	Result
Amount, Reference, files, Deposit to bank	`POST /finance/deposits`	Amount above zero and never more than the box holds; reference 2 to 100 characters; idempotency key per click; books lock.	Entry kind `bank_deposit`; the box balance drops at once (Maadi went from 4,087.00 to 2,087.00 in the figures).



Both difference cards; the note typed on Hany's card enables Write off and Keep on rider.



Deposit to bank from the Maadi box: 2,000 EGP, reference DEP-MAADI-1111 and a slip photo uploaded. The reference box is narrow in the three column layout, so the start of the text is cut off while typing.

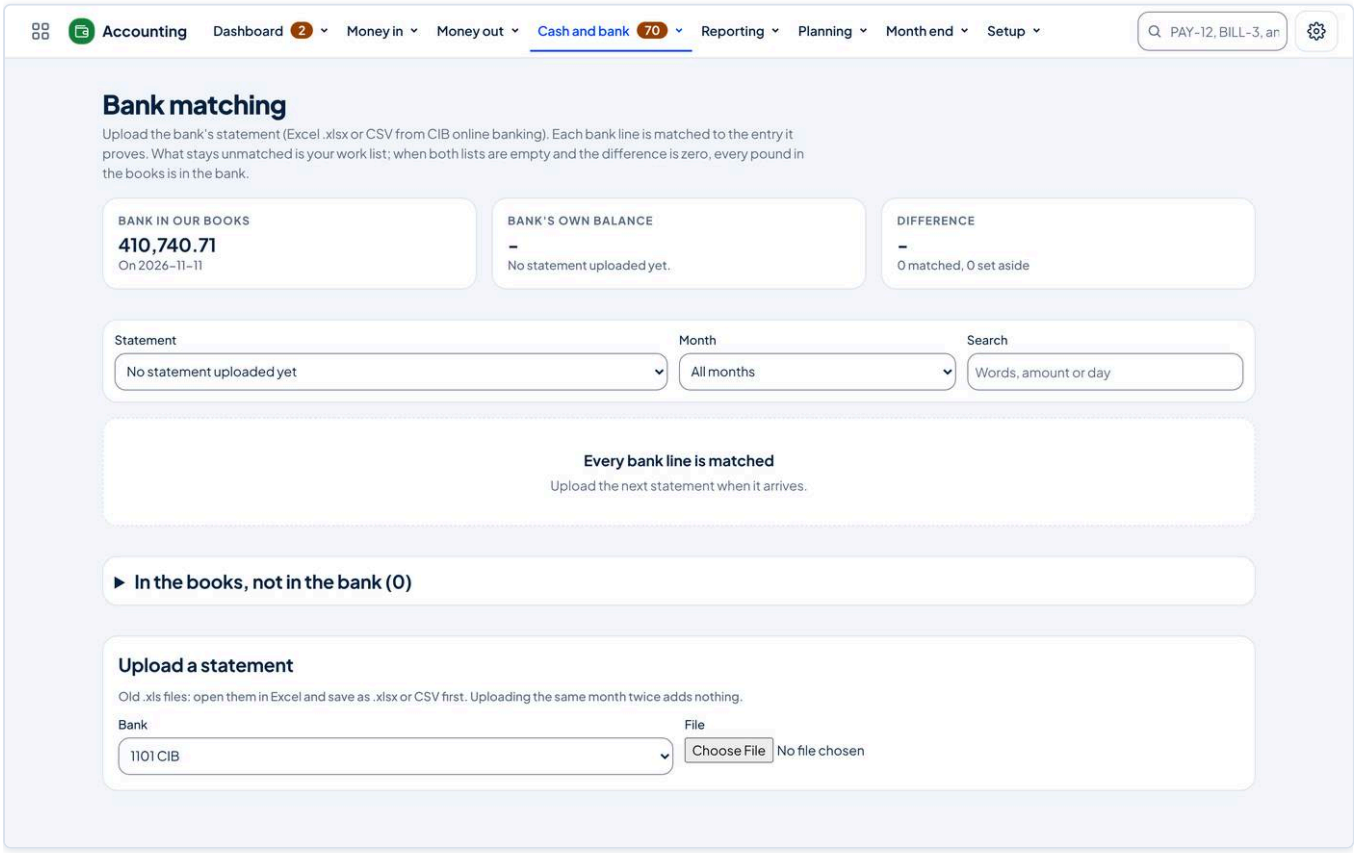


After the decisions and the deposit: no differences waiting, Maadi box down to 2,087.00, and the handovers show every status: Written off (with the note), On rider (with the note) and Balanced.

15.3 Bank matching

Address `/finance/bank` (optional query `?account=<id>&statement=&month=&q=`). Menu: Cash and bank, Bank matching. Opens with Accounting view; Match, Book as a bank charge, Set aside and Import statement need Accounting use (the matching buttons are hidden without it). Components `BankReconcile` and `StatementUpload` (`PeriodReports.tsx`) and `BankMatching` (`BankMatching.tsx`). Data: `GET /finance/bank-reconciliation/:accountId`, `GET /finance/bank-statements`, `GET /finance/bank-matching/:accountId`.

Before any statement



Bank matching before the first statement: the bank's own balance and the difference show a dash with "No statement uploaded yet.", and the matching area already says "Every bank line is matched".

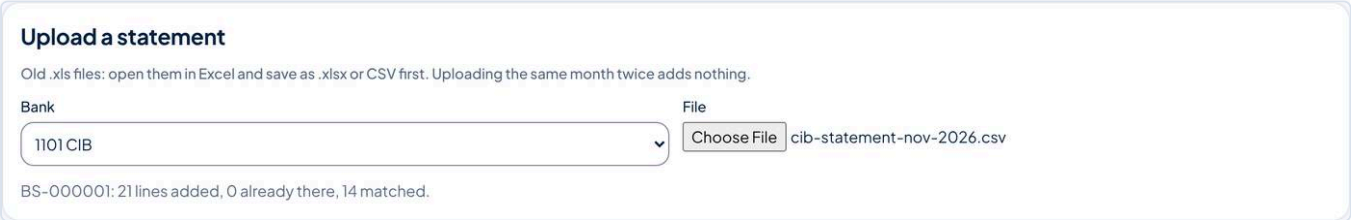
Uploading a statement and picking the columns

The guide used a CSV in the CIB layout created for the purpose: a title row, an account row, then the header `Date, Value Date, Description, Reference, Debit, Credit, Balance` and 21 lines from 3 to 11 November (cash deposits, InstaPay batches, merchant payouts, a 25.00 maintenance fee, a 150.00 duplicate posting and its reversal, the Maadi deposit and the Basata transfer recorded on this chapter's other screens, a 849.50 card purchase and a 1,000.00 transfer from an unknown sender).

Upload a statement after choosing the file: the header row was found on row 3, the columns were guessed from their names, and the first five lines are previewed.

#	Part	What it shows	What clicking it does
1	Bank	The open bank accounts of group 1100 ("1101 CIB").	Chooses which bank account the lines belong to.
2	File	The browser's own file button; accepts .xlsx, .csv and .txt.	Sends the file to `POST /finance/bank-statements/read`, which answers its rows; nothing is stored yet.
3	Header row	The first 30 rows as "n: first three cells".	Choosing another row guesses the columns again from that row.
4	Date	Required column. The guess took "Value Date" here (both columns hold the same days in this file).	Pick the column holding the day.
5	Description	Optional (None allowed).	When empty the reference is used, else "-".
6	Reference	Optional.	Kept with the line, up to 100 characters.
7	Amount (one column, out is minus)	None when the file has separate debit and credit columns.	Choosing a column hides Money out and Money in.
8	Money out	Shown only while Amount is None.	Debit column.
9	Money in	Shown only while Amount is None.	Credit column.
10	Balance	Optional running balance.	The last balance becomes "Bank's own balance".
11	Count line	"21 lines read from cib-statement-nov-2026.csv. First lines:" Rows without a valid day or amount are skipped.	Nothing.
12	Preview	Date (ISO form), Description, Amount, Balance of the first 5 lines.	Nothing.

#	Part	What it shows	What clicking it does
13	Import statement	Blue button, disabled with no lines.	`POST /finance/bank-statements` ; message "BS-000001: 21 lines added, 0 already there, 14 matched."

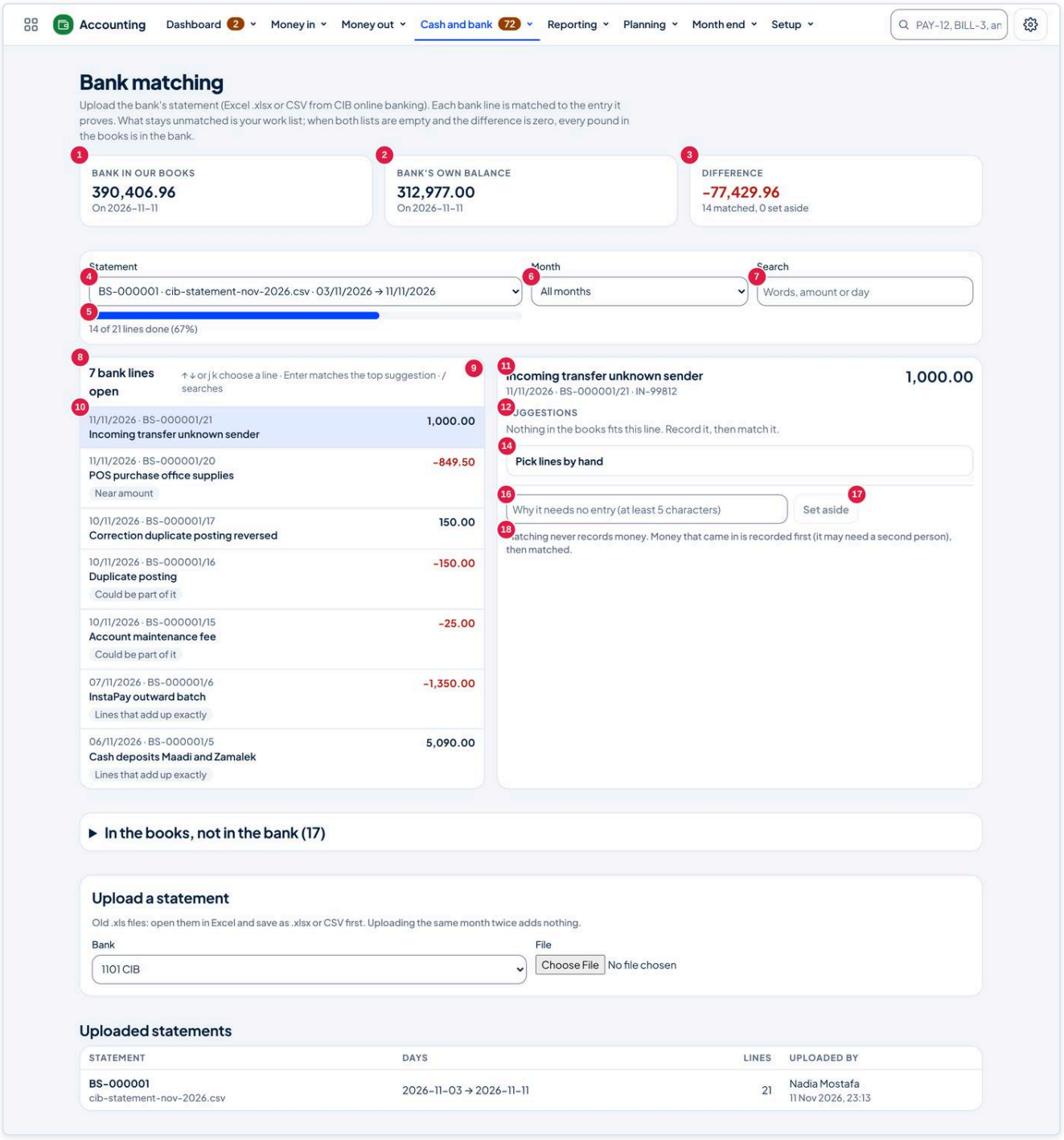


The import result. 14 of 21 lines were matched automatically because each had a single closest ledger line of the same amount within 3 days.

RULE

Rules of the upload: old .xls files are refused with "This is an old .xls file. Open it in Excel and save it as .xlsx or CSV."; an unreadable file answers "This file could not be read. Save it as .xlsx or CSV and try again."; an empty one "The file is empty.". Each line is stored with a fingerprint, so uploading overlapping days adds only the new lines ("N already there"). At most 5,000 lines per import. Statements are numbered BS-000001 onward.

The matching screen



Bank matching after the import: 7 bank lines open on the left, the selected line's suggestions and actions on the right, the books only list folded, the upload panel and the uploaded statements.

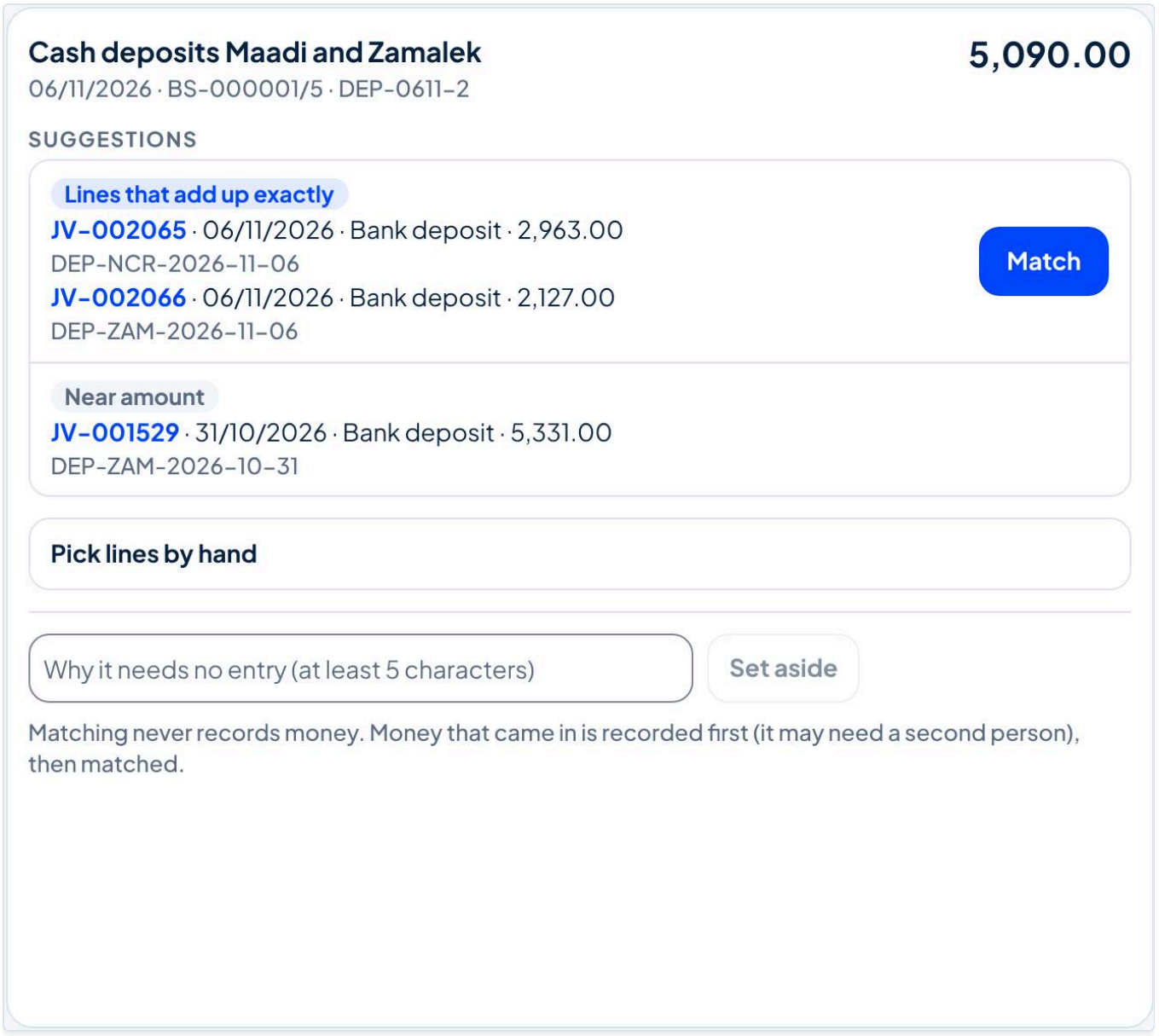
#	Part	What it shows	What clicking it does
1	Bank in our books	The bank account balance in the ledger on the day shown.	Nothing. With several banks a row of bank buttons appears above the cards; the chosen one has a blue border.
2	Bank's own balance	The last balance of the newest statement and its day; a dash before any statement.	Nothing.

#	Part	What it shows	What clicking it does
3	Difference	Books minus bank: green "Matches" at zero, red amount otherwise; under it "N matched, N set aside". (Here the difference is large because the made up statement starts from an invented opening balance.)	Nothing.
4	Statement	"BS-000001 · file name · from → to"; "No statement uploaded yet" before the first.	Shows the lines of that statement only (`?statement=`).
5	Progress bar	Blue bar and "14 of 21 lines done (67%)": matched and set aside lines count as done.	Nothing.
6	Month	All months or one month of bank lines.	Filters the open lines (`?month=`).
7	Search	Placeholder "Words, amount or day".	Enter applies it (`?q=`); the / key jumps here.
8	N bank lines open	Count of open lines (all statements of the account).	Nothing.
9	Keys hint	"↑ ↓ or j k choose a line · Enter matches the top suggestion · / searches".	Nothing.
10	Bank line	Day and statement/line number in grey, description in bold, amount on the right (money out in red with a minus), and a grey chip with the reason of its best suggestion. The selected line has a light blue background.	Selects the line; the right panel follows.
11	Selected line	Description, day, statement/line and bank reference, amount in large type.	Nothing.
12	Suggestions	Ranked ledger lines that could prove this bank line, each with a reason chip. Matchable reasons have a blue chip and a Match button; Near amount and Could be part of it have grey chips and no button. Empty: "Nothing in the books fits this line. Record it, then match it."	The JV numbers open the journal entry.
13	Match (top suggestion)	The first matchable suggestion has a blue filled Match button; later ones a white one. (Not visible on the figure because the selected line has no suggestion.)	Matches at once; see the controls table.
14	Pick lines by hand	A folded panel listing open ledger lines of the same direction within 10 days of the bank line (3 days window plus 7).	Opens the list with check boxes and a running total.
15	Book as a bank charge	Shown only for money out lines.	Books the line as a bank charges expense and matches it.
16	Why it needs no entry	Text box, placeholder "(at least 5 characters)".	Enables Set aside at 5 characters.
17	Set aside	Grey outline button.	Marks the line as needing no entry, with the reason.
18	Reminder	"Matching never records money. Money that came in is recorded first (it may need a second person), then matched."	Nothing.

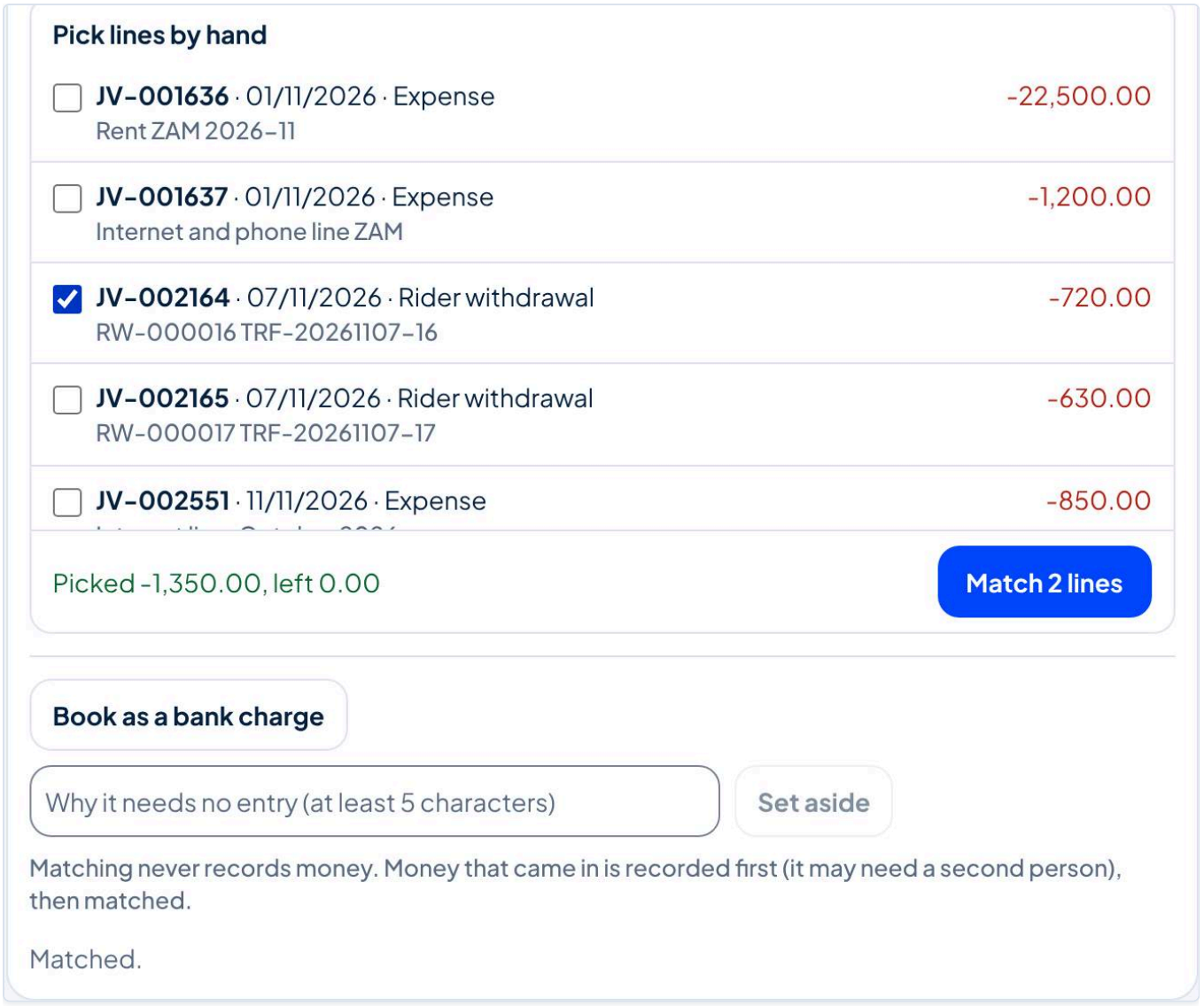
Controls on this screen

Control	What it does	Rule / limits	Result
Match (a suggestion)	`POST /finance/bank-lines/:id/match-many` with the suggestion's ledger line ids	The lines must add up exactly to the bank amount (`amounts_differ`), belong to this bank account (`ledger_line_not_found`), not prove another bank line already (`ledger_line_matched` , also enforced by a trigger on `bank_line_matches`); the bank line must still be open (`bank_line_not_open`). At most 20 ledger lines.	"Matched."; the line leaves the open list, progress moves on. No money is recorded.
Enter key	Same as Match on the top matchable suggestion	Ignored while typing in a box or with Ctrl, Alt or Cmd held.	Same.
Up and Down arrows, j and k	Move the selection through the open lines	Same exceptions.	The list scrolls to keep the line in view.
/ key	Focuses the search box		
Pick lines by hand, check boxes, Match N lines	Matches the ticked lines with `match-many`	The button is enabled only when the ticked amounts add up exactly; the total turns green ("Picked –1,350.00, left 0.00").	"Matched."
Book as a bank charge	`POST /finance/bank-lines/:id/bank-charge`	Money out only (`not_money_out`); uses the expense path, so from `finance.approval_threshold_piastres` it waits for a second person.	"Booked as a bank charge and matched." or "Saved and sent for approval. A second person must approve it before it is posted."
Why + Set aside	`POST /finance/bank-lines/:id/ignore`	Reason 5 to 300 characters (`note_required`).	"Set aside."; counted in "N set aside" and as done in the progress.
In the books, not in the bank (N)	Folded list of ledger lines on this bank since the first statement day that no bank line proves	Hint: "a deposit not made, a cheque not cleared, or a typo. Find out why."	Each JV number opens the entry.
Uploaded statements	Table Statement, Days, Lines (with "N already uploaded"), Uploaded by and when		Read only.

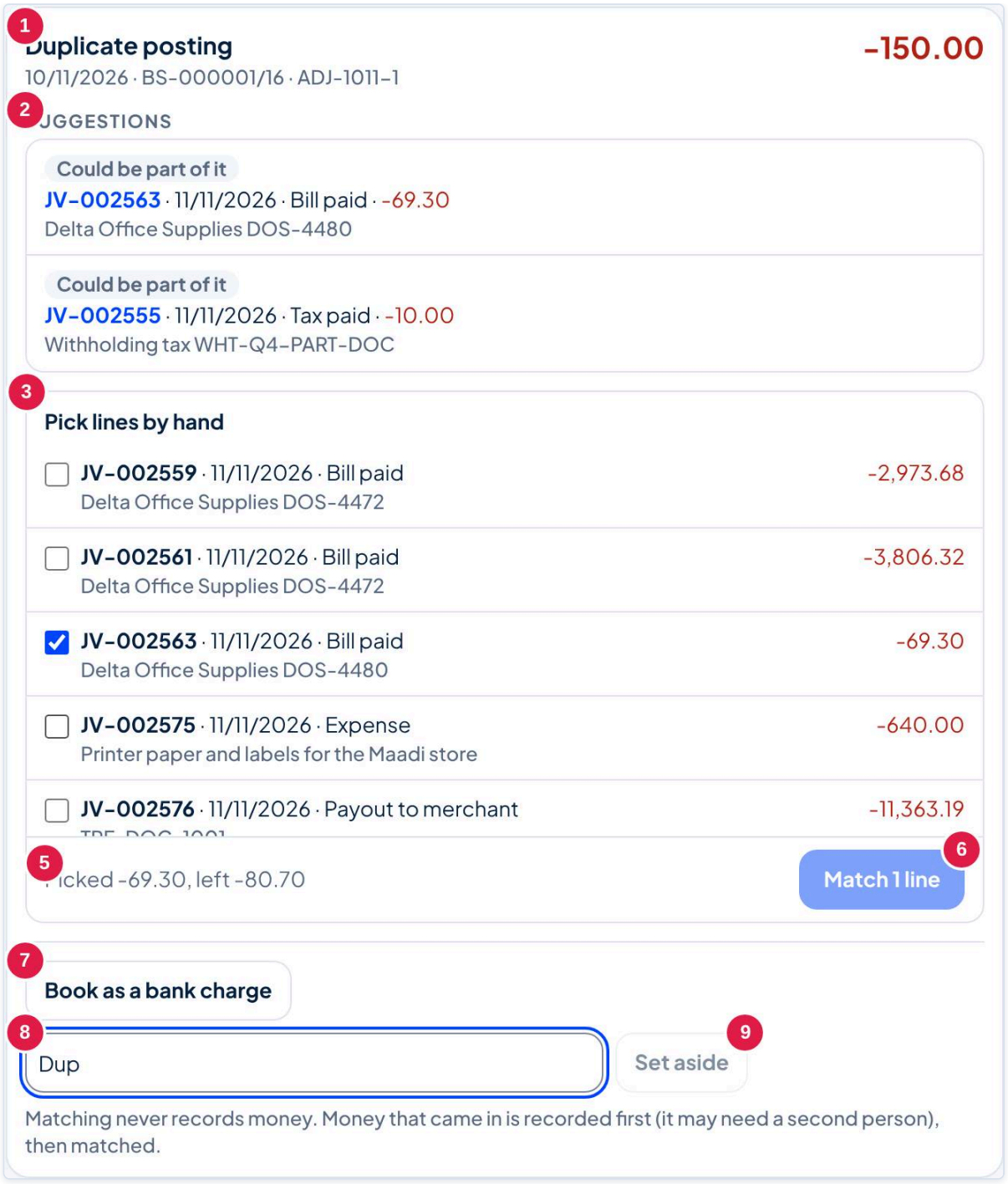
A bank line has three states: **open** (listed on the left), **matched** and **set aside**. Matched and set aside lines are no longer listed; they show only as the counts in the Difference card, the progress bar and the statement's lines. The API has `POST /finance/bank-lines/:id/reopen` , but no button on this screen calls it.



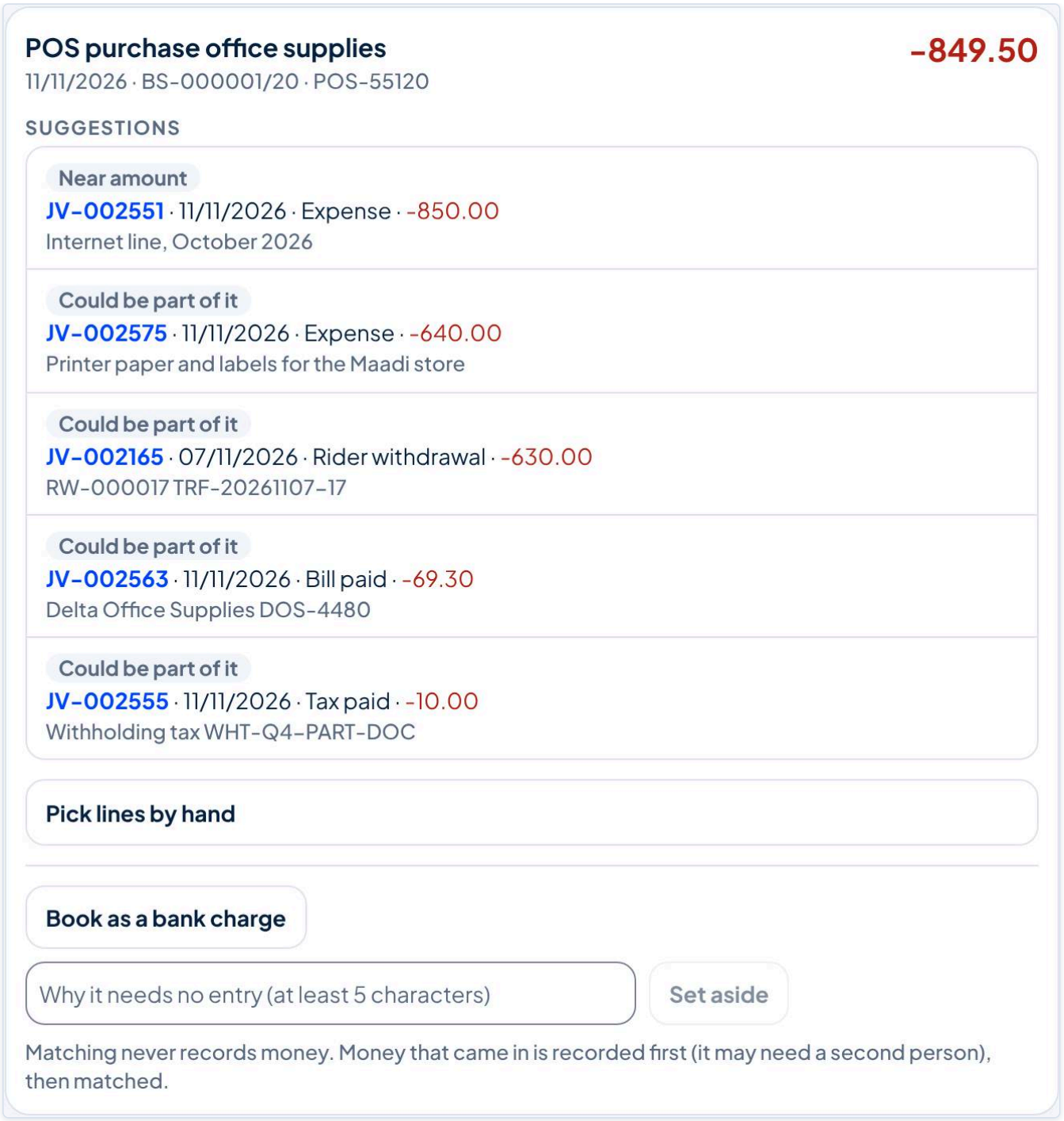
A credit of 5,090.00 whose top suggestion is two deposits of the same day that add up exactly (blue chip "Lines that add up exactly", blue Match button); a near amount is shown in grey without a button.



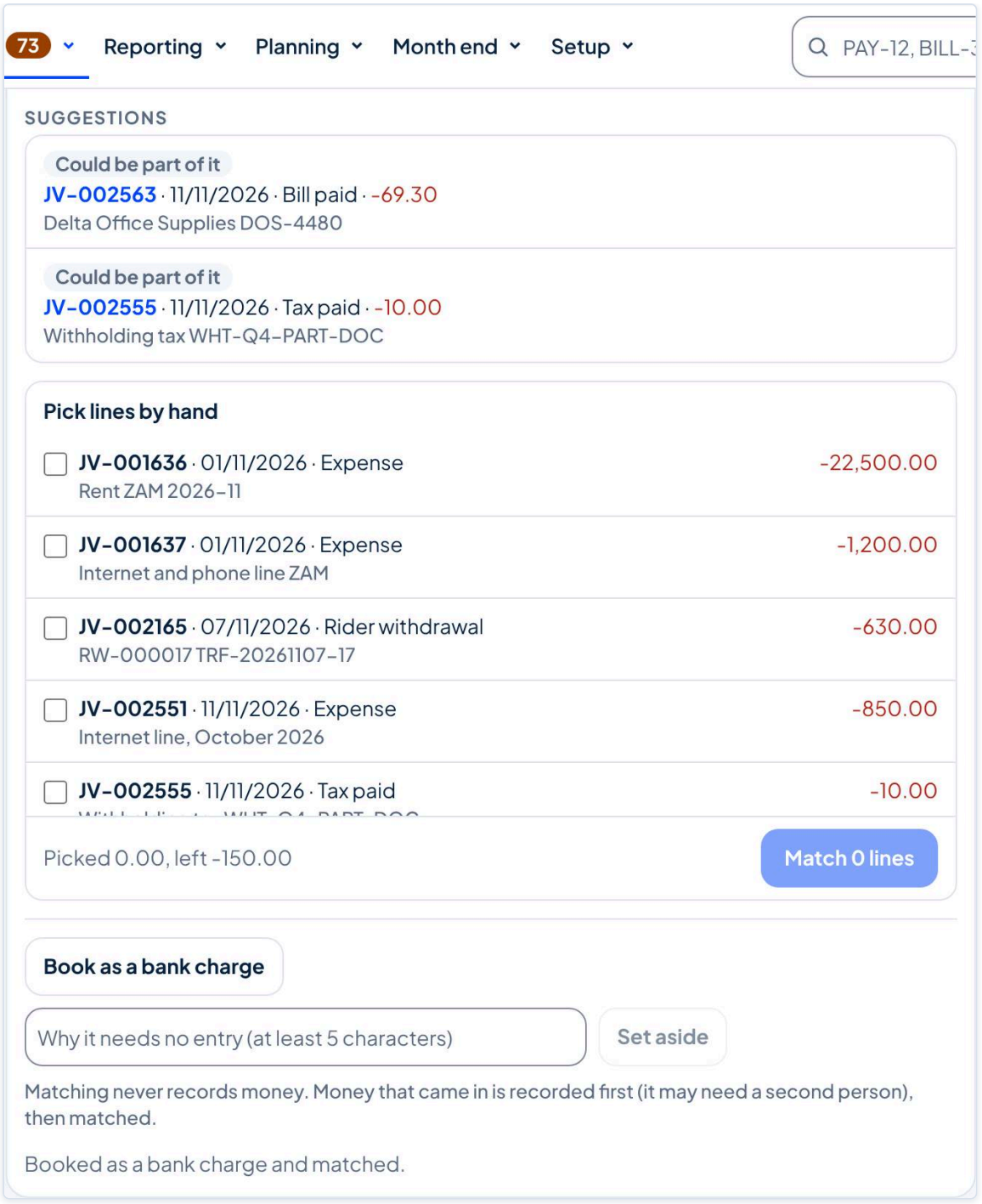
Pick lines by hand for the 1,350.00 InstaPay batch: two rider withdrawals ticked, the total turns green at "left 0.00" and Match 2 lines is enabled. After clicking, the panel says "Matched."



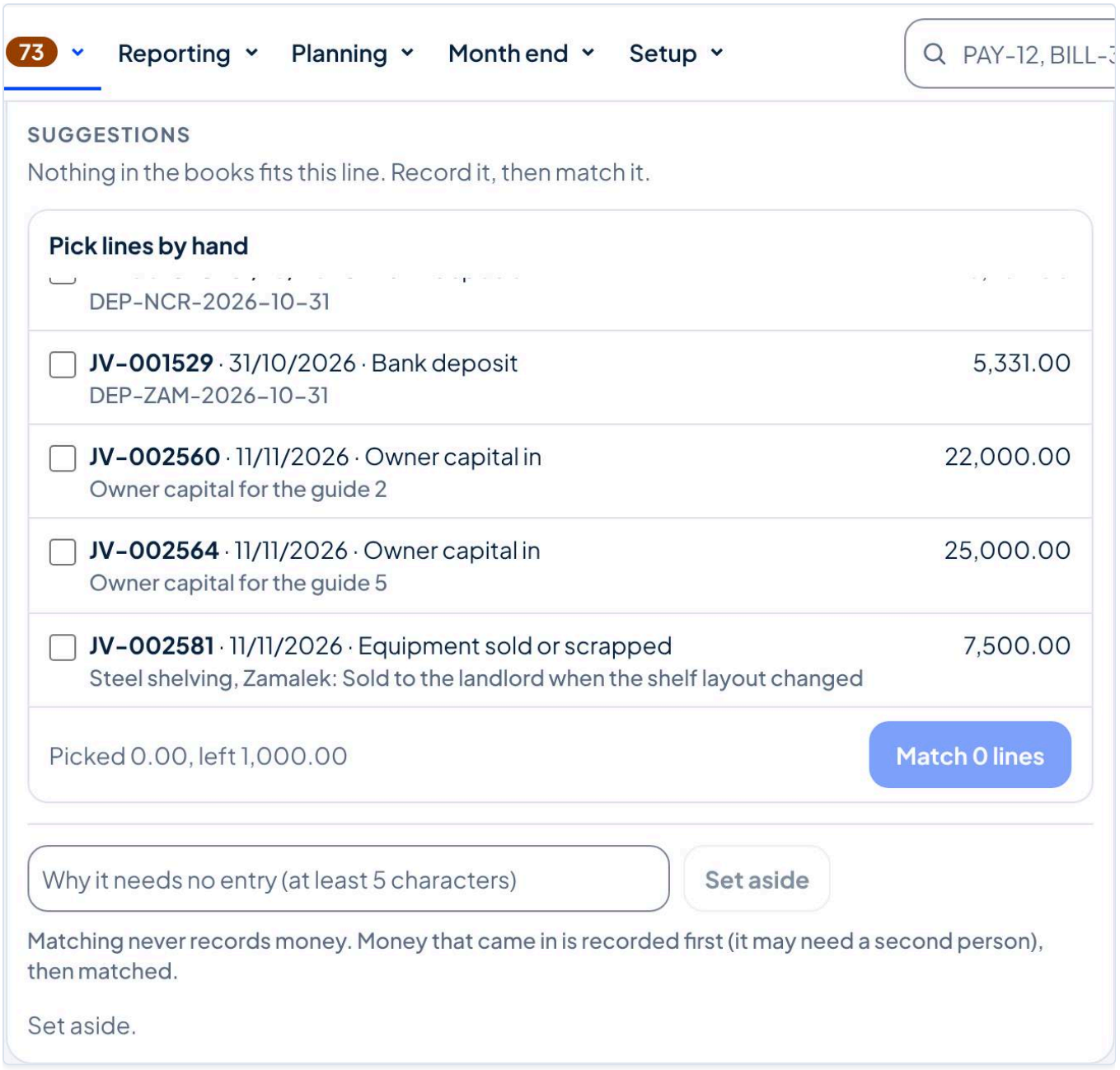
A money out line (−150.00 duplicate posting) with every action numbered: 1 the line, 2 suggestions that could be part of it (no button), 3 Pick lines by hand open, 4 a ticked line, 5 the running total in grey because it does not add up, 6 Match 1 line disabled, 7 Book as a bank charge, 8 a reason that is too short, 9 Set aside still disabled.



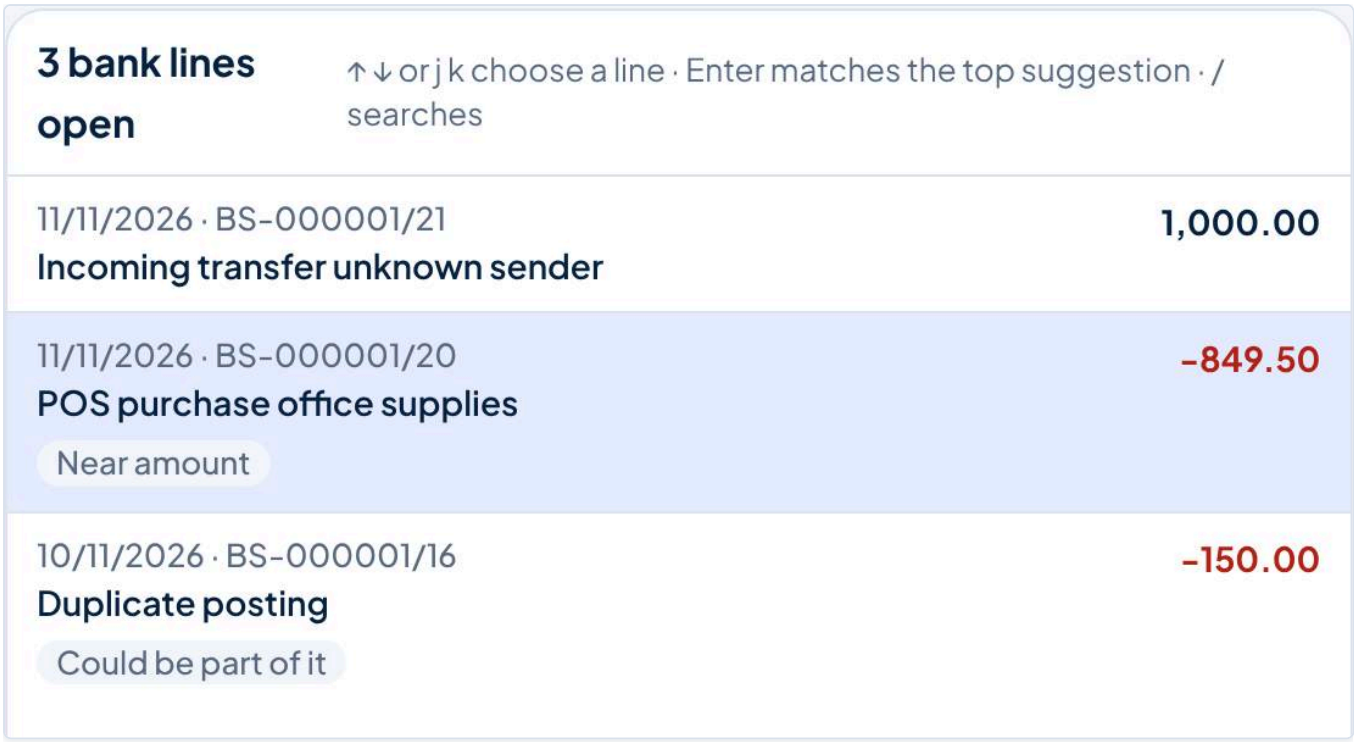
The 849.50 card purchase: the only close ledger line is 850.00, shown as "Near amount" with no Match button, because amounts must be exact.



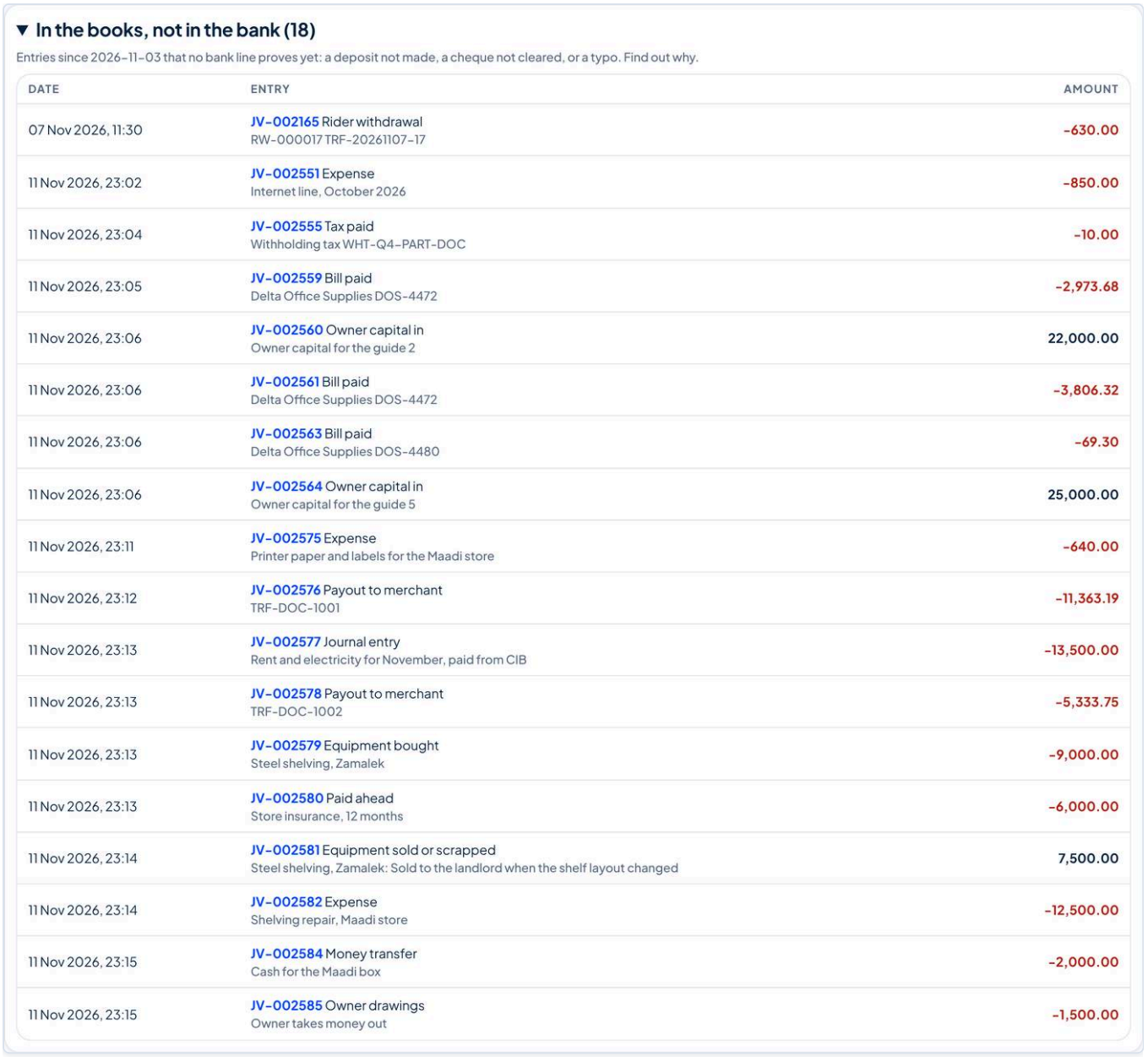
After Book as a bank charge on the 25.00 maintenance fee: "Booked as a bank charge and matched." (posted at once, below the approval limit).



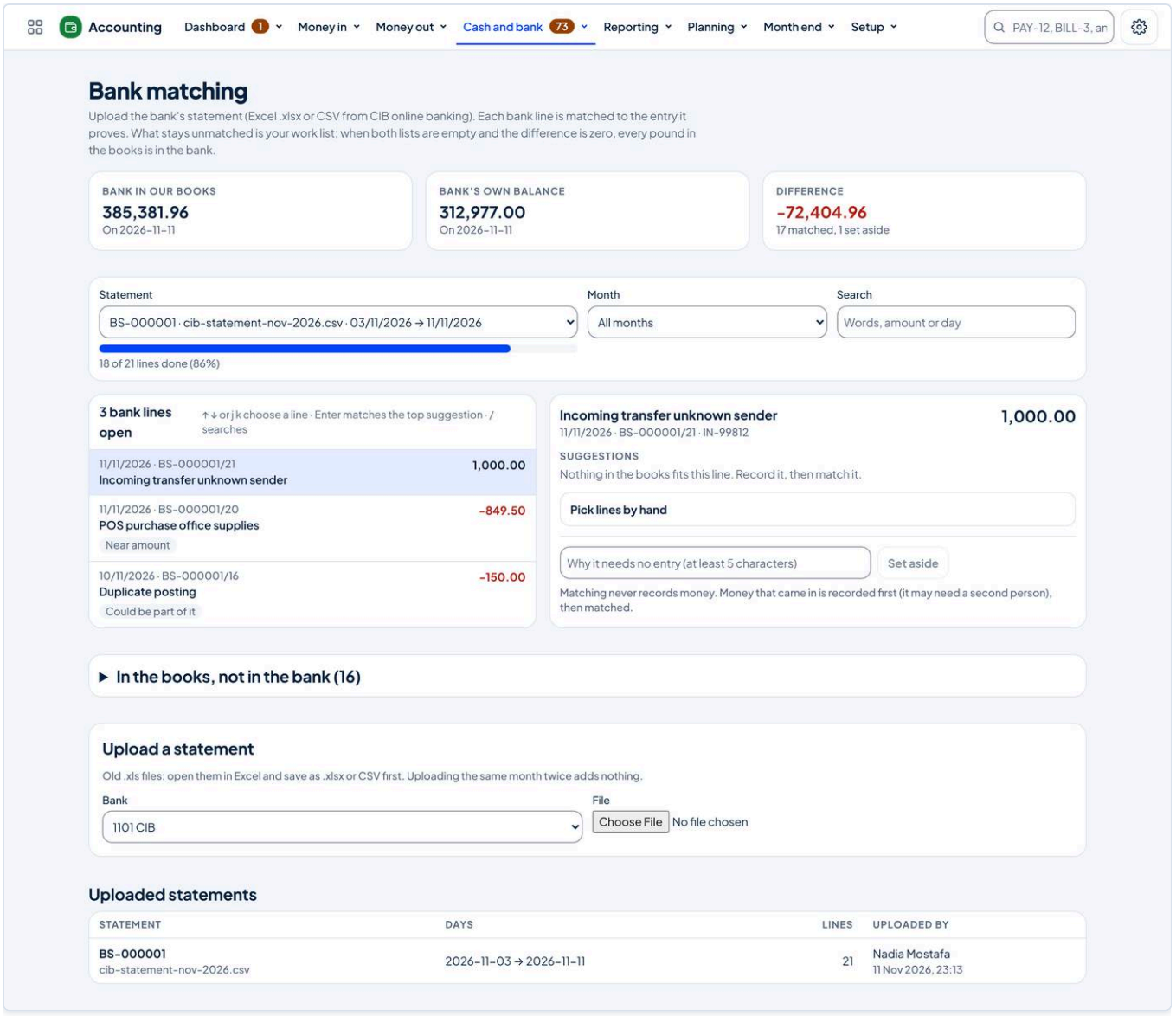
The 1,000.00 transfer from an unknown sender: "Nothing in the books fits this line. Record it, then match it." Money in is never created from a bank line.



The open list after pressing k: the selection moved up one line (light blue background).



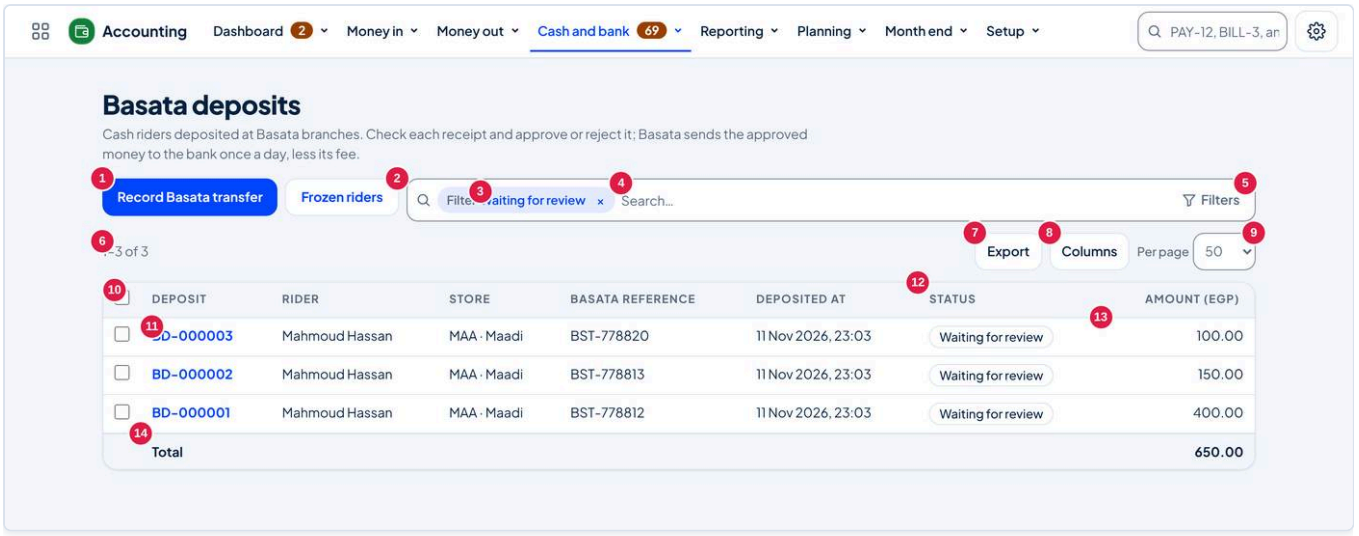
"In the books, not in the bank" unfolded: ledger lines on CIB since the statement's first day that no bank line proves yet.



The screen after the work: 18 of 21 lines done (86%), "17 matched, 1 set aside", three lines left open.

15.4 Basata deposits

Address `/finance/basata`. Menu: Cash and bank, Basata deposits. Opens with Accounting view; Record Basata transfer and the bulk Approve deposits need Accounting use. It is a standard list view (`AppListPage`, list `accounting/basata_deposits`, data `GET /lists/accounting/basata_deposits`; definition in `packages/shared/src/app-lists.ts`). It opens on the filter Waiting for review.

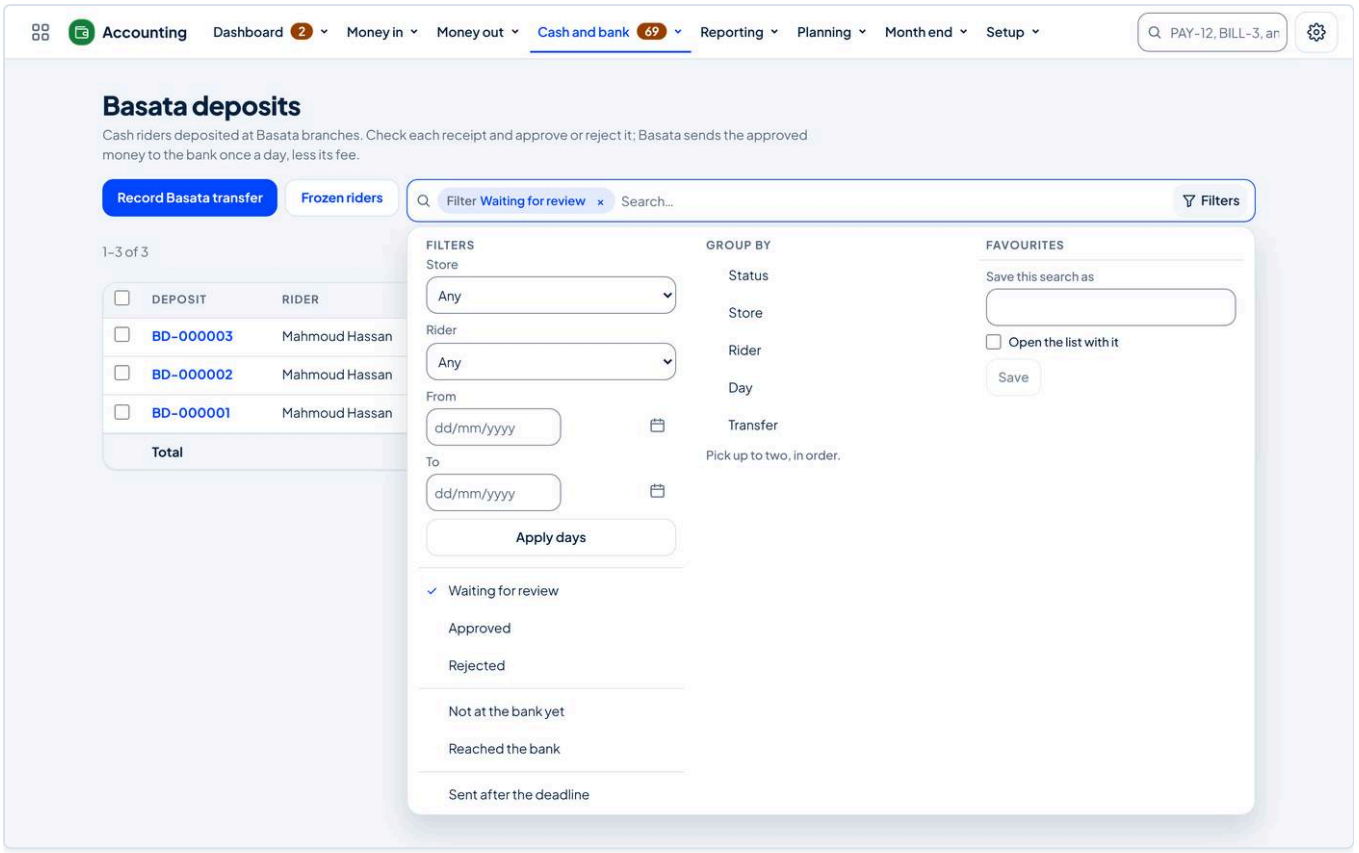


Basata deposits at /finance/basata, three deposits waiting for review.

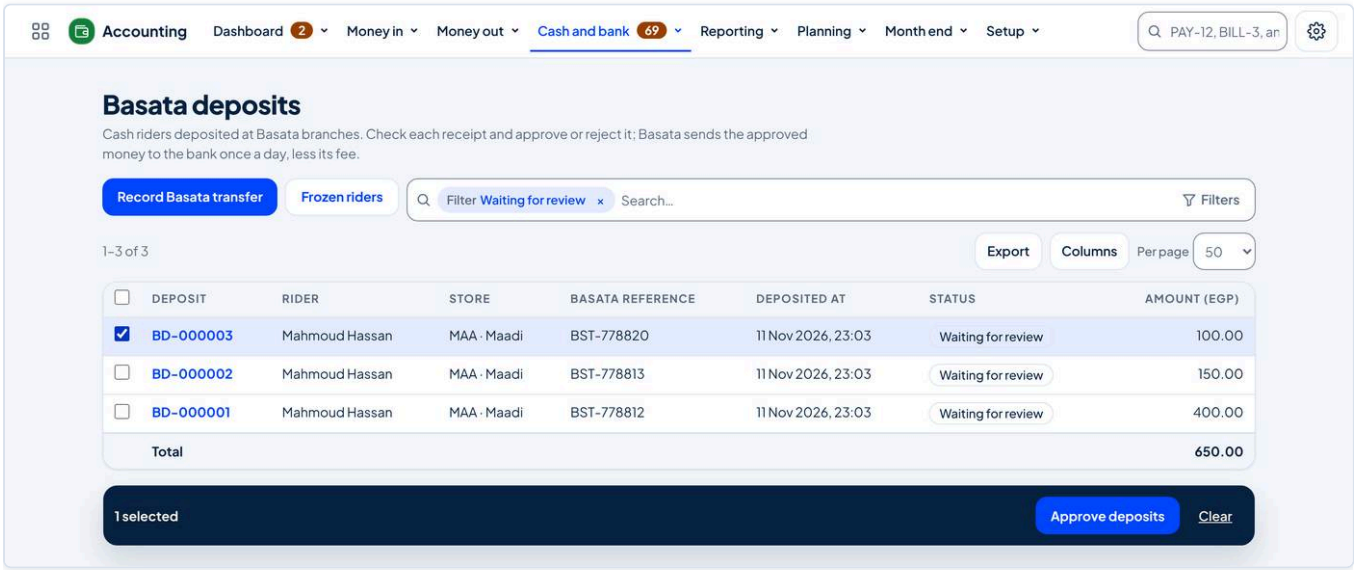
#	Part	What it shows	What clicking it does
1	Record Basata transfer	Blue filled button (Accounting use only).	Opens `/finance/basata/transfer`.
2	Frozen riders	White button with blue text.	Opens `/finance/frozen-riders`.
3	Filter chip	Light blue chip "Filter Waiting for review" with a cross.	The cross removes the filter (the list then shows every deposit).
4	Search	Typed search over number, rider, badge, store and reference.	Enter adds a search chip.
5	Filters	Funnel icon.	Opens the Filters, Group by and Favourites panel.
6	Count	"1–3 of 3".	Nothing.
7	Export	White button.	CSV or Excel of the passing rows (only the ticked rows when some are ticked).
8	Columns	White button.	Shows the optional columns: Badge, Sent at, Decided by, Reason, Transfer.
9	Per page	50 by default.	Changes the page size.
10	Select all	Header check box.	Ticks every row on the page and shows the bulk bar.
11	Deposit number	Blue BD number.	Opens the deposit page `/finance/basata/<id>`.
12	Status	Column of pills.	Sorting is by number, deposited, sent or amount (column headers with a sort key).
13	Status pill	Rounded pill with a thin grey border and dark text. The three statuses (Waiting for review, Approved, Rejected) use the same neutral look; there is no color per status in this list.	Nothing.
14	Total	Sum of Amount (EGP) over every passing row.	Nothing.

Controls on this screen

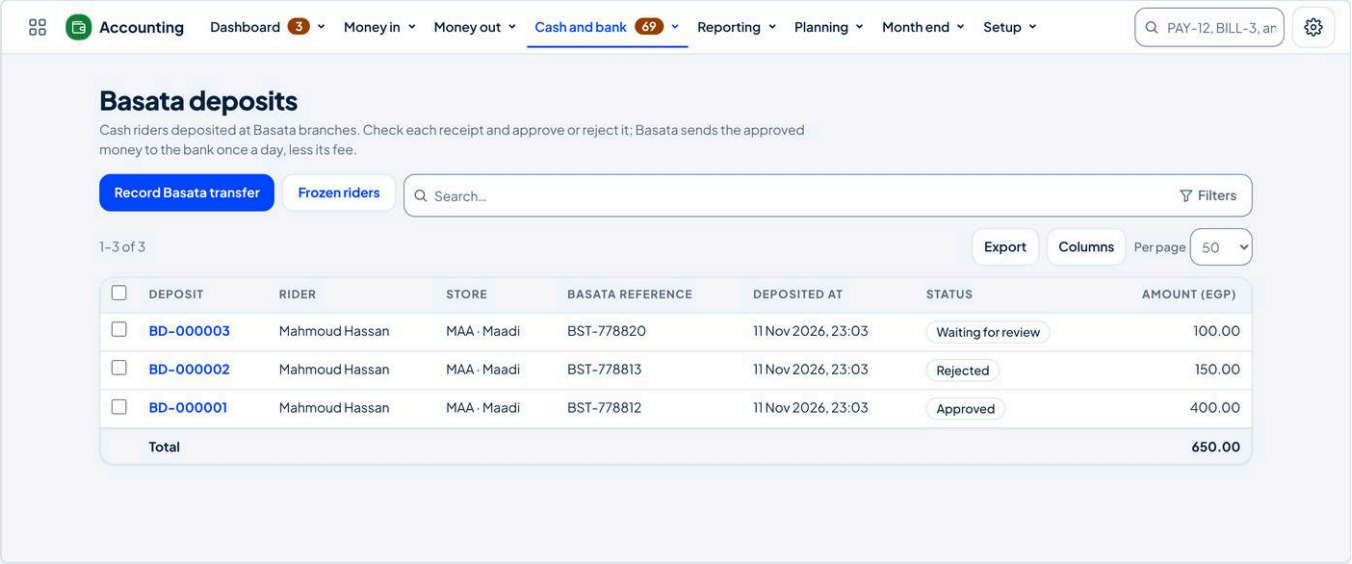
Control	What it does	Rule / limits	Result
Filters: Store, Rider	Pick lists (values `v=store:<id>`, `v=rider:<id>`)	Only stores and riders in the person's scope.	Applied at once, shown as a chip such as "Rider: Mahmoud Hassan (RDR-0002)".
Filters: From, To, Apply days	Deposited day range (dd/mm/yyyy)	Cairo days.	Chip with the days.
Filters: Waiting for review, Approved, Rejected	Status group	OR inside the group.	
Filters: Not at the bank yet, Reached the bank	Settled group: whether a Basata transfer (BT) covers the deposit		
Filters: Sent after the deadline	Timing	Deposits sent after their deadline day's hour.	
Filters: Today ... Last month	Date presets		
Group by	Status, Store, Rider, Day, Transfer	Up to two levels, in order.	Group rows with counts and amount subtotals.
Favourites	Save this search as, Open the list with it, Save	Per person.	Saved view in the Favourites list.
Row check boxes, Approve deposits, Clear	`POST /finance/basata/deposits/bulk-approve`	Accounting use; each deposit checked on its own (scope, not the person's own deposit, still pending, not more than the rider's cash).	Navy bulk bar "1 selected" with a blue Approve deposits button; per row results.



The Filters panel of Basata deposits: values (Store, Rider, From, To and Apply days), the status, settled and timing filters, Group by and Favourites.



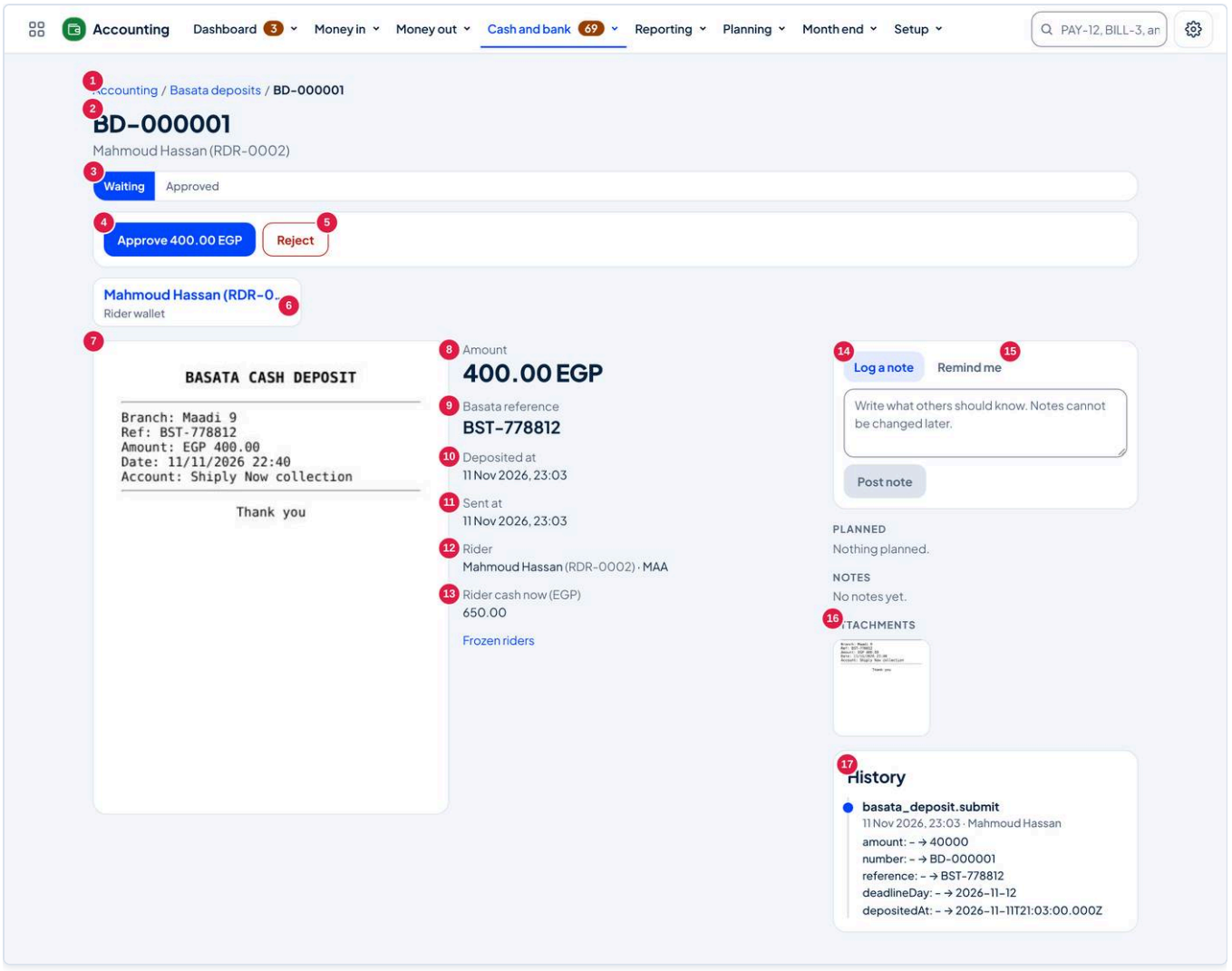
One row ticked: the row turns light blue and the navy bulk bar offers Approve deposits and Clear.



With the filter removed: one deposit of each status, Waiting for review, Rejected and Approved, all in the same neutral pill.

15.5 A Basata deposit

Address `/finance/basata/<id>`, opened from the list. Opens with Accounting view (the deposit's store must be in scope); Approve and Reject need Accounting use and show only while the deposit is waiting. Built on `DocumentPage` with `GET /finance/documents/basata_deposit/:id` and `GET /finance/basata/deposits/:id`; the buttons are `BasataActions`.



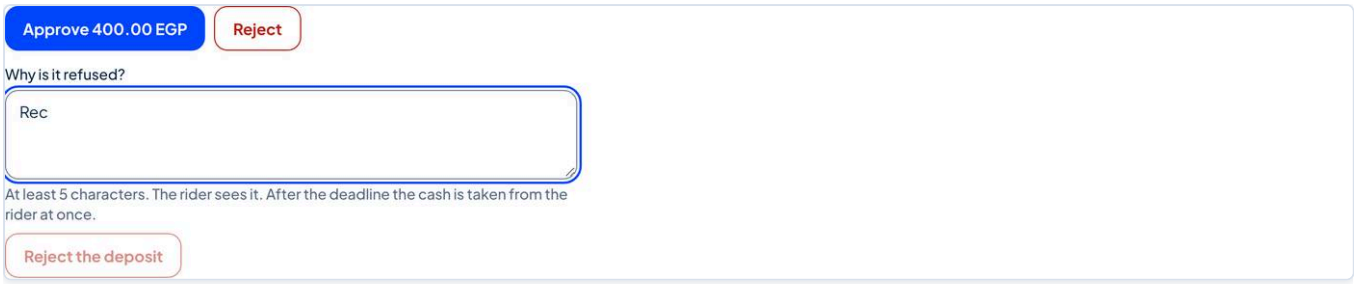
Deposit BD-000001 waiting for review, with the receipt photo on the left, the rider's figures in the middle and the chatter on the right.

#	Part	What it shows	What clicking it does
1	Breadcrumbs	Accounting / Basata deposits / BD-000001.	Back to the dashboard or to the list with its filters.
2	Title	Deposit number and, under it, the rider and badge.	Nothing.
3	Status bar	Waiting then Approved; the current step is a blue filled tab, a passed step has a tick; Rejected appears as a red filled tab after the other two.	Nothing.
4	Approve N EGP	Blue filled button with the amount.	Approves; see controls.
5	Reject	White button with red border and text.	Opens the reason box.
6	Rider wallet	Smart button with the rider name (cut with an ellipsis) and "Rider wallet". After approval a second smart button "JV-002570 Journal entry" appears.	Opens the rider's wallet page or the entry.
7	Receipt photo	The rider's photo, large.	Opens the photo in a new tab at full size.
8	Amount	Large "400.00 EGP".	Nothing.

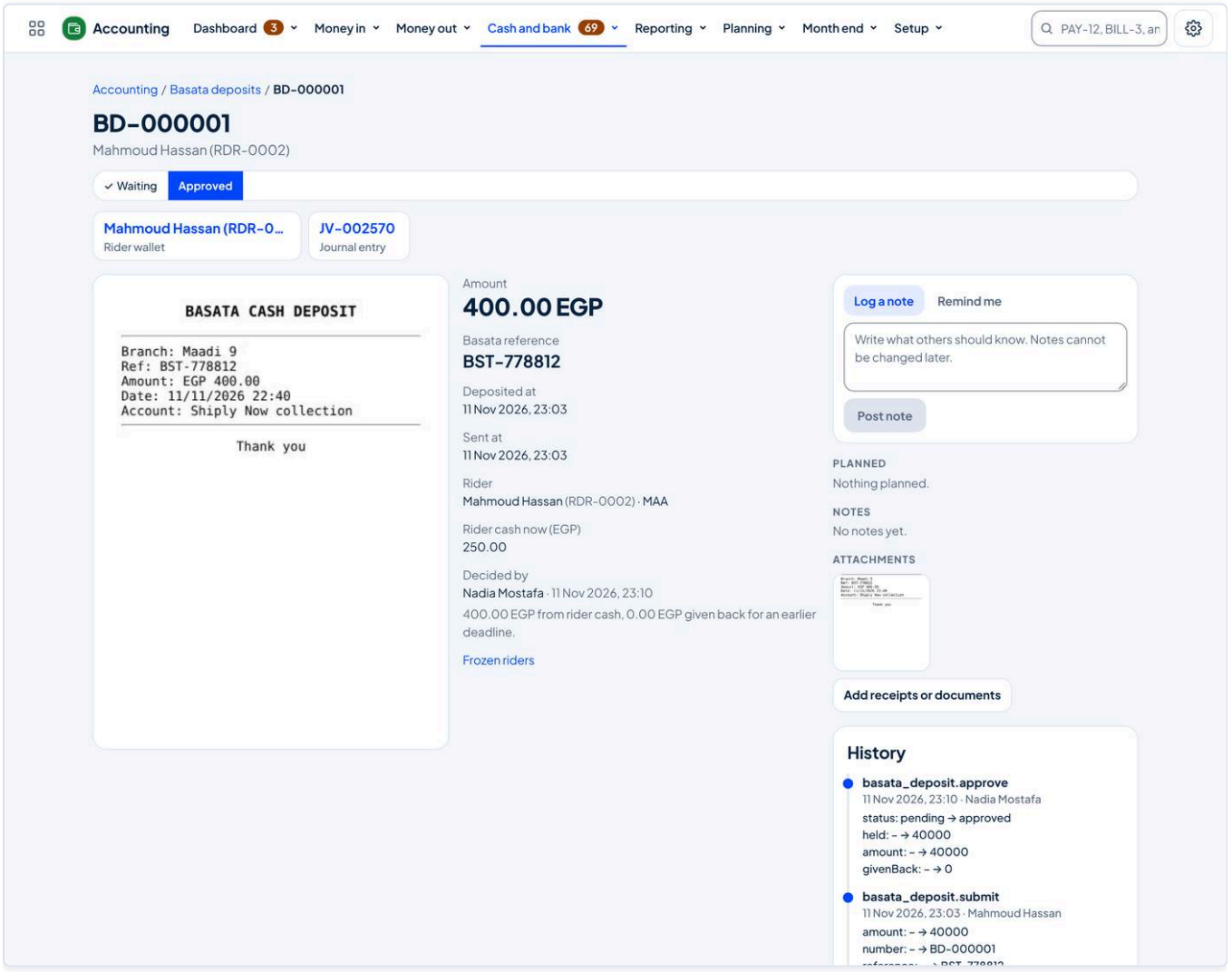
#	Part	What it shows	What clicking it does
9	Basata reference	As typed by the rider.	Nothing.
10	Deposited at	The time the rider gave.	Nothing.
11	Sent at	When the rider sent it.	Nothing.
12	Rider	Name, badge and store code; a bold red "Frozen" when the rider is frozen.	Nothing.
13	Rider cash now (EGP)	The rider's 1300 balance now. "Taken at a deadline, not given back (EGP)" in red appears under it when a deadline took cash.	Nothing.
14	Log a note	Chatter tab: note box and Post note (notes cannot be changed later).	Adds a note to the record.
15	Remind me	Chatter tab: a day, a note and a person.	Plans an activity; listed under Planned.
16	Attachments	The receipt thumbnail; after a decision an "Add receipts or documents" button appears.	A thumbnail opens the file in a new browser tab.
17	History	Audit rows newest first, with the action code (`basata_deposit.submit` , `basata_deposit.approve`) and the changed values (amounts in piastres).	Nothing.

Controls on this screen

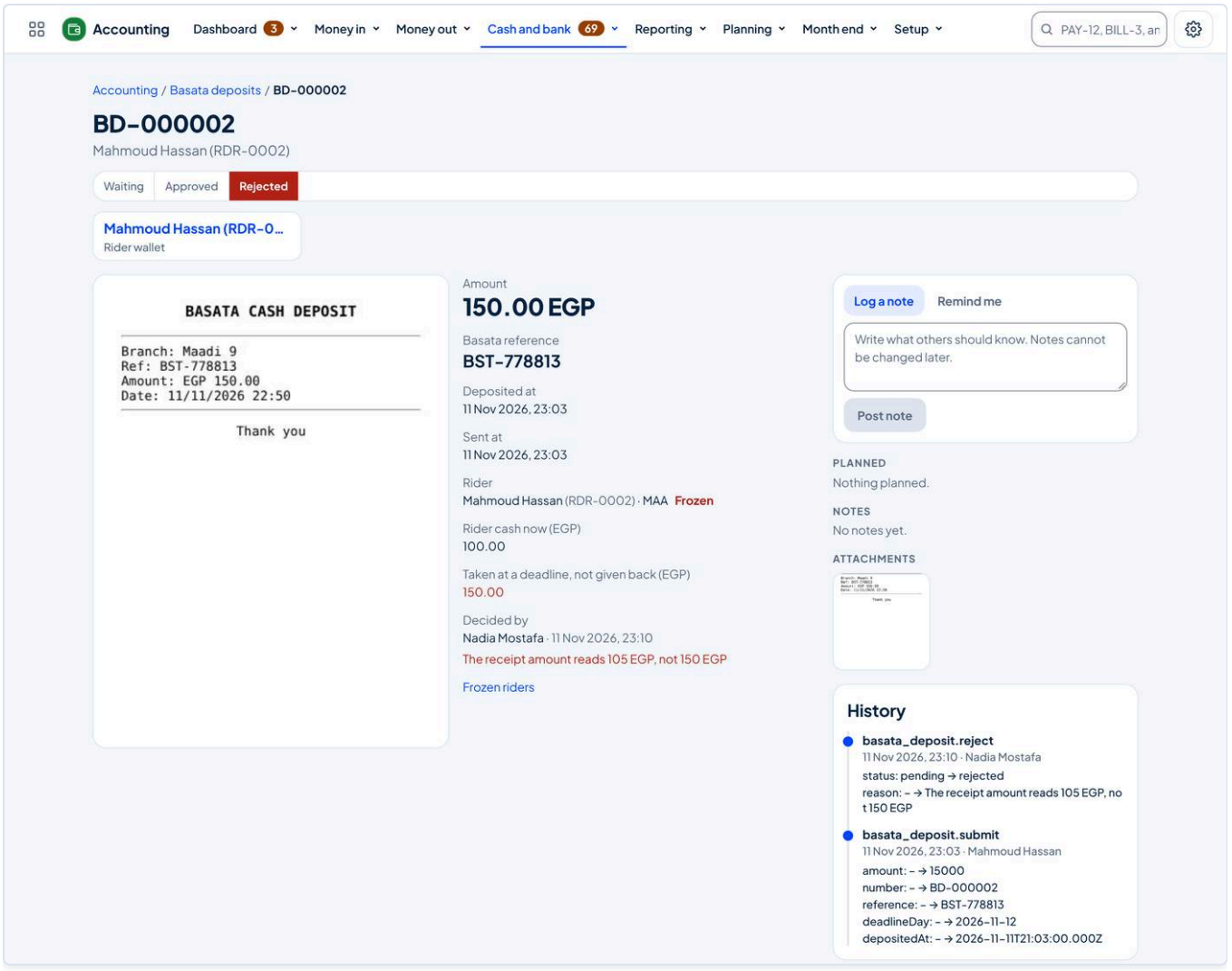
Control	What it does	Rule / limits	Result
Approve N EGP	`POST /finance/basata/deposits/:id/approve`	Accounting use; never the rider himself ("You cannot decide your own deposit." replaces the buttons); only waiting (`deposit_not_pending`); amount not above the rider's cash plus what a deadline took (`more_than_cash`); books lock.	"Approved." Entry Dr Money at Basata (1205), Cr 1300 rider cash; any part beyond what he still held gives back earlier deadline deductions, oldest first. The page shows "400.00 EGP from rider cash, 0.00 EGP given back for an earlier deadline."
Reject, Why is it refused?, Reject the deposit	`POST /finance/basata/deposits/:id/reject`	Reason 5 to 300 characters; the confirm button stays pale until then. The hint says: "At least 5 characters. The rider sees it. After the deadline the cash is taken from the rider at once."	"Rejected." The reason shows in red. When the deposit's deadline has passed, the deposit amount is taken from the rider (to his office wallet, else his 1350 dues) and he is frozen.



Reject opened with a 3 character reason: Reject the deposit stays disabled (pale red).



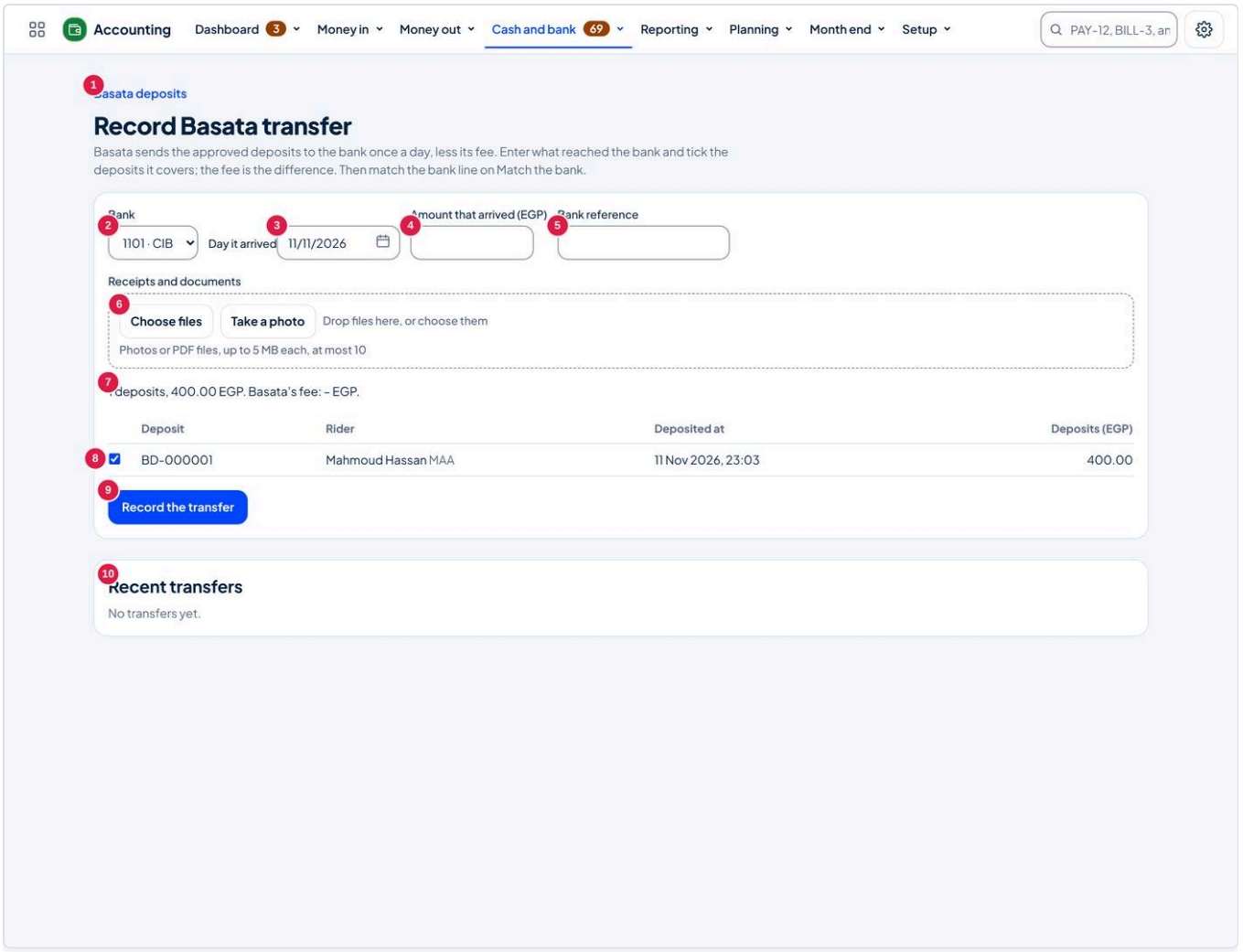
After Approve: status bar on Approved with Waiting ticked, a journal entry smart button, Decided by with the split of the amount, and the approval in History.



BD-000002 rejected after its deadline: red Rejected tab, the reason in red, the rider tagged Frozen and 150.00 taken at the deadline. (For this figure the deposit's deadline day was set to the day before, so the rejection fell after its deadline.)

15.6 Record Basata transfer

Address `/finance/basata/transfer`. Reached from the button on Basata deposits (no menu entry). Needs Accounting use to save. Data `GET /finance/basata/settlements/new`; component `BasataSettlementForm`.



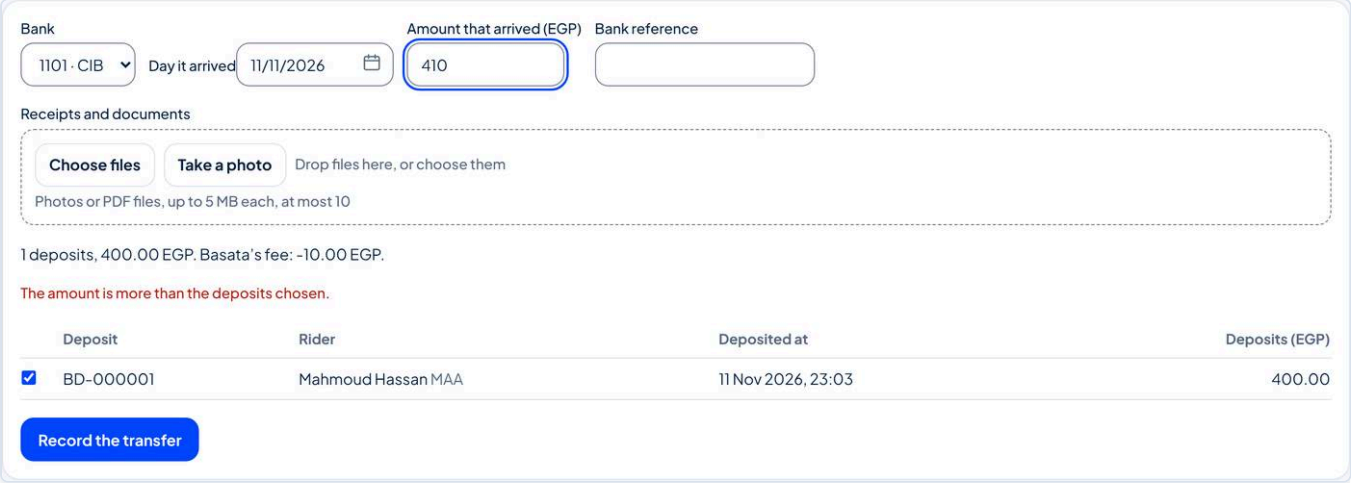
Record Basata transfer with one approved deposit waiting to be covered.

#	Part	What it shows	What clicking it does
1	Basata deposits	Blue back link.	Back to the list.
2	Bank	EGP bank accounts of group 1100, CIB first.	Chooses where the money arrived.
3	Day it arrived	Day first date box, today by default, never in the future.	Calendar icon opens the date picker.
4	Amount that arrived (EGP)	What Basata actually transferred.	Typing updates the fee line.
5	Bank reference	Optional, up to 100 characters.	Typing.
6	Receipts and documents	Attachment field (the transfer advice).	Upload.
7	Summary	"1 deposits, 400.00 EGP. Basata's fee: 5.00 EGP." (a dash before an amount is typed).	Nothing.
8	Deposit check boxes	Every approved deposit not yet settled, all ticked at first: number, rider and store code, deposited at, amount.	Untick deposits the transfer did not cover.
9	Record the transfer	Blue button, pale when no deposit is ticked or no bank.	Saves; see controls.

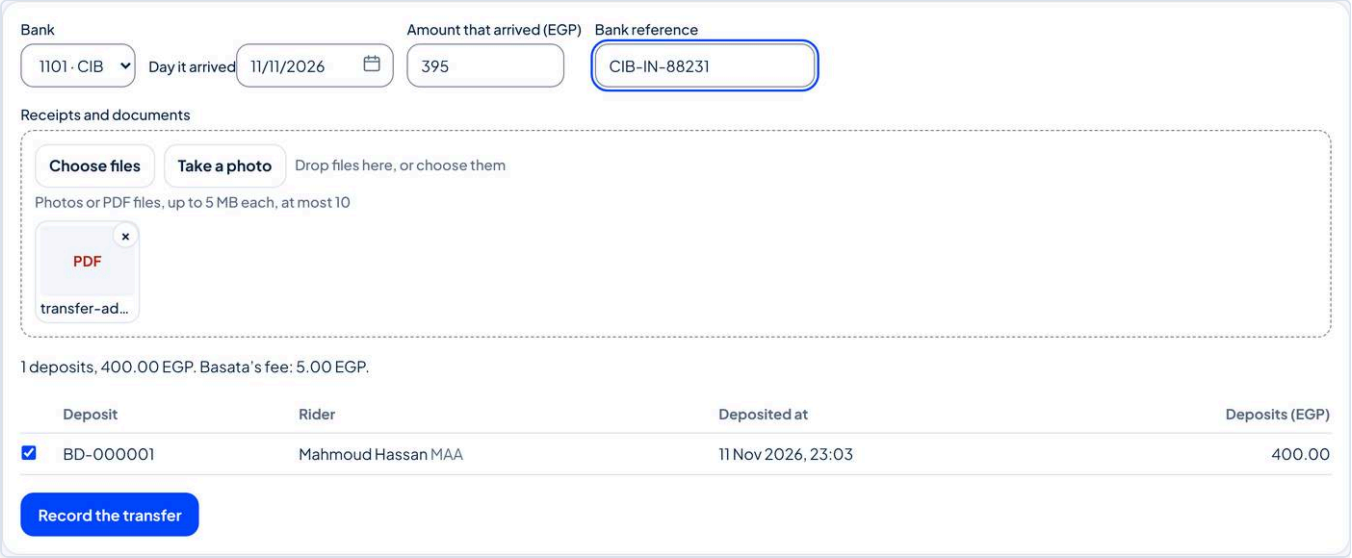
#	Part	What it shows	What clicking it does
10	Recent transfers	Transfer (BT number, links to its journal entry), Day it arrived (ISO form), Bank, Deposits, Amount that arrived, Fee. Empty: "No transfers yet."	The BT number opens the entry.

Controls on this screen

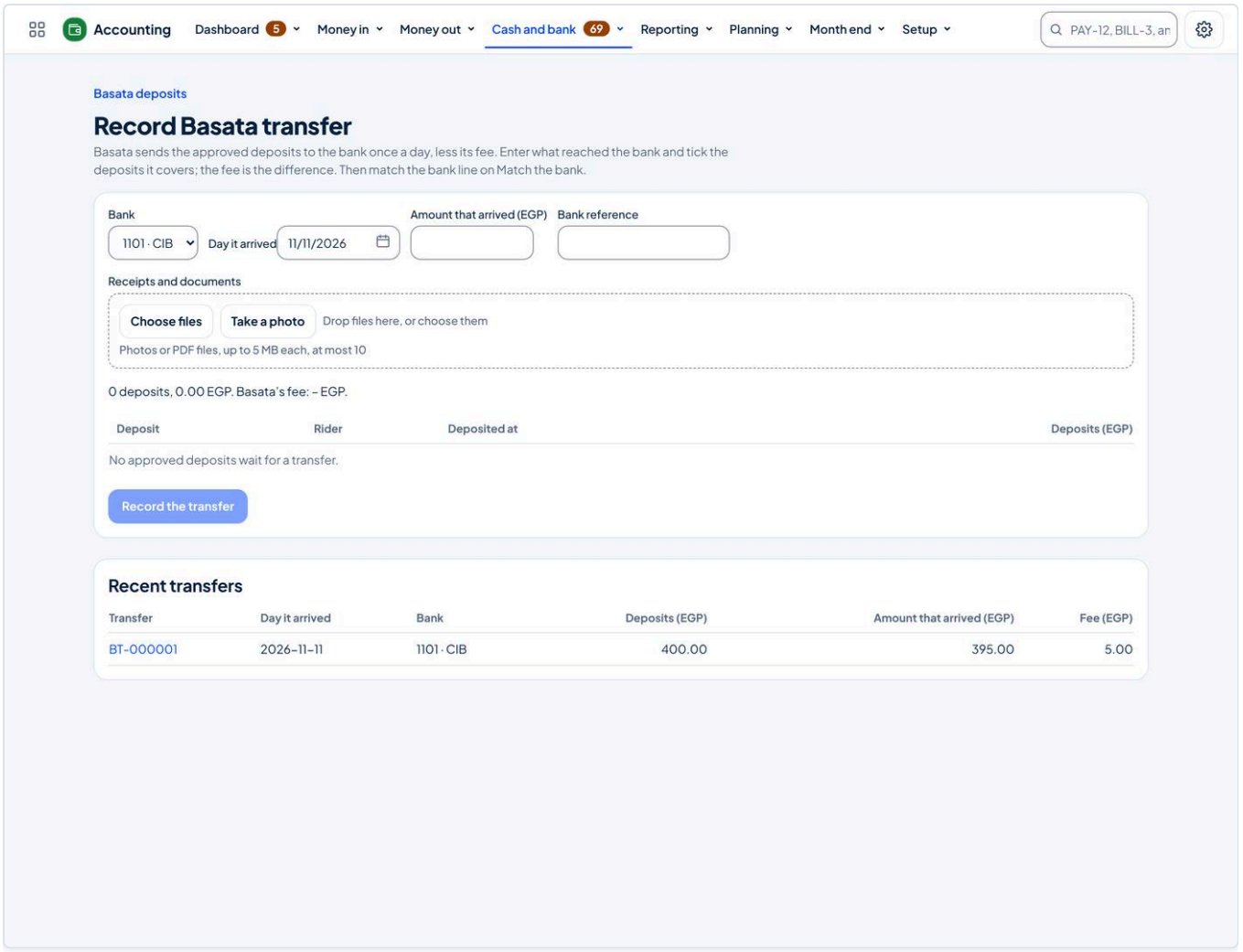
Control	What it does	Rule / limits	Result
Record the transfer	`POST /finance/basata/settlements`	Amount above zero ("Enter the amount that reached the bank."); never more than the ticked deposits ("The amount is more than the deposits chosen.", shown as soon as typed); only approved and unsettled deposits; day not in the future; books lock; no second person (money already ours moving between our accounts).	"Recorded." Entry Dr bank 395.00, Dr Basata fees 5.00, Cr Money at Basata 400.00; transfer BT-000001; the deposits become Reached the bank. The bank line is then matched on Bank matching.



An amount above the deposits: the fee reads −10.00 and the red message "The amount is more than the deposits chosen." appears.



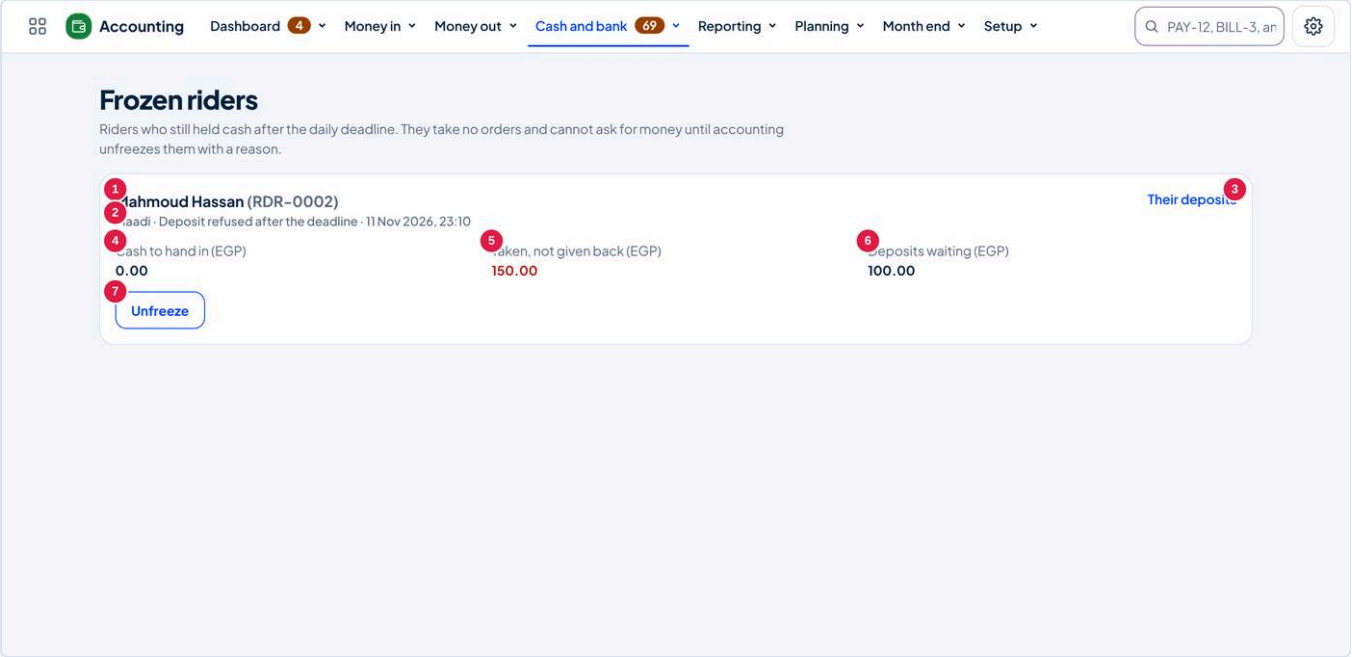
Filled: 395 EGP arrived at CIB with reference CIB-IN-88231 and the transfer advice PDF; the fee is 5.00 EGP.



After saving: "No approved deposits wait for a transfer.", the button is pale, and BT-000001 is listed under Recent transfers.

15.7 Frozen riders

Address `/finance/frozen-riders`. Menu: Cash and bank, Frozen riders (also a button on Basata deposits and a link on each deposit page). Opens with Accounting view; Unfreeze needs Accounting use (a store manager cannot unfreeze). Data `GET /finance/frozen-riders`; component `FrozenRiders`.



One frozen rider after his deposit was refused after the deadline.

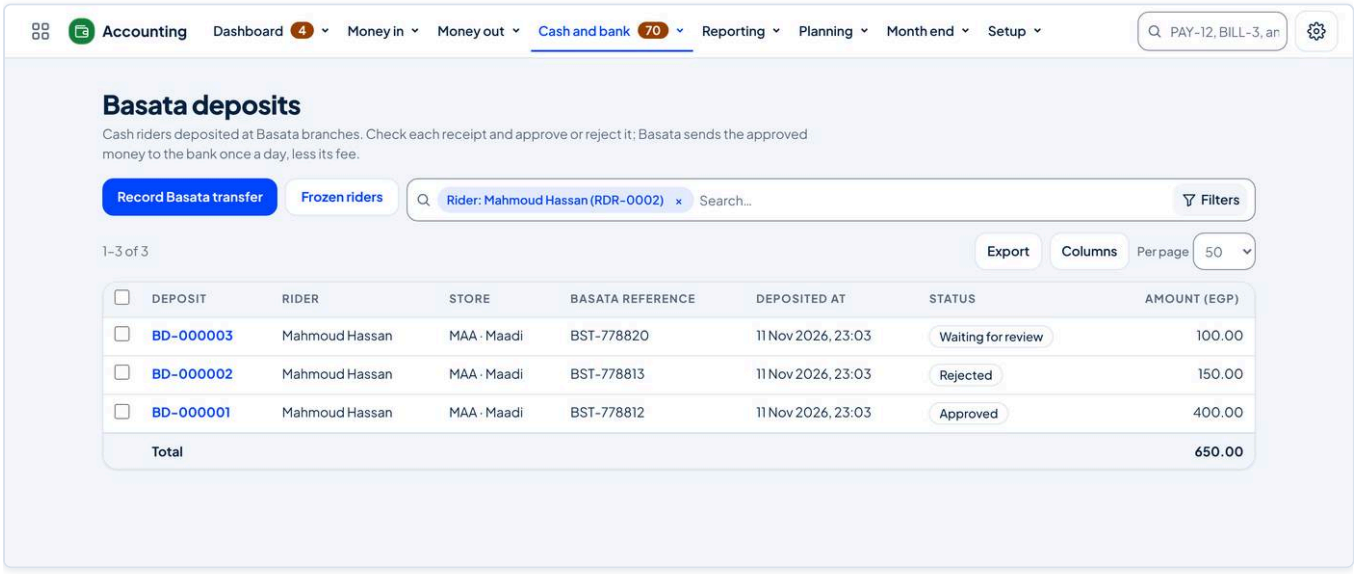
#	Part	What it shows	What clicking it does
1	Rider	Bold name and grey badge.	Nothing.
2	Why and when	Store · reason · time frozen. Reasons: "Cash held after the deadline" or "Deposit refused after the deadline". "Office: name" appears for a rider supplied by an office.	Nothing.
3	Their deposits	Blue link, top right.	Opens Basata deposits filtered on this rider (all statuses).
4	Cash to hand in (EGP)	What he still holds.	Nothing.
5	Taken, not given back (EGP)	Red: what the deadline took and no later deposit gave back.	Nothing.
6	Deposits waiting (EGP)	His Basata deposits waiting for review.	Nothing.
7	Unfreeze	Button with blue border and text.	Opens the reason box with Unfreeze the rider and Cancel.

Controls on this screen

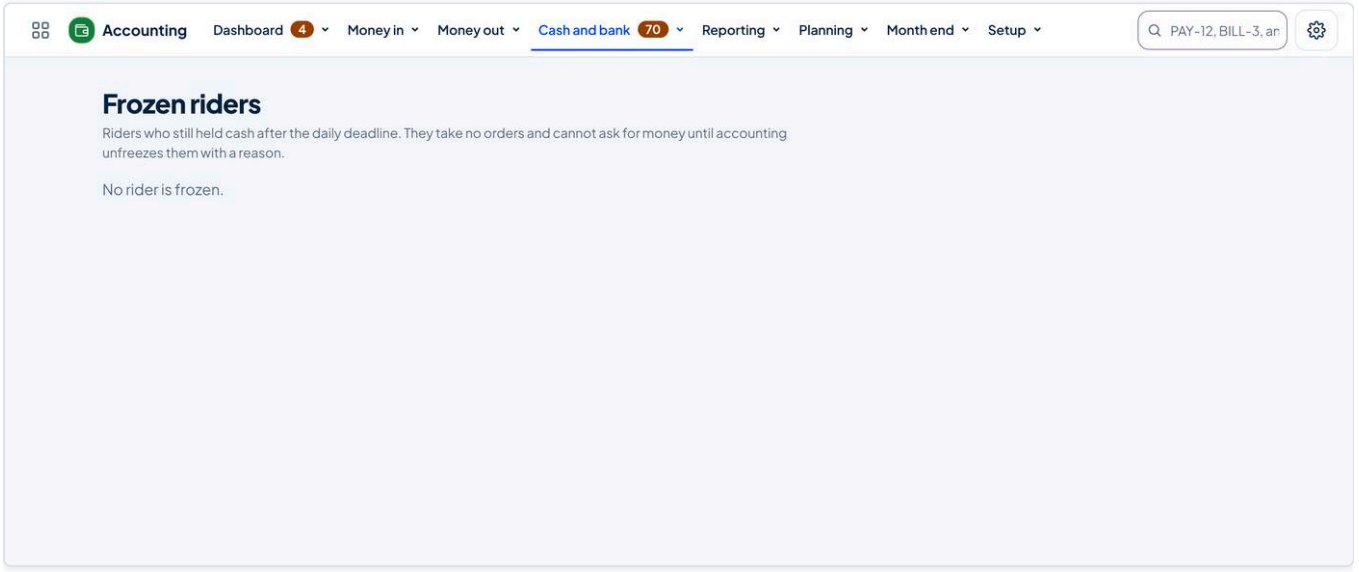
Control	What it does	Rule / limits	Result
Unfreeze, reason, Unfreeze the rider	`POST /finance/frozen-riders/:riderId/unfreeze`	Reason 5 to 300 characters ("Why may the rider work again? (at least 5 characters)"); the rider must be frozen (`rider_not_frozen`); his store in scope. Audited `rider.unfreeze` , kept in the freeze history.	The rider may take orders and ask for money again. The money taken is not given back by unfreezing; only a later approved deposit gives it back.

Control	What it does	Rule / limits	Result
Cancel	Closes the reason box		

The reason box opened and filled.



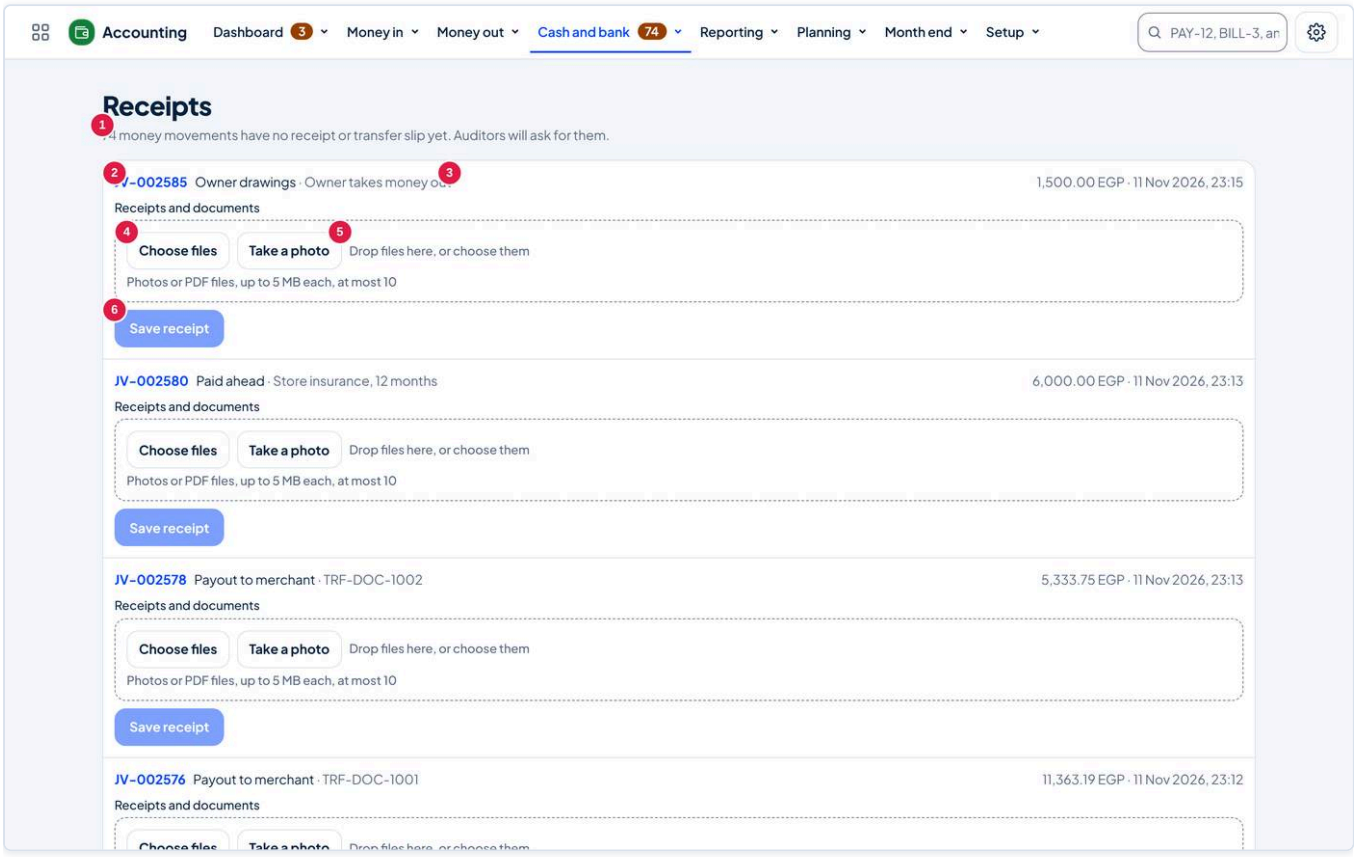
Their deposits: the Basata list opened on the rider value chip with every status.



After unfreezing the only frozen rider: the empty state "No rider is frozen." (The confirmation "The rider can work again." is not shown when the list becomes empty.)

15.8 Missing receipts

Address `/finance/receipts`, page title **Receipts**. Menu: Cash and bank, Missing receipts. Opens with Accounting view; Save receipt needs Accounting use. Data `GET /finance/attachments/missing` (function `missingReceipts`): money movements of the kinds listed in `RECEIPT_KINDS` (expense, bank deposit, money transfer, capital in and out, other income, payouts, merchant receipts, bill and tax payments, prepayment, equipment, staff advance and bonus, petty cash, expense claim, Basata deposit, cheques, security deposits, loan received and loan payment) that have no receipt on the entry, on its approval request or on its payout. The newest 200 are listed.



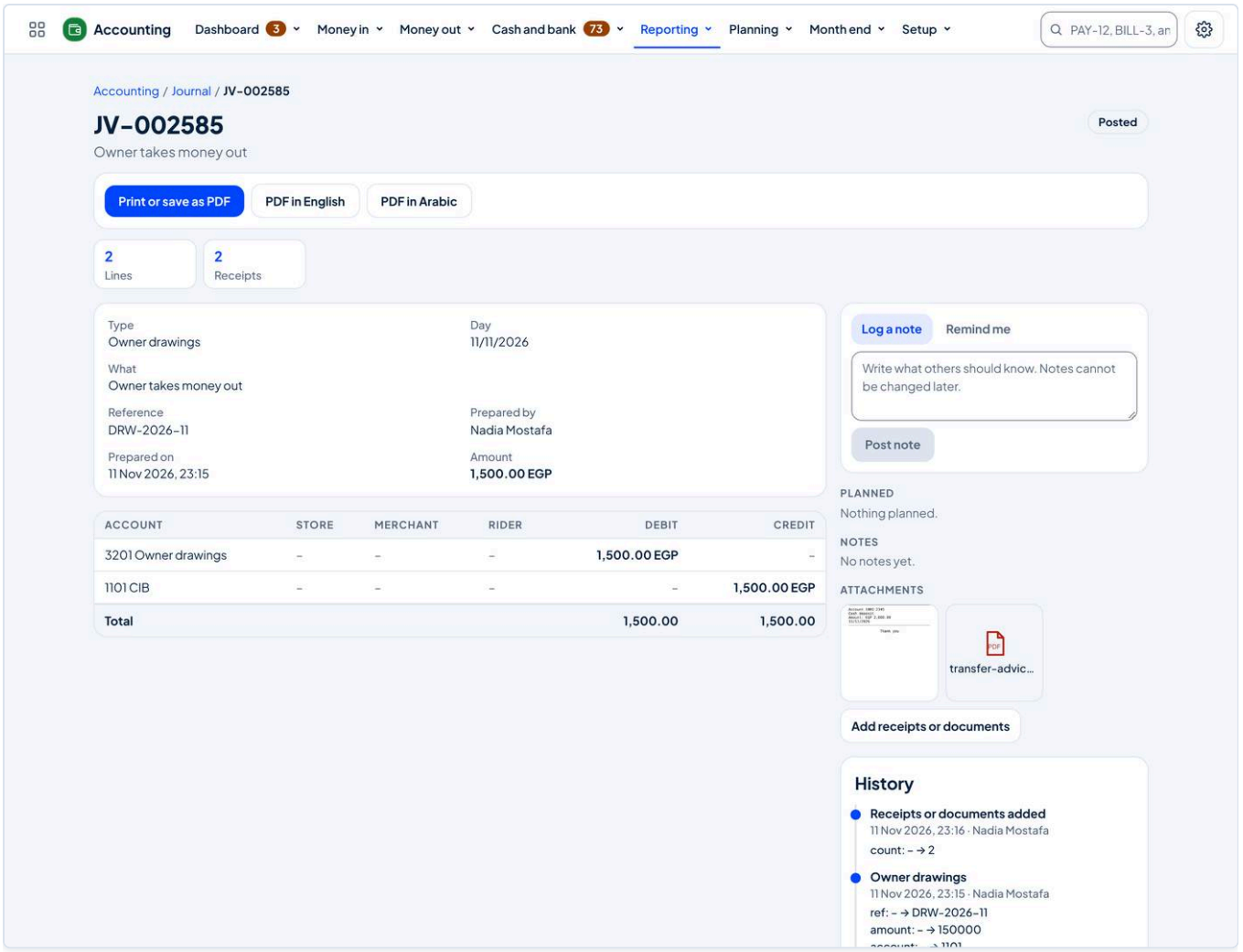
Missing receipts with the count in the intro and one row per entry.

#	Part	What it shows	What clicking it does
1	Count	"74 money movements have no receipt or transfer slip yet. Auditors will ask for them." Empty state: "Every money movement has its receipt."	Nothing.
2	Entry number	Blue JV number, entry kind name and memo in grey.	Opens the journal entry page.
3	Amount and time	Amount in EGP (sum of the debit lines) and when it was booked, on the right.	Nothing.
4	Choose files	Attachment field (photos and PDF, 5 MB each, at most 10).	File picker; drop also works.
5	Take a photo	Opens the phone camera.	Adds the photo.
6	Save receipt	Blue button, pale until at least one file has uploaded.	Links the files to the entry.

Controls on this screen

Control	What it does	Rule / limits	Result
Choose files or Take a photo	Uploads each file at once to `POST /finance/files`	Photos are shrunk in the browser; PDF up to 5 MB; other types refused ("name is not a photo or a PDF file."); more than 10 refused ("At most 10 files.").	Thumbnails (PDF shows a red PDF tile), each with a cross that deletes the upload.
Save receipt	`POST /finance/attachments` with `entryId` and `photoIds`	Accounting use; the entry must exist.	The files are linked in one transaction; History on the entry shows "Receipts or documents added, count 2". The row stays until the page is reloaded, then leaves the list.

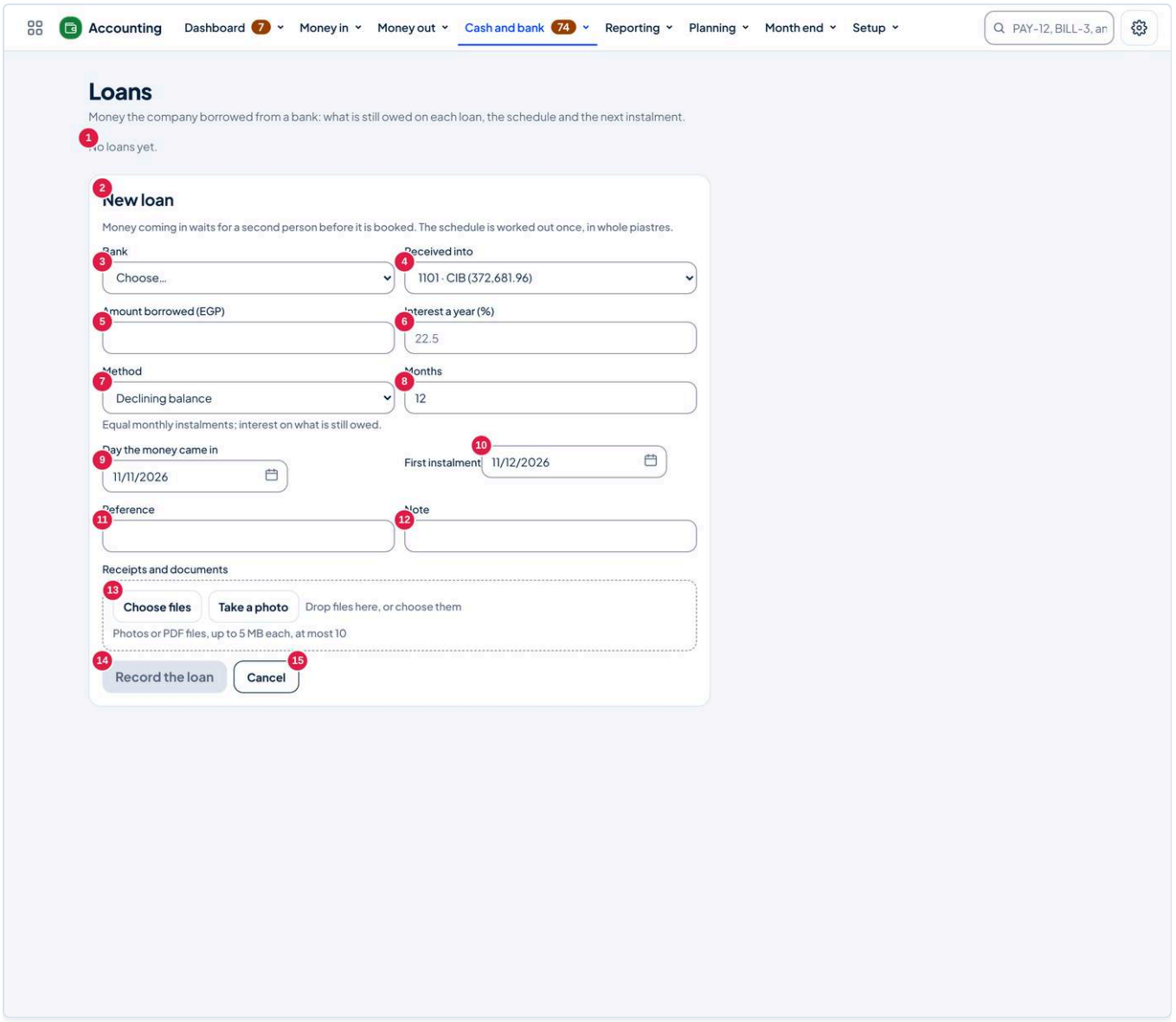
A photo and a PDF chosen for JV-002585, Save receipt enabled.



The journal entry after saving: the Receipts smart button counts 2, both files show under Attachments (a thumbnail and a PDF tile), and History records the addition. Clicking a file opens it in a new browser tab; there is no viewer inside the page.

15.9 Loans

Address `/finance/loans`. Menu: Cash and bank, Loans. Opens with Accounting view; the New loan form is shown only with Accounting use. Data `GET /finance/loans`; components `LoansTable` and `NewLoan` (`LoansFx.tsx`). The calculation of the schedule is in chapter 12, Loans.



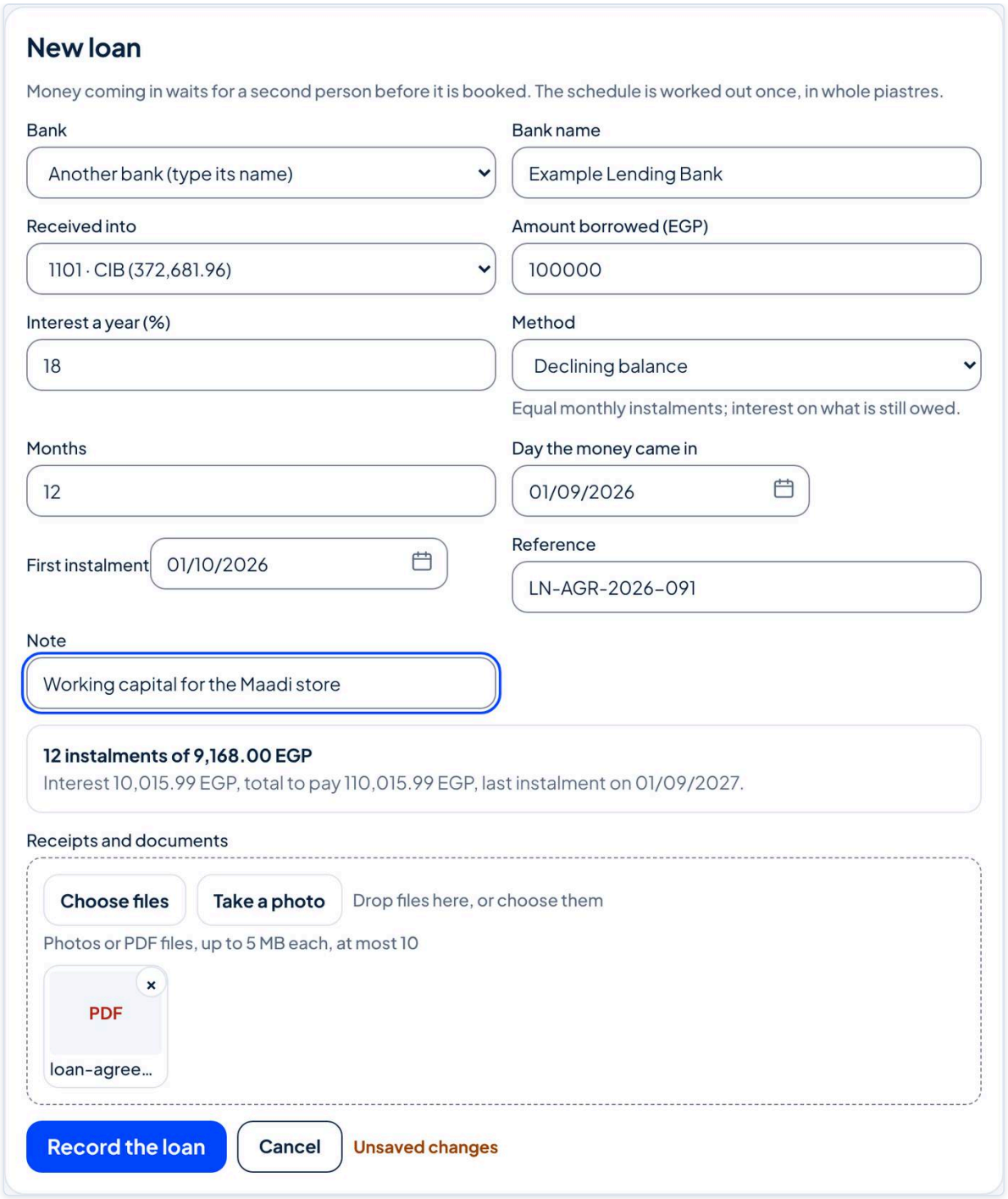
Loans before the first loan: "No loans yet." and the empty New loan form, every field numbered.

#	Part	What it shows	What clicking it does
1	Empty state	"No loans yet."; replaced by the loans table once a loan exists.	Nothing.
2	New loan	Section with the hint "Money coming in waits for a second person before it is booked. The schedule is worked out once, in whole piastres."	Nothing.
3	Bank	Choose…, then the 19 Egyptian banks of `EGYPT_BANKS` (names in the page language), then "Another bank (type its name)".	Another bank adds a Bank name box beside it.
4	Received into	Our open EGP bank accounts (code 11××) with their balance, CIB 1101 chosen by default.	Chooses where the money came in.
5	Amount borrowed (EGP)	Pounds and piastres, read without floats.	Typing.
6	Interest a year (%)	Placeholder 22.5; stored as basis points.	Typing.

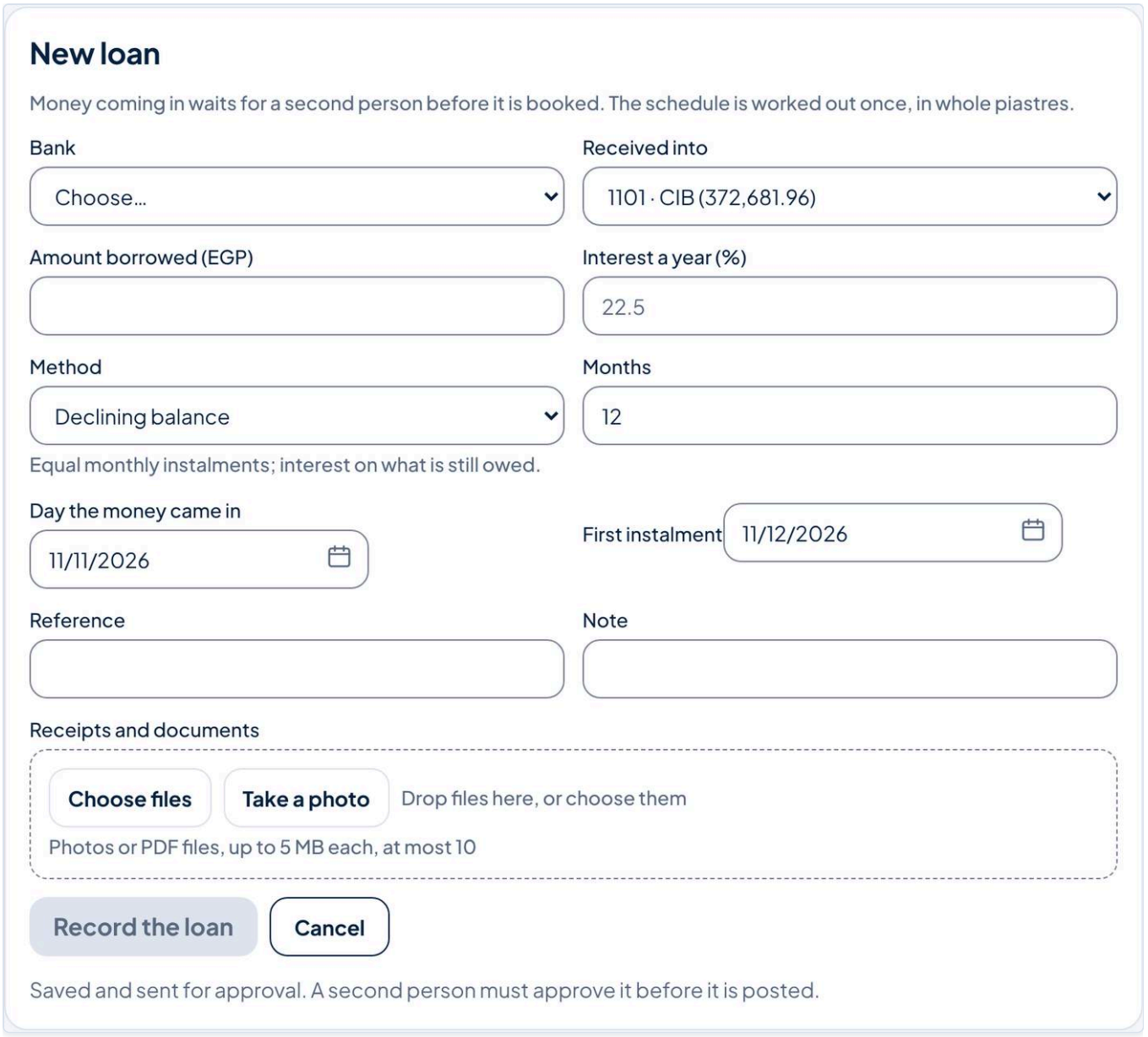
#	Part	What it shows	What clicking it does
7	Method	Declining balance (hint "Equal monthly instalments; interest on what is still owed.") or Flat ("Interest on the whole amount every month; principal in equal parts.").	Changes the method and its hint.
8	Months	12 by default, 1 to 600.	Typing.
9	Day the money came in	Day first date box, today by default, never in the future.	Date picker.
10	First instalment	30 days after today by default; must be after the day the money came in.	Date picker.
11	Reference	At least 2 characters.	Typing.
12	Note	Optional, up to 300 characters.	Typing.
13	Receipts and documents	The loan agreement or bank advice.	Upload.
14	Record the loan	Blue button, grey until the bank name, a valid preview, Received into and a reference are present.	Sends the loan for approval.
15	Cancel	White button; an orange "Unsaved changes" appears beside the buttons when anything was typed.	Clears the form (asks first when there are changes).

Controls on this screen

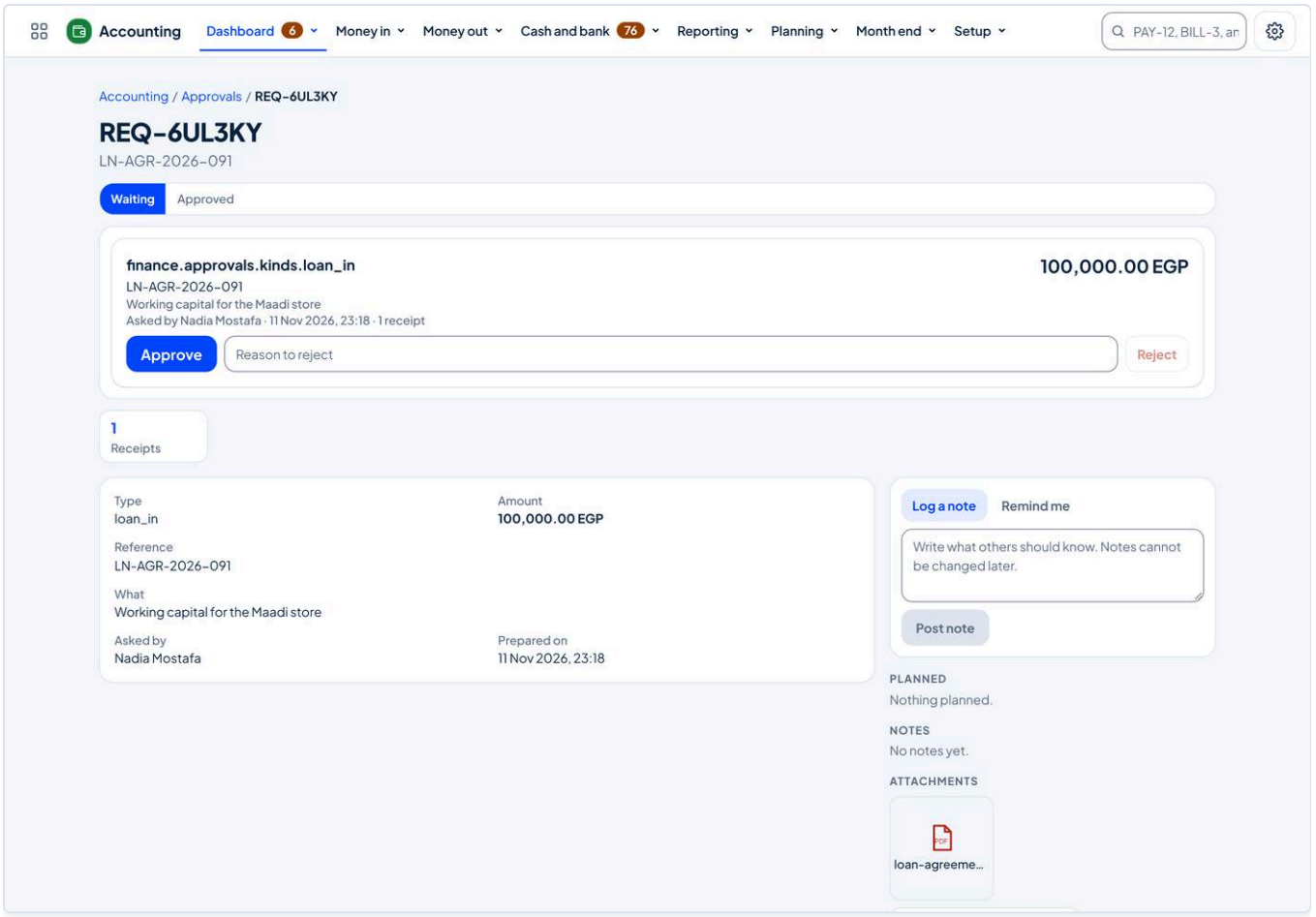
Control	What it does	Rule / limits	Result
Schedule preview	Appears as soon as amount, rate, months and both days are valid	Worked out in the browser with the same `loanSchedule` as the server.	"12 instalments of 9,168.00 EGP" and "Interest 10,015.99 EGP, total to pay 110,015.99 EGP, last instalment on 01/09/2027."
Record the loan	`POST /finance/loans`	Lender is always a bank; name 2 to 120 characters; first instalment after the start day (`first_payment_too_early`); money in always waits for a second person while `finance.approve_money_in` is 1.	"Saved and sent for approval. A second person must approve it before it is posted." A request of kind `loan_in` appears in Approvals.
Approve in Approvals	`POST /finance/requests/:id/decide` by another person with Accounting manage	Never the person who asked (`same_person`); the start day must be in open books.	Loan LN-000001 is created with its own 2700 account (2701), entry Dr bank, Cr 2701, and the full schedule.



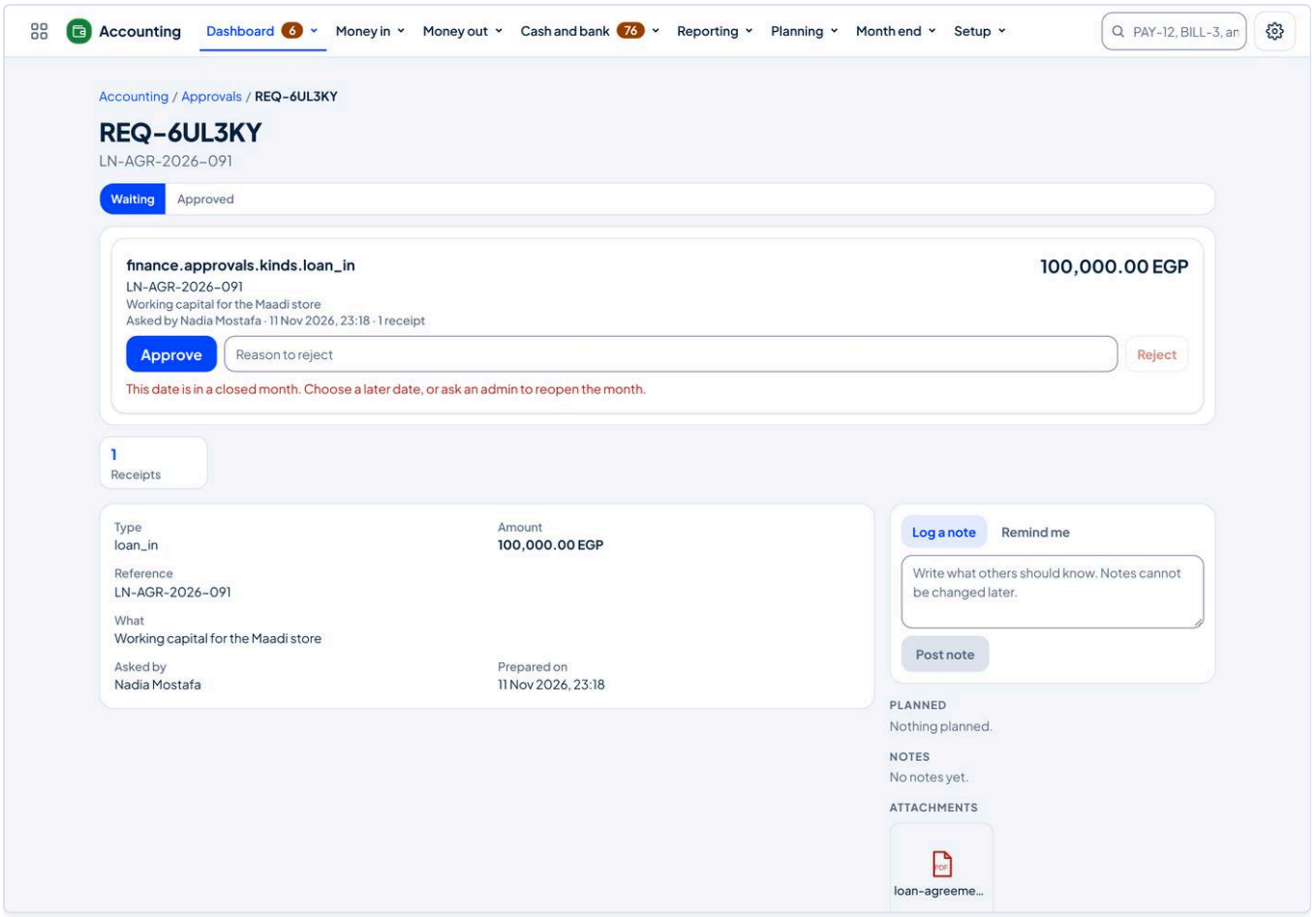
New loan filled with Another bank, 100,000 EGP at 18% declining over 12 months, the preview, the agreement PDF and the Unsaved changes marker.



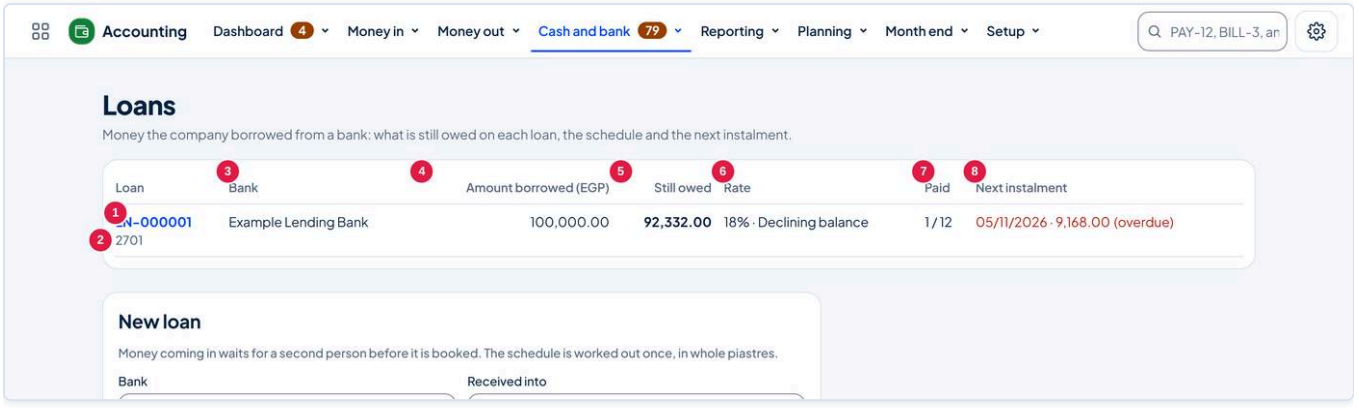
After Record the loan: the form is cleared and the message says it waits for a second person.



The loan request as the second person sees it in Approvals (REQ page): Approve, a reason box and Reject. The heading and Type show the request kind `loan_in` as stored.



An error state: approving a loan whose start day (1 Sep 2026) lies in closed books answers "This date is in a closed month. Choose a later date, or ask an admin to reopen the month." The New loan form accepted that day; the lock is only checked at approval. The request was then rejected with a reason and the loan recorded again from 1 Oct 2026.



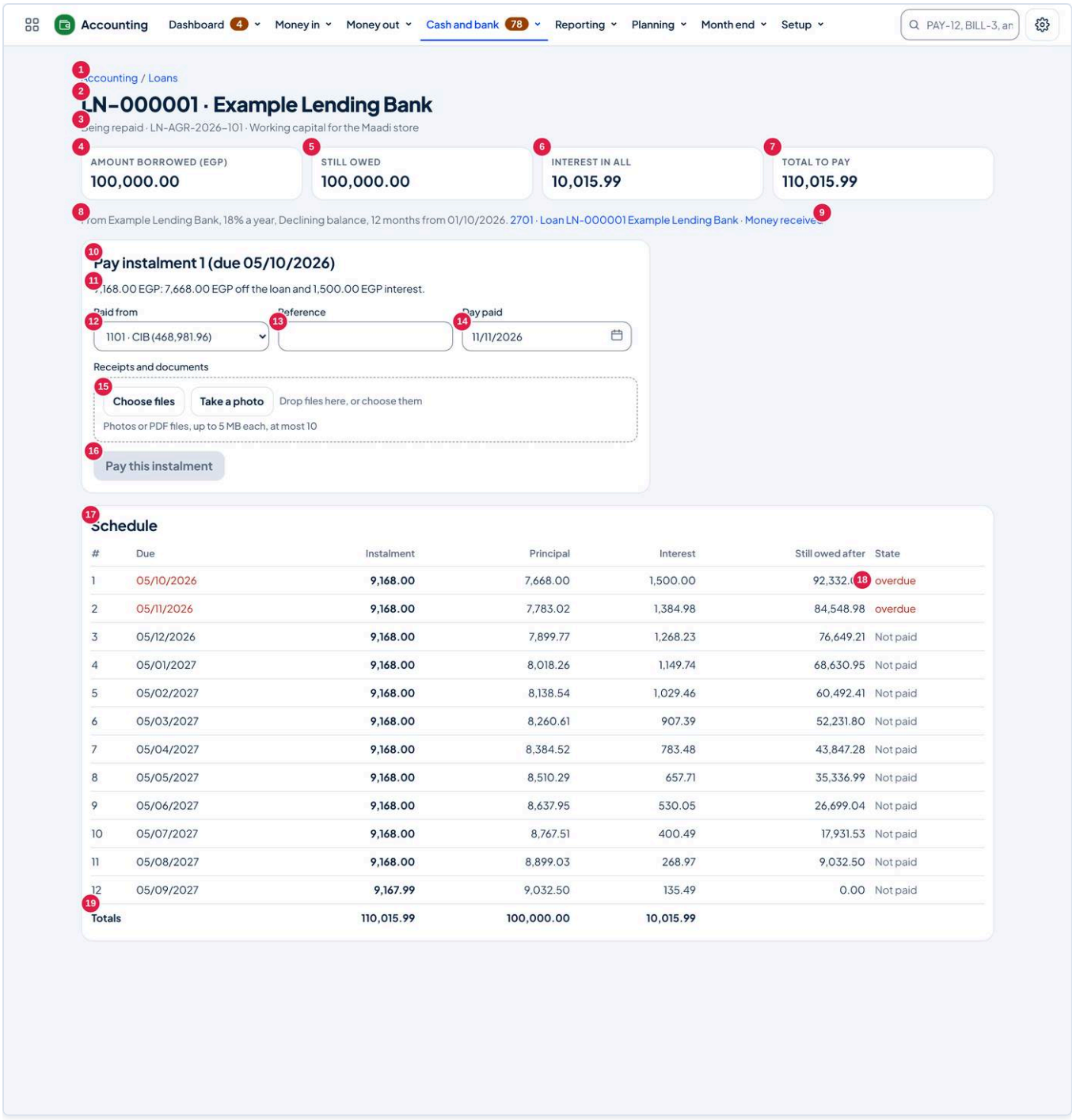
The loans table once the loan is approved, every column numbered.

#	Part	What it shows	What clicking it does
1	Loan	Blue LN number.	Opens the loan page.
2	Account	The loan's own account code (2701) in grey.	Nothing.
3	Bank	The lender, in the page language for a listed bank.	Nothing.
4	Amount borrowed (EGP)	The principal.	Nothing.

#	Part	What it shows	What clicking it does
5	Still owed	Bold: principal not yet repaid (the 2701 balance).	Nothing.
6	Rate	"18% · Declining balance".	Nothing.
7	Paid	Instalments paid out of the total, "0 / 12".	Nothing.
8	Next instalment	Due day and amount; red with "(overdue)" when the due day has passed; "Repaid" when nothing is left.	Nothing.

15.10 A loan page

Address `/finance/loans/<id>`. Opens with Accounting view; Pay next is shown only with Accounting use. Data `GET /finance/loans/:id`; component `LoanDetail`.



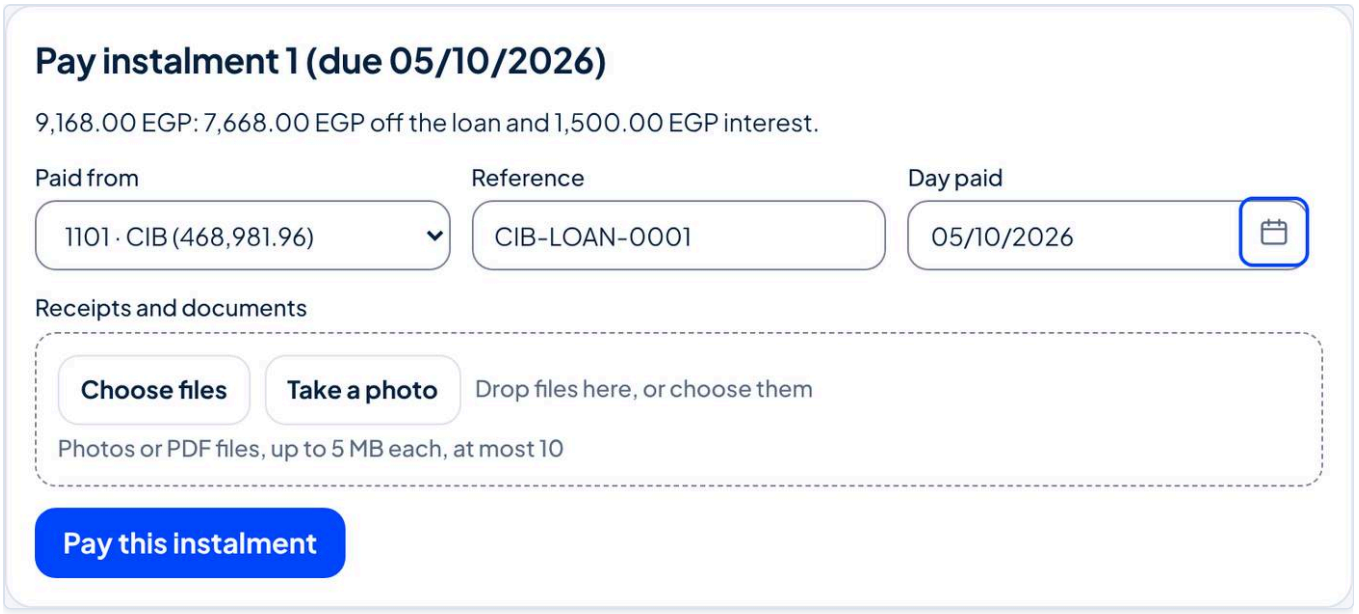
Loan LN-000001 before any payment: both past instalments are overdue (red), the rest Not paid.

#	Part	What it shows	What clicking it does
1	Breadcrumbs	Accounting / Loans.	Back.
2	Title	LN number and bank.	Nothing.
3	State line	"Being repaid" or "Repaid", reference and note.	Nothing.
4	Amount borrowed (EGP)	Card.	Nothing.
5	Still owed	Card; falls by the principal of each paid instalment.	Nothing.
6	Interest in all	Card: total interest of the schedule.	Nothing.

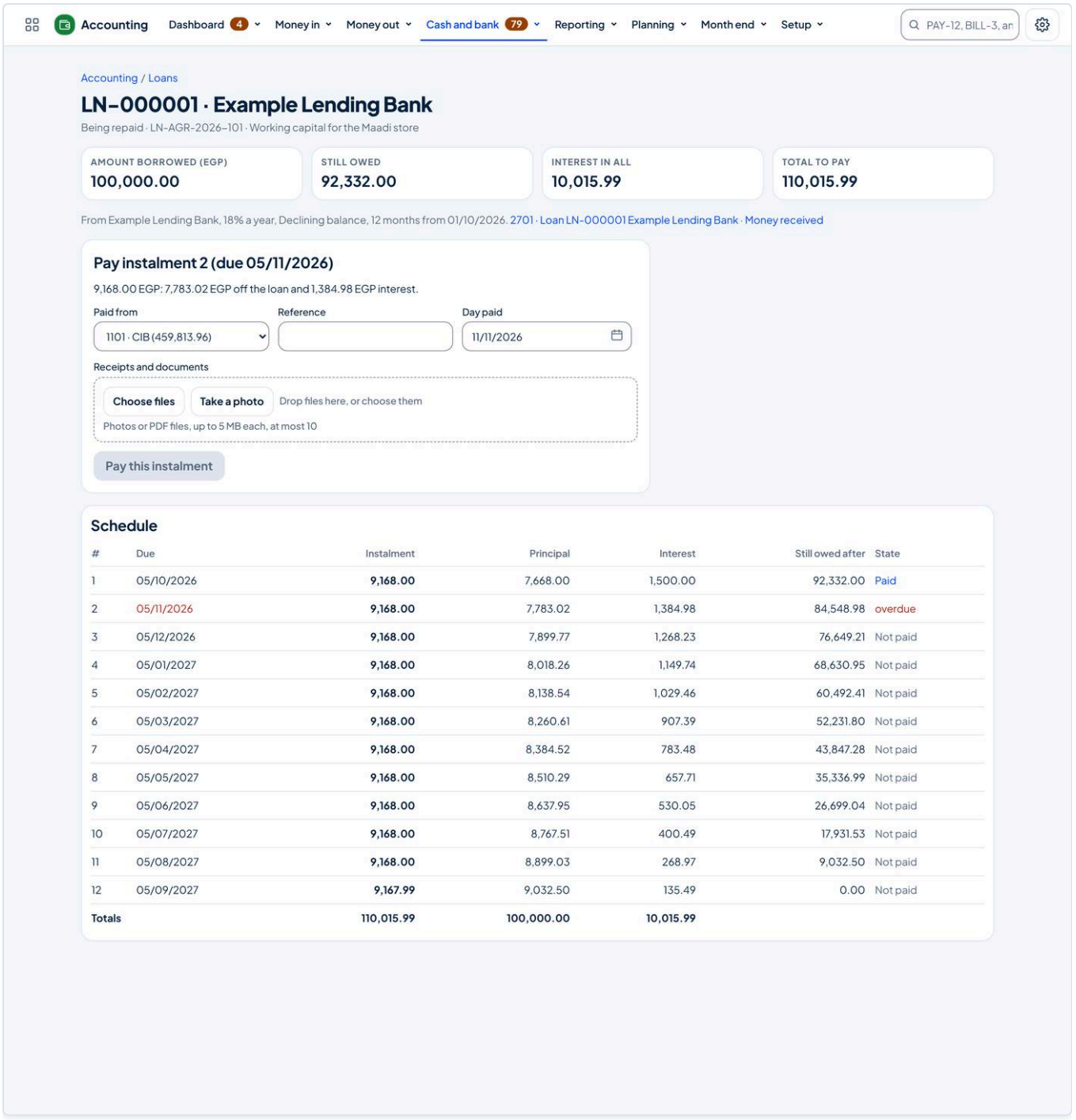
#	Part	What it shows	What clicking it does
7	Total to pay	Card: principal plus interest.	Nothing.
8	Summary	"From Example Lending Bank, 18% a year, Declining balance, 12 months from 01/10/2026."	Nothing.
9	Links	"2701 · Loan LN-000001 Example Lending Bank" and "Money received".	Open the loan account and the entry that received the money.
10	Pay instalment N (due day)	Section for the next unpaid instalment only (instalments are paid in order).	Nothing.
11	Split	"9,168.00 EGP: 7,668.00 EGP off the loan and 1,500.00 EGP interest."	Nothing.
12	Paid from	Every open EGP bank or cash box with its balance; the account the loan came into is chosen first.	Chooses the account.
13	Reference	At least 2 characters.	Enables the button.
14	Day paid	Today by default, never in the future.	Date picker.
15	Receipts and documents	Attachment field.	Upload.
16	Pay this instalment	Blue button, grey until a reference is typed.	Pays; see controls.
17	Schedule	#, Due, Instalment (bold), Principal, Interest, Still owed after, State; Totals row.	Nothing.
18	State	"Paid" as a blue link to its entry; "overdue" in red (and the due day in red) when unpaid and past due; "Not paid" in grey otherwise.	Paid opens the payment's journal entry.
19	Totals	Sums of instalments, principal and interest; the last instalment absorbs the rounding (9,167.99).	Nothing.

Controls on this screen

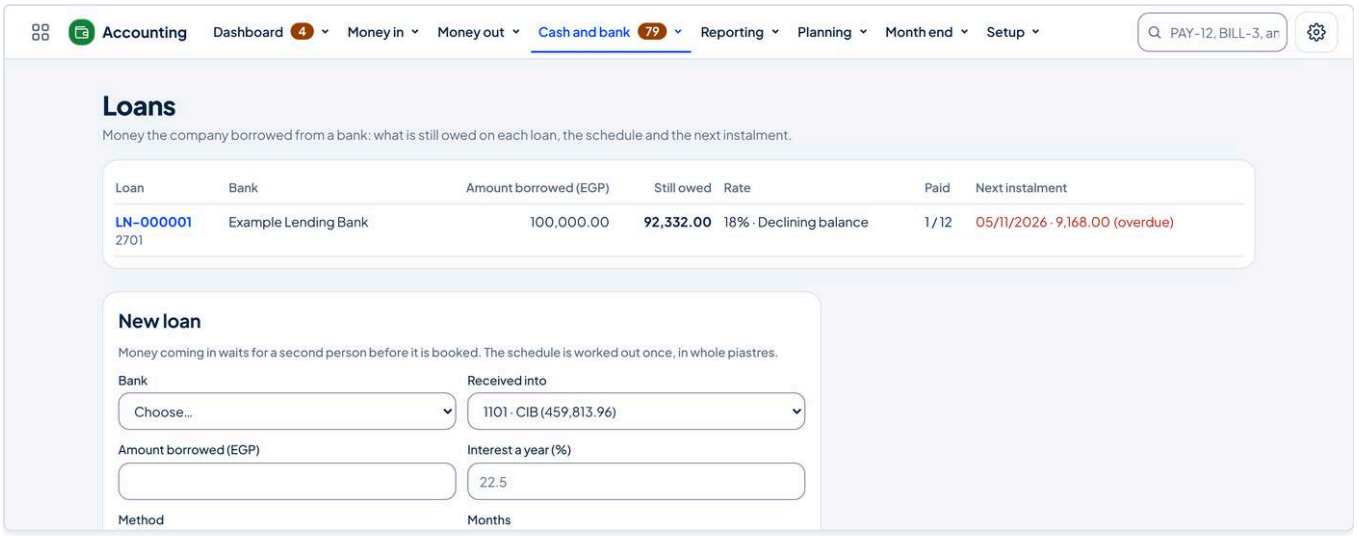
Control	What it does	Rule / limits	Result
Pay this instalment	`POST /finance/loans/:id/pay`	Accounting use; the account must hold the money now and at every later moment (`not_enough_money`); books lock; at once below `finance.approval_threshold_piastres` (10,000 EGP in the demo), otherwise it waits in Approvals for another person.	"Paid JV-002603". Entry Dr 2701 principal, Dr 5904 interest, Cr bank. The section moves to the next instalment.



Pay instalment 1 filled: paid from CIB, reference CIB-LOAN-0001, day 05/10/2026.



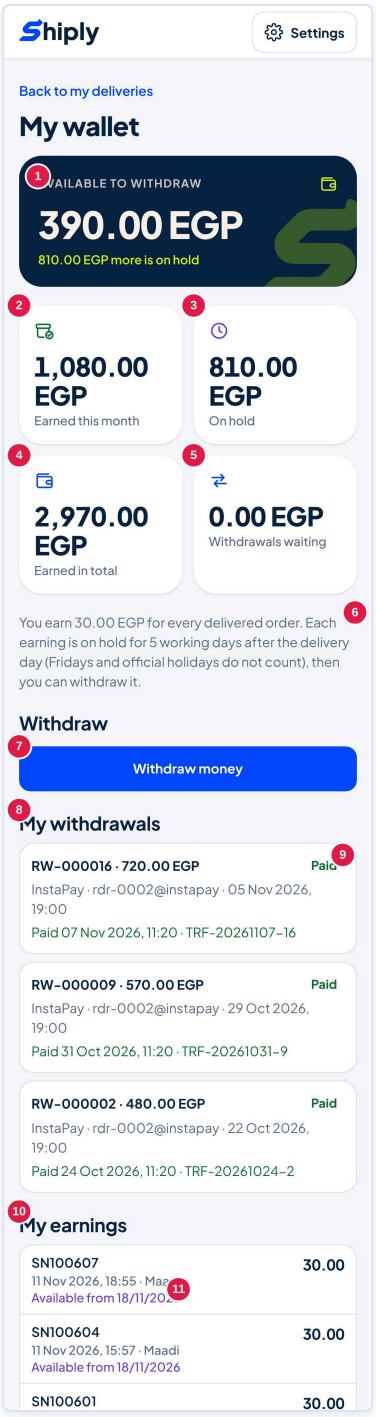
After paying instalment 1: Still owed 92,332.00, instalment 1 Paid (blue link), instalment 2 overdue in red, the others Not paid, and the form now offers instalment 2.



The loans table after the payment: Paid 1 / 12, still owed 92,332.00, next instalment 05/11/2026 overdue in red.

15.11 Rider app: My wallet

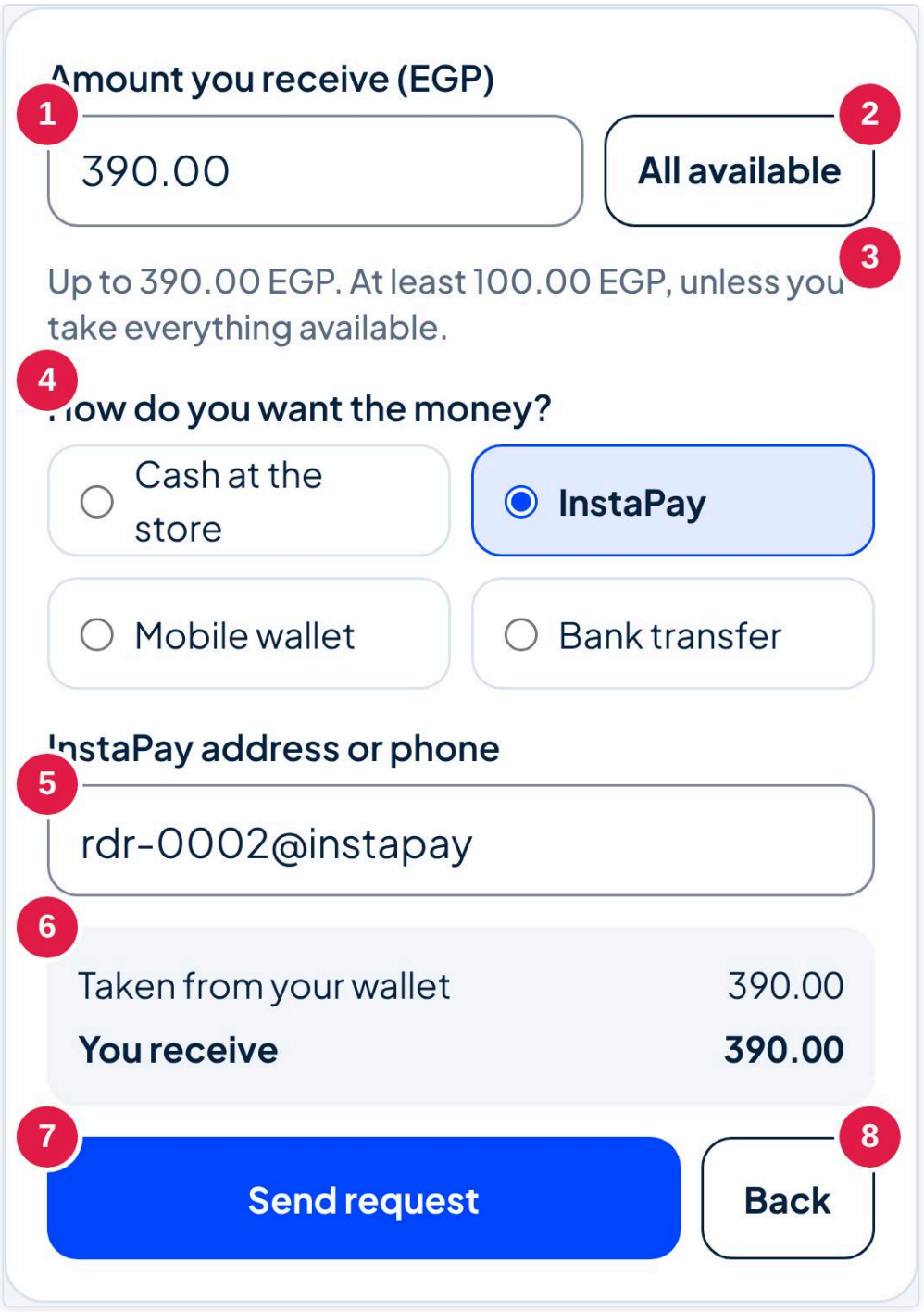
Address `/rider/wallet`, opened from the rider home (phone). Only the rider himself; data `GET /rider/wallet`, component `RiderWallet`. Rules in chapter 11, sections The withdrawal hold, The wallet figures and Withdrawals.



My wallet on a phone (rider RDR-0002) with every part numbered.

#	Part	What it shows	What clicking it does
1	Available to withdraw	Navy wallet card, label in spaced capitals, amount in Cream, Lime note "810.00 EGP more is on hold", faint S mark.	Nothing.
2	Earned this month	Card with a green delivered icon.	Nothing.
3	On hold	Card with a violet clock icon: earnings still inside their hold.	Nothing.
4	Earned in total	Card with a blue wallet icon.	Nothing.
5	Withdrawals waiting	Card with arrows icon: open requests.	Nothing.

#	Part	What it shows	What clicking it does
6	Rules	"You earn 30.00 EGP for every delivered order. Each earning is on hold for 5 working days after the delivery day (Fridays and official holidays do not count), then you can withdraw it."	Nothing.
7	Withdraw money	Full width blue button. Replaced by "You already have a withdrawal waiting..." or "Nothing is available yet..." when a request is not possible.	Opens the withdrawal form.
8	My withdrawals	Cards: RW number and amount, method and details, request time; "Paid ... · reference" in green; "Not approved: reason" for a rejection; "N EGP taken for what you owed" when dues were netted. Empty: "No withdrawals yet."	Cancel request on a waiting one; Show more after 5.
9	Status	Status word at the right of each card: Waiting, Approved, Paid (green), Not approved, Cancelled (grey pill).	Nothing.
10	My earnings	One row per delivered order: order number, time and store, 30.00.	Show more adds 30 rows.
11	Earning state	"Available from 18/11/2026" in violet while on hold, "Available" in green after.	Nothing.
12	Show more	Outline button.	Shows more rows.



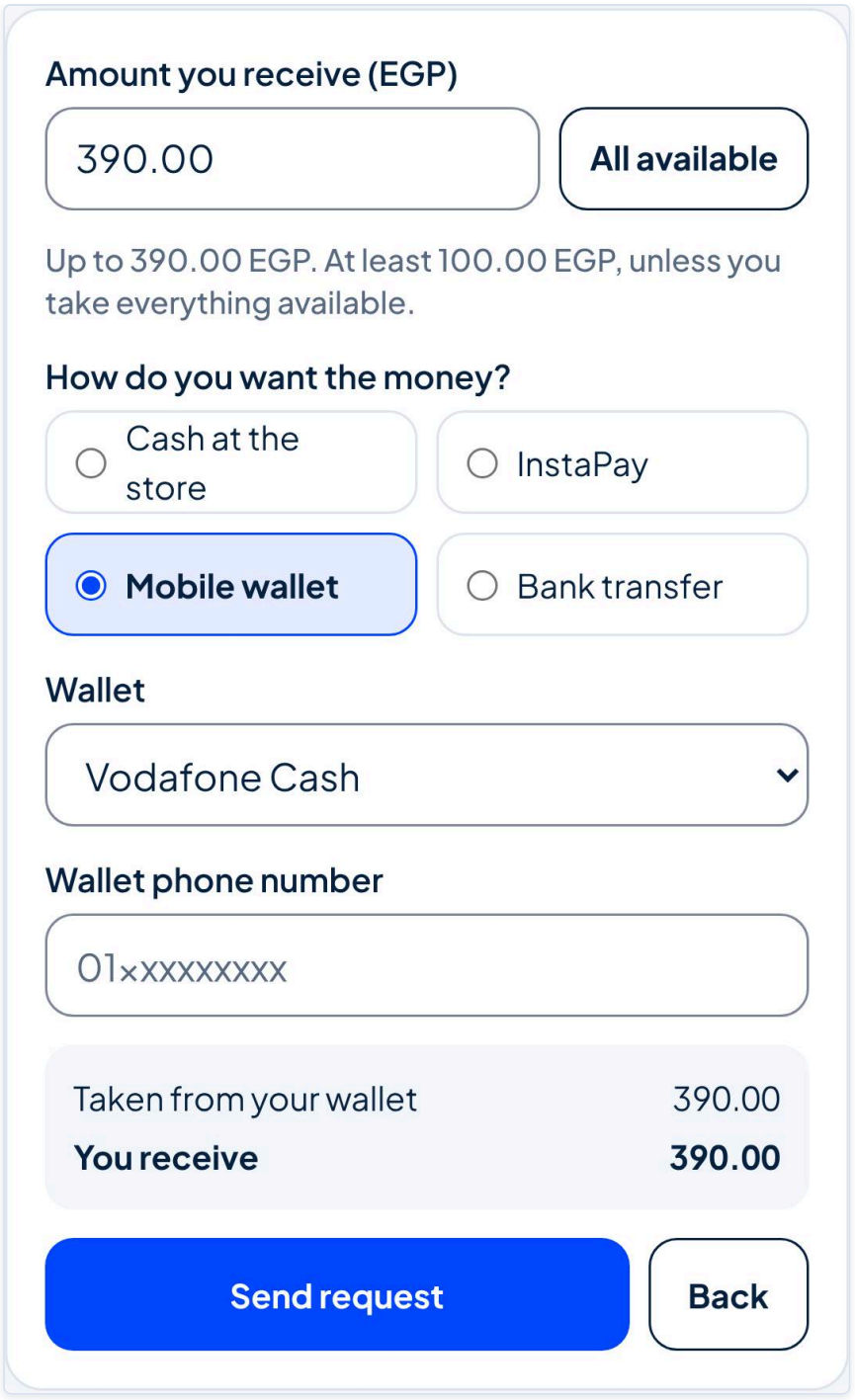
The withdrawal form opened; the last method used (InstaPay) is offered again.

#	Part	What it shows	What clicking it does
1	Amount you receive (EGP)	Filled with everything available.	Typing updates the summary.
2	All available	Outline button.	Puts the whole available amount back.
3	Limits	"Up to 390.00 EGP. At least 100.00 EGP, unless you take everything available."	Nothing.

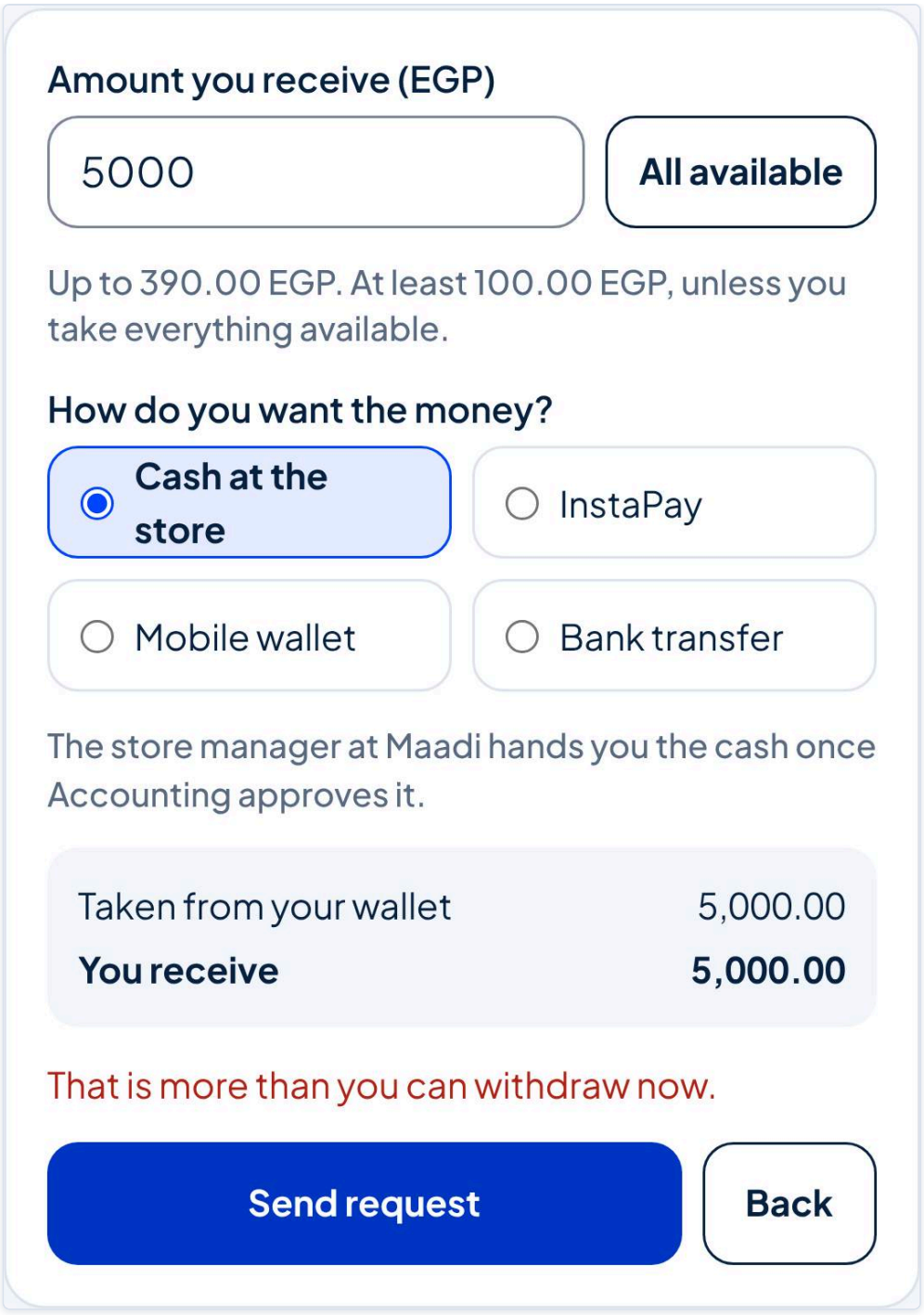
#	Part	What it shows	What clicking it does
4	How do you want the money?	Four radio tiles: Cash at the store, InstaPay, Mobile wallet, Bank transfer; the chosen tile has a blue border and light blue ground.	Shows the fields of that method.
5	Method field	InstaPay address or phone (name@instapay). Cash at the store shows "The store manager at Maadi hands you the cash once Accounting approves it."; Mobile wallet shows Wallet (Vodafone Cash, Etisalat Cash, Orange Cash, WE Pay) and Wallet phone number; Bank transfer shows Bank, IBAN (EG and 27 digits) and Account holder.	Typing.
6	Summary	Taken from your wallet, "What you owe, taken first" in orange when dues exist, and You receive in bold.	Nothing.
7	Send request	Blue button.	`POST /rider/withdrawals` : "Request sent. Accounting will review it."
8	Back	Outline button.	Closes the form.

Controls on this screen

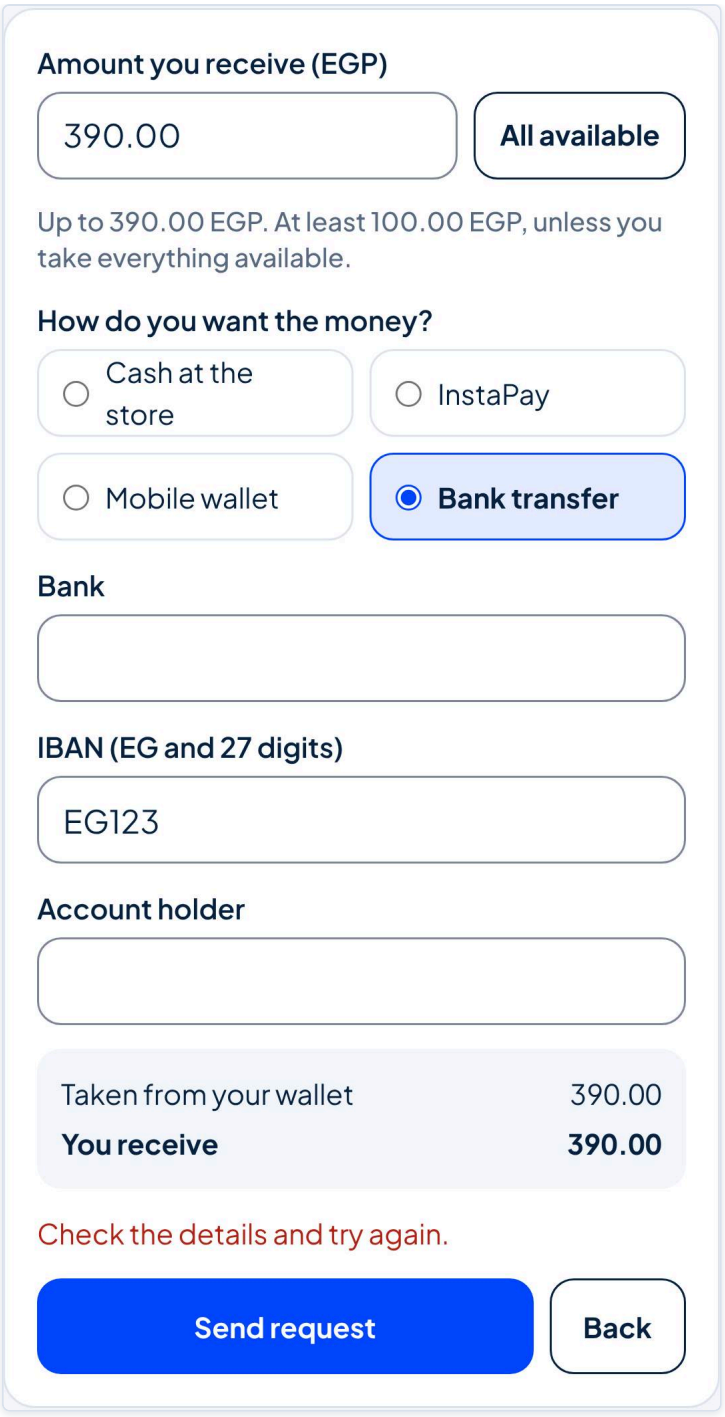
Control	What it does	Rule / limits	Result
Send request	Creates RW-nnnnnn	Not more than available ("That is more than you can withdraw now."); at least `rider.min_withdrawal_piastres` unless everything ("That is below the smallest withdrawal."); at most `rider.max_open_withdrawals` open; frozen riders refused (`rider_frozen`); IBAN EG plus 27 digits, Egyptian mobile, InstaPay handle checked; idempotency key.	Status Waiting; the amount leaves Available at once.
Cancel request	`POST /rider/withdrawals/:id/cancel`	Only while Waiting (before approval).	"Request cancelled. The amount is available again."; status Cancelled.



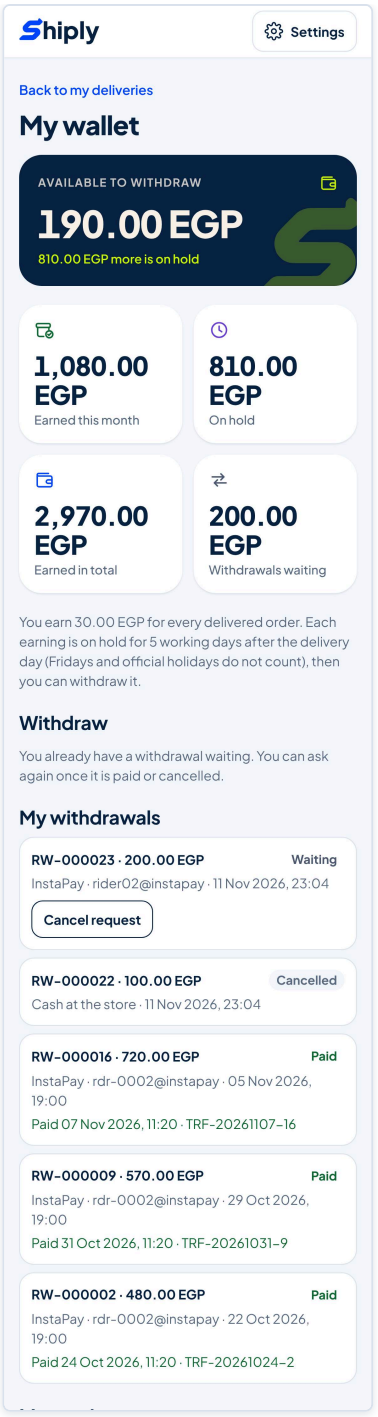
Mobile wallet chosen: the wallet list and the wallet phone number.



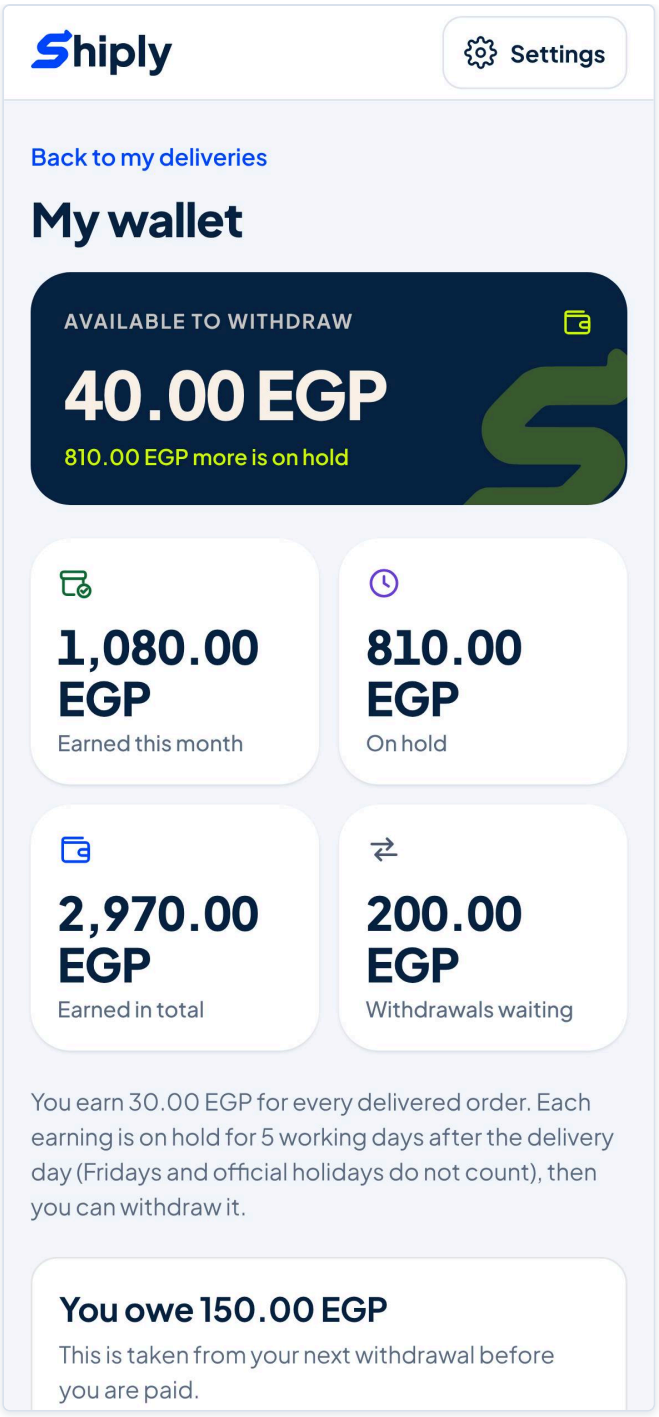
Error: 5,000 EGP is more than available; the red message appears above the buttons.



Error: a short IBAN. The server refuses the request and the screen shows the general "Check the details and try again." rather than the specific IBAN message.



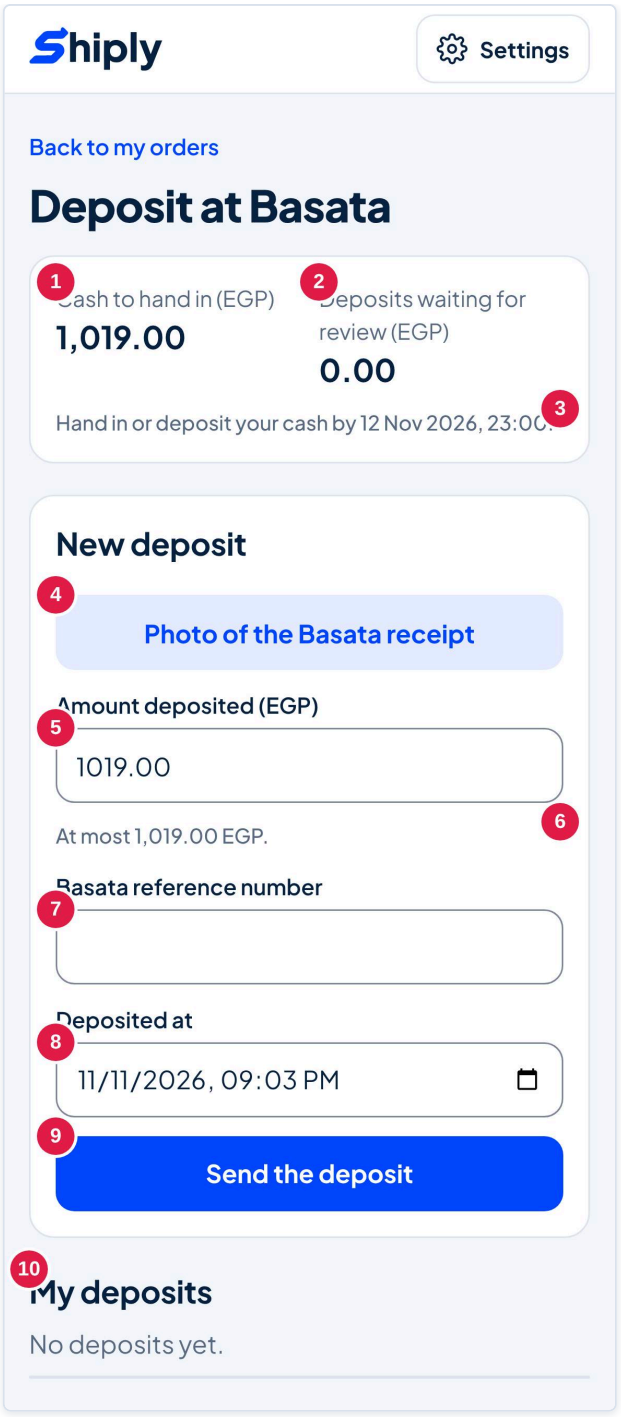
After a request and a cancelled one: Withdrawals waiting 200.00, the hint "You already have a withdrawal waiting...", RW-000023 Waiting with Cancel request, RW-000022 Cancelled (grey pill) and earlier Paid ones. The Waiting and Paid status words have no tinted background; only Cancelled shows a pill ground.



The same wallet after the deadline charge: Available drops to 40.00 and a "You owe 150.00 EGP" panel lists what will be taken from the next withdrawal.

15.12 Rider app: Deposit at Basata

Address `/rider/basata`, from the rider home. Only the rider himself; data `GET /rider/basata`, component `RiderBasata`. Works also while frozen (a frozen rider can still deposit).



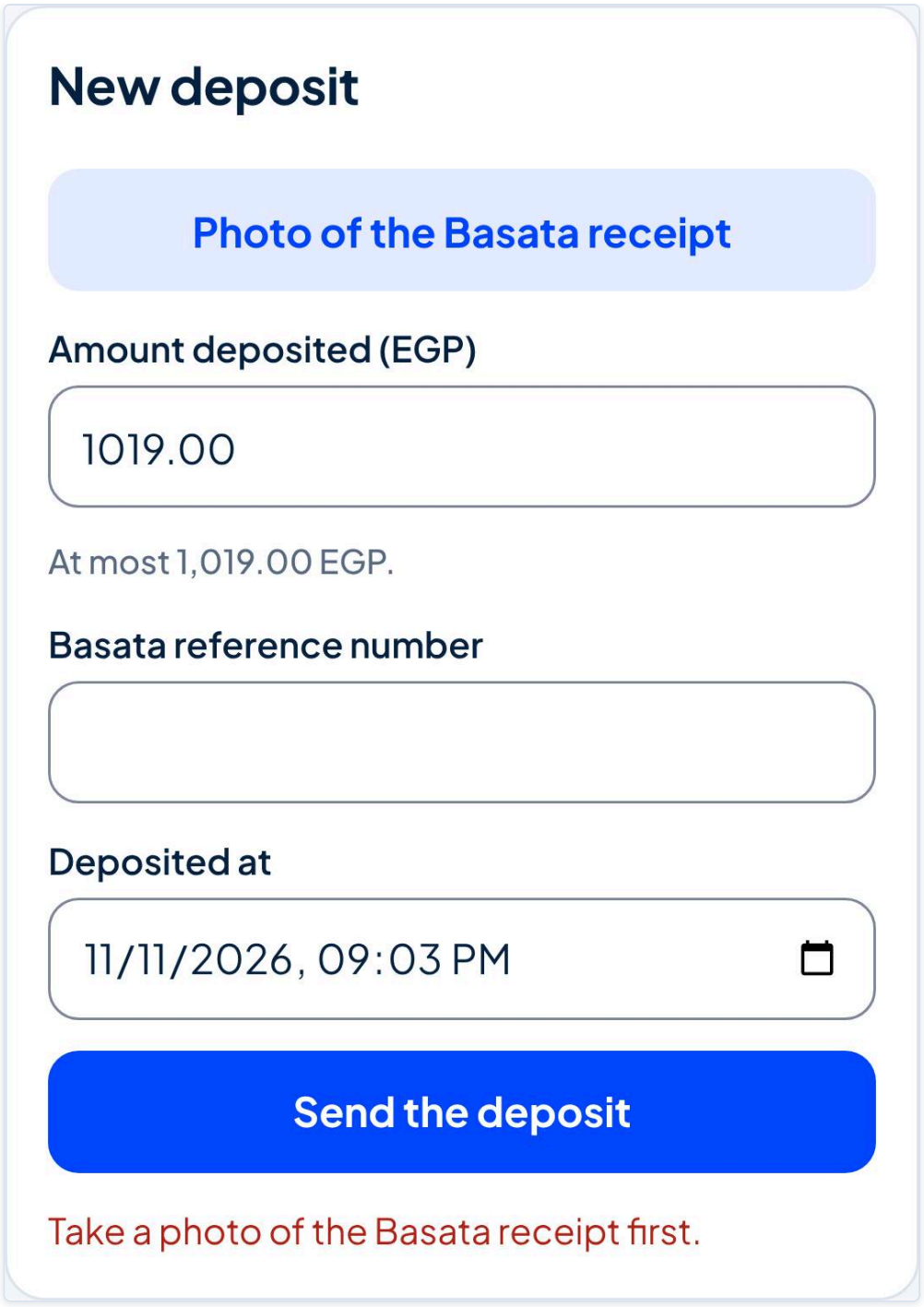
Deposit at Basata before any deposit, every part numbered.

#	Part	What it shows	What clicking it does
1	Cash to hand in (EGP)	Rider cash minus deposits waiting and handover differences waiting.	Nothing.
2	Deposits waiting for review (EGP)	Sum of his pending deposits. "Taken at the deadline (EGP)" in red with "Deposit it at Basata and it is given back once accounting approves." appears when a deadline took cash.	Nothing.
3	Deadline	"Hand in or deposit your cash by 12 Nov 2026, 23:00." (the next `cash.deposit_deadline_hour`).	Nothing.

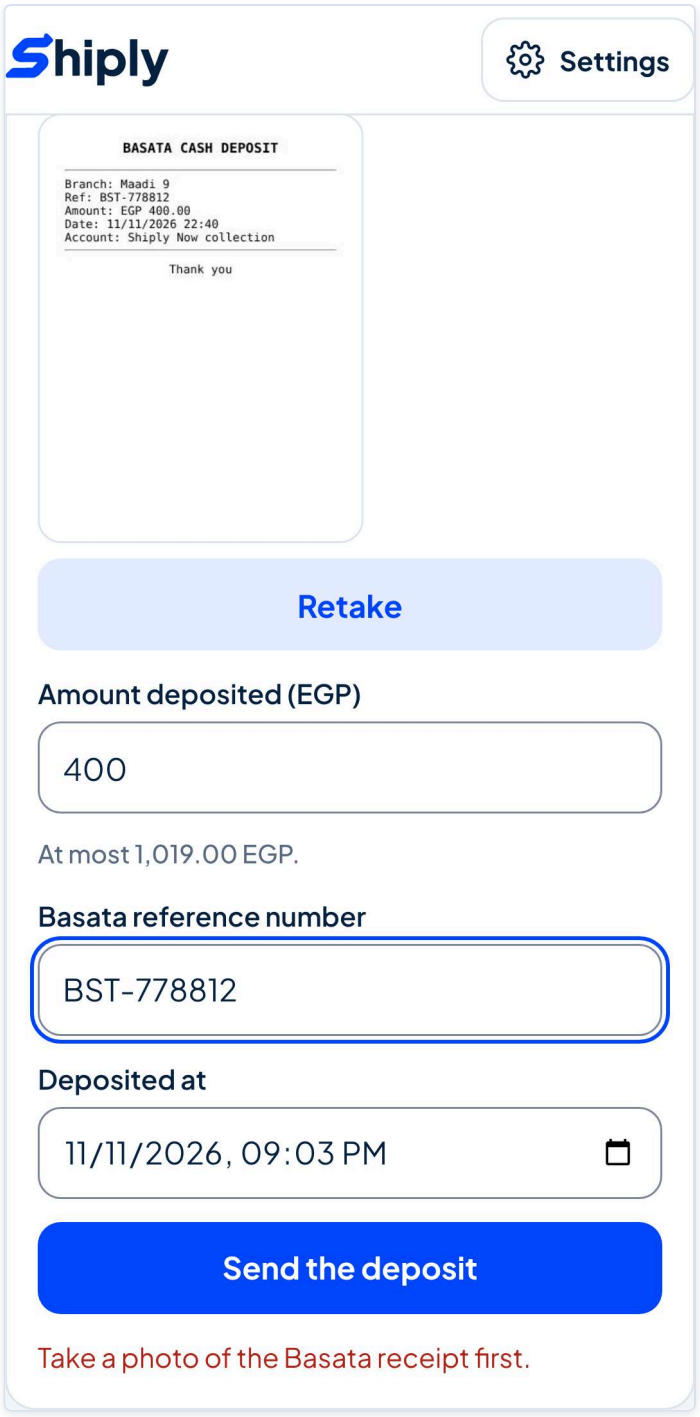
#	Part	What it shows	What clicking it does
4	Photo of the Basata receipt	Light blue camera button; after a photo it shows a preview and becomes "Retake".	Opens the camera (or the file picker on a computer); uploads a photo of kind `basata_receipt`.
5	Amount deposited (EGP)	Filled with the most he may deposit.	Typing.
6	At most	"At most 1,019.00 EGP." (cash to hand in plus what a deadline took).	Nothing.
7	Basata reference number	Capitals, up to 60 characters.	Typing.
8	Deposited at	Date and time box, now by default, never later than now.	Picker.
9	Send the deposit	Full width blue button ("Sending…" while busy); disabled when nothing may be deposited.	`POST /rider/basata/deposits`.
10	My deposits	Number, status word in color (Waiting for review orange, Approved green, Rejected red), reference, amount, time, and the rejection reason in red. Empty: "No deposits yet."	Nothing.

Controls on this screen

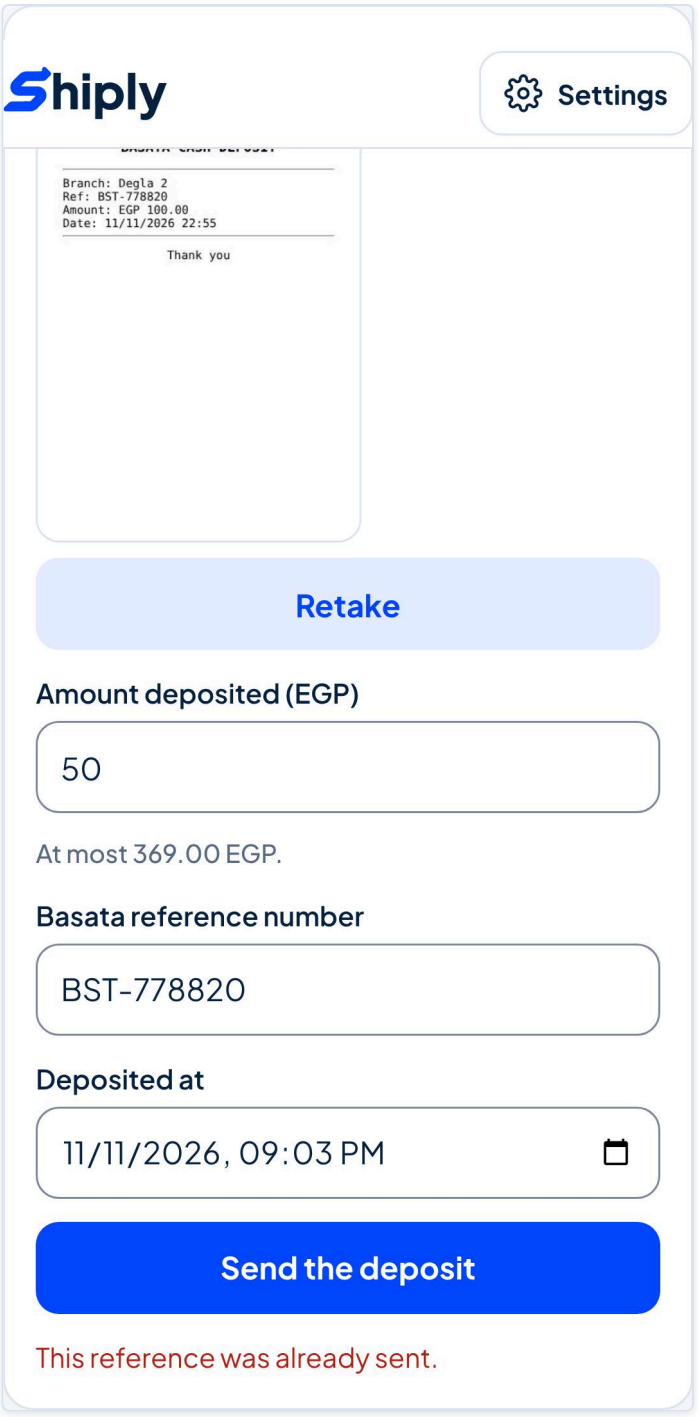
Control	What it does	Rule / limits	Result
Send the deposit	Creates BD-nnnnnn	Photo required ("Take a photo of the Basata receipt first."); amount above zero and not above the maximum ("That is more than the cash you have to hand in."); reference at least 3 characters and used once ("This reference was already sent."); time not in the future; Basata channel open for the store (`cash.basata_enabled`, else the form is replaced by "Basata deposits are closed for your store right now. Hand your cash to the store."); idempotency key.	"Sent. Accounting will check the receipt." The deposit is listed as Waiting for review and counts as handed in for the cash limit, the overdue alert and the deadline.



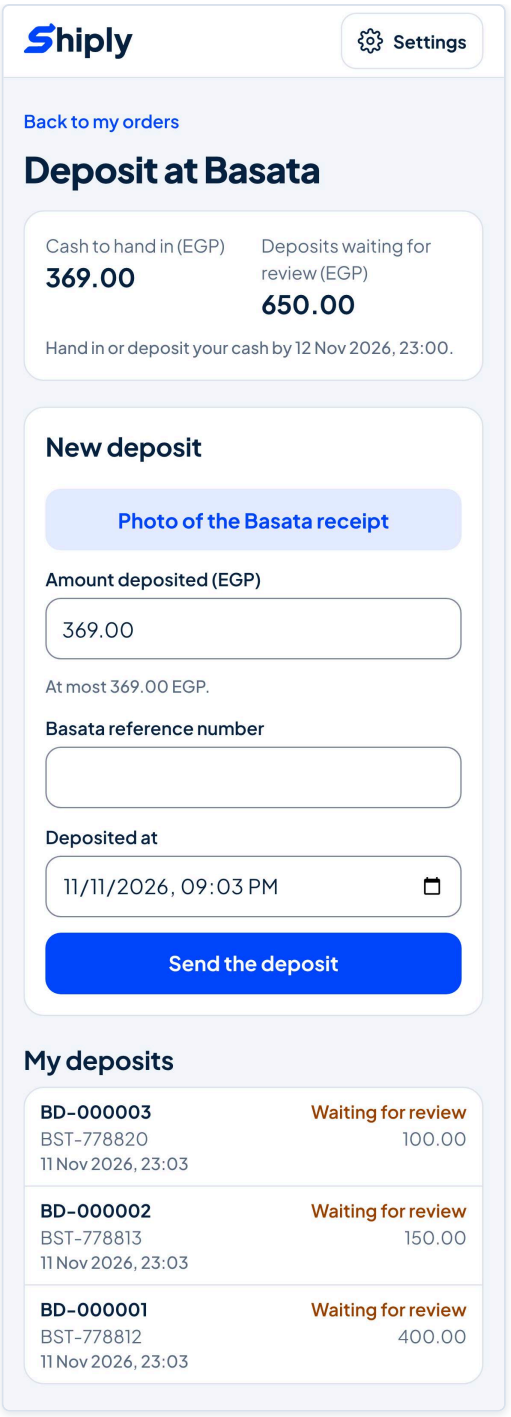
Error: Send without a photo.



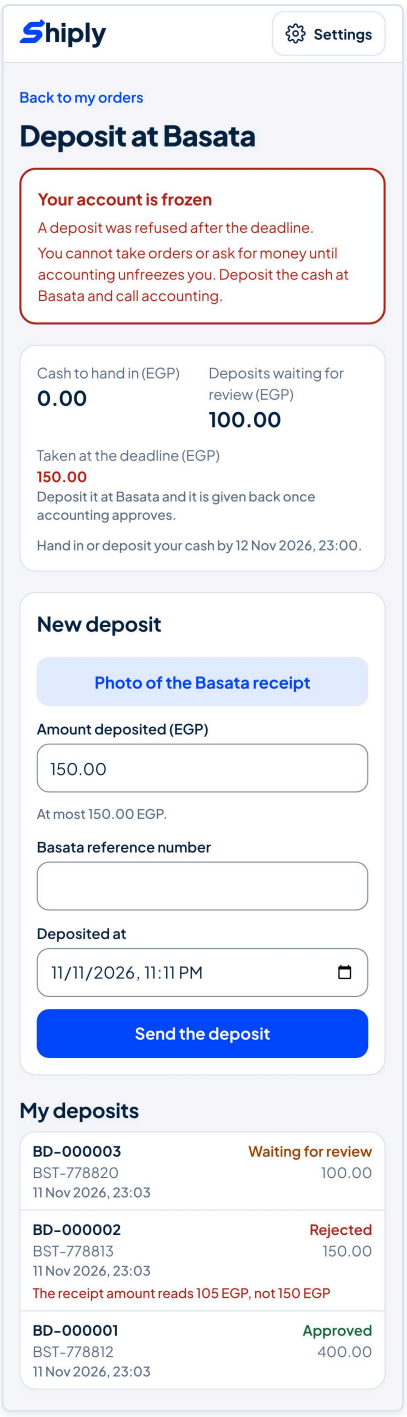
The form filled: receipt photo taken (preview and Retake), 400 EGP, reference BST-778812.



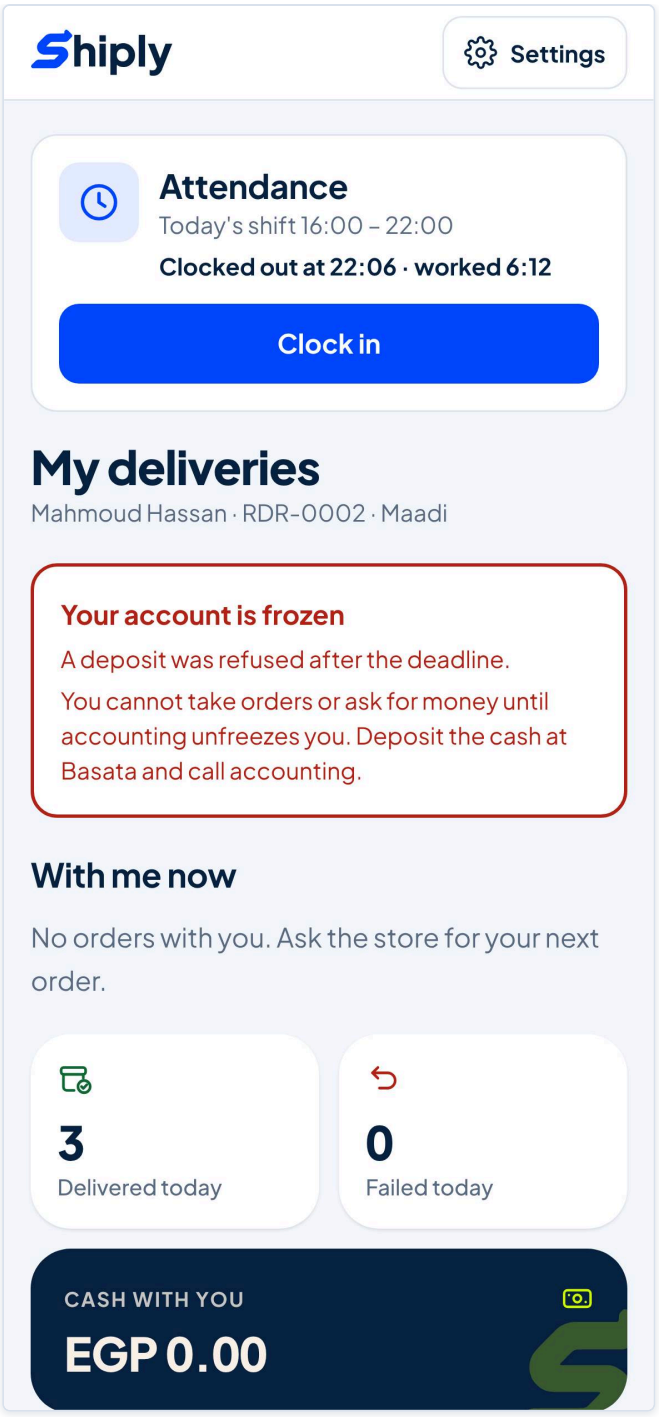
Error: a reference that was already sent.



Three deposits sent: 650.00 waiting for review, cash to hand in down to 369.00.



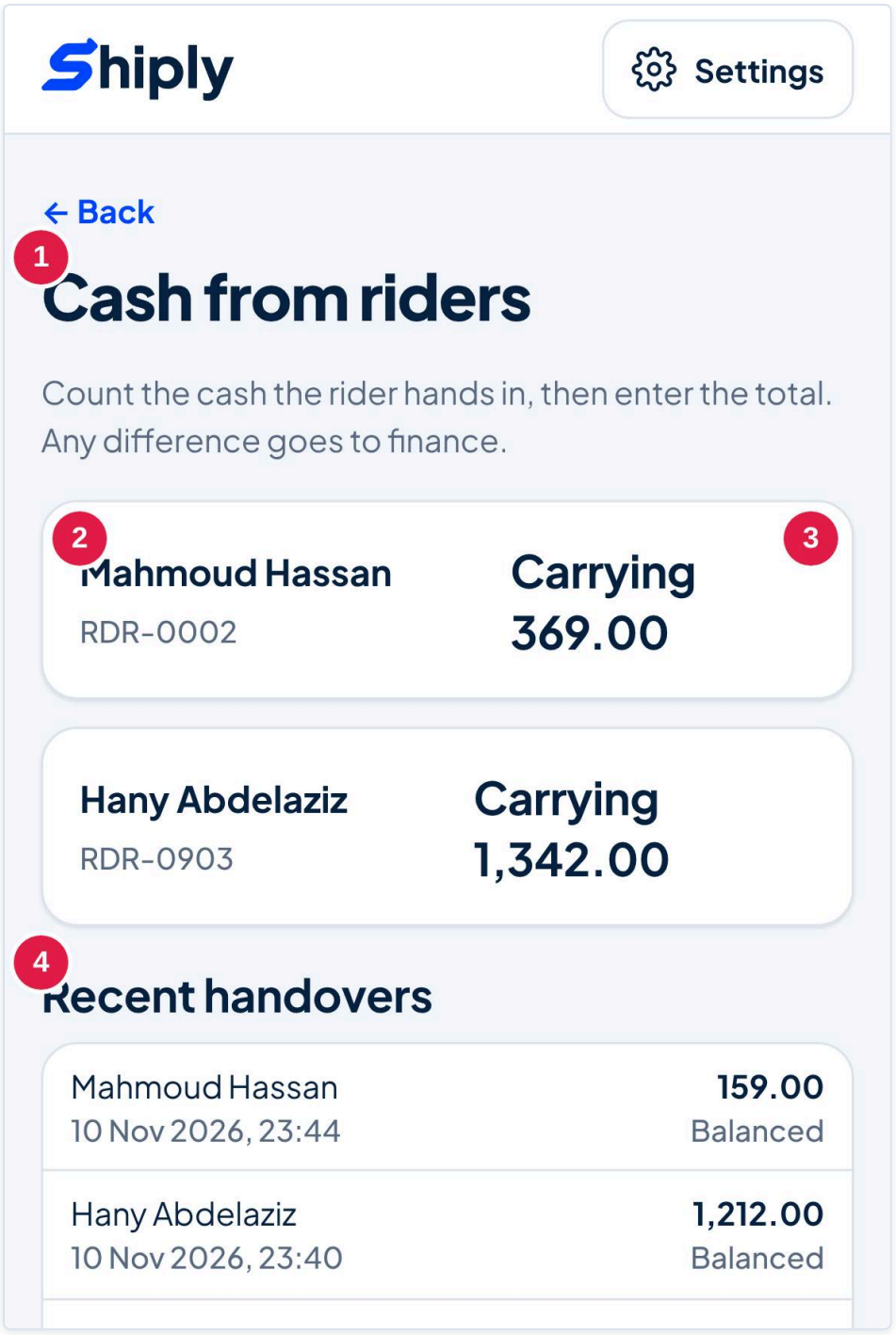
After finance approved one deposit and rejected another after its deadline: the red frozen banner ("Your account is frozen", the reason and what to do), 150.00 taken at the deadline, and the three statuses in their colors.



The rider home shows the same frozen banner; a frozen rider gets no orders at handover until Accounting unfreezes him.

15.13 Store app: Cash from riders

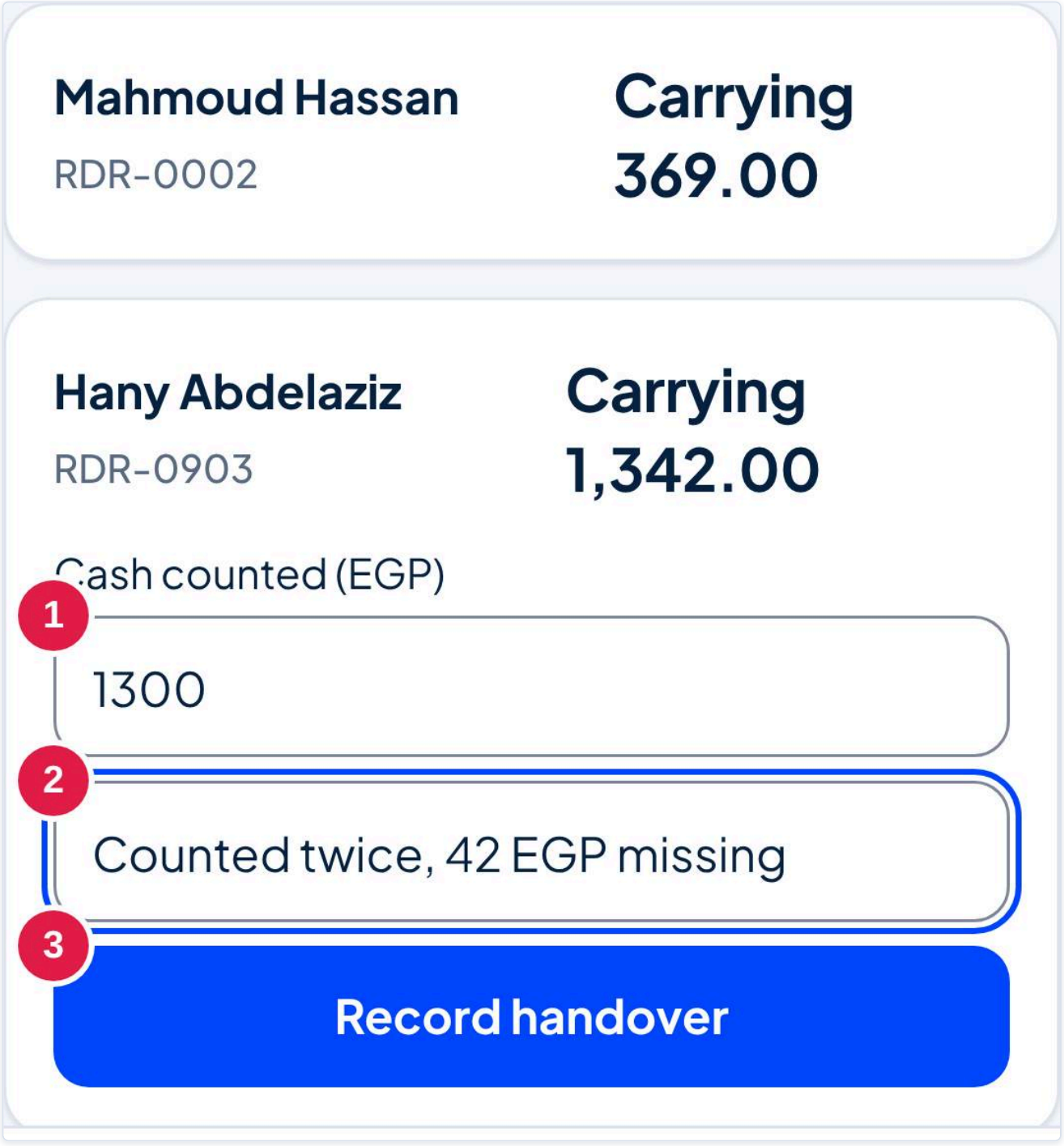
Address `/store/<storeId>/cash`, from the store home tile Rider cash, the Riders holding cash card ("Collect cash") or the Cash tab of the bottom bar. Store staff of that store (store use); data `GET /stores/:id/cash`, component `RiderCashScreen`. Store managers (store manage) also see "Rider withdrawals in cash" under it, where they record cash given for approved withdrawals (`StoreWithdrawals`); the demo staff account is not a manager, so that block does not show here.



Cash from riders on the Maadi store phone before counting, every part numbered.

#	Part	What it shows	What clicking it does
1	Title and intro	"Count the cash the rider hands in, then enter the total. Any difference goes to finance." A red box "Handing cash to the store is closed for now. Riders deposit their cash at Basata instead." appears when `cash.store_handover_enabled` is 0.	Nothing.
2	Rider card	Name and badge; "Frozen: cash not handed in by the deadline" in red for a frozen rider; "N orders still out" in orange when he still carries orders.	Opens the counting fields under the card (tap again to close).
3	Carrying	The cash he should hand in now (cash minus differences and deposits waiting).	Nothing.

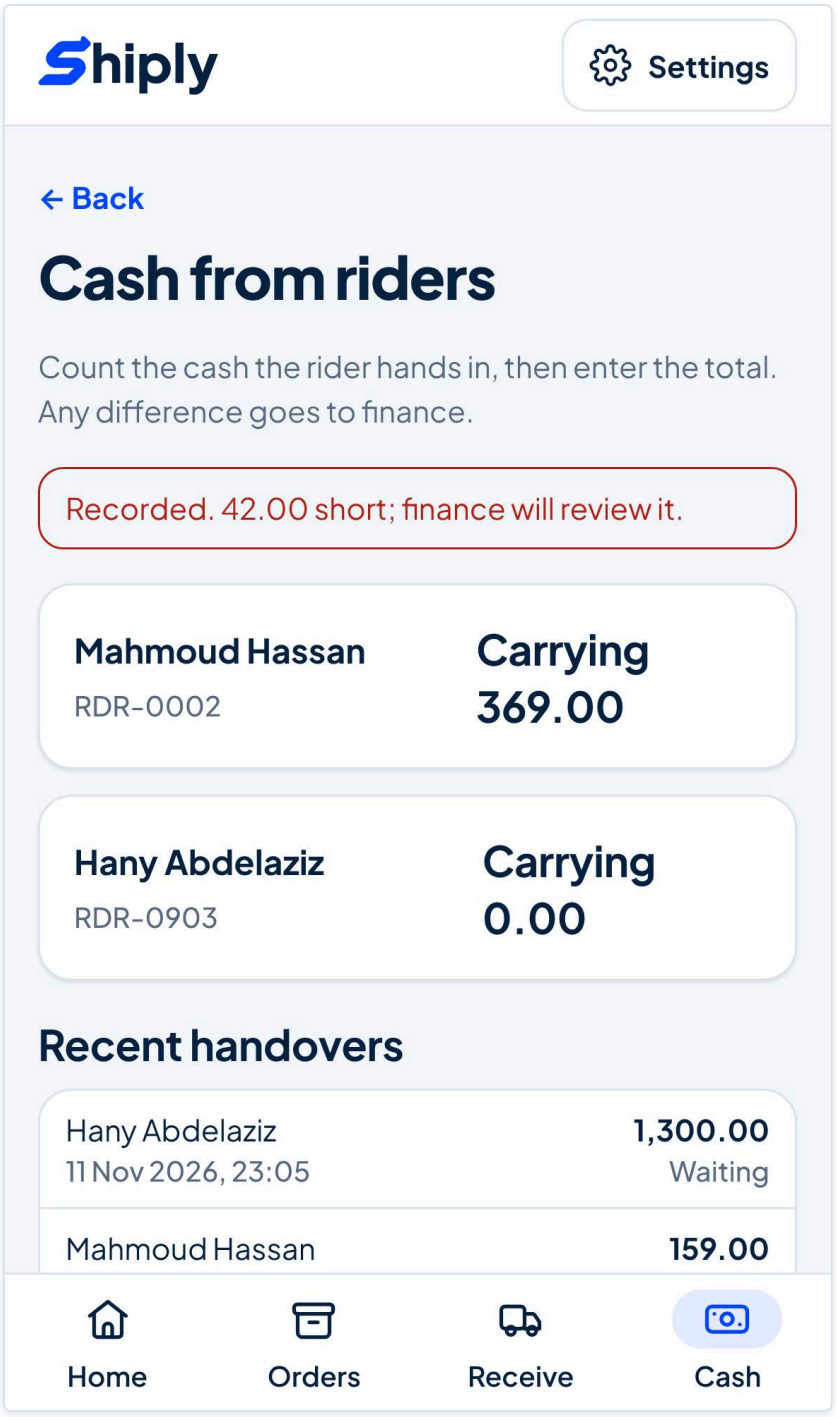
#	Part	What it shows	What clicking it does
4	Recent handovers	Name, time, counted amount and status (Balanced, Waiting, Written off, On rider).	Nothing.



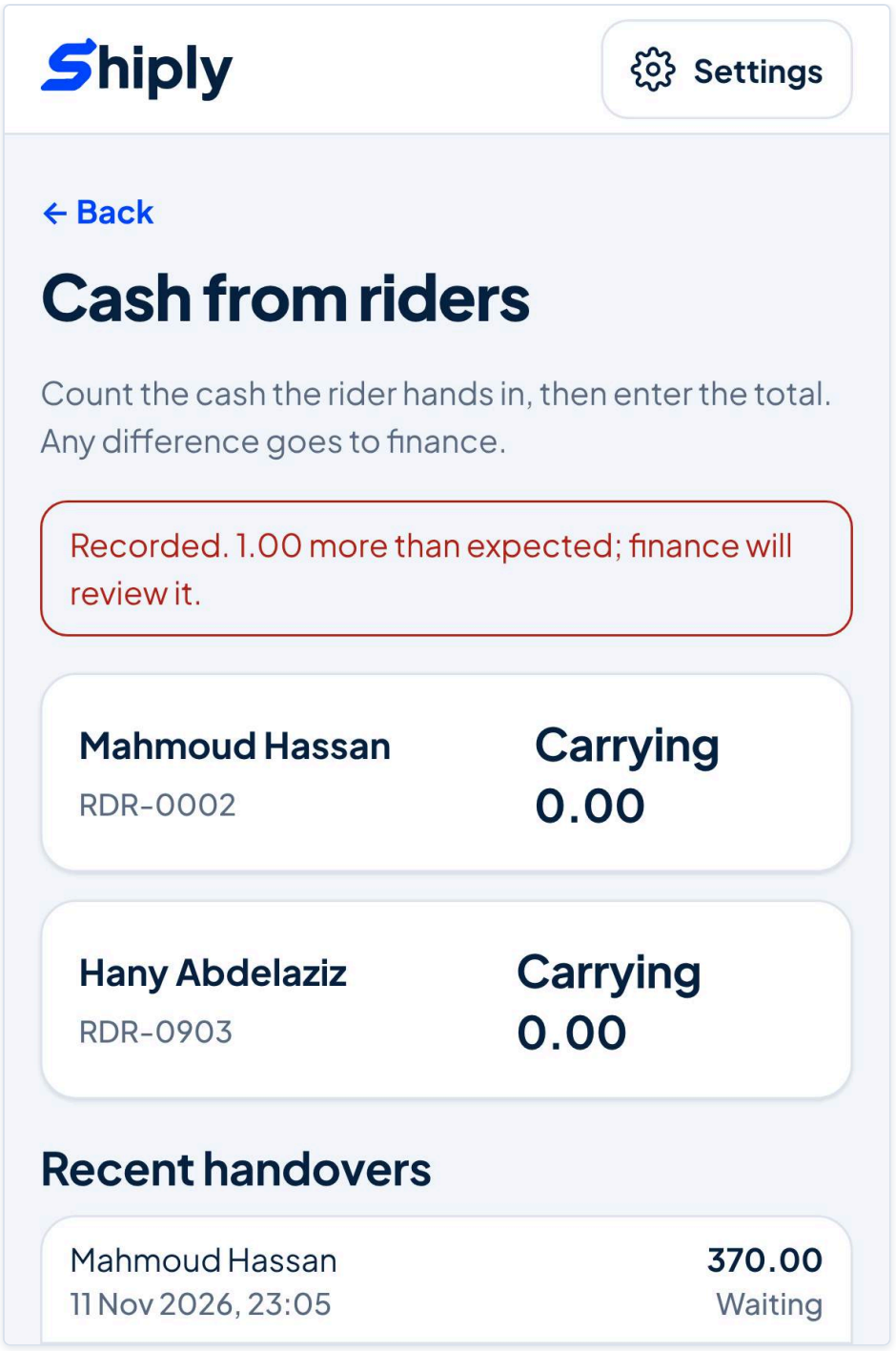
Counting Hany: 1 Cash counted (EGP), 2 the note (placeholder "Why?"), 3 Record handover.

Controls on this screen

Control	What it does	Rule / limits	Result
Record handover	`POST /stores/:id/cash/handovers`	Counted amount required ("Enter the amount counted."); the rider must belong to the store (`rider_other_store`); refused while the store channel is closed; note up to 300 characters; idempotency key per attempt.	Counted cash moves to the store box at once (CH number). Balanced: "Recorded. It matches." in grey. Short: red box "Recorded. 42.00 short; finance will review it." Over: "Recorded. 1.00 more than expected; finance will review it." Differences wait on Store cash.



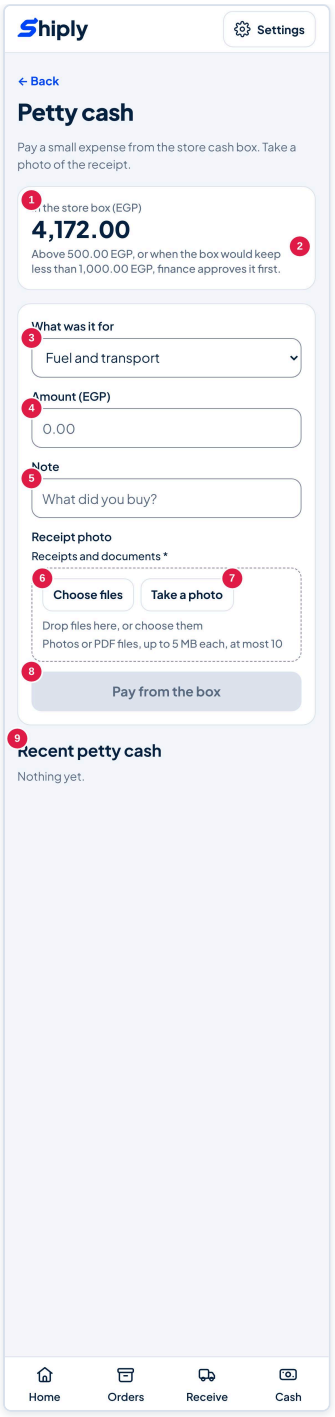
After recording 1,300.00 for Hany: the red difference message, Hany now carrying 0.00 and the handover Waiting.



After recording 370.00 for Mahmoud against 369.00 expected: "Recorded. 1.00 more than expected; finance will review it."

15.14 Store app: Petty cash

Address `/store/<storeId>/petty-cash`, from the store home tile Petty cash. Store staff (store use); data `GET /stores/:id/petty-cash`, component `PettyCashScreen`. Finance reviews these at Accounting, Money out, Petty cash (another chapter).



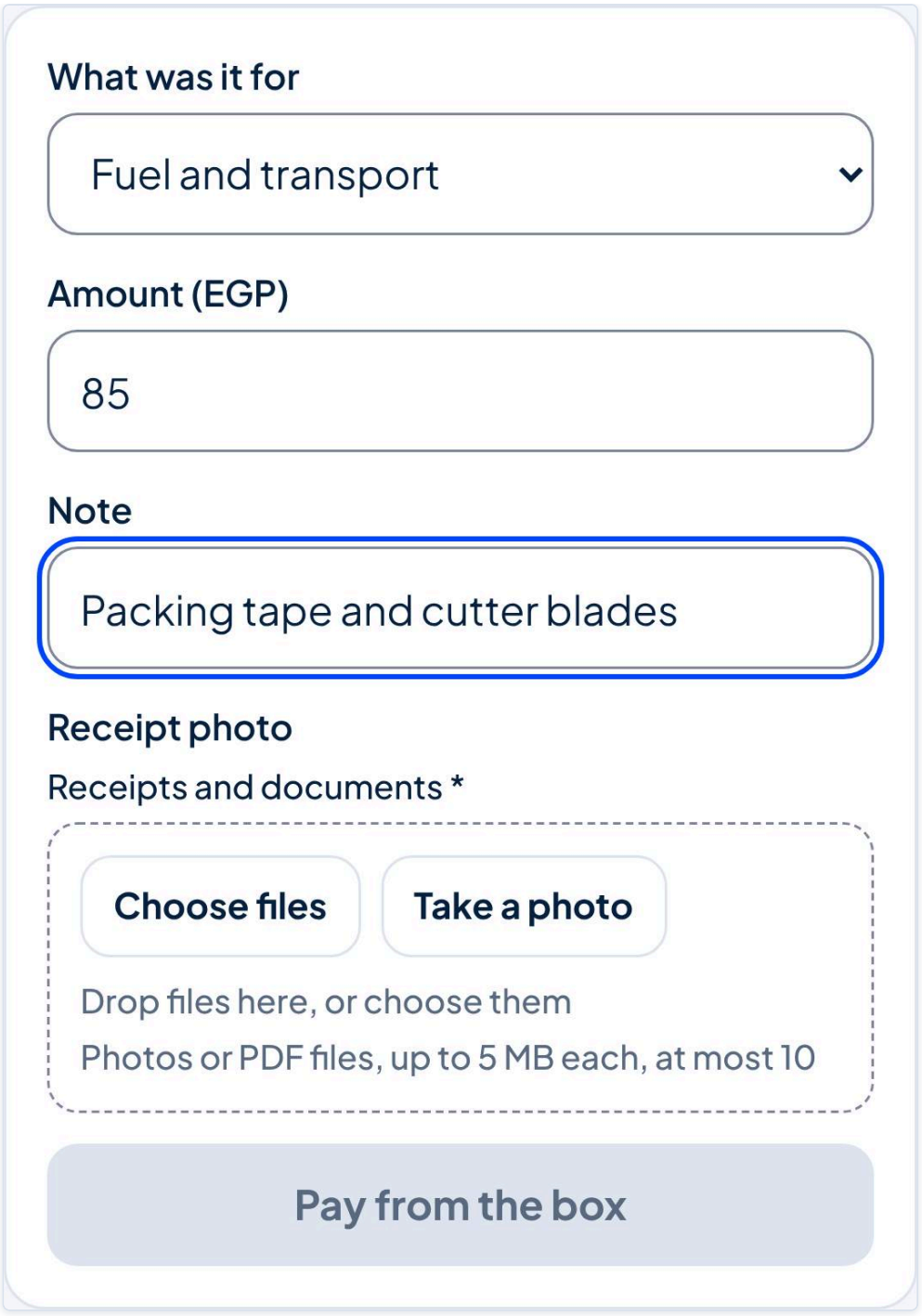
Petty cash on the Maadi store phone, every part numbered.

#	Part	What it shows	What clicking it does
1	In the store box (EGP)	The box balance in large type.	Nothing.
2	Limit hint	"Above 500.00 EGP, or when the box would keep less than 1,000.00 EGP, finance approves it first." (`finance.petty_cash_limit_piastres` and `finance.petty_cash_float_piastres`).	Nothing.

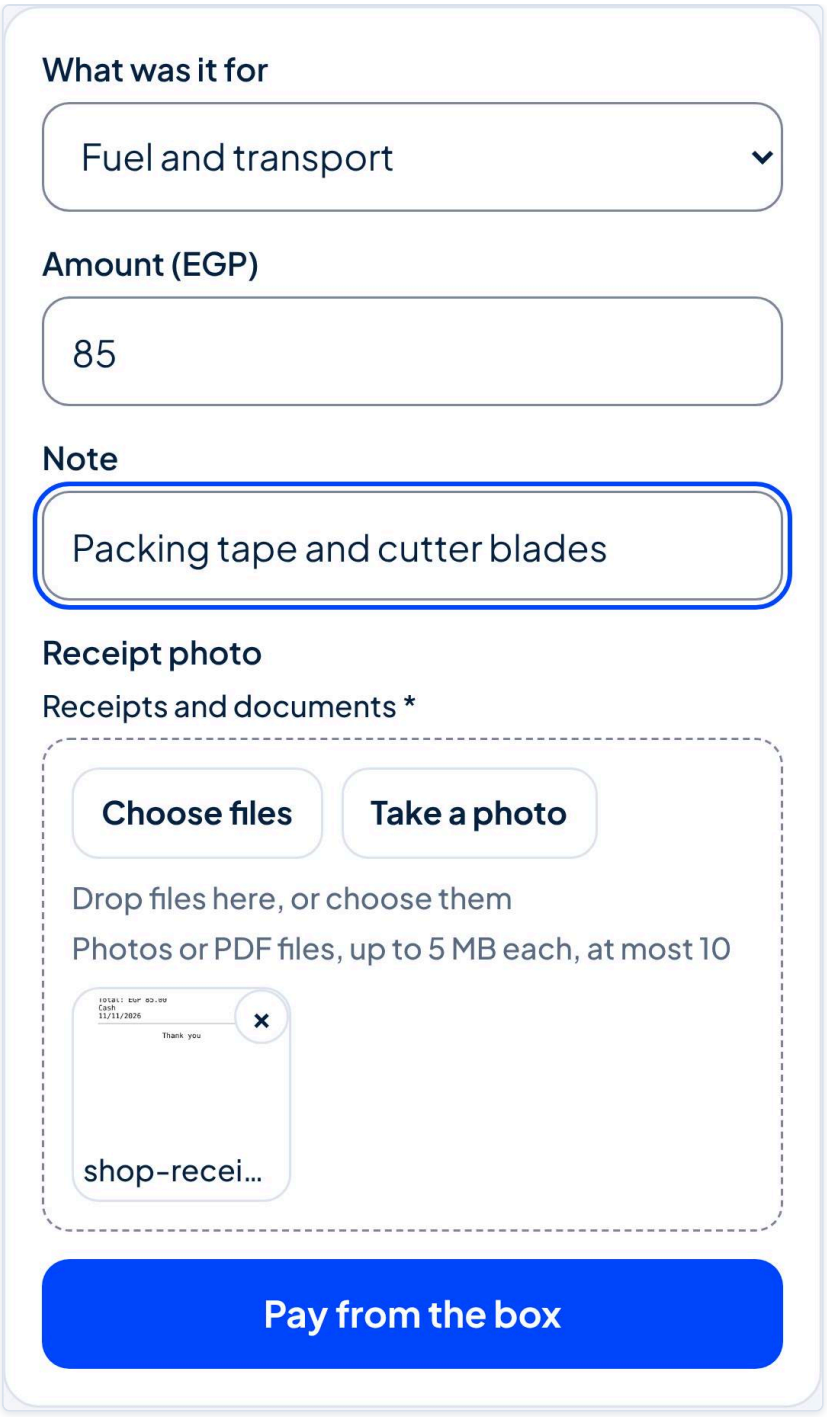
#	Part	What it shows	What clicking it does
3	What was it for	The expense categories finance opened for stores (ledger accounts marked petty). Without any: "Finance has not opened any petty cash category yet." and no form.	Chooses the category. A Cost tag (optional) list appears under it when cost tags exist.
4	Amount (EGP)	Placeholder 0.00.	Typing.
5	Note	Placeholder "What did you buy?"; at least 3 characters.	Typing.
6	Choose files	Under "Receipt photo", the attachment field marked with * (required).	File picker; the first file is the receipt photo, the rest extra attachments.
7	Take a photo	Camera.	Adds the photo.
8	Pay from the box	Full width button, grey until a receipt, an amount and a 3 character note are present.	Saves.
9	Recent petty cash	PC number, status (Paid, Waiting for approval, Rejected, Withdrawn, Reversed), note and time, amount. Empty: "Nothing yet."	Nothing.

Controls on this screen

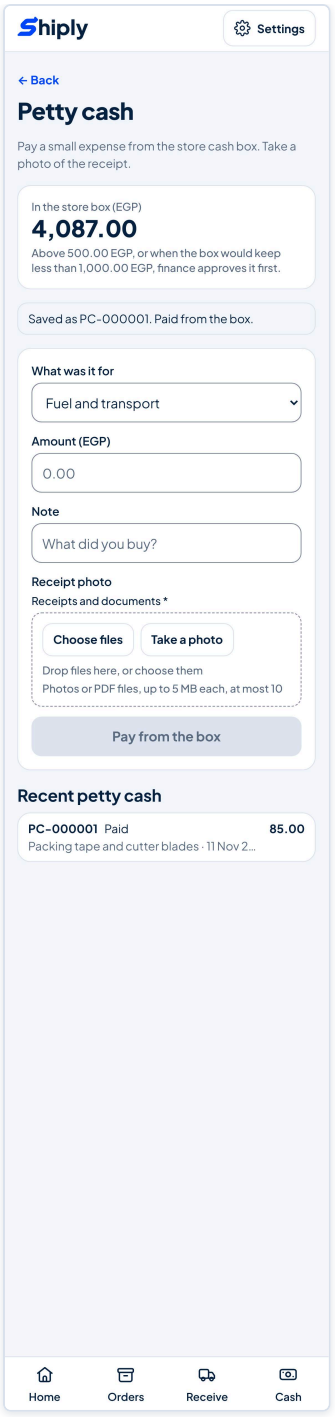
Control	What it does	Rule / limits	Result
Pay from the box	`POST /stores/:id/petty-cash`	Receipt required ("Take a photo of the receipt first."); amount above zero; never more than the box holds ("The box does not hold that much cash."); category open for petty cash; above the limit or below the float it waits for a second person; idempotency key.	"Saved as PC-000001. Paid from the box." (posted: Dr category, Cr store box, store tag) or "Saved as PC-nnnnnn. Waiting for finance to approve it."



Amount and note typed but no receipt yet: the button stays grey.



The receipt photo chosen (thumbnail with a cross); Pay from the box is enabled.



After saving 85.00: "Saved as PC-000001. Paid from the box.", the box down to 4,087.00 and the expense listed as Paid.

15.15 Statuses on these screens at a glance

Object	Status	How it looks and where
Cash handover (CH)	Balanced	Plain text "Balanced" in Recent handovers (finance desk and store phone).
	Waiting (pending)	"Waiting" as plain text; the handover also has an orange bordered card under Differences to review.
	Written off	Plain text with the note in grey under it.
	On rider (charged_to_rider)	Plain text with the note; the rider sees "You owe" in his wallet.

Object	Status	How it looks and where
Basata deposit (BD)	Waiting for review	Finance list: neutral grey bordered pill; deposit page: blue Waiting tab; rider phone: orange text.
	Approved	Neutral pill; blue Approved tab with Waiting ticked; rider phone: green text.
	Rejected	Neutral pill; red Rejected tab; reason in red on the page and on the rider phone.
Bank line	Open	Listed on the left of Bank matching with its best reason chip.
	Matched	Leaves the list; counted in "N matched" and in the progress bar.
	Set aside	Leaves the list; counted in "N set aside" and in the progress bar.
Loan (LN)	Being repaid	Grey text under the title on the loan page; the table shows the next instalment.
	Repaid	Grey "Repaid" under the title and in the Next instalment column.
Loan instalment	Paid	Blue link "Paid" to its entry.
	Overdue	Red "overdue" and red due day; red "(overdue)" in the loans table.
	Not paid	Grey "Not paid".
Rider withdrawal (RW)	Waiting (requested)	Status word at the card's right, with Cancel request under the card.
	Approved	Status word; for cash at the store the manager sees it with Cash given.
	Paid	Green word and a green "Paid ... · reference" line.
	Not approved (rejected)	Status word and "Not approved: reason".
	Cancelled	Grey pill.
Rider	Frozen	Red banner on the rider home and Basata screen; red "Frozen" tag on deposit pages; red line on the store rider card; a card on Frozen riders.
	Not frozen	No banner; "No rider is frozen." on Frozen riders.

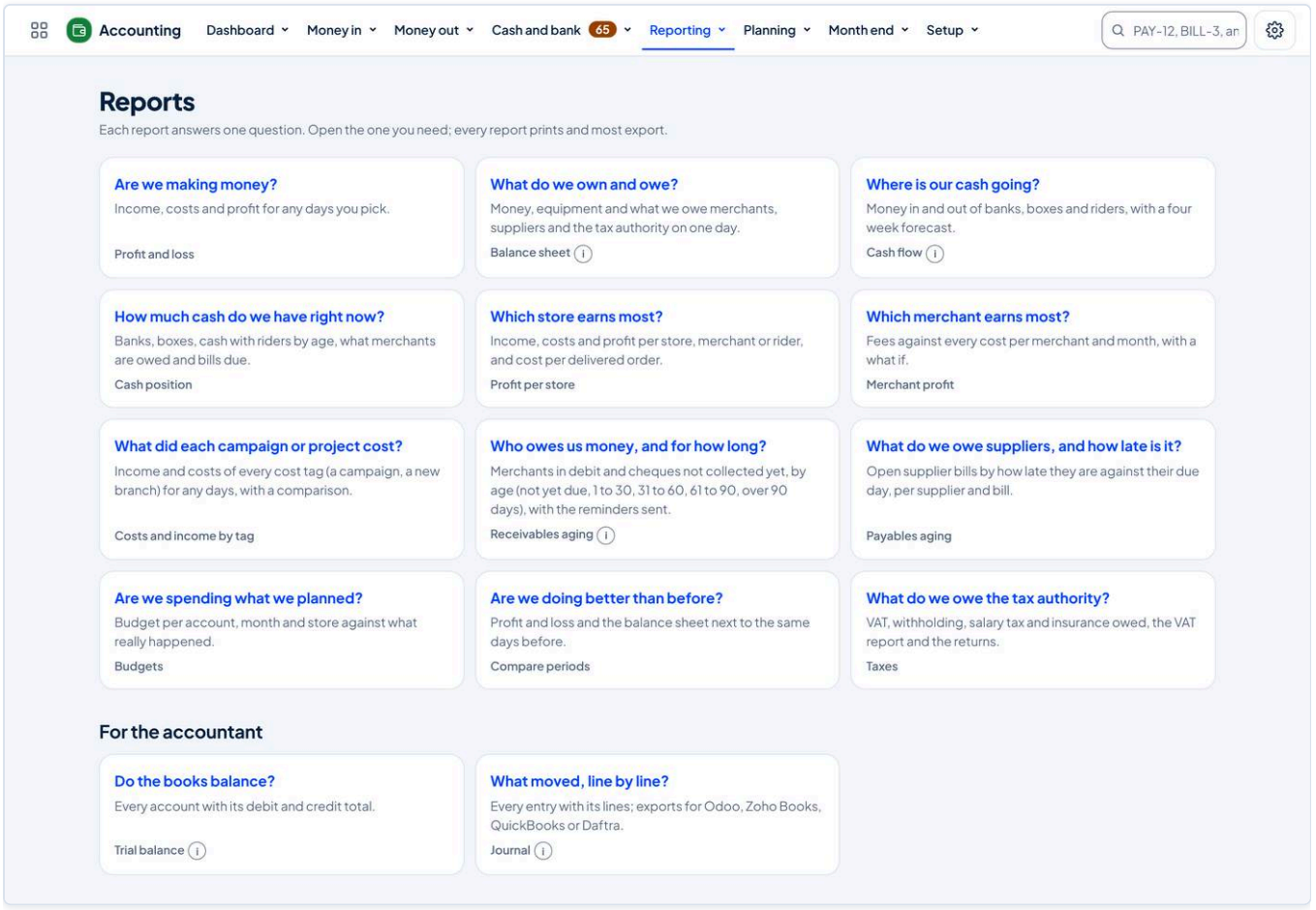
16 Screens: Reporting and Planning

This chapter walks through every screen of the Accounting app's **Reporting** and **Planning** menus exactly as a user sees them, plus the journal entry page and the approval request page that the reports lead to. For each screen it gives the address, the menu path, who may open it and which buttons need more rights, a picture of the whole screen, a table of every control, and pictures of the important states. The numbers behind the reports (how a profit is summed, how the forecast and budgets compute) are explained in the chapter *Taxes, reports, reconciliation and closing*; this chapter is about the screens.

> All screens here need **Accounting view** to open (every GET route of the API defaults to view). Anything that writes needs **Accounting use**; approving a forecast, opening it again, writing it into the budget and deciding an approval request need **Accounting manage**. People limited to some stores see only the lines and boxes of their stores. The pictures use the throwaway demo database; the demo people (for example the finance user and the second approver) are fictional.

16.1 All reports

Address: `/finance/reports`. **Menu:** Accounting, Reporting, All reports. **Who:** Accounting view. The page holds no data of its own: it is a list of cards, each titled with the plain question the report answers (`apps/web/src/app/finance/reports/page.tsx`, texts under `finance.reportsPage`).



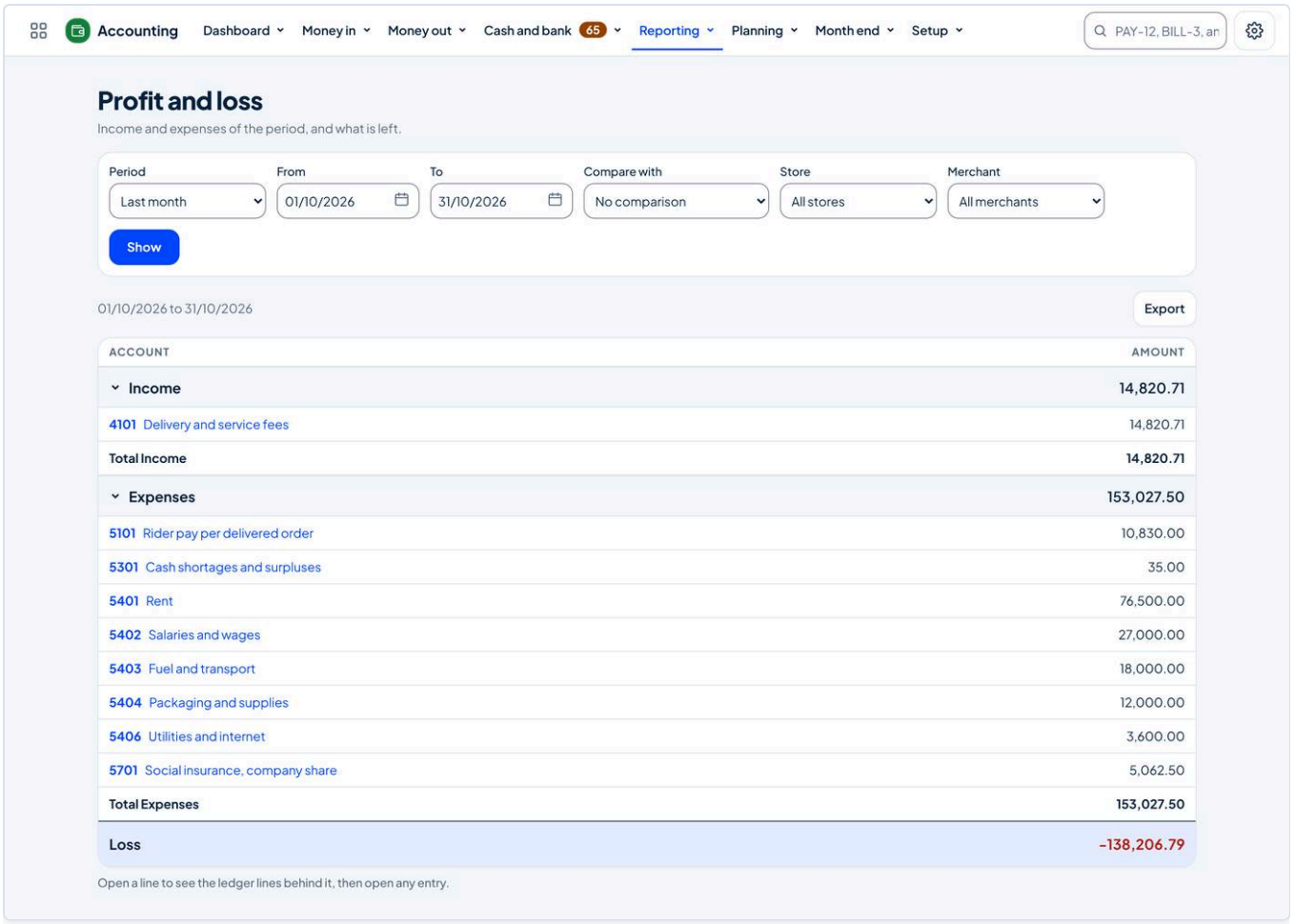
All reports: twelve question cards, then two cards "For the accountant" (/finance/reports).

Controls on this screen

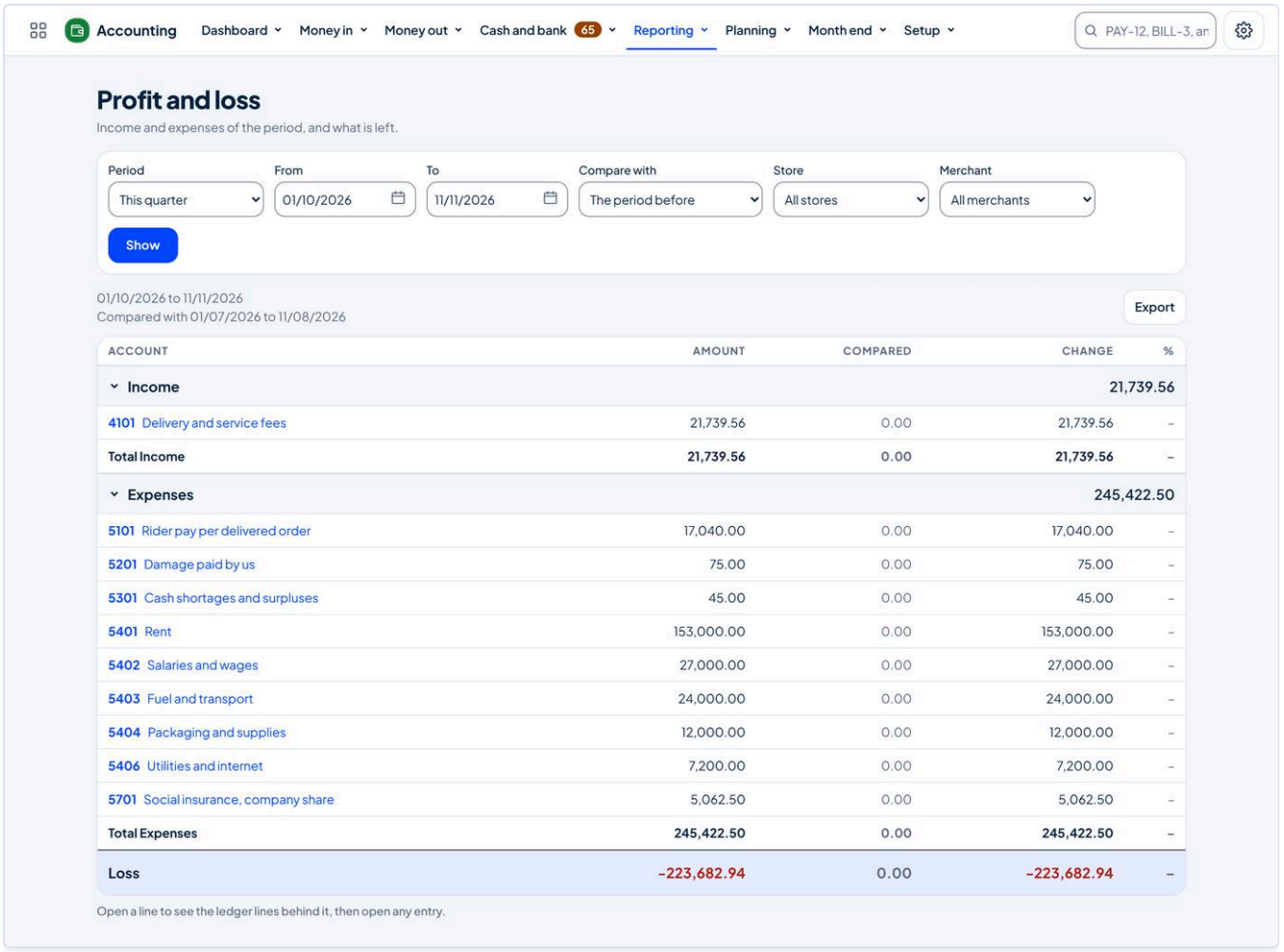
Control	What it does	Rule / limits	Result
Card title (the question, blue link)	Opens the report	Every card shows the question, one sentence of what it contains and the report's usual name below	Opens: Profit and loss, Balance sheet, Cash flow, Cash position, Profit per store, Merchant profit, Costs and income by tag, Receivables aging, Payables aging, Budgets, Compare periods, Taxes
(i) InfoTip beside a name	Explains an accountant's word (balance sheet, cash flow, receivables, trial balance, journal)	Hover or tap	A small bubble with the plain explanation
For the accountant	Trial balance and Journal	Same rights	Opens `/finance/trial` or `/finance/journal`

16.2 Profit and loss, Balance sheet and Trial balance (the drill down report page)

Addresses: `/finance/profit-and-loss` , `/finance/balance-sheet` (the earlier `?day=` still opens that day), `/finance/trial` . **Menu:** Reporting, then Profit and loss, Balance sheet or Trial balance. **Who:** Accounting view. The three are one component, `DrillReportPage` with `ReportFilterBar` and `DrillReportView` (`apps/web/src/components/finance/reports`), fed by `GET /finance/reports/:report` (`drillReport` in `packages/db/src/finance/drill.ts`). With no filters in the address the period is this month up to today and there is no comparison.



Profit and loss for last month, no comparison: Income and Expenses sections, each account a link, then the Loss line (/finance/profit-and-loss?preset=last_month).



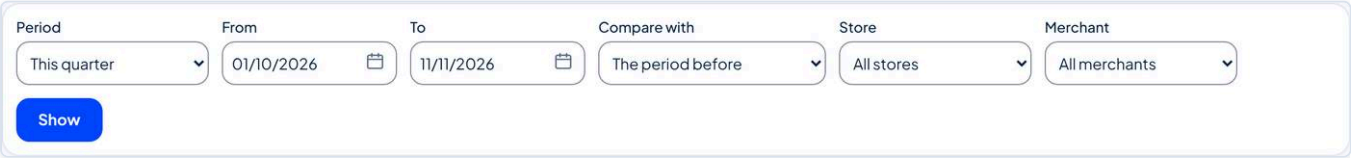
The same report for this quarter compared with the period before: Amount, Compared, Change and % columns; the line under the dates names the comparison days (/finance/profit-and-loss?from=2026-10-01&to=2026-11-11&compare=previous).

Controls on this screen

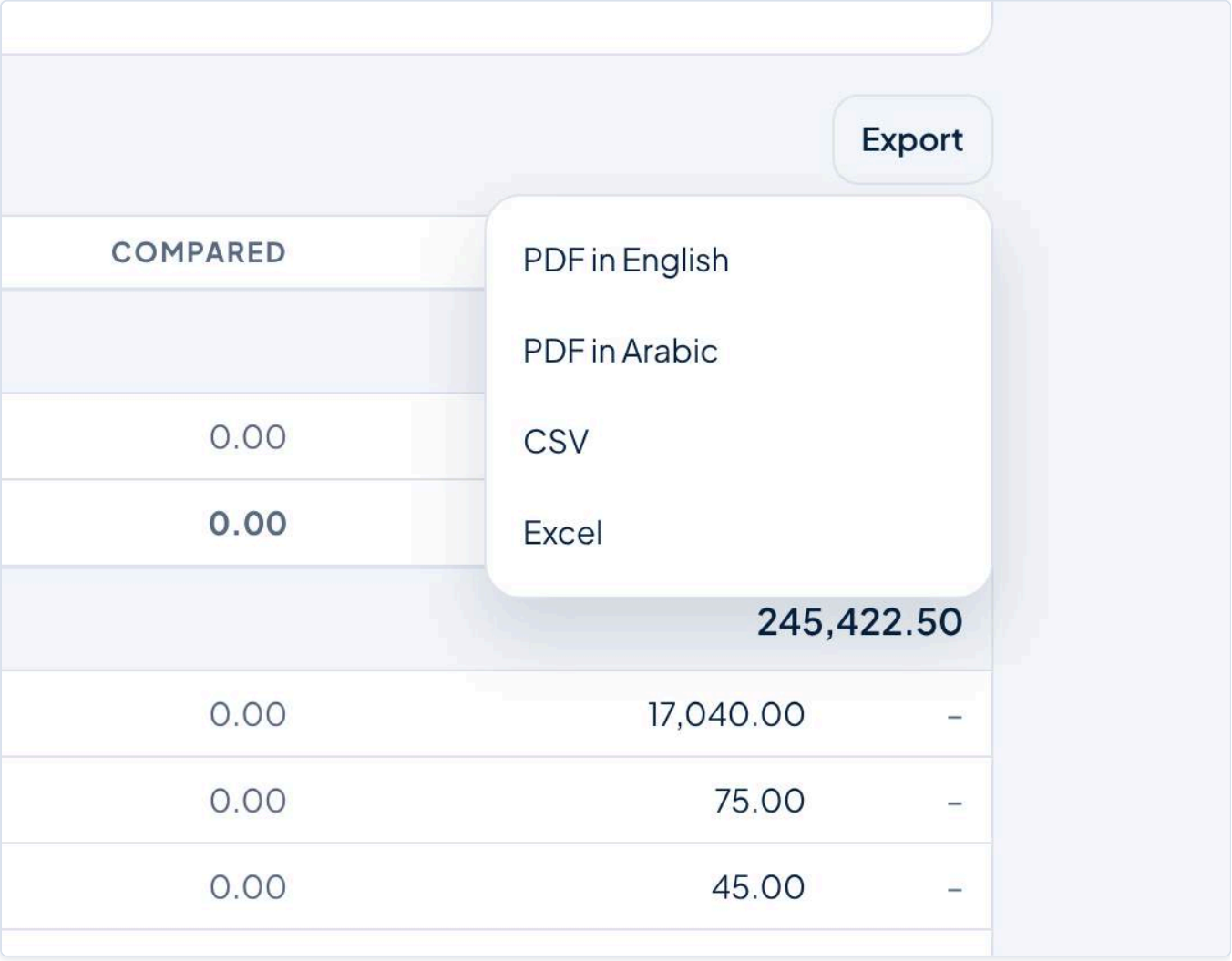
Control	What it does	Rule / limits	Result
Period	Presets: This month, Last month, This quarter, This year, Last year, Choose days	A preset fills From and To at once (`presetRange` in `packages/shared/src/report-period.ts`); typing a day switches the preset to Choose days	Nothing reloads until Show
From, To (DateInput, day first)	The Cairo days of the report	Never after today; From after To is swapped; the API refuses more than 400 days (`range_too_long`)	Written as `from` and `to` in the address
As of (balance sheet only)	The day the balance sheet is taken at the end of	Replaces From and To; the balance sheet sums every line up to that day	Header reads "At the end of dd/mm/yyyy"

Control	What it does	Rule / limits	Result
Compare with	No comparison, The period before, The same days last year	The period before: a range starting on the 1st moves back by its months (this quarter to date goes back one quarter), any other range by its own length	Adds the Compared, Change and % columns and a "Compared with" line
Store	All stores or one store	Counts only lines tagged with that store, so the totals may not balance; the hint says so	The balance sheet's "balances" pill is hidden while filtered
Merchant	All merchants or one merchant	Same rule as Store	Same
Show (blue)	Writes the filters into the address and reloads	The address is the state: Back and a shared link open the same report	New figures
Export	Opens a menu: PDF in English, PDF in Arabic, CSV, Excel	PDFs are made on the API (`GET /finance/reports/:report/export.pdf?lang=` , pdfkit, Arabic laid out right to left); CSV and Excel are built in the browser from the same rows (`exportCsv` , `exportXlsx` ; Excel sheets are right to left for Arabic readers)	A download named `profit-and-loss-<to day>` , `balance-sheet-…` or `trial-balance-…`
Section header (Income, Expenses; Assets, Liabilities, Equity)	Folds or opens the section	The chevron turns; the section total stays visible when folded	Only the header row shows
Account line (code and name, blue)	Opens the ledger lines behind the line	Wallet groups (merchant balances, rider bonuses owed, staff advances...) are one line "Group: N wallets"; Profit so far (balance sheet) opens all income and cost lines	Opens `/finance/reports/lines` with the same days, store and merchant and a way back
Amount / Balance column	Income and liabilities are shown with the sign flipped so a normal balance is positive	A negative figure is red	None
Compared, Change, %	The comparison figure, the change and its percent of the earlier figure	% is a dash when the earlier figure was 0; a rise is green, a fall red	None

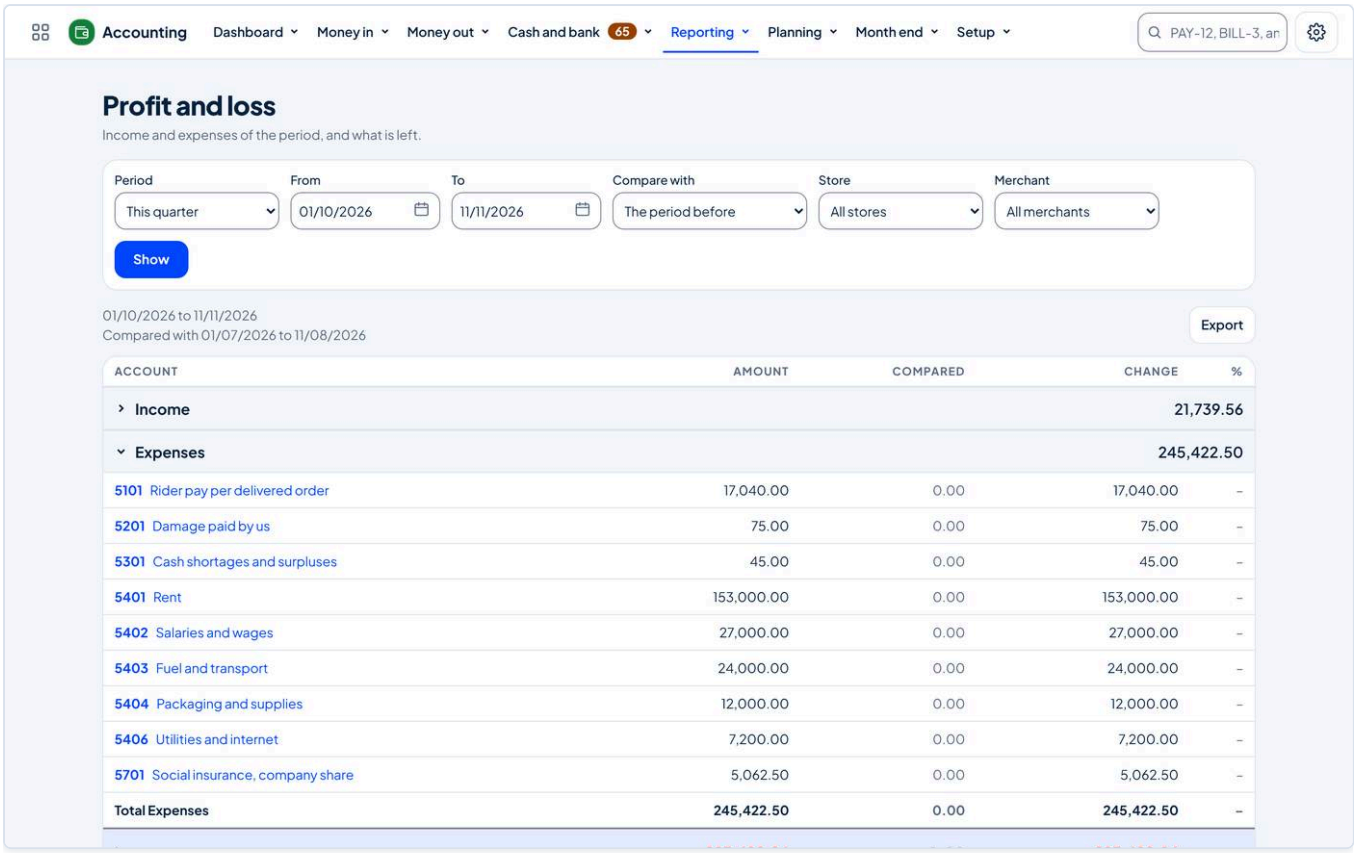
Control	What it does	Rule / limits	Result
Debit, Credit (trial balance only)	Each account's debit and credit totals	The two totals must be equal	Bottom line "Difference (must be 0)"
Bottom line	Profit or Loss (P&L), Difference (must be 0) on the balance sheet and trial balance	The balance sheet also shows the pill "Assets equal liabilities plus equity" or, in red, "Does not balance, call support"	None



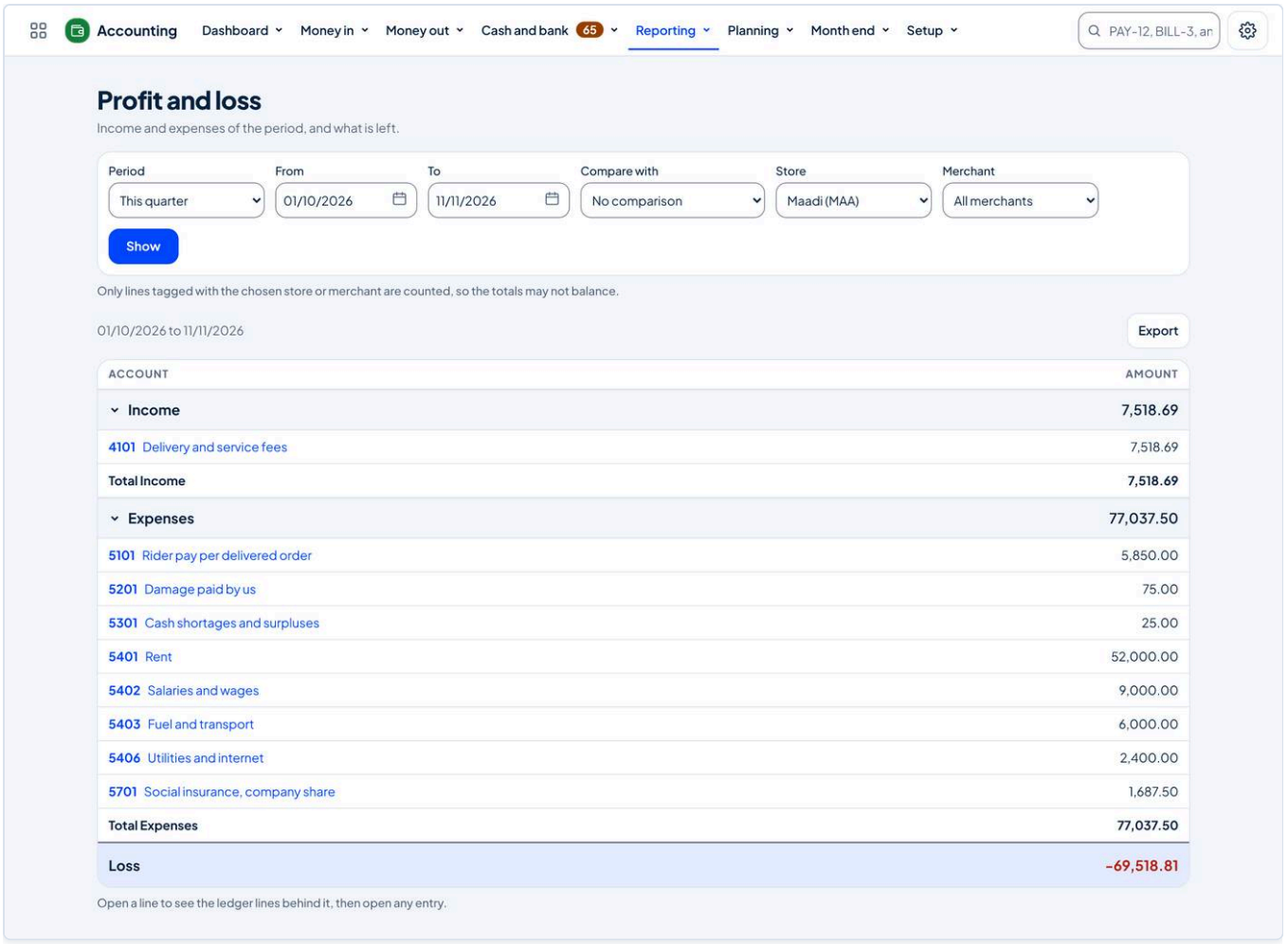
The filter bar every drill down report shares: Period, From, To, Compare with, Store, Merchant and Show.



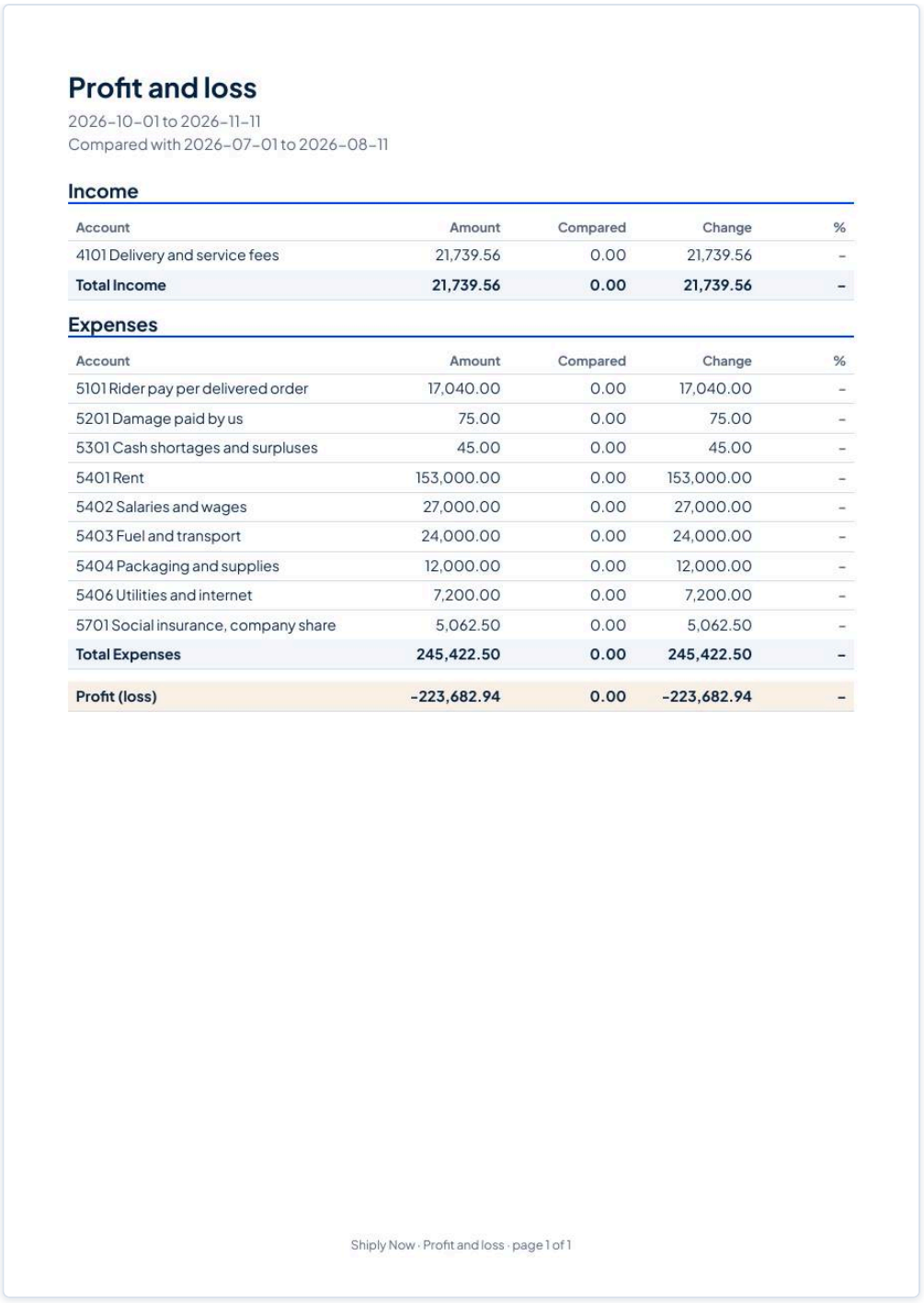
The Export menu open: PDF in English, PDF in Arabic, CSV, Excel.



The Income section folded: only its header and total stay.



Filtered to one store: the hint "Only lines tagged with the chosen store or merchant are counted, so the totals may not balance" appears under the bar.



Profit and loss

2026-10-01 to 2026-11-11
Compared with 2026-07-01 to 2026-08-11

Income

Account	Amount	Compared	Change	%
4101 Delivery and service fees	21,739.56	0.00	21,739.56	-
Total Income	**21,739.56**	**0.00**	**21,739.56**	**-**

Expenses

Account	Amount	Compared	Change	%
5101 Rider pay per delivered order	17,040.00	0.00	17,040.00	-
5201 Damage paid by us	75.00	0.00	75.00	-
5301 Cash shortages and surpluses	45.00	0.00	45.00	-
5401 Rent	153,000.00	0.00	153,000.00	-
5402 Salaries and wages	27,000.00	0.00	27,000.00	-
5403 Fuel and transport	24,000.00	0.00	24,000.00	-
5404 Packaging and supplies	12,000.00	0.00	12,000.00	-
5406 Utilities and internet	7,200.00	0.00	7,200.00	-
5701 Social insurance, company share	5,062.50	0.00	5,062.50	-
Total Expenses	**245,422.50**	**0.00**	**245,422.50**	**-**
Profit (loss)	**-223,682.94**	**0.00**	**-223,682.94**	**-**

Shiply Now · Profit and loss · page 1 of 1

First page of the PDF in English made by the API.

الأرباح والخسائر

من 01-10-2026 إلى 11-11-2026

مقارنة بالفترة من 01-07-2026 إلى 11-08-2026

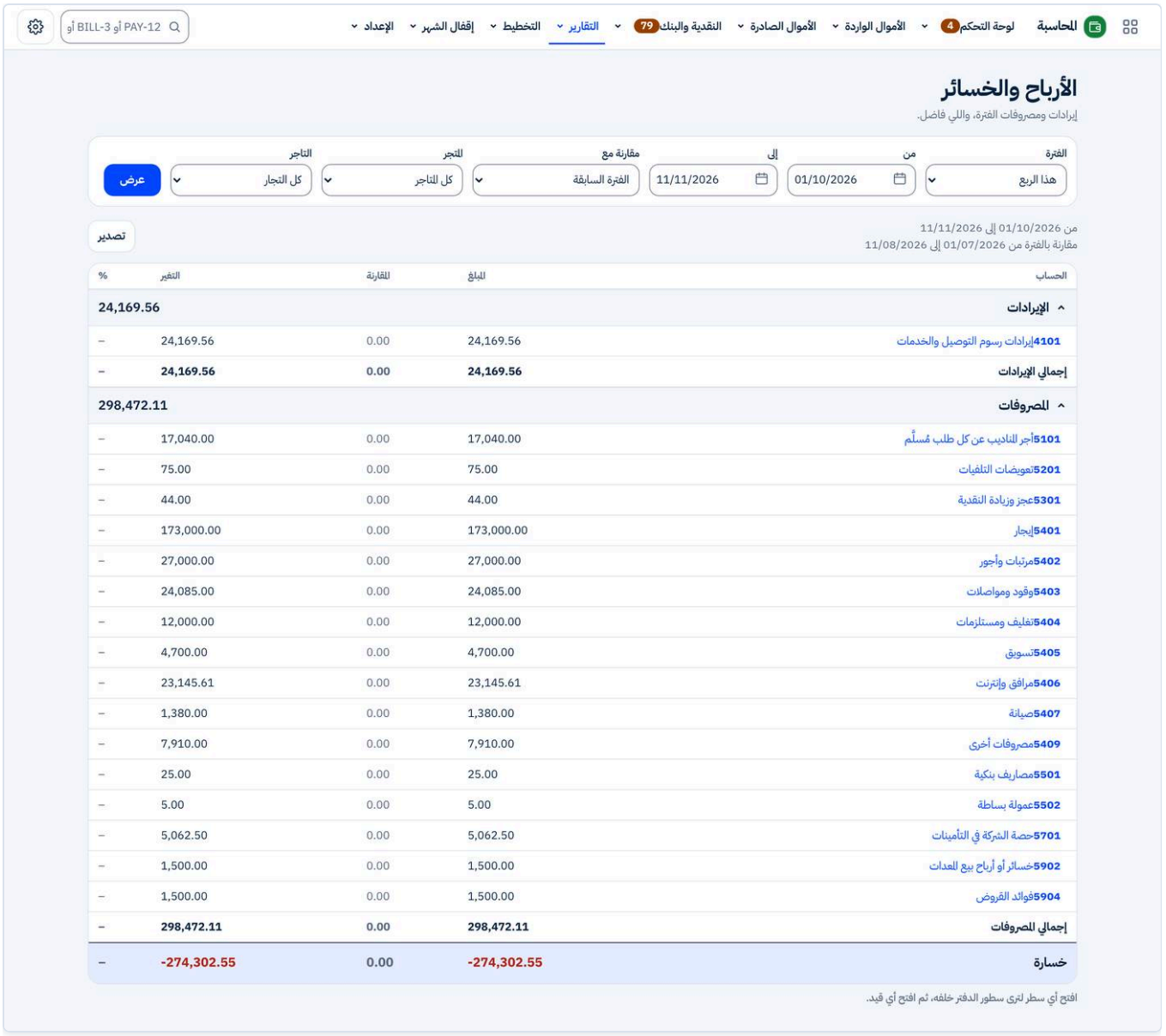
الإيرادات

الحساب	المبلغ	المقارنة	التغيير	٪
4101 إيرادات رسوم التوصيل والخدمات	21,739.56	0.00	21,739.56	–
الإجمالي الإيرادات	**21,739.56**	**0.00**	**21,739.56**	**–**

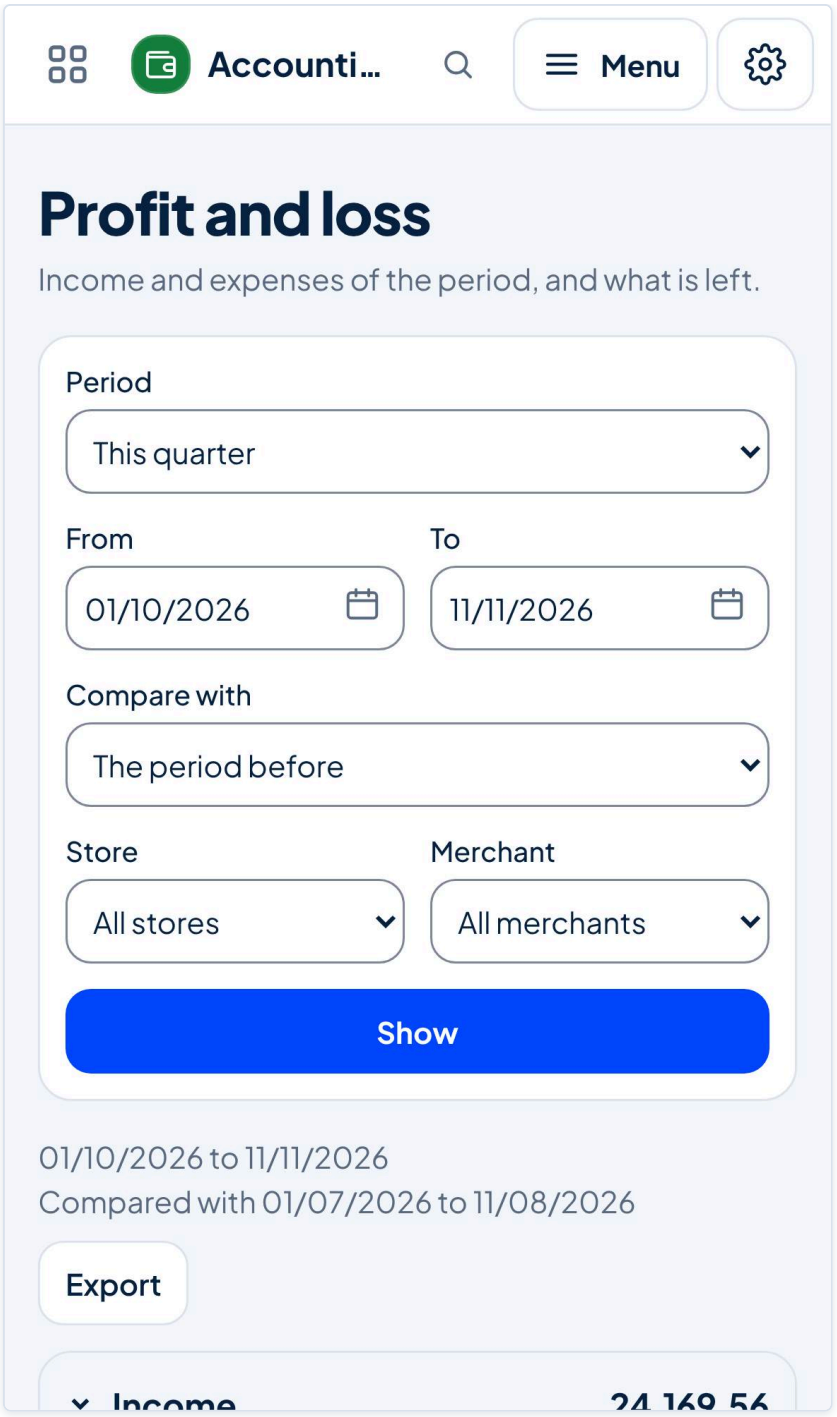
المصروفات

الحساب	المبلغ	المقارنة	التغيير	٪
5101 أجر المناديب عن كل طلب مُسلَّم	17,040.00	0.00	17,040.00	–
5201 تعويضات التلفيات	75.00	0.00	75.00	–
5301 عجز وزيادة النقدية	45.00	0.00	45.00	–
5401 إيجار	153,000.00	0.00	153,000.00	–
5402 مرتبات وأجور	27,000.00	0.00	27,000.00	–
5403 وقود ومواصلات	24,000.00	0.00	24,000.00	–
5404 تغليف ومستلزمات	12,000.00	0.00	12,000.00	–
5406 مرافق وإنترنت	7,200.00	0.00	7,200.00	–
5701 حصة الشركة في التأمينات	5,062.50	0.00	5,062.50	–
الإجمالي المصروفات	**245,422.50**	**0.00**	**245,422.50**	**–**
الربح (الخسارة)	**-223,682.94**	**0.00**	**-223,682.94**	**–**

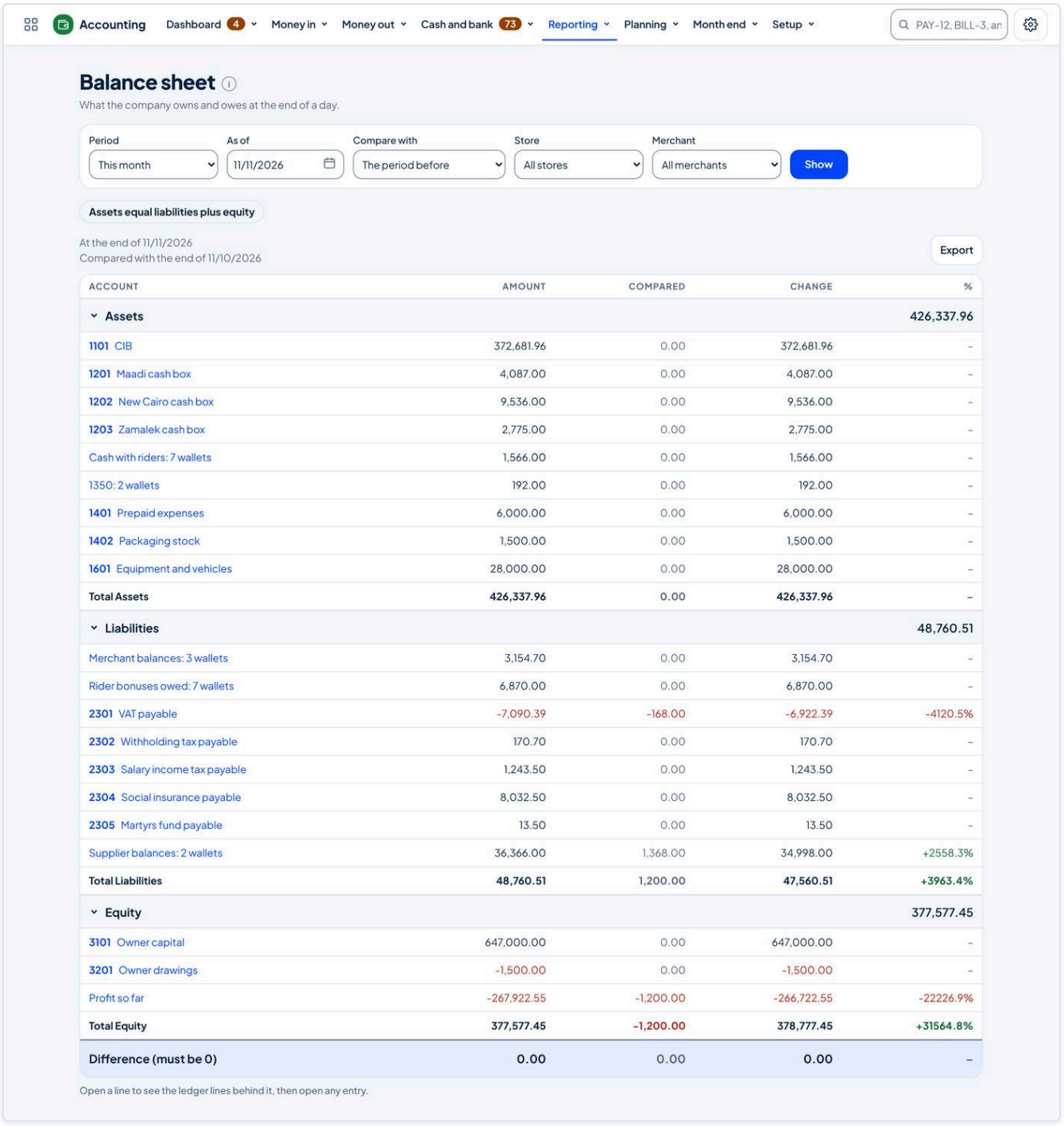
The same report as the PDF in Arabic, laid out right to left.



The page itself in Arabic (cookie shiply_locale=ar): the whole layout mirrors, figures stay left to right.



On a phone each section is a card; every line shows its amount and, under it, the compared figure and percent.

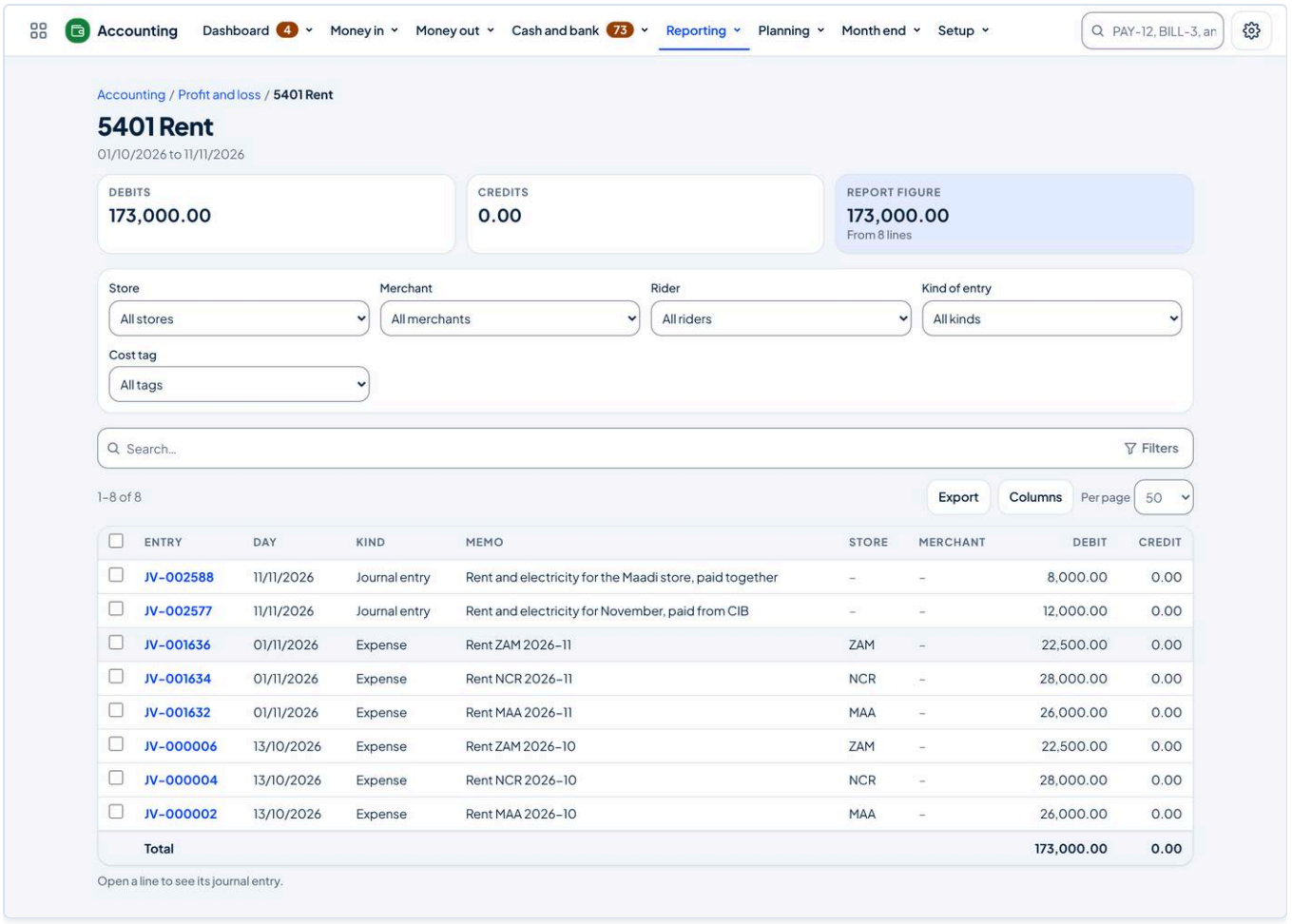


Balance sheet as of today compared with the end of the month before, with the green "Assets equal liabilities plus equity" pill (/finance/balance-sheet?compare=previous).

Trial balance for this quarter: Debit, Credit and Balance per account; the difference row must read 0.00 (/finance/trial).

The ledger lines behind a report line

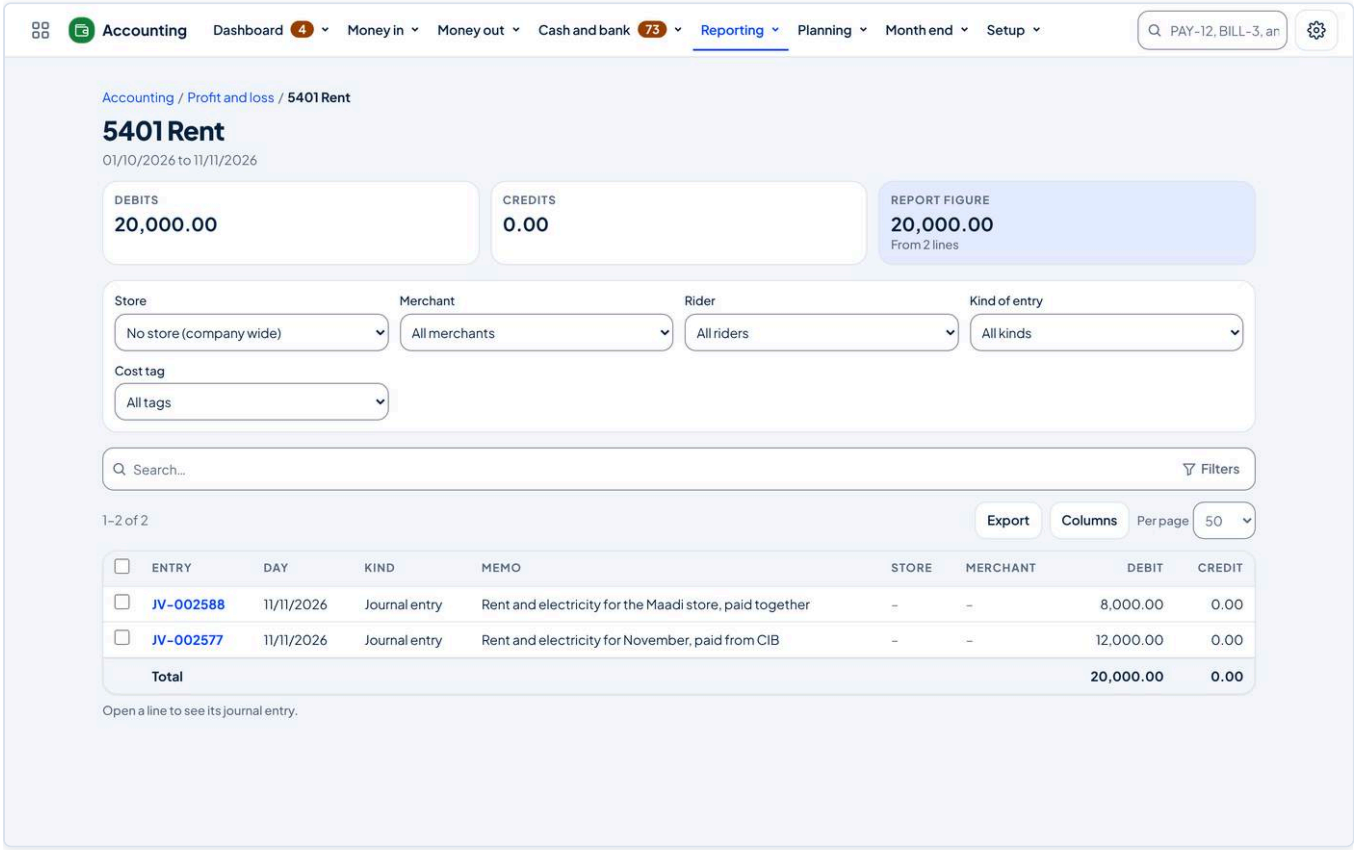
Address: `/finance/reports/lines?account=<id>` (or `group=` a wallet group, `set=income_costs` or `set=money`), with `from`, `to`, `store`, `merchant`, `rider`, `kind`, `tag` and `back`. It is opened from a report line, a profit per store row, a cash flow kind or a cost tag. Data: `GET /finance/ledger-lines`. Breadcrumbs lead back to the report with its filters (only for our own report pages, listed in `BACKS`).



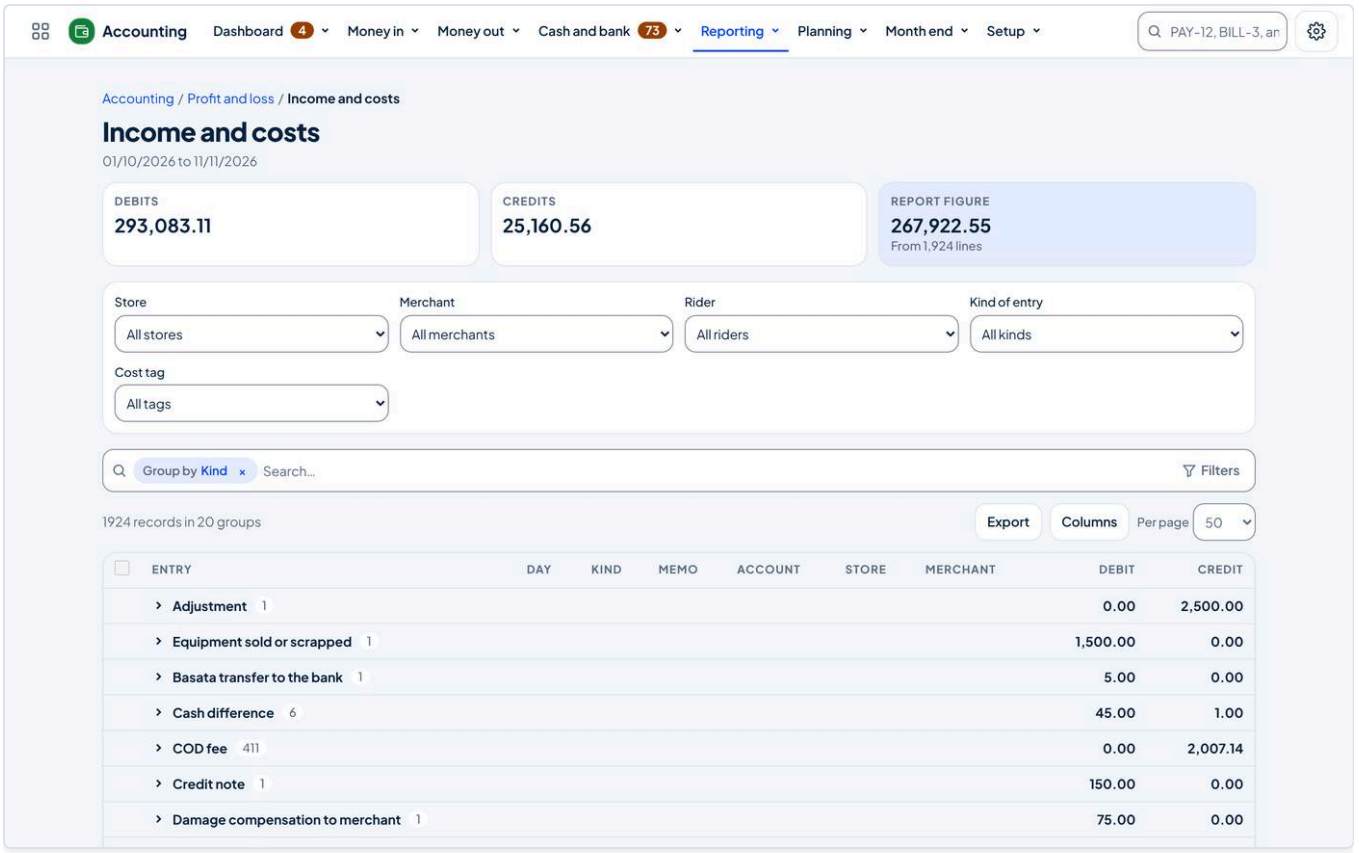
5401 Rent opened from the profit and loss: three figure cards (Debits, Credits, Report figure from 8 lines), the drill filters, then the list of lines; each row opens its journal entry.

Control	What it does	Rule / limits	Result
Breadcrumbs (Accounting / report / what)	Back to the report with the same filters	Only addresses under `/finance/` named in the page's list are trusted as a way back	The report reopens as it was
Debits, Credits, Report figure cards	Totals of the passing lines; Report figure is the net in the report's sign (credit accounts flipped)	"From N lines" counts the passing lines	Matches the report line it came from
Store	All stores, No store (company wide), or one store	Each change rewrites only its own parameter and goes back to page 1	The list reloads at once
Merchant	All merchants, No merchant, or one merchant	Same	Same
Rider	All riders or one rider (name and badge)	Same	Same
Kind of entry	All kinds or one kind (Order fee, Expense, Payout to merchant...)	Same	Same
Cost tag	All tags, No tag, or one tag	Shown only when cost tags exist	Same

Control	What it does	Rule / limits	Result
Search and Filters (list view)	Search entry number, memo, order, merchant, store; filters Debits, Credits, Deliveries and fees, Cash, Payouts, Bills, Journal entries by hand, With an order, No store; group by Month, Day, Store, Merchant, Rider, Kind, Account, Cost tag	Filters of one group are joined by "or", groups by "and"	See "The list view" in the journal section
Columns	Entry, Day, Kind, Memo, Account (hidden when one account is drilled), Store, Merchant, Rider, Order, Cost tag, Debit, Credit, Net	Optional columns are switched on in Columns	None
A row	Opens the journal entry	The row ids are kept for previous and next on the entry page	`/finance/journal/<id>`
"Only the newest lines are listed" (red)	Shown when more than 5,000 lines match	`LIST_MAX_ROWS`	Narrow the days to see all



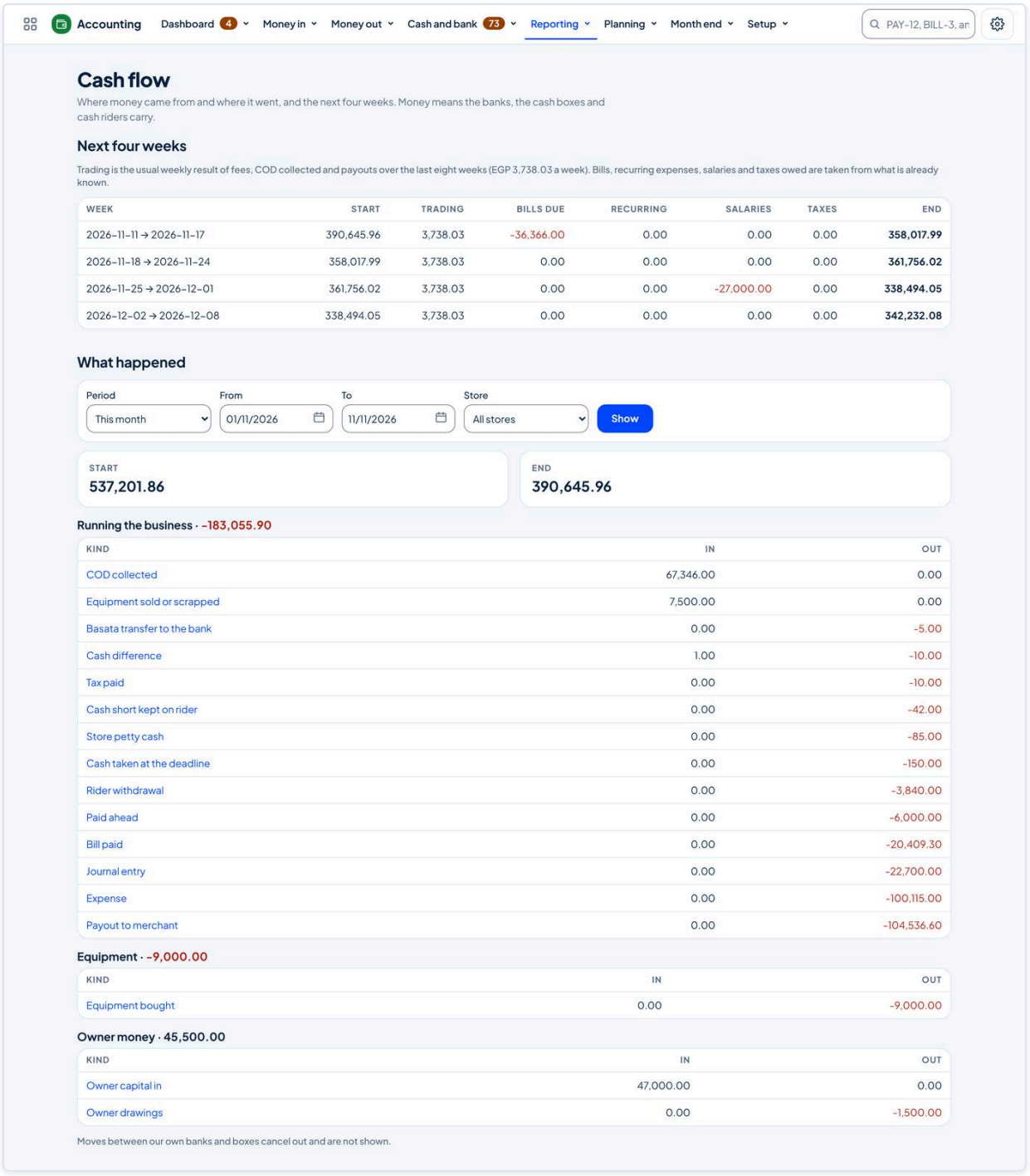
The same drill with Store set to "No store (company wide)": the figures change at once.



All income and cost lines of the period grouped by kind of entry, with a subtotal per group.

16.3 Cash flow and the four week forecast

Address: `/finance/money-flow`. **Menu:** Reporting, Cash flow. **Who:** Accounting view. Data: `GET /finance/cash-flow` (direct method over banks, cash boxes and rider cash) and `GET /finance/cash-forecast`. The forecast always covers the next four weeks from today; the filters change only "What happened".



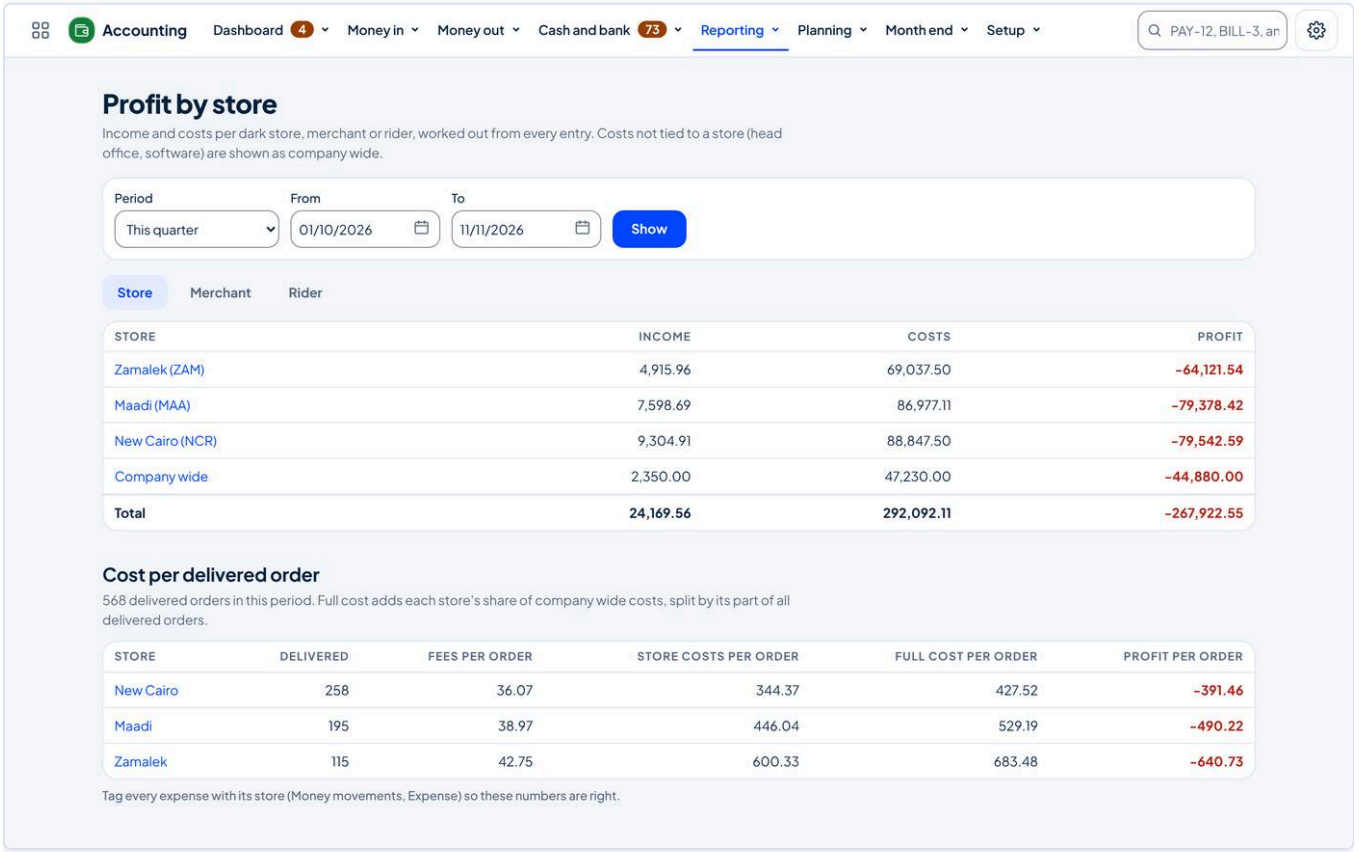
Cash flow: Next four weeks table at the top, then What happened with Start and End cards and the three sections (/finance/money-flow).

Control	What it does	Rule / limits	Result
Next four weeks: Week, Start, Trading, Bills due, Recurring, Salaries, Taxes, End	One row per week	Trading is the average weekly result of fees, COD collected and payouts over the last eight weeks (the hint names the amount); bills, recurring expenses, salaries at month end and taxes owed come from what is known	A red line "Money runs out in this forecast: plan a payout date, a delay or owner money now" appears when any End is below zero
Period, From, To, Store, Show	The period of What happened	No comparison and no merchant on this page	Address `from`, `to`, `store`

Control	What it does	Rule / limits	Result
Start, End cards	Money at the start and at the end of the period	Money means banks, cash boxes and rider cash	None
Running the business, Equipment, Owner money	Sections with their net total in the title	Moves between our own banks and boxes cancel out and are not shown	None
Kind (blue link), In, Out	Money in and out per kind of entry	Out is shown negative in red	Opens the money lines of that kind (`set=money&kind=`)

16.4 Profit per store

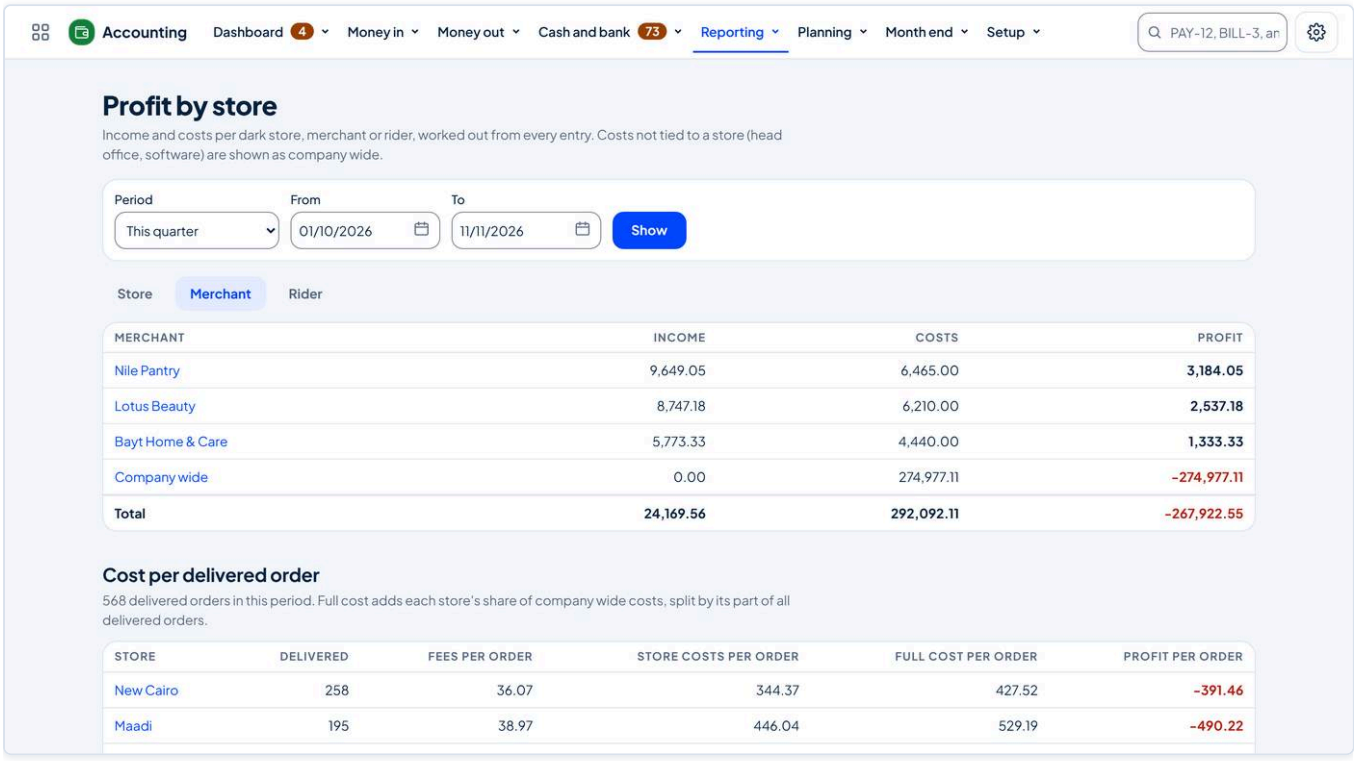
Address: `/finance/profit?by=store|merchant|rider` . **Menu:** Reporting, Profit per store (the page title reads "Profit by store"). **Who:** Accounting view. Data: `GET /finance/profit` and `GET /finance/cost-per-order` .



Profit by store for this quarter, then Cost per delivered order per store (/finance/profit).

Control	What it does	Rule / limits	Result
Period, From, To, Show	The days	No comparison, store or merchant filter here	Address `from` , `to` , `by`
Store, Merchant, Rider tabs	Changes the dimension	Keeps the days	Rows per store, merchant or rider

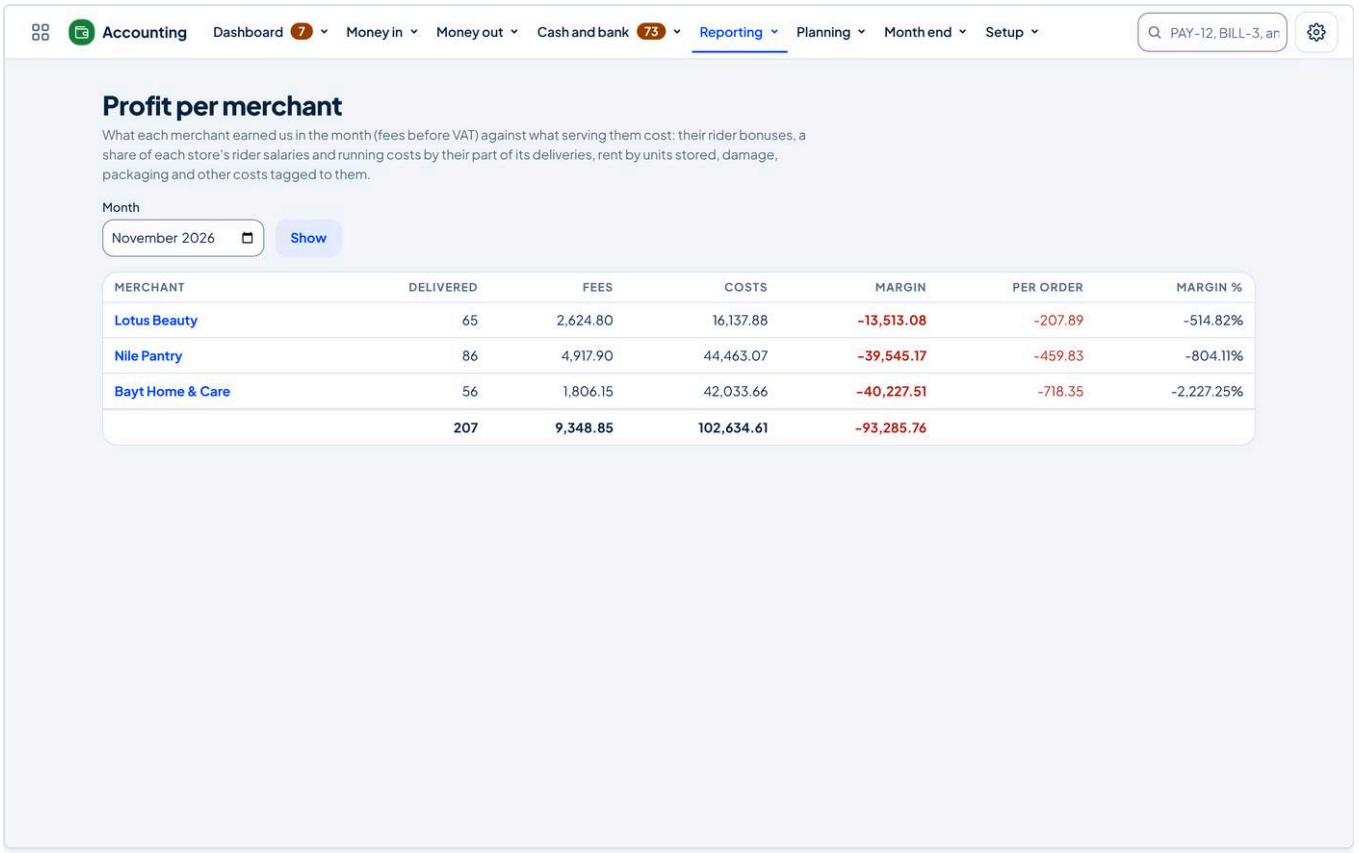
Control	What it does	Rule / limits	Result
Name (blue)	Opens every income and cost line of that store, merchant or rider	"Company wide" opens lines without a store; for riders the company wide row is plain text	`/finance/reports/lines?set=income_costs&store=…`
Income, Costs, Profit	Sums of the tagged lines	A loss is red	Total row at the bottom
Cost per delivered order: Delivered, Fees per order, Store costs per order, Full cost per order, Profit per order	Per store	Full cost adds each store's share of company wide costs, by its part of all delivered orders	The hint asks to tag every expense with its store



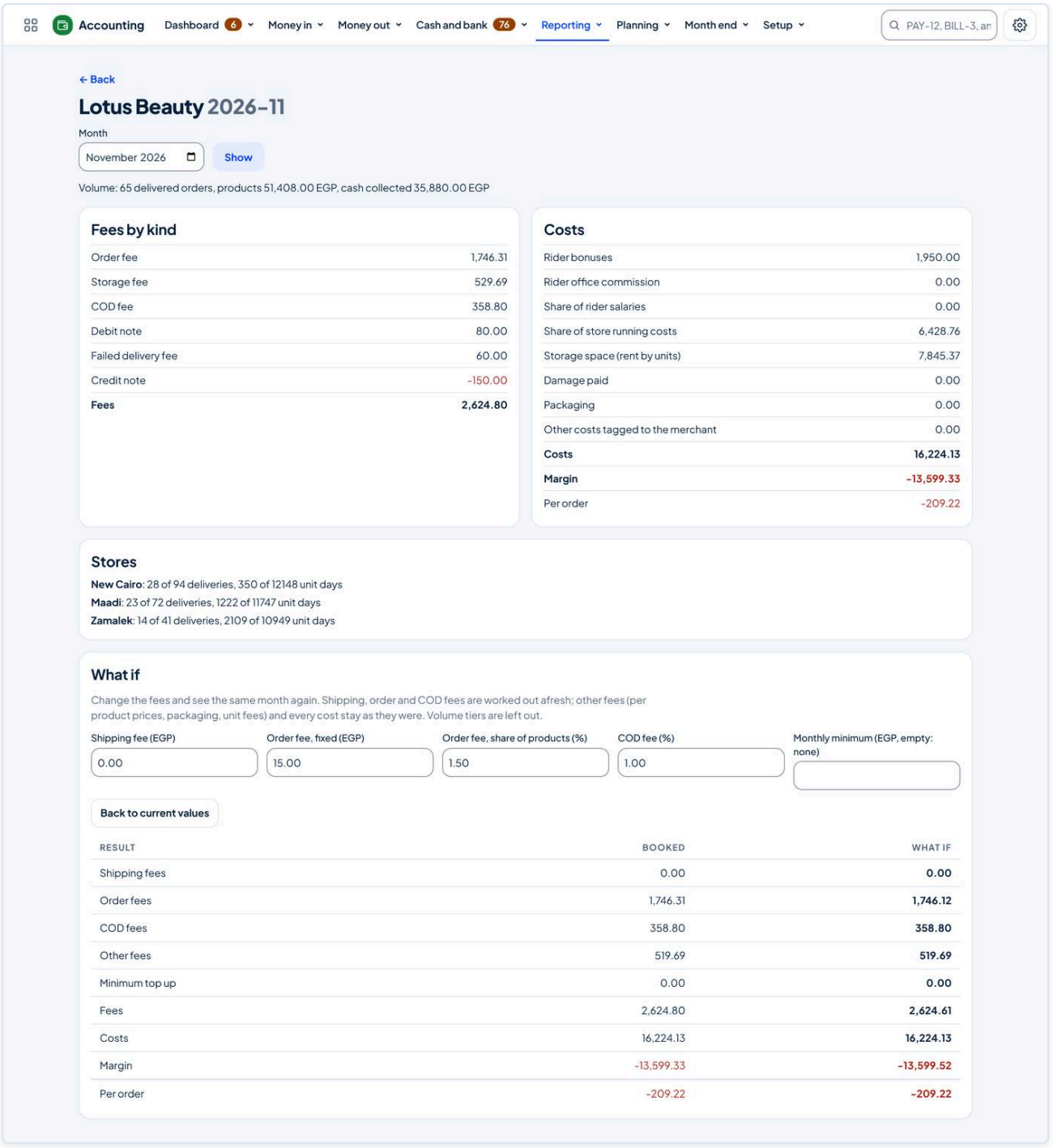
The Merchant tab: income, costs and profit per merchant.

16.5 Merchant profit and the what if

Addresses: `/finance/merchant-profit?month=YYYY-MM` and `/finance/merchant-profit/<merchantId>?month=` . **Menu:** Reporting, Merchant profit. **Who:** Accounting view. Data: `GET /finance/merchant-profit` . The method is explained in *Merchant money*.



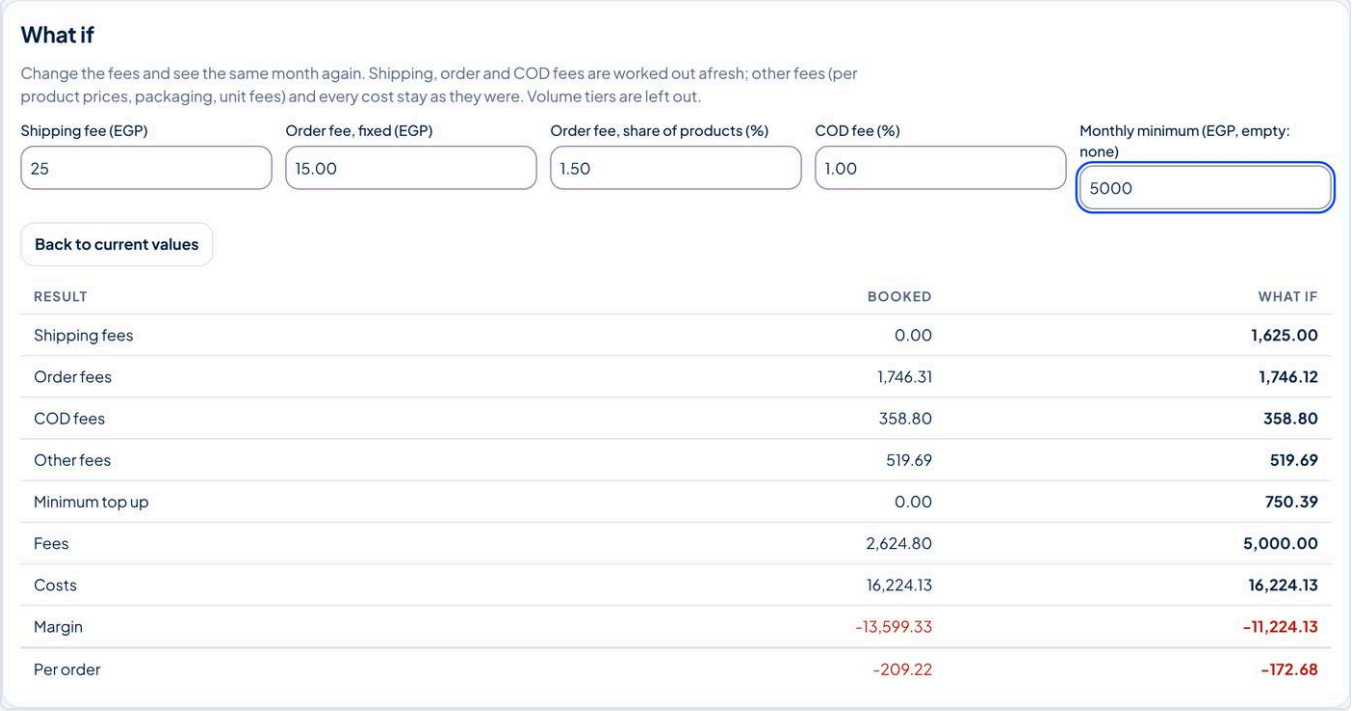
Profit per merchant for this month: Delivered, Fees, Costs, Margin, Per order, Margin % and a total row (/finance/merchant-profit).



One merchant's month: volume, fees by kind, every cost line, margin, stores with their share of deliveries and unit days, and the What if (/finance/merchant-profit/<id>?month=2026-11).

Control	What it does	Rule / limits	Result
Month, Show	Picks the Cairo month	One month at a time	Address `month`
Merchant name	Opens the merchant's month	None	Detail page
← Back	Back to the list	Keeps the month	List
Fees by kind	Fees before VAT per kind, credit notes negative, debit notes positive	From the merchant's tagged revenue lines	None

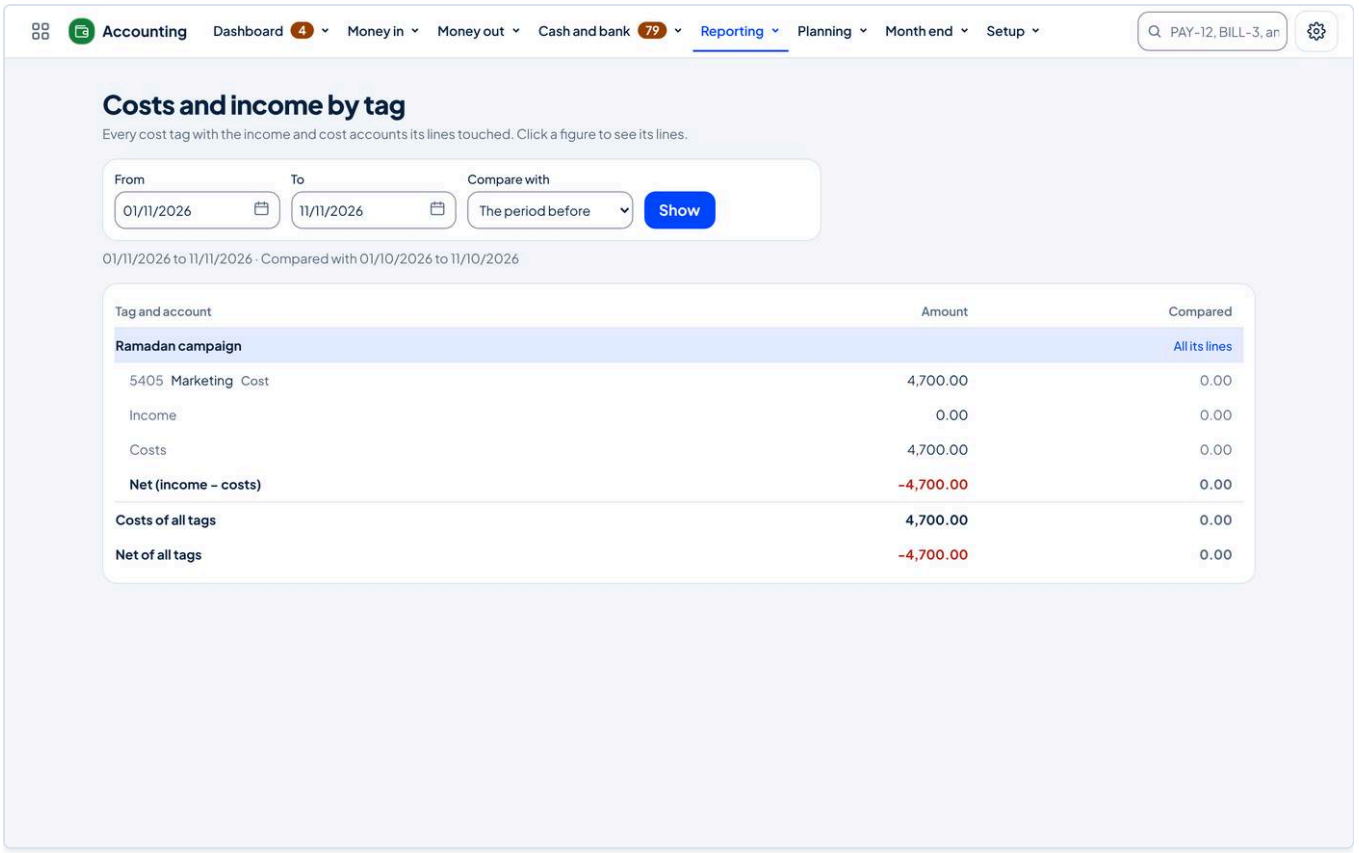
Control	What it does	Rule / limits	Result
Costs	Rider bonuses, office commission, share of rider salaries, share of store running costs, storage space (rent by unit days), damage paid, packaging, other costs tagged to the merchant	Shares follow the merchant's part of each store's deliveries and unit days (shown under Stores)	Margin and Per order
What if: Shipping fee, Order fee fixed, Order fee share of products, COD fee, Monthly minimum	Type new prices	Recalculates at once in the browser on the same month's volume; other fees and every cost stay as booked; volume tiers are left out (a note says so when the merchant has tiers)	The Result table: Booked against What if for shipping, order, COD, other fees, minimum top up, fees, costs, margin, per order
Back to current values	Refills the fields with the contract's prices	None	What if equals Booked again



What if with a 25 EGP shipping fee and a 5,000 EGP monthly minimum: shipping fees 1,625.00 and a minimum top up of 750.39 lift the fees to exactly 5,000.00.

16.6 Costs and income by tag

Address: `/finance/reports/tags?from=&to=&compare=` . **Menu:** Reporting, Costs and income by tag. **Who:** Accounting view (tags are opened by Accounting manage under Setup, Cost tags). Data: `GET /finance/cost-tags/report` . Default period: this month to today.

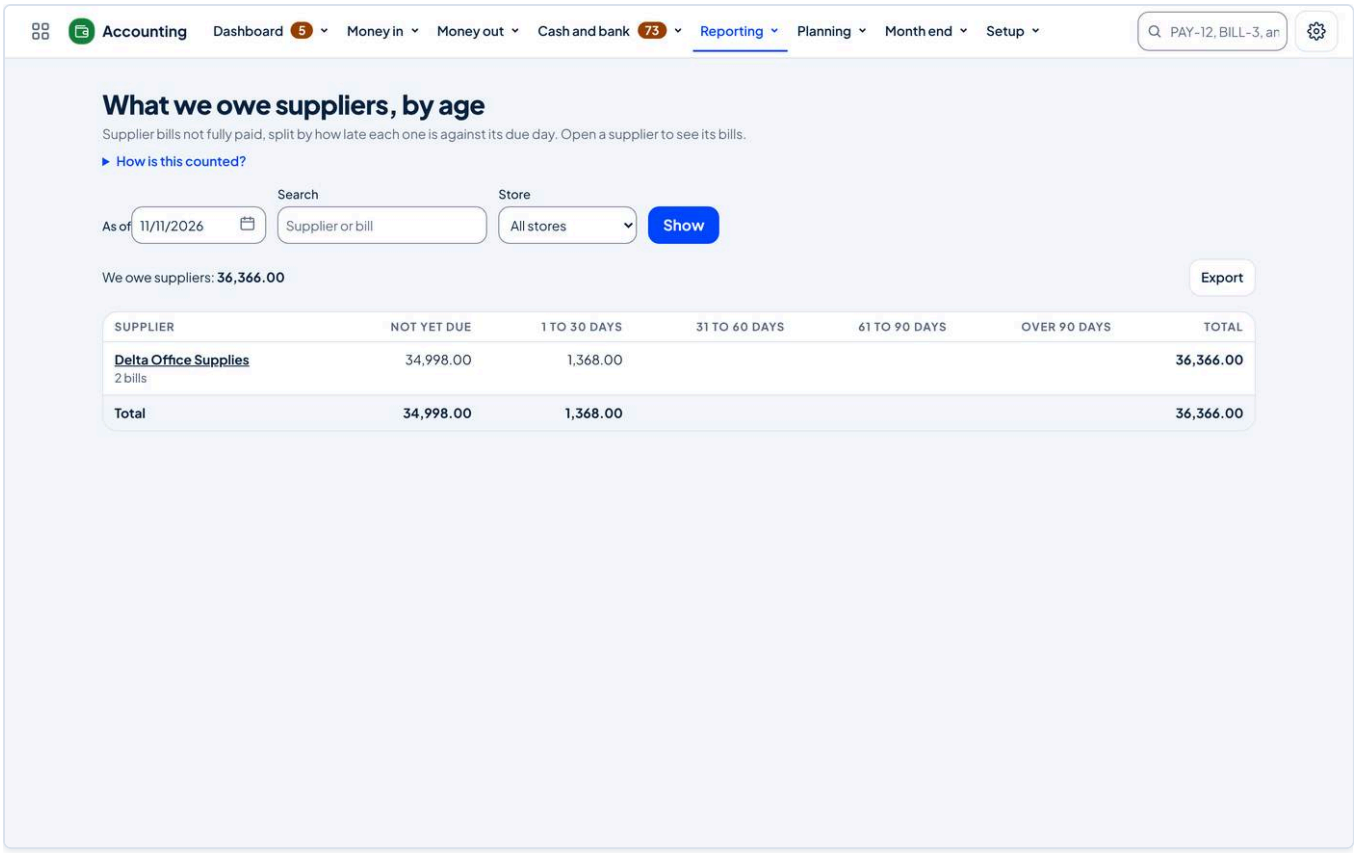


The Ramadan campaign tag with the marketing account its two expenses touched, its income, costs and net, compared with the period before.

Control	What it does	Rule / limits	Result
From, To, Compare with, Show	A plain GET form	Same compare modes as the drill reports	The address holds the three values
Tag row (blue band)	The tag's name; "(switched off)" when the tag is inactive	Only tags with lines in the days are listed	None
All its lines	Opens every income and cost line carrying the tag	None	Ledger lines with `tag=`
Account rows, Income, Costs, Net	Per account the amount (a link to its lines), then the tag's totals	Net is income minus costs	None
Costs of all tags, Net of all tags	Totals over every tag	None	None
Empty state	"No tagged lines in these days."	None	None

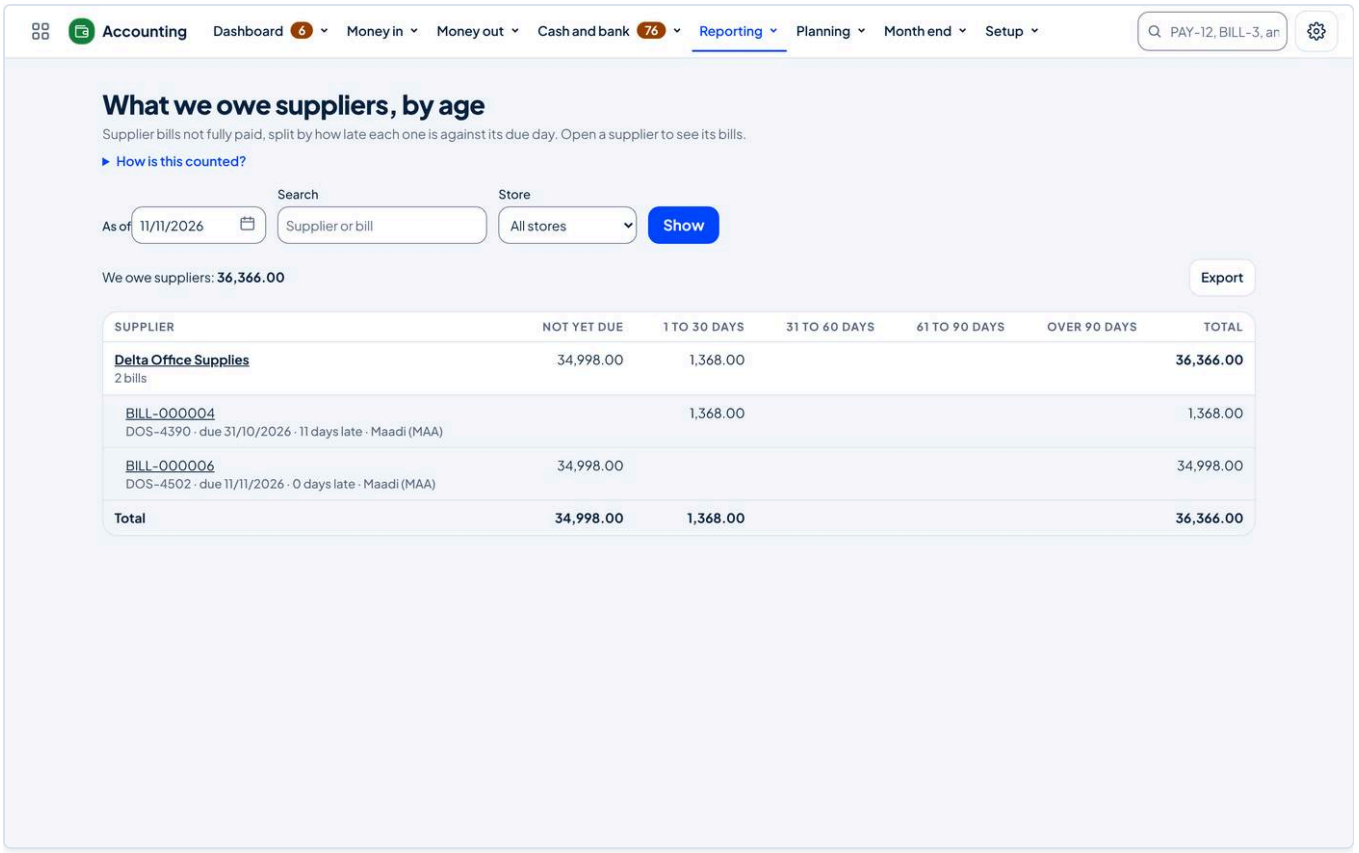
16.7 Payables aging

Address: `/finance/payables-aging?day=&q=&store=` . **Menu:** Reporting, Payables aging. **Who:** Accounting view. Data: `GET /finance/aging/payables` . Each open supplier bill counts from its due day with what was still unpaid at the end of the chosen day; a bill in another currency shows its EGP value at the bill's rate.

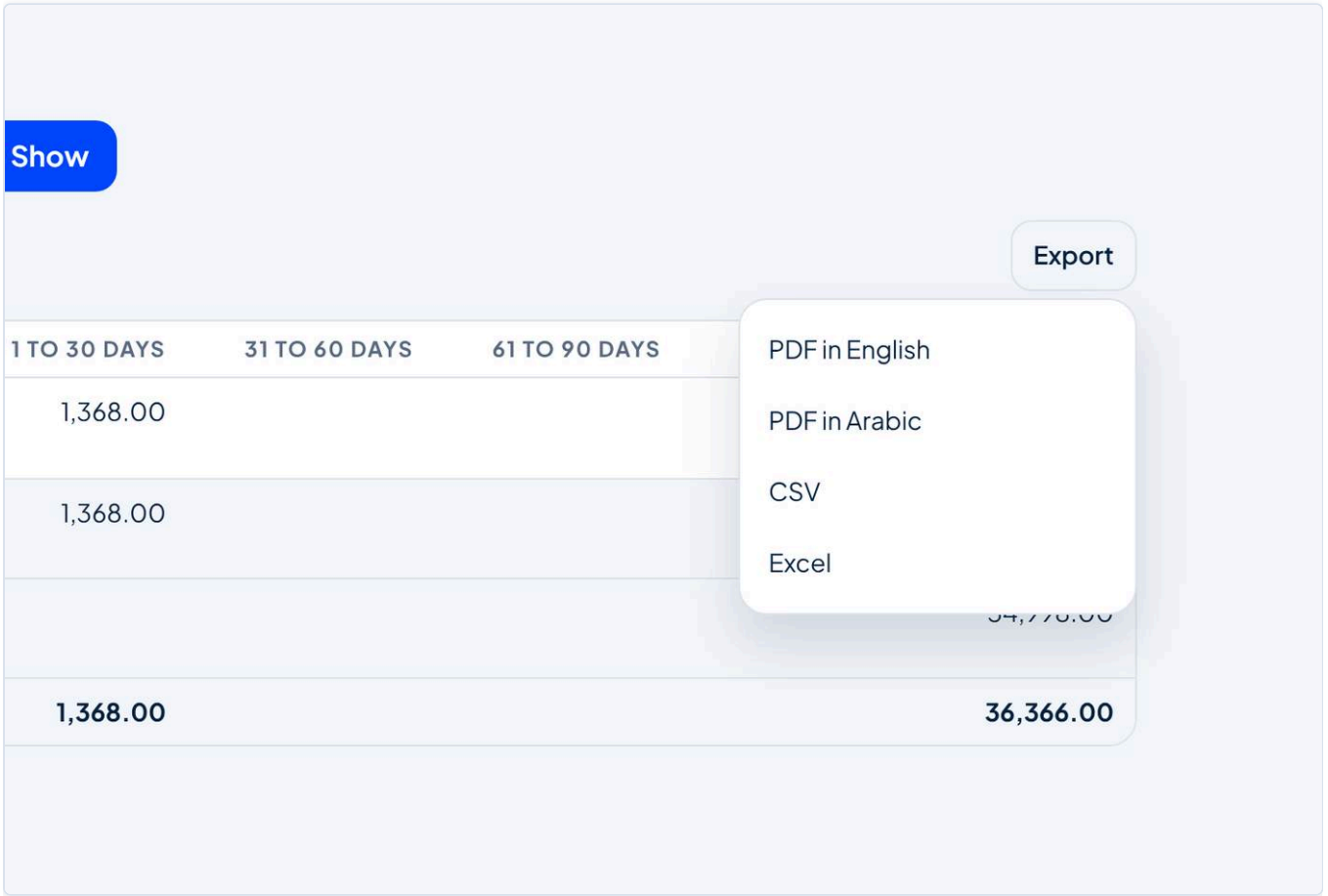


What we owe suppliers, by age: one row per supplier in the five bands (/finance/payables-aging).

Control	What it does	Rule / limits	Result
How is this counted?	Opens the method in one paragraph	None	Text folds open
As of, Search (supplier or bill), Store, Show	The day, a name or bill number, a store	Store keeps only bills tagged with it	Reloads
We owe suppliers	The grand total	None	None
Supplier name (underlined)	Opens or folds the supplier's bills	Each bill shows its number, supplier reference, due day, days late and store	Bill number opens the bill
Not yet due, 1 to 30, 31 to 60, 61 to 90, Over 90 days, Total	Amounts per band	Not yet due includes a bill due on the chosen day (0 days late)	Total row
Export	PDF in English, PDF in Arabic, CSV, Excel	PDF from `GET /finance/aging/payables/export.pdf`	A file named `owed-to-suppliers`



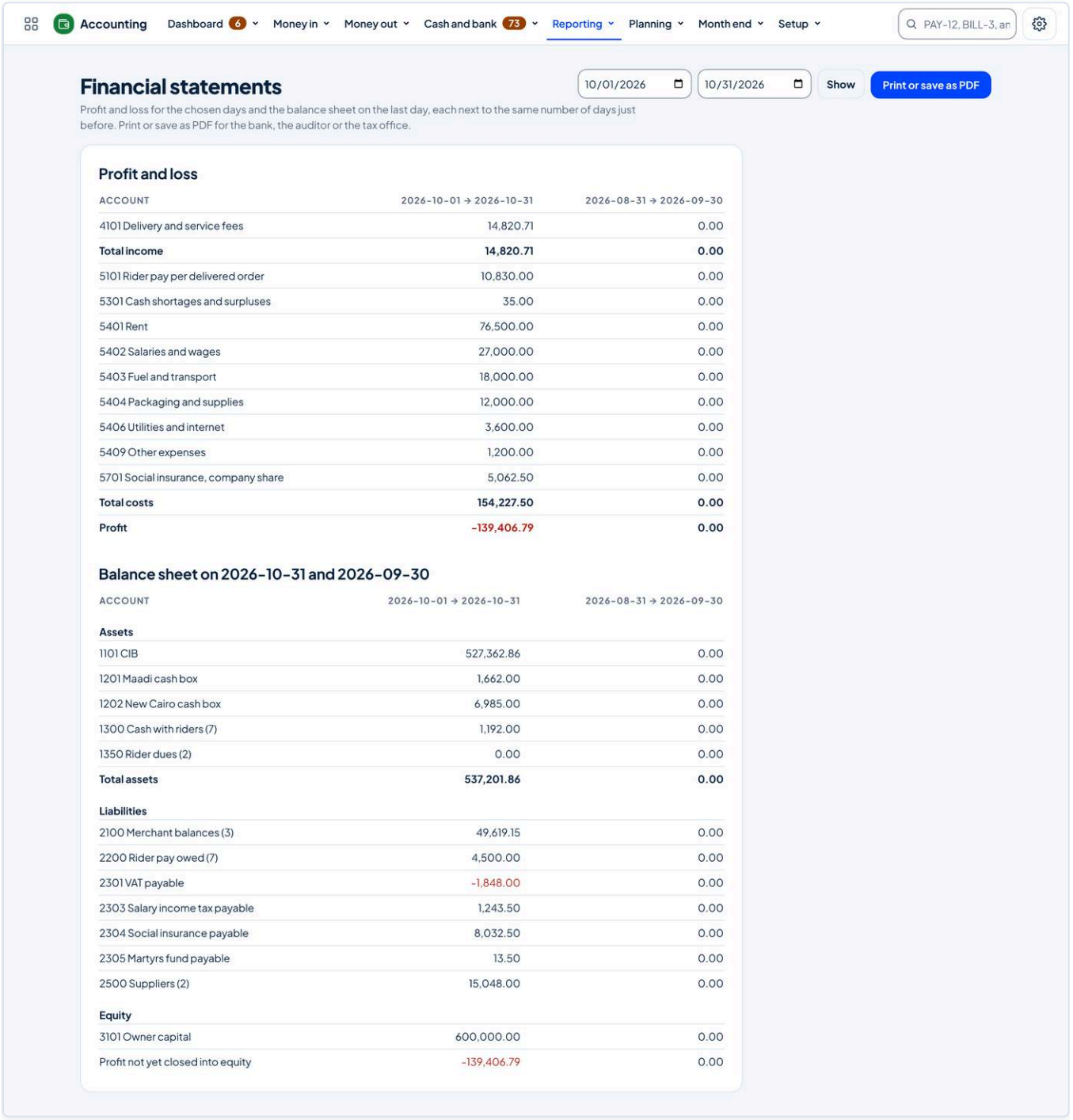
A supplier opened: two bills, one 11 days late, one due today.



The Export menu of the aging report.

16.8 Compare periods (financial statements)

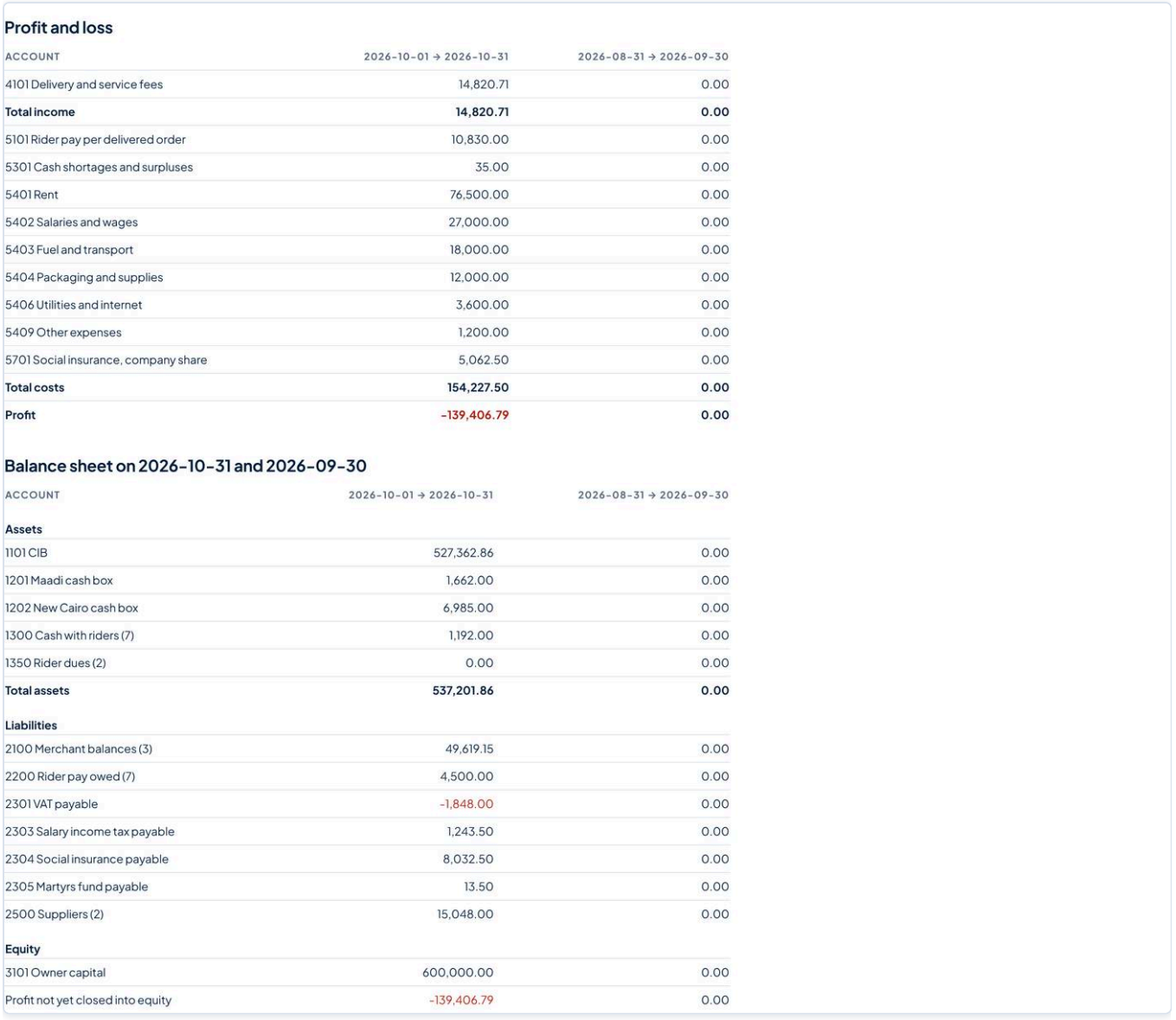
Address: `/finance/statements?from=&to=` . **Menu:** Reporting, Compare periods. **Who:** Accounting view. Data: `GET /finance/comparative-statements` . Default: last month. The page prints profit and loss for the chosen days and the balance sheet on the last day, each next to the same number of days just before.



Financial statements for October next to the 31 days before (/finance/statements).

Control	What it does	Rule / limits	Result
From, To (browser date boxes), Show	The period	These are the browser's own date fields	Address `from` , `to`

Control	What it does	Rule / limits	Result
Print or save as PDF	Opens the browser's print dialog	Printing hides the menus and the form	A clean two statement print
Profit and loss table	Every income and cost account, totals and profit, current and earlier column	Column headers show the two ranges	None
Balance sheet table	Assets, liabilities and equity on the last day of each range; wallet groups with their count, "Profit not yet closed into equity"	None	None



Profit and loss

ACCOUNT	2026-10-01 → 2026-10-31	2026-08-31 → 2026-09-30
4101 Delivery and service fees	14,820.71	0.00
Total income	**14,820.71**	**0.00**
5101 Rider pay per delivered order	10,830.00	0.00
5301 Cash shortages and surpluses	35.00	0.00
5401 Rent	76,500.00	0.00
5402 Salaries and wages	27,000.00	0.00
5403 Fuel and transport	18,000.00	0.00
5404 Packaging and supplies	12,000.00	0.00
5406 Utilities and internet	3,600.00	0.00
5409 Other expenses	1,200.00	0.00
5701 Social insurance, company share	5,062.50	0.00
Total costs	**154,227.50**	**0.00**
Profit	**-139,406.79**	**0.00**

Balance sheet on 2026-10-31 and 2026-09-30

ACCOUNT	2026-10-01 → 2026-10-31	2026-08-31 → 2026-09-30
Assets		
1101 CIB	527,362.86	0.00
1201 Maadi cash box	1,662.00	0.00
1202 New Cairo cash box	6,985.00	0.00
1300 Cash with riders (7)	1,192.00	0.00
1350 Rider dues (2)	0.00	0.00
Total assets	**537,201.86**	**0.00**
Liabilities		
2100 Merchant balances (3)	49,619.15	0.00
2200 Rider pay owed (7)	4,500.00	0.00
2301 VAT payable	-1,848.00	0.00
2303 Salary income tax payable	1,243.50	0.00
2304 Social insurance payable	8,032.50	0.00
2305 Martyrs fund payable	13.50	0.00
2500 Suppliers (2)	15,048.00	0.00
Equity		
3101 Owner capital	600,000.00	0.00
Profit not yet closed into equity	-139,406.79	0.00

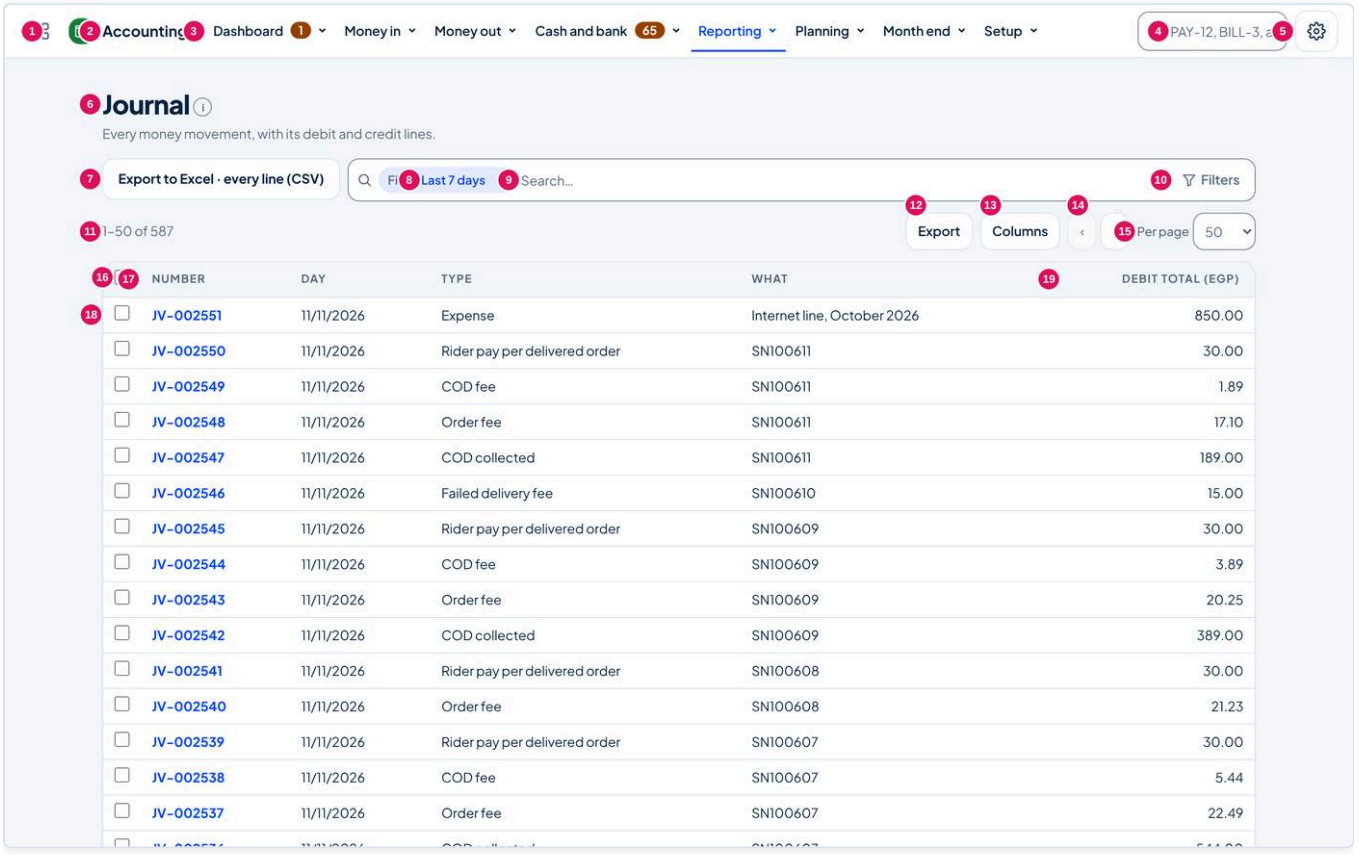
The print view: the menus and the date form are hidden.

16.9 The journal list

Address: `/finance/journal` . **Menu:** Reporting, Journal (also "What moved, line by line?" on All reports). **Who:** Accounting view; people limited to some stores see only entries with a line of their stores. It is the generic list view (`ListView` , `apps/web/src/components/odoo/ListView.tsx`) fed by `GET /finance/lists/journal` ; the server side definition is `journal` in `packages/db/src/finance/lists.ts` and its keys are

`FINANCE_LISTS.journal` in `packages/shared/src/finance-lists.ts`. It opens on "Last 7 days", newest first, 50 rows a page.

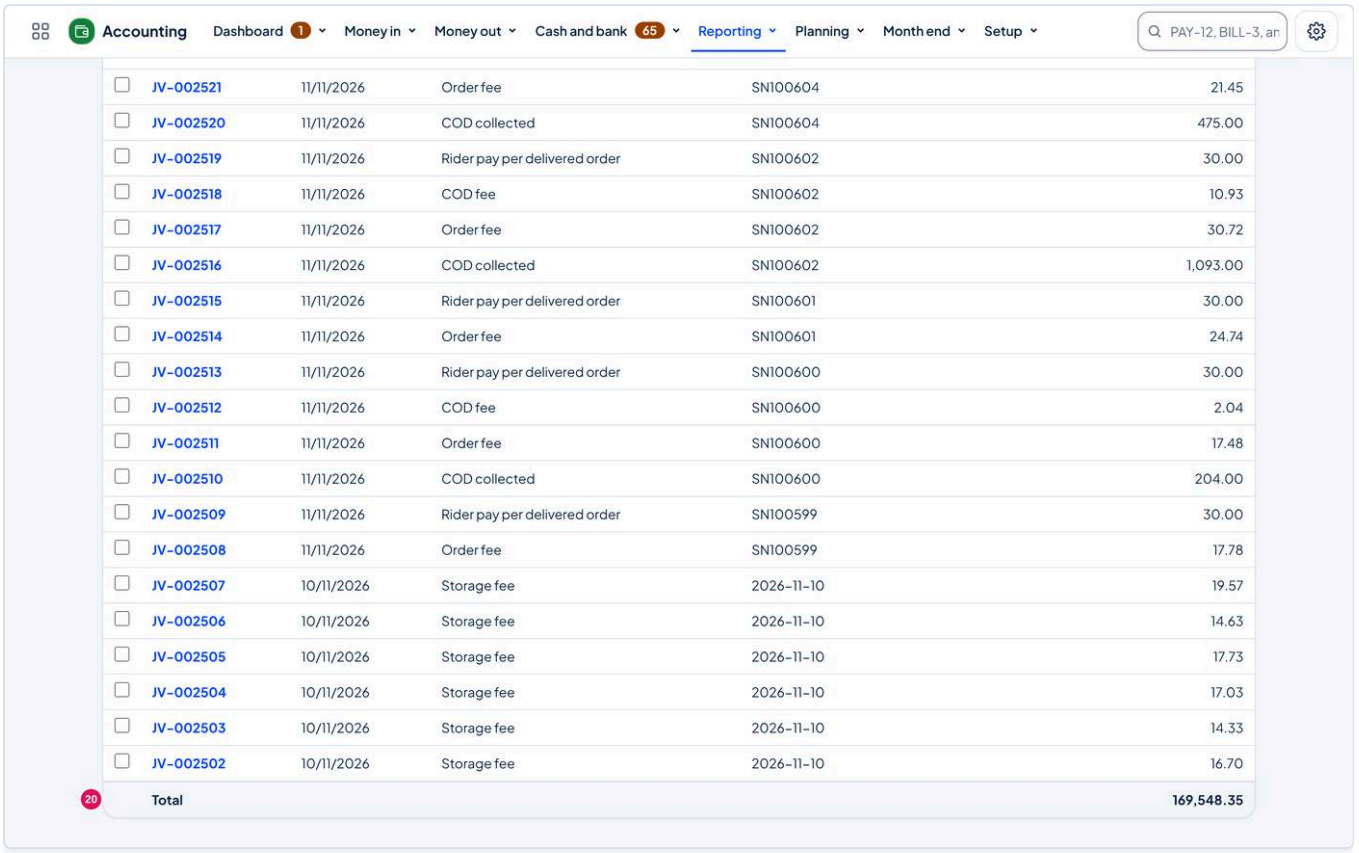
How the page looks, top to bottom



The journal list with its parts numbered (/finance/journal).

#	Part	What it shows	What clicking it does
1	App grid button	Nine square icon, top left of every app	Back to `/home` and its app tiles
2	App name	Green Accounting icon and the word Accounting	Opens the Accounting dashboard `/finance`
3	Menus	Dashboard (with an amber count of what waits for me), Money in, Money out, Cash and bank (count of open items), Reporting (underlined blue: the current menu), Planning, Month end, Setup	Each opens its list of pages; a menu with one page is a link
4	One search	Box with the hint "PAY-12, BILL-3, an order or a name"	Finds any document by number, an order or a merchant or supplier (see "Searching and filtering the journal")
5	Settings button	Gear	Account menu: Help, Tickets, language, theme, Sign out
6	Title	Journal and an (i) that explains the word	The (i) opens the explanation
7	Every line CSV	A white button "Export to Excel · every line (CSV)"	Downloads `/api/finance/ledger/export.csv`: one row per ledger line of the whole books (the accountant's file for Odoo, Zoho Books, QuickBooks, Daftra)

#	Part	What it shows	What clicking it does
8	Facet chip	Light blue chip "Filter Last 7 days" with ×	× removes the filter; Backspace in an empty search box removes the last chip
9	Search box	"Search…"	Typing searches; the text becomes a Search chip and `q` in the address
10	Filters	Funnel and the word Filters	Opens the panel Filters, Group by, Favourites
11	Range	"1–50 of 587", or "17 records in 4 groups" when grouped	None
12	Export	White button	Menu: CSV (Excel, any program), Excel workbook (.xlsx), and "Only the N selected" when rows are ticked
13	Columns	White button	Ticks for Number (fixed), Day, Type, What, Order, Accounts, By, Receipts, Debit total; the choice is saved per person
14	Previous and next page	‹ and › (greyed at the ends)	Changes `page`
15	Per page	25, 50, 100 or 200	Changes `size`
16	Select all	Checkbox in the header	Ticks every row of the page
17	Sortable header	NUMBER, DAY and DEBIT TOTAL (EGP) are buttons; the sorted one shows ▲ or ▼	First click sorts ascending, second descending (`sort` , `desc`)
18	A row	Number (bold blue), Day, Type, What (the memo), Debit total. Hover turns the row light grey; a ticked row is light blue	Anywhere on the row opens the entry; Ctrl or Cmd click opens it in a new tab; the checkbox only ticks
19	Debit total (EGP)	The sum of the entry's debit lines	Sort by amount
20	Totals row	"Total" and the sum of Debit total over every passing row, not only this page	None

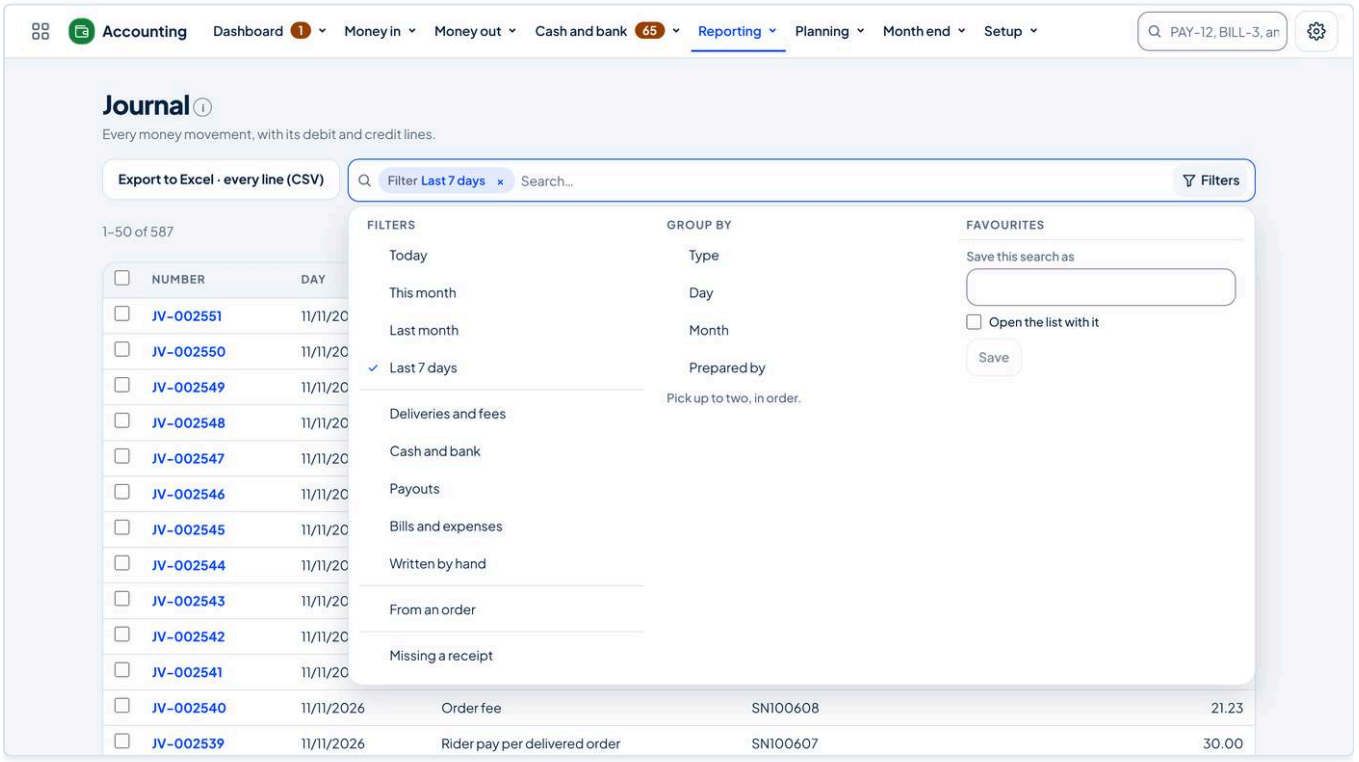


The bottom of the list: the grey Total row sums every row that passes the filters (part 20).

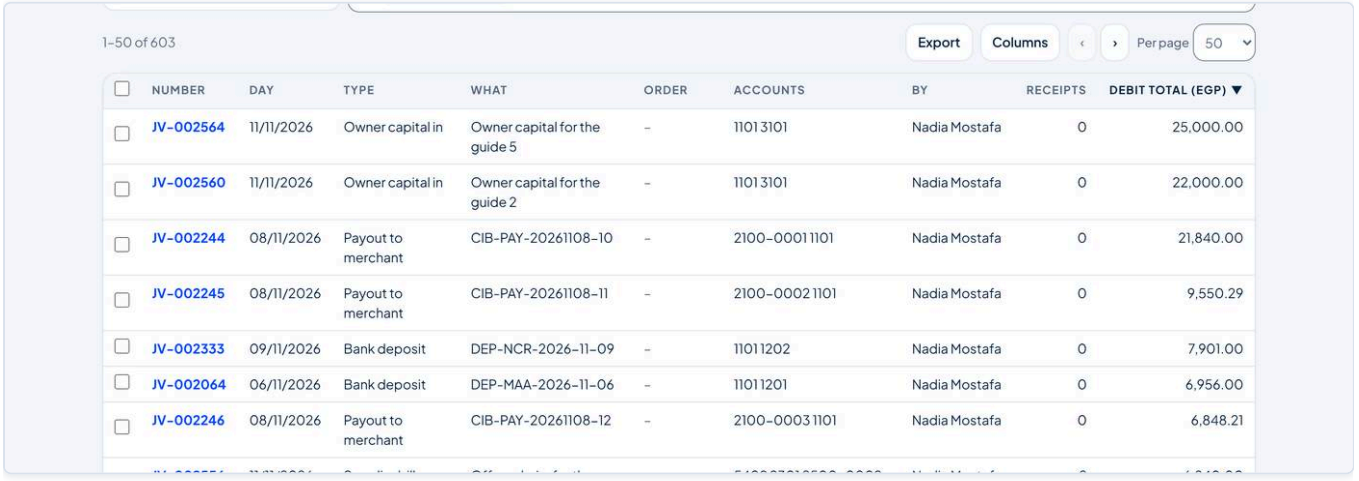
Controls on this screen

Control	What it does	Rule / limits	Result
Search	Matches number, memo, order number, kind key, account codes and the preparer's name	Case insensitive, inside the newest 5,000 entries	Search chip
Filters (date group)	Today, This month, Last month, Last 7 days	Joined by "or" inside the group	Filter chip, `f=`
Filters (kind group)	Deliveries and fees, Cash and bank, Payouts, Bills and expenses, Written by hand	Kind families listed below	Same
From an order	Entries linked to an order	Own group	Same
Missing a receipt	Entries made by a person, not from an order, with no receipt attached	Own group	Same
Group by	Type, Day, Month, Prepared by	Up to two levels, in the order picked ("Pick up to two, in order")	Group rows with a count and a subtotal; click a group to open it

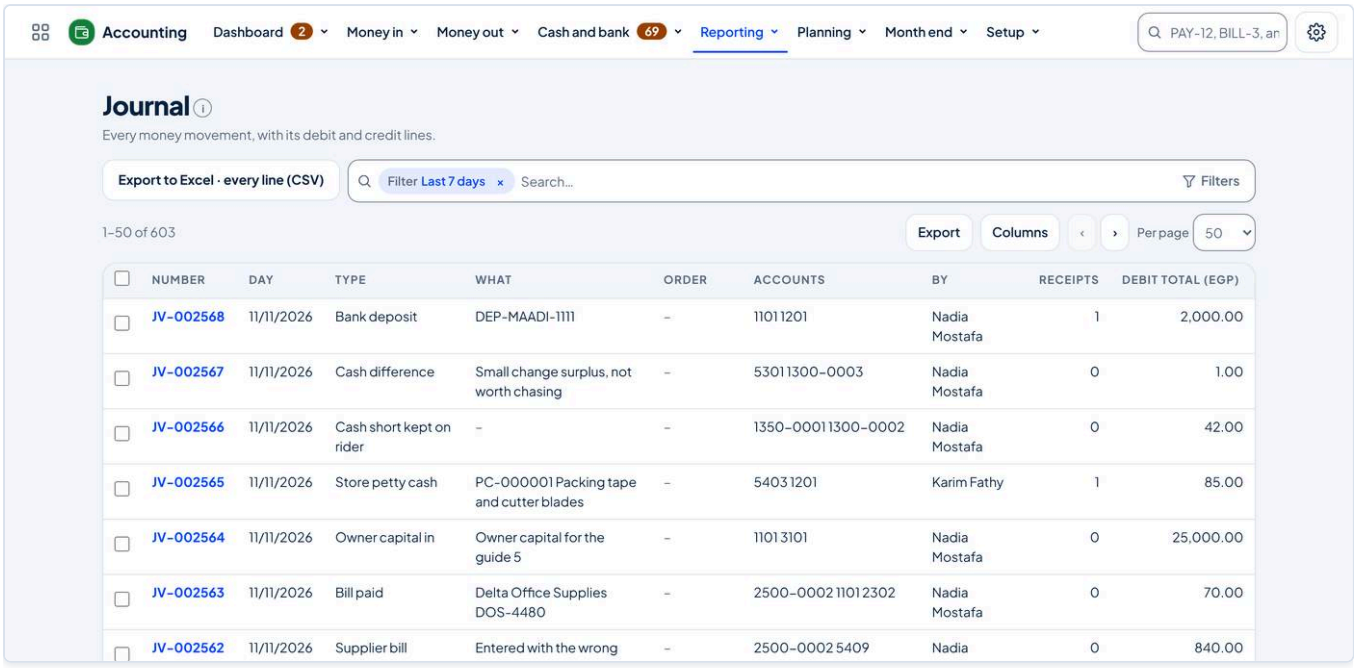
Control	What it does	Rule / limits	Result
Favourites: Save this search as, Open the list with it, Save	Saves the filters, grouping, sort and search under a name	Per person (`user_list_favorites`); one may be the default, which opens when the address names no view; audited `list.favorite_save`	A starred favourite in the panel; × deletes it
Export	CSV or .xlsx of the passing rows (or the ticked rows)	Money as EGP numbers; Arabic sheets right to left; at most the 5,000 loaded rows	File `journal`
Checkboxes and the dark bar	Shows "N selected" and Clear	The journal has no bulk actions: selection is for exporting only	Export offers "Only the N selected"
Phone	Each row is a card (number, up to four fields, amount)	None	Tap opens the entry



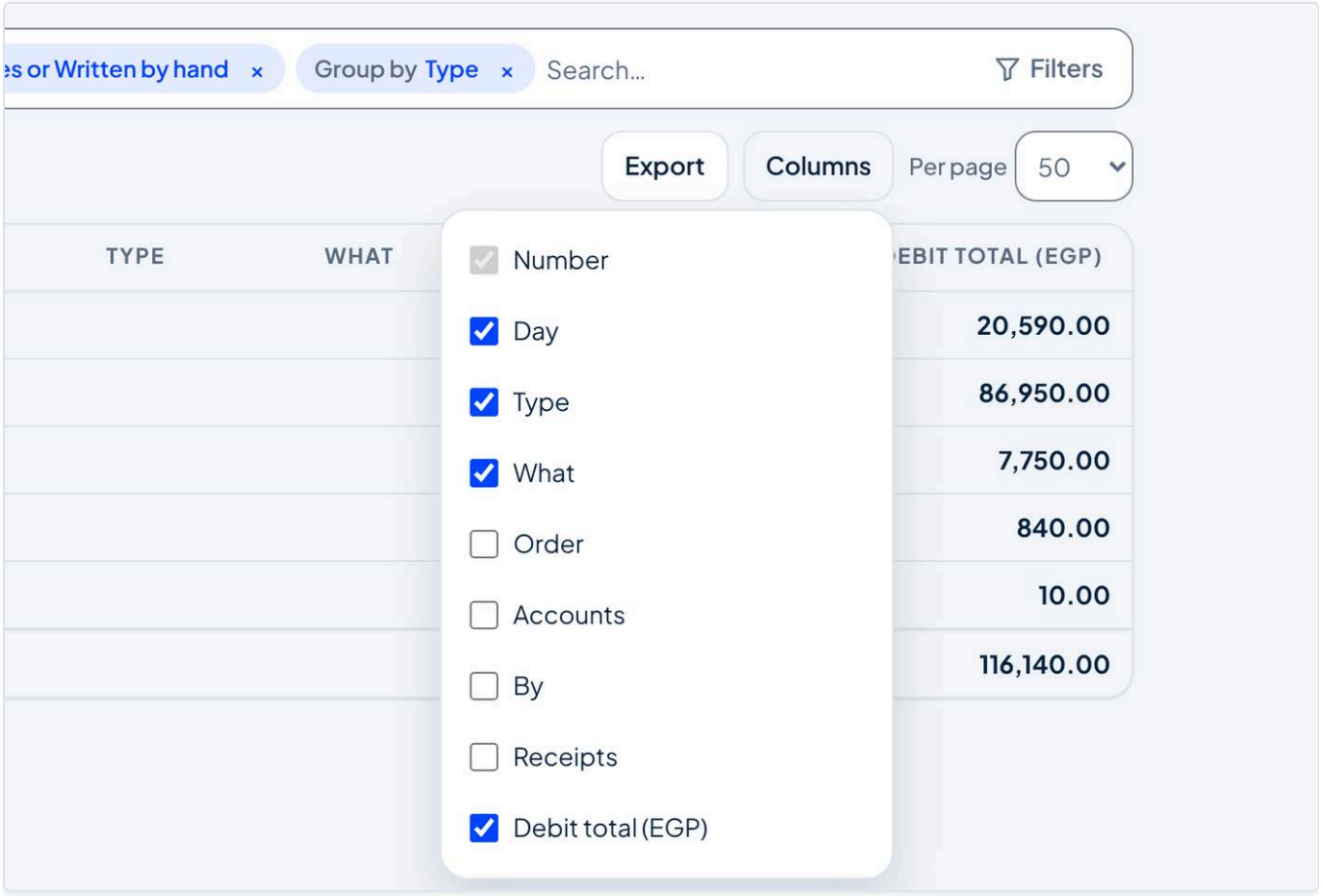
The Filters panel open: Filters (the date group, the kind group, From an order, Missing a receipt), Group by, Favourites.



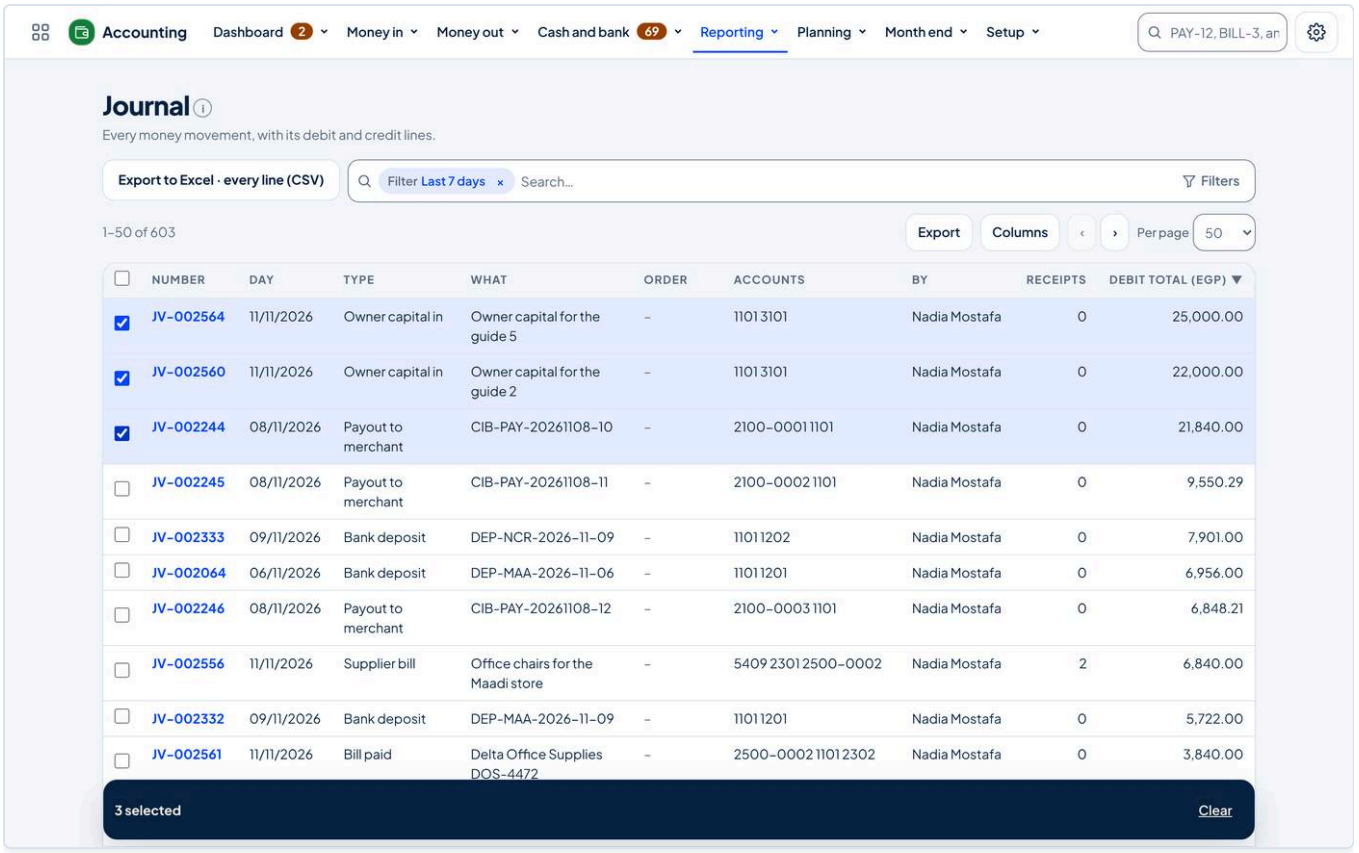
Sorted by Debit total, highest first (▼ on the header), with the optional columns Order, Accounts, By and Receipts switched on.



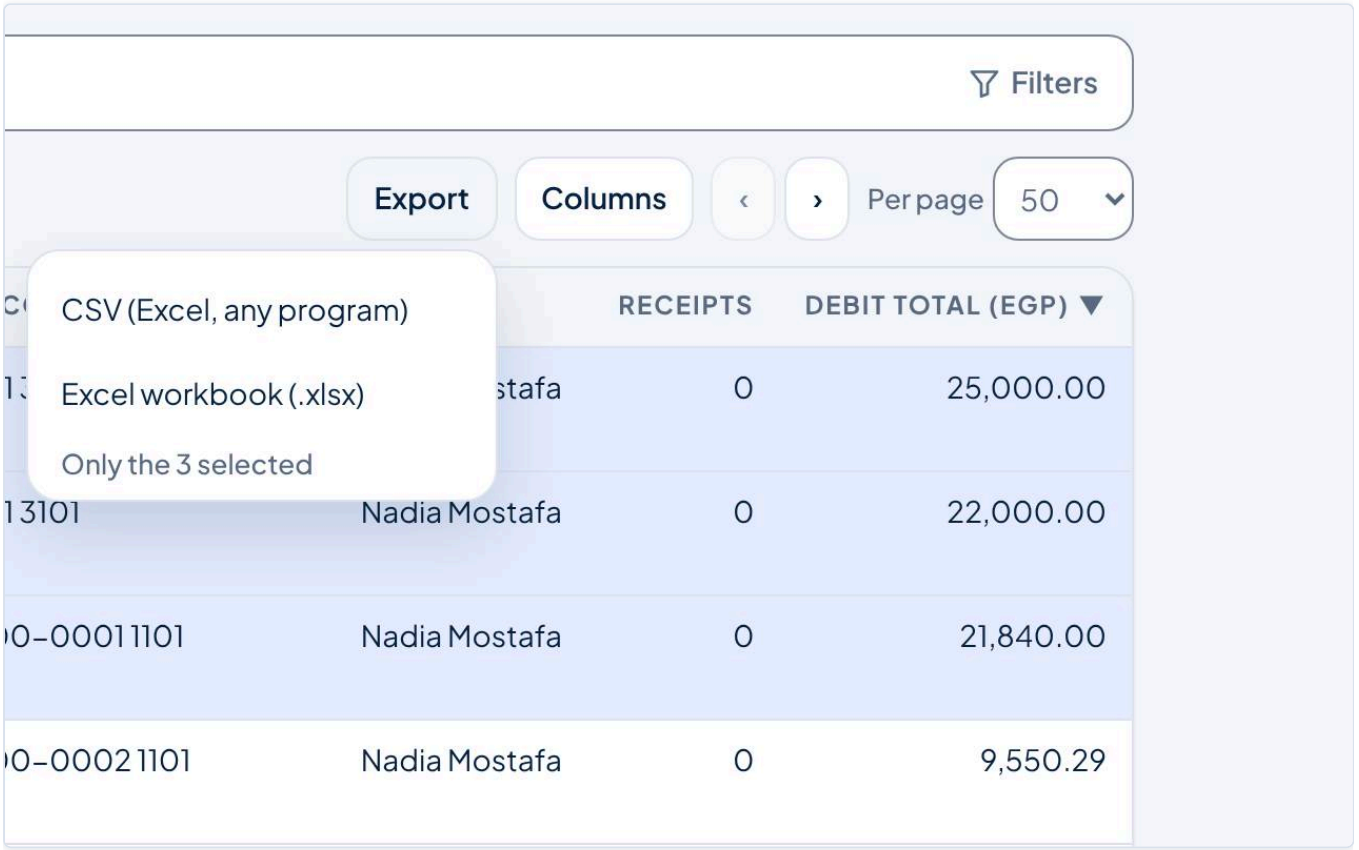
The optional columns switched on: Order, Accounts (codes of the lines), By (who prepared it) and Receipts.



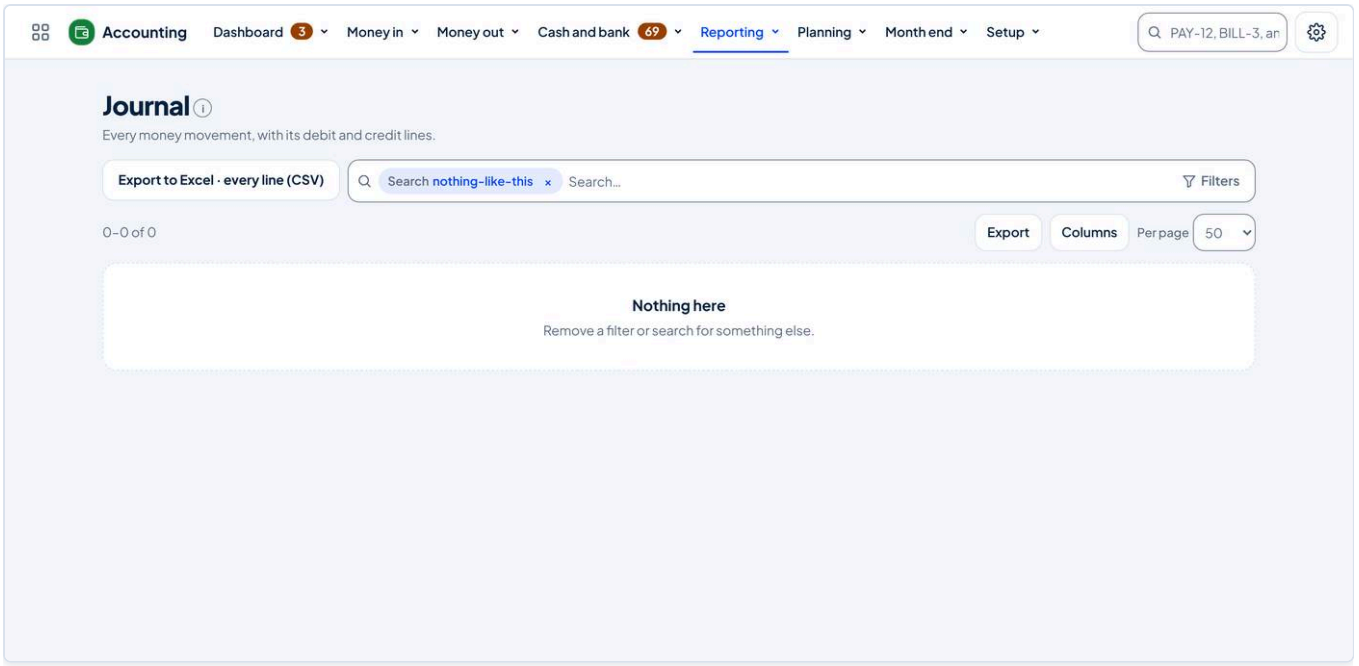
The Columns menu.



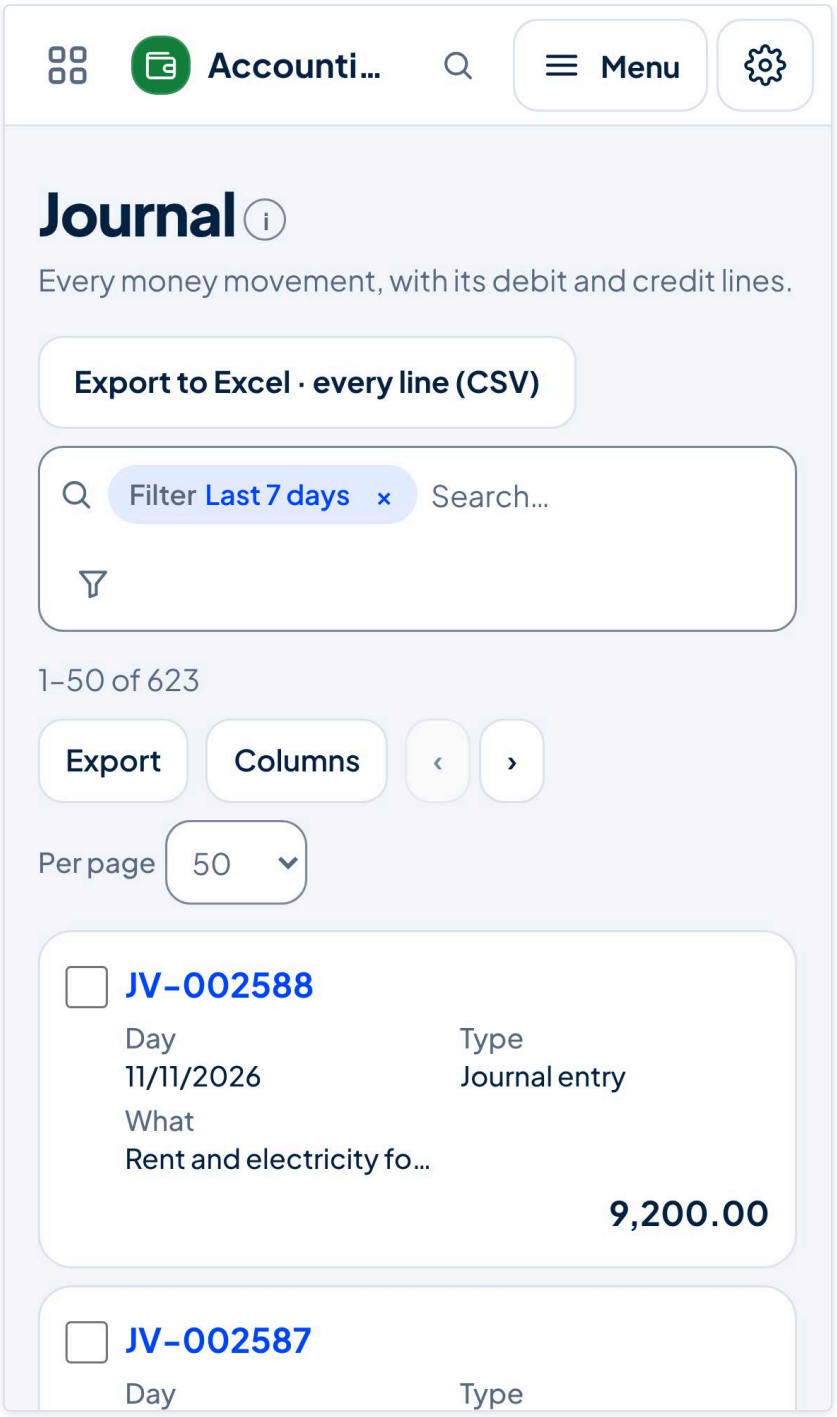
Three rows ticked: they turn light blue and the dark bar "3 selected" with Clear sticks to the bottom.



The Export menu with "Only the 3 selected".



Empty state: "Nothing here. Remove a filter or search for something else."



The journal on a phone: one card per entry.

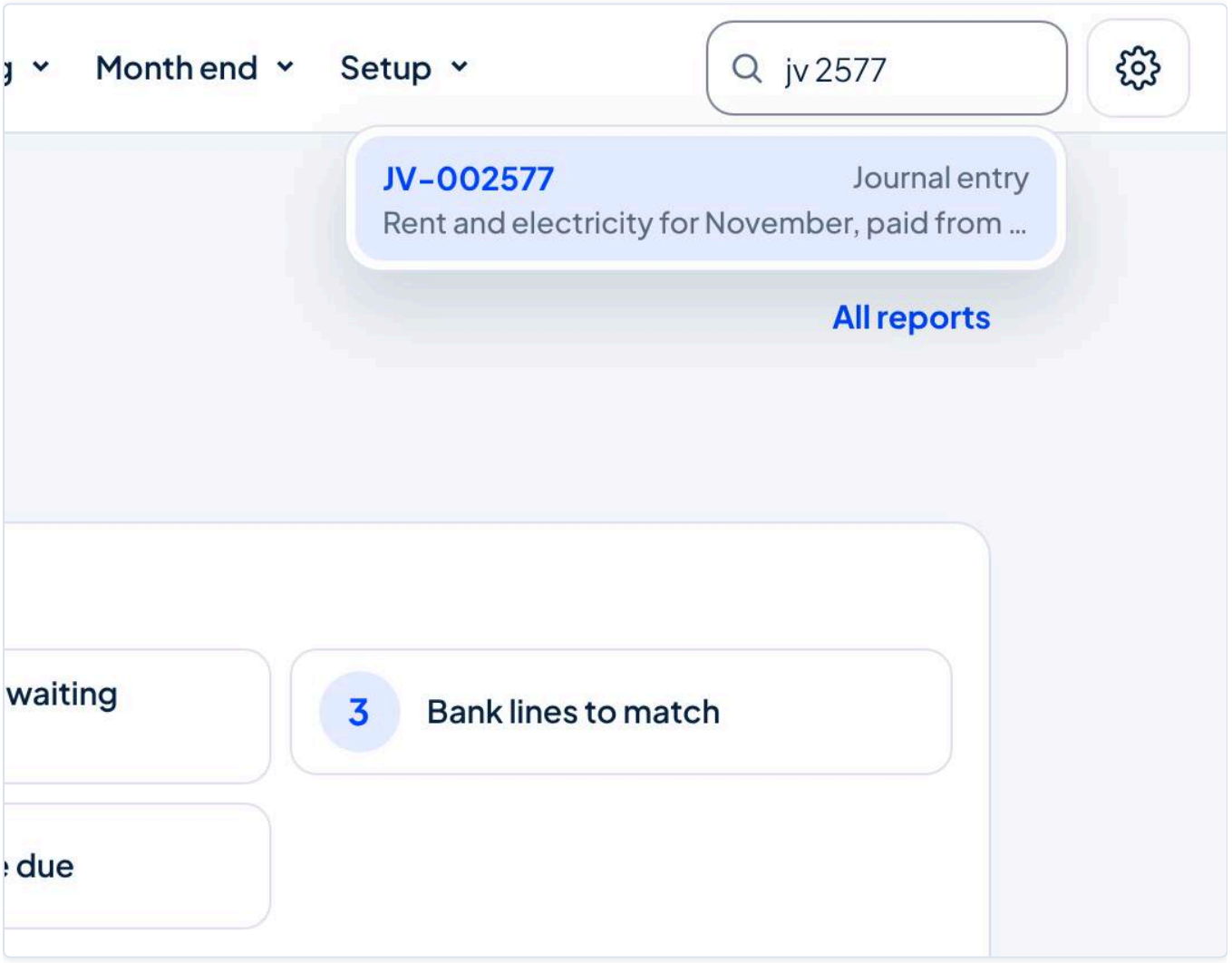
16.10 Searching and filtering the journal

There are four ways to find an entry: the Accounting top bar search, the journal list's search and filters, a report's drill down, and the smart buttons of other documents.

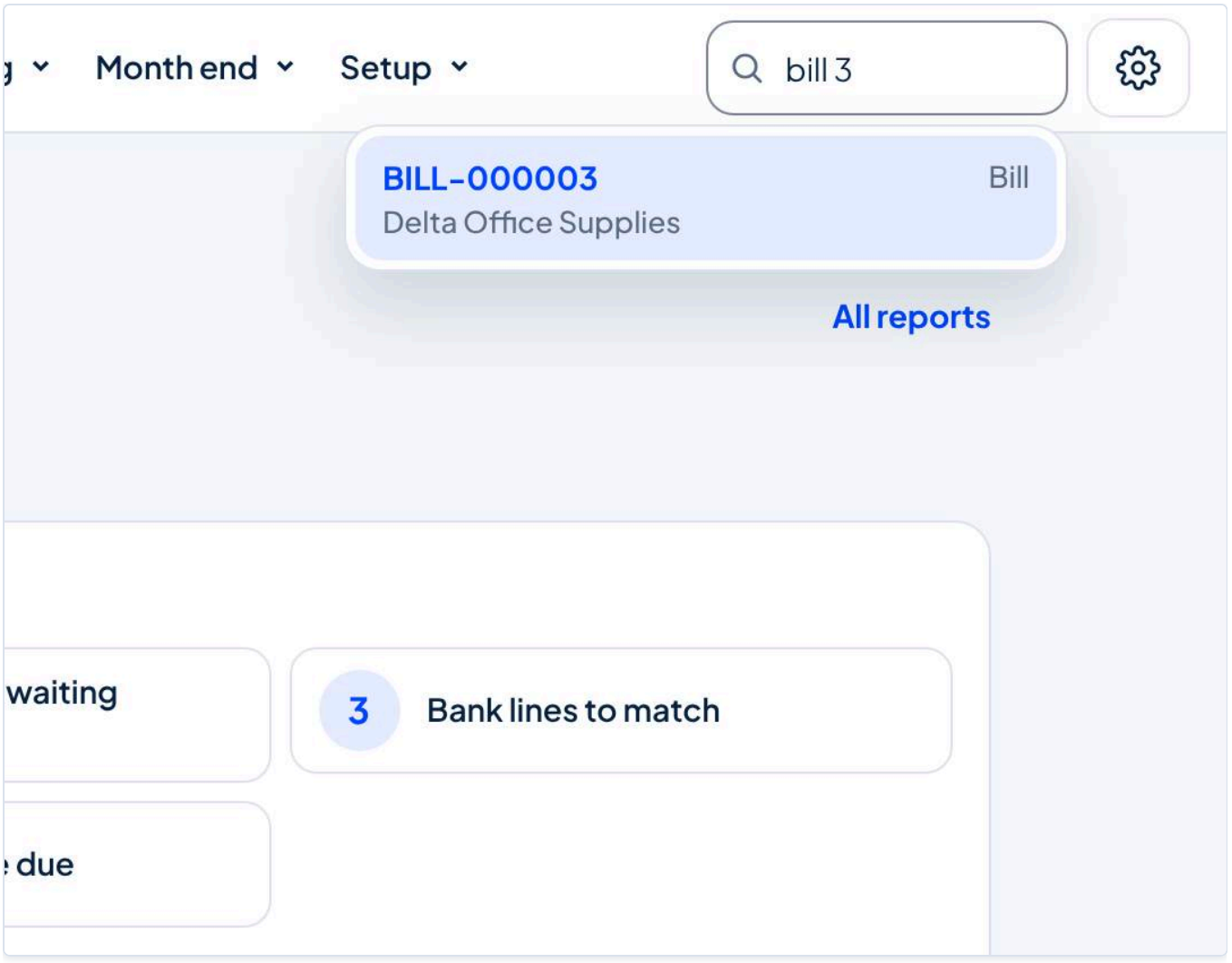
1. The top bar search (every Accounting page)

`GET /finance/search?q=` (`packages/db/src/finance/search.ts`). A document number is read by `parseDocNumber` (`packages/shared/src/documents.ts`) in loose forms: the prefix in any case, then spaces, a hyphen or # or nothing, leading zeros optional. "JV-002577", "jv 2577", "JV2577" and "jv-#2577" all find JV-002577. Prefixes: JV journal entry, PAY payout, CH cash handover, BILL bill, INV invoice, CN credit note, DN

debit note, PO purchase order, CHQ cheque, PC petty cash, RW rider withdrawal, BS bank statement, EC expense claim, BD Basata deposit. Other text finds order numbers (SN…) and merchant or supplier names. People limited to some stores or merchants find only theirs. A result shows the number, its kind and a detail line; Enter or a click opens it; the panel ends with "All reports".



"jv 2577" typed in the top bar: JV-002577, Journal entry, with its memo.



"bill 3" finds BILL-000003.

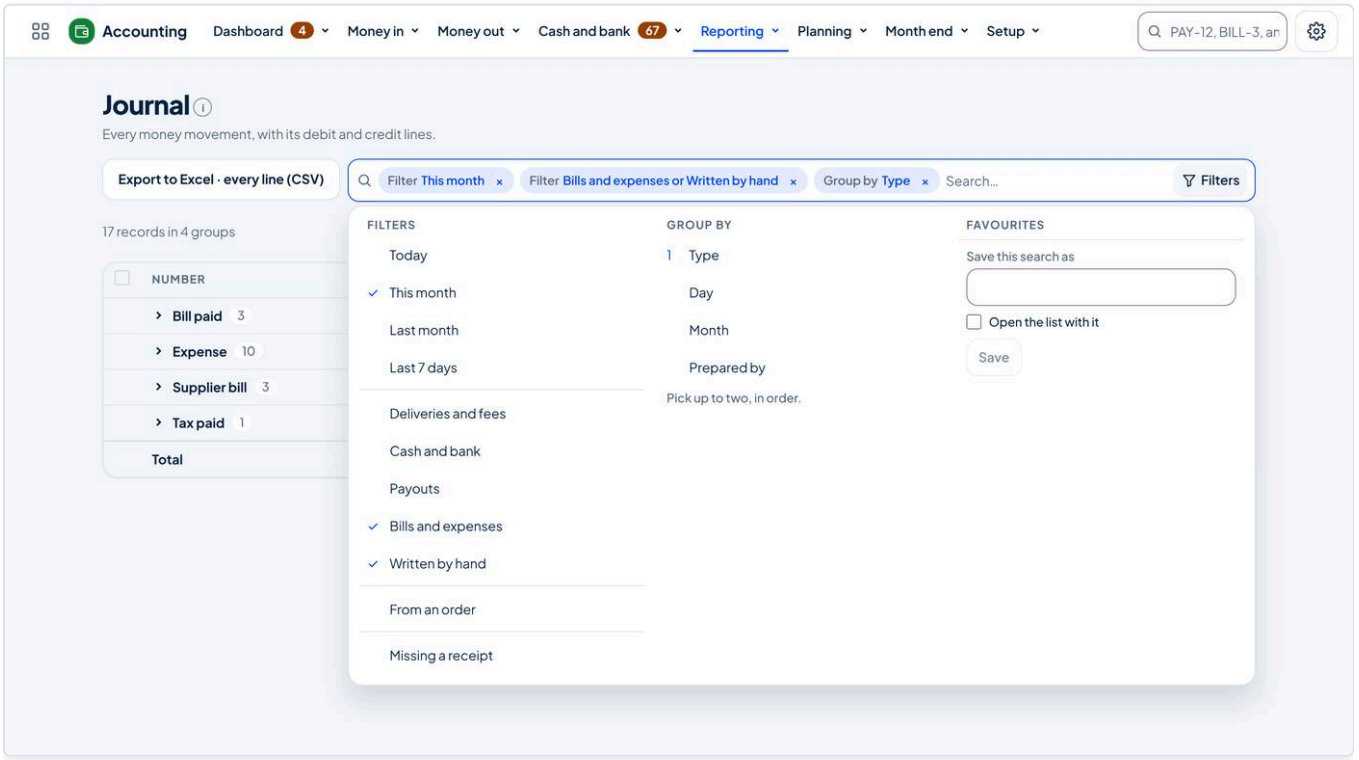
2. The journal list

The search box matches, for each entry: the number (JV-00×xxx), the memo, the order number, the kind key (for example `expense`, `order_fee`), the account codes of its lines (so "1101" finds every entry touching CIB) and the name of the person who prepared it. The predefined filters and what they mean:

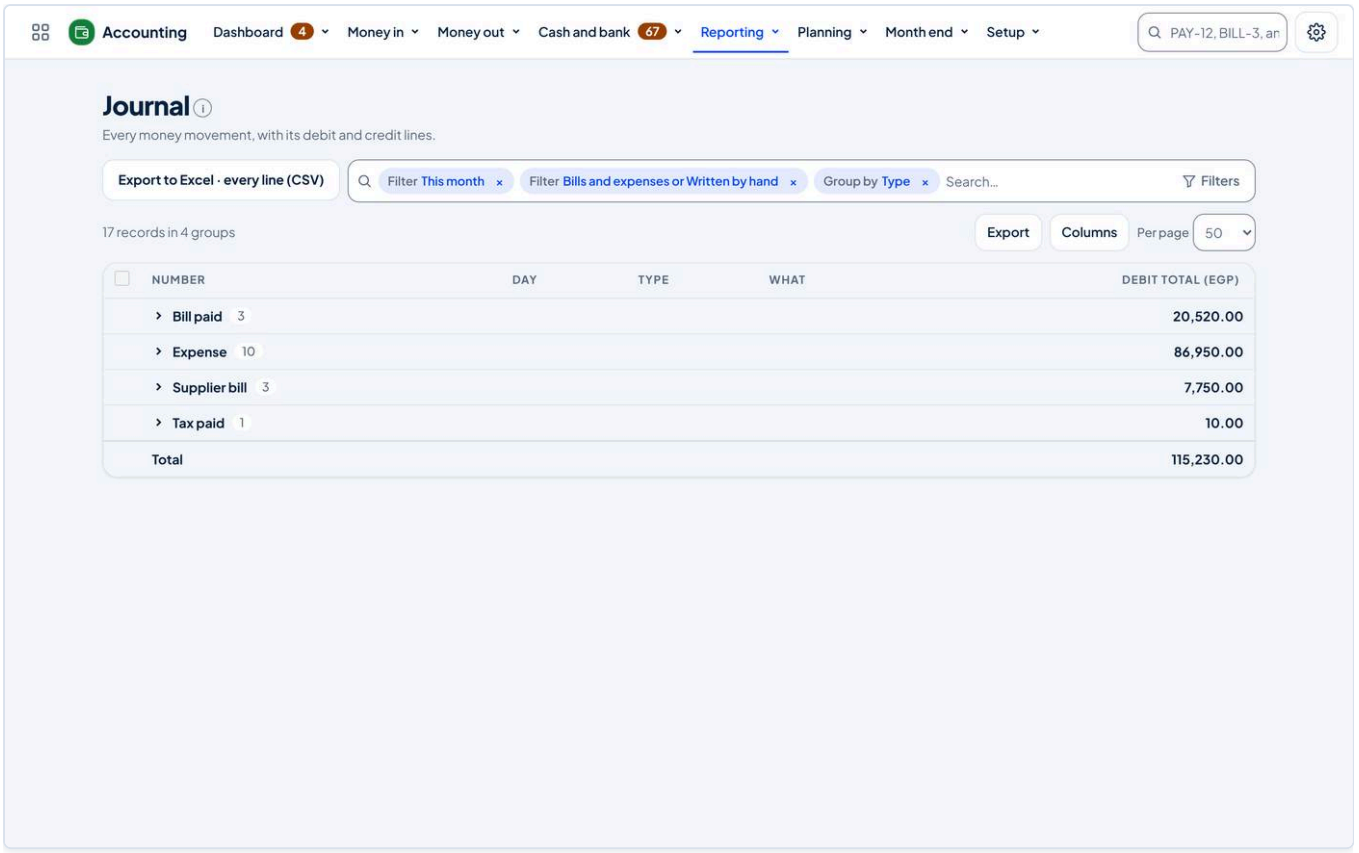
Filter	Group	Meaning (code)
Today	date	The entry's Cairo day is today
This month	date	Its Cairo month is this month
Last month	date	Its Cairo month is the month before
Last 7 days	date	Its day is today or one of the six days before (the default)
Deliveries and fees	kind	cod_collected, order_fee, cod_fee, failed_fee, receiving_fee, storage_fee, rider_bonus, return_fee, minimum_fee
Cash and bank	kind	cash_handover, cash_difference, bank_deposit, money_transfer, petty_cash, petty_cash_reversal, basata_deposit, basata_deadline_charge, basata_settlement
Payouts	kind	merchant_payout, rider_bonus_payout, rider_withdrawal
Bills and expenses	kind	supplier_bill, supplier_bill_cancel, bill_payment, expense, expense_claim, tax_payment, po_receipt, packaging_purchase

Filter	Group	Meaning (code)
Written by hand	kind	journal, adjustment
From an order	source	The entry has an order
Missing a receipt	receipt	No receipt attached, made by a person (not a job) and not from an order

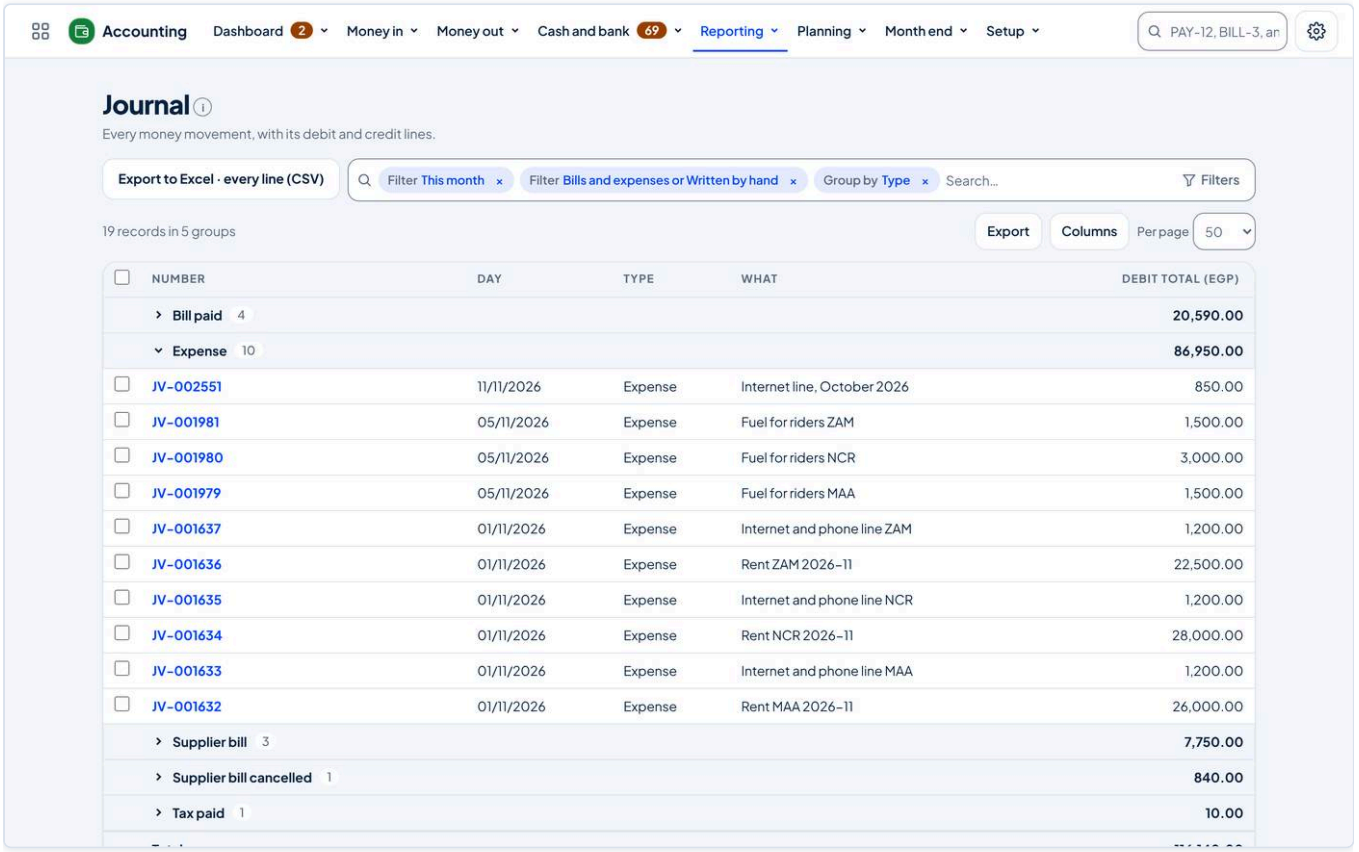
Filters in one group are joined by "or" ("Bills and expenses or Written by hand" is one chip), groups by "and". The journal list has no value filters (no store or day pickers): exact days are reached through the drill down lines list, which takes `from` and `to`. Group by: Type, Day, Month, Prepared by (up to two levels). Sorting: Number, Day (by the exact moment), Debit total. The state lives in the address: `q`, `f` (always written, so removing the last filter does not bring the default back), `g`, `sort`, `desc`, `page`, `size`.



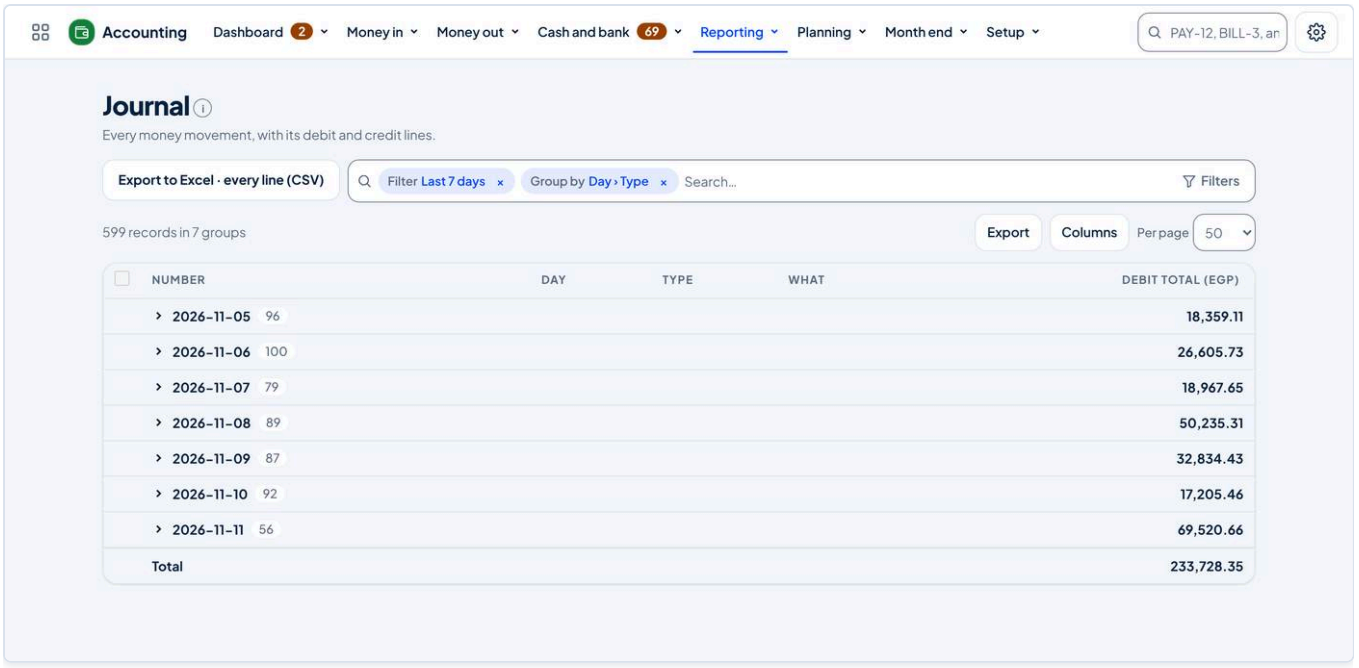
This month and "Bills and expenses or Written by hand", grouped by Type: three chips in the search bar, ticks in the panel, "1" beside Type.



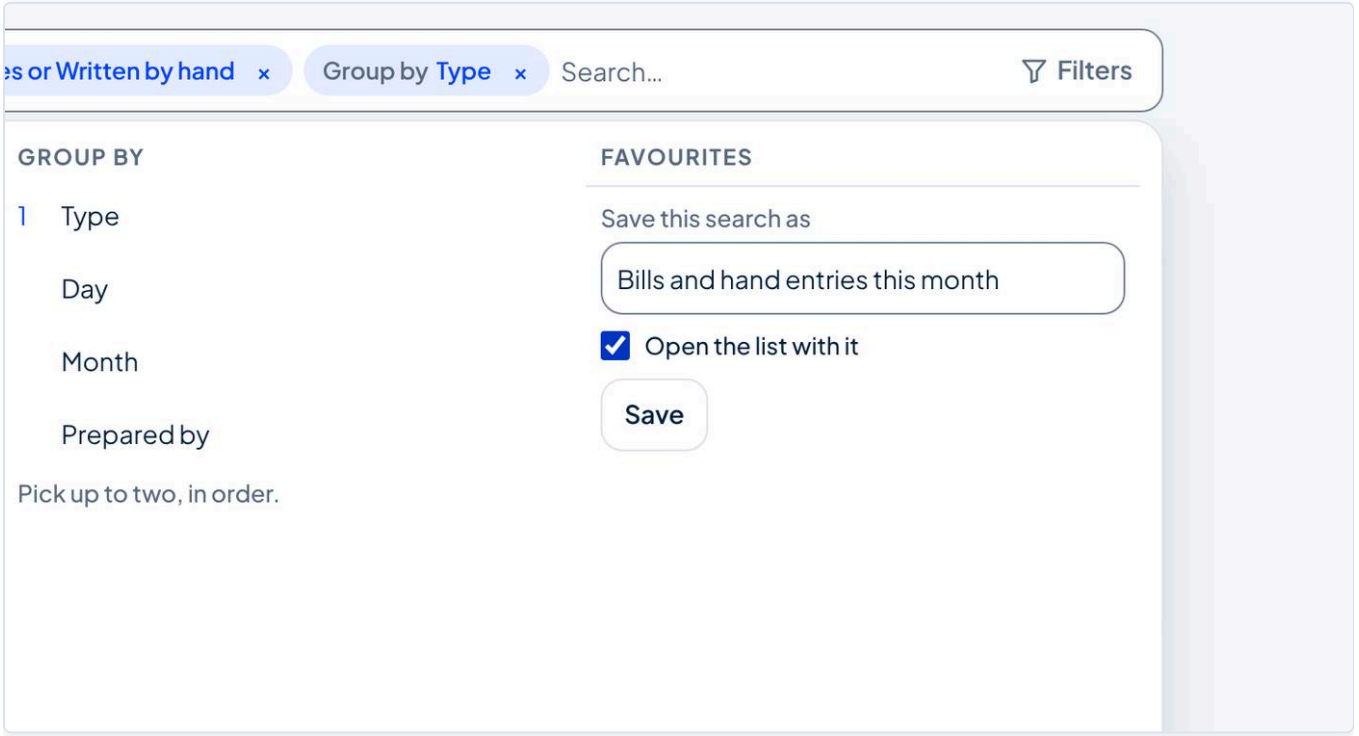
The same list grouped by Type: Bill paid 3, Expense 10, Supplier bill 3, Tax paid 1, each with its subtotal, and the total.



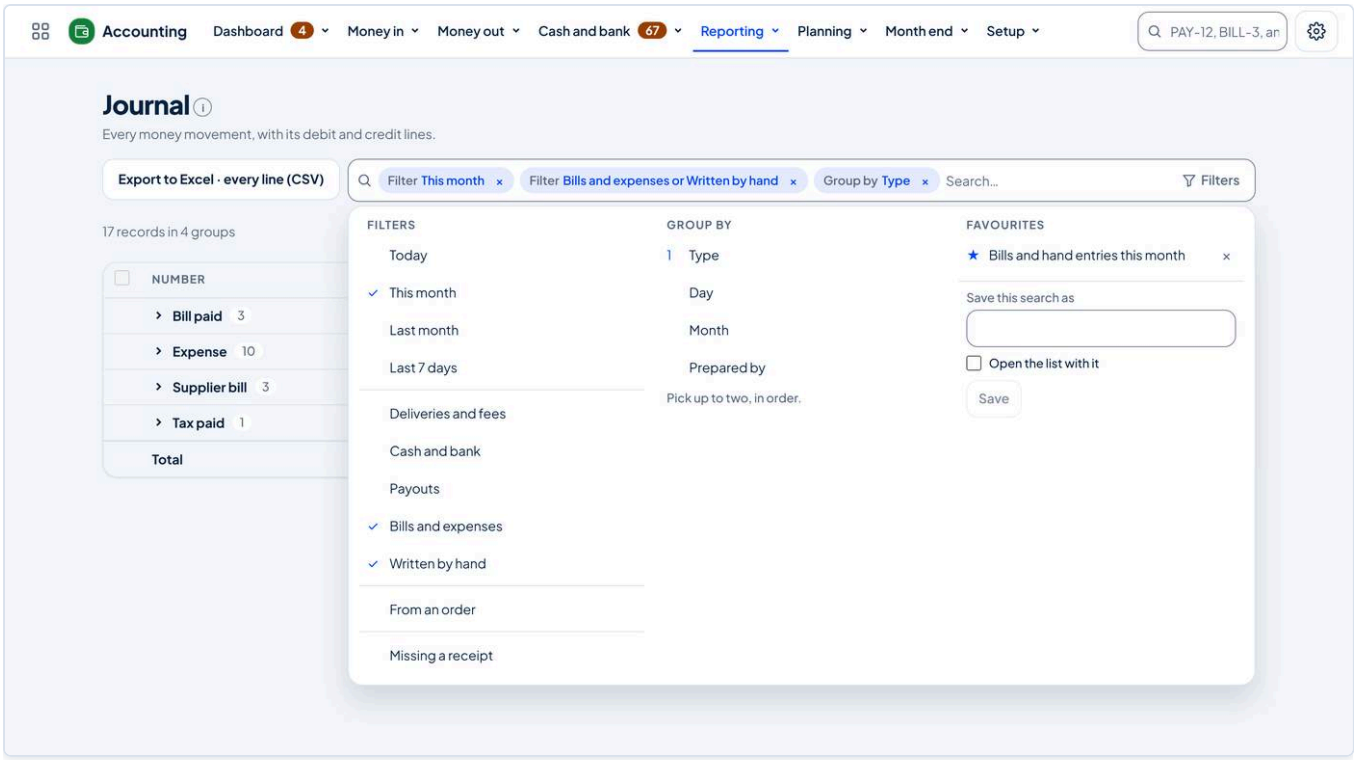
The Expense group opened: its entries listed under the group row.



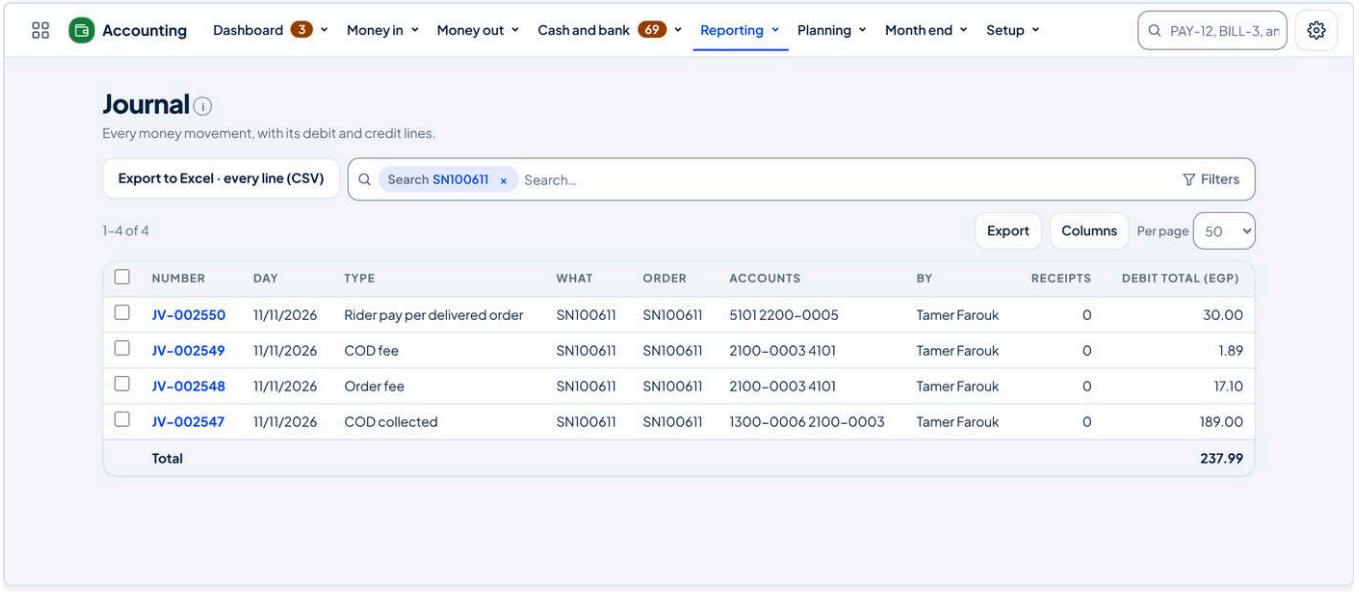
Two levels: Last 7 days grouped by Day, then Type.



Favourites: the name typed and "Open the list with it" ticked.



The saved favourite with its star; clicking it applies the search, × deletes it.



Searching an order number (SN100611) finds its delivery entries: COD collected, order fee, COD fee, rider pay.

3. The ledger lines drill list

From any report line (see the drill down report page above) the ledger lines list adds what the journal lacks: a day range (`from` , `to`), Store (including No store), Merchant (including No merchant), Rider, Kind of entry and Cost tag pickers, and the filters Debits, Credits, Deliveries and fees, Cash, Payouts, Bills, Journal entries by hand, With an order, No store, grouped by Month, Day, Store, Merchant, Rider, Kind, Account or Cost tag. Each row opens its journal entry.

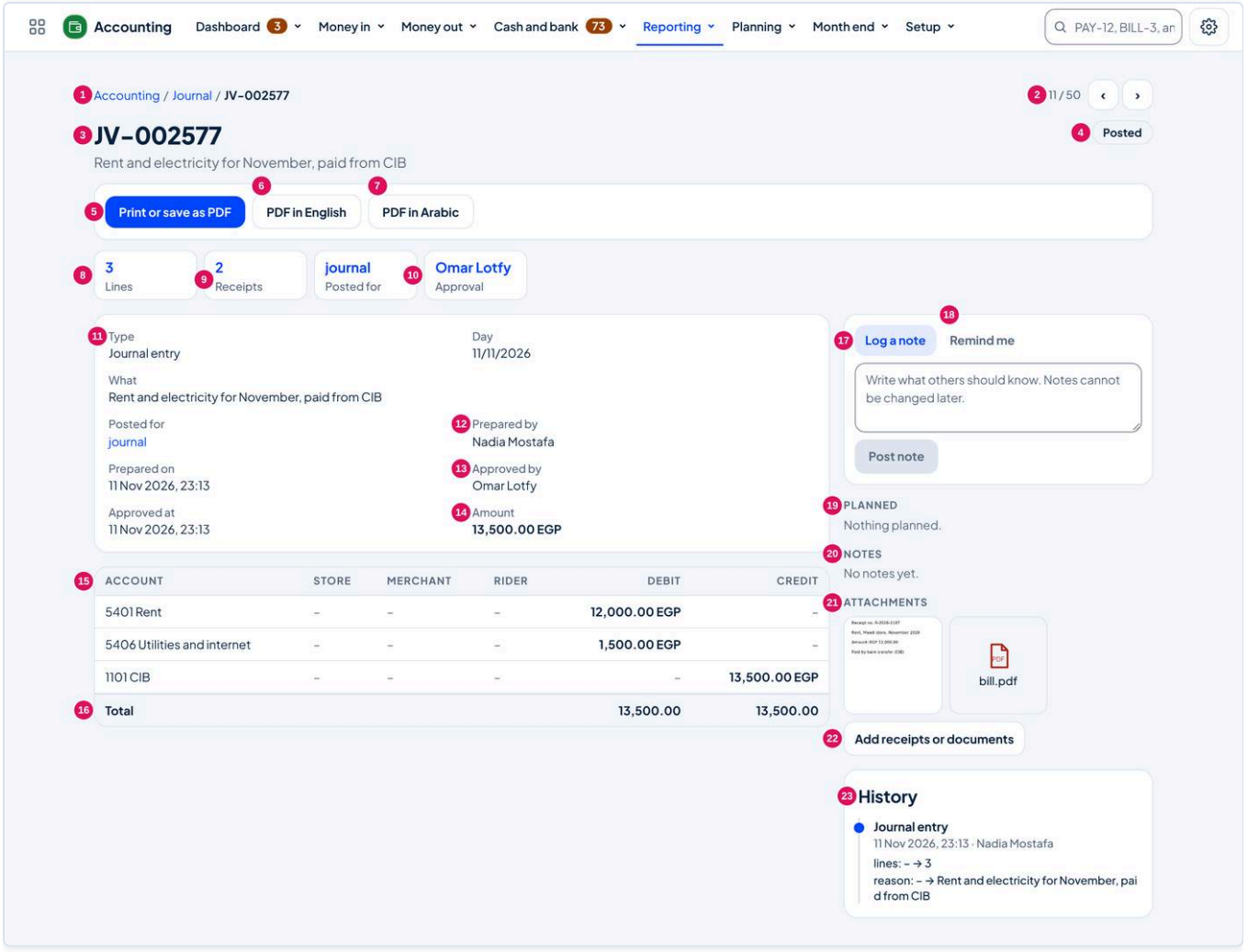
4. Smart buttons of other documents

A bill, payout, cheque, approval request or order shows a "Journal entries" or "Journal entry" tile that opens its own entries.

16.11 The journal entry page

Address: `/finance/journal/<id>`. Opened from the journal list, a ledger line, a search result or a smart button. **Who:** Accounting view; writing a note, a reminder or adding a file needs Accounting use. Data: `GET /finance/documents/ledger_entry/:id` (`entryDoc` in `packages/db/src/finance/documents.ts`), framed by `DocumentFrame`. An entry has no edit button: ledger rows are append only (database triggers refuse UPDATE and DELETE).

How the page looks, top to bottom

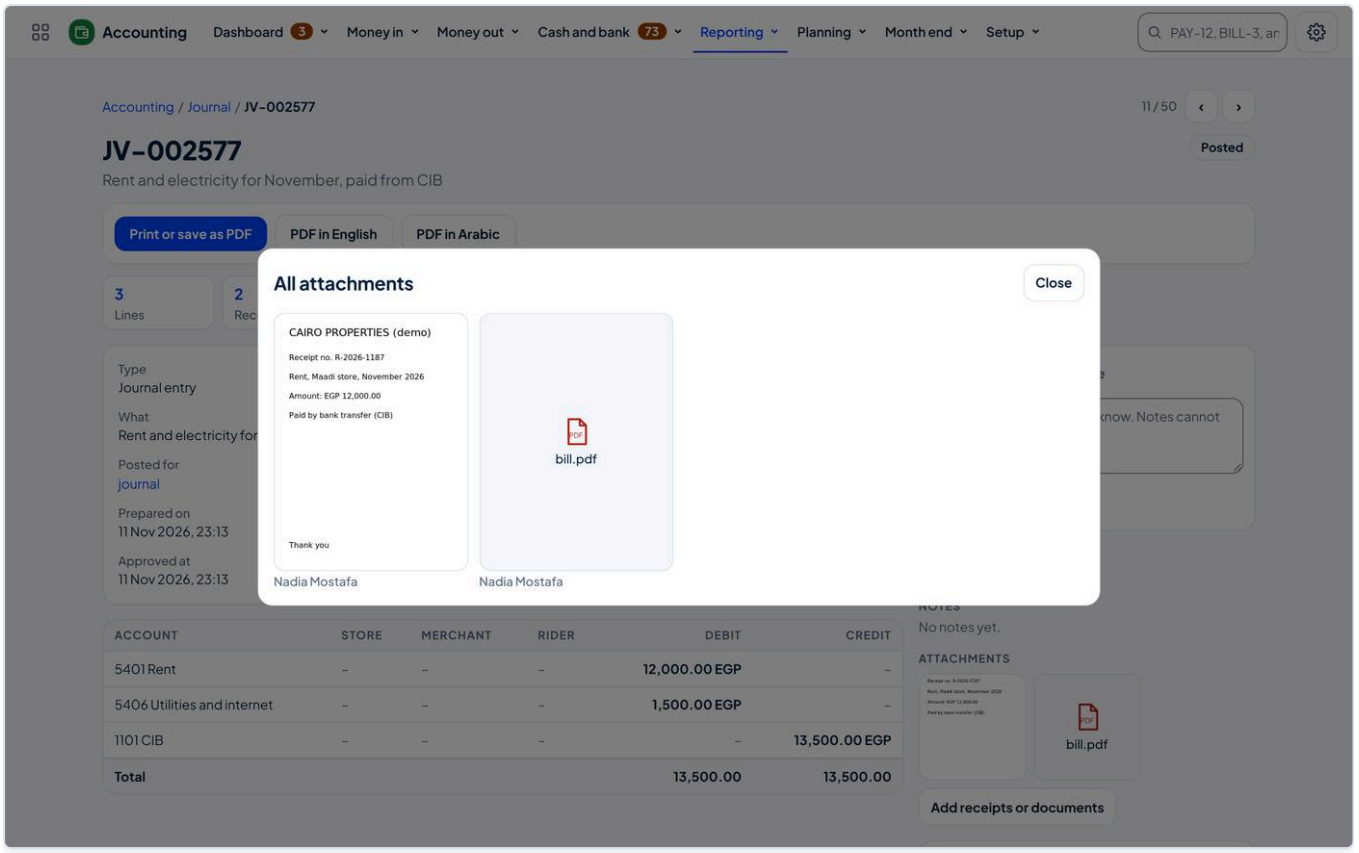


JV-002577, a journal entry made by hand from the CIB bank, approved by a second person, with two attachments; its parts numbered.

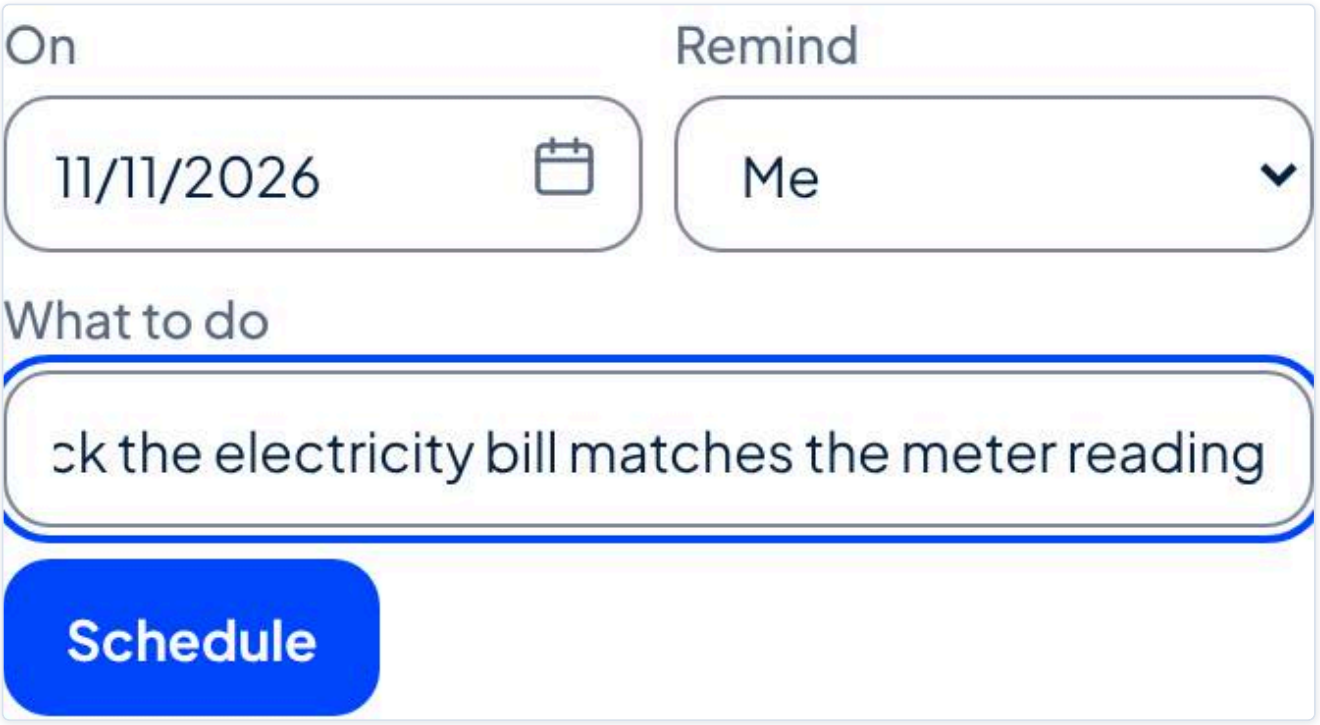
#	Part	What it shows	What clicking it does
1	Breadcrumbs	Accounting / Journal / JV-002577	Journal returns to the list with the filters it was opened from
2	Position, previous, next	"2 / 50" and ‹ › buttons	Opens the previous or next entry of the list the person came from (the row ids are kept in sessionStorage); "1 / 1" when opened from a search
3	Number and title	JV-002577 in large bold, the memo below in grey	None

#	Part	What it shows	What clicking it does
4	Status pill	"Posted" in a white pill with a grey outline. An entry is always Posted: there is no draft, cancelled or reversed state on the entry itself, and no status bar (no workflow)	None
5	Print or save as PDF (blue)	Browser print of the A4 voucher	Print dialog; the screen page is hidden and the voucher shown
6	PDF in English	White button	Downloads `/api/finance/journal/<id>/voucher.pdf?lang=en`
7	PDF in Arabic	White button	The same voucher in Arabic (`lang=ar`)
8	Lines tile	Count of lines (3) and "Lines"	Jumps to the lines table
9	Receipts tile	Count of attachments (2) and "Receipts"	0 files: jumps to Attachments to add one; 1 file: opens it in a new tab; several: opens the viewer "All attachments"
(10)	Posted for tile	The source document (a bill, payout, petty cash, invoice, cheque, withdrawal, claim, handover or the request kind)	Opens that document
10	Approval tile	Name of the person who decided and "Approval"; only when the entry went through a second person	Opens the approval request
(order)	Order tile	The order number, on delivery entries	Opens the order page
11	Type, Day, What, Reference, Posted for, Order	Kind in words, Cairo day, memo, reference when the entry has one	Posted for and Order are links
12	Prepared by, Prepared on	The person who asked when the entry went through a second person, else the poster; the posting moment	None
13	Approved by, Approved at	The second person and the decision moment; absent when no approval was needed	None
14	Amount	Sum of the debit lines, bold, in EGP	None
15	Lines table	Account (code and name; a wallet shows its owner), Store (code), Merchant, Rider (name and badge), Debit, Credit; largest debit first	None
16	Totals row	"Total" with the debit and credit totals, always equal	None

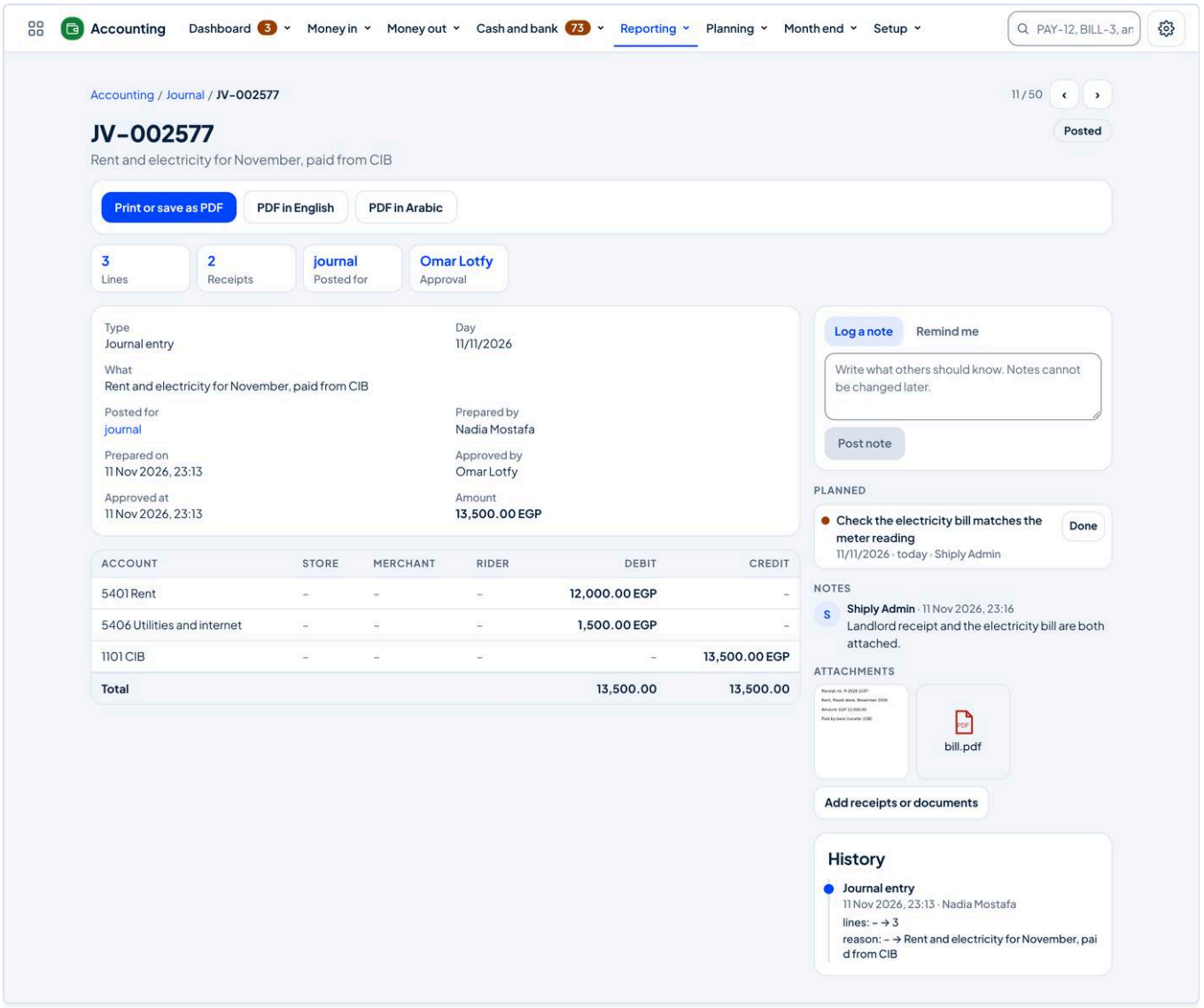
#	Part	What it shows	What clicking it does
17	Log a note (tab)	Text box "Write what others should know. Notes cannot be changed later." and Post note	Adds an append only note (audited `record.note`)
18	Remind me (tab)	On (a Cairo day), Remind (Me or another person who may open the record), What to do, Schedule	Adds a planned activity; a push `activity_due` is sent once on its day
19	Planned	Open reminders with day, "today" or "late", the person, and Done	Done closes it (only the person reminded or who planned it)
20	Notes	Notes newest first with the writer's initial, name and time	None
21	Attachments	A square thumbnail per photo, a tile with a red PDF icon and the file name per PDF; "No receipt or photo yet." when empty	One click opens the file in a new tab
22	Add receipts or documents	Opens the attachment field (drop files, Choose files, Take a photo; photos or PDF up to 5 MB, at most 10) and Save files	The files are linked to the entry (audited "Receipts or documents added")
23	History	The audit rows of the entry: action in words, time, person, each changed field "before → after"	None



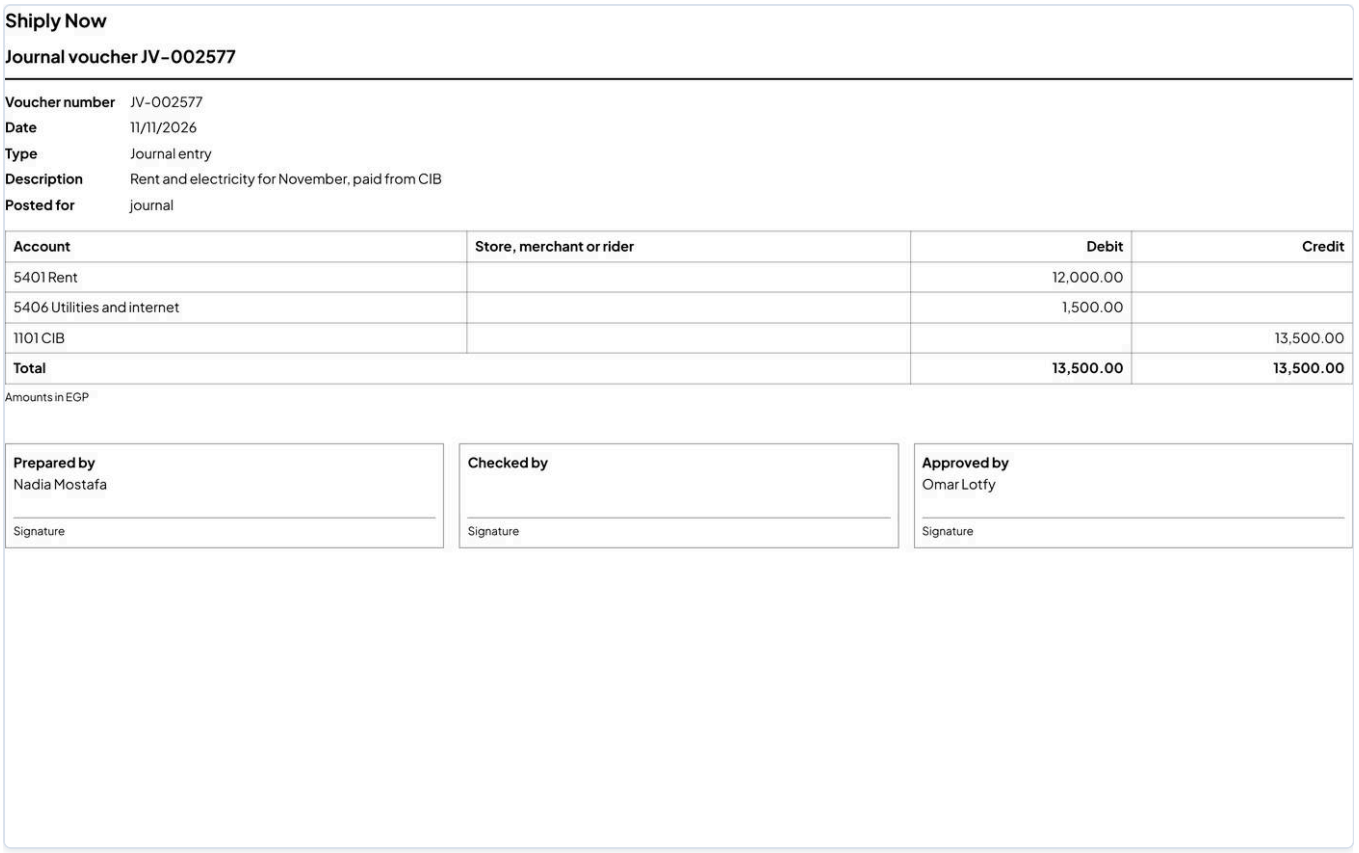
The viewer "All attachments", opened by the Receipts tile when there are several files: the photo and the PDF tile, each with who added it, and Close.



Remind me filled in: day, person and what to do.



After Schedule and Post note: the reminder under Planned (brown dot, "today", Done) and the note under Notes.



Shiply Now

Journal voucher JV-002577

Voucher number JV-002577
Date 11/11/2026
Type Journal entry
Description Rent and electricity for November, paid from CIB
Posted for journal

Account	Store, merchant or rider	Debit	Credit
5401 Rent		12,000.00	
5406 Utilities and internet		1,500.00	
1101 CIB			13,500.00
Total		**13,500.00**	**13,500.00**

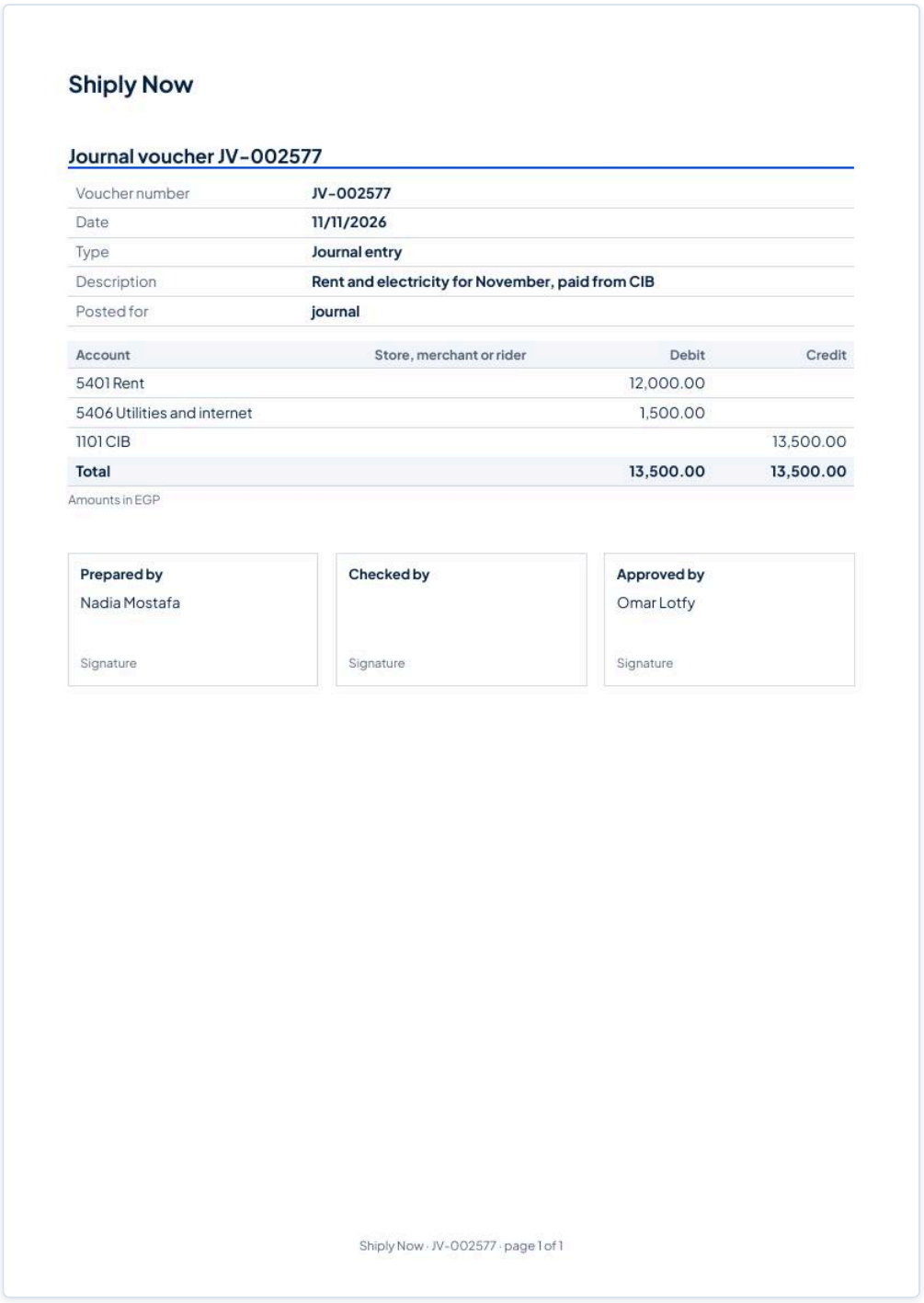
Amounts in EGP

Prepared by
Nadia Mostafa
Signature

Checked by
Signature

Approved by
Omar Lotfy
Signature

The print view (Print or save as PDF): company, "Journal voucher JV-002577", number, date, type, description, posted for, the lines with Store, merchant or rider, totals, "Amounts in EGP", and three boxes Prepared by, Checked by, Approved by with signature lines.



Shiply Now

Journal voucher JV-002577

Voucher number	JV-002577
Date	11/11/2026
Type	Journal entry
Description	Rent and electricity for November, paid from CIB
Posted for	journal

Account	Store, merchant or rider	Debit	Credit
5401 Rent		12,000.00	
5406 Utilities and internet		1,500.00	
1101 CIB			13,500.00
Total		**13,500.00**	**13,500.00**

Amounts in EGP

Prepared by	Checked by	Approved by
Nadia Mostafa		Omar Lotfy
Signature	Signature	Signature

Shiply Now · JV-002577 · page 1 of 1

The voucher PDF in English made by the API.

Shiply Now

قيد يومية JV-002577

رقم القيد	**JV-002577**
التاريخ	**11/11/2026**
النوع	**Journal entry**
البيان	**Rent and electricity for November, paid from CIB**
سُجّل من أجل	**journal**

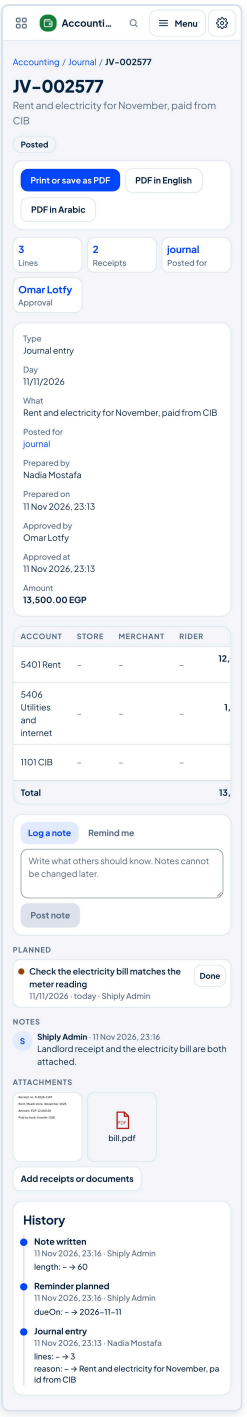
الحساب	المتجر أو التاجر أو المندوب	مدين	دائن
5401 إيجار		12,000.00	
5406 مرافق وإنترنت		1,500.00	
1101 البنك التجاري الدولي CIB			13,500.00
الإجمالي		**13,500.00**	**13,500.00**

المبالغ بالجنيه المصري

أعدّه	راجعه	اعتمده
Nadia Mostafa		Omar Lotfy
التوقيع	التوقيع	التوقيع

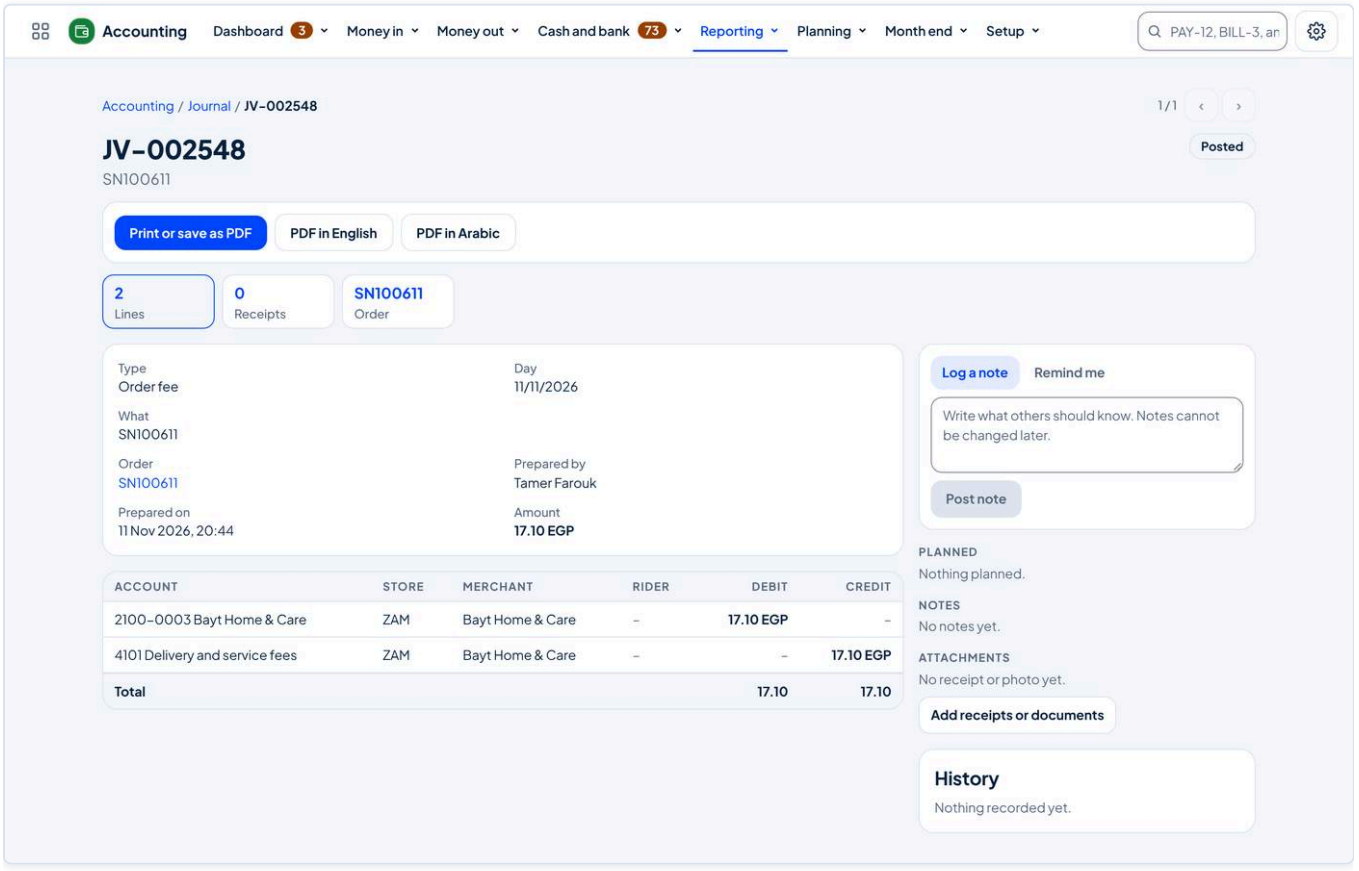
شيبلي ناو · · JV-002577 صفحة 1 من 1

The voucher PDF in Arabic.

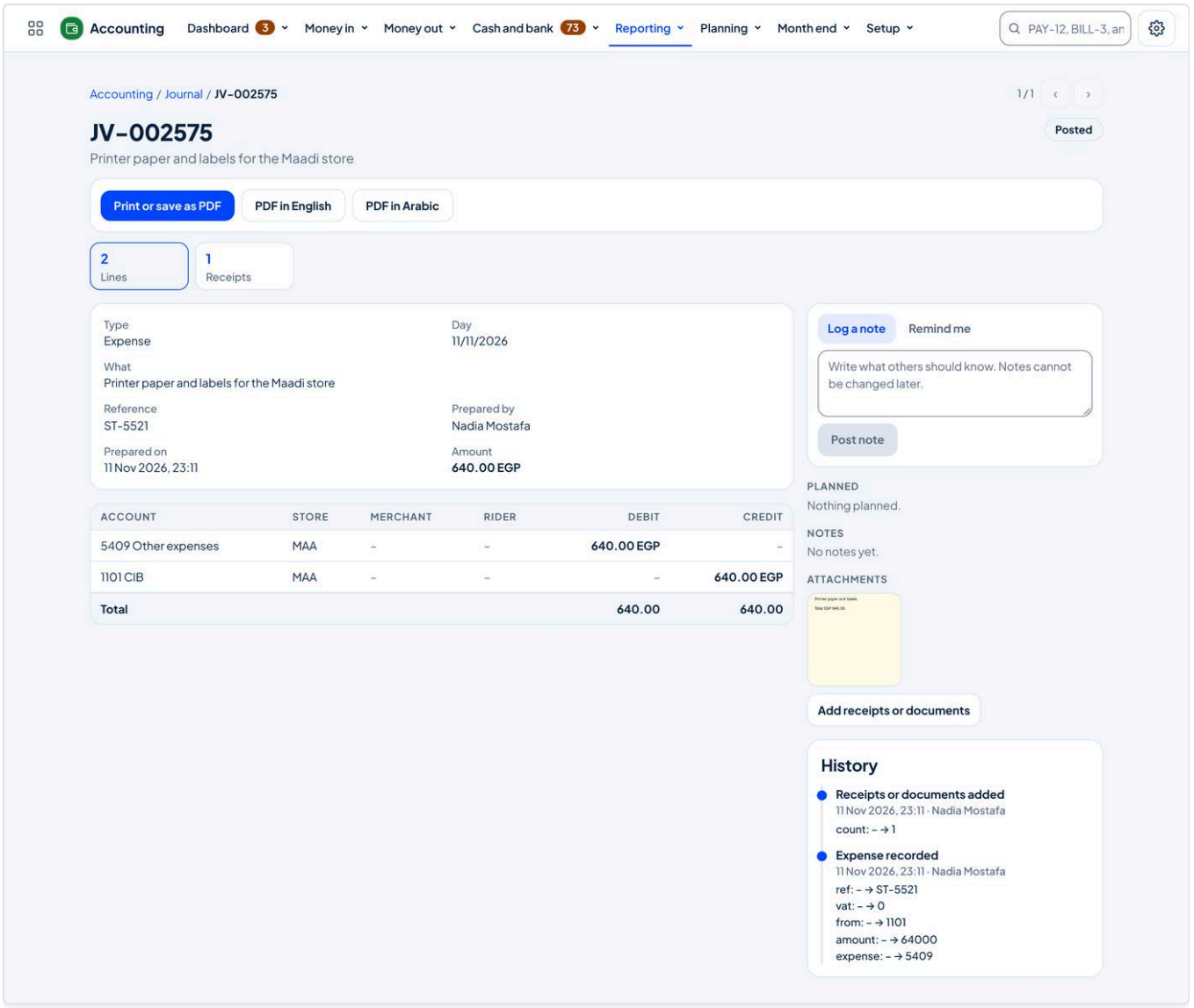


The entry on a phone: tiles three to a row, fields in one column, the lines table scrolls sideways, the chatter below.

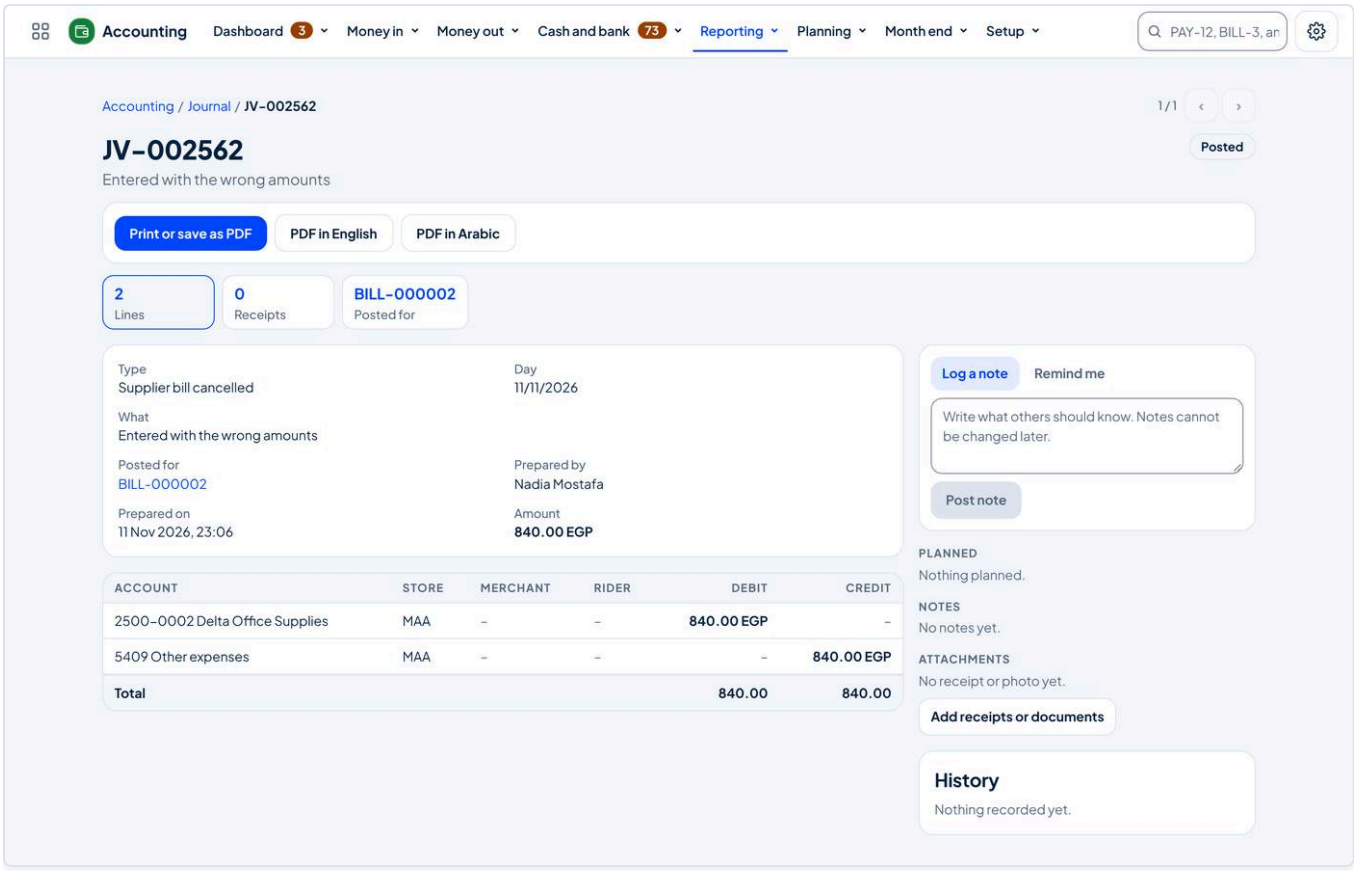
The same page for other kinds of entry



A delivery entry (JV-002548, Order fee): two lines on the merchant's wallet and fee income tagged with store ZAM and the merchant; tiles Lines 2, Receipts 0, Order SN100611; no approval; History "Nothing recorded yet." because jobs and deliveries are audited on the order.



An expense below the approval limit (JV-002575): posted at once, so no Approval tile; Receipts 1 opens the single photo directly; the photo appears under Attachments.



A correction entry (JV-002562, Supplier bill cancelled): the reversing lines of the bill, with "Posted for BILL-000002" linking to the bill it cancels.

16.12 Journal entry status

An entry is written to the books only when it is posted. Posting (`post()` in `packages/db/src/finance/ledger.ts`) inserts the entry and its lines in one transaction; the lines must add up to zero (a deferred trigger refuses anything else) and the rows can never be changed afterwards. That is why the entry page always shows the pill **Posted** and nothing else.

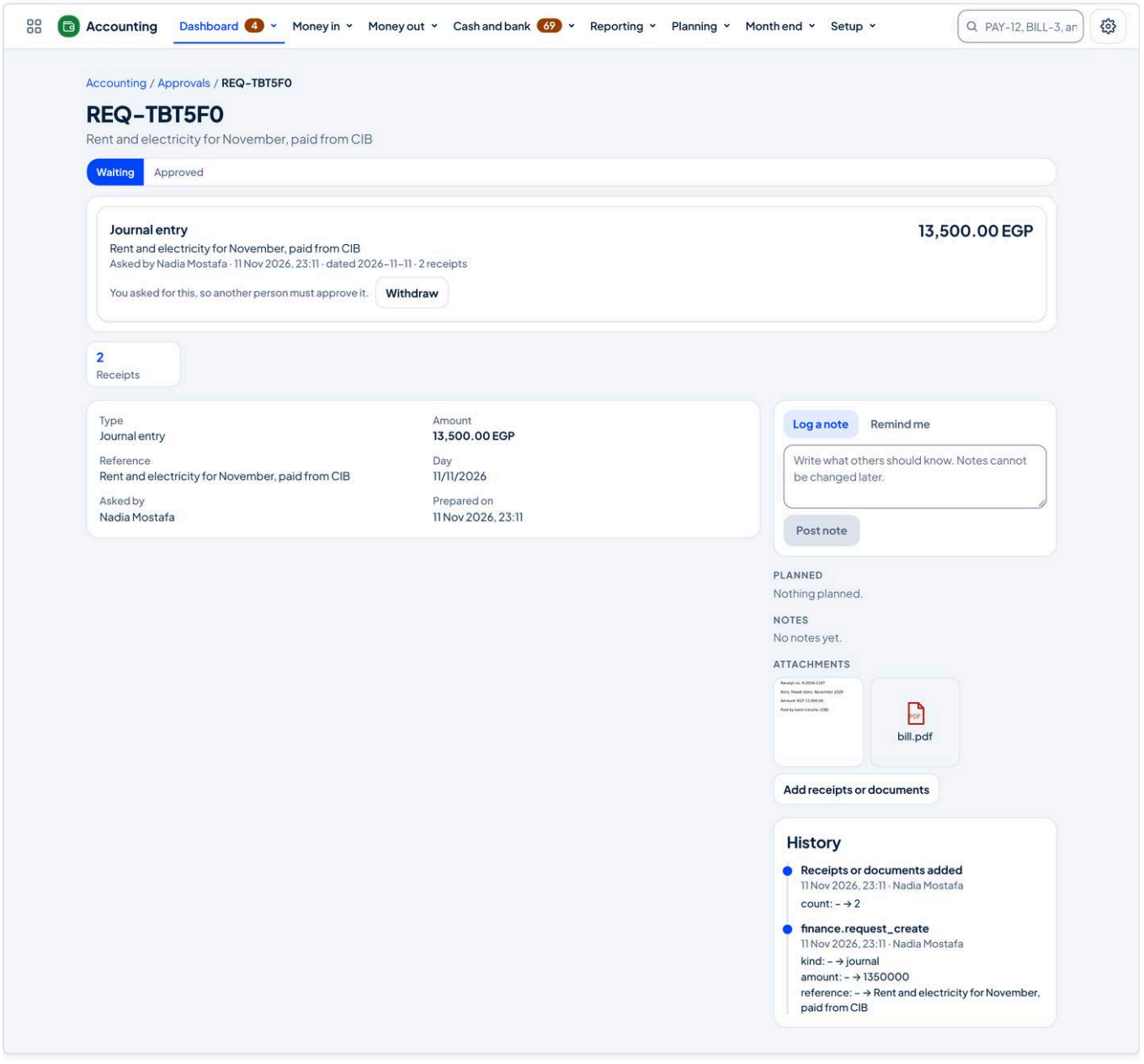
Money that needs a second person

Money going out from `finance.approval_threshold_piastres` (10,000 EGP by default) and every claimed money in (while `finance.approve_money_in` is 1) is not posted when it is asked. It is stored as a **finance request** (`finance_requests`) and waits in Money in or Dashboard, Approvals. Nothing appears in the journal, the reports or the balances until a second person approves it.

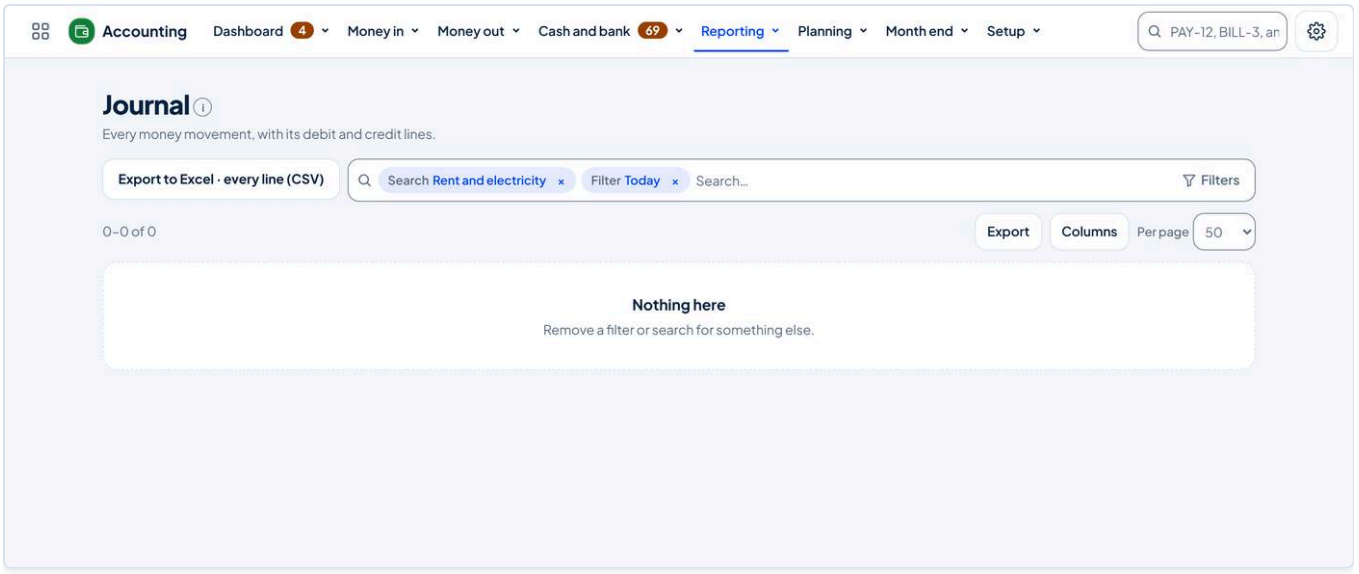
Request state	How it looks	What happened	Journal
Waiting (pending)	Number REQ-xxxxxx, status bar "Waiting" filled blue, "Approved" grey; the request card with Approve, Reason to reject and Reject (or, for the person who asked, "You asked for this, so another person must approve it." and Withdraw)	Asked by, day, amount, kind and receipts are stored	Nothing yet

Request state	How it looks	What happened	Journal
Approved	"Waiting" with a check mark, "Approved" filled blue; the title becomes the entry number; tiles Receipts and Journal entry; fields Decided by and Decided on	The entry was posted in the approval's transaction	The posted entry with Prepared by (the asker) and Approved by (the decider)
Rejected	Steps "Waiting", "Approved" grey, side state "Rejected" filled red; field Decision note	A reason is required	Nothing is posted
Withdrawn (cancelled)	Side state "Withdrawn" filled red	The asker withdrew it before a decision	Nothing is posted

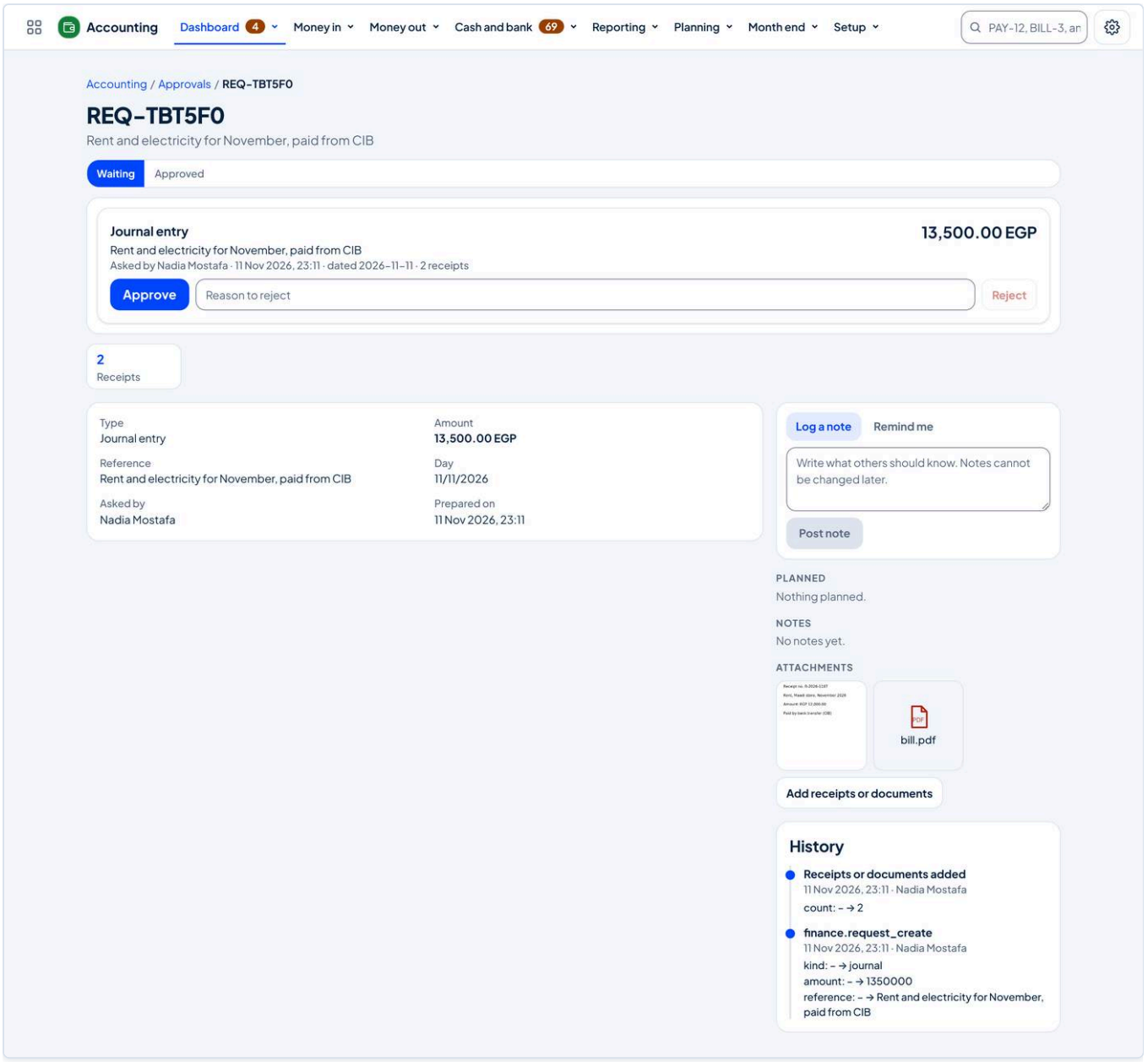
Deciding needs **Accounting manage** (`POST /finance/requests/:id/decide`) and never by the person who asked (`same_person`).



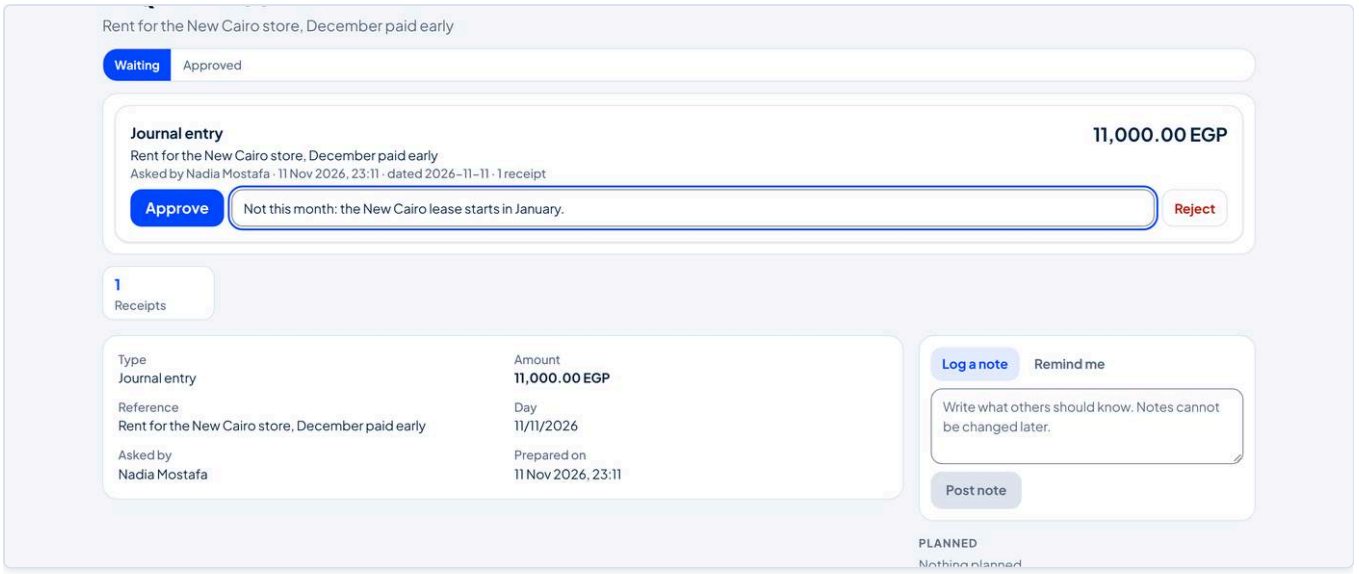
The request as its asker sees it: no Approve button, the note "You asked for this, so another person must approve it." and Withdraw.



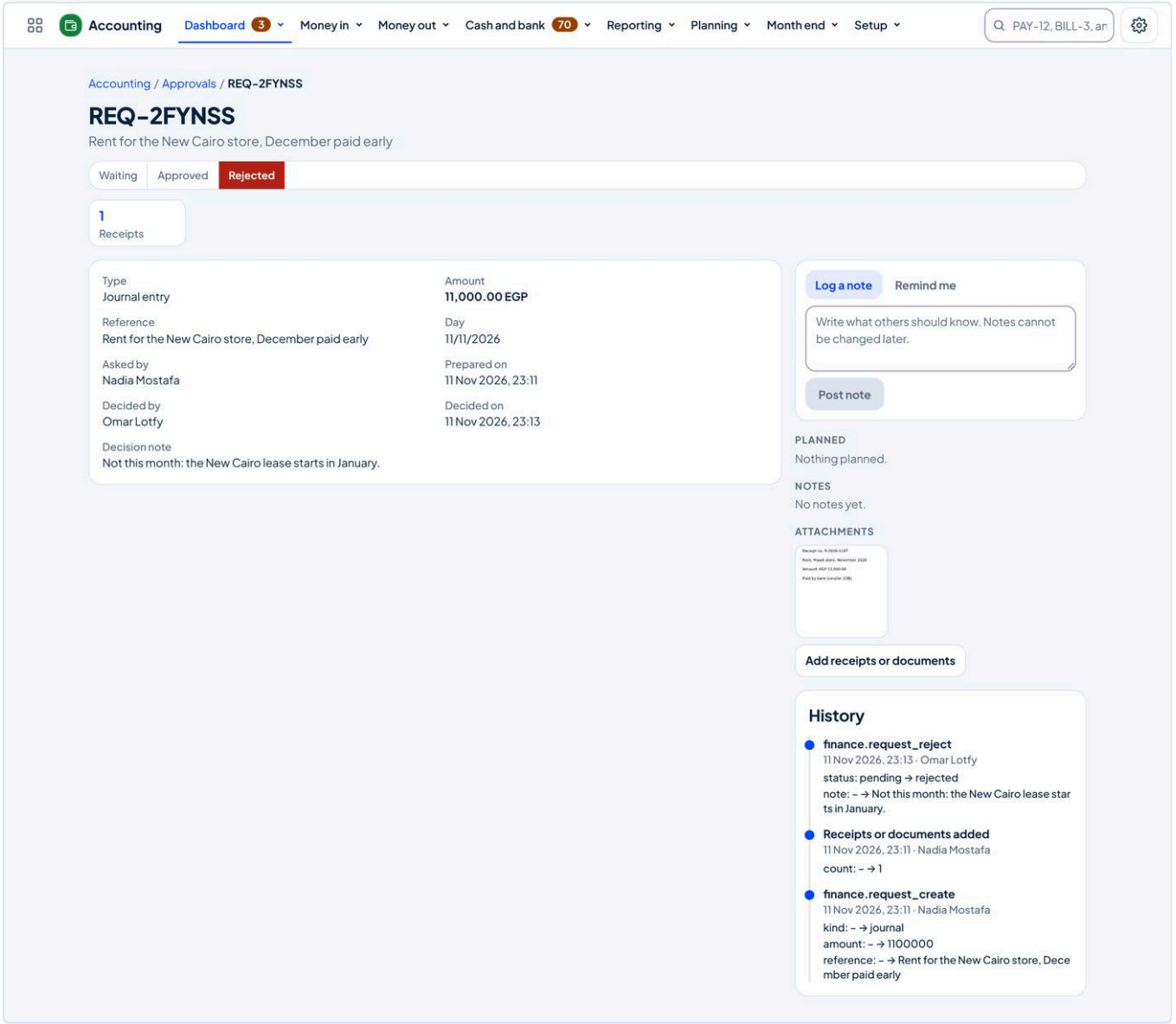
While it waits, the journal finds nothing for it (Today, search "Rent and electricity").



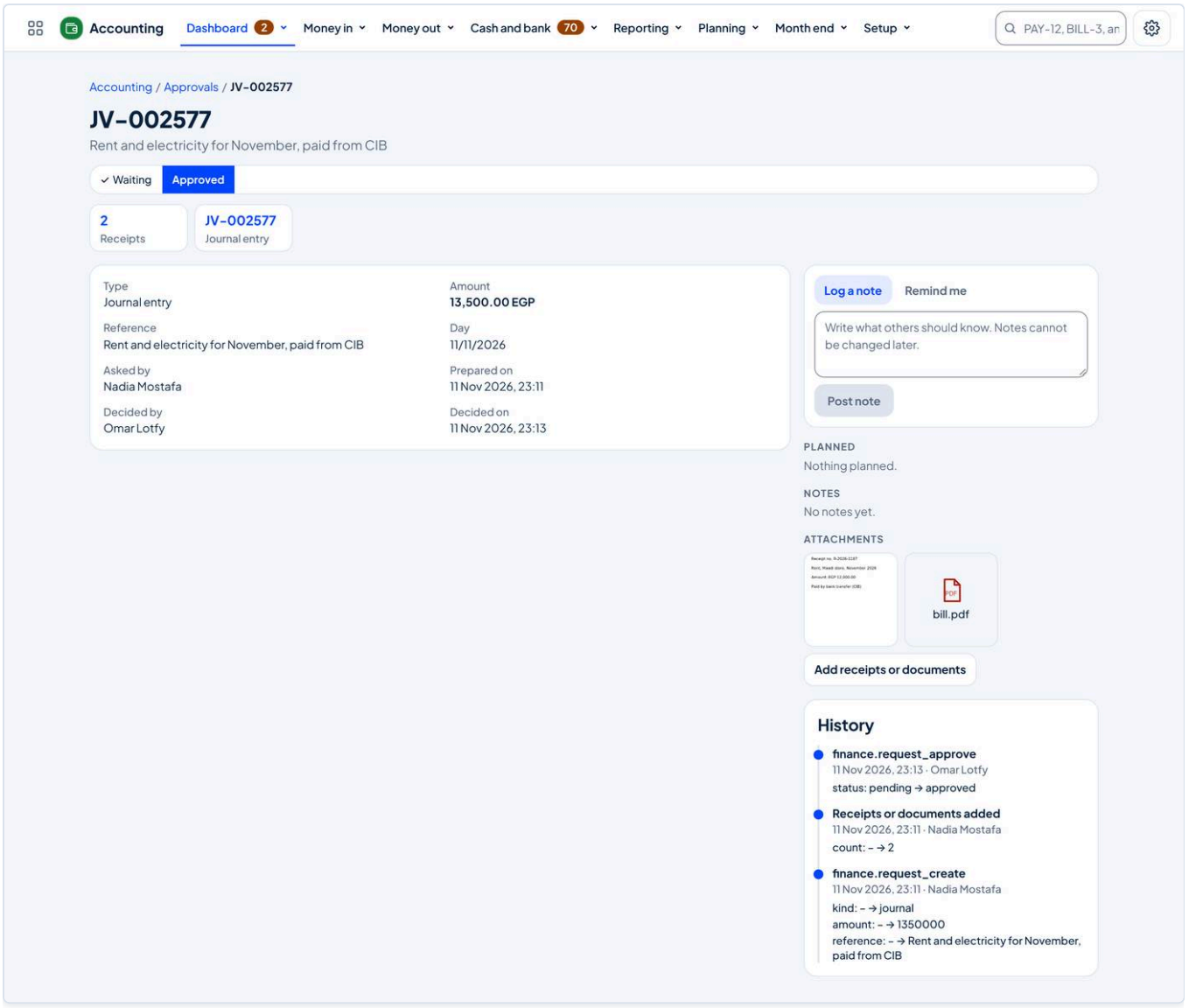
The same request as the second person sees it (/finance/approvals/<id>): REQ number, status bar Waiting, the card with the amount, "Asked by", Approve, Reason to reject and Reject; tile Receipts 2; fields Type, Amount, Reference, Day, Asked by, Prepared on; the chatter with both attachments and the History.



The second person types a reason; Reject stays disabled until a reason is written.



A rejected request: "Rejected" in red, Decided by, Decided on and the Decision note; History "finance.request_reject" with the note.



An approved request: the status bar moved to Approved, the tile JV-002577 Journal entry opens the posted entry.

The posted entry after approval is the one shown at the top of the journal entry page section: Prepared by is the asker, Approved by and Approved at the decider, and its Approval tile leads back to this request.

Corrections are new entries

Nothing is ever edited. A correction is a new entry whose lines turn the first one round, or an adjustment with a written reason:

Correction	Kind of the new entry	Where the link shows
Cancelling an unpaid bill	supplier_bill_cancel	"Posted for BILL-…" tile and field on the reversing entry; the bill lists both entries
Reversing a store petty cash expense	petty_cash_reversal	"Posted for PC-…" on the reversing entry; the petty cash page lists both
Accrual reversed on the 1st of next month (daily job, key `accrual_reversal:<id>`)	accrual_reversal	Only the memo "Reversal: …" names the accrual

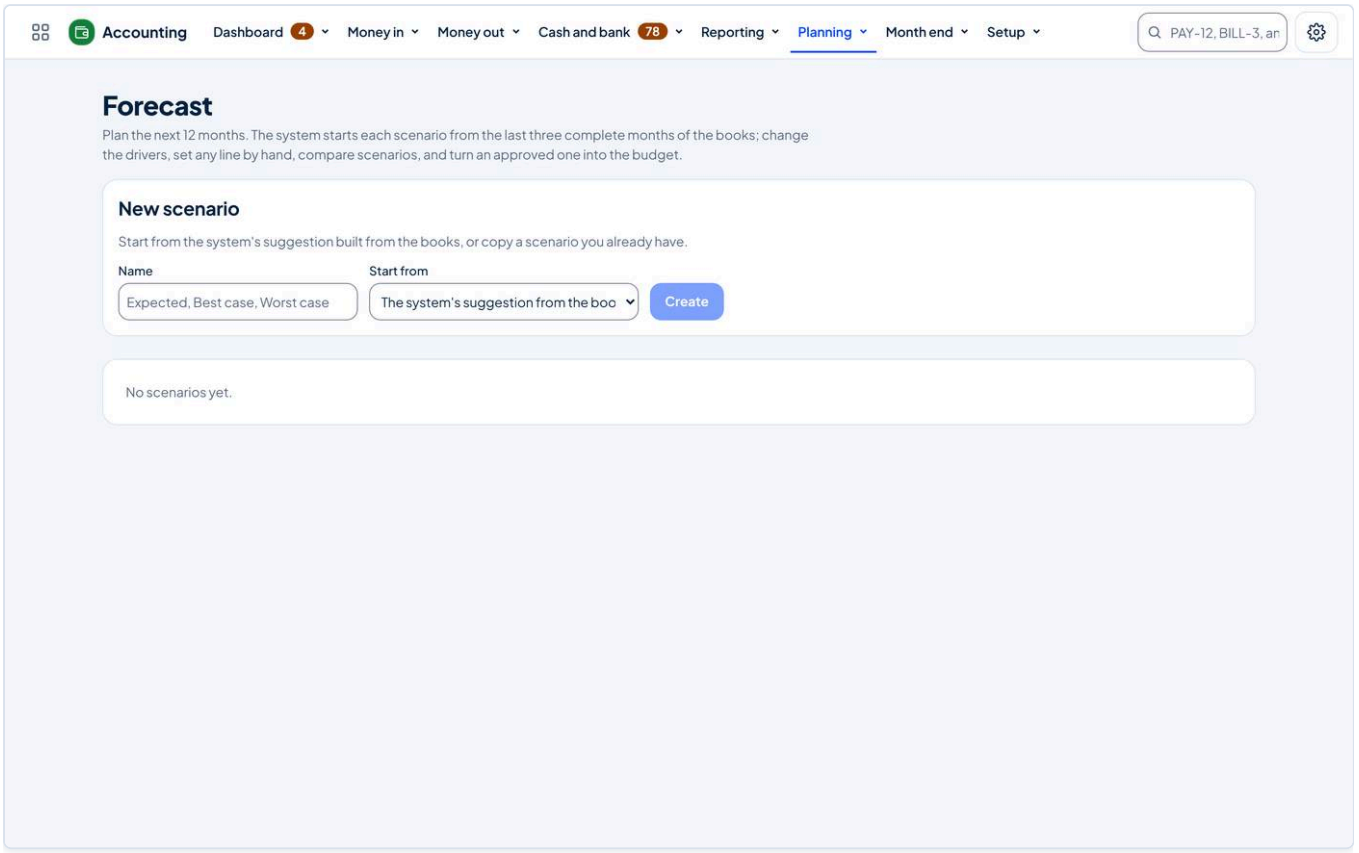
Correction	Kind of the new entry	Where the link shows
A bounced or cancelled cheque	the booking turned round	"Posted for CHQ-…" on both entries
Credit note or debit note on an invoice	credit_note, debit_note	"Posted for CN-… / DN-…"
Any other mistake	journal or adjustment (Money movements, Journal entry, with a reason)	The reason is the memo; no automatic link

Closed books

Once the books are closed through a day (`books_lock.locked_before`), `post()` and a database trigger refuse any line dated before the lock: the forms answer "This date is in a closed month. Choose a later date, or ask an admin to reopen the month." (`books_closed`). Entries already in the closed days stay readable and can still receive notes, reminders and attachments, but no new lines; a correction is posted on an open day. Jobs that would date a line inside closed books move it to the first open day (accrual reversals, storage catch up, depreciation).

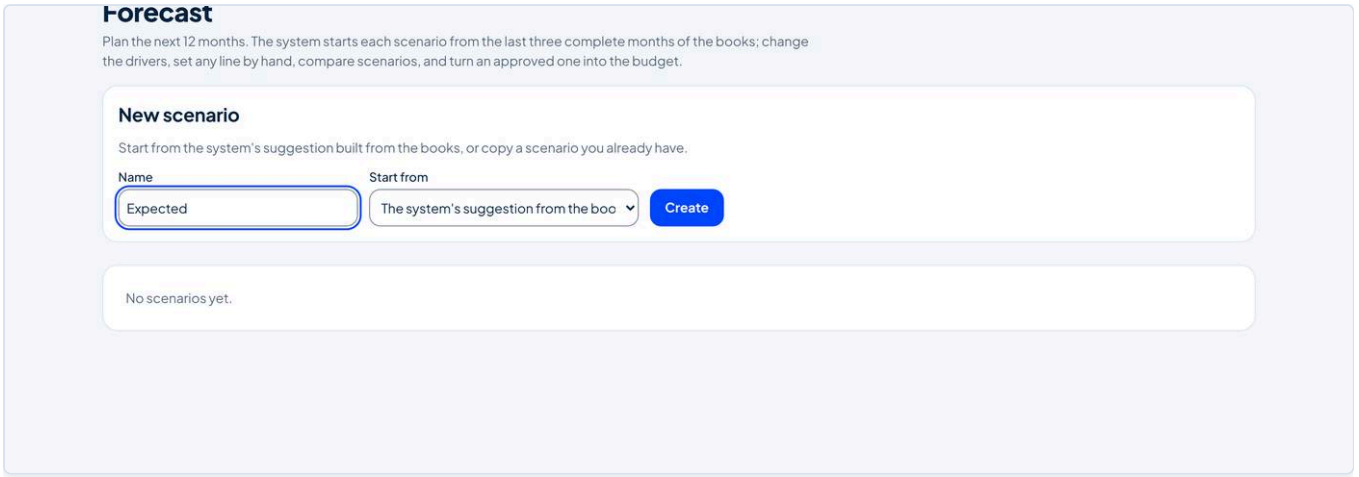
16.13 Forecast

Addresses: `/finance/forecast` (list and new scenario) and `/finance/forecast/<id>?tab=drivers|pl|bs|cf|compare` . **Menu:** Planning, Forecast. **Who:** Accounting view reads; Accounting use creates, edits, copies, renames and deletes drafts; Accounting manage approves, opens again and uses as budget (`apps/api/src/finance/forecast.controller.ts`). A scenario covers the next 12 months from the current month and starts from the last three complete months of the books (fewer when the books are younger).

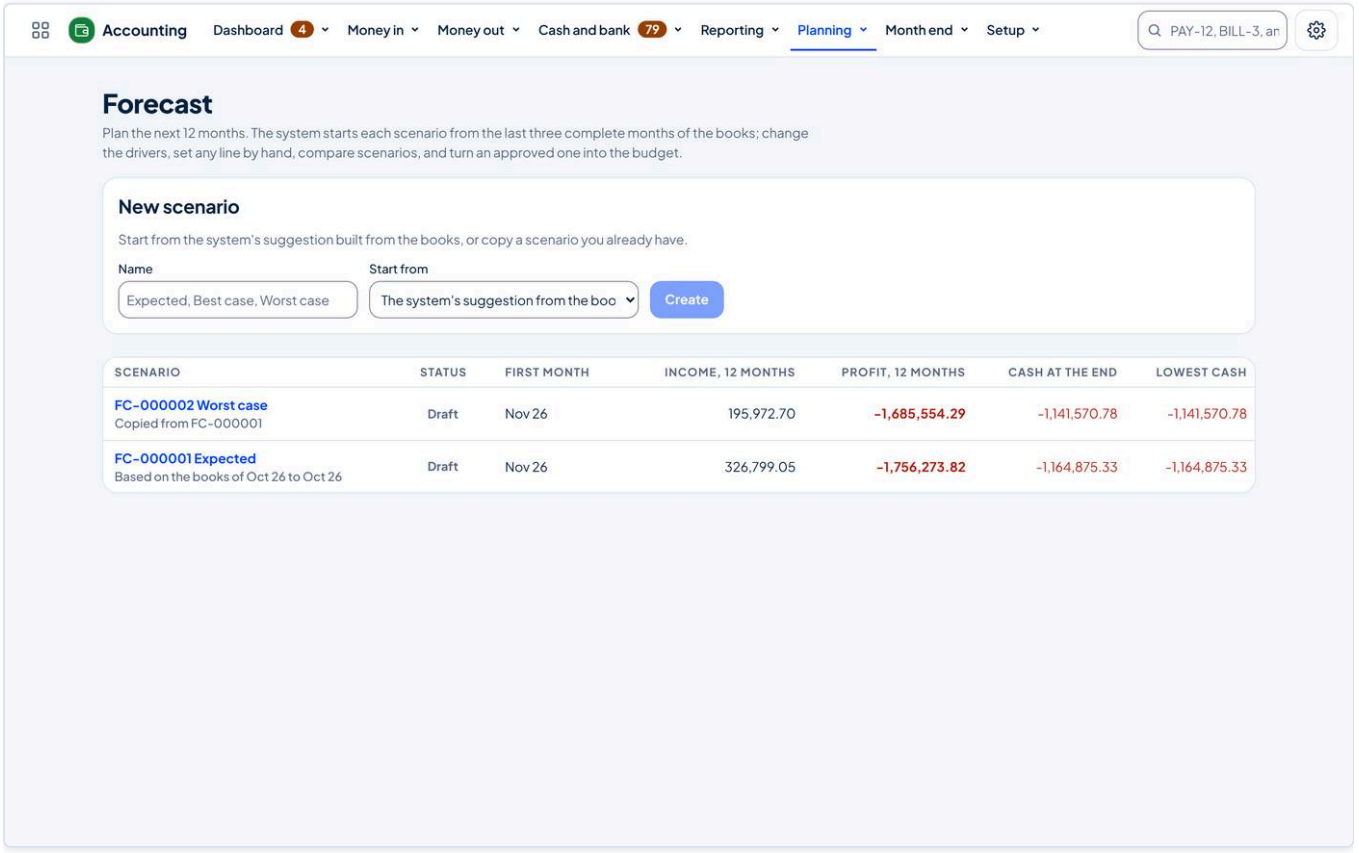


The forecast page before any scenario: New scenario form (Name, Start from) and "No scenarios yet." (/finance/forecast).

Control	What it does	Rule / limits	Result
Name	The scenario's name (Expected, Best case, Worst case)	Required	None
Start from	"The system's suggestion from the books" or "A copy of FC-… name"	The suggestion is built by `GET /finance/forecasts/suggestion`	None
Create	`POST /finance/forecasts`	Accounting use	Opens the new scenario FC-00000N
List columns	Scenario, Status, First month, Income 12 months, Profit 12 months, Cash at the end, Lowest cash, Delivered orders 12 months	None	A row opens the scenario



New scenario filled in: Name "Expected", Start from the system's suggestion.



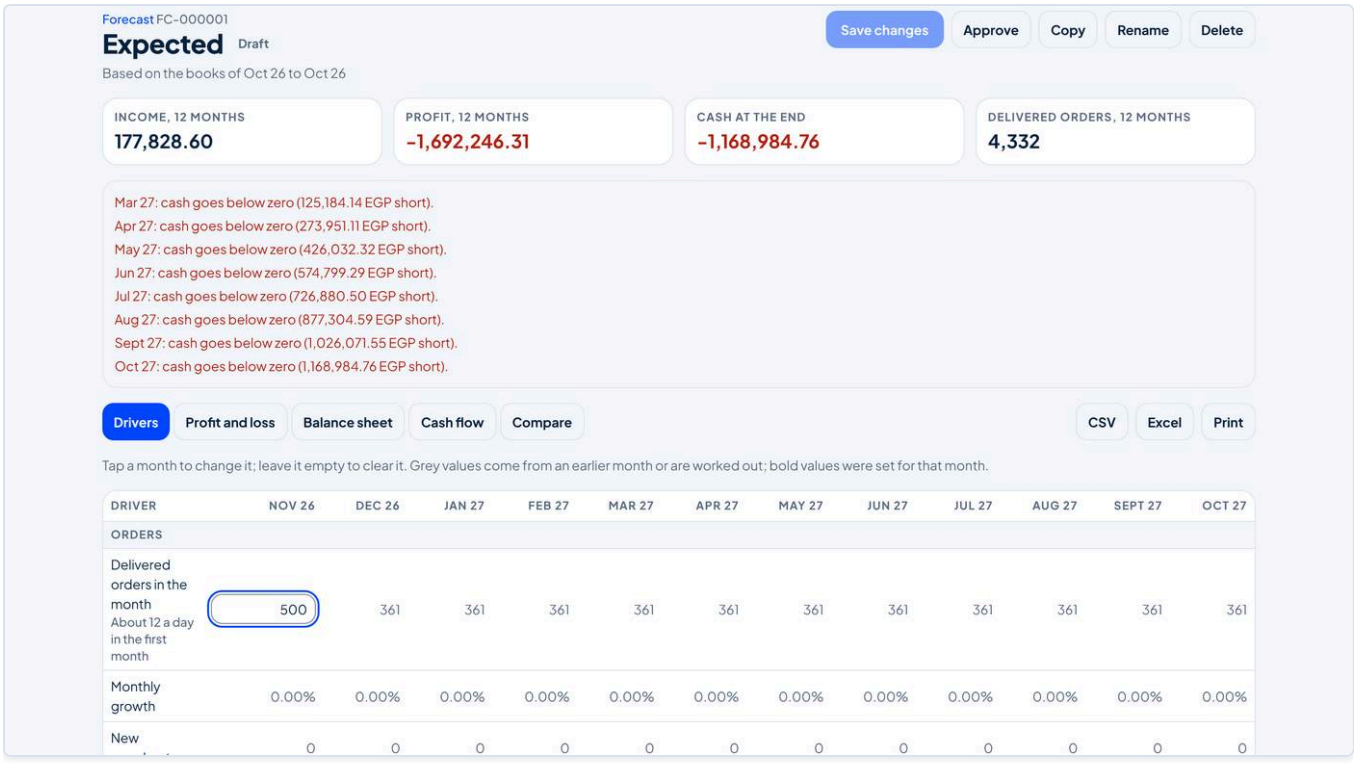
The list with two scenarios.

The scenario page

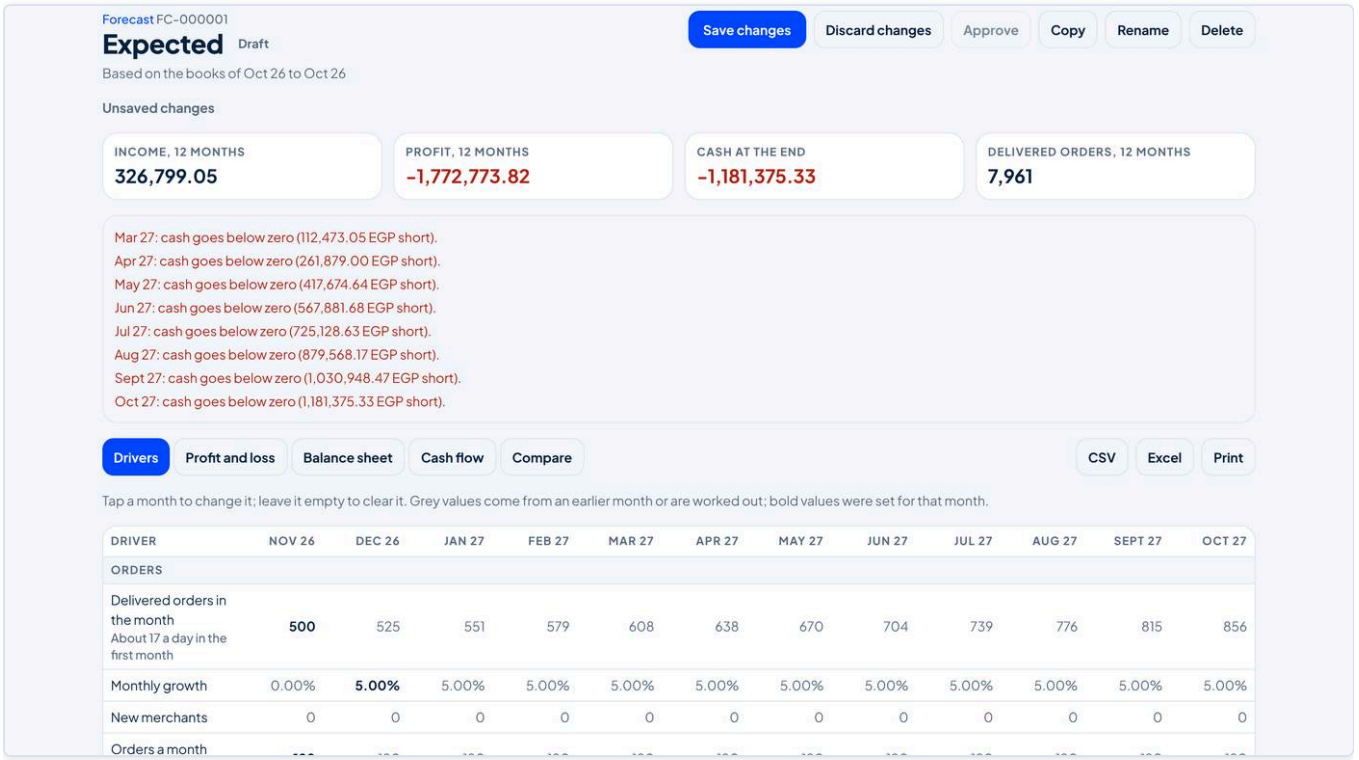
A new scenario on the Drivers tab: header with Save changes, Approve, Copy, Rename, Delete; four summary cards; the cash warnings in red; tabs; the drivers by month.

Control	What it does	Rule / limits	Result
Forecast FC-000001 (link), name, status pill	Back to the list; Draft (grey pill) or Approved (green pill)	"Based on the books of … to …", "Copied from …", "Approved by …"	None

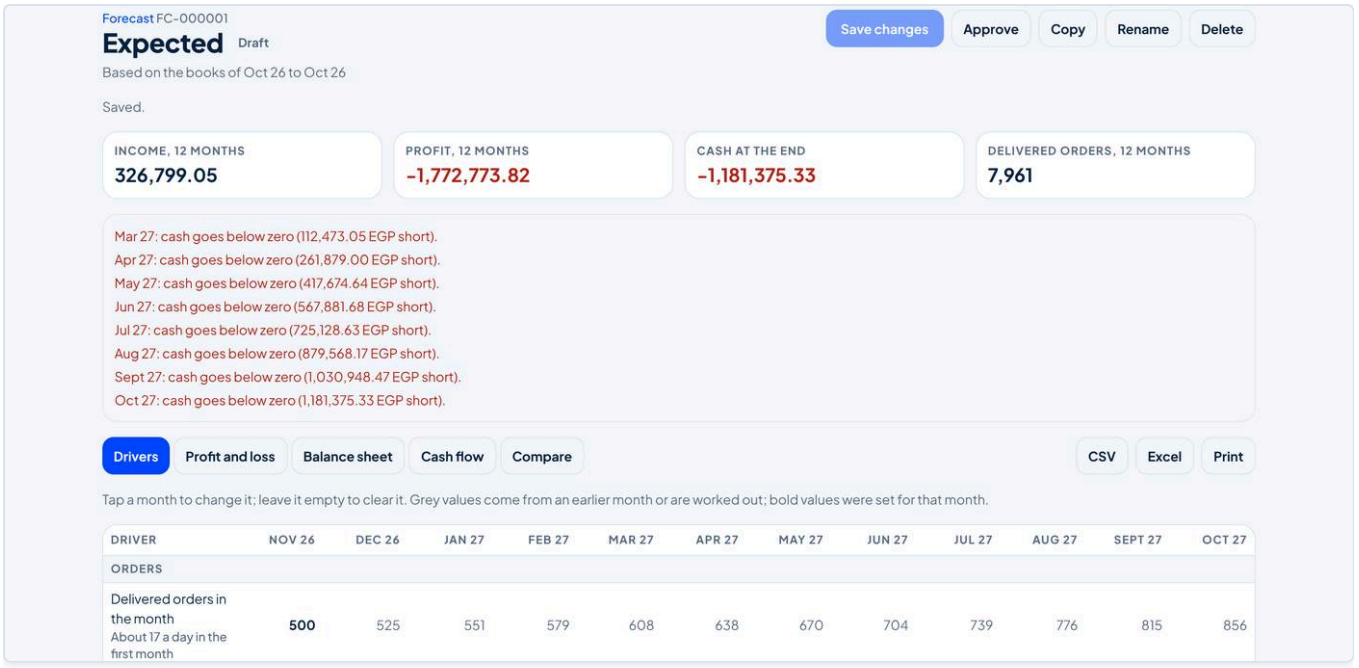
Control	What it does	Rule / limits	Result
Save changes	Saves changed drivers (`PUT :id/drivers`) and lines set by hand (`PUT :id/overrides`)	Draft only, Accounting use; disabled while nothing changed	"Saved."
Discard changes	Back to the saved values	Shown only while changed; leaving the page with changes asks first	None
Approve	`POST :id/approve`	Accounting manage; disabled while there are unsaved changes; an approved scenario cannot be changed (`forecast_approved`)	"Approved. It can now be used as the budget."
Open again	`POST :id/reopen`	Approved only, Accounting manage	Back to Draft
Use as budget	Opens the preview, then writes the budget	Approved only, Accounting manage	See below
Copy	Asks "Name of the copy" (proposes "name (copy)")	Accounting use	Opens the copy as a new draft
Rename	Asks "New name"	Accounting use	None
Delete	"Delete this scenario? This cannot be undone."	Draft only	Back to the list
Summary cards	Income 12 months, Profit 12 months, Cash at the end, Delivered orders 12 months	Recomputed live in the browser (`forecastCompute`) with every change	None
Warnings (red)	"Mar 27: cash goes below zero (… EGP short).", more repaid than borrowed, merchants would owe us	One line per month that breaks	None
Tabs	Drivers, Profit and loss, Balance sheet, Cash flow, Compare	The tab is kept in the address (`?tab=`)	None
CSV, Excel, Print	Exports or prints the tab shown	None	A file or the print dialog
Driver cells	Tap a month to change it; leave it empty to clear it	Bold: set for that month; grey: carried from an earlier month or worked out. Groups: Orders, Fee income per delivered order (before VAT), Cash and VAT, Costs, Equipment, loans and owner; "Add a fee per order", "Add an income or cost account"	The whole forecast recalculates at once



Editing a driver: the November cell of "Delivered orders in the month" becomes an input.



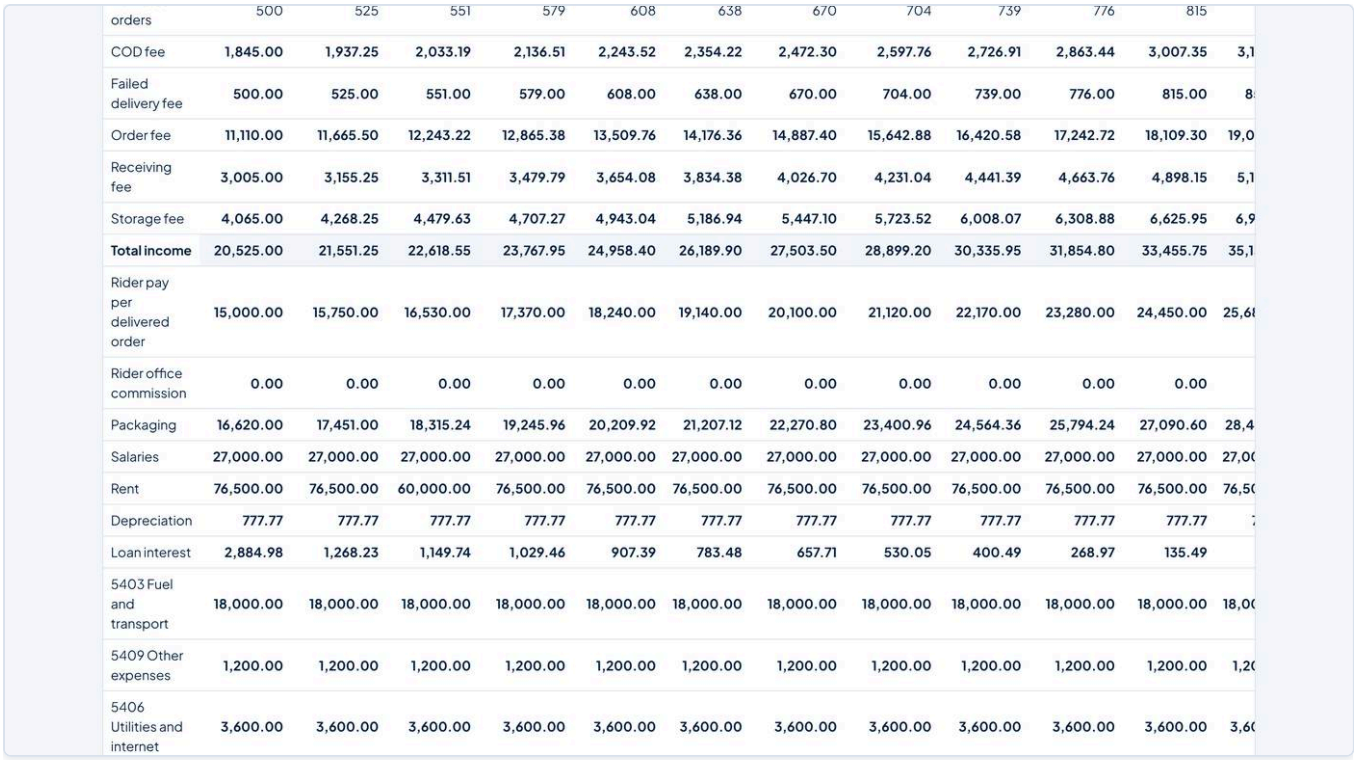
After 500 orders in November and 5% monthly growth from December: later months are worked out (525, 551…), the cards and warnings change at once, "Unsaved changes" appears with Save changes and Discard changes; Approve is disabled.



After Save changes.

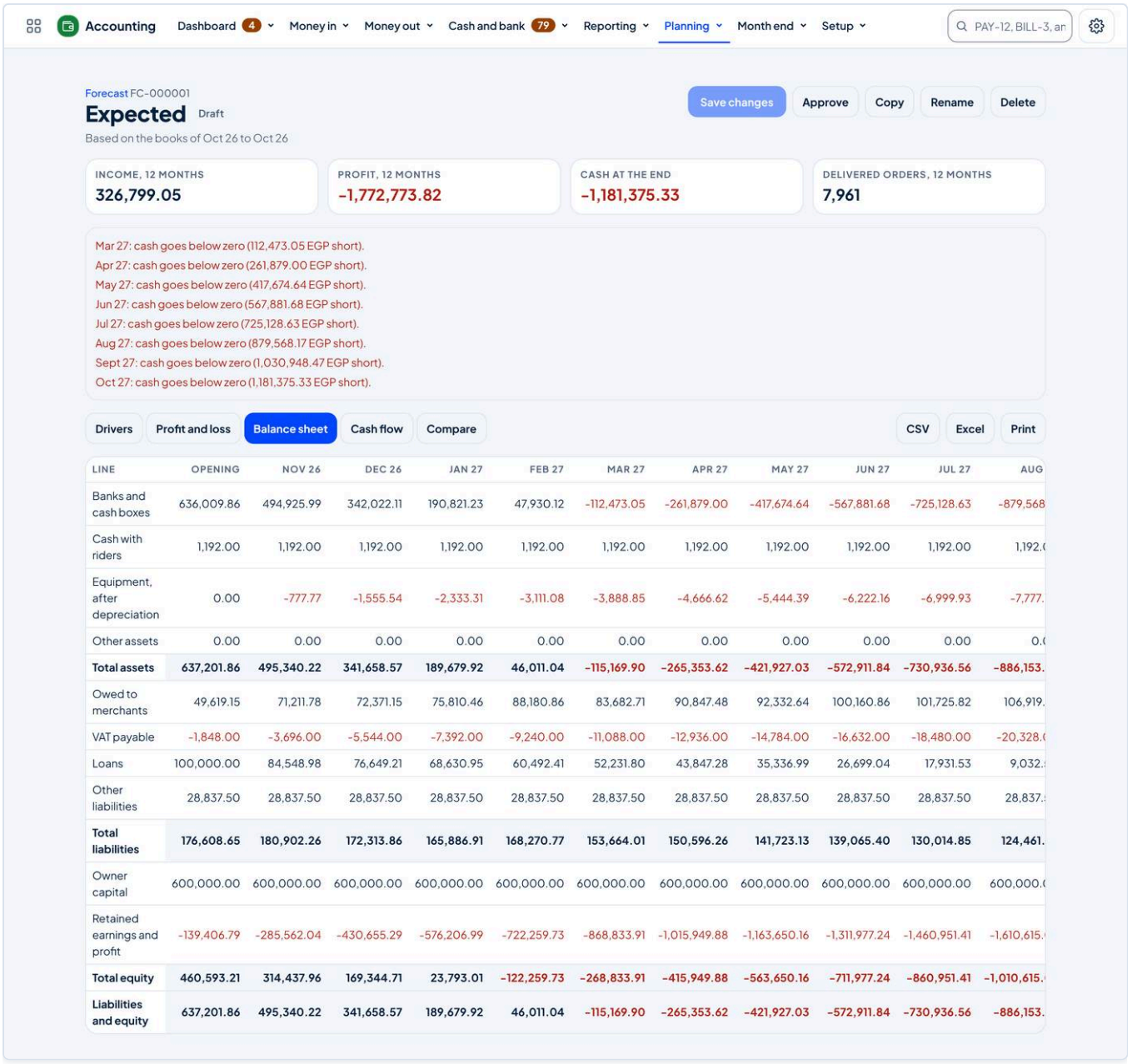
LINE	NOV 26	DEC 26	JAN 27	FEB 27	MAR 27	APR 27	MAY 27	JUN 27	JUL 27	AUG 27	SEPT 27	C
Delivered orders	500	525	551	579	608	638	670	704	739	776	815	
COD fee	1,845.00	1,937.25	2,033.19	2,136.51	2,243.52	2,354.22	2,472.30	2,597.76	2,726.91	2,863.44	3,007.35	3,1
Failed delivery fee	500.00	525.00	551.00	579.00	608.00	638.00	670.00	704.00	739.00	776.00	815.00	8
Order fee	11,110.00	11,665.50	12,243.22	12,865.38	13,509.76	14,176.36	14,887.40	15,642.88	16,420.58	17,242.72	18,109.30	19,0
Receiving fee	3,005.00	3,155.25	3,311.51	3,479.79	3,654.08	3,834.38	4,026.70	4,231.04	4,441.39	4,663.76	4,898.15	5,1
Storage fee	4,065.00	4,268.25	4,479.63	4,707.27	4,943.04	5,186.94	5,447.10	5,723.52	6,008.07	6,308.88	6,625.95	6,9
Total income	20,525.00	21,551.25	22,618.55	23,767.95	24,958.40	26,189.90	27,503.50	28,899.20	30,335.95	31,854.80	33,455.75	35,1
Rider pay per delivered order	15,000.00	15,750.00	16,530.00	17,370.00	18,240.00	19,140.00	20,100.00	21,120.00	22,170.00	23,280.00	24,450.00	25,6
Rider office commission	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	
Packaging	16,620.00	17,451.00	18,315.24	19,245.96	20,209.92	21,207.12	22,270.80	23,400.96	24,564.36	25,794.24	27,090.60	28,4
Salaries	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,0
Rent	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,5
Depreciation	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	
Loan interest	2,884.98	1,268.23	1,149.74	1,029.46	907.39	783.48	657.71	530.05	400.49	268.97	135.49	
5403 Fuel and transport	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,0
5409 Other expenses	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,2
5406 Utilities and internet	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,6
5301 Cash shortages and surpluses	35.00	35.00	35.00	35.00	35.00	35.00	35.00	35.00	35.00	35.00	35.00	
5701 Social insurance, company share	5,062.50	5,062.50	5,062.50	5,062.50	5,062.50	5,062.50	5,062.50	5,062.50	5,062.50	5,062.50	5,062.50	5,0
Total costs	166,680.25	166,644.50	168,170.25	169,820.69	171,532.58	173,305.87	175,203.78	177,226.28	179,310.12	181,518.48	183,851.36	186,3
Profit	-146,155.25	-145,093.25	-145,551.70	-146,052.74	-146,574.18	-147,115.97	-147,700.28	-148,327.08	-148,974.17	-149,663.68	-150,395.61	-151,

Profit and loss tab: income by fee kind, every cost line, total income, total costs and profit by month.



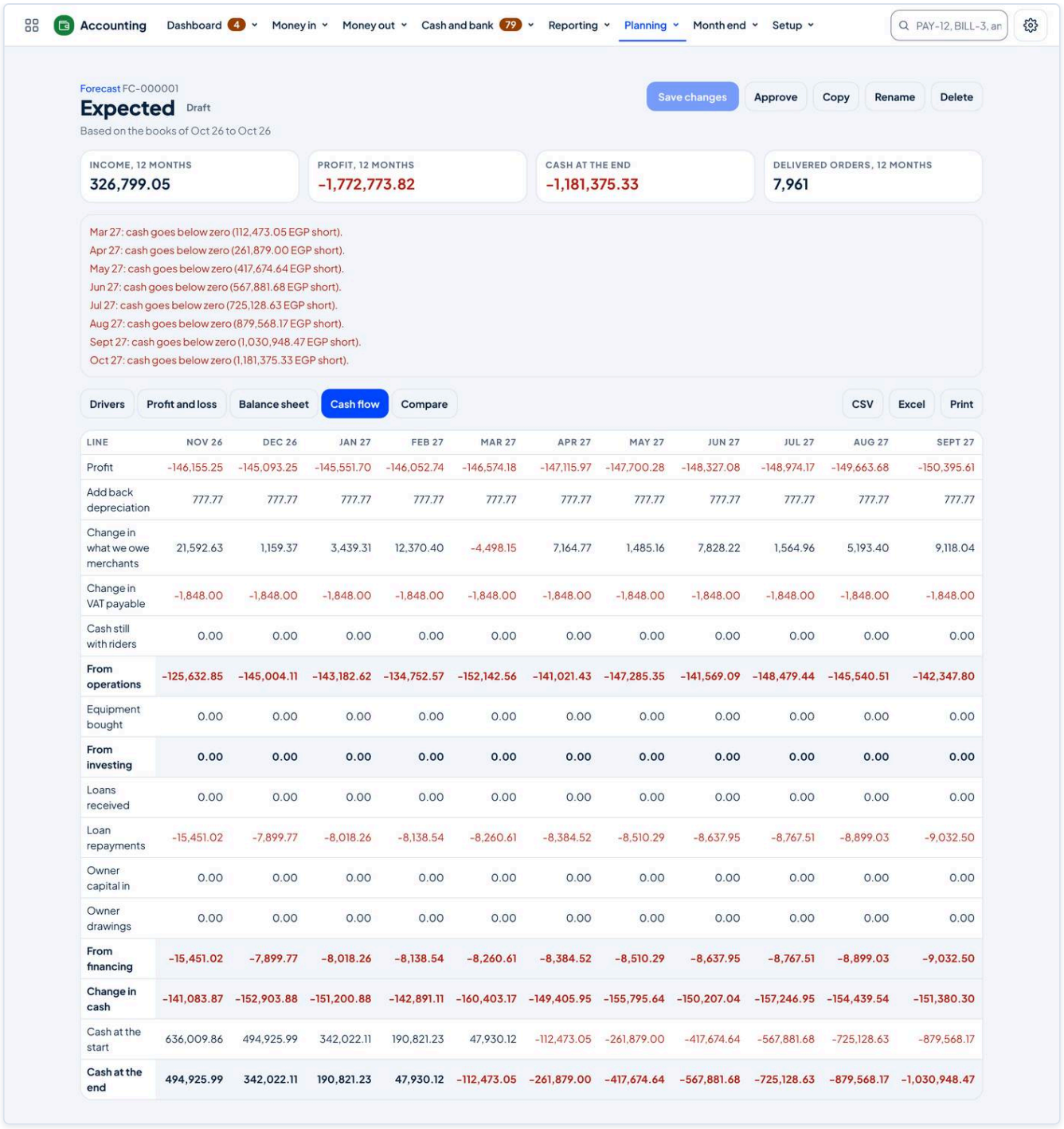
orders	500	525	551	579	608	638	670	704	739	776	815	
COD fee	1,845.00	1,937.25	2,033.19	2,136.51	2,243.52	2,354.22	2,472.30	2,597.76	2,726.91	2,863.44	3,007.35	3,1
Failed delivery fee	500.00	525.00	551.00	579.00	608.00	638.00	670.00	704.00	739.00	776.00	815.00	8
Order fee	11,110.00	11,665.50	12,243.22	12,865.38	13,509.76	14,176.36	14,887.40	15,642.88	16,420.58	17,242.72	18,109.30	19,0
Receiving fee	3,005.00	3,155.25	3,311.51	3,479.79	3,654.08	3,834.38	4,026.70	4,231.04	4,441.39	4,663.76	4,898.15	5,1
Storage fee	4,065.00	4,268.25	4,479.63	4,707.27	4,943.04	5,186.94	5,447.10	5,723.52	6,008.07	6,308.88	6,625.95	6,9
Total income	20,525.00	21,551.25	22,618.55	23,767.95	24,958.40	26,189.90	27,503.50	28,899.20	30,335.95	31,854.80	33,455.75	35,1
Rider pay per delivered order	15,000.00	15,750.00	16,530.00	17,370.00	18,240.00	19,140.00	20,100.00	21,120.00	22,170.00	23,280.00	24,450.00	25,6
Rider office commission	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	
Packaging	16,620.00	17,451.00	18,315.24	19,245.96	20,209.92	21,207.12	22,270.80	23,400.96	24,564.36	25,794.24	27,090.60	28,4
Salaries	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,000.00	27,0
Rent	76,500.00	76,500.00	60,000.00	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,500.00	76,5
Depreciation	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	
Loan interest	2,884.98	1,268.23	1,149.74	1,029.46	907.39	783.48	657.71	530.05	400.49	268.97	135.49	
5403 Fuel and transport	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,000.00	18,0
5409 Other expenses	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,200.00	1,2
5406 Utilities and internet	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,600.00	3,6

A line set by hand: rent of a later month typed on the Profit and loss tab, with "Why (kept with the lines you set by hand when you save)". A hand set line wins over the drivers and later shows "Set by hand by name: note".



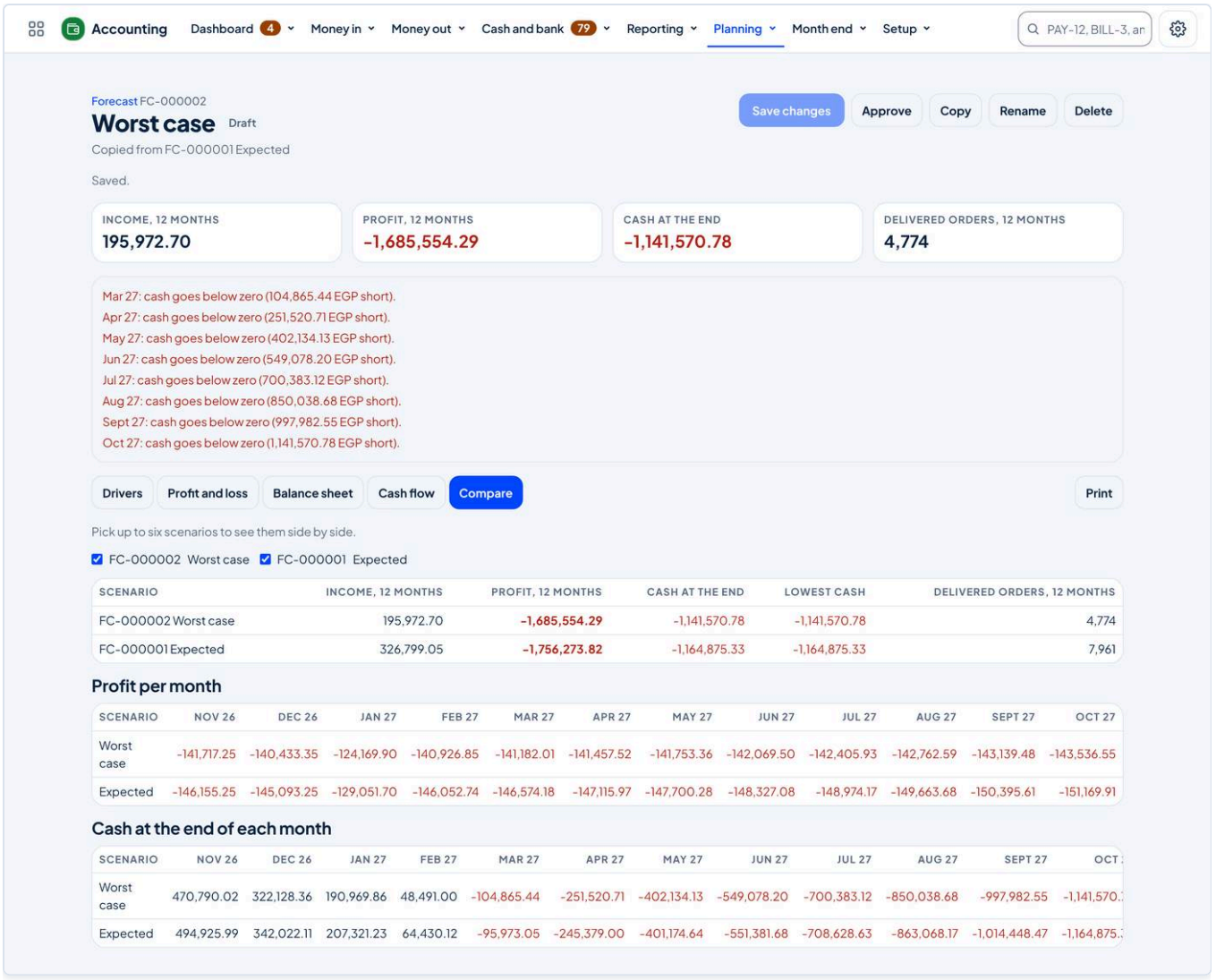
LINE	OPENING	NOV 26	DEC 26	JAN 27	FEB 27	MAR 27	APR 27	MAY 27	JUN 27	JUL 27	AUG
Banks and cash boxes	636,009.86	494,925.99	342,022.11	190,821.23	47,930.12	-112,473.05	-261,879.00	-417,674.64	-567,881.68	-725,128.63	-879,568
Cash with riders	1,192.00	1,192.00	1,192.00	1,192.00	1,192.00	1,192.00	1,192.00	1,192.00	1,192.00	1,192.00	1,192.
Equipment, after depreciation	0.00	-777.77	-1,555.54	-2,333.31	-3,111.08	-3,888.85	-4,666.62	-5,444.39	-6,222.16	-6,999.93	-7,777.
Other assets	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.
Total assets	**637,201.86**	**495,340.22**	**341,658.57**	**189,679.92**	**46,011.04**	**-115,169.90**	**-265,353.62**	**-421,927.03**	**-572,911.84**	**-730,936.56**	**-886,153.**
Owed to merchants	49,619.15	71,211.78	72,371.15	75,810.46	88,180.86	83,682.71	90,847.48	92,332.64	100,160.86	101,725.82	106,919.
VAT payable	-1,848.00	-3,696.00	-5,544.00	-7,392.00	-9,240.00	-11,088.00	-12,936.00	-14,784.00	-16,632.00	-18,480.00	-20,328.
Loans	100,000.00	84,548.98	76,649.21	68,630.95	60,492.41	52,231.80	43,847.28	35,336.99	26,699.04	17,931.53	9,032.
Other liabilities	28,837.50	28,837.50	28,837.50	28,837.50	28,837.50	28,837.50	28,837.50	28,837.50	28,837.50	28,837.50	28,837.
Total liabilities	**176,608.65**	**180,902.26**	**172,313.86**	**165,886.91**	**168,270.77**	**153,664.01**	**150,596.26**	**141,723.13**	**139,065.40**	**130,014.85**	**124,461.**
Owner capital	600,000.00	600,000.00	600,000.00	600,000.00	600,000.00	600,000.00	600,000.00	600,000.00	600,000.00	600,000.00	600,000.
Retained earnings and profit	-139,406.79	-285,562.04	-430,655.29	-576,206.99	-722,259.73	-868,833.91	-1,015,949.88	-1,163,650.16	-1,311,977.24	-1,460,951.41	-1,610,615.
Total equity	**460,593.21**	**314,437.96**	**169,344.71**	**23,793.01**	**-122,259.73**	**-268,833.91**	**-415,949.88**	**-563,650.16**	**-711,977.24**	**-860,951.41**	**-1,010,615.**
Liabilities and equity	**637,201.86**	**495,340.22**	**341,658.57**	**189,679.92**	**46,011.04**	**-115,169.90**	**-265,353.62**	**-421,927.03**	**-572,911.84**	**-730,936.56**	**-886,153.**

Balance sheet tab: banks and cash boxes, cash with riders, equipment after depreciation, owed to merchants, VAT payable, loans, capital, retained earnings, and the check row.

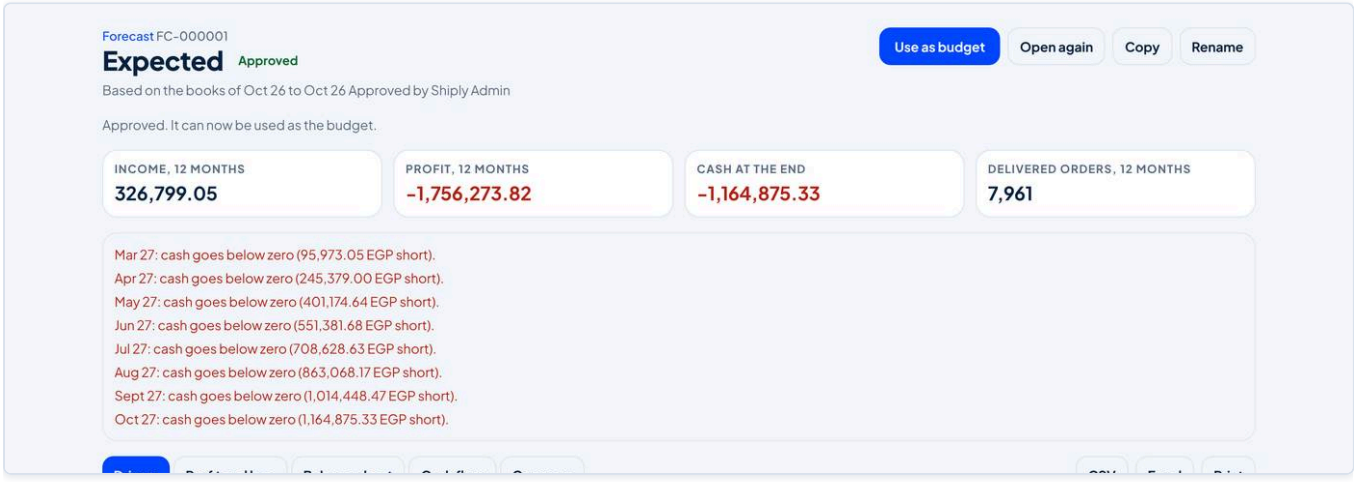


LINE	NOV 26	DEC 26	JAN 27	FEB 27	MAR 27	APR 27	MAY 27	JUN 27	JUL 27	AUG 27	SEPT 27
Profit	-146,155.25	-145,093.25	-145,551.70	-146,052.74	-146,574.18	-147,115.97	-147,700.28	-148,327.08	-148,974.17	-149,663.68	-150,395.61
Add back depreciation	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77	777.77
Change in what we owe merchants	21,592.63	1,159.37	3,439.31	12,370.40	-4,498.15	7,164.77	1,485.16	7,828.22	1,564.96	5,193.40	9,118.04
Change in VAT payable	-1,848.00	-1,848.00	-1,848.00	-1,848.00	-1,848.00	-1,848.00	-1,848.00	-1,848.00	-1,848.00	-1,848.00	-1,848.00
Cash still with riders	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
From operations	**-125,632.85**	**-145,004.11**	**-143,182.62**	**-134,752.57**	**-152,142.56**	**-141,021.43**	**-147,285.35**	**-141,569.09**	**-148,479.44**	**-145,540.51**	**-142,347.80**
Equipment bought	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
From investing	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
Loans received	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Loan repayments	-15,451.02	-7,899.77	-8,018.26	-8,138.54	-8,260.61	-8,384.52	-8,510.29	-8,637.95	-8,767.51	-8,899.03	-9,032.50
Owner capital in	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Owner drawings	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
From financing	**-15,451.02**	**-7,899.77**	**-8,018.26**	**-8,138.54**	**-8,260.61**	**-8,384.52**	**-8,510.29**	**-8,637.95**	**-8,767.51**	**-8,899.03**	**-9,032.50**
Change in cash	**-141,083.87**	**-152,903.88**	**-151,200.88**	**-142,891.11**	**-160,403.17**	**-149,405.95**	**-155,795.64**	**-150,207.04**	**-157,246.95**	**-154,439.54**	**-151,380.30**
Cash at the start	636,009.86	494,925.99	342,022.11	190,821.23	47,930.12	-112,473.05	-261,879.00	-417,674.64	-567,881.68	-725,128.63	-879,568.17
Cash at the end	**494,925.99**	**342,022.11**	**190,821.23**	**47,930.12**	**-112,473.05**	**-261,879.00**	**-417,674.64**	**-567,881.68**	**-725,128.63**	**-879,568.17**	**-1,030,948.47**

Cash flow tab with the cash warnings above: from operations (profit, add back depreciation, changes in merchant money, VAT and rider cash), investing, financing, change in cash, cash at the start and end.



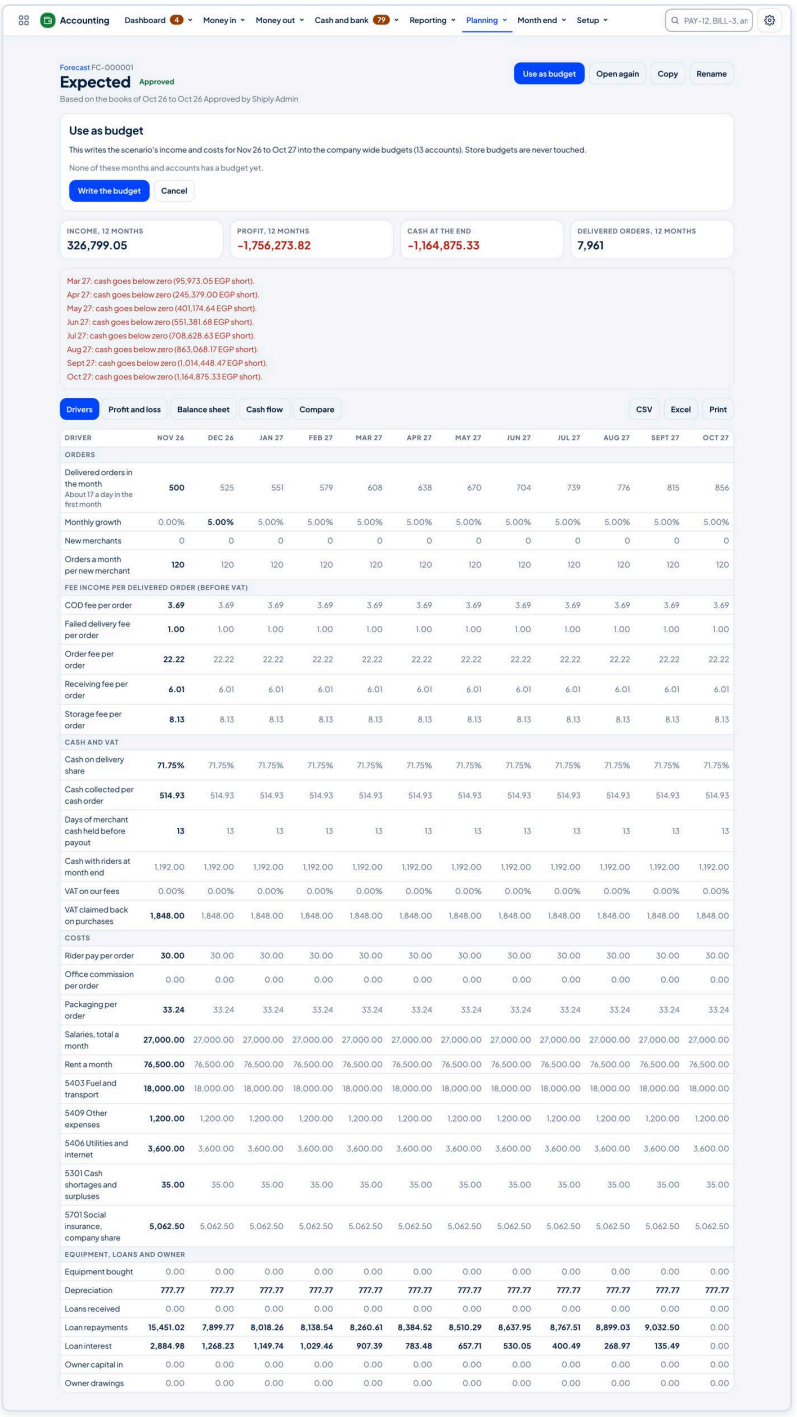
Compare tab: up to six scenarios ticked, a summary table, profit per month and cash at the end of each month side by side.



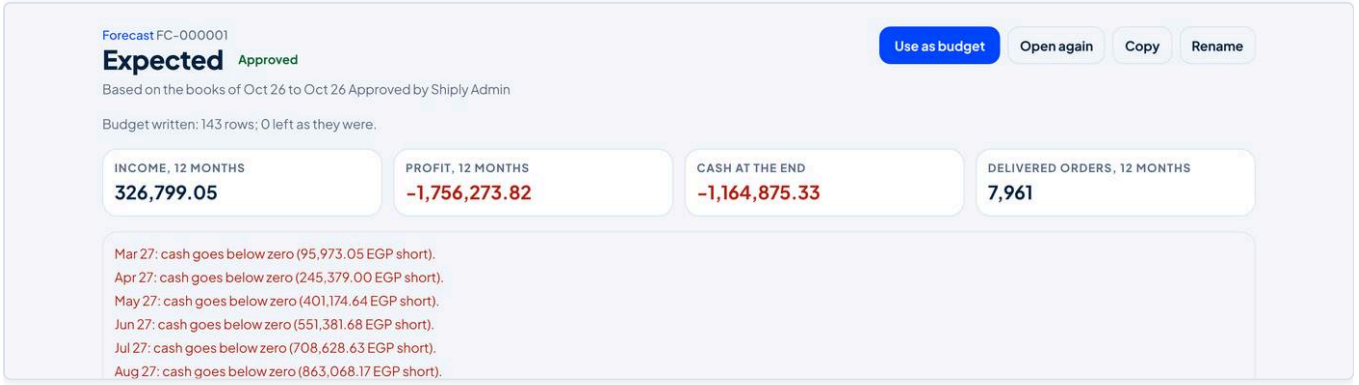
An approved scenario: green Approved pill, "Approved by", buttons Use as budget, Open again, Copy, Rename.

Use as budget

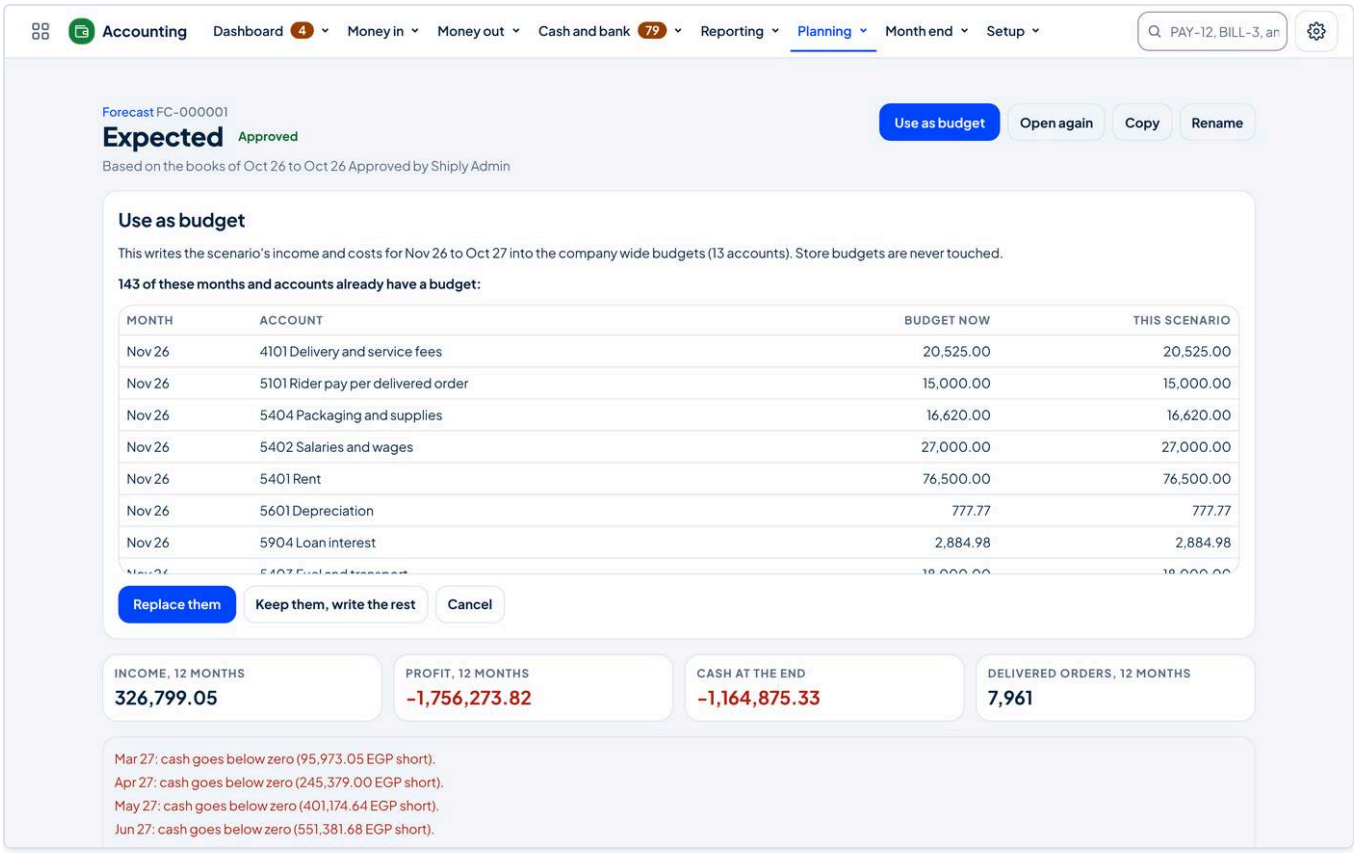
`GET :id/budget-preview` then `POST :id/to-budget`. It writes the scenario's income and costs of its 12 months into the **company wide** budgets; store budgets are never touched; amounts below zero are left out (a budget is never negative).



First time: "None of these months and accounts has a budget yet." with Write the budget and Cancel.



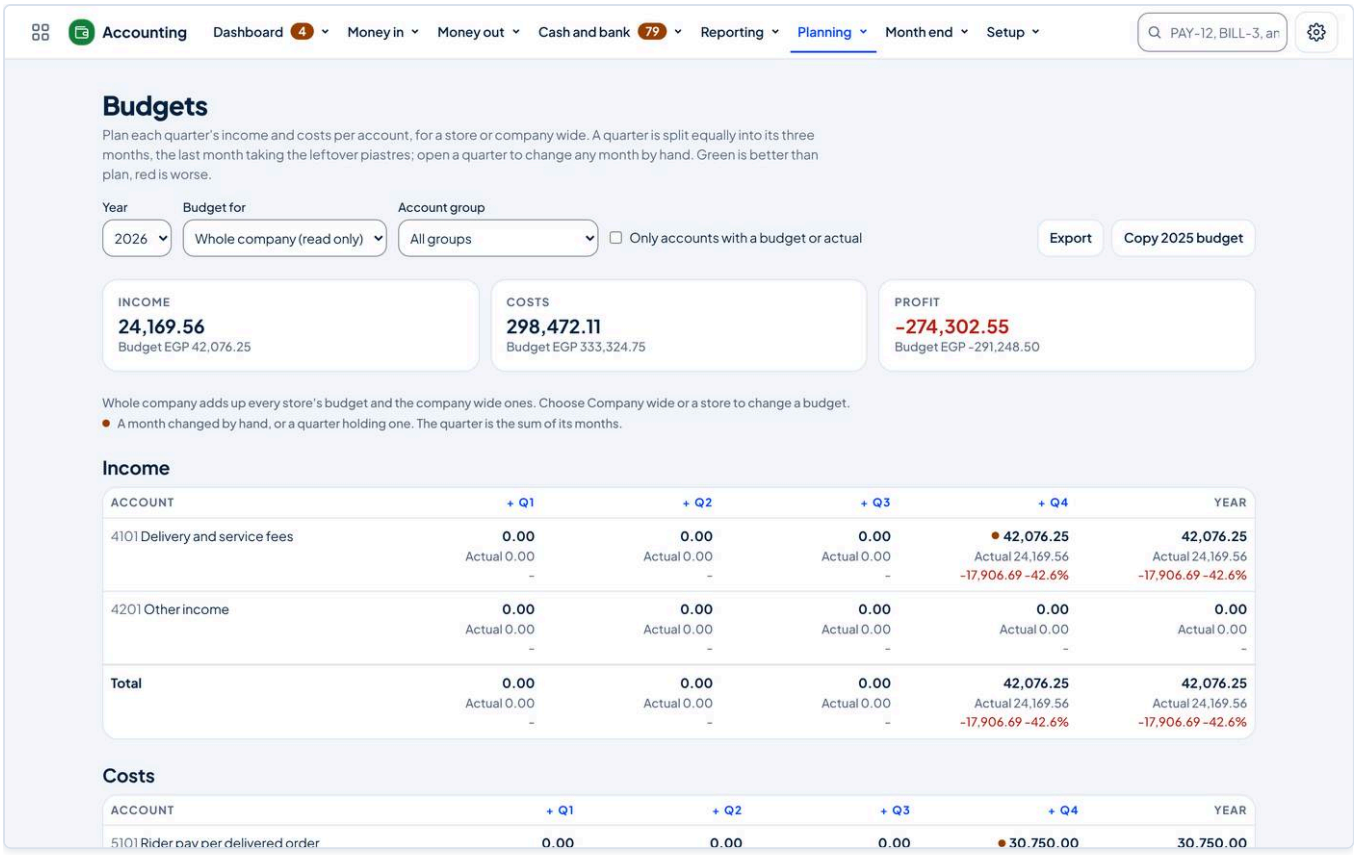
After Write the budget: "Budget written: N rows; M left as they were."



A second time: "143 of these months and accounts already have a budget:" with Month, Account, Budget now and This scenario, and the choice Replace them or Keep them, write the rest.

16.14 Budgets

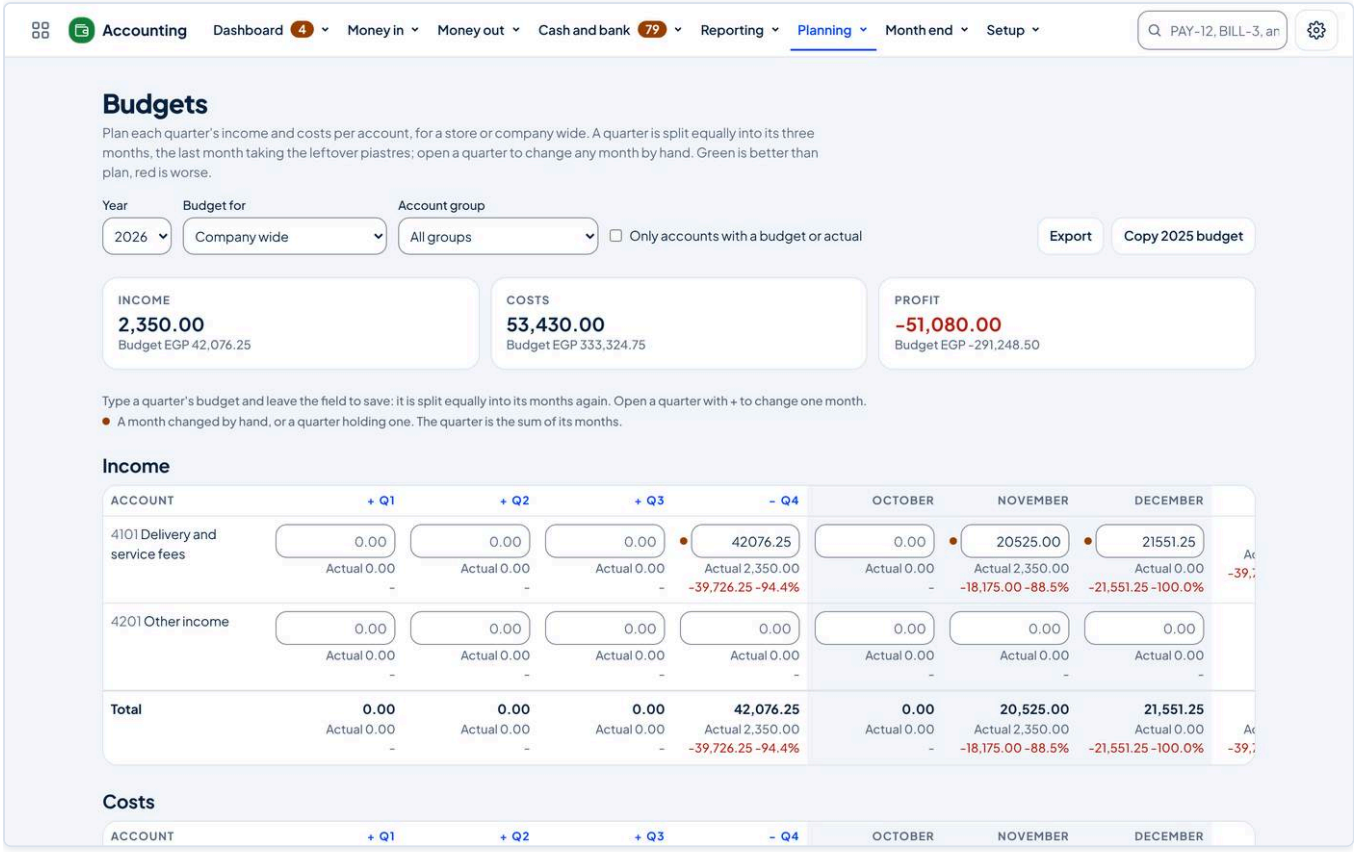
Address: `/finance/budgets?year=&scope=&group=`. **Menu:** Planning, Budgets. **Who:** Accounting view reads ("You can read budgets here; changing them needs accounting use."); Accounting use changes and copies. People limited to some stores see only their stores' budgets. Data: `GET /finance/budgets/year`, `PUT /finance/budgets/quarter`, `POST /finance/budgets/copy` (`apps/api/src/finance/budgets.controller.ts`).



Budgets for the whole company (read only): Income, Costs and Profit cards with their budget, then one row per account with Q1 to Q4 and the year (/finance/budgets).

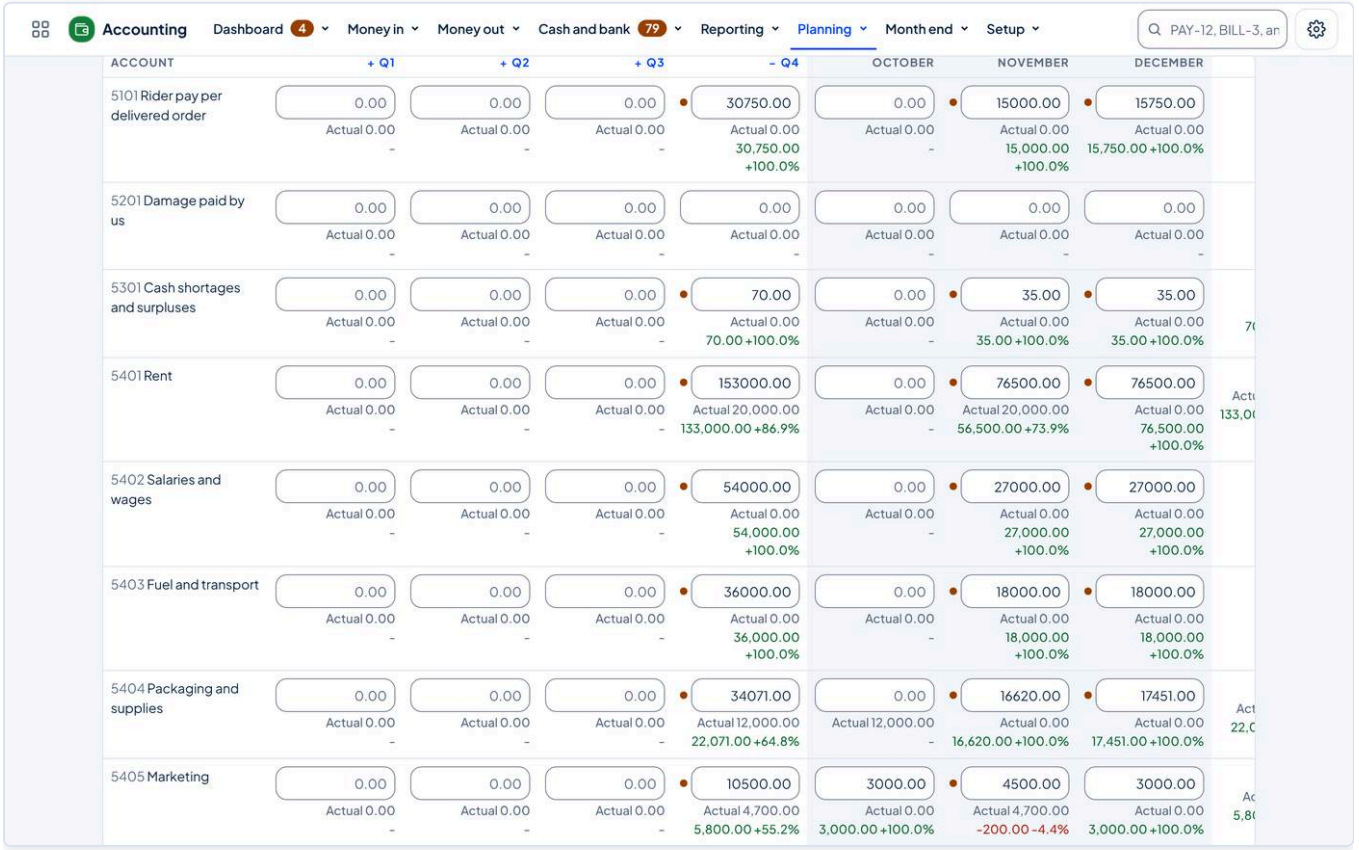
Control	What it does	Rule / limits	Result
Year	The calendar year	Address `year`	Reloads
Budget for	Whole company (read only), Company wide, or a store	Whole company adds every store's budget and the company wide ones; to change, choose Company wide or a store. Actuals follow the scope (Company wide counts only lines without a store)	Address `scope`
Account group	All groups or one group (4100 Fee income … 5900 Other costs)	Address `group`	Fewer rows
Only accounts with a budget or actual	Hides empty rows	None	None
Export	CSV or Excel of the table shown	None	A file
Copy <last year> budget	Opens the copy box: "Raise by %", Copy, Cancel	Copies last year of the same scope; the percent may be negative or empty; if the year already has a budget it asks "Replace it with the copy?" with Replace	"N months copied"
+ Q1 … + Q4 (headers)	Opens a quarter into its three months (– folds it)	Several quarters may be open	Month columns in grey appear after the quarter

Control	What it does	Rule / limits	Result
Quarter field	Type the quarter's budget and leave the field (or Enter)	Split equally into its months, the last month taking the leftover piastres; it replaces any month changed by hand	Saved at once
Month field	Change one month by hand	The quarter becomes the sum of its months	A small brown dot "Changed by hand" before the month and before its quarter
Under each figure	"Actual …" then the variance and its percent	Green is better than plan, red worse (for costs, spending less is better); a dash when there is no budget	None
Year column	Budget, actual and variance of the year	None	None

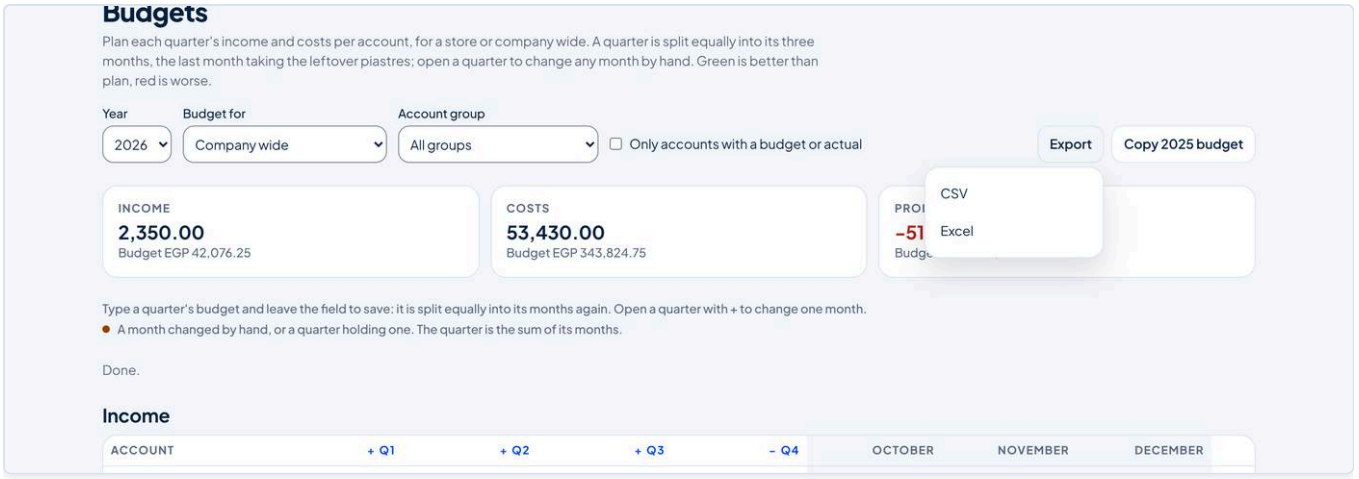


Company wide 2026 with Q4 opened into October, November and December.

Typing 9,000 into the Q4 field of 5405 Marketing: on leaving the field it is split 3,000 a month.

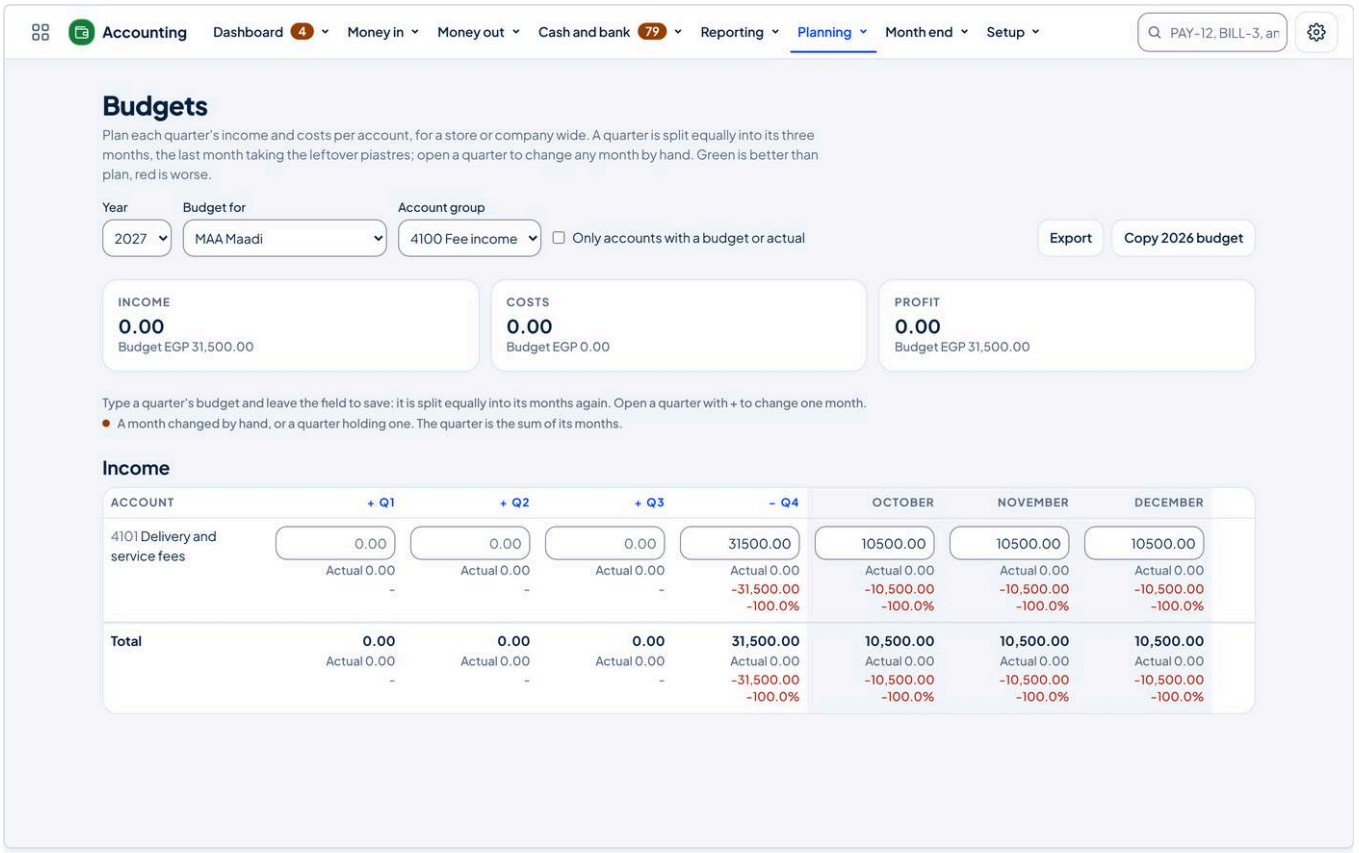


November of Marketing changed to 4,500 by hand: brown dots on November and Q4, Q4 now 10,500; months written from the forecast also carry dots.

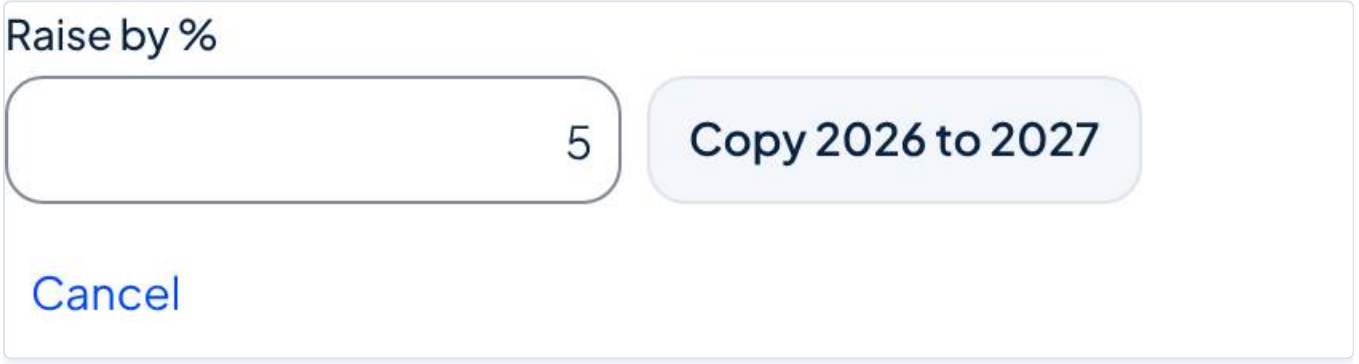


The Export menu: CSV and Excel.

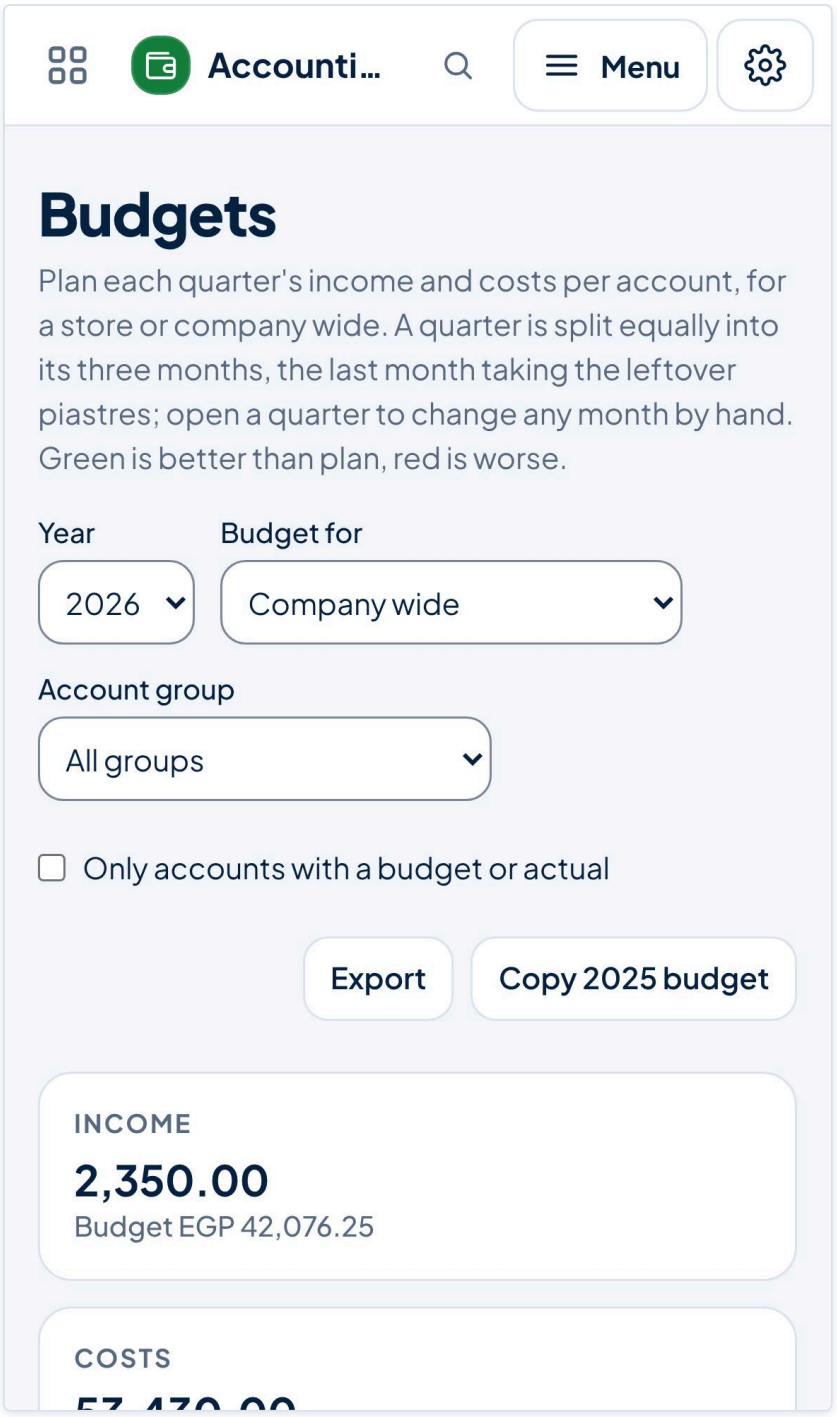
The copy box for the Maadi store: "Copies the 2026 budget of MAA to 2027. Leave the percent empty to copy it as it is.", Raise by % set to 5, Copy 2026 to 2027 and Cancel.



After the copy, 2027 for the Maadi store, Fee income group, Q4 open: 4101 is 31,500.00 (10,500.00 a month), 5% above the 30,000.00 of 2026.



Copying into a year that already has a budget here asks "2027 already has a budget here. Replace it with the copy?" with Replace (the budget is replaced only on that second click).



Budgets on a phone: the table scrolls sideways with the account column fixed.

17 Screens: Month end, Setup and common parts

This chapter shows the screens under the Accounting menus **Month end** and **Setup**, the Accounting settings page, and then the common parts that every Accounting screen is built from: the list view, the document page frame, the status bar and the status pills, smart buttons, the chatter, the attachment field and viewer, the form footer and the settings menu. It ends with how the same screens look for a person with fewer rights, on a phone, in Arabic and in the dark theme.

Every screenshot was taken on the demo copy of the system on 11 November 2026 (Cairo), signed in as the owner unless the caption says otherwise. Names of demo staff and riders are blurred in the screenshots. Numbered red markers on a screenshot are added for this guide; they are not part of the screen.

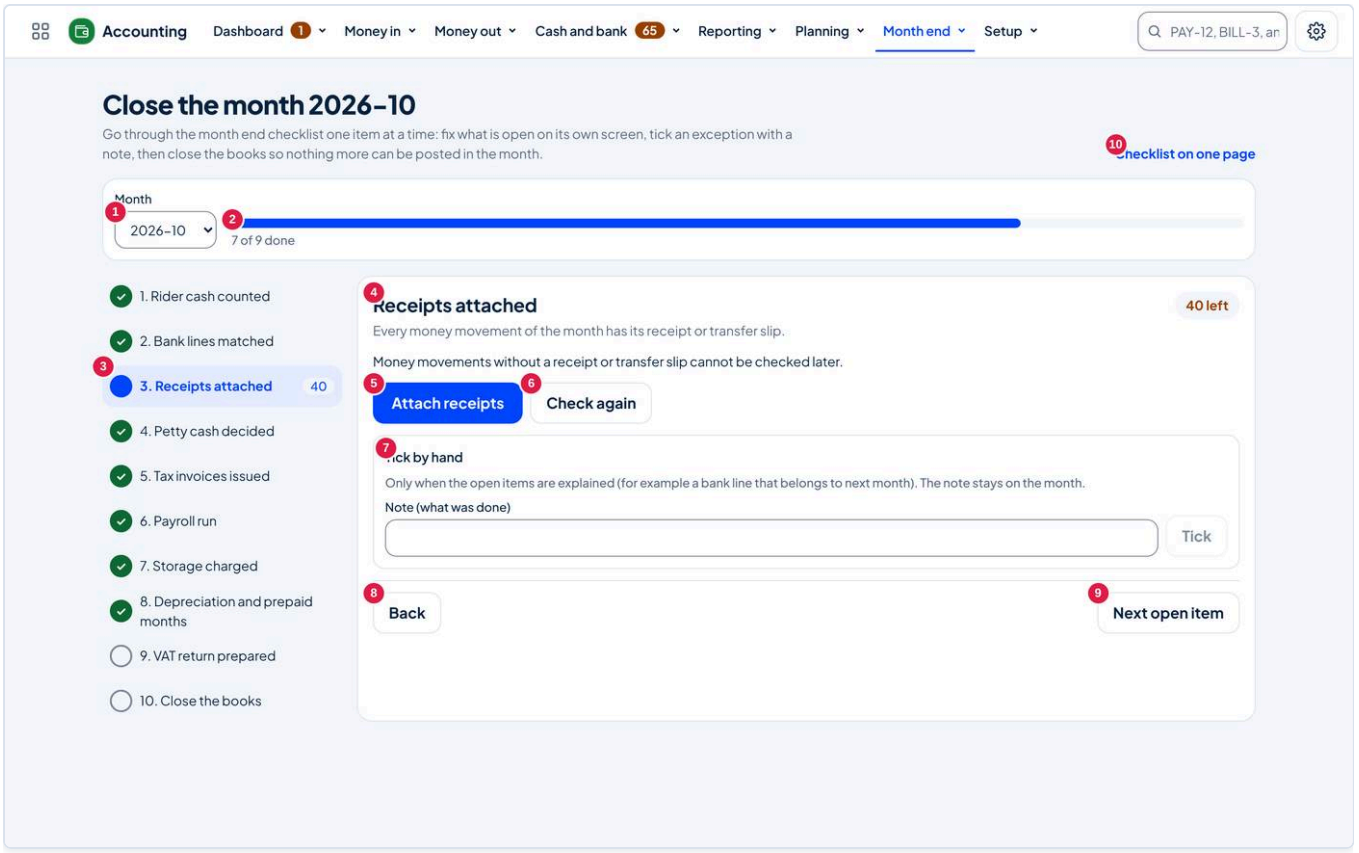
> Access words used in this chapter: **view**, **use** and **manage** are the levels a person holds on the Accounting app (`user_access.accounting`). The API decides every call: a GET needs view, any other call needs use, unless the route names another level (`@Access('accounting', 'manage')`). **Owner** means accounting manage plus settings manage (`isOwner()` in `apps/web/src/lib/apps.ts`). A page link is shown in a menu only to people who can open it.

17.1 Close the month (guided run)

Address: `/finance/month-end/run?month=YYYY-MM&step=<item>` . **Menu:** Accounting, Month end, Close the month. Also from the dashboard quick action *Close the month* and the dashboard tile *Month end for ...*.

Who: anyone with accounting view opens it. Ticking an item by hand and untick need accounting use. Closing the books needs accounting manage. Code: `apps/web/src/app/finance/month-end/run/page.tsx` , `components/finance/CloseMonthRun.tsx` , `lib/close-run.ts` . Data: `GET /finance/month-end?month=` (the live checklist, `monthEndChecklist` in `packages/db`).

The run walks the month end checklist one item at a time. Without `month` it opens last month; without `step` it opens the first item that is not done (`firstOpenStep`), or the closing step when all are done. Nothing on this screen decides a rule; each item is computed live from the books every time the page loads.



Close the month 2026-10, opened on its first open item (Receipts attached, 40 left), with numbered markers. Address /finance/month-end/run?month=2026-10.

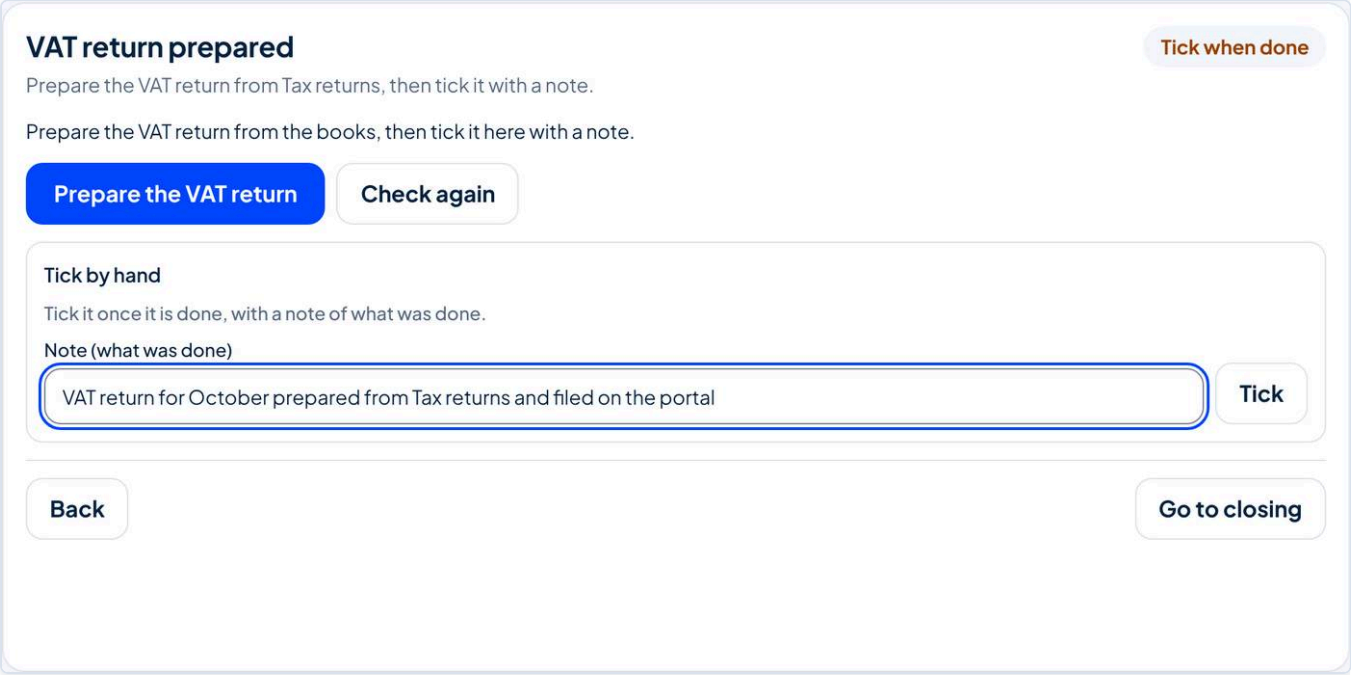
#	Control	What it does	Rule / limits	Result
1	Month	Chooses the month (this month and the 12 before it).	Changing it opens the first step, Rider cash counted.	The address changes to the new month; the checklist is read again.
2	Progress bar and "N of 9 done"	Shows how many items before closing are done.	Counts the nine items, not "Books closed". Adds "The month is not over yet." before the month ends and "The books of this month are closed." after closing.	Blue bar on a light grey track.
3	Step list (1 to 10)	Opens a step. The current step is highlighted (light blue ground, blue bold text, blue filled dot); a done step has a green dot with a white tick; an open step has an empty grey ring and, for an automatic item, its count of open records in a small pill.	Step 10 is always Close the books.	The right panel shows that step.

#	Control	What it does	Rule / limits	Result
4	Item panel: name, hint and state pill	The item's name, what it checks, and its state.	State pill: *Done* (green text on light blue), *N left* (orange text), *Tick when done* (orange, manual item), *After the month ends* (grey).	Below it a line explains why the item matters (`finance.closeRun.why.*`).
5	Fix button (for example Attach receipts)	Opens the screen that fixes the item.	Shown only while the item is not done. Targets: Count rider cash `/finance/cash` ; Match bank lines `/finance/bank?month=` ; Attach receipts `/finance/receipts` ; Decide petty cash `/finance/petty-cash?status=pending` ; Issue tax invoices `/finance/invoices` ; Run payroll `/finance/payroll` ; Check storage entries `/finance/journal` ; Check equipment and prepaid `/finance/assets` ; Prepare the VAT return `/finance/taxes?tab=returns` .	The other screen opens; come back and press Check again.
6	Check again	Reads the checklist again.	None.	The page refreshes in place (it fades while loading).
7	Tick by hand (note and Tick)	Marks an exception as done with a note.	Accounting use. Not for Books closed, not when the item is already ticked, not when an automatic item is already done. Tick stays grey until the note has 3 characters. `POST /finance/month-end/:month/marks` with `{ item, note }` ; audited.	"Ticked." and the line "Ticked by <name>: <note> · <time>"; the item counts as done.
8	Back	Opens the step before.	Hidden on step 1.	Previous step.
9	Next open item / Go to closing	Opens the next step that is not done, or closing when none is left.	Blue when the current item is done, white otherwise.	Next step.
10	Checklist on one page	Opens the plain checklist for the same month.	None.	`/finance/month-end?month=`
(after a tick)	Untick	Removes the hand tick.	Accounting use. `DELETE /finance/month-end/:month/marks/:item` ; audited.	The item goes back to its live state.

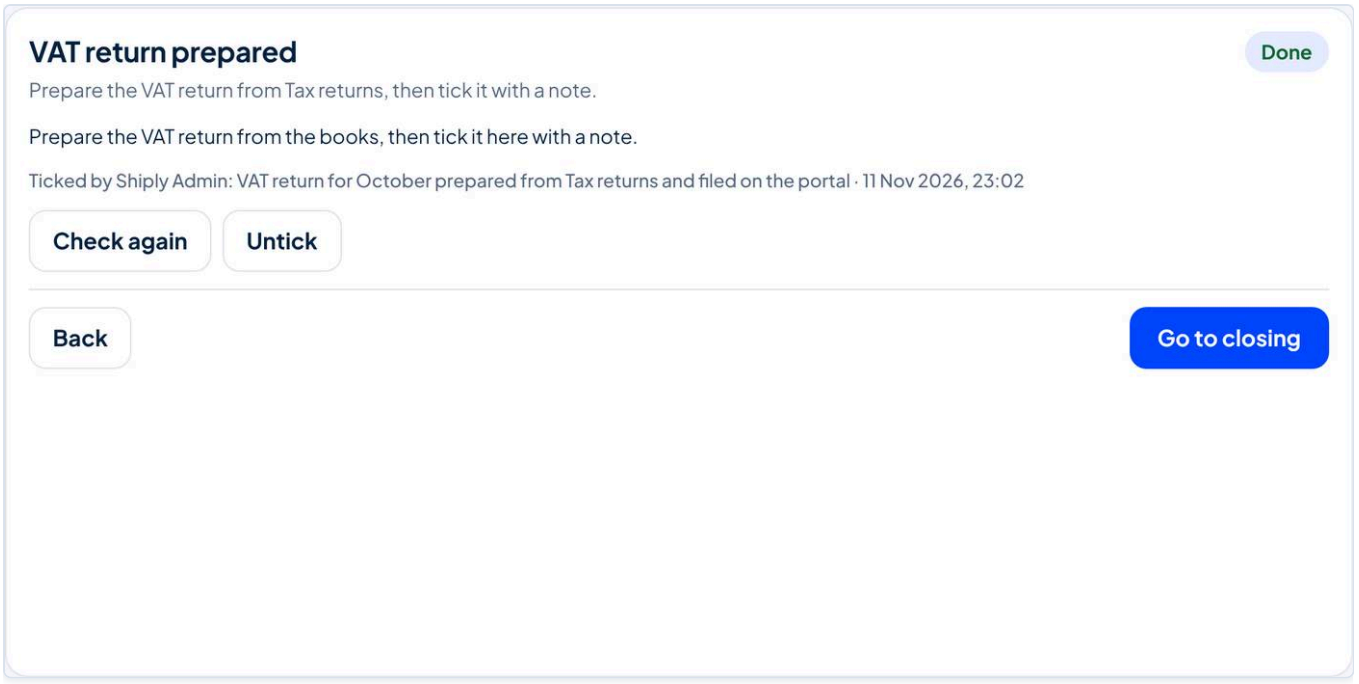
The nine items and the closing step

Step	Item (`MONTH_END_ITEMS`)	Done when	Kind
1	Rider cash counted (`cash_handovers`)	Every rider handed in the month's cash and every handover difference is settled.	Automatic

Step	Item (`MONTH_END_ITEMS`)	Done when	Kind
2	Bank lines matched (`bank_lines`)	Every bank statement line of the month is matched or set aside.	Automatic
3	Receipts attached (`receipts`)	Every money movement of the month has a receipt or transfer slip.	Automatic
4	Petty cash decided (`petty_cash`)	No petty expense of the month waits for approval.	Automatic
5	Tax invoices issued (`invoices`)	Every merchant charged fees in the month has its invoice.	Automatic, after the month ends
6	Payroll run (`payroll`)	The month's salaries are booked and paid.	Automatic, after the month ends
7	Storage charged (`storage`)	Storage was charged for every day of the month.	Automatic
8	Depreciation and prepaid months (`depreciation`)	Depreciation and prepaid months are charged for the month.	Automatic, after the month ends
9	VAT return prepared (`vat_return`)	Ticked by hand once the return is prepared.	Manual
10	Close the books	The books lock covers the month's last day.	The closing step

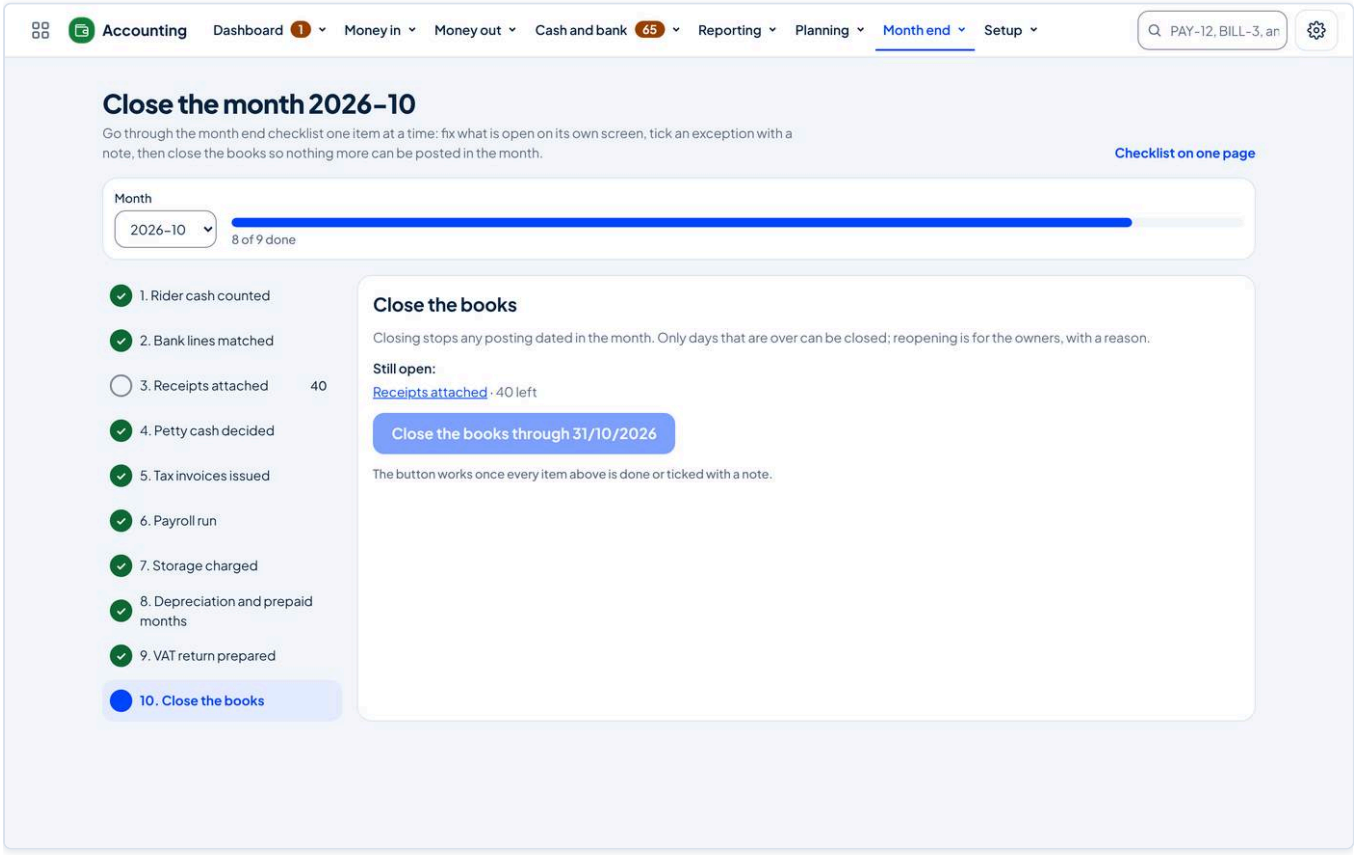


Step 9, VAT return prepared, with the hand tick note typed and the Tick button now enabled.



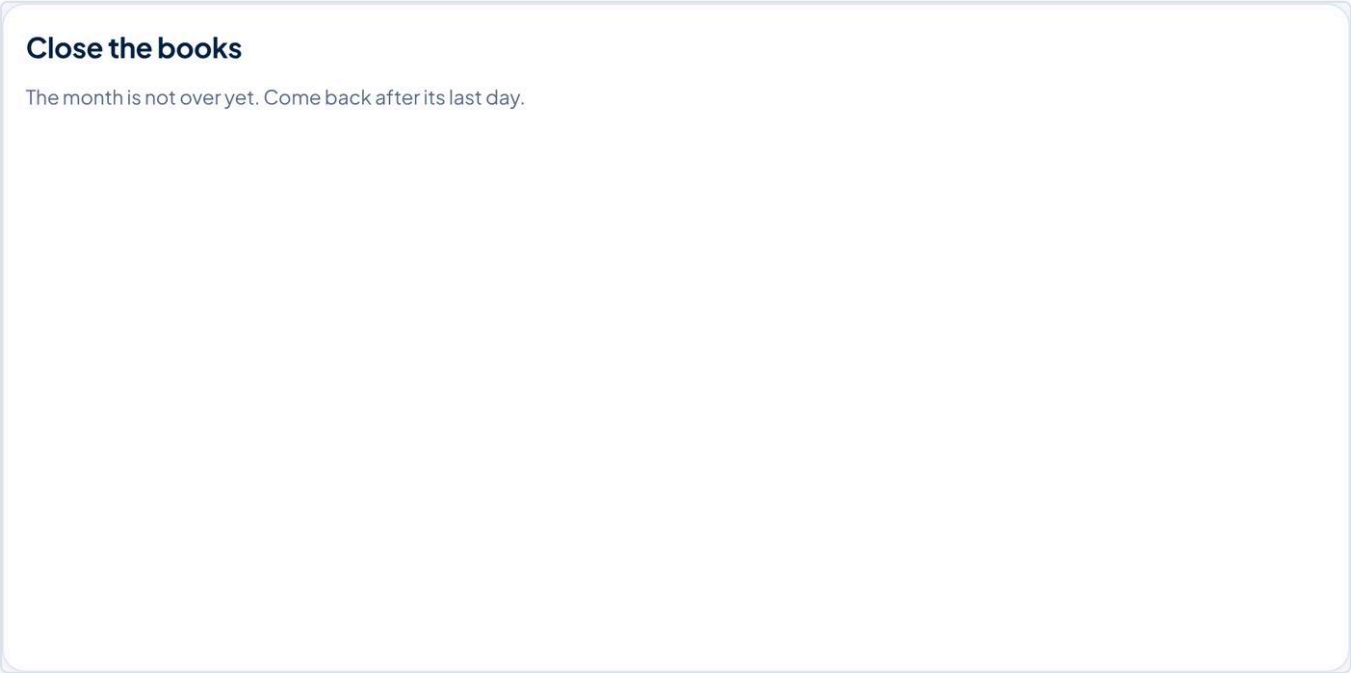
After Tick: the state pill reads Done, the line "Ticked by ... : note · time" appears, Untick replaces the tick box and the next button reads Go to closing.

The closing step



Step 10, Close the books, while one item is still open: the open item is listed as a link with its count and the close button stays pale (disabled).

Control	What it does	Rule / limits	Result
Still open list	Each item not yet done, as a link to its step, with its state.	Shown only while items are open.	Click opens the step.
Close the books through dd/mm/yyyy	Closes the books through the month's last day.	Accounting manage (others see "Closing the books needs accounting manage."). Enabled only when every item is done or ticked (`readyToClose`) and the month is over. `POST /finance/books-lock { day }` , the same lock as Close the books.	"The books are closed through dd/mm/yyyy." The panel then shows the closed line on a light blue ground.
Hint	"The button works once every item above is done or ticked with a note."	Text only.	None.



The closing step for the running month 2026–11: "The month is not over yet. Come back after its last day." and no button.

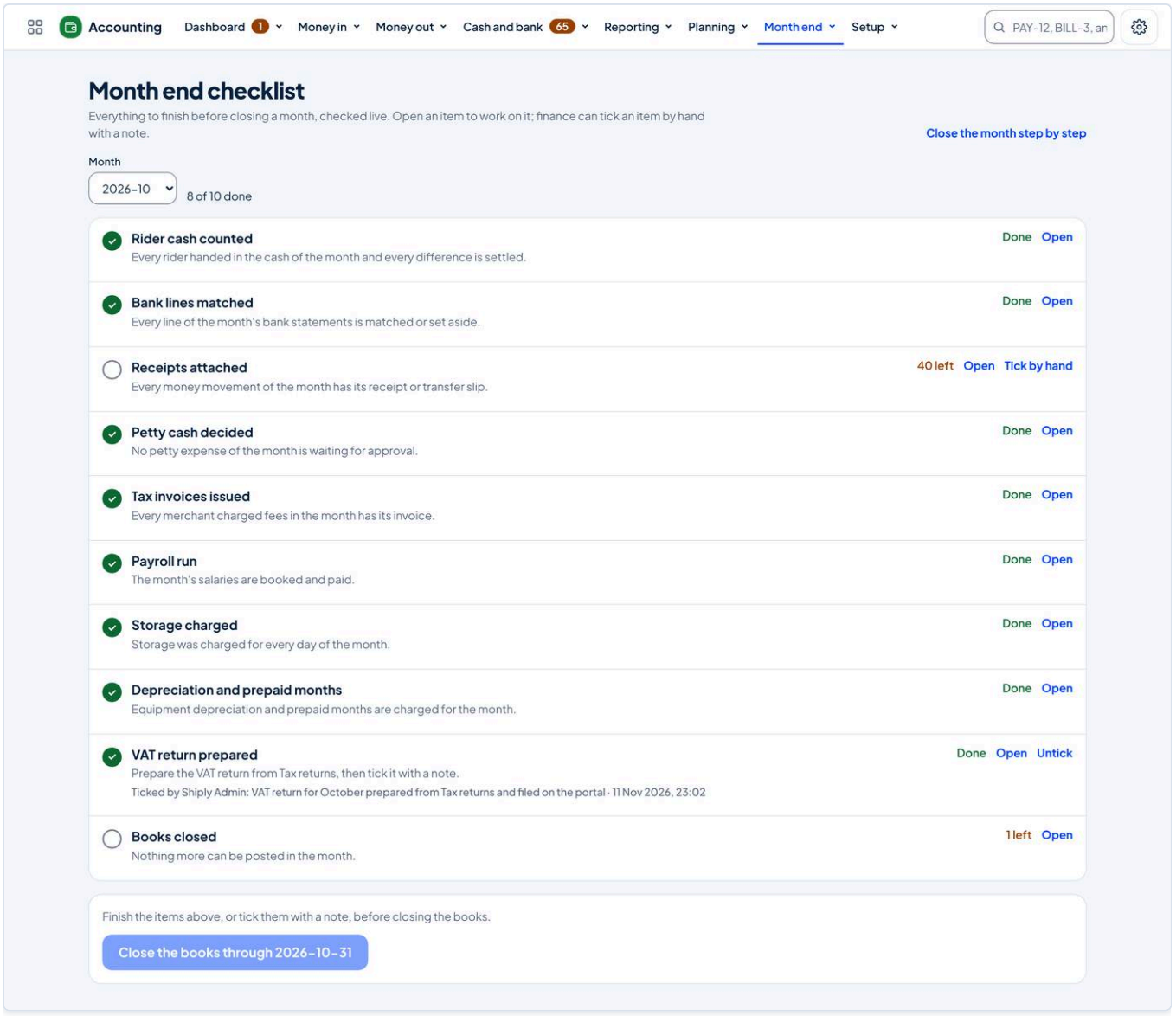
RULE

Closing stops every posting dated on or before the closed day: `post()` and a database trigger refuse such lines with `books_closed` ("This date is in a closed month. Choose a later date, or ask an admin to reopen the month."). See the chapter Taxes, reports, reconciliation and closing for the lock itself.

17.2 Month end checklist

Address: `/finance/month-end?month=YYYY-MM` . **Menu:** Accounting, Month end, Month end checklist. **Who:** accounting view opens; Tick by hand and Untick need use; the close button needs manage. Code: `app/finance/month-end/page.tsx` , `components/finance/MonthEndChecklist.tsx` .

The same live checklist as the run, on one page. Without `month` it shows last month.



Month end checklist for 2026-10: eight items done, Receipts attached with 40 left, the VAT return ticked by hand, and the close section at the bottom.

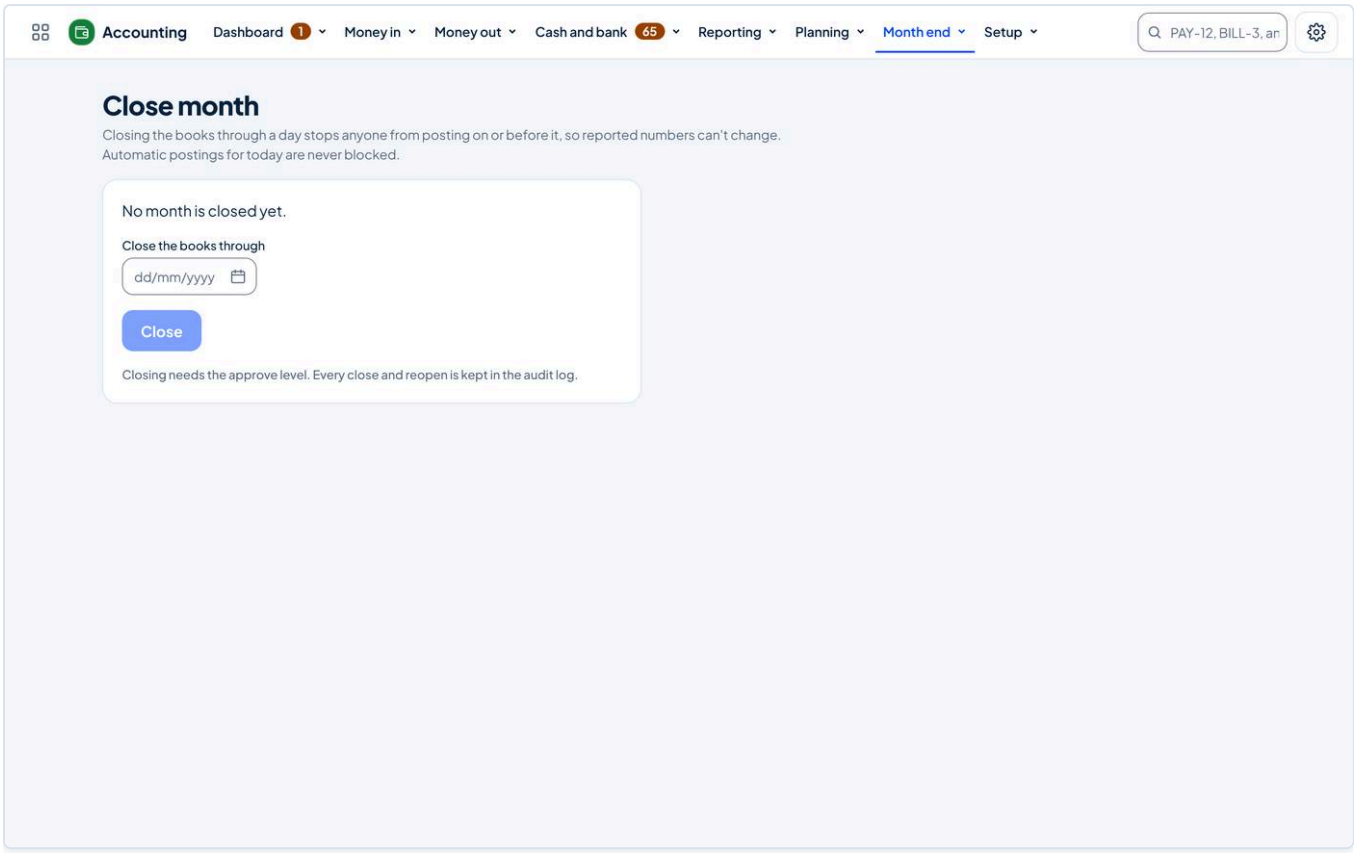
Control	What it does	Rule / limits	Result
Close the month step by step	Opens the guided run for this month.	None.	`/finance/month-end/run?month=`
Month	Chooses the month (this month and 12 before).	None.	The page reloads for that month.
"N of 10 done"	Counts all ten items, Books closed included.	Note: the guided run counts nine (without Books closed), so the two screens show different totals for the same month.	"The month is not over yet." is added before the month ends.
Item row: dot, name, hint	Green dot with a white tick when done, an empty grey ring when not.	None.	A ticked item also shows "Ticked by <name>: <note> · time".

Control	What it does	Rule / limits	Result
State text	*Done* (green), *N left* (orange), *Tick when done* (orange), *After the month ends* (grey).	Computed live.	None.
Open	Opens the screen where the item is worked on.	Links: the same targets as the run, except VAT return opens `/finance/tax-returns` (which redirects to the Tax returns tab) and Books closed opens `/finance/close`.	Another screen.
Tick by hand	Opens a note field and a Tick button under the row.	Accounting use; same limits as in the run (note of 3 characters or more, never Books closed).	The item is ticked with the note.
Untick	Removes a hand tick.	Accounting use.	Back to the live state.
Close section	"Every item is done. Close the books to stop any change in this month." or "Finish the items above, or tick them with a note, before closing the books."	Button *Close the books through YYYY-MM-DD*: accounting manage, enabled only when ready.	"The books are closed through YYYY-MM-DD."

Tick by hand opened on Receipts attached, with a note typed.

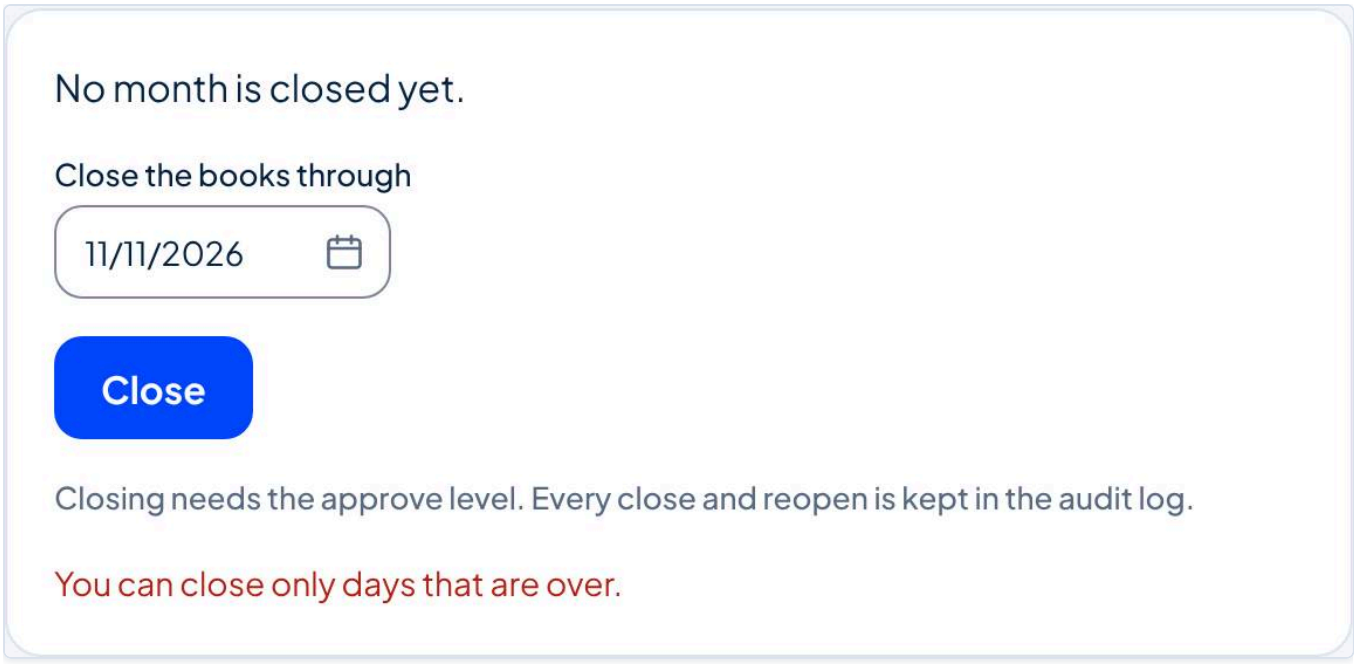
17.3 Close the books

Address: `/finance/close`. **Menu:** Accounting, Month end, Close the books (page title "Close month"). **Who:** the page opens with accounting view; closing needs accounting manage; moving the date back (reopening) also needs settings manage and a reason. Code: `app/finance/close/page.tsx`, `components/finance/CloseBooks.tsx`; API `POST /finance/books-lock` (`closeBooks`).

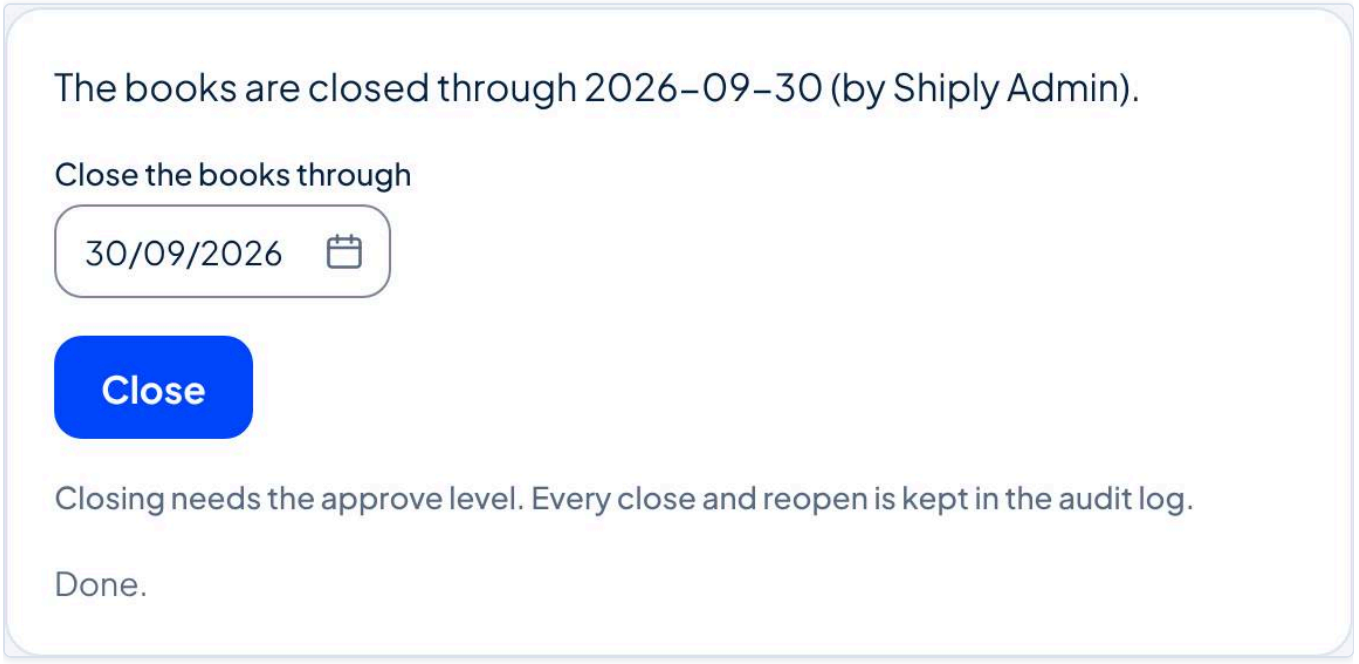


Close the books before anything was closed: "No month is closed yet."

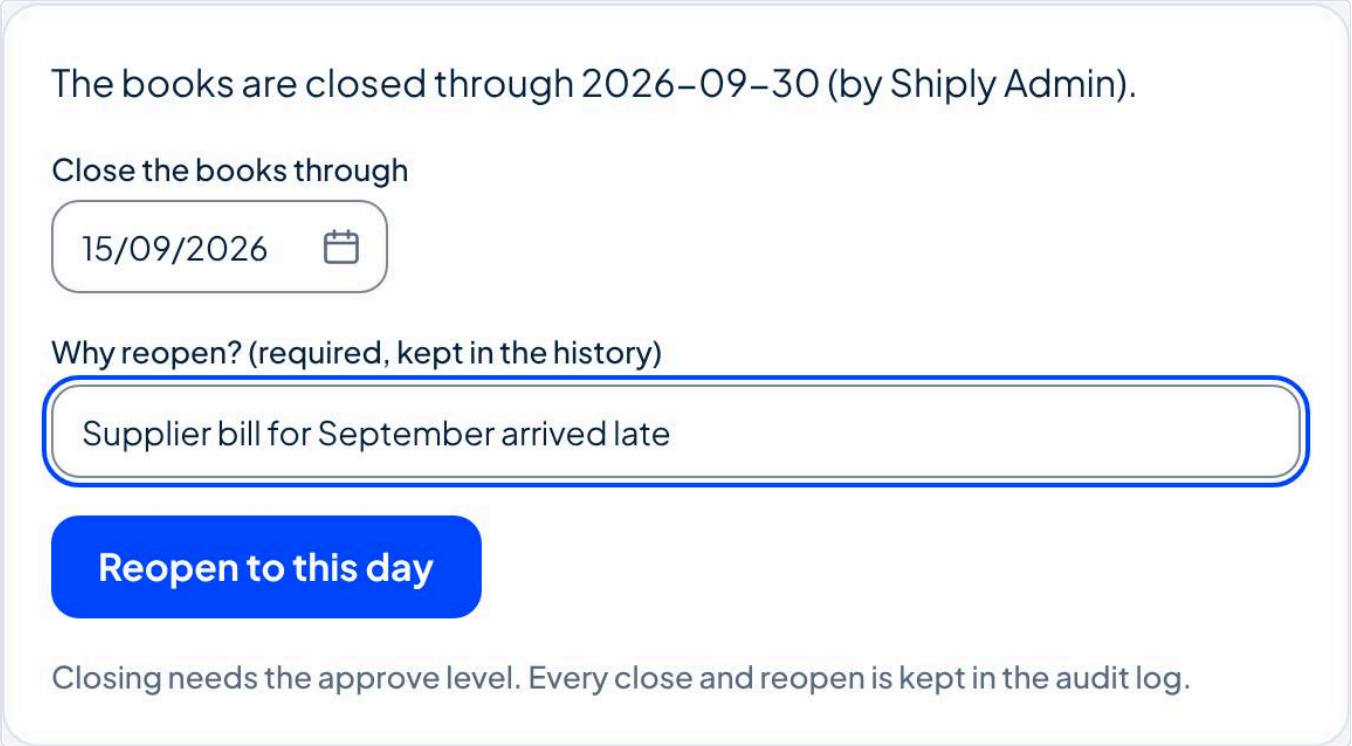
Control	What it does	Rule / limits	Result
Status line	"The books are closed through YYYY-MM-DD (by <name>)." or "No month is closed yet."	From `GET /finance/controls`.	None.
Close the books through (date box)	The last day to close. Typed day first (dd/mm/yyyy) or picked with the calendar button.	The calendar stops at yesterday; a typed later day is refused by the API with `day_not_over`.	None until the button is pressed.
Why reopen? (required, kept in the history)	Appears only when the chosen day is before the closed day (reopening).	Only for people with settings manage; others see "Moving the date back reopens closed days: only someone who manages settings can do it." The button stays disabled until the reason has 5 characters.	The reason is stored in the audit log.
Close / Reopen to this day	Sends the lock.	Accounting manage. Only days that are over. Reopen: settings manage and a reason (`reason_required` otherwise).	"Done." and the status line shows the new day; every close and reopen is audited.



Error: today typed as the day to close. The API answers day_not_over, shown in red as "You can close only days that are over."



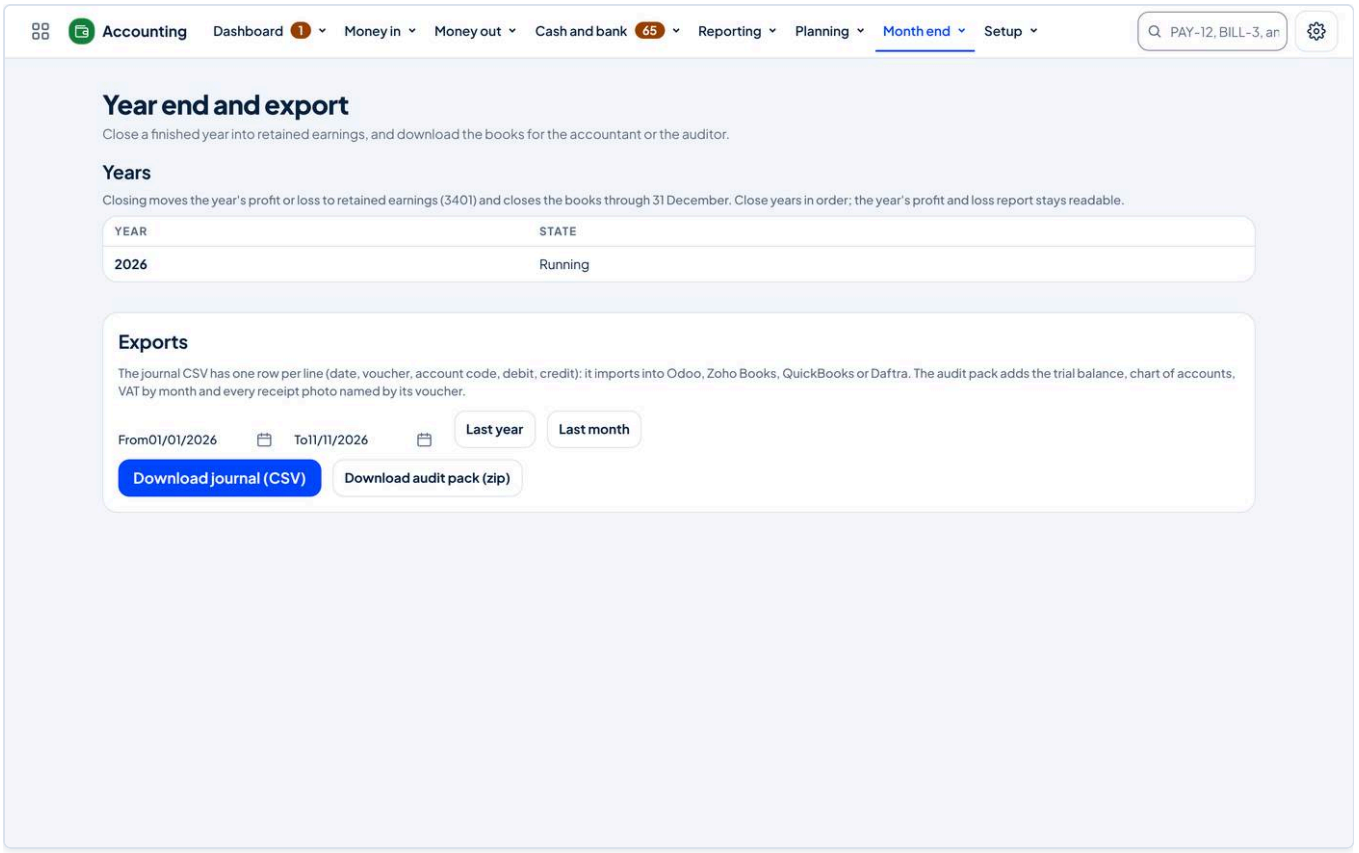
After closing through 30/09/2026: "Done." and the status line names the day and who closed it.



An earlier day chosen while September is closed: the reason field appears and the button becomes Reopen to this day.

17.4 Year end and export

Address: `/finance/year-end`. **Menu:** Accounting, Month end, Year end. **Who:** accounting view opens and downloads; Close year needs accounting manage (`POST /finance/year-close`). Code: `YearEnd` in `components/finance/PeriodReports.tsx`.



Year end and export: the only year with entries, 2026, is Running, so no Close year button shows. Below, the exports with the day range.

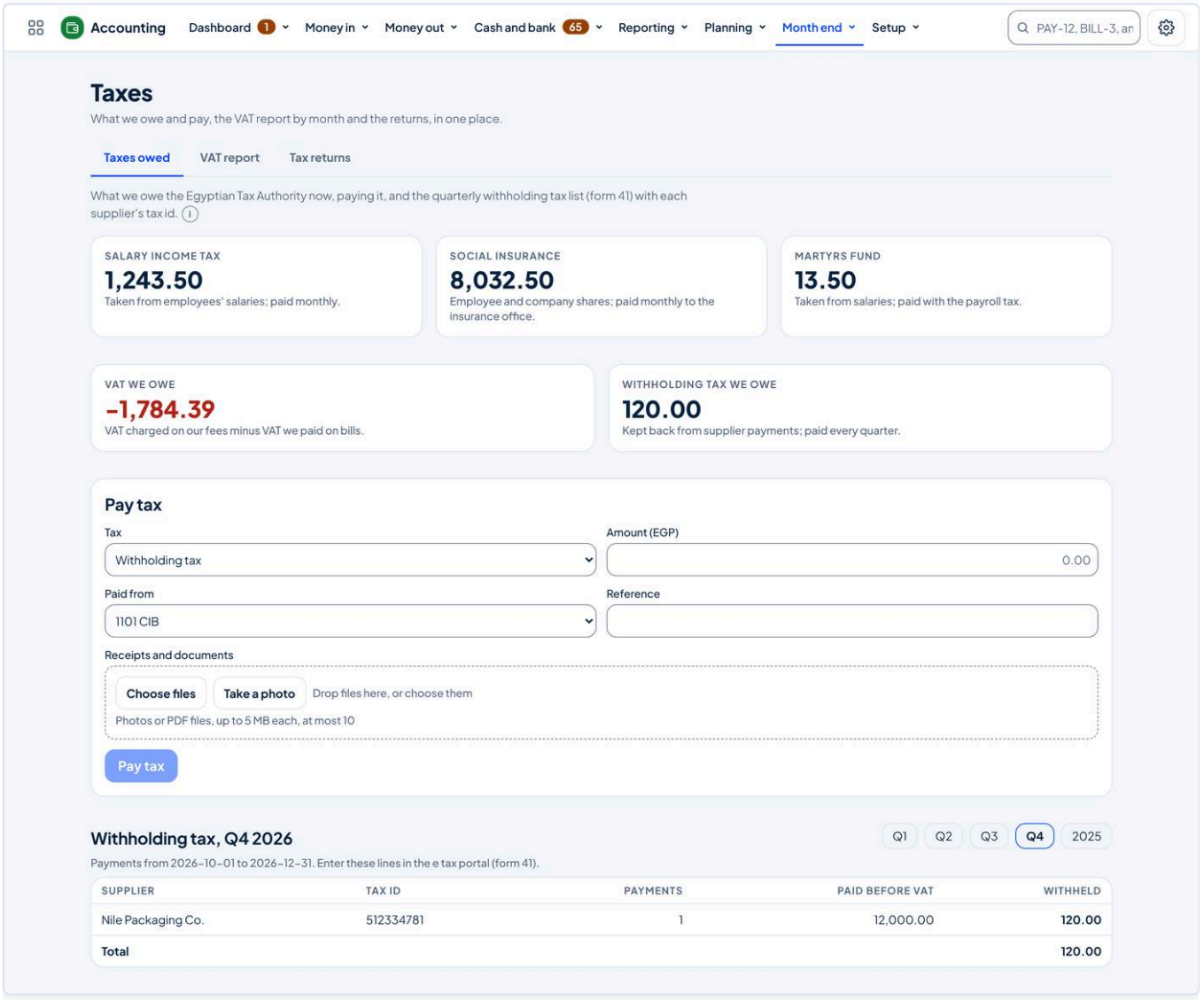
Control	What it does	Rule / limits	Result
Years table: Year, State	Every year that has entries. State: *Running* (not over), *Over, not closed*, *Closed (JV-xxxxxx)*.	From `GET /finance/year-close`.	"No entries yet." when empty.
Close year	Shown on a year that is over and not closed. First press shows *Yes, close YYYY* and *Cancel*.	Accounting manage. Years in order (`earlier_year_open`), once (`year_already_closed`), only when over (`year_not_over`). Book the company income tax first (Taxes, Tax returns).	"Closed with JV-xxxxxx; profit EGP n moved to retained earnings." Income and expense balances move to 3401 at the year's last moment and the books close through 31 December.
From, To	The day range of the exports (default 1 January to today).	Days not later than today.	None.
Last year, Last month	Set the range to last calendar year or last month.	None.	The two boxes change.

Control	What it does	Rule / limits	Result
Download journal (CSV)	One row per ledger line: date, voucher, account code, debit, credit.	Accounting view. `/api/finance/export/journal.csv?from=&to=` .	A CSV that imports into common accounting programs.
Download audit pack (zip)	Journal, trial balance, chart of accounts, VAT by month, every receipt photo named by voucher.	`/api/finance/export/audit-pack.zip?from=&to=` .	A zip file.

17.5 Taxes

Address: `/finance/taxes?tab=owed|vat|returns` . The old addresses `/finance/vat` and `/finance/tax-returns` open the matching tab. **Menu:** Accounting, Month end, Taxes owed. **Who:** accounting view reads all tabs; Pay tax needs use (and the second person rule from the approval limit); Book the tax needs manage; the salary lists need the salaries switch. Code: `app/finance/taxes/page.tsx` , `ReturnsTab.tsx` , `Taxes` in `components/finance/Payables.tsx` , `CorporateTaxForm.tsx` .

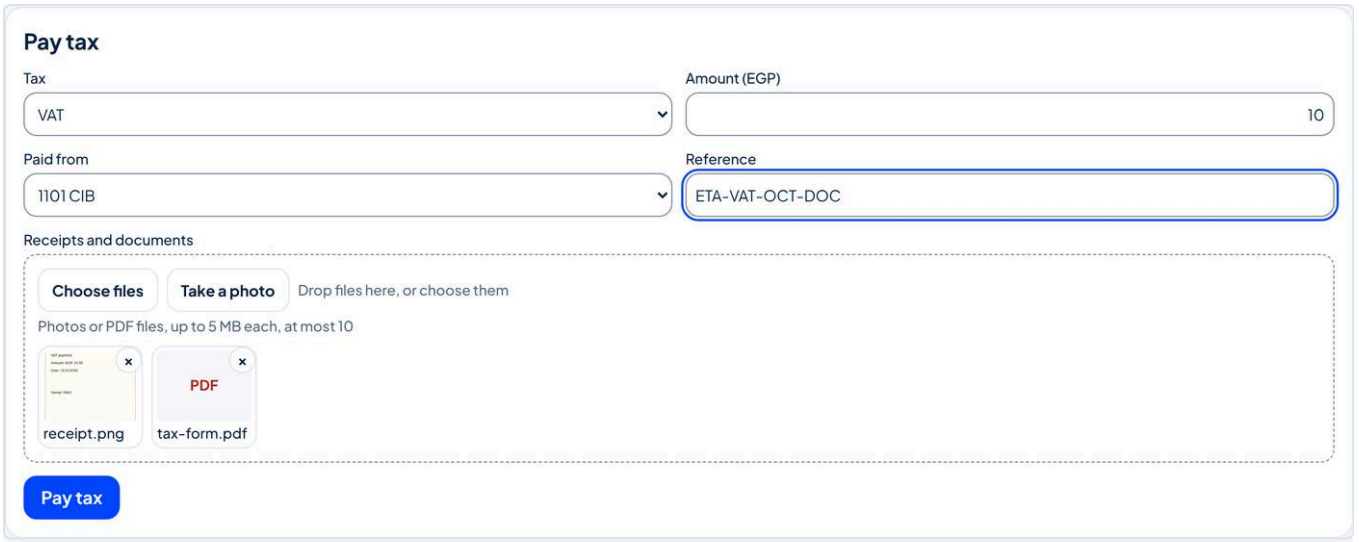
Tab Taxes owed



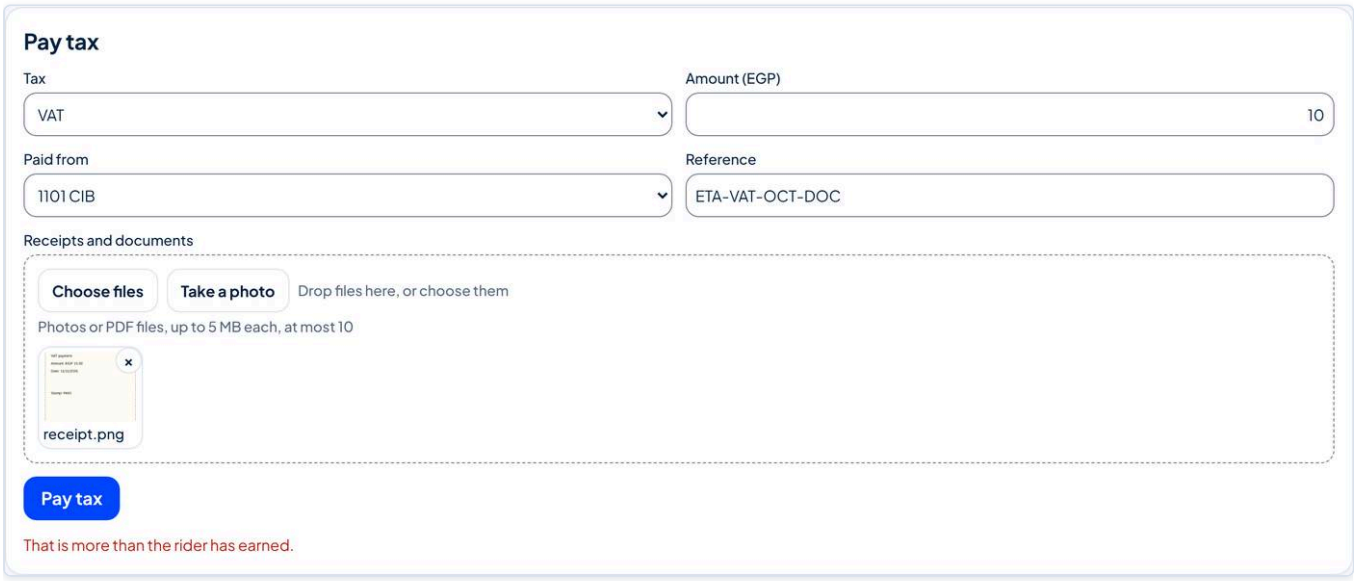
Taxes owed: five balance cards, the Pay tax form and the withholding list of the current quarter. VAT we owe is negative (a credit), shown in red.

Control	What it does	Rule / limits	Result
Tabs Taxes owed, VAT report, Tax returns	Switch the tab; the current one is blue with a blue underline.	Kept in the address (`?tab=`).	None.
(i) after the intro	InfoTip explaining withholding tax.	None.	A small popup.
Cards Salary income tax, Social insurance, Martyrs fund	What we owe now on 2303, 2304 and 2305, with how it is paid.	Balances summed from ledger lines.	None.
Cards VAT we owe, Withholding tax we owe	Balance of 2301 and 2302.	A negative amount is shown in red: VAT claimed back exceeds VAT charged.	None.

Control	What it does	Rule / limits	Result
Pay tax: Tax	Withholding tax, VAT, Salary income tax, Social insurance, Martyrs fund, Company income tax.	None.	None.
Amount (EGP)	Amount paid.	Pounds with up to two decimals; converted to piastres without floats.	None.
Paid from	An open bank or cash box (default the first, 1101 CIB).	None.	None.
Reference	The payment reference (for example the e tax portal number).	At least 2 characters.	None.
Receipts and documents	The attachment field (see Common parts).	The button waits while files upload.	Files are linked to the posted entry.
Pay tax	Posts the payment: Dr the tax account, Cr the bank or box.	Accounting use. Never more than owed (`more_than_owed`), never more than the bank holds (`not_enough_money`). From finance.approval_threshold_piastres it waits in Approvals for a second person. Idempotency key per press.	"Done." (posted) or "Sent for approval." The form empties.
Withholding tax, Q n YYYY: Q1 to Q4, previous year	Chooses the quarter of the withholding list.	The current quarter is outlined in blue.	Address `?year=&quarter=` .
Columns Supplier, Tax id, Payments, Paid before VAT, Withheld; Total	Withholding kept back per supplier in the quarter.	A supplier without a tax id shows "No tax id" in red, and a red line counts them above the table.	"No withholding tax in this quarter." when empty.

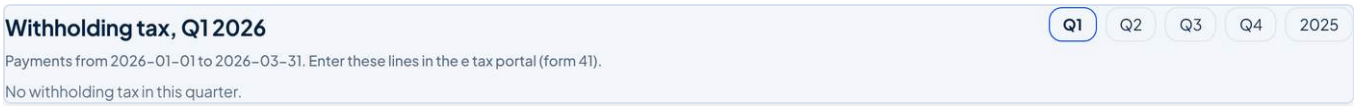


Pay tax filled in: VAT, 10 EGP from 1101 CIB, a reference and two files chosen.



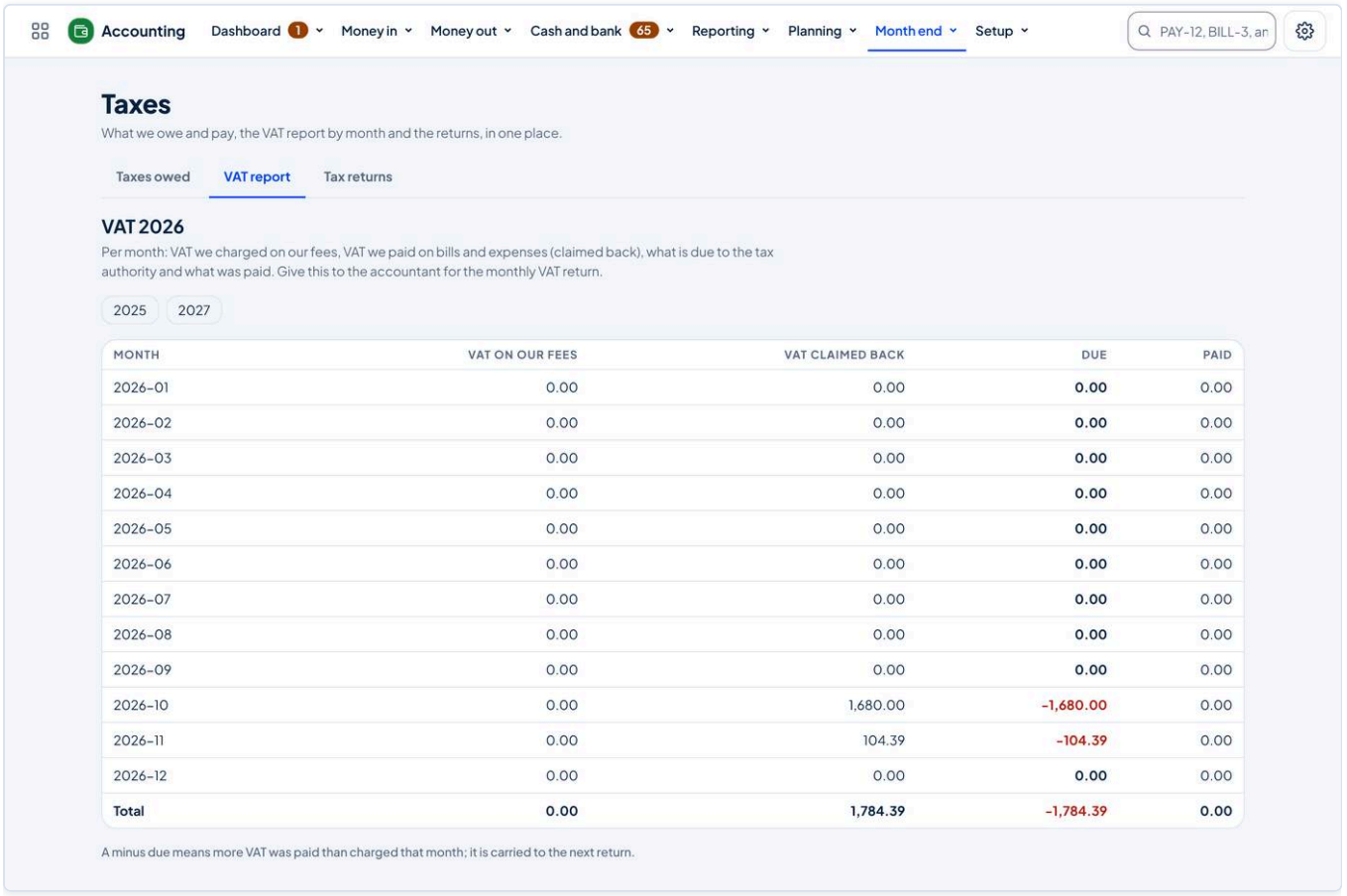
Error: paying VAT while VAT we owe is a credit. The API refuses with more_than_owed and the form shows its refusal message.

Result: 10 EGP of withholding tax paid with a receipt photo. Below the approval limit it posts at once ("Done."); the entry JV-002555 is shown under Common parts, document page.



Empty state of the withholding list (Q1 2026): "No withholding tax in this quarter."

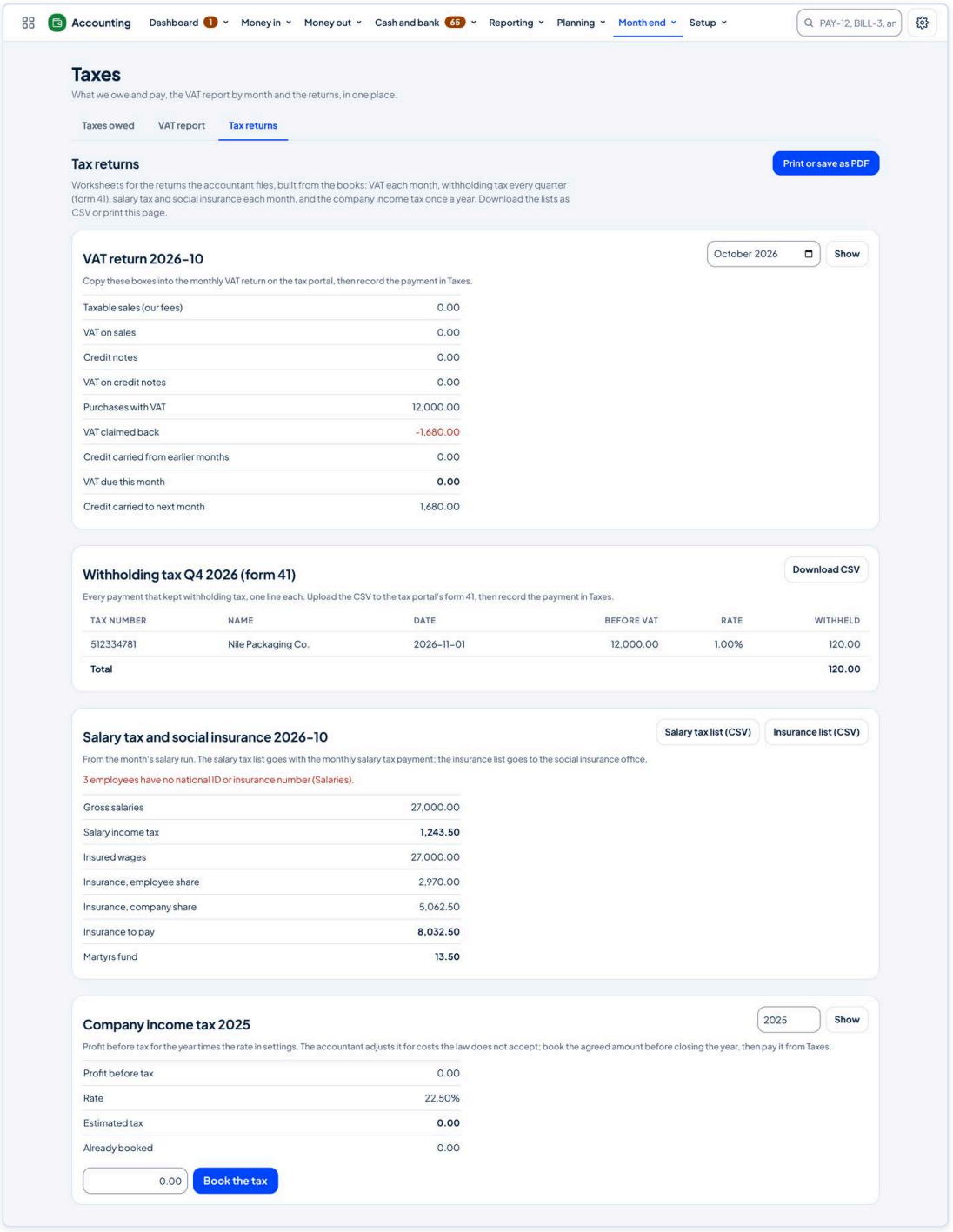
Tab VAT report



VAT report 2026 by month: VAT on our fees, VAT claimed back, Due and Paid.

Control	What it does	Rule / limits	Result
Year buttons (year before, year after)	Open another year.	`?tab=vat&year=`	None.
Columns Month, VAT on our fees, VAT claimed back, Due (bold), Paid; Total row	Monthly VAT from the 2301 lines, by entry kind (see the chapter Taxes, reports, reconciliation and closing).	A minus Due is a credit carried to the next return.	None.

Tab Tax returns



Tax returns: the VAT return of 2026-10, withholding Q4 2026 (form 41), salary tax and social insurance 2026-10, and company income tax 2025.

Control	What it does	Rule / limits	Result
Print or save as PDF	Prints the tab; menus and buttons are hidden in print.	None.	The browser print dialog.

Control	What it does	Rule / limits	Result
VAT return: month box and Show	Chooses the month (default last month).	`?tab=returns&month=`	Boxes: Taxable sales, VAT on sales, (debit notes when any), Credit notes, VAT on credit notes, Purchases with VAT, VAT claimed back, Credit carried from earlier months, VAT due this month (bold), Credit carried to next month. Negative boxes show in red.
Withholding tax Q n (form 41): Download CSV	One line per payment that kept withholding.	Quarter from the address (default current). Columns Tax number (Missing in red), Name, Date, Before VAT, Rate, Withheld; Total.	CSV for the portal's form 41. "Nothing in this period." when empty.
Salary tax and social insurance: Salary tax list (CSV), Insurance list (CSV)	Totals of the month's salary run.	Needs the salaries switch. A red line counts employees without national ID or insurance number.	Gross salaries, Salary income tax, Insured wages, Insurance employee and company share, Insurance to pay, Martyrs fund. "No salary run for this month." when none.
Company income tax: year box and Show	Profit before tax × finance.corporate_tax_bp.	Default last year.	Profit before tax, Rate, Estimated tax, Already booked.
Amount to book and Book the tax	Books the amount agreed with the accountant at the year's last moment (5801 against 2306); booking again books only the difference.	Accounting manage; before the year is closed.	"Done."

17.6 Chart of accounts

Address: `/finance/accounts` (`?all=true` shows closed accounts). **Menu:** Accounting, Setup, Chart of accounts. **Who:** accounting view reads; opening, editing, closing and reopening an account need accounting use (`POST /finance/accounts`, `PATCH /finance/accounts/:id`; the buttons are shown to everyone who can open the page and the API refuses the rest). Code: `components/finance/AccountsManager.tsx`.

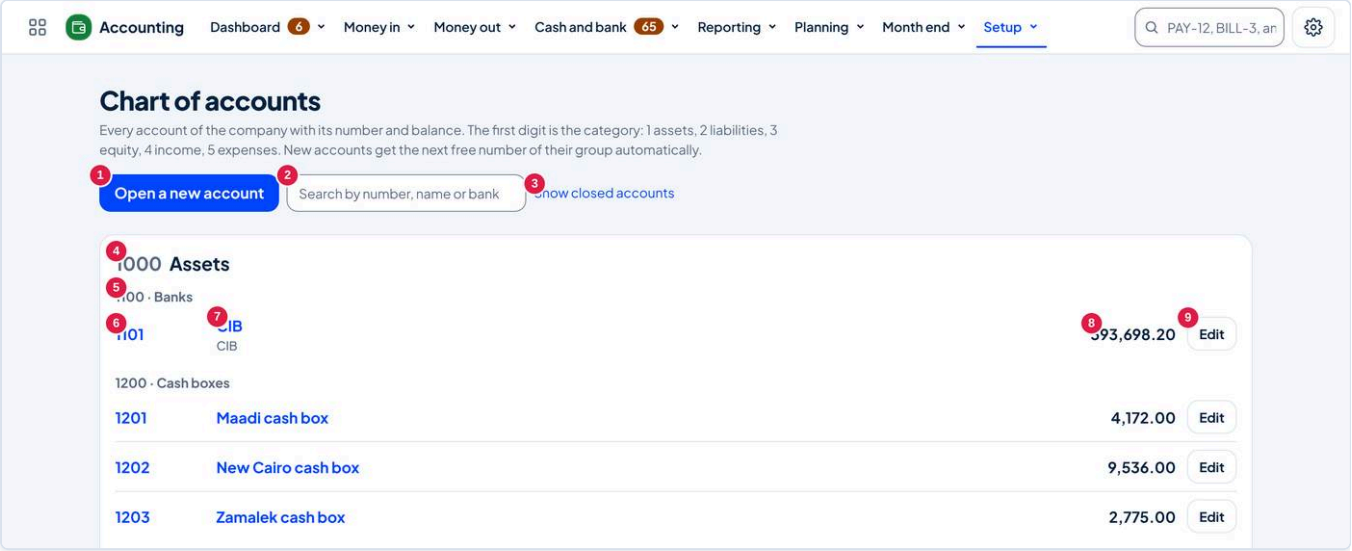


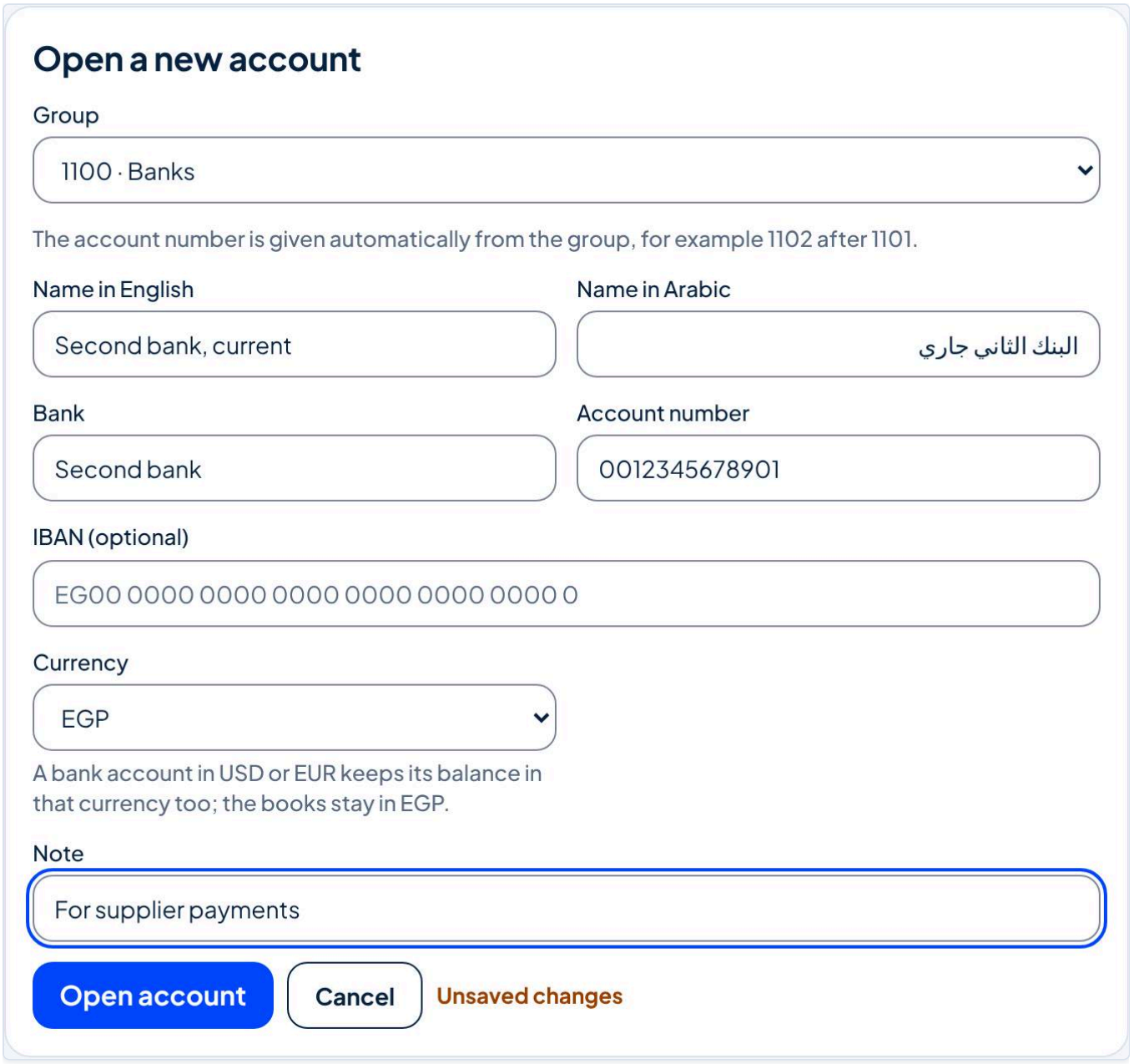
Chart of accounts, top of the page, with markers.

#	Control	What it does	Rule / limits	Result
1	Open a new account	Opens the new account form.	Hidden while the form is open.	The form appears under the toolbar.
2	Search by number, name or bank	Filters the rows as you type (code, English and Arabic names, bank, account number).	In the browser only.	Categories with no match disappear. With no match at all the page shows nothing, with no message.
3	Show closed accounts / Hide closed accounts	Includes or leaves out closed accounts.	`?all=true`	Closed rows are pale with a "Closed" pill.
4	Category card (1000 Assets, 2000 Liabilities, 3000 Equity, 4000 Income, 5000 Expenses)	Groups the accounts.	The first digit is the category.	None.
5	Group heading (for example 1100 · Banks)	The account group.	Groups from `ACCOUNT_GROUPS`.	None.
6	Code	The account number. Company accounts take the next number of their group (1101, 1102); party wallets take <group>-0001 onward.	Given automatically.	None.
7	Name (link)	Opens the account statement.	Bank details and the note show under it in grey.	`/finance/accounts/:id`

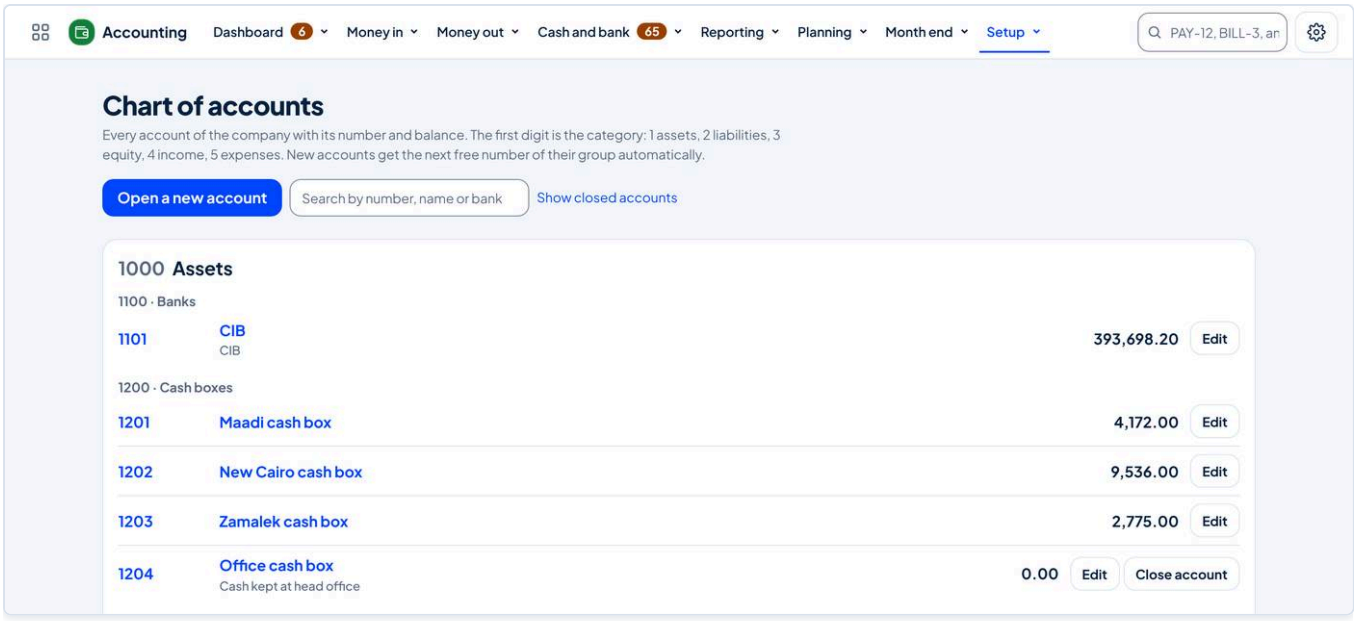
#	Control	What it does	Rule / limits	Result
8	Balance	Summed from ledger lines; liabilities, equity and income shown with the sign flipped. A foreign bank also shows its USD or EUR balance.	Never stored.	None.
9	Edit	Opens the edit form under the row.	Wallet names cannot be changed; only bank accounts show bank fields.	Save or Cancel (form footer).
(row)	Revalue	Month end revaluation of a USD or EUR bank at a rate typed.	Foreign currency banks only.	Posts the difference to 5901.
(row)	Close account / Reopen	Closes or reopens the account. A confirm dialog asks "Close this account? It must have a zero balance, and no new money can use it."	Hidden on system accounts (`locked`). Only an empty account closes.	The row disappears (closed) unless closed accounts are shown.

Open a new account

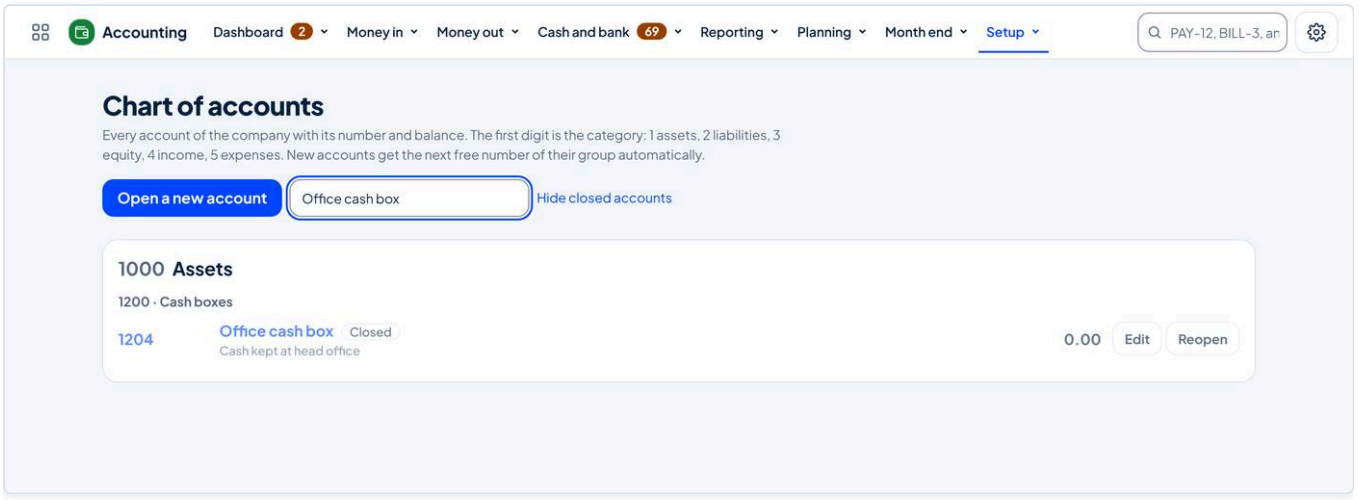
Field	What it does	Rule / limits
Group	The group the account belongs to; it decides the code.	Only groups that may be opened by hand: 1100 Banks, 1200 Cash boxes, 1400 Other assets, 2400 Other liabilities, 3100 Owner capital, 3200 Owner drawings, 4200 Other income, 5400 Operating expenses, 5500 Bank charges.
Name in English, Name in Arabic	The two names.	Required, at least 2 characters each.
Bank, Account number	Bank details.	Banks only; required.
IBAN (optional)	Bank IBAN.	Banks only.
Currency	EGP, USD or EUR.	Banks only. A foreign bank keeps its balance in that currency too; the books stay in EGP.
Note	Free note.	Optional.
Open account, Cancel, Unsaved changes	The form footer.	Cancel asks before throwing changes away.



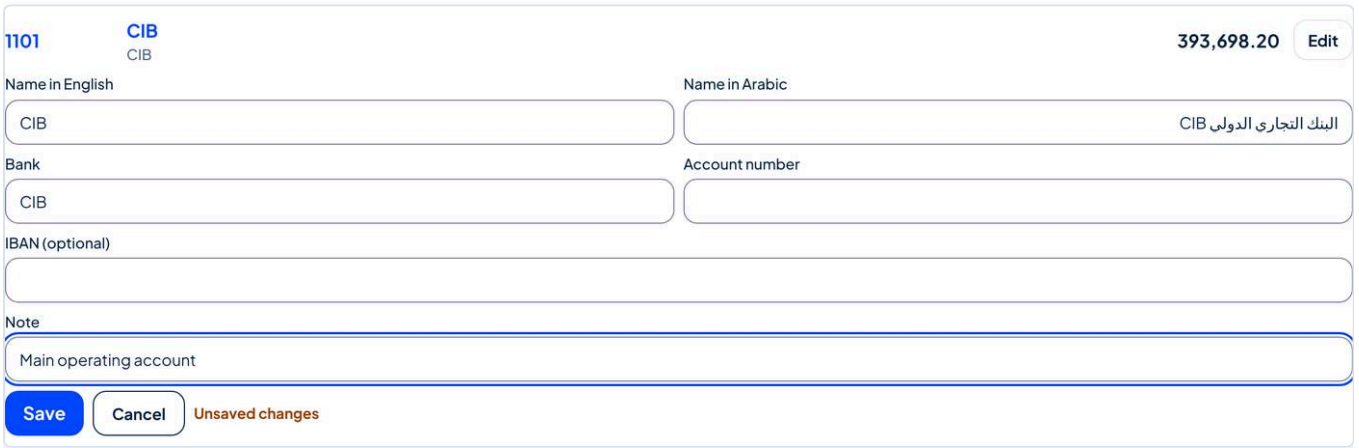
The new account form for a bank, filled in, with "Unsaved changes" in orange beside the buttons.



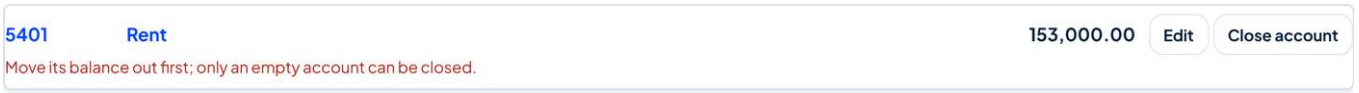
After opening an Office cash box: it received the next code of its group, 1204, and shows a Close account button because it is empty and not a system account. The form closes on success, so the message "Account 1204 opened." is never seen.



The same account after Close account, seen with Show closed accounts: pale, with a Closed pill and a Reopen button.



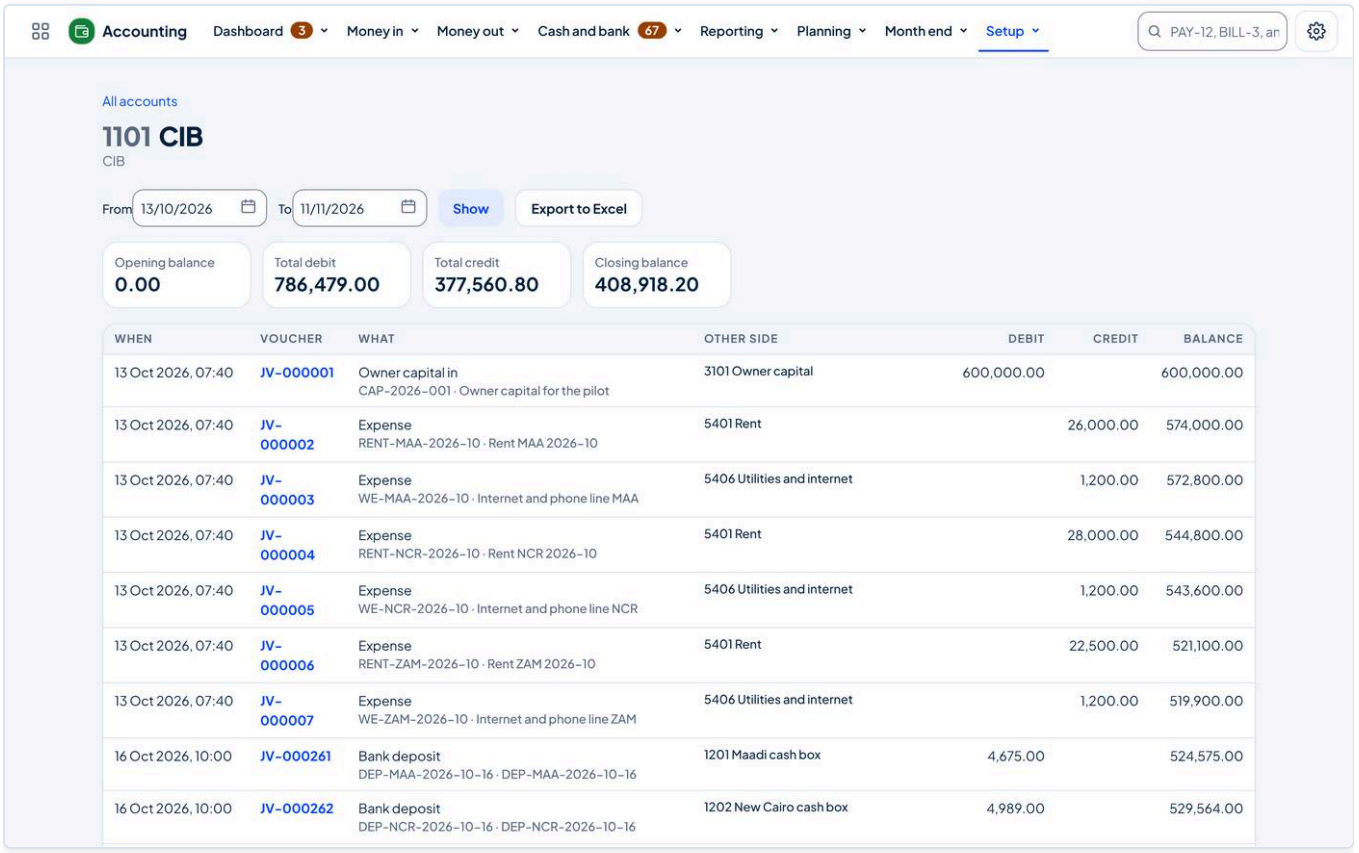
Edit opened on 1101 CIB: names, bank, account number, IBAN and note, with Save and Cancel.



Error: Close account on 5401 Rent, which holds a balance: "Move its balance out first; only an empty account can be closed."

Account statement

Address: `/finance/accounts/:id?from=&to=` (default the last 30 days). Accounting view.



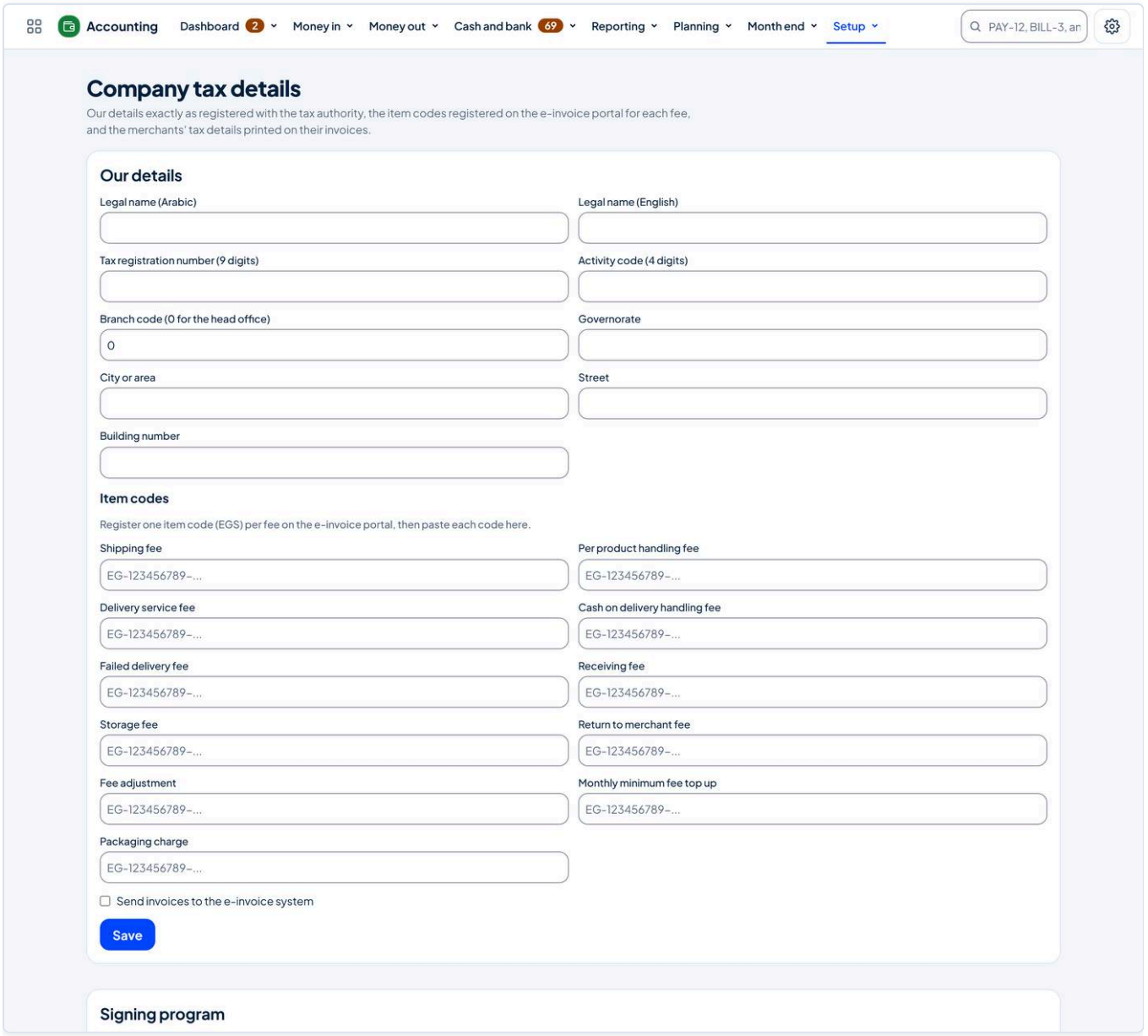
Statement of 1101 CIB: opening balance, total debit, total credit, closing balance, and one row per line with its voucher.

Control	What it does	Rule / limits	Result
All accounts	Back to the chart.	None.	`/finance/accounts`
From, To, Show	The days of the statement.	Cairo days.	The page reloads.
Export to Excel	The same rows as CSV.	`/api/finance/accounts/:id/statement.csv`	A CSV file.
Columns When, Voucher (link), What, Other side, Debit, Credit, Balance	Each line with its running balance.	The voucher opens the journal entry.	None.

17.7 Company tax details

Address: `/finance/company` . **Menu:** Accounting, Setup, Company tax details. **Who:** accounting view reads; Save of our details needs accounting manage (`PUT /finance/company-profile`); New signing key is owner

only (shown only to owners); merchant tax and bank details need accounting use. Code: `CompanyTax` in `components/finance/Invoices.tsx`.



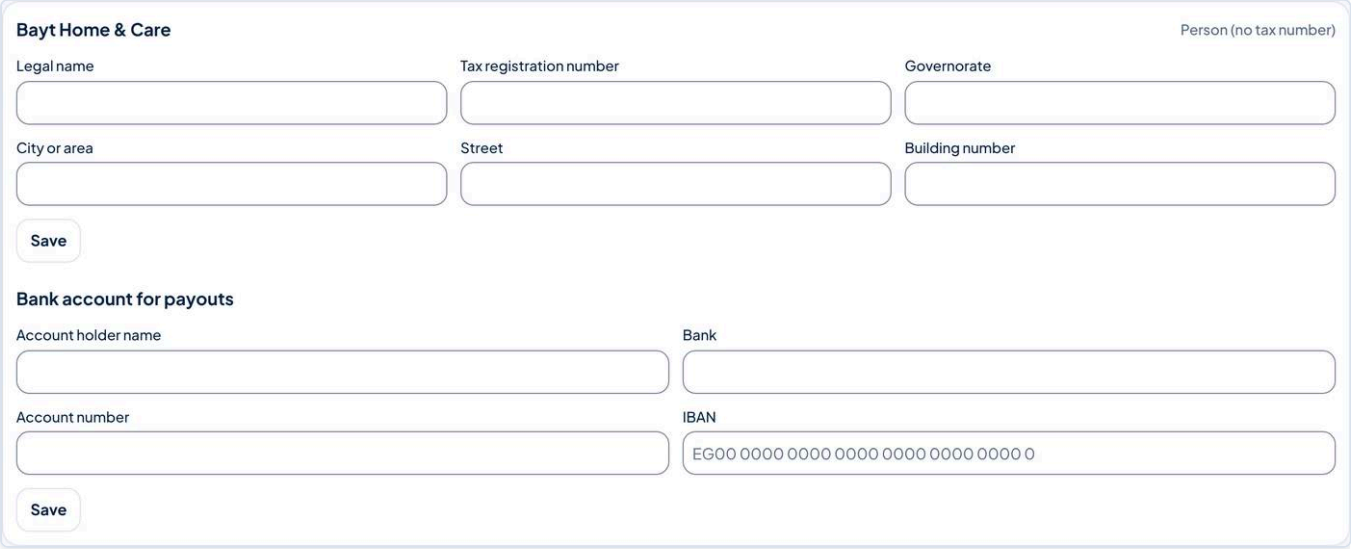
Company tax details: our legal details, the item code per fee kind and the e invoice switch. In the demo the details are still empty.

Control	What it does	Rule / limits	Result
Legal name (Arabic), Legal name (English)	Our legal names as registered.	Printed on invoices and sent to the tax authority.	None.
Tax registration number (9 digits), Activity code (4 digits), Branch code (0 for the head office)	Our tax identity.	Checked on save.	None.
Governorate, City or area, Street, Building number	Our address.	Required for e invoices.	None.
Item codes (one per fee kind: Shipping fee, Per product handling fee, Delivery service fee, Cash on delivery handling fee, Failed delivery fee, Receiving fee, Storage fee, Return to merchant fee, Fee adjustment, Monthly minimum fee top up, Packaging charge)	The item code (EGS) registered on the e invoice portal for each fee.	Needed only once a contract, the standard price or a booking uses that fee.	Used on each invoice line.

Control	What it does	Rule / limits	Result
Send invoices to the e invoice system	The e invoice switch.	Nothing is sent without ETA credentials.	None.
Save	Saves the details, codes and switch.	Accounting manage; audited.	"Done."

Signing program: whether a signing key exists, and New signing key for owners.

Control	What it does	Rule / limits	Result
New signing key	Makes the key the signing program on the finance PC uses.	Owner only. The old key stops working.	The key is shown once in a grey box: "Copy this key into the signing program now; it is shown only once."
Merchants' tax details (one row per merchant)	Click a row to open it. The right side reads "Company, complete", "Company, address missing" or "Person (no tax number)".	A merchant with a tax number is invoiced as a company and needs its full address.	Fields Legal name, Tax registration number, Governorate, City or area, Street, Building number with Save (`PUT /finance/merchants/:id/tax`), then Bank account for payouts (`/finance/merchants/:id/bank`).

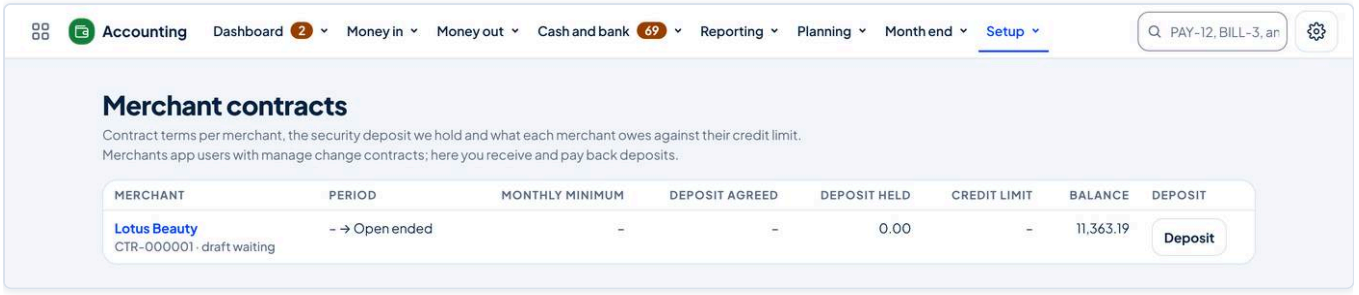


A merchant row opened: tax fields, Save and the payout bank account form.

17.8 Merchant contracts and deposits

Address: `/finance/contracts` . **Menu:** Accounting, Setup, Merchant contracts. **Who:** accounting view reads; the Deposit column shows only with accounting use; the merchant name links to the merchant page only with merchants view. Contracts are changed only in the Merchants app (merchants manage). Data: `GET /finance/contracts` (`listContracts`).

The demo copy had no contract at first; the page then shows its empty state "No contracts yet. Open a merchant in the Merchants app to add one." A draft version was made for Lotus Beauty to show a row.



Merchant contracts with one contract that has only a draft version: CTR-000001 · draft waiting, no version in force, so the period reads "– → Open ended" and the agreed terms are dashes.

Column / control	What it shows or does	Rule / limits
Merchant	Name (link to the merchant), the version in force (CTR-000001 v2) or the contract number, "draft waiting" when a draft exists, and "Over limit" in red when the balance owed passes the credit limit.	Version in force on today's Cairo day.
Period	Start → end (or "Open ended") and "Notice by" (end minus notice days).	From the version in force.
Monthly minimum, Deposit agreed, Credit limit	Terms of the version in force; a dash when not set.	Amounts in EGP.
Deposit held	Balance of the merchant's 2600 deposit wallet.	Summed from ledger lines.
Balance	What the merchant owes us (positive) or we owe them.	From the merchant wallet.
Deposit	Opens the deposit form in the row.	Accounting use.

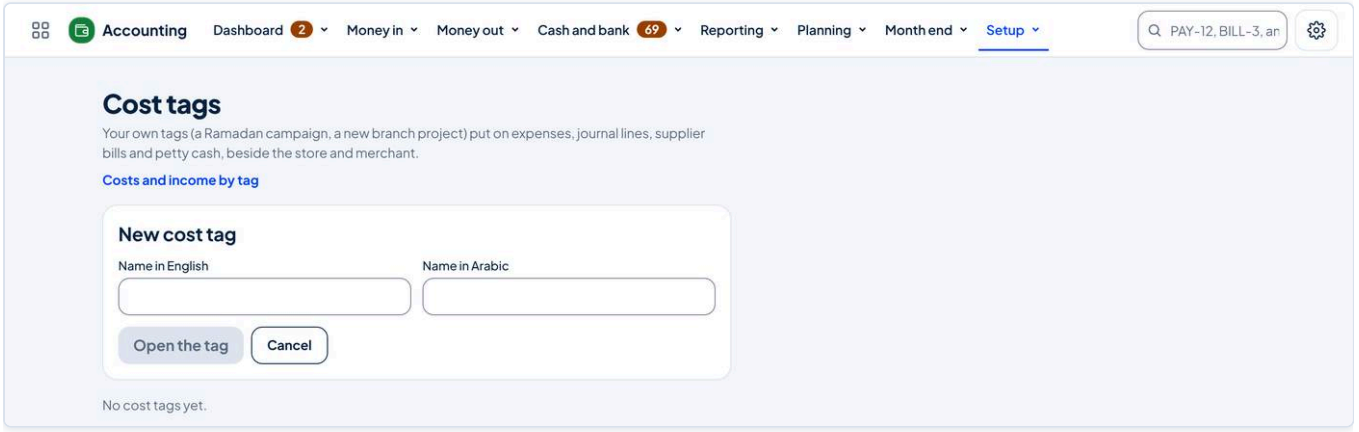
The deposit form opened: Movement Received, Through Our bank, 5000 EGP, a reference and a receipt photo.

Field	What it does	Rule / limits	Result
Movement	Received or Paid back.	None.	None.

Field	What it does	Rule / limits	Result
Through	Our bank, or The merchant's balance.	From the balance moves money between the merchant wallet and the deposit wallet.	None.
Amount (EGP), Reference	Amount and reference.	Reference 2 to 100 characters.	None.
Receipts and documents	Attachment field.	Optional.	Linked to the entry or request.
Record, Cancel	Sends `POST /finance/merchants/:id/deposit` with an idempotency key; Cancel closes the form.	Second person rule: money received waits while finance.approve_money_in is 1; paid back waits from the approval limit.	"Sent for approval." or "Done."; Deposit held changes once posted.

17.9 Cost tags

Address: `/finance/cost-tags` . **Menu:** Accounting, Setup, Cost tags. **Who:** accounting view sees a plain list of tag names; accounting manage sees the form and the switches (`POST` and `PATCH /finance/cost-tags`).

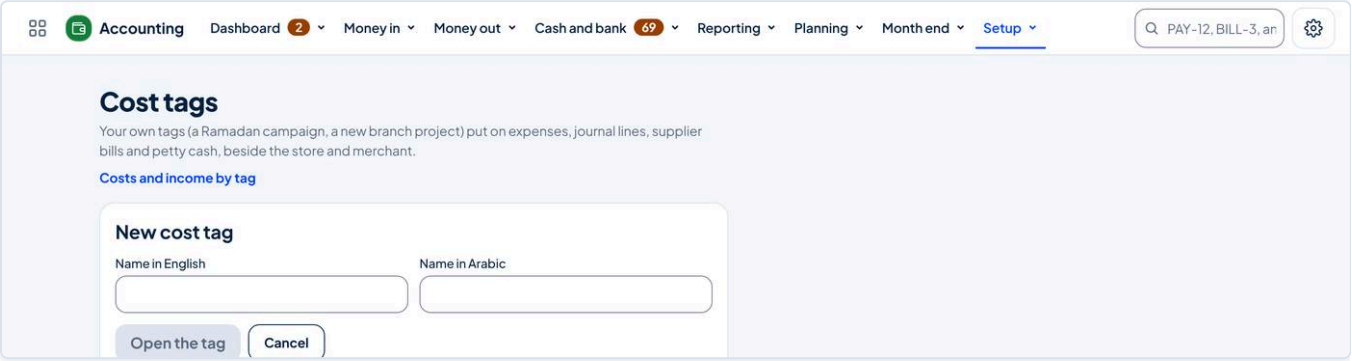


Cost tags before any tag exists: the New cost tag form and the empty state "No cost tags yet."

Control	What it does	Rule / limits	Result
Costs and income by tag	Opens the tag report.	None.	`/finance/reports/tags`
Name in English, Name in Arabic	The tag's names.	Each at least 2 characters.	None.
Open the tag, Cancel	Form footer: saves the tag or empties the fields.	Accounting manage; audited.	The tag appears in the list; "Done."
Tag row: name and "Not used yet" / "On n lines"	How many ledger lines carry it.	None.	None.

Control	What it does	Rule / limits	Result
Switch off / Switch on	Stops or allows choosing the tag on new expenses, journal lines, bills and petty cash.	Accounting manage. Lines already tagged keep it.	An off tag is pale.

New cost tag filled in.

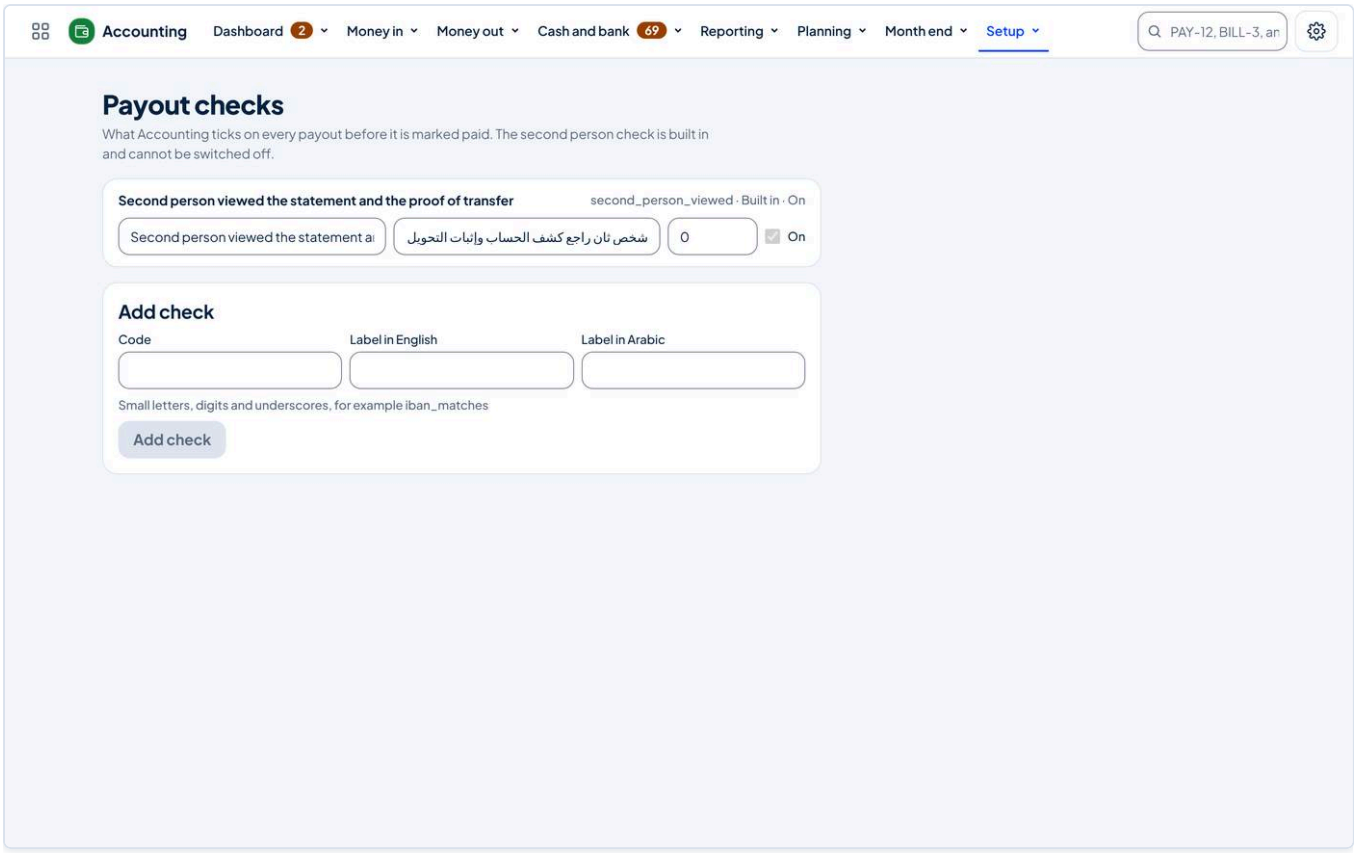


The tag Ramadan campaign opened: "Done." and the row "Not used yet" with Switch off.

17.10 Payout checks

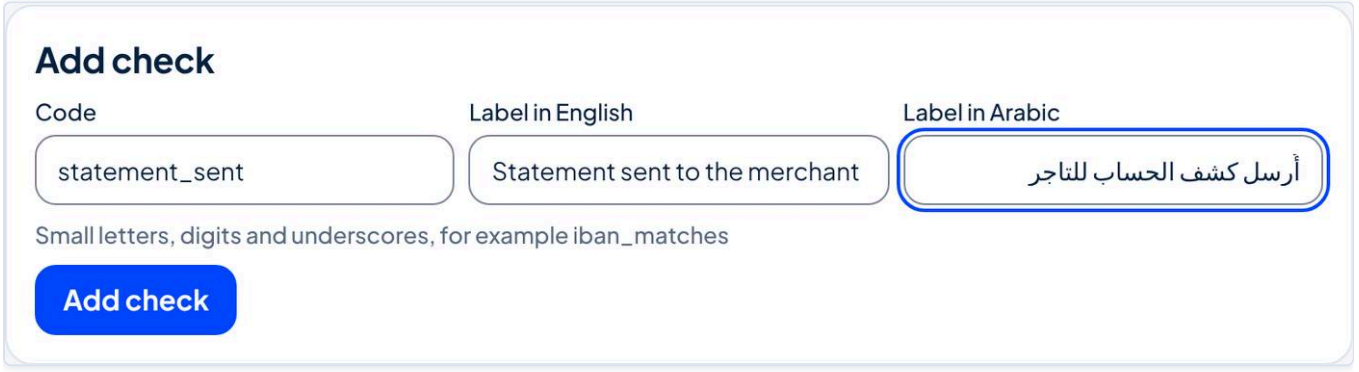
Address: `/finance/payout-checks`. **Menu:** Accounting, Setup, Payout checks. **Who:** accounting view reads; editing and adding need accounting manage. Code: `components/finance/PayoutChecksEditor.tsx`.

These are the boxes Accounting ticks on every payout before it can be marked paid (the payout page shows "Some checks are not ticked yet." until all active checks are ticked). The built in check *Second person viewed the statement and the proof of transfer* (`second_person_viewed`) always exists and cannot be switched off.

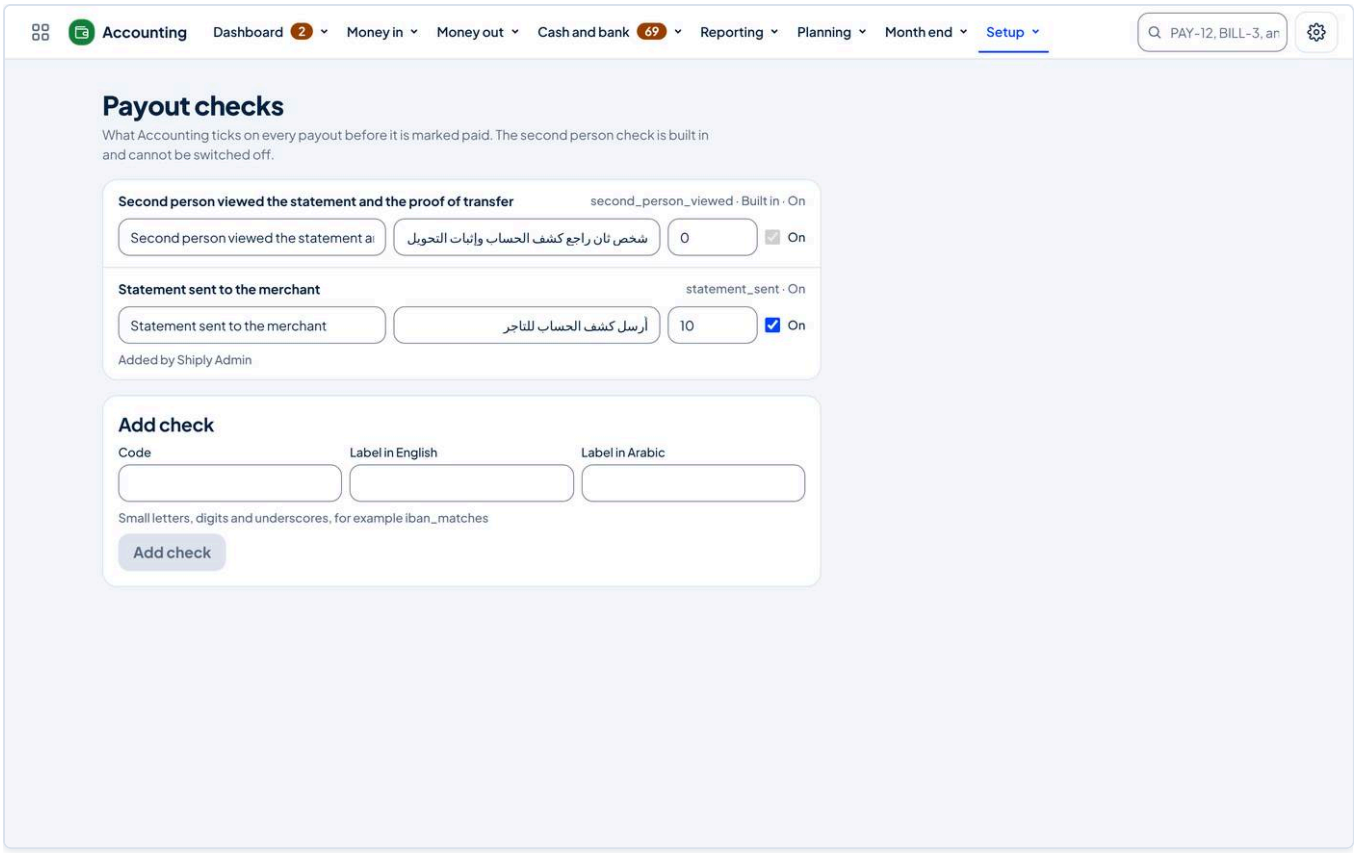


Payout checks with only the built in check: its code, "Built in · On", the English and Arabic labels, the order 0 and the greyed On box.

Control	What it does	Rule / limits	Result
Row header	Label, code, "Built in" for the system check, On or Off.	None.	None.
Label in English, Label in Arabic, Order, On	Edit a check. A Save button appears once something changed.	Accounting manage. On cannot be switched off on the built in check (`check_is_system`).	Saved; "Added by <name>" under checks people added.
Add check: Code, Label in English, Label in Arabic	A new check.	Code: small letters, digits and underscores, unique (`check_code_taken` : "A check with this code exists."). All three required. Order is the last one plus 10.	The new check appears, switched on.



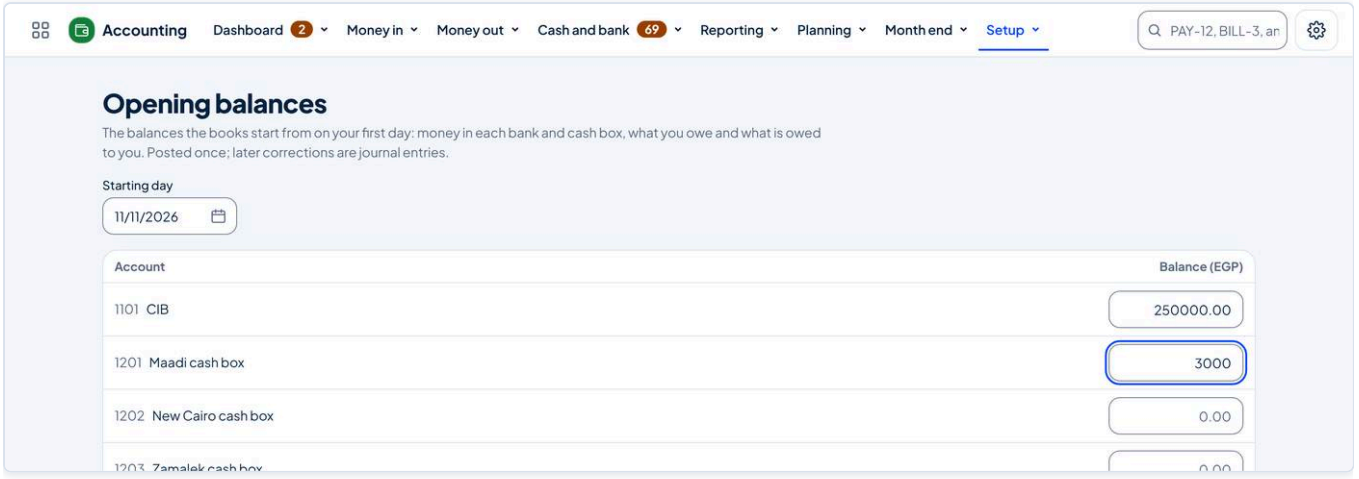
Add check filled in: statement_sent.



After Add check: the new check with order 10, On, and "Added by Shiply Admin".

17.11 Opening balances

Address: `/finance/opening`. **Menu:** Accounting, Setup, Opening balances, shown to owners only; anyone else gets the not found page. **API:** `POST /finance/opening-balances` needs accounting manage and settings manage.



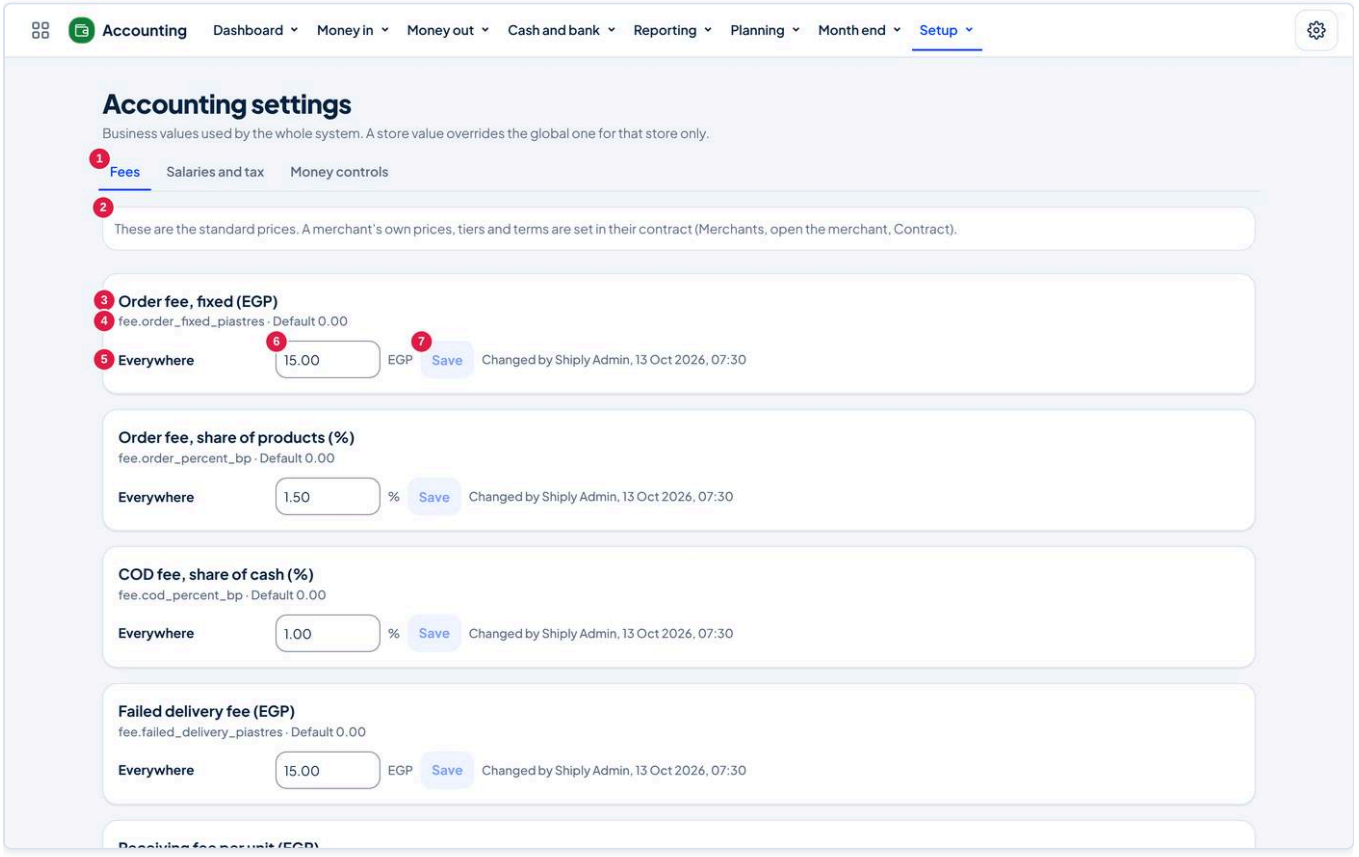
Opening balances with two balances typed (not posted in the demo). Every open account except Opening balances (3300), income and expenses is listed.

Control	What it does	Rule / limits	Result
Starting day	The day the books start (default today).	Not later than today.	Posted at the start of that day.

Control	What it does	Rule / limits	Result
Balance (EGP) per account	Type each balance as an accountant reads it: money held, or the amount owed.	A minus sign only for the opposite side (for example an overdrawn bank). A wrong number disables the button.	None.
Post opening balances	Posts one entry; the difference goes to 3301 Opening balances.	Owner only, once (`opening_already_posted`). Disabled with no amount.	Afterwards the page shows only "Opening balances were already posted. Correct them with a journal entry."

17.12 Accounting settings

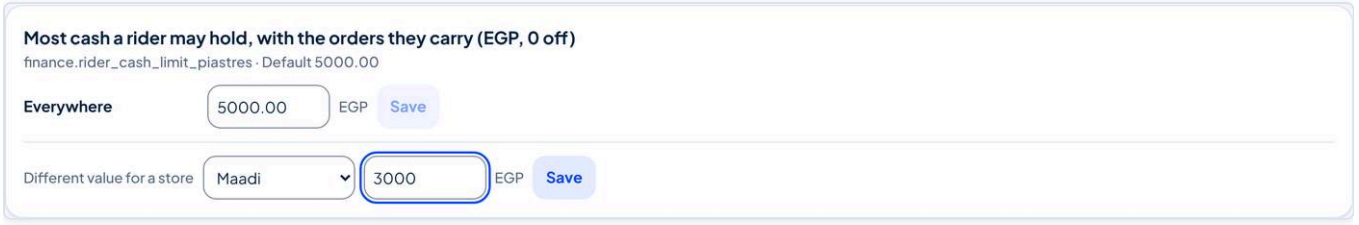
Address: `/admin/settings/accounting` . **Menu:** Accounting, Setup, Settings (shown only to people with settings manage). **Who:** every team member's pages may read settings (`GET /admin/settings`); changing one needs settings manage (`PUT /admin/settings` , audited). Code: `components/admin/SettingsEditor.tsx` ; definitions and defaults in `packages/shared/src/settings.ts` .



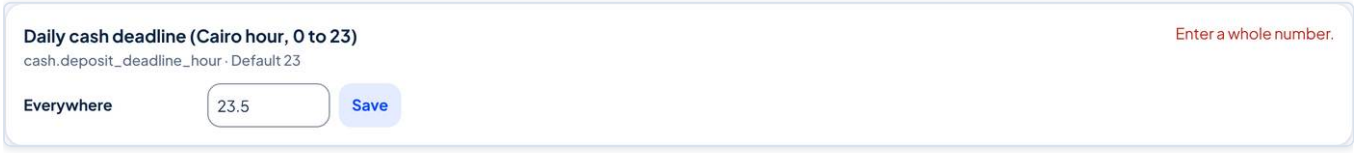
Accounting settings, tab Fees, with markers on one setting card.

#	Control	What it does	Rule / limits	Result
1	Tabs Fees, Salaries and tax, Money controls	Show one group of settings.	The tab is kept only while on the page.	None.

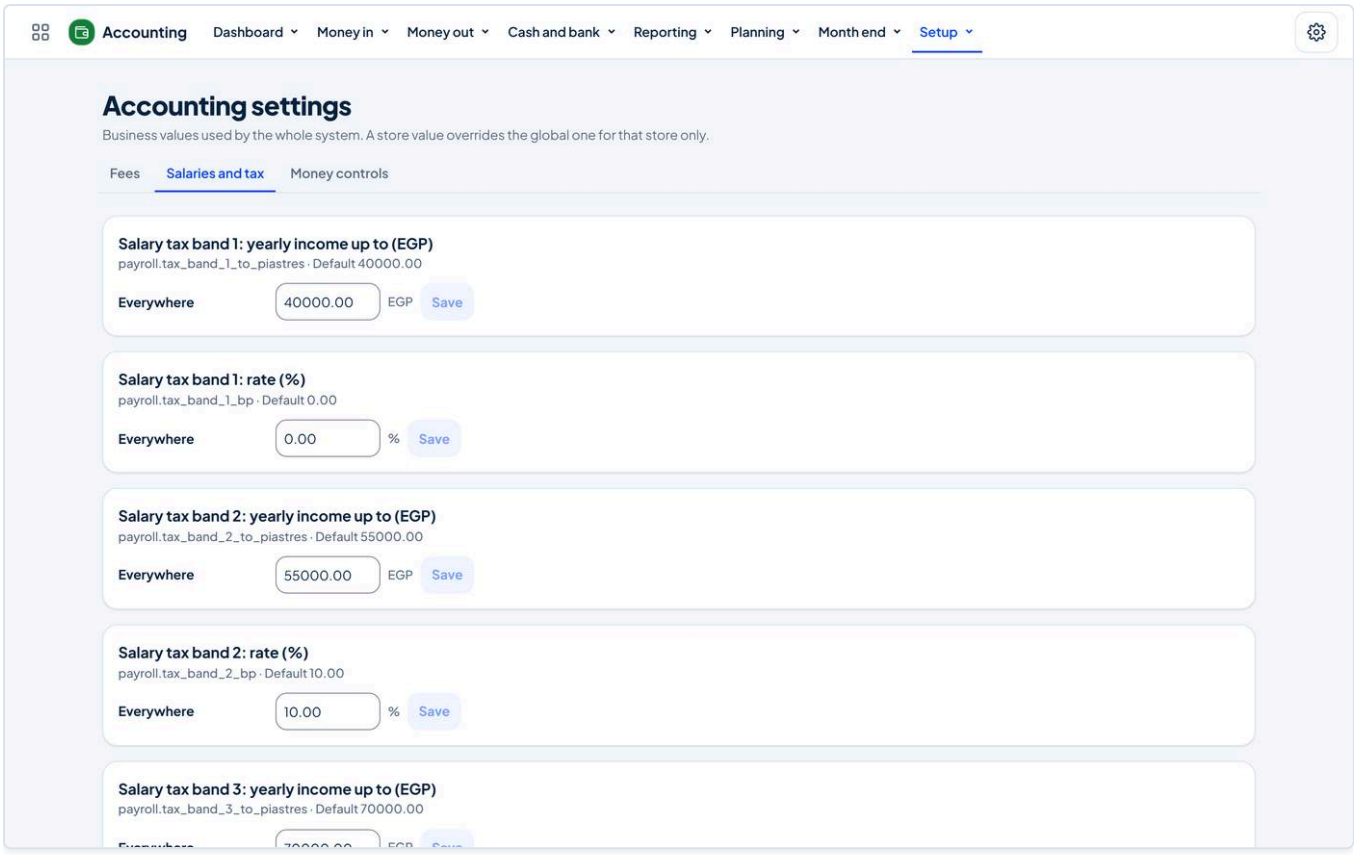
#	Control	What it does	Rule / limits	Result
2	Fees note	"These are the standard prices. A merchant's own prices, tiers and terms are set in their contract."	Fees tab only.	None.
3	Setting name	Plain name with its unit.	Two settings have no plain name and show their key (finance.corporate_tax_bp, finance.payout_minimum_piastres).	None.
4	Key and Default	The setting key and its default value.	The default is used until a value is saved.	None.
5	Everywhere	The global value.	None.	None.
6	Value box with unit	Money in EGP (stored as piastres), shares in percent (stored as basis points), other values as whole numbers.	"Enter a whole number." for a value that is not one; "That value is outside what this setting allows." for one outside the range.	None.
7	Save	Saves the value.	Pale until the value changes. Settings manage.	"Saved." and "Changed by <name>, <time>".
(card)	Different value for a store: store list, value, Save; Remove on an override	A store's own value.	Only settings whose override is "store".	The store row appears under Everywhere; Remove deletes it.



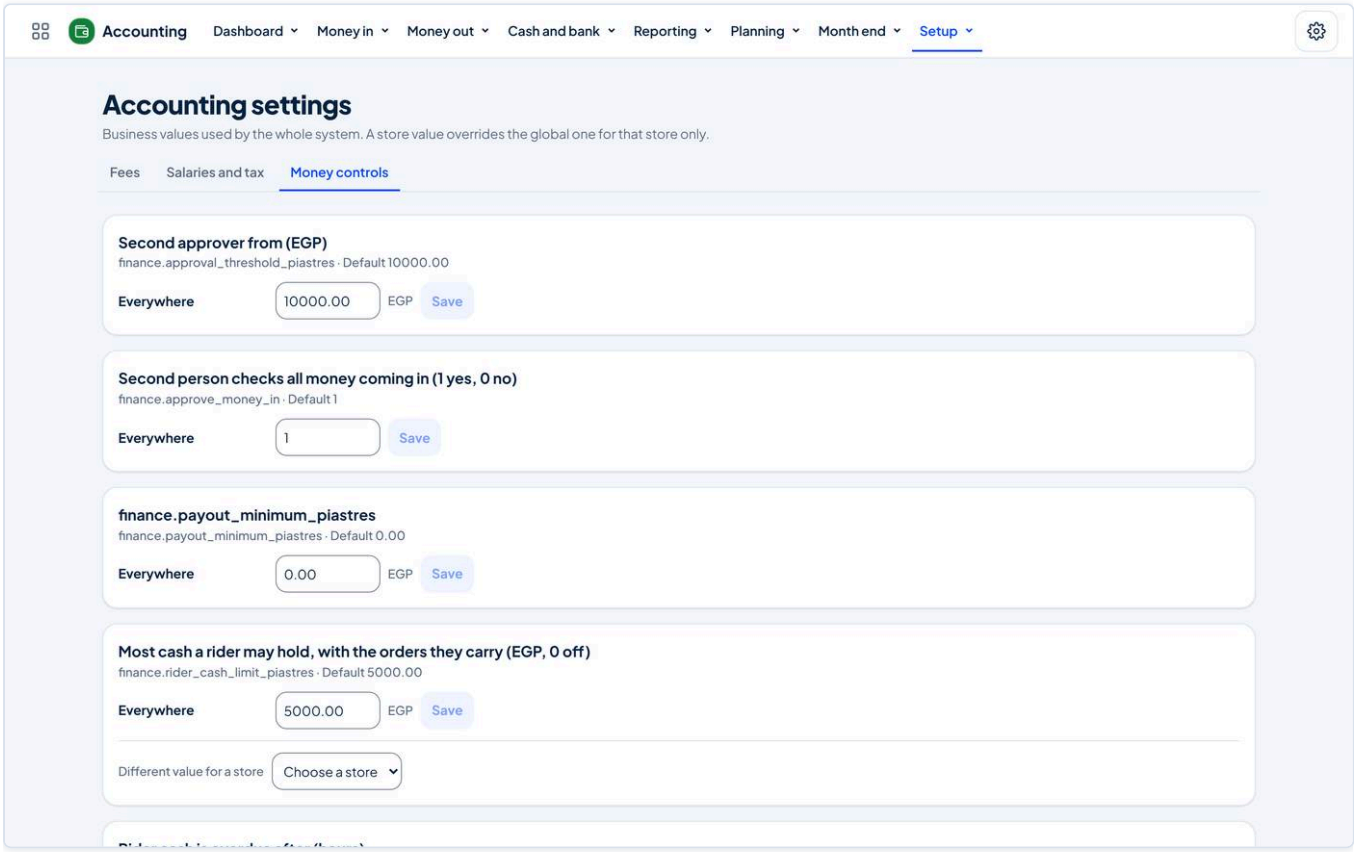
A store value being added: Maadi, 3000 EGP for the rider cash limit.



Error: 23.5 typed for the daily cash deadline hour: "Enter a whole number." in red at the top right of the card.



Tab Salaries and tax (top of the page).



Tab Money controls (top of the page).

Every Accounting setting and its default

"Store" in the last column means a dark store may have its own value. Money is shown in EGP, shares in percent.

Tab Fees

Key	Name on screen	Default	Store
`fee.order_fixed_piastres`	Order fee, fixed (EGP)	0.00	No
`fee.order_percent_bp`	Order fee, share of products (%)	0.00%	No
`fee.cod_percent_bp`	COD fee, share of cash (%)	0.00%	No
`fee.failed_delivery_piastres`	Failed delivery fee (EGP)	0.00	No
`fee.receiving_per_unit_piastres`	Receiving fee per unit (EGP)	0.00	No
`fee.storage_per_unit_month_piastres`	Storage per unit per month (EGP)	0.00	No
`fee.return_per_unit_piastres`	Return to merchant fee per unit (EGP)	0.00	No
`fee.shipping_fee_piastres`	Shipping fee per delivered order (EGP)	0.00	No
`fee.packaging_charge_piastres`	Packaging charge per delivered order (EGP, 0: our cost)	0.00	No
`fee.failed_delivery_percent_bp`	Failed delivery, share of the shipment for contracts with a shipping fee (%)	100.00%	No
`fee.postal_bp`	Postal fee (share of our fees per shipment)	10.00%	No
`fee.vat_bp`	VAT on our fees (%)	0.00% (the owner set 14% on 2026-11-11)	No
`rider.bonus_per_delivery_piastres`	Rider pay per delivered order (EGP)	0.00	Store
`rider.withdraw_hold_working_days`	Rider pay hold (working days)	5	No
`rider.weekend_days`	Days off for the hold (weekday digits, 5 = Friday)	5	No
`rider.min_withdrawal_piastres`	Smallest rider withdrawal (EGP)	100.00	No
`rider.max_open_withdrawals`	Open withdrawals per rider	1	No
`rider.penalty_day_piastres`	A day of rider pay for penalties (EGP)	200.00	Store

Tab Salaries and tax

Key	Name on screen	Default
`payroll.tax_band_1_to_piastres` / `_1_bp`	Salary tax band 1: yearly income up to / rate	40,000.00 EGP / 0%
`payroll.tax_band_2_to_piastres` / `_2_bp`	Band 2	55,000.00 EGP / 10%
`payroll.tax_band_3_to_piastres` / `_3_bp`	Band 3	70,000.00 EGP / 15%

Key	Name on screen	Default
`payroll.tax_band_4_to_piastres / _4_bp`	Band 4	200,000.00 EGP / 20%
`payroll.tax_band_5_to_piastres / _5_bp`	Band 5	400,000.00 EGP / 22.5%
`payroll.tax_band_6_to_piastres / _6_bp`	Band 6	1,200,000.00 EGP / 25%
`payroll.tax_top_bp`	Salary tax rate above the last band (%)	27.5%
`payroll.tax_skip_1_from_piastres to _5_`	Above this yearly income, bands 1 to n lose their lower rate (EGP)	600,000 / 700,000 / 800,000 / 900,000 / 1,200,000 EGP (0 turns one off)
`finance.corporate_tax_bp`	(key shown, no plain name) company income tax rate	22.5%
`payroll.personal_exemption_piastres`	Salary tax: yearly personal exemption (EGP)	20,000.00
`payroll.si_employee_bp`	Social insurance, employee share (%)	11%
`payroll.si_employer_bp`	Social insurance, company share (%)	18.75%
`payroll.si_min_wage_piastres`	Lowest monthly insured wage (EGP)	2,700.00
`payroll.si_max_wage_piastres`	Highest monthly insured wage (EGP)	16,700.00
`payroll.martyrs_fund_bp`	Martyrs fund, share of gross salary (%)	0.05%

Tab Money controls

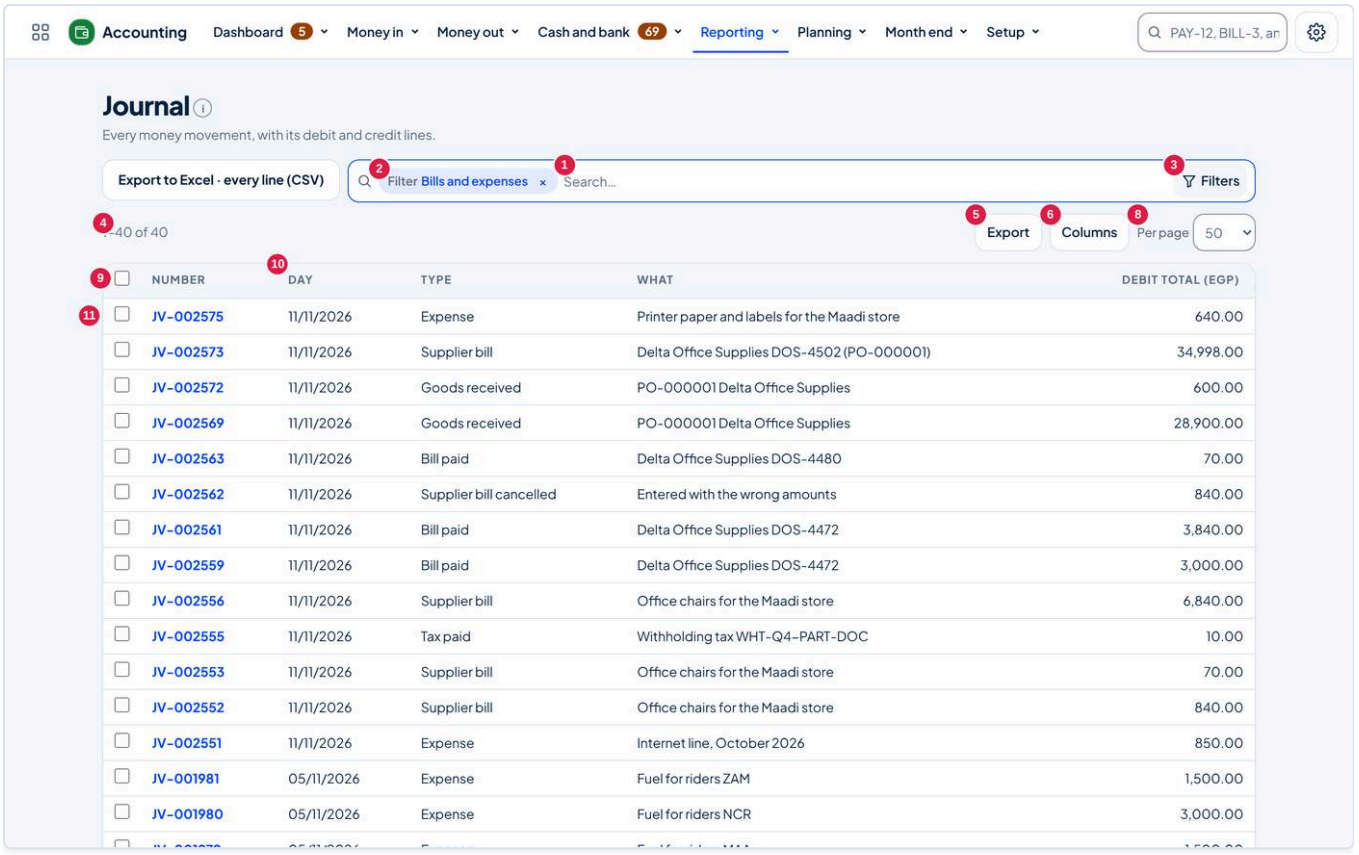
Key	Name on screen	Default	Store
`finance.approval_threshold_piastres`	Second approver from (EGP)	10,000.00 (0 turns it off)	No
`finance.approve_money_in`	Second person checks all money coming in (1 yes, 0 no)	1	No
`finance.payout_minimum_piastres`	(key shown, no plain name) payouts wait until we owe at least this	0.00	No
`finance.rider_cash_limit_piastres`	Most cash a rider may hold, with the orders they carry (EGP, 0 off)	5,000.00	Store
`finance.rider_cash_max_hours`	Rider cash is overdue after (hours)	12	Store
`finance.rider_cash_warn_hours`	Cash position: rider cash is getting old after (hours)	4	No
`finance.petty_cash_limit_piastres`	Petty cash above this waits for finance (EGP, 0 off)	500.00	Store

Key	Name on screen	Default	Store
`finance.petty_cash_float_piastres`	Cash a store box keeps; petty cash below it waits for finance (EGP)	1,000.00	Store
`cash.store_handover_enabled`	Riders may hand cash to the store (1 open, 0 closed)	1	Store
`cash.basata_enabled`	Riders may deposit cash at Basata (1 open, 0 closed)	1	Store
`cash.deposit_deadline_enabled`	Take cash held at the deadline and freeze the rider (1 on, 0 off)	1	No
`cash.deposit_deadline_hour`	Daily cash deadline (Cairo hour, 0 to 23)	23	No
`claims.max_age_days`	Expense claims: oldest day paid that may be claimed (days back)	60	No
`storage.catch_up_days`	Storage days charged after the server slept (most)	7	No
`finance.po_bill_tolerance_bp`	Bill above purchase order allowed by (%)	2%	No
`receivables.reminders_enabled`	Automatic payment reminders to merchants who owe us (1 on, 0 off)	1	No
`receivables.reminder_first_days`	First payment reminder after this many days in debit	3	No
`receivables.reminder_every_days`	Then a payment reminder every (days)	7	No
`receivables.reminder_max`	Most payment reminders per debit period (hand sent ones count)	4	No
`receivables.reminder_hour`	Payment reminders go out from this Cairo hour	10	No
`finance.due_reminder_days`	Phone reminder of a payment this many days before it is due (and on the day)	3	No

> Settings are keyed by (key, scope). A merchant scope is refused (`scope_not_allowed`): a merchant's own prices live in the merchant contract. The order of the tabs follows `tabOf()` in `SettingsEditor.tsx` : keys starting fee. or rider. go to Fees; payroll. and finance.corporate_tax_bp to Salaries and tax; finance., claims., storage., receivables. and cash. to Money controls.

17.13 Common part: the list view

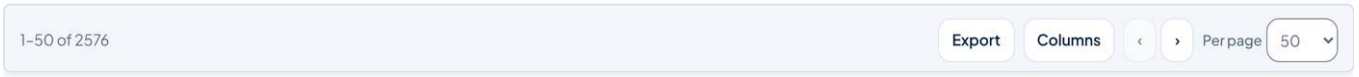
Every Accounting list (payouts, bills, journal, cheques, rider withdrawals, approvals, petty cash, invoices, purchase orders and more) is the same `ListView` (`components/odoo/ListView.tsx`), fed by `GET /finance/lists/:id` . The whole state lives in the address: `q` search, `f` filters, `v` value filters, `g` group by, `sort` / `desc` , `page` , `size` . The server loads at most 5,000 newest rows inside the person's stores and merchants, then searches, filters, groups, totals and pages them.



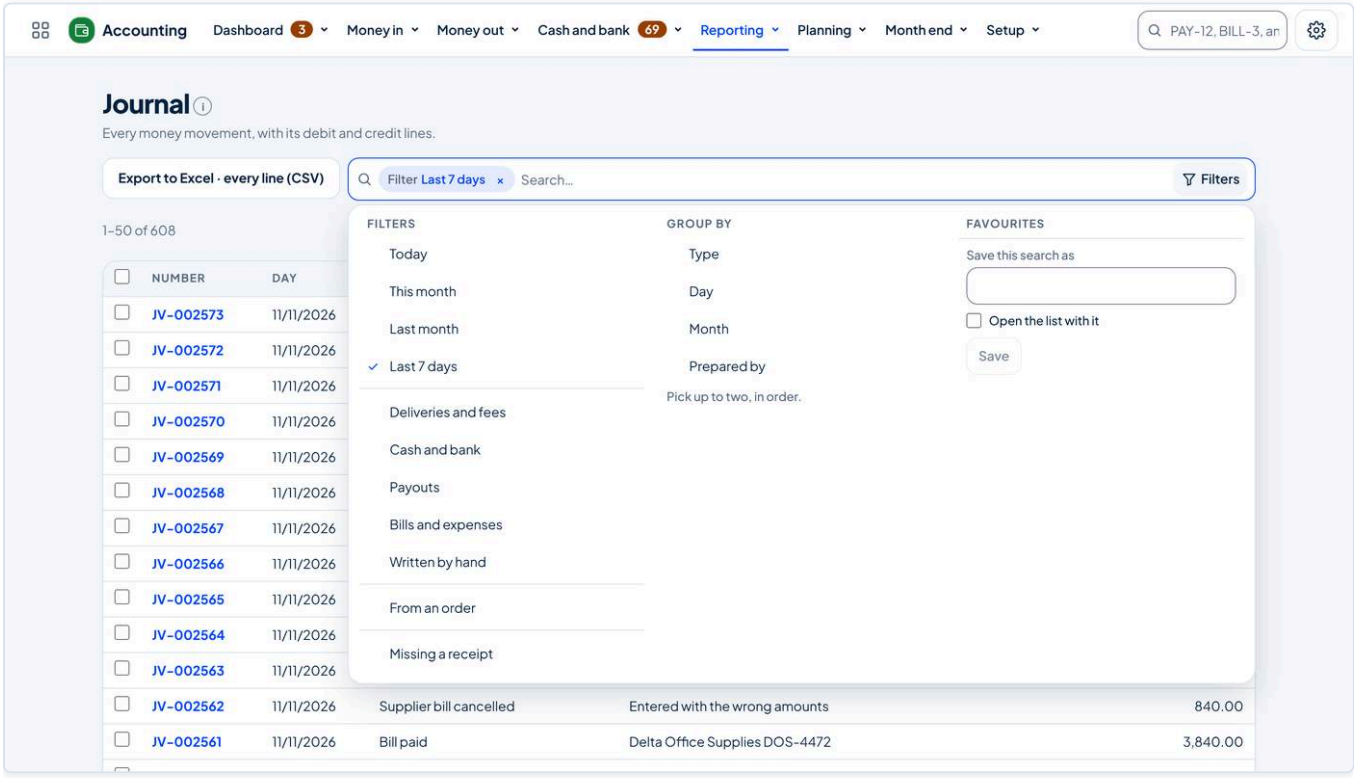
The journal list filtered to Bills and expenses, with markers. Address /finance/journal?f=bills.

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Search box	Magnifier icon, placeholder "Search…", a white box with a grey border that turns blue when focused.	Type and press Enter: adds a Search chip; Backspace in an empty box removes the last chip.
2	Facet chip	A pill on light blue (#E5ECFF) with a grey word (Search, Filter, Group by) and the value in blue; several filters of one group are joined by "or"; a small × at the end.	× removes that chip. Removing the last filter keeps the list unfiltered (the defaults never come back by themselves).
3	Filters	Funnel icon and the word Filters (icon only on phones).	Opens the panel Filters, Group by, Favourites.
4	Range	Grey text "1–40 of 40", or "40 records in 6 groups" when grouped.	None.
5	Export	White button.	Opens "CSV (Excel, any program)", "Excel workbook (.xlsx)" and, when rows are ticked, "Only the n selected".
6	Columns	White button.	Opens the column chooser.
7	Previous and next page (‹ ›)	Two small square buttons; a pale one is disabled. Shown only when there is more than one page and the list is not grouped.	Moves one page.
8	Per page	A select (20, 50, 100).	Changes the page size and goes to page 1.
9	Select all	Checkbox in the header.	Ticks every row on the page and shows the bulk bar.

#	Part	What it shows (colors, icons, wording)	What clicking it does
10	Column header	Grey capitals; money columns are right aligned with " (EGP)".	Sorts by that column; a second click reverses.
11	Row	Checkbox, the number in blue bold (a link), other cells in normal text; status cells as pills.	The number opens the document page; the list's rows are remembered for previous and next.

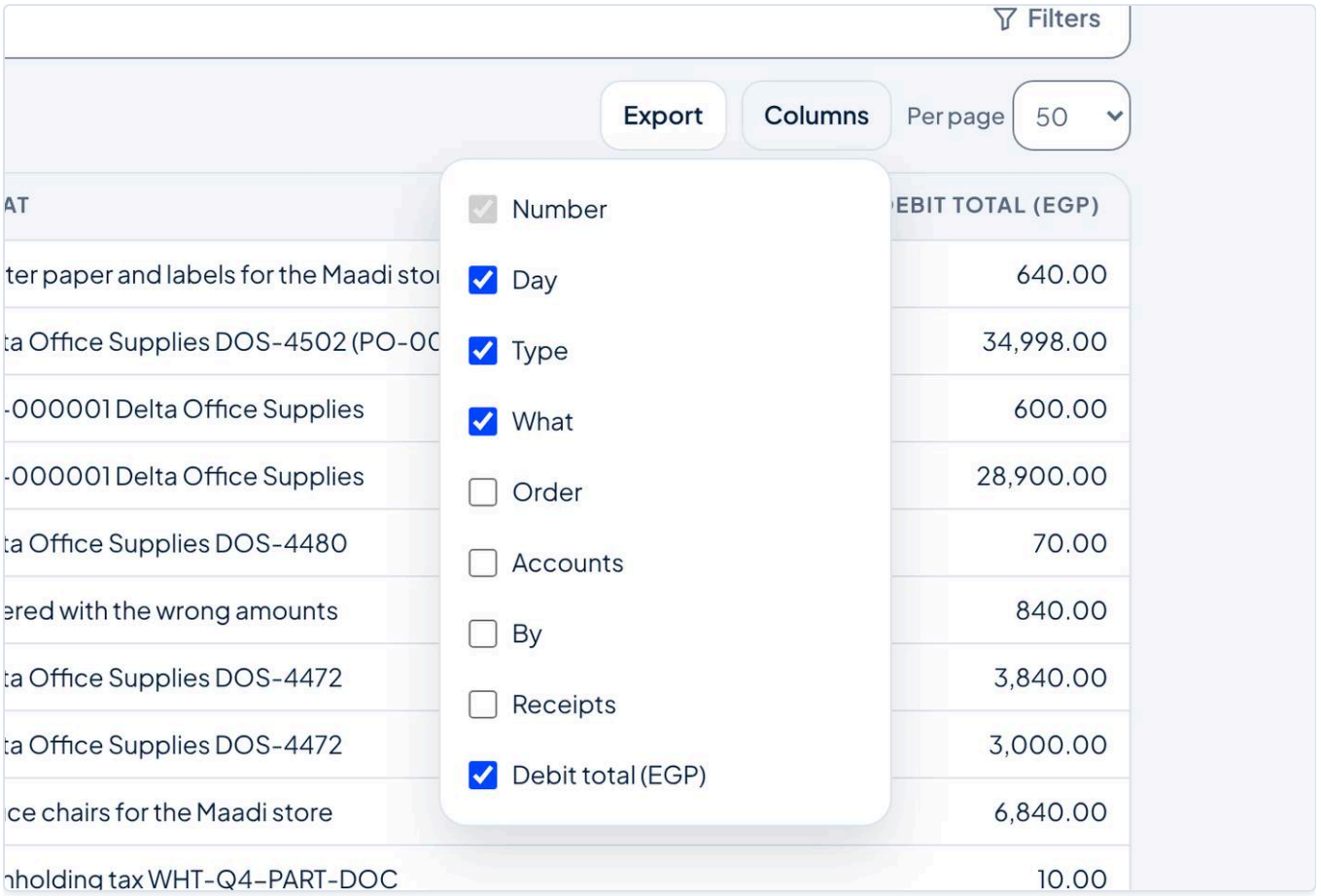


The toolbar of the unfiltered journal (2,576 entries): range, Export, Columns, previous and next page, and Per page.

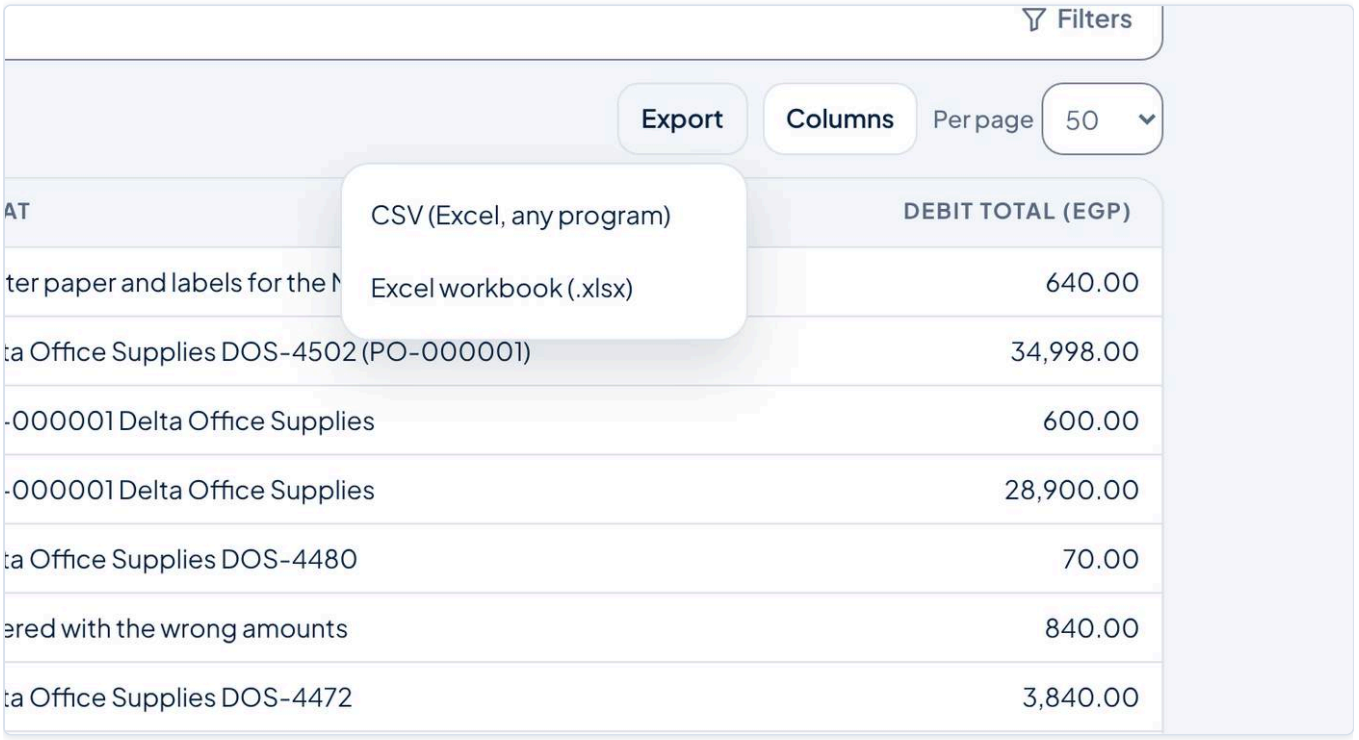


The Filters panel open: Filters in groups separated by lines (a tick marks the ones on), Group by (numbers 1 and 2 mark the order), Favourites with "Save this search as", "Open the list with it" and Save.

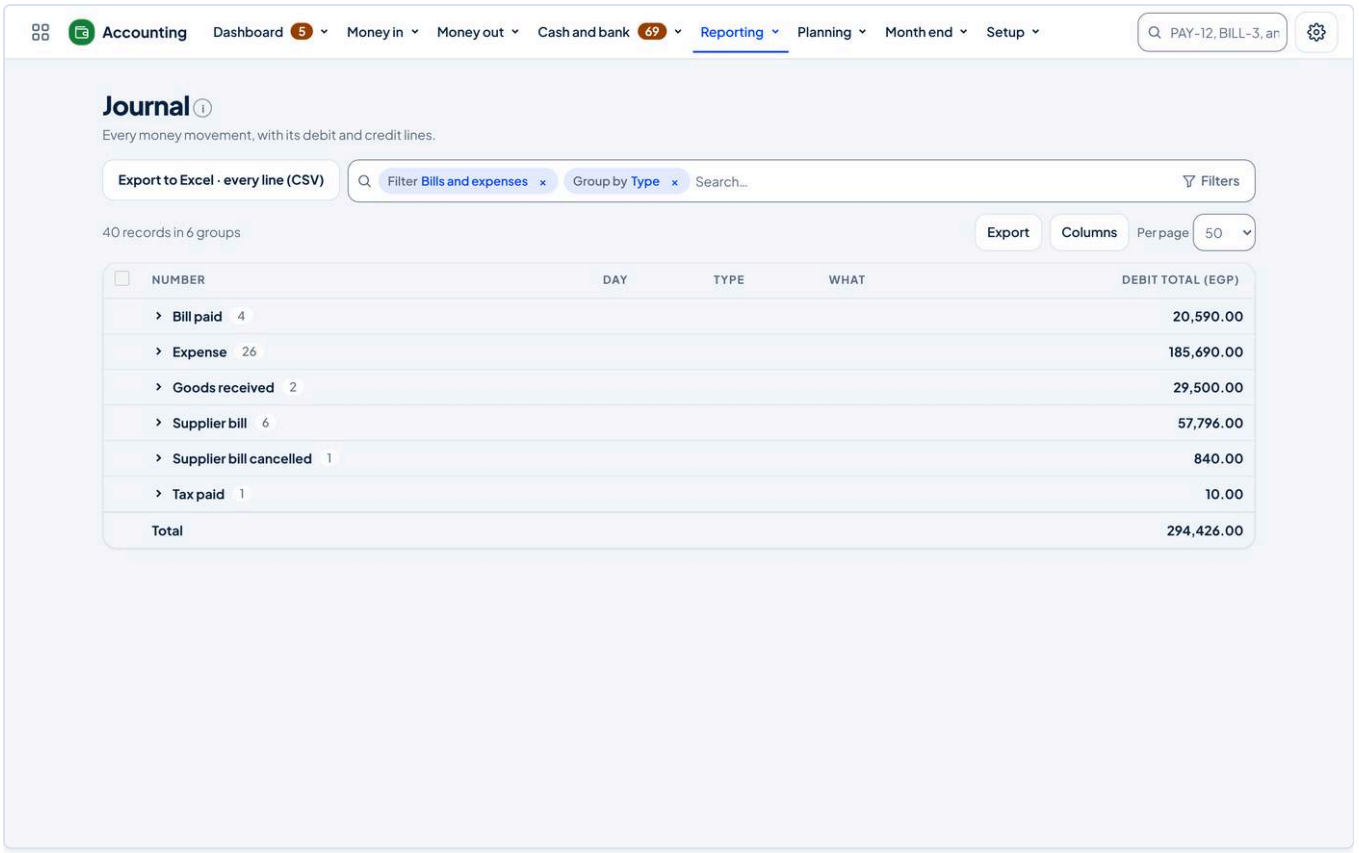
Panel part	What it does	Rule
Filters	Each filter toggles on or off.	Filters in the same group are joined by OR, groups by AND. Some lists add pick lists and day boxes (store, merchant, from, to) with an Apply days button.
Group by	Groups the rows; pick up to two in order.	Groups show a count and the subtotal of money columns; a group opens with its chevron.
Favourites	Saves the current search under a name; ★ marks the default (opens when the address names no view), ☆ the others; × deletes.	Per person and list (`user_list_favorites`), audited.



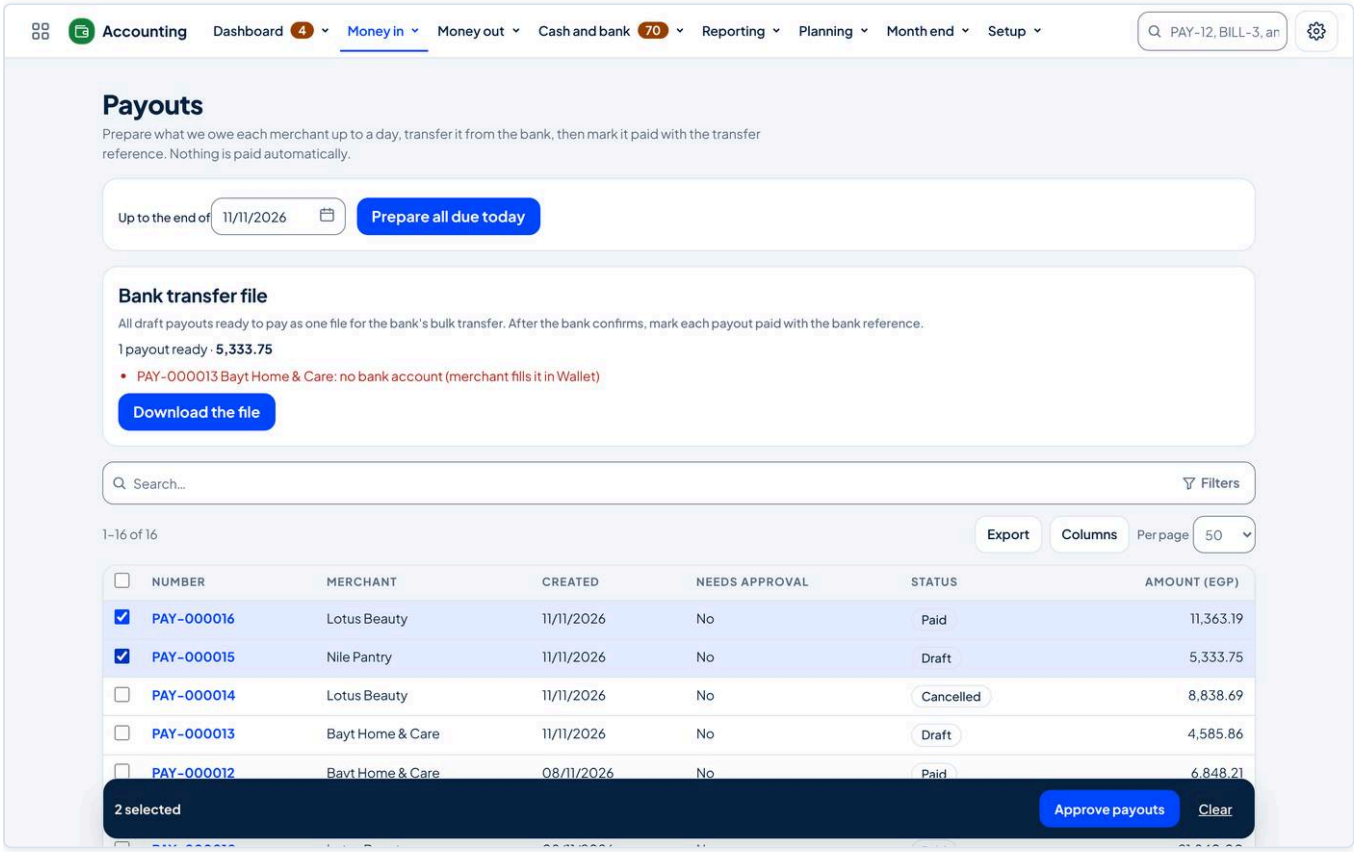
The column chooser: Number is always shown (greyed tick); the rest can be ticked on or off. The choice is kept per person (user_list_prefs) and in the browser.



The Export menu. Excel files are written without any library; money is exported as EGP numbers and Arabic sheets read right to left.



Grouped by Type: each group with its count in a grey pill and its subtotal, and a Total row at the bottom.

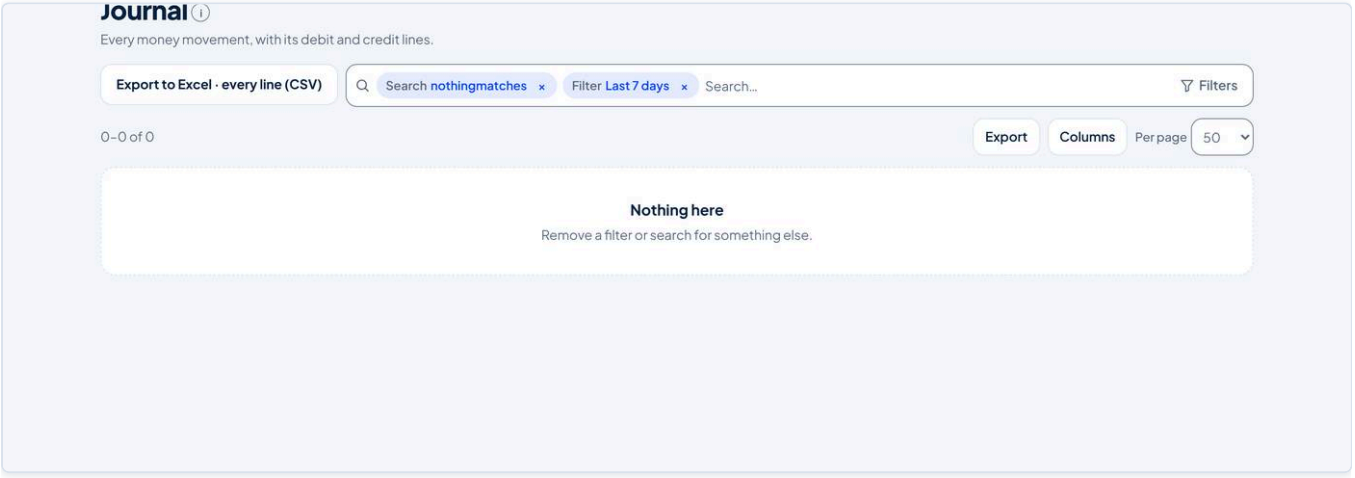


Payouts with two rows ticked: ticked rows turn light blue and the bulk bar sticks to the bottom of the screen.

The bulk bar: Navy ground, white text "2 selected", the list's actions as blue buttons (a danger action is outlined white), and Clear.

Bulk bar part	What it does	Rule
"n selected"	How many rows are ticked.	None.
Action buttons (for example Approve payouts)	Sends `{ ids, note? }` to the action's endpoint.	Each record is still checked on its own; an action that needs a note or a choice first opens its field, the next press runs it. Payouts: only those at or above the approval limit, never one's own.
Reason or note field	"Reason (needed to reject)" or "Note (optional; needed to reject)".	Up to 300 characters.
Clear	Unticks all rows.	None.

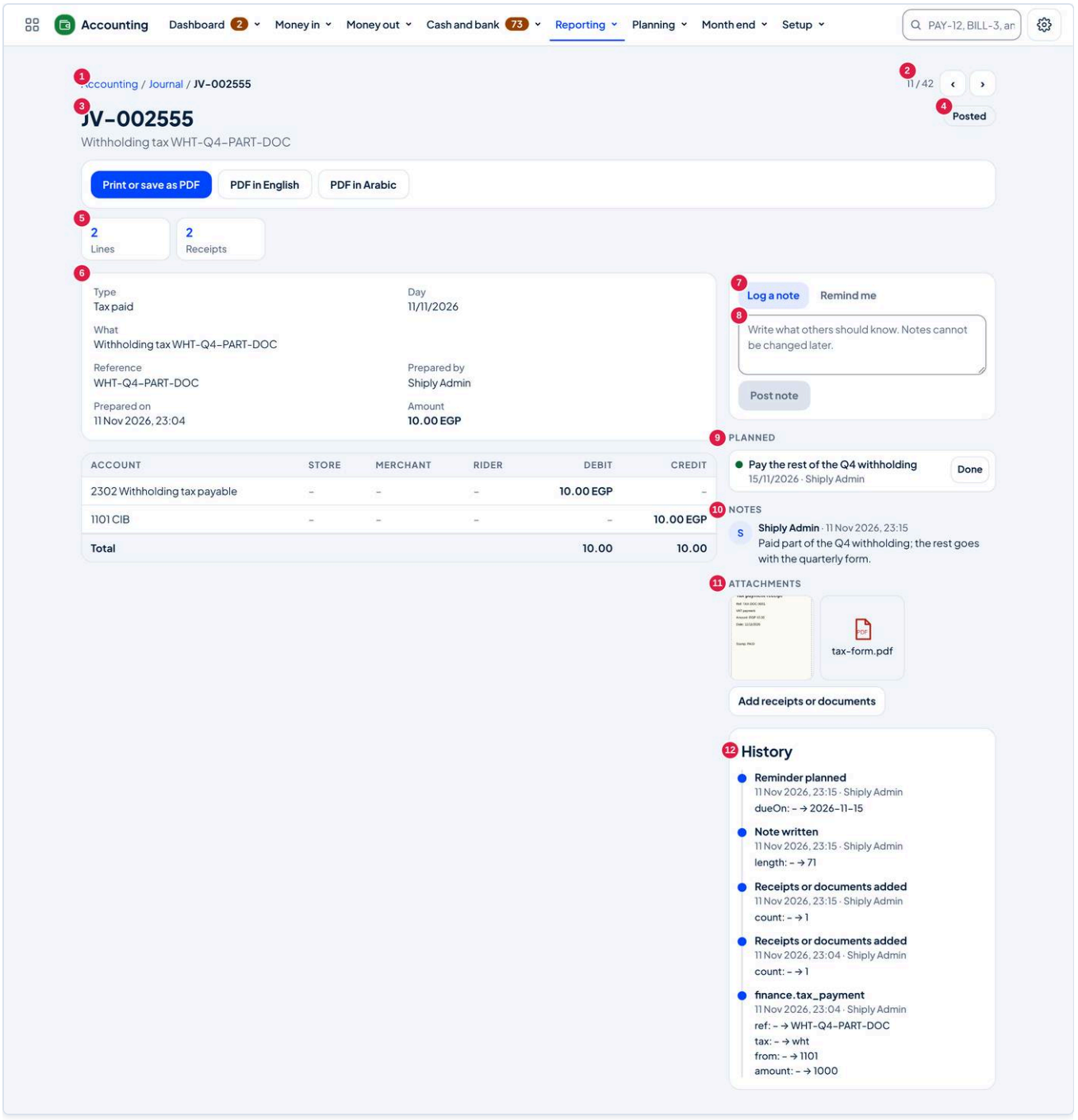
The result box after a bulk action: "0 done, 2 refused" with each reason and its count; an orange border when something was refused, × closes it.



Empty state of a list: "Nothing here" and "Remove a filter or search for something else."

17.14 Common part: the document page frame

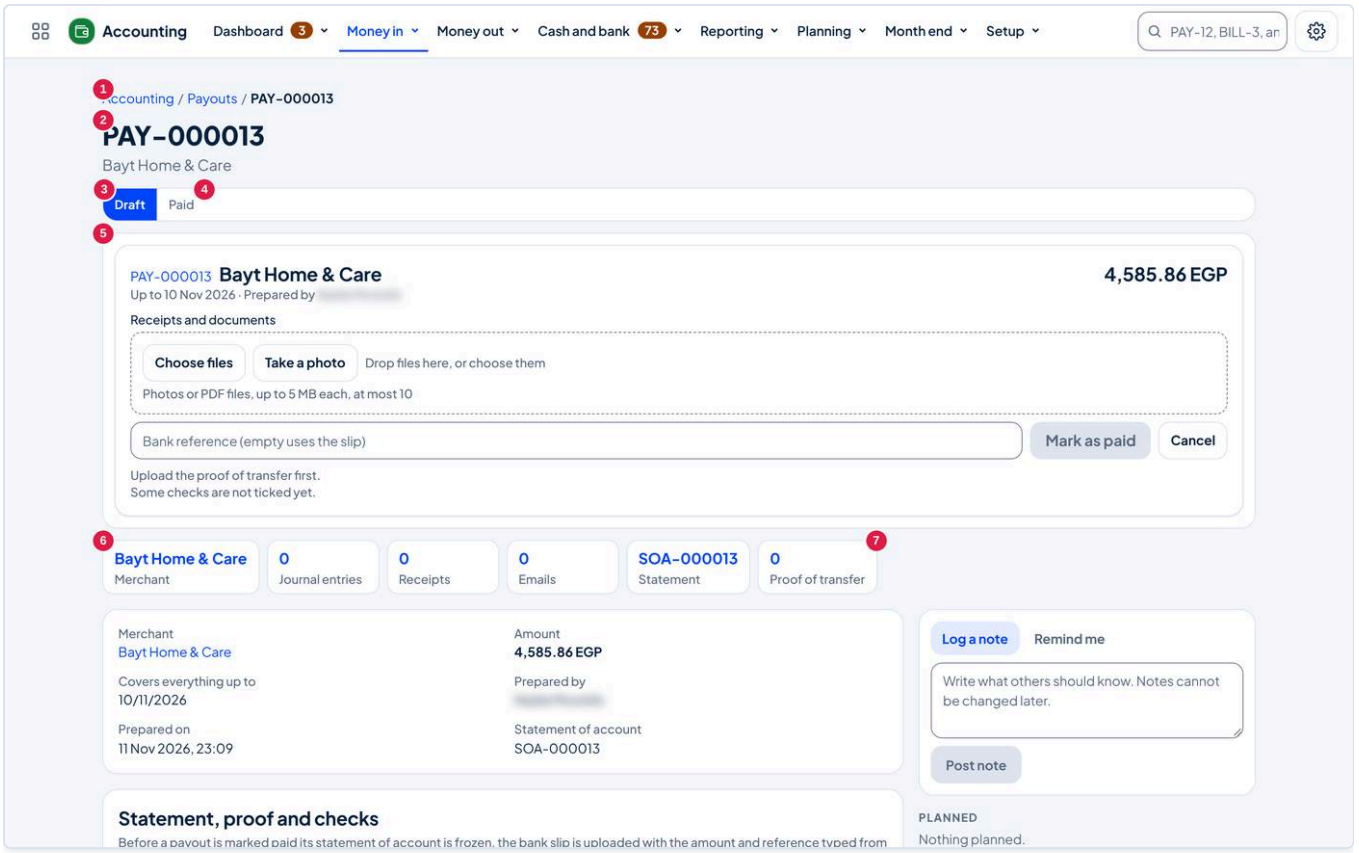
Every document page (payout, bill, journal entry, cheque, rider withdrawal, approval, petty cash, invoice, purchase order, expense claim, Basata deposit, handover) is drawn by `DocumentFrame` (`components/odoo/DocumentFrame.tsx`) from `GET /finance/documents/:type/:id`. The chatter sits beside the content from the lg width, below it on narrower screens.



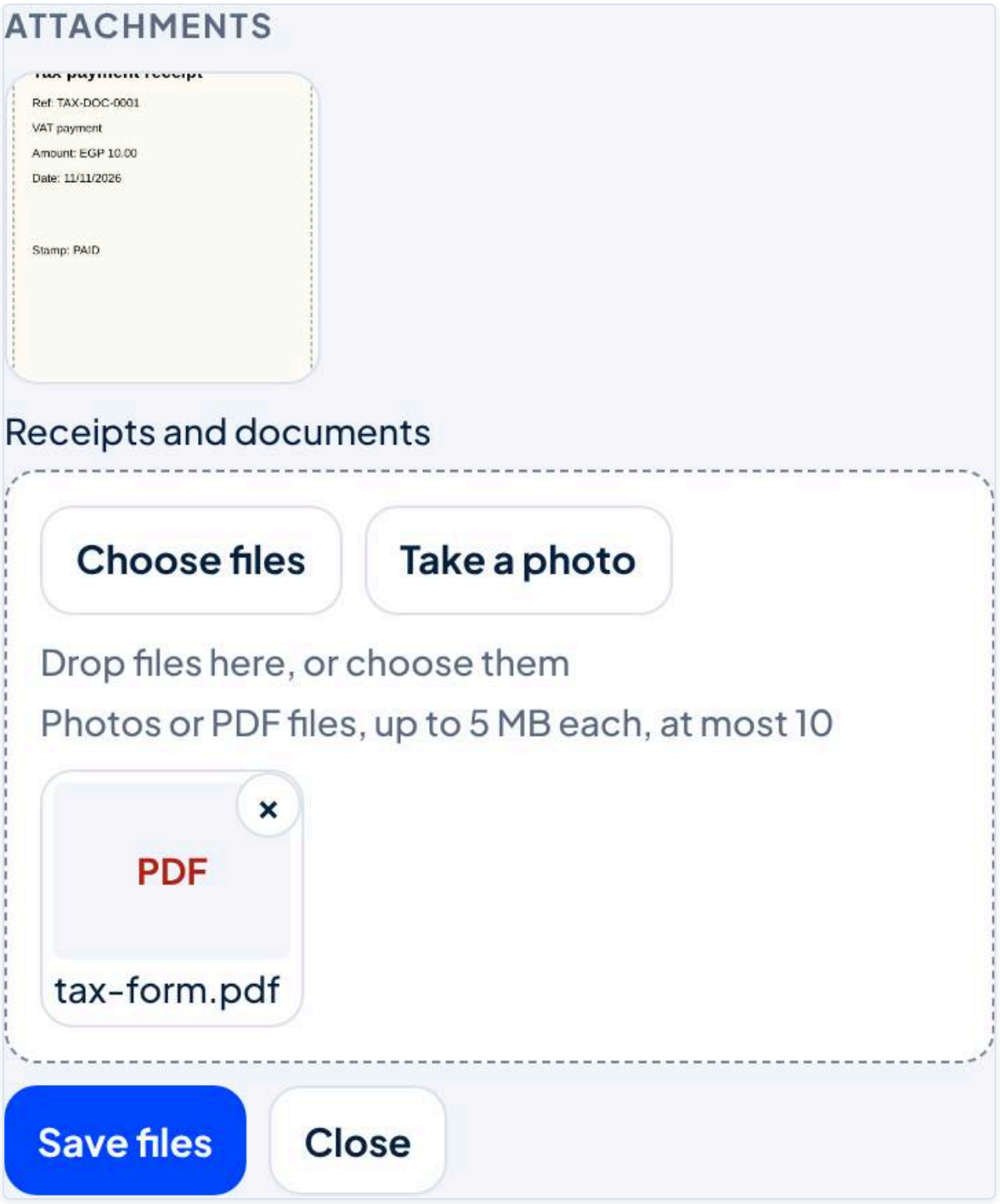
The journal entry JV-002555 (the tax payment made above), opened from the filtered journal list, with markers. Address /finance/journal/<id>.

#	Part	What it shows (colors, icons, wording)	What clicking it does
1	Breadcrumbs	"Accounting / Journal / JV-002555": the app and list in blue links, the number in bold.	Accounting opens the dashboard; the list name opens the list with the filters the person came from.
2	Previous and next record	"11 / 42" and two square arrow buttons. Shown only when the page was opened from a list.	Opens the previous or next row of that list.
3	Number and title	The number large and bold; the title in grey under it.	None.

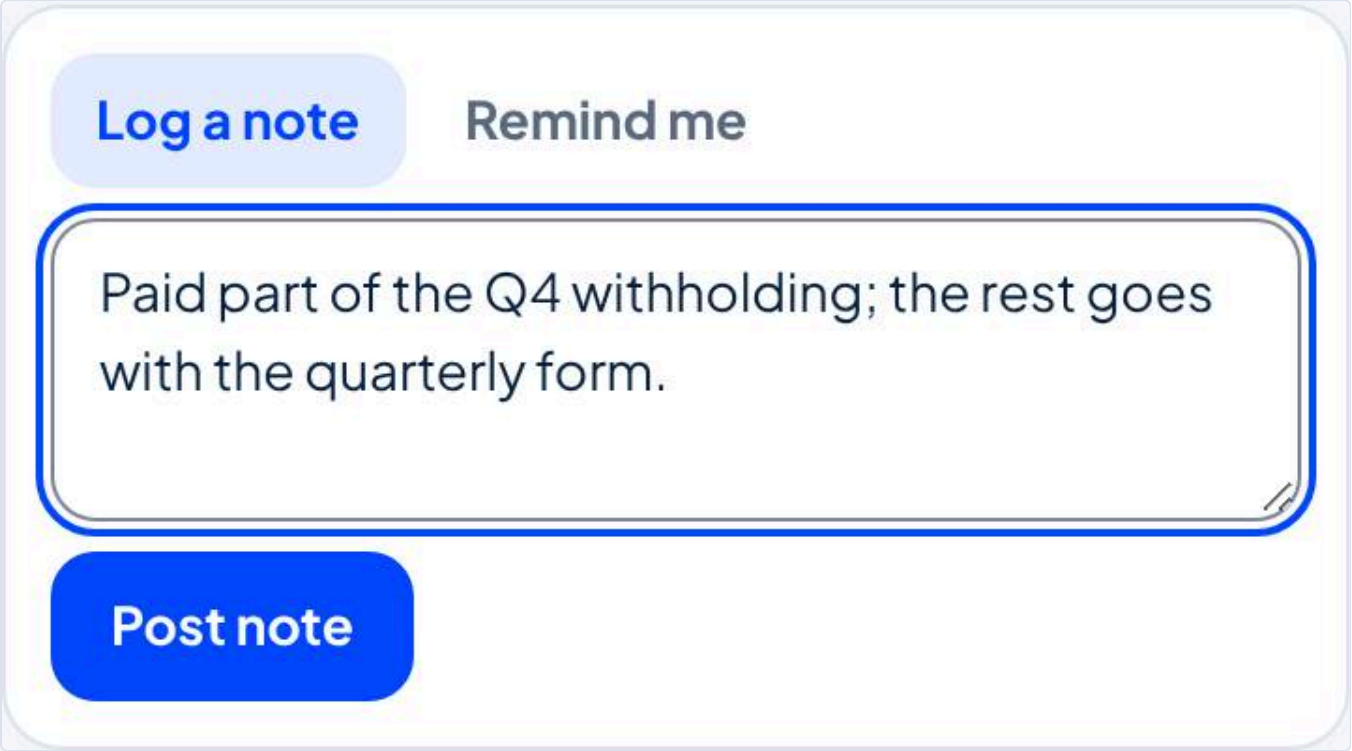
#	Part	What it shows (colors, icons, wording)	What clicking it does
4	Status pill (documents without a path)	A white pill with a grey border and bold text, here "Posted".	None. Documents with a path show the status bar instead.
(actions)	Action bar	A white card with the document's buttons (here Print or save as PDF in blue, PDF in English, PDF in Arabic).	The actions of that document.
5	Smart buttons	White tiles: the count or value in blue bold on top, the label in grey below (here 2 Lines, 2 Receipts). A tile turns blue bordered on hover.	Opens the related records; Receipts and Attachments open the files themselves (one file opens in a new tab, several open the attachment viewer).
6	Details	Label in grey, value under it; amounts bold with EGP. Then the lines table with Total.	Links in values open the record.
7	Chatter tabs	"Log a note" and "Remind me"; the chosen one has a light blue ground and blue text. Shown only with use on the record's app.	Switches the form.
8	Note box	Placeholder "Write what others should know. Notes cannot be changed later."; Post note is grey until text is typed.	Post note adds an append only note (audited record.note).
9	Planned	Reminders not done: a green dot (orange when late, "late"/"today" added), the text, day and who, and a Done button. "Nothing planned." when empty.	Done closes it (only the person reminded or who planned it).
10	Notes	Each note with the writer's initial in a light blue circle, name in bold, time in grey. "No notes yet." when empty.	None.
11	Attachments	Thumbnails three to a row: a photo preview, or a red PDF icon with the file name. "No receipt or photo yet." when empty. Add receipts or documents below (with use).	A tile opens the file; Add receipts or documents opens the attachment field with Save files.
12	History	A timeline from the audit log: a blue dot, the action in bold, time and person in grey, then each changed field as "field: before → after".	None.



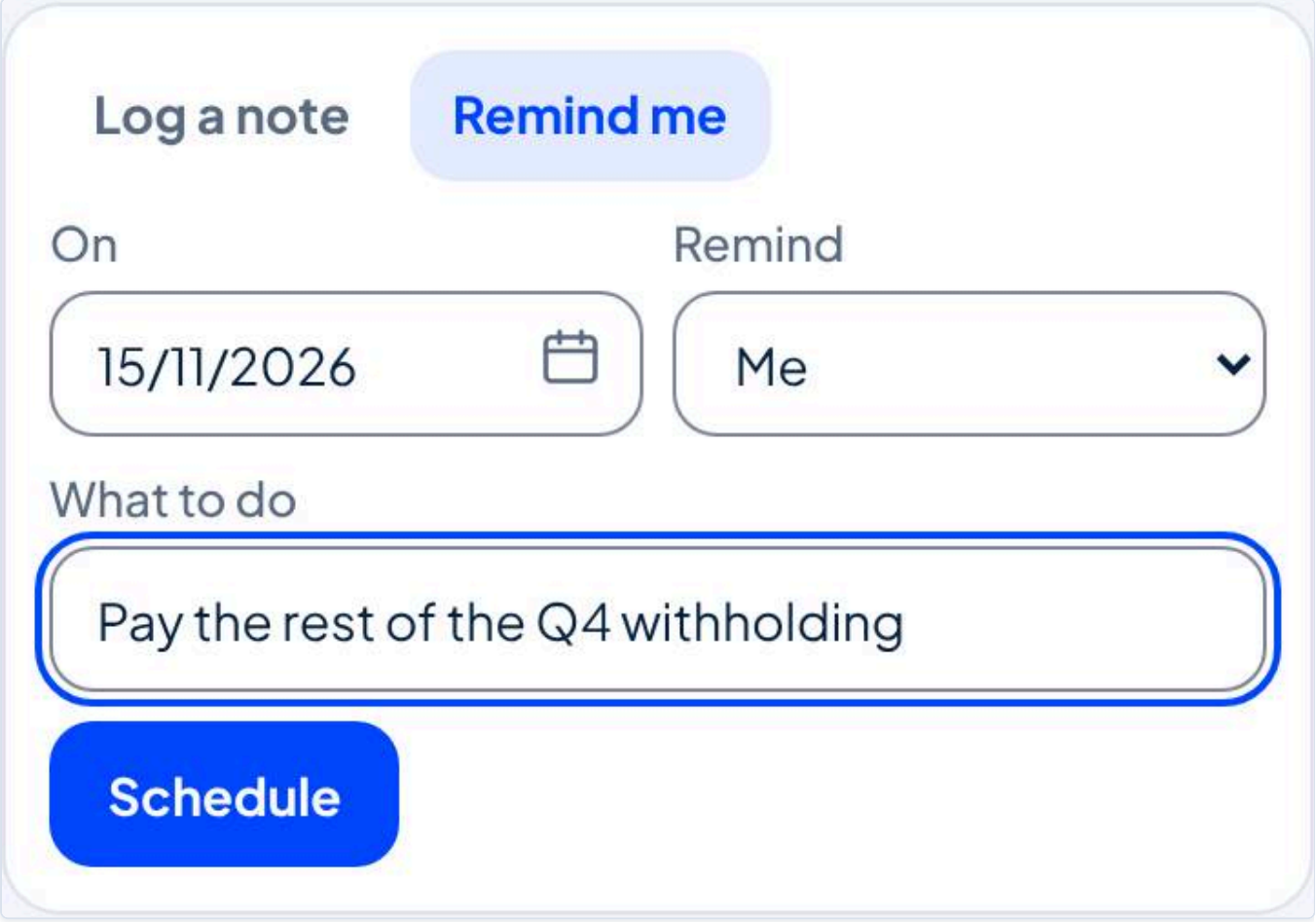
A payout document (PAY-000013, draft) with markers: (1) breadcrumbs, (2) number and merchant, (3) the status bar's current step in blue, (4) the next step in grey, (5) the action card (attachment field, bank reference, Mark as paid, Cancel, and the reasons it cannot be paid yet), (6, 7) smart buttons Merchant, Journal entries, Receipts, Emails, Statement and Proof of transfer.



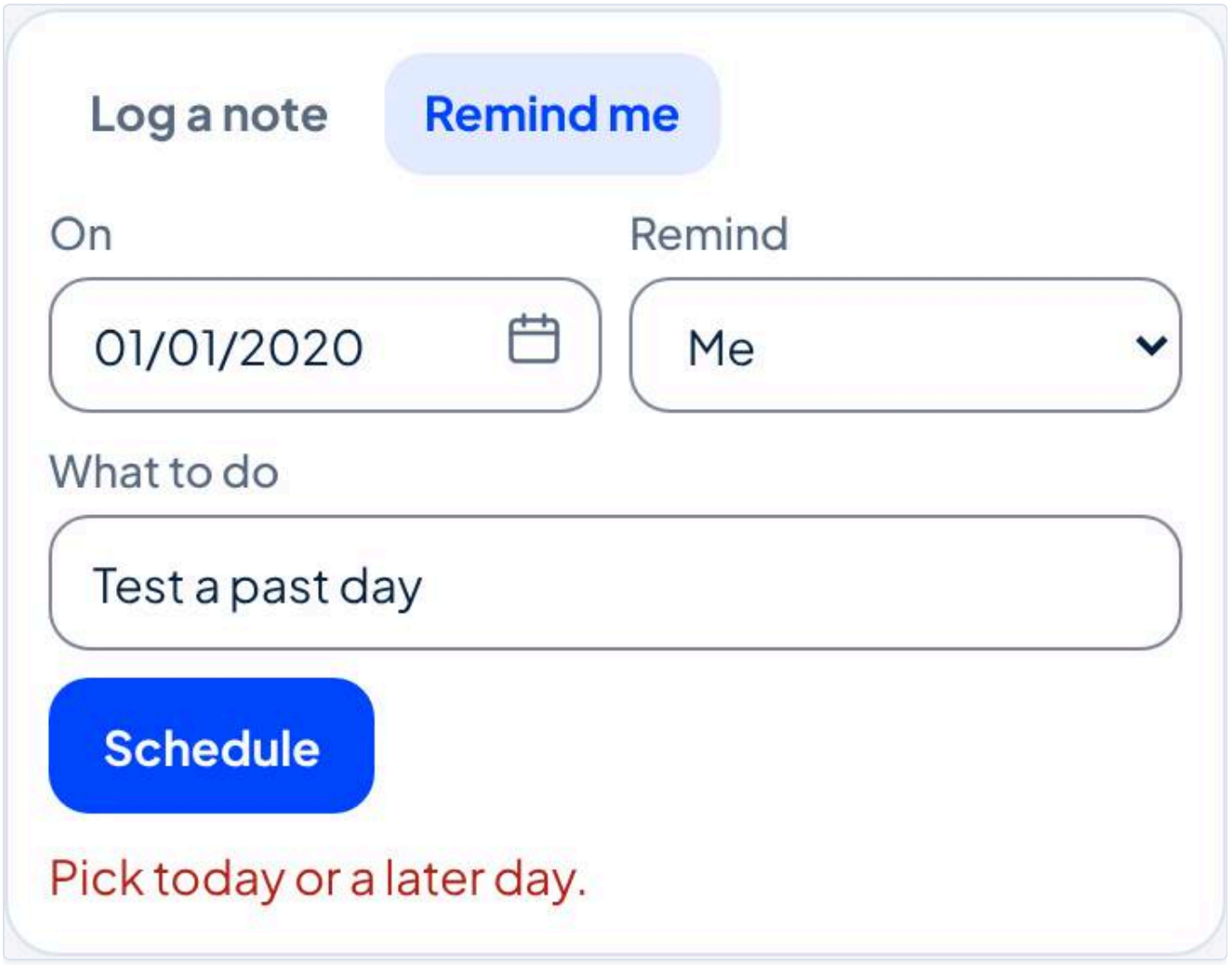
Add receipts or documents opened under Attachments, with a PDF chosen and Save files.



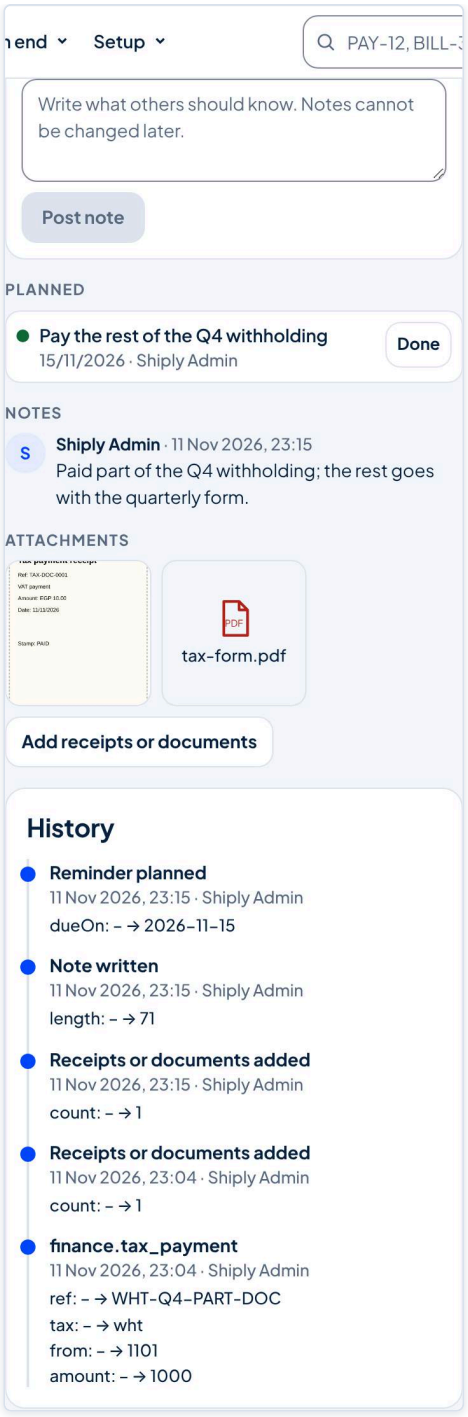
A note typed in Log a note; Post note is now blue.



Remind me: On (a day), Remind (Me or another person who may open the record), What to do, and Schedule.



Error: a reminder on a past day: "Pick today or a later day." Other chatter errors: "That person cannot open this record.", "You can read this record but not write on it.", "Only the person reminded or who planned it can close it."



The chatter after a note and a reminder: Planned, Notes, Attachments (photo and PDF) and History.

17.15 Common part: the attachment field and viewer

`AttachmentField` (`components/finance/AttachmentField.tsx`) is used in every finance form that takes receipts: Pay tax, deposits, payouts, expenses, bills, petty cash and more.

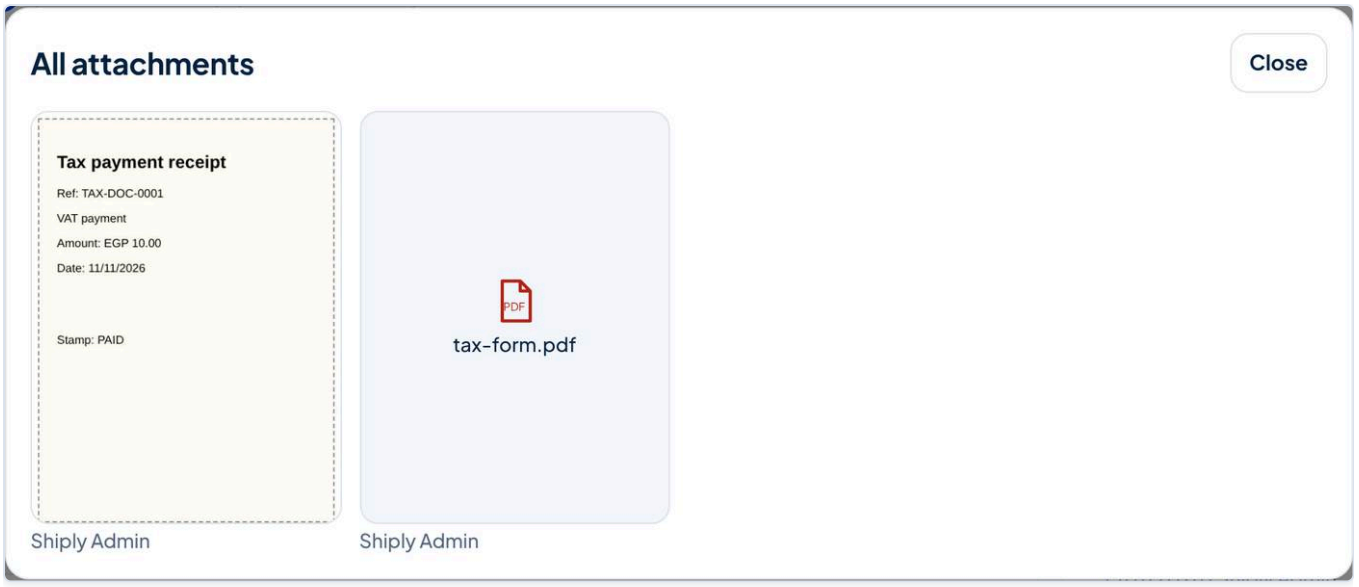
The attachment field with a photo and a PDF chosen, with markers.

#	Part	What it shows	What clicking it does
1	Choose files	White button.	Opens the file picker (several files; JPEG, PNG, WebP or PDF).
2	Take a photo	White button.	Opens the phone camera (back camera).
3	Drop zone	Dashed grey border; "Drop files here, or choose them" and "Photos or PDF files, up to 5 MB each, at most 10". The border turns blue while a file is dragged over it.	Dropping files adds them.
4	Photo tile	A preview with the file name; "Uploading…" while it uploads; a red border if the upload failed.	None.
5	PDF tile	"PDF" in red bold on grey, with the file name.	None.
6	× on a tile	A round white button at the top corner.	Removes the file; an uploaded file is deleted at once (`DELETE /finance/files/:id`).

RULE

Each file uploads as soon as it is chosen (`POST /finance/files`) and waits unlinked; the form sends the ids as `attachmentIds` and its save links them in the same transaction. Photos are shrunk in the browser like every receipt photo; PDFs are sent as they are, at most 5 MB. The save waits while files upload ("Wait until the files finish uploading."). Errors: "{name} is not a photo or a PDF file.", "{name} is larger than 5 MB.", "At most 10 files.", "Upload failed. Try again.", and for a required field "Add the receipt before saving.". Unlinked files are deleted by the daily job.

After × on the PDF: only the photo remains.



The attachment viewer ("All attachments"), opened from the Receipts smart button of a record with two files: large tiles four to a row on a wide screen, who added each, and Close. Clicking outside also closes it; a tile opens the file.

17.16 Common part: the form footer

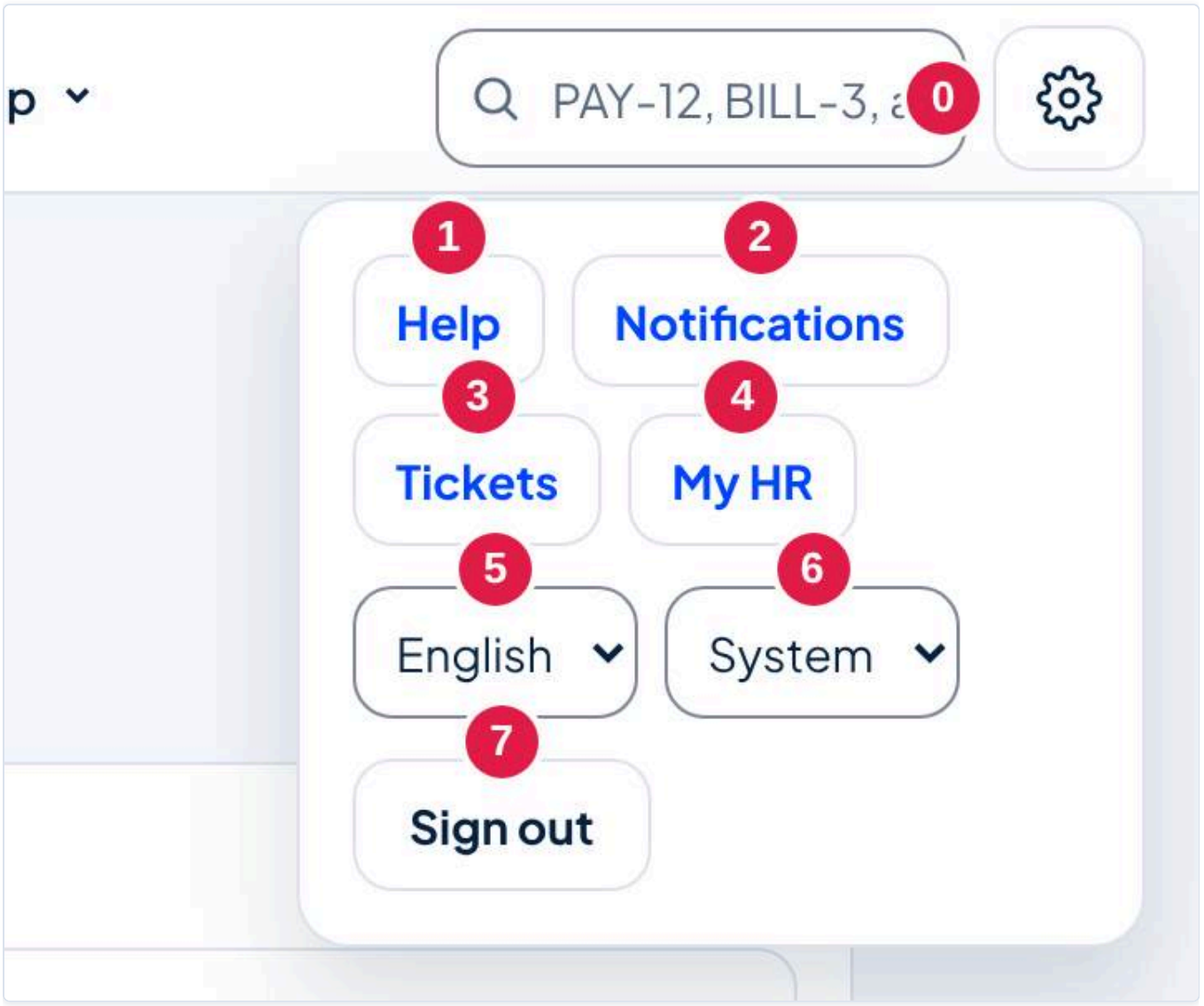
`FormFooter` (`components/workflow/FormFooter.tsx`) ends every create and edit form.

The form footer of New cost tag after typing: (1) the save button (pale while the form is not valid), (2) Cancel, (3) "Unsaved changes" in bold orange.

#	Part	What it shows	What clicking it does
1	Save (its own label per form, for example Open the tag, Open account)	Blue; grey while disabled; "Saving…" while busy.	Saves the form.
2	Cancel	Navy outlined.	When there are changes the browser asks "Throw away the changes you made?"; then empties or leaves the form.
3	Unsaved changes	Bold orange text, only while the form differs from what is saved.	None. Leaving the page by a link or closing the tab asks "You have changes that are not saved. Leave anyway?"

17.17 Common part: the settings menu

The gear button at the right of the top bar (`AccountMenu` , `components/AccountMenu.tsx`) holds the personal settings. Escape or a click outside closes it.



The settings menu open, with markers.

#	Part	What it shows	What clicking it does
0	Gear button	A gear icon (with the person's photo beside it once their HR photo exists).	Opens and closes the panel.
1	Help	Blue link button.	`/help` : the SOP of the person's role (admins can read every role).
2	Notifications	Blue link button.	`/notifications` : phone notification switches.
3	Tickets	Blue link button, with a count when tickets wait (not for merchants).	`/tickets`
4	My HR	Blue link button (team members).	`/my-file`
5	Language	English or العربية.	Sets the `shiply_locale` cookie and reloads the page in that language and direction.

#	Part	What it shows	What clicking it does
6	Theme	System, Light or Dark.	Sets the `shiply_theme` cookie.
7	Sign out	Bold dark text button.	Ends the session.

17.18 Statuses and their colors

Accounting shows a status in three ways.

- **Status bar** on a document page with a path (`StatusBar` , `lib/workflows.ts`): steps left to right in one white bar with grey borders. A done step has a ✓ and normal dark text; the current step has a Shiply Blue (#0046FC) ground with white bold text; later steps are grey (#5B6B82). A side state (cancelled, rejected...) is added at the end on a red (#B42318) ground with white bold text, and then no step is lit. On phones only the step before, the current one and the next show.
- **Status pill** in lists and on documents without a path: a white pill with a grey (#DFE5EE) border and dark text, the same for every status. Exceptions: cheques (cleared in green, bounced and cancelled in red, the rest grey) and forecasts (Approved green on a light green tint, Draft grey on a light grey tint).
- **Text states**: the month end checklist (Done green #0F6A35, n left and Tick when done orange #9A4100, After the month ends grey) and loans (Active or Repaid in grey under the title).

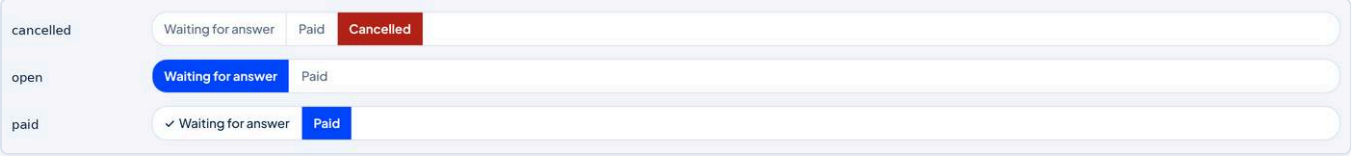
In the dark theme the same roles use lighter tones (for example green #5FD38D, orange #FFB35C) on Navy grounds.

Document	Path (blue when current)	Side states (red)	When each appears
Payout (`payout`)	Draft → Paid	Cancelled	Draft when prepared (it may still need approval: "Needs approval" column); Paid when marked paid with the bank reference; Cancelled when a draft is cancelled or a payout cheque bounces.
Bill (`bill`)	Waiting for answer → Paid (the label of the state open is shared with tickets)	Cancelled	Open when booked; Paid when fully paid; Cancelled by a reversing entry.
Rider withdrawal (`rider_withdrawal`)	Requested → Approved → Paid	Rejected, Cancelled	Requested by the rider; approved by accounting use; paid from a bank or box; rejected with a reason; cancelled by the rider before approval.
Approval request (`finance_request`)	Waiting → Approved	Rejected, Cancelled	Waits for a second person; approved posts the entry; rejected with a note; cancelled by the person who asked.
Petty cash (`petty_cash`)	Waiting → Posted	Rejected, Cancelled, Reversed	Waiting when above the limit or below the float; posted; reversed by accounting manage with a reason.
Tax invoice e invoice state (`eta`)	Not sent → Signed → Submitted → Accepted	Invalid, Rejected	Signed by the signing program, submitted by EtaSync, accepted (valid) or invalid or rejected by the tax authority.

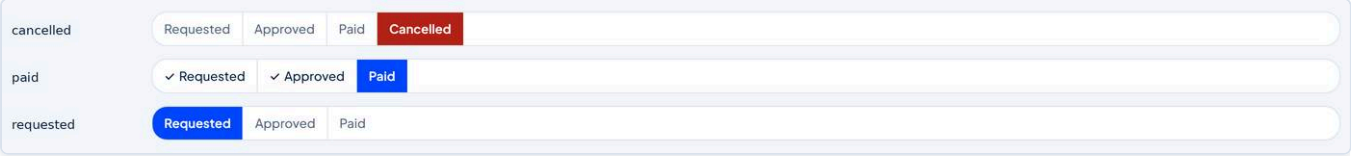
Document	Path (blue when current)	Side states (red)	When each appears
Purchase order (`purchase_order`)	Requested → Approved → Ordered → Received → Billed → Closed	Rejected, Cancelled	Approved at once below the limit, else by another person; received in parts; billed; closed.
Cheque received (`cheque_received`)	Waiting for approval → Received → Deposited → Cleared	Bounced, Cancelled	Always waits for a second person; deposited at the bank; cleared never before its due day.
Cheque issued (`cheque_issued`)	Waiting for approval → Issued → Cleared	Bounced, Cancelled	Booked against 2451; cleared out of the bank.
Expense claim (`expense_claim`)	Manager → Waiting for accounting → Paid	Rejected, Cancelled	The manager in force approves, accounting pays.
Basata deposit (`basata_deposit`)	Waiting → Approved	Rejected	Sent by the rider with the receipt; approved or rejected by accounting use.
Cash handover (`handover`)	Waiting → Balanced	Written off, On the rider	A difference waits for finance, who write it off or keep it on the rider.
Journal entry	No path: a grey pill Posted (or Reversed)		Every posted entry.
Loan	No pill: "Active" or "Repaid" in the grey line under the title		Repaid once the last instalment is paid.
Forecast	Pill Draft (grey tint) or Approved (green tint)		Approved by accounting manage; Reopen makes it a draft again.

The strips below are the real status bars of demo records for each status that exists in the demo copy (cheques, expense claims, loans and forecasts had no record).

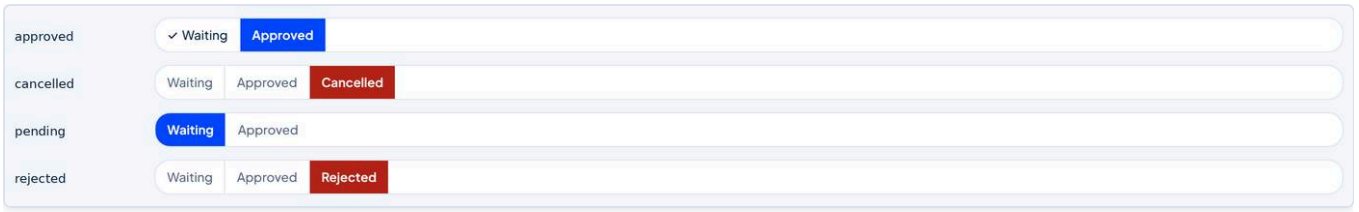
Payout status bars: cancelled, draft, paid.



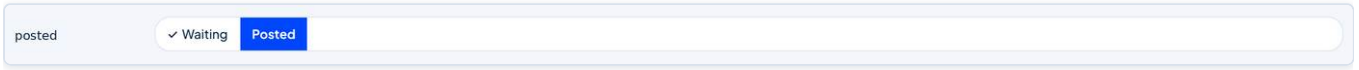
Bill status bars: cancelled, open (labelled "Waiting for answer"), paid.



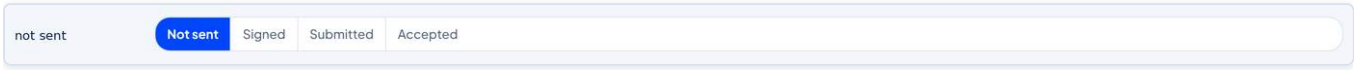
Rider withdrawal status bars: cancelled, paid, requested.



Approval request status bars: approved, cancelled, pending, rejected.



Petty cash status bar: posted.



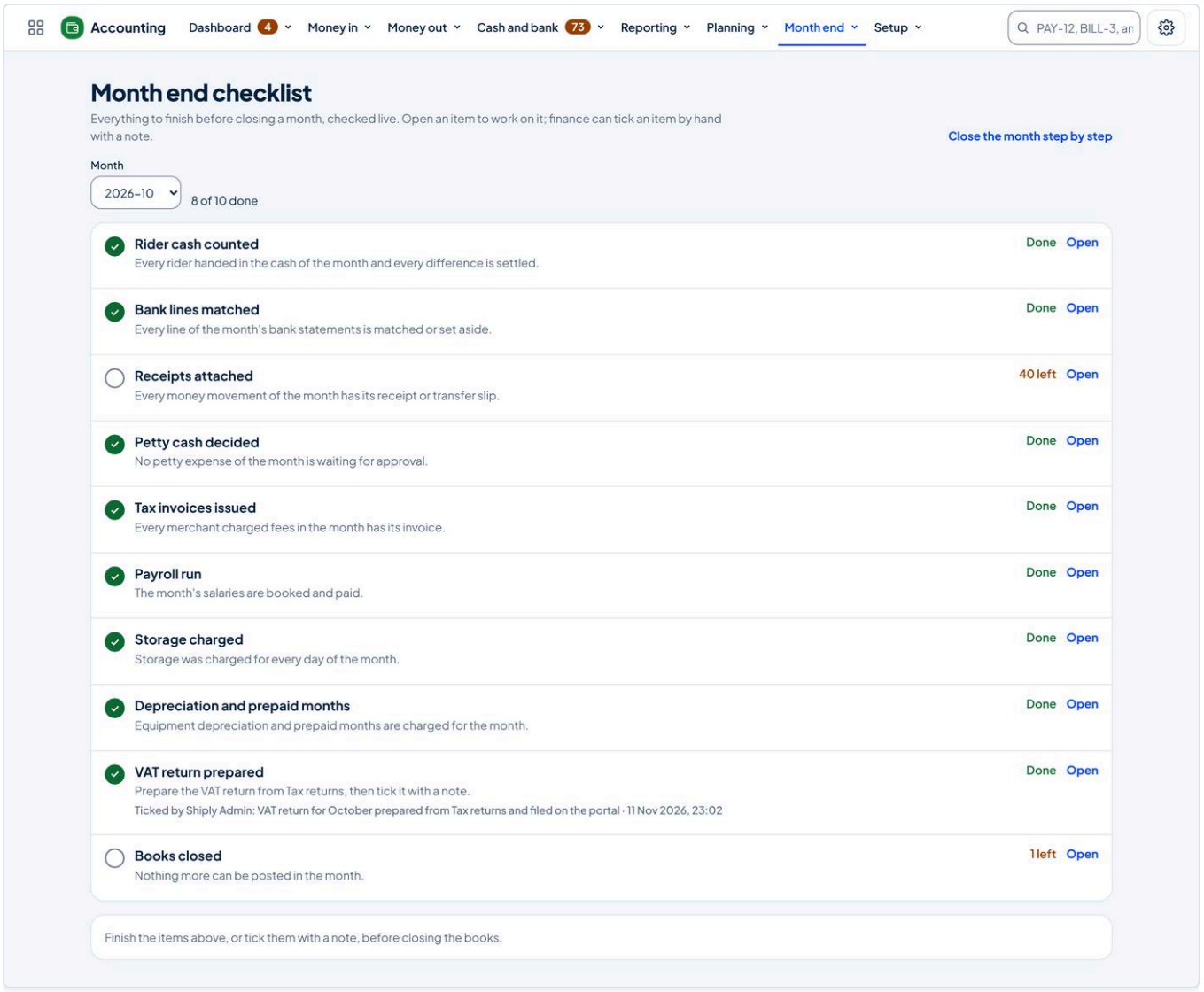
Tax invoice e invoice status bar: not sent.

Purchase order status bars: cancelled, closed, rejected.

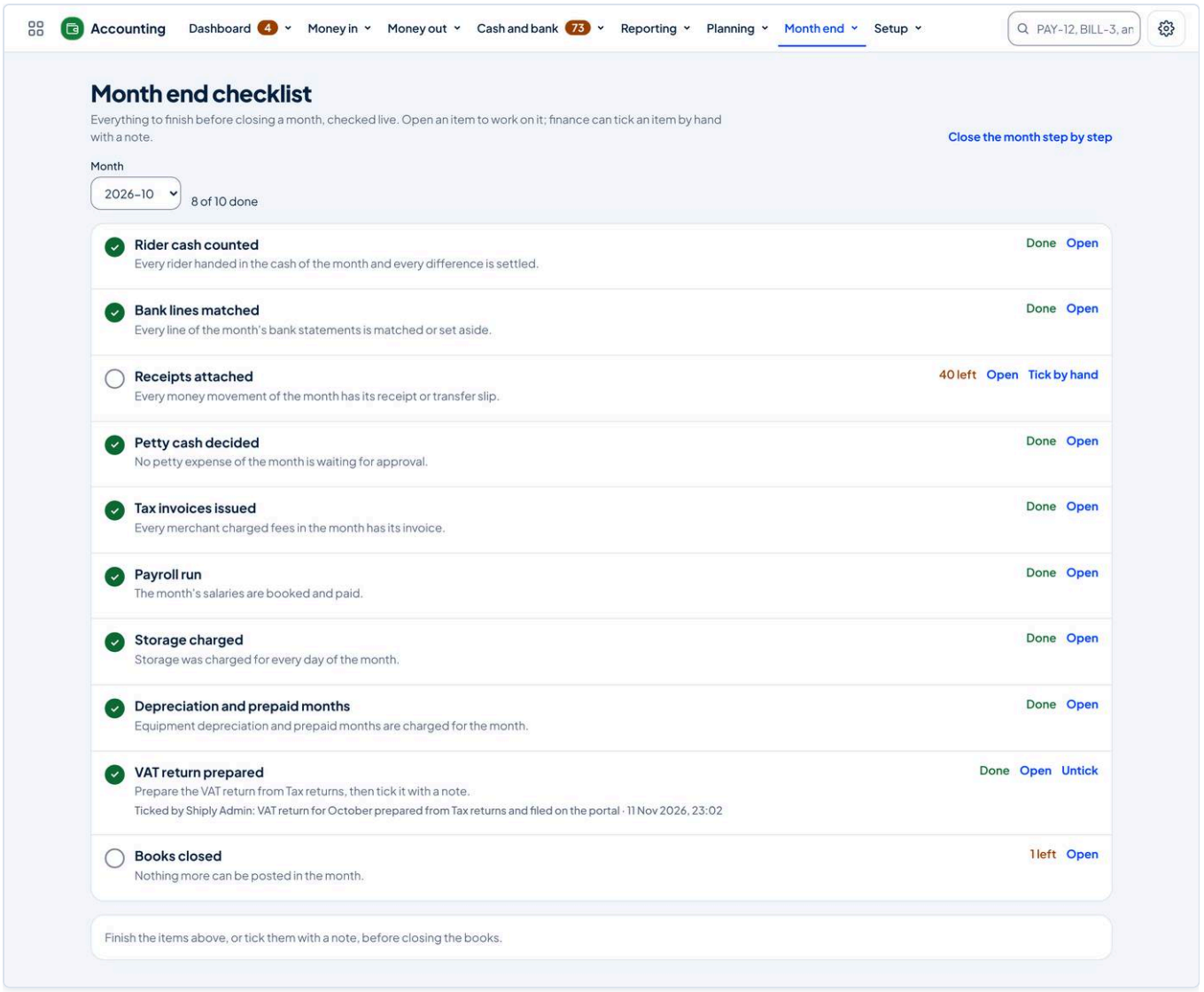
Basata deposit status bars: approved, pending, rejected.

17.19 Access: the same screen for view and for use

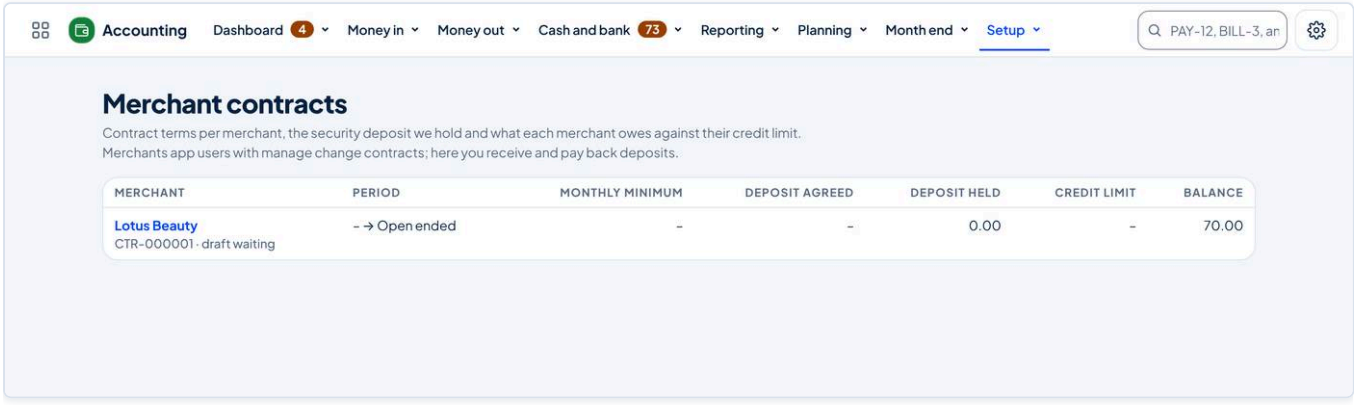
The pages hide what a person cannot do, and the API refuses it anyway. Below, the same demo user with accounting **view**, then with accounting **use**.



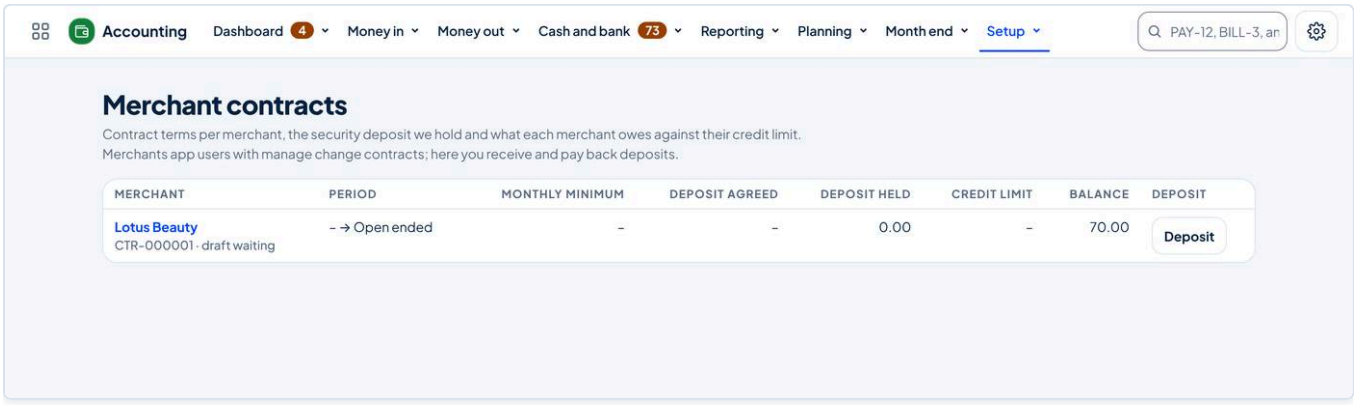
Month end checklist with accounting view: Open links only, no Tick by hand, no close button.



The same page with accounting use: Tick by hand appears on items that may be ticked; the close button still needs manage, so only the "Finish the items above..." line shows.



Merchant contracts with accounting view: no Deposit column, and the merchant name is plain text because this user has no merchants view.

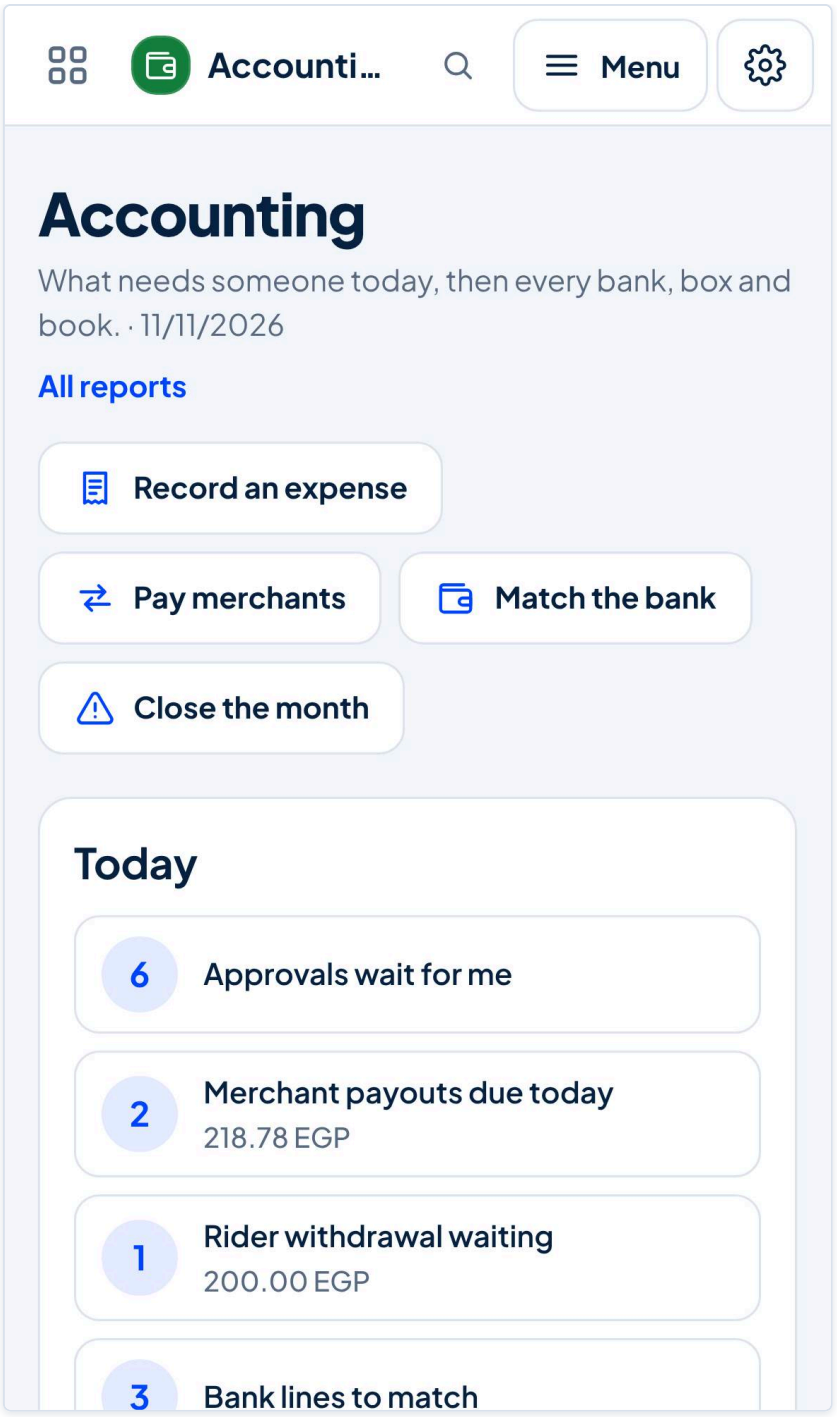


The same page with accounting use: the Deposit column and button appear.

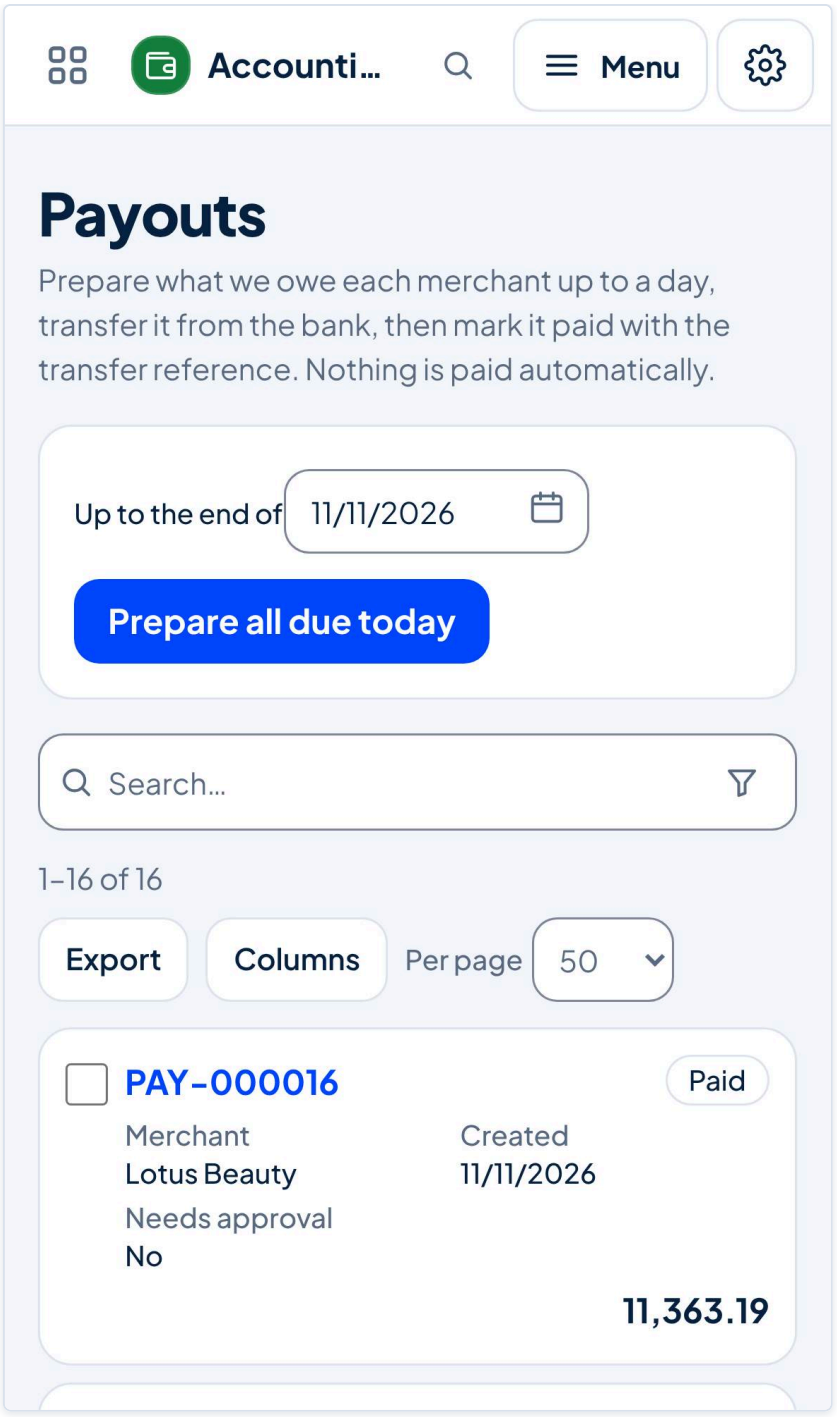
Screen	View	Use adds	Manage adds
Close the month, Month end checklist	Read, Open, Check again	Tick by hand, Untick	Close the books
Close the books	Read the closed day	(button shown, refused)	Close; reopen also needs settings manage
Year end	Read, downloads	None	Close year
Taxes	Read every tab	Pay tax	Book the company income tax
Chart of accounts	Read, statements	Open, edit, close, reopen, revalue	None
Company tax details	Read	Merchant tax and bank details	Save our details; signing key: owner
Merchant contracts	Read	Deposit	None
Cost tags	List of names	None	Form, switch on and off
Payout checks	Read	None	Edit, add
Opening balances	Owner only (accounting manage and settings manage)		
Accounting settings	Settings manage only		

17.20 Phone layout

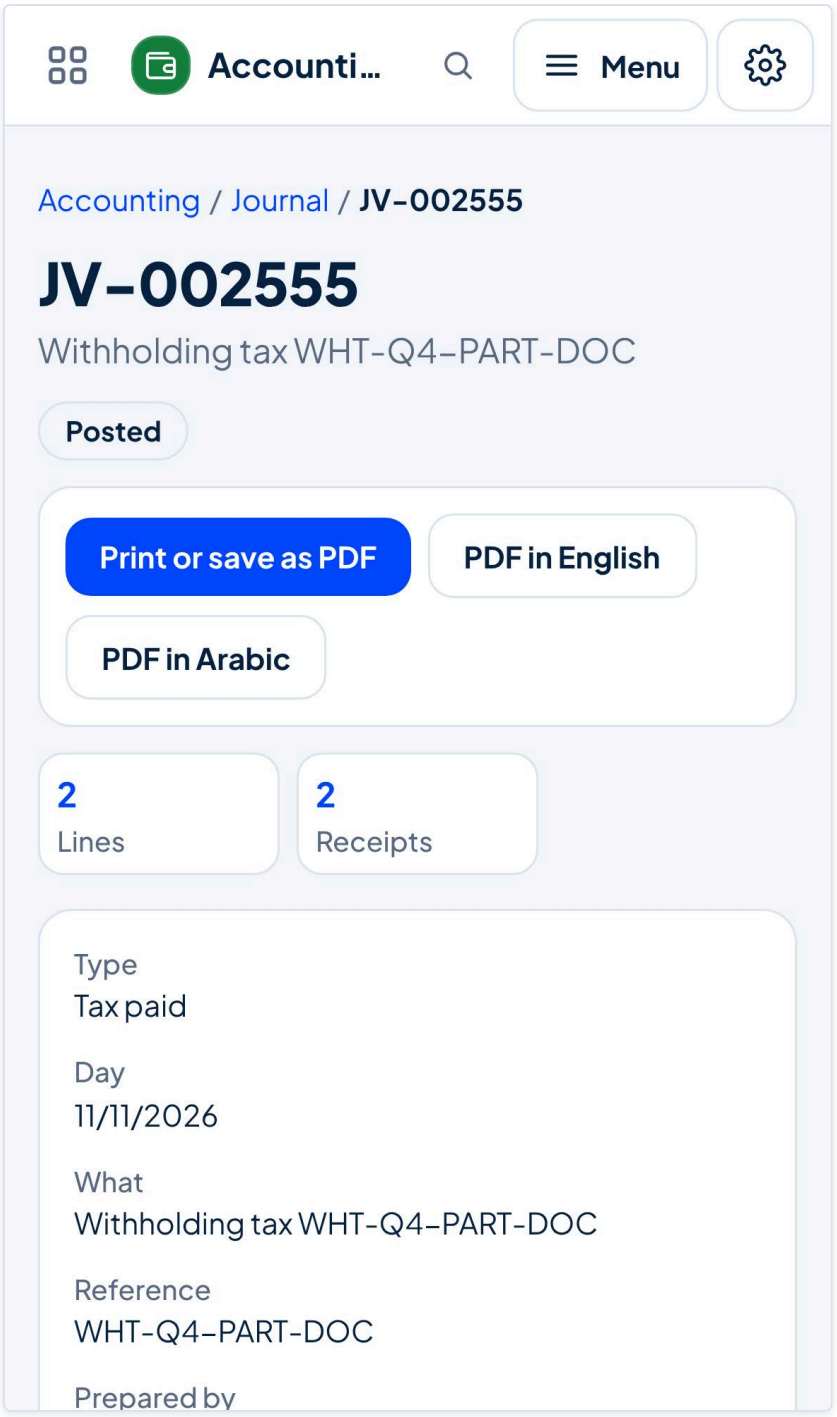
On a phone (iPhone 13 width) the top bar keeps the grid button, the app name, a search icon, one Menu button for all the menus, and the gear. Lists become cards, and the document chatter moves below the content.



The Accounting dashboard on a phone.



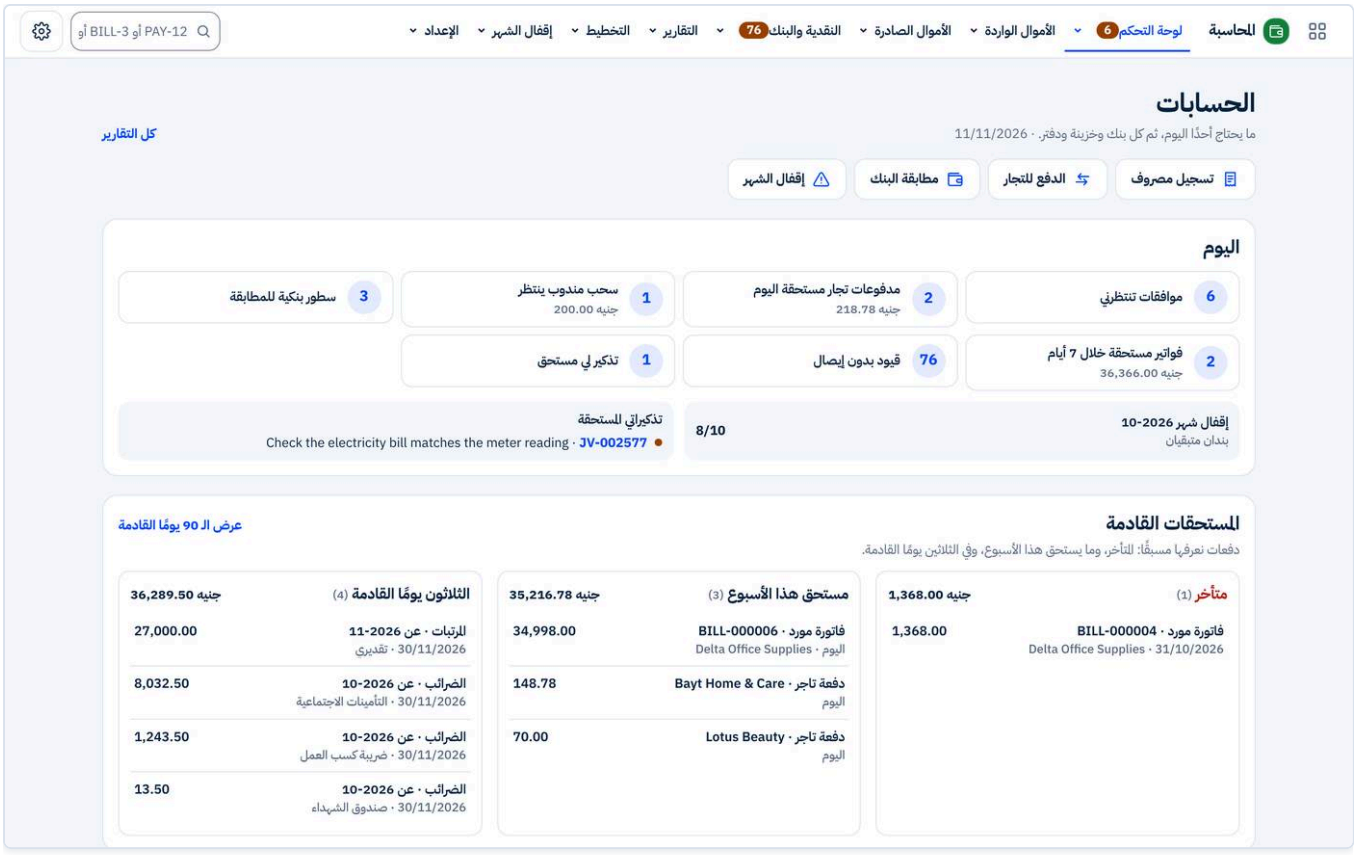
The payouts list on a phone: the Filters button is an icon, and each row is a card with a checkbox, the number, the status pill, the other columns as label and value, and the amount at the bottom right.



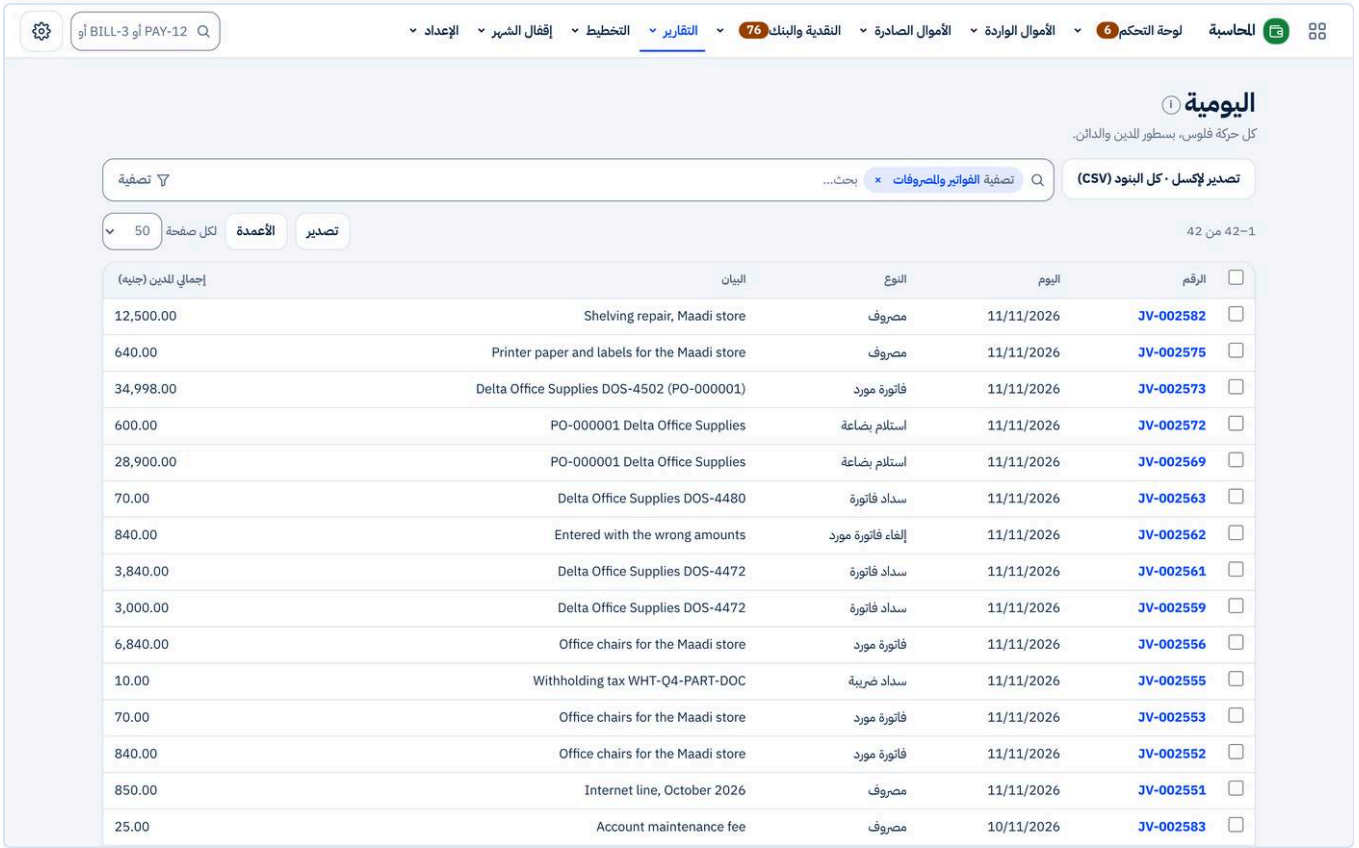
A journal entry on a phone: breadcrumbs, number, pill, action buttons wrapped, smart buttons three to a row and the details one under the other.

17.21 Arabic, right to left

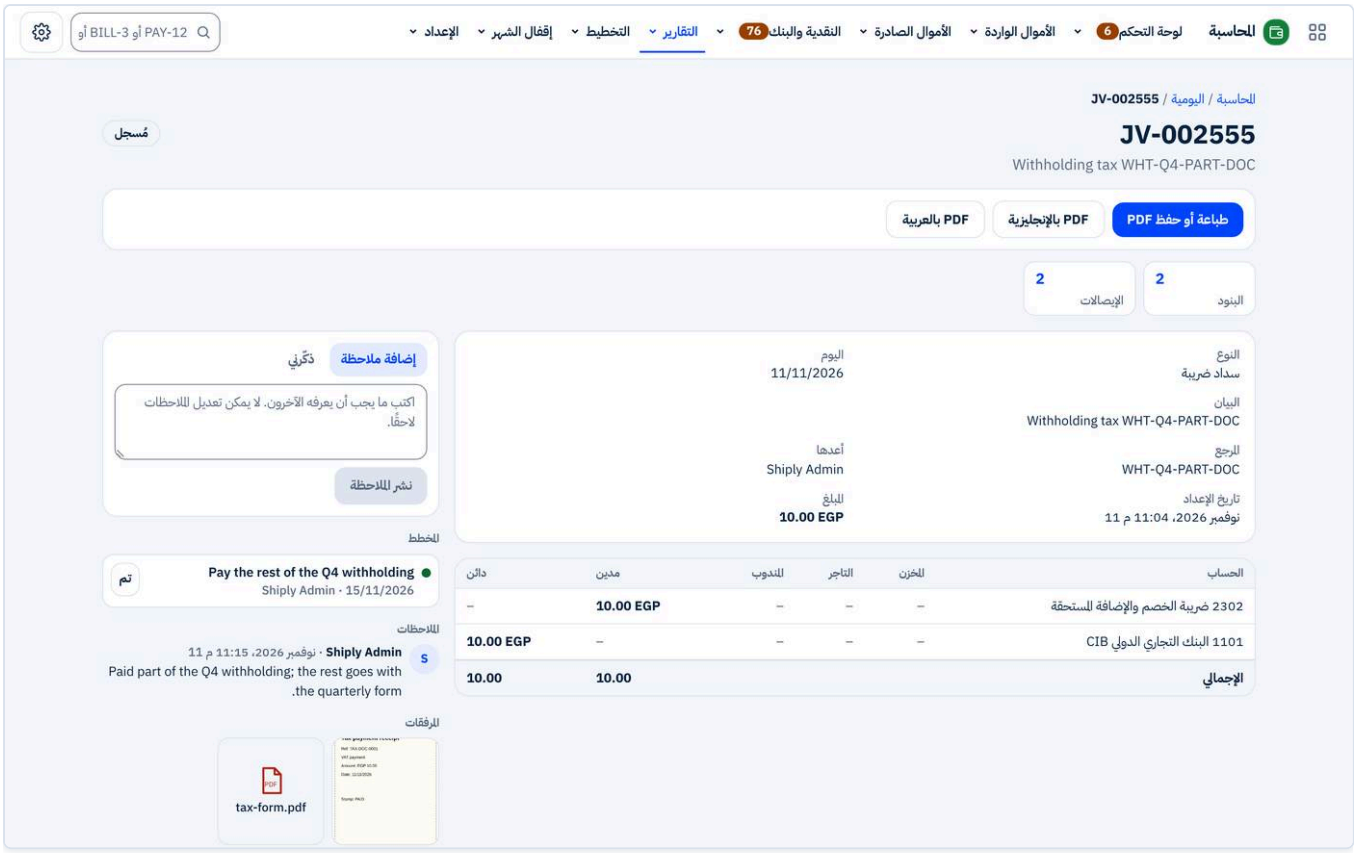
With the language Arabic (cookie `shiply_locale=ar`) every screen mirrors: menus start at the right, the grid button and app name sit at the right, the gear and search at the left, and the chatter is on the left of the content. Numbers, codes and days stay left to right (`ltr-iso`). Text people typed (notes, memos) is shown as typed.



The Accounting dashboard in Arabic.

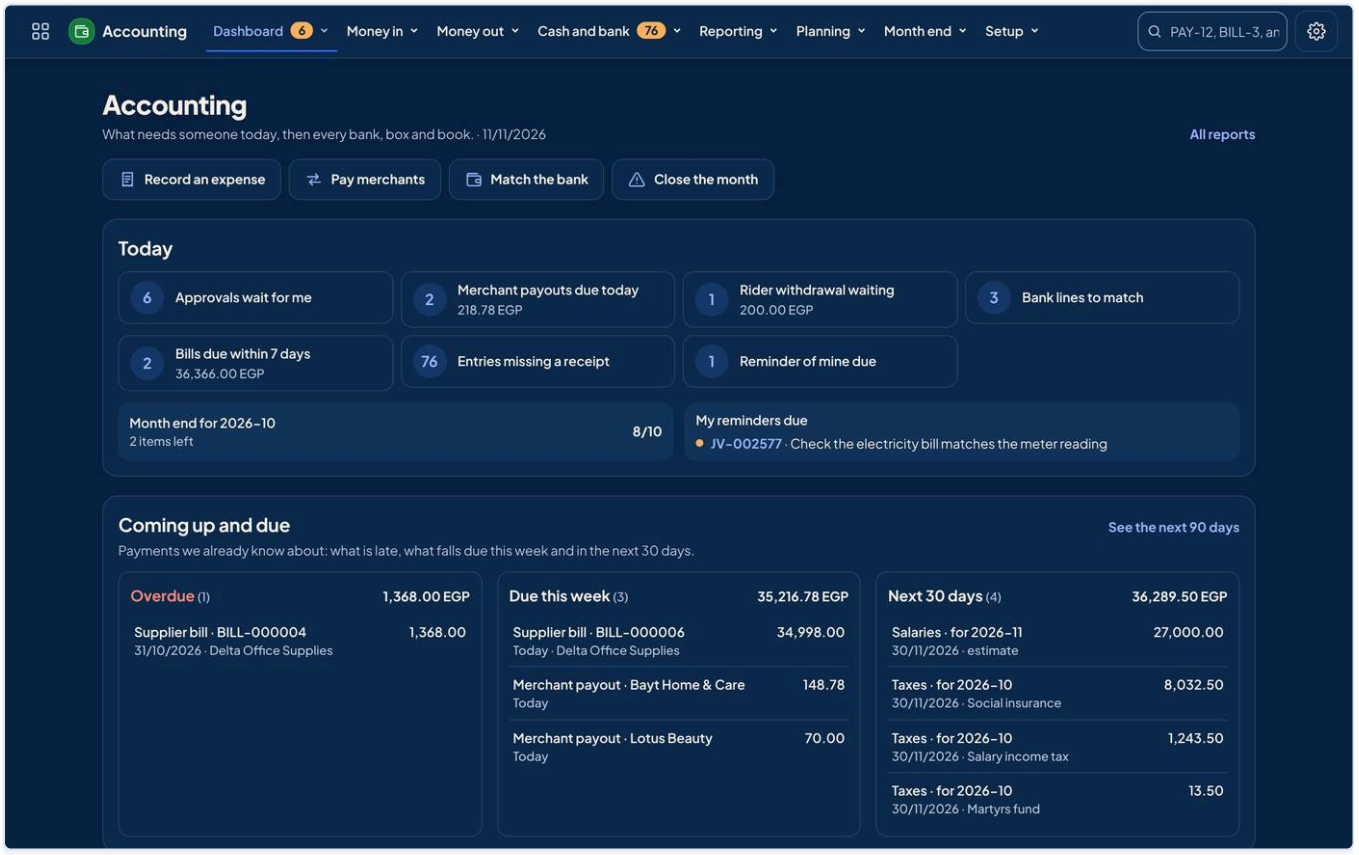


The journal list in Arabic, filtered to bills and expenses.



The journal entry JV-002555 in Arabic: breadcrumbs and number at the right, the status pill at the left, the chatter on the left.

17.22 Dark theme



The Accounting dashboard in the dark theme (cookie shiply_theme=dark): Navy grounds, lighter blue links, and the count badges in amber.